

INTERNATIONAL STANDARD



Universal Serial Bus interfaces for data and power – Part 3-1: Universal Serial Bus 3.1 Specification



THIS PUBLICATION IS COPYRIGHT PROTECTED

Copyright © 2017 IEC, Geneva, Switzerland

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either IEC or IEC's member National Committee in the country of the requester. If you have any questions about IEC copyright or have an enquiry about obtaining additional rights to this publication, please contact the address below or your local IEC member National Committee for further information.

IEC Central Office
3, rue de Varembe
CH-1211 Geneva 20
Switzerland

Tel.: +41 22 919 02 11
Fax: +41 22 919 03 00
info@iec.ch
www.iec.ch

About the IEC

The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

About IEC publications

The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

IEC Catalogue - webstore.iec.ch/catalogue

The stand-alone application for consulting the entire bibliographical information on IEC International Standards, Technical Specifications, Technical Reports and other documents. Available for PC, Mac OS, Android Tablets and iPad.

IEC publications search - www.iec.ch/searchpub

The advanced search enables to find IEC publications by a variety of criteria (reference number, text, technical committee,...). It also gives information on projects, replaced and withdrawn publications.

IEC Just Published - webstore.iec.ch/justpublished

Stay up to date on all new IEC publications. Just Published details all new publications released. Available online and also once a month by email.

Electropedia - www.electropedia.org

The world's leading online dictionary of electronic and electrical terms containing 20 000 terms and definitions in English and French, with equivalent terms in 16 additional languages. Also known as the International Electrotechnical Vocabulary (IEV) online.

IEC Glossary - std.iec.ch/glossary

65 000 electrotechnical terminology entries in English and French extracted from the Terms and Definitions clause of IEC publications issued since 2002. Some entries have been collected from earlier publications of IEC TC 37, 77, 86 and CISPR.

IEC Customer Service Centre - webstore.iec.ch/csc

If you wish to give us your feedback on this publication or need further assistance, please contact the Customer Service Centre: csc@iec.ch.



IEC 62680-3-1

Edition 1.0 2017-03

INTERNATIONAL STANDARD



**Universal Serial Bus interfaces for data and power –
Part 3-1: Universal Serial Bus 3.1 Specification**

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

ICS 29.220; 35.200

ISBN 978-2-8322-3913-1

Warning! Make sure that you obtained this publication from an authorized distributor.

INTERNATIONAL ELECTROTECHNICAL COMMISSION

UNIVERSAL SERIAL BUS INTERFACES FOR DATA AND POWER –

Part 3-1: Universal Serial Bus 3.1 Specification

FOREWORD

- 1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as "IEC Publication(s)"). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
- 3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
- 4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
- 5) IEC itself does not provide any attestation of conformity. Independent certification bodies provide conformity assessment services and, in some areas, access to IEC marks of conformity. IEC is not responsible for any services carried out by independent certification bodies.
- 6) All users should ensure that they have the latest edition of this publication.
- 7) No liability shall attach to IEC or its directors, employees, servants or agents including individual experts and members of its technical committees and IEC National Committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this IEC Publication or any other IEC Publications.
- 8) Attention is drawn to the Normative references cited in this publication. Use of the referenced publications is indispensable for the correct application of this publication.
- 9) Attention is drawn to the possibility that some of the elements of this IEC Publication may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

International Standard IEC 62680-3-1 has been prepared by technical area 14: Interfaces and methods of measurement for personal computing equipment, of IEC technical committee 100: Audio, video and multimedia systems and equipment.

The text of this standard was prepared by the USB Implementers Forum (USB-IF). The structure and editorial rules used in this publication reflect the practice of the organization which submitted it.

The text of this standard is based on the following documents:

CDV	Report on voting
100/2589/CDV	100/2684/RVC

Full information on the voting for the approval of this International Standard can be found in the report on voting indicated in the above table.

A list of all parts in the IEC 62680 series, published under the general title *Universal serial bus interfaces for data and power*, can be found on the IEC website.

The committee has decided that the contents of this document will remain unchanged until the stability date indicated on the IEC website under "<http://webstore.iec.ch>" in the data related to the specific document. At this date, the document will be

- reconfirmed,
- withdrawn,
- replaced by a revised edition, or
- amended.

IMPORTANT – The 'colour inside' logo on the cover page of this publication indicates that it contains colours which are considered to be useful for the correct understanding of its contents. Users should therefore print this document using a colour printer.

INTRODUCTION

The IEC 62680 series is based on a series of specifications that were originally developed by the USB Implementers Forum (USB-IF). These specifications were submitted to the IEC under the auspices of a special agreement between the IEC and the USB-IF.

The USB Implementers Forum, Inc. (USB-IF) is a non-profit corporation founded by the group of companies that developed the Universal Serial Bus specification. The USB-IF was formed to provide a support organization and forum for the advancement and adoption of Universal Serial Bus technology. The Forum facilitates the development of high-quality compatible USB peripherals (devices), and promotes the benefits of USB and the quality of products that have passed compliance testing.

ANY USB SPECIFICATIONS ARE PROVIDED TO YOU "AS IS", WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE. THE USB IMPLEMENTERS FORUM AND THE AUTHORS OF ANY USB SPECIFICATIONS DISCLAIM ALL LIABILITY, INCLUDING LIABILITY FOR INFRINGEMENT OF ANY PROPRIETARY RIGHTS, RELATING TO USE OR IMPLEMENTATION OR INFORMATION IN THIS SPECIFICATION.

THE PROVISION OF ANY USB SPECIFICATIONS TO YOU DOES NOT PROVIDE YOU WITH ANY LICENCE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS.

Entering into USB Adopters Agreements may, however, allow a signing company to participate in a reciprocal, RAND-Z licensing arrangement for compliant products. For more information, please see:

<http://www.usb.org/developers/docs/>

http://www.usb.org/developers/devclass_docs#approved

IEC DOES NOT TAKE ANY POSITION AS TO WHETHER IT IS ADVISABLE FOR YOU TO ENTER INTO ANY USB ADOPTERS AGREEMENTS OR TO PARTICIPATE IN THE "USB IMPLEMENTERS FORUM".

Universal Serial Bus 3.1 Specification

Hewlett-Packard Company

Intel Corporation

Microsoft Corporation

Renesas Corporation

ST-Ericsson

Texas Instruments

Revision 1.0

July 26, 2013

Revision History

Revision	Comments	Issue Date
1.0	Initial release. USB 3.0	November 12, 2008
	Incorporated errata and ECNs	June 6, 2011
1.0	Initial release. USB 3.1	July 26, 2013

INTELLECTUAL PROPERTY DISCLAIMER

THIS SPECIFICATION IS PROVIDED TO YOU “AS IS” WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE. THE AUTHORS OF THIS SPECIFICATION DISCLAIM ALL LIABILITY, INCLUDING LIABILITY FOR INFRINGEMENT OF ANY PROPRIETARY RIGHTS, RELATING TO USE OR IMPLEMENTATION OF INFORMATION IN THIS SPECIFICATION. THE PROVISION OF THIS SPECIFICATION TO YOU DOES NOT PROVIDE YOU WITH ANY LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS.

Please send comments to techsup@usb.org

For industry information, refer to the USB Implementers Forum web page at <http://www.usb.org>

All product names are trademarks, registered trademarks, or servicemarks of their respective owners.

Copyright © 2007-2013, Hewlett-Packard Company, Intel Corporation, Microsoft Corporation, Renesas Corporation, ST-Ericsson, and Texas Instruments.

All rights reserved.

Acknowledgement of Technical Contribution

Dedication

Dedicated to the memory of Brad Hosler, the impact of whose accomplishments made the Universal Serial Bus one of the most successful technology innovations of the Personal Computer era.

The authors of this specification would like to recognize the following people who participated in the USB 3.0 Bus Specification technical workgroups. We would also like to acknowledge the many others throughout the industry who provided feedback and contributed to the development of this specification.

Promoter Company Employees

Alan Berkema	Hewlett-Packard Company
Walter Fry	Hewlett-Packard Company
Anthony Hudson	Hewlett-Packard Company
David Roderick	Hewlett-Packard Company
Kok Hong Chan	Intel Corporation
Huimin Chen	Intel Corporation
Bob Dunstan	Intel Corporation
Dan Froelich	Intel Corporation
Howard Heck	Intel Corporation
Brad Hosler	Intel Corporation
John Howard	Intel Corporation
Rahman Ismail	Intel Corporation
John Keys	Intel Corporation
Yun Ling	Intel Corporation
Andy Martwick	Intel Corporation
Steve McGowan	Intel Corporation
Ramin Neshati	Intel Corporation
Duane Quiet	Intel Corporation
Jeff Ravencraft	Intel Corporation
Brad Saunders	Intel Corporation
Joe Schaefer	Intel Corporation
Sarah Sharp	Intel Corporation
Micah Sheller	Intel Corporation
Gary Solomon	Intel Corporation
Karthi Vadivelu	Intel Corporation
Clint Walker	Intel Corporation
Jim Walsh	Intel Corporation
Randy Aull	Microsoft Corporation
Fred Bhesania	Microsoft Corporation
Martin Borge	Microsoft Corporation
Jim Bovee	Microsoft Corporation
Stephen Cooper	Microsoft Corporation
Lars Giusti	Microsoft Corporation
Robbie Harris	Microsoft Corporation

Allen Marshall	Microsoft Corporation
Kiran Muthabatulla	Microsoft Corporation
Tomas Perez-Rodriguez	Microsoft Corporation
Mukund Sankaranarayan	Microsoft Corporation
Nathan Sherman	Microsoft Corporation
Glen Slick	Microsoft Corporation
David Wooten	Microsoft Corporation
Rob Young	Microsoft Corporation
Nobuo Furuya	NEC Corporation
Hiroshi Kariya	NEC Corporation
Masami Katagiri	NEC Corporation
Yuichi Mizoguchi	NEC Corporation
Kats Nakazawa	NEC Corporation
Nobuyuki Mizukoshi	NEC Corporation
Yutaka Noguchi	NEC Corporation
Hajime Nozaki	NEC Corporation
Kenji Oguma	NEC Corporation
Satoshi Ohtani	NEC Corporation
Takanori Saeki	NEC Corporation
Eiji Sakai	NEC Corporation
Hiro Sakamoto	NEC Corporation
Hajime Sakuma	NEC Corporation
Makoto Sato	NEC Corporation
Hock Seow	NEC Corporation
"Peter" Chu Tin Teng	NEC Corporation
Yoshiyuki Tomoda	NEC Corporation
Satomi Yamauchi	NEC Corporation
Yoshiyuki Yamada	NEC Corporation
Susumu Yasuda	NEC Corporation
Alan Chang	ST-NXP Wireless
Wing Yan Chung	ST-NXP Wireless
Socol Constantin	ST-NXP Wireless
Knud Holtvoeth	NXP Semiconductors, B.V.
Linus Kerk	ST-NXP Wireless
Martin Klein	NXP Semiconductors, B.V.
Geert Knapen	NXP Semiconductors, B.V.
Chee Ee Lee	ST-NXP Wireless
Christian Paquet	NXP Semiconductors, B.V.
Veerappan Rajaram	ST-NXP Wireless
Shaun Reemeyer	ST-NXP Wireless
Dave Sroka	ST-NXP Wireless
Chee-Yen TEE	ST-NXP Wireless
Jerome Tjia	ST-NXP Wireless
Bart Vertenten	NXP Semiconductors, B.V.
Hock Meng Yeo	ST-NXP Wireless
Olivier Alavoine	Texas Instruments.

David Arciniega	Texas Instruments
Richard Baker	Texas Instruments
Sujoy Chakravarty	Texas Instruments
T. Y. Chan	Texas Instruments
Romit Dasgupta	Texas Instruments.
Alex Davidson	Texas Instruments
Eric Desmarchelier	Texas Instruments
Christophe Gautier	Texas Instruments
Dan Harmon	Texas Instruments
Will Harris	Texas Instruments
Richard Hubbard	Texas Instruments
Ivo Huber	Texas Instruments
Scott Kim	Texas Instruments
Grant Ley	Texas Instruments
Karl Muth	Texas Instruments
Lee Myers	Texas Instruments
Julie Nirchi	Texas Instruments
Wes Ray	Texas Instruments
Matthew Rowley	Texas Instruments
Bill Sherry	Texas Instruments
Mitsuru Shimada	Texas Instruments
James Skidmore	Texas Instruments
Yoram Solomon	Texas Instruments.
Sue Vining	Texas Instruments
Jin-sheng Wang	Texas Instruments
Roy Wojciechowski	Texas Instruments

Contributor Company Employees

Glen Chandler	Acon
John Chen	Acon
Roger Hou	Acon
Charles Wang	Acon
Norman Wu	Acon
Steven Yang	Acon
George Yee	Acon
George Olear	Contech Research
Sophia Liu	Electronics Testing Center, Taiwan (ETC)
William Northey	FCI
Tom Sultzer	FCI
Garry Biddle	Foxconn
Kuan-Yu Chen	Foxconn
Jason Chou	Foxconn
Gustavo Duenas	Foxconn
Bob Hall	Foxconn
Jiayong He	Foxconn

Jim Koser	Foxconn
Joe Ortega	Foxconn
Ash Raheja	Foxconn
James Sabo	Foxconn
Pei Tsao	Foxconn
Kevin Walker	Foxconn
Tsuneki Watanabe	Foxconn
Chong Yi	Foxconn
Taro Hishinuma	Hirose Electric
Kaz Ichikawa	Hirose Electric
Ryozo Koyama	Hirose Electric
Karl Kwiat	Hirose Electric
Tadashi Sakaizawa	Hirose Electric
Shinya Tono	Hirose Electric
Eiji Wakatsuki	Hirose Electric
Takashi Ehara	Japan Aviation Electronics Industry Ltd. (JAE)
Ron Muir	Japan Aviation Electronics Industry Ltd. (JAE)
Kazuhiro Saito	Japan Aviation Electronics Industry Ltd. (JAE)
Hitoshi Kawamura	Mitsumi
Takashi Kawasaki	Mitsumi
Atsushi Nishio	Mitsumi
Yasuhiko Shinohara	Mitsumi
Tom Lu	Molex Inc.
Edmund Poh	Molex Inc.
Scott Sommers	Molex Inc.
Jason Squire	Molex Inc.
Dat Ba Nguyen	NTS/National Technical System
Jan Fahllund	Nokia
Richard Petrie	Nokia
Panu Ylihaavisto	Nokia
Martin Furuholm	Seagate Technology LLC
Julian Gorfajn	Seagate Technology LLC
Marc Hildebrandt	Seagate Technology LLC
Tony Priborsky	Seagate Technology LLC
Harold To	Seagate Technology LLC
Robert Lefferts	Synopsys, Inc.
Saleem Mohammad	Synopsys, Inc.
Matthew Myers	Synopsys, Inc.
Daniel Weinlander	Synopsys, Inc.
Mike Engbretson	Tektronix, Inc.
Thomas Grzysiewicz	Tyco Electronics
Masaaki Iwasaki	Tyco Electronics
Kazukiyo Osada	Tyco Electronics
Hiroshi Shirai	Tyco Electronics
Scott Shuey	Tyco Electronics
Masaru Ueno	Tyco Electronics

The authors of this specification would like to recognize the following people who participated in the USB 3.1 Bus Specification technical workgroups. Additionally we would like to acknowledge the many others throughout industry who provided feedback and contributed to the development of this specification.

Promoter Company Employees

Alan Berkema	Hewlett Packard
Norton Ewart	Hewlett Packard
Monji Jabori	Hewlett Packard
Rahul Lakdawala	Hewlett Packard
Jim Mann	Hewlett Packard
Linden McClure	Hewlett Packard
Mike Bell	Intel Corporation
Huimin Chen	Intel Corporation
Kuan-Yu Chen	Intel Corporation
Bob Dunstan	Intel Corporation
Benjamin Graniello	Intel Corporation
Howard Heck	Intel Corporation
John Howard	Intel Corporation
Rahman Ismail	Intel Corporation
Yun Ling	Intel Corporation
Steve McGowan	Intel Corporation
Sridharan Ranganathan	Intel Corporation
Kaleb Ruof	Intel Corporation
Brad Saunders	Intel Corporation
Sarah Sharp	Intel Corporation
Ronald Swartz	Intel Corporation
Jennifer Tsai	Intel Corporation
Karthi Vadivelu	Intel Corporation
Randy Aull	Microsoft Corporation
Vivek Gupta	Microsoft Corporation
Toby Nixon	Microsoft Corporation
Yang You	Microsoft Corporation
Nobuo Furuya	Renesas Electronics Corp.
Masami Katagiri	Renesas Electronics Corp.
Steven Kawamoto	Renesas Electronics Corp.
Kiichi Muto	Renesas Electronics Corp.
Peter Teng	Renesas Electronics Corp.
Hicham Bouzekri	ST-Ericsson
Morten Christiansen	ST-Ericsson
Grant Ley	Texas Instruments
James Skidmore	Texas Instruments
Sue Vining	Texas Instruments
Tod Wolf	Texas Instruments
Li Yang	Texas Instruments

Contributor Company Employees

Jason Chen	Aces Electronics Co., Ltd.
Andy Feng	Aces Electronics Co., Ltd.
Chris Kao	Aces Electronics Co., Ltd.
Glen Chandler	ACON, Advanced-Connectek, Inc.
Alan MacDougall	ACON, Advanced-Connectek, Inc.
Shadi Barakat	Advanced Micro Devices
Walter Fry	Advanced Micro Devices
Will Harris	Advanced Micro Devices
Jason Hawken	Advanced Micro Devices
Hugo Lamarche	Advanced Micro Devices
Yufei Ma	Advanced Micro Devices
Joseph Scanlon	Advanced Micro Devices
Vishant Tyagi	Advanced Micro Devices
Min Wang	Advanced Micro Devices
James Choate	Agilent Technologies, Inc.
Thorsten Goetzmann	Agilent Technologies, Inc.
Takuya Hirato	Agilent Technologies, Inc.
Hiroshi Kanda	Agilent Technologies, Inc.
Donald Schoenecker	Agilent Technologies, Inc.
Chi Chang	ASMedia Technology Inc.
Chin Chang	ASMedia Technology Inc.
Chiahsin Chen	ASMedia Technology Inc.
Weber Chuang	ASMedia Technology Inc.
Ming-Wei Hsu	ASMedia Technology Inc.
Han Sung Kuo	ASMedia Technology Inc.
ShuYu Lin	ASMedia Technology Inc.
Luke Peng	ASMedia Technology Inc.
Daniel Wei	ASMedia Technology Inc.
ShengChung Wu	ASMedia Technology Inc.
Ted Hsiao	Bizlink Technology, Inc.
Pete Burgers	DisplayLink (UK) Ltd.
Dan Ellis	DisplayLink (UK) Ltd.
Richard Petrie	DisplayLink (UK) Ltd.
Terry Little	Foxconn / Hon Hai
Steve Sedio	Foxconn / Hon Hai
Tim Barilovits	Fresco Logic Inc.
Bob McVay	Fresco Logic Inc.
Christopher Meyers	Fresco Logic Inc.
Jie Ni	Fresco Logic Inc.
Jeffrey Yang	Fresco Logic Inc.
Jing-Fan Zhang	Fresco Logic Inc.
Mike Engbretson	Granite River Labs
Kunia Aihara	Hirose Electric Co., Ltd.
Kazu Ichikawa	Hirose Electric Co., Ltd.
Masaru Kawamura	Hirose Electric Co., Ltd.

William MacKillop	Hirose Electric Co., Ltd.
Sho Nakamura	Hirose Electric Co., Ltd.
Toshiyuki Takada	Hirose Electric Co., Ltd.
Sid Tono	Hirose Electric Co., Ltd.
Tirumal Annamaneni	Intersil Corporation
Colby Keith	Intersil Corporation
Gourgen Oganessyan	Intersil Corporation
Michael Vrazel	Intersil Corporation
Toshiyuki Moritake	Japan Aviation Electronics Industry Ltd. (JAE)
Takeharu Naito	Japan Aviation Electronics Industry Ltd. (JAE)
Mark Saubert	Japan Aviation Electronics Industry Ltd. (JAE)
Toshio Shimoyama	Japan Aviation Electronics Industry Ltd. (JAE)
Takamitsu Wada	Japan Aviation Electronics Industry Ltd. (JAE)
Roy Chestnut	LeCroy Corporation
Christopher Forker	LeCroy Corporation
Linden Hsu	LeCroy Corporation
Daniel H Jacobs	LeCroy Corporation
David Li	LeCroy Corporation
Mike Micheletti	LeCroy Corporation
Michael Romm	LeCroy Corporation
Chris Webb	LeCroy Corporation
Tomoki Harada	Lenovo
Ariel Delos Reyes	Lotes Co., Ltd.
Smark Huo	Lotes Co., Ltd.
Regina Liu-Hwang	Lotes Co., Ltd.
Harvey Newman	LSI Corporation
Dave Thompson	LSI Corporation
Srinivas Vura	LSI Corporation
Josue Castillo	Luxshare-ICT
Alan Kinningham	Luxshare-ICT
John Lin	Luxshare-ICT
Stone Lin	Luxshare-ICT
Pat Young	Luxshare-ICT
John Garney	MCCI Corporation
Peter Harrison	Nokia Corporation
Mika Tolvanen	Nokia Corporation
Panu Ylihaavisto	Nokia Corporation
Jason Chen	NXP Semiconductors
Gerrit den Besten	NXP Semiconductors
Bart Vertenten	NXP Semiconductors
Ho Wai Wong-Lam	NXP Semiconductors
Jagoun Koo	Samsung Electronics Co., Ltd.
Cheolho Lee	Samsung Electronics Co., Ltd.
Jun Bum Lee	Samsung Electronics Co., Ltd.
Alvin Cox	Seagate Technology LLC
Steven Davis	Seagate Technology LLC

Bahar Ghaffari	Seagate Technology LLC
Henry (John) Hein	Seagate Technology LLC
Tony Priborsky	Seagate Technology LLC
Tom Skaar	Seagate Technology LLC
Dan Smith	Seagate Technology LLC
Mark Bohm	SMSC
Jerome DeRoo	STMicroelectronics
Benoit Mercier	STMicroelectronics
Subramaniam Aravindhan	Synopsys, Inc.
Bala Babu	Synopsys, Inc.
Sanjay Dave	Synopsys, Inc.
Gervais Fong	Synopsys, Inc.
Kevin Heilman	Synopsys, Inc.
Eric Huang	Synopsys, Inc.
Behram Minwalla	Synopsys, Inc.
Saleem Mohammad	Synopsys, Inc.
Matthew Myers	Synopsys, Inc.
Tri Nguyen	Synopsys, Inc.
John Stonick	Synopsys, Inc.
Zongyao Wen	Synopsys, Inc.
Paul Wyborny	Synopsys, Inc.
Sarah Boen	Tektronix, Inc.
Darren Gray	Tektronix, Inc.
Srikrishna N.H.	Tektronix, Inc.
Randy White	Tektronix, Inc.
Jim McGrath	Tyco Electronics Corp., a TE Connectivity Ltd. company
Josh Moody	Tyco Electronics Corp., a TE Connectivity Ltd. company
Scott Shuey	Tyco Electronics Corp., a TE Connectivity Ltd. company
Egbert Stellinga	Tyco Electronics Corp., a TE Connectivity Ltd. company
Noah Zhang	Tyco Electronics Corp., a TE Connectivity Ltd. company
Marvin DeForest	Western Digital Technologies, Inc.
Larry McMillan	Western Digital Technologies, Inc.
Cristian Roman Del Nido	Western Digital Technologies, Inc.
Curtis Stevens	Western Digital Technologies, Inc.

CONTENTS

FOREWORD.....	2
INTRODUCTION.....	4
1 Introduction	38
1.1 Background	38
1.2 Objective of the Specification.....	38
1.3 Scope of the Document.....	38
1.4 USB Product Compliance.....	39
1.5 Document Organization	39
1.6 Design Goals	39
1.7 Related Documents.....	39
2 Terms and Abbreviations	40
3 Architectural Overview.....	48
3.1 USB 3.1 System Description	49
3.1.1 USB 3.1 Physical Interface	50
3.1.2 USB 3.1 Power.....	50
3.1.3 USB 3.1 System Configuration.....	51
3.1.4 USB 3.1 Architecture Summary	51
3.2 Enhanced SuperSpeed Bus Architecture.....	51
3.2.1 Physical Layer.....	53
3.2.2 Link Layer.....	54
3.2.3 Protocol Layer	55
3.2.4 Robustness	57
3.2.5 Enhanced SuperSpeed Power Management	57
3.2.6 Devices	58
3.2.7 Hosts.....	62
3.3 Enhanced SuperSpeed Bus Data Flow Models.....	63
4 Enhanced SuperSpeed Data Flow Model.....	63
4.1 Implementer Viewpoints.....	63
4.2 Enhanced SuperSpeed Communication Flow	64
4.2.1 Pipes	64
4.3 Enhanced SuperSpeed Protocol Overview	64
4.3.1 Differences from USB 2.0	64
4.4 Generalized Transfer Description.....	66
4.4.1 Data Bursting.....	67
4.4.2 IN Transfers	67
4.4.3 OUT Transfers.....	68
4.4.4 Power Management and Performance	69
4.4.5 Control Transfers.....	69
4.4.6 Bulk Transfers	71
4.4.7 Interrupt Transfers.....	74
4.4.8 Isochronous Transfers	75
4.4.9 Device Notifications	79
4.4.10 Reliability	79
4.4.11 Efficiency.....	79
5 Mechanical	80
5.1 Objective	80

5.2	Significant Features	80
5.2.1	Connectors	80
5.2.2	Allowed Cable Assemblies.....	82
5.2.3	Raw Cables	82
5.3	Connector Mating Interfaces	82
5.3.1	USB 3.1 Standard-A Connector	82
5.3.2	USB 3.1 Standard-B Connector	98
5.3.3	USB 3.1 Micro Connector Family	103
5.4	Cable Construction and Wire Assignments.....	111
5.4.1	Cable Construction	111
5.4.2	Wire Assignments.....	112
5.4.3	Wire Gauges and Cable Diameters	113
5.5	Cable Assemblies	113
5.5.1	USB 3.1 Standard-A to USB 3.1 Standard-B Cable Assembly	113
5.5.2	USB 3.1 Standard-A to USB 3.1 Standard-A Cable Assembly	115
5.5.3	USB 3.1 Standard-A to USB 3.1 Micro-B Cable Assembly.....	115
5.5.4	USB 3.1 Micro-A to USB 3.1 Micro-B Cable Assembly	117
5.5.5	USB 3.1 Micro-A to USB 3.1 Standard-B Cable Assembly	119
5.5.6	USB 3.1 Icon Location	120
5.5.7	Cable Assembly Length	120
5.6	Electrical Requirements	120
5.6.1	Enhanced SuperSpeed Electrical Requirements	121
5.6.2	DC Electrical Requirements	130
5.7	Mechanical and Environmental Requirements	130
5.7.1	Mechanical Requirements.....	131
5.7.2	Environmental Requirements	134
5.7.3	Materials	134
5.8	Implementation Notes and Design Guides.....	135
5.8.1	Mated Connector Dimensions	135
5.8.2	EMI and RFI Management	137
5.8.3	Stacked Connectors	138
6	Physical Layer.....	139
6.1	Physical Layer Overview	139
6.2	Physical Layer Functions	139
6.2.1	Measurement Overview	143
6.2.2	Channel Overview	143
6.3	Symbol Encoding	144
6.3.1	Gen 1 Encoding.....	144
6.3.2	Gen 2 Encoding.....	146
6.3.3	Special Symbols for Framing and Link Management.....	150
6.4	Link Initialization and Training.....	151
6.4.1	Link Training.....	151
6.4.2	Lane Polarity Inversion	156
6.4.3	Elasticity Buffer and SKP Ordered Set.....	156
6.4.4	Compliance Pattern	158
6.5	Clock and Jitter	159
6.5.1	Informative Jitter Budgeting	159
6.5.2	Normative Clock Recovery Function	160
6.5.3	Normative Spread Spectrum Clocking (SSC)	162

6.5.4	Normative Slew Rate Limit.....	163
6.6	Signaling.....	163
6.6.1	Eye Diagrams.....	163
6.6.2	Voltage Level Definitions	164
6.6.3	Tx and Rx Input Parasitics.....	165
6.7	Transmitter Specifications.....	165
6.7.1	Transmitter Electrical Parameters	165
6.7.2	Low Power Transmitter	167
6.7.3	Transmitter Eye	167
6.7.4	Tx Compliance Reference Receiver Equalize Function	168
6.7.5	Informative Transmitter De-emphasis	168
6.7.6	Entry into Electrical Idle, U1	170
6.8	Receiver Specifications.....	170
6.8.1	Receiver Equalization Training	170
6.8.2	Informative Receiver CTLE Function.....	171
6.8.3	Receiver Electrical Parameters.....	174
6.8.4	Receiver Loopback	175
6.8.5	Normative Receiver Tolerance Compliance Test.....	176
6.9	Low Frequency Periodic Signaling (LFPS)	178
6.9.1	LFPS Signal Definition.....	178
6.9.2	Example LFPS Handshake for U1/U2 Exit, Loopback Exit, and U3 Wakeup	180
6.9.3	Warm Reset.....	182
6.9.4	SuperSpeedPlus Capability Declaration.....	183
6.9.5	SuperSpeedPlus LFPS Based PWM Message (LBPM).....	184
6.10	Transmitter and Receiver DC Specifications	186
6.10.1	Informative ESD Protection.....	186
6.10.2	Informative Short Circuit Requirements	186
6.10.3	Normative High Impedance Reflections	186
6.11	Receiver Detection	186
6.11.1	Rx Detect Overview	186
6.11.2	Rx Detect Sequence	187
6.11.3	Upper Limit on Channel Capacitance	187
6.12	Retimers	188
7	Link Layer	188
7.1	Byte Ordering	189
7.1.1	SuperSpeed USB Line Code.....	189
7.1.2	SuperSpeedPlus USB Line Code	189
7.2	Link Management and Flow Control	189
7.2.1	Packets and Packet Framing	190
7.2.2	Link Commands.....	198
7.2.3	Logical Idle.....	202
7.2.4	Link Command Usage for Flow Control, Error Recovery, and Power Management.....	202
7.3	Link Error Rules/Recovery	218
7.3.1	Overview of Enhanced SuperSpeed Bit Errors	218
7.3.2	Link Error Types, Detection, and Recovery	218
7.3.3	Link Error Statistics	218
7.3.4	Header Packet Errors	219

7.3.5	Link Command Errors	220
7.3.6	ACK Tx Header Sequence Number Error	221
7.3.7	Header Sequence Number Advertisement Error	222
7.3.8	SuperSpeed Rx Header Buffer Credit Advertisement Error	222
7.3.9	SuperSpeedPlus Type 1/Type 2 Rx Buffer Credit Advertisement Error	223
7.3.10	Training Sequence Error	223
7.3.11	SuperSpeed 8b/10b Errors	224
7.3.12	SuperSpeedPlus Block Header Errors	224
7.3.13	Summary of Error Types and Recovery	224
7.4	PowerOn Reset and Inband Reset	226
7.4.1	PowerOn Reset	226
7.4.2	Inband Reset	227
7.5	Link Training and Status State Machine (LTSSM)	228
7.5.1	eSS.Disabled	230
7.5.2	eSS.Inactive	232
7.5.3	Rx.Detect	233
7.5.4	Polling	236
7.5.5	Compliance Mode	247
7.5.6	U0	248
7.5.7	U1	249
7.5.8	U2	250
7.5.9	U3	251
7.5.10	Recovery	252
7.5.11	Loopback	256
7.5.12	Hot Reset	258
8	Protocol Layer	260
8.1	Enhanced SuperSpeed Transactions	261
8.1.1	Transactions on a SuperSpeed Bus Instance	261
8.1.2	Transactions on a SuperSpeedPlus Bus Instance	262
8.2	Packet Types	263
8.3	Packet Formats	263
8.3.1	Fields Common to all Headers	263
8.4	Link Management Packet (LMP)	265
8.4.1	Subtype Field	265
8.4.2	Set Link Function	266
8.4.3	U2 Inactivity Timeout	267
8.4.4	Vendor Device Test	267
8.4.5	Port Capabilities	268
8.4.6	Port Configuration	270
8.4.7	Port Configuration Response	270
8.4.8	Precision Time Measurement	271
8.5	Transaction Packet (TP)	284
8.5.1	Acknowledgement (ACK) Transaction Packet	284
8.5.2	Not Ready (NRDY) Transaction Packet	287
8.5.3	Endpoint Ready (ERDY) Transaction Packet	288
8.5.4	STATUS Transaction Packet	289
8.5.5	STALL Transaction Packet	289
8.5.6	Device Notification (DEV_NOTIFICATION) Transaction Packet	289
8.5.7	PING Transaction Packet	296

8.5.8	PING_RESPONSE Transaction Packet.....	296
8.6	Data Packet (DP).....	297
8.7	Isochronous Timestamp Packet (ITP).....	299
8.8	Addressing Triple.....	300
8.9	Route String Field.....	300
8.9.1	Route String Port Field	300
8.9.2	Route String Port Field Width	300
8.9.3	Port Number	300
8.10	Transaction Packet Usages	300
8.10.1	Flow Control Conditions.....	301
8.10.2	Burst Transactions.....	301
8.10.3	Short Packets	303
8.10.4	SuperSpeedPlus Transaction Reordering	303
8.11	TP or DP Responses	305
8.11.1	Device Response to TP Requesting Data	306
8.11.2	Host Response to Data Received from a Device	306
8.11.3	Device Response to Data Received from the Host	307
8.11.4	Device Response to a SETUP DP.....	308
8.12	TP Sequences	309
8.12.1	Bulk Transactions	309
8.12.2	Control Transfers.....	338
8.12.3	Bus Interval and Service Interval	341
8.12.4	Interrupt Transactions.....	341
8.12.5	Host Timing Information.....	347
8.12.6	Isochronous Transactions.....	348
8.13	Timing Parameters.....	360
9	Device Framework.....	363
9.1	USB Device States	363
9.1.1	Visible Device States	363
9.1.2	Bus Enumeration	368
9.2	Generic Device Operations	369
9.2.1	Dynamic Attachment and Removal	369
9.2.2	Address Assignment.....	369
9.2.3	Configuration	369
9.2.4	Data Transfer	370
9.2.5	Power Management.....	370
9.2.6	Request Processing.....	372
9.2.7	Request Error	374
9.3	USB Device Requests.....	374
9.3.1	bmRequestType	374
9.3.2	bRequest.....	375
9.3.3	wValue	375
9.3.4	wIndex.....	375
9.3.5	wLength.....	375
9.4	Standard Device Requests.....	376
9.4.1	Clear Feature	379
9.4.2	Get Configuration	379
9.4.3	Get Descriptor	380
9.4.4	Get Interface	381

9.4.5	Get Status	381
9.4.6	Set Address	384
9.4.7	Set Configuration	385
9.4.8	Set Descriptor	385
9.4.9	Set Feature	386
9.4.10	Set Interface	387
9.4.11	Set Isochronous Delay	388
9.4.12	Set SEL	388
9.4.13	Synch Frame	389
9.4.14	Events and Their Effect on Device Parameters	389
9.5	Descriptors	390
9.6	Standard USB Descriptor Definitions	391
9.6.1	Device	391
9.6.2	Binary Device Object Store (BOS)	394
9.6.3	Configuration	402
9.6.4	Interface Association	403
9.6.5	Interface	404
9.6.6	Endpoint	406
9.6.7	SuperSpeed Endpoint Companion	409
9.6.8	SuperSpeedPlus Isochronous Endpoint Companion	411
9.6.9	String	412
9.7	Device Class Definitions	412
9.7.1	Descriptors	412
9.7.2	Interface(s)	413
9.7.3	Requests	413
10	Hub, Host Downstream Port, and Device Upstream Port Specification	413
10.1	Hub Feature Summary	413
10.1.1	Connecting to an Enhanced SuperSpeed Capable Host	417
10.1.2	Connecting to a USB 2.0 Host	417
10.1.3	Hub Connectivity	418
10.1.4	Resume Connectivity	420
10.1.5	Hub Fault Recovery Mechanisms	421
10.1.6	Hub Buffer Architecture	421
10.2	Hub Power Management	423
10.2.1	Link States	423
10.2.2	Hub Downstream Port U1/U2 Timers	423
10.2.3	Downstream/Upstream Port Link State Transitions	423
10.3	Hub Downstream Facing Ports	424
10.3.1	Hub Downstream Facing Port State Descriptions	427
10.3.2	Disconnect Detect Mechanism	431
10.3.3	Labeling	432
10.4	Hub Downstream Facing Port Power Management	432
10.4.1	Downstream Facing Port PM Timers	432
10.4.2	Hub Downstream Facing Port State Descriptions	433
10.5	Hub Upstream Facing Port	436
10.5.1	Upstream Facing Port State Descriptions	437
10.5.2	Hub Connect State Machine	439
10.6	Upstream Facing Port Power Management	440
10.6.1	Upstream Facing Port PM Timer	441

10.6.2	Hub Upstream Facing Port State Descriptions	442
10.7	SuperSpeed Hub Header Packet Forwarding and Data Repeater	444
10.7.1	SuperSpeed Hub Elasticity Buffer	445
10.7.2	SKP Ordered Sets	445
10.7.3	Interpacket Spacing	445
10.7.4	SuperSpeed Header Packet Buffer Architecture	445
10.7.5	SuperSpeed Packet Connectivity	446
10.8	SuperSpeedPlus Store and Forward Behavior	447
10.8.1	Hub Elasticity Buffer	447
10.8.2	SKP Ordered Sets	447
10.8.3	Interpacket Spacing	447
10.8.4	Upstream Flowing Buffering	447
10.8.5	Downstream Flowing Buffering	448
10.8.6	SuperSpeedPlus Hub Arbitration of Packets	449
10.8.7	SuperSpeedPlus Upstream Flowing Packet Modifications	451
10.8.8	SuperSpeedPlus Downstream Controller	452
10.9	Port State Machines	452
10.9.1	Port Transmit State Machine	452
10.9.2	Port Transmit State Descriptions	453
10.9.3	Port Receive State Machine	454
10.9.4	Port Receive State Descriptions	455
10.10	Suspend and Resume	460
10.11	Hub Upstream Port Reset Behavior	460
10.12	Hub Port Power Control	461
10.12.1	Multiple Gangs	461
10.13	Hub Controller	461
10.13.1	Endpoint Organization	462
10.13.2	Hub Information Architecture and Operation	462
10.13.3	Port Change Information Processing	463
10.13.4	Hub and Port Status Change Bitmap	464
10.13.5	Over-current Reporting and Recovery	465
10.13.6	Enumeration Handling	466
10.14	Hub Configuration	466
10.15	Descriptors	467
10.15.1	Standard Descriptors for Hub Class	468
10.15.2	Class-specific Descriptors	473
10.16	Requests	475
10.16.1	Standard Requests	475
10.16.2	Class-specific Requests	476
10.17	Host Root (Downstream) Ports	493
10.18	Peripheral Device Upstream Ports	493
10.18.1	Peripheral Device Upstream Ports	493
10.18.2	Peripheral Device Upstream Port State Machine	494
10.19	Hub Chapter Parameters	496
11	Interoperability and Power Delivery	498
11.1	USB 3.1 Host Support for USB 2.0	499
11.2	USB 3.1 Hub Support for USB 2.0	499
11.3	USB 3.1 Device Support for USB 2.0	500
11.4	Power Distribution	500

11.4.1	Classes of Devices and Connections	500
11.4.2	Steady-State Voltage Drop Budget	503
11.4.3	Power Control During Suspend/Resume	504
11.4.4	Dynamic Attach and Detach.....	505
11.4.5	VBUS Electrical Characteristics	506
11.4.6	Powered-B Connector.....	506
11.4.7	Wire Gauge Table.....	506
A	Gen 1 Symbol Encoding	508
B	Symbol Scrambling.....	514
B.1.	Data Scrambling	514
C	Power Management.....	521
C.1.	SuperSpeed Power Management Overview	521
C.1.1.	Link Power Management	521
C.1.1.1.	Summary of Link States	521
C.1.1.2.	U0 – Link Active.....	522
C.1.1.3.	U1 – Link Idle with Fast Exit.....	522
C.1.1.3.1.	U1 Entry	522
C.1.1.3.2.	Exiting the U1 State.....	523
C.1.1.4.	U2 – Link Idle with Slow Exit.....	523
C.1.1.5.	U3 – Link Suspend.....	525
C.1.2.	Link Power Management for Downstream Ports	526
C.1.2.1.	Link State Coordination and Management.....	526
C.1.2.2.	Packet Deferring	526
C.1.2.3.	Software Interface.....	527
C.1.3.	Other Link Power Management Support Mechanisms	528
C.1.3.1.	Packets Pending Flag	528
C.1.3.2.	Support for Isochronous Transfers	529
C.1.3.3.	Support for Interrupt Transfers.....	529
C.1.4.	Device Power Management	529
C.1.4.1.	Function Suspend	529
C.1.4.2.	Device Suspend.....	530
C.1.4.3.	Host Initiated Suspend.....	530
C.1.4.4.	Host Initiated Wake from Suspend	530
C.1.4.5.	Device Initiated Wake from Suspend.....	531
C.1.5.	Platform Power Management Support.....	531
C.1.5.1.	System Exit Latency and BELT	532
C.1.5.2.	Maximum Exit Latency and PING	533
C.1.5.2.1.	Maximum Exit Latency t1 (tMEL1).....	533
C.1.5.2.2.	Maximum Exit Latency t2 (tMEL2).....	533
C.1.5.2.3.	Maximum Exit Latency t3 (tMEL3).....	533
C.1.5.2.4.	Maximum Exit Latency t4 (tMEL4).....	534
C.2.	Calculating U1 and U2 End to End Exit Latencies	534
C.2.1.	Device Connected Directly to Host.....	535
C.2.1.1.	Host Initiated Transition	535
C.2.1.2.	Device Initiated Transition.....	536

C.2.2.	Device Connected Through a Hub	536
C.2.2.1.	Host Initiated Transition	536
C.2.2.2.	Device Initiated Transition.....	538
C.3.	Device-Initiated Link Power Management Policies	539
C.3.1.	Overview and Background Information.....	539
C.3.2.	Entry Conditions for U1 and U2	539
C.3.2.1.	Control Endpoints	540
C.3.2.2.	Bulk Endpoints.....	540
C.3.2.3.	Interrupt Endpoints	540
C.3.2.4.	Isochronous Endpoints.....	541
C.3.2.5.	Devices That Need Timestamp Packets	541
C.4.	Latency Tolerance Message (LTM) Implementation Example	541
C.4.1.	Device State Machine Implementation Example	541
C.4.1.1.	LTM-Idle State BELT.....	542
C.4.1.2.	LTM-Active State BELT	542
C.4.1.3.	Transitioning Between LT-States	542
C.4.1.3.1.	Transitioning From LT-idle to LT-active.....	542
C.4.1.3.2.	Transitioning From LT-active to LT-idle.....	543
C.4.2.	Other Considerations.....	543
C.5.	SuperSpeed vs. High Speed Power Management Considerations	544
D	Example Packets	545
E	Repeaters.....	547
E.1.	Overview	547
E.1.1.	Definitions	547
E.1.2.	Scope	547
E.1.2.1.	Retimers	547
E.1.2.2.	Re-drivers	548
E.2.	Retimer Architectural Overview and Requirement	548
E.2.1.	Architectural Overview.....	549
E.2.2.	General requirements	549
E.2.2.1.	Physical Layer Requirements.....	549
E.2.2.2.	Link Layer Requirements	550
E.2.3.	Retimer Operation	550
E.3.	Retimer Training and Status State Machine (RTSSM)	551
E.3.1.	Rx.Detect	552
E.3.1.1.	Rx.Detect Requirement.....	552
E.3.1.2.	Exit from Rx.Detect.....	553
E.3.2.	eSS.Disabled.....	553
E.3.2.1.	eSS.Disabled Requirements	553
E.3.2.2.	Exit from eSS.Disabled	553
E.3.3.	Polling	553
E.3.3.1.	Polling.SpeedDetect	554
E.3.3.1.1.	Polling.SpeedDetect Requirements.....	554
E.3.3.1.2.	Exit from Polling.SpeedDetect	555
E.3.3.2.	Polling.PortConfig	555

E.3.3.2.1.	Polling.PortConfig Requirements	555
E.3.3.2.2.	Exit from Polling.PortConfig	556
E.3.3.3.	Polling.RxEQ	556
E.3.3.3.1.	Polling.RxEQ Requirements	556
E.3.3.3.2.	Exit from Polling.RxEQ	556
E.3.3.4.	Polling.TSx	556
E.3.3.4.1.	Polling.TSx Requirements	556
E.3.3.4.2.	Exit from Polling.TSx	557
E.3.3.5.	The tPollingConfigurationTimeout timer has expired.Polling.Idle	557
E.3.3.5.1.	Polling.Idle Requirements	557
E.3.3.5.2.	Exit from Polling.Idle	557
E.3.4.	Compliance Mode	558
E.3.5.	U0	558
E.3.5.1.	U0 Requirements	558
E.3.5.2.	Exit from U0	558
E.3.6.	U1	558
E.3.6.1.	U1 Requirements	559
E.3.6.2.	Exit from U1	559
E.3.7.	U2	559
E.3.7.1.	U2 Requirements	560
E.3.7.2.	Exit from U2	560
E.3.8.	U3	560
E.3.8.1.	U3 Requirements	560
E.3.8.2.	Exit from U3	561
E.3.9.	Recovery	561
E.3.10.	PassThrough Loopback	562
E.3.10.1.	PassThrough Loopback Requirements	562
E.3.10.2.	Exit from PassThrough Loopback	562
E.3.11.	Local Loopback	562
E.3.11.1.	Local Loopback Requirements	563
E.3.11.2.	Exit from Local Loopback.Active	563
E.3.11.3.	Exit from Local Loopback.Exit	563
E.3.12.	Hot Reset	563
E.3.12.1.	Hot Reset Requirements	563
E.3.12.2.	Exit from Hot Reset	563
E.4.	Clock Offset Compensation	564
E.4.1.	Gen 1 Operation	564
E.4.1.1.	Underflow Clock Offset Compensation	564
E.4.1.2.	Overflow Clock Offset Compensation	564
E.4.2.	Gen 2 Operation	565
E.5.	Compliance	565
E.5.1.	Host and Device Product Compliance	565
E.5.2.	Component-Level Retimer Compliance	565

Title: USB 3.1 CTLE	567
Applied to: USB_3_1r1.0_07_31_2013.....	567
Actual Change	568
(a) Section 6.4.4, Table 6-13, page 159	568
6.7.1Transmitter Electrical Parameters.....	570
6.7.1Transmitter Electrical Parameters.....	571
(c) Section 6.7.5, page 168	572
6.7.5Informative Transmitter De-emphasis	572
6.7.5.1Gen 1 (5GT/s)	572
6.7.5.2Gen 2 (10GT/s)	572
6.7.5Informative Transmitter De-emphasis	575
6.7.5.1Gen 1 (5GT/s)	575
6.7.5.2Gen 2 (10GT/s)	575
(d) Section 6.8.2.2.1, Figure 6-25, page 173	579
(e) Section 6.8.2.2.1, Figure 6-27, page 177	581
Title: HSEQ	583
Applied to: USB_3_1r1.0_07_31_2013.....	583
Actual Change	584
Title: USB 3.1 wHubDelay.....	591
Applied to: USB_3_1r1.0_07_31_2013.....	591
Actual Change	592
Table 10-19. Hub Parameters.....	592
Title: USB 3.1 LTM	593
Applied to: USB_3_1r1.0_07_31_2013.....	593
Actual Change	594
Title: Polling LFPS	597
Applied to: USB_3_1r1.0_07_31_2013.....	597
Actual Change	598
Title: USB 3.1 PTM Value	603
Applied to: USB_3_1r1.0_07_31_2013.....	603
Actual Change	604
Title: SKP OS Bytes During Compliance Clarification.....	605
Applied to: USB_3_1r1.0_07_31_2013.....	605
Actual Change	606
(a) From Text (and location): Table 6-12, Section 6.4.2, page158	606
(a) To Text (and location): Table 6-12, Section 6.4.2, page 158	606
(b) From Text (and location): Section 6.4.2, page 156	606
(c) From Text (and location): Section 6.4.2, page 156	606
Title: USB3.1 SKP Ordered Set Definition.....	607
Applied to: USB_3_1r1.0_07_31_2013.....	607
Actual Change	608
Section 6.3.2.2 Normative 128b/132b Decode Rules	608
Section 6.4.3 Elasticity Buffer and SKP Ordered Set.....	609
Section 6.4.3.2 SKP Rules (Host/Device/Hub) for Gen 2 Operation.....	610
6.8.4Receiver Loopback.....	613

Title: SLC IS SDS	614
Applied to: USB_3_1r1.0_07_31_2013.....	614
Actual Change	615
Section 6.3.3.....	615
Section 6.4.1.2.2	616
Title: SSP ping.LFPS tRepeat Requirement	618
Applied to: USB_3_1r1.0_07_31_2013.....	618
Actual Change	619
Title: SSP System Jitter Budget	620
Applied to: USB_3_1r1.0_07_31_2013.....	620
Actual Change	621
(b) From, in Section 6.7.3, Table 6-19, page 168.....	622
(b) To, in Section 6.7.3, Table 6-19, page 168.....	622
(c) From, in Section 6.8.5, Table 6-27, page 178.....	623
(c) To, in Section 6.8.5, Table 6-27, page 178.....	624
Title: USB3.1 tHubDriveResume	625
Applied to: USB_3_1r1.0_07_31_2013.....	625
Actual Change	626
Table 10-19, Hub Parameters.....	626
10.10Suspend and Resume.....	626
Title: TSEQ Gen2 Clarification	627
Applied to: USB_3_1r1.0_07_31_2013.....	627
Actual Change	628
(a) From Text (and location): Section 6.4.1.2.2, Page 154	628
6.4.1.2.2Training Sequence Values for Gen 2 Operation	628
(a) To Text (and location): Section 6.4.1.2.2, Page 154.....	628
6.4.1.2.2Training Sequence Values for Gen 2 Operation	628
(b) From Text (and location): Section 7.5.4.7.1, Page 242.....	629
7.5.4.7.1Polling.RxEQ Requirements.....	629
(b) To Text (and location): Section 7.5.4.7.1, Page 242.....	629
7.5.4.7.1Polling.RxEQ Requirements.....	629
(c) From Text (and location): Section 7.5.4.7.2, Page 242	630
7.5.4.7.2Exit from Polling.RxEQ	630
(c) To Text (and location): Section 7.5.4.7.2, Page 242	630
7.5.4.7.2Exit from Polling.RxEQ	630
(d) From Text (and location): Section 6.8.1, Page 170.....	631
(d) To Text (and location): Section 6.8.1, Page 170.....	631
Title: USB 3.1 VBUS Max Limit	632
Applied to: USB_3_1r1.0_07_31_2013.....	632
Actual Change	633
(a) Section 11.4.2 Steady State Voltage Drop Budget	633
(b) Section 11.4.5 VBUS Electical Characteristics.....	634
Title: USB 3.1 VBUS Max Limit	637
Applied to: USB_3_1r1.0_07_31_2013.....	637
Actual Change Requested	638
(a) Section 11.4.2 Steady State Voltage Drop Budget	638

(b) Section 11.4.5 VBUS Electrical Characteristics	639
Figure 2-1 – Port and Link Pictorial	48
Figure 3-1 – USB 3.1 Dual Bus System Architecture	49
Figure 3-2 – USB 3.1 Cable	50
Figure 3-3 – USB 3.1 Terminology Reference Model	52
Figure 3-4 – Enhanced SuperSpeed Bus Communications Layers and Power Management Elements	53
Figure 3-5 – Examples of Supported USB 3.1 USB Physical Device Topologies	59
Figure 3-6 – SuperSpeed Only Enhanced SuperSpeed Peripheral Device Configuration	60
Figure 3-7 – Enhanced SuperSpeed Device Configuration	60
Figure 3-8 – Multiple SuperSpeed Bus Instances in an Enhanced SuperSpeed System	62
Figure 4-1 – Enhanced SuperSpeed IN Transaction Protocol	68
Figure 4-2 – Enhanced SuperSpeed OUT Transaction Protocol	69
Figure 4-3 – Enhanced SuperSpeed IN Stream Example	72
Figure 5-1 – USB 3.1 Standard-A Receptacle Interface Dimensions	85
Figure 5-2 – Example USB 3.1 Standard-A Receptacle with Grounding Springs and Required contact zones on the Standard-A Plug	87
Figure 5-3 – Example USB 3.1 Standard-A Mid-Mount Receptacles with Insertion Detect	88
Figure 5-4 – USB 3.1 Standard-A Plug Interface Dimensions	91
Figure 5-5 – Example Footprint for the USB 3.1 Standard-A Receptacle – Through-Hole with Back-Shield	94
Figure 5-6 – Example Footprint for the USB 3.1 Standard-A Receptacle – Mid-Mount Standard Mount Through-Hole with Insertion Detect	95
Figure 5-7 – Example Footprint for the USB 3.1 Standard-A Receptacle – Mid-Mount Reverse Mount Through-Hole with Insertion Detect	96
Figure 5-8 – Illustration of Color Coding Recommendation for USB 3.1 Standard-A Connector	98
Figure 5-9 – USB 3.1 Standard-B Receptacle Interface Dimensions	100
Figure 5-10 – USB 3.1 Standard-B Plug Interface Dimensions	101
Figure 5-11 – Reference Footprint for the USB 3.1 Standard-B Receptacle	102
Figure 5-12 – USB 3.1 Micro-B and -AB Receptacles Interface Dimensions	105
Figure 5-13 – USB 3.1 Micro-B and Micro-A Plug Interface Dimensions	108
Figure 5-14 – Reference Footprint for the USB 3.1 Micro-B or Micro-AB Receptacle	110
Figure 5-15 – Illustration of a USB 3.1 Cable Cross-Section	112
Figure 5-16 – USB 3.1 Standard-A to USB 3.1 Standard-B Cable Assembly	114
Figure 5-17 – USB 3.1 Micro-B Plug Cable Overmold Dimensions	116
Figure 5-18 – USB 3.1 Micro-A Cable Overmold Dimensions	118
Figure 5-19 – Typical Plug Orientation	120
Figure 5-20 – Recommended Ground Void Dimension for USB Standard-A Receptacle	122
Figure 5-21 – Impedance Limits of a Mated Connector for Gen 2 Speed	123
Figure 5-22 – Illustration of Cable Assembly Mounted on Test Fixture	124
Figure 5-23 – Illustration of Cable Assembly with Reference Host and Device	124
Figure 5-24 – Illustration of Insertion Loss Fit at Nyquist Frequency	125

Figure 5-25 – Example of Insertion Loss Deviation	126
Figure 5-26 – Pass/Fail Examples.....	128
Figure 5-27 – Illustration of Peak-to-Peak Crosstalk	129
Figure 5-28 – Differential-to-Common-Mode Conversion Requirement for Gen 2	129
Figure 5-29 – Set Up For Cable SE Measurement (subject to change).....	130
Figure 5-30 – 4-Axes Continuity Test	133
Figure 5-31 – Mated USB 3.1 Standard-A Connector	136
Figure 5-32 – Mated USB 3.1 Standard-B Connector	136
Figure 5-33 – Mated USB 3.1 Micro-B Connector.....	137
Figure 5-34 – Examples of Connector Apertures	138
Figure 6-1 – SuperSpeed Physical Layer	139
Figure 6-2 – Transmitter Block Diagram	140
Figure 6-3 – Gen 1 Receiver Block Diagram	141
Figure 6-4 – Gen 2 Receiver Block Diagram	142
Figure 6-5 – Channel Models	143
Figure 6-6 – Character to Symbol Mapping	144
Figure 6-7 – Bit Transmission Order	144
Figure 6-8 – LFSR with Scrambling Polynomial.....	145
Figure 6-9 – Gen 2 Serialization and Deserialization Order	146
Figure 6-10 – Gen 2 Bit Transmission Order and Framing.....	147
Figure 6-11 – LFSR for use in Gen 2 operation	149
Figure 6-12 – Jitter Filtering – “Golden PLL” and Jitter Transfer Functions.....	160
Figure 6-13 – “Golden PLL” and Jitter Transfer Functions for Gen 1 Operation	161
Figure 6-14 – “Golden PLL” and Jitter Transfer Functions for Gen 2 Operation	161
Figure 6-15 – Example of Period Modulation from Triangular SSC	162
Figure 6-16 – Eye Masks	164
Figure 6-17 – Single-ended and Differential Voltage Levels	165
Figure 6-18 – Device Termination Schematic	165
Figure 6-19 – Tx Normative Setup with Reference Channel	168
Figure 6-20 – De-Emphasis Waveform.....	169
Figure 6-21 – 3-tap Transmit Equalizer Structure	169
Figure 6-22 – Example Output Waveform for 3-tap Transmit Equalizer	170
Figure 6-23 – Frequency Spectrum of TSEQ.....	171
Figure 6-24 – Gen 1 Tx Compliance Rx EQ Transfer Function	172
Figure 6-25 – Gen 2 Compliance Rx EQ Transfer Function	173
Figure 6-26 – Gen 2 reference DFE Function.....	174
Figure 6-27 – Rx Tolerance Setup	177
Figure 6-28 – Jitter Tolerance Curve.....	177
Figure 6-29 – LFPS Signaling	179
Figure 6-30 – U1 Exit, U2 Exit, and U3 Wakeup LFPS Handshake Timing Diagram	181
Figure 6-31 – Example of Warm Reset Out of U3.....	183
Figure 6-32 – Example of Binary Representation based on Polling.LFPS	183
Figure 6-33 – SCD1/SCD2 transmission	184

Figure 6-34 – Logic Representation of LBPS	185
Figure 6-35 – LBPM Transmission Examples	186
Figure 6-36 – Rx Detect Schematic.....	187
Figure 7-1 – Link Layer	188
Figure 7-2 – Byte Ordering.....	189
Figure 7-3 – Enhanced SuperSpeed Header Packet with HPSTART, Packet Header, and Link Control Word	190
Figure 7-4 – SuperSpeedPlus DPH Format	191
Figure 7-5 – Packet Header	191
Figure 7-6 – CRC-16 Remainder Generation.....	192
Figure 7-7 – Link Control Word	193
Figure 7-8 – CRC-5 Remainder Generation	193
Figure 7-9 – Data Packet Payload with CRC-32 and Framing.....	194
Figure 7-10 – CRC-32 Remainder Generation.....	195
Figure 7-11 – Data Packet with Data Packet Header Followed by Data Packet Payload. (a) SuperSpeed DP; (b). SuperSpeedPlus DP.....	197
Figure 7-12 – Link Command Structure.....	198
Figure 7-13 – Link Command Word Structure.....	198
Figure 7-14 – State Diagram of the Link Training and Status State Machine	230
Figure 7-15 – eSS.Disabled Substate Machine	232
Figure 7-16 – eSS.Inactive Substate Machine.....	233
Figure 7-17 – Rx.Detect Substate Machine	236
Figure 7-18 – Polling Substate Machine	247
Figure 7-19 – U1.....	250
Figure 7-20 – U2.....	251
Figure 7-21 – U3.....	252
Figure 7-22 – Recovery Substate Machine	256
Figure 7-23 – Loopback Substate Machine	258
Figure 7-24 – Hot Reset Substate Machine	260
Figure 8-1 – Protocol Layer Highlighted	261
Figure 8-2 – Example Transaction Packet.....	263
Figure 8-3 – Link Control Word Detail	264
Figure 8-4 – Link Management Packet Structure	265
Figure 8-5 – Set Link Function LMP	266
Figure 8-6 – U2 Inactivity Timeout LMP	267
Figure 8-7 – Vendor Device Test LMP.....	267
Figure 8-8 – Port Capability LMP	268
Figure 8-9 – Port Configuration LMP	270
Figure 8-10 – Port Configuration Response LMP	271
Figure 8-11 – Link Delay Measurement Protocol	272
Figure 8-12 – PTM ITP Protocol.....	273
Figure 8-13 – LDM State Machine Notation.....	274
Figure 8-14 – LDM Requester State Machine.....	275
Figure 8-15 – LDM Responder State Machine	277

Figure 8-16 – PTM Path Performance Contributors	281
Figure 8-17 – LDM LMP	283
Figure 8-18 – ACK Transaction Packet	285
Figure 8-19 – NRDY Transaction Packet.....	288
Figure 8-20 – ERDY Transaction Packet.....	288
Figure 8-21 – STATUS Transaction Packet.....	289
Figure 8-22 – STALL Transaction Packet	289
Figure 8-23 – Device Notification Transaction Packet	290
Figure 8-24 – Function Wake Device Notification	290
Figure 8-25 – Latency Tolerance Message Device Notification.....	291
Figure 8-26 – Bus Interval Adjustment Message Device Notification	292
Figure 8-27 – Sublink Speed Device Notification.....	295
Figure 8-28 – PING Transaction Packet.....	296
Figure 8-29 – PING_RESPONSE Transaction Packet	297
Figure 8-30 – Example Data Packet.....	297
Figure 8-31 – Isochronous Timestamp Packet.....	299
Figure 8-32 – Route String Detail.....	300
Figure 8-33 – Sample Concurrent BULK IN Transactions	304
Figure 8-34 – Sample Concurrent BULK and Isochronous IN Transactions	305
Figure 8-35 – Legend for State Machines.....	310
Figure 8-36 – Sample BULK IN Sequence.....	312
Figure 8-37 – Sample BULK OUT Sequence.....	313
Figure 8-38 – General Stream Protocol State Machine (SPSM).....	314
Figure 8-39 – Device IN Stream Protocol State Machine (DISPSM)	317
Figure 8-40 – Device IN Move Data State Machine (DIMDSM)	320
Figure 8-41 – Device OUT Stream Protocol State Machine (DOSPSM)	322
Figure 8-42 – Device OUT Move Data State Machine (DOMDSM).....	325
Figure 8-43 – Host IN Stream Protocol State Machine (HISPSM).....	328
Figure 8-44 – Host IN Move Data State Machine (HIMDSM)	330
Figure 8-45 – Host OUT Stream Protocol State Machine (HOSPSM).....	333
Figure 8-46 – Host OUT Move Data State Machine (HOMDSM)	336
Figure 8-47 – Control Read Sequence	339
Figure 8-48 – Control Write Sequence	340
Figure 8-49 – Host Sends Interrupt IN Transaction in Each Service Interval.....	343
Figure 8-50 – Host Stops Servicing Interrupt IN Transaction Once NRDY is Received	343
Figure 8-51 – Host Resumes IN Transaction after Device Sent ERDY.....	344
Figure 8-52 – Endpoint Sends STALL TP	344
Figure 8-53 – Host Detects Error in Data and Device Resends Data	344
Figure 8-54 – Host Sends Interrupt OUT Transaction in Each Service Interval	346
Figure 8-55 – Host Stops Servicing Interrupt OUT Transaction Once NRDY is Received	346
Figure 8-56 – Host Resumes Sending Interrupt OUT Transaction After Device Sent ERDY	346
Figure 8-57 – Device Detects Error in Data and Host Resends Data	347

Figure 8-58 – Endpoint Sends STALL TP	347
Figure 8-59 – Multiple Active Isochronous Endpoints with Aligned Service Interval Boundaries	348
Figure 8-60 – Enhanced SuperSpeed Isochronous IN Transaction Format	349
Figure 8-61 – Enhanced SuperSpeed Isochronous OUT Transaction Format	349
Figure 8-62 – Sample Enhanced SuperSpeed Isochronous IN Transaction	350
Figure 8-63 – Sample Enhanced SuperSpeed Isochronous OUT Transaction.....	351
Figure 8-64 – Sample Enhanced SuperSpeed Isochronous IN Transaction	352
Figure 8-65 – Sample Enhanced SuperSpeed Isochronous OUT Transaction.....	353
Figure 8-66 – Sample Smart Enhanced SuperSpeed Isochronous IN Transaction	355
Figure 8-67 – Sample Smart Enhanced SuperSpeed Isochronous OUT Transaction	356
Figure 8-68 – Sample Pipeline Isochronous IN Transactions.....	358
Figure 9-1 – Peripheral State Diagram and Hub State Diagram (Enhanced SuperSpeed Portion Only)	364
Figure 9-2 – wIndex Format when Specifying an Endpoint	375
Figure 9-3 – wIndex Format when Specifying an Interface	375
Figure 9-4 – Information Returned by a Standard GetStatus() Request to a Device	382
Figure 9-5 – Information Returned by a Standard GetStatus() Request to an Interface.....	383
Figure 9-6 – Information Returned by a Standard GetStatus() Request to an Endpoint	383
Figure 9-7 – Information Returned by a PTM GetStatus() Request to an Endpoint	384
Figure 9-8 – Example of Feedback Endpoint Relationships	409
Figure 10-1 – USB Hub Architecture	414
Figure 10-2 – SuperSpeed Portion of the USB Hub Architecture	415
Figure 10-3 – SuperSpeedPlus Portion of the Hub Architecture	416
Figure 10-4 – Simple USB Topology	417
Figure 10-5 – Route String Example	419
Figure 10-6 – SuperSpeed Hub Signaling Connectivity	420
Figure 10-7 – Resume Connectivity	421
Figure 10-8 – Typical SuperSpeed Hub Header Packet Buffer Architecture.....	422
Figure 10-9 – SuperSpeed Hub Data Buffer Traffic (Header Packet Buffer Only Shown for DS Port 1)	422
Figure 10-10 – Downstream Facing Hub Port State Machine.....	425
Figure 10-11 – Downstream Facing Hub Port Power Management State Machine.....	433
Figure 10-12 – Upstream Facing Hub Port State Machine	437
Figure 10-13 – Hub Connect (HCONNECT) State Machine	439
Figure 10-14 – Upstream Facing Hub Port Power Management State Machine	441
Figure 10-15 – Example SS Hub Header Packet Buffer Architecture – Downstream Traffic	445
Figure 10-16 – Example SS Hub Header Packet Buffer Architecture – Upstream Traffic	446
Figure 10-17 – Logical Representation of Upstream Flowing Buffers.....	448
Figure 10-18 – Logical Representation of Downstream Flowing Buffers	448
Figure 10-19 – Port Transmit State Machine	453
Figure 10-20 – Upstream Facing Port Rx State Machine	455
Figure 10-21 – Example Hub Controller Organization.....	462

Figure 10-22 – Relationship of Status, Status Change, and Control Information to Device States	463
Figure 10-23 – Port Status Handling Method.....	464
Figure 10-24 – Hub and Port Status Change Bitmap	465
Figure 10-25 – Example Hub and Port Change Bit Sampling.....	465
Figure 10-26 – Peripheral Upstream Device Port State Machine	494
Figure 11-1 – Compound Self-powered Hub.....	501
Figure 11-2 – Low-power Bus-powered Function.....	502
Figure 11-3 – High-power Bus-powered Function.....	502
Figure 11-4 – Self-powered Function	503
Figure 11-5 – Worst-case Voltage Drop Topology (Steady State).....	503
Figure 11-6 – Worst-case Voltage Drop Analysis Using Equivalent Resistance	504
Figure 11-7 – Typical Suspend Current Averaging Profile	505
Figure C.1 – Flow Diagram for Host Initiated Wakeup	531
Figure C.2 – Device Total Intrinsic Latency Tolerance	532
Figure C.3 – Host to Device Path Exit Latency Calculation Examples	534
Figure C.4 – Device Connected Directly to a Host.....	535
Figure C.5 – Device Connected Through a Hub	536
Figure C.6 – Downstream Host to Device Path Exit Latency with Hub	537
Figure C.7 – Upstream Device to Host Path Exit Latency with Hub	538
Figure C.8 – LT State Diagram	541
Figure C.9 – System Power during SuperSpeed and High Speed Device Data Transfers	544
Figure D.1 – Sample ERDY Transaction Packet.....	545
Figure D.2 – Sample Data Packet	545
Figure D.3 – Example placement of Gen 2 SKP Block, Idle Symbols, Link Command and Header Packet	545
Figure D.4 – Example placement of Gen 2 Data Packets and Idle Symbols.....	546
Figure E.1 – Link segment definition	547
Figure E.2 – Retimer implementation examples	548
Figure E.3 – Example high level retimer architecture in Gen 2 operation.....	549
Figure E.4 – Retimer Training and Status State Machine	552
Figure E.5 – Polling Substate Machine.....	554
Figure E.6 – Recovery Substate Machine	562
Figure E.7 – Example block diagram of a retimer operating at SuperSpeed	565
Figure 6-20 – De-Emphasis Waveform.....	572
Figure 6-21 – 3-tap Transmit Equalizer Structure	573
Figure 6-22 – Example Output Waveform for 3-tap Transmit Equalizer	573
Figure 6-20 – De-Emphasis Waveform.....	575
Figure 6-21 – 3-tap Transmit Equalizer Structure	576
Figure 6-22 – Example Output Waveform for 3-tap Transmit Equalizer	576
Figure 6-23 – Example waveforms for measuring transmitter equalization	578
Figure 6-25 – Gen 2 Compliance Rx EQ Transfer Function	579
Figure 6-25 – Gen 2 Compliance Rx EQ Transfer Function	580

Table 3-1 – Comparing Enhanced SuperSpeed Bus to USB 2.0 Bus	51
Table 5-1 – Plugs Accepted By Receptacles	81
Table 5-2 – USB 3.1 Standard-A Connector Pin Assignments	97
Table 5-3 – USB 3.1 Standard-B Connector Pin Assignments	103
Table 5-4 – USB 3.1 Micro-B Connector Pin Assignments.....	111
Table 5-5 – USB 3.1 Micro-AB/-A Connector Pin Assignments	111
Table 5-6 – Cable Wire Assignments	113
Table 5-7 – Reference Wire Gauges	113
Table 5-8 – USB 3.1 Standard-A to USB 3.1 Standard-B Cable Assembly Wiring.....	115
Table 5-9 – USB 3.1 Standard-A to USB 3.1 Standard-A Cable Assembly Wiring.....	115
Table 5-10 – USB 3.1 Standard-A to USB 3.1 Micro-B Cable Assembly Wiring	117
Table 5-11 – USB 3.1 Micro-A to USB 3.1 Micro-B Cable Assembly Wiring	119
Table 5-12 – USB 3.1 Micro-A to USB 3.1 Standard-B Cable Assembly Wiring	119
Table 5-13 – SDP Differential Insertion Loss Examples for Gen 2 speed.....	122
Table 5-14 – SDP Differential Insertion Loss Examples for Gen 2 speed with Coaxial Construction	122
Table 5-15 – Design Targets.....	124
Table 5-16 – Durability Ratings	131
Table 5-17 – Environmental Test Conditions	134
Table 5-18 – Reference Materials	135
Table 6-1 – Special Symbols.....	150
Table 6-2 – Gen 1 TSEQ Ordered Set.....	152
Table 6-3 – Gen 1 TS1 Ordered Set.....	152
Table 6-4 – Gen 1 TS2 Ordered Set.....	152
Table 6-5 – Gen 1/Gen 2 Link Configuration	153
Table 6-6 – Gen 2 TS1 Ordered Set.....	154
Table 6-7 – Gen 2 TS2 Ordered Set.....	155
Table 6-8 – Gen 2 TSEQ Ordered Set.....	155
Table 6-9 – Gen 2 SYNC Ordered Set	155
Table 6-10 – SDS Ordered Set	155
Table 6-11 – Gen 1 SKP Ordered Set Structure	157
Table 6-12 – Gen 2 SKP Ordered Set	158
Table 6-13 – Compliance Pattern Sequences.....	159
Table 6-14 – Gen 2 Compliance Pattern	159
Table 6-15 – Informative Jitter Budgeting at the Silicon Pads.....	160
Table 6-16 – SSC Parameters	162
Table 6-17 – Transmitter Normative Electrical Parameters.....	166
Table 6-18 – Transmitter Informative Electrical Parameters at Silicon Pads	167
Table 6-19 – Normative Transmitter Eye Mask at Test Point TP1	168
Table 6-20 – Informative Gen 2 Transmitter Equalization Settings	170
Table 6-21 – Receiver Normative Electrical Parameters.....	174
Table 6-22 – Receiver Informative Electrical Parameters	175

Table 6-23 – BRST	176
Table 6-24 – BDAT	176
Table 6-25 – BERC	176
Table 6-26 – BCNT	176
Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing	178
Table 6-28 – Normative LFPS Electrical Specification	179
Table 6-29 – LFPS Transmitter Timing for SuperSpeed Designs ¹	180
Table 6-30 – LFPS Handshake Timing for U1/U2 Exit, Loopback Exit, and U3 Wakeup	182
Table 6-31 – Binary Representation of Polling.LFPS	183
Table 6-32 – LBPS Transmit and Receive Specification	185
Table 7-1 – CRC-16 Mapping	192
Table 7-2 – CRC-32 Mapping	196
Table 7-3 – Link Command Ordered Set Structure	198
Table 7-4 – Link Command Bit Definitions	199
Table 7-5 – Link Command Definitions	201
Table 7-6 – Logical Idle Definition	202
Table 7-7 – Transmitter Timers Summary	213
Table 7-8 – Link Flow Control Timers Summary	214
Table 7-9 – Valid Packet Framing Symbol Order (Sx is One of SHP, DPHP, SDP, END or EDB)	220
Table 7-10 – Valid Link Command Symbol Order	221
Table 7-11 – Error Types and Recovery	225
Table 7-12 – LTSSM State Transition Timeouts	229
Table 7-13 – PHY Capability LBPM	240
Table 8-1 – Type Field Description	264
Table 8-2 – Link Control Word Format	265
Table 8-3 – Link Management Packet Subtype Field	266
Table 8-4 – Set Link Function	267
Table 8-5 – U2 Inactivity Timer Functionality	267
Table 8-6 – Vendor-specific Device Test Function	268
Table 8-7 – Port Capability LMP Format	269
Table 8-8 – Port Type Selection Matrix	269
Table 8-9 – Port Configuration LMP Format (Differences with Port Capability LMP)	270
Table 8-10 – Port Configuration Response LMP Format (Differences with Port Capability LMP)	271
Table 8-11 – LDM LMP	284
Table 8-12 – Transaction Packet Subtype Field	284
Table 8-13 – ACK TP Format	286
Table 8-14 – NRDY TP Format (Differences with ACK TP)	288
Table 8-15 – ERDY TP Format (Differences with ACK TP)	288
Table 8-16 – STATUS TP Format (Differences with ACK TP)	289
Table 8-17 – STALL TP Format (Differences with ACK TP)	289
Table 8-18 – Device Notification TP Format (Differences with ACK TP)	290
Table 8-19 – Function Wake Device Notification	291

Table 8-20 – Latency Tolerance Message Device Notification	291
Table 8-21 – Bus Interval Adjustment Message Device Notification.....	292
Table 8-22 – Sublink Speed Device Notification	295
Table 8-23 – PING TP Format (differences with ACK TP)	296
Table 8-24 – PING_RESPONSE TP Format (Differences with ACK TP)	297
Table 8-25 – Data Packet Format (Differences with ACK TP).....	298
Table 8-26 – Isochronous Timestamp Packet Format.....	299
Table 8-27 – Device Responses to TP Requesting Data (Bulk, Control, and Interrupt Endpoints)	306
Table 8-28 – Host Responses to Data Received from a Device (Bulk, Control, and Interrupt Endpoints)	307
Table 8-29 – Device Responses to OUT Transactions (Bulk, Control, and Interrupt Endpoints)	308
Table 8-30 – Device Responses to SETUP Transactions (Only for Control Endpoints)	309
Table 8-31 – Status Stage Responses	341
Table 8-32 – ACK TP and DPs for Pipelined Isochronous IN Transactions	357
Table 8-33 – Device Responses to Isochronous IN Transactions	359
Table 8-34 – Host Responses to IN Transactions.....	360
Table 8-35 – Device Responses to OUT Data Packets.....	360
Table 8-36 – Timing Parameters	361
Table 9-1 – Visible Enhanced SuperSpeed Device States.....	365
Table 9-2 – Preserved USB Suspend State Parameters	371
Table 9-3 – Format of Setup Data	374
Table 9-4 – Standard Device Requests	376
Table 9-5 – Standard Request Codes	377
Table 9-6 – Descriptor Types	378
Table 9-7 – Standard Feature Selectors.....	378
Table 9-8 – Standard Status Type Codes.....	382
Table 9-9 – Suspend Options.....	387
Table 9-10 – Device Parameters and Events	390
Table 9-11 – Standard Device Descriptor.....	393
Table 9-12 – BOS Descriptor	394
Table 9-13 – Format of a Device Capability Descriptor.....	394
Table 9-14 – Device Capability Type Codes.....	395
Table 9-15 – USB 2.0 Extension Descriptor	395
Table 9-16 – SuperSpeed Device Capability Descriptor	397
Table 9-17 – Container ID Descriptor	398
Table 9-18 – Platform Descriptor	399
Table 9-19 – SuperSpeedPlus Descriptor	400
Table 9-20 – PTM Capability Descriptor	402
Table 9-21 – Standard Configuration Descriptor.....	403
Table 9-22 – Standard Interface Association Descriptor.....	404
Table 9-23 – Standard Interface Descriptor.....	406
Table 9-24 – Standard Endpoint Descriptor.....	407

Table 9-25 – Example of Feedback Endpoint Numbers	408
Table 9-26 – SuperSpeed Endpoint Companion Descriptor	410
Table 9-27 – SuperSpeedPlus Isochronous Endpoint Companion Descriptor	411
Table 9-28 – String Descriptor Zero, Specifying Languages Supported by the Device	412
Table 9-29 – UNICODE String Descriptor	412
Table 10-1 – Downstream Facing Hub Port State Machine Diagram Legend	426
Table 10-2 – Downstream Port VBUS Requirements	428
Table 10-3 – Downstream Flowing Header Packet Processing Actions.....	456
Table 10-4 – Hub Power Operating Mode Summary	467
Table 10-5 – Enhanced SuperSpeed Hub Descriptor	474
Table 10-6 – Hub Responses to Standard Device Requests.....	476
Table 10-7 – Hub Class Requests	477
Table 10-8 – Hub Class Request Codes.....	477
Table 10-9 – Hub Class Feature Selectors	478
Table 10-10 – Hub Status Field, <i>wHubStatus</i>	480
Table 10-11 – Hub Change Field, <i>wHubChange</i>	481
Table 10-12 – Port Status Type Codes	482
Table 10-13 – Port Status Field, <i>wPortStatus</i>	483
Table 10-14 – Port Change Field, <i>wPortChange</i>	486
Table 10-15 – Extended Port Status Field, <i>dwExtPortStatus</i>	488
Table 10-16 – U1 Timeout Value Encoding	490
Table 10-17 – U2 Timeout Value Encoding	490
Table 10-18 – Downstream Port Remote Wake Mask Encoding	492
Table 10-19 – Hub Parameters	497
Table 11-1 – USB 3.0 and USB 2.0 Interoperability	499
Table 11-2 – DC Electrical Characteristics	506
Table 11-3 – VBUS/Gnd Wire Gauge vs. Maximum Length.....	507
Table A.1 – 8b/10b Data Symbol Codes.....	508
Table A.2 – 8b/10b Special Character Symbol Codes	514
Table C.1 – Link States and Characteristics Summary	522
Table 6-13 – Compliance Pattern Sequences.....	568
Table 6-13 – Compliance Pattern Sequences.....	569
Table 6-17 – Transmitter Normative Electrical Parameters.....	570
Table 6-17 – Transmitter Normative Electrical Parameters.....	571
Table 6-20 – Informative Gen 2 Transmitter Equalization Settings	574
Table 6-20 – Informative Gen 2 Transmitter Equalization Settings	577
Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing	581
Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing	582
Table 11-4 – Link Command Bit Definitions.....	587
Table 11-5 – Link Command Bit Definitions.....	588
Table 11-6 – Gen 2 SKP Ordered Set	610
Table 11-7 – Gen 2 SKP Ordered Set	611
Table 11-8 – SDS Ordered Set	616

Table 11-9 – SDS Ordered Set	617
Table 6-29 – LFPS Transmitter Timing for SuperSpeed Designs ¹	619
Table 6-29 – LFPS Transmitter Timing for SuperSpeed Designs ¹	619
Table 6-15 – Informative Jitter Budgeting at the Silicon Pads.....	621
Table 6-15 – Informative Jitter Budgeting at the Silicon Pads.....	621
Table 6-19 – Normative Transmitter Eye Mask at Test Point TP1	622
Table 6-19 – Normative Transmitter Eye Mask at Test Point TP1	622
Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing	623
Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing	624

NOTE All Engineering Change Notices (ECN) and Errata documents as of May 01, 2015 that pertain to this core specification follow the last page of the specification, starting on page 567.

1 Introduction

1.1 Background

The original Universal Serial Bus (USB) was driven by the need to provide a user-friendly plug-and-play way to attach external peripherals to a Personal Computer (PC). USB has gone beyond just being a way to connect peripherals to PCs. Printers use USB to interface directly to cameras. Mobile devices use USB connected keyboards and mice. USB technology commonly finds itself in automobiles, televisions, and set-top boxes. USB, as a protocol, is also being picked up and used in many nontraditional applications, such as industrial automation. And USB as a source of power has become the mobile device charging solution endorsed by international communities across the globe.

Initially, USB provided two speeds (12 Mbps and 1,5 Mbps) that peripherals could use. As PCs became increasingly powerful and able to process larger amounts of data, users needed to get more and more data into and out of their PCs. This led to the definition of the USB 2.0 specification in 2000 to provide a third transfer rate of 480 Mbps while retaining backward compatibility. By 2006, two things in the environment happened: the transfer rates of HDDs exceeded 100 MB/s, far outstripping USB 2.0's ~32 MB/s bandwidth and the amount of digital content users were creating was an ever increasing pace. USB 3.0 was the USB community's response and provided users with the ability to move data at rates up to 450 MB/s while retaining backward compatibility with USB 2.0.

Now, with the continued trend for more bandwidth driven by larger and faster storage solutions, higher resolution video, and broader use of USB as an external expansion/docking solution, USB 3.1 extends the performance range of USB up to 1 GB/s by doubling the SuperSpeed USB clock rate to 10 Gbps and enhancing data encoding efficiency.

1.2 Objective of the Specification

This document defines the latest generation USB industry standard, USB 3.1. The specification describes the protocol definition, types of transactions, bus management, and the programming interface required to design and build systems and peripherals that are compliant with this specification. USB 3.1 is primarily a performance enhancement to SuperSpeed USB 3.0 resulting in providing more than double the bandwidth for devices such as Solid State Drives and High Definition displays.

This specification refers to Enhanced SuperSpeed as a collection of features or requirements that apply to both USB 3.0 and USB 3.1 bus operation. Additionally, where specific differences exist with regard to the USB 3.0 definition of SuperSpeed features or requirements, those differences will be uniquely identified as SuperSpeedPlus (or SSP) features or requirements – generally, "SuperSpeed" is in reference to 5 Gbps operation and "SuperSpeedPlus" is in reference to 10 Gbps operation.

USB 3.1's goal remains to enable devices from different vendors to interoperate in an open architecture, while maintaining and leveraging the existing USB infrastructure (device drivers, software interfaces, etc.). The specification is intended as an enhancement to the PC architecture, spanning portable, business desktop, and home environments, as well as simple device-to-device communications. It is intended that the specification allow system OEMs and peripheral developers adequate room for product versatility and market differentiation without the burden of carrying obsolete interfaces or losing compatibility.

1.3 Scope of the Document

The specification is primarily targeted at peripheral developers and platform/adaptor developers, but provides valuable information for platform operating system/BIOS/device driver, adaptor IHVs/ISVs, and system OEMs. This specification can be used for developing new products and associated software.

Product developers using this specification are expected to know and understand the USB 2.0 Specification. Specifically, USB 3.1 devices must implement device framework commands and descriptors as defined in the USB 2.0 Specification. Devices operating at the new 10 Gbps (Gen 2) speed must implement the SuperSpeedPlus enhancements defined in this version of the specification.

1.4 USB Product Compliance

Adopters of the USB 3.1 specification have signed the USB 3.0 Adopters Agreement, which provides them access to a reasonable and nondiscriminatory (RANDZ) license from the Promoters and other Adopters to certain intellectual property contained in products that are compliant with the USB 3.1 specification. Adopters can demonstrate compliance with the specification through the testing program as defined by the USB Implementers Forum (USB-IF). Products that demonstrate compliance with the specification will be granted certain rights to use the USB-IF logos as defined in the logo license.

Starting with USB 3.1, product compliance requirements are being tightened up to prohibit non-certified cables and connectors. Use of any registered icons or logos on products, documentation or packaging will require a license and license requirements will include passing specific product certification.

1.5 Document Organization

Chapters 1 through 4 provide an overview for all readers, while Chapters 5 through 11 contain detailed technical information defining USB 3.1.

Readers should contact operating system vendors for operating system bindings specific to USB 3.1.

1.6 Design Goals

USB 3.0 was a revolutionary step for USB. USB 3.1 is the next evolutionary step to increase the bandwidth. The goal remains the same; end users view it as the same as they viewed USB 2.0 and USB 3.0, just faster. Several key design areas to meet this goal are listed below:

Preserve the USB model of smart host and simple device.

- Leverage the existing USB infrastructure. There are a vast number of USB products in use today. A large part of their success can be traced to the existence of stable software interfaces, easily developed software device drivers, and a number of generic standard device class drivers (HID, mass storage, audio, etc.). Enhanced SuperSpeed USB devices are designed to keep this software infrastructure intact so that developers of peripherals can continue to use the same interfaces and leverage all of their existing development work.
- Significantly improve power management. Reduce the active power when sending data and reduce idle power by providing a richer set of power management mechanisms to allow devices to drive the bus into lower power states.
- Ease of use has always been and remains a key design goal for all varieties of USB.
- Preserve the investment. There are a large number of PCs in use that support only USB 2.0. There are a larger number of USB 2.0 peripherals in use. Retaining backward compatibility at the Type-A connector to allow Enhanced SuperSpeed devices to be used, albeit at a lower speed, with USB 2.0 PCs and allow high speed devices with their existing cables to be connected to the USB 3.1 SuperSpeed Type-A connectors.
- Features that allow the host controller to take advantage of the USB 3.1 speed without any change to the OS.

1.7 Related Documents

Universal Serial Bus Specification, Revision 2.0

USB On-the-Go Supplement to the USB 2.0 Specification, Revision 1.3

USB On-the-Go and Embedded Host Supplement to the USB 3.0 Specification, Revision 1.0

Universal Serial Bus Micro-USB Cables and Connectors Specification, Revision 1.01

EIA-364-1000.01: Environmental Test Methodology for Assessing the Performance of Electrical Connectors and Sockets Used in Business Office Applications

USB 3.0 Connectors and Cable Assemblies Compliance Document

USB SuperSpeed Electrical Test Methodology white paper

USB 3.0 Jitter Budgeting white paper

INCITS TR-35-2004, INCITS Technical Report for Information Technology – Fibre Channel – Methodologies for Jitter and Signal Quality Specification (FC-MJSQ)

Universal Serial Bus 3.0 Specification (including errata and ECNs through May 1, 2011)

Universal Serial Bus Power Delivery Specification, Revision 1.0 Including Errata through 31-October-2012

2 Terms and Abbreviations

This chapter lists and defines terms and abbreviations used throughout this specification. Note, for terms and abbreviations not defined here, use their generally accepted or dictionary meaning.

Term/Abbreviation	Definition
ACK	Handshake packet indicating a positive acknowledgment.
ACK Tx Header Sequence Number	The expected header sequence number in the link control word to be acknowledged.
active device	A device that is powered and is not in the Suspend state.
asynchronous data	Data transferred at irregular intervals with relaxed latency requirements.
attached	A downstream device is attached to an upstream device when there is a physical connection between the two.
AWG#	The measurement of a wire's cross section, as defined by the American Wire Gauge standard.
bandwidth	The amount of data transmitted per unit of time, typically bits per second (bps) or bytes per second (Bps).
big endian	A method of storing data that places the most significant byte of multiple-byte values at a lower storage address. For example, a 16-bit integer stored in big endian format places the least significant byte at the higher address and the most significant byte at the lower address. See also little endian.
bit	A unit of information used by digital computers. Represents the smallest piece of addressable memory within a computer. A bit expresses the choice between two possibilities and is typically represented by a logical one (1) or zero (0).
bps	Transmission rate expressed in bits per second.
Bps	Transmission rate expressed in bytes per second.
buffer	Storage used to compensate for a difference in data rates or time of occurrence of events, when transmitting data from one device to another.
bulk transfer	One of the four USB transfer types. Bulk transfers are non-periodic, large bursty communication typically used for a transfer that can use any available bandwidth and can also be delayed until bandwidth is available. See also transfer type.

Term/Abbreviation	Definition
bus enumeration	Detecting, identifying, and configuring USB devices.
bus interval	The period that establishes the integral boundary of service intervals. It is equivalent to the Microframe interval (THSFRAM) defined in the USB 2.0 specification Table 7-8.
bus instance	A bus instance refers to a link and its children operating at the same Gen X speed.
byte	A data element that is 8 bits in size.
cable	Raw cable with no plugs attached.
cable assembly	Cable attached with plugs.
captive cable	Cable assembly that has a Type-A plug on one end and that is either permanently attached or has a vendor specific connector on the other end.
capabilities	Those attributes of a USB device that are administrated by the host.
CDR	Circuit that performs the Clock and Data Recovery function.
characteristics	Those qualities of a USB device that are unchangeable; for example, the device class is a device characteristic.
client	Software resident on the host that interacts with the USB system software to arrange data transfer between a function and the host. The client is often the data provider and consumer for transferred data.
component	A physical chip or circuit that contains a port.
configuring software	Software resident on the host that is responsible for configuring a USB device.
control endpoint	A pair of device endpoints with the same endpoint number that are used by a control pipe. Control endpoints transfer data in both directions and, therefore, use both endpoint directions of a device address and endpoint number combination. Thus, each control endpoint consumes two endpoint addresses.
control pipe	Same as a message pipe.
connected	A downstream device is connected to an upstream device when it is attached to the upstream device, and when the downstream device has asserted Rx terminations for SuperSpeed signaling or has asserted the D+ or D- data line in order to enter low-speed, full-speed, or high-speed signaling.
control transfer	One of the four USB transfer types. Control transfers support configuration/command/status type communications between client and function. See also transfer type.
Controlling Hub	A controlling hub is any hub whose upstream link is not in U3.
CRC	CRC-5, CRC-16, CRC-32. See Cyclic Redundancy Check.
Cyclic Redundancy Check (CRC)	A check performed on data to see if an error has occurred in transmitting, reading, or writing the data. The result of a CRC is typically stored or transmitted with the checked data. The stored or transmitted result is compared to a CRC calculated from the data to determine if an error has occurred.
D codes	The data type codes used in 8b/10b encoding.
D+ and D-	Differential pair defined in the USB 2.0 specification.
default address	An address defined by the USB Specification and used by a USB device when it is first powered or reset. The default address is 00H.
default pipe	The message pipe created by the USB system software to pass control and status information between the host and a USB device's endpoint zero.
descrambling	Restoring the pseudo-random 8-bit character to the original state. See scrambling.
detached	A downstream device is detached from an upstream device when the physical cable between the two is removed.
device	A logical or physical entity that performs one or more functions. The actual entity described depends on the context of the reference. At the lowest level, device may refer to a single hardware component, as in a memory device. At a higher level, it may refer to a collection of hardware components that perform a particular function, such as a USB interface device. At an even higher level, device may refer to the function performed by an entity attached to the USB. Devices may be physical, electrical, addressable, and logical. When used as a non-specific reference, a USB device is either a hub or a peripheral device.

Term/Abbreviation	Definition
device address	A 7-bit value representing the address of a device on the USB. The device address is the default address (00H) when the USB device is first powered or the device is reset. Devices are assigned a unique device address by the USB system software.
device endpoint	A uniquely addressable portion of a USB device that is the source or sink of information in a communication flow between the host and device. See also endpoint address.
device software	Software that is responsible for using a USB device. This software may or may not also be responsible for configuring the device for use.
DFP	Downstream Facing Port (aka downstream port). See Figure 2-1.
disconnected (unconnected)	A downstream device is disconnected from an upstream device when it is attached to the upstream device, and when the downstream device has not asserted Rx terminations for SuperSpeed signaling or has not asserted either the D+ or D- data line in order to enter low-speed, full-speed, or high-speed signaling.
downstream	The direction of data flow from the host or away from the host. A downstream port is the port on a hub electrically farthest from the host that generates downstream data traffic from the hub. Downstream ports receive upstream data traffic.
downstream port	The port on a host or a hub to which a device is connected. For example, a system's root ports are downstream ports.
downstream sublink	The collection of lanes between the DFP Tx and the UFP Rx. See Figure 2-1.
DP	Data Packet which consists of a Data Packet Header followed by a Data Packet Payload.
DPH	Data Packet Header. Contains the data packet's address, route string, length, and other information about the packet.
DPP	Data Packet Payload. Contains the data packet's data and a 32-bit CRC.
DPPABORT	Frame ordered set used to abort a data packet payload.
DPPEND	Frame ordered set used to denote the end of a data packet payload.
DPPSTART	Frame ordered set used to denote the start of a data packet payload.
driver	When referring to hardware, an I/O pad that drives an external load. When referring to software, a program responsible for interfacing to a hardware device, that is, a device driver.
DSPORT	Notation indicating the state machine of a downstream facing port of a hub. Refer to Section 10.3.
dual simplex	Two data paths that independently carry traffic in each direction.
DWORD	Double word. A data element that is two words (i.e., 4 bytes or 32 bits) in size.
dynamic insertion and removal	The ability to attach and remove devices while the host is in operation.
endpoint	See device endpoint.
endpoint address	The combination of an endpoint number and an endpoint direction on a USB device. Each endpoint address supports data transfer in one direction.
endpoint direction	The direction of data transfer on the USB. The direction can be either IN or OUT. IN refers to transfers to the host; OUT refers to transfers from the host.
endpoint number	A four-bit value between 0H and FH, inclusive, associated with an endpoint on a USB device.
Enhanced SuperSpeed	An adjective referring to any valid collection of USB defined features defined for the bus that runs over the SSRx and SSTx differential pairs in a USB 3.0 or USB 3.1 system. It is used in place of phrases like SuperSpeed/SuperSpeedPlus.
external port	See port.
frame number	The bus interval counter value within the ITP divided by 8 (integer division).
full-duplex	Computer data transmission occurring in both directions simultaneously.
full-speed	USB operation at 12 Mbps. See also low-speed and high-speed.
function	A set of one or more related interfaces on a USB device that exposes a capability to a software client.

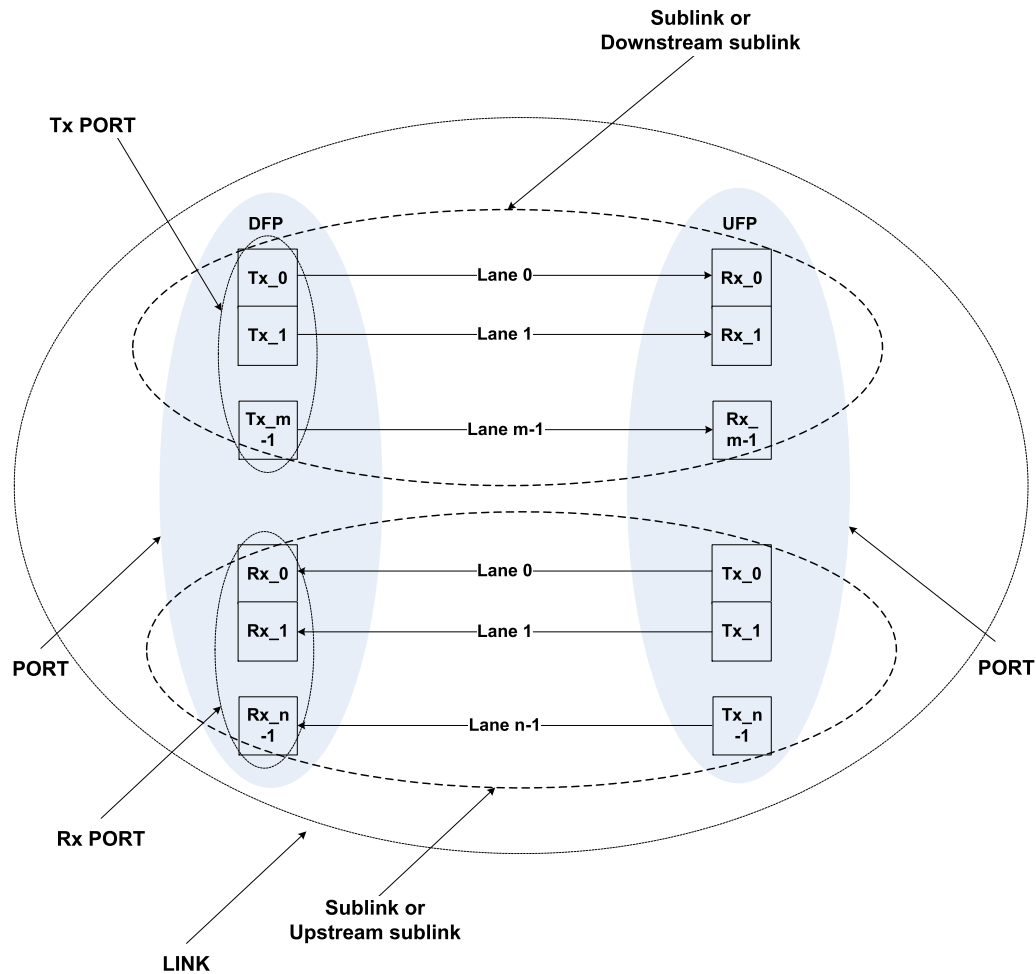
Term/Abbreviation	Definition
Gbps	Transmission rate expressed in gigabits per second (1,000,000,000 bits per second).
Gen 1	Gen 1 is an adjective used to refer to the Physical layer associated with a 5.0 Gbps signaling rate. The original USB SuperSpeed Phy and a Gen 1 Phy refer to the same Phy.
Gen 2	Gen 2 is an adjective used to refer to the Physical layer associated with a 10 Gbps signaling rate.
Gen X	Gen X is a generic term used to refer to any of the combinations Gen 1, Gen 2 or Gen 1/Gen 2 when the topic is specific to the phy layers but does not need to be specific to either Gen 1 or Gen 2. Examples include: Gen X phy/connection.
handshake packet	A packet that acknowledges or rejects a specific condition. For examples, see ACK, NRDY, or ERDY.
header	Packet header. For example, DPH, LMP, and TP are all headers.
Header Sequence Number Advertisement	The exchange of the ACK Tx Header Sequence Numbers between the link partners upon entry to U0.
high-speed	USB operation at 480 Mbps. See also low-speed and full-speed.
host	The host computer system where the USB host controller is installed. This includes the host hardware platform (CPU, bus, etc.) and the operating system in use.
host controller	The interface provided to the system to support devices on the USB.
Hot Reset	Reset mechanism using TS1/TS2 ordered sets.
HPSTART	Frame ordered set used to denote the start of a header packet.
hub	A USB device that provides additional connections to the USB.
Hub Delay Measurement (HDM)	The HDM mechanism of PTM defines a set of hub features that increases the accuracy of the Isochronous Timestamps in the ITPs that they forward downstream.
hub tier	One plus the number of USB links in a communication path between the host and a peripheral device.
ID pin	Denotes the pin on the USB 3.1 Micro connector family that is used to differentiate a USB 3.1 Micro-A plug from a USB 3.1 Micro-B plug.
Inband Reset	Mechanism that relies on SuperSpeed and/or LFPS signaling to propagate the reset across the link.
informative	Information given for illustrative purposes only and contains no requirements. See normative.
interrupt transfer	One of the four USB transfer types. Interrupt transfers have a bounded latency and are typically used to handle service needs. See also transfer type.
isochronous data	A stream of data whose timing is implied by its delivery rate.
isochronous device	An entity with isochronous endpoints, as defined in the USB Specification, that sources or sinks sampled analog streams or synchronous data streams.
isochronous sink endpoint	An endpoint that is capable of consuming an isochronous data stream that is sent by the host.
isochronous source endpoint	An endpoint that is capable of producing an isochronous data stream and sending it to the host.
isochronous transfer	One of the four USB transfer types. Isochronous transfers are used when working with isochronous data. Isochronous transfers provide periodic, continuous communication between host and device. See also transfer type.
ITP	Isochronous Timestamp Packet, sent periodically by a host to inform devices on the USB of the current bus time.
jitter	A tendency toward lack of synchronization caused by mechanical or electrical changes. More specifically, the phase shift of digital pulses over a transmission medium.
KB	Kilobyte or 1,024 bytes.

Term/Abbreviation	Definition
K codes	The control type codes used in 8b/10b encoding. SHP – start header packet SDP – start data packet END – end header or data packet EDB – end of nullified (bad) packet SLC – start link command COM – comma SKP – skip EPF – end packet framing
lane	The connection between a transmitter (Tx) of one port to the receiver (Rx) in another port. See Figure 2-1.
LBPM	LFPS Based PWM Messaging
LCSTART	Frame ordered set used to denote the start of a link command.
LDM	See Link Delay Measurement.
LDM Context	Timestamps recorded by an LDM Requester or Responder.
LDM Link Delay	The delay across the Link between a Responder and a Requester.
LDM Requester	A USB hub or device that uses the LDM protocol to communicate with an upstream host controller or hub and measure the upstream link delay.
LDM Responder	A USB host controller or hub that uses the LDM protocol to communicate with a downstream hub or device.
LFPS	Low frequency periodic signal. Used to communicate information across a link without using Enhanced SuperSpeed signaling.
LFSR	Linear feedback shift register. Used to create pseudo-random characters for scrambling.
LI	Logical Idle
Link	The connection between 2 ports. Includes both the Upstream and Downstream sublinks. See Figure 2-1.
link command	An eight-symbol sequence used for link-level flow control, retries, power management, and device removal.
Link Control Word	Two bytes with 11 bits to define the link level flow control and a 5-bit CRC5 to ensure data integrity.
Link Delay Measurement (LDM)	To perform LDM a device utilizes the PTM link level protocol to poll and automatically determine if an upstream port supports PTM, and to precisely calculate the delay of its upstream link. PTM LMPs shall be used to measure the link delay of a device upstream link.
Link Error Count	The number of events that result in entry to Recovery due to bit errors.
little endian	Method of storing data that places the least significant byte of multiple-byte values at lower storage addresses. For example, a 16-bit integer stored in little endian format places the least significant byte at the lower address and the most significant byte at the next address. See also big endian.
LMP	Link Management Packet. A type of header packet used to communicate information between a pair of link partners.
Local Rx Header Buffer Credit	The availability of a single free Rx Header Buffer of a port itself.
Logical Idle	Period of one or more symbol times when no information (packets or link commands) is being transmitted when link is in U0.
low-speed	USB operation at 1.5 Mbps. See also full-speed and high-speed.
LSb	Least significant bit.
LSB	Least significant byte.
LTSSM	Link Training and Status State Machine.

Term/Abbreviation	Definition
message pipe	A bi-directional pipe that transfers data using a request/data/status paradigm. The data has an imposed structure that allows requests to be reliably identified and communicated.
MOD	The modulus function MOD (Number, Divisor) returns the remainder portion of dividing the Number by the Divisor.
MSb	Most significant bit.
MSB	Most significant byte.
normative	Required by the specification. See also informative.
NRDY	Handshake packet indicating a negative acknowledgment.
packet	A bundle of data organized in a group for transmission. Packets typically contain three elements: control information (e.g., source, destination, and length), the data to be transferred, and error detection and correction bits.
peripheral	A physical entity that is attached to a USB cable and is currently operating as a “device” as defined in this specification.
peripheral device	A non-hub USB device that provides one or more functions to the host, such as a mass storage device.
persistent	State information (e.g., a descriptor field) that is retained and persistent through entry into and exit from D3.
Phase Locked Loop	A circuit that acts as a phase detector to keep an oscillator in phase with an incoming frequency.
physical device	A device that has a physical implementation; e.g., speakers, microphones, and CD players.
pipe	A logical abstraction representing the association between an endpoint on a device and software on the host. A pipe has several attributes; for example, a pipe may transfer data as streams (stream pipe) or messages (message pipe). See also stream pipe and message pipe.
PLL	See Phase Locked Loop.
plug	Connector attached to the cable, to be mated with the receptacle
port	Point of access to or from a system or circuit. Consists of a Tx port and an Rx port. For the USB, the point where a USB device is attached. See Figure 2-1.
PowerOn Reset (POR)	An event to restore a device to its initial state.
PPM	Parts Per Million.
Precision Time Measurement	A protocol for determining propagation delays through the USB topology with a high degree of accuracy.
PRBS	Pseudo-Random Bit Stream.
protocol	A specific set of rules, procedures, or conventions relating to format and timing of data transmission between two devices.
PTM	Refer to Precision Time Measurement.
PTM Clock	A signal source with a period of <code>tlsochTimestampGranularity</code> units, used to advance various PTM time clocks and time sources.
PTM Domain	The set of LDM Responders associated with a PTM Root.
PTM Local Time Source	The time clock associated with a LDM Requester or Responder which is advanced by PTM Clock transitions.
PTM Root	A PTM Root is a LDM Responder and the source of the bus interval boundary for a PTM Domain, i.e. the USB host controller.
receptacle	Connector mounted on the host or device, to be mated with the plug.
Remote Rx Header Buffer Credit	The availability of a single free Rx Header Buffer from a link partner.
request	A request made to a USB device contained within the data portion of a SETUP packet.
root hub	A USB hub directly attached to or integrated into the host controller.
root port	The downstream port on a root hub.

Term/Abbreviation	Definition
reserved	Reserved identifies fields and values that are not defined for use by this specification. A transmitter sets all reserved fields to zero and a receiver ignores reserved fields. A transmitter does not set a reserved value and a receiver ignores any reserved values.
Rx Header Buffer Credit Advertisement	The exchange of the Remote Rx Header Buffer Credits between the link partners upon entry to U0.
Rx Header Sequence Number	The expected header sequence number of a header packet received from a link partner.
Rx Port	The collection of receivers attached lanes in a port. See Figure 2-1.
scrambling	The process of changing an eight-bit character in a pseudo-random way. See descrambling.
SCD	SuperSpeedPlus Capability Declaration
SDP	Shielded Differential Pair.
service interval	An integral multiple of bus intervals within which a periodic endpoint must be serviced.
service jitter	The deviation of service delivery from its scheduled delivery time.
Soft Error Count	The count of the number of single bit errors that have been recovered without Recovery entry. See also Link Error Count.
SSC	Spread Spectrum Clock.
stage	One part of the sequence composing a control transfer; stages include the Setup stage, the Data stage, and the Status stage.
stream endpoint	A SuperSpeed bulk endpoint whose SuperSpeed Endpoint Companion Descriptor bmAttributes field declares a MaxStreams value that is greater than 0.
stream pipe	A pipe that transfers data as a stream of samples with no defined USB structure.
sublink	The collection Rx or Tx lanes between a DFP and a UFP. See Figure 2-1.
SuperSpeed	An adjective referring to the architectural layer portions of a device defined in this specification when operating with a Gen 1 PHY. Note previously defined as USB operation at 5 Gbps.
SuperSpeed bus instance	A bus instance operating at Gen 1 speed.
SuperSpeedPlus	An adjective referring to the architectural layer portions of a device defined in this specification when operating with a Gen 2 PHY
SuperSpeedPlus bus instance	A bus instance operating at some Gen X speed above Gen 1 speed.
synchronization type	A classification that characterizes an isochronous endpoint's capability to connect to other isochronous endpoints.
termination	Passive components attached at the end of the connections to prevent signals from being reflected or echoed.
timeout	A time interval within which an expected event shall occur.
TP	Transaction Packet. A type of header packet used to communicate information between a device and the host.
training sequences	Ordered sets for initializing bit and symbol alignment and receiver equalization. Examples are TS1, TS2, and TSEQ.
transaction	The delivery of service to an endpoint: The IN consists of an ACK TP with a response of NRDY TP, DP, or STALL TP. The OUT consists of a DP with a response of NRDY TP, an ACK TP, or STALL TP.
transfer	One or more bus transactions to move information between a software client and its function.
transfer type	Determines the characteristics of the data flow between a software client and its function. Four standard transfer types are defined: control, interrupt, bulk, and isochronous.

Term/Abbreviation	Definition
Type-A connector	The standard-A connector defined in this specification.
Type A packet	See Type A traffic class
Type B packet	See Type B traffic class
Type A traffic class	Header packet or periodic data packet
Type B traffic class	Asynchronous data packet
Type A Rx Buffer Credit	The availability of a single free Rx Buffer from a link partner to store a periodic data packet of maximum data packet payload
Type B Rx Buffer Credit	The availability of a single free Rx Buffer from a link partner to store an asynchronous data packet of maximum data packet payload
Type A Rx Buffer Credit Advertisement	The exchange of the Type A Remote Rx Buffer Credits between the link partners upon entry to U0
Type B Rx Buffer Credit Advertisement	The exchange of the Type B Remote Rx Buffer Credits between the link partners upon entry to U0
Tx Header Sequence Number	The header sequence number to be added to a header packet to be transmitted.
Tx Port	The collection of transmitters driving lanes in a port. See Figure 2-1.
upstream	The direction of data flow towards the host. An upstream port is the port on a device electrically closest to the host. Upstream ports receive downstream data traffic.
upstream port	A port that a device uses to connect to a host or a hub. The port on all devices is an upstream port.
upstream sublink	The collection of lanes between the DFP Rx and the UFP Tx. See Figure 2-1.
UFP	Upstream Facing Port (aka upstream port). See Figure 2-1.
USB 3.1 Standard-A connector	USB 3.1 host connector, supporting both SuperSpeed and SuperSpeedPlus modes.
USB 3.1 Standard-B connector	USB 3.1 standard Type-B device connector, supporting both SuperSpeed and SuperSpeedPlus modes.
USB 3.1 Micro-A plug	Part of the USB 3.1 Micro connector family for OTG use; it can be plugged into a USB 3.1 Micro-AB receptacle; it differs from the USB 3.1 Micro-B plug only in keying and ID pin connection.
USB 3.1 Micro-AB receptacle	Part of the USB 3.1 Micro connector family; it accepts either a USB 3.1 Micro-B plug or a USB 3.1 Micro-A plug.
USB 3.1 Micro-B connector	USB 3.1 device connector, supporting both SuperSpeed and SuperSpeedPlus modes.
USB 3.1 Micro connector family	All the receptacles and plugs supporting both SuperSpeed and SuperSpeedPlus modes that are used on devices, including the USB 3.1 Micro-B, USB 3.1 Micro-AB, and USB 3.1 Micro-A connectors.
USB 2.0 Standard-A connector	The Type-A connector defined by the USB 2.0 specification.
USB 2.0 Standard-B connector	The standard Type-B connector defined by the USB 2.0 specification.
USB-IF	USB Implementers Forum, Inc. is a nonprofit corporation formed to facilitate the development of USB compliant products and promote the technology.
USDPORT	Notation indicating the state machine of the upstream facing port of a peripheral device. Refer to Section 10.18.2.
USPORT	Notation indicating the state machine of the upstream facing port of a hub. Refer to Section 10.5.
UTP	Unshielded Twisted Pair.
Warm Reset	Reset mechanism using LFPS.
WORD	A data element that is 2 bytes (16 bits) in size.



IEC

Figure 2-1 – Port and Link Pictorial

Figure 2-1 illustrates the parts of a port and the connection between ports. The USB 3.1 specification only defines a port with a single Tx and Rx. The inclusion of more than one Rx/Tx pair in a port is for harmonization with the SSIC specification where multiple Rx/Tx pairs are defined.

NOTE The meanings of the terms used in this figure are not the same as used in PCIe.

3 Architectural Overview

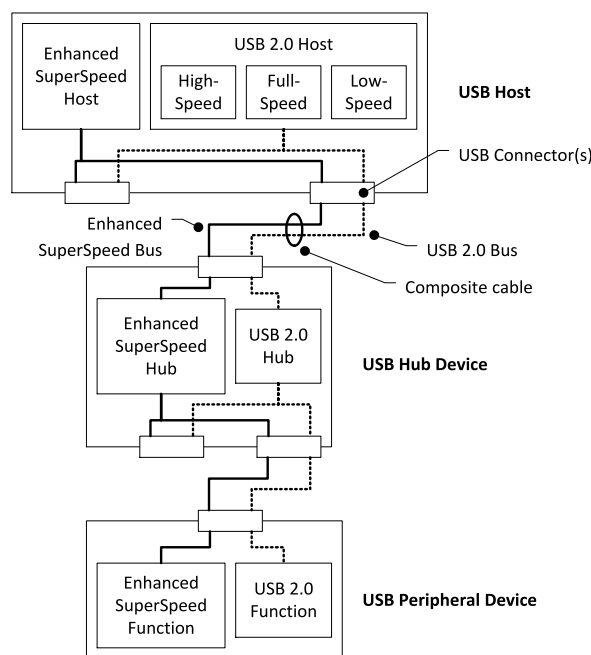
This chapter presents an overview of Universal Serial Bus 3.1 architecture and key concepts. USB 3.1 is similar to earlier versions of USB in that it is a cable bus supporting data exchange between a host computer and a wide range of simultaneously accessible peripherals. The attached peripherals share bandwidth through a host-scheduled protocol. The bus allows peripherals to be attached, configured, used, and detached while the host and other peripherals are in operation.

USB 3.1 is a dual-bus architecture that provides backward compatibility with USB 2.0. One bus is a USB 2.0 bus (see *Universal Serial Bus Specification, Revision 2.0*) and the other is an Enhanced SuperSpeed bus (see Section 3.1). This specification uses the term *Enhanced SuperSpeed* as a generic adjective referring to any valid collection of USB defined features that were defined for the bus that runs in parallel to the USB 2.0 bus in a USB 3.1 system, as defined below. This chapter is organized into several focus areas. The first focuses on architecture and concepts related to elements which span the USB 3.1 system (Section 3.1). The remaining sections focus on Enhanced SuperSpeed USB specific architecture and concepts.

Later chapters describe the various components and specific requirements of Enhanced SuperSpeed USB in greater detail. The reader is expected to have a fundamental understanding of the architectural concepts of USB 2.0. Refer to the *Universal Serial Bus Specification, Revision 2.0* for complete details.

3.1 USB 3.1 System Description

The USB 3.1 system architecture (Figure 3-1) is comprised of two simultaneously active buses: a USB 2.0 bus and an Enhanced SuperSpeed bus. The Enhanced SuperSpeed bus has similar architectural components to USB 2.0, namely:



IEC

Figure 3-1 – USB 3.1 Dual Bus System Architecture

- USB 3.1 interconnect
- USB 3.1 devices
- USB 3.1 host

The USB 3.1 interconnect is the manner in which USB 3.1 and USB 2.0 devices connect to and communicate with the USB 3.1 host. The USB 3.1 interconnect inherits core architectural elements from USB 2.0, although several are augmented to accommodate the dual bus architecture.

The baseline structural topology is the same as USB 2.0. It consists of a tiered star topology with a single host at tier 1 and hubs at lower tiers to provide bus connectivity to devices.

The USB 3.1 connection model accommodates backward and forward compatibility for connecting USB 3.1 or USB 2.0 devices into a USB 3.1 connector. Similarly, USB 3.1 devices can be attached to a USB 2.0 connector. The mechanical and electrical backward/forward compatibility for USB 3.1 is accomplished via a composite cable and associated connector assemblies that form the mechanical infrastructure for the dual-bus architecture. USB 3.1 peripheral devices accomplish backward compatibility by including both Enhanced SuperSpeed and USB 2.0 interfaces. USB 3.1 hosts have both Enhanced SuperSpeed and USB 2.0 interfaces, which are essentially parallel buses that may be active simultaneously.

The USB 3.1 connection model allows for the discovery and configuration of USB devices at the highest signaling speed supported by the peripheral device, the highest signaling rate supported by hubs between the host and peripheral device, and the current host capability and configuration.

USB 3.1 hubs are a specific class of USB device whose purpose is to provide additional connection points to the bus beyond those provided by the host. In this specification, non-hub devices are referred to as peripheral devices in order to differentiate them from hub devices. In addition, in USB 2.0 the term “function” was sometimes used interchangeably with device. In this specification a function is a logical entity within a device, see Figure 3-4.

The architectural implications of Enhanced SuperSpeed bus support on hosts, hub devices and peripheral devices are described in detail in Section 3.2.

3.1.1 USB 3.1 Physical Interface

The physical interface of USB 3.1 is comprised of USB 2.0 and Enhanced SuperSpeed portions. The USB 2.0 definitions for Electrical can be found in Chapter 7 of the USB 2.0 specification. The Enhanced SuperSpeed definitions are contained in this USB 3.1 specification and comprised of Mechanical (Chapter 5), and Physical Layer (Chapter 6) specifications. The physical layer for the Enhanced SuperSpeed bus is described in Section 3.2.1.

3.1.1.1 USB 3.1 Mechanical

The mechanical specifications for USB 3.1 cables and connector assemblies are provided in Chapter 5. All USB devices have an upstream connection. Hosts and hubs (defined below) have one or more downstream connections. Upstream and downstream connectors are not mechanically interchangeable, thus eliminating illegal loopback connections at hubs.

USB 3.1 cables have eight primary conductors: three twisted signal pairs for USB data paths and a power pair. Figure 3-2 illustrates the basic signal arrangement for the USB 3.1 cable. In addition to the twisted signal pair for USB 2.0 data path, two twisted signal pairs are used to provide the Enhanced SuperSpeed data path, one for the transmit path and one for the receive path.

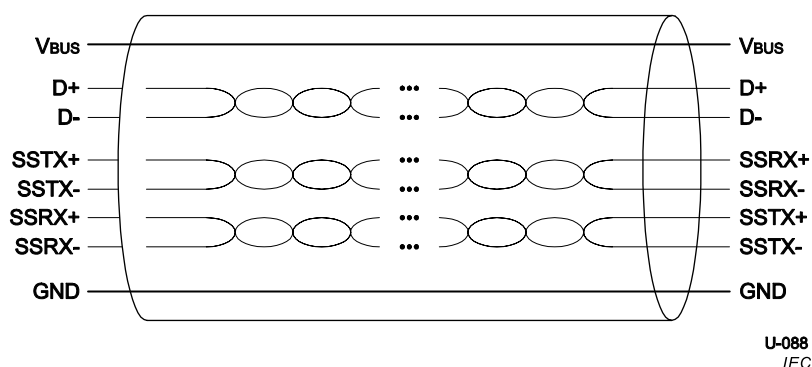


Figure 3-2 – USB 3.1 Cable

USB 3.1 receptacles (both upstream and downstream) are backward compatible with USB 2.0 connector plugs. USB 3.1 cables and plugs are not intended to be compatible with USB 2.0 upstream receptacles. As an aid to users, USB 3.1 recommends standard coloring for plastic portions of USB 3.1 plugs and receptacles.

Electrical (insertion loss, return loss, crosstalk, etc.) performance for USB 3.1 is defined with regard to raw cables, mated connectors, and mated cable assemblies, with compliance requirements using industry test specifications established for the latter two categories. Similarly, mechanical (insertion/extraction forces, durability, etc.) and environmental (temperature life, mixed flowing gas, etc.) requirements are defined and compliance established via recognized industry test specifications.

3.1.2 USB 3.1 Power

The specification covers two aspects of power:

- Power distribution over the USB deals with the issues of how USB devices consume power provided by the downstream ports to which they are connected. USB 3.1 power distribution is similar to USB 2.0, with increased supply budgets for devices operating on an Enhanced SuperSpeed bus.
- Power management deals with how hosts, devices, hubs, and the USB system software interact to provide power efficient operation of the bus. The power management of the USB 2.0 bus portion is unchanged. The use model for power management of the Enhanced SuperSpeed bus is described in Appendix C.

3.1.3 USB 3.1 System Configuration

USB 3.1 supports USB devices (all speeds) attaching and detaching from the USB 3.1 at any time. Consequently, system software must accommodate dynamic changes in the physical bus topology. The architectural elements for the discovery of attachment and removal of devices on USB 3.1 are identical to those in USB 2.0. There are enhancements provided to manage the specifics of the Enhanced SuperSpeed bus for configuration and power management.

The independent, dual-bus architecture allows for activation of each of the buses independently and provides for the attachment of USB devices to the highest speed bus available for the device.

3.1.4 USB 3.1 Architecture Summary

USB 3.1 is a dual-bus architecture that incorporates USB 2.0 and an Enhanced SuperSpeed bus. Table 3-1 summarizes the key architectural differences between an Enhanced SuperSpeed bus and a USB 2.0 bus.

Table 3-1 – Comparing Enhanced SuperSpeed Bus to USB 2.0 Bus

Characteristic	Enhanced SuperSpeed USB	USB 2.0
Data Rate	Gen 1 (5.0 Gbps), Gen 2 (10 Gbps)	low-speed (1.5 Mbps), full-speed (12 Mbps), and high-speed (480 Mbps)
Data Interface	Dual-simplex, four-wire differential signaling separate from USB 2.0 signaling Simultaneous bi-directional data flows	Half-duplex two-wire differential signaling Unidirectional data flow with negotiated directional bus transitions
Cable signal count	Six: Four for Enhanced SuperSpeed data path, two for USB 2.0 data path	Two: Two for low-speed/full-speed/high-speed (USB 2.0) data path
Bus transaction protocol	Host directed, asynchronous traffic flow Packet traffic is explicitly routed	Host directed, polled traffic flow Packet traffic is broadcast to all devices
Power management	Multi-level link power management supporting idle, sleep, and suspend states. Link-, Device-, and Function-level power management.	Port-level suspend with two levels of entry/exit latency Device-level power management
Bus power	Same as for USB 2.0 with a 50 % increase for unconfigured power and an 80 % increase for configured power	Support for low/high bus-powered devices with lower power limits for un-configured and suspended devices
Port State	Port hardware detects connect events and brings the port into operational state ready for Enhanced SuperSpeed data communication.	Port hardware detects connect events. System software uses port commands to transition the port into an enabled state (i.e., can do USB data communication flows)
Data transfer types	USB 2.0 types with Enhanced SuperSpeed constraints. Bulk has streams capability (refer to Section 3.3).	Four data transfer types: control, bulk, Interrupt, and Isochronous

3.2 Enhanced SuperSpeed Bus Architecture

Figure 3-3 illustrates the reference model for the terminology in this specification.

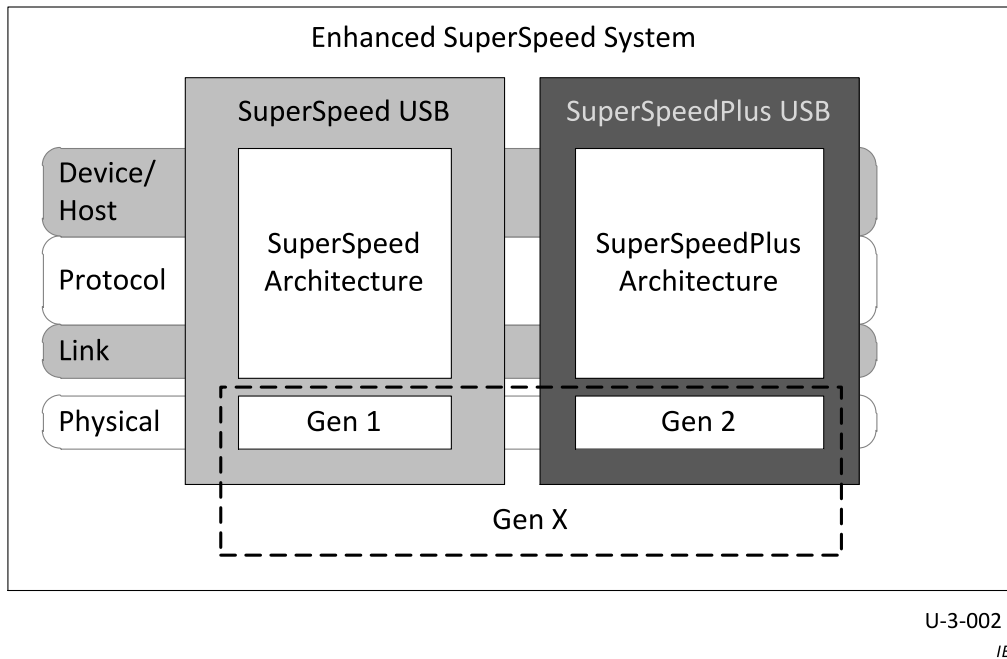


Figure 3-3 – USB 3.1 Terminology Reference Model

The Enhanced SuperSpeed bus is a layered communications architecture that is comprised of the following elements:

- **Enhanced SuperSpeed Interconnect.** The Enhanced SuperSpeed interconnect is the manner in which devices are connected to and communicate with the host over the Enhanced SuperSpeed bus. This includes the topology of devices connected to the bus, the communications layers, the relationships between them and how they interact to accomplish information exchanges between the host and devices.
- **Devices.** Enhanced SuperSpeed devices are sources or sinks of information exchanges. They implement the required device-end, Enhanced SuperSpeed communications layers to accomplish information exchanges between a driver on the host and one or more logical functions on the device.
- **Host.** An Enhanced SuperSpeed host is a source or sink of information. It implements the required host-end, Enhanced SuperSpeed communications layers to accomplish information exchanges over the bus. It owns the Enhanced SuperSpeed data activity schedule and management of the Enhanced SuperSpeed bus and all devices connected to it.

Figure 3-4 illustrates a reference diagram of the Enhanced SuperSpeed interconnect represented as communications layers through a topology of host, zero to five levels of hubs, and devices.

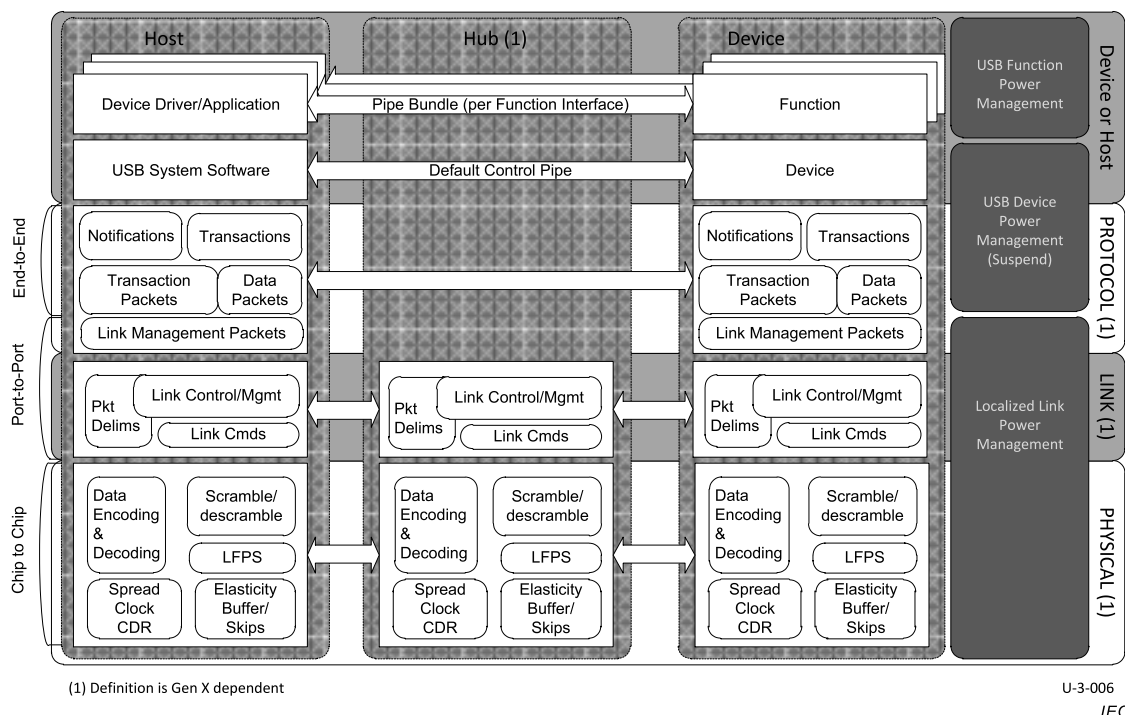


Figure 3-4 – Enhanced SuperSpeed Bus Communications Layers and Power Management Elements

The rows (device or host, protocol, link, physical) realize the communications layers of the Enhanced SuperSpeed interconnect. Sections 3.2.1 through 3.2.3 provide architectural overviews of each of the communications layers. The three, left-most columns (host, hub, and device) illustrate the topological relationships between devices connected to the Enhanced SuperSpeed bus; refer to the overview in Sections 3.2.6 through 3.2.7. The right-most column illustrates the influence of power management mechanisms over the communications layers; refer to the overview in Section 3.2.5.

3.2.1 Physical Layer

The Gen X physical layer specifications are detailed in Chapter 6. The physical layer defines the PHY portion of a port and the physical connection between a downstream facing port (on a host or hub) and the upstream facing port on a device. The Gen X physical connection is comprised of two differential data pairs, one transmit path and one receive path (see Figure 3-2).

The electrical aspects of each path are characterized as a transmitter, channel, and receiver; these collectively represent a unidirectional differential sublink. Each differential sublink is AC-coupled with capacitors located on the transmitter side of the differential sublink. The channel includes the electrical characteristics of the cables and connectors.

At an electrical level, each differential sublink is initialized by enabling its receiver termination. The transmitter is responsible for detecting the far end receiver termination as an indication of a bus connection and informing the link layer so the connect status can be factored into link operation and management.

When receiver termination is present but no signaling is occurring on the differential sublink, it is considered to be in the electrical idle state. When in this state, low frequency periodic signaling (LFPS) is used to signal initialization and power management information. The LFPS is relatively simple to generate and detect and uses very little power.

Each PHY has its own clock domain with Spread Spectrum Clocking (SSC) modulation. The USB 3.1 cable does not include a reference clock so the clock domains on each end of the

physical connection are not explicitly connected. Bit-level timing synchronization relies on the local receiver aligning its bit recovery clock to the remote transmitter's clock by phase-locking to the signal transitions in the received bit stream.

The receiver needs to reliably recover clock and data from the bit stream. For Gen 1 operation the transmitter encodes data and control characters into symbols, see section 3.2.1.1. Control symbols are used to achieve byte alignment and are used for framing data and managing the link. Special characteristics make control symbols uniquely identifiable from data symbols. For Gen 2 operation the transmitter block encodes the data and control bytes, see section 3.2.1.2. Special control blocks are used to achieve block alignment in the receiver and for managing the link.

A number of techniques are employed to improve channel performance. For example, to avoid overdriving and improve eye margin at the receiver, transmitter de-emphasis may be applied when multiple bits of the same polarity are sent. Also, equalization may be used in the receiver with the characteristics of the equalization profile being established adaptively as part of link training.

Signal (timing, jitter tolerance, etc.) and electrical (DC characteristics, channel capacitance, etc.) performance of Gen X links are defined with compliance requirements specified in terms of transmit and receive signaling eyes.

The specific Gen X physical layers are summarized in the following sections.

3.2.1.1 Gen 1 Physical Layer

The nominal signaling data rate for Gen 1 physical layer is 5 Gbps.

A Gen 1 transmitter encodes data and control characters into symbols using an 8b/10b code.

The physical layer receives 8-bit data from the link layer and scrambles the data to reduce EMI emissions. It then encodes the scrambled 8-bit data into 10-bit symbols for transmission over the physical connection. The resultant data are sent at a rate that includes spread spectrum to further lower the EMI emissions. The bit stream is recovered from the differential sublink by the receiver, assembled into 10-bit symbols, decoded and descrambled, producing 8-bit data that are then sent to the link layer for further processing.

3.2.1.2 Gen 2 Physical Layer

The nominal signaling data rate for the Gen 2 physical layer is 10 Gbps.

A Gen 2 transmitter frames data and control bytes (referred to as Symbols) by prepending a 4-bit block identifier to 16 symbols (128 bits) to create a 128b132b block. The symbols of the block may be scrambled or not depending upon their source (whether they are data or which type of control symbol). As in Gen 1 operation the resultant data are sent out across the electrical interconnect using spread spectrum clocking to lower EMI emissions. The bit stream is recovered from the electrical interconnect by the receiver and then assembled and aligned into 132 bit blocks. The data is descrambled and the identifier information and the descrambled bits are passed onto the link layer for further processing.

A Gen 2 PHY uses a protocol over LFPS signaling to negotiate to the highest common data rate capability of two connected PHYs.

3.2.2 Link Layer

The Enhanced SuperSpeed link layer specifications are detailed in Chapter 7. An Enhanced SuperSpeed link is a logical and physical connection of two ports. The connected ports are called link partners. A port has a physical part (refer to Section 3.2.1) and a logical part. The link layer defines the logical portion of a port and the communications between link partners.

The logical portion of a port has:

- State machines for managing its end of the physical connection. These include physical layer initialization and event management, i.e., connect, removal, and power management.
- State machines and buffering for managing information exchanges with the link partner. It implements protocols for flow control, reliable delivery (port to port) of packet headers, and link power management. The different link packet types are defined in Chapter 7.
- Buffering for data and protocol layer information elements.

The logical portion of a port also:

- Provides correct framing of sequences of bytes into packets during transmission; e.g., insertion of packet delimiters
- Detects received packets, including packet delimiters and error checks of received header packets (for reliable delivery)
- Provides an appropriate interface to the protocol layer for pass-through of protocol-layer packet information exchanges

The physical layer provides the logical port an interface through which it is able to:

- Manage the state of its PHY (i.e., its end of the physical connection), including power management and events (connection, removal, and wake).
- Transmit and receive byte streams, with additional signals that qualify the byte stream as control sequences or data. The physical layer includes discrete transmit and receive physical links, therefore, a port is able to simultaneously transmit and receive control and data information.

The protocol between link partners uses specific encoded control sequences. Note that control sequences are encoded to be tolerant to a single bit error. Control sequences are used for port-to-port command protocol, framing of packet data (packet delimiters), etc. There is a link-partner protocol for power management that uses packet headers.

3.2.3 Protocol Layer

The protocol layer specifications for Enhanced SuperSpeed are detailed in Chapter 7. This protocol layer defines the “end-to-end” communications rules between a host and device (see Figure 3-4).

The Enhanced SuperSpeed protocol provides for application data information exchanges between a host and a device endpoint. This communications relationship is called a pipe. It is a host-directed protocol, which means the host determines when application data is transferred between the host and device. The Enhanced SuperSpeed protocol is not a polled protocol, as a device is able to asynchronously request service from the host on behalf of a particular endpoint.

All protocol layer communications are accomplished via the exchange of packets. Packets are sequences of data bytes with specific control sequences which serve as delimiters managed by the link layer. Host transmitted protocol packets are routed through intervening hubs directly to a peripheral device. They do not traverse bus paths that are not part of the direct path between the host and the target peripheral device. A peripheral device expects it has been targeted by any protocol layer packet it receives. Device transmitted protocol packets simply flow upstream through hubs to the host.

Packet headers are the building block of the protocol layer. They are fixed size packets with type and subtype field encodings for specific purposes. A small record within a packet header is utilized by the link layer (port-to-port) to manage the flow of the packet from port to port. Packet headers are delivered through the link layer (port-to-port) reliably. The remaining fields are utilized by the end-to-end protocol.

Application data is transmitted within data packet payloads. Data packet payloads are preceded (in the protocol) by a specifically encoded data packet headers. Data packet payloads are not delivered reliably through the link layer (however, the accompanying data packet headers are delivered reliably). The protocol layer supports reliable delivery of data packets via explicit acknowledgement (header) packets and retransmission of lost or corrupt data. Not all data information exchanges utilize data acknowledgements. Packets moving over the Enhanced SuperSpeed bus (e.g. through multiple hubs) are strongly ordered, end-to-end. They arrive at the recipient device or host in the same order that the host or device endpoint originally transmitted them.

Data may be transmitted in bursts of back-to-back sequences of data packets (depending on the scheduling by the host). The protocol allows efficient bus utilization by concurrently transmitting and receiving over the link. For example, a transmitter (host or device) can burst multiple packets of data back-to-back while the receiver can transmit data acknowledgements without interrupting the burst of data packets. The number of data packets in a specific burst is scheduled by the host. Furthermore, an Enhanced SuperSpeed host may simultaneously schedule multiple OUT bursts to be active at the same time as at least one IN burst. See section 3.2.6.1 for a summary of valid combinations of Enhanced SuperSpeed topologies and section 3.2.7 for limitations of hosts to schedule combinations of bursts to those devices.

The protocol provides flow control support for some transfer types. A device-initiated flow control is signaled by a device via a defined protocol packet. A host-initiated flow control event is realized via the host schedule (host will simply not schedule information flows for a pipe unless it has data or buffering available). On reception of a flow control event, the host will remove the pipe from its schedule. Resumption of scheduling information flows for a pipe may be initiated by the host or device. A device endpoint will notify a host of its readiness (to source or sink data) via an asynchronously transmitted “ready” packet. On reception of the “ready” notification, the host will add the pipe to its schedule, assuming that it still has data or buffering available.

Independent information streams can be explicitly delineated and multiplexed on the bulk transfer type. This means through a single pipe instance, more than one data stream can be tagged by the source and identified by the sink. The protocol provides for the device to direct which data stream is active on the pipe.

Devices may asynchronously transmit notifications to the host. These notifications are used to convey a change in the device or function state.

3.2.3.1 SuperSpeed Protocol

All packets of a SuperSpeed burst on a SuperSpeed bus will not have packets from other endpoint flows intermingled within the burst.

A SuperSpeed host transmits a special packet header to the SuperSpeed bus (from the root port) that includes the host's timestamp. The value in this packet is used to keep SuperSpeed devices (that need to) in synchronization with the host. In contrast to other packet types, the timestamp packet is forwarded down all paths not in a low power state. The SuperSpeed timestamp packet transmission is scheduled by the host at a specification determined period.

3.2.3.2 SuperSpeedPlus Protocol

The SuperSpeedPlus protocol inherits almost all of the SuperSpeed protocol. SuperSpeedPlus protocol defines the following features on the SuperSpeed protocol base:

- ACK Transaction Packets (TPs) and Data Packets (DPs) are annotated with the transfer type of the endpoint and upstream flowing asynchronous DPs on SuperSpeedPlus bus segments are annotated with an arbitration weight (AW) used by SuperSpeedPlus hub arbiters for fair service.

- Relaxed Enhanced SuperSpeed host concurrent endpoint scheduling rules for SuperSpeedPlus endpoints. This decouples asynchronous and periodic transaction scheduling and allows concurrent IN endpoint scheduling for SuperSpeedPlus endpoints and for SuperSpeed endpoints on different SuperSpeed bus-instances (see section 3.2.6.4 for more information).
- Packets to or from simultaneously active endpoints moving over a SuperSpeedPlus bus can be intermingled with each other and reordered (with respect to different endpoints flows) by each SuperSpeedPlus hub they transit.

The SuperSpeedPlus bus also uses the host timestamp packet feature defined for the SuperSpeed bus as described in Section 3.2.3.1 it uses the Precision Time Measurement (PTM) feature defined in Section 8.4.8 to determine the link delay. In addition, SuperSpeedPlus hubs are required to update the host timestamp packet based on the link delay and the delay in the hub before forwarding it as described in Section 10.9.4.4.1.

3.2.4 Robustness

There are several attributes of Enhanced SuperSpeed USB that contribute to its robustness:

- Signal integrity using differential drivers, receivers, and shielding
- CRC protection for header and data packets
- Link level header packet retries to ensure their reliable delivery
- End-to-end protocol retries of data packets to ensure their reliable delivery
- Detection of attach and detach and system-level configuration of resources
- Data and control pipe constructs for ensuring independence from adverse interactions between functions

3.2.4.1 Error Detection

The Gen X physical layer bit error rate is expected to be less than one in 10^{12} bits. To provide protection against occasional bit errors, packet framing and link commands have sufficient redundancy to tolerate single-bit errors. Each packet includes a CRC to provide error detection of multiple bit errors. When data integrity is required an error recovery procedure may be invoked in hardware or software.

The protocol includes separate CRCs for headers and data packet payloads. Additionally, the link control word (in each packet header) has its own CRC. A failed CRC in the header or link control word is considered a serious error which will result in a link level retry to recover from the error. A failed CRC in a data packet payload is considered to indicate corrupted data and can be handled by the protocol layer with a request to resend the data packet.

The link and physical layers work together to provide reliable packet header transmission. The physical layer provides an error rate that does not exceed (on average) one bit error in every 10^{12} bits. The link layer uses error checking to catch errors and retransmission of the packet header further reducing the packet header error rate.

3.2.4.2 Error Handling

Errors may be handled in hardware or software. Hardware error handling includes reporting and retrying of failed header packets. A USB host controller will try a transmission that encounters errors up to three times before informing the client software of the failure. The client software can recover in an implementation-specific way.

3.2.5 Enhanced SuperSpeed Power Management

Enhanced SuperSpeed provides power management at distinct areas in the bus architecture, link, device, and function (refer to Figure 3-4). These power management areas are not tightly

coupled but do have dependencies; these mostly deal with allowable power state transitions based on dependencies with power states of links, devices, and functions.

Link power management occurs asynchronously on every link (i.e., locally) in the connected hierarchy. The link power management policy may be driven by the device, the host or a combination of both. The link power state may be driven by the device or by the downstream port inactivity timers that are programmable by host software. The link power states are propagated upwards by hubs (e.g., when all downstream ports are in a low power state, the hub is required to transition its upstream port to a low power state). The decisions to change link power states are made locally. The host does not directly track the individual link power states. Since only those links between the host and device are involved in a given data exchange, links that are not being utilized for data communications can be placed in a lower power state.

The host does not directly control or have visibility of the individual links' power states. This implies that one or more links in the path between the host and device can be in reduced power state when the host initiates a communication on the bus. There are in-band protocol mechanisms that force these links to transition to the operational power state and notify the host that a transition has occurred. The host knows (can calculate) the worst-case transition time to bring a path to any specific device to an active, or ready state, using these mechanisms. Similarly, a device initiating a communication on the bus with its upstream link in a reduced power state, will first transition its link into an operational state which will cause all links between it and the host to transition to the operational state.

The key points of link power management include:

- Devices send asynchronous ready notifications to the host.
- Packets are routed, allowing links that are not involved in data communications to transition to and/or remain in a low power state.
- Packets that encounter ports in low power states cause those ports to transition out of the low power state with indications of the transition event.
- Multiple host or device driven link states with progressively lower power at increased exit latencies.

As with the USB 2.0 bus, devices can be explicitly suspended via a similar port-suspend mechanism. This sets the link to the lowest link power state and sets a limit on the power draw requirement of the device.

Enhanced SuperSpeed provides support for function power management in addition to device power management. For multi-function (composite) devices, each function can be independently placed into a lower power state. Note that a device shall transition into the suspended state when directed by the host via a port command. The device shall not automatically transition into the suspended state when all the individual functions within it are suspended.

Functions on devices may be capable of being remote wake sources. The remote-wake feature on a function must be explicitly enabled by the host. Likewise, a protocol notification is available for a function to signal a remote wake event that can be associated with the source function. All remote-wake notifications are functional across all possible combinations of individual link power states on the path between the device and host.

3.2.6 Devices

All Enhanced SuperSpeed devices share their base architecture with USB 2.0. They are required to carry information for self-identification and generic configuration. They are also required to demonstrate behavior consistent with the defined Enhanced SuperSpeed Device States.

All devices are assigned a USB address when enumerated by the host. Each device supports one or more pipes through which the host may communicate with the device. All devices must support a designated pipe at endpoint zero to which the device's Default Control Pipe is attached. All devices support a common access mechanism for accessing information through this control pipe. Refer to Chapter 8 for a complete definition of a control pipe.

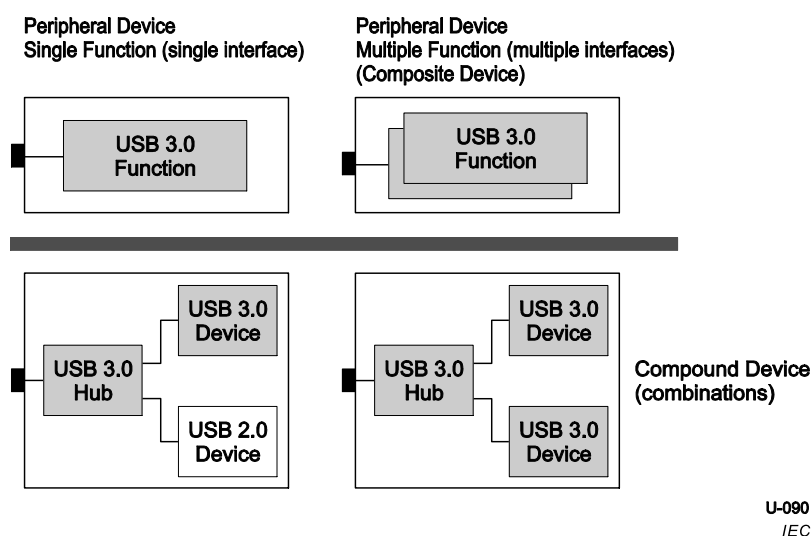
Enhanced SuperSpeed inherits the categories of information that are supported on the default control pipe from USB 2.0.

The USB 3.1 specification defines two types of USB devices that can be connected to an Enhanced SuperSpeed host. These are described briefly below.

3.2.6.1 Peripheral Devices

A USB 3.1 peripheral device must provide support for both Enhanced SuperSpeed and at least one of the USB 2.0 speeds. The minimal functional requirement for the USB 2.0 speed implementation is for a device to be detected on a USB 2.0 host and allow system software to direct the user to attach the device to an Enhanced SuperSpeed port. A device implementation may provide appropriate full functionality when operating in the implemented USB 2.0 speed mode. Simultaneous operation of Enhanced SuperSpeed and USB 2.0 speed modes is not allowed for peripheral devices.

USB 3.1 devices within a single physical package (i.e., a single peripheral) can consist of a number of functional topologies including single function, multiple functions on a single peripheral device (composite device), and permanently attached peripheral devices behind an integrated hub (compound device) (see Figure 3-5).



**Figure 3-5 – Examples of Supported USB 3.1
USB Physical Device Topologies**

An Enhanced SuperSpeed portion of a peripheral device may only be assembled into one of the following configurations:

- **SuperSpeed Only Peripheral Device.** This device implementation is comprised of a Gen 1 only PHY and conforms to the SuperSpeed link, protocol and device specifications; see Figure 3-6.

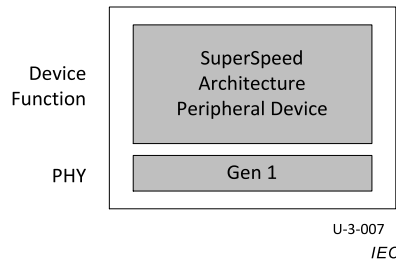


Figure 3-6 – SuperSpeed Only Enhanced SuperSpeed Peripheral Device Configuration

- Enhanced SuperSpeed Device. This is an attachable device that must implement both SuperSpeed and SuperSpeedPlus device architecture and at all Gen X speeds; see Figure 3-7.

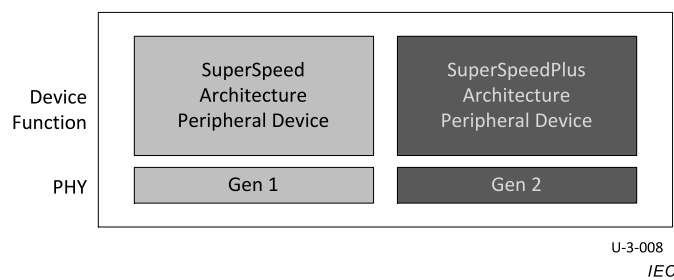


Figure 3-7 – Enhanced SuperSpeed Device Configuration

For additional information about USB 3.1 Hubs, see section 3.2.6.2.

3.2.6.2 Hubs

The specifications for the Enhanced SuperSpeed portion of a USB 3.1 hub are detailed in Chapter 9. Hubs have always been a key element in the plug-and-play architecture of the USB. Hosts provide an implementation-specific number of downstream ports to which devices can be attached. Hubs provide additional downstream ports so they provide users with a simple connectivity expansion mechanism for the attachment of additional devices to the USB.

In order to support the dual-bus architecture of USB 3.1, a USB 3.1 hub is the logical combination of two hubs: a USB 2.0 hub and an Enhanced SuperSpeed hub (see the hub in Figure 3-1). The power and ground from the cable connected to the upstream port are shared across both units within the USB 3.1 hub. The USB 2.0 hub unit is connected to the USB 2.0 data lines and the Enhanced SuperSpeed hub is connected to the SuperSpeed data lines. A USB 3.1 hub connects upstream as two devices; an Enhanced SuperSpeed hub on the Enhanced SuperSpeed bus and a USB 2.0 hub on the USB 2.0 bus.

A USB 3.1 hub has one upstream port and one or more downstream ports. All ports operate at all USB 2.0 speeds and at all Gen X speeds. The Enhanced SuperSpeed hub manages the Enhanced SuperSpeed portions of the downstream ports and the USB 2.0 hub manages the USB 2.0 portions of the downstream ports. Each physical port has bus-specific control/status registers. Refer to the Universal Serial Bus Specification, Revision 2.0 for details on the USB 2.0 hub. Hubs detect device attach, removal, and remote-wake events on downstream ports and enable the distribution of power to downstream devices. It also has hardware support for reset and suspend/resume signaling.

An Enhanced SuperSpeed hub has a hub controller that responds to standard, hub-specific status/control commands that are used by a host to configure the hub and to monitor and control its downstream ports.

An Enhanced SuperSpeed hub operates as a SuperSpeed hub when its upstream facing port is operating at Gen 1 speed and operates as a SuperSpeedPlus hub when its upstream facing port is operating in any Gen X speeds beyond Gen 1.

3.2.6.3 SuperSpeed Hub

A SuperSpeed hub consists of two logical components: a SuperSpeed hub controller and a SuperSpeed repeater/forwarder. The hub repeater/forwarder is a protocol-controlled router between the SuperSpeed upstream port and downstream ports. The repeater architecture allows a host to schedule simultaneous out-bound bursts to different endpoints on a SuperSpeed bus. It limits the number of simultaneous in-bound bursts from different endpoints on a SuperSpeed bus to one.

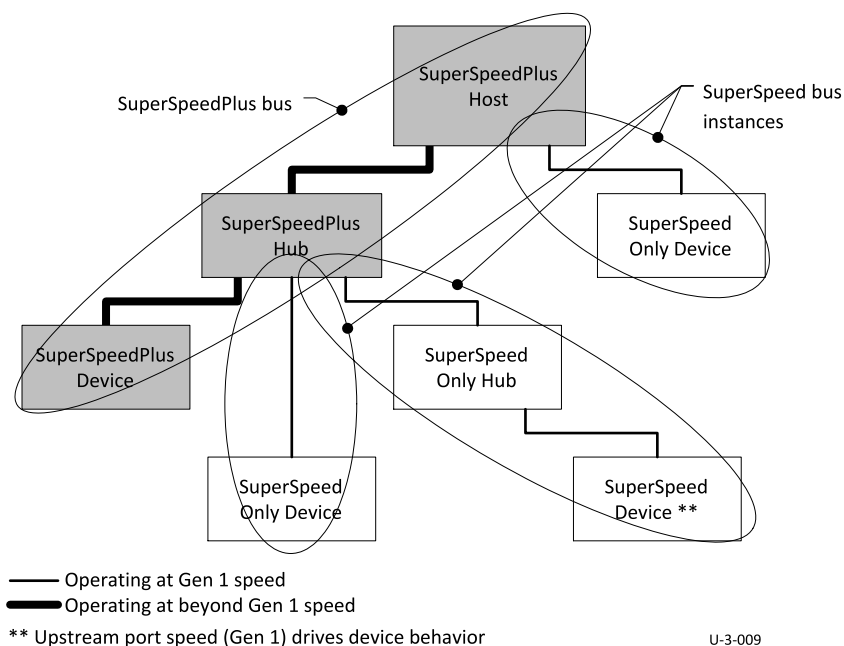
SuperSpeed hubs actively participate in the (end-to-end) protocol in several ways, including:

- Routes out-bound packets to explicit downstream ports.
- Routes in-bound packets from a downstream port to the upstream port.
- Propagates the timestamp packet to all downstream ports not in a low-power state.
- Detects when packets encounter a port that is in a low-power state. The hub transitions the targeted port out of the low-power state and notifies the host and device (in-band) that the packet encountered a port in a low-power state.

3.2.6.4 SuperSpeedPlus Hub

A SuperSpeedPlus hub serves a special role when its upstream facing port is operating at a Gen 2 or beyond speed (not Gen 1 speed). A SuperSpeedPlus hub isolates downstream signaling environments from the upstream signaling environment utilizing a store-and-forward architecture. Figure 3-8 illustrates a SuperSpeedPlus hub with an upstream facing port running in beyond Gen 1 speed, supporting downstream devices operating at both Gen 1 speed and beyond Gen 1 speeds.

In contrast to the SuperSpeed hub, which is characterized as a repeater/forwarder hub architecture, the SuperSpeedPlus hub is characterized as a store-and-forward hub because it can receive one or more entire DPs before transmitting up or downstream. The store-and-forward architecture of a SuperSpeedPlus hub allows a host to schedule multiple endpoint bursts, across multiple endpoints, for both in-bound and out-bound flows, as long as they are to SuperSpeedPlus endpoints. The SuperSpeedPlus host is also able to use the same SuperSpeedPlus scheduling rules across SuperSpeed endpoints on different downstream SuperSpeed bus instances. The SuperSpeedPlus host must use SuperSpeed only bus scheduling rules for all SuperSpeed endpoints on the same SuperSpeed bus instance.



**Figure 3-8 – Multiple SuperSpeed Bus Instances
in an Enhanced SuperSpeed System**

A SuperSpeedPlus hub consists of three logical components: a SuperSpeedPlus hub controller, a SuperSpeedPlus upstream controller and a SuperSpeedPlus downstream controller (one for each downstream facing port).

A SuperSpeedPlus hub is required to implement USB PTM (Precision Time Management).

SuperSpeedPlus hubs actively participate in the (end-to-end) protocol in several ways, including:

- Routes and preserves ordering (within an endpoint flow) of out-bound packets (TPs, DPs) from the upstream port to specific downstream ports
- Routes in-bound packets and preserves ordering (within an endpoint flow) to the upstream port, via:
 - Providing fair-service for simultaneously active, in-bound, asynchronous transfer type endpoint data flows, independent of device operating speed or location within the topology.
 - Providing strict-priority for simultaneously active, in-bound, periodic transfer type endpoint data flows, independent of device operating speed or location within the topology.
- SuperSpeedPlus hubs ensure compatibility with SuperSpeed devices connected to its downstream facing ports.
- Detects when packets encounter a port that is in a low-power state. The hub transitions the targeted port out of the low-power state and notifies the host and device (in-band) that the packet encountered a port in a low-power state.
- Updates the host timestamp packet based on the link delay and the delay in the hub before forwarding it to all downstream ports that are not in a low-power state.

3.2.7 Hosts

A USB 3.1 host interacts with USB devices through a host controller. To support the dual-bus architecture of USB 3.1, a host controller must include both Enhanced SuperSpeed and USB 2.0 elements, which can simultaneously manage control, status and information exchanges between the host and devices over each bus.

The host includes an implementation-specific number of root downstream ports for Enhanced SuperSpeed and USB 2.0. Through these ports the host:

- Detects the attachment and removal of USB devices
- Manages control flow between the host and USB devices
- Manages data flow between the host and USB devices
- Collects status and activity statistics
- Provides power to attached USB devices
- A SuperSpeedPlus host is required to implement USB PTM (Precision Time Management)

USB System Software inherits its architectural requirements from USB 2.0, including:

- Device enumeration and configuration
- Scheduling of periodic and asynchronous data transfers
- Device and function power management
- Device and bus management information

3.3 Enhanced SuperSpeed Bus Data Flow Models

The data flow models for the Enhanced SuperSpeed bus are described in Chapter 4. The Enhanced SuperSpeed bus inherits the data flow models from USB 2.0, including:

- Data and control exchanges between the host and devices are via sets of either unidirectional or bi-directional pipes.
- Data transfers occur between host software and a particular endpoint on a device. The endpoint is associated with a particular function on the device. These associations between host software to endpoints related to a particular function are called pipes. A device may have more than one active pipe. There are two types of pipes: stream and message. Stream data has no USB-defined structure, while message does. Pipes have associations of data bandwidth, transfer service type (see below), and endpoint characteristics, like direction and buffer size.
- Most pipes come into existence when the device is configured by system software. However, one message pipe, the Default Control Pipe, always exists once a device has been powered and is in the default state, to provide access to the device's configuration, status, and control information.
- A pipe supports one of four transfer types as defined in USB 2.0 (bulk, control, interrupt, and isochronous). The basic architectural elements of these transfer types are unchanged from USB 2.0.
- The bulk transfer type has an extension for Enhanced SuperSpeed protocol called Streams. Streams provide in-band, protocol-level support for multiplexing multiple independent logical data streams through a standard bulk pipe.

4 Enhanced SuperSpeed Data Flow Model

This chapter presents a high-level description of how data and information move across the Enhanced SuperSpeed bus. Consult the Protocol Layer Chapter for details on the low-level protocol. This chapter provides device framework overview information that is further expanded in the Device Framework Chapter. All implementers should read this chapter to understand the key concepts of the Enhanced SuperSpeed bus.

4.1 Implementer Viewpoints

The Enhanced SuperSpeed bus is very similar to USB 2.0 in that it provides communication services between a USB Host and attached USB Devices. The communication model view preserves the USB 2.0 layered architecture and basic components of the communication flow

(i.e., point-to-point, same transfer types, etc.). Refer to Chapter 5 in the *Universal Serial Bus Specification, Revision 2.0* for more information about the USB 2.0 communication flow.

This chapter describes the differences (from USB 2.0) of how data and control information are communicated between an Enhanced SuperSpeed Host and its attached Enhanced SuperSpeed Devices. In order to understand Enhanced SuperSpeed data flow, the following concepts are useful:

- **Communication Flow Models:** Section 4.2 describes how communication flows between the host and devices over the Enhanced SuperSpeed bus.
- **Enhanced SuperSpeed Protocol Overview:** Section 4.3 gives a high level overview of the Enhanced SuperSpeed protocol and compares it to the USB 2.0 protocol.
- **Generalized Transfer Description:** Section 4.4 provides an overview of how data transfers work using the Enhanced SuperSpeed protocol and subsequent sections define the operating constraints for each transfer type.
- **Device Notifications:** Section 4.4.9 provides an overview of Device Notifications, a feature which allows a device to asynchronously notify its host of events or status on the device.
- **Reliability and Efficiency:** Sections 4.4.10 and 4.4.11 summarize the information and mechanisms available for the Enhanced SuperSpeed bus to ensure reliability and increase efficiency.

4.2 Enhanced SuperSpeed Communication Flow

The Enhanced SuperSpeed Bus retains the familiar concepts, mechanisms and support for endpoints, pipes, and transfer types. Refer to the *Universal Serial Bus Specification, Revision 2.0* for details. As in USB 2.0, the ultimate consumer/producer of data is an endpoint.

The endpoint's characteristics (Max Packet Size, Burst Size, etc.) are reported in the endpoint descriptor and the SuperSpeed Endpoint Companion Descriptor. As in USB 2.0, the endpoint is identified using an addressing triple {Device Address, Endpoint Number, Direction}.

All Enhanced SuperSpeed devices must implement at least the Default Control Pipe (endpoint zero). The Default Control Pipe is a control pipe as defined in the *Universal Serial Bus Specification, Revision 2.0*.

4.2.1 Pipes

An Enhanced SuperSpeed pipe is an association between an endpoint on a device and software on the host. Pipes represent the ability to move data between software on the host via a memory buffer and an endpoint on a device and have the same behavior as defined in the *Universal Serial Bus Specification, Revision 2.0*. The main difference is that when a non-isochronous Enhanced SuperSpeed endpoint is busy it returns a Not Ready (NRDY) response and must send an Endpoint Ready (ERDY) notification when it wants to be serviced again. The host will then reschedule the transaction at the next available opportunity within the constraints of the transfer type.

4.3 Enhanced SuperSpeed Protocol Overview

As mentioned in the Architecture Overview Chapter, the Enhanced SuperSpeed protocol is architected to take advantage of the dual-simplex physical layer. All USB 2.0 transfer types are supported by the Enhanced SuperSpeed protocol. The differences between the USB 2.0 protocol and the Enhanced SuperSpeed protocol are first discussed followed by a brief description of the packets used in the Enhanced SuperSpeed protocol.

4.3.1 Differences from USB 2.0

The Enhanced SuperSpeed bus is backward compatible with USB 2.0 at the framework level. However, there are some fundamental differences between the USB 2.0 and the Enhanced SuperSpeed protocol:

- USB 2.0 uses a three-part transaction (Token, Data, and Handshake) while the Enhanced SuperSpeed protocol uses the same three parts differently. For OUTs, the token is incorporated in the data packet; while for INs, the Token is replaced by a handshake.
- USB 2.0 does not support bursting while the Enhanced SuperSpeed protocol supports continuous bursting.
- USB 2.0 is a half-duplex broadcast bus while the Enhanced SuperSpeed bus is a dual-simplex unicast bus which allows concurrent IN and OUT transactions.
- USB 2.0 uses a polling model while the Enhanced SuperSpeed protocol uses asynchronous notifications.
- USB 2.0 does not have a Streaming capability while the Enhanced SuperSpeed protocol supports Streaming for bulk endpoints.
- USB 2.0 offers no mechanism for isochronous capable devices to enter the low power USB bus state between service intervals. The Enhanced SuperSpeed bus allows isochronous capable devices to autonomously enter low-power link states between service intervals or within a service interval. An Enhanced SuperSpeed host shall transmit a PING packet to the targeted isochronous device before the service interval to allow time for the path to transition back to the active power state before initiating the isochronous transfer.
- USB 2.0 offers no mechanism for a device to inform the host how much latency the device can tolerate if the system enters lower system power states. Thus a host may not enter lower system power states as it might impact a device's performance because it lacks an understanding of a device's power policy. USB 3.1 provides a mechanism to allow Enhanced SuperSpeed devices to inform the host of their latency tolerance using Latency Tolerance Messaging. The host may use this information to establish a system power policy that accounts for the devices' latency tolerance.
- USB 2.0 transmits SOF/uSOF at fixed 1 ms/125 μ s intervals, with very tight duration and jitter specifications. Enhanced SuperSpeed links have a similar mechanism called an Isochronous Timestamp Packet (ITP) that is transmitted by a host. The USB host may send an Isochronous Timestamp Packet (ITP) within a relaxed timing window from a bus interval boundary. USB 3.0 added a mechanism for devices to send a Bus Interval Adjustment Message that is used by the host to adjust its 125 μ s bus interval up to $\pm 13,333 \mu$ s. A device may change the interval with small finite adjustments.
- USB 3.1 defines an optional-normative for hosts and hubs operating at Gen 1 speed and required for hosts and hubs operating at Gen 2 speed, Precision Time Measurement (PTM) capability for Enhanced SuperSpeed devices, enabling the host, hubs, and devices to accurately determine propagation delays through the USB topology.
- USB 2.0 power management, including Link Power Management, is always directly initiated by the host. The Enhanced SuperSpeed bus supports link-level power management that may be initiated from either end of the link. Thus, each link can independently enter low-power states whenever idle and exit whenever communication is needed.
- USB 2.0 handles transaction error detection and recovery and flow control only at the end-to-end level for each transaction. The Enhanced SuperSpeed protocol splits these functions between the end-to-end and link levels.

4.3.1.1 Comparing USB 2.0 and Enhanced SuperSpeed Transactions

The Enhanced SuperSpeed dual-simplex physical layer allows information to travel simultaneously in both directions. The Enhanced SuperSpeed protocol allows the transmitter to send multiple data packets before receiving a handshake. For OUT transfers, the information contained in the USB 2.0 Token is incorporated in the data packet header so a separate Token is not required. For IN transfers, a handshake is sent to the device to request data. The device may respond by either returning data, returning a STALL handshake, or by returning a Not Ready (NRDY) handshake to defer the transfer until the device is ready.

The USB 2.0 broadcasts packets to all enabled downstream ports. Every device is required to decode the address triple {device address, endpoint, and direction} of each packet to determine if it needs to respond. The Enhanced SuperSpeed bus unicasts the packets;

downstream packets are sent over a directed path between the host and the targeted device while upstream packets are sent over the direct path between the device and the host. Enhanced SuperSpeed packets contain routing information that the hubs use to determine which downstream port the packet needs to traverse to reach the device. There is one exception; the Isochronous Timestamp Packet (ITP) is multicast to all active ports.

USB 2.0 style polling has been replaced with asynchronous notifications. The Enhanced SuperSpeed transaction is initiated by the host making a request followed by a response from the device. If the device can honor the request, it either accepts or sends data. If the endpoint is halted, the device shall respond with a STALL handshake. If it cannot honor the request due to lack of buffer space or data, it responds with a Not Ready (NRDY) to tell the host that it is not able to process the request at this time. When the device can honor the request, it will send an Endpoint Ready (ERDY) to the host which will then reschedule the transaction.

The move to unicasting and the limited multicasting of packets together with asynchronous notifications allows links that are not actively passing packets to be put into reduced power states. Upstream and downstream ports cooperate to place their link into a reduced power state that hubs will propagate upstream. Allowing link partners to control their independent link power state and a hub's propagating the highest link power state seen on any of its downstream ports to its upstream port, puts the bus into the lowest allowable power state rapidly.

4.3.1.2 Introduction to Enhanced SuperSpeed Packets

Enhanced SuperSpeed packets start with a 16-byte header. Some packets consist of a header only. All headers begin with the Packet Type information used to decide how to handle the packet. The header is protected by a 16-bit CRC (CRC-16) and ends with a 2-byte link control word. Depending on the Type, most packets contain routing information (Route String) and a device address triple {device address, endpoint number, and direction}. The Route String is used to direct packets sent by the host on a directed path through the topology. Packets sent by the device are implicitly routed as the hub always forwards a packet seen on any downstream port to its upstream port. There are four basic types of packets: Link Management Packets, Transaction Packets, Data Packets, and Isochronous Timestamp Packets:

- A Link Management Packet (LMP) only traverses a pair of directly connected ports and is primarily used to manage that link.
- A Transaction Packet (TP) traverses all the links in the path directly connecting the host and a device. It is used to control the flow of data packets, configure devices and hubs, etc. Note that a Transaction Packet does not have a data payload.
- A Data Packet (DP) traverses all the links in the path directly connecting the host and a device. Data Packets consist of two parts: a Data Packet Header (DPH) which is similar to a TP and a Data Packet Payload (DPP) which consists of the data block plus a 32-bit CRC (CRC-32) used to ensure the data's integrity.
- An Isochronous Timestamp Packet (ITP) is a multicast packet sent by an Enhanced SuperSpeed host/hub to all active links.

4.4 Generalized Transfer Description

Each non-isochronous data packet sent to a receiver is acknowledged by a handshake (called an ACK transaction packet). However, due to the fact that the Enhanced SuperSpeed bus has independent transmit and receive paths, the transmitter does not have to wait for an explicit handshake for each data packet transferred before sending the next packet.

The Enhanced SuperSpeed bus preserves all of the basic data flow and transfer concepts defined in USB 2.0, including the transfer types, pipes, and basic data flow model. The differences with USB 2.0 are discussed in this section, starting at the protocol level, followed by transfer type constraints.

The USB 2.0 specification utilizes a serial transaction model. This essentially means that a host starts and completes one bus transaction {Token, Data, Handshake} before starting the next transaction. Split transactions also adhere to this same model since they are comprised of complete high-speed transactions {Token, Data, Handshake} that are completed under the same model as all other transactions.

The Enhanced SuperSpeed protocol improves on the USB 2.0 transaction protocol by using the independent transmit and receive paths. The result is that the Enhanced SuperSpeed USB transaction protocol is essentially a split-transaction protocol that generally allows more than one IN or OUT “bus transaction to be active on the bus at the same time. Note that a SuperSpeed link has a restriction that at most one IN “bus transaction” can be active on that SuperSpeed bus instance. The order in which a device responds to transactions is fixed on a per endpoint basis (for example, if an endpoint received three DPs, the endpoint must return ACK TPs for each one, in the order that the DPs were received). The order a device responds to ACKs or DPs that are sent to different endpoints on the device is device implementation dependent and software can not expect them to occur/complete in any particular order. The split-transaction protocol scales well (across multiple transactions to multiple function endpoints) with signaling bit-rates as it is not subject to propagation delays.

The USB 2.0 protocol completes an entire IN or OUT transaction {Token, Data, Handshake} before continuing to the next bus transaction for the next scheduled function endpoint. All transmissions from the host are essentially broadcast on the USB 2.0 bus. In contrast, the Enhanced SuperSpeed protocol does not broadcast any packets (except for ITPs) and packets traverse only the links needed to reach the intended recipient. The host starts all transactions by sending handshakes or data and devices respond with either data or handshakes. If the device does not have data available, or cannot accept the data, it responds with a packet that states that it is not able to do so. Subsequently, when the device is ready to either receive or transmit data it sends a notification to the host that indicates that it is ready to resume transactions. In addition, the Enhanced SuperSpeed bus provides the ability to transition links into and out of specific low power states. Lower power link states are entered either under software control or under autonomous hardware control after being enabled by software. Mechanisms are provided to automatically transition all links in the path between the host and a device from a non-active power state to the active power state.

Devices report the maximum packet size for each endpoint in its endpoint descriptor. The size indicates data payload length only and does not include any of the overhead for link and protocol level. Bandwidth allocation for SuperSpeed is similar to USB 2.0.

4.4.1 Data Bursting

Data Bursting enhances efficiency by eliminating the wait time for acknowledgements on a per data packet basis. Each endpoint on an Enhanced SuperSpeed device indicates the number of packets that it can send/receive (called the maximum data burst size) before it has to wait for an explicit handshake. Maximum data burst size is an individual endpoint capability; a host determines an endpoint's maximum data burst size from the SuperSpeed Endpoint Companion descriptor associated with this endpoint (refer to Section 9.6.7).

The host may dynamically change the burst size on a per-transaction basis up to the configured maximum burst size. Examples of when a host may use different burst sizes include, but are not limited to, a fairness policy on the host and retries for an interrupt stream. When the endpoint is an OUT, the host can easily control the burst size (the receiver must always be able to manage a transaction burst size). When the endpoint is an IN, the host can limit the burst size for the endpoint on a per-transaction basis via a field in the acknowledgement packet sent to the device.

4.4.2 IN Transfers

The host and device shall adhere to the constraints of the transfer type and endpoint characteristics.

A host initiates a transfer by sending an acknowledgement packet (IN) to the device. This acknowledgement packet contains the addressing information required to route the packet to the intended endpoint. The host tells the device the number of data packets it can send and the sequence number of the first data packet expected from the device. In response the endpoint will transmit data packet(s) with the appropriate sequence numbers back to the host. The acknowledgement packet also implicitly acknowledges the previous data packet that was received successfully.

Note that even though the host is required to send an acknowledgement packet for every data packet received, the device can send up to the number of data packets requested without waiting for any acknowledgement packet.

The Enhanced SuperSpeed IN transaction protocol is illustrated in Figure 4-1. An IN transfer on the Enhanced SuperSpeed bus consists of one, or more, IN transactions consisting of one, or more, packets and completes when any one of the following conditions occurs:

- All the data for the transfer is successfully received.
- The endpoint responds with a packet that is less than the endpoint's maximum packet size.
- The endpoint responds with an error.

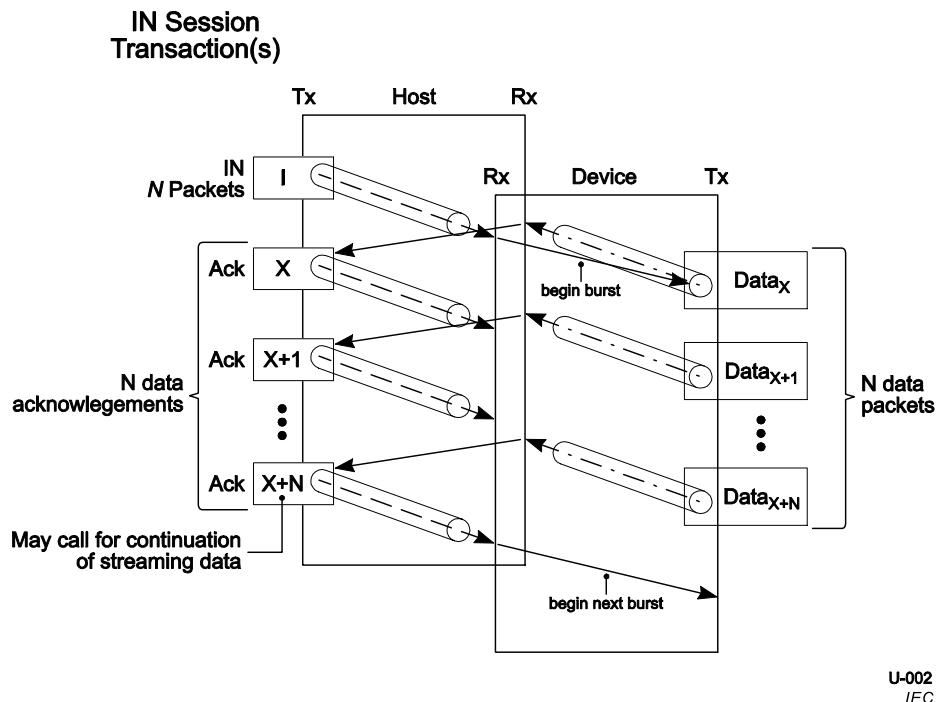


Figure 4-1 – Enhanced SuperSpeed IN Transaction Protocol

4.4.3 OUT Transfers

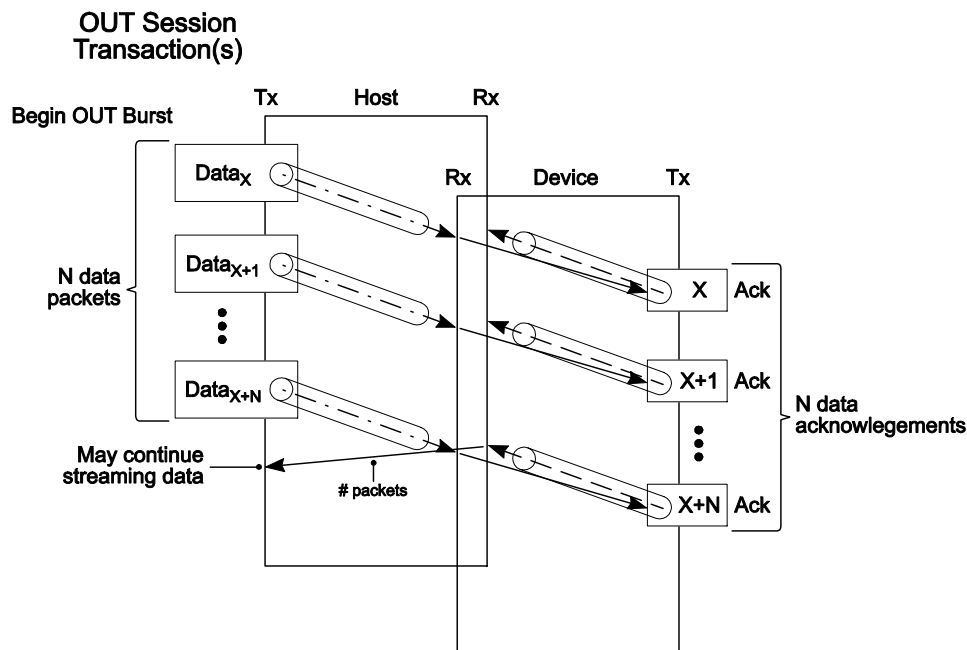
The host and device shall adhere to the constraints of the transfer type and endpoint characteristics.

A host initiates a transfer by sending a burst of data packets to the device. Each data packet contains the addressing information required to route the packet to the intended endpoint. It also includes the sequence number of the data packet. For a non-isochronous transaction, the device returns an acknowledgement packet including the sequence number for the next data packet and implicitly acknowledging the current data packet.

Note that even though the device is required to send an acknowledgement packet for every data packet received, the host can send up to the maximum burst size number of data packets to the device without waiting for an acknowledgement.

The Enhanced SuperSpeed OUT transaction protocol is illustrated in Figure 4-2. An OUT transfer on the Enhanced SuperSpeed bus consists of one, or more, OUT transactions consisting of one, or more, packets and completes when any one of the following conditions occurs:

- All the data for the transfer is successfully transmitted.
- The host sends a packet that is less than the endpoints maximum packet size.
- The endpoint responds with an error.



U-003
IEC

Figure 4-2 – Enhanced SuperSpeed OUT Transaction Protocol

4.4.4 Power Management and Performance

The use of inactivity timers and device-driven link power management provides the ability for very aggressive power management. When the host sends a packet to a device behind a hub with a port whose link is in a non-active state, the packet will not be able to traverse the link until it returns to the active state. In the case of an IN transaction on a SuperSpeed bus instance, the host will not be able to start another IN transaction on that SuperSpeed bus instance until the current one completes. The effect of this behavior could have a significant impact on overall performance.

To balance power management with good performance, the concept of a deferral (to both INs and OUTs) is used. When a host initiates a transaction that encounters a link in a non-active state, a deferred response is sent by the hub to tell the host that this particular path is in a reduced power managed state and that the host should go on to schedule other transactions. In addition, the hub sends a deferred request to the device to notify it that a transaction was attempted. This mechanism informs the host of added latency due to power management and allows the host to mitigate performance impacts that result from the link power management.

4.4.5 Control Transfers

The purpose and characteristics of Control Transfers are identical to those defined in Section 5.5 of the *Universal Serial Bus Specification, Revision 2.0*. The Protocol Layer chapter of this specification describes the details of the packets, bus transactions, and transaction sequences used to accomplish Control transfers. The Device Framework chapter of this specification defines the complete set of standard command codes used for devices.

Each device is required to implement the default control pipe as a message pipe. This pipe is intended for device initialization and management. This pipe is used to access device descriptors and to make requests of the device to manipulate its behavior (at a device-level). Control transfers must adhere to the same request definitions described in the *Universal Serial Bus Specification, Revision 2.0*.

The EnhancedSuperSpeed system will make a “best effort” to support delivery of control transfers between the host and devices. As with USB 2.0, a function and its client software cannot request specific bandwidth for control transfers.

4.4.5.1 Control Transfer Packet Size

Control endpoints have a fixed maximum control transfer data payload size of 512 bytes and have a maximum burst size of one. These maximums apply to all data transactions during the data stage of the control transfer. Refer to Section 8.12.2 for detailed information on the Setup and Status stages of an Enhanced SuperSpeed control transfer.

An Enhanced SuperSpeed device must report a value of 09H in the bMaxPacketSize field of its Device Descriptor. The rule for decoding the default maximum packet size for the Default Control Pipe is given in Section 9.6.1. The Default Control Pipe must support a maximum sequence value of 32 (i.e., sequence values in the range [0-31] are used).

The requirements for data delivery and completion of device-to-host and host-to-device Data stages are generally not changed between USB 2.0 and the Enhanced SuperSpeed bus (refer to Section 5.5.3 of the *Universal Serial Bus Specification, Revision 2.0*).

4.4.5.2 Control Transfer Bandwidth Requirements

A device has no way to indicate the desired bandwidth for a control pipe. A host balances the bus access requirements of all control pipes and pending transactions on those pipes to provide a “best effort” delivery between client software and functions on the device. This policy is the same as the USB 2.0 policy.

The Enhanced SuperSpeed bus requires that bus bandwidth be reserved to be available for use by control transfers as follows:

- The transactions of a control transfer may be scheduled coincident with transactions for other function endpoints of any defined transfer type.
- Retries of control transfers are not given priority over other best effort transactions.
- If there are control and bulk transfers pending for multiple endpoints, control transfers for different endpoints are selected for service according to a fair access policy that is host controller implementation-dependent.
- When a control endpoint delivers a flow control event (as defined in Section 8.10.1), the host will remove the endpoint from the actively scheduled endpoints. The host will resume the transfer to the endpoint upon receipt of a ready notification from the device.

These requirements allow control transfers between a host and devices to regularly move data across the Enhanced SuperSpeed bus with “best effort.” System software’s discretionary behavior defined in Section 5.5.4 of the *Universal Serial Bus Specification, Revision 2.0* applies equally to Enhanced SuperSpeed control transfers.

4.4.5.3 Control Transfer Data Sequences

The Enhanced SuperSpeed protocol preserves the message format and general stage sequencing of control transfers defined in Section 5.5.5 of the *Universal Serial Bus Specification, Revision 2.0*. The Enhanced SuperSpeed protocol defines some changes to the Setup and Status stages of a control transfer. However, all of the sequencing requirements for normal and error recovery scenarios defined in Section 5.5.5 of the *Universal Serial Bus Specification, Revision 2.0* directly map to the SuperSpeed Protocol.

4.4.6 Bulk Transfers

The purpose and characteristics of Bulk Transfers are similar to those defined in Section 5.8 of the *Universal Serial Bus Specification, Revision 2.0*. Section 8.12.1 of this specification describes the details of the packets, bus transactions and transaction sequences used to accomplish Bulk transfers. The Bulk transfer type is intended to support devices that want to communicate relatively large amounts of data at highly variable times where the transfer can use any available Enhanced SuperSpeed bandwidth. An Enhanced SuperSpeed Bulk function endpoint provides the following:

- Access to the Enhanced SuperSpeed bus on a bandwidth available basis
- Guaranteed delivery of data, but no guarantee of bandwidth or latency

The Enhanced SuperSpeed bus retains the following characteristics of bulk pipes:

- No data content structure is imposed on the communication flow for bulk pipes.
- A bulk pipe is a stream pipe and, therefore, always has communication flow either into or out of the host for any pipe instance. If an application requires a bi-directional bulk communication flow, two bulk pipes must be used (one IN and one OUT).

Standard USB bulk pipes provide the ability to move a stream of data. The Enhanced SuperSpeed bus adds the concept of Streams that provide protocol-level support for a multi-stream model.

4.4.6.1 Bulk Transfer Data Packet Size

An endpoint for bulk transfers shall set the maximum data packet payload size in its endpoint descriptor to 1 024 bytes. It also specifies the burst size that the endpoint can accept from or transmit on the Enhanced SuperSpeed bus. The allowable burst size for a bulk endpoint shall be in the range of 1 to 16. All Enhanced SuperSpeed bulk endpoints shall support sequence values in the range [0-31].

A host is required to support any Enhanced SuperSpeed bulk endpoint. A host shall support all bulk burst sizes. The host ensures that no data payload of any data packet in a burst transaction will be sent to the endpoint that is larger than the maximum packet size. Additionally, it shall not send more data packets than the reported maximum burst size.

A bulk function endpoint must always transmit data payloads with data fields less than, or equal to, 1024 bytes. If the bulk transfer has more data than that, all data payloads in the burst transaction are required to be 1024 bytes in length except for the last data payload in the burst, which may contain the remaining data. A bulk transfer may span multiple bus transactions. A bulk transfer is complete when the endpoint does one of the following:

- Has transferred exactly the amount of data expected.
- Transfers a data packet with a payload less than 1 024 bytes.
- Responds with a STALL handshake.

4.4.6.2 Bulk Transfer Bandwidth Requirements

As with USB 2.0 a bulk function endpoint has no way to indicate a desired bandwidth for a bulk pipe. Bulk transactions occur on the Enhanced SuperSpeed bus only on a bandwidth available basis. The Enhanced SuperSpeed host provides a “good effort” delivery of bulk data between client software and device functions. Moving control transfers over the Enhanced SuperSpeed bus has priority over moving bulk transactions. When there are bulk transfers pending for multiple endpoints, the host will provide transaction opportunities to individual endpoints according to a fair access policy, which is host implementation dependent.

All bulk transfers pending in a system contend for the same available bus time. An endpoint and its client software cannot assume a specific rate of service for bulk transfers. Bus time

made available to a software client and its endpoint can be changed as other devices are inserted into and removed from the system or as bulk transfers are requested for other function endpoints. Client software cannot assume ordering between bulk and control transfers; i.e., in some situations, bulk transfers can be delivered ahead of control transfers.

The host can use any burst size between 1 and the reported maximum in transactions with a bulk endpoint to more effectively utilize the available bandwidth. For example, there may be more bulk transfers than bandwidth available, so a host can employ a policy of using smaller data bursts per transactions to provide fair service to all pending bulk data streams.

When a bulk endpoint delivers a flow control event (as defined in Section 8.10.1), the host will remove it from the actively scheduled endpoints. The host will resume the transfer to the endpoint upon receipt of a ready notification from the device.

4.4.6.3 Bulk Transfer Data Sequences

Bulk transactions use the standard burst sequence for reliable data delivery defined in Section 8.10.2. Bulk endpoints are initialized to the initial transmit or receive sequence number and burst size (refer to Section 8.12.1.2 and Section 8.12.1.3) by an appropriate control transfer (SetConfiguration, SetInterface, ClearEndpointFeature). Likewise, a host assumes the initial transmit or receive sequence number and burst size for bulk pipes after it has successfully completed the appropriate control transfer as mentioned above.

Halt conditions for an Enhanced SuperSpeed bulk pipe have the identical side effects as defined for a USB 2.0 bulk endpoint. Recovery from halt conditions are also identical to USB 2.0 (refer to Section 5.8.5 of the *Universal Serial Bus Specification, Revision 2.0*). A bulk pipe halt condition includes a STALL handshake response to a transaction or exhaustion of the host's transaction retry policy due to transmission errors.

4.4.6.4 Bulk Streams

A standard USB Bulk Pipe represents the ability to move single stream of (FIFO) data between the host and a device via a host memory buffer and a device endpoint. Enhanced SuperSpeed streams provide protocol-level support for a multi-stream model and utilize the "stream" pipe communications mode (refer to Section 5.3.2 of the *Universal Serial Bus Specification, Revision 2.0*).

Streams are managed between the host and a device using the *Stream Protocol*. Each Stream is assigned a *Stream ID* (SID).

The Stream Protocol defines a handshake, which allows the device or host to establish the *Current Stream* (CStream) ID associated with an endpoint. The host uses the CStream ID to select the command or operation-specific Endpoint Buffer(s) that will be used for subsequent data transfers on the pipe (see Figure 4-3). The device uses the CStream ID to select the Function Data buffer(s) that will be used.

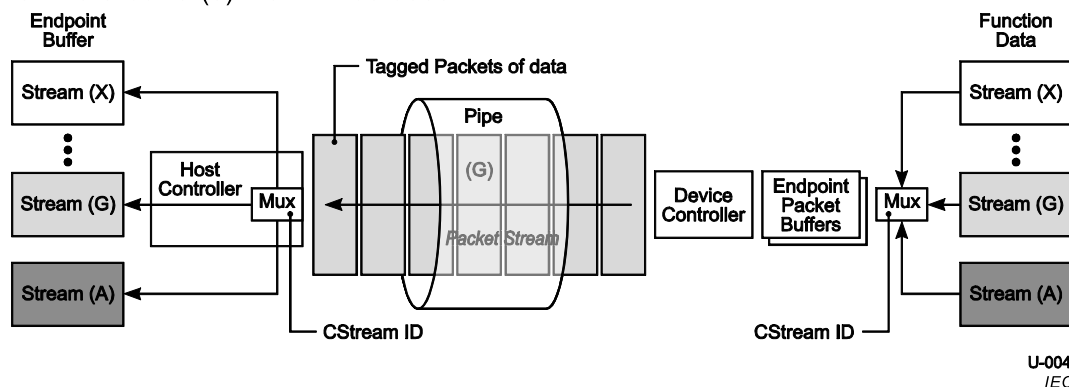


Figure 4-3 – Enhanced SuperSpeed IN Stream Example

The example in Figure 4-3 represents an IN Bulk pipe, where a large number of Streams have been established. Associated with each Stream in host memory is one or more Endpoint Buffers to receive the Stream data. In the device, there is a corresponding command or operation-specific Function Data to be transmitted to the host.

When the device has data available for a specific Stream (G in this example), it issues an ERDY tagged with the CStream ID, and the host will begin issuing IN ACK TP's to the device that is tagged with the CStream ID. The device will respond by returning DPs that contain the Function Data associated with the CStream ID that is also tagged with the CStream ID. When the host receives the data, it uses the CStream ID to select the set of Endpoint Buffers that will receive the data.

When the Function Data is exhausted, the device terminates the Stream (refer to Section 8.12.1.4). The host is also allowed to terminate the Stream if it runs out of Endpoint Buffer space.

Streams may be used, for example, to support out-of-order data transfers required for mass storage device command queuing.

A standard bulk endpoint has a single set of Endpoint Buffers associated with it. Streams extend the number of host buffers accessible by an endpoint from 1 to up to 65533. There is a 1:1 mapping between a host buffer and a Stream ID.

Device Class defined methods are used for coordinating the Stream IDs that are used by the host to select Endpoint Buffers and the device to select Function Data associated with a particular Stream. Typically this is done via an out-of-band mechanism (e.g., another endpoint) that is used to pass the list of valid Stream IDs between the host and the device.

The selection of the Current Stream may be initiated by the host or the device and, in either case, the Stream Protocol provides a method for a selection to be rejected. For example, the host may reject a Stream selection initiated by the device if it has no Endpoint Buffers available for it. Or the device may reject a Stream selection initiated by the host if it has no Function Data available for it. The Device Class defines when a stream may be selected by the host or the device, and the actions that will be taken when a Stream is rejected (refer to Section 8.12.1.4).

A combination of vendor and Device Class defined algorithms determine how Streams are scheduled by a device. The Stream protocol provides methods for starting, stopping, and switching Streams (refer to Section 8.12.1.4).

Mechanisms defined by the Stream protocol allow the device or the host to flow control a Stream. These mechanisms overlap with the standard bulk flow control mechanism.

The host also may start or stop a Stream. For instance, the host will stop a Stream if it runs out of buffer space for the Stream. When the host controller informs the device of this condition, the device may switch to another Stream or wait and continue the same Stream when the host receives more buffers.

The Stream Protocol also provides a mechanism which allows the host to asynchronously inform the device when Endpoint Buffers have been added to the pipe. This is useful in cases in which the host must terminate a stream because it ran out of Endpoint Buffers; however, the device still has more Function Data to transfer. Without this mechanism, the device would have to periodically retry starting the Stream (impacting power management), or a long latency out-of-band method would be required.

Since Streams are run over a standard bulk pipe, an error will halt the pipe, stopping all stream activity. Removal of the halt condition is achieved via software intervention through a separate control pipe as it is for a standard bulk pipe.

Finally, Streams significantly increase the functionality of a bulk endpoint, while having a minimal impact on the additional hardware required to support the feature in hosts and devices.

4.4.7 Interrupt Transfers

The purpose and characteristics of interrupt transfers are similar to those defined in USB 2.0 (see Section 5.7 of the *Universal Serial Bus Specification, Revision 2.0*). The Enhanced SuperSpeed interrupt transfer types are intended to support devices that require a high reliability method to communicate a small amount of data with a bounded service interval. The Protocol Layer chapter of this specification describes the details of the packets, bus transactions and transaction sequences used to accomplish Interrupt transfers. The Enhanced SuperSpeed Interrupt transfer type nominally provides the following:

- Guaranteed maximum service interval
- Guaranteed retry of transfer attempts in the next service interval

Interrupt transfers are attempted each service interval for an interrupt endpoint. Bandwidth is reserved to guarantee a transfer attempt each service interval. Once a transfer is successful, another transfer attempt is not made until the next service interval. If the endpoint responds with a not ready notification, or an acknowledgement indicating that it cannot accept any more packets, the host will not attempt another transfer to that endpoint until it receives a ready notification. The host must then service the endpoint within the larger of (a) twice the service interval, and (b) the device's last reported BELT, after receipt of the ready notification. The requested service interval for the endpoint is described in its endpoint descriptor.

The Enhanced SuperSpeed architecture retains the following characteristics of interrupt pipes:

- No data content structure is imposed on communication flow for interrupt pipes
- An interrupt pipe is a stream pipe and, therefore, is always unidirectional

4.4.7.1 Interrupt Transfer Packet Size

An endpoint for interrupt transfers specifies the maximum data packet payload size that it can accept from or transmit on the SuperSpeed bus. The only allowable maximum data payload size for interrupt endpoints is 1 024 bytes for interrupt endpoints that support a burst size greater than one and can be any size from 1 to 1 024 for an interrupt endpoint with a burst size equal to one. The maximum allowable burst size for interrupt endpoints is three. All Enhanced SuperSpeed interrupt endpoints shall support sequence values in the range [0-31].

Enhanced SuperSpeed interrupt endpoints are only intended for moving small amounts of data with a bounded service interval. The Enhanced SuperSpeed protocol does not require the interrupt transactions to be maximum size.

A host is required to support Enhanced SuperSpeed interrupt endpoints. A host shall support all allowed combinations of interrupt packet sizes and burst sizes. The host ensures that no data payload of any data packet in a burst transaction shall be sent to the endpoint that is larger than the endpoint's maximum packet size. Also, the host shall not send more data packets in a burst transaction than the endpoint's maximum burst size.

An interrupt endpoint shall always transmit data payloads with data fields less than, or equal to, the endpoint's maximum packet size. If the interrupt transfer has more information than will fit into the maximum packet size for the endpoint, all data payloads in the burst transaction are required to be maximum packet size except for the last data payload in the burst transaction, which may contain the remaining data. An interrupt transfer may span multiple burst transactions.

An interrupt transfer is complete when the endpoint does one of the following:

- Has transferred exactly the amount of data expected

- Transfers a data packet with a payload less than the maximum packet size
- Responds with a STALL handshake

4.4.7.2 Interrupt Transfer Bandwidth Requirements

Periodic endpoints may be allocated up to 90 % of the total available bandwidth on an Enhanced SuperSpeed bus.

An endpoint for an interrupt pipe specifies its desired service interval bound via its endpoint descriptor. An interrupt endpoint can specify a desired period $2^{(bInterval-1)} \times 125 \mu s$, where $bInterval$ is in the range 1 up to (and including) 16. The USB System Software will use this information during configuration to determine a period that can be sustained. The period provided by the system may be shorter than that desired by the device up to the shortest period defined by the Enhanced SuperSpeed architecture (125 μs which is also referred to as a bus interval). Note that errors on the bus can prevent an interrupt transaction from being successfully delivered over the bus and consequently exceed the desired period.

An Enhanced SuperSpeed interrupt endpoint can move up to three maximum sized packets (3×1024 bytes) per service interval. Interrupt transfers are moved over the USB by accessing an interrupt endpoint every service interval. For interrupt endpoints, the host has no way to determine whether the endpoint will source/sync data without accessing the endpoint and requesting an interrupt transfer. If an interrupt IN endpoint has no interrupt data to transmit, or an interrupt OUT endpoint has insufficient buffer to accept data when accessed by the host, it responds with a flow control response.

An endpoint should only provide interrupt data when it has interrupt data pending to avoid having a software client erroneously notified of a transfer completion. A zero-length data payload is a valid transfer and may be useful for some implementations. The host may access an endpoint at any point during the service interval. The interrupt endpoint should not assume a fixed spacing between transaction attempts. The interrupt endpoint can assume only that it will receive a transaction attempt within the service interval bound. Errors can prevent the successful exchange of data within the service interval bound and a host is not required to retry the transaction in the same service interval and is only required to retry the transaction in the next service interval.

4.4.7.3 Interrupt Transfer Data Sequences

Interrupt transactions use the standard burst sequence for reliable data delivery protocol defined in Section 8.10.2. Interrupt endpoints are initialized to the initial transmit, or receive, sequence number and burst size (refer to Section 8.12.4.1 and Section 8.12.4.2) by an appropriate control transfer (SetConfiguration, SetInterface, ClearEndpointFeature). A host sets the initial transmit or receive sequence number and burst size for interrupt pipes after it has successfully completed the appropriate control transfer.

Halt conditions for a SuperSpeed interrupt pipe have the identical side effects as defined for a USB 2.0 interrupt endpoint. Recovery from halt conditions are also identical to the USB 2.0, refer to Section 5.7.5 in the *Universal Serial Bus Specification, Revision 2.0*. An interrupt pipe halt condition includes a STALL handshake response to a transaction, or exhaustion, of the host's transaction retry policy due to transmission errors.

4.4.8 Isochronous Transfers

The purpose of Enhanced SuperSpeed isochronous transfers is similar to those defined in USB 2.0 (refer to Section 5.6 of the *Universal Serial Bus Specification, Revision 2.0*). As in USB 2.0, the Enhanced SuperSpeed isochronous transfer type is intended to support streams that want to perform error tolerant, periodic transfers within a bounded service interval. The Enhanced SuperSpeed bus does not transmit start of frames as on USB 2.0. Timing information is transmitted to devices using Isochronous Timestamp Packets (ITPs). The Protocol Layer chapter of this specification describes the details of the packets, bus transactions, and transaction sequences used to accomplish isochronous transfers. It also

describes how the timing information is conveyed to devices. The Enhanced SuperSpeed isochronous transfer type provides the following:

- Guaranteed bandwidth for transaction attempts on the Enhanced SuperSpeed bus with bounded latency
- Guaranteed data rate through the pipe as long as data is provided to the pipe

Isochronous transactions are attempted each service interval for an isochronous endpoint. Isochronous endpoints that are admitted on the Enhanced SuperSpeed bus are guaranteed the bandwidth they require on the bus. The host can request data from the device, or send data to the device, at any time during the service interval for a particular endpoint on that device. The requested service interval for the endpoint is described in its endpoint descriptor. The Enhanced SuperSpeed isochronous transfer type is designed to support a source and sink that produce and consume data at the same average rate.

An Enhanced SuperSpeed isochronous pipe is a stream pipe and is always unidirectional. The endpoint description identifies whether a given isochronous pipe's communication flow is into or out of the host. If a device requires bi-directional isochronous communication flows, two isochronous pipes must be used, one in each direction.

Enhanced SuperSpeed power management may interfere with isochronous transfers whenever an isochronous transfer needs to traverse a non-active link. The resultant delay could result in the data not arriving within the service interval. To overcome this, the Enhanced SuperSpeed protocol defines a PING and PING_RESPONSE mechanism (refer to Section 8.5.7). Before initiating an isochronous transfer the host shall send a PING packet to the device. The device responds with a PING_RESPONSE packet that tells the host that all the links in the path to the device are in the active state.

4.4.8.1 Isochronous Transfer Packet Size

An endpoint for isochronous transfers specifies the maximum data packet payload size that the endpoint can accept from or transmit on SuperSpeed. The only allowable maximum data payload size for isochronous endpoints is 1 024 bytes for isochronous endpoints that support a burst size greater than one and can be any size from 0 to 1 024 for an isochronous endpoint with a burst size equal to one. The maximum allowable burst size for isochronous endpoints is 16. However an isochronous endpoint can request up to six burst transactions in the same service interval.

The Enhanced SuperSpeed protocol does not require the isochronous data packets to be maximum size. If an amount of data less than the maximum packet size is being transferred, the data packet shall not be padded.

A host shall support Enhanced SuperSpeed isochronous endpoints for all allowed combinations of isochronous packet sizes and burst sizes. The host shall ensure that no data payload of any data packet in a burst transaction be sent to the endpoint that is larger than the reported maximum packet size. Also, the host shall not send more data packets in a burst transaction than the endpoint's maximum burst size.

An isochronous endpoint shall always transmit data payloads with data fields less than, or equal to, the endpoint's maximum packet size. If the isochronous transfer has more information than will fit into the maximum packet size for the endpoint, all data payloads in the burst transaction are required to be maximum packet size except for the last data payload in the burst transaction, which may contain the remaining data. An isochronous transfer may span multiple burst transactions.

4.4.8.2 Isochronous Transfer Bandwidth Requirements

Periodic endpoints can be allocated up to 90 % of the total available bandwidth on the Enhanced SuperSpeed bus.

An endpoint for an isochronous pipe specifies its desired service interval bound via its endpoint descriptor. An isochronous endpoint can specify a desired period $2^{(bInterval-1)} \times 125 \mu s$, where $bInterval$ is in the range 1 to 16. The system software will use this information during configuration to determine whether the endpoint can be added to the host schedule. Note that errors on the bus can prevent an isochronous transaction from being successfully delivered over the bus.

A SuperSpeed isochronous endpoint can move up to three burst transactions of up to 16 maximum sized packets ($3 \times 16 \times 1\,024$ bytes) per service interval. A SuperSpeedPlus isochronous endpoint can move up to six burst transactions of up to 16 maximum sized packets ($6 \times 16 \times 1\,024$ bytes) per service interval. Isochronous transfers are moved over the USB by accessing an isochronous endpoint every service interval. The host will send data to, or request data from, the endpoint every service interval. Note, if an endpoint has no isochronous data to transmit when accessed by the host, it shall send a zero length packet in response to the request for data.

The host may access an endpoint at any point during the appropriate service interval. The isochronous endpoint should not assume a fixed spacing between transaction attempts. The isochronous endpoint can assume only that it will receive a transaction attempt within the service interval bound. Errors may prevent the successful exchange of data within the service interval bound; however, since the packets in an isochronous transaction are not acknowledged, a host/device has no way of knowing which packets were not received successfully and hence will not retry packets.

4.4.8.3 Isochronous Transfer Data Sequences

Isochronous endpoints always transmit data packets starting with sequence number zero in each service interval. Each successive data packet transmitted in the same service interval is sent with the next higher sequence number. The sequence number shall roll over from thirty one to zero when transmitting the thirty second packet. Isochronous endpoints do not support retries and cannot respond with flow control responses.

4.4.8.4 Special Considerations for Isochronous Transfers

For a general overview of isochronous data movements over USB, USB clock model, clock synchronization, and the different types of USB-defined synchronization types and their specific requirements, refer to the *USB 2.0 Specification*, Section 5.12. The following section presents the information necessary to implement Enhanced SuperSpeed isochronous endpoints that need an explicit feedback isochronous endpoint.

4.4.8.4.1 Explicit Feedback

An Enhanced SuperSpeed asynchronous isochronous sink endpoint must provide explicit feedback to the host by indicating accurately what its desired data rate (F_f) is, relative to the USB bus interval frequency. This allows the host to continuously adjust the number of samples sent to the sink so that neither underflow, nor overflow, of the data buffer occurs. Likewise, an Enhanced SuperSpeed adaptive source endpoint must receive explicit feedback from the host so that it can accurately generate the number of samples required by the host. Feedback endpoints can be specified as described in Section 9.6.6 for the *bmAttributes* field of the endpoint descriptor.

To generate the desired data rate F_f , the device must measure its actual sampling rate F_s , referenced to the USB notion of time, i.e., the USB bus interval frequency. This specification requires the data rate F_f to be resolved to better than one sample per second (1 Hz) in order to allow a high-quality source rate to be created and to tolerate delays and errors in the feedback loop. To achieve this accuracy, the measurement time T_{meas} must be at least 1 second. Therefore:

$$T_{meas} = 2^K$$

where T_{meas} is now expressed in USB bus intervals and $K \geq 13$ for Enhanced SuperSpeed devices (125 μs bus intervals). However, in most devices, the actual sampling rate F_s is derived from a master clock F_m through a binary divider. Therefore:

$$F_m = F_s * 2^P$$

where P is a positive integer (including 0 if no higher-frequency master clock is available). The measurement time T_{meas} can now be decreased by measuring F_m instead of F_s and:

$$T_{\text{meas}} = \frac{2^K}{2^P} = 2^{(K-P)}$$

In this way, a new estimate for F_f becomes available every $2^{(K-P)}$ bus intervals. P is practically bound to be in the range $[0, K]$ because there is no point in using a clock slower than F_s ($P=0$), and no point in trying to update F_f more than once per bus interval ($P=K$). A sink can determine F_f by counting cycles of the master clock F_m for a period of $2^{(K-P)}$ bus intervals. The counter is read into F_f and reset every $2^{(K-P)}$ bus intervals. As long as no clock cycles are skipped, the count will be accurate over the long term.

Each bus interval, an adaptive source adds F_f to any remaining fractional sample count from the previous bus interval, sources the number of samples in the integer part of the sum, and retains the fractional sample count for the next bus interval. The source can look at the behavior of F_f over many bus intervals to determine an even more accurate rate, if it needs to.

F_f is expressed in number of samples per bus interval. The F_f value consists of an integer part that represents the (integer) number of samples per bus interval and a fractional part that represents the “fraction” of a sample that would be needed to match the sampling frequency F_s to a resolution of 1 Hz or better. The fractional part requires at least K bits to represent the “fraction” of a sample to a resolution of 1 Hz or better. The integer part must have enough bits to represent the maximum number of samples that can ever occur in a single bus interval. Assuming that the minimum sample size is one byte, then this number is currently limited to $48 \times 1024 = 49152$ and 16 bits are needed.

For Enhanced SuperSpeed endpoints, the F_f value shall be encoded in an unsigned 32.K ($K \geq 13$) format, encoded into eight bytes (for future extensibility). The value shall be aligned into these eight bytes so that the binary point is located between the fourth and the fifth byte so that it has a 32.32 format. Only the first K bits behind the binary point are required. The lower 32- K bits may be optionally used to extend the precision of F_f , otherwise, they shall be reported as zero.

An endpoint needs to implement only the number of bits that it effectively requires for its maximum F_f .

The choice of P is endpoint-specific. Use the following guidelines when choosing P :

- P must be in the range $[0, K]$.
- Larger values of P are preferred, because they reduce the size of the frame counter and increase the rate at which F_f is updated. More frequent updates result in a tighter control of the source data rate, which reduces the buffer space required to handle F_f changes.
- P should be less than K so that F_f is averaged across at least two frames in order to reduce SOF jitter effects.
- P should not be zero in order to keep the deviation in the number of samples sourced to less than 1 in the event of a lost F_f value.

Isochronous transfers are used to read F_f from the feedback register. The desired reporting rate for the feedback should be $2^{(K-P)}$ bus intervals. F_f will be reported at most once per

update period. There is nothing to be gained by reporting the same F_f value more than once per update period. The endpoint may choose to report F_f only if the updated value has changed from the previous F_f value. If the value has not changed, the endpoint may report the current F_f value or a zero length data payload. It is strongly recommended that an endpoint always report the current F_f value any time it is polled.

It is possible that the source will deliver one too many or one too few samples over a long period due to errors or accumulated inaccuracies in measuring F_f . The sink must have sufficient buffer capability to accommodate this. When the sink recognizes this condition, it should adjust the reported F_f value to correct it. This may also be necessary to compensate for relative clock drifts. The implementation of this correction process is endpoint-specific and is not specified.

4.4.9 Device Notifications

Device notifications are a standard method for a device to communicate asynchronous device- and bus-level event information to the host. This feature does not map to the pipe model defined for the standard transfer types. Device notifications are always initiated by a device and the flow of data information is always device to host.

Device notifications are message-oriented data communications that have a specific data format structure as defined in Section 8.5.6. Device notifications do not have any data payload. Devices can send a device notification at any time.

4.4.10 Reliability

To ensure reliable operation, several layers of protection are used. This provides reliability for both flow control and data end to end.

4.4.10.1 Physical Layer

The Enhanced SuperSpeed physical layer provides bit error rates less than 1 bit in 10^{12} bits.

4.4.10.2 Link Layer

The Enhanced SuperSpeed link layer has mechanisms that ensure a bit error rate less than 1 bit in 10^{20} bits for header packets. The link layer uses a number of techniques including packet framing ordered sets, link level flow control and retries to ensure reliable end-to-end delivery for header packets.

4.4.10.3 Protocol Layer

The Enhanced SuperSpeed protocol layer depends on a 32-bit CRC appended to the Data Payload and a timeout coupled with retries to ensure that reliable data is provided to the application.

4.4.11 Efficiency

Enhanced SuperSpeed communications efficiency is dependent on a number of factors, including line encoding, packet structure and framing, link level flow control and protocol overhead.

Links that operate at Gen 1 speed (5 Gbps and 8b/10b line encoding) the raw throughput is 500 MBps. Accounting for flow control, packet framing and protocol overheads reduces the effective bandwidth down to 450 MBps or less to be delivered to an application.

Links that operate at Gen 2 speed (10 Gbps and 128b/132b line encoding) the raw throughput is approximately 1,2 GBps. Accounting for flow control, packet framing, and protocol overheads reduces the best case effective bandwidth down to approximately 1.1 GBps or less

to be delivered to an application. Also, effective bandwidth of individual endpoint flows can be affected by interaction with other simultaneously active endpoint flows traversing through SuperSpeedPlus hub arbiters.

5 Mechanical

This chapter defines form, fit, and function of the USB 3.1 connectors and cable assemblies. It contains the following:

- Connector mating interfaces
- Cables and cable assemblies
- Electrical requirements
- Mechanical and environmental requirements
- Implementation notes and guidelines

The intention of this chapter is to enable connector, system, and device designers and manufacturers to build, qualify, and use the USB 3.1 connectors, cables, and cable assemblies.

If any part of this chapter conflicts with the USB 2.0 specification, the USB 3.1 specification supersedes the USB 2.0 specification.

With some clearly noted exceptions, USB 3.1 connectors and cable assemblies provide the same functionality as the previously defined USB 3.0 connectors and cable assemblies. The primary differences are that USB 3.1 connectors and cable assemblies are specifically intended to support 10 Gbps SuperSpeed USB operation and USB 3.1 connectors and cable assemblies include features and requirements associated with improved EMI/RFI performance. From the perspective of interface mating interoperability, USB 3.0 connectors and cable assemblies are the same as USB 3.1 connectors and cable assemblies therefore, only USB 3.1 is listed for interface interoperability requirements and is inclusive of USB 3.0.

5.1 Objective

The mechanical layer specification has been developed with the following objectives:

- Supporting Gen 1 speed (5 Gbps) and Gen 2 speed (10 Gbps)
- Backward compatible with USB 2.0
- Minimizing connector form factor variations
- Providing the system designer features that allow implementations to comply with EMI and RFI requirements
- Supporting On-The-Go (OTG)
- Supporting compatibility with the Universal Serial Bus Power Delivery Specification
- Low cost

5.2 Significant Features

This section identifies the significant features of the USB 3.1 (Enhanced SuperSpeed) connectors and cable assemblies' specification. The purpose of this section is not to present all technical details associated with each major feature, but rather to highlight their existence. Where appropriate, this section references other parts of the document where additional details are found.

5.2.1 Connectors

The USB 3.1 specification defines the following connectors:

- Enhanced SuperSpeed Standard-A plug and receptacle
- Enhanced SuperSpeed Standard-B plug and receptacle
- Enhanced SuperSpeed Micro-B plug and receptacle
- Enhanced SuperSpeed Micro-A plug
- Enhanced SuperSpeed Micro-AB receptacle

All SuperSpeed connectors have the same mating interfaces and are compatible with each other. The USB Enhanced SuperSpeed Gen 2 connectors have unique electrical requirements to support the Gen 2 speed in addition to the Gen 1 speed.

Table 5-1 lists the compatible plugs and receptacles.

Table 5-1 – Plugs Accepted By Receptacles

Receptacle (Standard or PD)	Plugs Accepted (Standard or PD)
USB 2.0 Standard-A	USB 2.0 Standard-A or USB 3.1 Standard-A
USB 3.1 Standard-A	USB 3.1 Standard-A or USB 2.0 Standard-A
USB 2.0 Standard-B	USB 2.0 Standard-B
USB 3.1 Standard-B	USB 3.1 Standard-B or USB 2.0 Standard-B
USB 2.0 Micro-B	USB 2.0 Micro-B
USB 3.1 Micro-B	USB 3.1 Micro-B or USB 2.0 Micro-B
USB 2.0 Micro-AB	USB 2.0 Micro-B or USB 2.0 Micro-A
USB 3.1 Micro-AB	USB 3.1 Micro-B, USB 3.1 Micro-A, USB 2.0 Micro-B, or USB 2.0 Micro-A

5.2.1.1 USB 3.1 Standard-A Connector

The USB 3.1 Standard-A connector is defined as the host connector. It has the same mating interface as the USB 2.0 Standard-A connector, but with additional pins for two more differential pairs and a drain. Refer to Section 5.3.1.3 for pin assignments and descriptions.

A USB 3.1 Standard-A receptacle accepts either a USB 3.1 Standard-A plug or a USB 2.0 Standard-A plug. Similarly, a USB 3.1 Standard-A plug may be mated with either a USB 3.1 Standard-A receptacle or a USB 2.0 Standard-A receptacle.

A unique color coding is recommended for the USB 3.1 Standard-A connector plastic housings to help users distinguish the USB 3.1 Standard-A connector from the USB 2.0 Standard-A connector (refer to Section 5.3.1.4 for details).

5.2.1.2 USB 3.1 Standard-B Connector

The USB 3.1 Standard-B connector is defined for relatively large, stationary peripherals, such as external hard drives and printers. It is defined so that the USB 3.1 Standard-B receptacle accepts either a USB 3.1 Standard-B plug, or a USB 2.0 Standard-B plug. Inserting a USB 3.1 Standard-B plug into a USB 2.0 Standard-B receptacle is physically disallowed (refer to Section 5.3.2 for details).

5.2.1.3 USB 3.1 Micro-B Connector

The USB 3.1 Micro-B plug and USB 3.1 Micro-B receptacle connectors are defined for small handheld devices and other applications where a small connector size may be used. It is defined so that the USB 3.1 Micro-B receptacle accepts either a USB 3.1 Micro-B plug, or a USB 2.0 Micro-B plug. Inserting a USB 3.1 Micro-B plug into a USB 2.0 Micro-B receptacle or USB 2.0 Micro-AB receptacle is physically disallowed by the implementation.

5.2.1.4 USB 3.1 Micro-AB and USB 3.1 Micro-A Connectors

The USB 3.1 Micro-AB receptacle is similar to the USB 3.1 Micro-B receptacle except for different keying. It accepts a USB 3.1 Micro-A plug, a USB 3.1 Micro-B plug, a USB 2.0 Micro-A plug, or a USB 2.0 Micro-B plug. The USB 3.1 Micro-AB receptacle is only allowed on OTG products which may function as either a host or device. All other uses of the USB 3.1 Micro-AB receptacle are prohibited.

The USB 3.1 Micro-A plug is similar to the USB 3.1 Micro-B plug except for different keying and ID pin connections. Similar to the USB 2.0 Micro-A plug, the USB 3.1 Micro-A plug is defined for OTG applications only. Section 5.3.3 defines the USB 3.1 Micro connector family

5.2.2 Allowed Cable Assemblies

The USB 3.1 specification defines the following cable assemblies:

- USB 3.1 Standard-A plug to USB 3.1 Standard-B plug
- USB 3.1 Standard-A plug to USB 3.1 Micro-B plug
- USB 3.1 Standard-A plug to USB 3.1 Standard-A plug
- USB 3.1 Micro-A plug to USB 3.1 Micro-B plug
- USB 3.1 Micro-A plug to USB 3.1 Standard-B plug
- Captive cable with USB 3.1 Standard-A plug
- Captive cable with USB 3.1 Micro-A plug

A captive cable is a cable assembly that has a Standard-A plug on one end and that is either hardwired or has a vendor-specific connector on the other end. A hardwired cable is directly wired to the device and it is not detachable from the device. This specification does not define how the vendor-specific connector or hardwired attachment is done on the device side.

For electrical compliance purposes, a USB 3.1 captive cable (hardwired or with vendor-specific connector on the device end) shall be considered part of the USB 3.1 device.

No other types of cable assemblies are allowed by this specification. Section 5.5 provides detailed discussion on USB 3.1 cable assemblies.

5.2.3 Raw Cables

Due to EMI, RFI, and signal integrity requirements, each cable differential pair used for the SuperSpeed lines in a USB 3.1 cable assembly shall be shielded; the Unshielded Twisted Pair (UTP) used for USB 2.0 is not allowed for SuperSpeed lines. Section 5.4 defines the cable construction for USB 3.1.

5.3 Connector Mating Interfaces

This section defines the connector mating interfaces, including the connector interface drawings, pin assignments, and descriptions.

5.3.1 USB 3.1 Standard-A Connector

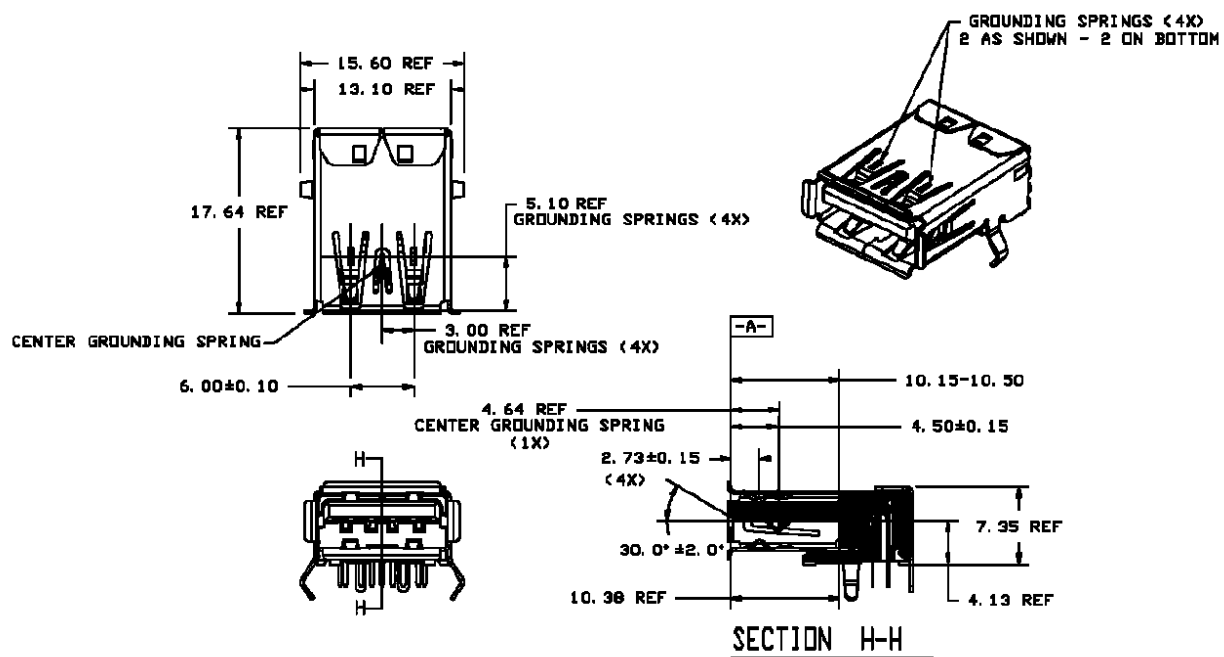
5.3.1.1 Interface Definition

Figure 5-1 and Figure 5-2 show the USB 3.1 Standard-A receptacle and required ground spring mating areas, respectively. Figure 5-4 shows the Standard-A plug interface dimensions for USB 3.1. Only the dimensions that govern the mating interoperability are specified. All REF dimensions are informative.

The *Universal Serial Bus Power Delivery Specification* defines the mechanical and electrical requirements for the Insertion Detect feature to support cold socket capability. It may be implemented in a Standard-A receptacle or a PD Standard-A receptacle. Implementation is vendor-specific. The Insertion Detect feature shall be implemented for cold socket Standard-A applications and is optional for all other Standard-A implementations. See *the Universal Serial Bus Power Delivery Specification* for complete Insertion Detect requirements. Example connector configurations including Insertion Detect features are shown in Figure 5-3.

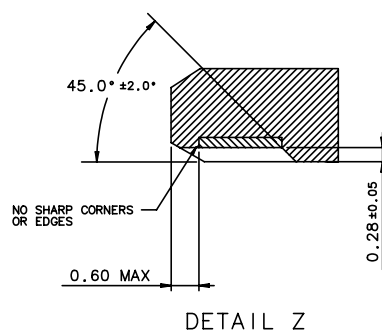
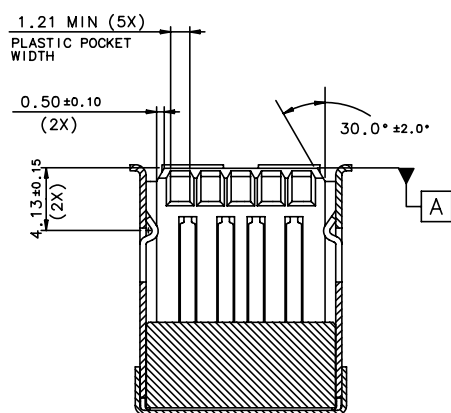
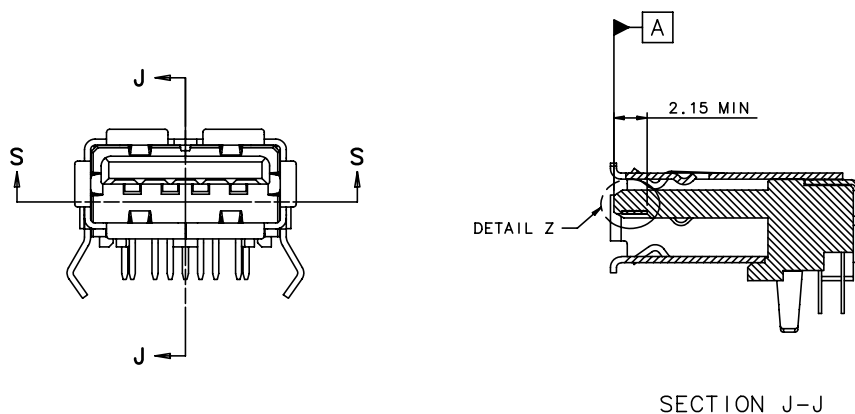
Although the USB 3.1 Standard-A connector has basically the same form factor as the USB 2.0 Standard-A connector, it has significant differences inside. Below are the key features and design areas that need attention:

- In addition to the Vbus, D-, D+, and GND pins that are required for USB 2.0, the USB 3.1 Standard-A connector includes five more pins: two differential signal pairs plus one ground (GND_DRAIN). The two added differential signal pairs are for SuperSpeed data transfer, supporting dual simplex SuperSpeed signaling. The added GND_DRAIN pin is for drain wire termination and managing EMI, RFI, and signal integrity.
- The contact areas of the five SuperSpeed pins are located towards the front of the receptacle as blades, while the four USB 2.0 pins towards the back of the receptacle as beams or springs. Accordingly, in the plug, the SuperSpeed contacts are beams located behind the USB 2.0 blades. In other words, the USB 3.1 Standard-A connector has a two-tier contact system.
- The tiered-contact approach within the Standard-A connector form factor results in less contact area as compared to the USB 2.0 Standard-A connector. The connector interface dimensions take into consideration contact mating requirements between the USB 3.1 Standard-A receptacle and USB 3.1 Standard-A plug, the USB 3.1 Standard-A receptacle and USB 2.0 Standard-A plugs, and the USB 2.0 Standard-A receptacles and USB 3.1 Standard-A plug.
- The connector interface definition avoids shorting between the SuperSpeed and USB 2.0 pins during insertion when plugging a USB 2.0 Standard-A plug into a USB 3.1 Standard-A receptacle or a USB 3.1 Standard-A plug into a USB 2.0 Standard-A receptacle.
- There may be some increase in the USB 3.1 Standard-A receptacle connector depth (into a system board) to support the two-tiered-contacts as compared to the USB 2.0 Standard-A receptacle.
- Drawings for stacked USB 3.1 Standard-A receptacles are not shown in this specification. They are allowed as long as they meet all the electrical and mechanical requirements defined in this specification. When designing a stacked USB 3.1 Standard-A receptacle, efforts need to be made to minimize impedance discontinuity of the top connector in the stack because of its long electrical path. Attention to the high speed electrical design of USB 3.1 Standard-A connectors is required. In addition to minimizing the connector impedance discontinuities, crosstalk between the SuperSpeed differential signal pairs and USB 2.0 D+/D- pair should also be minimized.
- The receptacle connector should have a back-shield to ensure that the receptacle connector is fully enclosed. The USB 3.1 receptacle should also make good contact to the PCB ground by providing sufficient number of ground tabs to ensure a low impedance path to PCB ground. The USB 3.1 receptacle connector should have a robust mating interface to the shield of the USB 3.1 plug when it is inserted. Previous versions of this specification required providing a grounding spring tab in the middle of the side closest to the USB SuperSpeed signal contacts and grounding springs on both sides of the shell for USB 3.0 Standard-A receptacles. New designs shall have three grounding spring tabs on the side closest to the USB SuperSpeed signal contacts, two grounding spring tabs on the side opposite the USB SuperSpeed signal contacts, and a grounding spring on both sides of the shell of the USB 3.1 Standard-A receptacle. See Figure 5-2.



IEC

Continued on next page



SECTION S-S

DETAIL Z

IEC

Continued on next page

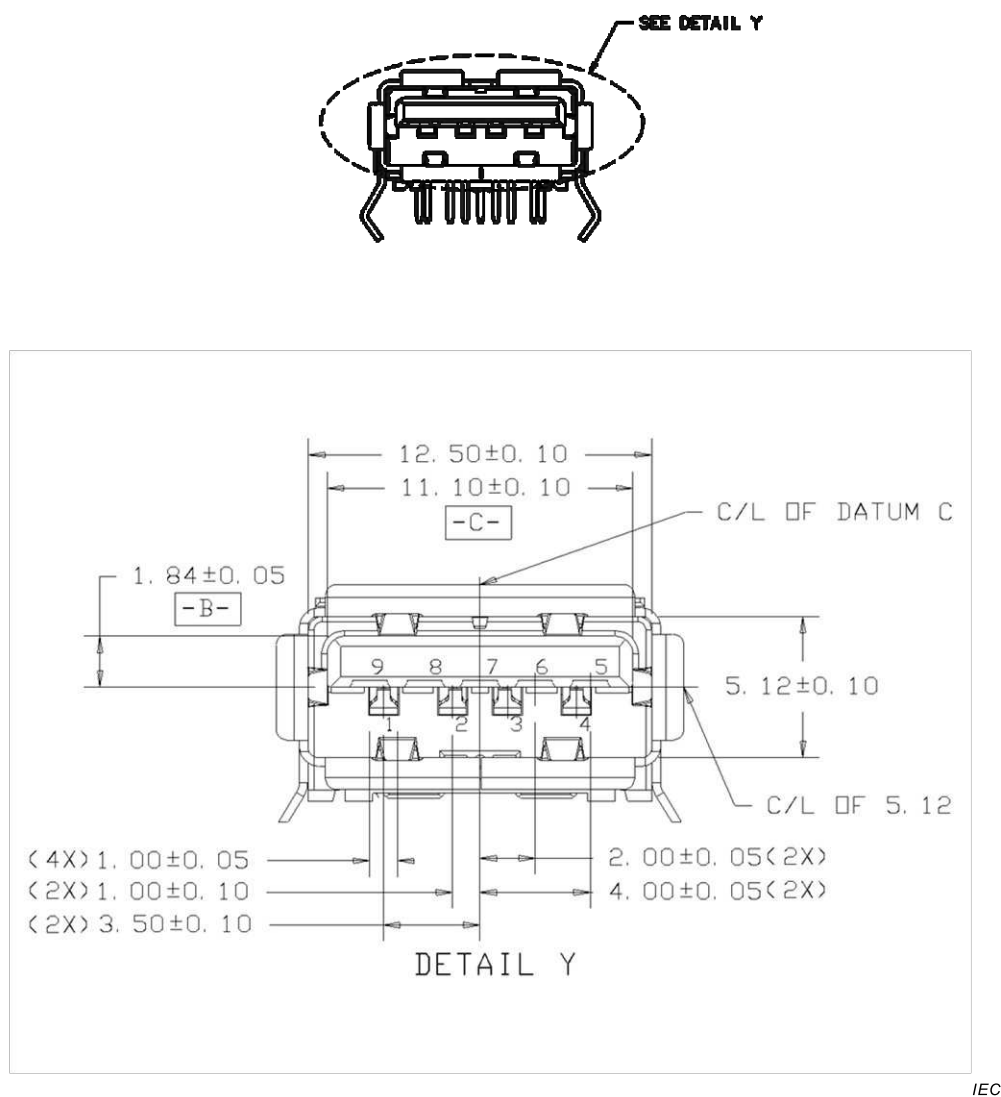
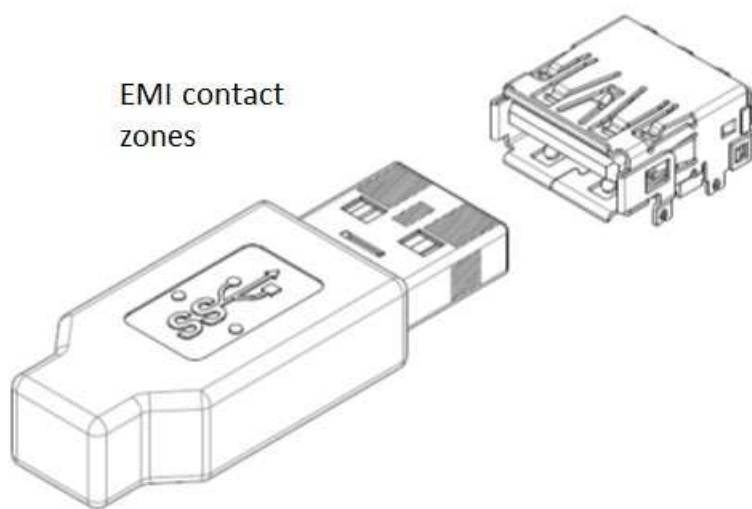
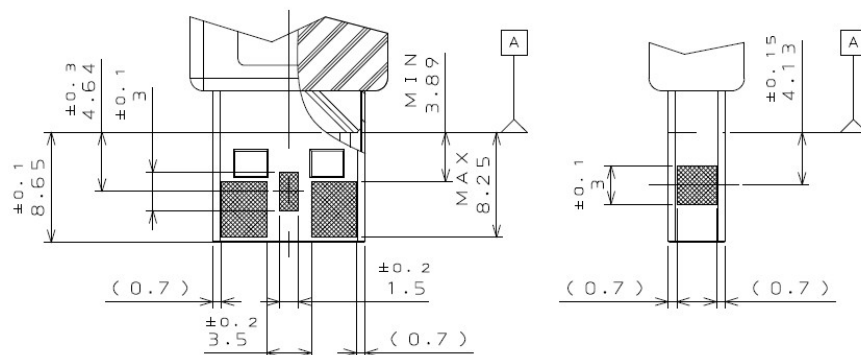


Figure 5-1 – USB 3.1 Standard-A Receptacle Interface Dimensions

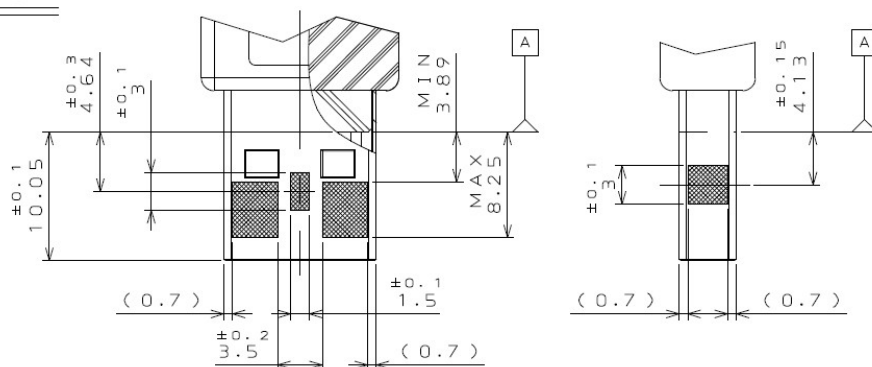


TOP VIEW (SIDE NEAREST SUPERSPEED CONTACTS)

USB 3.1 PLUG

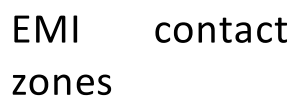


USB 3.1 PD PLUG

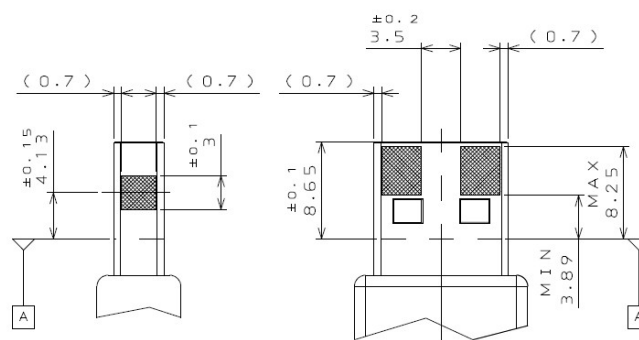


IEC

Continued on next page



U S B 3 . 1 P L U G



USB 3.1 PD PLUG

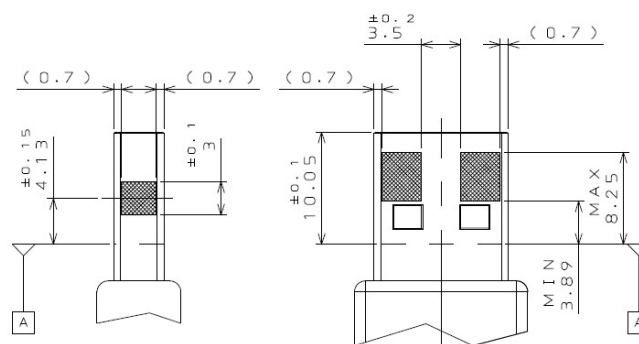
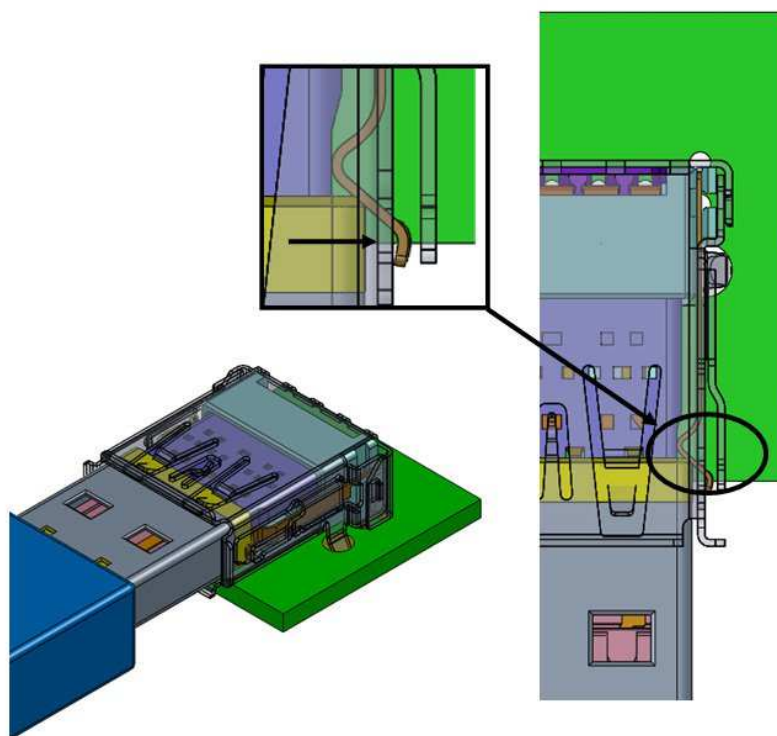
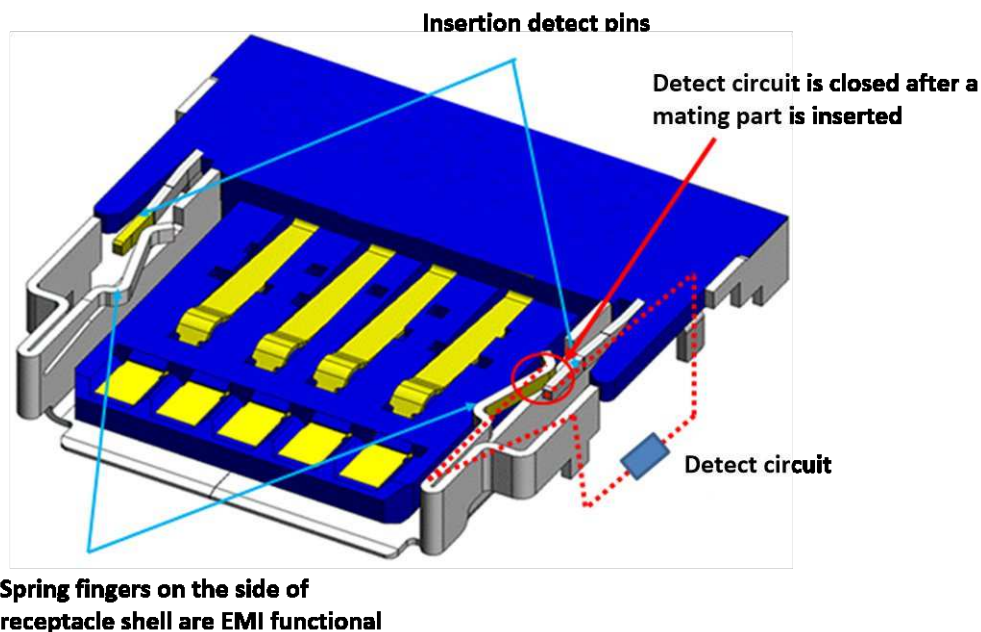
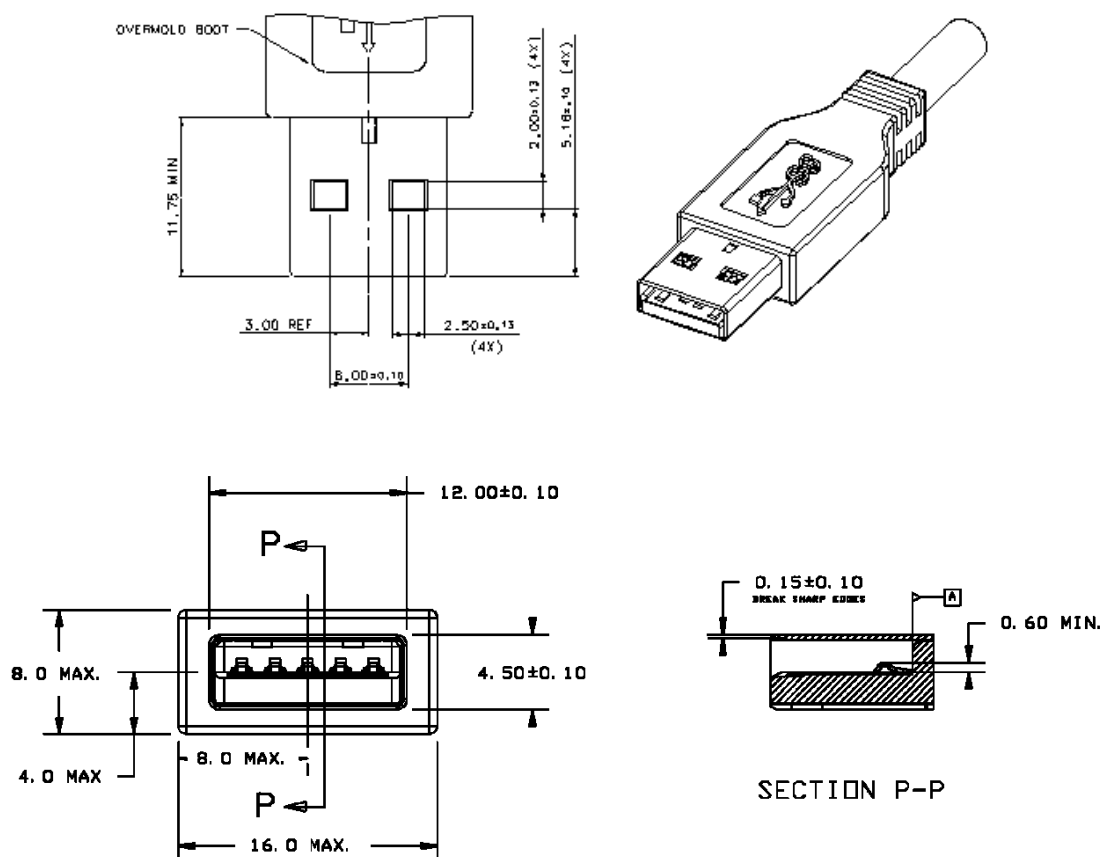


Figure 5-2 – Example USB 3.1 Standard-A Receptacle with Grounding Springs and Required contact zones on the Standard-A Plug



IEC

Figure 5-3 – Example USB 3.1 Standard-A Mid-Mount Receptacles with Insertion Detect

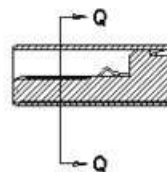
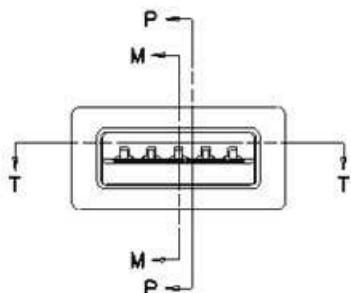


NOTES:

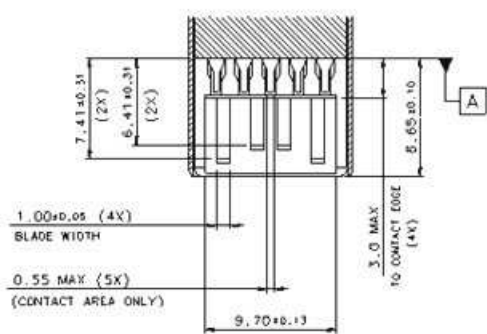
- 1) NON-DIMENSIONED GEOMETRY FOR REFERENCE ONLY, SUBJECT TO CHANGE
- 2) DRAWING FOR MATING INTERFACE DIMENSIONS ONLY. VIEWS MAY NOT SHOW REALISTIC MANUFACTURING CONDITION.

IEC

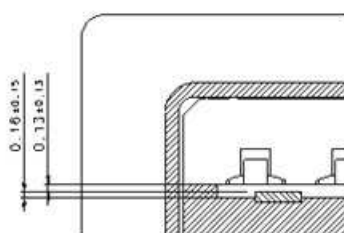
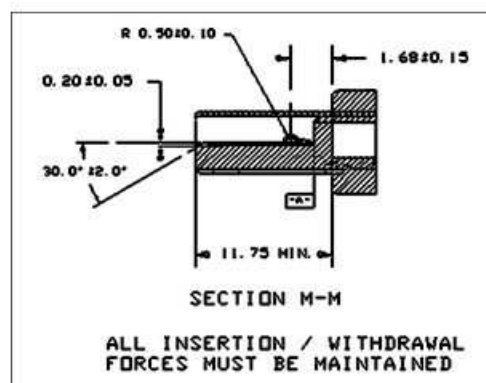
Continued on next page



SECTION P-P



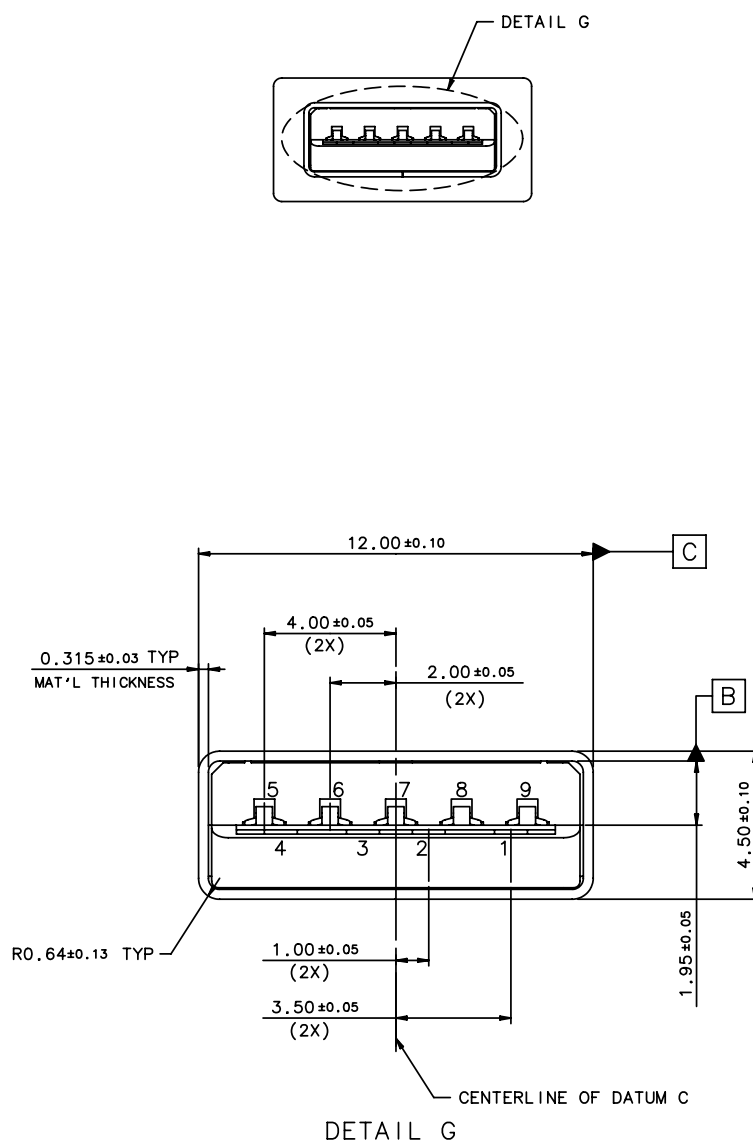
SECTION T-T



SECTION Q-Q

IEC

Continued on next page



IEC

Figure 5-4 – USB 3.1 Standard-A Plug Interface Dimensions

5.3.1.2 USB 3.1 Standard-A Reference Footprints

This specification does not define standard footprints. Any footprint may be used as long as all mechanical and electrical requirements are met. Example footprints are provided for reference only.

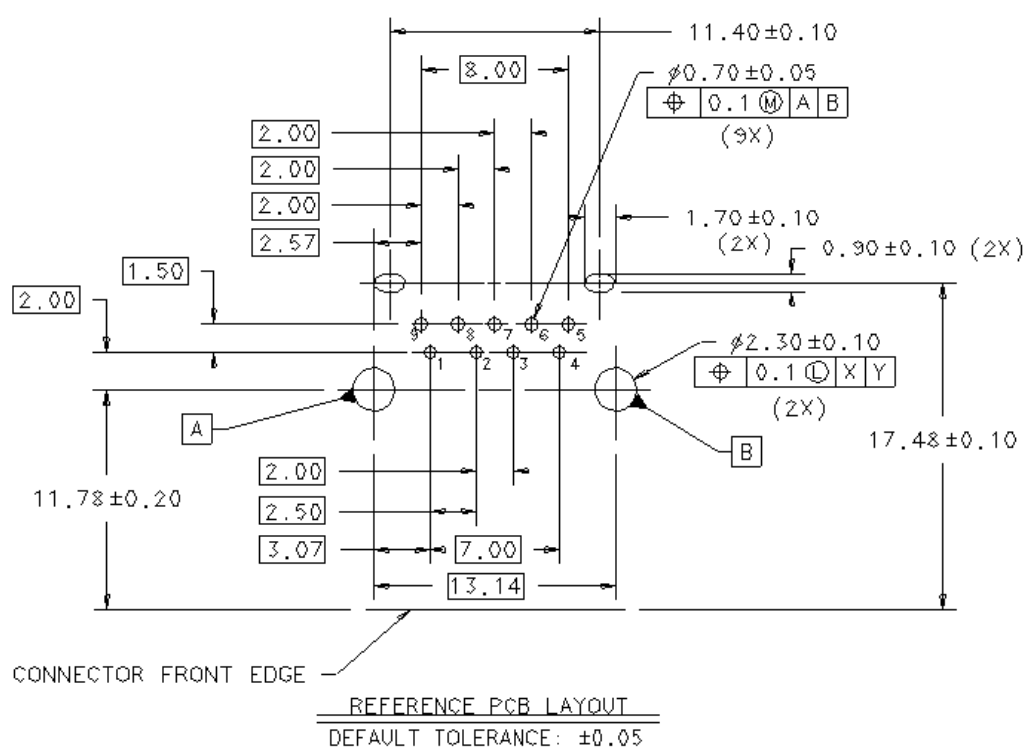
Figure 5-5 shows through-hole example footprints for the USB 3.1 Standard-A receptacle with a back-shield. Pin numbers are marked.

Figure 5-6 shows an example footprint for a mid-mount standard mount (mounted on the top of the PCB) Standard-A receptacle that includes Insertion Detect.

Figure 5-7 shows an example mid-mount reverse mount (mounted on the bottom of the PCB) with Insertion Detect. The reverse mount configuration locates the SuperSpeed signals between the USB 2.0 signals and the PCB edge, making the SuperSpeed signal routing more challenging.

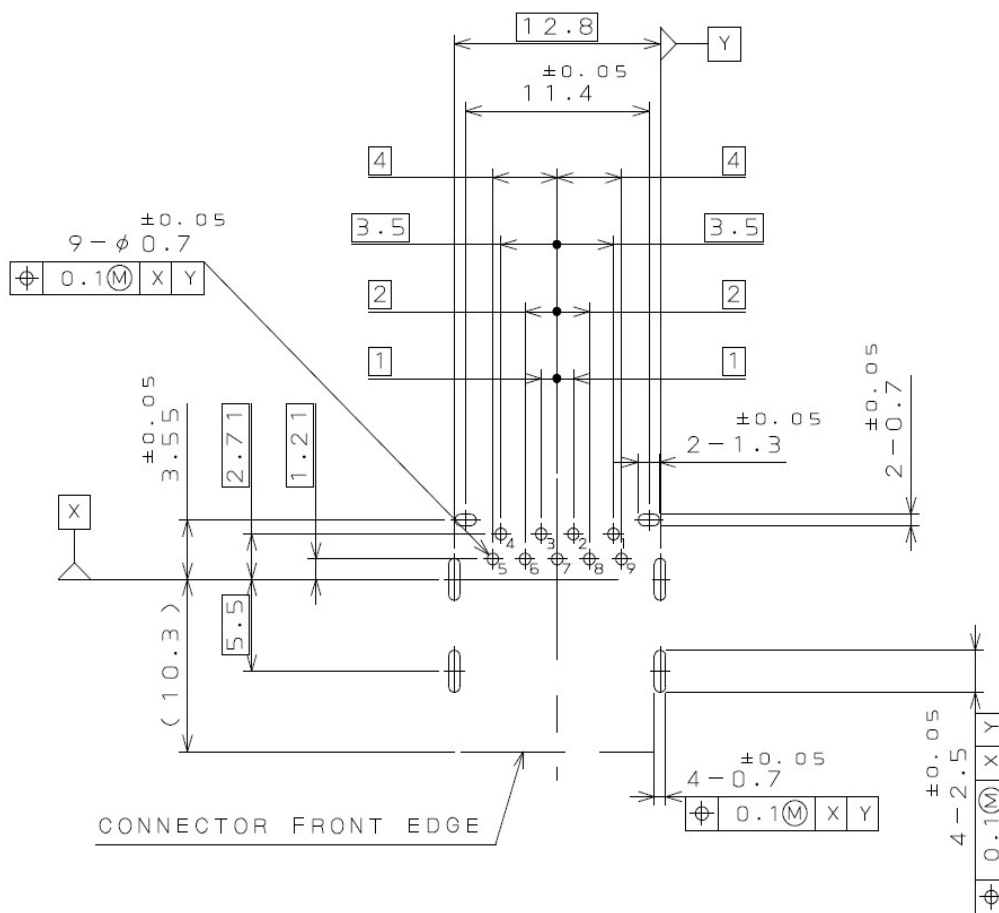
PCB LAYOUT (REF.)
(STANDARD MOUNT TYPE)

Continued on next page



IEC

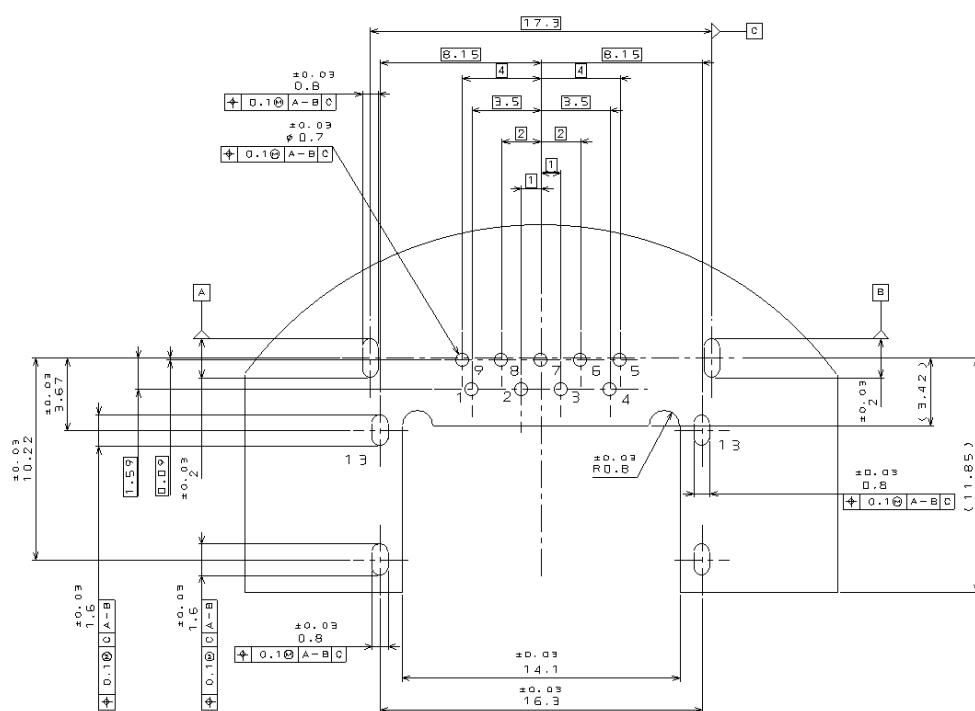
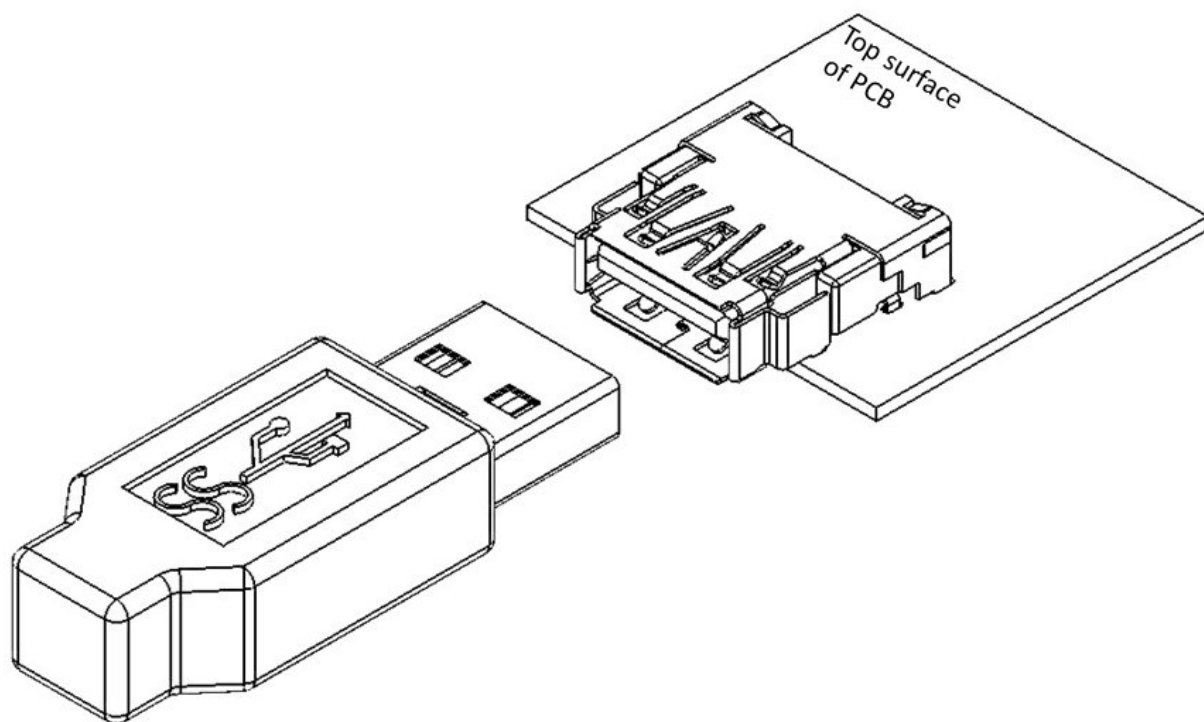
Continued on next page



PCB LAYOUT (REF.)
(REVERSE MOUNT TYPE)

IEC

Figure 5-5 – Example Footprint for the USB 3.1 Standard-A Receptacle – Through-Hole with Back-Shield



IEC

Figure 5-6 – Example Footprint for the USB 3.1 Standard-A Receptacle – Mid-Mount Standard Mount Through-Hole with Insertion Detect

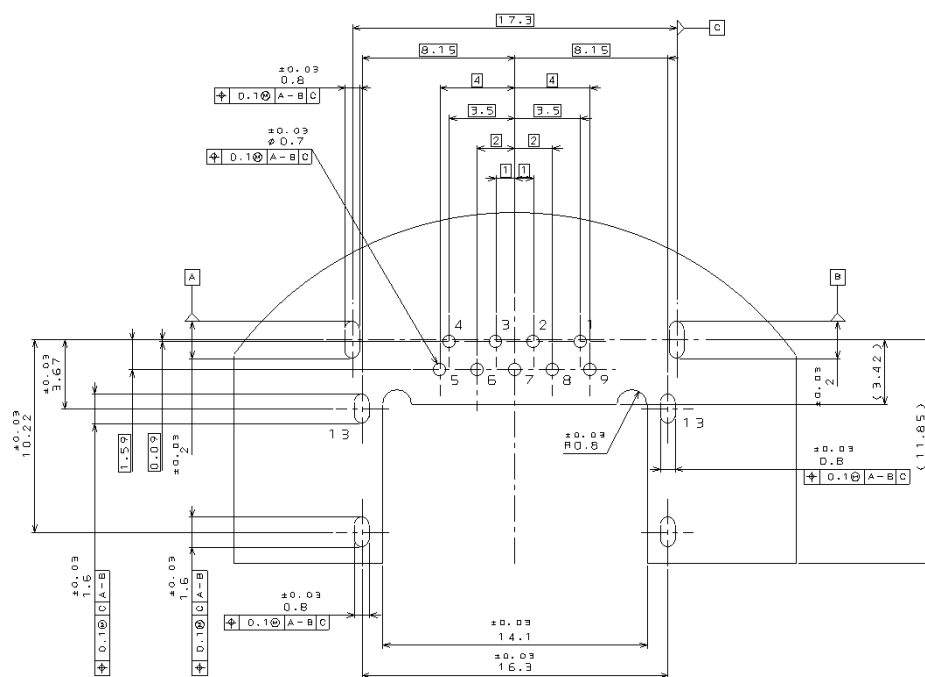


Figure 5-7 – Example Footprint for the USB 3.1 Standard-A Receptacle – Mid-Mount Reverse Mount Through-Hole with Insertion Detect

The usage and assignments of the nine pins in the USB 3.1 Standard-A connector are defined in Table 5-2.

Table 5-2 – USB 3.1 Standard-A Connector Pin Assignments

Pin Number ¹	Signal Name ²	Description	Mating Sequence ³
1	VBUS	Power	Third
2	D-	USB 2.0 differential pair	Fourth
3	D+		
4	GND	Ground for power return	Third
5	StdA_SSRX-	SuperSpeed receiver differential pair	Last
6	StdA_SSRX+		
7	GND_DRAIN	Ground for signal return	
8	StdA_SSTX-	SuperSpeed transmitter differential pair	
9	StdA_SSTX+		
12 ⁴ , 13	INSERTION DETECT	Receptacle only. Detects insertion of a plug into the receptacle. Optional except for cold socket applications. See the Universal Serial Bus Power Delivery Specification for details.	Second
Shell	Shield	Connector metal shell	First

¹ Pin numbers not included in this table do not have contacts present.

² Tx and Rx are defined from the host perspective.

³ The mating sequence assumes support of INSERTION DETECT.

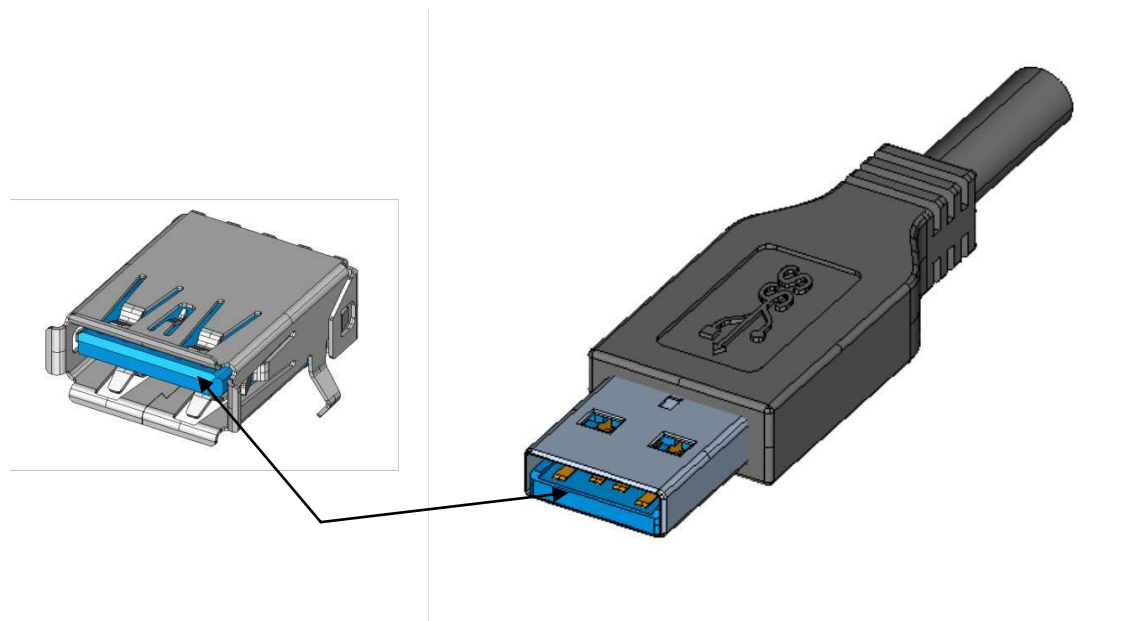
⁴ Pin 12, if present, shall be connected to Shield.

The physical location of the pins in the connector is illustrated in Figure 5-1 to Figure 5-7. Pins 1 to 4 are referred to as the USB 2.0 pins, while pins 5 to 9 are referred to as the SuperSpeed pins. See the Universal Serial Bus Power Delivery Specification for location of pins 12 and 13.

5.3.1.4 USB 3.1 Standard-A Connector Color Coding

Since both the USB 2.0 Standard-A and USB 3.1 Standard-A receptacles may co-exist on a host, color coding is recommended for the USB 3.1 Standard-A connector (receptacle and plug) housings to help users distinguish it from the USB 2.0 Standard-A connector.

Blue (Pantone 300C) is the recommended color for the USB 3.1 Standard-A receptacle and plug plastic housings. When the recommended color is used, connector manufacturers and system integrators should make sure that the blue-colored receptacle housing is visible to users. Figure 5-8 illustrates the color coding recommendation for the USB 3.1 Standard-A connector.



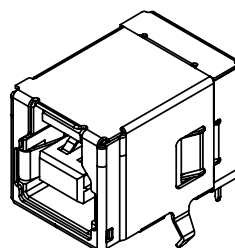
IEC

Figure 5-8 – Illustration of Color Coding Recommendation for USB 3.1 Standard-A Connector

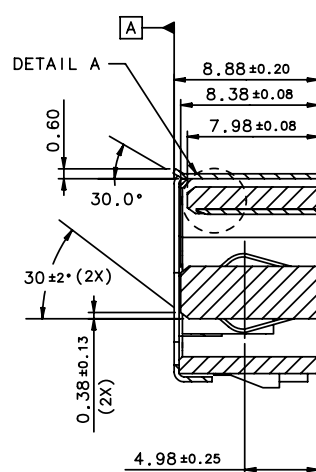
5.3.2 USB 3.1 Standard-B Connector

5.3.2.1 Interface Definition

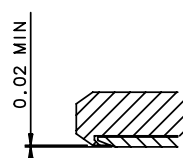
Figure 5-9, Figure 5-10, and Figure 5-11 show the USB Standard-B receptacle dimensions, the USB Standard-B plug dimensions, and a USB Standard-B receptacle reference footprint, respectively. See Section 5.6.1.2 for target characteristic impedance.



SECTION B-B



SECTION A-A

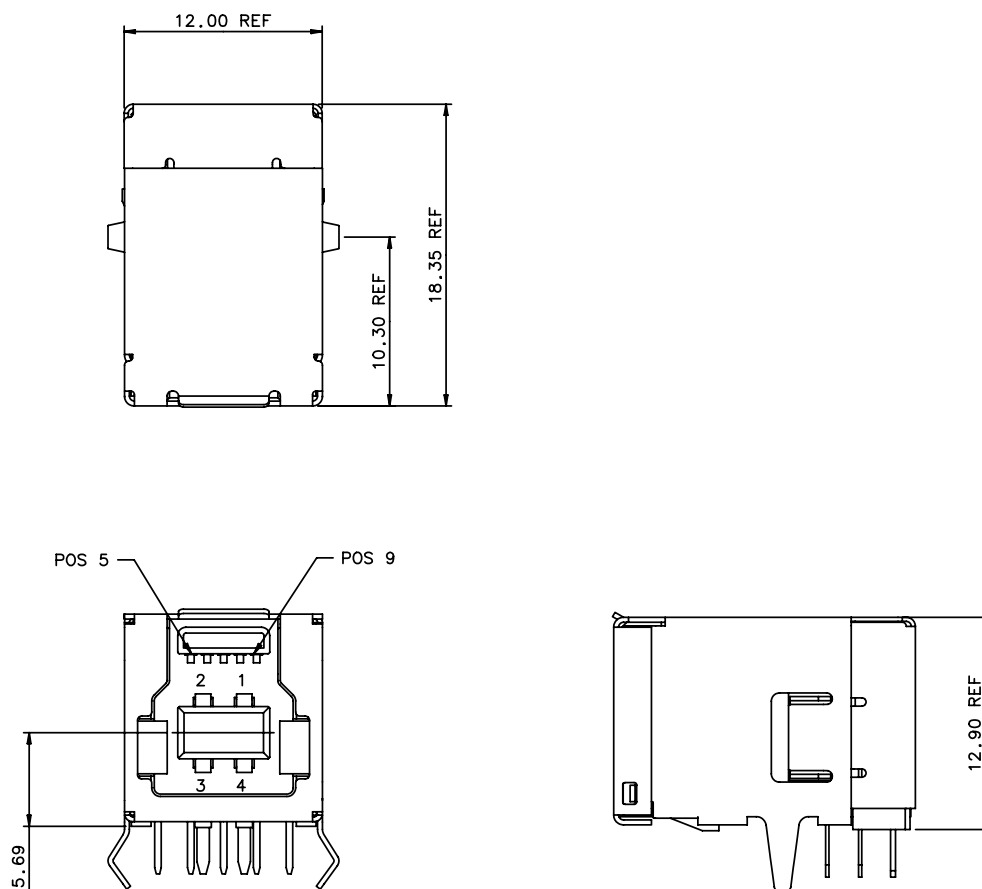


DETAIL A

SECTION C-C

IEC

Continued on next page



NOTES:

- 1) CRITICAL DIMENSIONS ARE TOLERANCED AND SHALL NOT BE DEVIATED.
- 2) GENERAL TOLERANCE IS ± 0.10 OTHERWISE THE SPECIFIED TOLERANCE APPLIES.
- 3) ALL DIMENSIONS ARE IN MILLIMETERS.
- 4) DIMENSIONS THAT ARE LABELED REF ARE TYPICAL DIMENSIONS AND MAY VARY FROM MANUFACTURER TO MANUFACTURER.
- 5) NON-DIMENSIONED GEOMETRY FOR REFERENCE ONLY, SUBJECT TO CHANGE.

IEC

Figure 5-9 – USB 3.1 Standard-B Receptacle Interface Dimensions

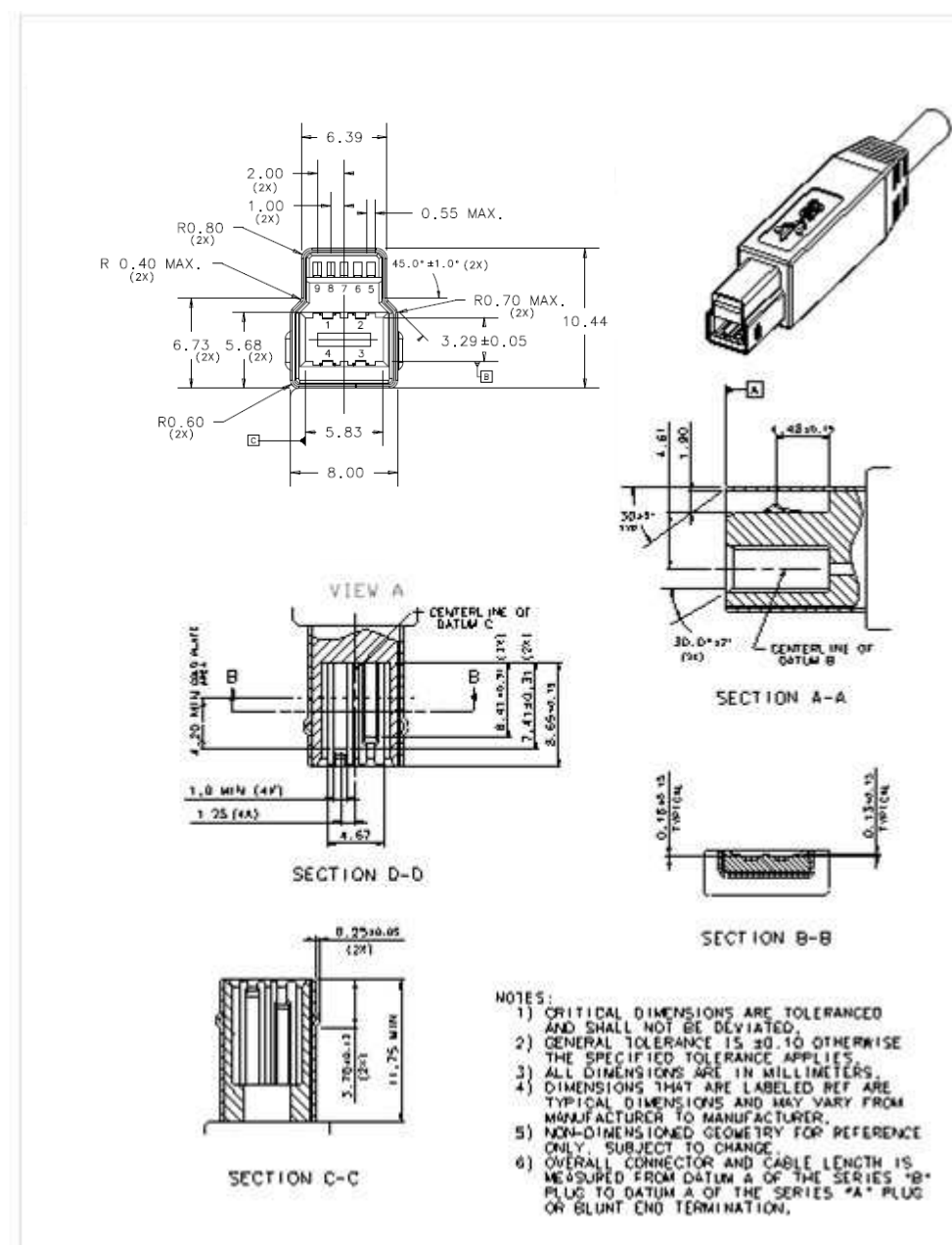
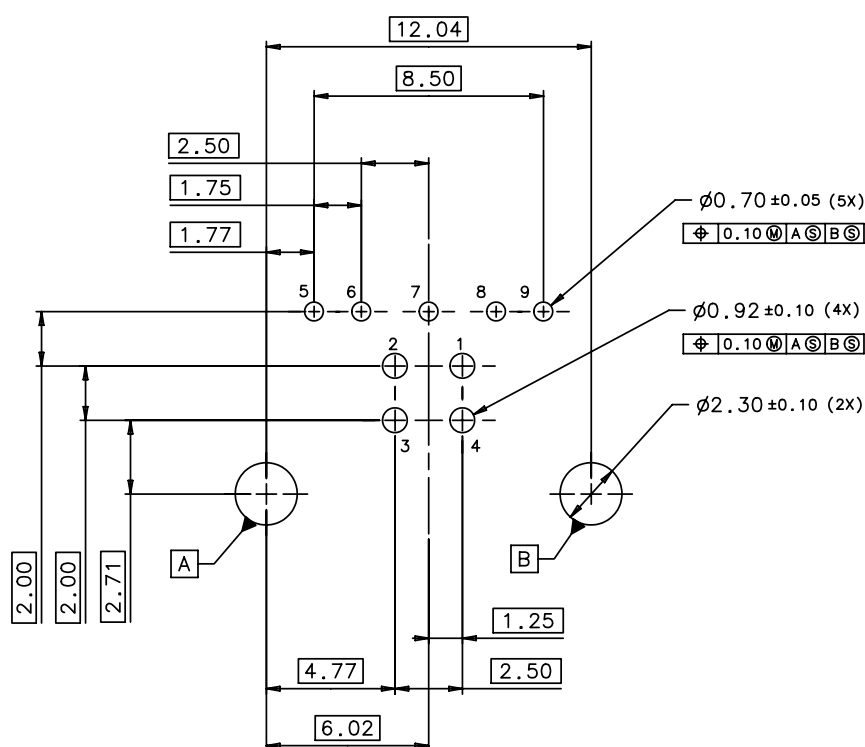


Figure 5-10 – USB 3.1 Standard-B Plug Interface Dimensions



REFERENCE PCB LAYOUT

IEC

Figure 5-11 – Reference Footprint for the USB 3.1 Standard-B Receptacle

The USB 3.1 Standard-B receptacle interfaces have two portions: the USB 2.0 interface and the SuperSpeed interface. The USB 2.0 interface consists of pins 1 to 4, while the SuperSpeed interface consists of pins 5 to 9.

When a USB 2.0 Standard-B plug is inserted into the USB 3.1 Standard-B receptacle, only the USB 2.0 interface is engaged and the link will not take advantage of the Enhanced SuperSpeed capability. Since the USB 3.1 SuperSpeed portion is visibly not mated when a USB 2.0 Standard-B plug is inserted in the USB 3.1 Standard-B receptacle, users have the visual feedback that the cable plug is not matched with the receptacle. Only when a USB 3.1 Standard-B plug is inserted into the USB 3.1 Standard-B receptacle, is the interface completely visibly engaged.

5.3.2.2 Pin Assignments and Description

The usage and assignments of the nine pins in the USB 3.1 Standard-B connector are defined in Table 5-3.

Table 5-3 – USB 3.1 Standard-B Connector Pin Assignments

Pin Number	Signal Name	Description	Mating Sequence
1	VBUS	Power	Second
2	D-	USB 2.0 differential pair	Third or beyond
3	D+		
4	GND	Ground for power return	Second
5	StdB_SSTX-	SuperSpeed transmitter differential pair	Third or beyond
6	StdB_SSTX+		
7	GND_DRAIN	Ground for signal return	
8	StdB_SSRX-	SuperSpeed receiver differential pair	
9	StdB_SSRX+		
Shell	Shield	Connector metal shell	First

NOTE Tx and Rx are defined from the device perspective.

The physical location of the pins in the connector is illustrated in Figure 5-9 to Figure 5-11.

5.3.3 USB 3.1 Micro Connector Family

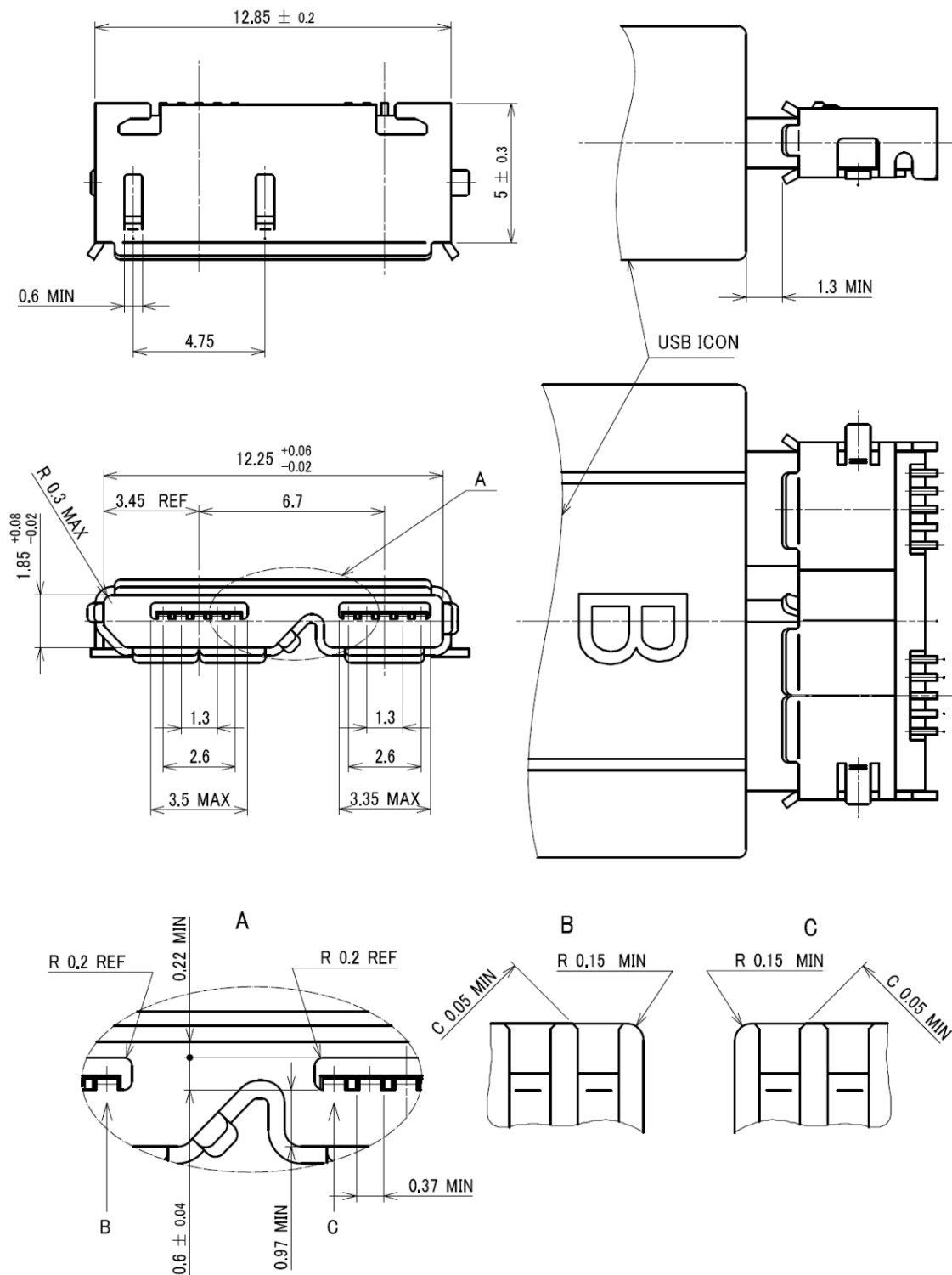
5.3.3.1 Interfaces Definition

The USB 3.1 Micro connector family consists of the USB 3.1 Micro-B receptacle, USB 3.1 Micro-AB receptacle, USB 3.1 Micro-B plug, and USB 3.1 Micro-A plug. Figure 5-12 and Figure 5-13 show the USB 3.1 Micro family receptacle and plug interface dimensions, respectively. Only dimensions that govern the mating interoperability are specified.

The USB 3.1 Micro connector family has the following characteristics:

- The USB 3.1 Micro-B connector may be considered a combination of USB 2.0 Micro-B interface and the USB 3.1 SuperSpeed contacts. The USB 3.1 Micro-B receptacle accepts a USB 2.0 Micro-B plug, maintaining backward compatibility.
- The USB 3.1 Micro-B connector maintains the same connector height and contact pitch as the USB 2.0 Micro-B connector.
- The USB 3.1 Micro-B connector uses the same latch design as the USB 2.0 Micro-B connector.
- The USB 3.1 Micro-AB receptacle is identical to the USB 3.1 Micro-B receptacle except for a keying difference in the connector shell outline.
- The USB 3.1 Micro-A plug is similar to the USB 3.1 Micro-B plug with different keying and ID pin connections. The Universal Serial Bus Power Delivery Specification discusses the ID pin connections.

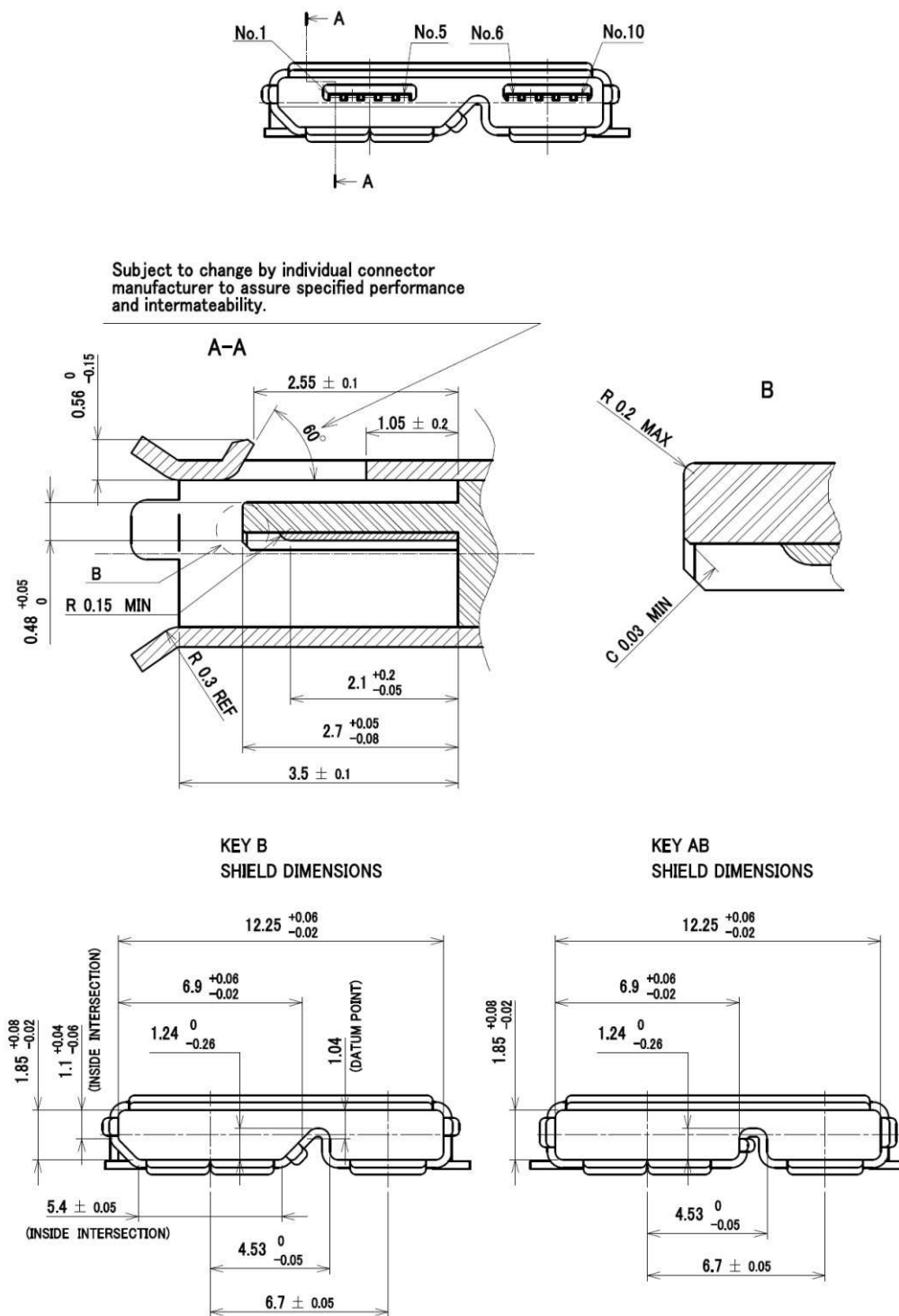
There is no required footprint for the USB 3.1 Micro connector family. Figure 5-14 shows reference Micro-B and -AB connector footprints.



NOTE : General tolerance is $\pm 0.05 \text{ mm}$, otherwise the specified tolerances apply.

IEC

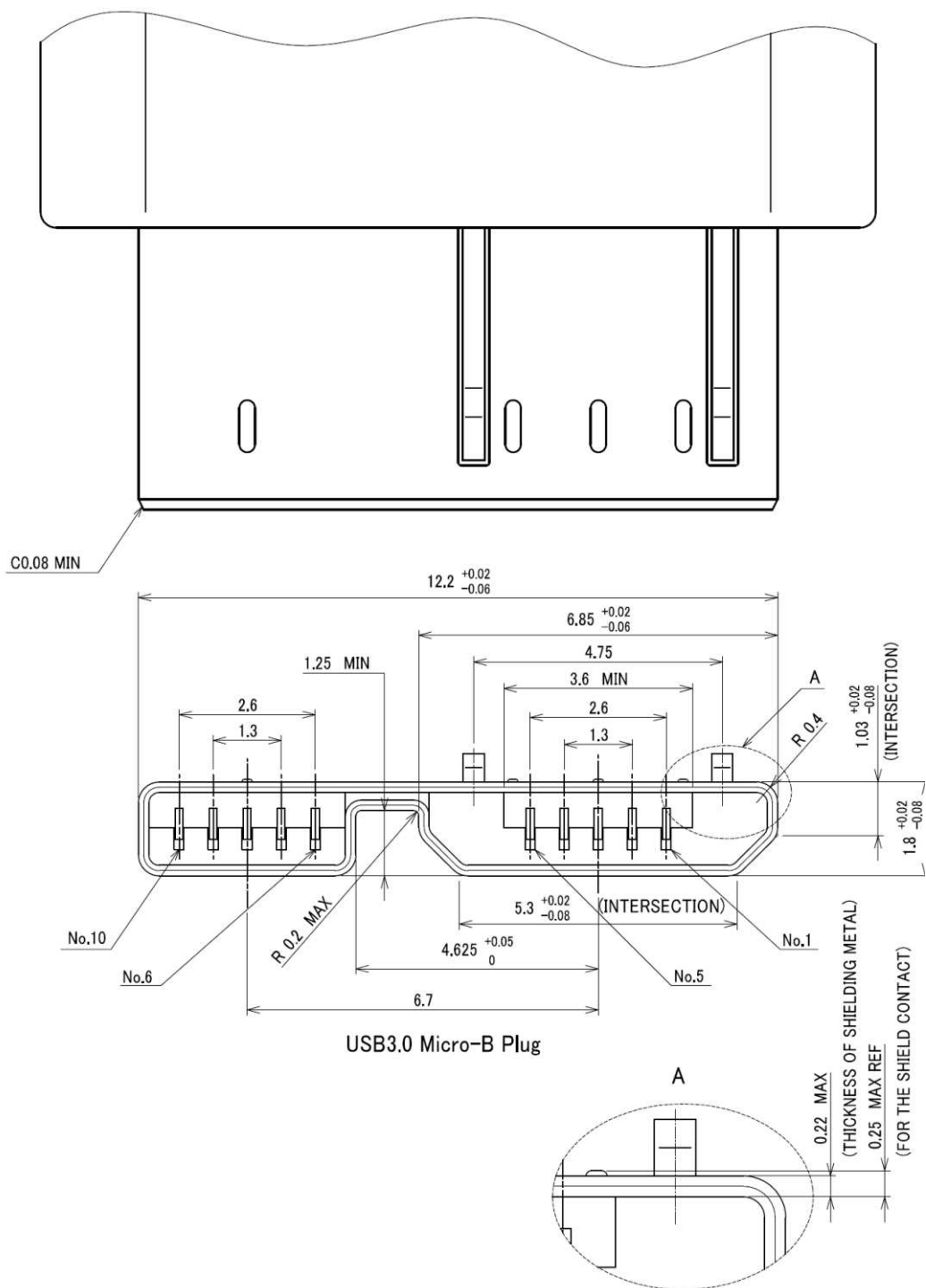
Continued on next page



NOTE : Chamfer metals are optional with no sharp edges.

IEC

Figure 5-12 – USB 3.1 Micro-B and -AB Receptacles Interface Dimensions

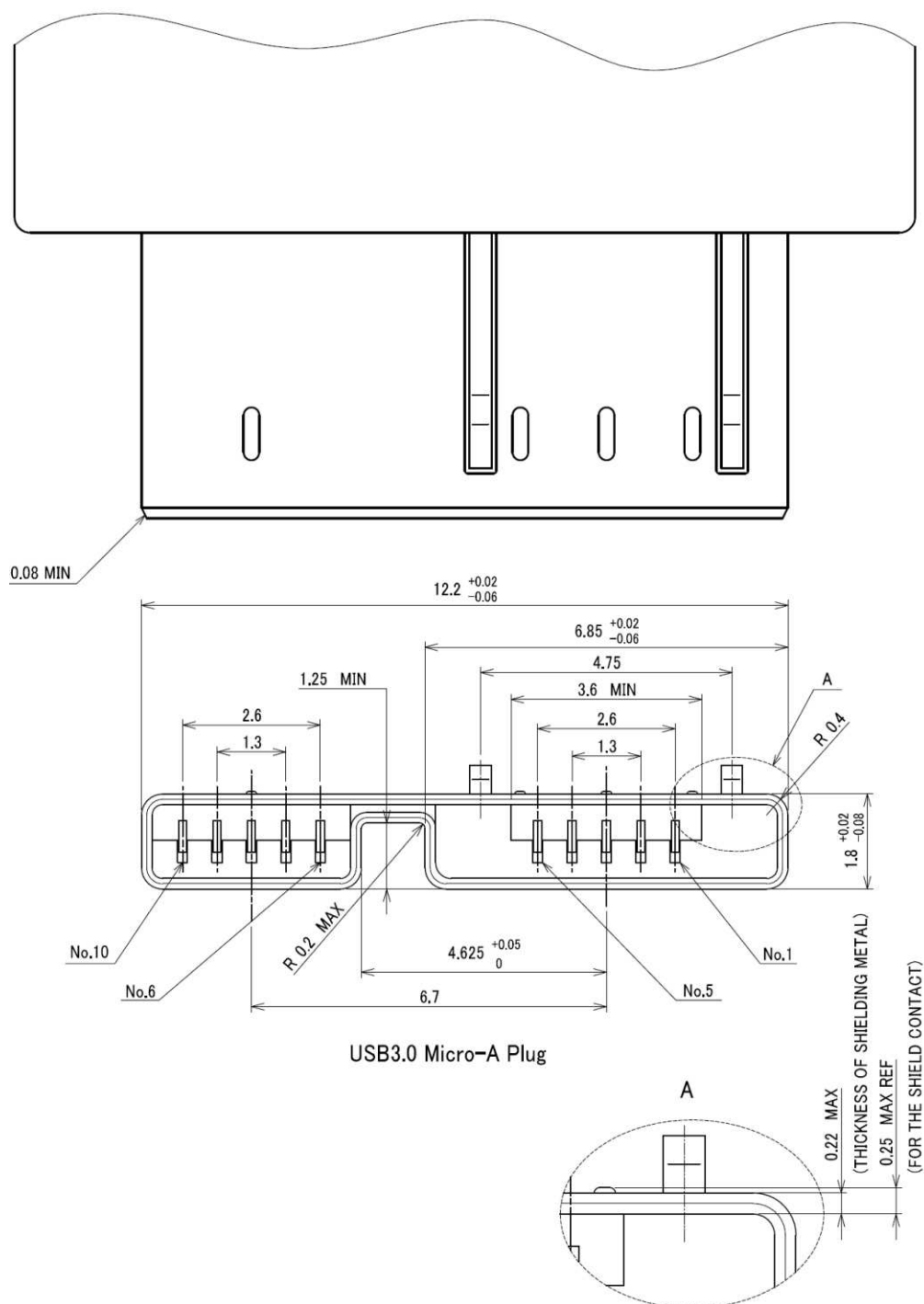


NOTES:

1. Dimensions that are labeled REF may vary from manufacturer to manufacturer.
2. General tolerance is ± 0.05 mm, otherwise the specified tolerances apply.

IEC

Continued on next page



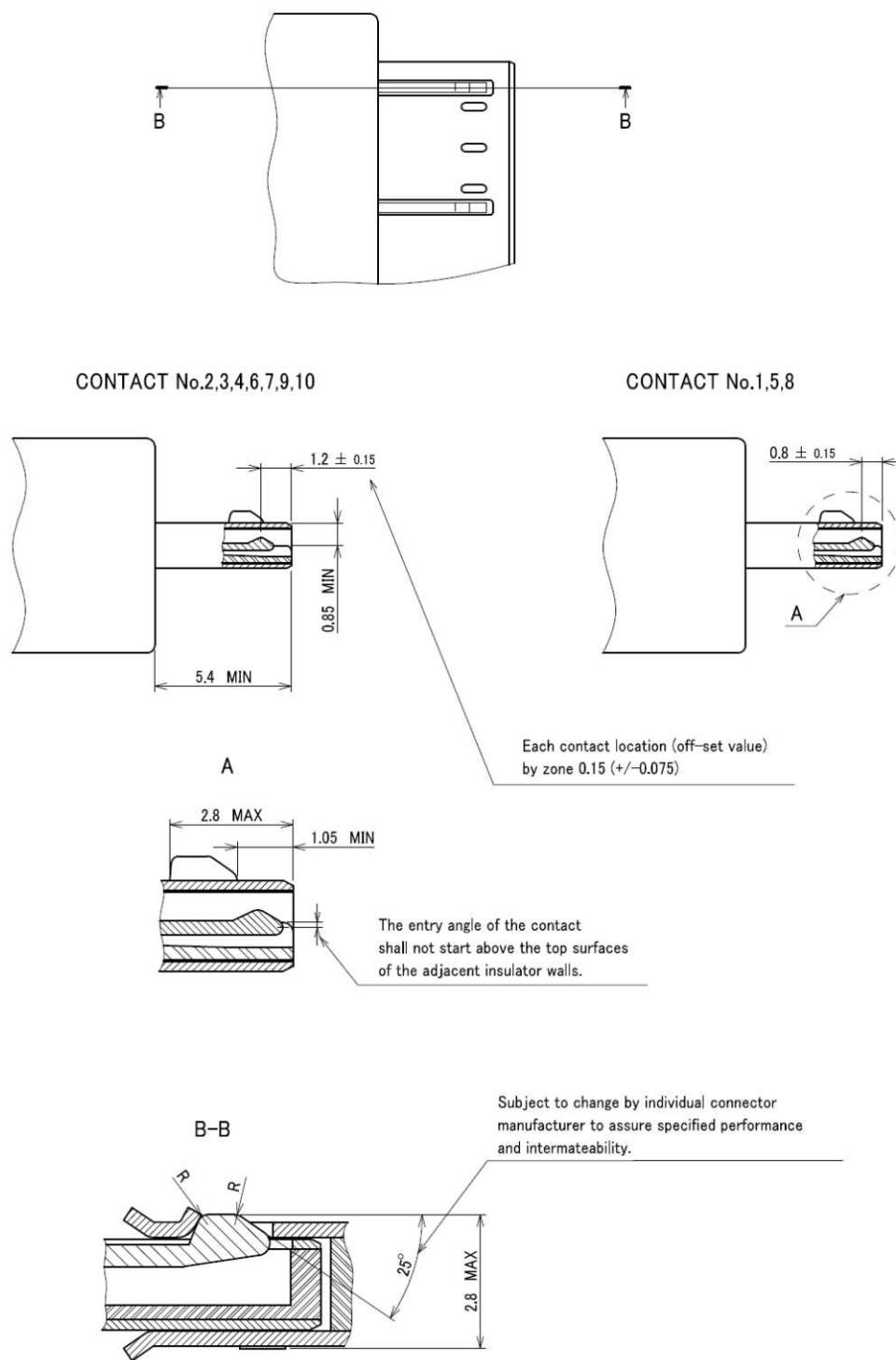
USB3.0 Micro-A Plug

NOTES:

1. Dimensions that are labeled REF may vary from manufacturer to manufacturer.
2. General tolerance is $\pm 0.05\text{mm}$, otherwise the specified tolerances apply.

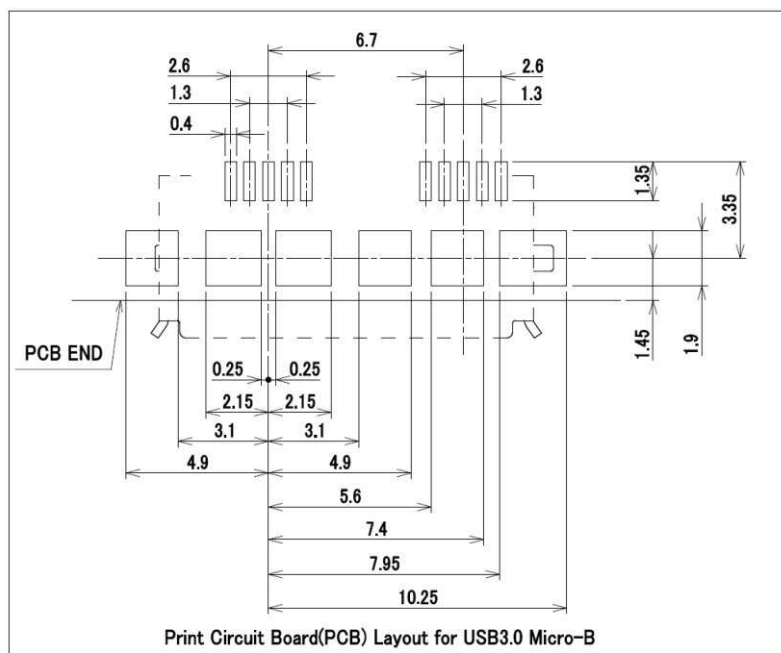
IEC

Continued on next page

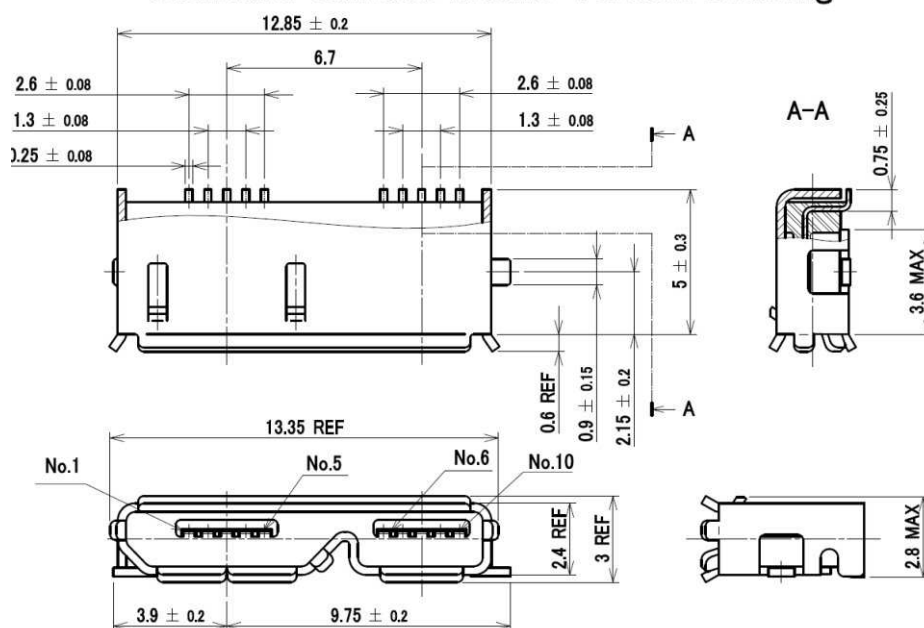


IEC

Figure 5-13 – USB 3.1 Micro-B and Micro-A Plug Interface Dimensions



Standard-Surface Mount-Version Drawing



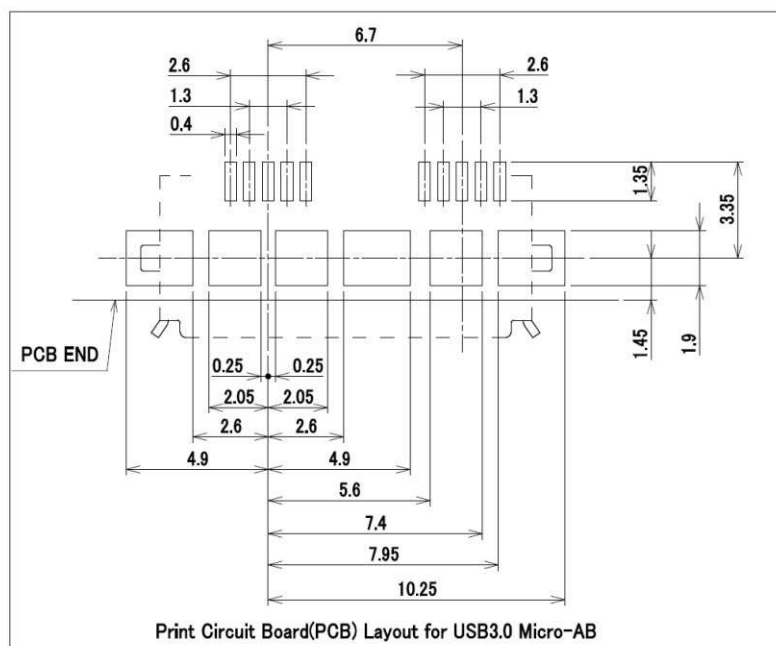
USB3.0 Micro-B RECEPTACLE

NOTES :

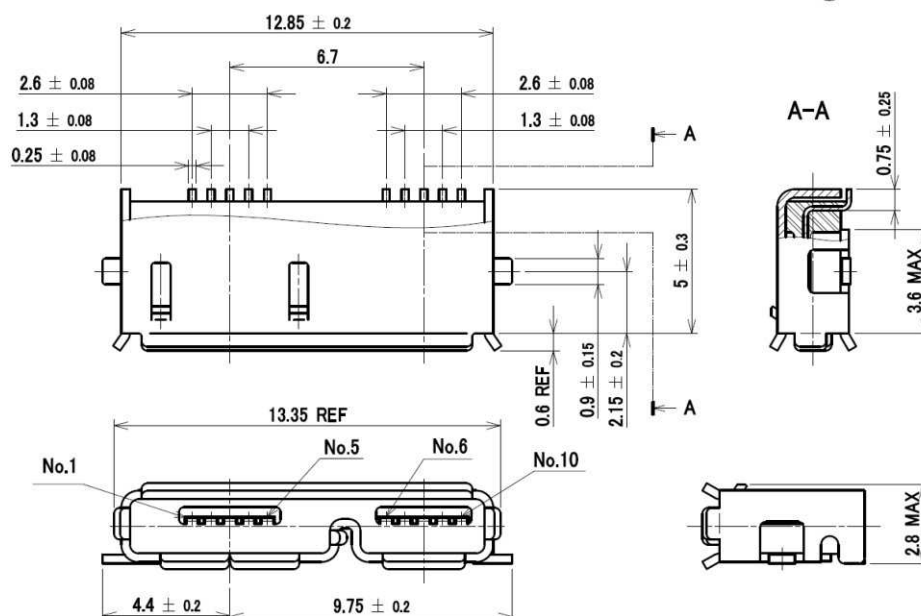
1. Critical Dimensions are TOLERANCED and shall not be deviated.
2. Dimensions that labeled REF are typical dimensions and may vary from manufacturer to manufacturer.
3. General tolerance is ± 0.05 mm, otherwise the specified tolerances apply.
4. Chamfer metals are optional with no sharp edges.

IEC

Continued on next page



Standard-Surface Mount-Version Drawing



USB3.0 Micro-AB RECEPTACLE

NOTES :

1. Critical Dimensions are TOLERANCED and shall not be deviated.
2. Dimensions that labeled REF are typical dimensions and may vary from manufacturer to manufacturer.
3. General tolerance is ± 0.05 mm, otherwise the specified tolerances apply.
4. Chamfer metals are optional with no sharp edges.

IEC

Figure 5-14 – Reference Footprint for the USB 3.1 Micro-B or Micro-AB Receptacle

5.3.3.2 Pin Assignments and Description

Table 5-4 and Table 5-5 show the pin assignments for the USB 3.1 Micro connector family.

Table 5-4 – USB 3.1 Micro-B Connector Pin Assignments

Pin Number	Signal Name	Description	Mating Sequence
1	VBUS	Power	Second
2	D-	USB 2.0 differential pair	Last
3	D+		
4	ID		
5	GND	Ground for power return	Second
6	MicB_SSTX-	SuperSpeed transmitter differential pair	Last
7	MicB_SSTX+		
8	GND_DRAIN	Ground for SuperSpeed signal return	Second
9	MicB_SSRX-	SuperSpeed receiver differential pair	Last
10	MicB_SSRX+		
Shell	Shield	Connector metal shell	First

NOTE Tx and Rx are defined from the device perspective.

Table 5-5 – USB 3.1 Micro-AB/-A Connector Pin Assignments

Pin Number	Signal Name	Description	Mating Sequence
1	VBUS	Power	Second
2	D-	USB 2.0 differential pair	Last
3	D+		
4	ID		
5	GND	Ground for power return	Second
6	MicA_SSTX-	SuperSpeed receiver differential pair	Last
7	MicA_SSTX+		
8	GND_DRAIN	Ground for SuperSpeed signal return	Second
9	MicA_SSRX-	SuperSpeed transmitter differential pair	Last
10	MicA_SSRX+		
Shell	Shield	Connector metal shell	First

NOTE Tx and Rx are defined when an OTG device serves as a host.

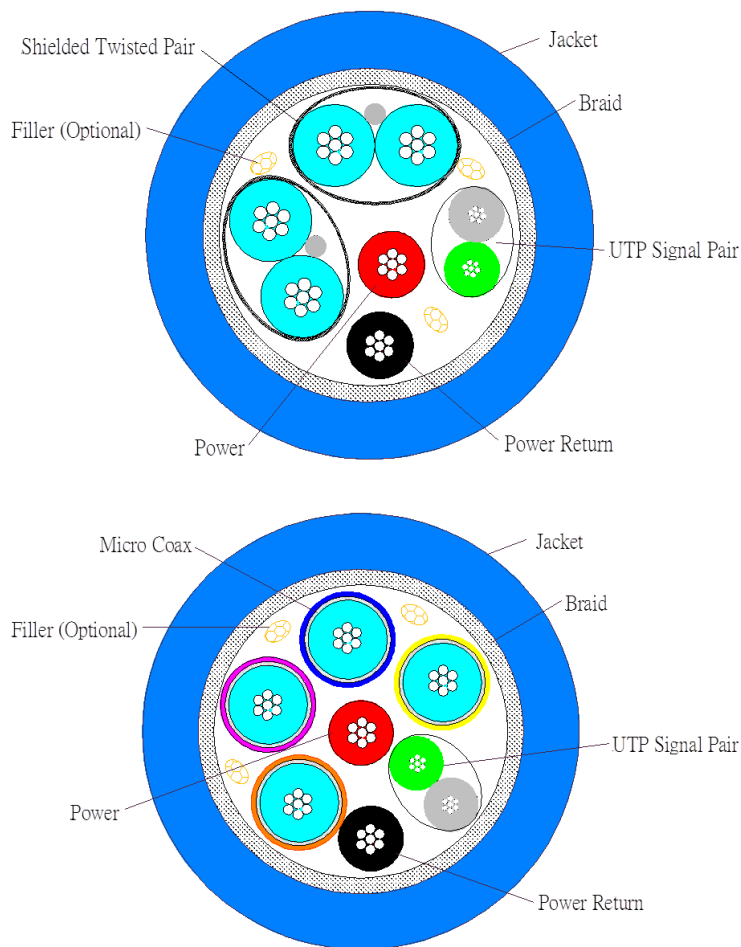
The physical location of the pins in the connector is illustrated in Figure 5-12 to Figure 5-14.

5.4 Cable Construction and Wire Assignments

This section discusses the USB 3.1 cables, including cable construction, wire assignments, and wire gauges. The performance requirements are specified in Section 5.6.1.

5.4.1 Cable Construction

Figure 5-15 illustrates a USB 3.1 cable cross-section. There are three groups of wires: D+/D-signal pair (typically unshielded twisted pair (UTP)), Enhanced SuperSpeed signal pairs (typically Shielded Differential Pair (SDP), twisted, twinax, or coaxial signal pairs), and power and ground wires.



IEC

Figure 5-15 – Illustration of a USB 3.1 Cable Cross-Section

The D+/D- signal pair is intended to transmit the USB 2.0 signaling while the Enhanced SuperSpeed signal pairs are used for SuperSpeed; the shield is needed for the SuperSpeed differential pairs for signal integrity and EMI performance. Each Enhanced SuperSpeed drain wire is connected to the system ground through the GND_DRAIN pin(s) in the connector.

A metal braid is required to enclose all the wires in the USB 3.1 cable. The braid shall be terminated to the plug metal shells, as close to 360° as possible, to reduce EMI.

5.4.2 Wire Assignments

Table 5-6 defines the wire number, signal assignments of the wires.

Table 5-6 – Cable Wire Assignments

Wire Number	Signal Name	Description
1	PWR	Power
2	D-	Unshielded twist pair, negative
3	D+	Unshielded twist pair, positive
4	GND_PWRrt	Ground for power return
5	P1-	Shielded differential pair 1, negative
6	P1+	Shielded differential pair 1, positive
7	P1_Drain	Drain wire for SDP1
8	P2-	Shielded differential pair 2, negative
9	P2+	Shielded differential pair 2, positive
10	P2_Drain	Drain wire for SDP2
Braid	Shield	Cable external braid to be 360° terminated on to plug metal shell

5.4.3 Wire Gauges and Cable Diameters

This specification does not specify wire gauges. Table 5-7 lists typical wire gauges for reference purposes only. A large gauge wire incurs less loss, but at the cost of cable flexibility. It is recommended to use the smallest possible wire gauges that meet the cable assembly electrical requirements.

To maximize cable flexibility, all wires should be stranded and the cable outer diameter should be minimized as much as possible. A typical non-USB 3.1 Power Delivery capable cable outer diameter may range from 3 mm to 6 mm.

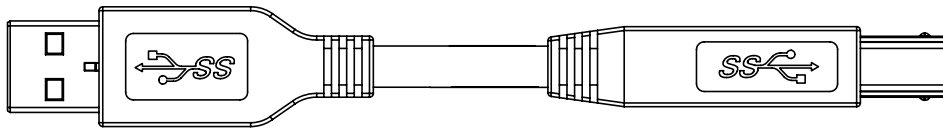
Table 5-7 – Reference Wire Gauges

Wire Number	Signal Name	Wire Gauge (AWG)
1	PWR	20-28
2	D-	28-34
3	D+	28-34
4	GND_PWRrt	20-28
5	P1-	26-34
6	SP1+	26-34
7	P1_Drain	28-34
8	P2-	26-34
9	P2+	26-34
10	P2_Drain	28-34

5.5 Cable Assemblies

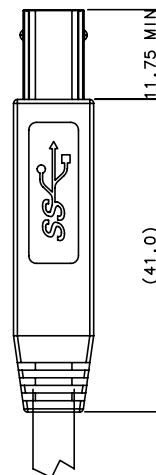
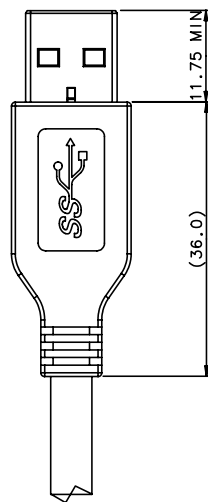
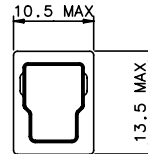
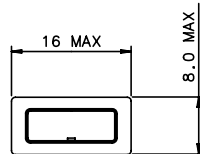
5.5.1 USB 3.1 Standard-A to USB 3.1 Standard-B Cable Assembly

Figure 5-16 shows a USB 3.1 Standard-A to USB 3.1 Standard-B cable assembly. Due to increased wire sizes required for some PD cable implementations, the overmold dimensions for PD cables have larger maximum dimensions than the non-PD cables specified. See the *Universal Serial Bus Power Delivery Specification* for the maximum overmold dimensions of PD cable assemblies.



STANDARD "A" PLUG
(ALWAYS UPSTREAM TOWARDS
THE "HOST" SYSTEM)

STANDARD "B" PLUG
(ALWAYS DOWNSTREAM
TOWARDS THE USB DEVICE)



STANDARD "A" PLUG

STANDARD "B" PLUG

IEC

Figure 5-16 – USB 3.1 Standard-A to USB 3.1 Standard-B Cable Assembly

Table 5-8 defines the wire connections for the USB 3.1 Standard-A to USB 3.1 Standard-B cable assembly.

Table 5-8 – USB 3.1 Standard-A to USB 3.1 Standard-B Cable Assembly Wiring

USB 3.1 Standard-A Plug		Wire		USB 3.1 Standard-B Plug	
Pin Number	Signal Name	Wire Number	Signal Name	Pin Number	Signal Name
1	VBUS	1	PWR	1	VBUS
2	D-	2	D-	2	D-
3	D+	3	D+	3	D+
4	GND	4	GND_PWRrt	4	GND
5	StdA_SSRX-	5	P1-	5	StdB_SSTX-
6	StdA_SSRX+	6	P1+	6	StdB_SSTX+
7	GND_DRAIN	7 and 10	P1_Drain P2_Drain	7	GND_DRAIN
8	StdA_SSTX-	8	P2-	8	StdB_SSRX-
9	StdA_SSTX+	9	P2+	9	StdB_SSRX+
Shell	Shield	Braid	Shield	Shell	Shield

5.5.2 USB 3.1 Standard-A to USB 3.1 Standard-A Cable Assembly

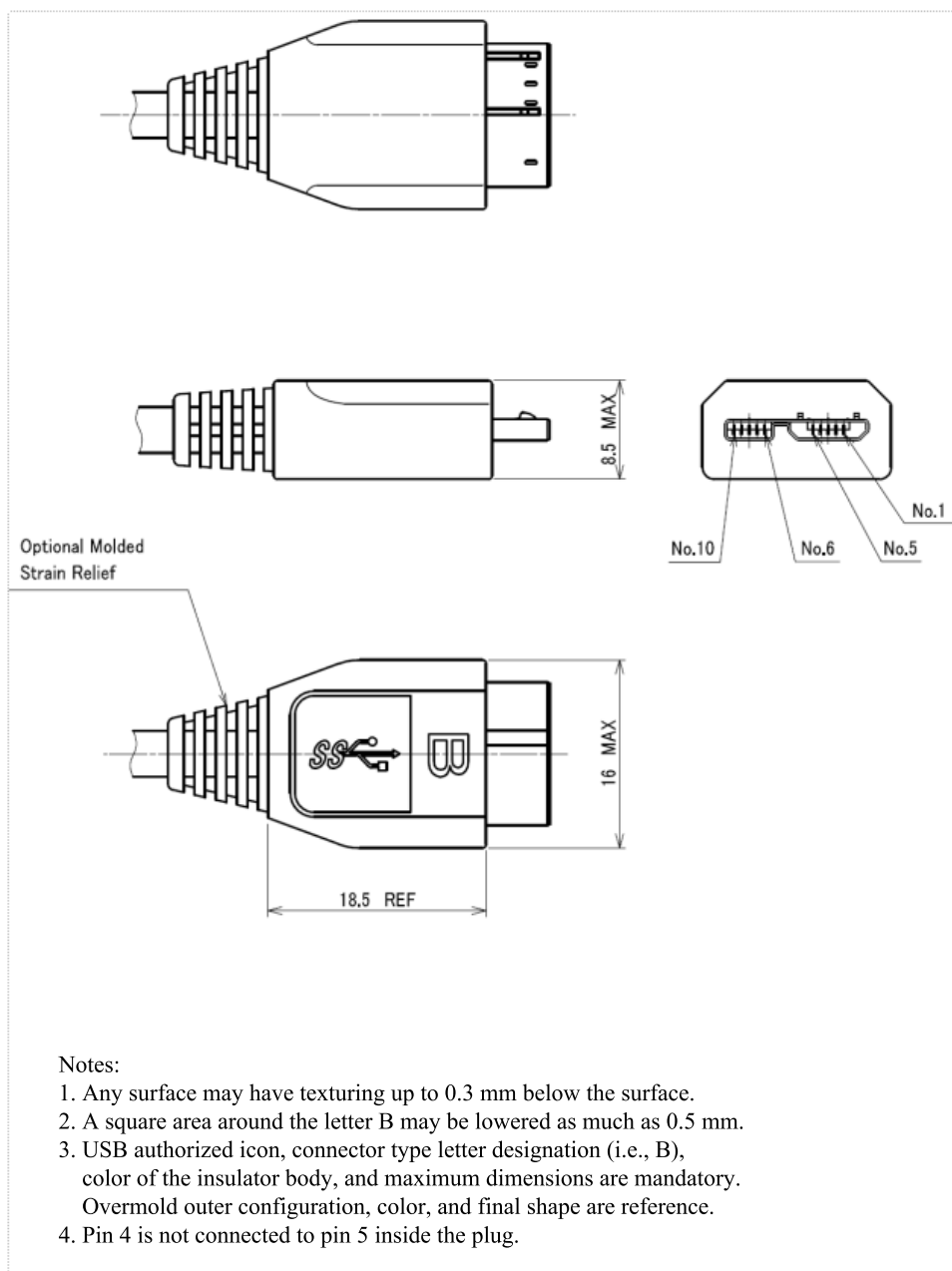
The USB 3.1 Standard-A to USB 3.1 Standard-A cable assembly is defined for operating system debugging and other host-to-host connection applications. Table 5-9 shows wire connections for such a cable assembly. Refer to Figure 5-16 for the USB 3.1 Standard-A plug cable overmold dimensions.

Table 5-9 – USB 3.1 Standard-A to USB 3.1 Standard-A Cable Assembly Wiring

USB 3.1 Standard-A Plug #1		Wire		USB 3.1 Standard-A Plug #2	
Pin Number	Signal Name	Wire Number	Signal Name	Pin Number	Signal Name
1	VBUS	No connect		1	VBUS
2	D-	No connect		2	D-
3	D+	No connect		3	D+
4	GND	4	GND_PWRrt	4	GND
5	StdA_SSRX-	5	P1-	8	StdA_SSTX-
6	StdA_SSRX+	6	P1+	9	StdA_SSTX+
7	GND_DRAIN	7 & 10	P1_Drain P2_Drain	7	GND_DRAIN
8	StdA_SSTX-	8	P2-	5	StdA_SSRX-
9	StdA_SSTX+	9	P2+	6	StdA_SSRX+
Shell	Shield	Braid	Shield	Shell	Shield

5.5.3 USB 3.1 Standard-A to USB 3.1 Micro-B Cable Assembly

Figure 5-17 shows the USB 3.1 Micro-B plug overmold dimensions for a USB 3.1 Standard-A to USB 3.1 Micro-B cable assembly. The USB 3.1 Standard-A plug overmold dimensions are found in Figure 5-16. Due to increased wire sizes required for some PD cable implementations, the overmold dimensions for PD cables have larger maximum dimensions than the non-PD cables specified. See the *Universal Serial Bus Power Delivery Specification* for the maximum overmold dimensions of PD cable assemblies.



IEC

Figure 5-17 – USB 3.1 Micro-B Plug Cable Overmold Dimensions

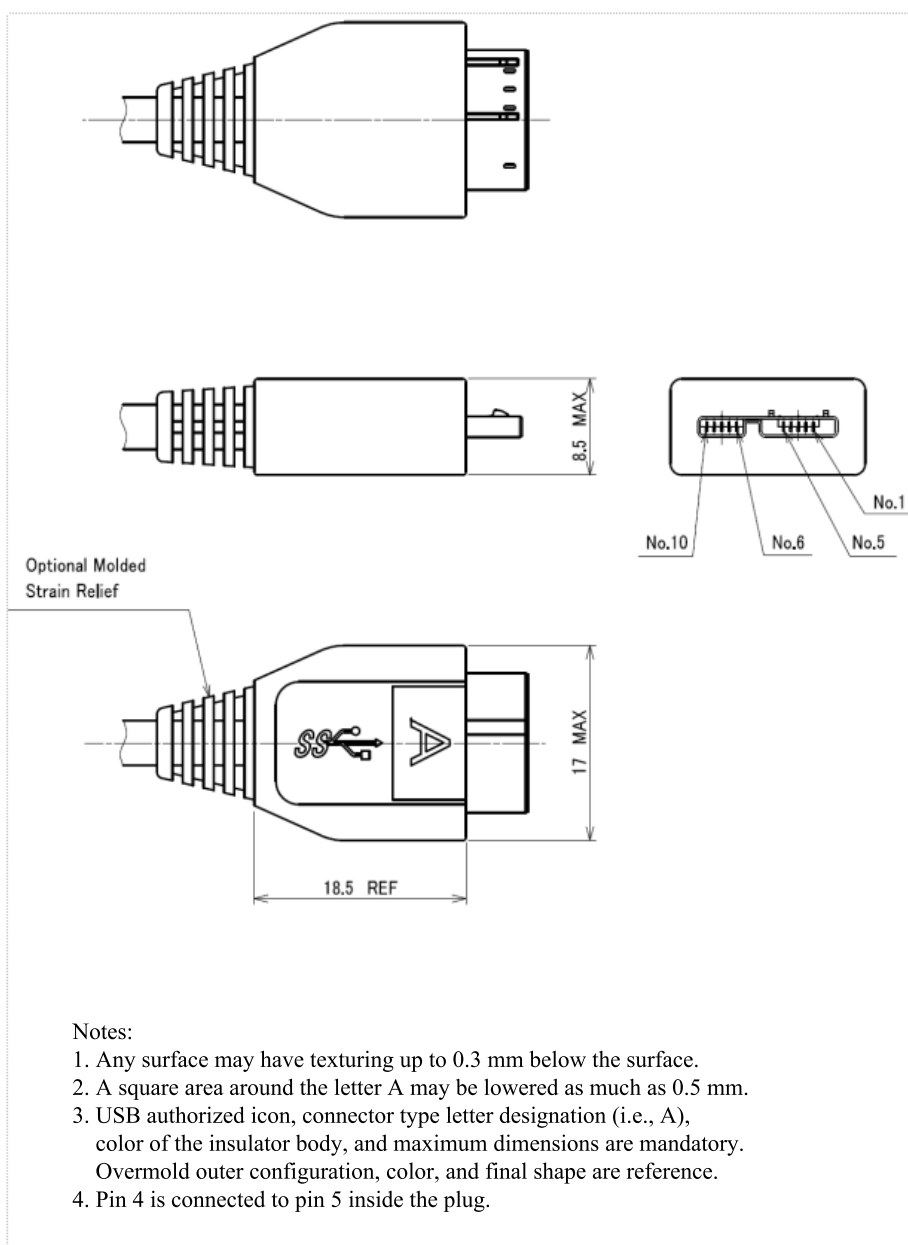
Table 5-10 shows the wire connections for the USB 3.1 Standard-A to USB 3.1 Micro-B cable assembly. Note that the ID pin in the USB 3.1 Micro-B plug shall not be connected, but left in the open condition.

Table 5-10 – USB 3.1 Standard-A to USB 3.1 Micro-B Cable Assembly Wiring

USB 3.1 Standard-A Plug		Wire		USB 3.1 Micro-B Plug	
Pin Number	Signal Name	Wire Number	Signal Name	Pin Number	Signal Name
1	VBUS	1	PWR	1	VBUS
2	D-	2	D-	2	D-
3	D+	3	D+	3	D+
4	GND	4	GND_PWRrt	5	GND
5	StdA_SSRX-	5	P1-	6	MicB_SSTX-
6	StdA_SSRX+	6	P1+	7	MicB_SSTX+
7	GND_DRAIN	7 and 10	P1_Drain P2_Drain	8	GND_DRAIN
8	StdA_SSTX-	8	P2-	9	MicB_SSRX-
9	StdA_SSTX+	9	P2+	10	MicB_SSRX+
				4	ID
Shell	Shield	Braid	Shield	Shell	Shield

5.5.4 USB 3.1 Micro-A to USB 3.1 Micro-B Cable Assembly

Figure 5-18 shows the USB 3.1 Micro-A plug cable overmold dimensions in a USB 3.1 Micro-A to USB 3.1 Micro-B cable assembly. The USB 3.1 Micro-B plug cable overmold dimensions are shown in Figure 5-17. Due to increased wire sizes required for some PD cable implementations, the overmold dimensions for PD cables have larger maximum dimensions than the non-PD cables specified. See the *Universal Serial Bus Power Delivery Specification* for the maximum overmold dimensions of PD cable assemblies.



IEC

Figure 5-18 – USB 3.1 Micro-A Cable Overmold Dimensions

Table 5-11 shows the wire connections for the USB 3.1 Micro-A to USB 3.1 Micro-B cable assembly. The ID pin on a USB 3.1 Micro-A plug shall be connected to the GND pin. The ID pin on a USB 3.1 Micro-B plug shall be a no-connect or connected to ground by a resistance of greater than $R_{b_PLUG_ID}$ (1 M Ω minimum). See the *Universal Serial Bus Power Delivery Specification* for additional details regarding electrical connections to ID pins. An OTG device is required to be able to detect whether a USB 3.1 Micro-A or USB 3.1 Micro-B plug is inserted by determining if the ID pin resistance to ground is less than $R_{a_PLUG_ID}$ (10 Ω maximum) or if the resistance to ground is greater than $R_{b_PLUG_ID}$. Any ID resistance less than $R_{a_PLUG_ID}$ shall be treated as ID = FALSE and any resistance greater than $R_{b_PLUG_ID}$ shall be treated as ID = TRUE.

Table 5-11 – USB 3.1 Micro-A to USB 3.1 Micro-B Cable Assembly Wiring

USB 3.1 Micro-A Plug		Wire		USB 3.1 Micro-B Plug	
Pin Number	Signal Name	Wire Number	Signal Name	Pin Number	Signal Name
1	VBUS	1	PWR	1	VBUS
2	D-	2	D-	2	D-
3	D+	3	D+	3	D+
4	ID (see Note 1)	No Connect		4	ID (see Note 2)
5	GND	4	GND_PWRrt	5	GND
6	MicA_SSTX-	5	P1-	9	MicB_SSRX-
7	MicA_SSTX+	6	P1+	10	MicB_SSRX+
8	GND_DRAIN	7 and 10	P1_Drain P2_Drain	8	GND_DRAIN
9	MicA_SSRX-	8	P2-	6	MicB_SSTX-
10	MicA_SSRX+	9	P2+	7	MicB_SSTX+
Shell	Shield	Braid	Shield	Shell	Shield

NOTES:

1. Connect to the GND.
2. No connect or connect to ground by a resistance greater than 1 MΩ minimum.

5.5.5 USB 3.1 Micro-A to USB 3.1 Standard-B Cable Assembly

A USB 3.1 Micro-A to USB 3.1 Standard-B cable assembly is also allowed. Figure 5-18 and Figure 5-16 show, respectively, the USB 3.1 Micro-A cable overmold and the USB 3.1 Standard-B cable overmold dimensions.

Table 5-12 shows the wire connections for the USB 3.1 Micro-A to USB 3.1 Standard-B cable assembly.

Table 5-12 – USB 3.1 Micro-A to USB 3.1 Standard-B Cable Assembly Wiring

USB 3.1 Micro-A Plug		Wire		USB 3.1 Standard-B Plug	
Pin Number	Signal Name	Wire Number	Signal Name	Pin Number	Signal Name
1	VBUS	1	PWR	1	VBUS
2	D-	2	D-	2	D-
3	D+	3	D+	3	D+
4	ID (see Note 1)	No Connect			
5	GND	4	GND_PWRrt	4	GND
6	MicA_SSTX-	5	P1-	8	StdB_SSRX-
7	MicA_SSTX+	6	P1+	9	StdB_SSRX+
8	GND_DRAIN	7 and 10	P1_Drain P2_Drain	7	GND_DRAIN
9	MicA_SSRX-	8	P2-	5	StdB_SSTX-
10	MicA_SSRX+	9	P2+	6	StdB_SSTX+
Shell	Shield	Braid	Shield	Shell	Shield

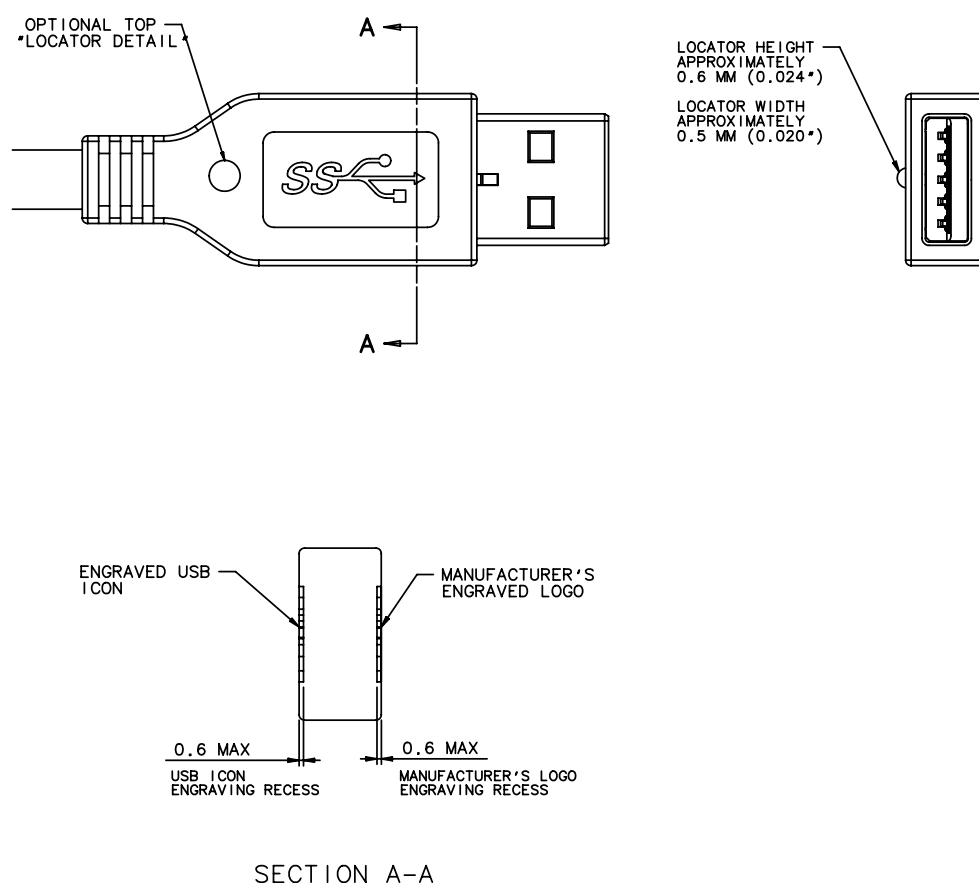
NOTES:

1. Connect to the GND

5.5.6 USB 3.1 Icon Location

USB 3.1 cable assemblies compliant with the *USB 3.1 Connectors and Cable Assemblies Compliance Specification* shall display the appropriate USB 3.1 Icon. A dimensioned drawing and allowable usage of the icon are supplied with the license from the USB-IF.

The USB 3.1 Icon is embossed in a recessed area on the side of the USB 3.1 plug. This provides easy user recognition and facilitates alignment during the mating process. The USB Icon and Manufacturer's logo should not project beyond the overmold surface. The USB 3.1 compliant cable assembly is required to have the USB 3.1 Icons on the plugs at both ends, while the manufacturer's logo is recommended. USB 3.1 receptacles should be orientated to allow the icon on the plug to be visible during the mating process. Figure 5-19 shows a typical plug orientation.



IEC

Figure 5-19 – Typical Plug Orientation

5.5.7 Cable Assembly Length

This specification does not specify cable assembly lengths. A USB 3.1 cable assembly may be of any length as long as it meets all the requirements defined in this specification. The cable assembly voltage drop budget defined in Section 11.4.2 and the cable assembly loss budget defined in Section 5.6.1.3.2, limit the cable assembly length.

5.6 Electrical Requirements

This section covers the electrical requirements for USB 3.1 raw cables, mated connectors, and mated cable assemblies. USB 3.1 signals, known as Enhanced SuperSpeed are

governed by this specification. The USB 2.0 signals are governed by the USB 2.0 specification, unless otherwise specified.

Compliance to the USB 3.1 specification is established through normative requirements of mated connectors and mated cable assemblies.

Enhanced SuperSpeed requirements supporting Gen 2 speed are specified in the frequency domain. Components and assemblies meeting Enhanced SuperSpeed Gen 2 speed electrical requirements do not require separate qualification testing for Gen 1 speed compliance.

DC requirements, such as contact resistance and current carrying capability, are also specified in this section.

Any informative specification for cable and connector products is for the purpose of design guidelines and manufacturing control.

In conjunction with performance requirements, the required test method is referenced for the parameter stated. A list of the industry standards for DC requirements is found in the Section 5.6.2. Additional supporting test procedures are found in the *USB 3.1 Connectors and Cable Assemblies Compliance Document*.

The requirements in the section apply to all USB 3.1 connectors and/or cable assemblies unless specified otherwise.

5.6.1 Enhanced SuperSpeed Electrical Requirements

The following sections outline the requirements for SuperSpeed signals.

5.6.1.1 Raw Cable

Informative raw cable electrical performance targets are provided to help cable assembly manufacturers manage raw cable suppliers. These targets are not part of the USB 3.1 compliance requirements. The mandatory requirements are that the mated cable assembly performance specified in Section 5.6.1.3 and other tests specified in the *USB 3.1 Connectors and Cable Assemblies Compliance Document*.

5.6.1.1.1 Characteristic Impedance

The differential characteristic impedance for the SDP pairs is recommended to be $90 \Omega \pm 5 \Omega$. The single-ended characteristic impedance of coaxial Enhanced SuperSpeed signal wires is recommended to be $45 \Omega \pm 3 \Omega$. It should be measured with a TDR in a differential mode using a 200 ps (10 %–90 %) rise time.

5.6.1.1.2 Intra-Pair Skew

The intra-pair skew for the SDP pairs is recommended to be less than 15 ps/m. It should be measured with a Time Domain Transmission (TDT) in a differential mode using a 200 ps (10 %–90 %) rise time with a crossing at 50 % of the input voltage.

5.6.1.1.3 Differential Insertion Loss

Cable loss depends on wire gauges, plating and dielectric materials. Table 5-13 and Table 5-14 show examples of average differential insertion loss for the SDP pairs for Enhanced SuperSpeed Gen 2 speed. To meet the cable assembly differential insertion loss target, support of the Gen 2 speed requires better performance from the raw cable than required for support of the Gen 1 speed.

Table 5-13 – SDP Differential Insertion Loss Examples for Gen 2 speed

Frequency	34AWG	32AWG	30AWG	28AWG
0,625 GHz	-1,8 dB/m	-1,4 dB/m	-1,2 dB/m	-1,0 dB/m
1,25 GHz	-2,5 dB/m	-2,0 dB/m	-1,7 dB/m	-1,4 dB/m
2,50 GHz	-3,7 dB/m	-2,9 dB/m	-2,5 dB/m	-2,1 dB/m
5,00 GHz	-5,5 dB/m	-4,5 dB/m	-3,9 dB/m	-3,1 dB/m
7,50 GHz	-7,0 dB/m	-5,9 dB/m	-5,0 dB/m	-4,1 dB/m

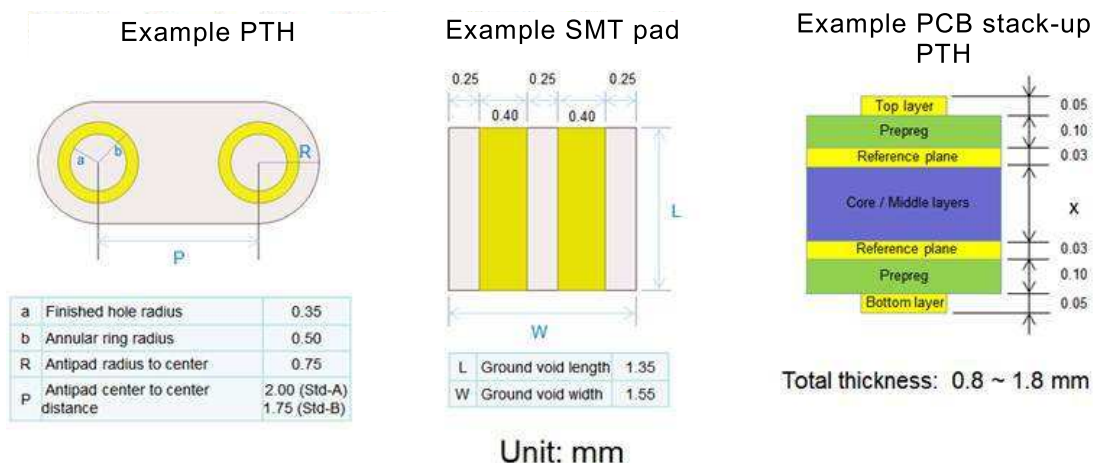
Table 5-14 – SDP Differential Insertion Loss Examples for Gen 2 speed with Coaxial Construction

Frequency	34AWG	32AWG	30AWG	28AWG
0,625 GHz	-1,6 dB/m	-1,3 dB/m	-1,1 dB/m	-1,0 dB/m
1,25 GHz	-2,3 dB/m	-1,8 dB/m	-1,5 dB/m	-1,3 dB/m
2,50 GHz	-3,5 dB/m	-2,7 dB/m	-2,3 dB/m	-1,9 dB/m
5,00 GHz	-5,3 dB/m	-4,2 dB/m	-3,5 dB/m	-3,1 dB/m
7,50 GHz	-7,2 dB/m	-5,5 dB/m	-4,9 dB/m	-4,2 dB/m

5.6.1.2 Mated Connector Impedance

SuperSpeed signal routing on the PCB should minimize the stub length and minimize impedance discontinuities in the signal path. It is recommended that the SuperSpeed signals be routed on the opposite side of the PCB from the side the lead is inserted for through-hole implementations. It is recommended that the SuperSpeed signals be routed on the same side of the PCB as the solder pads for SMT implementations.

For Enhanced SuperSpeed Gen 2 speed applications, electrical optimization is required to achieve the best performance. The PCB stack up, lead geometry, and solder pad geometry should be modeled in three dimensions. Example ground voids under pads shown in Figure 5-20 are based on pad geometry, mounting type, and PCB stack up.



IEC

NOTE See Section 5.6.1.3 for recommended electrical requirements in determining void dimensions

Figure 5-20 – Recommended Ground Void Dimension for USB Standard-A Receptacle

5.6.1.2.1 Mated Connector Impedance for Gen 2 Speed

The recommended mated connector impedance is needed to maintain signal integrity. The differential impedance of a mated connector should be $90\ \Omega \pm 10\ \Omega$, as seen from a 40 ps (20 %-80 %) risetime of a differential TDR. The impedance profile of a mated connector should fall within the limits shown in Figure 5-21. The impedance profile of the mated connector is defined from the receptacle footprints through the plug cable termination area. In a case where the plug is directly attached to a device PCB, the mated connector impedance profile includes the path from the receptacle footprints to the plug footprints.

The normative cable assembly requirements are specified in Section 5.6.1.3.2.

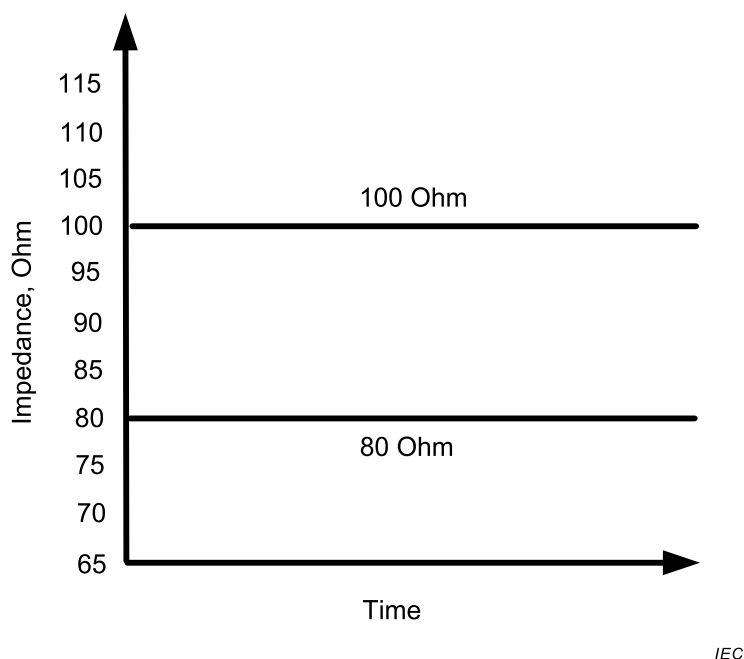


Figure 5-21 – Impedance Limits of a Mated Connector for Gen 2 Speed

5.6.1.3 Mated Cable Assemblies for Gen 2 speed

The requirements for mated cable assemblies for Gen 2 speed are divided into informative and normative requirements. The informative requirements are provided as design targets and normative requirements are pass/fail criteria for mated cable assembly compliance testing.

5.6.1.3.1 Design Targets

The design targets are summarized in Table 5-15. Cable assemblies that meet the target performance should pass the cable assembly compliance tests, but it is not guaranteed. Compliance is determined with the normative requirements.

Table 5-15 – Design Targets

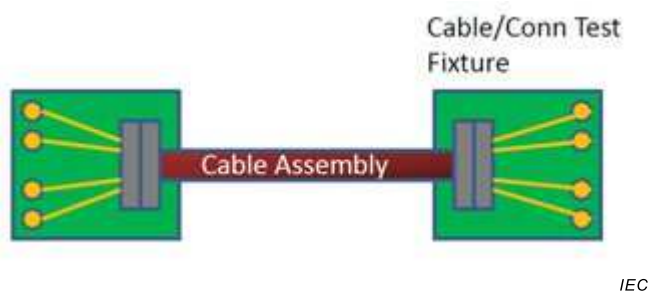
Components	Items	Design Targets
Raw Cable	Differential impedance	Section 5.6.1.1.1.
	Intra-pair skew	Section 5.6.1.1.2.
	Differential insertion loss	Section 5.6.1.1.3.
Mated Connector	Mated connector impedance	Section 5.6.1.2.1
Mated Cable Assembly	Differential insertion loss	≥ -6 dB from DC to 5 GHz and no strong resonance within DC to 5 GHz.
	Differential near-end crosstalk between SuperSpeed Gen 2 signal pairs	≤ -34 dB to 5 GHz.
	Differential near-end and far-end crosstalk between D+/D- pair and SuperSpeed Gen2 pairs	≤ -30 dB to 5 GHz.

5.6.1.3.2 Normative Requirements

The normative requirements manage the impact of insertion loss, multi-reflection, and crosstalk for the end-to-end link performance and mode conversion for EMI/RFI purposes.

5.6.1.3.2.1 Test Fixtures

The mated cable assembly electrical requirements for Gen 2 speed are mostly specified in the frequency domain. To accurately measure the frequency response of the Gen 2 speed mated cable assembly, the fixture design and its performance characteristics are critical items. Common fixtures defined by the USB-IF or equivalent shall be used. Refer to *USB 3.1 Connectors and Cable Assemblies Compliance Document* for detailed descriptions of test fixtures. Figure 5-22 illustrates an example of a cable assembly mounted on a test fixture.



IEC

Figure 5-22 – Illustration of Cable Assembly Mounted on Test Fixture

5.6.1.3.2.2 Reference Hosts and Devices

To avoid channel margin loss associated with assigning interconnect budgets to the host, cable assembly, and device, the compliance of a Gen 2 speed cable assembly is established with reference hosts and reference devices to emulate the end-to-end condition as illustrated in Figure 5-23.



IEC

Figure 5-23 – Illustration of Cable Assembly with Reference Host and Device

The USB 3.1 specification defines the standard reference hosts and devices in the form of S-parameter files, which are available for download from the USB website. The measured S-parameters of the Gen 2 speed cable assembly (with fixture effects removed) are cascaded with the S-parameters of the reference hosts and reference devices, resulting in frequency responses for total channels. The pass/fail criteria of the Gen 2 speed cable assembly are based on the channel frequency responses, as described in the following subsections.

5.6.1.3.2.3 Channel Metrics

There are three signal integrity impairments that impact the end-to-end link performance: attenuation, reflection, and crosstalk. Three parameters are used as the channel metrics to represent these three impairments: insertion loss fit at Nyquist frequency ($IL_{fitatNq}$), integrated multi-reflection (IMR), and integrated crosstalk (IXT).

To obtain the channel insertion loss fit at Nyquist frequency (5 GHz for SuperSpeed Gen 2), the measured (cascaded) differential insertion loss, $IL(f)$, is fitted with a smoothing function:

$$IL_{fit}(f) = a + b\sqrt{f} + cf + d\sqrt{f^3}, \quad (5-1)$$

where f is frequency and a , b , c , and d are fitting coefficients. Figure 5-24 shows an example of $IL(f)$ and $IL_{fit}(f)$ plotted together; $IL_{fitatNq} = -20.6$ dB, measured at 5 GHz along $IL_{fit}(f)$.

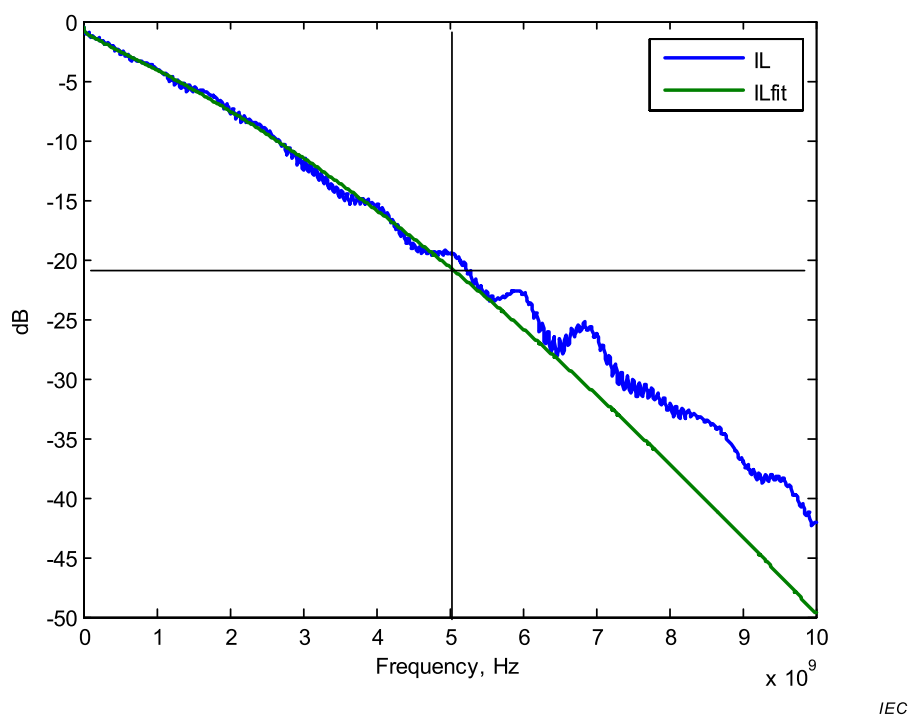


Figure 5-24 – Illustration of Insertion Loss Fit at Nyquist Frequency

The difference between $IL(f)$ and $IL_{fit}(f)$ is defined as the insertion loss deviation, ILD:

$$ILD(f) = IL(f) - IL_{fit}(f). \quad (5-2)$$

It measures the ripple of the insertion loss, or the multi-reflection. Figure 5-25 shows an example of insertion loss deviation plotted against frequency.

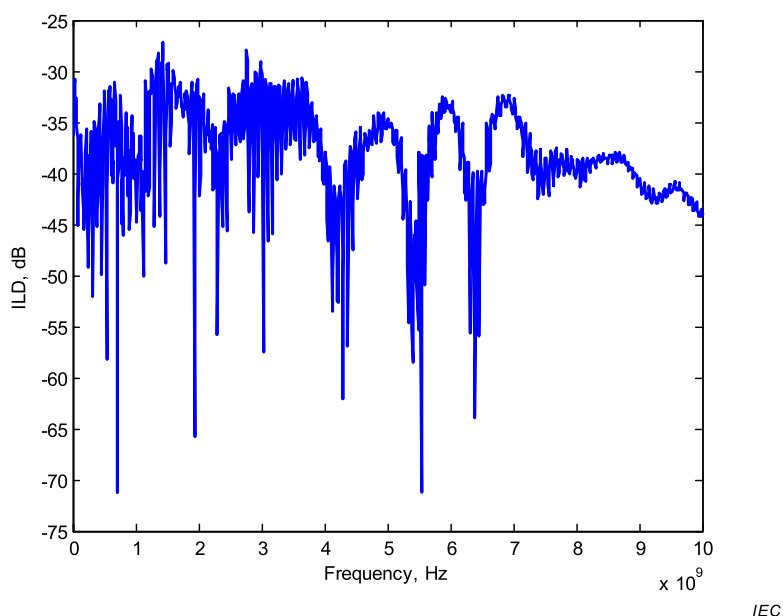


Figure 5-25 – Example of Insertion Loss Deviation

The integrated multi-reflection, IMR, is calculated with the equation below:

$$IMR = \sqrt{\int_0^{f_{\max}} |ILD(f)|^2 |V_{in}(f)|^2 df / f_{Nq}} * 1000 \text{ (in mV)}, \quad (5-3)$$

where f_{Nq} is the Nyquist frequency (5 GHz), $f_{\max}$ is chosen as the 2 times the Nyquist frequency (10 GHz), and $V_{in}(f)$ is the input trapezoidal pulse spectrum, defined as:

$$|V_{in}(\omega)| = \left| \frac{\sin\left(\frac{\omega T_r}{2}\right)}{\frac{\omega T_r}{2}} \cdot \frac{\sin\left(\frac{\omega T_b}{2}\right)}{\frac{\omega T_b}{2}} \right|$$

$T_b = \text{Unit Interval} = 100 \text{ ps}$
 $T_r = \text{Rise time (0-100\%)} = 0.2 T_b$
 $\omega = 2\pi f$

The integrated crosstalk, IXT, is defined as:

$$IXT = \sqrt{\int_0^{f_{\max}} |NEXT(f)|^2 |V_{in}(f)|^2 df / f_{Nq}} * 1000 \text{ (in mV)}, \quad (5-4)$$

where $NEXT(f)$ is the near-end crosstalk between the SuperSpeed Gen 2 signal pairs. The contribution of USB 2 D+/D- pair to SuperSpeed signal pairs is relatively small and is not included in IXT for simplicity.

More detailed discussion of ILfitatNq, IMR and IXT is given in a USB-IF whitepaper *Establishing USB SuperSpeed Gen 2 Channel and Cable Assembly High Speed Compliance Specification*. The USB-IF also provides a standard tool to convert measured cable assembly S-parameters into ILfitatNq, IMR and IXT.

5.6.1.3.2.4 Pass/Fail Criteria

SuperSpeed Gen 2 channel performance is based on ILfitatNq, IMR, and IXT. In general, a channel with more loss, more reflection, and more crosstalk has less margin. Channel margin is measured with BER (bit error ratio) eye height (eH) and eye width (eW) at BER=10⁻¹². The correlation between eH and eW and the channel metrics ILfitatNq, IMR, and IXT is established following the methodology described in the USB-IF whitepaper *Establishing USB SuperSpeed Gen 2 Channel and Cable Assembly High Speed Compliance Specification*. For each channel with a metrics ILfitatNq, IMR, and IXT, eH and eW is calculated:

$$\begin{aligned} eH &= f_H(ILfitatNq, IMR, IXT) \\ eW &= f_W(ILfitatNq, IMR, IXT) \end{aligned} \quad (5-5)$$

Note that the effect of Tx and Rx equalization, jitter, and sampling noise is included in the eye height and eye width calculations. The pass/fail criteria are then expressed as

$$\begin{aligned} eH &= f_H(ILfitatNq, IMR, IXT) > 0 \\ eW &= f_W(ILfitatNq, IMR, IXT) > 0 \end{aligned} \quad (5-6)$$

and

$$\begin{aligned} ILfitatNq &\geq -22 \text{ dB} \\ IMR &\leq 60 \text{ mV} \\ IXT &\leq 25 \text{ mV} \end{aligned} \quad (5-7)$$

Equation (5-6) is an open-eye (at BER=10⁻¹²) requirement, while Equation (5-7) constrains the maximum channel loss, multi-reflection, and crosstalk. A SuperSpeed Gen 2 cable assembly is considered pass if both Equations (5-6) and (5-7) are satisfied. Figure 5-26 shows pass and fail examples.

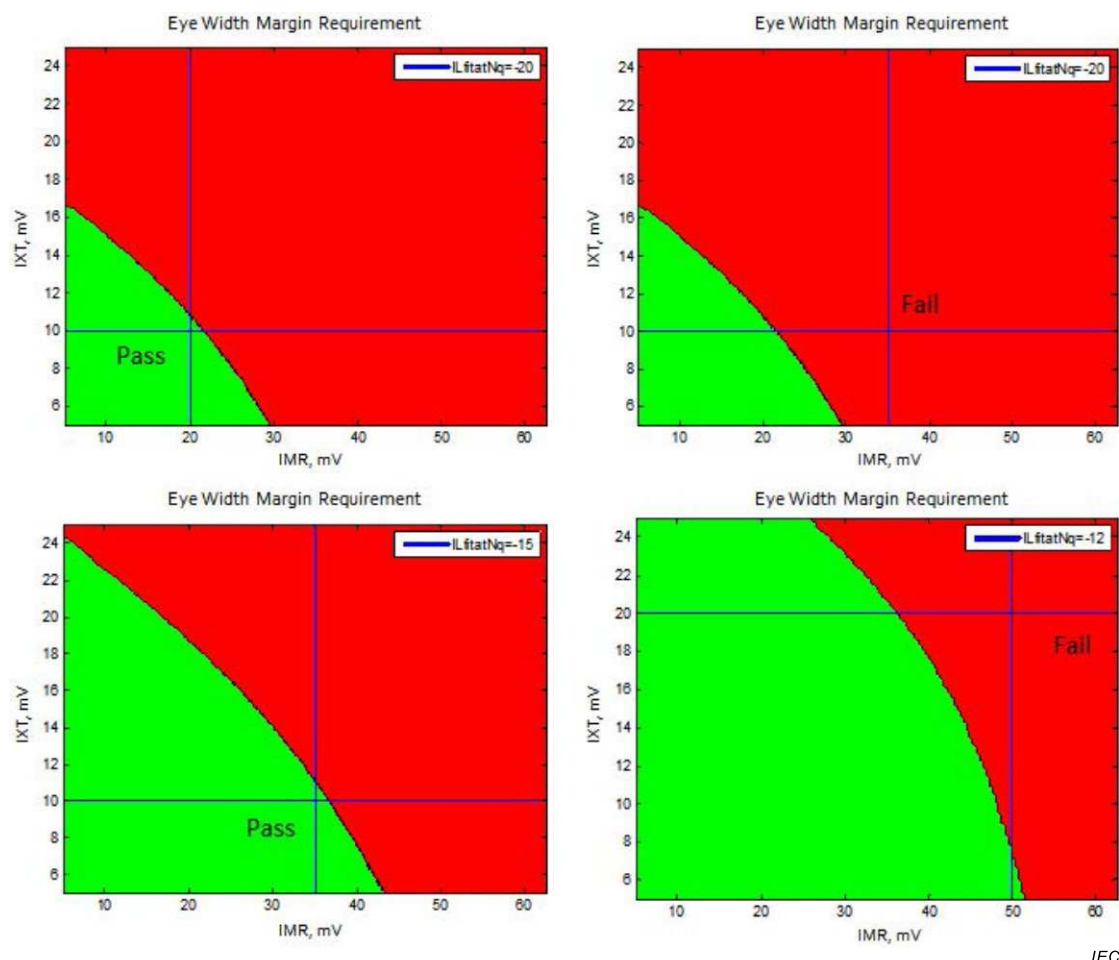


Figure 5-26 – Pass/Fail Examples

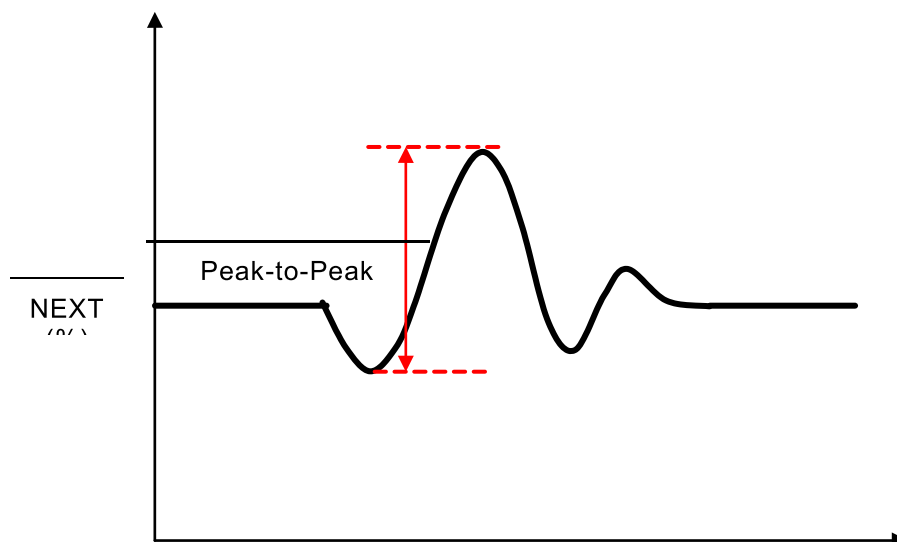
The x-axis is IMR in mV and the y-axis is IXT in mV. The “green” part represents the passing region with open eyes, while the “red” area the failing region with closed eyes. The passing area increases as $|IL_{fitatNq}|$ decreases. This pass/fail criteria allows tradeoffs among $IL_{fitatNq}$, IMR, and IXT. For example, a cable assembly may have more loss if IMR and/or IXT is smaller.

USB-IF provides a standard tool to calculate eH and eW based on the input cable assembly S-parameters. This tool, or an equivalent, is integrated into the USB SuperSpeed Gen 2 compliance test suite in the USB CabCon compliance program.

5.6.1.3.2.5 Differential Crosstalk between D+/D- and SuperSpeed Gen 2 Signal Pairs (EIA-360-90)

The differential near-end crosstalk (DDNEXT) and far-end crosstalk (DDFEXT) between the D+/D- pair and the SuperSpeed Gen 2 signal pairs shall be measured in time domain with a rise time of 500 ps (10-90 %) entering the connector under test. The mated cable assembly meets the DDNEXT/DDFEXT requirement if its peak-to-peak value does not exceed the limits below (see Figure 5-27 for illustration of the peak-to-peak crosstalk):

- USB 3.1 Standard-A connector: 2 %
- USB 3.1 Standard-B connector: 2 %
- USB 3.1 Micro connector family: 2 %

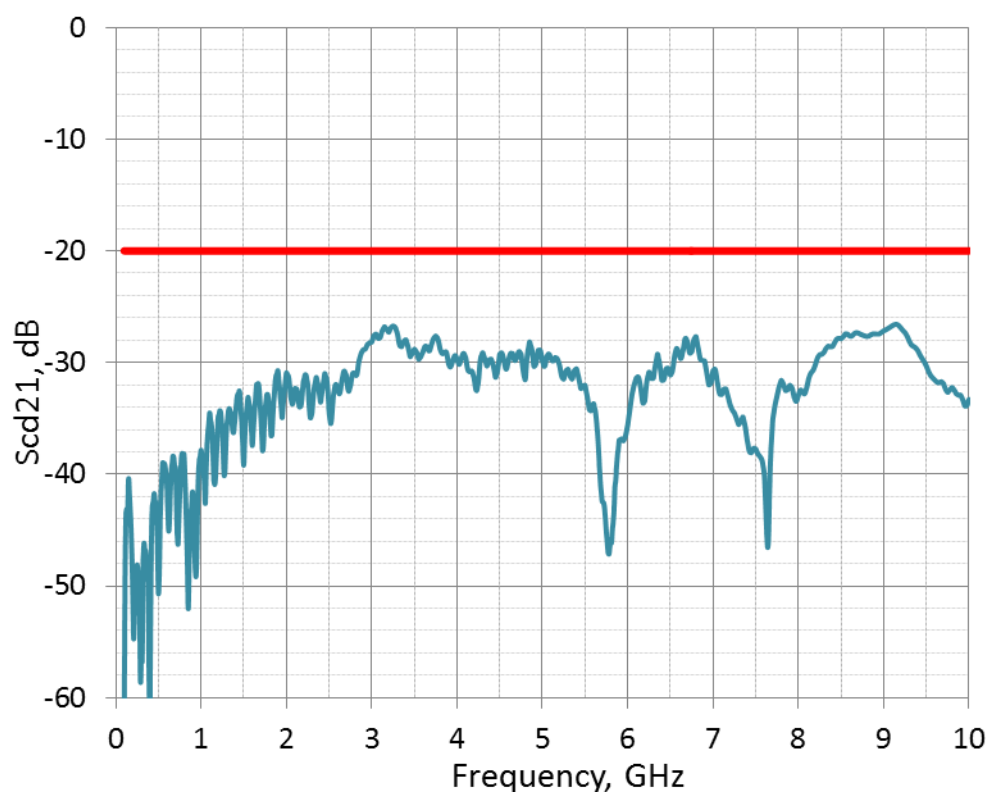


IEC

Figure 5-27 – Illustration of Peak-to-Peak Crosstalk

5.6.1.3.2.6 Differential to Common Mode Conversion

The differential to common mode conversion (SCD21 or SCD12) does not require embedding the reference host and the reference device with the mated cable assembly; it is for the mated cable assembly only. A mated cable assembly passes the SCD12 requirement if its SCD12 is less than or equal to -20 dB across the frequency range of 100 MHz to 10 GHz, as shown in Figure 5-28.



IEC

Figure 5-28 – Differential-to-Common-Mode Conversion Requirement for Gen 2

5.6.1.3.2.7 Cable Shielding Effectiveness

The cable assembly shielding effectiveness (SE) test measures the radio frequency interference (RFI) level from the cable assembly. To perform the measurement, the cable assembly shall be installed in the cable SE test fixture as shown in Figure 5-29. The coupling factors from the cable to the fixture are characterized. The Cable SE test fixtures and specification values are under development and are to be included in future updates to this specification.

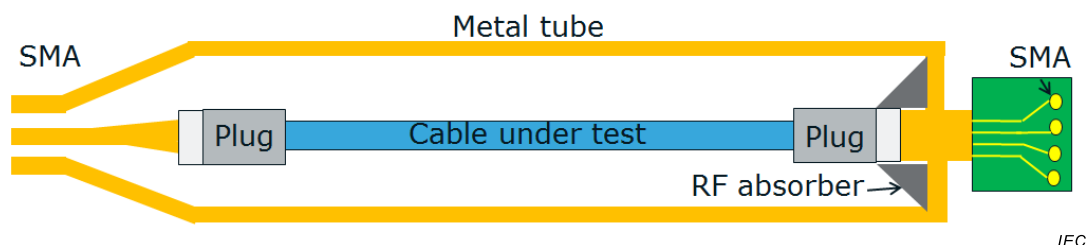


Figure 5-29 – Set Up For Cable SE Measurement (subject to change)

5.6.2 DC Electrical Requirements

5.6.2.1 Low Level Contact Resistance (EIA 364-23B)

The following requirement applies to both the power and signal contacts:

- 30 mΩ (Max) initial for Vbus and GND contacts.
- 50 mΩ (Max) initial for all other contacts.
- Maximum change (delta) of +10 mΩ after environmental stresses.
- Measure at 20 mV (Max) open circuit at 100 mA.
- Refer to Section 5.7 for environmental requirements and test sequences.

5.6.2.2 Dielectric Strength (EIA 364-20)

No breakdown shall occur when 100 Volts AC (RMS) is applied between adjacent contacts of unmated and mated connectors.

5.6.2.3 Insulation Resistance (EIA 364-21)

A minimum of 100 MΩ insulation resistance is required between adjacent contacts of unmated and mated connectors.

5.6.2.4 Contact Current Rating (EIA 364-70, Method 2)

A current of 1.8 A shall be applied to VBUS pin and its corresponding GND pin (pin 1 and pin 4 of the USB 3.1 Standard-A and Standard-B connectors; pin 1 and pin 5 of the USB 3.1 Micro connector family). Additionally, a minimum current of 0,25 A shall be applied to all the other contacts. When the current is applied to the contacts, the delta temperature shall not exceed +30 °C at any point on the USB 3.1 connectors under test, when measured at an ambient temperature of 25 °C.

5.7 Mechanical and Environmental Requirements

The requirements in this section apply to all USB 3.1 connectors and/or cable assemblies unless specified otherwise.

5.7.1 Mechanical Requirements

5.7.1.1 Insertion Force (EIA 364-13)

The connector insertion force shall not exceed 35 N at a maximum rate of 12,5 mm (0,492") per minute.

It is recommended to use a non-silicon based lubricant on the latching mechanism to reduce wear. The effects of lubricants should be restricted to insertion and extraction characteristics.

5.7.1.2 Extraction Force Requirements (EIA 364-13)

5.7.1.2.1 Extraction Force (EIA 364-13, USB3.1 Standard Connector)

The connector extraction force shall not be less than 10 N initial and 8 N after the specified insertion/extraction or durability cycles (at a maximum rate of 12,5 mm (0,492") per minute).

No burrs or sharp edges are allowed on top of locking latches (hook surfaces that will rub against the receptacle shield).

It is recommended to use a non-silicon based lubricant on the latching mechanism to reduce wear. The effects of lubricants should be restricted to insertion and extraction characteristics.

5.7.1.2.2 Extraction Force (EIA 364-13, USB 3.1 Micro Connector Family Only)

The connector extraction force shall not be less than 10 N or more than 25 N initial and less than 8 N and more than 25 N after the specified insertion/extraction or durability cycles (at a maximum rate of 12,5 mm (0,492") per minute).

No burrs or sharp edges are allowed on top of locking latches (hook surfaces that will rub against the receptacle shield).

It is recommended to use a non-silicon based lubricant on the latching mechanism to reduce wear. The effects of lubricants should be restricted to insertion and extraction characteristics.

5.7.1.3 Durability or Insertion/Extraction Cycles (EIA 364-09)

The durability ratings listed in Table 5-16 are specified for the USB 3.1 connectors.

Table 5-16 – Durability Ratings

Connector	Standard Durability Class	High Durability Class
USB 3.1 Standard-A connector	1 500 cycles min	5 000 cycles min
USB 3.1 Standard-B connector	1 500 cycles min	5 000 cycles min
USB 3.1 Micro connector family	10 000 cycles min	

The durability test shall be done at a maximum rate of 200 cycles per hour and no physical damage to any part of the connector or cable assembly shall occur.

5.7.1.4 Cable Flexing (EIA 364-41, Condition I)

No physical damage or discontinuity over 1 ms during flexing shall occur to the cable assembly with Dimension X = 3,7 times the cable diameter and 100 cycles in each of two planes.

5.7.1.5 Cable Pull-Out (EIA 364-38, Condition A)

No physical damage to the cable assembly shall occur when it is subjected to a 40 N axial load for a minimum of 1 minute while clamping one end of the cable plug.

5.7.1.6 Peel Strength (USB 3.1 Micro Connector Family Only)

No visible physical damage shall be noticed to a soldered receptacle when it is pulled up from the PCB in the vertical direction with a minimum force of 150 N.

5.7.1.7 4-Axes Continuity Test (USB 3.1 Micro Connector Family Only)

The USB 3.1 Micro connector family shall be tested for continuity under stress using the test configurations shown below. Plugs shall be supplied in a cable assembly with a representative overmold. A USB 3.1 Micro-B or -AB receptacle shall be mounted on a 2-layer printed circuit board (PCB) between 0,8 and 1,0 mm thickness. The PCB shall be clamped on either side of the receptacle no further than 5 mm away from the solder tails. The PCB shall initially be placed in a horizontal plane, and an 8-N tensile force shall be applied to the cable in a downward direction, perpendicular to the axis of insertion, for a period of at least 10 seconds.

The continuity across each contact shall be measured throughout the application of the tensile force. The PCB shall then be rotated 90 degrees such that the cable is still inserted horizontally and the 8 N tensile force will be applied again in the downward direction and continuity measured as before. This test is repeated for 180-degree and 270-degree rotations. Passing parts shall not exhibit any discontinuities greater than 1 μ s duration in any of the four orientations.

One method for measuring the continuity through the contacts is to short all the wires at the end of the cable pigtail and apply a voltage through a pull-up to each of VBUS, D+, D-, ID, and the SuperSpeed pins, with the GND pins connected to ground.

When testing a USB 3.1 Micro-A plug, all the sense resistors shall stay pulled down for the length of the test. When testing a USB 3.1 Micro-B plug, the ID pin shall stay high and the other pins shall remain low for the duration of the test. Alternate methods are allowed to verify continuity through all pins.

The 4 axes continuity tests shall be done with a USB 3.1 Micro-B/-A plug in a USB 3.1 Micro-B/-AB receptacle and with a USB 2.0 Micro-B/-A plug in a USB 3.1 Micro-B/-AB receptacle, as illustrated in Figure 5-30.

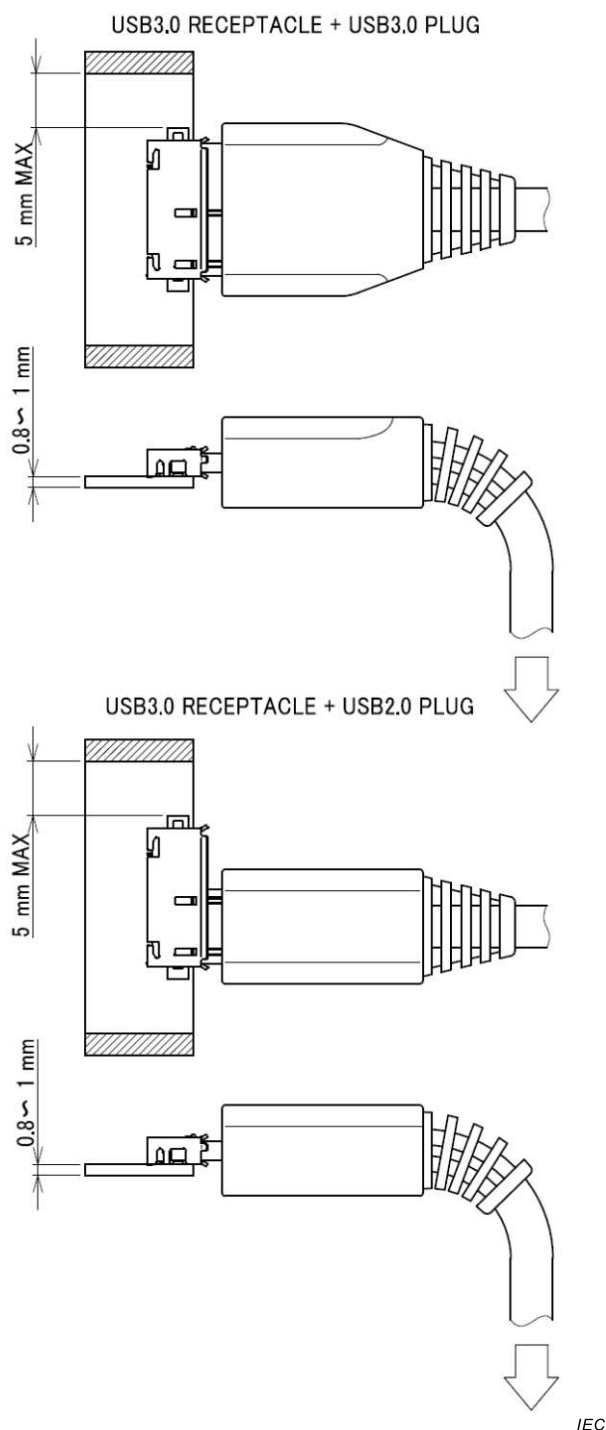


Figure 5-30 – 4-Axes Continuity Test

5.7.1.8 Wrenching Strength (Reference, USB 3.1 Micro Connector Family Only)

The wrenching strength test shall be performed using virgin parts. Perpendicular forces (F_p) are applied to a plug when inserted at a distance (L) of 15 mm from the edge of the receptacle. Testing conditions and method shall be agreed to by all parties. These forces shall be applied in all four directions (i.e., left, right, up, and down). Compliant connectors shall meet the following force thresholds:

- No plug or receptacle damage shall occur when a force of 0-25 N is applied.

The plug may be damaged, but only in such a way that the receptacle does not sustain damage when a force of 25-50 N is applied.

5.7.1.9 Lead Co-Planarity

Co-planarity of all SMT leads shall be within a 0.08 mm range.

5.7.1.10 Solderability

Solder shall cover a minimum of 95 % of the surface being immersed, when soldered at a temperature $255\text{ }^{\circ}\text{C} \pm 5\text{ }^{\circ}\text{C}$ for an immersion duration of 5 s.

5.7.1.11 Restriction of Hazardous Substances (RoHS) Compliance

It is recommended that components be RoHS compliant. Environmental Requirements

5.7.2 Environmental Requirements

The connector interface environmental tests shall follow EIA-364-1000.01, Environmental Test Methodology for Assessing the Performance of Electrical Connectors and Sockets Used in Business Office Applications.

Since the connector defined has far more than 0,127 mm wipe length, Test Group 6 in EIA 364-1000.01 is not required. The temperature life test duration and the mixed flowing gas test duration values are derived from EIA 364-1000.01 based on the field temperature per the following.

Table 5-17 – Environmental Test Conditions

Temperature Life test temperature and duration	105 °C for 120 hours
Temperature Life test temperature and duration for preconditioning	105 °C for 72 hours
Mixed flowing gas test duration	7 days

The pass/fail criterion for the low-level contact resistance (LLCR) is as defined in Section 5.6.2.1. The durability ratings are defined in Section 5.7.1.3.

5.7.3 Materials

This specification does not specify materials for connectors and cables. Connector and cable manufactures should select appropriate materials based on performance requirements. Table 5-18 below is provided for reference only.

Note:

Connector and cable manufacturers shall comply with contact plating requirements per the following options:

Option I**Receptacle**

Contact area: (Min) 0.05 µm Au + (Min) 0.75 µm Ni-Pd on top of (Min) 2.0 µm Ni

Plug

Contact area: (Min) 0.05 µm Au + (Min) 0.75 µm Ni-Pd on top of (Min) 2.0 µm Ni

Option II

Receptacle

Contact area: (Min) 0,75 µm Au on top of (Min) 2,0 µm Ni

Plug

Contact area: (Min) 0,75 µm Au on top of (Min) 2,0 µm Ni.

Other material parameters, which connector and cable manufacturers select based on performance parameters, are listed below in Table 5-18 for reference only.

Table 5-18 – Reference Materials¹

Component	Materials	Comments
Cable	Conductor: copper with tin or silver plating	
	SDP Shield: AL foil or AL/mylar foil	
	Coxial shield: copper strand	
	Braid: Tin plated copper or aluminum	
	Jacket: PVC or halogen free substitute material	
Cable Overmold	Thermoset or thermoplastic	
Connector Shell	Copper alloy or stainless steel, depending on durability requirement	
Housing	Thermoplastics capable of withstanding lead-free soldering temperature.	

5.8 Implementation Notes and Design Guides

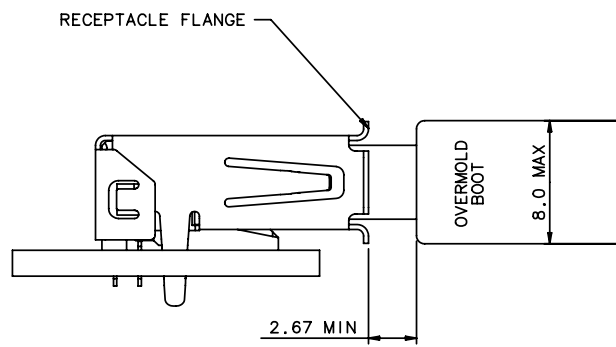
This section discusses a few implementation notes and design guides to help users design and use the USB 3.1 connectors and cables.

5.8.1 Mated Connector Dimensions

Figure 5-31, Figure 5-32, and Figure 5-33 show the mated plugs and receptacles for the USB 3.1 Standard-A, USB 3.1 Standard-B, and USB 3.1 Micro connectors, respectively. The distance between the receptacle front surface and the cable overmold should be observed by system designers to avoid interference between the system enclosure and the cable plug overmold.

Provisions shall be made in connectors and chassis to ground the connector metal shells to the metal chassis to reduce EMI and RFI.

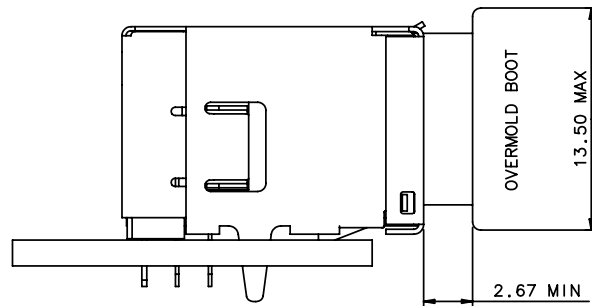
¹ Halogen-free materials should be considered for all plastics.



FULLY MATED PLUG AND RECEPTACLE

IEC

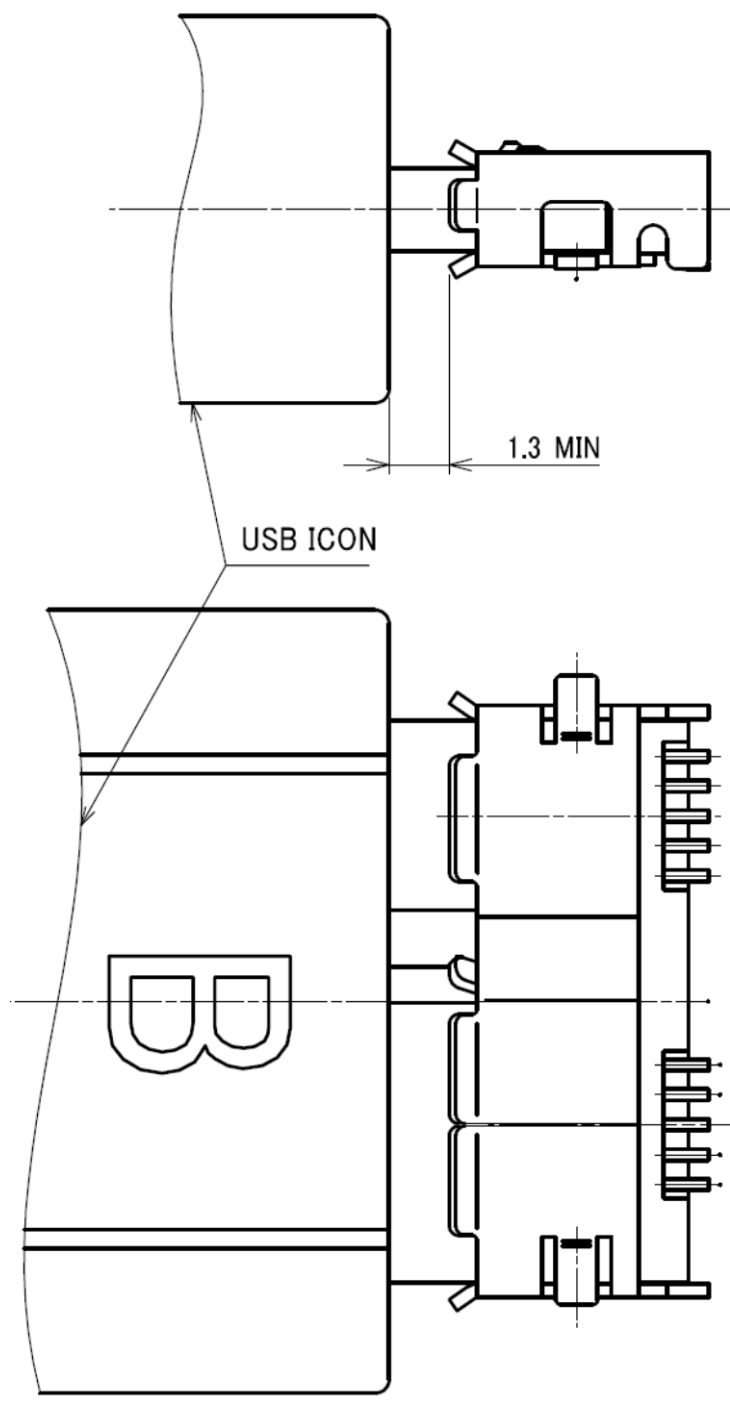
Figure 5-31 – Mated USB 3.1 Standard-A Connector



FULLY MATED PLUG AND RECEPTACLE

IEC

Figure 5-32 – Mated USB 3.1 Standard-B Connector



IEC

Figure 5-33 – Mated USB 3.1 Micro-B Connector

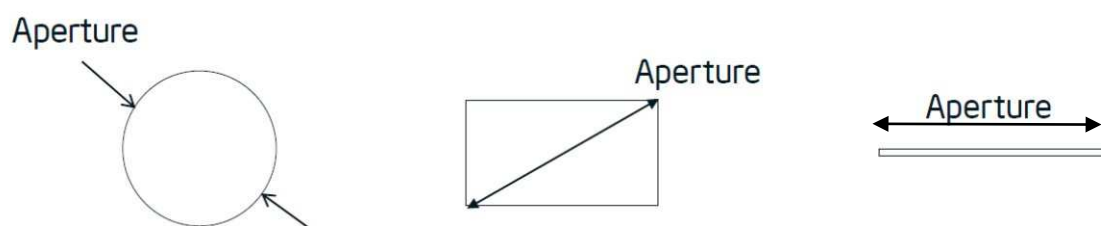
5.8.2 EMI and RFI Management

Systems that include USB 3.1 connectors and cable assemblies need to meet the relevant EMI/EMC regulations.

Connector and cable assembly designers, as well as system implementers should pay attention to receptacle and cable plug shielding to ensure a low-impedance grounding path. The following are guidelines for EMI and RFI management:

- The quality of raw cables should be ensured. The intra-pair skew or the differential to common mode conversion of the SuperSpeed pairs has a significant impact on cable EMI performance and should be controlled within the limits of this specification.

- The cable external braid should be terminated to the cable plug metal shell as close to 360° as possible. Without appropriate shielding termination, even a perfect cable with zero intra-pair skew may not meet EMI requirements.
- The wire termination contributes to the differential-to-common-mode conversion. The breakout distance for the wire termination should be kept as small as possible for EMI, RFI, and signal integrity. If possible, symmetry should be maintained for the two lines within a differential pair.
- The mating interface between the receptacle and cable plug should have a sufficient number of grounding fingers, or springs, to provide a continuous return path from the cable plug to system ground. Friction locks should not compromise ground return connections.
- The receptacle connectors should be designed with a back-shield as part of the receptacle connector metal shell. The back-shield should have connections to adjacent shell surfaces and provide multiple connections for ground termination. The back-shield should be designed with a short return path to the chassis ground.
- The receptacle connectors should be connected to metal chassis or enclosures through grounding fingers, screws, or any other way to mitigate EMI and RFI.
- Plug connectors should have back-shields when possible.
- Outer connector shell surfaces in the mated configuration should have a maximum aperture size of 2 mm. This applies to both mounted connectors (e.g., soldered to a circuit board) and connectors used in cable assemblies. See Figure 5-34.



IEC

Figure 5-34 – Examples of Connector Apertures

5.8.3 Stacked Connectors

Stacked USB connectors are commonly used in PC systems. This specification does not explicitly define the stacked USB 3.1 Standard-A receptacles, but they are allowed. The following are a few points that should be taken into account when designing a stacked USB 3.1 connector:

- A stacked connector introduces additional crosstalk between the top and bottom connectors. Such crosstalk should be minimized when designing a stacked USB 3.1 connector. Enhanced SuperSpeed Gen 2 Standard-A stacked connectors should limit the total crosstalk sum from all sources within the connector to –40 dB (up to the fundamental frequency of 5,0 GHz and limiting the power sum to include aggressors having a magnitude greater than or equal to –50 dB).
- Due to the additional electrical length, the top connector generally does not perform as well as the bottom connector. Connector designers should carefully design the top connector contact geometries and materials to minimize impedance discontinuity.
- Regardless of how many connectors within a stack one may choose to design, the electrical requirements defined in Section 5.6 shall be met.

6 Physical Layer

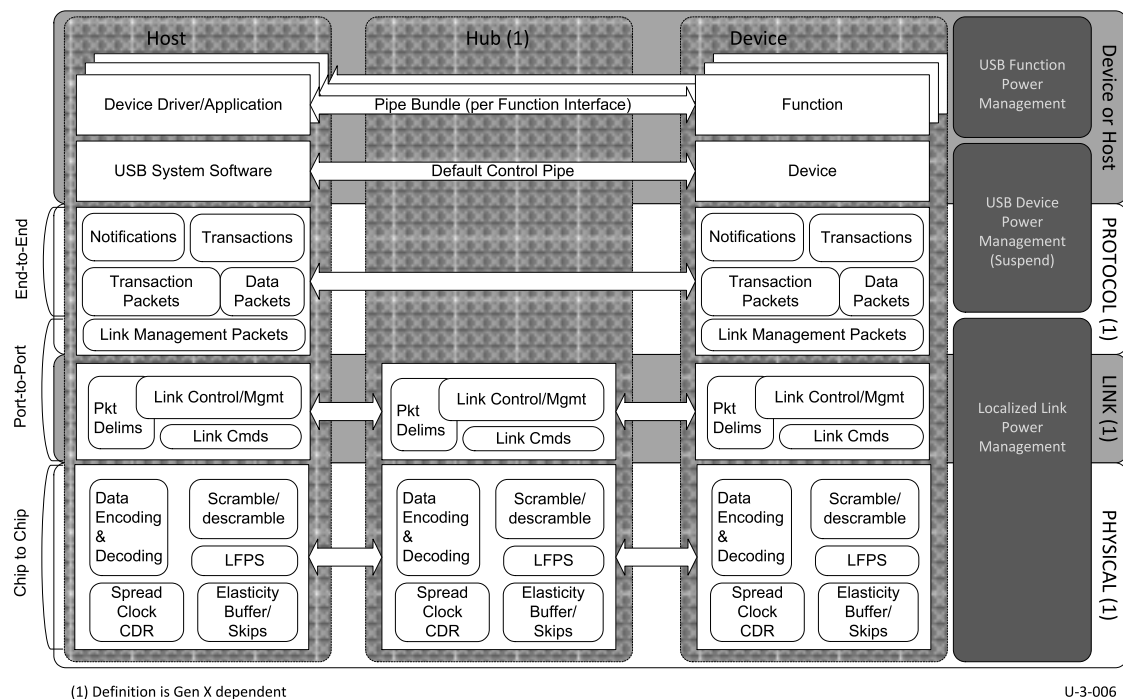


Figure 6-1 – SuperSpeed Physical Layer

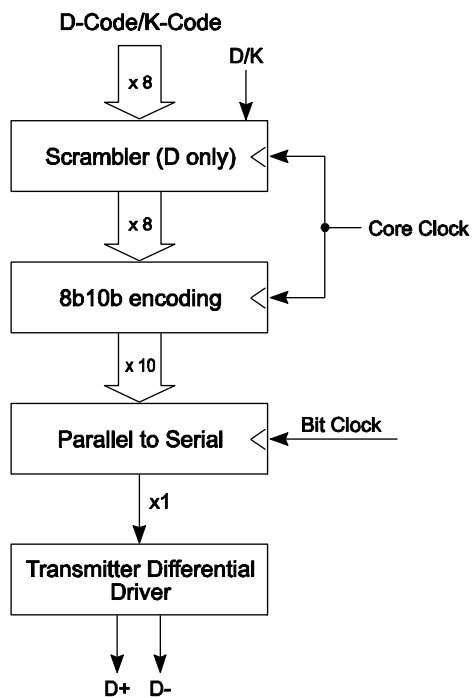
6.1 Physical Layer Overview

The physical layer defines the signaling technology for the SuperSpeed and SuperSpeedPlus busses. This chapter defines the electrical requirements of the SuperSpeed and SuperSpeedPlus physical layers.

This section defines the electrical-layer parameters required for operation of SuperSpeed and SuperSpeedPlus components. Normative specifications are required. Informative specifications may assist product designers and testers in understanding the intended behavior of the SuperSpeed and SuperSpeedPlus busses.

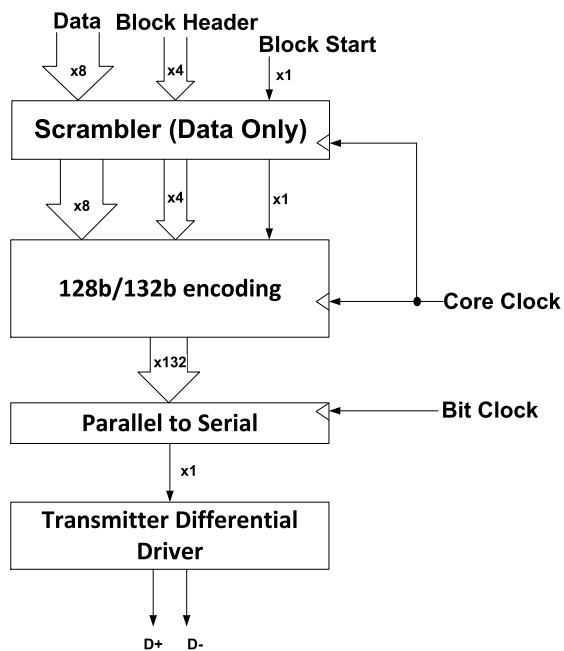
6.2 Physical Layer Functions

The functions of the physical layer are shown in Figure 6-2, Figure 6-3, Figure 6-4, and Figure 6-5.



U-014
IEC

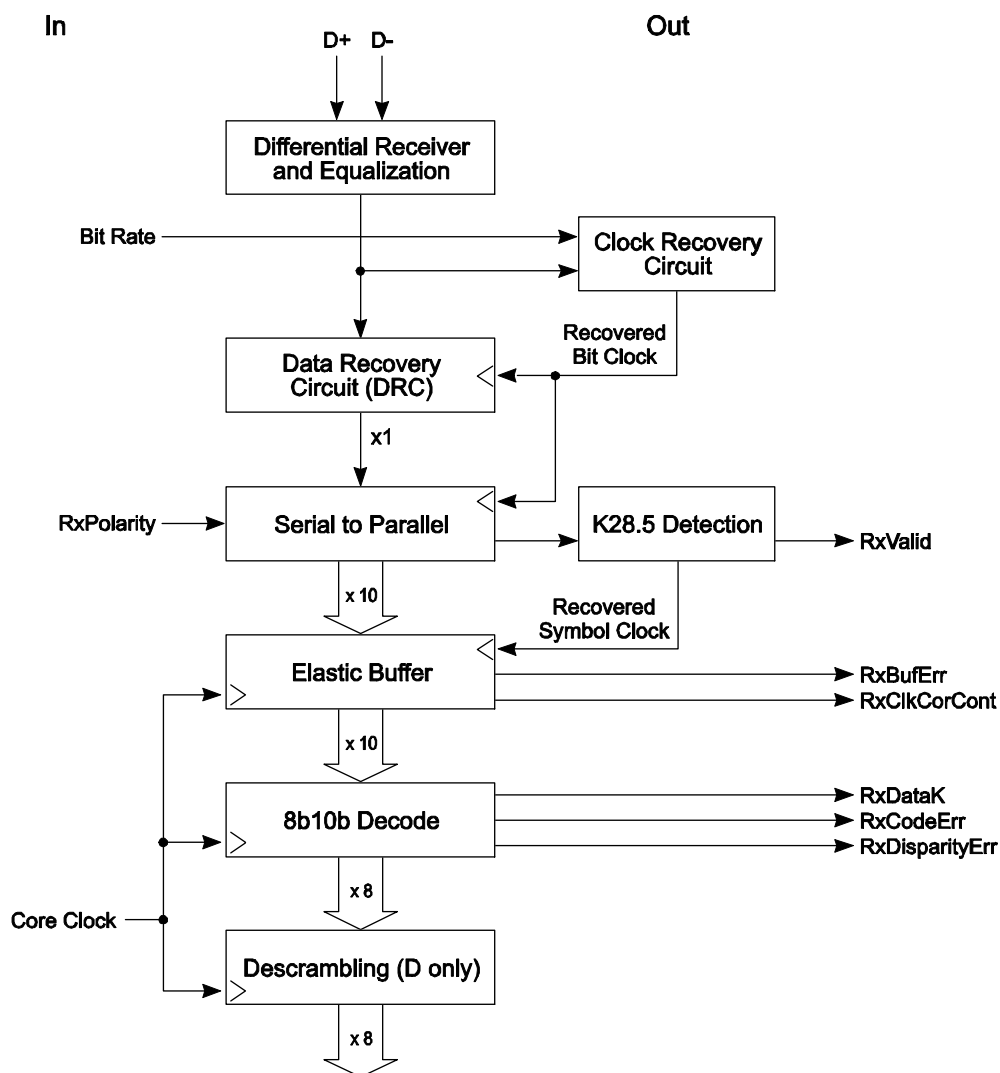
Gen 1 transmitter



IEC

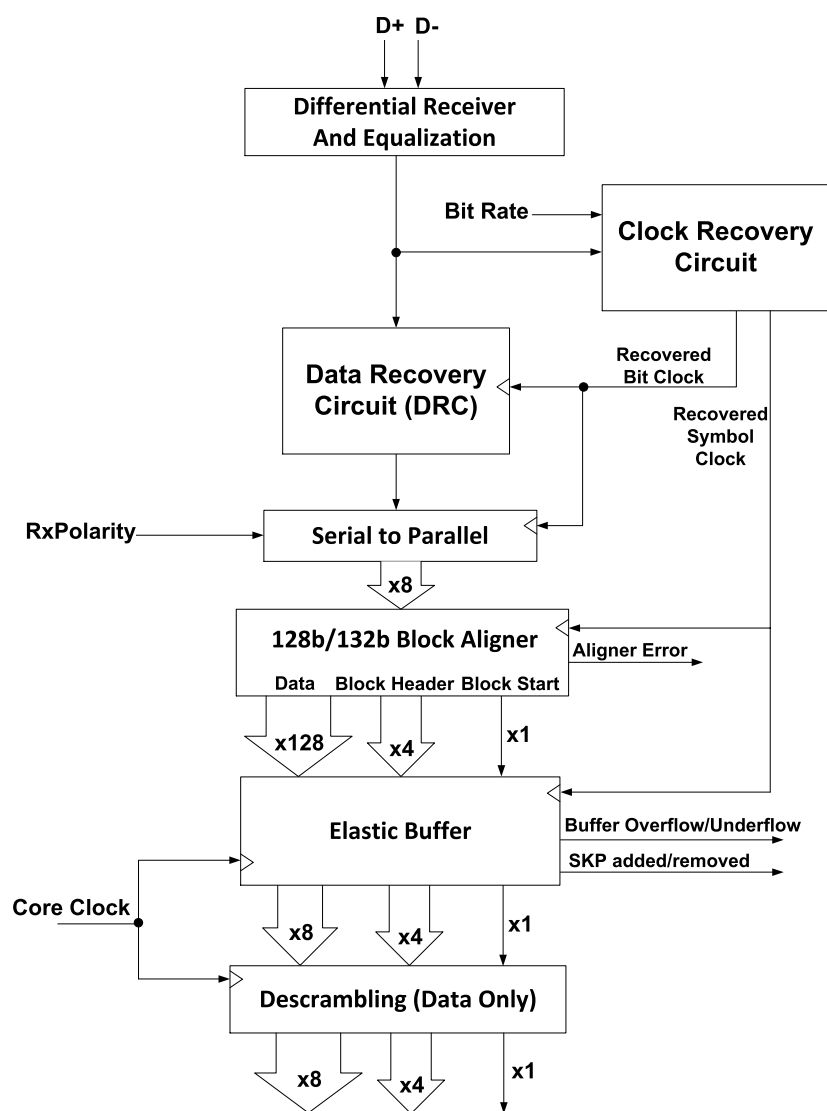
Gen 2 transmitter

Figure 6-2 – Transmitter Block Diagram



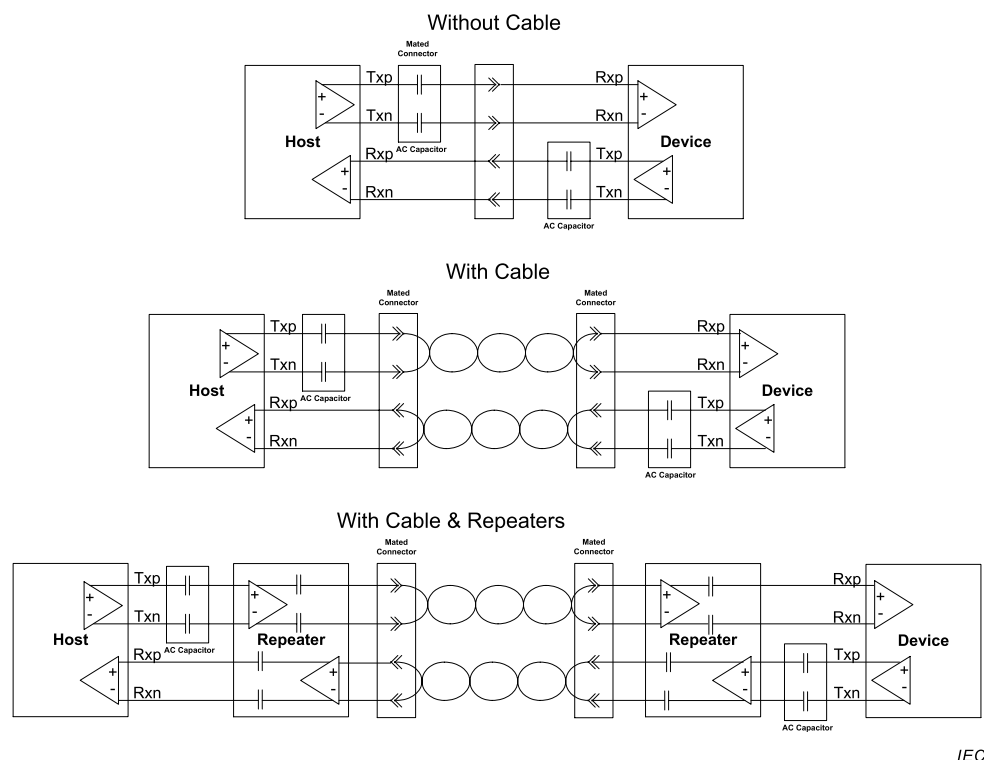
U-015
IEC

Figure 6-3 – Gen 1 Receiver Block Diagram



IEC

Figure 6-4 – Gen 2 Receiver Block Diagram



IEC

Figure 6-5 – Channel Models

6.2.1 Measurement Overview

The normative eye diagram is to be measured through compliance channels that represent long and short channels in order to cover the range of losses seen by real applications. These reference channels for testing at Gen 1 speed are described in the *USB 3.0 SuperSpeed Equalizer Design Guidelines* white paper. Reference channels for testing Gen 2 speed are described in a companion white paper posted on the USB-IF website. The eye diagram is measured using the appropriate clock recovery function described in Section 6.5.2.

Due to non-ideal channel characteristics, the eye diagram at the receiver may be completely closed. Informative receiver equalization functions are provided in Section 6.8.2 that are optimized for the compliance channels and are used to open the receiver eyes.

This methodology allows a silicon vendor to design the channel and the component as a matched pair. It is expected that a silicon component will have layout guidelines that must be followed in order for the component to meet the overall specification and the eye diagram at the end of the compliance channel.

Note that simultaneous USB 2.0 and SuperSpeed or SuperSpeedPlus operation is a testing requirement for compliance.

6.2.2 Channel Overview

A PHY is a transmitter and receiver that operate together and are located on the same component. A channel connects two PHYs together with two unidirectional differential pairs of pins for a total of four wires. The PHYs are required to be AC coupled. The AC coupling capacitors are associated with the transmitter.

6.3 Symbol Encoding

6.3.1 Gen 1 Encoding

The Gen 1 PHY uses the 8b/10b transmission code. The definition of this transmission code is identical to that specified in ANSI X3.230-1994 (also referred to as ANSI INCITS 230-1994), clause 11. As shown in Figure 6-6, ABCDE maps to abcdei and FGH maps to fghj.

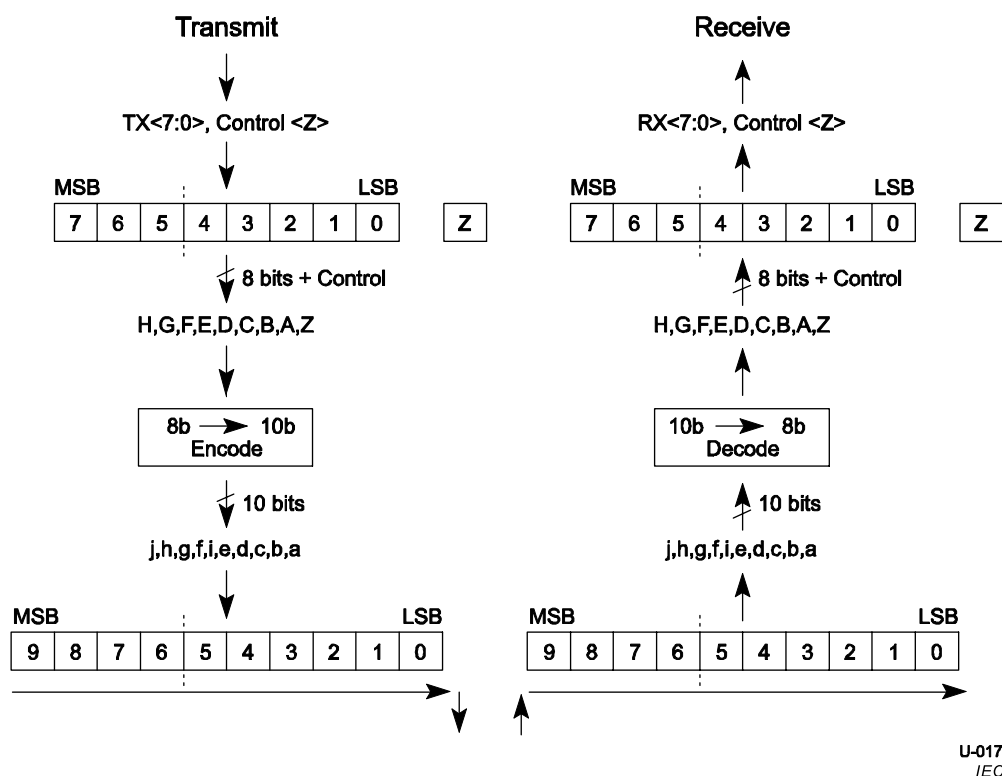


Figure 6-6 – Character to Symbol Mapping

6.3.1.1 Serialization and Deserialization of Data

The bits of a Symbol are placed starting with bit “a” and ending with bit “j.” This is shown in Figure 6-7.

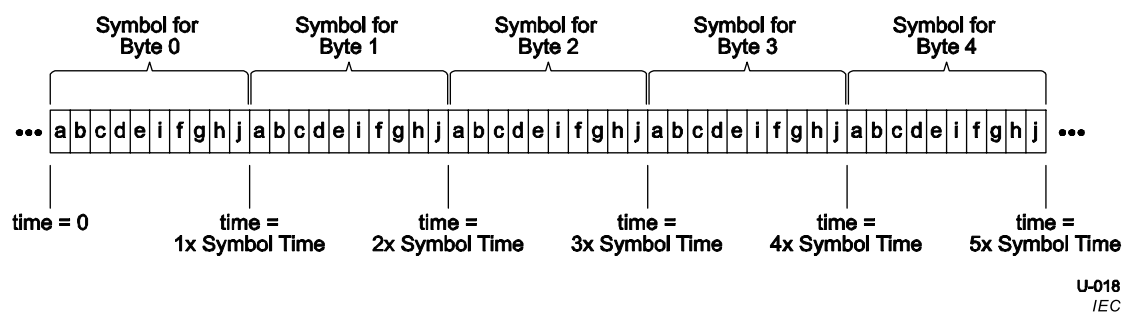


Figure 6-7 – Bit Transmission Order

6.3.1.2 Normative 8b/10b Decode Rules for Gen 1 Operation

- 1) A Transmitter is permitted to pick any disparity when first transmitting differential data after being in an Electrical Idle state. The Transmitter shall then follow proper 8b/10b encoding rules until the next Electrical Idle state is entered.
- 2) The initial disparity for a Receiver is the disparity of the first Symbol used to obtain Symbol lock.

- 3) Disparity may also be re-initialized if Symbol lock is lost and regained during the transmission of differential information due to a burst error event.
- 4) All following received Symbols after the initial disparity is set shall be in the proper column corresponding to the current running disparity.
- 5) Receive disparity errors do not directly cause the link to retrain.
- 6) If a disparity error or 8b/10 Decode error is detected, the physical layer shall inform the link layer.

6.3.1.3 Gen 1 Data Scrambling

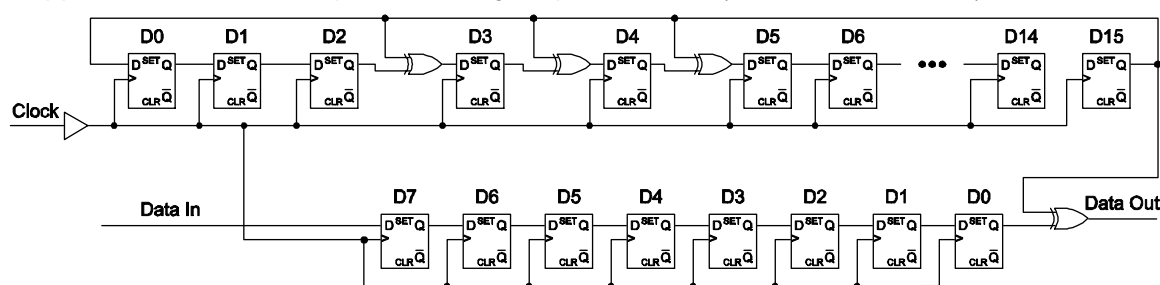
The scrambling function is implemented using a free running Linear Feedback Shift Register (LFSR). On the Transmit side, scrambling is applied to characters prior to the 8b/10b encoding. On the receive side, descrambling is applied to characters after 8b/10b decoding. The LFSR is reset whenever a COM symbol is sent or received.

The LFSR is graphically represented in Figure 6-8. Scrambling or unscrambling is performed by serially XORing the 8-bit (D0-D7) character with the 16-bit (D0-D15) output of the LFSR. An output of the LFSR, D15, is XORed with D0 of the data to be processed. The LFSR and data register are then serially advanced and the output processing is repeated for D1 through D7. The LFSR is advanced after the data is XORed.

The mechanism to notify the physical layer to disable scrambling is implementation specific and beyond the scope of this specification.

The data scrambling rules are as follows:

- 1) The LFSR implements the polynomial: $G(X) = X^{16} + X^5 + X^4 + X^3 + 1$
- 2) The LFSR value shall be advanced eight serial shifts for each Symbol except for SKP.
- 3) All 8b/10b D-codes, except those within the Training Sequence Ordered Sets shall be scrambled.
- 4) K codes shall not be scrambled.
- 5) The initialized value of an LFSR seed (D0-D15) shall be FFFFh. After COM leaves the Transmitter LFSR, the LFSR on the transmit side shall be initialized. Every time COM enters the Receive LFSR, the LFSR on the receive side shall be initialized. This also applies to the BRST sequence during loopback mode (see section 6.8.4.1).



U-019
IEC

Figure 6-8 – LFSR with Scrambling Polynomial



IMPLEMENTATION NOTE

Disabling Scrambling

Disabling scrambling is intended to help simplify test and debug equipment. Control of the exact data patterns is useful in a test and debug environment. Since scrambling is reset at the physical layer, there is no reasonable way to reliably control the state of the data transitions

through software. The Disable Scrambling bit is provided in the training sequence for this purpose.

The mechanism(s) and/or interface(s) used to notify the physical layer to disable scrambling is component implementation specific and beyond the scope of this specification.

For more information on scrambling, refer to Appendix B.

6.3.1.4 8b/10b Decode Errors for Gen 1 Operation

An 8b/10b Decode error shall occur when a received Symbol does not match any of the valid 8b/10b Symbols listed in Appendix A. Any received 8b/10b Symbol that does not match any of the valid 8b/10b Symbols listed in Appendix A shall be forwarded to the link layer by substituting a K28.4 symbol (refer to Table 6-1). 8b/10b errors may not directly initiate Recovery.

6.3.2 Gen 2 Encoding

A Gen 2 link, operating at 10 Gb/s, shall use the encoding rules described in this subsection. The encoding is a scrambled 128b/132b encoding.

6.3.2.1 Serialization and Deserialization of Data

Data is serialized and transmitted from LSB to MSB as shown below. For Gen 2 operation a Symbol is defined to be 1 byte of information that may or may not be scrambled according to the scrambling rules.

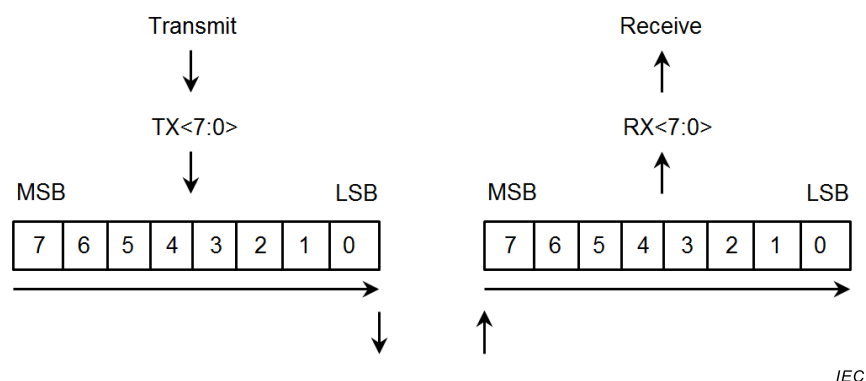
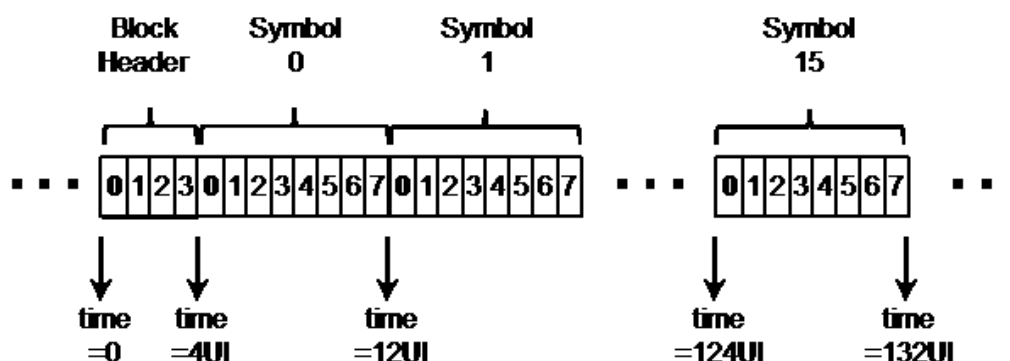


Figure 6-9 – Gen 2 Serialization and Deserialization Order



IEC

Figure 6-10 – Gen 2 Bit Transmission Order and Framing

6.3.2.2 Normative 128b/132b Decode Rules

The physical layer shall encode the data on a per block basis. Each block shall comprise a 4-bit Block Header and a 128-bit payload. The 4-bit header is set to 0011b for data and 1100b for control blocks. This header format allows for the correction of single bit errors in the header information.

Ordered sets are control blocks, and all data is sent in data blocks. The following is a list of the control blocks.

- TS1 Ordered Set
- TS2 Ordered Set
- TSEQ Ordered Set
- SYNC Ordered Set
- SKP Ordered Set
- SDS Ordered Set

6.3.2.3 Data Scrambling for Gen 2 Operation

The scrambler used for Gen 2 operation is different than the scrambler used for Gen 1 operation. The LFSR uses the following polynomial: $G(X) = X^{23} + X^{21} + X^{16} + X^8 + X^5 + X^2 + 1$.

The scrambler has the following modes of operation:

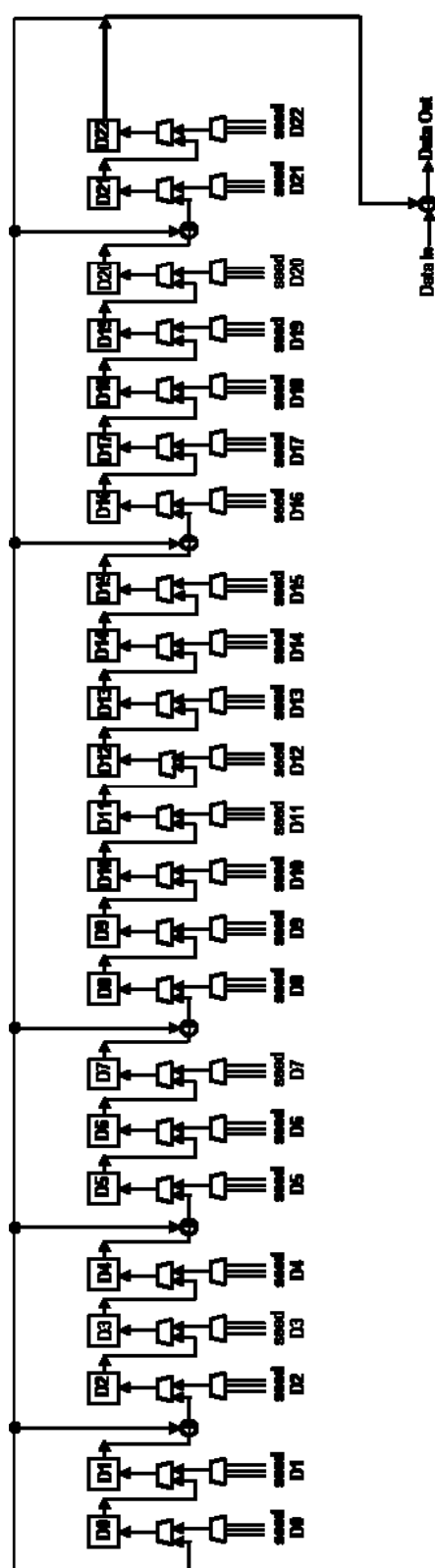
- 1) The scrambler advances and is XORed with the data.
- 2) The scrambler advances and is bypassed (not XORed with the data).
- 3) The scrambler does not advance and is bypassed (not XORed with the data).

The scrambling rules are as follows:

- 1) The 4 bits of the Block Header bypass and do not advance the scrambler.
- 2) TS1, TS2 and TSEQ:
 - a) Symbol 0 of a TS1, TS2, or TSEQ Ordered Set bypass and advances the scrambler.

- b) Symbols 1 to 13 are scrambled.
- c) Symbols 14 and 15 bypass the scrambler and the scrambler advances if being used for DC balance. If they are not being used for DC balance then they are scrambled.
- 3) SKP Ordered Sets bypass and do not advance the scrambler.
- 4) SDS Ordered Sets bypass the scrambler, but the scrambler advances.
- 5) All symbols of a SYNC Ordered Set bypass the scrambler. The scrambling LFSR is initialized after the last Symbol of a SYNC Ordered Set is transmitted. The descrambling LFSR is initialized after the last Symbol of a SYNC Ordered Set is received.
- 6) Receivers evaluate Symbol 0 of Control Blocks to determine whether to advance their LFSR. If Symbol 0 of the Block is SKP or SKPEND then the LFSR is not advanced for any Symbol of that Block. Otherwise, the LFSR is advanced for all Symbols of the Block.
- 7) All 16 Symbols of a Data Block are scrambled and advance the scrambler.
- 8) For Symbols that need to be scrambled the least significant bit is scrambled first and the most significant bit is scrambled last.
- 9) The seed value for the LFSR is 1D BFBCh.
- 10) Every 16384 TSEQ sets a SYNC Ordered Set shall be inserted to reset scrambler and to aid in block alignment.

Figure 6-11 depicts the LFSR for the scrambling polynomial.



IEC

Figure 6-11 – LFSR for use in Gen 2 operation

6.3.2.4 128b/132b Decode Errors

The Block Header decode error rules are as follows:

- 1) Single bit errors in the Block Header shall be reported to the link layer and corrected.

2) Double bit errors in the Block Header shall be reported to the link layer.

6.3.3 Special Symbols for Framing and Link Management

The 8b/10b encoding scheme provides Special Symbols that are distinct from the Data Symbols used to represent characters. These Special Symbols are used for various Link Management mechanisms described later. Table 6-1 lists the Special Symbols used and provides a brief description for each. Special Symbols shall follow the proper 8b/10b disparity rules. The compliance tests are defined in the *USB SuperSpeed Compliance Methodology* white paper. For Gen 2 operation the block header identifies whether the following 16 symbols have special meaning or if they represent data. In Gen 2 operation a receiver shall always perform single bit error correction on the special symbols when they are part of a control block. For Gen 1 and Gen 2 the following special symbols are defined.

Table 6-1 – Special Symbols

Symbol	Name	Gen 1 Symbol	Gen 2 Symbol	Description
SKP	Skip	K28.1	CCh	Compensates for different bit rates between two communicating ports. SKPs may be dynamically inserted or removed from the data stream. For SuperSpeedPlus operation, unscrambled
SKPEND	Skip End	Not applicable	33h	Marks the boundary between SKP symbols and the remainder of the SKP OS. Unscrambled.
SDP	Start Data Packet	K28.2	96h	Marks the start of a Data Packet Payload. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
EDB	End Bad	K28.3	69h	Marks the end of a nullified Packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SUB	Decode Error Substitution	K28.4	Not applicable	Symbol substituted by the 8b/10b decoder when a Decode error is detected.
COM	Comma	K28.5	Not applicable	Used for symbol alignment.
-----	-----	K28.6	Not applicable	Reserved
SHP	Start Header Packet	K27.7	9Ah	Marks the start of a Data Packet (Gen 1 operation only), Transaction Packet or Link Management Packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
DPHP	Start Data Packet Header	Not applicable	95h	Marks the start of a Data Packet (SuperSpeedPlus only). Scrambled and transmitted only in data block.
END	End	K29.7	65h	Marks the end of a packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SLC	Start Link Command	K30.7	5Ah	Marks the start of a Link Command. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
EPF	End Packet Framing	K23.7	36h	Marks the end of a packet framing. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SDS	Start of Data Stream	Not applicable	63h	Marks the start of an SDS Ordered Set. Unscrambled.

6.4 Link Initialization and Training

6.4.1 Link Training

6.4.1.1 Gen 1 Operation

This section defines the sequences that are used for configuration and initialization. The sequences are used by the Initialization State Machine (refer to Chapter 7) for the following functions:

- Configuring and initializing the link
- Bit-lock and symbol lock
- Rx equalization training
- Lane polarity inversion

Training sequences are composed of Ordered Sets used for initializing bit alignment, Symbol alignment and optimizing the equalization. Training sequence Ordered Sets are never scrambled but are always 8b/10b encoded.

Bit lock refers to the ability of the Clock/Data Recovery (CDR) circuit to extract the phase and frequency information from the incoming data stream. Bit lock is accomplished by sending a sufficiently long sequence of bits (D10.2 symbol containing alternating 0s and 1s) so the CDR roughly centers the clock within the bit.

Once the CDR is properly recovering data bits, the next step is to locate the start and end of a 10-bit symbol. For this purpose, the special K-Code called COMMA is selected from the 8b/10b codes. The bit pattern of the COMMA code is unique, so that it is never found in other data patterns, including any combination of a D-Code appended to any other D-Code or appended to any K-Code. This applies to any polarity of code. The only exception is for various bit patterns that include a bit error.

Training sequences (TS1 or TS2) are transmitted consecutively and can only be interrupted by SKP Ordered Sets occurring between Ordered Sets (between consecutive TS1 sets, consecutive TS2 sets, or when TS1 is followed by TS2).

6.4.1.1.1 Normative Training Sequence Rules for Gen 1 Operation

Training sequences are composed of Ordered Sets used for initializing bit alignment, symbol alignment, and receiver equalization.

The following rules apply to the training sequences:

- Training sequence Ordered Sets shall be 8b/10b encoded.
- Transmission of a TS1 or TS2 Ordered Set shall not be interrupted by SKP Ordered Sets. SKP Ordered Sets shall be inserted before, or after, completion of any TS1 or TS2 Ordered Set.
- No SKP Ordered Sets are to be transmitted during the entire TSEQ time (65,536 ordered sets). This means that the PHY must manage its elasticity buffer differently than during normal operation.

Additional rules for the use of TSEQ, TS1, and TS2 Ordered Sets can be found in Chapter 7.

6.4.1.1.2 Training Control Bits for Gen 1 Operation

The training control bits are found in the Link Functionality symbol within the TS1 and TS2 ordered sets. They are described in Table 6-5.

Bit 0 and bit 2 of the link configuration field shall not be set to 1 simultaneously. If a receiver detects this condition in the received Link configuration field, then all of the training control bits shall be ignored.

6.4.1.1.3 Training Sequence Values for Gen 1 Operation

The TSEQ training sequence repeats 65 536 times to allow for testing many coefficient settings.

Table 6-2 – Gen 1 TSEQ Ordered Set

Symbol Number	Name	Value
0	K28.5	COM (Comma)
1	D31.7	FFh
2	D23.0	17h
3	D0.6	C0h
4	D20.0	14h
5	D18.5	B2h
6	D7.7	E7h
7	D2.0	02h
8	D2.4	82h
9	D18.3	72h
10	D14.3	6Eh
11	D8.1	28h
12	D6.5	A6h
13	D30.5	BEh
14	D13.3	6Dh
15	D31.5	BFh
16-31	D10.2	4Ah

Table 6-3 – Gen 1 TS1 Ordered Set

Symbol Number	Encoded Values	Description
0-3	K28.5	COM (Comma)
4	D0.0	Reserved for future use
5	See Table 6-5	Link Functionality
6-15	D10.2	TS1 Identifier

Table 6-4 – Gen 1 TS2 Ordered Set

Symbol Number	Encoded Values	Description
0-3	K28.5	COM (Comma)
4	D0.0	Reserved
5	See Table 6-5	Link Functionality
6-15	D5.2	TS2 Identifier

Table 6-5 – Gen 1/Gen 2 Link Configuration

Field	TS1 Symbol 5	Description
Bit 0	0 = Normal Training 1 = Reset	Reset is set by the Host only in order to reset the device.
Bit 1	Set to 0	Reserved for future use.
Bit 2	0 = Loopback de-asserted 1 = Loopback asserted	When set, the receiving component enters digital loopback.
Bit 3	0 = Disable Scrambling de-asserted 1 = Disable Scrambling asserted	When set, the receiving component disables scrambling. When this is asserted during Gen 2 operation the training Ordered Sets are still scrambled and the disabling of scrambling begins with the first Data Block after the SDS.
Bit 4	0 = Local loopback in repeater de-asserted 1 = Local loopback in repeater asserted	When set, the nearest repeater in the link is placed into local loopback mode.
Bit 5:7	Set to 0	Reserved for future use.

6.4.1.2 Gen 2 Operation

This section defines the sequences that are used for configuration and initialization of a link operating at Gen 2 rates. The sequences are used by the Initialization State Machine (refer to Chapter 7) for the following functions:

- Configuring and initializing the link
- Bit-lock and symbol lock
- Rx equalization training
- Lane polarity inversion
- Block alignment

Training sequences are composed of Ordered Sets used for initializing bit alignment, Symbol alignment, block alignment and optimizing the equalization.

Bit lock refers to the ability of the Clock/Data Recovery (CDR) circuit to extract the phase and frequency information from the incoming data stream. Bit lock is accomplished by sending a pattern sufficiently rich in transitions so that the CDR roughly centers the clock within the bit.

6.4.1.2.1 Normative Training Sequence Rules for Gen 2 Operation

Training sequences are composed of Ordered Sets used for initializing bit alignment, symbol alignment, block alignment, scrambler synchronization and receiver equalization.

The following rules apply to the training sequences:

- 1) Training sequence Ordered Sets shall comprise 16 Symbols and be 128b/132b encoded.
- 2) Transmission of a TSEQ, TS1 or TS2 Ordered Set can only be interrupted by a SKP Ordered Set or a SYNC Ordered Set.
- 3) A SYNC Ordered Set shall be transmitted every 16 384 TSEQ sets during a Gen 2 training session.
- 4) A SYNC Ordered Set shall be transmitted every 32 ordered sets in Gen 2 operation when sending TS1 or TS2 ordered sets (during Recovery, Polling.Active, Recovery.Configuration, Hot Reset and Polling.Config).

6.4.1.2.2 Training Sequence Values for Gen 2 Operation

Transmitters are required to track the running DC Balance of the bits transmitted on the wire (after scrambling) for TSEQ, TS1 and TS2 Ordered Sets. The running DC Balance is the difference between the number of 1s transmitted and the number of 0s transmitted.

The PHY shall be capable of tracking a difference of at least 511 bits in either direction: 511 more 1s than 0s, and 511 more 0s than 1s. Any counters used shall saturate at their limit (not roll-over) and continue to track reductions after their limit is reached. For example, a counter that can track a difference of 511 bits will saturate at 511 if a difference of 513 is detected, and then change to 509 if the difference is reduced by 2 in the future.

The running DC Balance is set to 0 at the start of Gen 2 data block transmission.

For every TSEQ, TS1 or TS2 Ordered Set transmitted, Transmitters shall evaluate the running DC Balance and transmit one of the DC Balance Symbols defined for Symbols 14 and 15 as defined by the algorithm below. If the number of 1s needs to be reduced, the DC Balance Symbols 20h (for Symbol 14) and 08h (for Symbol 15) are transmitted. If the number of 0s needs to be reduced, the DC Balance Symbols DFh (for Symbol 14) and F7h (for Symbol 15) are transmitted. If no change is required, the appropriate TS Identifier Symbol is transmitted. Any DC Balance Symbols transmitted for Symbols 14 or 15 bypass scrambling, while TS Identifier Symbols follow the standard scrambling rules. The following algorithm shall be used to control the DC Balance:

- 1) If the running DC Balance is > 31 at the end of Symbol 11 of the TS Ordered Set, transmit DFh for Symbol 14 and F7h for Symbol 15 to reduce the number of 0s, or 20h for Symbol 14 and 08h for Symbol 15 to reduce the number of 1s.
- 2) Else, if the running DC Balance is > 15 at the end of Symbol 11 of the TS Ordered Set, transmit F7h for Symbol 15 to reduce the number of 0s, or 08h for Symbol 15 to reduce the number of 1s. Transmit the normal TS Identifier Symbol (scrambled) for Symbol 14.
- 3) Else, transmit the normal TS Identifier Symbol (scrambled) for Symbols 14 and 15.

Receivers may check Symbols 14 and 15 for the following values when determining whether a TS Ordered Set is valid: The appropriate TS Identifier Symbol after de-scrambling, or a valid DC Balance Symbol of DFh or 20h before de-scrambling for Symbol 14, or a valid DC Balance Symbol of F7h or 08h before de-scrambling for Symbol 15.

A new ordered set required for Gen 2 operation is the Start of Data Stream (SDS) Ordered set. This is only defined for Gen 2 operation and does not have a Gen 1 counterpart. It shall be transmitted during Polling.Idle, Recovery.Idle, and Hot Reset.Exit to define the transition from Ordered Set Blocks to a Data Stream. It shall not be transmitted at any other time. While not in the Loopback state, the Block following an SDS Ordered Set shall be a Data Block and the first Symbol of that Data Block is the first Symbol of the Data Stream.

Table 6-6 – Gen 2 TS1 Ordered Set

Symbol Number	Symbol	Description
0-3	1Eh	TS1 Identifier
4	00h	Reserved
5	See Table 6-5	Link Functionality
6-13	1Eh	TS1 Identifier
14-15	TS1 Identifier (1Eh) or a DC Balance Symbol	TS1 Identifier or DC balance

Table 6-7 – Gen 2 TS2 Ordered Set

Symbol Number	Symbol	Description
0-3	2Dh	TS2 Identifier
4	00h	Reserved
5	See Table 6-5	Link Functionality
6-13	2Dh	TS2 Identifier
14-15	TS2 Identifier (2Dh) or a DC Balance Symbol	TS2 Identifier or DC balance

Table 6-8 – Gen 2 TSEQ Ordered Set

Symbol Number	Symbol	Description
0-3	87h	TSEQ Identifier
4-5	00h	Reserved
6-13	87h	TSEQ Identifier
14-15	TSEQ Identifier (87h) or a DC Balance Symbol	TSEQ Identifier or DC balance

Table 6-9 – Gen 2 SYNC Ordered Set

Symbol Number	Symbol	Description
0,2,4,6,8,10,12,14	00h	Symbol 0 SYNC identifier
1,3,5,7,9,11,13,15	FFh	

Table 6-10 – SDS Ordered Set

Symbol Number	Symbol	Description
0	E1h	SDS Ordered Set Identifier
2 through 15	55h	Body of SDS Ordered Set

6.4.1.2.3 Training Control Bits for Gen 2 Operation

The training control bits are found in the Link Functionality symbol within the TS1 and TS2 ordered sets. They are described in Table 6-5.

Bit 0 and bit 2 of the link configuration field shall not be set to 1 simultaneously. If a receiver detects this condition in the received Link configuration field, then all of the training control bits shall be ignored.

6.4.1.2.4 Informative Block Alignment for Gen 2 Operation

During Link training, the 132 bits of the SYNC block are a unique bit pattern that Receivers use to determine the location of the Block Headers in the received bit stream. Conceptually, Receivers can be in three different phases of Block alignment: Unaligned, Aligned, and Locked. These phases are defined to illustrate the required behavior, but are not meant to specify a required implementation.

Unaligned Phase: Receivers enter this phase when they exit a low-power Link state, or if directed. In this phase, Receivers monitor the received bit stream for the SYNC OS. When one is detected, they adjust their alignment to it and proceed to the Aligned phase.

Aligned Phase: During this phase, receivers monitor the received bit stream for SYNC Ordered Sets. If a SYNC OS is detected with an alignment that does not match the current alignment, Receivers shall adjust its alignment to the newly received SYNC OS. Once an SDS OS is received, Receivers proceed to the Locked phase. Receivers are permitted to return to the Unaligned phase if an undefined Block Header is received. Receivers shall adjust the alignment in this phase as needed when receiving SKP ordered sets of lengths other than 16 symbols.

Locked Phase: Receivers shall not adjust their Block alignment while in this phase. Data Blocks are expected to be received with the given alignment, and adjusting the Block alignment would interfere with the processing of these Blocks. Receivers shall return to the Unaligned or Aligned phase if an undefined Block Header is received. Receivers shall adjust the alignment in this phase as needed when receiving SKP ordered sets of lengths other than 16 symbols.

Upon entering U1 a transmitter may send out unintended data before powering down. These bits have no meaning and may arise due to a block boundary ending in the middle of a transmitter's internal parallel data path.

6.4.2 Lane Polarity Inversion

6.4.2.1 Gen 1 Operation

During the TSEQ training sequence, the Receiver shall use the D10.2 Symbol within the TSEQ Ordered Set to determine lane polarity inversion (Rxp and Rxn are swapped). If polarity inversion has occurred, the D10.2 symbols within the TSEQ ordered set will be received as D21.5 instead of D10.2 and the receiver shall invert the polarity of the received bits. This shall be done before the TSEQ symbols 1-15 are used since these symbols are not all symmetric under inversion in the 8b/10b domain. If the receiver does not use the TSEQ training sequence then the polarity inversion may be checked against the D10.2 symbol in the TS1 ordered set.

6.4.2.2 Gen 2 Operation

During reception of SYNC ordered sets the symbols of the SYNC Ordered Set shall be used to determine whether a polarity inversion has occurred. If the SYNC identifier (and symbols 2,4,6,8,10,12,14) are received as FFh instead of 00h then a polarity inversion has occurred and the receiver shall invert the polarity of the received bits.

6.4.3 Elasticity Buffer and SKP Ordered Set

The Enhanced SuperSpeed architecture supports a separate reference clock source on each side of the Enhanced SuperSpeed link. The accuracy of each reference clock is required to be within ± 300 ppm. This gives a maximum frequency difference between the two devices of the link of ± 600 ppm. In addition, SSC creates a frequency delta that has a maximum difference of 5 000 ppm. The total magnitude of the frequency delta can range from $-5\,300$ to 300 ppm. This frequency delta is managed by an elasticity buffer that consumes or inserts SKP ordered sets.

SKP Ordered Sets shall be used to compensate for frequency differences between the two ends of the link. The transmitter sends SKP ordered sets at an average of every 354 symbols. However, SKP ordered sets shall not be inserted within any packet. The transmitter is allowed to buffer the SKP ordered sets up to a maximum of four SKP ordered sets. For Gen 1 operation the receiver shall implement an elasticity buffer capable of buffering (or starving) eight symbols of data. For Gen 2 operation, due to the presence of retimers along the signal path, a receiver shall tolerate not receiving any SKP Symbols for up to 180 blocks. Thus during Gen 2 operation a receiver must implement an elasticity buffer capable of buffering (or starving) twenty two symbols of data.

6.4.3.1 SKP Rules (Host/Device/Hub) for Gen 1 Operation

- The SKP Ordered Set shall consist of a SKP K-Symbol followed by a SKP K-Symbol. A SKP Ordered Set represents two Symbols that can be used for clock compensation.
- A device shall keep a running count of the number of transmitted symbols since the last SKP Ordered set. The value of this count will be referred to as Y. The value of Y is reset whenever the transmitter enters Polling.Active.
- Unless otherwise specified, a transmitter shall insert the integer result of $Y/354$ calculation Ordered sets immediately after each transmitted TS1, TS2 Ordered Set, LMP, TP Data Packet Payload, or Logical idle. During training only, a transmitter is allowed the option of waiting to insert 2 SKP ordered sets when the integer result of $Y/354$ reaches 2. A transmitter shall not transmit SKP Ordered Sets at any other time.

NOTE The non-integer remainder of the $Y/354$ SKP calculation shall not be discarded and shall be used in the calculation to schedule the next SKP Ordered Set.

- SKP Commands do not count as interruptions when monitoring for Ordered Sets (i.e., consecutive TS1, TS2 Ordered Sets in Polling and Recovery).

Table 6-11 – Gen 1 SKP Ordered Set Structure

Symbol Number	Encoded Values	Description
0	K28.1	SKP
1	K28.1	SKP

6.4.3.2 SKP Rules (Host/Device/Hub) for Gen 2 Operation:

Table 6-12 describes the layout of the SKP Ordered Set when using 128b/132b encoding. A transmitted SKP Ordered Set always starts out as 16 Symbols long. The granularity for which SKP Symbols can be added or removed by a Port is two symbols. A port may add or remove more than 2 SKP symbols, but the number of SKP symbols that is added or removed shall be a multiple of two. This includes retimers within the signal path. Thus, a receiver may receive a SKP OS with anywhere from 0 to thirty six SKP symbols with the number of SKP symbols being a multiple of two. A SKP OS with 0 SKP symbols has only a SKPEND symbol followed by the three symbols that describe the LFSR state. Another impact of receiving variable length SKP OS is that a receiver is always allowed to add up to 12 SKP symbols to any SKP OS regardless of the length of the received SKP OS.

The SKPEND Symbol indicates the last four Symbols of SKP Ordered Set so that receivers can identify the location of the next Block Header in the bit stream. The three Symbols following the SKPEND Symbol contain different information depending on the LTSSM state.

A receiver must always perform single bit error correction on the SKP and SKPEND (and all other special) symbols. However, since the Hamming distance between the SKP and SKPEND symbols is 8, once a receiver has determined that it is dealing with a non-empty SKP OS (by proper detection of a first SKP symbol) it may be beneficial to use multiple bit (up to 3-bit) error correction in differentiating between a SKP and a SKPEND symbol.

Table 6-12 – Gen 2 SKP Ordered Set

Symbol Number	Value	Description
0 through $2*N-1$ [N can be 0 through 18]	CCh	SKP Symbol Symbol 0 is the SKP Ordered Set Identifier NOTE For an empty SKP OS, the first symbol will be a SKPEND.
$2*N$	33h	SKPEND Symbol
$2*N+1$	00-FFh	(i) If prior block was a Data Block: Bit[7] = Even Data Parity Bit[6:0] = LFSR[22:16] (ii) Else: Bit[7] = $\sim$ LFSR[22] Bit[6:0] = LFSR[22:16]
$2*N+2$	00-FFh	(i) If LTSSM state is Polling.Compliance: Error_Status[7:0] (ii) Else LFSR[15:8]
$2*N+3$	00-FFh	(i) If LTSSM state is Polling.Compliance: $\sim$ Error_Status[7:0] (ii) Else LFSR[7:0]

The following rules apply for SKP insertion for Gen 2 operation:

- 1) A transmitter shall keep a running count of the number of transmitted blocks since the last SKP Ordered set. The value of this count will be referred to as Y. The value of Y is reset whenever the transmitter enters Polling.Active or when a SKP OS is transmitted.
- 2) Once the count, Y, gets to 21 a transmitter must insert a SKP OS at the next legitimate opportunity. The fastest a transmitter can insert SKP OS is once every 22 blocks. Situations that delay the immediate insertion of a SKP OS are the following: a transmitter shall not interrupt a data packet or a SYNC OS to insert a SKP OS. In the worst case it may take 90 blocks before there is an opportunity to insert a SKP OS. In Gen 2 operation there is no accumulation of SKP OS, each time a SKP OS is transmitted the SKP counter, Y, is reset to 0.
- 3) SKP Ordered Sets do not count as interruptions when monitoring for Ordered Sets (i.e., consecutive TS1, TS2 Ordered Sets in Polling and Recovery).
- 4) SYNC ordered sets have priority of SKP ordered sets. A SKP OS that is scheduled to be sent at the same time as a SYNC OS shall have to be delayed until the SYNC OS is transmitted.
- 5) The Data parity bit should be even parity for last three symbols in the SKP OS. The parity is a check of the LFSR seed value.

6.4.4 Compliance Pattern

Entry to the Polling.Compliance substate is described in Chapter 7. This initiates the transmission of the pseudo-random data pattern generated by the scrambled D0.0 compliance sequence. SKPs are not sent during the transmission of any compliance pattern. The compliance pattern shall be transmitted continuously or until a ping LFPS (refer to Section 6.9) is detected at the receiver. Upon detection of a ping LFPS, the compliance pattern shall advance to the next compliance pattern. Upon detection of a reset, LFPS the compliance pattern shall be terminated. The compliance pattern sequences are described in Table 6-13.

In the table, patterns CP0 through CP8 are transmitted at Gen 1 rate, while CP9 through CP12 are transmitted at Gen 2 rate.

Table 6-13 – Compliance Pattern Sequences

Compliance Pattern	Value	Description
CP0	D0.0 scrambled	A pseudo-random data pattern that is exactly the same as logical idle (refer to Chapter 7) but does not include SKP sequences.
CP1	D10.2	Nyquist frequency
CP2	D24.3	Nyquist/2
CP3	K28.5	COM pattern
CP4	LFPS	The low frequency periodic signaling pattern
CP5	K28.7	With de-emphasis
CP6	K28.7	Without de-emphasis
CP7	50-250 1's and 0's	With de-emphasis. Repeating 50-250 1's and then 50-250 0's.
CP8	50-250 1's and 0's	With without de-emphasis. Repeating 50-250 1's and then 50-250 0's.
CP9		Pseudo-random data pattern (see section 6.4.4.1)
CP10	AAh	Nyquist pattern at 10 Gb/s. This is not 128b132b encoded.
CP11	CCh	Nyquist/2 at 10 Gb/s, This is not 128b132b encoded.
CP12	LFSR15	Uncoded LFSR15 for PHY level testing and fault isolation. This is not 128b132b encoded.

NOTE Unless otherwise noted, scrambling is disabled for compliance patterns.

6.4.4.1 Gen 2 Compliance Pattern CP9

The Gen 2 compliance pattern comprises a pseudo-random data pattern that is used to test transmitter and receiver compliance. The pattern repeats every 65 536 symbols and starts with a SYNC Ordered Set so as to reset the scrambler and mark the beginning of the pattern.

Table 6-14 – Gen 2 Compliance Pattern

Symbol Number	Value	Description
0-15	SYNC ordered set	Compliance pattern starts with a SYNC to signal the beginning of the pattern and reset the scrambler
16-65535	00h	Data symbol 00h that is scrambled.

6.5 Clock and Jitter

6.5.1 Informative Jitter Budgeting

The jitter for USB 3.1 is budgeted among the components that comprise the end to end connections: the transmitter, channel (including packaging, connectors, and cables), and the receiver. The jitter budget is derived at the silicon pads. The Dj distribution is the dual Dirac method. Table 6-15 lists Tx, Rx, and channel jitter budgets.

Table 6-15 – Informative Jitter Budgeting at the Silicon Pads

Jitter Contribution (ps)	Gen 1 (5 GT/s)			Gen 2 (10 GT/s)		
	Rj ^{1,2}	Dj ³	Tj ⁴ at 10 ⁻¹²	Rj ^{1,2}	Dj ³	Tj ⁴ at 10 ⁻¹²
Tx ⁶	2,42	41	75	1,31	17	35,4
Media ⁵	2,13	45	75	0,00	36	36,0
Rx	2,42	57	91	1,31	21	39,4
Total:	4,03	143	200	1,85	74	100

¹ Rj is the sigma value assuming a Gaussian distribution.

² Rj Total is computed as the Root Sum Square of the individual Rj components.

³ Dj budget is using the Dual Dirac method.

⁴ Tj at a 10⁻¹² BER is calculated as $14.068 \times Rj + Dj$.

⁵ The media budget includes the cancellation of ISI from the appropriate Rx equalization function.

⁶ Tx is measured after application of the JTF.

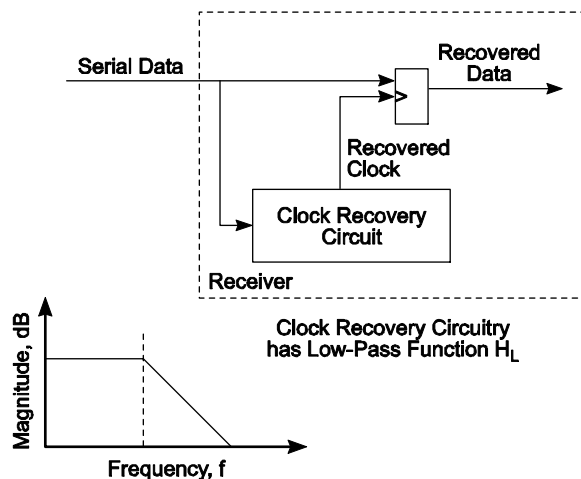
**NOTE****Captive Cables**

Captive cables must meet the mated connector requirements specified in Section 5.6.1.2. But a captive cable is not considered a stand-alone component. For electrical budgeting purposes, a captive cable is considered to be part of a device, and must meet the device jitter requirements listed in Table 6-15.

6.5.2 Normative Clock Recovery Function

The Tx Phase jitter measurement is performed using a standard clock recovery, shown in Figure 6-12. For information on the golden PLL measurement refer to the latest version of INCITS TR-35-2004, *INCITS Technical Report for Information Technology – Fibre Channel – Methodologies for Jitter and Signal Quality Specification (FC-MJSQ)*.

The clock recovery function is given by Equations 1-3. A schematic of the general clock recovery function is shown in Figure 6-12. As shown, the clock recovery circuit has a low pass response. After the recovered clock is compared (subtracted) to the data, the overall clock recovery becomes a high pass function. This is shown with the appropriate bandwidths in Figure 6-13 for Gen 1 operation and in Figure 6-14 for Gen 2 operation.

U-020
IEC**Figure 6-12 – Jitter Filtering – “Golden PLL” and Jitter Transfer Functions**

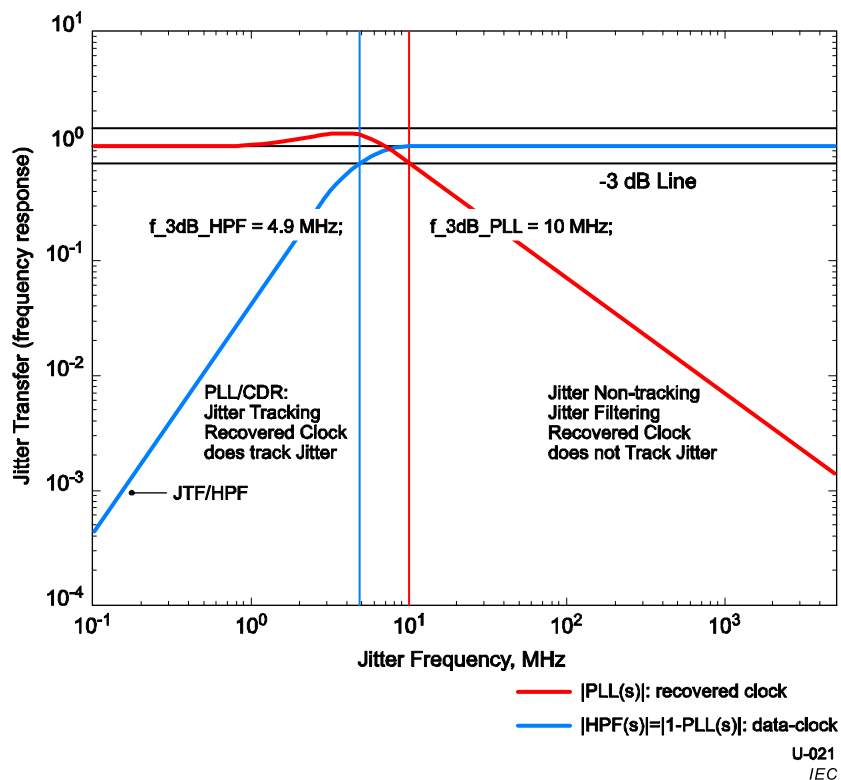


Figure 6-13 – “Golden PLL” and Jitter Transfer Functions for Gen 1 Operation

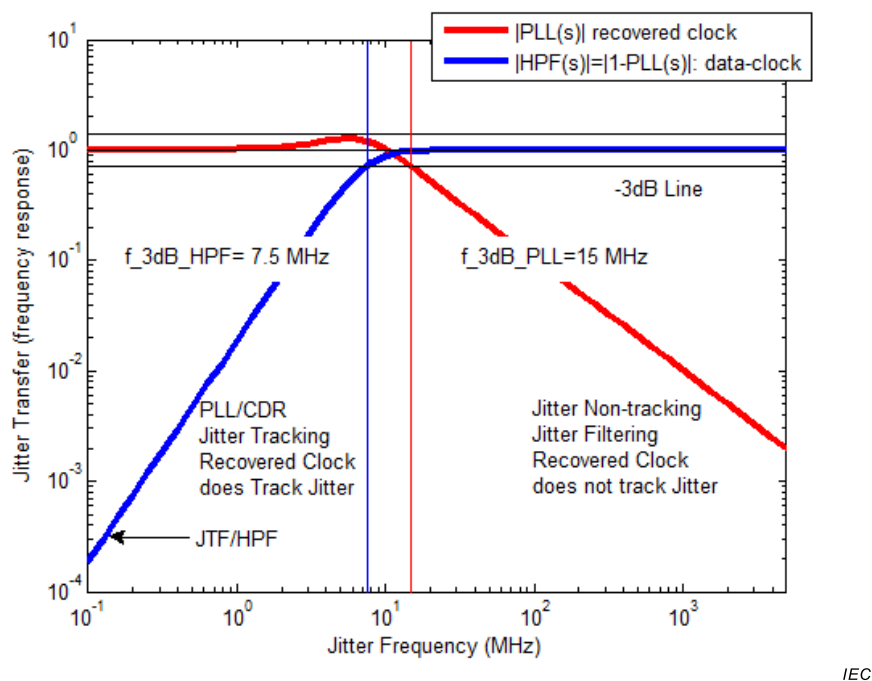


Figure 6-14 – “Golden PLL” and Jitter Transfer Functions for Gen 2 Operation

The equations for these functions are:

$$H_{CDR}(s) = \frac{2s\zeta\omega_n + \omega_n^2}{s^2 + 2s\zeta\omega_n + \omega_n^2} \quad (1)$$

and

$$JTF(s) = \frac{s^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \quad (2)$$

where ω_n is the natural frequency and ζ is the damping factor. The relationship to the 3 dB frequency is

$$\omega_{3dB} = \omega_n \left(1 + 2\zeta^2 + \left[(1 + 2\zeta^2)^2 + 1 \right]^{\frac{1}{2}} \right)^{\frac{1}{2}} \quad (3)$$

As shown in Figure 6-13, for Gen 1 operation the corner frequency is $\omega_{3dB} = 2\pi 10^7$ and $\zeta = 0.707$. For Gen 2 operation refer to Figure 6-14 with $\omega_{3dB} = 2\pi 1.5 \times 10^7$ and $\zeta = 0.707$. These transfer functions have a maximum peaking of 2 dB.

6.5.3 Normative Spread Spectrum Clocking (SSC)

All ports are required to have Spread Spectrum Clocking (SSC) modulation. Providing the same SSC clock to two different components is allowed but not required, the SSC can be generated asynchronously. The SSC profile is not specified and is vendor specific. The SSC modulation requirement is listed in Table 6-16. The SSC modulation may not violate the phase slew rate described in Section 6.5.4.

Table 6-16 – SSC Parameters

Symbol	Description	Limits		Units	Note
		Min	Max		
$t_{SSC-MOD-RATE}$	Modulation Rate	30	33	kHz	
$t_{SSC-FREQ-DEVIATION}$	SSC deviation	+0/-4000	+0/-5000	ppm	1, 2

NOTES:

- 1) The data rate is modulated from 0 ppm to -5000 ppm of the nominal data rate frequency and scales with data rate.
- 2) This is measured below 2 MHz only.

An example of the period modulation from triangular SSC is shown in Figure 6-15.

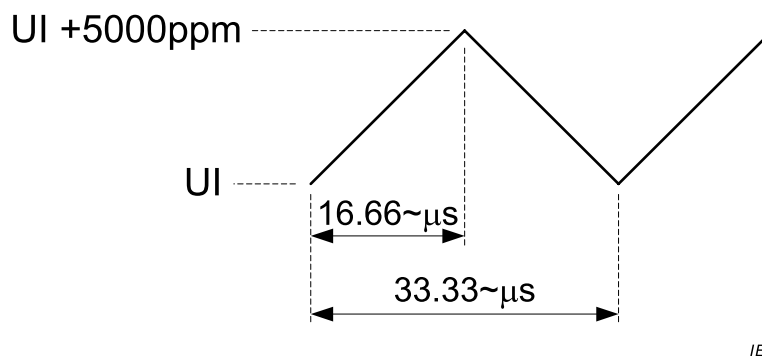


Figure 6-15 – Example of Period Modulation from Triangular SSC

6.5.4 Normative Slew Rate Limit

The CDR is a slew rate limited phase tracking device. The combination of SSC and all other jitter sources within the bandwidth of the CDR shall not exceed the maximum allowed slew rate.

This measurement is performed by filtering the phase jitter with the CDR transfer function and taking the first difference of the phase jitter to obtain the filtered period jitter. The peak of the period jitter shall not exceed $T_{\text{CDR_SLEW_MAX}}$ listed in Table 6-17.

Additional details on the slew rate measurement are available in the white paper titled *USB 3.0 Jitter Budgeting*.

6.6 Signaling

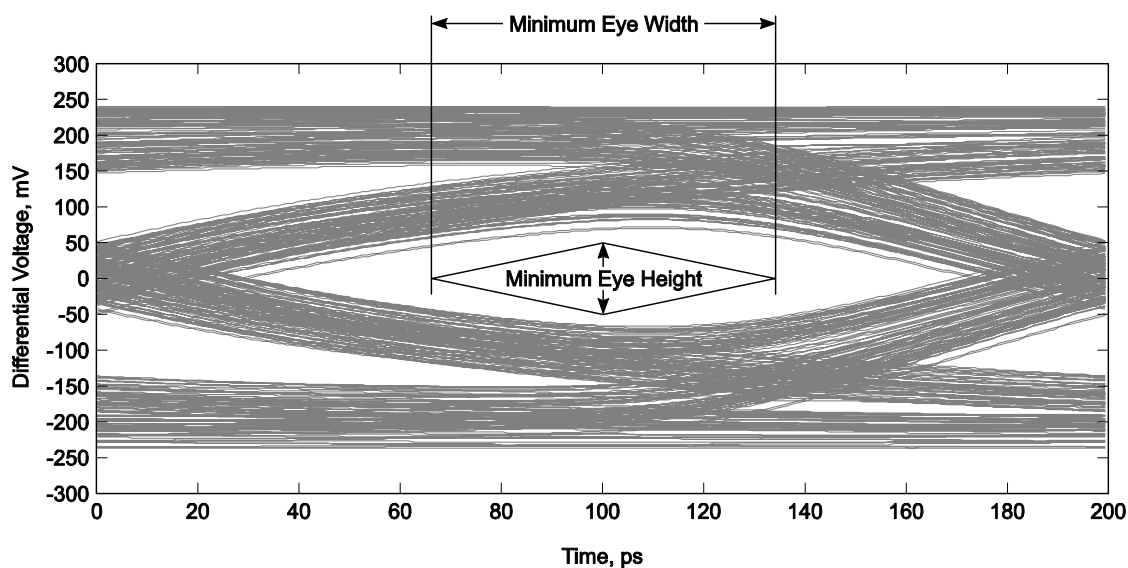
6.6.1 Eye Diagrams

The eye diagrams are a graphical representation of the voltage and time limits of the signal. This eye mask applies to jitter after the application of the appropriate jitter transfer function and reference receiver equalization. In all cases, the eye is to be measured for 10^6 consecutive UI. The budget for the link is derived assuming a total 10^{-12} bit error rate and is extrapolated to a measurement of 10^6 UI assuming the random jitter is Gaussian.

Figure 6-16 shows the eye mask used for all eye diagram measurements. Referring to the figure, the time is measured from the crossing points of Txp/Txn. The time is called the eye width, and the voltage is the eye height. The eye height is to be measured at the maximum opening (at the center of the eye width ± 0.05 UI). Specific eye mask requirements are defined in Table 6-19.

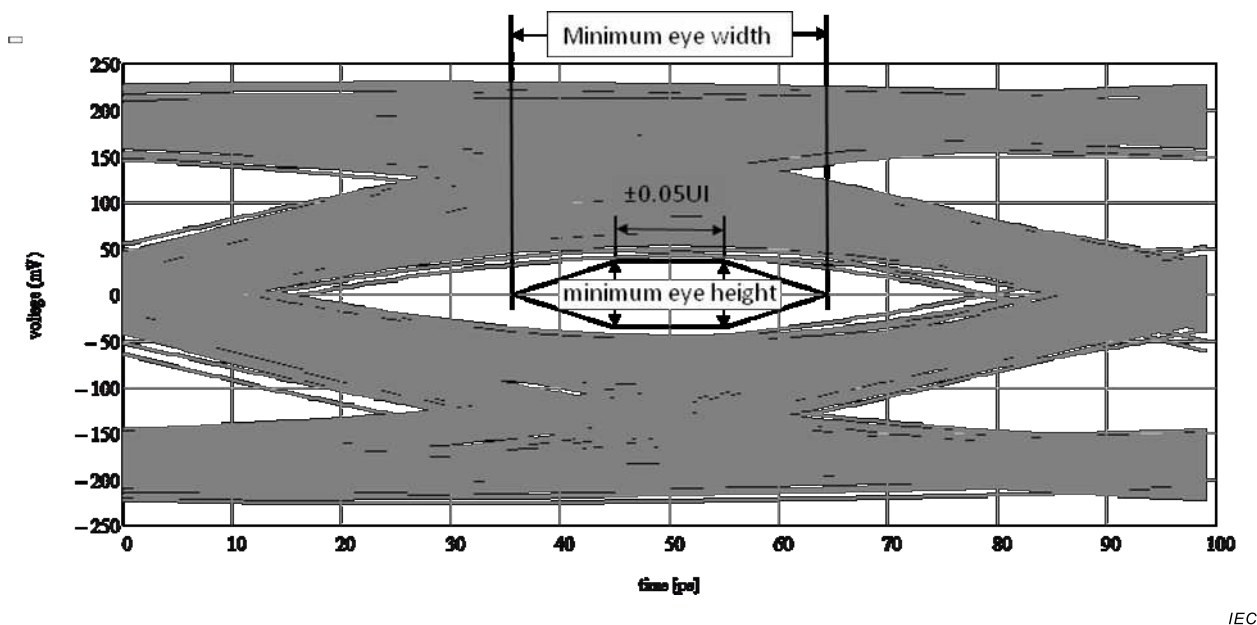
The eye diagrams are to be centered using the jitter transfer function (JTF). The recovered clock is obtained from the data and processed by the JTF. The center of the recovered clock is used to position the center of the data in the eye diagram.

The eye diagrams are to be measured into 50- Ω single-ended loads.



U-023
IEC

Gen 1 eye mask



Gen 2 eye mask

Figure 6-16 – Eye Masks

6.6.2 Voltage Level Definitions

Referring to Figure 6-17, the differential voltage, V_{DIFF} , is the voltage on Txp (Rxp at the receiver) with respect to Txn (Rxn at the receiver). V_{DIFF} is the same voltage as the swing on the single signal of one conductor. The differential voltage is

(4) $V_{DIFF} = T_{xp} - T_{xn}$

The total differential voltage swing is the peak to peak differential voltage, $V_{DIFF-PP}$. This is twice the differential voltage. The peak to peak differential voltage is

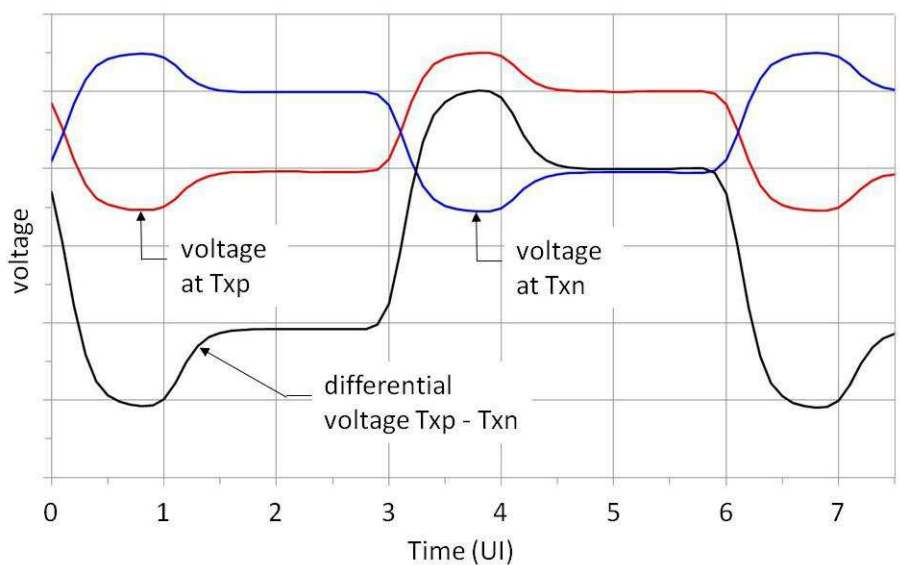
(5) $V_{DIFF-PP} = 2 \times V_{DIFF}$

The Common Mode Voltage (V_{CM}) is the average voltage present on the same differential pair with respect to ground. This is measured, with respect to ground, as

(6) $V_{CM} = (T_{xp} + T_{xn}) / 2$.

DC is defined as all frequency components below $F_{DC} = 30$ kHz. AC is defined as all frequency components at or above $F_{DC} = 30$ kHz. These definitions pertain to all voltage and current specifications.

An example waveform is shown in Figure 6-17. In this waveform, the peak-to-peak differential voltage, $V_{DIFF-PP}$ is 800 mV. The differential voltage, V_{DIFF} , is 400 mVPP. Note that while the center crossing point for both Txp and Txn is shown at 300 mV, the corresponding crossover point for the differential voltage is at 0.0 V. The center crossing point at 300 mV is also the common mode voltage, V_{CM} . Note these waveforms include de-emphasis. The actual amount of de-emphasis can vary depending on the transmitter setting according to the allowed ranges in Table 6-17.

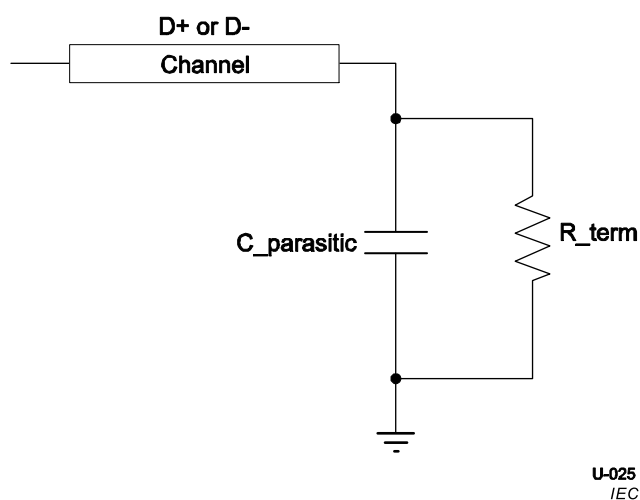


IEC

Figure 6-17 – Single-ended and Differential Voltage Levels

6.6.3 Tx and Rx Input Parasitics

Tx and Rx input parasitics are specified by the lumped circuit shown in Figure 6-18.



U-025
IEC

Figure 6-18 – Device Termination Schematic

In this circuit, the input buffer is simplified to a termination resistance in parallel with a parasitic capacitor. This simplified circuit is the load impedance.

6.7 Transmitter Specifications

6.7.1 Transmitter Electrical Parameters

Peak (p) and peak-peak (p-p) are defined in Section 6.6.2.

Table 6-17 – Transmitter Normative Electrical Parameters

Symbol	Parameter	Gen 1 (5,0 GT/s)	Gen 2 (10 GT/s)	Units	Comments
UI	Unit Interval	199,94 (min) 200,06 (max)	99,97 (min) 100,03 (max)	ps	The specified UI is equivalent to a tolerance of ± 300 ppm for each device. Period does not account for SSC induced variations.
$V_{TX-DIFF-PP}$	Differential p-p Tx voltage swing	0,8 (min) 1,2 (max)	0,8 (min) 1,2 (max)	V	Nominal is 1 V p-p
$V_{TX-DIFF-PP-LOW}$	Low-Power Differential p-p Tx voltage swing	0,4 (min) 1,2 (max)	0,4 (min) 1,2 (max)	V	Refer to Section 6.7.2. There is no de-emphasis requirement in this mode. De-emphasis is implementation specific for this mode.
$V_{TX-DE-RATIO}$	Tx de-emphasis	3,0 (min) 4,0 (max)	Not applicable	dB	Nominal is 3.5 dB for Gen 1 operation. Gen 2 transmitter equalization recommendations are described in section 6.7.5.2.
$R_{TX-DIFF-DC}$	DC differential impedance	72 (min) 120 (max)	72 (min) 120 (max)	Ω	
$V_{TX-RCV-DETECT}$	The amount of voltage change allowed during Receiver Detection	0,6 (max)	0,6 (max)	V	Detect voltage transition should be an increase in voltage on the pin looking at the detect signal to avoid a high impedance requirement when an "off" receiver's input goes below ground.
$C_{AC-COUPLING}$	AC Coupling Capacitor	75 (min) 200 (max)	75 (min) 265 (max)	nF	All Transmitters shall be AC coupled. The AC coupling is required either within the media or within the transmitting component itself.
$t_{CDR_SLEW_MAX}$	Maximum slew rate	10	Not applicable	ms/s	See the jitter white paper for details on this measurement. This is a df/ft specification; refer to Section 6.5.4 for details.
SSC_{dfdt}	SSC df/dt	Not applicable	1 250 (max)	ppm/ μ s	See ¹ .

¹ Measured over a 0.5 μ s interval using CP10. The measurements shall be low pass filtered using a filter with 3 dB cutoff frequency that is 60 times the modulation rate. The filter stopband rejection shall be greater or equal to a second order low-pass of 20 dB per decade. Evaluation of the maximum df/dt is achieved by inspection of the low-pass filtered waveform.

The values in Table 6-18 are informative and not normative. They are included in this document to provide some guidance beyond the normative requirements in Table 6-17 for transmitter design and development. A transmitter can be fully compliant with the normative requirements of the specification and not meet all the values in this table (many of which are immeasurable in a finished product). Similarly, a transmitter that meets all the values in this table is not guaranteed to be in full compliance with the normative part of this specification.

Table 6-18 – Transmitter Informative Electrical Parameters at Silicon Pads

Symbol	Parameter	5,0 GT/s	10 GT/s	Units	Comments
$t_{\text{MIN-PULSE-Dj}}$	Deterministic min pulse	0,96	0,96	UI	Tx pulse width variation that is deterministic
$t_{\text{MIN-PULSE-Tj}}$	Tx min pulse	0,90	0,90	UI	Min Tx pulse at 10^{-12} including Dj and Rj
$t_{\text{TX-EYE}}$	Transmitter Eye	0,625 (min)	0,646 (min)	UI	Includes all jitter sources
$t_{\text{TX-DJ-DD}}$	Tx deterministic jitter	0,205 (max)	0,170 (max)	UI	Deterministic jitter only assuming the Dual Dirac distribution
$C_{\text{TX-PARASITIC}}$	Tx input capacitance for return loss	1,25 (max)	1,1 (max)	pf	Parasitic capacitance to ground
$R_{\text{TX-DC}}$	Transmitter DC common mode impedance	18 (min) 30 (max)	18 (min) 30 (max)	Ω	DC impedance limits to guarantee Receiver detect behavior. Measured with respect to AC ground over a voltage of 0-500 mV.
$I_{\text{TX-SHORT}}$	Transmitter short-circuit current limit	60 (max)	60 (max)	mA	The total current Transmitter can supply when shorted to ground.
$V_{\text{TX-DC-CM}}$	Transmitter DC common-mode voltage	0 (min) 2,2 (max)	0 (min) 2,2 (max)	V	The instantaneous allowed DC common-mode voltages at the connector side of the AC coupling capacitors.
$V_{\text{TX-CM-AC-PP-ACTIVE}}$	Tx AC common mode voltage active	100	100 (max)	mVp-p	Maximum mismatch from Txp + Txn for both time and amplitude.
$V_{\text{TX-CM-DC-ACTIVE-IDLE-DELTA}}$	Absolute DC Common Mode Voltage between U1 and U0	200 (max)	200 (max)	mV	
$V_{\text{TX-IDLE-DIFF-AC-PP}}$	Electrical Idle Differential Peak –Peak Output Voltage	0 (min) 10 (max)	0 (min) 10 (max)	mV	
$V_{\text{TX-IDLE-DIFF-DC}}$	DC Electrical Idle Differential Output Voltage	0 (min) 10 (max)	0 (min) 10 (max)	mV	Voltage shall be low pass filtered to remove any AC component. This limits the common mode error when resuming U1 to U0.

6.7.2 Low Power Transmitter

In addition to the full swing transmitter specification, an optional low power swing transmitter is also specified for SuperSpeed applications. A low power swing transmitter is typically used in systems that are sensitive to power and noise interference, and have a relatively short channel. The requirement as to whether a transmitter needs to support full swing, low power swing, or both swings, is dependent on its usage model. All SuperSpeed transmitters must support full swing, while support for low power swing is optional. The method by which the output swing is selected is not defined in the specification, and is implementation specific.

While two different transmitters are specified, only a single receiver specification is defined. This implies that receiver margins (as specified in Table 6-21) shall be met if a low power transmitter is used.

6.7.3 Transmitter Eye

The eye mask is measured using the compliance data patterns as described in Section 6.4.3.2. The transmitter compliance test for Gen 1 uses compliance patterns CP0 and CP1 for TJ and RJ, respectively. The transmitter compliance test for Gen 2 uses compliance patterns CP9 and CP10 for TJ and RJ, respectively. Eye height is measured for 10^6 consecutive UI. Jitter is extrapolated from 10^6 UI to 10^{-12} BER.

Table 6-19 – Normative Transmitter Eye Mask at Test Point TP1

Signal Characteristic	5GT/s			10GT/s			Units	Note
	Minimum	Nominal	Maximum	Minimum	Nominal	Maximum		
Eye Height	100		1 200	70		1 200	mV	2,4
Dj			0,43			0,530	UI	1,2,3
Rj			0,23			0,184	UI	1,2,3,5
Tj			0,66			0,714	UI	1,2,3

NOTES:

- 1) Measured over 10^6 consecutive UI and extrapolated to 10^{-12} BER.
- 2) Measured after receiver equalization function.
- 3) Measured at end of reference channel and cables at TP1 in Figure 6-19.
- 4) The eye height is to be measured at the minimum opening over the range from the center of the eye $\pm 0,05$ UI.
- 5) The Rj specification is calculated as 14,069 times the RMS random jitter for 10^{-12} BER.

The compliance testing setup is shown in Figure 6-19. All measurements are made at the test point (TP1), and the Tx specifications are applied after processing the measured data with the compliance reference equalizer transfer function described in the next section.

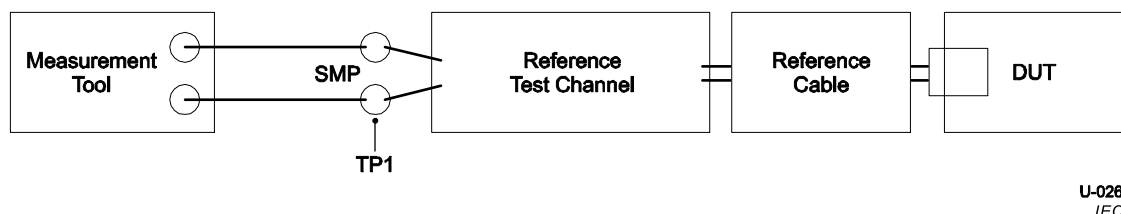


Figure 6-19 – Tx Normative Setup with Reference Channel

6.7.4 Tx Compliance Reference Receiver Equalize Function

The normative transmitter eye is captured at the end of the reference channel. At this point the eye may be closed. To open the eye so it can be measured a reference Rx equalizer, is applied to the signal. Details of the reference equalizer are contained in Section 6.8.2.

6.7.5 Informative Transmitter De-emphasis

6.7.5.1 Gen 1 (5GT/s)

The channel budgets and eye diagrams were derived using a $V_{TX-DE-RATIO}$ of transmit de-emphasis for both the Host and the Device reference channels. An example differential peak-to-peak de-emphasis waveform is shown in Figure 6-20.

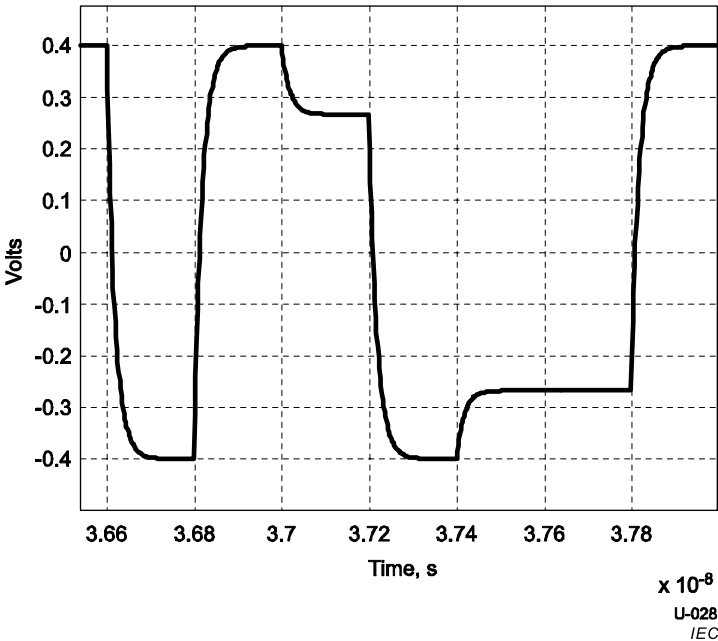
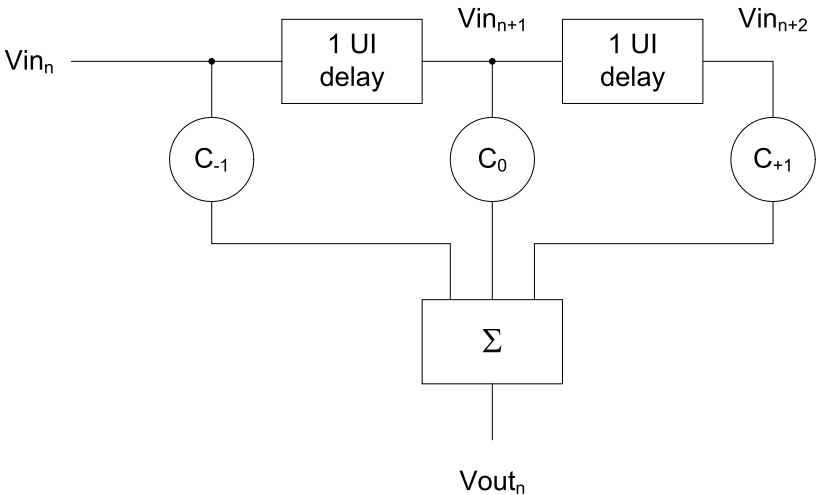


Figure 6-20 – De-Emphasis Waveform

6.7.5.2 Gen 2 (10GT/s)

Gen 2 transmitters employ a 3-tap FIR-based equalizer, the structure of which is shown in Figure 6-21. An example waveform from the 3-tap equalizer is shown in Figure 6-22. In the figure, the pre-cursor (V_c) is referred to as pre-shoot, while the post-cursor (V_b) is referred to as de-emphasis. This convention allows pre-shoot and de-emphasis to be defined independently of one another. The maximum swing, V_d , is also shown to illustrate that, when both C_{+1} and C_{-1} are nonzero, the swing of V_a does not reach the maximum as defined by V_d . Figure 6-22 is shown as an example of TxEQ and is not intended to represent the signal as it would appear for measurement purposes.

Table 6-20 provides recommended (informative) tap coefficient values (C_{-1} and C_1) along with the corresponding pre-shoot, de-emphasis and output amplitudes. The host/device loss referred to in the table refers to the differential insertion loss in the conductor path from the silicon die pad to the connector, and includes parasitic I/O capacitance, the chip package (routing, vias and I/O pins), and printed circuit board (routing and vias).



IEC

Figure 6-21 – 3-tap Transmit Equalizer Structure

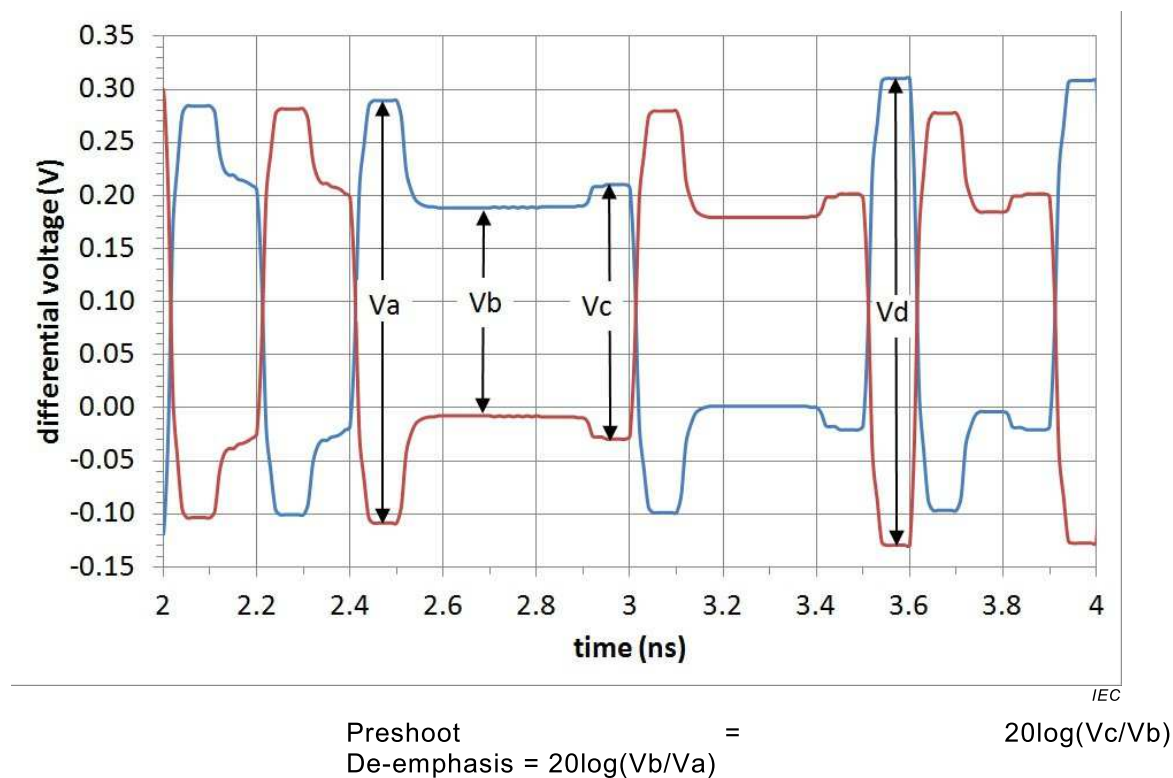


Figure 6-22 – Example Output Waveform for 3-tap Transmit Equalizer

Table 6-20 – Informative Gen 2 Transmitter Equalization Settings

Host/Device Loss @ 5 GHz (dB)	<3,5	≥3,5
C_{-1}	0,000	-0,100
C_1	-0,125	-0,125
Preshoot (dB)	0,0	2,7
De-emphasis (dB)	-2,5	-3,3
V_a/V_d	1,000	0,800
V_b/V_d	0,750	0,550
V_c/V_d	0,750	0,750

6.7.6 Entry into Electrical Idle, U1

Electrical Idle is a steady state condition where the Transmitter Txp and Txn voltages are held constant at the same value and the Receiver Termination is within the range specified by Z_{RX-DC} . Electrical Idle is used in the power saving state of U1.

The low impedance common mode and differential Receiver terminations values (see Table 6-21) must be met in Electrical Idle. The Transmitter can be in either a low or high impedance mode during Electrical Idle.

6.8 Receiver Specifications

6.8.1 Receiver Equalization Training

The receiver equalization training sequence, detailed in Section 6.4.1.1 for Gen 1 operation and in Section 6.4.1.2 for Gen 2 operation, can be used to train the receiver equalizer. The TSEQ training sequence is designed to provide spectrally rich data patterns that are useful for training typical receiver equalization architectures. For Gen 1 operation, a high edge density

pattern is interleaved with the data to help the CDR maintain bit lock. For Gen 2 operation the sequence is deemed sufficiently rich on its own.

During Gen 1 operation the TSEQ training sequence repeats 65 536 times to allow for testing many coefficient settings. Also during Gen 1 operation no SKPs are inserted during the TSEQ training sequence. The frequency spectrum of the TSEQ sequence is shown in Figure 6-23.

During Gen 2 operation, the training period is ~8 ms. The training pattern is periodic with a period of 16 384 132-bit blocks (2162688UI). The much longer pattern greatly increases the richness of the pattern compared to Gen 1. The Gen 2 training pattern spectrum is essentially white. Due to the length of the Gen 2 training interval and the potential desire to examine the data, SKPs are inserted during polling.TSEQ.

Receiver equalization training is implementation specific.

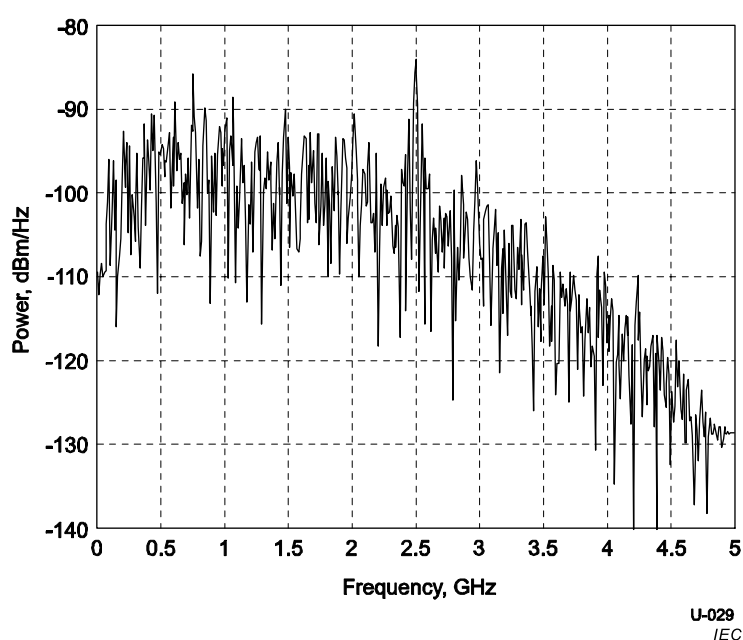


Figure 6-23 – Frequency Spectrum of TSEQ

6.8.2 Informative Receiver CTLE Function

USB 3.1 allows the use of receiver equalization to meet system timing and voltage margins. For long cables and channels the eye at the Rx is closed, and there is no meaningful eye without first applying an equalization function. The Rx equalizer may be required to adapt to different channel losses using the Rx EQ training period. The exact Rx equalizer and training method is implementation specific.

6.8.2.1 Gen 1 Reference CTLE

The equation for the continuous time linear equalizer (CTLE) used to develop the specification is the compliance Rx EQ transfer function described below.

$$H(s) = \frac{A_{dc} \omega_{p1} \omega_{p2}}{\omega_z} \cdot \frac{s + \omega_z}{(s + \omega_{p1})(s + \omega_{p2})} \quad (10)$$

where

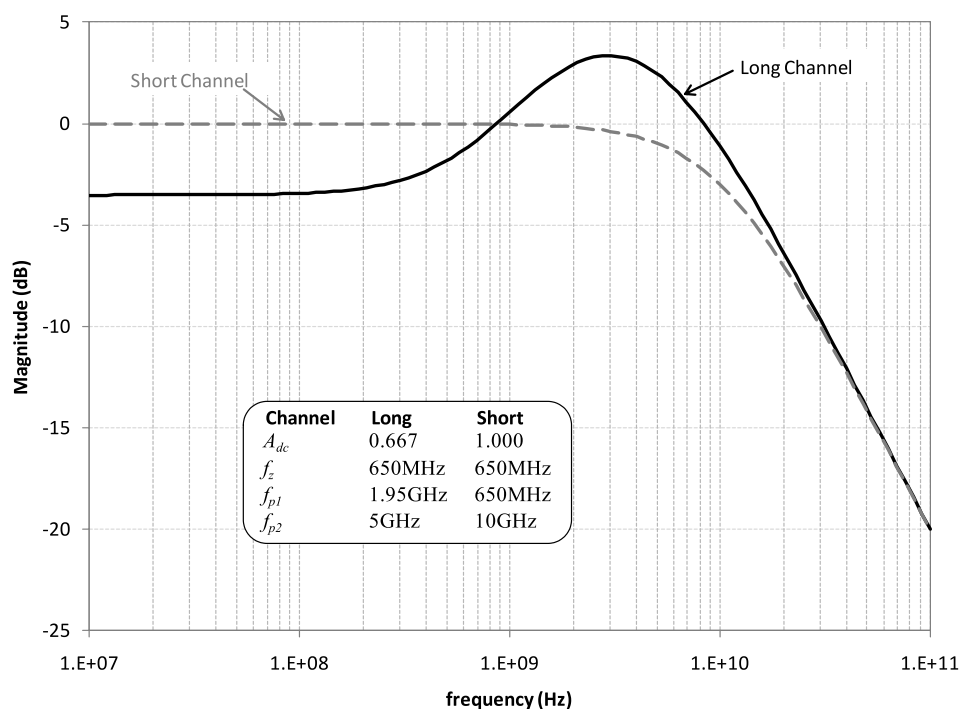
A_{dc} is the DC gain

$\omega_z = 2\pi f_z$ is the zero frequency

$\omega_{p1} = 2\pi f_{p1}$ is the first pole frequency

$\omega_{p2} = 2\pi f_{p2}$ is the second pole frequency

Figure 6-24 is a plot of the Compliance EQ transfer functions with the values for each of the input parameters.



IEC

Figure 6-24 – Gen 1 Tx Compliance Rx EQ Transfer Function

6.8.2.2 Gen 2 Reference Equalizer Function

6.8.2.2.1 Reference CTLE

Equation (11) describes the frequency response for the Gen 2 reference continuous time linear equalizer (CTLE) that is used for compliance testing. The equation describes the same first order CTLE as contained in Equation (10).

$$H(s) = A_{ac} \omega_{p2} \frac{s + \frac{A_{dc}}{A_{ac}} \omega_{p1}}{(s + \omega_{p1})(s + \omega_{p2})} \quad (11)$$

Where

A_{ac} is the high frequency peak gain

A_{dc} is the DC gain

$\omega_{p1} = 2\pi f_{p1}$ is the first pole frequency

$\omega_{p2} = 2\pi f_{p2}$ is the second pole frequency

Figure 6-25 is a plot of the Compliance EQ transfer functions with the values for each of the input parameters.

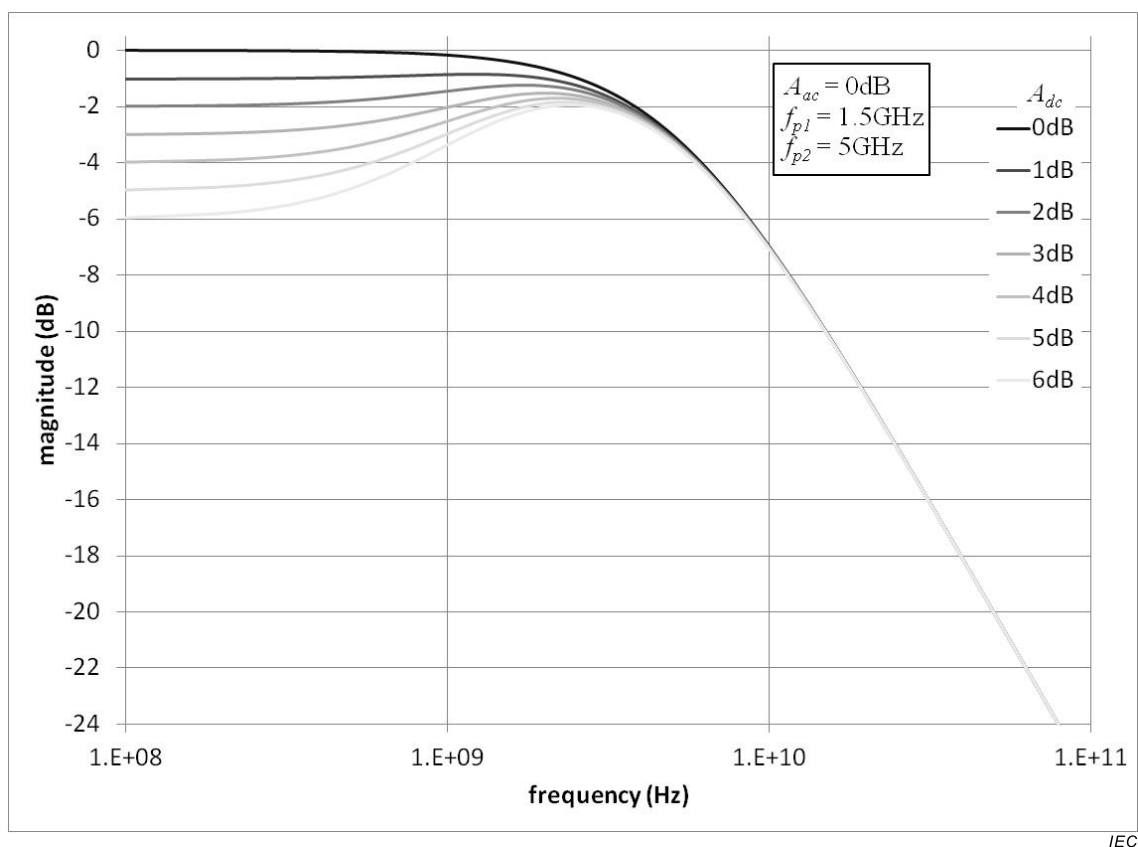


Figure 6-25 – Gen 2 Compliance Rx EQ Transfer Function

6.8.2.2.2 Reference DFE

In addition to the 1st order CTLE, a one-tap reference DFE is used in transmitter compliance testing. The DFE behavior is described by equation (12) and Figure 6-26. The limits on d_1 are 0 to 50 mV.

$$y_k = x_k - d_1 \operatorname{sgn}(y_{k-1}) \quad (12)$$

Where

- y_k is the DFE differential output voltage
- y_k^* is the decision function output voltage, $|y_k^*| = 1$
- x_k is the DFE differential input voltage
- d_1 is the DFE feedback coefficient
- k is the sample index in UI

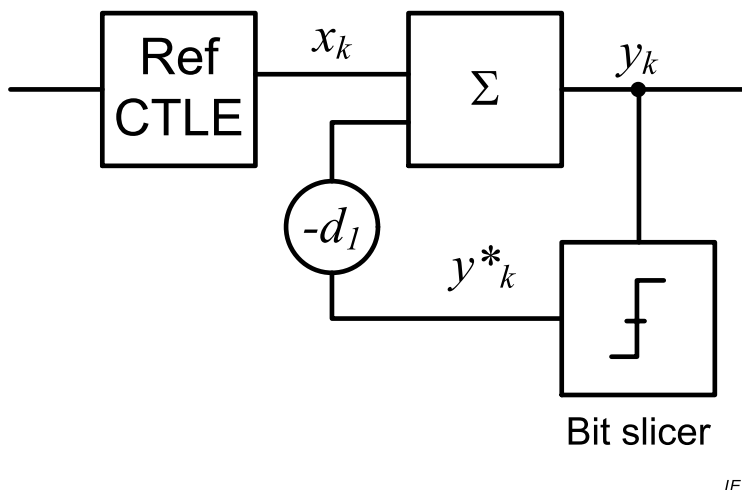


Figure 6-26 – Gen 2 reference DFE Function

6.8.3 Receiver Electrical Parameters

Normative specifications are to be measured at the connector. Peak (p) and peak- peak (p-p) are defined in Section 6.6.2.

Table 6-21 – Receiver Normative Electrical Parameters

Symbol	Parameter	Gen 1 (5.0 GT/s)	Gen 2 (10 GT/s)	Units	Comments
UI	Unit Interval	199,94 (min) 200,06 (max)	99,97 (min) 100,03 (max)	ps	UI does not account for SSC caused variations.
R_{RX-DC}	Receiver DC common mode impedance	18 (min) 30 (max)	18 (min) 30 (max)	Ω	DC impedance limits are needed to guarantee Receiver detect. Measured with respect to ground over a voltage of 500 mV maximum.
$R_{RX-DIFF-DC}$	DC differential impedance	72 (min) 120 (max)	72 (min) 120 (max)	Ω	
$Z_{RX-HIGH-IMP-DC-POS}^1$	DC Input CM Input Impedance for $V > 0$ during Reset or power down	25k (min)	25k (min)	Ω	Rx DC CM impedance with the Rx terminations not powered, measured over the range 0 – 500 mV with respect to ground.
$V_{RX-LFPS-DET-DIFFp-p}$	LFPS Detect Threshold	100 (min) 300 (max)	100 (min) 300 (max)	mV	Below the minimum is noise. Must wake up above the maximum.

¹ Only DC Input CM Input Impedance for $V > 0$ is specified. DC Input CM Input Impedance for $V < 0$ is not guaranteed and could be as low as 0 Ω .

The values in Table 6-22 are informative and not normative. They are included in this document to provide some guidance beyond the normative requirements in Table 6-21 for receiver design and development. A receiver can be fully compliant with the normative requirements of the specification and not meet all the values in this table (many of which are not measurable in a finished product). Similarly, a receiver that meets all the values in this table is not guaranteed to be in full compliance with the normative part of this specification.

Table 6-22 – Receiver Informative Electrical Parameters

Symbol	Parameter	Gen 1 (5.0 GT/s)	Gen 2 (10 GT/s)	Units	Comments
$V_{RX-DIFF-PP-POST-EQ}$	Differential Rx peak-to-peak voltage	30 (min)	30 (min)	mV	Measured after the Rx EQ function (Section 6.8.2).
t_{RX-TJ}	Max Rx inherent timing error	0,45 (max)	0,394 (max)	UI	Measured after the Rx EQ function (Section 6.8.2).
$t_{RX-DJ-DD}$	Max Rx inherent deterministic timing error	0,285 (max)	0,21 (max)	UI	Maximum Rx inherent deterministic timing error
$C_{RX-PARASITIC}$	Rx input capacitance for return loss	1,1 (max)	1,0 (max)	pF	
$V_{RX-CM-AC-P}$	Rx AC common mode voltage	150 (max)	150 (max)	mV Peak	Measured at Rx pins into a pair of 50 Ω terminations into ground. Includes Tx and channel conversion, AC range up to 5 GHz
$V_{RX-CM-DC-ACTIVE-IDLE-DELTA_P}$	Rx AC common mode voltage during the U1 to U0 transition	200 (max)	200 (max)	mV Peak	Measured at Rx pins into a pair of 50 Ω terminations into ground. Includes Tx and channel conversion, AC range up to 5 GHz

6.8.4 Receiver Loopback

The entry and exit process for receiver loopback is described in Chapter 7.

Receiver loopback must be retimed. Direct connection from the Rx amplifier to the transmitter is not allowed for loopback mode. The receiver shall continue to process SKPs as appropriate. SKP symbols shall be consumed or inserted as required for proper clock tolerance compensation. Over runs or under runs of the clock tolerance buffers will reset the buffers to the neutral position.

During loopback the receiver shall process the Bit Error Rate Test (BERT) commands.

Loopback shall occur in the 10-bit domain for Gen 1 operation and in the 132-bit domain for Gen 2 operation. No error correction is allowed. All symbols shall be transmitted as received with the exception of SKP and BERT commands.

6.8.4.1 Loopback BERT for Gen 1 Operation

During loopback the receiver processes the BERT ordered sets BRST, BDAT, and BERC. These ordered sets are described in Table 6-23 through Table 6-26. BRST and BDAT are looped back as received. BERC ordered sets are not looped back but are replaced with BCNT ordered sets. Any time a BRST is received, the error count register EC is set to 0 and the scrambling LFSR is set to 0FFFFh. Any number of consecutive BRST ordered sets may be received.

BRST followed by BDAT starts the bit error rate test. The BDAT sequence is the output of the scrambler and is equivalent to the logical idle sequence. It consists of scrambled 0 as described in Appendix B. As listed in Appendix B, the first 16 characters of the sequence are reprinted here:

FF	17	C0	14	B2	E7	02	82	72	6E	28	A6	BE	6D	BF	8D
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

The receiver shall compare the received data to the BDAT sequence. Errors increment the error count register (EC) by 1. EC may not roll over but shall be held at FFh. The LFSR is advanced once for every character except SKPs. The LFSR rolls over after $2^{16} - 1$ symbols. SKPs shall be inserted or deleted as necessary for clock tolerance compensation.

The BERC command does not increment the error count register. The LFSR is advanced. The BERC ordered set is replaced by the BCNT ordered set. The BCNT ordered set includes the non-scrambled 8b/10b encoded error count (EC) register based on the running disparity. Following the return of the BCNT ordered set, the loopback slave shall continue to repeat symbols as received.

BERC may be sent multiple times. The EC register is not cleared by BERC ordered sets.

BERT continues until the loopback mode is terminated as described in Chapter 7.

Table 6-23 – BRST

Symbol Number	Encoded Values	Description
0	K28.5	COM
1	K28.7	BRST

Table 6-24 – BDAT

Symbol Number	Encoded Values	Description
D0.0 <0:n>	Logical Idle (refer to Appendix B)	Scrambled 0 Rolls over after $2^{16} - 1$ symbols

Table 6-25 – BERC

Symbol Number	Encoded Values	Description
0	K28.3	BERC
1	K28.3	BERC
2	K28.3	BERC
3	K28.3	BERC

Table 6-26 – BCNT

Symbol Number	Encoded Values	Description
0	K28.3	BERC
0	K28.3	BERC
EC<0:7>	DCODE	Error count (not scrambled)
EC<0:7>	DCODE	Error count (not scrambled)

6.8.5 Normative Receiver Tolerance Compliance Test

The receiver tolerance test is tested using the appropriate compliance reference channel for Gen 1 or Gen 2 operation depending upon the rate being tested. A pattern generator shall send the rate appropriate compliance test pattern with added jitter through the compliance reference channels to the receiver. The receiver shall loop back the data and any difference in the pattern sent from the pattern generator and returned will be an error. When running the compliance tests, the receiver shall be put into loopback mode.

Additional details on the receiver compliance test are contained in the reference document, *USB SuperSpeed Compliance Methodology*.

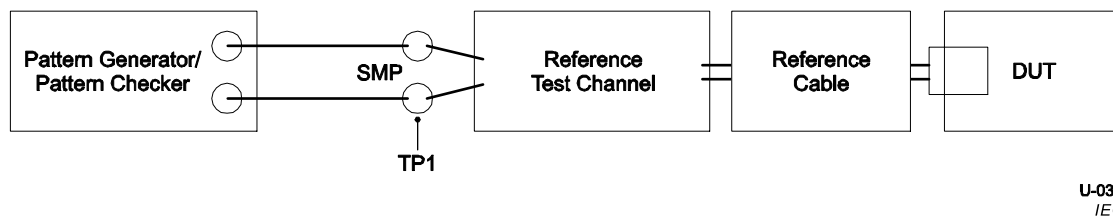


Figure 6-27 – Rx Tolerance Setup

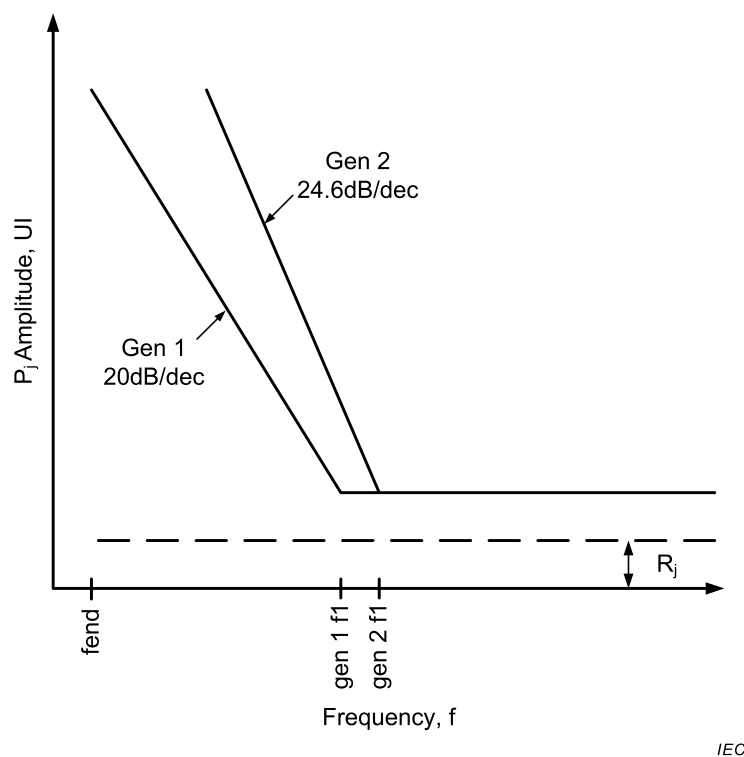


Figure 6-28 – Jitter Tolerance Curve

The jitter components used to test the receiver shall meet the requirements of Table 6-27.

Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing

Symbol	Parameter	Gen 1 (5GT/s)	Gen 2 (10GT/s)	Units	Notes
f1	Tolerance corner	4,9	7,5	MHz	
J _{Rj}	Random Jitter	0,0121	0,01308	UI rms	1
J _{Rj_p-p}	Random Jitter peak- peak at 10 ⁻¹²	0,17	0,184	UI p-p	1,4
J _{Pj_500kHz}	Sinusoidal Jitter	2	4,76	UI p-p	1,2,3
J _{Pj_1Mhz}	Sinusoidal Jitter	1	2,03	UI p-p	1,2,3
J _{Pj_2MHz}	Sinusoidal Jitter	0,5	0,87	UI p-p	1,2,3
J _{Pj_4MHz}	Sinusoidal Jitter	N/A	0,37	UI p-p	1,2,3
J _{Pj_f1}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_50MHz}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_100MHz}	Sinusoidal Jitter	N/A	0,17	UI p-p	1,2,3
V _{full_swing}	Transition bit differential voltage swing	0,75	0,8	V p-p	1
V _{EQ_level}	Non transition bit voltage (equalization)	-3	Preshoot=2,7 De-emphasis= -3,3	dB	1

NOTES:

- 1) All parameters measured at TP1. The test point is shown in Figure 6-19.
- 2) Due to time limitations at compliance testing, only a subset of frequencies can be tested. However, the Rx is required to tolerate Pj at all frequencies between the compliance test points.
- 3) During the Rx tolerance test, SSC is generated by test equipment and present at all times. Each J_{Pj} source is then added and tested to the specification limit one at a time.
- 4) Random jitter is also present during the Rx tolerance test, though it is not shown in Figure 6-20.
- 5) The JTOL specs for Gen 2 comprehend jitter peaking with re-timers in the system and has a 25 dB/decade slope.

6.9 Low Frequency Periodic Signaling (LFPS)

Low frequency periodic signaling (LFPS) is used for side band communication between the two ports across a link that is in a low power link state. It is also used when a link is under training, or when a downstream port issues Warm Reset to reset the link.

6.9.1 LFPS Signal Definition

Table 6-28 defines the LFPS electrical specification at the transmitter. An example differential LFPS waveform is shown in Figure 6-29. t_{Period} is the period of an LFPS cycle. An LFPS burst is the transmission of continuous LFPS signal over a period of time defined by t_{Burst}. An LFPS sequence is defined by the transmission of a single LFPS burst of duration t_{Burst} over a period of time defined by t_{Repeat}. The link is in electrical idle between the two contiguous LFPS bursts.

An LFPS message is encoded based on the variation of t_{Burst}. t_{Repeat} is defined as a time interval when the next LFPS message is transmitted. The LFPS messages include Polling.LFPS and Ping.LFPS, as defined in Table 6-29. There are also LFPS signaling defined by a for U1/U2 and Loopback exit, U3 wakeup, and Warm Reset.

The detailed use of LFPS signaling is specified in the following sections and Chapter 7.

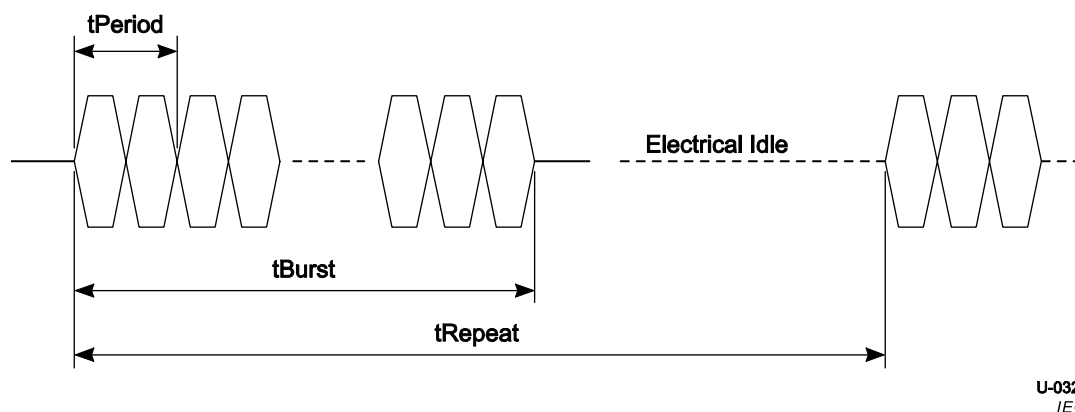


Figure 6-29 – LFPS Signaling

Table 6-28 – Normative LFPS Electrical Specification

Symbol	Minimum	Typical	Maximum	Units	Comments
tPeriod	20		100	ns	
tPeriod for SuperSpeedPlus	20		80	ns	
$V_{CM-AC-LFPS}$			$V_{TX-CM-AC-PP-ACTIVE}$	mV	See Table 6-18
$V_{CM-LFPS-Active}$			10	mV	
$V_{TX-DIFF-PP-LFPS}$	800		1 200	mV	Peak-peak differential amplitude
$V_{TX-DIFF-PP-LFPS-LP}$	400		600	mV	Low power peak-peak differential amplitude
tRiseFall2080			4	ns	Measured at compliance TP1, as shown in Figure 6-19.
Duty cycle	40		60	%	Measured at compliance TP1, as shown in Figure 6-19.

Table 6-29 – LFPS Transmitter Timing for SuperSpeed Designs¹

	tBurst				tRepeat		
	Min	Typ	Max	Minimum Number of LFPS Cycles ²	Min	Typ	Max
Polling.LFPS	0,6 µs	1,0 µs	1,4 µs		6 µs	10 µs	14 µs
Ping.LFPS ⁸	40 ns		200 ns	2	160 ms	200 ms	240 ms
Ping.LFPS for SuperSpeedPlus ⁹	40 ns		160 ns	2			
tReset ³	80 ms	100 ms	120 ms				
U1 Exit ^{4,5}	600 ns ⁷		2 ms				
U2 / Loopback Exit ^{4,5}	80 µs ⁷		2 ms				
U3 Wakeup ^{4,5}	80 µs ⁷		10 ms				

¹ If the transmission of an LFPS signal does not meet the specification, the receiver behavior is undefined.

² Only Ping.LFPS has a requirement for minimum number of LFPS cycles.

³ The declaration of Ping.LFPS depends on only the Ping.LFPS burst.

⁴ Warm Reset, U1/U2/Loopback Exit, and U3 Wakeup are all single burst LFPS signals. tRepeat is not applicable.

⁵ The minimum duration of an LFPS burst shall be transmitted as specified. The LFPS handshake process and timing are defined in Section 6.9.2.

⁶ A Port in U2 or U3 is not required to keep its transmitter DC common mode voltage. When a port begins U2 exit or U3 wakeup, it may start sending LFPS signal while establishing its transmitter DC common mode voltage. To make sure its link partner receives a proper LFPS signal, a minimum of 80 µs tBurst shall be transmitted. The same consideration also applies to a port receiving LFPS U2 exit or U3 wakeup signal.

⁷ A port is still required to detect U1 LFPS exit signal at a minimum of 300 ns. The extra 300 ns is provided as the guard band for successful U1 LFPS exit handshake.

⁸ This requirement applies to SuperSpeed only designs (are only capable of operating at 5 Gb/s).

⁹ This requirement applies to SuperSpeedPlus designs (capable of operating at 10 Gb/s and higher speeds).



IMPLEMENTATION NOTE

Detect and differentiate between Ping.LFPS and U1 LFPS exit signaling for a downstream port in U1 or U2

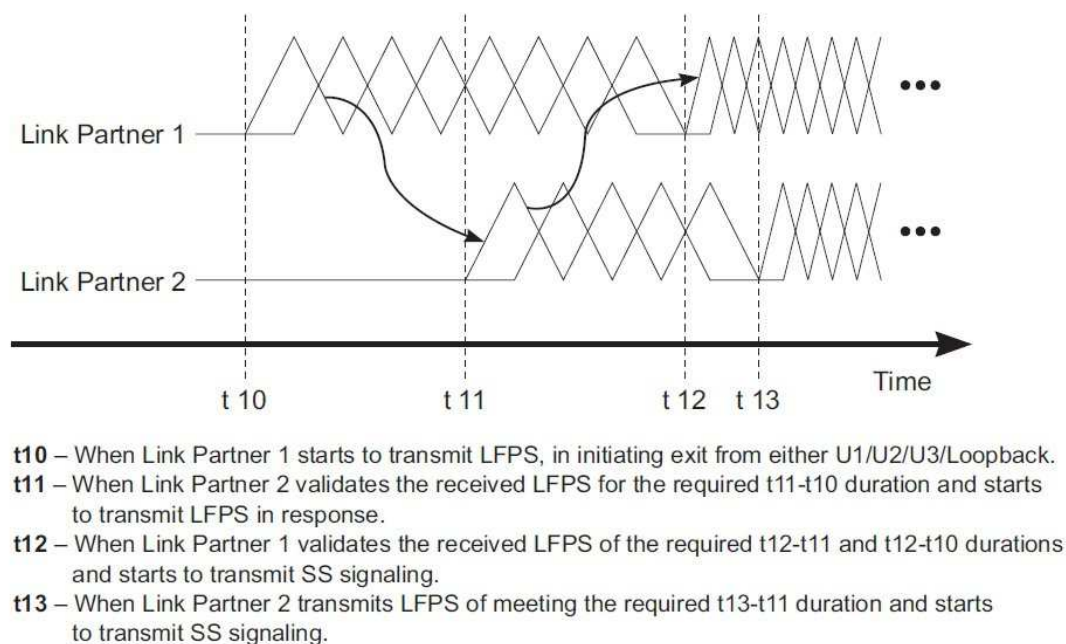
When a downstream port is in U1, it may receive either a Ping.LFPS as a message from its link partner to inform its presence, or an U1 LFPS exit signal to signal that its link partner is attempting exit from U1. This will also occur when a downstream port is in U2, since there are situations where a downstream port enters U2 from U1 when its U2 inactivity timer times out, and its link partner is still in U1.

Upon detecting the break of electrical idle due to receiving an LFPS signal, a downstream port may start a timer to measure the duration of the LFPS signal. If an electrical idle condition does not occur when the timer expires at 300 ns, a downstream port can declare the received LFPS signal is U1 exit and then respond to U1 exit by sending U1 LFPS exit handshake signal. If an electrical idle condition is detected before the timer reaches 300 ns, a downstream port can declare that the received LFPS signal is Ping.LFPS.

6.9.2 Example LFPS Handshake for U1/U2 Exit, Loopback Exit, and U3 Wakeup

The LFPS signal used for U1/U2 exit, Loopback exit, and U3 wakeup is defined the same as continuous LFPS signals with the exception of timeout values defined in Table 6-30. The handshake process for U1/U2 exit and U3 wakeup is illustrated in Figure 6-30. The timing

requirements are different for U1 exit, U2 exit, Loopback exit, and U3 wakeup. They are listed in Table 6-30.



U-033A

IEC

Figure 6-30 – U1 Exit, U2 Exit, and U3 Wakeup LFPS Handshake Timing Diagram

NOTE The timing diagram in Figure 6-22 is for illustration of the LFPS handshake process only.

The handshake process is as follows:

- Link partner 1 initiates exit by transmitting LFPS at time t10 (see Figure 6-30). LFPS transmission shall continue until the handshake is declared either successful or failed.
- Link partner 2 detects valid LFPS on its receiver and responds by transmitting LFPS at time t11. LFPS transmission shall continue until the handshake is declared either successful or failed.
- A successful handshake is declared for link partner 1 if the following conditions are met within “tNoLFPSResponseTimeout” after t10 (see Figure 6-30 and Table 6-30):
 - 1) Valid LFPS is received from link partner 2.

NOTE In case of concurrent U1 exit, where both ports initiate U1 exit simultaneously, both ports will assume to be Link partner 1. Both ports will start receiving LFPS signal before t10. And received U1 LFPS exit signal may be validated around t10. This may result in a minimum duration of U1 exit LFPS signal. To ensure successful U1 exit under such situations, both ports shall transmit U1 LFPS exit signal for 600 ns before exiting U1.

- 2) For U1 exit, U2 exit, U3 Wakeup and not Loopback exit, link partner 1 is ready to transmit the training sequences and the maximum time gap after an LFPS transmitter stops transmission and before a SuperSpeed transmitter starts transmission is 20 ns.

NOTE There is no Near End Cross Talk (NEXT) specification for SuperSpeed transmitters and receivers. Therefore, when a port enters Recovery and starts transmitting TS1 Ordered Sets and its link partner is in electrical idle after successful LFPS handshake, a port may potentially train its receiver using its own TS1 Ordered Sets due to NEXT. The intention of adding the second exit condition is to prevent a port from electrical idle before transitioning to Recovery.

- A successful handshake is declared for link partner 2 if the following conditions are met:
 - 1) Link partner 2 has transmitted the minimum LFPS defined as (t13 – t11) in Table 6-30.
 - 2) For U1 exit, U2 exit, U3 Wakeup, and not Loopback exit, link partner 2 is ready to transmit the training sequences and the maximum time gap after an LFPS transmitter stops transmission and before a SuperSpeed transmitter starts transmission is 20 ns.

- A U1 exit, U2 exit, Loopback exit, and U3 wakeup handshake failure shall be declared if the conditions for a successful handshake are not met.
- Link partner 1 shall declare a failed handshake if its successful handshake conditions were not met.
- Link partner 2 shall declare a failed handshake if its successful handshake conditions were not met.

NOTE Except for Ping.LFPS, when an upstream port in Ux or Loopback.Active receives an LFPS signal, it shall proceed with U1/U2 exit, or U3 wakeup, or Loopback exit handshake even if the LFPS is later determined to be a Warm Reset. If the LFPS is a Warm Reset, an upstream port, if in Ux, will enter Recovery and then times out to SS.Inactive, or if in Loopback Active, will enter Rx.Detect and then transitions to Polling.LFPS. When Warm Reset is detected, an upstream port will enter Rx.Detect.

Table 6-30 – LFPS Handshake Timing for U1/U2 Exit, Loopback Exit, and U3 Wakeup

	U1 Exit		U2/Loopback Exit		U3 Wakeup	
	Min	Max	Min	Max	Min	Max
t11 – t10	0,3 μ s	0,9 μ s/2 ms ¹ –900 ns)	0,3 μ s	2 ms	0,3 μ s	10 ms
t13 – t10	0,9 μ s	2ms		2 ms		20 ms
t12 – t11	0,3 μ s	0,9 μ s ¹	0	2 ms	0	10 ms
t13 – t11	0,6 μ s	0,9 μ s ¹	80 μ s	2 ms	80 μ s	10 ms
t12 – t10	0,6 μ s	2ms ¹	80 μ s	2 ms	80 μ s	10 ms
tNoLFPSResponseTimeout		2 ms		2 ms		10 ms

NOTE:

- 1) There are two sets of maximum timing requirements. The set with short timing requirement applies to normal operations when U2_Inactivity_Timer is disabled. The set with relaxed timing requirement applies to operations when U2_Inactivity_Timer is enabled. It also includes one corner case where U2_Inactivity_Timer is disabled and the port, upon entry to U1, initiating U1 exit immediately.
- 2) In a case where U2_Inactivity_Timer is enabled, it is the responsibility of each link partner to respond accordingly depending on its U1 or U2 state. For example, when link partner 1 initiates exit in U1 and link partner 2 is in U2, it is expected that both link partners will eventually enter U0 with respective timings starting from different U1/U2 states. Essentially, t12-t10 of link partner 1 follows U1 Exit tBurst timing and t13-t11 of link partner 2 follows U2 Exit tBurst timing.

6.9.3 Warm Reset

A Warm Reset is a reset generated only by a downstream port to an upstream port. A downstream port may issue a Warm Reset at any Link states except SS.Disabled. An upstream port is required to detect a Warm Reset at any link states except SS.Disabled.

NOTE Warm Reset is defined to be able to reset a hardware failure of a device, such as the LTSSM hanging. Under this assumption, Warm Reset may be detected in any link states except SS.Disabled.

A Warm Reset shares the same continuous LFPS signal as a low power Link state exit handshake signal. In order for an upstream port to be able to differentiate between the two signals, the tBurst of a Warm Reset is extended, as is defined in Table 6-20.

The Warm Reset assertion is asynchronous between a downstream port and an upstream port since it has to take a certain period of time for an upstream port to declare that a Warm Reset is detected. However, the de-assertion of the Warm Reset between a downstream port and an upstream port shall be made synchronous. Figure 6-31 shows a timing diagram of Warm Reset generation and detection when a port is U3. Once a Warm Reset is issued by a downstream port, it will take at least tResetDelay for an upstream port to declare the detection of Warm Reset. Once a Warm Reset is detected, an upstream port shall continue to assert the Warm Reset until it no longer receives any LFPS signals from a downstream port.

- An upstream port shall declare the detection of Warm Reset within $t_{ResetDelay}$. The minimum $t_{ResetDelay}$ shall be 18 ms; the maximum $t_{ResetDelay}$ shall be 50 ms.

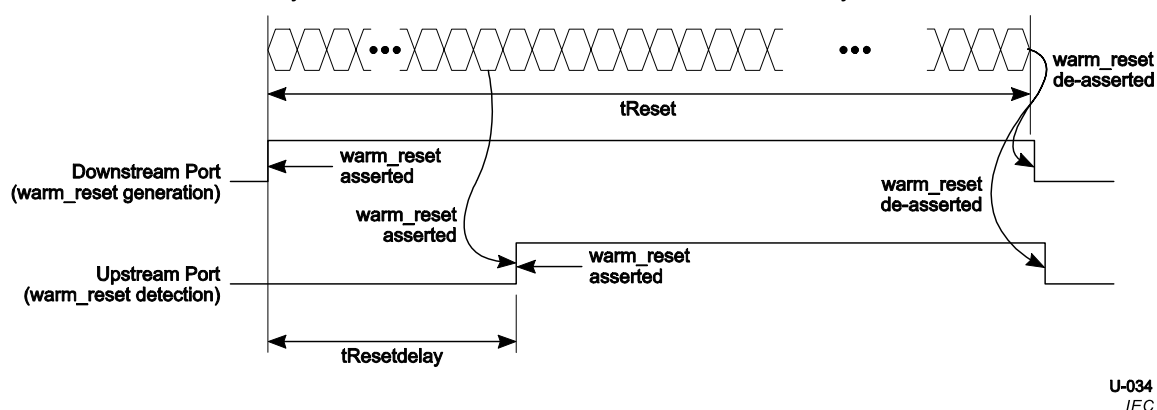


Figure 6-31 – Example of Warm Reset Out of U3

6.9.4 SuperSpeedPlus Capability Declaration

SuperSpeedPlus Capability Declaration (SCD) is a step for a SuperSpeedPlus port, while in the Polling.LFPS substate, to identify itself as SuperSpeedPlus capable by transmitting Polling.LFPS signals with specific patterns unique to SuperSpeedPlus ports. This section defines SuperSpeedPlus specific patterns in SCD1 and SCD2. The use of SCD1 and SCD2 is described in Chapter 7.

6.9.4.1 Binary Representation of Polling.LFPS

Binary representation of Polling.LFPS refers to logic presentation based on Polling.LFPS signal by the measure of t_{Repeat} duration. As shown in Table 6-31, logic '0' is represented with t_{Repeat} varying between 6~9 μs , and logic '1' is represented with t_{Repeat} varying between 11~14 μs . t_{Repeat} between 9~11 μs is defined as a guard band. Figure 6-32 is an example of Polling.LFPS based binary representation in the time domain.

Table 6-31 – Binary Representation of Polling.LFPS

t_{Repeat} (μs)	Logic value
6~9	'0'
11~14	'1'
9~11	illegal

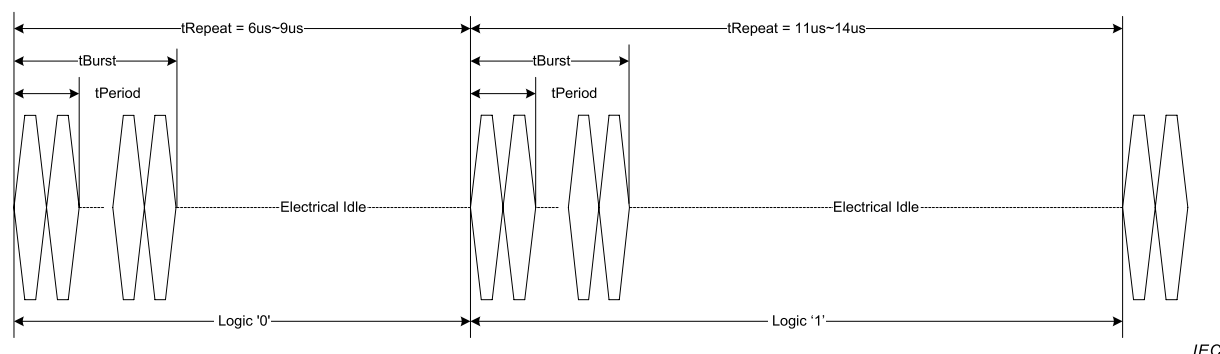


Figure 6-32 – Example of Binary Representation based on Polling.LFPS

6.9.4.2 SCD1/SCD2 Definitions and Transmission

SCD1 is defined as “0010” and SCD2 is defined as “1101”. The transmission of SCD1/SCD2 shall be based on the following.

- The transmission shall be LSb first, and consecutive SCD1/SCD2 shall be transmitted back to back.
- The transmission shall be completed with an extra t_{Burst} followed by electrical idle (EI) of at least 2x the maximum allowable t_{Repeat} value.

Figure 6-33 is an example of SCD1/SCD2 waveform in the time domain.

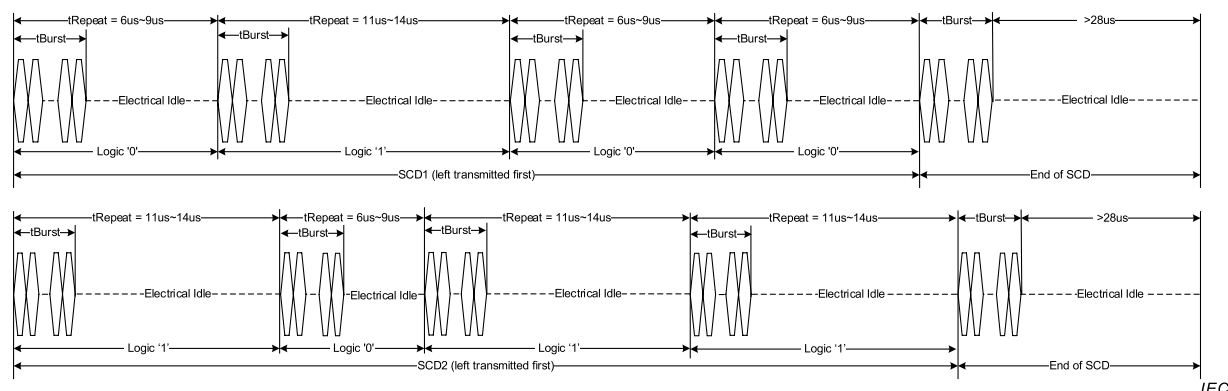


Figure 6-33 – SCD1/SCD2 transmission

6.9.5 SuperSpeedPlus LFPS Based PWM Message (LBPM)

LBPM is defined as a low power signaling mechanism for two SuperSpeedPlus ports to communicate with each other based on LFPS signals. The adoption of Pulse Width Modulation (PWM) is to embed the transmitting clock in data and to allow for easy data recovery at the receiver based on LFPS clock defined in Table 6-28. This section describes the concept and construction of LBPM. Refer to Chapter 7 for use of LBPM.

6.9.5.1 Introduction to LFPS Based PWM Signaling (LBPS)

LBPS is based on PWM with embedded transmit clock and is basically constructed with two distinctive electrical states, which are LFPS signaling state and EI state. As is shown in Figure 6-34, two logic states are defined based on LBPS.

- Logic ‘0’ is defined within the unit interval of t_{PWM} as one-third of LFPS signal followed by two-third of EI.
- Logic ‘1’ is defined within the unit interval of t_{PWM} as two-third of LFPS signal followed by one-third of EI.

The specification of the transmit and receive LBPS is defined in Table 6-32.

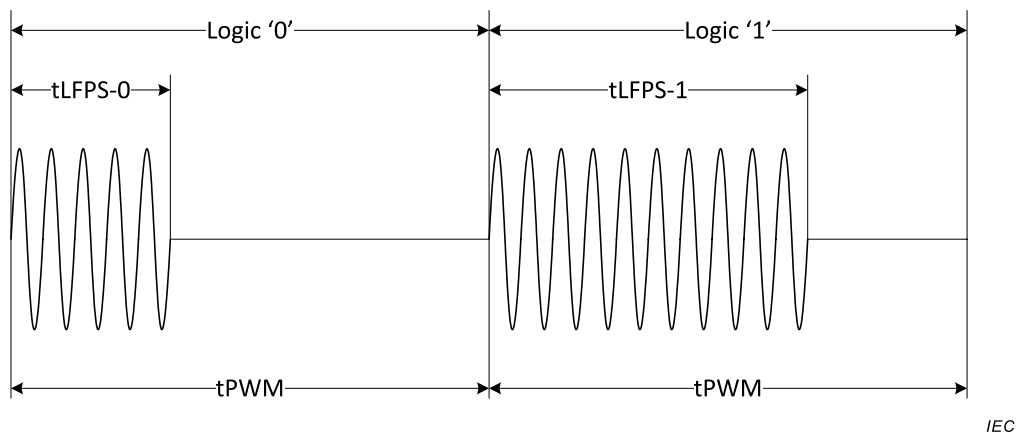


Figure 6-34 – Logic Representation of LBPS

Table 6-32 – LBPS Transmit and Receive Specification

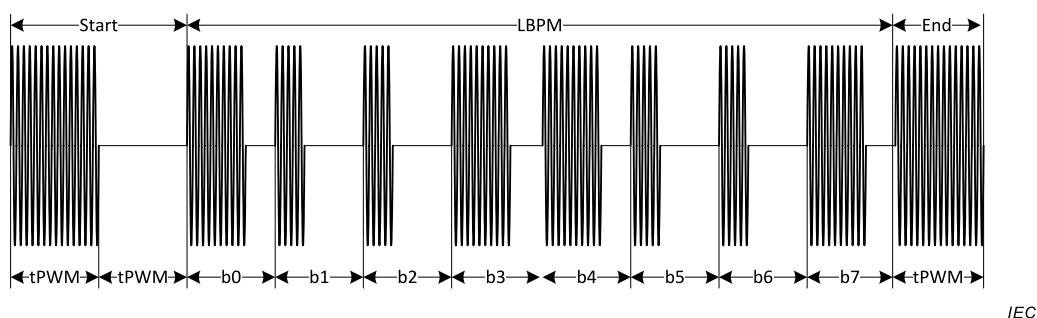
	Unit	Transmit			Receive		
		Min	TYP	Max	Min	TYP	Max
tPWM	μs	2	2,2	2,4			
tLFPS-0	μs	0,5		0,80	0,45		0,85
tLFPS-1	μs	1,33		1,80	1,28		1,85

6.9.5.2 LBPM Definition and Transmission

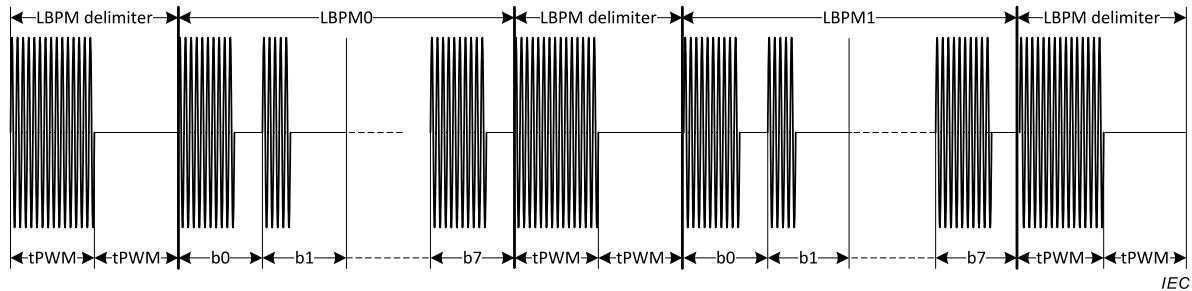
LBPM is byte based with LSb transmitted first. A port may transmit a single LBPM, or consecutive LBPMs. The transmission of LBPM shall adhere to the following conventions.

- A LBPM delimiter is defined with one tPWM of LFPS followed by one tPWM of EI.
- The LPBM transmission shall start with a LBPM delimiter.
- The LBPM transmission shall end with a LBPM delimiter.
- The transmission of LBPM shall be LSb first.
- Consecutive LBPM transmission shall be LSB first with LBPM delimiter in between each LBPMs.

Examples of LBPM transmission are shown in Figure 6-35.



(a). Single LBPM Transmission



(b). Consecutive LBPMs Transmitted Back to Back

Figure 6-35 – LBPM Transmission Examples

6.10 Transmitter and Receiver DC Specifications

6.10.1 Informative ESD Protection

It is recommended that all signal and power pins withstand a minimum ESD value using the human body model and the charged device model, without damage, as defined by the semiconductor industry.

This is a suggested ESD tolerance. The ASIC designer is expected to apply current technology and knowledge of ESD prevention circuits to protect from environmental hazards that could create long term failure, and possibly, catastrophic failure in a system. It is up to the designer to use good design practices to implement appropriate ESD protection. With ESD protection in place, the ASIC design shall meet the other electrical requirements of this specification.

6.10.2 Informative Short Circuit Requirements

All Transmitters and Receivers shall support surprise hot insertion/removal without damage to the component. The Transmitter and Receiver shall be capable of withstanding sustained short circuit to ground of Txp (Rxp) and Txn (Rxn).

6.10.3 Normative High Impedance Reflections

During an asynchronous reset event, one device may be reset while the other device is transmitting. The device under reset is required to disconnect the receiver termination. During this time, the device under reset may be receiving active data. Since the data is not terminated, the differential voltage into the receiver will be doubled. For a short channel, the receiver may experience a total of $2 \times V_{DIFF}$.

The receiver shall tolerate this doubling of the negative voltage that can occur if the Rx termination is disconnected. A part shall tolerate a 20 ms event that doubles the voltage on the receiver input when the termination is disconnected 10 000 times over the life time of the part.

6.11 Receiver Detection

6.11.1 Rx Detect Overview

The Receiver Detection circuit is implemented as part of a Transmitter and shall correctly detect whether a load impedance equivalent to a DC impedance R_{RX-DC} (Table 6-21) is present. The Rx detection operates on the principle of the RC time constant of the circuit. This time constant changes based on the presence of the receiver termination. This is conceptually illustrated in Figure 6-36. In this figure, R_{Detect} is the implementation specific charging resistor. C_{AC} is the AC capacitor that is in the circuit only if R_{Term} is also present, otherwise, only $C_{Parasitic}$ is present.

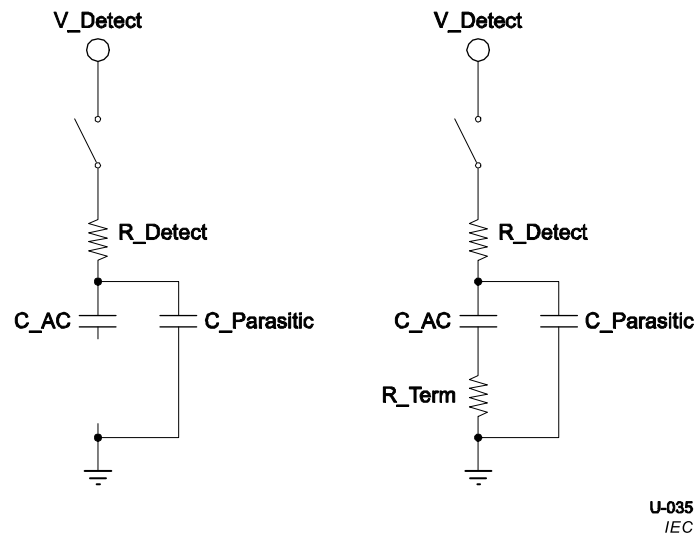


Figure 6-36 – Rx Detect Schematic

The left side of Figure 6-36 shows the Receiver Detection circuit with no termination present. The right side of the figure is the same circuit with termination.

Detect voltage transition must be common mode. Detect voltage transition shall conform to $V_{TX_RCV_DETECT}$ as described in Table 6-17.

The receiver detect sequence shall be in the positive common mode direction only. Negative receiver detection is not allowed.

6.11.2 Rx Detect Sequence

The recommended behavior of the Receiver Detection sequence is:

- 1) A Transmitter shall start at a stable voltage prior to the detect common mode shift.
- 2) A Transmitter changes the common mode voltage on Txp and Txn consistent with detection of Receiver high impedance which is bounded by parameter $Z_{RX_HIGH-IMP-DC-POS}$ listed in Table 6-21.
- 3) A Receiver is detected based on the rate that the lines change to the new voltage.
 - The Receiver is not present if the voltage at the Transmitter charges at a rate dictated only by the Transmitter impedance and the capacitance of the interconnect and series capacitor.
 - The Receiver is present if the voltage at the Transmitter charges at a rate dictated by the Transmitter impedance, the series capacitor, the interconnect capacitance, and the Receiver termination.

Any time Electrical Idle is exited the detect sequence does not have to execute or may be aborted. During the Device connect, the Device receiver has to guarantee it is always in high impedance state while its power plane is stabilizing. This is required to avoid the Host falsely detecting the Device and starting the training sequence before the Device is ready. Similarly a disabled port has to keep its receiver termination in high impedance which is bounded by parameters $Z_{RX_HIGH-IMP-DC-POS}$ until directed by higher layer to exit from the Disabled state. In contrast, a port which is at U1/U2/U3 Electrical Idle shall have its Receiver Termination turned on and meet the R_{RX-DC} specification.

6.11.3 Upper Limit on Channel Capacitance

The interconnect total capacitance to ground seen by the Receiver Detection circuit shall not exceed 3 nF to ground, including capacitance added by attached test instrumentation. This

limit is needed to guarantee proper operation during Receiver detect. Note that this capacitance is separate and distinct from the AC coupling capacitance value.

6.12 Retimers

Requirements for retimers are defined in Appendix E.

7 Link Layer

The Enhanced SuperSpeed USB consists of SuperSpeed USB operating at the Gen 1 speed, and SuperSpeedPlus USB operating at the Gen 2 speed. The link layer of the Enhanced SuperSpeed USB has the responsibility of maintaining the link connectivity so that successful data transfers between the two link partners are ensured. A robust link flow control is defined based on packets and link commands. Packets are prepared in the link layer to carry data and different information between the host and a device. Link commands are defined for communications between the two link partners. Packet frame ordered sets and link command ordered sets are also constructed such that they are tolerant to one symbol error. In addition, error detection capabilities are also incorporated into a packet and a link command to verify packet and link command integrity.

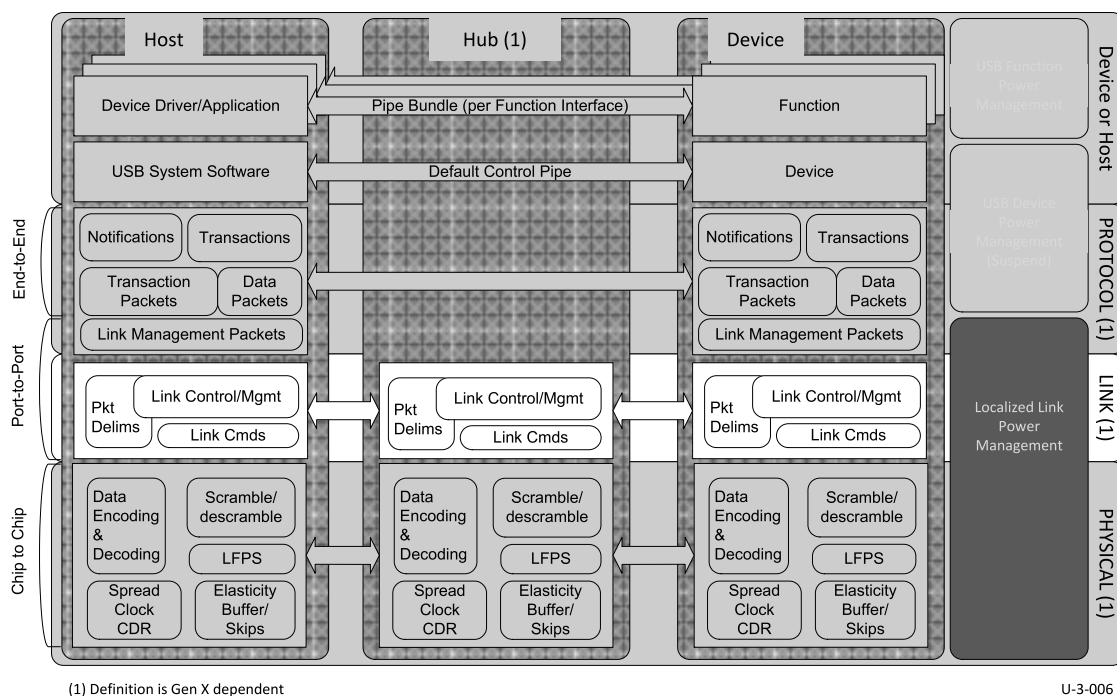


Figure 7-1 – Link Layer

The link layer also facilitates link training, link testing/debugging, and link power management. This is accomplished by the introduction of Link Training Status State Machine (LTSSM).

The focus of this chapter is to address the following in detail:

- Packet Framing
- Link command definition and usage
- Link initialization and flow control
- Link power management
- Link error rules/recovery
- Resets

- LTSSM specifications

7.1 Byte Ordering

Multiple byte fields in a packet or a link command are moved over to the bus in little-endian order, i.e., the least significant byte (LSB) first, and the most significant byte (MSB) last. Figure 7-2 shows an example of byte ordering.

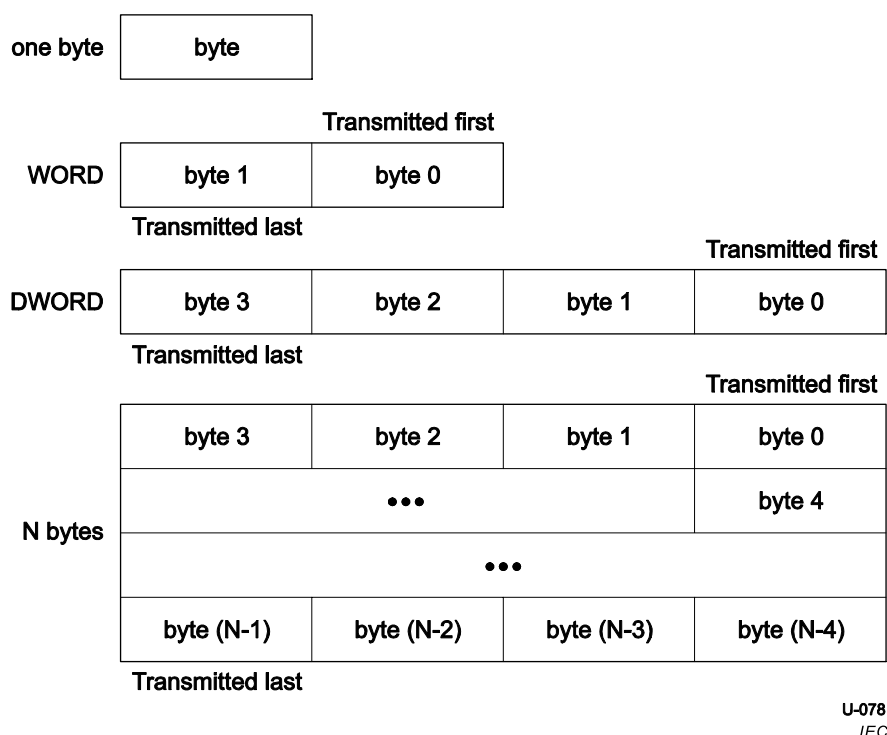


Figure 7-2 – Byte Ordering

7.1.1 SuperSpeed USB Line Code

Each byte of a packet or link command will be encoded in the physical layer using 8b/10b encoding. Refer to Section 6.3.1.3 regarding 8b/10b encoding and bit ordering.

7.1.2 SuperSpeedPlus USB Line Code

To improve the effective throughput for SuperSpeedPlus USB, 128b/132b line code is employed to replace 8b/10b line code used in SuperSpeed USB. Two block types are defined based on 128b/132b line code. A control block is defined to transmit TSEQ, TS1, TS2, SYNC, SDS, and SKP ordered sets. A data block is defined to transmit packets, link commands, and Idle Symbols. Refer to Section 6.3 for 128b/132b block definition and bit ordering. Each symbol within a data block is scrambled by default, unless disabled otherwise by the Disabling Scrambling bit asserted in the TS2 ordered set received in Polling.Configuration or Recovery.Configuration. Refer to Sections 7.5.4.9 and 7.5.10.4 for details.

7.2 Link Management and Flow Control

This section contains information regarding link data integrity, flow control, and link power management.

- The packet and packet framing section defines packet types, packet structures, and CRC requirements for each packet.
- The link command section defines special link command structures that control various functionalities at the link level.
- The logical idle defines a special symbol used in U0.

- The flow control defines a set of handshake rules for packet transactions.

7.2.1 Packets and Packet Framing

The Enhanced SuperSpeed USB uses packets to transfer information. Detailed packet formats for Link Management Packets (LMP), Transaction Packets (TP), Isochronous Timestamp Packets (ITP), and Data Packets (DP) are defined in Section 7.2.

7.2.1.1 Header Packet Structure

For SuperSpeed USB, all header packets are 20 symbols long, as is formatted in Figure 7-3. This includes LMPs, TPs, ITPs, and DPHs. A header packet consists of three parts, a header packet framing, a packet header, and a Link Control Word.

For SuperSpeedPlus USB, all header packets except for non-deferred DPH are the same as SuperSpeed USB. The non-deferred SuperSpeedPlus DPH is a header packet with its own framing ordered set, and contains a length field replica immediately after the Link Control word. The purpose of this special construction is to allow the non-deferred SuperSpeedPlus DPH to be processed differently from all other header packets and to achieve single bit error tolerance in its length field. The non-deferred SuperSpeedPlus DPH format is shown in Figure 7-4.

7.2.1.1.1 Header Packet Framing

Header packet framing, HPSTART ordered set, is a four-symbol header packet starting frame ordered set. For SuperSpeed USB, it is defined as three consecutive K symbols of SHP followed by a single K-symbol of EPF. For SuperSpeedPlus USB, HPSTART ordered set is the framing ordered set for all header packets except for non-deferred DPH, and is defined as three consecutive symbols of SHP followed by a single symbol of EPF. A non-deferred SuperSpeedPlus DPH uses DPHSTART ordered set, which is defined as three consecutive symbols of DPHP followed by a single symbol of EPF. Refer to Table 6-1 for framing symbol definition.

- All header packets except for non-deferred SuperSpeedPlus data packet header shall always begin with HPSTART ordered set.
- A non-deferred SuperSpeedPlus data packet header shall always begin with DPHSTART ordered set.
- A deferred SuperSpeedPlus data packet header shall always begin with HPSTART ordered set and it shall contain the length field replica.

The construction of the header packet framing is to achieve one symbol error tolerance.

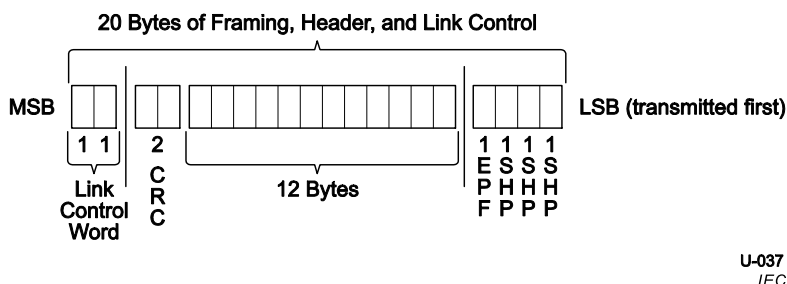
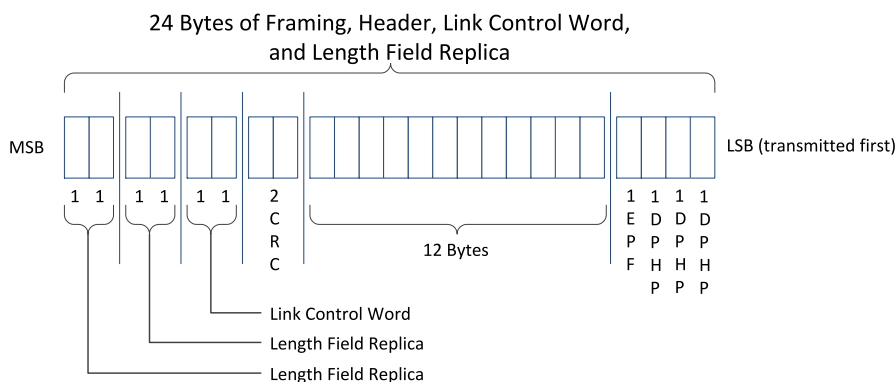


Figure 7-3 – Enhanced SuperSpeed Header Packet with HPSTART, Packet Header, and Link Control Word

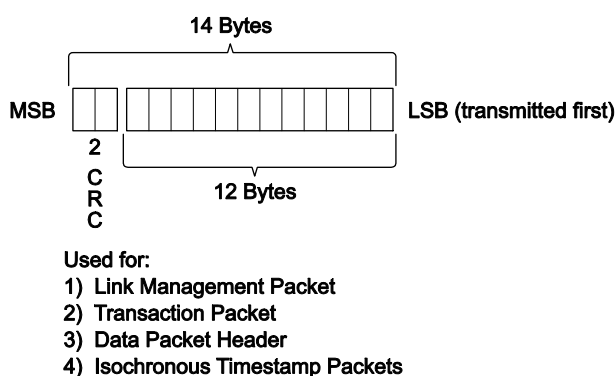


IEC

Figure 7-4 – SuperSpeedPlus DPH Format

7.2.1.1.2 Packet Header

A packet header consists of 14 bytes as formatted in Figure 7-5. It includes 12 bytes of header information and a 2-byte CRC-16. CRC-16 is used to protect the data integrity of the 12-byte header information.



U-038
IFC

Figure 7-5 – Packet Header

The implementation of CRC-16 on the packet header is defined below:

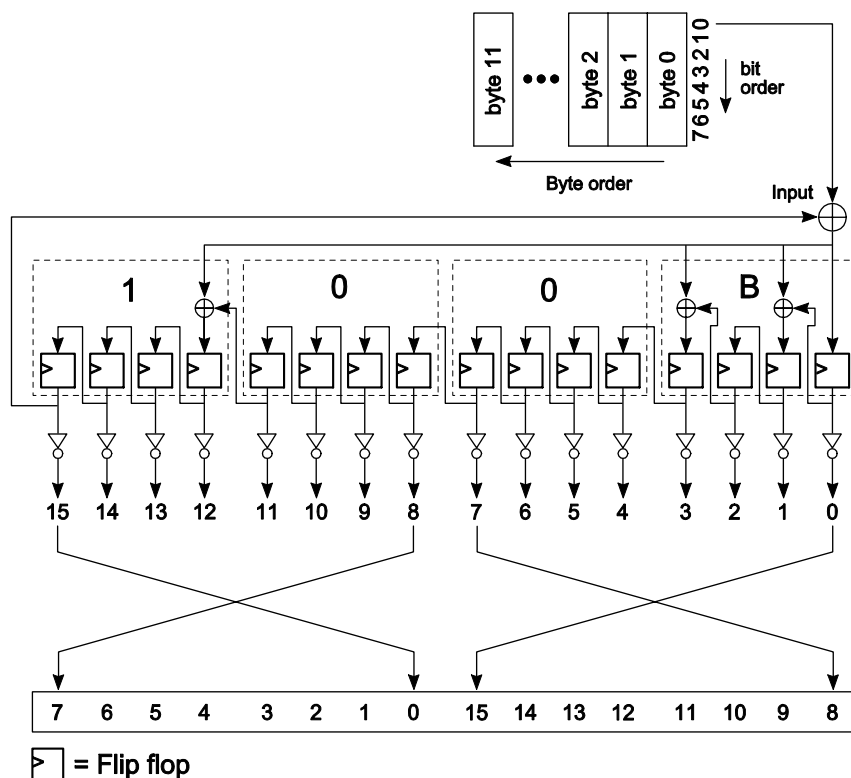
- The polynomial for CRC-16 shall be 100Bh.

NOTE The CRC-16 polynomial is not the same as the one used for USB 2.0.

- The initial value of CRC-16 shall be FFFFh.
- CRC-16 shall be calculated for all 12 bytes of the header information, not inclusive of any packet framing symbols.
- CRC-16 calculation shall begin at byte 0, bit 0 and continue to bit 7 of each of the 12 bytes.
- The remainder of CRC-16 shall be complemented.
- The residual of CRC-16 shall be F6AAh.

NOTE The inversion of the CRC-16 remainder adds an offset of FFFFh that will create a constant CRC-16 residual of F6AAh at the receiver side.

Figure 7-6 is an illustration of CRC-16 remainder generation. The output bit ordering is listed in Table 7-1.



U-039
IEC

Figure 7-6 – CRC-16 Remainder Generation

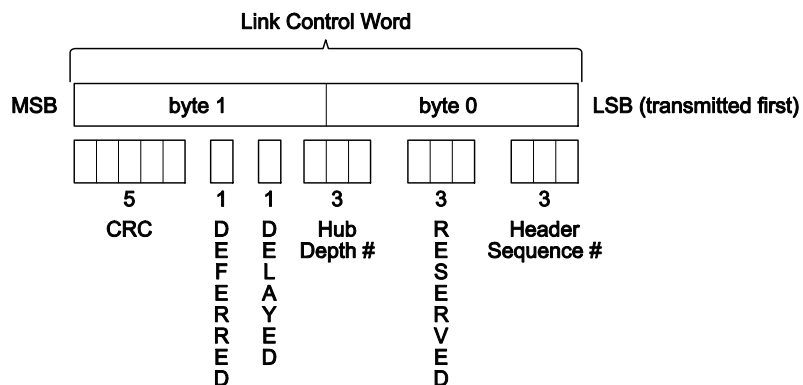
Table 7-1 – CRC-16 Mapping

CRC-16 Result Bit	Position in CRC-16 Field
0	15
1	14
2	13
3	12
4	11
5	10
6	9
7	8
8	7
9	6
10	5
11	4
12	3
13	2
14	1
15	0

7.2.1.1.3 Link Control Word

The 2-byte Link Control Word is formatted as shown in Figure 7-7. It is used for both link level and end-to-end flow control.

The Link Control Word shall contain a 3-bit Header Sequence Number, 3-bit Reserved, a 3-bit Hub Depth Index, a Delayed bit (DL), a Deferred bit (DF), and a 5-bit CRC-5.



U-040
IEC

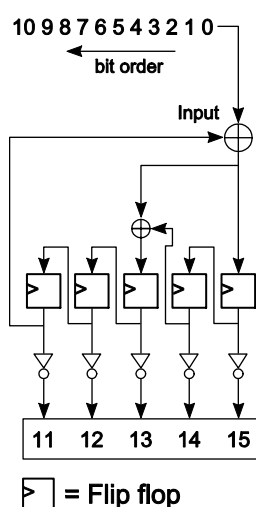
Figure 7-7 – Link Control Word

CRC-5 protects the data integrity of the Link Control Word. The implementation of CRC-5 is defined below:

- The CRC-5 polynomial shall be 00101b.
- The Initial value for the CRC-5 shall be 11111b.
- CRC-5 is calculated for the remaining 11 bits of the Link Control Word.
- CRC-5 calculation shall begin at bit 0 and proceed to bit 10.
- The remainder of CRC-5 shall be complemented, with the MSb mapped to bit 11, the next MSb mapped to bit 12, and so on, until the LSb mapped to bit 15 of the Link Control Word.
- The residual of CRC-5 shall be 01100b.

NOTE The inversion of the CRC-5 remainder adds an offset of 11111b that will create a constant CRC-5 residual of 01100b at the receiver side.

Figure 7-8 is an illustration of CRC-5 remainder generation.



U-041
IEC

Figure 7-8 – CRC-5 Remainder Generation

7.2.1.2 Data Packet Payload Structure

Data packets are a special type of packet consisting of a Data Packet Header (DPH) and a Data Packet Payload (DPP). The DPH is defined in Section 7.2.1.1. The DPP, on the other hand, consists of a data packet payload framing, and a variable length of data followed by 4 bytes of CRC-32. Figure 7-9 describes the format of a DPP.

7.2.1.2.1 Data Packet Payload Framing

DPP framing ordered sets consist of a starting framing ordered set called DPPSTART OS, and one of two ending framing ordered sets called DPPEND OS or DPPABORT OS. As indicated by Figure 7-9, a DPPSTART ordered set, which is a DPP starting frame ordered set, consists of three consecutive symbols of SDP followed by a single symbol of EPF. A DPP ending frame ordered set has two different types. The first type, DPPEND ordered set, is a DPP ending frame ordered set which consists of three consecutive symbol of END followed by a single symbol of EPF. The second type, DPPABORT ordered set, is a DPP aborting frame ordered set which consists of three consecutive symbol of EDB (end of nullified packet) followed by a single symbol of EPF. The DPPEND ordered set is used to indicate a normal ending of a complete DPP. For SuperSpeed USB, the DPPABORT ordered set is used to indicate an abnormal ending of a DPP. For SuperSpeedPlus USB, a DPPABORT OS is used to notify either a partially nullified DPP or nullified DPP.

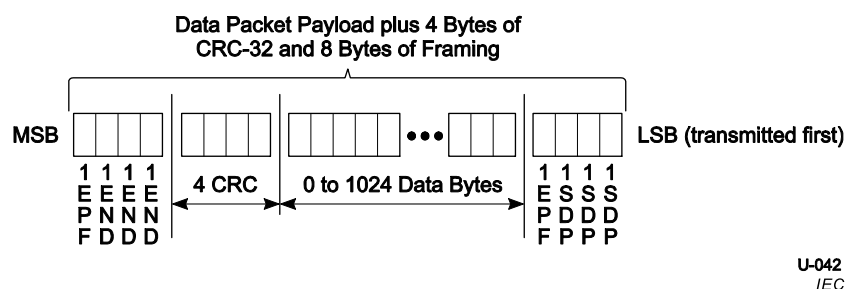


Figure 7-9 – Data Packet Payload with CRC-32 and Framing

7.2.1.2.2 Data Packet Payload

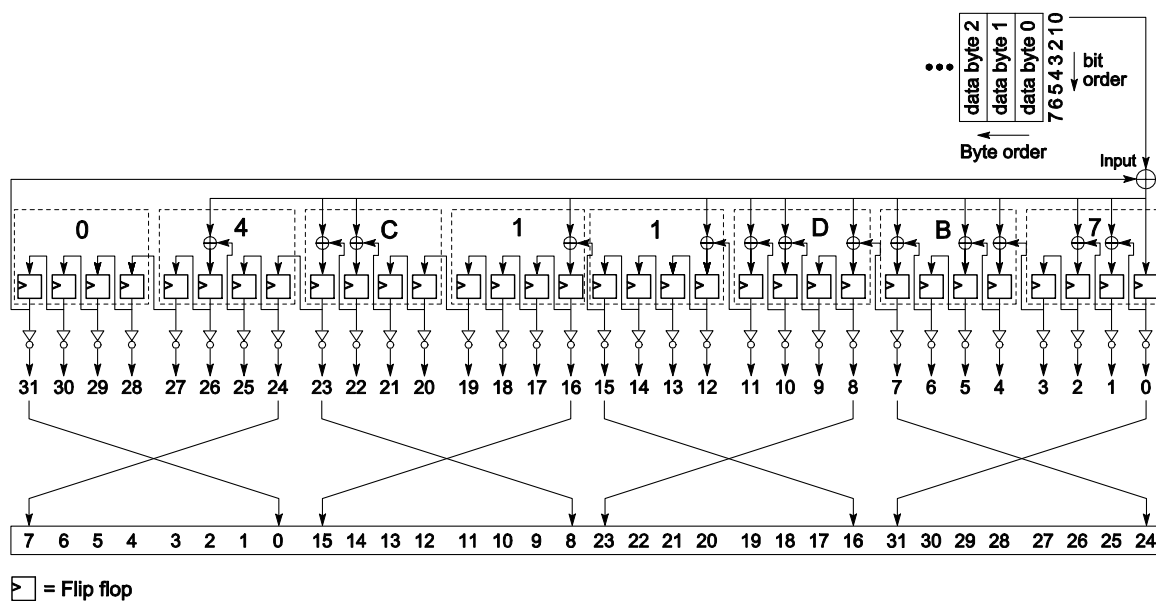
The DPP section consists of 0 to 1 024 bytes of data payload followed by 4 bytes CRC-32.

CRC-32 protects the data integrity of the data payload. CRC-32 is as follows:

- The CRC-32 polynomial shall be 04C1 1DB7h.
- The CRC-32 Initial value shall be FFFF FFFFh.
- CRC-32 shall be calculated for all bytes of the DPP, not inclusive of any packet framing symbols.
- CRC-32 calculation shall begin at byte 0, bit 0 and continue to bit 7 of each of the bytes of the DPP.
- The remainder of CRC-32 shall be complemented.
- The residual of CRC-32 shall be C704DD7Bh.

NOTE The inversion of the CRC-32 remainder adds an offset of FFFF FFFFh that will create a constant CRC-32 residual of C704DD7Bh at the receiver side.

Figure 7-10 is an illustration of CRC-32 remainder generation. The output bit ordering is listed in Table 7-2.



U-043
IEC

Figure 7-10 – CRC-32 Remainder Generation

Table 7-2 – CRC-32 Mapping

CRC-32 Result Bit	Position in CRC-32 Field
0	31
1	30
2	29
3	28
4	27
5	26
6	25
7	24
8	23
9	22
10	21
11	20
12	19
13	18
14	17
15	16
16	15
17	14
18	13
19	12
20	11
21	10
22	9
23	8
24	7
25	6
26	5
27	4
28	3
29	2
30	1
31	0

For SuperSpeed USB, any premature termination of a DPP shall end with a DPPABORT ordered set.

For SuperSpeedPlus USB, a port shall always preserve the DPP boundary by completing the DPP transmission meeting the length field specification defined in its associated DPH except for the following conditions.

- 1) A downstream port is directed to issue a Warm Reset.

NOTE An upstream port, before declaring the detection of Warm Reset, may already enter Recovery.

- 2) A port is directed to enter Recovery.

NOTE A port may also complete the DPP transmission under any of the above conditions.

In all other cases, a port in SuperSpeedPlus operation shall perform one of the following.

- It shall append DPPEND OS upon completing the transmission of DPP.
- In the case of a nullified DPP, it shall append DPPABORT OS immediately after its DPH.
- In the case of partially nullified DPP, it shall append DPPABORT OS after completing the DPP as defined by the length field in its associated DPH, similar to normal ending of DPP. A port shall fill with Idle Symbols in DPP if intended data for transmission are not available. The condition to transmit a partially nullified DPP is implementation specific.

7.2.1.2.3 Data Payload Structure and Spacing between DPH and DPP

There shall be no spacing between a DPH and its corresponding DPP. This is illustrated in Figure 7-11.

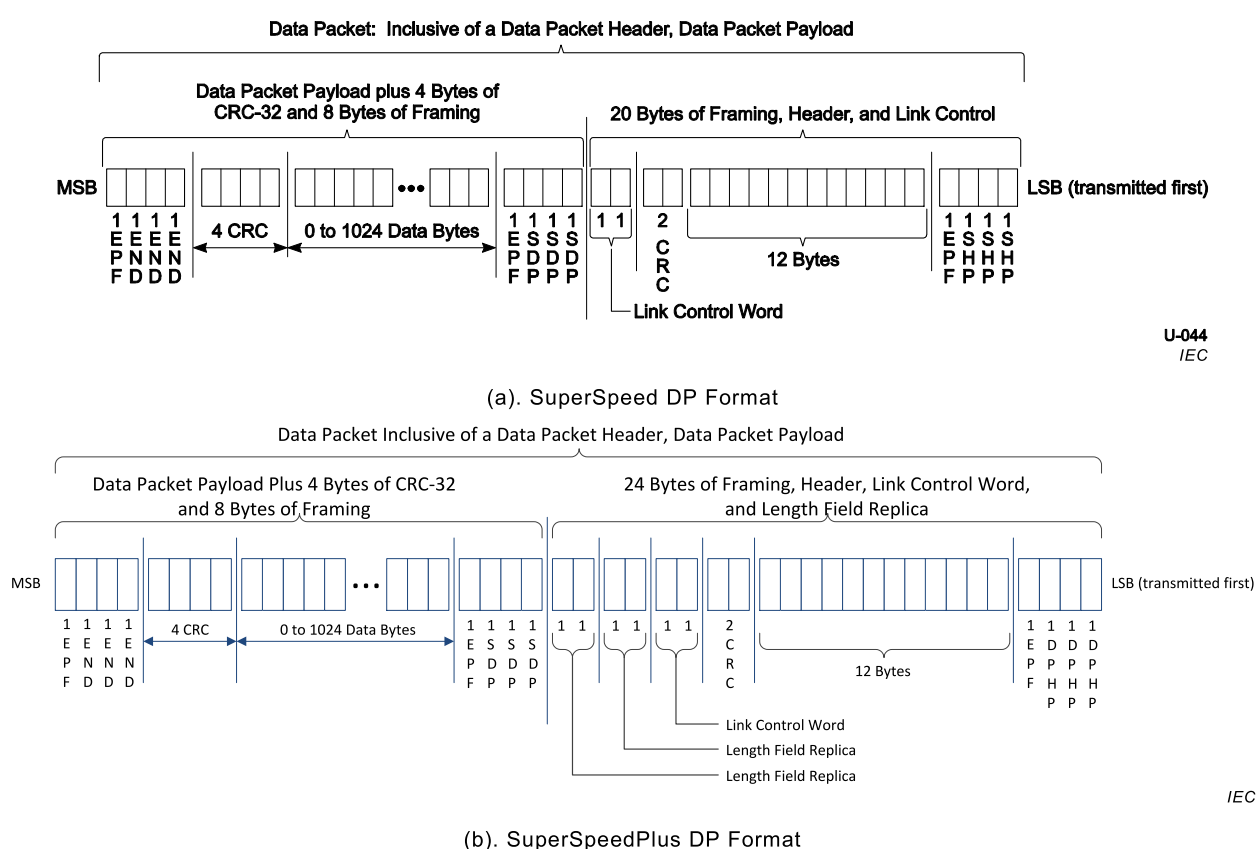


Figure 7-11 – Data Packet with Data Packet Header Followed by Data Packet Payload. (a) SuperSpeed DP; (b). SuperSpeedPlus DP

Additional details on how header packets are transmitted and received at the link level are described in Section 7.2.4.

7.2.1.3 SuperSpeedPlus Packet Placement

For SuperSpeedPlus USB, packet placement shall meet the following rules:

- All packets shall be placed in data blocks.
- The placement of a packet may start in any symbol position within a data block, and may cross over to the next consecutive data blocks.

Refer to Appendix D for examples of SuperSpeedPlus packet placement.

7.2.2 Link Commands

Link commands are used for link level data integrity, flow control and link power management. Link commands are a fixed length of eight symbols and contain repeated symbols to increase the error tolerance. Refer to Section 7.3 for more details. Link command names have the L-preface to differentiate their link level usage and to avoid confusion with packets.

7.2.2.1 Link Command Structure

Link command shall be eight symbols long and constructed with the following format shown in Figure 7-12. The first four symbols, LCSTART, are the link command starting frame ordered set consisting of three consecutive SLCs followed by EPF. The second four symbols consist of a two-symbol link command word and its replica. Table 7-3 summarizes the link command structure.

Table 7-3 – Link Command Ordered Set Structure

Symbol Number	Description
0	SLC (Start Link Command)
1	SLC (Start Link Command)
2	SLC (Start Link Command)
3	EPF
4~5	Link Command Word
6~7	Link Command Word

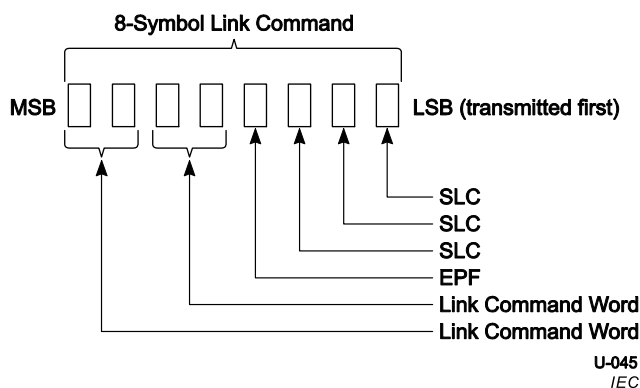


Figure 7-12 – Link Command Structure

7.2.2.2 Link Command Word Definition

Link command word is 16 bits long with the 11-bit link command information protected by a 5-bit CRC-5 (see Figure 7-13). The 11-bit link command information is defined in Table 7-4. The calculation of CRC-5 is the same as Link Control Word illustrated in Figure 7-7.

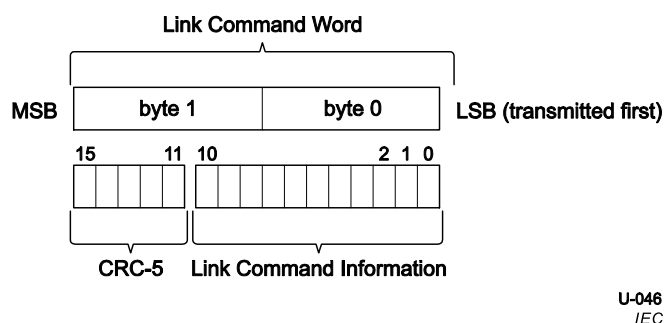


Figure 7-13 – Link Command Word Structure

Table 7-4 – Link Command Bit Definitions

Class		Type	b6~4	Sub-Type
b10~9	Link Command	b8~7		b3~0
00	LGOOD_n LRTY LBAD LCRD_x LCRD1_x LCRD2_x	00: LGOOD_n	Reserved (000)	b3: Reserved b2~0: HP Sequence Number 000: LGOOD_0 001: LGOOD_1 010: LGOOD_2 011: LGOOD_3 100: LGOOD_4 101: LGOOD_5 110: LGOOD_6 111: LGOOD_7
		01: LCRD_x LCRD1_x LCRD2_x		b3: Reserved b2: Credit series 0: LCRD_x or LCRD1_x 1: LCRD2_x b1~0: Rx Header Buffer Credit 00: LCRD_A/LCRD1_A/LCRD2_A 01: LCRD_B/LCRD1_B/LCRD2_B 10: LCRD_C/LCRD1_C/LCRD2_C 11: LCRD_D/LCRD1_D/LCRD2_D
		10: LRTY 11: LBAD		Reserved (0000)
01	LGO_Ux LAU LXU LPMA	00: LGO_Ux		0001: LGO_U1 0010: LGO_U2 0011: LGO_U3 Others: Reserved
		01: LAU 10: LXU 11: LPMA		Reserved (0000)
10	LDN LUP	00: LUP 11: LDN Others: Reserved		Reserved (0000)
11: Reserved	Reserved	Reserved (0000)		Reserved (0000)

Link commands are defined for four usage cases. First, link commands are used to ensure the successful transfer of a packet. Second, link commands are used for link flow control. Third, link commands are used for link power management. And finally, special link commands are defined for a port to signal its presence in U0.

Successful header packet transactions between the two link partners require proper header packet acknowledgement. Rx Header Buffer Credit exchange facilitates link flow control. Header packet acknowledgement and Rx Header Buffer Credit exchange are realized using different link commands. LGOOD_n (n = 0 to 7) and LBAD are used to acknowledge whether a header packet has been received properly or not. LRTY is used to signal that a header packet is re-sent.

For SuperSpeed USB, LCRD_A, LCRD_B, LCRD_C, and LCRD_D are the link commands used to signal the availability of Rx Header Buffers in terms of Credit.

For SuperSpeedPlus USB, Type 1 and Type 2 traffic classes are defined. Type 1 traffic class, or namely Type 1 packet, includes the following packet types: periodic DPs, TPs, ITPs, and LMPs. Type 2 traffic class, or namely Type 2 packet, includes only the asynchronous DPs. LCRD1_A, LCRD1_B, LCRD1_C, and LCRD1_D are link commands used for Type 1 traffic class to signal the availability of Rx Buffers for Type 1 header packets or data packets in terms of Credit. LCRD2_A, LCRD2_B, LCRD2_C, and LCRD2_D are link commands used for Type 2 traffic class to signal the availability of Rx Buffers for Type 2 packets in terms of Credit.

In the following sections, LCRD_x or LCRD1_x/LCRD2_x is used with x denoting either A, B, C, or D. See Table 7-5 for details. LGOOD_n uses an explicit numerical index called Header Sequence Number to represent the sequencing of a header packet. The Header Sequence Number starts from 0 and is incremented by one based on modulo-8 addition with each header packet. The index corresponds to the received Header Sequence Number and is used for flow control and detection of lost or corrupted header packets.

LCRD_x and LCRD1_x/LCRD2_x use an explicit alphabetical index. The index A, B, C, D, A, B, C... is advanced by one with each header packet being processed and an Rx Header Buffer Credit is available. The index is used to ensure Rx Header Buffer Credits are received in order such that missing of an LCRD_x or LCRD1_x/LCRD2_x can be detected. The index operations of LCRD1_x and LCRD2_x are independent.

LBAD and LRTY do not use indexes.

LGO_U1, LGO_U2, LGO_U3, LAU, LXU, and LPMA are link commands used for link power management.

LDN and LUP are special link commands used by a downstream port and an upstream port to indicate their port presence in U0. The usage of LDN and LUP is described in Table 7-5.

Additional requirements and examples on the use of link commands are found in Section 7.2.4.

Table 7-5 – Link Command Definitions

Link Command	Definition – See Sections 7.2.4.1, 7.2.4.2, and 7.5.6 for detailed use and requirements.
LGOOD_n	n (0, 1, 2,7): Header Sequence Number. Sent by a port receiving a header packet when all of the following conditions are true: • The header packet has a valid structure and can be recognized by the receiver. • CRC-5 and CRC-16 are valid. • Header Sequence Number in the received header packet matches the expected Rx Header Sequence Number. • An Rx Header Buffer in the receiver is available for storing the received header packet. Mismatch between a Header Sequence Number in the received header packet and the expected Rx Header Sequence Number will result in a port transitioning to Recovery. Received by a port sending a header packet. This is an acknowledgement from a link partner that a header packet with the Header Sequence Number of "n" is received properly. Receipt of LGOOD_n mismatching the expected ACK Tx Header Sequence Number will result in a port transitioning to Recovery. Also sent by a port upon entry to U0 as the Header Sequence Number Advertisement to initialize the ACK Tx Header Sequence Number of the two ports. Refer to Section 7.2.4.1 for details.
LBAD	Bad header packet. Sent by a port receiving the header packet in response to an invalid header packet. Packet that was received has corrupted CRC-5 and/or CRC-16. Receipt of LBAD will cause a port to resend all header packets after the last header packet that has been acknowledged with LGOOD_n. Refer to Section 7.2.4.1 for details.
LCRD_x LCRD1_x LCRD2_x	LCRD_x: Rx Header Buffer Credit Index for all SuperSpeed header packets. It is to signify that a single Rx Header Buffer Credit has been made available. LCRD1_x: Type 1 Rx Buffer Credit Index for all SuperSpeedPlus Type 1 traffic class. It is to signify that a single Type 1 Rx Buffer Credit has been made available LCRD2_x: Type 2 Rx Buffer Credit Index for all SuperSpeedPlus Type 2 traffic class. It is to signify that a single Type 2 Rx Buffer Credit has been made available. x (A, B, C, D): Rx Header Buffer Credit Index. For SuperSpeed USB, LCRD_x to be sent by a port after receiving a header packet that meets the following criteria: • LGOOD_n has been or will be sent. • The header packet has been processed, and an Rx Header Buffer Credit is available. For SuperSpeedPlus USB, LCRD1_x to be sent by a port after receiving a Type 1 packet that meets all the following criteria: • LGOOD_n has been or will be sent. • The packet (HP or DP) has been processed and a Type 1 Rx Buffer Credit is available. For SuperSpeedPlus USB, LCRD2_x to be sent by a port after receiving a DP of Type 2 traffic class that meets all the following criteria: • LGOOD_n has been or will be sent. • The data packet has been processed and a Type 2 Rx Buffer Credit is available. LCRD_x or LCRD1_x/LCRD2_x is sent in the alphabetical order of A, B, C, D, and back to A without skipping. Missing LCRD_x or LCRD1_x/LCRD2_x will cause the link to transition to Recovery. Refer to Section 7.2.4.1 for details.
LRTY	Sent by a port before resending the first header packet in response to receipt of LBAD.
LGO_U1	Sent by a port requesting entry to U1.
LGO_U2	Sent by a port requesting entry to U2.
LGO_U3	Sent by a downstream port requesting entry to U3. An upstream port shall accept the request.
LAU	Sent by a port accepting the request to enter U1, U2, or U3.
LXU	Sent by a port rejecting the request to enter U1 or U2.
LPMA	Sent by a port upon receiving L AU. Used in conjunction with LGO_Ux and LAU handshake to guarantee both ports are in the same state.

Link Command	Definition – See Sections 7.2.4.1, 7.2.4.2, and 7.5.6 for detailed use and requirements.
LDN	Downstream port present in U0. Sent by a downstream port every 10 μ s when there are no packets or other link commands to be transmitted. Refer to Section 7.5.6.1 for details.
LUP	Device present in U0. Sent by an upstream port every 10 μ s when there are no packets or other link commands to be transmitted. Refer to Section 7.5.6.1 for details.

7.2.2.3 Link Command Placement

The link command placement shall meet the following rules:

- Link commands shall not be placed inside header packet structures (i.e., within LMPs, TPs, ITPs, or DPHs).
- Link commands shall not be placed within the DPP of a DP structure.
- Link commands shall not be placed between the DPH and the DPP.
- Link commands may be placed before and after a header packet with the exception that they shall not be placed in between a DPH and its DPP.
- Multiple link commands are allowed to be transmitted back to back.
- Link commands shall not be sent until all scheduled SKP ordered sets have been transmitted.

NOTE Additional rules regarding scheduling of link commands are found in Section 10.9.

For SuperSpeedPlus USB, the link command placement shall meet the following additional rules:

- All link commands shall be placed in data blocks.
- The placement of a link command may start in any symbol position within a data block, and may cross over to the next consecutive data blocks.

Refer to Appendix D for examples of SuperSpeedPlus link command placement.

7.2.3 Logical Idle

Logical Idle is defined to be a period of one or more symbol periods when no information (packets or link commands) is being transferred on the link. For SuperSpeed USB, a special D-Symbol (00h), is defined as Idle Symbol (IS). For SuperSpeedPlus USB, a special symbol (5Ah) is defined as Idle Symbol. Idle Symbol shall be transmitted by a port at any time in U0 meeting the logical idle definition.

By default, the IS shall be scrambled according to rules described in Section 6.3.

Table 7-6 – Logical Idle Definition

Symbol	Data Byte Value		Definition
IS	SuperSpeed operation	00h	Represents Idle state on the bus.
	SuperSpeedPlus operation	5Ah	

7.2.4 Link Command Usage for Flow Control, Error Recovery, and Power Management

Link commands are used for link level header packet flow control, to identify lost/corrupted header packets and to initiate/acknowledge link level power management transitions. The construction and descriptions for each link command are found in Section 7.2.2.

7.2.4.1 Header Packet Flow Control and Error Recovery

Header packet flow control is used for all header packets. It requires each side of the link to follow specific header buffer and transmission ordering constraints to guarantee a successful packet transfer and link interoperability. This section describes, in detail, the rules of packet flow control.

7.2.4.1.1 Initialization

The link initialization refers to initialization of a port once a link transitions to U0 from Polling, Recovery, or Hot Reset. The initialization, for SuperSpeed USB, includes the Header Sequence Number Advertisement and the Rx Header Buffer Credit Advertisement between the two ports before a header packet can be transmitted. For SuperSpeedPlus USB, the initialization includes the Header Sequence Number Advertisement, the Type 1 Rx Buffer Credit Advertisement, and the Type 2 Rx Buffer Credit Advertisement.

- The following requirements shall be applied to a port:
 - 1) A port shall maintain two Tx Header Sequence Numbers. One is the Tx Header Sequence Number that is defined as the Header Sequence Number that will be assigned to a header packet when it is first transmitted (not a re-transmission). The other is the ACK Tx Header Sequence Number that is defined as the expected Header Sequence Number to be acknowledged with LGOOD_n that is sent by a port receiving the header packet.
 - 2) A port shall have an Rx Header Sequence Number. It is defined as the expected Header Sequence Number when a header packet is received.
 - 3) A port in SuperSpeed operation shall maintain two Rx Header Buffer Credit Counts. One is the Local Rx Header Buffer Credit Count that is defined as the number of the available Rx Header Buffer Credits of its receiver. The other is the Remote Rx Header Buffer Credit Count that is defined as the number of the available Rx Header Buffer Credits from its link partner. A port in SuperSpeedPlus operation shall maintain two Type 1 Rx Buffer Credit Counts for Type 1 traffic class. One is the Local Type 1 Rx Buffer Credit Count that is defined as the number of the available Rx Buffer Credits of its receiver. The other is the Remote Type 1 Rx Buffer Credit Count that is defined as the number of the available Rx Buffer Credits from its link partner. A port in SuperSpeedPlus operation shall also maintain two Type 2 Rx Buffer Credit Counts for Type 2 traffic class. One is the Local Type 2 Rx Buffer Credit Count that is defined as the number of the available Rx Buffer Credits of its receiver. The other is the Remote Type 2 Rx Buffer Credit Count that is defined as the number of the available Rx Buffer Credits from its link partner.
 - 4) A port in SuperSpeed operation shall have enough Tx Header Buffers in its transmitter to hold up to four unacknowledged header packets. A port in SuperSpeedPlus operation shall have enough Type 1/Type 2 Tx Header Buffers in its transmitter to hold up to four unacknowledged header packets of Type 1 traffic class, and another four unacknowledged data packet headers of Type 2 traffic class.
 - 5) A port in SuperSpeed operation shall not transmit any header packet if its Remote Rx Header Buffer Credit Count is zero. A port in SuperSpeedPlus operation shall not transmit any Type 1 packets if its Remote Type 1 Rx Buffer Credit Count is zero, or any Type 2 packets if its Remote Type 2 Rx Buffer Credit Count is zero.
 - 6) A port in SuperSpeed operation shall have enough Rx Header Buffers in its receiver to receive up to four header packets. A port in SuperSpeedPlus operation shall have enough Rx Buffers in its receiver to receive up to four Type 1 packets of maximum DPP size, and another four Type 2 packets of maximum DPP size.
 - 7) Upon entry to U0, the following shall be performed in the sequence presented:
 - a) A port in SuperSpeed operation shall start the PENDING_HP_TIMER and CREDIT_HP_TIMER in expectation of the Header Sequence Number Advertisement, and the Rx Header Buffer Credit Advertisement. A port in SuperSpeedPlus operation shall start the PENDING_HP_TIMER and Type 1 and Type 2

CREDIT_HP_TIMERS in expectation of the Header Sequence Number Advertisement, and the Type 1 and Type 2 Rx Buffer Credit Advertisements

- b) A port shall initiate the Header Sequence Number Advertisement.
 - c) A port in SuperSpeed operation shall initiate the Rx Header Buffer Credit Advertisement. A port in SuperSpeedPlus operation shall initiate the Type 1 and Type 2 Rx Buffer Credit Advertisements.
- The Header Sequence Number Advertisement refers to ACK Tx Header Sequence Number initialization by exchanging Header Sequence Numbers between the two ports. This Header Sequence Number is the Header Sequence Number of the last header packet a port has received properly. The main purpose of the Header Sequence Number Advertisement is to maintain the link flow before and after Recovery such that a port upon re-entry to U0 is aware what the last header packet is that was sent successfully prior to Recovery, and decides what header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that can be flushed or need to be retransmitted. The following rules shall be applied during the Header Sequence Number Advertisement:
 - 1) A port shall set its initial Rx Header Sequence Number defined in the following:
 - a) If a port enters U0 from Polling or Hot Reset, the Rx Header Sequence Number is zero.
 - b) If a port enters U0 from Recovery, the Rx Header Sequence Number is the header Sequence Number of the next expected header packet.
 - 2) A port shall set its initial Tx Header Sequence Number defined in the following:
 - a) If a port enters U0 from Polling or Hot Reset, its Tx Header Sequence Number is zero.
 - b) If a port enters U0 from Recovery, its Tx Header Sequence Number is the same as the Tx Header Sequence Number before Recovery.

NOTE A header packet that is re-transmitted shall maintain its originally assigned Header Sequence Number.

 - 3) A port shall initiate the Header Sequence Number Advertisement by transmitting LGOOD_n with “n” equal to the Rx Header Sequence Number minus one.

NOTE The decrement is based on modulo-8 operation.
 - 4) A port shall set its initial ACK Tx Header Sequence Number to the Sequence Number received during the Rx Header Sequence Number Advertisement plus one.

NOTE The increment is based on modulo-8 operation.
 - 5) A port in SuperSpeed operation shall not send any header packets until the Header Sequence Number Advertisement has been received and a Remote Rx Header Buffer Credit is available. A port in SuperSpeedPlus operation shall not send any Type 1 or Type 2 packet until the Header Sequence Number Advertisement has been received and their respective Remote Type 1 or Type 2 Rx Buffer Credit is available.
 - 6) A port shall not request for a low power link state entry before receiving and sending the Header Sequence Number Advertisement.

NOTE The rules of Low Power Link State Initiation (refer to Section 7.2.4.2) still apply.
 - 7) A port shall flush the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers upon receiving the Header Sequence Number Advertisement. A port shall do one of the following:
 - a) If a port enters U0 from Polling or Hot Reset, it shall flush all the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers.
 - b) If a port enters U0 from Recovery, it shall flush all the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have been sent before Recovery except for those with the Header Sequence Number greater than (modulo 8) the Header Sequence Number received in Header Sequence Number Advertisement.

NOTE If for example, the Header Sequence Number Advertisement of LGOOD_1 is received, a port shall flush the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers with Header Sequence Numbers of 1, 0, 7, 6.

- For SuperSpeed USB, the Rx Header Buffer Credit Advertisement refers to Remote Rx Header Buffer Credit Count Initialization by exchanging the number of available Local Rx Header Buffer Credits between the two ports. The main purpose of this advertisement is for a port to align its Remote Rx Header Buffer Credit Count with its link partner upon entry to U0. The following rules shall be applied during the Rx Header Buffer Credit Advertisement:
 - 1) A port shall initiate the Rx Header Buffer Credit Advertisement after sending LGOOD_n during Header Sequence Number Advertisement.
 - 2) A port shall initialize the following before sending the Rx Header Buffer Credit:
 - a) A port shall initialize its Tx Header Buffer Credit index to A.
 - b) A port shall initialize its Rx Header Buffer Credit index to A.
 - c) A port shall initialize its Remote Rx Header Buffer Credit Count to zero.
 - d) A port shall continue to process those header packets in its Rx Header Buffers that have been either acknowledged with LGOOD_n prior to entry to Recovery, or validated during Recovery, and then update the Local Rx Header Buffer Credit Count.
 - e) A port shall set its Local Rx Header Buffer Credit Count defined in the following:
 - 1) If a port enters U0 from Polling or Hot Reset, its Local Rx Header Buffer Credit Count is 4.
 - 2) If a port enters U0 from Recovery, its Local Rx Header Buffer Credit Count is the number of Rx Header Buffers available for incoming header packets.
 - 3) A port shall perform the Rx Header Buffer Credit Advertisement by transmitting LCRD_x to notify its link partner. A port shall transmit one of the following based on its Local Rx Header Buffer Credit Count:
 - a) LCRD_A if the Local Rx Header Buffer Credit Count is one.
 - b) LCRD_A and LCRD_B if the Local Rx Header Buffer Credit Count is two.
 - c) LCRD_A, LCRD_B, and LCRD_C if the Local Rx Header Buffer Credit Count is three.
 - d) LCRD_A, LCRD_B, LCRD_C and LCRD_D if the Local Rx Header Buffer Credit Count is four.
 - 4) A port receiving LCRD_x from its link partner shall increment its Remote Rx Header Buffer Credit Count by one each time an LCRD_x is received up to four.
 - 5) A port shall not transmit any header packet if its Remote Rx Header Buffer Credit Count is zero.
 - 6) A port shall not request for a low power link state entry before receiving and sending LCRD_x during the Rx Header Buffer Credit Advertisement.

Note: The rules of Low Power Link State Initiation (refer to Section 7.2.4.2) still apply.

- For SuperSpeedPlus USB, the process of the Type 1/Type 2 Rx Buffer Credit Advertisements is the same as the process of the Rx Header Buffer Credit Advertisement defined for SuperSpeed USB. There is no specific order requirement to perform the Rx Buffer Credit Advertisement between the two traffic classes. Mixture of LCRD1_x and LCRD2_x may be sent. The following rules shall be applied to SuperSpeedPlus USB during the Type 1/Type 2 Rx Buffer Credit Advertisements:
 1. A port shall initiate the Type 1/Type 2 Rx Buffer Credit Advertisement after sending LGOOD_n during Header Sequence Number Advertisement.
 2. A port shall initialize the following before sending the Type 1/Type 2 Rx Buffer Credit:
 - a. A port shall initialize its Type 1/Type 2 Tx Header Buffer Credit index to A.

- b. A port shall initialize its Type 1/Type 2 Rx Buffer Credit index to A.
- c. A port shall initialize its Remote Type 1/Type 2 Rx Buffer Credit Count to zero.
- d. A port shall continue to process the packets in its Type 1/Type 2 Rx Buffers that have been either acknowledged with LGOOD_n prior to entry to Recovery, or validated during Recovery, and then update the Local Type 1/Type 2 Rx Buffer Credit Count.
- e. A port shall set its Local Type 1/Type 2 Rx Buffer Credit Count defined in the following:
 1. If a port enters U0 from Polling or Hot Reset, its Local Type 1/Type 2 Rx Buffer Credit Count is 4.
 2. If a port enters U0 from Recovery, its Local Type 1/Type 2 Rx Buffer Credit Count is the number of Type 1/Type 2 Rx Buffers available for incoming packets.
3. A port shall perform the Type 1/Type 2 Rx Buffer Credit Advertisement by transmitting LCRD1_x/LCRD2_x to notify its link partner. A port shall transmit one of the following based on its Local Type 1/Type 2 Rx Buffer Credit Count:
 - a. LCRD1_A/LCRD2_A if the Local Type 1/Type 2 Rx Buffer Credit Count is one.
 - b. LCRD1_A and LCRD1_B/ LCRD2_A and LCRD2_B if the Local Type 1/Type 2 Rx Buffer Credit Count is two.
 - c. LCRD1_A, LCRD1_B, and LCRD1_C/ LCRD2_A, LCRD2_B, and LCRD2_C if the Local Type 1/Type 2 Rx Buffer Credit Count is three.
 - d. LCRD1_A, LCRD1_B, LCRD1_C and LCRD1_D/ LCRD2_A, LCRD2_B, LCRD2_C and LCRD2_D if the Local Type 1/Type 2 Rx Buffer Credit Count is four.
4. A port receiving LCRD1_x/LCRD2_x from its link partner shall increment its respective Remote Type 1/Type 2 Rx Buffer Credit Count by one each time a LCRD1_x/LCRD2_x is received up to four.
5. A port shall not transmit any Type 1 or Type 2 packet if the respective Remote Type 1 or Type 2 Rx Buffer Credit Count is zero.
6. A port shall not request for a low power link state entry before receiving and sending LCRD1_x and LCRD2_x during the Type 1 and Type 2 Rx Buffer Credit Advertisements.
Note: The rules of Low Power Link State Initiation (refer to Section 7.2.4.2) still apply.
- The following rules shall be applied additionally when a port enters U0 from Recovery:
 1. A port sending LBAD before Recovery shall not expect to receive LRTY before a retried header packet from its link partner upon entry to U0.
 2. A port receiving LBAD before Recovery shall not send LRTY before a retried header packet to its link partner upon entry to U0.
Note: There exists a situation where an LBAD was sent by a port before Recovery and it may or may not be received properly by its link partner. Under this situation, the rules of LBAD/LRTY do not apply. Refer to Sections 7.2.4.1.4 and 7.2.4.1.12 for details.
- Upon entry to Recovery and the next state is Hot Reset or Loopback, a port may optionally continue its processing of all the packets received properly.

7.2.4.1.2 General Rules of LGOOD_n and LCRD_x/LCRD1_x/LCRD2_x Usage

- For SuperSpeed USB, the Rx Header Buffer Credit shall be transmitted in the alphabetical order of LCRD_A, LCRD_B, LCRD_C, LCRD_D, and back to LCRD_A. LCRD_x received out of alphabetical order is considered as missing of a link command, and transition to Recovery shall be initiated.

- For SuperSpeedPlus USB, the Type 1 Rx Buffer Credit shall be transmitted in the alphabetical order of LCRD1_A, LCRD1_B, LCRD1_C, LCRD1_D, and back to LCRD1_A. LCRD1_x received out of alphabetical order is considered as missing of a link command, and transition to Recovery shall be initiated. The same shall apply to the Type 2 Rx Buffer Credit based on LCRD2_x.
- Header packets shall be sent with the Header Sequence Number in the numerical order from 0 to 7, and back to 0. LGOOD_n received out of the numerical order is considered as missing of a link command, and the transition to Recovery shall be initiated.
- Header packet transmission may be delayed. When this occurs, the DL bit shall be set in the Link Control Word by a hub and optionally by a peripheral device or host. Some, but not necessarily all, of the conditions that will cause this delay follow:
 - 1) When a header packet is resent.
 - 2) When the link is in Recovery.
 - 3) For SuperSpeed USB, when the Remote Rx Header Buffer Credit Count is zero. For SuperSpeedPlus USB, when the Remote Type 1 or Type 2 Rx Buffer Credit Count is zero.
 - 4) For SuperSpeed USB when the Tx Header Buffer is not empty. For SuperSpeedPlus USB when the Type 1/Type 2 Tx Header Buffer is not empty

NOTE The delayed bit only has significance if it is set in an ITP. If a device uses the ITP to synchronize its internal clock, then it should ignore any ITPs with the delayed bit set.

7.2.4.1.3 Transmitting Packets

This Section describes header packet transmission in SuperSpeed operation, and Type 1/Type 2 packet transmission in SuperSpeedPlus operation.

- Before sending a header packet or a Type 1/Type 2 Packet, a port shall add the Tx Header Sequence Number corresponding to the Header Sequence Number field in the Link Control Word.
- Transmission of a header packet or a Type 1/Type 2 Packet shall consume a Tx Header Buffer or a Type 1/Type 2 Tx Header Buffer. Accordingly, the Tx Header Sequence Number shall be incremented by one after the transmission or roll over to zero if the maximum Header sequence number is reached.
- Transmission of a retried header packet or a Type 1/Type 2 Packet shall not consume an additional Tx Header Buffer or Type 1/Type 2 Tx Header Buffer and the Tx Header Sequence Number shall remain unchanged.
- Upon receiving LBAD, a port shall send LRTY followed by resending all the header packets that have not been acknowledged with LGOOD_n except for Recovery. Refer to Section 7.2.4.1.1 for additional rules applicable when a port enters U0 from Recovery.
- Prior to resending a header packet, a port shall set the Delay bit within the Link Control word and re-calculate CRC-5.

NOTE CRC-16 within header packet remains unchanged.

- For SuperSpeed USB, the Remote Rx Header Buffer Credit Count shall be incremented by one if a valid LCRD_x is received.
- For SuperSpeedPlus USB, the Remote Type 1 Rx Buffer Credit Count shall be incremented by one if a valid LCRD1_x is received. The Remote Type 2 Rx Buffer Credit shall be incremented by one if a valid LCRD2_x is received.
- For SuperSpeed USB, the Remote Rx Header Buffer Credit Count shall be decremented by one if a header packet is sent for the first time after entering U0, including when it is resent following Recovery.
- For SuperSpeedPlus USB, Remote Type 1 Rx Buffer Credit Count shall be decremented by one if a Type 1 packet is sent for the first time after entering U0, including when it is resent following Recovery. The same operation applies to Remote Type 2 Rx Buffer Credit Count with regard to Type 2 packet.

- The Remote Rx Header Buffer Credit Count or the Remote Type 1/Type 2 Rx Buffer Credit Count shall not be changed when a header packet is retried following LRTY.

7.2.4.1.4 Deferred DPH

The Deferred DPH shall be treated as a TP for buffering and credit purposes. Refer to Section 7.2.1.1.1 for deferred DPH format.

7.2.4.1.5 Receiving Header Packets

This section covers receiving all header packets except for SuperSpeedPlus DPH, which is described in Section 7.2.4.1.6.

- Upon receiving a header packet, the following verifications shall be performed:
 - 1) CRC-5
 - 2) CRC-16
 - 3) Matching between the Header Sequence Number in the received header packet and the Rx Header Sequence Number
 - 4) The availability of an Rx Header Buffer to store a header packet
- A header packet is defined as “received properly” when it has passed all four criteria described above.
- When a header packet has been received properly, a port shall issue a single LGOOD_n with “n” corresponding to the Rx Header Sequence Number and increment the Rx Header Sequence Number by one (or roll over to 0 if the maximum Header Sequence Number is reached).
- In SuperSpeed operation, a port shall consume one Rx Header Buffer until it has been processed.
- In SuperSpeedPlus operation, a port shall consume one Type 1 Rx Buffer Credit until it has been processed.
- When a header packet is not “received properly”, one of the following shall occur:
 - 1) If the header packet has one or more CRC-5 or CRC-16 errors, a port shall issue a single LBAD. A port shall ignore all the header packets received subsequently until an LRTY has been received, or the link has entered Recovery. Refer to Section 7.2.4.1.1 for additional rules applicable when a port enters U0 from Recovery.
 - 2) If the Header Sequence Number in the received header packet does not match the Rx Header Sequence Number, or a port does not have an Rx Header Buffer available to store a header packet, a port shall transition to Recovery.
- In SuperSpeed operation, after transmitting LBAD, a port shall continue to issue LCRD_x if an Rx Header Buffer Credit is made available.
- In SuperSpeedPlus operation, after transmitting LBAD, a port shall continue to issue LCRD1_x/LCRD2_x if its respective Type 1/Type 2 Rx Buffer Credit is made available.
- A port shall transition directly to Recovery if it fails to receive a header packet three consecutive times. A port shall not issue the third LBAD upon the third error.

7.2.4.1.6 Receiving Data Packet Header in SuperSpeedPlus Operation

- Upon receiving a DPH, the following verifications shall be performed by a port:
 - 1) CRC-5
 - 2) CRC-16
 - 3) Matching between the Header Sequence Number in the received header packet and the Rx Header Sequence Number
 - 4) The availability of an Rx Buffer to store a header packet or a data packet.
- A DPH is defined as “received properly” when it has passed all four criteria described above. A port shall ignore the length field replica.

- When a DPH has been received properly, a port shall issue a single LGOOD_n with “n” corresponding to the Rx Header Sequence Number and increment the Rx Header Sequence Number by one (or roll over to 0 if the maximum Header Sequence Number is reached).
- A port shall consume one Type 1 or Type 2 Rx Buffer Credit until it has been processed and a Rx Buffer is available.
- When a DPH is not “received properly”, one of the following shall occur:
 - 1) If the DPH has one or more CRC-5 or CRC-16 errors, but the two length field replica are valid and identical, a port shall issue a single LBAD and track the associated DPP that immediately follows the DPH. A port shall ignore all the packets received subsequently until an LRTY has been received, or the link has entered Recovery. Refer to Section 7.2.4.1.1 for additional rules applicable when a port enters U0 from Recovery.

NOTE A valid length field is 0~1 024. Refer to Section 7.2.1.2 for details.

- 2) If any one of the following conditions occurs, a port shall transition to Recovery.
 - a) The two length field replica is not identical.
 - b) The Header Sequence Number in the received DPH does not match the Rx Header Sequence Number.
 - c) The Rx Buffer does not have enough space to store the received DP
- After transmitting LBAD, a port shall continue to issue LCRD1_x or LCRD2_x if its corresponding Type 1 or Type 2 Rx Buffer Credit is made available.
- A port shall transition directly to Recovery if it fails to receive a data packet header three consecutive times. A port shall not issue the third LBAD upon the third error.

7.2.4.1.7 SuperSpeed Rx Header Buffer Credit

Each port is required to have four Rx Header Buffer Credits in its receiver. This is referred to the Local Rx Header Buffer Credit. The number of the Local Rx Header Buffer Credits represents the number of header packets a port can accept and is managed by the Local Rx Header Buffer Credit Count.

- A port shall consume one Local Rx Header Buffer Credit if a header packet is “received properly”. The Local Rx Header Buffer Credit Count shall be decremented by one.
- Upon completion of a header packet processing, a port shall restore a Local Rx Header Buffer Credit by:
 - 1) Sending a single LCRD_x
 - 2) Advancing the Credit index alphabetically (or roll over to A if the Header Buffer Credit index of D is reached) and
 - 3) Incrementing the Local Rx Header Buffer Credit Count by one.

NOTE The LCRD_x index is used to ensure Rx Header Buffer Credits are sent in an alphabetical order such that missing of an LCRD_x can be detected.

7.2.4.1.8 SuperSpeedPlus Type 1/Type 2 Rx Buffer Credit

Each port shall have the following two classes of Rx Buffer Credits.

- A port shall have four Type 1 Rx Buffer Credits for Type 1 traffic class in its receiver. This is referred to the Local Type 1 Rx Buffer Credit.
- A port shall have four Type 2 Rx Buffer Credits for Type 2 traffic class in its receiver. This is referred to the Local Type 2 Rx Buffer Credit.

The operation rules of the Local Type 1 and Type 2 Rx Buffer Credits are the same. They each represent the number of Type 1 or Type 2 packets a port can accept and are managed by their respective Local Type 1 and Type 2 Rx Buffer Credit Count. The following descriptions refer to the Local Type 1/Type 2 Rx Buffer Credit management.

- A port shall consume one Local Type 1 or Type 2 Rx Buffer Credit if the respective Type 1 or Type 2 packet is “received properly”. The Local Type 1/Type 2 Rx Buffer Credit Count shall be decremented by one.
- Upon completion of a Type 1 or Type 2 packet processing, and the respective Type 1 or Type 2 Rx Buffer is made available, a port shall restore accordingly a Local Type 1 or Type 2 Rx Buffer Credit by:
 - 1) Sending a single LCRD1_x or LCRD2_x
 - 2) Advancing the Credit index alphabetically (or roll over to A if the Rx Buffer Credit index of D is reached) and
 - 3) Incrementing the Local Type 1 or Type 2 Rx Buffer Credit Count by one.

7.2.4.1.9 Receiving Data Packet Payload

For SuperSpeed USB, the processing of DPP shall adhere to the following rules:

- A DPP processing shall be started if the following two conditions are met:
- A DPH is received properly.
 - 1) A DPPSTART ordered set is received properly immediately after its DPH.
- The DPP processing shall be completed when a valid DPPEND ordered set is detected.
- The DPP processing shall be aborted when one of the following conditions is met:
 - 1) A valid DPPABORT ordered set is detected.
 - 2) A K-symbol that does not belong to a valid DPPEND or DPPABORT ordered set is detected before a valid DPPEND or DPPABORT ordered set. A port shall then ignore the corresponding DPPEND or DPPABORT ordered set associated with the DPP.
 - 3) A DPP of length exceeding sDataSymbolsBabble (see Table 10-19) has been reached and no valid DPPEND or DPPABORT ordered set is detected.
- A DPP shall be dropped if its DPH is corrupted.
- A DPP shall be dropped when it does not immediately follow its DPH.

For SuperSpeedPlus USB, the processing of DPP shall adhere to the following rules:

- A DPP processing shall be started if the following conditions are met.
 - 1) A DPH is received properly or a DPH is not received properly but a valid DPP length field replica is declared.
 - 2) A DPPSTART ordered set or DPPABORT ordered set is received immediately after its DPH.
- The DPP processing shall adhere to the following rules:
 - 1) The DPP processing shall be completed when a valid DPPEND OS or DPPABORT OS is detected at the expected end of DPP indicated by valid length field plus 4.
 - 2) The DPP processing shall be aborted if a DPPEND ordered set or a DPPABORT ordered set is not detected at the expected end of DPP indicated by valid length field plus 4. The port shall transition to Recovery.
 - 3) The DPP processing shall be completed if a DPPABORT ordered set is detected immediately after DPH without DPPSTART ordered set.

7.2.4.1.10 Receiving LGOOD_n

- A port shall maintain every header packet transmitted within its Tx Header Buffer or Type 1/Type 2 Tx Header Buffer until it receives an LGOOD_n. Upon receiving LGOOD_n, a port shall do one of the following:
 - 1) If LGOOD_n is the Header Sequence Number Advertisement and a port is entering U0 from Recovery, a port shall flush all the header packets retained in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have their Header Sequence Numbers equal to or less than the received Header Sequence Number, and initialize

its ACK Tx Header Sequence Number to be the received Header Sequence Number plus one.

NOTE The comparison and increment are based on modulo-8 operation.

- 2) If a port receives an LGOOD_n and this LGOOD_n is not Header Sequence Number Advertisement, it shall flush the header packet in its Tx Header Buffer or Type 1/Type 2 Tx Header Buffer with its Header Sequence Number matching the received Header Sequence Number and increment the ACK Tx Header Sequence Number by one based on modulo-8 operation.
- 3) If a port receives an LGOOD_n and this LGOOD_n is not Header Sequence Number Advertisement, it shall transition to Recovery if the received Header Sequence Number does not match the ACK Tx Header Sequence Number. The ACK Tx Header Sequence Number shall be unchanged.

NOTE A port that has received an out of order LGOOD_n implies a lost or corrupted link command and shall initiate transition to Recovery.

7.2.4.1.11 Receiving LCRD_x/LCRD1_x/LCRD2_x

- A port in SuperSpeed operation shall adjust its Remote Rx Header Buffer Credit Count based on the received LCRD_x:
 - 1) A port shall increment its Remote Rx Header Buffer Credit Count by one upon receipt of LCRD_x.
 - 2) A port shall transition to Recovery if it receives an out of order LCRD_x.
- NOTE A port that has received an out of order credit implies a lost or corrupted link command and shall transition to Recovery.
- A port in SuperSpeedPlus operation shall adjust accordingly its Remote Type 1 and Type 2 Rx Buffer Credit Count based on the received LCRD1_x and LCRD2_x:
 - 1) A port shall increment accordingly its Remote Type 1 or Type 2 Rx Buffer Credit Count by one upon receipt of LCRD1_x or LCRD2_x.
 - 2) A port shall transition to Recovery if it receives an out of order LCRD1_x or LCRD2_x.

7.2.4.1.12 Receiving LBAD

Upon receipt of LBAD, a port shall send a single LRTY before retransmitting all the header packets in the Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have not been acknowledged with LGOOD_n. Additional rules in the following shall apply.

- 1) A hub shall set the DL bit in the Link Control Word on all resent header packets and recalculate CRC-5. When retransmitting a DP in SuperSpeed operation, a hub shall drop the DPP. When retransmitting a DP in SuperSpeedPlus operation, a hub shall drop the DPP and replace it with a nullified DPP. Refer to Section 7.2.1.2.2 for definition of a nullified DPP.
- 2) The host or a peripheral device may optionally set the DL bit in the Link Control Word on any resent header packets and recalculate CRC-5. If the retried packet is a DP and the DL bit in DPH is clear, the DPH shall be followed by a DPP.

NOTE Resending an ITP invalidates the isochronous timestamp value. CRC-16 is unchanged in a retried header packet.

Upon receipt of LBAD, a port shall send a single LRTY if there is no unacknowledged header packet in the Tx Header Buffers or Type 1/Type 2 Tx Header Buffers.

NOTE This is an error condition where LBAD is created due to a link error.

7.2.4.1.13 Transmitter Timers

A PENDING_HP_TIMER is specified to cover the period of time from when a header packet is sent to a link partner, to when the header packet is acknowledged by a link partner. The purpose of this time limit is to allow a port to detect if the header packet acknowledgement

sent by its link partner is lost or corrupted. The timeout value for the PENDING_HP_TIMER is listed in Table 7-7. The operation of the PENDING_HP_TIMER shall be based on the following rules:

- A port shall have a PENDING_HP_TIMER that is active only in U0 and if one of the following conditions is met:
 - 1) A port has a header packet transmitted but not acknowledged by its link partner, except during the period between receipt of LBAD and retransmission of the oldest header packet in the Tx Header Buffer or Type 1/Type 2 Tx Header Buffer.
 - 2) A port is expecting the Header Sequence Number Advertisement from its link partner.
- The PENDING_HP_TIMER shall be started if one of the following conditions is met:
 - 1) When a port enters U0 in expectation of the Header Sequence Number Advertisement.
 - 2) When a header packet is transmitted and there are no prior header packets transmitted but unacknowledged in the Tx Header Buffers or Type 1/Type 2 Tx Header Buffers.
 - 3) When the oldest header packet is retransmitted in response to LBAD.
- The PENDING_HP_TIMER shall be reset and restarted when a header packet is acknowledged with LGOOD_n and there are still header packets transmitted but unacknowledged in the Tx Header Buffers or or Type 1/Type 2 Tx Header Buffers.
- The PENDING_HP_TIMER shall be reset and stopped if one of the following conditions is met:
 - 1) When a Header Sequence Number Advertisement is received.
 - 2) When a header packet acknowledgement of LGOOD_n is received and all the transmitted header packets in the Tx Header Buffers or Type 1/Type 2 Tx Header Buffers are acknowledged.
 - 3) When a header packet acknowledgement of LBAD is received.
- A port shall transition to Recovery if the following two conditions are met:
 - 1) PENDING_HP_TIMER times out.
 - 2) Additionally for SuperSpeed USB, the transmission of an outgoing header packet is completed or the transmission of an outgoing DPP is either completed with DPPEND or terminated with DPPABORT.

NOTE This is to allow a graceful transition to Recovery without a header packet being truncated.

For SuperSpeed USB, a CREDIT_HP_TIMER is also specified to cover the period of time from when a header packet has been transmitted and its Remote Rx Header Buffer Credit count is less than four, to when a Remote Rx Header Buffer Credit is received and its Remote Rx Header Buffer Credit count is back to four. The purpose of this timer is to make sure that a Remote Rx Header Buffer Credit is received within a reasonable time limit. This will allow a port sending the header packet to reclaim a Remote Rx Header Buffer Credit within a time limit in order to continue the process of packet transmission. This will also allow a port receiving the header packet enough time to process the header packet.

Similarly for SuperSpeedPlus USB, two CREDIT_HP_TIMERS are specified. A Type 1 CREDIT_HP_TIMER is specified to cover the period of time from when a Type 1 packet has been transmitted and its Remote Type 1 Rx Buffer Credit count is less than four, to when a Remote Type 1 Rx Buffer Credit is received and its Remote Type 1 Rx Buffer Credit count is back to four. A Type 2 CREDIT_HP_TIMER is specified to cover the period of time from when a Type 2 packet has been transmitted and its Remote Type 2 Rx Buffer Credit count is less than four, to when a Remote Type 2 Rx Buffer Credit is received and its Remote Type 2 Rx Buffer Credit count is back to four. The timeout value for the CREDIT_HP_TIMER is listed in Table 7-7.

For SuperSpeed USB, the operation of the CREDIT_HP_TIMER shall be based on the following rules:

- A port shall have a CREDIT_HP_TIMER that is active only in U0 and if one of the following conditions is met:
 - 1) A port has its Remote Rx Header Buffer Credit Count less than four.
 - 2) A port is expecting the Header Sequence Number Advertisement and the Rx Header Buffer Credit Advertisement from its link partner.
- The CREDIT_HP_TIMER shall be started when a header packet or a retried header packet is sent, or when a port enters U0.
- The CREDIT_HP_TIMER shall be reset when a valid LCRD_x is received.
- The CREDIT_HP_TIMER shall be restarted if a valid LCRD_x is received and the Remote Rx Header Buffer Credit Count is less than four.
- A port shall transition to Recovery if the following two conditions are met:
 - 1) CREDIT_HP_TIMER times out.
 - 2) The transmission of an outgoing header packet is completed or the transmission of an outgoing DPP is either completed with DPPEND or terminated with DPPABORT.

NOTE This is to allow a graceful transition to Recovery without a header packet being truncated.

For SuperSpeedPlus USB, the operation of the Type 1/Type 2 CREDIT_HP_TIMERS shall be based on the following rules:

- A port shall have its Type 1/Type 2 CREDIT_HP_TIMERS that are active only in U0 and if one of the following conditions is met:
 - 1) A port has its respective Remote Type 1/Type 2 Rx Buffer Credit Count less than four.
 - 2) A port is expecting the Header Sequence Number Advertisement and the Type 1/Type 2 Rx Buffer Credit Advertisements from its link partner.
- The Type 1/Type 2 CREDIT_HP_TIMERS shall be started when their respective packet or retried packet is sent, or when a port enters U0.
- The Type 1 or Type 2 CREDIT_HP_TIMER shall be reset when the respective LCRD1_x or LCRD2_x is received.
- The Type 1 or Type 2 CREDIT_HP_TIMER shall be restarted if a valid LCRD1_x or LCRD2_x is received and the respective Remote Type 1 or Type 2 Rx Buffer Credit Count is less than four.
- A port shall transition to Recovery if the Type 1 or Type 2 CREDIT_HP_TIMER times out.

Table 7-7 – Transmitter Timers Summary

Timers	Timeout Value (µs)
PENDING_HP_TIMER	3
CREDIT_HP_TIMER	5 000
Type 1/Type 2 CREDIT_HP_TIMER	

7.2.4.2 Link Power Management and Flow

Requests to transition to low power link states are done at the link level during U0. Link commands LGO_U1, LGO_U2, and LGO_U3 are sent by a port as a request to enter a low power link state. LAU or LXU is sent by the other port as the response. LPMA is sent by a port in response only to LAU. Details on exit/wake from a low power link state are described in Sections 7.5.7, 7.5.8, and 7.5.9.

7.2.4.2.1 Power Management Link Timers

A port shall have three timers for link power management. First, a PM_LC_TIMER is used for a port initiating an entry request to a low power link state. It is designed to ensure a prompt

entry to a low power link state. Second, a PM_ENTRY_TIMER is used for a port accepting the entry request to a low power link state. It is designed to ensure that both ports across the link are in the same low power link state regardless if the LAU or LPMA is lost or corrupted. Finally, a Ux_EXIT_TIMER is used for a port to initiate the exit from U1 or U2. It is specified to ensure that the duration of U1 or U2 exit is bounded and the latency of a header packet transmission is not compromised. The timeout values of the three timers are specified in Table 7-8.

A port shall operate the PM_LC_TIMER based on the following rules:

- A port requesting a low power link state entry shall start the PM_LC_TIMER after the last symbol of the LGO_Ux link command is sent.
- A port requesting a low power link state entry shall disable and reset the PM_LC_TIMER upon receipt of the LAU or LXU.

A port shall operate the PM_ENTRY_TIMER based on the following rules:

- A port accepting the request to enter a low power link state shall start the PM_ENTRY_TIMER after the last symbol of the LAU is sent.
- A port accepting the request to enter a low power link state shall disable and reset the PM_ENTRY_TIMER upon receipt of an LPMA or a TS1 ordered set.

A port shall operate the Ux_EXIT_TIMER based on the following rules.

- A port initiating U1 or U2 exit shall start the Ux_EXIT_TIMER when it starts to send LFPS Exit handshake signal.
- A port initiating U1 or U2 exit shall disable and reset the Ux_EXIT_TIMER upon entry to U0.

Table 7-8 – Link Flow Control Timers Summary

Timers	Timeout Value (µs)
PM_LC_TIMER	3
PM_ENTRY_TIMER	6
Ux_EXIT_TIMER	6 000

7.2.4.2.2 Low Power Link State Initiation

- A port shall not send a LGO_U1, LGO_U2 or LGO_U3 unless it meets all of the following:
 - 1) It has transmitted LGOOD_n and LCRD_x or LCRD1_x/LCRD2_x for all the packets received.
 - 2) It has received LGOOD_n and LCRD_x or LCRD1_x/LCRD2_x for all the packets transmitted.

NOTE This implies all credits must be received and returned before a port can initiate a transition to a low power link state.

- 3) It has no pending packets for transmission.
- 4) It has completed the Header Sequence Number Advertisement and the Rx Header Buffer Credit Advertisement or Type 1/Type 2 Rx Buffer Credit Advertisements upon entry to U0.

NOTE This implies that a port has sent the Header Sequence Number Advertisement and the Rx Header Buffer Credit Advertisement or Type 1/Type 2 Rx Buffer Credit Advertisements to its link partner, and also received the Header Sequence Number Advertisement and the Rx Header Buffer Credit Advertisement or Type 1/Type 2 Rx Buffer Credit Advertisements from its link partner.

- 5) It is directed by a higher layer to initiate entry. Examples of when a higher layer may direct the link layer to initiate entry are: (a) the U1 or U2 inactivity timer expires (refer to PORT_U1_TIMEOUT, PORT_U2_TIMEOUT in Chapter 10); (b) reception of a

SetPortFeature(PORT_LINK_STATE) request; and (c) device implementation specific mechanisms.

- 6) It has met higher layer conditions for initiating entry. Examples are: (a) U1_enable/U2_enable is set or U1_TIMEOUT/U2_TIMEOUT is not equal zero; (b) device has received an ACK TP for each and every previously transmitted packet; (c) device is not waiting for a TP following a PING; and (d) device is not waiting for a timestamp following a timestamp request (for these and any other examples, refer to Chapter 8).
- A port shall do one of the following in response to receiving an LGO_U1 or LGO_U2:
 - 1) A port shall send an LAU if the Force Link PM Accept field is asserted due to having received a Set Link Function LMP.
 - 2) A port shall send an LAU if all of the following conditions are met:
 - a) It has transmitted an LGOOD_n, LCRD_x or LCRD1_x/LCRD2_x sequence for all packets received.
 - b) It has received an LGOOD_n, LCRD_x or LCRD1_x/LCRD2_x sequence for all packets transmitted.
 - c) It has no pending packets for transmission.
 - d) It is not directed by a higher layer to reject entry. Examples of when a higher layer may direct the link layer to reject entry are: (1) Downstream port is not enabled for U1 or U2 (i.e., PORT_U1_TIMEOUT or PORT_U2_TIMEOUT reset to zero); (2) When a device has not received an ACK TP for a previously transmitted packet (refer to Chapter 8); and (3) When a device receives a ping TP (refer to the ping packet definition in Chapter 8 for more information).
 - 3) A port shall send an LXU if any of the above conditions are not met.

7.2.4.2.3 U1/U2 Entry Flow

Either a downstream port or an upstream port may initiate U1/U2 entry or exit. Entry to a low power U1 or U2 link state is accomplished by using the link commands defined in Table 7-5.

- A port shall send a single LGO_U1 or LGO_U2 to request a transition to a low power link state.
- Upon issuing LGO_Ux, a port shall start its PM_LC_TIMER.
- A port shall either accept LGO_Ux with a single LAU or shall reject LGO_U1 or LGO_U2 with a single LXU and remain in U0.
- Upon sending LGO_U1 or LGO_U2, a port shall not send any packets until it has received LXU or re-entered U0.
- Upon sending LGO_U1 or LGO_U2, a port shall continue receiving and processing packets and link commands.
- Upon receiving LXU, a port shall remain in U0.
- A port shall initiate transition to Recovery if a single LAU or LXU is not received upon PM_LC_TIMER timeout.
- Upon issuing LAU, a port shall start PM_ENTRY_TIMER.
- Upon receiving LAU, a port shall send a single LPMA and then enter the requested low power link state.
- Upon issuing LAU or LPMA, a port shall not send any packets or link commands.
- A port that sends LAU shall enter the corresponding low power link state upon receipt of LPMA before PM_ENTRY_TIMER timeout.
- A port that sends LAU shall enter the requested low power link state upon PM_ENTRY_TIMER timeout and if all of the following conditions are met:
 - 1) LPMA is not received.
 - 2) No TS1 ordered set is received.

NOTE This implies LPMA is corrupted and the port issuing LGO_Ux has entered Ux.

- A port that has sent LAU shall enter Recovery before PM_ENTRY_TIMER timeout if a TS1 ordered set is received.

NOTE This implies LAU was corrupted and the port issuing LGO_Ux has entered Recovery.

- A port that has sent LAU shall not respond with Ux LFPS exit handshake defined in Section 6.9.2 before PM_ENTRY_TIMER timeout and if LFPS Ux_Exit signal is received.

NOTE This implies LPMA was corrupted and the port issuing LGO_Ux has initiated Ux exit. Under this situation, the port sending LAU shall complete the low power link state entry process and then respond to Ux exit.

There also exists a situation where a port transitions from U1 to U2 directly.

- A port in U1 shall enter U2 directly if the following two conditions are met:
 - 1) The port's U2 inactivity timer is enabled.
 - 2) The U2 inactivity timer times out and no U1 LFPS exit signal is received.

7.2.4.2.4 U3 Entry Flow

Only a downstream port can initiate U3 entry. An upstream port shall not reject U3 entry.

- Upon directed, a downstream port shall initiate U3 entry process by sending LGO_U3.
- Upon issuing LGO_U3, a downstream port shall start PM_LC_TIMER.
- An upstream port shall send LAU in response to LGO_U3 request by a downstream port.
- An upstream port shall not send any packets or link commands subsequent to sending an LAU.
- Upon issuing LGO_U3, a downstream port shall ignore any packets sent by an upstream port.

NOTE This is a corner condition that an upstream port is sending a header packet before receiving LGO_U3.

- Upon Receiving LGO_U3, an upstream port shall respond with an LAU. The processing of all the unacknowledged packets shall be aborted.
- Upon issuing LAU, an upstream port shall start PM_ENTRY_TIMER.
- A downstream port shall send a single LPMA and then transition to U3 when LAU is received.
- A downstream port shall transition to Recovery and reinitiate U3 entry after re-entry to U0 if all of the following three conditions are met:
 - 1) The PM_LC_TIMER times out.
 - 2) LAU is not received.
 - 3) The number of consecutive U3 entry attempts is less than three.
- An upstream port shall transition U3 when one of the following two conditions is met:
 - 1) LPMA is received
 - 2) The PM_ENTRY_TIMER times out and LPMA is not received
- A downstream port shall transition to eSS.Inactive when it fails U3 entry on three consecutive attempts.

7.2.4.2.5 Concurrent Low Power Link Management Flow

Concurrent low power link management flow applies to situations where a downstream port and an upstream port both issue a request to enter a low power link state.

- If a downstream port has sent an LGO_U1, LGO_U2, or LGO_U3 and also received an LGO_U1 or LGO_U2, it shall send an LXU.

- If an upstream port has sent an LGO_U1 or LGO_U2 and also received an LGO_U1, LGO_U2, it shall wait until receipt of an LXU and then send either an LAU or LXU.
- If an upstream port has sent an LGO_U1 or LGO_U2 and also received an LGO_U3 from a downstream port, it shall wait until the reception of an LXU and then send an LAU.
- If a downstream port is directed by a higher layer to initiate a transition to U3, and a transition to U1 or U2 has been initiated but not yet completed, the port shall first complete the in-process transition to U1 or U2, then return to U0 and request entry to U3.

7.2.4.2.6 Concurrent Low Power Link Management and Recovery Flow

Concurrent low power link management and Recovery flow applies to situations where a port issues a low power link state entry and another port issues Recovery. The port that issues the low power link state entry shall meet the following rules:

- Upon issuing LGO_Ux, the port shall transition to Recovery if a TS1 ordered set is received.
- The port shall reinitiate low power link state entry process described in Section 7.2.4.2.3 and 7.2.4.2.4 upon re-entry to U0 from Recovery if the conditions to enter a low power link state are still valid.

7.2.4.2.7 Low Power Link State Exit Flow

Exit from a low power link state refers to exit from U1/U2, or wakeup from U3. It is accomplished by the LFPS Exit signaling defined in Section 6.9.2. A successful LFPS handshake process will lead both a downstream port and an upstream port to Recovery.

A Ux_EXIT_TIMER defined in Section 7.2.4.2.1 is only applied when a port is attempting an exit from U1 or U2. It shall not be applied when a port is initiating a U3 wakeup.

The exit from U1/U2 shall meet the following flow. The U3 wakeup follows the same flow with the exception that Ux_EXIT_TIMER is disabled during U3 wakeup.

- If a port is initiating U1/U2 Exit, it shall start sending U1/U2 LFPS Exit handshake signal defined in Section 6.9.2 and start the Ux_EXIT_TIMER.
- If a port is initiating U3 wakeup, it shall start sending U3 LFPS wakeup handshake signal defined in Section 6.9.2.
- A port upon receiving U1/U2 Exit or U3 wakeup LFPS handshake signal shall start U1/U2 exit or U3 wakeup by responding with U1/U2 Exit or U3 wakeup LFPS signal defined in Section 6.9.2.
- Upon a successful LFPS handshake before tNoLFPSResponseTimeout defined in Table 6-30, a port shall transition to Recovery.
- A port initiating U1 or U2 Exit shall transition to eSS.Inactive if one of the following two conditions is met:
 - 1) Upon tNoLFPSResponseTimeout and the condition of a successful LFPS handshake is not met.
 - 2) Upon Ux_EXIT_TIMER timeout, the link has not transitioned to U0.
- A port initiating U3 wakeup shall remain in U3 when the condition of a successful LFPS handshake is not met upon tNoLFPSResponseTimeout and it may initiate U3 wakeup again after a minimum of 100-ms delay.
- A root port not able to respond to U3 LFPS wakeup within tNoLFPSResponseTimeout shall initiate U3 LFPS wakeup when it is ready to return to U0.

7.3 Link Error Rules/Recovery

7.3.1 Overview of Enhanced SuperSpeed Bit Errors

The Enhanced SuperSpeed timing budget is based on a link's statistical random bit error probability less than 10^{-12} . Packet framings and link command framing are tolerant to one symbol error. Details on bit error detection under link flow control are described in Section 7.2.4.

7.3.2 Link Error Types, Detection, and Recovery

Data transfers between the two link partners are carried out using the form of a packet. A set of link commands is defined to ensure the successful packet flow across the link. Other link commands are also defined to manage the link connectivity. When symbol errors occur on the link, the integrity of a packet or a link command can be compromised. Therefore, not only a packet or a link command needs to be constructed to increase the error tolerance, but the link data integrity handling also needs to be specified such that any errors that will invalidate or corrupt a packet or a link command can be detected and a link error can be recovered.

There are various types of errors at the link layer. This includes an error on a packet or a link command, or an error during the link training process, or an error when a link is in transition from one state to another. The detection and recovery from those link errors are described with details in this section.

7.3.3 Link Error Statistics

To facilitate the quality of the link operation, two counts of link error statistics are implemented and accessible by the upper layer for its decision if port re-configuration is needed.

7.3.3.1 Link Error Count

The Link Error Count is defined to record the number of events when a port transitions from U0 to Recovery to recover an error event. Except for the upstream port in SuperSpeed operation, all ports shall implement the Link Error Count.

The operation of Link Error count shall adhere to the following rules.

- A port in SuperSpeedPlus operation shall implement the Link Error Count that counts up to 65 535 error events. The Link Error Count shall saturate if it has reached its maximum count value.
- The Link Error Count shall be reset to zero in any one of the following conditions.
 - 1) PowerOn Reset
 - 2) Entry to Polling.Idle
 - 3) Directed
 - 4) Hot Reset
- The Link Error Count shall be incremented by one each time a port transitions from U0 to Recovery to recover an error event.

7.3.3.2 Soft Error Count

The Soft Error Count is defined to record the number of error events that are either correctable or detectable and do not require the link to recover through transition to Recovery. Only a port in SuperSpeedPlus operation may optionally implement the Soft Error Count.

The operation of the Soft Error Count shall adhere to the following rules.

- A port in SuperSpeedPlus operation shall count up to 65 535 error events. The Soft Error Count shall saturate if it has reached its maximum count value
- The Soft Error Count shall be reset to zero in any one of the following conditions.
 - 1) PowerOn Reset
 - 2) Entry to Polling.Idle
 - 3) Directed
 - 4) Hot Reset
- The Soft Error Count shall increment by one if any of the following errors is detected.
 - 1) Single-bit error in the block header.
 - 2) CRC-5 or CRC-16 or CRC-32 error.
 - 3) Single symbol framing error.
 - 4) Idle Symbol error.
 - 5) Single SKP symbol error.
 - 6) Optionally for error in the length field replica of DPH.

7.3.4 Header Packet Errors

Several types of header packet errors are detected. They are:

- 1) Missing of a header packet
- 2) Invalid header packet due to CRC errors
- 3) Mismatch of a Rx Header Sequence Number

Regardless, the Link Error Count is incremented for only one class of errors in the link layer, and those are errors which will cause the link to transition to Recovery. For errors that will not cause the link to enter Recovery, the Link Error Count shall remain unchanged.

7.3.4.1 Packet Framing Error

A packet framing ordered set is constructed such that any single symbol corruption within the ordered set will not prevent its packet framing recognition.

Header packet framing and DPP framing are all constructed using four symbol ordered sets. A header packet contains only one packet framing ordered set at the beginning of the packet defined in Section 7.2.1. A DPP begins with start packet framing ordered set and ends with end packet framing ordered set as defined in Section 7.2.2.

- A valid HPSTART ordered set, or a valid DPHSTART ordered set, or a valid DPP framing ordered set shall be declared if the following two conditions are met:
 - 1) At least three of the four symbols in the four consecutive symbol periods are valid packet framing symbols.
 - 2) The four symbols are in the order defined in Table 7-9.

NOTE If an HPSTART ordered set or a DPHSTART ordered set has two or more symbols corrupted, a header packet will not be detectable and, therefore, result in missing of a header packet. Similarly, if a DPP framing ordered set is corrupted in SuperSpeed operation, it will result in missing of a data packet payload. For SuperSpeed operation, a corruption of a DPP framing ordered set will result in the loss of data packet payload boundary.

- Missing of a header packet shall result in a port transitioning to Recovery depending on which one of the following conditions becomes true first:
 - 1) A port transmitting the header packet upon its PENDING_HP_TIMER timeout.
 - 2) A port receiving the header packet upon detection of a Rx Header Sequence Number error.

- Missing of a DPP framing ordered set in SuperSpeedPlus operation shall result in a port transitioning to Recovery.
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs.

**Table 7-9 – Valid Packet Framing Symbol Order
(Sx is One of SHP, DPHP, SDP, END or EDB)**

Symbol 0	Symbol 1	Symbol 2	Symbol 3	Comment
Sx	Sx	Sx	EPF	All symbols are valid
Corrupt	Sx	Sx	EPF	First symbol corrupted
Sx	Corrupt	Sx	EPF	Second symbol corrupted
Sx	Sx	Corrupt	EPF	Third symbol corrupted
Sx	Sx	Sx	Corrupt	EPF corrupted

7.3.4.2 Header Packet Error

Each header packet contains a CRC-5 and a CRC-16 to ensure that the data integrity of a header packet can be verified. A CRC-5 is used to detect bit errors in the Link Control Word. A CRC-16 is used to detect bit errors in the packet header. A header packet error can be detected using CRC-5 or CRC-16 checks.

- A header packet error shall be declared if the following conditions are true:
 - 1) A valid HPSTART ordered set or DPHSTART ordered set is detected.
 - 2) Either CRC-5 or CRC-16 check fails as defined in Section 7.2.1 or additionally for SuperSpeed USB, any K-symbol occurrence in the packet header or Link Control Word that prevents CRC-5 or CRC-16 checks from being completed.
- A port receiving the header packet shall send an LBAD as defined in Section 7.2.4.1 if it detects a header packet error. The Link Error Count shall remain unchanged.
- If a port fails to receive a header packet for three consecutive times, it shall transition to Recovery. The Link Error Count shall be incremented by one. Refer to Section 7.2.4.1.4 for details.

7.3.4.3 Rx Header Sequence Number Error

Each port contains an Rx Header Sequence Number that is defined in Section 7.2.4.1 and initialized upon entry to U0. Upon receiving a header packet, a port is required to compare the Header Sequence Number embedded in the header packet with the Rx Header Sequence Number stored in its receiver. This ensures that header packets are transmitted and received in an orderly manner. A missing or corrupted header packet can be detected.

- An Rx Header Sequence Number error shall occur if the following conditions are met:
 - 1) A header packet is received and no header packet error is detected.
 - 2) The Header Sequence Number in the received header packet does not match the Rx Header Sequence Number.
- A port detecting an Rx Header Packet Sequence Number error shall transition to Recovery.
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs.

7.3.5 Link Command Errors

A link command consists of four-symbol link command frame ordered set, LCSTART, followed by a two-symbol link command word, and its repeat. A link command is constructed such that any single symbol corruption within the link command frame ordered set will not invalidate the

recognition of a link command. Additionally for SuperSpeedPlus USB, any single-bit error in the two link command words will not corrupt the correct parsing of a link command.

- A detection of a link command shall be declared if the following two conditions are met:
 - 1) At least three of the four symbols in four consecutive symbol periods are valid link command symbols.
 - 2) The four symbols are in the order described in Table 7-10.
- For SuperSpeed USB, a valid link command is declared if both link command words are the same, they both contain valid link command information as defined in Table 7-4, and they both pass the CRC-5 check.
- For SuperSpeedPlus USB, a valid link command is declared if one of the following conditions is met:
 - 1) Both link command words are the same, they contain valid link command information as defined in Table 7-4, and they pass the CRC-5 check.
 - 2) One of the link command words contains valid link command information as defined in Table 7-4, and passes the CRC-5 check, and the other link command word either contains invalid link command information, or fails the CRC-5 check.
- An invalid link command is declared upon detection of a link command and the conditions to meet a valid link command are not met.
- An invalid link command shall be ignored.
- A port detecting missing of LGOOD_n or LCRD_x or LCRD1_x/LCRD2_x shall transition to Recovery.

NOTE Missing LGOOD_n is declared when two consecutive LGOOD_n received are not in numerical order. Missing LGOOD_n, or LBAD, or LRTY can also be inferred upon PENDING_HP_TIMER timeout. Missing LCRD_x or LCRD1_x/LCRD2_x is declared when two consecutive LCRD_x or LCRD1_x/LCRD2_x received are not in alphabetical order, or upon CREDIT_HP_TIMER or Type 1/Type 2 CREDIT_HP_TIMER times out and LCRD_x or LCRD1_x/LCRD2_x is not received.

- A port detecting missing of LGO_Ux, or LAU, or LXU shall transition to Recovery.

NOTE Detection of missing LGO_Ux, or LAU, or LXU is declared upon PM_LC_TIMER timeout and LAU or LXU is not received.

- A downstream port detecting missing of LUP shall transition to Recovery (refer to Section 7.5.6 for LUP detection).

NOTE Missing of LPMA will not transition the link to Recovery. It will only cause an Ux entry delay for the port accepting LGO_Ux (refer to Section 7.2.4.2 for details).

- An upstream port detecting missing of LDN shall transition to Recovery (refer to Section 7.5.6 for LDN detection).
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs due to an error.

Table 7-10 – Valid Link Command Symbol Order

Symbol 0	Symbol 1	Symbol 2	Symbol 3	Comment
SLC	SLC	SLC	EPF	All symbols are valid
Corrupt	SLC	SLC	EPF	First SLC corrupted
SLC	Corrupt	SLC	EPF	Second SLC corrupted
SLC	SLC	Corrupt	EPF	Third SLC corrupted
SLC	SLC	SLC	Corrupt	EPF corrupted

7.3.6 ACK Tx Header Sequence Number Error

Each port has an ACK Tx Header Sequence Number that is defined in Section 7.2.4.1. The ACK Tx Header Sequence Number is initialized during the Header Sequence Number

Advertisement. After a header packet is transmitted, a port is expecting to receive an LGOOD_n from its link partner as an explicit acknowledgement that the header packet is received properly. Upon receiving LGOOD_n, the Header Sequence Number contained in LGOOD_n will be compared with the ACK Tx Header Sequence Number. The outcome of the comparison will determine if an ACK Tx Header Sequence Number error has occurred.

- An ACK Tx Header Sequence Number error shall be declared if the following conditions are met:
 - 1) A valid LGOOD_n is received.
 - 2) The Header Sequence Number in the received LGOOD_n does not match the ACK Tx Header Sequence Number.
 - 3) The LGOOD_n is not for Header Sequence Number Advertisement.
- A port detecting an ACK Tx Header Sequence Number error shall transition to Recovery.
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs.

7.3.7 Header Sequence Number Advertisement Error

Each port is required to first perform a Header Sequence Number Advertisement upon entry to U0. The details of a Header Sequence Number Advertisement are described in Section 7.2.4. A Header Sequence Number Advertisement is the first step of the link initialization to ensure that the link flow is maintained un-interrupted before and after Recovery. Any errors occurred during the Header Sequence Number Advertisement must be detected and proper error recovery must be initiated.

- A Header Sequence Number Advertisement error shall occur if one of the following conditions is true:
 - 1) Upon PENDING_HP_TIMER timeout and the Header Sequence Number Advertisement not received
 - 2) A header packet received before sending Header Sequence Number Advertisement
 - 3) LCRD_x or LCRD1_x/LCRD2_x or LGO_Ux received before receiving Header Sequence Number Advertisement
- A port detecting any Header Sequence Number Advertisement error shall transition to Recovery.
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs.

7.3.8 SuperSpeed Rx Header Buffer Credit Advertisement Error

Each port is required to perform the Rx Header Buffer Credit Advertisement after Header Sequence Number Advertisement upon entry to U0. The details of Rx Header Buffer Credit Advertisement are described in Section 7.2.4.

- An Rx Header Buffer Credit Advertisement error shall occur if one of the following conditions is true:
 - 1) Upon CREDIT_HP_TIMER timeout and no LCRD_x received.
 - 2) A header packet received before sending LCRD_x.
 - 3) LGO_Ux received before receiving LCRD_x.
- A port detecting an Rx Header Buffer Credit Advertisement Error shall transition to Recovery.
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs.

7.3.9 SuperSpeedPlus Type 1/Type 2 Rx Buffer Credit Advertisement Error

For SuperSpeedPlus USB, the operation of Type 1 Rx Buffer Credit Advertisement error detection is the same as Type 2 Rx Buffer Credit Advertisement.

Each port is required to perform the Type 1 and Type 2 Rx Buffer Credit Advertisements after Header Sequence Number Advertisement upon entry to U0. The details of Type 1/Type 2 Rx Buffer Credit Advertisements are described in Section 7.2.4.

- A Type 1/Type 2 Rx Buffer Credit Advertisement error shall occur if one of the following conditions is true:
 - 1) Upon Type 1 or Type 2 CREDIT_HP_TIMER timeout and its respective LCRD1_x or LCRD2_x is not received.
 - 2) A Type 1 packet received before sending LCRD1_x, or Type 2 packet received before sending LCRD2_x.
 - 3) LGO_Ux received before receiving LCRD1_x or LCRD2_x.
- A port detecting a Type 1/Type 2 Rx Buffer Credit Advertisement Error shall transition to Recovery.
- The Link Error Count shall be incremented by one each time a transition to Recovery occurs.

7.3.10 Training Sequence Error

Symbol corruptions during the TS1 and TS2 ordered sets in Polling.Active, Polling.Configuration, Recovery.Active, and Recovery.Configuration substates are expected until the requirements are met to transition to the next state. A timeout from any one of these substates is considered a Training Sequence error.

- A timeout from either Polling.Active, Polling.Configuration, Recovery.Active, or Recovery.Configuration substate shall result in a Training Sequence error.
- For SuperSpeedPlus operation, upon detecting a Training Sequence error in Polling.Active or Polling.Configuration, the port shall transition to Polling.PortMatch to negotiate for SuperSpeed operation. Refer to Section 7.5.4 for details.
- For SuperSpeed operation, upon detecting a Training Sequence error, one of the following link state transitions shall be followed:
 - 1) A downstream port shall transition to Rx.Detect if a Training Sequence error occurs during Polling and *cPollingTimeout* is less than two.
 - 2) A downstream port shall transition to eSS.Inactive if a Training Sequence error occurs during Polling and *cPollingTimeout* is two.
 - 3) An upstream port of a hub shall transition to Rx.Detect if a Training Sequence error occurs during Polling.
 - 4) An upstream port of a peripheral device shall transition to eSS.Disabled if a Training Sequence error occurs during Polling.
 - 5) A downstream port shall transition to eSS.Inactive if a Training Sequence error occurs during Recovery and the transition to Recovery is not an attempt for Hot Reset.
 - 6) A downstream port shall transition to Rx.Detect if a Training Sequence error occurs during Recovery.Active and Recovery.Configuration and the transition to Recovery is an attempt for Hot Reset.
 - 7) An upstream port shall transition to eSS.Inactive if a Training Sequence error occurs during Recovery.
- The Link Error Count shall remain unchanged.

7.3.11 SuperSpeed 8b/10b Errors

There are two types of errors when a receiver decodes 8b/10b symbols. One is a disparity error that is declared when the running disparity of the received 8b/10b symbols is not +2, or 0, or –2. The other is a decode error when an unrecognized 8b/10b symbol is received.

Upon receiving notification of an 8b/10b error:

- A port may optionally do the following:
 - 1) If the link is receiving a header packet, it shall send LBAD.
 - 2) If the link is receiving a link command, it shall ignore the link command.
 - 3) If the link is receiving a DPP, it shall drop the DPP.
- The Link Error Count shall remain unchanged.

7.3.12 SuperSpeedPlus Block Header Errors

There are two types of block header errors, a correctable single bit block header error, and a detectable but not correctable two-bit block header error.

- Upon detecting a single-bit block header error, the PHY shall correct it and report the error event to the link layer. The Soft Error Count shall be incremented by one if the Soft Error Count is implemented.
- Upon detecting two-bit block header error, the PHY shall report the error event to the link layer. The port shall transition Recovery. The Link Error Count shall be incremented by one.

7.3.13 Summary of Error Types and Recovery

Table 7-11 summarizes the link error types, error count, and different error paths to restore the link.

- The link error shall be counted each time a link transitions to Recovery due to an error.
- The link error shall be counted by a downstream port.
- The Link Error Count shall be reset upon PowerOn Reset, Warm Reset, Hot Reset, or whenever a port enters Polling.Idle.

Situations also exist where an unexpected link command or header packet is received. These include but are not limited to the following:

- 1) Receiving an unexpected link command such as LBAD, LRTY, LAU, LXU, or LPMA before receiving the Header Sequence Number Advertisement and the Remote Rx Header Buffer Credit Advertisement or Type 1/Type 2 Rx Buffer Credit Advertisements.
- 2) Receiving the Header Sequence Number Advertisement after entry to U0 from Recovery with its ACK Tx Header Sequence Number not corresponding to any header packets in the Tx Header Buffers or Type 1/Type 2 Tx Header Buffers.
- 3) Receiving LRTY without sending LBAD.
- 4) Receiving LGOOD_n that is neither a Header Sequence Number Advertisement, nor for header packet acknowledgement.
- 5) Receiving LAU, or LXU without sending LGO_Ux.
- 6) Receiving LPMA without sending LAU.
- 7) Receiving an unexpected header packet during link initialization.

These error situations are largely not due to link errors. A port's behavior under these situations is undefined and implementation specific. It is recommended that a port ignore those unexpected link commands or header packets.

If the ports are directed to different link states based on TS2 ordered set, the downstream port's TS2 ordered set overrides the upstream port's TS2 ordered set. For example, if a downstream port issues Hot Reset in its TS2 ordered set, and an upstream port issues Loopback mode, Hot Reset overrides Loopback. The ports shall enter Hot Reset.

Table 7-11 – Error Types and Recovery

Error Type	Description/Example	Error Recovery Path	Update Link Error Count?	Update Soft Error Count? (SuperSpeedPlus USB)
Missing Header Packet Framing	Only a valid packet framing ordered set will be declared in the receiver side.	Delayed transition to Recovery	Yes	No
Header Packet Error	Any header packet CRC is bad.	Header packet retry process	No	Yes
Rx Header Sequence Number Error	The Header Sequence Number in the received header packet does not match the Rx Header Sequence Number.	Recovery	Yes	No
ACK Tx Header Sequence Number Error	The Header Sequence Number in the received LGOOD_n (not Header Sequence Number Advertisement) does not match ACK Tx Header Sequence Number.	Recovery	Yes	No
Header Sequence Number Advertisement Error	1) LGOOD_n not received upon PENDING_HP_TIMER timeout. 2) A header packet received before sending LGOOD_n. 3) LCRD_x or LCRD1_x/LCRD2_x or LGO_Ux received before receiving LGOOD_n.	Recovery	Yes	No
Rx Header Buffer Credit Advertisement Error (SuperSpeed USB)	1) LCRD_x not received upon CREDIT_HP_TIMER timeout. 2) A header packet received before sending LCRD_x. 3) LGO_Ux received before receiving LCRD_x.	Recovery	Yes	No
Type 1/Type 2 Rx Buffer Credit Advertisement Error (SuperSpeedPlus USB)	1) LCRD1_x/LCRD2_x not received upon Type 1/Type 2 CREDIT_HP_TIMER timeout. 2) A packet received before sending LCRD1_x/LCRD2_x. 3) LGO_Ux received before receiving LCRD1_x/LCRD2_x.	Recovery	Yes	No
Training Sequence Error	1) Timeout from Polling to Rx.Detect or eSS.Disabled without reaching U0. 2) Timeout from Recovery to eSS.Inactive without reaching U0. 3) Timeout from Recovery to Rx.Detect without reaching U0. 4) Timeout from Polling.Active or Polling.Configuration to Polling.PortMatch (SuperSpeedPlus USB only)	Timeout from Recovery to eSS.Inactive requires software intervention.	No	No
Invalid link command	Valid link command framing but invalid link command word.	Ignored	No	Yes

Error Type	Description/Example	Error Recovery Path	Update Link Error Count?	Update Soft Error Count? (SuperSpeedPlus USB)
Missing link command	No valid link command framing is detected.	Delayed transition to Recovery if missing LGOOD_n or LCRD_x or LCRD1_x/LCRD2_x	Yes	No
8b/10b Error (SuperSpeed USB)	Detected in the PHY layer	N.A.	No	N.A.
128b/132b Soft Error (SuperSpeedPlus USB)	Detected and corrected in PHY layer	Correctable	No	Yes
128b/132b Two-bit Error (SuperSpeedPlus USB)	Detectable	Recovery	Yes	No
Single Bit SKP/SKPEND Error (SuperSpeedPlus USB)	Detectable	Correctable	No	Yes

7.4 PowerOn Reset and Inband Reset

There are two categories of reset associated with a link. The first, PowerOn Reset, restores storage elements, registers, or memories to predetermined states when power is applied. Upon PowerOn Reset, the LTSSM (described in Section 7.5) shall enter Rx.Detect. The second, Inband Reset, uses Enhanced SuperSpeed or LFPS signaling to propagate the reset across the link. There are two mechanisms to complete an Inband Reset, Hot Reset and Warm Reset. Upon completion of either a PowerOn Reset or an Inband Reset, the link shall transition to U0 as described in Section 7.4.2.

7.4.1 PowerOn Reset

PowerOn Reset restores a storage element, register, or memory to a predetermined state when power is applied (refer to Section 9.1.1.2 for clarification of when power is applied for self-powered devices). A port must be responsible for its own internal Reset signaling and timing.

The following shall occur when PowerOn Reset is asserted or while VBUS is off:

- 1) Receiver termination shall meet the $Z_{RX-HIGH-IMP-DC-POS}$ specification defined in Table 6-21.
- 2) Transmitters shall hold a constant DC common mode voltage ($V_{TX-DC-CM}$) defined in Table 6-18.

The following shall occur when PowerOn Reset is completed and VBUS is valid:

- 1) The LTSSM of a port shall be initialized to Rx.Detect.
- 2) The LTSSM and the PHY level variables (such as Rx equalization settings) shall be reset to their default values.
- 3) The receiver termination of a port shall meet the R_{RX-DC} specification defined in Table 6-21.

NOTE Rx termination shall always be maintained throughout operation except for eSS.Disabled

7.4.2 Inband Reset

An Inband Reset shall be generated by a downstream port only when it is directed.

There are two mechanisms to generate an Inband Reset. The first mechanism; Hot Reset, is defined by sending TS2 ordered sets with the Reset bit asserted. A Hot Reset shall cause the LTSSM to transition to the Hot Reset state. Upon completion of Hot Reset, the following shall occur:

- A downstream port shall reset its Link Error Count.
- The port in SuperSpeedPlus operation shall reset the Soft Error Count if implemented.
- The port shall reset its PM timers and the associated U1 and U2 timeout values to zero.
- The port configuration information of an upstream port shall remain unchanged. Refer to Sections 8.4.5 and 8.4.6 for details.
- The PHY level variables (such as Rx equalization settings) shall remain unchanged.
- The LTSSM of a port shall transition to U0.

The second mechanism of an Inband Reset is Warm Reset. The signaling of a Warm Reset is defined as an LFPS signaling meeting the tReset requirements (see Table 6-29). A Warm Reset will cause the LTSSM to transition to Rx.Detect, retrain the link including the receiver equalizer, reset an upstream port, and then transition to U0. An upstream port shall enable its LFPS receiver and Warm Reset detector in all link states except eSS.Disabled. A completion of a Warm Reset shall result in the following.

- A downstream port shall reset its Link Error Count.
- Port configuration information of an upstream port shall be reset to default values. Refer to Sections 8.4.5 and 8.4.6 for details.
- The PHY level variables (such as Rx equalization settings) shall be reinitialized or retrained.
- The LTSSM of a port shall transition to U0 through RxDetect and Polling.

A downstream port may be directed to reset the link in two ways, "PORT_RESET", or "BH_PORT_RESET" as described in Section 10.3.1.6. When a "PORT_RESET" is directed, a downstream port shall issue either a Hot Reset, or a Warm Reset, depending on its LTSSM state. When a "BH_PORT_RESET" is directed, a downstream port shall issue a Warm Reset in any of its LTSSM states except eSS.Disabled.

If a "PORT_RESET" is directed, a downstream port shall issue either a Hot Reset or a Warm Reset based on the following conditions:

- If the downstream port is U3, or Loopback, or Compliance Mode, or eSS.Inactive, it shall use Warm Reset.
- If the downstream port is in U0, it shall use Hot Reset.
- If a downstream port is in a transitory state of Polling or Recovery, it shall use Hot Reset.
- If the downstream port is in U1 or U2, it shall exit U1 or U2 using the LFPS exit handshake, transition to Recovery and then transition to Hot Reset. The following two additional rules apply when the downstream port fails to enter Hot Reset.
 - 1) If a Hot Reset fails due to an LFPS handshake timeout in U1 or U2, a downstream port shall transition to eSS.Inactive until software intervention or upon detection of removal of an upstream port.
 - 2) If a Hot Reset fails due to a TS1/TS2 handshake timeout, a downstream port shall transition to Rx.Detect and attempt a Warm Reset.
- If the downstream port is in eSS.Disabled, an Inband Reset is prohibited.

If a “BH_PORT_RESET” is directed, Warm Reset shall be issued, and the following shall occur:

- A downstream port shall initiate a Warm Reset in all the link states except eSS.Disabled and transition to Rx.Detect.
- An upstream port shall enable its LFPS receiver and Warm Reset detector in all the link states except eSS.Disabled.
- An upstream port receiving Warm Reset shall transition to Rx.Detect. Refer to Section 6.9.3 for Warm Reset Detection.

7.5 Link Training and Status State Machine (LTSSM)

Link Training and Status State Machine (LTSSM) is a state machine defined for link connectivity and the link power management. LTSSM consists of 12 different link states that can be characterized based on their functionalities. First, there are four operational link states, U0, U1, U2, and U3. U0 is a state where an Enhanced SuperSpeed link is enabled. Packet transfers are in progress or the link is idle. U1 is low power link state where no packet transfer is carried out and the Enhanced SuperSpeed link connectivity can be disabled to allow opportunities for saving the link power. U2 is also a low power link state. Compared with U1, U2 allows for further power saving opportunities with a penalty of increased exit latency. U3 is a link suspend state where aggressive power saving opportunities are possible.

Second, there are four link states, Rx.Detect, Polling, Recovery, and Hot Reset, that are introduced for link initialization and training. Rx.Detect represents the initial power-on link state where a port is attempting to determine if its Enhanced SuperSpeed link partner is present. Upon detecting the presence of an Enhanced SuperSpeed link partner, the link training process will be started. Polling is a link state that is defined for the two link partners to have their Enhanced SuperSpeed transmitters and receivers trained, synchronized, and ready for packet transfer. Recovery is a link state defined for retraining the link when the two link partners exit from a low power link state, or when a link partner has detected that the link is not operating in U0 properly and the link needs to be retrained, or when a link partner decides to change the mode of link operation. Hot Reset is a state defined to allow a downstream port to reset its upstream port.

Third, two other link states, Loopback and Compliance Mode, are introduced for bit error test and transmitter compliance test. Finally, two more link states are defined. eSS.Inactive is a link error state where a link is in a non-operable state and software intervention is needed. eSS.Disabled is a link state where Enhanced SuperSpeed connectivity is disabled and the link may operate under USB 2.0 mode.

Configuration information and requests to initiate LTSSM state transitions are mainly controlled by software. All LTSSM references to “directed” refers to upper layer mechanisms.

There are also various timers defined and implemented for LTSSM in order to ensure the successful operation of LTSSM. The timeout values are summarized in Table 7-12. All timers used in the link layer have a tolerance of 0~+50 % accuracy with exception of the U2 inactivity timer (refer to Section 9.4.1 for U2 inactivity timer accuracy). All timeout values must be set to the specified values after PowerOn Reset or Inband Reset. All counters must be also initialized after PowerOn Reset or Inband Reset.

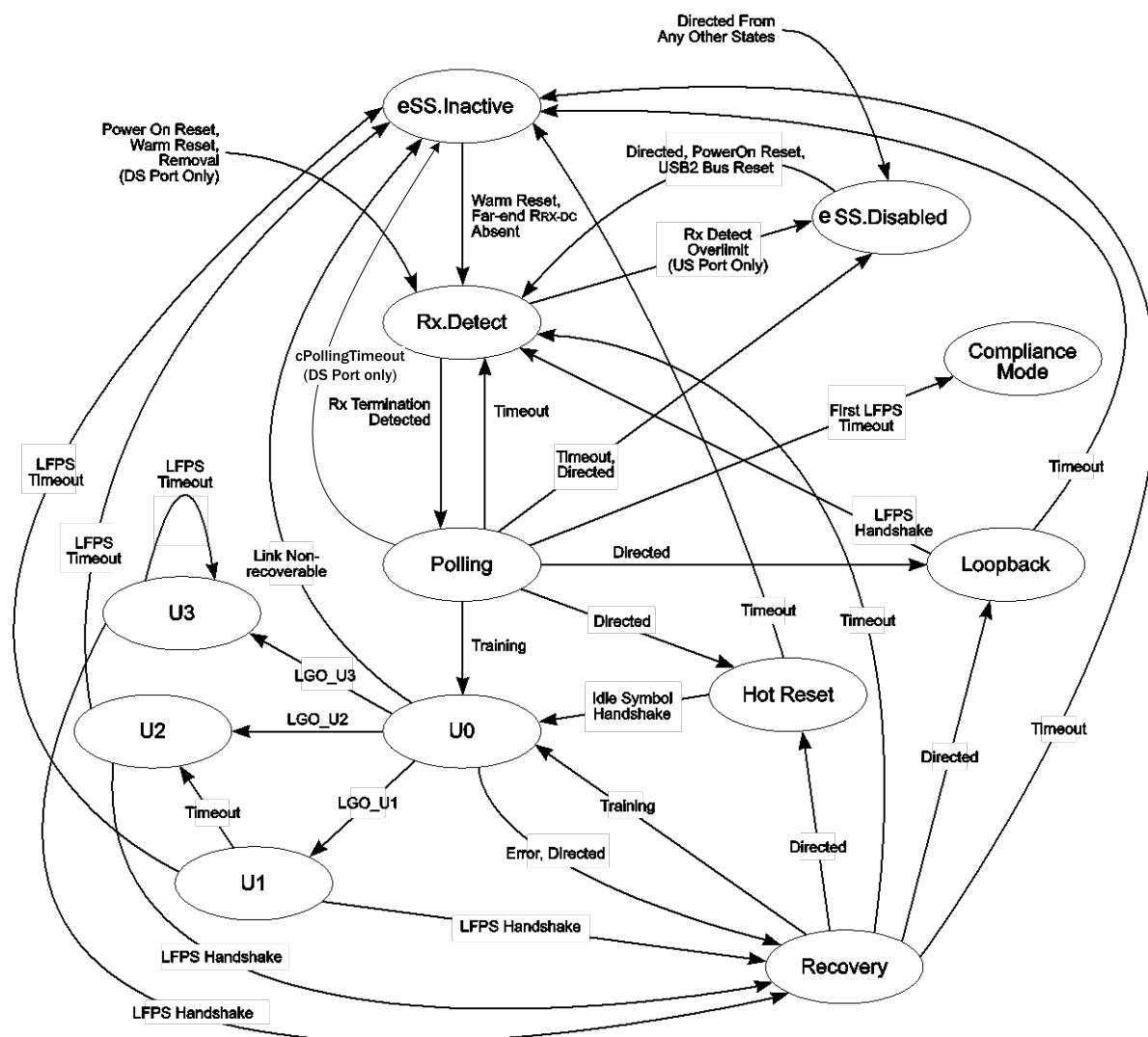
In the state machine descriptions, lists of state entry and exit conditions are not prioritized.

State machine diagrams are overviews and may not include all the transition conditions.

Table 7-12 – LTSSM State Transition Timeouts

Name	Initial State	Timeout to Next State	Timeout Values
teSSInactiveQuietTimeout	eSS.Inactive.Quiet	eSS.Inactive.Disconnect.Detect	12 ms
tRxDetectQuietTimeoutDFP ¹	Rx.Detect.Quiet	Rx.Detect.Active	12 ms (min) 120 ms (max)
tRxDetectQuietTimeoutUFP	Rx.Detect.Quiet	Rx.Detect.Active	12 ms
tPollingLFPSTimeout	Polling.LFPS /Polling.LFSPPlus ²	Compliance/Rx.Detect/ eSS.Disabled/eSS.Inactive	360 ms
tPollingSCDLFPSTimeout (SuperSpeedPlus operation)	Polling.LFPS or Polling.LFSPPlus	Polling.RxEQ	60 µs
tPollingLBPMFLFPSTimeout (SuperSpeedPlus operation)	Polling.PortMatch or Polling.PortConfig	Rx.Detect/ eSS.Disabled/eSS.Inactive	12 ms
tPollingActiveTimeout	Polling.Active ²	Rx.Detect/Polling.PortMatch/ eSS.Disabled/eSS.Inactive	12 ms
tPollingConfigurationTimeout	Polling.Configuration ²	Rx.Detect/Polling.PortMatch/ eSS.Disabled/eSS.Inactive	12 ms
tPollingIdleTimeout	Polling.Idle ²	Rx.Detect/Polling.PortMatch/ eSS.Disabled/ eSS.Inactive	2 ms
tU0RecoveryTimeout	U0	Recovery	1 ms
tU0LTimeout	U0	U0	10 µs
tNoLFPSResponseTimeout	U1	eSS.Inactive	2 ms
PORT_U2_TIMEOUT	U1 ³	U2	U2 Inactivity field set in LMP (refer to Section 7.4 for details)
tU1PingTimeout	U1	Rx.Detect	300 ms
tNoLFPSResponseTimeout	U2	eSS.Inactive	2 ms
tNoLFPSResponseTimeout	U3	U3	10 ms
tRecoveryActiveTimeout	Recovery.Active	eSS.Inactive, Rx.Detect	12 ms
tRecoveryConfigurationTimeout	Recovery.Configuration	eSS.Inactive, Rx.Detect	6 ms
tRecoveryIdleTimeout	Recovery.Idle	eSS.Inactive	2 ms
tLoopbackExitTimeout	Loopback.Exit	eSS.Inactive	2 ms
tHotResetActiveTimeout	Hot Reset.Active	eSS.Inactive	12 ms
tHotResetExitTimeout	Hot Reset.Exit	eSS.Inactive	2 ms
tU3WakeupRetryDelay	U3	U3	100 ms
tU2RxdetDelay	U2	U2	100 ms
tU3RxdetDelay	U3	U3	100 ms
¹ Implementations are recommended to consider system states when choosing when to use a larger value of the tRxDetectQuietTimeoutDFP timer. Using a smaller value when the system is active and a larger value when the system is in a standby/sleep or other idle state allows for higher responsiveness to connect events during active states while enabling power savings in an idle system state. ² Upon Polling timeout, a port shall transition to different states. Refer to Section 7.5.4.3 for details. ³ The accuracy of U2 inactivity timer is specified in Section 9.4.1.			

All state machines diagrams have descriptions for transition conditions. These descriptions are informative only. The exact implementation of the state transitions shall follow the description in each section.



Note: Transition conditions are illustrative only. Not all of the transition conditions are listed.

U-047A
IEC

Figure 7-14 – State Diagram of the Link Training and Status State Machine

7.5.1 eSS.Disabled

eSS.Disabled is a state with a port's low-impedance receiver termination removed. It is a state where a port's Enhanced SuperSpeed connectivity is disabled. Refer to Section 10.18 for details regarding the behavior of a peripheral device. Refer to Sections 9.3 to 9.6 for behaviors regarding a hub's upstream port and downstream port.

eSS.Disabled does not contain any substates in the case of downstream ports and hub upstream ports. For a peripheral upstream port, it contains two substates, eSS.Disabled.Default and eSS.Disabled.Error.

eSS.Disabled is also a logical power-off state for a self-powered hub upstream port.

eSS.Disabled.Default is also a logical power-off state for a self-powered peripheral upstream port.

A downstream port shall transition to this state from any other state when directed.

A self-powered hub or peripheral upstream port shall transition to this state when VBUS is not valid.

7.5.1.1 eSS.Disabled for Downstream Ports and Hub Upstream Ports

eSS.Disabled for Downstream Ports and Hub Upstream Ports does not contain any substates.

7.5.1.1.1 eSS.Disabled Requirements

- VBUS may be present during eSS.Disabled.
- The port's receiver termination shall present high impedance to ground of $Z_{RX-HIGH-IMP-DC-POS}$ defined in Table 6-21.
- The port shall be disabled from transmitting and receiving LFPS and Enhanced SuperSpeed signals.

7.5.1.1.2 Exit from eSS.Disabled

- A downstream port shall transition to Rx.Detect when directed.
- An upstream port shall transition to Rx.Detect only when VBUS transitions to valid or a USB 2.0 bus reset is detected.

7.5.1.2 eSS.Disabled for Upstream Ports of Peripheral Devices

eSS.Disabled of a peripheral device operates similarly to hub upstream ports, except that it only attempts a limited number of Enhanced SuperSpeed attempts upon USB 2.0 bus reset.

7.5.1.2.1 eSS.Disabled Substate Machine

eSS.Disabled of a peripheral device has two substates shown in Figure 7-15.

- eSS.Disabled.Default
- eSS.Disabled.Error

eSS.Disabled.Default is a logical power-off state for a self-powered peripheral device.

7.5.1.2.2 eSS.Disabled Requirements

The requirements of a peripheral upstream port are the same as defined in Section 7.5.1.1.1. In addition, a peripheral upstream port shall implement a tDisabledCount counter. The operation of the tDisabledCount counter shall meet the following requirement.

- The tDisabledCount counter shall be reset to zero upon one of the following two conditions:
 - 1) Invalid VBUS
 - 2) Successful port configuration exchange
- The tDisabledCount counter shall be incremented upon entry to eSS.Disabled.Default.

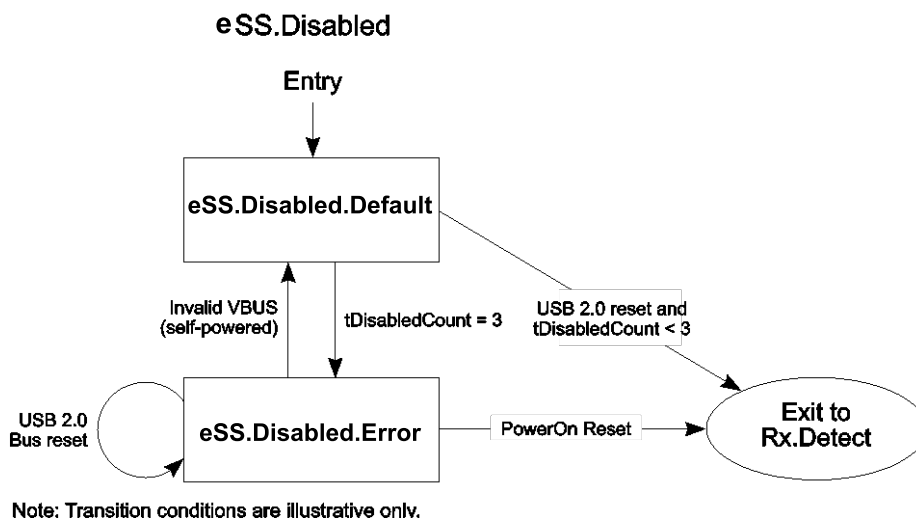
7.5.1.2.3 Exit from eSS.Disabled.Default

- A peripheral upstream port shall transition to Rx.Detect if one of the following conditions are met:
 - 1) When VBUS transitions to valid.
 - 2) When a USB 2.0 bus reset is detected and tDisabledCount is less than 3.
- A peripheral upstream port shall transition to eSS.Disabled.Error if tDisabledCount is 3.

7.5.1.2.4 Exit from eSS.Disabled.Error

- A peripheral upstream port shall transition to Rx.Detect upon PowerOn reset.
- A self-powered peripheral upstream port shall transition to eSS.Disabled.Default upon detection of invalid VBUS.

- A peripheral upstream port shall remain in eSS.Disabled.Error upon detection of USB 2.0 bus reset.



U-183
IEC

Figure 7-15 – eSS.Disabled Substate Machine

7.5.2 eSS.Inactive

eSS.Inactive is a state where a link has failed Enhanced SuperSpeed operation. A downstream port can only exit from this state when directed, or upon detection of an absence of a far-end receiver termination (R_{RX-DC}) specified in Table 6-21, or upon a Warm Reset. An upstream port can only exit to Rx.Detect upon a Warm Reset, or upon detecting an absence of a far-end receiver termination (R_{RX-DC}) specified in Table 6-21.

During eSS.Inactive, a port periodically performs a far-end receiver termination detection. If a disconnection is detected, a port will return to Rx.Detect. If a disconnect is not detected, the link will stay in eSS.Inactive until software intervention.

7.5.2.1 eSS.Inactive Substate Machines

eSS.Inactive contains the following substate machines shown in Figure 7-16:

- eSS.Inactive.Disconnect.Detect
- eSS.Inactive.Quiet

7.5.2.2 eSS.Inactive Requirements

- VBUS shall be present.
- The receiver termination shall meet the requirement (R_{RX-DC}) specified in Table 6-21.
- The transmitter common mode is not required to be within specification during this state.

7.5.2.3 eSS.Inactive.Quiet

eSS.Inactive.Quiet is a substate defined in which a port has disabled its far-end receiver termination detection so that extra power can be saved while waiting for software intervention.

7.5.2.3.1 eSS.Inactive.Quiet Requirements

- The function of the far-end receiver termination detection shall be disabled.
- A 12-ms timer (teSSInactiveQuietTimeout) shall be started upon entry to the substate.

7.5.2.3.2 Exit from eSS.Inactive.Quiet

- The port shall transition to eSS.Inactive.Disconnect.Detect upon the 12-ms timer timeout (teSSInactiveQuietTimeout).
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when Warm Reset is issued.
- An upstream port shall transition to Rx.Detect upon detection of Warm Reset.

7.5.2.4 eSS.Inactive.Disconnect.Detect

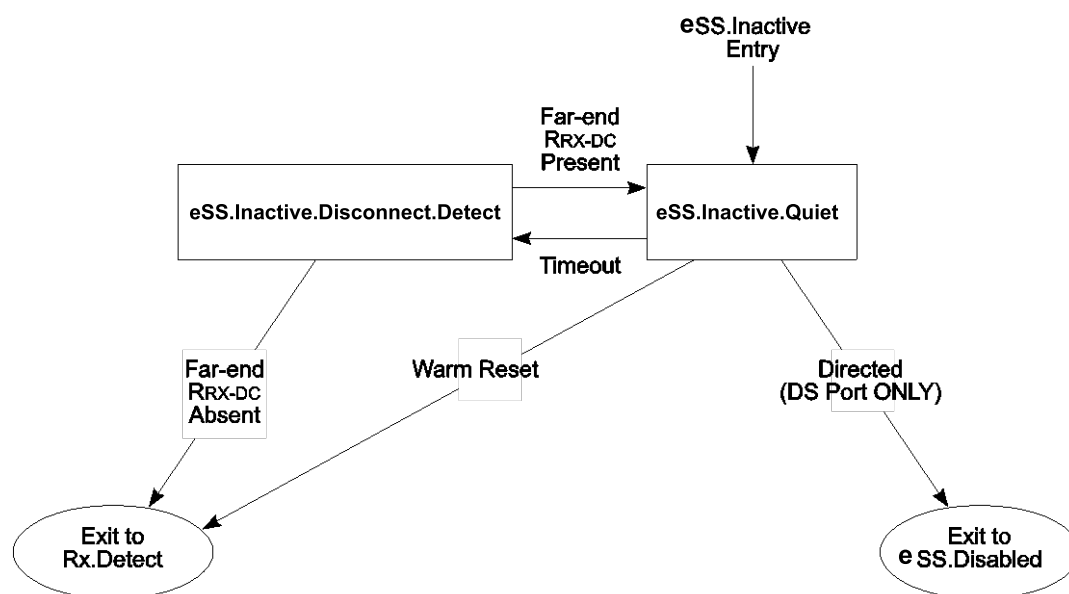
eSS.Inactive.Disconnect.Detect is a substate in which a port will perform the far-end receiver termination detection in order to determine if its link partner is disconnected during eSS.Inactive, or if the transition to eSS.Inactive is due to a disconnect from its link partner.

7.5.2.4.1 eSS.Inactive.Disconnect.Detect Requirements

The transmitter shall perform the far-end receiver termination detection described in Section 6.11.

7.5.2.4.2 Exit from eSS.Inactive.Disconnect.Detect

- The port shall transition to Rx.Detect when a far-end low-impedance receiver termination (R_{RX-DC}) meeting specification defined in Table 6-21 is not detected.
- The port shall transition to eSS.Inactive.Quiet when a far-end low-impedance receiver termination (R_{RX-DC}) meeting specification defined in Table 6-21 is detected.



Note: Transition conditions are illustrative only. Not all of the transition conditions are listed.

U-048
IEC

Figure 7-16 – eSS.Inactive Substate Machine

7.5.3 Rx.Detect

Rx.Detect is the power on state of the LTSSM for both a downstream port and an upstream port. It is also the state for a downstream port upon issuing a Warm Reset, and the state for an upstream port upon detecting a Warm Reset from any other link state except eSS.Disabled. The purpose of Rx.Detect is to detect the impedance of far-end receiver termination to ground. Rx.Detect.Reset is a default reset state used by the two ports to synchronize the operation after a Warm Reset; this substate exits immediately if Warm Reset is not present. Rx.Detect.Active is a substate for far-end receiver termination detection. Rx.Detect.Quiet is a power saving substate in which the function of a far-end receiver termination detection is

disabled. A port will perform the far-end receiver termination detection periodically during Rx.Detect.

7.5.3.1 Rx.Detect Substate Machines

Rx.Detect contains a substate machine shown in Figure 7-17 with the following substates:

- Rx.Detect.Reset
- Rx.Detect.Active
- Rx.Detect.Quiet

7.5.3.2 Rx.Detect Requirements

- The transmitter common mode is not required to be within specification during this state.
- The low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21 shall be maintained.

7.5.3.3 Rx.Detect.Reset

Rx.Detect.Reset is a substate designed for the two ports to synchronize their operations on Warm Reset. In this substate, a downstream port shall generate Warm Reset when directed. If an upstream port enters Rx.Detect upon detection of Warm Reset, it shall remain in this substate until the completion of Warm Reset.

For a port entering Rx.Detect not due to a Warm Reset, it shall exit immediately.

7.5.3.3.1 Rx.Detect.Reset Requirements

If a port enters Rx.Detect upon a Warm Reset, the following requirements shall be applied. Refer to Section 6.9.3 for details.

- A downstream port shall transmit Warm Reset for the duration of t_{Reset} as defined in Table 6-29.

NOTE This includes the case when Hot Reset attempt fails in Recovery. Refer to Section 7.4.2 for details.

- An upstream port shall remain in this state until it detects the completion of Warm Reset.

7.5.3.3.2 Exit from Rx.Detect.Reset

- The port shall transition directly to Rx.Detect.Active if the entry to Rx.Detect is not due to a Warm Reset.

NOTE Warm Reset is not present during power-on.

- A downstream port shall transition to Rx.Detect.Active after it transmits Warm Reset for the duration of t_{Reset} as defined in Table 6-29.
- A downstream port shall transition to eSS.Disabled when directed.
- An upstream port shall transition to Rx.Detect.Active when it receives no more LFPS Warm Reset signaling from the downstream port as defined in Section 6.9.3.

7.5.3.4 Rx.Detect.Active

Rx.Detect.Active is a substate to detect the presence of an Enhanced SuperSpeed link partner. A port will perform a far-end receiver termination detection as defined in Section 6.11.

7.5.3.5 Rx.Detect.Active Requirements

- The transmitter shall initiate a far-end receiver termination detection described in Section 6.11.

- The number of far-end receiver termination detection events shall be counted by an upstream port. The detection of far-end receiver termination is defined in Section 6.11.

NOTE This count value is used by a peripheral device to determine when it needs to transition to eSS.Disabled. It is also used by a hub to control its downstream port state machine. Refer to Section 10.3.1.1 for details.

7.5.3.6 Exit from Rx.Detect.Active

- The port shall transition to Polling upon detection of a far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- A downstream port shall transition to Rx.Detect.Quiet when a far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21 is not detected.
- A downstream port shall transition to eSS.Disabled when directed.
- An upstream port of a hub shall transition to Rx.Detect.Quiet when a far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21 is not detected.
- An upstream port of a peripheral device shall transition to Rx.Detect.Quiet when the following two conditions are met:
 - 1) A far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21 is not detected.
 - 2) The number of far-end receiver termination detection events is less than eight.
- An upstream port of a peripheral device shall transition to eSS.Disabled when the following two conditions are met:
 - 1) A far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21 is not detected.
 - 2) The number of far-end receiver termination detection events has reached eight.

NOTE This limit on the number of the far-end receiver termination detections is to allow an Enhanced SuperSpeed peripheral device on a legacy platform to transition to USB 2.0 after 80 ms.

7.5.3.7 Rx.Detect.Quiet

Rx.Detect.Quiet is a substate where a port has disabled its far-end receiver termination detection.

7.5.3.7.1 Rx.Detect.Quiet Requirements

- The far-end receiver termination detection shall be disabled.
- A downstream port shall start a timer ($t_{RxDetectQuietTimeoutDFP}$) with a timeout between 12 ms and 120 ms upon entry to the substate.
- An upstream port shall start a timer ($t_{RxDetectQuietTimeoutUFP}$) with a timeout of 12 ms upon entry to the substate.

7.5.3.7.2 Exit from Rx.Detect.Quiet

- A downstream port shall transition to Rx.Detect.Active upon the timeout of the $t_{RxDetectQuietTimeoutDFP}$ timer (12 ms to 120 ms).
- An upstream port shall transition to Rx.Detect.Active upon the timeout of the $t_{RxDetectQuietTimeoutUFP}$ timer (12 ms).
- A downstream port shall transition to eSS.Disabled when directed.

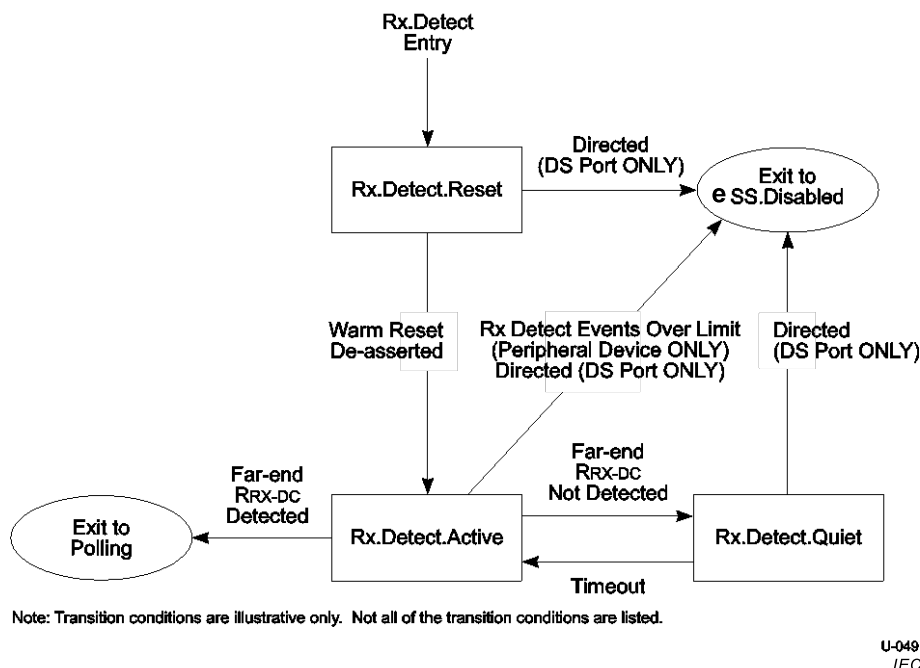


Figure 7-17 – Rx.Detect Substate Machine

7.5.4 Polling

Polling is a state for port capability negotiation and link training. During Polling, a Polling.LFPS handshake shall take place between the two ports in SuperSpeed operation before the link training is started. Similarly for SuperSpeedPlus operation, Polling.LFPS based SCD1/SCD2 handshakes, LBPM based port capability negotiation and match, and subsequent port configuration shall take place before SuperSpeedPlus link training is started. Bit lock, block alignment for SuperSpeedPlus operation, symbol lock, lane polarity inversion, and Rx equalization trainings are achieved using TSEQ, SYNC, TS1, and TS2 training ordered sets.

7.5.4.1 Polling Substate Machines

Polling contains a substate machine shown in Figure 7-18 with the following substates:

- Polling.LFPS
- Polling.LFPSPlus
- Polling.PortMatch
- Polling.PortConfig
- Polling.RxEQ
- Polling.Active
- Polling.Configuration
- Polling.Idle

7.5.4.2 Polling Requirements

- The port shall maintain its low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- A downstream port shall implement a counter ($cPollingTimeout$) to count the number of consecutive transition events from any Polling substate to Rx.Detect due to timeout. The operation of $cPollingTimeout$ shall adhere to the following rules.
 - 1) It shall be reset to zero upon any of the following conditions.
 - i) PowerOn Reset.

- ii) Warm Reset on the port.
 - iii) Upon exit to eSS.Disabled, or eSS.Inactive, or U0.
 - iv) Upon detection of the removal of far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- 2) It shall be incremented by one or saturated at two if a transition to Rx.Detect is due to timeout in any of the Polling substates.

7.5.4.3 Polling.LFPS

Polling.LFPS is a substate designed to establish the PHY's DC operating point, and to synchronize the operations between the two link partners after exiting from Rx.Detect. This is also a substate for a port to identify itself based on various Polling.LFPS signatures.

7.5.4.3.1 Polling.LFPS Requirements

- Upon entry, a LFPS receiver shall be enabled to receive the Polling.LFPS signals defined in Section 6.9.1.
- Upon entry, a port shall establish its LFPS operating condition within 80 μ s.
- A downstream port shall disable its transition path to Compliance Mode upon PowerOn Reset or Warm Reset.
- A downstream port shall enable its transition path to Compliance Mode, if directed.
- An upstream port always has its transition path to Compliance Mode enabled upon PowerOn Reset.
- A port in SuperSpeed operation shall transmit Polling.LFPS.
- An upstream port in SuperSpeedPlus operation shall transmit SCD1 defined in Table 6-32. If no signature of SCD1 is found in sixteen consecutive Polling.LFPS received, the port shall switch to SuperSpeed operation and transmit Polling.LFPS instead of SCD1.
- A downstream port in SuperSpeedPlus operation shall transmit SCD1 defined in Table 6-32 if Compliance Mode is disabled. If no signature of SCD1 is found in sixteen consecutive Polling.LFPS received, the port shall switch to SuperSpeed operation and transmit Polling.LFPS instead of SCD1.
- A downstream port in SuperSpeedPlus operation shall transmit SCD2 if Compliance Mode is enabled.

NOTE In case a retimer is connected to the downstream port, SCD2 is used for the retimer to enable Compliance Mode. Refer to Appendix E for details.

- A port in SuperSpeedPlus operation shall implement a 60- μ s timer ($t_{PollingSCDLFPSTimeout}$) to monitor the absence of LFPS signal after the completion of SuperSpeed Polling.LFPS handshake.
- A port in SuperSpeedPlus operation shall be ready for SuperSpeed operation if it has detected that its link partner operates at SuperSpeed.

NOTE There is no time allocated for a SuperSpeedPlus port to re-configure itself for SuperSpeed operation upon detection of its link partner operating at SuperSpeed. A SuperSpeedPlus port shall switch to SuperSpeed operation as quickly as possible for its receiver equalization training. Any time for configuration will result in receiving fewer TSEQ ordered sets from its link partner.

- A port shall disable its transition path to Compliance Mode when it has successfully completed Polling.LFPS handshake or has entered Compliance Mode.
- A 360-ms timer ($t_{PollingLFPSTimeout}$) shall be started upon entry to the substate.
- The operating condition of an eSS PHY shall be established when a port is ready to exit to Polling.RxEQ.
- An eSS receiver in SuperSpeed operation may optionally be enabled to receive TSEQ ordered sets for receiver equalizer training.

NOTE The port first entering Polling.RxEQ will start transmitting TSEQ ordered sets while the other port is still in Polling.LFPS. Enabling a SuperSpeed receiver in Polling.LFPS will allow a port to start the receiver equalizer training while completing the requirement for Polling.LFPS exit handshake.

7.5.4.3.2 Exit from Polling.LFPS

- The port in SuperSpeed operation shall transition to Polling.RxEQ when the following three conditions are met:
 - 1) At least 16 consecutive Polling.LFPS bursts meeting the Polling.LFPS specification defined in Section 6.9 are sent.
 - 2) Two consecutive Polling.LFPS bursts are received.
 - 3) Four consecutive Polling.LFPS bursts are sent after receiving one Polling.LFPS burst.
- The port in SuperSpeedPlus operation shall transition to Polling.LFPSPlus if two SCD1 are transmitted after one SCD1 or SCD2 as defined in Section 6.9.4.2 is received.
- The port in SuperSpeedPlus operation shall transition to Polling.RxEQ and switch to SuperSpeed operation if the following conditions are met:
 - 1) At least two consecutive Polling.LFPS bursts are received.
 - 2) Twenty Polling.LFPS bursts are transmitted, and no SCD1 is detected.

NOTE This condition guarantees that, in the case of a port in SuperSpeedPlus operation connecting to a port in SuperSpeed operation, a port in SuperSpeed operation will receive twenty consecutive Polling.LFPS to exit from this substate if it is unable to recognize Polling.LFPS with varying tRepeat in SCD1.

- 3) No LFPS signal for more than tPollingSCDLFPSTimeout is observed.

NOTE This condition implies the SuperSpeed link partner has entered Polling.RxEQ transmitting TSEQ ordered sets.

- An upstream port shall transition to Compliance Mode upon the 360-ms timer timeout (tPollingLFPSTimeout) and the following two conditions are met:
 - 1) The port has never successfully completed Polling.LFPS after PowerOn Reset.
 - 2) The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.

NOTE If the very first attempt in Polling.LFPS handshake fails after PowerOn Reset, it implies that a passive test load may be present and compliance test should be initiated. If the very first attempt in Polling.LFPS handshake succeeds after PowerOn Reset, it implies the presence of the Enhanced SuperSpeed ports on each side of the link and no compliance test is intended. Therefore, any subsequent handshake timeout in Polling.LFPS when the link is retrained is only an indication of link training failure, not a signal to enter Compliance Mode.

- A downstream port shall transition to Compliance Mode upon the 360-ms timer timeout (tPollingLFPSTimeout) if the following three conditions are met:
 - 1) The Compliance Mode is enabled.
 - 2) The port has never successfully completed Polling.LFPS handshake after Compliance Mode is enabled.
 - 3) The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.

NOTE In case Compliance mode is disabled, a downstream port may enter Rx.Detect attempting Polling.LFPS handshake again, or enter eSS.Inactive for SW intervention based on the count value of cPollingTimeout.

- A downstream port shall transition to Rx.Detect upon the 360-ms timer timeout (tPollingLFPSTimeout) if cPollingTimeout is less than two and Compliance Mode is disabled.
- A downstream port shall transition to eSS.Inactive upon the 360-ms timer timeout (tPollingLFPSTimeout) and cPollingTimeout is two.
- An upstream port of a hub shall transition to Rx.Detect upon the 360-ms timer timeout (tPollingLFPSTimeout) after having trained once since PowerOn Reset and the conditions to transition to Polling.RxEQ are not met.

- A peripheral device shall transition to eSS.Disabled upon the 360-ms timeout (tPollingLFPSTimeout) after having trained once since PowerOn Reset and the conditions to transition to Polling.RxEQ are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.4 Polling.LFPSPlus

Polling.LFPSPlus is a substate where the port SuperSpeedPlus operation performs SCD2 handshake, for additional confirmation of the SuperSpeedPlus capability of its link partner.

7.5.4.4.1 Polling.LFPSPlus Requirements

- The port in SuperSpeedPlus operation shall transmit SCD2 defined in Table 6-32. If SCD2 cannot be found in sixteen consecutive Polling.LFPS received, it shall transmit Polling.LFPS instead of SCD2.

NOTE This is an extreme case where a port in SuperSpeed operation transmits Polling.LFPS coinciding with SCD1 and remains in Polling.LFPS.

- The operation of the 360-ms timer (tPollingLFPSTimeout) shall continue without reset upon entry to this substate from Polling.LFPS.
- A port in SuperSpeedPlus operation shall be ready for SuperSpeed operation if it has detected that its link partner operates at SuperSpeed.
- A port in SuperSpeedPlus operation shall implement a 60-us timer (tPollingSCDLFPSTimeout) to monitor the absence of LFPS signal after the completion of SuperSpeed Polling.LFPS handshake.

7.5.4.4.2 Exit from Polling.LFPSPlus

- The port in SuperSpeedPlus operation shall transition to Polling.PortMatch if two SCD2 are transmitted after one SCD2 as defined in Section 6.9.4.2 is received.
- A port in SuperSpeedPlus operation shall transition to Polling.RxEQ and switch to SuperSpeed operation if the following two conditions are met:

- 1) No LFPS signal for more than tPollingSCDLFPSTimeout is observed.

NOTE This condition implies the SuperSpeed link partner has entered Polling.RxEQ transmitting TSEQ ordered sets.

- 2) Twenty Polling.LFPS bursts are transmitted, after finding no SCD2 is detected.

NOTE This condition guarantees that, in the case of a port in SuperSpeedPlus operation connecting to a port in SuperSpeed operation, a port in SuperSpeed operation will receive twenty consecutive Polling.LFPS to exit from this substate if it is unable to recognize Polling.LFPS with varying tRepeat in SCD1 and SCD2, and it happens to transmit Polling.LFPS matching SCD1.

- A downstream port shall transition to Rx.Detect upon the 360-ms timer timeout (tPollingLFPSTimeout) and cPollingTimeout is less than two.
- A downstream port shall transition to eSS.Inactive upon the 360-ms timer timeout (tPollingLFPSTimeout) and cPollingTimeout is two.
- An upstream port of a hub shall transition to Rx.Detect upon the 360-ms timeout (tPollingLFPSTimeout).
- A peripheral device shall transition to eSS.Disabled upon the 360-ms timeout (tPollingLFPSTimeout).
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.5 Polling.PortMatch

Polling.PortMatch is a substate where the two ports in SuperSpeedPlus operation perform the LBPM handshake, for announcing, matching, and deciding the operation on the highest common capability between the two link partners. The LBPM handshake includes two stages of operation. The first stage is to announce LBPM PHY Capability as defined in Table 7-13 below. The second stage is for each port to decode LBPM PHY Capability and adjust to the highest common PHY Capability by transmitting PHY Capability Match.

7.5.4.5.1 PHY Capability LBPM Definition

SuperSpeedPlus PHY Capability is defined based on the following LBPM. Refer to Section 6.9.5 for LBPM details.

Table 7-13 – PHY Capability LBPM

b1~0: LBPM Type	b3~2: LBPM Subtype	b7~b4: Reserved (0000)
00: PHY Capability	PHY Rate	
	00: 5 Gbps	
	01: 10 Gbps	
01: PHY Ready	Reserved (00)	

Two types of LBPM are defined. PHY Capability LBPM is used to announce a port's PHY capabilities. The initial value shall describe a port's highest PHY capabilities. The announced capabilities may later be adjusted in order to match the link partner's capabilities or if the link fails to reach U0 during training and the two ports need to fall back to a lower rate.

PHY Ready LBPM is defined for a port to send a notification that it has completed its PHY re-configuration and ready for link training at the matched port capability defined by PHY Capability LBPM. Refer to Section 7.5.4.6 for use of PHY Ready LBPM.

7.5.4.5.2 Polling.PortMatch Requirements

- A 12-ms timer (tPollingLBPM LFPSTimeout) shall be started upon entry to the substate.
- Upon entry to this substate from Polling.LFPSPlus, the port shall transmit continuous PHY Capability LBPMs defined in Table 7-13 to announce its highest PHY Capability.
- Upon entry to this substate from Polling.Active, or Polling.Configuration, or Polling.Idle, the port shall transmit continuous PHY Capability LBPMs defined in Table 7-13 to announce its next highest PHY Capability from its previous PHY Capability.
- The port shall decode received PHY Capability LBPM or PHY Ready LBPM and compare to its own PHY Capability.
- The port with higher PHY capability shall adjust its PHY capability by transmitting PHY Capability LBPM that matches its link partner's.
- The port with lower PHY capability shall continue transmitting its own PHY Capability LBPMs and monitoring the PHY Capability LBPMs from its link partner.
- The two ports shall continue the interactive process of PHY Capability LBPM exchange as described above until they match the PHY Capability.

7.5.4.5.3 Exit from Polling.PortMatch

- The port shall transition to Polling.PortConfig when four consecutive and matched PHY Capability LBPMs are sent after two consecutive and matched PHY Capability LBPMs or PHY Ready LBPMs are received.

NOTE A port exiting from Polling.PortMatch and ready for PHY operation without re-configuration may immediately transmit PHY Ready LBPMs.

- A downstream port shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and cPollingTimeout is less than two.
- A downstream port shall transition to eSS.Inactive upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and cPollingTimeout is two.
- An upstream port of a hub shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and the conditions to transition to Polling.PortConfig are not met.
- A peripheral device shall transition to eSS.Disabled upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and the conditions to transition to Polling.PortConfig are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.6 Polling.PortConfig

Polling.PortConfig is a substate where a port configures its PHY according to PHY Capability LBPM matched in Polling.PortMatch, and synchronizes with its link partner in exiting from this substate to Polling.RxEQ. Depending on matched PHY Capability LBPM, a port may configure its PHY for SuperSpeed operation or SuperSpeedPlus operation.

7.5.4.6.1 Polling.PortConfig Requirements

- The operation of the 12-ms timer (tPollingLBPM LFPSTimeout) shall continue without reset upon entry to the substate from Polling.PortMatch.
- Upon entry to this state, the port shall place its transmitter in electrical idle if it is preparing its PHY re-configuration according to PHY Capability LBPM negotiated in Polling.PortMatch. The port shall perform the following PHY re-configuration.
 - 1) The transmitter DC common mode voltage shall be within specification (VTX-CM-DCACTIVE-IDLE-DELTA) defined in Table 6-18.
 - 2) The port shall maintain its low-impedance receiver termination (RRX-DC) defined in Table 6-21
 - 3) The port shall be ready to transmit TSEQ OS.
 - 4) The port shall be ready to receive TSEQ OS for receiver equalization training.
- Upon completion of PHY re-configuration, the port shall transmit consecutive PHY Ready LBPMs to notify its link partner. Refer to Table 7-13 for PHY Ready LBPM definition.

7.5.4.6.2 Exit from Polling.PortConfig

- The port shall transition to Polling.RxEQ when four consecutive and matched PHY Ready LBPMs are sent after two consecutive and matched PHY Ready LBPMs are received.
- A downstream port shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and cPollingTimeout is less than two.
- A downstream port shall transition to eSS.Inactive upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and cPollingTimeout is two.
- An upstream port of a hub shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and the conditions to transition to Polling.RxEQ are not met.
- A peripheral device shall transition to eSS.Disabled upon the 12-ms timer timeout (tPollingLBPM LFPSTimeout) and the conditions to transition to Polling.RxEQ are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.7 Polling.RxEQ

Polling.RxEQ is a substate for receiver equalization training. A port is required to complete its receiver equalization training.

7.5.4.7.1 Polling.RxEQ Requirements

- The detection and correction of the lane polarity inversion in SuperSpeed operation shall be enabled, as is described in Section 6.4.2.
- The port shall transmit the corresponding TSEQ ordered sets defined in Table 6-2.
- The port shall complete receiver equalizer training upon exit from this substate.

NOTE A situation may exist where the port entering Polling.RxEQ earlier is transmitting TSEQ ordered sets while its link partner is still sending Polling.LFPS to satisfy the exit conditions from Polling.LFPS or Polling.LFPSPlus to Polling.RxEQ. In this situation, if its link partner is in electrical idle, near-end cross talk may cause the port to train its Rx equalizer using its own TSEQ ordered sets. To avoid a receiver from training itself, a port may either ignore the beginning part (about 30 μ s) of the TSEQ ordered sets, or continue the equalizer training until it completes the transmission of TSEQ ordered sets.

7.5.4.7.2 Exit from Polling.RxEQ

- The port in SuperSpeed operation shall transition to Polling.Active after 65 536 consecutive TSEQ ordered sets defined in Table 6-2 are transmitted.
- The port in SuperSpeedPlus operation shall transition to Polling.Active after 262 143 TSEQ ordered sets defined in Table 6-18 are transmitted. Refer to Section 6.4.1.2.1 for SYNC ordered set insertion while transmitting TSEQ ordered sets.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.8 Polling.Active

Polling.Active is a substate that continues the link's Enhanced SuperSpeed training.

7.5.4.8.1 Polling.Active Requirements

- A 12-ms timer (tPollingActiveTimeout) shall be started upon entry to this substate.
- The port shall transmit TS1 ordered sets.
- The port in SuperSpeedPlus operation shall insert a SYNC ordered set every 32 TS1 ordered sets.
- The port in SuperSpeedPlus operation shall perform block alignment and scrambler synchronization as defined in Sections 6.3.2.3 and 6.4.1.2.4.
- Lane polarity detection and correction shall be completed.
- The port that fails to achieve a successful training with its link partner shall reconfigure itself for the next capability it supports.

NOTE An example of this is, when a SuperSpeedPlus port fails to reach successful handshake with its link partner, it shall re-configure itself for SuperSpeed operation.

- The receiver is in training using TS1 or TS2 ordered sets.

NOTE Depending on the link condition and different receiver implementations, one port's receiver may train faster than the other. When this occurs, the port whose receiver trains first will enter Polling.Configuration and start transmitting TS2 ordered sets while the port whose receiver is not yet trained is still in Polling.Active using TS2 ordered sets to train its receiver.

7.5.4.8.2 Exit from Polling.Active

- The port in SuperSpeed operation shall transition to Polling.Configuration upon receiving eight consecutive and identical TS1 or TS2 ordered sets.

- The port in SuperSpeedPlus operation shall transition to Polling.Configuration upon receiving eight consecutive and identical TS1 or TS2 ordered sets, excluding symbols 14 and 15 of TS1 or TS2 ordered sets.

NOTE SYNC OS and SKP OS in between TS1 OS and/or TS2 OS do not disqualify the consecutive detection of TS1 OS and TS2 OS. Symbols 14 and 15 are used for TS1 or TS2 ordered set identifier or DC balance adjustment.

- A downstream port in SuperSpeed operation shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingActiveTimeout) and the following two conditions are met.
 - 1) The conditions to transition to Polling.Configuration are not met.
 - 2) cPollingTimeout is less than two.
- A downstream port in SuperSpeed operation shall transition to eSS.Inactive upon the 12-ms timer timeout (tPollingActiveTimeout) and the following two conditions are met.
 - 1) The conditions to transition to Polling.Configuration are not met.
 - 2) cPollingTimeout is two.
- An upstream port of a hub in SuperSpeed operation shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingActiveTimeout) and the conditions to transition to Polling.Configuration are not met.
- An upstream port of a peripheral device in SuperSpeed operation shall transition to eSS.Disabled upon the 12-ms timer timeout (tPollingActiveTimeout) and the conditions to transition to Polling.Configuration are not met.
- A downstream port in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 12-ms timer timeout (tPollingActiveTimeout) and the conditions to transition to Polling.Configuration are not met.
- An upstream port of a hub in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 12-ms timer timeout (tPollingActiveTimeout) and the conditions to transition to Polling.Configuration are not met.
- An upstream port of a peripheral device in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 12-ms timer timeout (tPollingActiveTimeout) and the conditions to transition to Polling.Configuration are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.9 Polling.Configuration

Polling.Configuration is a substate where the two link partners complete the Enhanced SuperSpeed training.

7.5.4.9.1 Polling.Configuration Requirements

- The port shall transmit identical TS2 ordered sets upon entry to this substate and set the link configuration field in the TS2 ordered set based on the following.
 - 1) When directed, a downstream port shall set Reset bit in the TS2 ordered set.

NOTE An upstream port can only set the Reset bit in the TS2 ordered set when in Hot Reset. Active. Refer to Section 7.5.12.3 for details.

- 2) When directed, the port shall set Loopback bit in the TS2 ordered set.
 - 3) When directed, the port shall set the Disabling Scrambling bit in the TS2 ordered set.
- The port in SuperSpeedPlus operation shall insert a SYNC ordered set every 32 TS2 ordered sets.

- The port in SuperSpeedPlus operation shall perform block alignment and scrambler synchronization as defined in Sections 6.3.2.3 and 6.4.1.2.4.
- The port that fails to achieve a successful training with its link partner shall reconfigure itself for the next capability it supports.

NOTE An example of this is, when a SuperSpeedPlus port fails to reach successful handshake with its link partner, it shall re-configure itself for SuperSpeed operation.

- A 12-ms timer (tPollingConfigurationTimeout) shall be started upon entry to this substate.

7.5.4.9.2 Exit from Polling.Configuration

- The port in SuperSpeed operation shall transition to Polling.Idle when the following two conditions are met:
 - 1) Eight consecutive and identical TS2 ordered sets are received.
 - 2) Sixteen TS2 ordered sets are sent after receiving the first of the eight consecutive and identical TS2 ordered sets.
- The port in SuperSpeedPlus operation shall transition to Polling.Idle when the following two conditions are met:
 - 1) Eight consecutive and identical TS2 ordered sets, excluding symbols 14 and 15, are received.
 - 2) Sixteen TS2 ordered sets are sent after receiving the first of the eight consecutive and identical TS2 ordered sets, excluding symbols 14 and 15.

NOTE SYNC OS and SKP OS in between TS2 OS do not disqualify the consecutive detection of TS2 OS.

- A downstream port in SuperSpeed operation shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the following two conditions are met.
 - 1) The conditions to transition to Polling.Idle are not met.
 - 2) cPollingTimeout is less than two.
- A downstream port in SuperSpeed operation shall transition to eSS.Inactive upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the following two conditions are met.
 - 1) The conditions to transition to Polling.Idle are not met.
 - 2) cPollingTimeout is two.
- An upstream port of a hub in SuperSpeed operation shall transition to Rx.Detect upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the conditions to transition to Polling.Idle are not met.
- An upstream port of a peripheral device in SuperSpeed operation shall transition to eSS.Disabled upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the conditions to transition to Polling.Idle are not met.
- A downstream port in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the conditions to transition to Polling.Idle are not met.
- An upstream port of a hub in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the conditions to transition to Polling.Idle are not met.
- An upstream port of a peripheral device in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 12-ms timer timeout (tPollingConfigurationTimeout) and the conditions to transition to Polling.Idle are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.4.10 Polling.Idle

Polling.Idle is a substate where the port decodes the TS2 ordered set received in Polling.Configuration and determines the next state.

7.5.4.10.1 Polling.Idle Requirements

- The port shall decode the TS2 ordered set received during Polling.Configuration and proceeds to the next state.
- A downstream port shall reset its Link Error Count.
- An upstream port shall reset its port configuration information to default values. Refer to Sections 8.4.5 and 8.4.6 for details.
- The port in SuperSpeed operation shall enable the scrambling by default if the Disabling Scrambling bit is not asserted in the TS2 ordered set received in Polling.Configuration.
- The port in SuperSpeed operation shall disable the scrambling if directed, or if the Disabling Scrambling bit is asserted in the TS2 ordered set received in Polling.Configuration.
- The port in SuperSpeed operation shall transmit Idle Symbols if the next state is U0. The port may transmit Idle Symbols if the next state is Loopback or HotReset.
- The port in SuperSpeedPlus operation shall transmit a single SDS ordered set before the start of the data blocks with Idle Symbols if the next state is U0. The port may transmit SDS ordered set if the next state is Loopback or Hot Reset.

NOTE Under situation where a SKP ordered set is also scheduled at the same time with SDS ordered set, SKP ordered set shall be transmitted first.

- The port in SuperSpeedPlus operation may ignore SDS ordered set if corrupted and continue to process the following data block. The port may optionally choose to recover SDS ordered set if error is detected.
- The port in SuperSpeedPlus operation shall disable the scrambling upon completion of SDS ordered set transmission if directed, or if the Disabling Scrambling bit is asserted in the TS2 ordered set received in Polling.Configuration.
- The port that fails to achieve a successful training with its link partner shall reconfigure itself for the next capability it supports.

NOTE An example of this is, when a SuperSpeedPlus port fails to reach successful handshake with its link partner, it shall re-configure itself for SuperSpeed operation.

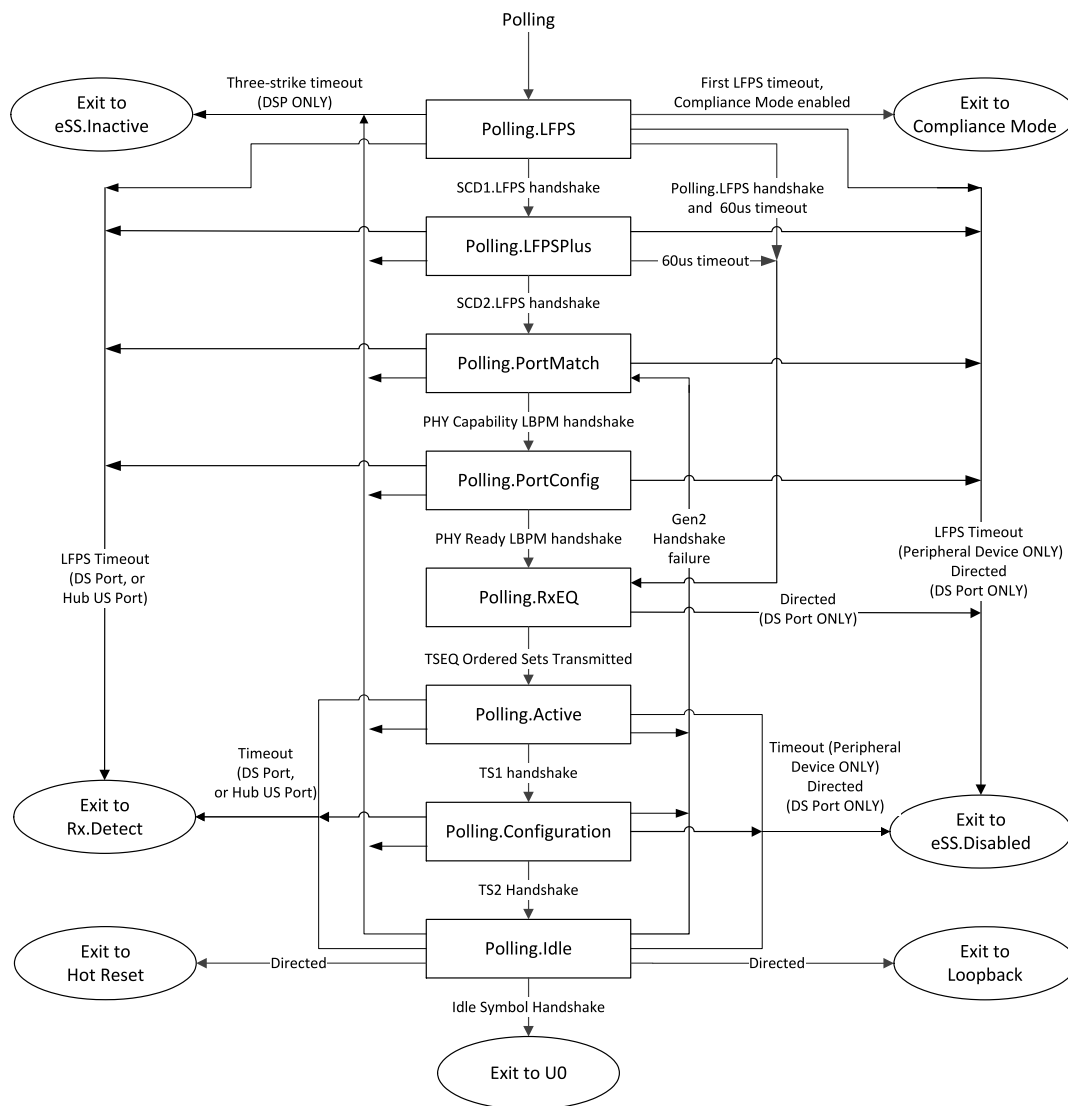
- A 2-ms timer (tPollingIdleTimeout) shall be started upon entry to this state.
- The port shall be able to receive the Header Sequence Number Advertisement from its link partner.

NOTE The exit time difference between the two ports will result in one port entering U0 first and starting the Header Sequence Number Advertisement while the other port is still in Polling.Idle.

7.5.4.10.2 Exit from Polling.Idle

- The port shall transition to Loopback when directed as a loopback master and the port is capable of being a loopback master. Refer to Section 7.5.11 for details.
- The port shall transition to Loopback as a loopback slave if the Loopback bit is asserted in the TS2 ordered set received in Polling.Configuration. Refer to Section 7.5.4.9 for details.
- A downstream port shall transition to Hot Reset when directed.
- An upstream port shall transition to Hot Reset when the Reset bit is asserted in the TS2 ordered set received in Polling.Configuration.
- The port shall transition to U0 when the following two conditions are met:
 - 1) Eight consecutive Idle Symbols are received.
 - 2) Sixteen Idle Symbols are sent after receiving one Idle Symbol.

- A downstream port in SuperSpeed operation shall transition to eSS.Disabled when directed.
- A downstream port in SuperSpeed operation shall transition to Rx.Detect upon the 2-ms timer timeout (tPollingIdleTimeout) and the following two conditions are met.
 - 1) The conditions to transition to U0 are not met.
 - 2) cPollingTimeout is less than two.
- A downstream port in SuperSpeed operation shall transition to eSS.Inactive upon the 2-ms timer timeout (tPollingIdleTimeout) and the following two conditions are met.
 - 1) The conditions to transition to U0 are not met.
 - 2) cPollingTimeout is two.
- An upstream port of a hub in SuperSpeed operation shall transition to Rx.Detect upon the 2-ms timer timeout (tPollingIdleTimeout) and the conditions to transition to U0 are not met.
- An upstream port of a peripheral device in SuperSpeed operation shall transition to eSS.Disabled upon the 2-ms timer timeout (tPollingIdleTimeout) and the conditions to transition to U0 are not met.
- A downstream port in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 2-ms timer timeout (tPollingIdleTimeout) and the conditions to transition U0 are not met.
- An upstream port of a hub in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 2-ms timer timeout (tPollingIdleTimeout) and the conditions to transition to U0 are not met.
- An upstream port of a peripheral device in SuperSpeedPlus operation shall transition to Polling.PortMatch to negotiate for alternative operation upon the 2-ms timer timeout (tPollingIdleTimeout) and the conditions to transition to U0 are not met.
- A downstream port shall transition to Rx.Detect when Warm Reset is directed.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.



Note: Transition conditions are illustrative only. Not all of the transition conditions are listed.

IEC

Figure 7-18 – Polling Substate Machine

7.5.5 Compliance Mode

Compliance Mode is used to test the transmitter for compliance to voltage and timing specifications. Several different test patterns are transmitted as defined in Table 6-13. Compliance Mode does not contain any substate machines.

Note that for a downstream port, the default setting for entry to Compliance Mode is disabled. It may optionally be enabled when directed. This is to prevent the automatic entry to Compliance Mode due to connection of a bad device that fails to exit from Polling.LFPS upon power-on, or a downstream port fails to respond in time.

7.5.5.1 Compliance Mode Requirements

- The port shall maintain its low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- The LFPS receiver is used to control the test pattern sequencing.
- Upon entry to Compliance Mode, the port shall wait until its eSS Tx DC common mode voltage meets the $V_{TX-DC-CM}$ specification defined in Table 6-18 before it starts to send the first compliance test pattern defined in Table 6-13.

- The port shall transmit the next compliance test pattern continuously upon detection of a Ping.LFPS as defined in Section 6.9.1.
- The port shall transmit the first compliance test pattern continuously upon detection of a Ping.LFPS and the test pattern has reached the final test pattern.

7.5.5.2 Exit from Compliance Mode

- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect upon detection of Warm Reset.
- A downstream port shall transition to eSS.Disabled when directed.

7.5.6 U0

U0 is the normal operational state where packets can be transmitted and received. U0 does not contain any substate machines.

7.5.6.1 U0 Requirements

- The port shall meet the transmitter specifications defined in Table 6-17.
- The port shall maintain the low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- The LFPS receiver shall be enabled.
- The port shall enable a 1-ms timer ($t_{U0RecoveryTimeout}$) to measure the time interval between two consecutive link commands. This timer will be reset and restarted every time a link command is received.
- The port shall enable a 10- μ s timer ($t_{U0LTimeout}$). This timer shall be reset when the first symbol of any link command or packet is sent and restarted after the last symbol of any link command or packet is sent. This timer shall be active when the link is in logical idle.
- A downstream port shall transmit a single LDN when the 10- μ s timer ($t_{U0LTimeout}$) expires.
- An upstream port shall transmit a single LUP when the 10- μ s timer ($t_{U0LTimeout}$) expires.

7.5.6.2 Exit from U0

- The port shall transition to U1 upon successful completion of LGO_U1 entry sequence. Refer to Section 7.2.4.2 for details.
- The port shall transition to U2 upon successful completion of LGO_U2 entry sequence. Refer to Section 7.2.4.2 for details.
- The port shall transition to U3 upon successful completion of LGO_U3 entry sequence. Refer to Section 7.2.4.2 for details.
- A downstream port shall transition to eSS.Inactive when it fails U3 entry on three consecutive attempts.
- The port shall transition to Recovery upon any errors stated in Section 7.3 that will cause a link to transition to Recovery.
- The port shall transition to Recovery upon detection of a TS1 ordered set.
- The port shall transition to Recovery when directed.
- The port shall transition to eSS.Inactive when PENDING_HP_TIMER times out for the fourth consecutive time.

NOTE This implies the link has transitioned to Recovery for three consecutive times and each time the transition to Recovery is due to PENDING_HP_TIMER timeout.

- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to eSS.Inactive when directed.
- An upstream port shall transition to eSS.Disabled when directed.

NOTE After entry to U0 and the successful completion of training and link initialization, both ports are required to exchange port capabilities information using Port Capability LMPs within tPortConfiguration time as defined in Section 8.4.5. This includes the following scenarios:

- 1) Entry to U0 from polling directly;
- 2) Entry to U0 indirectly from Polling through Hot Reset;
- 3) Entry to U0 from Recovery and port configuration has not been successfully completed after exiting from Polling. In this case, both ports shall continue the port configuration process by completing the remaining LMP exchanges.

If the port has not received a Port Capability LMP within tPortConfiguration time, a downstream port shall be directed to transition to eSS.Inactive and an upstream port shall be directed to transition to eSS.Disabled.

- A downstream port shall transition to Recovery upon not receiving any link commands within 1 ms (tU0RecoveryTimeout).

NOTE Not receiving any link commands including LUP within 1 ms implies either a link is under serious error condition, or an upstream port has been removed. To accommodate for both situations, a downstream port will transition to Recovery and attempt to retrain the link. If the retraining fails, it will then transition to eSS.Inactive. During eSS.Inactive, a downstream port will attempt a far-end receiver termination detection. If it determines that a far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21 is not present, it will enter Rx.Detect. Otherwise, it will wait for software intervention.

- An upstream port shall transition to Recovery if it does not receive any link command or any packet (as specified in Section 7.2.4.1.4) within 1 ms (tU0RecoveryTimeout).
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.
- An upstream port shall transition to eSS.Disabled upon detection of VBUS off.

NOTE This condition only applies to a self-powered upstream port. eSS.Disabled is a logical power-off state for a self-powered upstream port.

7.5.7 U1

U1 is a low power state where no packets are to be transmitted and both ports agree to enter a link state where an Enhanced SuperSpeed PHY can be placed into a low power state.

U1 does not contain any substates. Transitions to other states are shown in Figure 7-19.

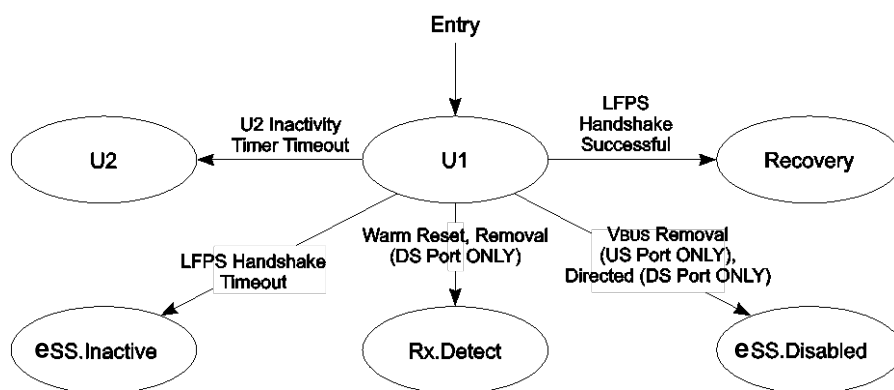
7.5.7.1 U1 Requirements

- The Enhanced SuperSpeed transmitter DC common mode voltage shall be within specification ($V_{TX-CM-DC-ACTIVE-IDLE-DELTA}$) defined in Table 6-18.
- The port shall maintain its low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- The port shall enable its U1 exit detect functionality as defined in Section 6.9.2.
- The port shall enable its LFPS transmitter when it initiates the exit from U1.
- The port shall enable its U2 inactivity timer upon entry to this state if the U2 inactivity timer has a non-zero timeout value.
- A downstream port shall enable its Ping.LFPS detection.
- A downstream port shall enable a 300-ms timer (tU1PingTimeout). This timer will be reset and restarted when a Ping.LFPS is received.
- An upstream port shall transmit Ping.LFPS as defined in Table 6-29.

7.5.7.2 Exit from U1

- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when the 300-ms timer (tU1PingTimeout) expires.

- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.
- A self-powered upstream port shall transition to eSS.Disabled upon not detecting valid Vbus as defined in Section 11.4.5
- The port shall transition to U2 upon the timeout of the U2 inactivity timer defined in Sections 10.4.2.4 and 10.6.2.4
- The port shall transition to Recovery upon successful completion of a LFPS handshake meeting the U1 LFPS exit handshake signaling in Section 6.9.2
- The port shall transition to eSS.Inactive upon the 2-ms (tNoLFPSResponseTimeout) LFPS handshake timer timeout and a successful LFPS handshake meeting the U1 LFPS exit handshake signaling in Section 6.9.2 is not achieved.



Note: Transition conditions are illustrative only. Not all of the transition conditions are listed.

U-051
IEC

Figure 7-19 – U1

7.5.8 U2

U2 is a link state where more power saving opportunities are allowed compare to U1, but with an increased exit latency.

U2 does not contain any substates. The transitions to other states are shown in Figure 7-20.

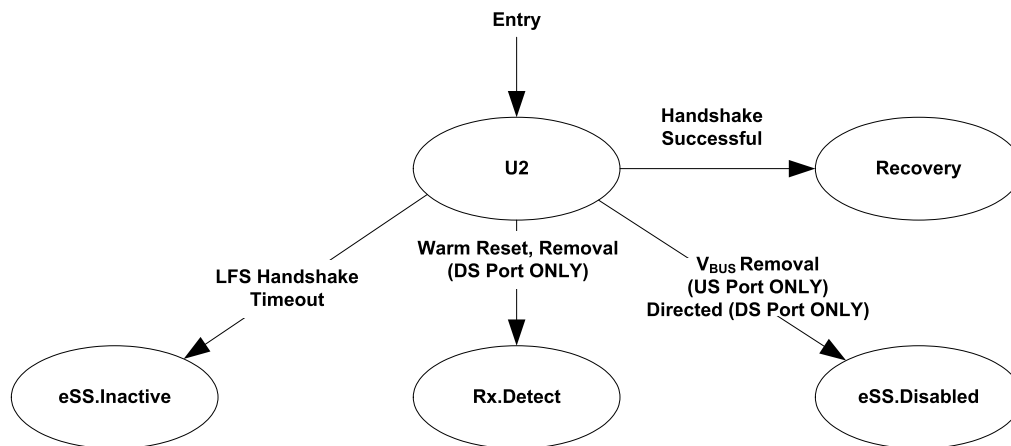
7.5.8.1 U2 Requirements

- The Enhanced SuperSpeed transmitter DC common mode voltage does not need to be within specification (VTX-CM-DC-ACTIVE-IDLE-DELTA) defined in Table 6-18
- The port shall maintain its low-impedance receiver termination (RRX-DC) defined in Table 6-21
- When a downstream port is in U2, its upstream port may be in U1 or U2. If the upstream port is in U1, it will send Ping.LFPS periodically. A downstream port shall differentiate between Ping.LFPS and U1 LFPS exit handshake signaling.
- The port shall enable its U2 exit detect functionality as defined in Section 6.9.2
- The port shall enable its LFPS transmitter when it initiates the exit from U2.
- A downstream port shall perform a far-end receiver termination detection every 100 ms (tU2RxdetDelay).

7.5.8.2 Exit from U2

- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect upon detection of a far-end high-impedance receiver termination (ZRX-HIGH-IMP-DC-POS) defined in Table 6-21.

- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.
- A self-powered upstream port shall transition to eSS.Disabled upon not detecting valid Vbus as defined in Section 11.4.5.
- The port shall transition to Recovery upon successful completion of a LFPS handshake meeting the U2 LFPS exit signaling defined in Section 6.9.2.
- The port shall transition to eSS.Inactive upon the 2-ms LFPS handshake timer timeout (tNoLFPSResponseTimeout) and a successful LFPS handshake meeting the U2 LFPS exit handshake signaling in Section 6.9.2 is not achieved.



Note: Transition conditions are illustrative only, Not all of the transition conditions are listed.

IEC

Figure 7-20 – U2

7.5.9 U3

U3 is a link state where a device is put into a suspend state. Significant link and device powers are saved.

U3 does not contain any substates. Transitions to other states are shown in Figure 7-21.

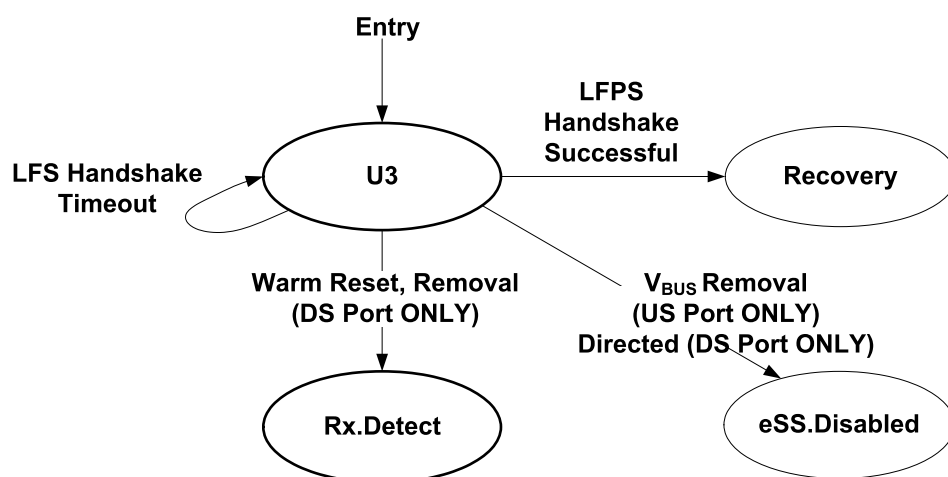
7.5.9.1 U3 Requirements

- The Enhanced SuperSpeed transmitter DC common mode voltage does not need to be within specification ($V_{TX-CM-DC-ACTIVE-IDLE-DELTA}$) defined in Table 6-18.
- The port shall maintain its low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.
- LFPS Ping detection shall be disabled.
- The port shall enable its U3 wakeup detect functionality as defined in Section 6.9.2
- The port shall enable its LFPS transmitter when it initiates the exit from U3.
- A downstream port shall perform a far-end receiver termination detection every 100 ms ($t_{U3RxdetDelay}$).
- The port not able to respond to U3 LFPS wakeup within $t_{NoLFPSResponseTimeout}$ may initiate U3 LFPS wakeup when it is ready to return to U0.

7.5.9.2 Exit from U3

- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect upon detection of a far-end high-impedance receiver termination ($Z_{RX-HIGH-IMP-DC-POS}$) defined in Table 6-21

- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.
- A self-powered upstream port shall transition to eSS.Disabled upon not detecting valid Vbus as defined in Section 11.4.5
- The port shall transition to Recovery upon successful completion of a LFPS handshake meeting the U3 wakeup signaling defined in Section 6.9.2
- The port shall remain in U3 when the 10-ms LFPS handshake timer times out (tNoLFPSResponseTimeout) and a successful LFPS handshake meeting the U3 wakeup handshake signaling in Section 6.9.2 is not achieved. 100 ms (tU3WakeupRetryDelay) after an unsuccessful LFPS handshake and the requirement to exit U3 still exists, then the port shall initiate the U3 wakeup LFPS Handshake signaling to wake up the host.



Note: Transition conditions are illustrative only, Not all of the transition conditions are listed.

IEC

Figure 7-21 – U3

7.5.10 Recovery

The Recovery link state is entered to retrain the link, or to perform Hot Reset, or to switch to Loopback mode. In order to retrain the link and also minimize the recovery latency, the two link partners do not train the receiver equalizers. Instead, the last trained equalizer configurations are maintained. Only TS1 and TS2 ordered sets are transmitted to synchronize the link and to exchange the link configuration information defined in Table 6-5.

7.5.10.1 Recovery Substate Machines

Recovery contains a substate machine shown in Figure 7-22 with the following substates:

- Recovery.Active
- Recovery.Configuration
- Recovery.Idle

7.5.10.2 Recovery Requirements

- The port shall meet the transmitter specifications as defined in Table 6-17.
- The port shall maintain the low-impedance receiver termination (R_{RX-DC}) as defined in Table 6-21.
- For SuperSpeed USB, all header packets in the Tx Header Buffers and the Rx Header Buffers shall be handled based on the requirements specified in Section 7.2.4.

- For SuperSpeedPlus USB, all header packets and data packet headers in the Type 1/Type 2 Tx Header Buffers and the Type 1/Type 2 Rx Buffers shall be handled based on the requirements specified in Section 7.2.4.

7.5.10.3 Recovery.Active

Recovery.Active is a substate to train the Enhanced SuperSpeed link by transmitting the TS1 ordered sets.

7.5.10.3.1 Recovery.Active Requirements

- A 12-ms timer (tRecoveryActiveTimeout) shall be started upon entry to this substate.
- The port shall transmit the TS1 ordered sets upon entry to this substate.
- The port shall train its receiver with TS1 or TS2 ordered sets.

NOTE Depending on the link condition and different receiver implementations, one port's receiver may train faster than the other. When this occurs, the port whose receiver trains first will enter Recovery.Configuration and start transmitting TS2 ordered sets while the port whose receiver is not yet trained is still in Recovery.Active using the TS2 ordered sets to train its receiver.

- The port in SuperSpeedPlus operation shall insert a SYNC ordered set every 32 TS1 ordered sets.
- The port in SuperSpeedPlus operation shall perform block alignment and scrambler synchronization as defined in Sections 6.3.2.3 and 6.4.1.2.4.

7.5.10.3.2 Exit from Recovery.Active

- The port in SuperSpeed operation shall transition to Recovery.Configuration after eight consecutive and identical TS1 or TS2 ordered sets are received.
- The port in SuperSpeedPlus operation shall transition to Recovery.Configuration upon receiving eight consecutive and identical TS1 or TS2 ordered sets, excluding symbols 14 and 15 of TS1 or TS2 ordered sets.

NOTE SYNC OS and SKP OS in between TS1 OS and/or TS2 OS do not disqualify the consecutive detection of TS1 OS and TS2 OS. Symbols 14 and 15 are used for TS1 or TS2 ordered set identifier or DC balance adjustment.

- The port shall transition to eSS.Inactive when the following conditions are met:
 - 1) Either the Ux_EXIT_TIMER or the 12-ms timer (tRecoveryActiveTimeout) times out.
 - 2) For a downstream port, the transition to Recovery is not to attempt a Hot Reset.
- A downstream port shall transition to Rx.Detect when the following conditions are met:
 - 1) Either the Ux_EXIT_TIMER or the 12-ms timer (tRecoveryActiveTimeout) times out.
 - 2) The transition to Recovery is to attempt a Hot Reset.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.10.4 Recovery.Configuration

Recovery.Configuration is a substate designed to allow the two link partners to achieve the Enhanced SuperSpeed handshake by exchanging the TS2 ordered sets.

7.5.10.4.1 Recovery.Configuration Requirements

- The port shall transmit identical TS2 ordered sets upon entry to this substate and set the link configuration field in the TS2 ordered set based on the following:
 - 1) When directed, a downstream port shall set Reset bit in the TS2 ordered set.

NOTE An upstream port can only set the Reset bit in the TS2 ordered set when in Hot Reset. Active. Refer to Section 7.5.12.3 for details.

- 2) When directed, the port shall set Loopback bit in the TS2 ordered set.
- 3) When directed, the port shall set the Disabling Scrambling bit in the TS2 ordered set.
- A 6-ms timer (tRecoveryConfigurationTimeout) shall be started upon entry to this substate.
- The port in SuperSpeedPlus operation shall insert a SYNC ordered set every 32 TS2 ordered sets.
- The port in SuperSpeedPlus operation shall perform block alignment and scrambler synchronization as defined in Sections 6.3.2.3 and 6.4.1.2.1.

7.5.10.4.2 Exit from Recovery.Configuration

- The port in SuperSpeed operation shall transition to Recovery.Idle after the following two conditions are met:
 - 1) Eight consecutive and identical TS2 ordered sets are received.
 - 2) Sixteen TS2 ordered sets are sent after receiving the first of the eight consecutive and identical TS2 ordered sets.
- The port in SuperSpeedPlus operation shall transition to Recovery.Idle when the following two conditions are met:
 - 1) Eight consecutive and identical TS2 ordered sets, excluding symbols 14 and 15, are received.
 - 2) Sixteen TS2 ordered sets are sent after receiving the first of the eight consecutive and identical TS2 ordered sets, excluding symbols 14 and 15.
- The port shall transition to eSS.Inactive when the following conditions are met:
 - 1) Either the Ux_EXIT_TIMER or the 6-ms timer (tRecoveryConfigurationTimeout) times out.
 - 2) For a downstream port, the transition to Recovery is not to attempt a Hot Reset.
- A downstream port shall transition to Rx.Detect when the following conditions are met:
 - 1) Either the Ux_EXIT_TIMER or the 6-ms timer (tRecoveryConfigurationTimeout) times out.
 - 2) The transition to Recovery is to attempt a Hot Reset.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.10.5 Recovery.Idle

Recovery.Idle is a substate where a port decodes the link configuration field defined in the TS2 ordered set received during Recovery.Configuration and determines the next state.

7.5.10.5.1 Recovery.Idle Requirements

- A 2-ms timer (tRecoveryIdleTimeout) shall be started upon entry to this substate.
- The port in SuperSpeed operation shall transmit Idle Symbols if the next state is U0. The port may transmit Idle Symbols if the next state is Loopback or HotReset.
- The port shall decode the link configuration field defined in the TS2 ordered sets received during Recovery.Configuration and proceed to the next state.
- The port in SuperSpeed operation shall enable the scrambling by default if the Disabling Scrambling bit is not asserted in the TS2 ordered set received in Recovery.configuration.
- The port in SuperSpeed operation shall disable the scrambling if directed, or if the Disabling Scrambling bit is asserted in the TS2 ordered set received in Recovery.configuration.

- The port in SuperSpeedPlus operation shall transmit a single SDS ordered set before the start of the data blocks with Idle Symbols if the next state is U0. The port may transmit SDS ordered set if the next state is Loopback or Hot Reset.

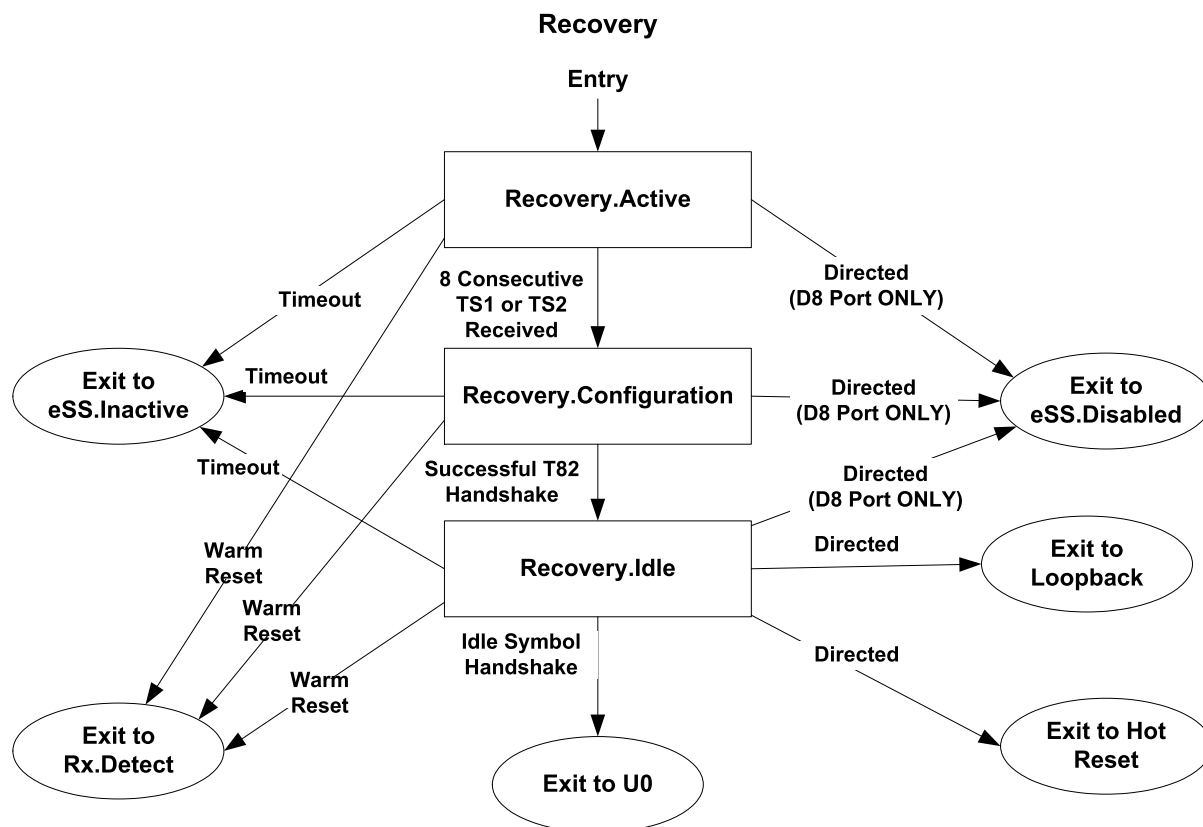
NOTE Under situation where a SKP ordered set is also scheduled at the same time with SDS ordered set, SKP ordered set shall be transmitted first.

- The port in SuperSpeedPlus operation shall disable the scrambling upon completion of SDS ordered set transmission if directed, or if the Disabling Scrambling bit is asserted in the TS2 ordered set received in Recovery.Configuration.
- The port in SuperSpeedPlus operation may ignore SDS ordered set if corrupted and continue to process the following data block. The port may optionally choose to recover SDS ordered set if error is detected.
- The port shall be able to receive the Header Sequence Number Advertisement from its link partner.

NOTE The exit time difference between the two ports will result in one port entering U0 first and starting the Header Sequence Number Advertisement while the other port is still in Recovery.Idle.

7.5.10.5.2 Exit from Recovery.Idle

- The port shall transition to Loopback when directed as a loopback master and the port is capable of being a loopback master.
- The port shall transition to Loopback as a loopback slave if the Loopback bit is asserted in TS2 ordered sets.
- The port shall transition to U0 when the following two conditions are met:
 - 1) Eight consecutive Idle Symbols are received.
 - 2) Sixteen Idle Symbols are sent after receiving one Idle Symbol.
- The port shall transition to eSS.Inactive when one of the following timers times out and the conditions to transition to U0 are not met:
 - 1) Ux_EXIT_TIMER
 - 2) The 2-ms timer (tRecoveryIdleTimeout)
- A downstream port shall transition to Hot Reset when directed.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.
- An upstream port shall transition to Hot Reset if the Reset bit is asserted in TS2 ordered sets.



Note: Transition conditions are illustrative only. Not all transition conditions are listed.

IEC

Figure 7-22 – Recovery Substate Machine

7.5.11 Loopback

Loopback is intended for test and fault isolation. Loopback includes a bit error rate test (BERT) state machine, described in Chapter 6.

A loopback master is the port requesting loopback. A loopback slave is the port that retransmits the symbols received from the loopback master.

During Loopback.Active, the loopback slave must support the BERT protocol described in Chapter 6. The loopback slave must respond to the command for BERT error counter reset and BERT report error count. The loopback slave must check the incoming data for the loopback data pattern.

7.5.11.1 Loopback Substate Machines

Loopback contains a substate machine shown in Figure 7-23 with the following substates:

- Loopback.Active
- Loopback.Exit

7.5.11.2 Loopback Requirements

- There shall be one loopback master and one loopback slave. The loopback master is the port that has the Loopback bit asserted in TS2 ordered sets.
- The port shall maintain its transmitter specifications defined in Table 6-17.
- The port shall maintain its low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21.

7.5.11.3 Loopback.Active

Loopback.Active is a substate where the loopback test is active. The loopback master is sending data/commands to its loopback slave. The loopback slave is either looping back the data or detecting/executing the commands it received from the loopback master.

7.5.11.3.1 Loopback.Active Requirements

- The loopback master shall send valid symbols with SKPs as necessary.
- The loopback slave shall retransmit the received symbols.
- The loopback slave shall not modify the received symbols, other than lane polarity inversion if necessary, and SKP ordered set, which may be added or dropped as required.

NOTE For SuperSpeed operation this implies that the loopback slave should disable or bypass its own 8b/10b encoder/decoder and scrambler/descrambler. For SuperSpeedPlus operation this implies that the loopback slave should disable or bypass its own scrambler/descrambler.

- The loopback slave must process the BERT commands as defined in Section 6.8.4.
- The LFPS receiver shall be enabled.

7.5.11.3.2 Exit from Loopback.Active

- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.
- When directed, the loopback master shall transition to Loopback.Exit.
- The loopback slave shall transition to Loopback.Exit upon detection of Loopback LFPS exit handshake signal meeting Loopback LFPS exit signaling defined in Section 6.9.2

7.5.11.4 Loopback.Exit

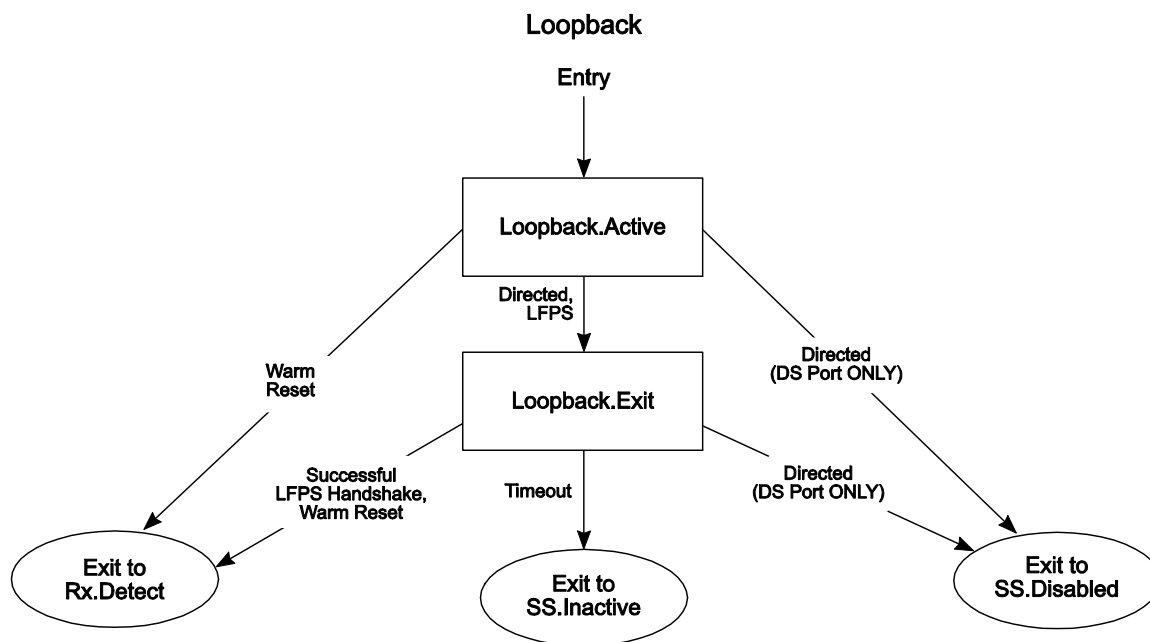
Loopback.Exit is a substate where a loopback master has completed the loopback test and starts the exit from Loopback.

7.5.11.4.1 Loopback.Exit Requirements

- A 2-ms timer (tLoopbackExitTimeout) shall be started upon entry to the substate.
- The LFPS transmitter and the LFPS receiver shall be enabled.
- The port shall transmit and receive Loopback LFPS exit handshake signal defined in Section 6.9.2

7.5.11.4.2 Exit from Loopback.Exit

- The port shall transition to Rx.Detect upon a successful Loopback LFPS exit handshake defined in Section 6.9.2
- The port shall transition to eSS.Inactive upon the 2-ms timer timeout (tLoopbackExitTimeout) and the condition to transition to Rx.Detect is not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.



Note: Transition conditions are illustrative only. Not all of the transition conditions are listed.

U-055
IEC

Figure 7-23 – Loopback Substate Machine

7.5.12 Hot Reset

Only a downstream port can be directed to initiate a Hot Reset.

When the downstream port initiates reset, it shall transmit TS2 ordered sets with the Reset bit asserted. The upstream port shall respond by sending the TS2 ordered sets with Reset bit asserted. Upon completion of Hot Reset processing, the upstream port shall signal the downstream port by sending the TS2 ordered sets with the Reset bit de-asserted. The downstream port shall respond with the Reset bit de-asserted in the TS2 ordered sets. Once both ports receive the TS2 ordered sets with the Reset bit de-asserted, they shall exit from Hot Reset.Active and transition to Hot Reset.Exit. Once a successful idle symbol handshake is achieved, the port shall return to U0.

7.5.12.1 Hot Reset Substate Machines

Hot Reset contains a substate machine shown in Figure 7-24 with the following substates:

- Hot Reset.Active
- Hot Reset.Exit

7.5.12.2 Hot Reset Requirements

- A downstream port shall reset its Link Error Count as defined in Section 7.4.2.
- A downstream port shall reset its PM timers and the associated U1 and U2 timeout values to zero.
- The port in SuperSpeedPlus operation shall reset the Soft Error Count if implemented.
- The port Configuration information shall remain unchanged (refer to Section 8.4.6 for details).
- The port shall maintain its transmitter specifications defined in Table 6-17.
- The port shall maintain its low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21

7.5.12.3 Hot Reset.Active

Hot Reset.Active is a substate where a port will perform the reset as defined in Section 7.4.2.

7.5.12.3.1 Hot Reset.Active Requirements

- Upon entry to this substate, the port shall first transmit at least 16 TS2 ordered sets continuously with the Reset bit asserted.

NOTE Depending on the time delay between the two ports entering Hot Reset, when the downstream port is transmitting the first 16 TS2 ordered sets with the Reset bit asserted, it may still receive part of the TS2 ordered sets from the upstream port exiting from Polling.Configuration or Recovery.Configuration. The downstream port shall ignore those TS2 ordered sets. Also upon entry to this substate, both ports shall ignore the Disabling Scrambling bit in the link configuration field of the TS2 Ordered Set. This bit is only decoded in Polling.Idle or Recovery.Idle.

- A 12-ms timer (tHotResetActiveTimeout) shall be started upon entry to this substate.
- A downstream port shall continue to transmit TS2 ordered sets with the Reset bit asserted until the upstream port transitions from sending TS2 ordered sets with the Reset bit asserted to sending the TS2 ordered sets with the Reset bit de-asserted.
- An upstream port shall transmit TS2 ordered sets with the Reset bit asserted while performing the Hot Reset.
- An upstream port shall transmit TS2 ordered sets with the Reset bit de-asserted after completing the Hot Reset.
- The port shall perform Hot Reset described in Hot Reset requirement of this section.

7.5.12.3.2 Exit from Hot Reset.Active

- The port shall transition to Hot Reset.Exit when the following three conditions are met.
 - 1) At least 16 TS2 ordered sets with the Reset bit asserted are transmitted.
 - 2) Two consecutive TS2 ordered sets are received with the Reset bit de-asserted.
 - 3) Four consecutive TS2 ordered sets with the Reset bit de-asserted are sent after receiving one TS2 ordered set with the Reset bit de-asserted.
- The port shall transition to eSS.Inactive upon the 12-ms timer timeout (tHotResetActiveTimeout) and the conditions to transition to Hot Reset.Exit are not met.
- The port in SuperSpeedPlus operation may ignore SDS OS if corrupted.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

7.5.12.4 Hot Reset.Exit

Hot Reset.Exit is a substate where the port has completed Hot Reset and is ready to exit from Hot Reset.

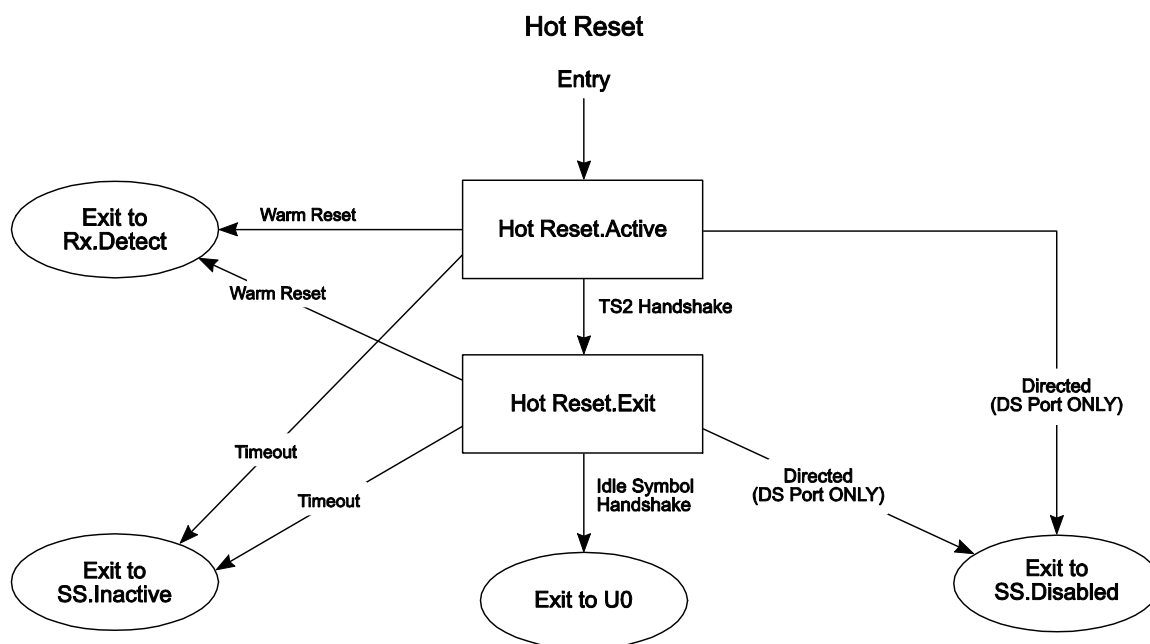
7.5.12.4.1 Hot Reset.Exit Requirements

- The port in SuperSpeed operation shall transmit idle symbols.
- The port in SuperSpeedPlus operation shall transmit a single SDS ordered set before the start of the data block with Idle Symbols.
- The port in SuperSpeedPlus operation may ignore SDS ordered set if corrupted and continue to process the following data block. The port may optionally choose to recover SDS ordered set if error is detected.
- A 2-ms timer (tHotResetExitTimeout) shall be started upon entry to this substate.
- The port shall be able to receive the Header Sequence Number Advertisement from its link partner.

NOTE The exit time difference between the two ports will result in one port entering U0 first and starting the Header Sequence Number Advertisement while the other port is still in Hot Reset.Exit.

7.5.12.4.2 Exit from Hot Reset.Exit

- The port shall transition to U0 when the following two conditions are met:
- Eight consecutive Idle Symbols are received.
- Sixteen Idle Symbols are sent after receiving one Idle Symbol.
- The port shall transition to eSS.Inactive upon the 2-ms timer timeout (tHotResetExitTimeout) and the conditions to transition to U0 are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.



Note: Transition conditions are illustrative only. Not all of the transition conditions are listed.

U-056
IEC

Figure 7-24 – Hot Reset Substate Machine

8 Protocol Layer

The protocol layer manages the end-to-end flow of data between a device and its host. This layer is built on the assumption that the link layer guarantees delivery of header packets and this layer adds on end to end reliability for the rest of the packets depending on the transfer type.

Where not specifically noted, requirements apply to both the SuperSpeed and SuperSpeedPlus architectures. Gen 2 speed capable devices operating at Gen 1 speed, regardless of their additional capabilities (e.g. Gen 2 speed), shall only use features of the SuperSpeed architecture.

The chapter describes the following in detail:

- Types of packets
- Format of the packets
- Expected responses to packets sent by the host and a device

- The four USB defined transfer types
- Support for Streams for the bulk transfer type
- Timing parameters for the various responses and packets the host or a device may receive or transmit

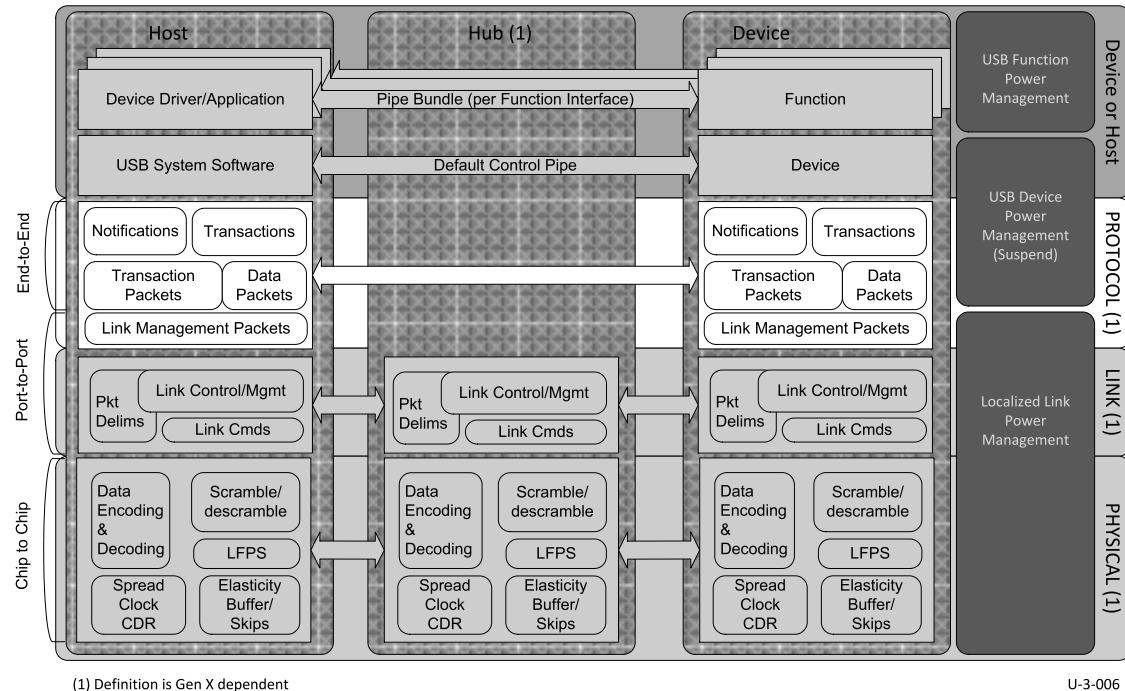


Figure 8-1 – Protocol Layer Highlighted

8.1 Enhanced SuperSpeed Transactions

The Enhanced SuperSpeed Bus defines multiple speeds at which the bus can operate. The rules for transactions on a SuperSpeed bus instance are defined in Section 8.1.1. The rules for transactions on a SuperSpeedPlus bus instance are defined in Section 8.1.2.

8.1.1 Transactions on a SuperSpeed Bus Instance

Transactions are initiated by the host when it either requests or sends data to an endpoint on a device and are completed when the endpoint sends the data or acknowledges receipt of the data. A transfer on the SuperSpeed bus instance is a request for data made by a device application to the host which then breaks it up into one or more burst transactions. A host may initiate one or more OUT bus transactions to one or more endpoints while it waits for the completion of the current bus transaction. However, a host shall not initiate another IN bus transaction to any endpoint on the same SuperSpeed bus instance until the host:

- For non-isochronous endpoint
 - 1) receives all requested Data Packets (DPs) or
 - 2) receives a short packet or
 - 3) receives a DP with EOB flag set or
 - 4) receives an NRDY or a STALL Transaction Packet (TP) or
 - 5) times out the transaction for the current ACK TP
- For isochronous endpoint
 - 1) receives all the DPs that were requested or
 - 2) receives a short packet or

- 3) receives a DP with last packet flag field set or
- 4) times out the transaction for the current ACK TP.

For non-isochronous transactions, an endpoint may respond to valid transactions by:

- Returning an NRDY Transaction Packet
- Accepting it by returning an ACK Transaction Packet in the case of an OUT transaction
- Returning one or more data packets in the case of an IN transaction
- Returning a STALL Transaction Packet if there is an internal endpoint error

An NRDY Transaction Packet (TP) response indicates that an endpoint is not ready to sink or source data. This allows the links between the device and the host to be placed in a reduced power state until an endpoint is ready to receive or send data. However, as mentioned in Section 8.10.1, the host may continue to perform transactions with the endpoint on the device even before the endpoint notifies the host that it is ready. When ready, the endpoint asynchronously sends an ERDY TP to the host to tell it that it is now ready to move data and the host responds by rescheduling the request. Note that isochronous transactions do not use ERDY or NRDY TPs as they are serviced by the host at periodic intervals. Additionally, data packets sent to or received from an isochronous endpoint are not acknowledged, i.e., no ACK TPs are sent to acknowledge the receipt of data packets.

Endpoints only respond to requests made by the host. The host is responsible for scheduling transactions on the bus and maintaining the priority and fairness of the data movement on the bus; it does this by the timing and ordering of IN and OUT requests. Transactions are not broadcast; packets traverse a direct path between the host and device. Any unused links may be placed into reduced power states making the bus amenable to aggressive power management.

8.1.2 Transactions on a SuperSpeedPlus Bus Instance

Transactions on a SuperSpeedPlus bus instance follow the rules defined in Section 8.1.1 with the following modifications:

- A SuperSpeedPlus host may issue simultaneous IN requests
- A SuperSpeedPlus host should pipeline Isochronous IN transactions as described in Section 8.12.6.3.1
- A SuperSpeedPlus device shall support simultaneous IN requests to different endpoints
- Transactions may arrive/complete in a different order than they were initiated

8.1.2.1 Simultaneous IN Transactions

A SuperSpeedPlus host may initiate simultaneous IN transactions. However, a SuperSpeedPlus host shall not initiate simultaneous IN transactions to:

- The same endpoint
- A SuperSpeed bus instance

NOTE An ACK TP to continue a burst does not constitute a new IN transaction.

8.1.2.2 Transaction Reordering

Packets (DPs and TPs) may be delivered to the intended recipient in a different order than they were originated. This may happen due to the following:

- A SuperSpeedPlus hub may reorder due to the SuperSpeedPlus packet ordering rules. Refer to Section 10.8.6.
- A SuperSpeedPlus device may reorder asynchronous and periodic IN requests.

- SuperSpeedPlus devices and hubs will transmit TPs before DPs (for both periodic and asynchronous packets)

8.2 Packet Types

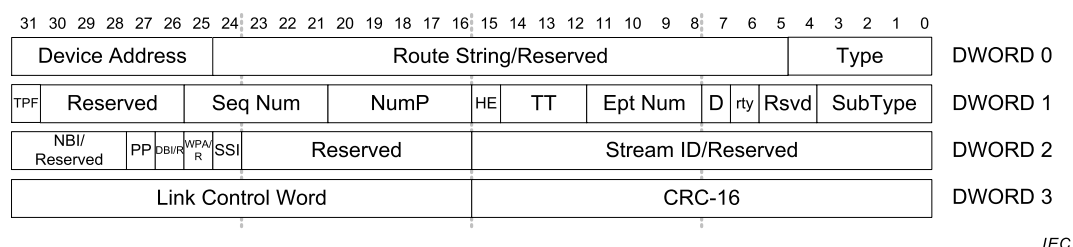
Enhanced SuperSpeed USB uses four basic packet types each with one or more subtypes. The four packet types are:

- **Link Management Packets** (LMP) only travel between a pair of links (e.g., a pair of directly connected ports) and is primarily used to manage that link.
- **Transaction Packets** (TP) traverse all the links directly connecting the host to a device. They are used to control the flow of data packets, configure devices, and hubs, etc. Transaction Packets have no data payload.
- **Data Packets** (DP) traverse all the links directly connecting the host to a device. Data Packets have two parts: a Data Packet Header (DPH) and a Data Packet Payload (DPP).
- **Isochronous Timestamp Packets** (ITP) are multicast on all the active links.

All packets consist of a 14-byte header, followed by a 2-byte Link Control Word at the end of the packet (16 bytes total). All headers have a Type field that is used by the receiving entity (e.g., host, hub, or device) to determine how to process the packet. All headers include a 2-byte CRC (CRC-16).

All devices (including hubs) and the host consume the LMPs they receive.

If the value of the **Type** field is *Transaction Packet* or *Data Packet Header*, the **Route String** and **Device Address** fields follow the **Type** field. The **Route String** field is used by hubs to route packets which appear on their upstream port to the appropriate downstream port. Packets flowing from a device to the host are always routed from a downstream port on a hub to its upstream port. The **Device Address** field is provided to the host so that it can identify the source of a packet. All other fields are discussed further in this chapter.



IEC

Figure 8-2 – Example Transaction Packet

Data Packets include additional information in the header that describes the data block. The Data Block is always followed by a 4-byte CRC-32 used to determine the correctness of the data. The Data Block and the CRC-32 together are referred to as the Data Packet Payload.

8.3 Packet Formats

Packet byte and bit definitions in this section are described in an un-encoded data format. The effects of symbols added to the serial stream (i.e., to frame packets or control or modify the link), bit encoding, bit scrambling, and link level framing have been removed for the sake of clarity. Refer to Chapters 6 and 7 for detailed information.

8.3.1 Fields Common to all Headers

All Enhanced SuperSpeed headers start with the **Type** field that is used to determine how to interpret the packet. At a high level this tells the recipient of the packet what to do with it: either to use it to manage the link or to move and control the flow of data between a device and the host.

8.3.1.1 Reserved Values and Reserved Field Handling

Reserved fields and Reserved values shall not be used in a vendor-specific manner.

A transmitter shall set all **Reserved** fields to zero and a receiver shall ignore any **Reserved** field.

A transmitter shall not set a defined field to a reserved value and a receiver shall ignore any packet that has any of its defined fields set to a reserved value. Note that the receiver shall acknowledge the packet and return credit for the same as per the requirement specified in Section 7.2.4.1.

NOTE SuperSpeedPlus hosts, hubs and devices use some fields previously marked as Reserved.

8.3.1.2 Type Field

The **Type** field is a 5-bit field that identifies the format of the packet. The type is used to determine how the packet is to be used or forwarded by intervening links.

Table 8-1 – Type Field Description

Width (bits)	Offset (DW:bit)	Description
5	0:0	Type. These 5 bits identify the packet's Type.
		<u>Value</u> <u>Description</u>
		00000bLink Management Packet.
		00100bTransaction Packet
		01000bData Packet Header
		01100bIsochronous Timestamp Packet
		All other values are Reserved.

8.3.1.3 CRC-16

All header packets have a 16-bit CRC field. This field is the CRC calculated over the preceding 12 bytes in the header packet. Refer to Section 7.2.1.1.2 for the polynomial used to calculate this value.

8.3.1.4 Link Control Word

The usage of the Link Control Word is defined in Section 7.2.1.1.3.

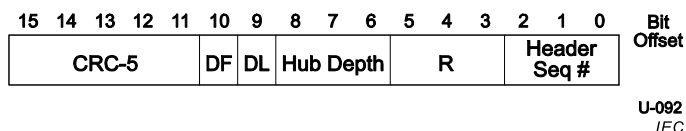


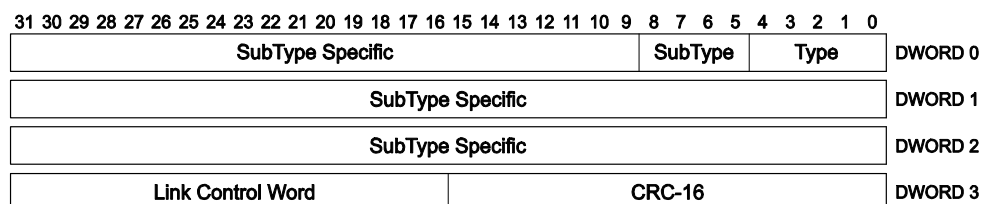
Figure 8-3 – Link Control Word Detail

Table 8-2 – Link Control Word Format

Width (bits)	Offset (DW:bit)	Description
3	3:16	Header Sequence Number. The valid values in this field are 0 through 7.
3	3:19	Reserved (R).
3	3:22	Hub Depth. This field is only valid when the Deferred bit is set and identifies to the host the hierarchical on the USB that the hub is located at in the deferred TP or DPH returned to the host. This informs the host that the port on which the packet was supposed to be forwarded on is currently in a low power state (either U1 or U2). The only valid values in this field are 0 through 4.
1	3:25	Delayed (DL). This bit may be set if a Header Packet is resent or the transmission of a Header Packet is delayed. Chapter 7 and Chapter 10 provide more details on when this bit shall be set. This bit shall not be reset by any subsequent hub that this packet traverses.
1	3:26	Deferred (DF). This bit may only be set by a hub. This bit shall be set when the downstream port on which the packet needs to be sent is in a power managed state. This bit shall not be reset by any subsequent hub that this packet traverses. Refer to sections 10.9.4.4.1 and 10.9.4.4.2 for hub packet deferral process.
5	3:27	CRC-5. This field is the CRC used to verify the correctness of the preceding 11 bits in this word. Refer to Section 7.2.1.1.3 for the polynomial used to calculate this value.

8.4 Link Management Packet (LMP)

Packets that have the **Type** field set to *Link Management Packet* are referred to as LMPs. These packets are used to manage a single link. They carry no addressing information and as such are not routable. They may be generated as the result of hub port commands. For example, a hub port command is used to set the U2 inactivity timeout. In addition, they are used to exchange port capability information and may be used for testing purposes.



U-093
IEC

Figure 8-4 – Link Management Packet Structure

8.4.1 Subtype Field

The value in the LMP **Subtype** field further identifies the content of the LMP.

Table 8-3 – Link Management Packet Subtype Field

Width (bits)	Offset (DW:bit)	Description																		
4	0:5	Subtype. These 4 bits identify the Link Packet Subtype. <table><tr><th><u>Value</u></th><th><u>Type of LMP</u></th></tr><tr><td>0000b</td><td>Reserved</td></tr><tr><td>0001b</td><td>Set Link Function</td></tr><tr><td>0010b</td><td>U2 Inactivity Timeout</td></tr><tr><td>0011b</td><td>Vendor Device Test</td></tr><tr><td>0100b</td><td>Port Capability</td></tr><tr><td>0101b</td><td>Port Configuration</td></tr><tr><td>0110b</td><td>Port Configuration Response</td></tr><tr><td>0111b-1111b</td><td>Reserved</td></tr></table>	<u>Value</u>	<u>Type of LMP</u>	0000b	Reserved	0001b	Set Link Function	0010b	U2 Inactivity Timeout	0011b	Vendor Device Test	0100b	Port Capability	0101b	Port Configuration	0110b	Port Configuration Response	0111b-1111b	Reserved
<u>Value</u>	<u>Type of LMP</u>																			
0000b	Reserved																			
0001b	Set Link Function																			
0010b	U2 Inactivity Timeout																			
0011b	Vendor Device Test																			
0100b	Port Capability																			
0101b	Port Configuration																			
0110b	Port Configuration Response																			
0111b-1111b	Reserved																			
16	3:0	CRC-16. This field is the CRC calculated over the preceding 12 bytes. Refer to Section 7.2.1.1.2 for the polynomial used to calculate this value.																		

8.4.2 Set Link Function

The **Set Link Function** LMP shall be used to configure functionality that can be changed without leaving the active (U0) state.

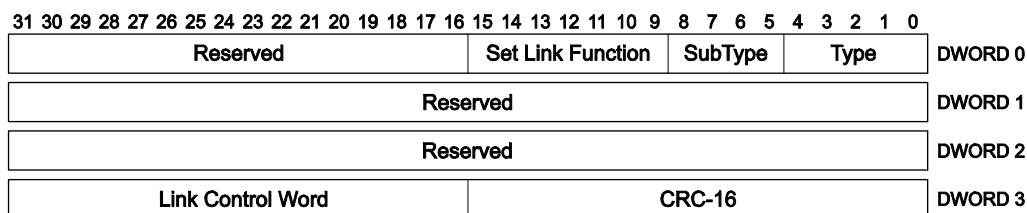
Upon receipt of an LMP with the Force_LinkPM_Accept bit asserted, the port shall accept all LGO_U1 and LGO_U2 Link Commands until the port receives an LMP with the Force_LinkPM_Accept bit de-asserted. After port receives an LMP with the Force_LinkPM_Accept bit de-asserted, port will function in normal mode doing power management based on packet pending state of device's endpoints.

The device must stay in U1 or U2 until the downstream port initiates exit to U0. Software must ensure that there are no pending packets at the link level before issuing a SetPortFeature command that generates an LGO_U1 or LGO_U2 link command.

During normal operation, this feature shall only be used if all other means of lowering the link state from U0 to U1 or U2 fail.

This LMP is sent by a hub to a device connected on a specific port when it receives a SetPortFeature (FORCE_LINKPM_ACCEPT) command. Refer to Section 10.16.2.2 and Section 10.16.2.10 for more details.

NOTE Improper use of the Force_LinkPM_Accept functionality can impact the performance of the link significantly and in some cases (when used during normal operation only) may lead to the device being unable to return to proper operation.



U-094
IEC

Figure 8-5 – Set Link Function LMP

Table 8-4 – Set Link Function

Width (bits)	Offset (DW:bit)	Description																
4	0:5	Subtype. This field shall be set to <i>Set Link Function</i> for a Set Link Function LMP.																
7	0:9	Set Link Function. These 7 bits identify the Set Link Function. <table><tr><th><u>Bits</u></th><th><u>Description</u></th></tr><tr><td>0</td><td>Reserved.</td></tr><tr><td>1</td><td>Force_LinkPM_Accept</td></tr><tr><td></td><td><table><tr><th><u>Value</u></th><th><u>Meaning</u></th></tr><tr><td>0</td><td>De-assert</td></tr><tr><td>1</td><td>Assert</td></tr></table></td></tr><tr><td>6:2</td><td>Reserved.</td></tr></table>	<u>Bits</u>	<u>Description</u>	0	Reserved.	1	Force_LinkPM_Accept		<table><tr><th><u>Value</u></th><th><u>Meaning</u></th></tr><tr><td>0</td><td>De-assert</td></tr><tr><td>1</td><td>Assert</td></tr></table>	<u>Value</u>	<u>Meaning</u>	0	De-assert	1	Assert	6:2	Reserved.
<u>Bits</u>	<u>Description</u>																	
0	Reserved.																	
1	Force_LinkPM_Accept																	
	<table><tr><th><u>Value</u></th><th><u>Meaning</u></th></tr><tr><td>0</td><td>De-assert</td></tr><tr><td>1</td><td>Assert</td></tr></table>	<u>Value</u>	<u>Meaning</u>	0	De-assert	1	Assert											
<u>Value</u>	<u>Meaning</u>																	
0	De-assert																	
1	Assert																	
6:2	Reserved.																	

8.4.3 U2 Inactivity Timeout

The U2 Inactivity Timeout LMP shall be used to define the timeout from U1 to U2. Refer to Section 10.6 for details on this LMP.

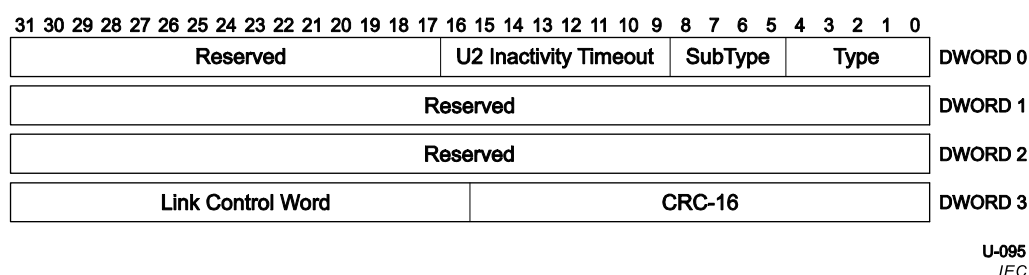


Figure 8-6 – U2 Inactivity Timeout LMP

Table 8-5 – U2 Inactivity Timer Functionality

Width (bits)	Offset (DW:bit)	Description
4	0:5	Subtype. This field shall be set to <i>U2 Inactivity Timeout</i> for a U2 Inactivity Timeout LMP.
8	0:9	U2 Inactivity Timeout. These 8 bits represent the U2 Inactivity Timeout value. The value placed in this field is the same value that is sent to the hub in a Set Port Feature (PORT_U2_TIMEOUT) command. Refer to Section 10.16.2.10 for details on the encoding of this field.

8.4.4 Vendor Device Test

Use of this LMP is intended for vendor-specific device testing and shall not be used during normal operation of the link.

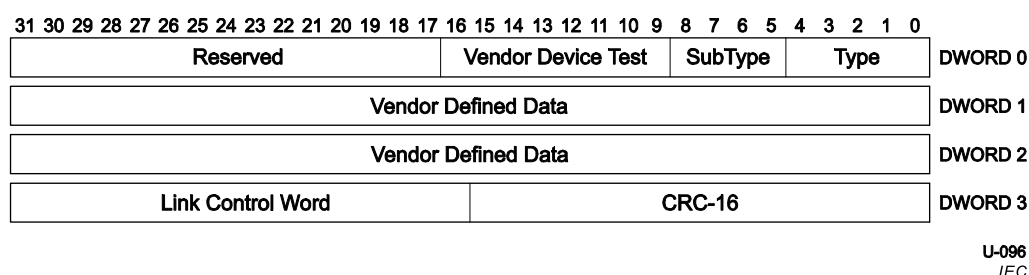


Figure 8-7 – Vendor Device Test LMP

Table 8-6 – Vendor-specific Device Test Function

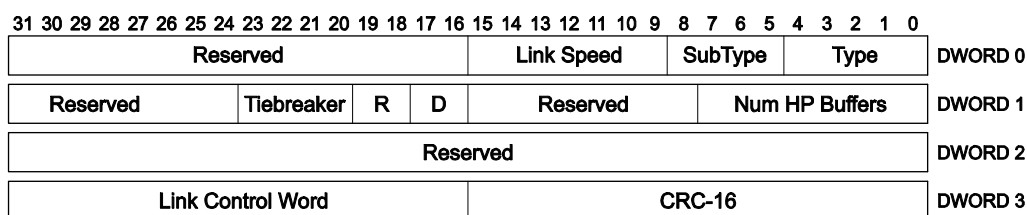
Width (bits)	Offset (DW:bit)	Description
4	0:5	Subtype. This field shall be set to <i>Vendor Device Test</i> .
8	0:9	Vendor-specific device test. The function of these 8 bits is vendor specific.
64	1:0	Vendor-defined data. This value is vendor-defined.

8.4.5 Port Capabilities

The Port Capability LMP describes each port's link capabilities and is sent by both link partners after the successful completion of training and link initialization. After the port enters U0 from Polling, the port shall send Port Capability LMP within tPortConfiguration time once link initialization (refer to Section 7.2.4.1.1) is completed. Note the port may not always transition directly from Polling to U0, but may transition through other intermediate states (e.g., Recovery or Hot Reset) before entering U0. Regardless of states passed through between Polling and entry into U0, the device shall send a Port Capability LMP immediately upon entering U0.

If a link partner does not receive this LMP within tPortConfiguration time then:

- If the link partner has downstream capability, it shall signal an error as described in Section 10.16.2.6.
- If the link partner only supports upstream capability, see Sections 10.5 and 10.18 which define the hub and peripheral upstream port connect states.



U-097
IEC

Figure 8-8 – Port Capability LMP

Table 8-7 – Port Capability LMP Format

Width (bits)	Offset (DW:bit)	Description						
4	0:5	SubType. This field shall be set to <i>Port Capability</i> .						
7	0:9	Link speed. When operating at Gen 1 speed, this field is a bitmask that describes the link speeds supported by this device. <table><tr><th>Bits</th><th>Description</th></tr><tr><td>0</td><td>This bit shall be set to 1 to indicate this device supports signaling at 5 Gbps</td></tr><tr><td>6:1</td><td>Reserved.</td></tr></table> When not operating at Gen 1 speed, this field is Reserved and shall be set to zero.	Bits	Description	0	This bit shall be set to 1 to indicate this device supports signaling at 5 Gbps	6:1	Reserved.
Bits	Description							
0	This bit shall be set to 1 to indicate this device supports signaling at 5 Gbps							
6:1	Reserved.							
16	0:16	Reserved (R).						
8	1:0	Num HP Buffers. When operating at Gen 1 speed this field specifies the number of header packet buffers (in each direction Transmit or Receive) this device supports. All devices that are compliant to this revision of the specification shall return a value of 4 in this field. All other values are reserved. When not operating at Gen 1 speed, this field is Reserved and shall be set to zero.						
8	1:8	Reserved (R).						
2	1:16	Direction (D). This field is used to identify the upstream or downstream capabilities of the port. All ports shall have at least one of these bits set. <table><tr><th>Bits</th><th>Description</th></tr><tr><td>0</td><td>If this bit is set to 1, then this port can be configured to be a downstream port.</td></tr><tr><td>1</td><td>If this bit is set to 1, then this port can be configured to be an upstream port.</td></tr></table>	Bits	Description	0	If this bit is set to 1, then this port can be configured to be a downstream port.	1	If this bit is set to 1, then this port can be configured to be an upstream port.
Bits	Description							
0	If this bit is set to 1, then this port can be configured to be a downstream port.							
1	If this bit is set to 1, then this port can be configured to be an upstream port.							
1	1:18	USB 3.0 OTG Capable (OTG). This field shall be set to 1 if the port supports the OTG Capability. Refer to Section 6.1 of the <i>On-The-Go and Embedded Host Supplement to the USB 3.0 Specification (Revision 3.0)</i> for more information.						
1	1:19	Reserved (R).						
4	1:20	Tiebreaker. This field is only valid when both bits 0 and 1 of the Direction field are set. This field is used to determine the port type when two devices with both upstream and downstream capability are connected to each other. See Table 8-8 for details. This field shall be set to zero in all other cases.						
40	1:24	Reserved.						

After exchanging Port Capability LMPs, the link partners shall determine which of the link partners shall be configured as the downstream facing port as specified in Table 8-8.

Table 8-8 – Port Type Selection Matrix

		Port 2		
		Upstream Only	Downstream Only	Both
Port 1	Upstream Only	Not Defined	Port 2 is the downstream port.	Port 2 is the downstream port.
	Downstream Only	Port 1 is the downstream port.	Not Defined	Port 1 is the downstream port.
	Both	Port 1 is the downstream port.	Port 2 is the downstream port.	The port with the higher value in the Tiebreaker field shall become the downstream port ¹ .

¹ If the **TieBreaker** field contents are equal, then the two link partners shall exchange Port Capability LMPs again with new and different value in the **TieBreaker** field. The sequence of TieBreaker field values generated by a port shall be sufficiently random.

8.4.6 Port Configuration

Only the fields that are different from the Port Capability LMP are described in this section.

All Enhanced SuperSpeed ports that support downstream port capability shall be capable of sending this LMP.

If the port that was to be configured in the upstream facing mode does not receive this LMP within *tPortConfiguration* time after link initialization, then the upstream port shall transition to *eSS.Disabled* and a peripheral device shall try and connect at the other speeds this device supports.

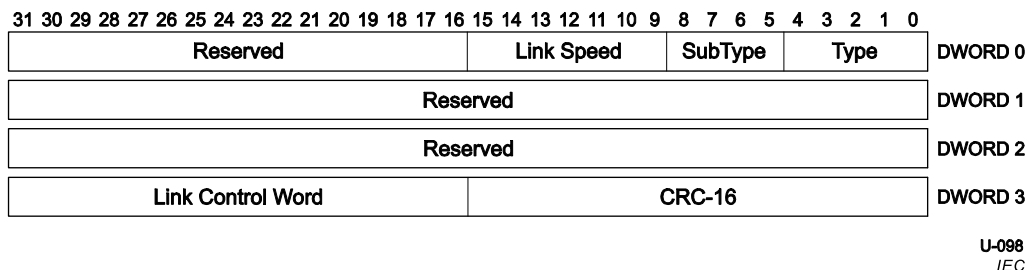


Figure 8-9 – Port Configuration LMP

Table 8-9 – Port Configuration LMP Format (Differences with Port Capability LMP)

Width (bits)	Offset (DW:bit)	Description						
4	0:5	SubType. This field shall be set to <i>Port Configuration</i> .						
7	0:9	Link speed. When operating at Gen 1 speed, this field describes the link speed at which the upstream port shall operate. Only one of the bits in this field shall be set in the Port Configuration LMP sent by the link partner configured in the downstream mode. <table><tr><th><u>Bits</u></th><th><u>Description</u></th></tr><tr><td>0</td><td>If this bit is set to 1, then this device shall operate at 5 Gbps.</td></tr><tr><td>6:1</td><td>Reserved.</td></tr></table> When not operating at Gen 1 speed, this field is Reserved and shall be set to zero.	<u>Bits</u>	<u>Description</u>	0	If this bit is set to 1, then this device shall operate at 5 Gbps.	6:1	Reserved.
<u>Bits</u>	<u>Description</u>							
0	If this bit is set to 1, then this device shall operate at 5 Gbps.							
6:1	Reserved.							
80	0:16b	Reserved.						

A port configured in the downstream mode shall send the Port Configuration LMP to the upstream port. The port sending this LMP shall select only one bit for the **Link Speed** field. The **Link Speed** field shall only be used when the port is operating at Gen 1 speed.

If a downstream capable port cannot work with its link partner, then the downstream capable port shall signal an error as described in Section 10.16.2.6.

8.4.7 Port Configuration Response

This LMP is sent by the upstream port in response to a Port Configuration. It is used to indicate acceptance or rejection of the Port Configuration LMP. Only the fields that are different from the Port Capability LMP are described in this section.

All Enhanced SuperSpeed ports that support upstream port capability shall be capable of sending this LMP.

If the downstream port does not receive this LMP within *tPortConfiguration* time, it shall signal an error as described in Section 10.16.2.6.

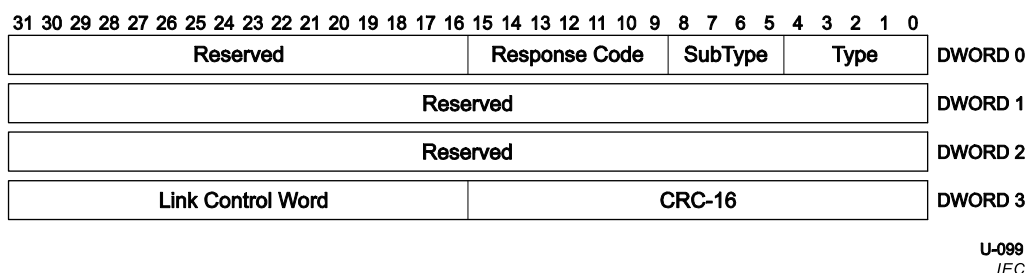


Figure 8-10 – Port Configuration Response LMP

**Table 8-10 – Port Configuration Response LMP Format
(Differences with Port Capability LMP)**

Width (bits)	Offset (DW:bit)	Description						
4	0:5	SubType. This field shall be set to <i>Port Configuration Response</i> .						
7	0:9	Response Code. When operating at Gen 1 speed, this field indicates the settings that were accepted in the Port Configuration LMP that was sent to a device. <table><tr><th><u>Bits</u></th><th><u>Description</u></th></tr><tr><td>0</td><td>If this bit is set to 1, then this device accepted the Link Speed setting.</td></tr><tr><td>6:1</td><td>Reserved.</td></tr></table> When not operating at Gen 1 speed, this this field is Reserved and shall be set to zero.	<u>Bits</u>	<u>Description</u>	0	If this bit is set to 1, then this device accepted the Link Speed setting.	6:1	Reserved.
<u>Bits</u>	<u>Description</u>							
0	If this bit is set to 1, then this device accepted the Link Speed setting.							
6:1	Reserved.							
80	0:16	Reserved.						

If the Response Code indicates that the Link Speed was rejected by the upstream port, the downstream port shall signal an error as described in Section 10.16.2.6.

8.4.8 Precision Time Measurement

PTM enables USB devices to have more precise notion of time by providing a method of precisely characterizing link delays, and the propagation delays through a hub. The PTM capability is discovered by software through the PTM Capability Descriptor described in Section 9.6.2.6.

Precision Time Measurement consists of two separate mechanisms: Link Delay Measurement (LDM) and Hub Delay Measurement (HDM). These mechanisms complement each other to provide highly accurate bus interval boundary timing for devices; however, HDM may be used to improve device bus interval boundary timing accuracy even if LDM timing information is not available.

SuperSpeedPlus hosts and hubs shall support PTM. PTM support is optional normative for all peripheral devices and SuperSpeed only hosts and hubs. Ideally, PTM is supported by all components of a USB topology; however, PTM capable hubs will still improve the overall accuracy of a device's notion of the bus interval boundary timing.

8.4.8.1 PTM Bus Interval Boundary Counters

A bus interval boundary shall be defined as a pair of counters, referred to as the PTM Bus Interval Boundary Counters, that use a format similar to the 27-bit *Isochronous Timestamp* of an ITP:

- *PTM Delta Counter, 13 bits.*
- *PTM Bus Interval Counter, 14 bits.*

The *PTM Clock* has a period of *tlsochTimestampGranularity* units.

The *PTM Delta Counter* shall be incremented by the *PTM Clock* to measure the delay from present time to the previous bus interval boundary. The *PTM Delta Counter* is a modulus 7 500 counter, wrapping on microframe boundaries, i.e. incrementing from 0 to 7 499 (~125 µs), then wrapping back to 0.

The *PTM Bus Interval Counter* shall be incremented when the *PTM Delta Counter* wraps. The *PTM Bus Interval Counter* is a modulus 16k counter, i.e. incrementing from 0 to 16 383, then wrapping back to 0.

The time synchronization mechanism within the device of the *PTM Clock* to the bus interval boundary is implementation-specific.

Hosts shall implement a set of *PTM Bus Interval Boundary Counters*. The host is the source of the bus interval boundary for a *PTM Domain*.

Hubs are not required to implement *PTM Bus Interval Boundary Counters*.

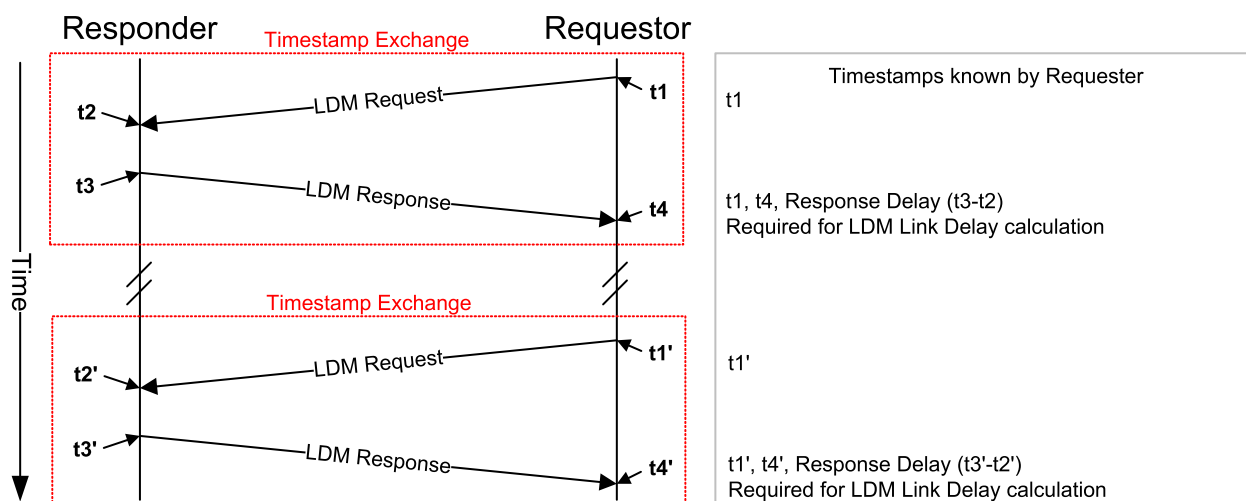
PTM capable devices shall implement *PTM Bus Interval Boundary Counters*.

8.4.8.2 LDM Protocol

The LDM protocol is executed with a series of *Exchanges* between a Requester and a Responder, and ITPs transmitted by Responders to measure the *LDM Link Delay* illustrates the *LDM Timestamp Exchanges*.

The following rules apply to LDM Requesters and Responders:

- A LDM Requester is an upstream facing port.
- A LDM Responder is a downstream facing port.
- A LDM Requester and a LDM Responder are paired on a USB link.

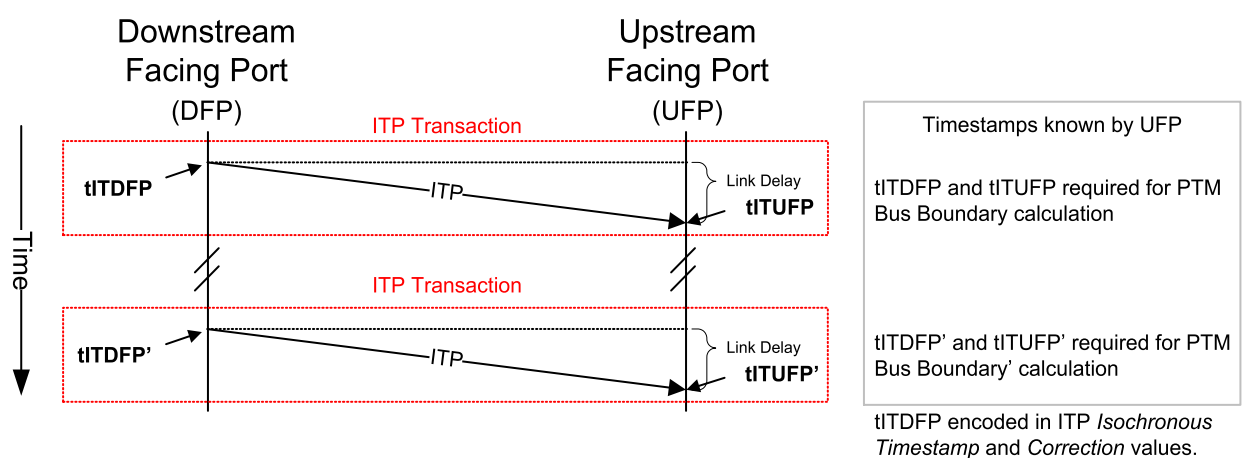


IEC

Figure 8-11 – Link Delay Measurement Protocol

The following rules apply to LDM Requests and Responses, and LDM Requesters and Responders:

- When a LDM Requester transmits a LDM Request LMP, it uses the value of its PTM Local Time Source as the t1 timestamp. When the LDM Requester receives a LDM Response LMP it uses the value of its PTM Local Time Source as the t4 timestamp.
- When a LDM Responder receives a LDM Request LMP it uses the value of its PTM Local Time Source as the t2 timestamp, and when it transmits a LDM Response LMP it uses the value of its PTM Local Time Source as the t3 timestamp.
- Each LDM Exchange defines a set of timestamps which the LDM Requester may use to calculate the LDM Link Delay.
- If an LDM Message is received by a port that does not support PTM, then the Message shall be treated as an unsupported LMP by the port and silently dropped. Note that the port shall acknowledge the packet and return credit for the same as per the requirement specified in Section 7.2.4.1.
- LDM timestamps shall reference the first framing symbol of a received or transmitted LDM LMP.



IEC

Figure 8-12 – PTM ITP Protocol

The following rules apply to ITPs and PTM capable hosts, hubs, and devices:

- The t1TDFP timestamp represents the time that a PTM downstream facing port transmits an ITP.
- The t1TUFP timestamp represents the time that a PTM upstream facing port receives an ITP.

8.4.8.2.1 LDM Timestamp Exchange

A Timestamp Exchange consists of a LDM Request LMP being generated by a LDM Requester and the LDM Responder returning a LDM Response LMP.

As illustrated in Figure 8-11, the timestamps t1, t2, t3 and t4 are created by the Requester and the Responder.

At time t4 in a Timestamp Exchange, a LDM Requester has all the information that it needs to compute the LDM Link Delay. In the example of Figure 8-11, at time t4, the Requester has recorded timestamps for t1 and t4, and received *Response Delay* from the Responder. Refer to Section 8.4.8.3 for how these timestamps are used to compute the LDM Link Delay.

8.4.8.2.2 PTM ITP Transfer

Once a LDM Timestamp Exchange is complete, the Link Delay may be computed and PTM may be applied by a Requester to received ITPs. A PTM ITP Transfer consists of an ITP being generated by a LDM Responder and being received by a LDM Requester.

As illustrated in Figure 8-19, the timestamps tTDFP and tTUFp are recorded by the Requester and Responder.

At time tTUFp in a PTM ITP Transfer a LDM Requester has all the information that it needs to compute the bus interval boundary. In the example of Figure 8-12, at time tTUFp the Requester has recorded timestamp for tTUFp and received timestamp tTDFP (encoded in the Isochronous Timestamp and Correction fields of an ITP) from the Responder. Refer to Section 8.4.8.4 for how these timestamps are used to compute the bus interval boundary.

8.4.8.3 LDM State Machines

The LDM state machines utilize the following notation:

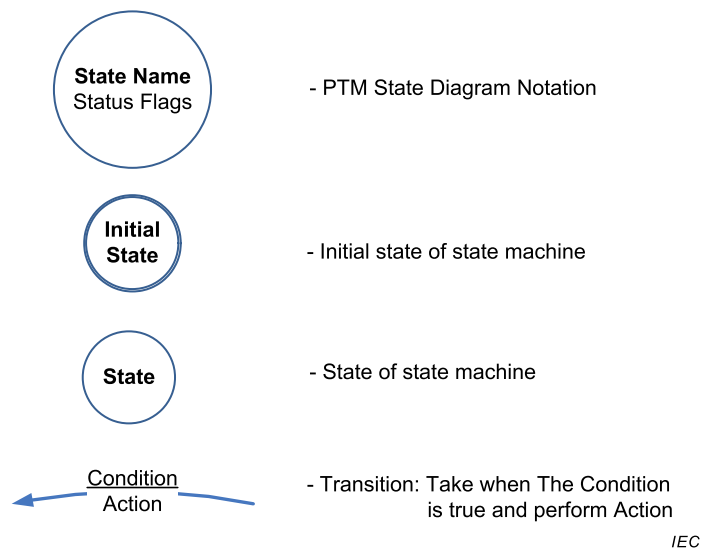


Figure 8-13 – LDM State Machine Notation

Where the State Name is an informative name defined for the state, the Status Flags values are:

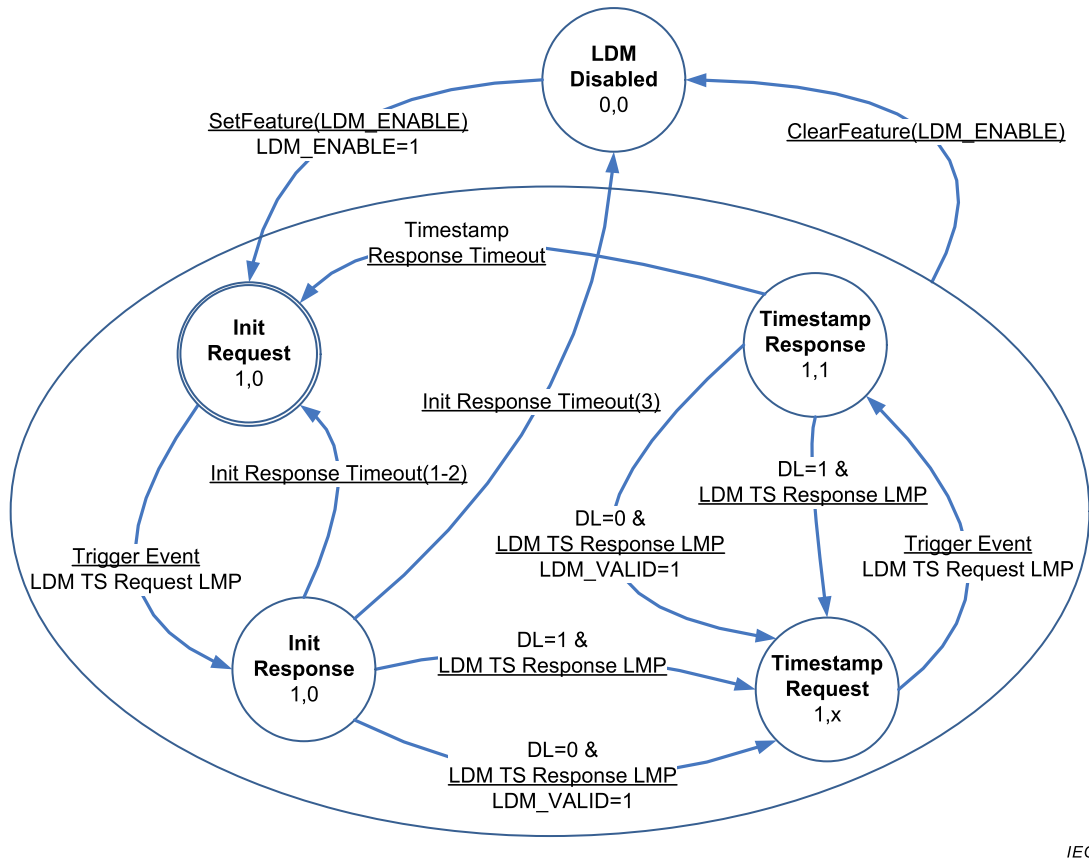
LDM Enabled and *LDM Valid*, respectively, e.g. the Status Flags value 0,0 is interpreted as *LDM Enabled* = false (0) and *LDM Valid* = false (0).

NOTE Transitions associated with a large bubble may occur from any state defined within the bubble as long as the Conditions match.

NOTE The figures in this section are provided to illustrate state transition conditions and actions; however, refer to the textual descriptions of the respective states in the sections below for their explicit definition.

8.4.8.3.1 Requester Operation

This section describes the Timestamp Exchange operations that a Requester (Upstream Facing Port) performs to participate in the LDM protocol.



IEC

Figure 8-14 – LDM Requester State Machine

The LDM Requester State Machine shall maintain the following local variable: *Init Response Timeout Counter*.

The LDM Requester State Machine shall maintain the following local timer: *Response Timer*. All local timers are set to 0 when they are “started”.

8.4.8.3.1.1 Init Request

This is the initial state of the Requester after power-up, Hot Reset, or Warm Reset.

Upon entering this state, the Requester shall set the *LDM Enabled* flag to 1.

The *Init Response Timeout Counter* shall be initialized to 0 at power up, or if the **Init Request** state is entered from the **LDM Disabled** or **Timestamp Response** states. If the **Init Request** state is entered from the **Init Response** state, then the *Init Response Timeout Counter* shall not be changed.

In this state the LDM Protocol is enabled (*LDM Enabled* = 1); the local copy of the bus interval boundary is Invalid (*LDM Valid* = 0) and the Requester waits for a Trigger Event.

Trigger Event – When a Trigger Event occurs, the Requester shall transmit a LDM TS Request LMP to the Responder, save timestamp t1 from the Local Time Source in the LDM Context to record the time that the LDM Request was transmitted, and transition to the **Init Response** state.

A typical Trigger Event for the Requester **Init Request** state would be the transition of the device to the Address state.

8.4.8.3.1.2 Init Response

Upon entering this state, the Requester shall increment the *Init Response Timeout Counter*, start the *Response Timer* and wait for a TS Response LMP or a timeout.

LDM TS Response LMP & DL=0 – If a LDM TS Response LMP and LCW Delayed (DL) = 0 is received, the Requester shall save timestamp t4 from the Local Time Source in the LDM Context to record the time that the LDM Response was received, then calculate the LDM Link Delay (refer to Section 8.4.8.4) and set the *LDM Valid* flag to 1, and transition to the **Timestamp Request** state.

LDM TS Response LMP & DL=1 – If a LDM TS Response LMP and LCW Delayed (DL) = 1 is received, the Requester shall consider the *Response Delay* field of the LDM TS Response LMP invalid, invalidate the LDM Context t1, t2, and t3 timestamps, immediately generate a *Trigger Event* to initiate another Exchange, and transition to the **Timestamp Request** state. Refer to Sections 7.2.4.1.2 and 7.2.4.1.12 for the conditions that may set the Delayed bit.

Init Response Timeout(1-2) – If an Init Response Timeout (tLDMRequestTimeout) occurs and the *Init Response Timeout Counter* is less than 3, then the Requester shall increment the *Init Response Timeout Counter* and transition to the **Init Request** state.

Init Response Timeout(3) – If an Init Response Timeout occurs and the *Init Response Timeout Counter* is equal to 3, then the Requester shall transition to the **LDM Disabled** state

8.4.8.3.1.3 Timestamp Request

Upon entering this state, the Requester shall wait for a Trigger Event.

Trigger Event – When a Trigger Event occurs, the Requester shall transmit a LDM Timestamp (TS) Request LMP to the Responder, save timestamp t1 from the Local Time Source in the LDM Context to record the time that the LDM Request was transmitted, and transition to the **Timestamp Response** state.

A typical Trigger Event for the Requester Timestamp Request state would be to first transition to the Timestamp Request state, or execute an (averaging) algorithm to improve the accuracy of the Link Delay.

NOTE Timestamp t1 shall be adjusted for the TS Delay. Refer to Section 8.4.8.6 for more information.

8.4.8.3.1.4 Timestamp Response

Upon entering this state, the Requester shall start the *Response Timer* and wait for a LDM TS Response LMP or a timeout.

LDM TS Response LMP & DL=0 – If a LDM TS Response LMP is received and the LCW Delayed (DL) flag is zero, the Requester shall save timestamp t4 from the Local Time Source in the LDM Context to record the time that the LDM Response was received, then calculate the LDM Link Delay (refer to Section 8.4.8.4), set the *LDM Valid* flag to 1, and transition to the **Timestamp Request** state.

LDM TS Response LMP & DL=1 – If a LDM TS Response LMP is received and the LCW Delayed (DL) flag is one, the Requester shall consider the *Response Delay* field of the LDM TS Response LMP invalid, invalidate the LDM Context t1, t2, t3 timestamps, immediately generate a *Trigger Event* to initiate another Timestamp Exchange, and transition to the **Timestamp Request** state. Refer to Sections 7.2.4.1.2 and 7.2.4.1.12 for the conditions that may set the Delayed bit.

Response Timeout – If a Response Timeout occurs the Requester shall transition to the **Init Request** state, where the LDM state machine will attempt to retry the Timestamp Exchange with the Responder.

NOTE Timestamp t_4 shall be adjusted for the TS Delay. Refer to Section 8.4.8.6 for more information.

8.4.8.3.1.5 LDM Disabled

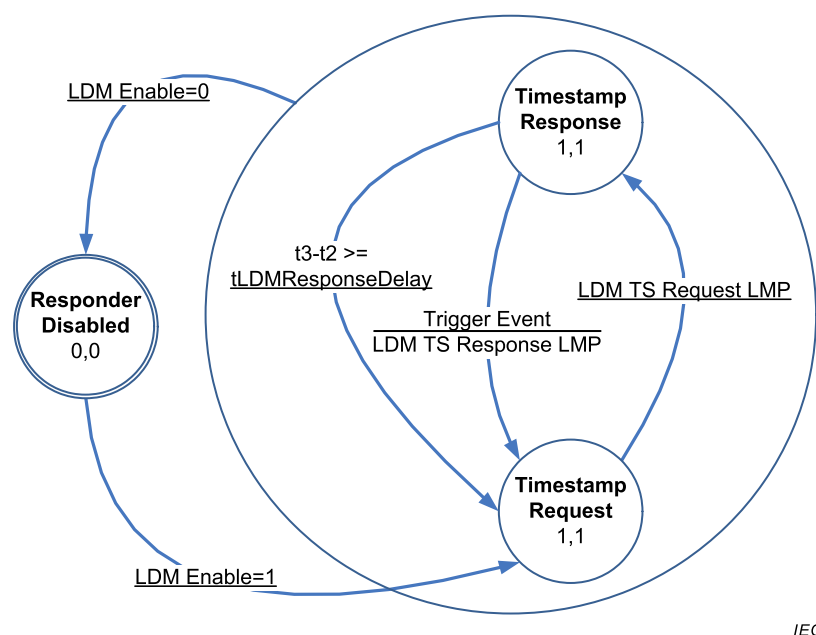
Upon entering this state, the Requester shall clear the LDM_ENABLE flag and terminate all LDM protocol activity.

CLEAR_FEATURE(LDM_ENABLE) – When this request is received by the device it shall transition from any other LDM state to the **LDM Disabled** state.

SET_FEATURE(LDM_ENABLE) – If this request is received by the device it shall transition to the **Init Request** state.

8.4.8.3.2 Responder Operation

This section describes the operations that a Responder (i.e. the Downstream Facing Port of a hub or host controller) performs to participate in the LDM protocol.



IEC

Figure 8-15 – LDM Responder State Machine

The LDM Responder State Machine shall maintain the following local variable: *Responder Response Delay Overflow*.

8.4.8.3.2.1 Responder Disabled

This is the initial state of the Responder after power-up, Hot Reset, or Warm Reset.

Upon entering this state the Responder shall terminate all LDM protocol activity.

LDM Enabled=0 – If *LDM Enabled* equals 0, then the Responder shall transition from any other LDM state to the **Responder Disabled** state.

LDM Enabled=1 – If *LDM Enabled* equal 1, then the Responder shall transition to the **Timestamp Request** state.

8.4.8.3.2.2 Timestamp Request

Upon entering this state, the Responder shall wait for a LDM TS Request LMP.

LDM TS Request LMP – If a LDM TS Request LMP is received, the Responder shall capture timestamp t2 from the PTM Local Time Source to record the time that the LDM TS Request was received and transition to the **Timestamp Response** state.

NOTE Timestamp t2 shall be adjusted for the TS Delay. Refer to Section 8.4.8.6 for more information.

8.4.8.3.2.3 Timestamp Response

Upon entering this state the Responder shall wait for a Trigger Event.

Trigger Event – When a Trigger Event occurs, the Responder shall capture timestamp t3 from the PTM Local Time Source to record the time that the LDM Response was transmitted. If the value t3 – t2 is less than tLDMResponseDelay, the Responder shall form a LDM TS Response LMP by initializing the *Response Delay* field with the value t3 – t2, transmit the LDM TS Response LMP to the Requester, and transition to the **Timestamp Request** state. If the value t3 – t2 is equal to or greater than tLDMResponseDelay, then the Responder shall transition to the **Timestamp Request** state.

A typical Trigger Event for the Responder **Timestamp Response** state would be the next opportunity to schedule a LDM TS Response LMP on its downstream link after a LDM TS Request LMP has been received.

NOTE The Timestamp t3 shall be adjusted for the TS Delay. Refer to Section 8.4.8.6 for more information.

NOTE A Responder shall set the LCW *Delayed* (DL) flag and re-calculate CRC-5 if a LDM TS Response LMP is delayed if its adjusted *Response Delay* value exceeds tLDMRequestTimeout. Refer to Section 8.4.8.6 for more information.

8.4.8.4 LDM Link Delay

LDM defines the set of PTM capabilities which support the measurement of the LDM Link Delay.

LDM Link Delay identifies the delay between the first symbol of a packet being transmitted on a Responder's downstream facing port and the first symbol of the same packet being received on the Requester's upstream facing port. In a hub or device the LDM Link Delay is derived from Timestamp Exchanges with its upstream Responder.

A Requester may execute multiple Timestamp Exchanges to refine its LDM Link Delay value through averaging.

NOTE Field, or subfield names that reference a received ITP shall use the subscript _(RxITP) and field, or subfield names that reference a transmitted ITP shall use the subscript _(TxITP).

8.4.8.4.1 Calculation

The LDM Link Delay is calculated by the Requester when a Timestamp Exchange completes. The LDM Link Delay is the measured link delay adjusted to ensure compatibility with non-PTM aware software.

The *TP Transmission Time* is the time it takes to transmit a TP including framing and encoding at *UI nominal*.

Where *UI nominal* is defined as:

$$UI\ nominal = \frac{(UI\ max) + (UI\ min)}{2}$$

Refer to Section 6.7 for the definition of *UI min* and *UI max*.

The *TP Transmission Time* for a link operating at Gen 1 speed is $200 \text{ UI} * \text{UI nominal}$ (i.e. 40 ns @ 5 Gbps).

It takes either 164 or 168 *UI*, depending on block alignment, to transfer a TP over a link operating at Gen 2 speed. The statistical average is 165 *UI*. Therefore, the *TP Transmission Time* for a link operating at Gen 2 speed is $165 \text{ UI} * \text{UI nominal}$ (i.e. 16.5 ns @ 10 Gbps).

If *LDM Valid* is zero, then the *LDM Link Delay* shall be calculated using the following formula:

$$\text{LDM Link Delay} = \text{TP Transmission Time} - t_{\text{TPTransmissionDelay}}$$

If *LDM Valid* is one, then the received LDM TS Response LMP of a Timestamp Exchange provides the Requester with the *Response Delay* field which defines the delay between when the LDM Request was received and the LDM Response was transmitted by the Responder ((t3 – t2).

When the LDM TS Response LMP of a Timestamp Exchange is received, a Requester has accumulated the timestamps t1 and t4 that can be combined with the *Response Delay* (t3 – t2 timestamp) received from the Responder to calculate the value of LDM Link Delay using the following formula:

$$\text{LDM Link Delay} = \frac{(t4 - t1) - (\text{Response Delay})}{2} + \text{TP Transmission Time} - t_{\text{TPTransmissionDelay}}$$

The values t1, t4, and *Response Delay* indicate the timestamps captured during the LDM Exchanges as illustrated in Figure 8-11. The default TP transmission time (*tTPTransmissionDelay*) is corrected by the actual *TP Transmission Time* of the link. After the LDM Link Delay calculation is complete, the values of timestamps t1 and t4 in the Requester LDM Context will be overwritten with the values of the next Timestamp Exchange.

8.4.8.5 PTM Bus Interval Boundary Device Calculation

The bus interval boundary is calculated by a device when an ITP is received.

An ITP provides a device with three values:

- The *Bus Interval Counter*_(RxITP) subfield contains the current Frame Number.
- The *Delta*_(RxITP) subfield contains the delay from the start of the currently received ITP to the previous bus interval boundary.
- The *Correction*_(RxITP) field contains any negative delay that the ITP accumulated as it passed through hubs.

If *Delta*_(RxITP) is greater than or equal to 7500, the device shall ignore the ITP.

If *Delta*_(RxITP) is less than 7500, the device shall apply *Delta*_(RxITP) and *Correction*_(RxITP) values and the *LDM Link Delay* (determined from preceding TS Exchanges) to set the value of the *PTM Delta Counter* at the time an ITP is received (tITUFP) using the following formula:

$$\text{PTM Delta Counter}_{(t\text{ITUFP})} =$$

$$\text{MODULUS}(\text{ISOCH_DELAY} + \text{LDM Link Delay} + \text{Delta}_{(RxITP)} - \text{Correction}_{(RxITP)}, 7500)$$

Where $\text{MODULUS}(\text{number}, \text{divisor})$ returns the integer remainder after *number* is divided by *divisor* and ISOCH_DELAY is the value written to the device by a SET_ISOCH_DELAY request.

At the same time, a device shall use the values of *Bus Interval Counter*_(RxITP) and *Delta*_(RxITP) subfields received in the ITP to set the value of the *PTM Bus Interval Counter* at the time an ITP is received (tITUFP) using the following formula:

$$PTM\ Bus\ Interval\ Counter_{(tITUFP)} = \\ Bus\ Interval\ Counter_{(RxITP)} + \text{ROUNDDOWN}((LDM\ Link\ Delay + \Delta_{(RxITP)}) / 7500))$$

Where **ROUNDDOWN** (*n*) rounds *n* down, towards zero, to the nearest integer value.

This combination of setting the PTM Delta Counter and the PTM Bus Interval Counter defines the bus interval boundary time in the device.

The time synchronization mechanism within the device itself (e.g. to the PTM Local Time Source) is implementation-specific.

Figure 8-12 illustrates the tITUFP and tITDFP timing points of an ITP transaction.

8.4.8.6 PTM Bus Interval Boundary Host Calculation

A host shall maintain a *PTM Delta Counter* and a *PTM Bus Interval Counter*. The host shall transmit downstream ITPs using these current values for the ITP Isochronous Timestamp *Bus Interval Counter*_(TxITP) and *Delay*_(TxITP) fields, and set the *Correction*_(TxITP) field to zero.

8.4.8.7 PTM Hub ITP Regeneration

A hub shall maintain an *ITP Delay Counter* that is incremented by the PTM Clock.

If an ITP is received by a PTM capable hub and the Delayed (DL) bit is not set, then the hub shall apply the following rules:

- Set the *ITP Delay Counter* to zero.
A PTM capable hub shall apply the following rules independently for each downstream port in U0:
- When an ITP is received, then the hub shall queue an ITP for transmission on this downstream facing port.
- When transmitting an ITP, a hub shall:
 - 1) Copy the value of *Bus Interval Boundary*_(RxITP) to the *Bus Interval Boundary*_(TxITP) subfield of the *Isochronous Timestamp* field in the downstream ITP.
 - 2) Calculate the value of *Delta*_(TxITP) subfield using the following method:
Determine the *Delta* value at time the ITP shall be transmitted (tITPDFP) using the formula:

$$\Delta_{(tITPDFP)} = LDM\ Link\ Delay + \Delta_{(RxITP)} + (ITP\ Delay\ Counter - wHubDelay) - Correction_{(RxITP)}$$
 Where *Delta*_(tITPDFP) is the *Delta* value at time tITPDFP.
 If *Delta*_(tITPDFP) is greater than or equal to zero and less than 7500, the hub shall set *Delta*_(TxITP) equal to *Delta*_(tITPDFP) and the *Correction*_(TxITP) value to zero.
 If *Delta*_(tITPDFP) is greater than or equal to 7500, the hub shall set *Delta*_(TxITP) equal to 7500.
 If *Delta*_(tITPDFP) is negative the hub shall set *Delta*_(TxITP) to zero and calculate the *Correction*_(TxITP) value using the following formula:

$$Correction_{(TxITP)} = -\Delta_{(tITPDFP)}$$
 - 3) Re-calculate the CRC-16 for the modified ITP.

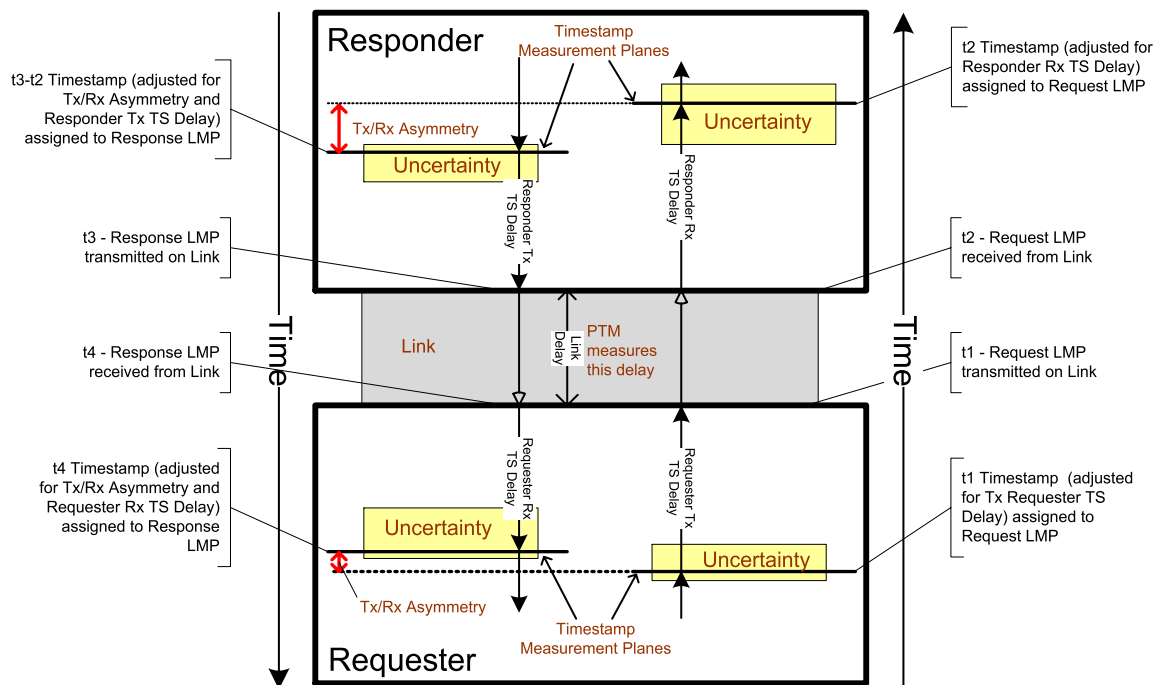
If an ITP is received by a PTM capable hub and the Delayed (DL) bit is set, then the hub shall apply the following rules:

- Forward the received ITP without modification.

The Isochronous Timestamp values used in the ITP being transmitted shall be the values present at the time of transmission; e.g., if other packets are queued for transmission at the time the ITP is queued, the values used shall not be the values present at the time of queuing, but adjusted for the actual time of transmission (tITPDFP). Refer to Section 8.4.8.6 for more information.

8.4.8.8 Performance

Figure 8-16 illustrates the various components of a LDM Exchange path that contribute to the overall performance of the LDM mechanism. The Requester to Responder and Responder to Requester paths apply to LDM TS Request and TS Response LMPs, respectively. The Requester Rx path applies to ITPs received by an upstream facing port and the Responder Tx path applies to ITPs transmitted by a downstream facing port.



IEC

Figure 8-16 – PTM Path Performance Contributors

The following requirements should be met to achieve optimal propagation delay measurement:

- The Link Delay between Requester and Responder should be symmetric.
- Timestamp values are assigned to ITPs and TS LMPs to capture when the packet was actually received or transmitted on the Link. LDM timestamp values are also captured in ITPs (*Bus Interval Counter/Delta fields*) and TS Response LMPs (*Response Delay*) when they are transmitted.
- An implementation dependent *TS Delay* occurs between assigning Timestamp values to an ITP, or LDM LMP, and the packet actually being received or transmitted on the Link. Each Timestamp value should be adjusted for the respective TS Delay so that the Timestamp captured for a LDM LMP or ITP approximates the actual time that the last symbol of the respective packet crosses the boundary between a Requester or Responder and the Link.
- A port may contain asymmetric delays in its timestamp mechanism or protocol path (e.g. Rx/Tx Asymmetry). If these asymmetries are not negligible:

- 1) The Responder shall adjust the TS Response LMP Response Delay ($t_3 - t_2$ Timestamp) value appropriately for its Rx/Tx Asymmetry, such that its t_2 and t_3 Timestamps appear to have been captured at equal TS Delays from the Link boundary.
 - 2) The Requester shall account for its Rx/Tx Asymmetry when computing the t_4 Timestamp.
- The Link Delay between Requester and Responder should be constant over the time interval between LDM Request and Response LMPs.
 - The worst case delay fluctuation (Uncertainty) of timestamps shall be bounded by $t_{\text{PropagationDelayJitterLimit}}$.
 - Delay fluctuation (Uncertainty) of timestamps due to link components and due to the protocol stack within clocks should be reduced by two techniques:
 - 1) The Timestamp Measurement Planes used in PTM should be generated as close to the physical Link boundary as practical for a given clock implementation, i.e. minimize TS Delay.
 - 2) Remaining delay fluctuation (Uncertainty) introduced by the protocol stack and by link components can be reduced by averaging Link Delay values over multiple Timestamp Exchanges. The averaging algorithms are outside the scope of this specification.
 - The inherent stability and precision of a clock's oscillator must be within the clock accuracy requirements defined for the Unit Interval in Table 6-17.

IMPLEMENTATION NOTE

LDM Timestamp Capture Mechanisms

LDM uses services from both the Data Link and Transaction Layers. LDM accuracy requires that time measurements be taken as close to the Physical Layer as possible. Conversely, the messaging protocol itself properly belongs to the Transaction Layer. The LDM message protocol applies to a single Link, where the Upstream Facing Port is the Requestor and the Downstream Facing Port is the Responder.

For most implementations, the logic within the Transaction Layer and Data Link Layers is essentially non-deterministic. Implementation details and current conditions have considerable impact on exactly when a particular packet may encounter any particular processing step. This makes it effectively impossible to capture any timestamp that accurately records the time of a particular physical event within these layers.

Ideally time measurements should be taken with the symbol level accuracy as close to the D+/D- outputs of the Transmitter Differential Driver block (Figure 6-2), or the D+/D- inputs of the Differential Receiver and Equalization block (Figure 6-3). Typically this will require an implementation specific adjustment to compensate for the inability to directly measure the time at the actual pins, because the time will typically be measured (i.e. the Timestamp captured) at some internal point in the Rx or Tx path. The designer should approximate these delays and accommodate them by the timestamp values that they record and the time values that they generate (e.g. the Response Delay value in a LDM TS Response) appropriately. The accuracy and consistency of this measurement are not bounded by this specification, but it is strongly recommended that the highest practical level of accuracy and consistency be achieved.

8.4.8.9 LDM Rules

A Responder shall respond to each LDM Request LMP with a LDM Response LMP according to the following rules:

- A Responder shall not send a LDM Response without first receiving a LDM Request LMP.
- A Responder shall capture the PTM Local Clock Source timestamps (t_2 and t_3) when transmitting LDM Response and when receiving LDM Request LMPs.

- A Responder shall issue LDM Response LMP when it possesses the timing values required to populate the LDM Response LMP: timestamps ($t_2 - t_3$ in Figure 8-11).
- A Requester shall capture t_1 timestamps upon transmitting the last symbol of a LDM Request.
- A Responder shall capture t_2 timestamps upon receiving the last symbol of a LDM Request.
- A Responder shall capture t_3 timestamps upon transmitting the last symbol of a LDM Response.
- A Requester shall capture t_4 timestamps upon receiving the last symbol of a LDM Response.

Note that these rules assume that the Tx and Rx TS Delays in Figure 8-16 are zero, i.e. the Timestamp Measurement Planes and the TS LMP link boundary transmit and receive times are identical. Refer to Section 8.4.8.8 for how to adjust for actual Timestamp Delays.

8.4.8.10 LDM and Hubs

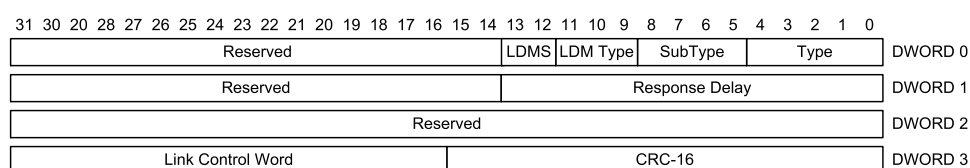
A hub is both a Requester and a Responder, and acts as intermediary between its Upstream and Downstream Facing Ports. As a Requester, a hub utilizes the PTM LDM mechanism to issue LDM Requests on its Upstream Facing Port to identify the LDM Link Delay between itself and its upstream Responder. A hub utilizes the PTMHDM mechanism to update the *Isochronous Timestamps* of ITPs that it forwards downstream.

A hub implementation has *LDM Enabled* and *LDM Valid* flags. Their states are determined by the Hub's Requester State Machine. The values of the *LDM Enabled* or *LDM Valid* flags in the hub's Responder State Machines track the Requester State Machine values; e.g., if *LDM Enabled* or *LDM Valid* in the Requester State Machine transition to 0, then all of the hub's Responder State Machine shall transition to the **Responder Disabled** state. When both *LDM Enabled* and *LDM Valid* in the Requester State Machine transition to 1, then all of the hub's Responder State Machine shall transition to the **Init Request** state. A hub does not have *LDM Enabled* or *LDM Valid* flags per downstream facing port Responder State Machine.

The USB topology is enumerated from the Root Hub port down; i.e., a hub must be in the Configured state before its downstream ports are active. The PTM timing parameters are selected so that, for typical enumeration sequences, the LDM Link Delay is established in a hub before it transitions to the Configured state. This means that the hub's Responder State Machines should be in the **Init Request** state as soon as its downstream ports are operational and that no software intervention is required to enable LDM. If the Requester of a device attached to a Downstream Facing Port is unable to establish the LDM Link Delay in timely manner, it may transition to the **LDM Disabled** state and require software to manually start LDM Exchanges. An alternative is for PTM aware enumeration software to verify that PTM capable hubs have established the LDM Link Delay before configuring them.

8.4.8.11 Link Delay Measurement (LDM) LMP

LDM LMPs shall be used in a LDM Timestamp Exchange to measure the link delay on an upstream facing port.



U-???

IEC

Figure 8-17 – LDM LMP

Table 8-11 – LDM LMP

Width (bits)	Offset (DW:bit)	Description
4	0:5	SubType. This field shall be set to <i>Precision Time Measurement</i> .
3	0:9	LDM Type. This field identifies the type of LDM LMP. This field shall be set to <i>TS Request</i> if the LDM Request LMP initiates an LDM Timestamp Exchange. Value Description 0 TS Request – Initiates a LDM Timestamp Exchange. 1 TS Response – Completes a Timestamp Exchange. Carries timing information. 2-3 Reserved
2	0:12	LDM Sequence Number (LDMS). This field identifies the sequence number of a LDM Exchange. The value of this field shall be set by the Requester in a LDM Request, and returned by Responder in the associated LDM Response. The Requester may use this value to explicitly verify that a Response LMP is associated with the Request LMP of an Exchange. This value shall be incremented on each Exchange initiated by the Requester.
14	1:0	Response Delay. In a LDM Response LMP this field shall report in <i>tlsochTimestampGranularity</i> units the delay between receiving the last symbol of a LDM Request and transmitting the last symbol of the associated LDM Response.

8.5 Transaction Packet (TP)

Transaction Packets (TPs) traverse the direct path between the host and a device. TPs are used to control data flow and manage the end-to-end connection. The value in the **Type** field shall be set to *Transaction Packet*. The **Route String** field is used by hubs to route a packet that appears on its upstream port to the correct downstream port. The route string is set to zero for a TP sent by a device. When the host sends a TP, the **Device Address** field contains the address of the intended recipient. When a device sends a TP to the host then it sets the **Device Address** field to its own address. This field is used by the host to identify the source of the TP. The **SubType** field in a TP is used by the recipient to determine the format and usage of the TP.

Table 8-12 – Transaction Packet Subtype Field

Width (bits)	Offset (DW:bit)	Description																						
4	1:0	Subtype. The subtype field is used to identify a specific type of TP. <table><tr><th><u>Value</u></th><th><u>Type of TP</u></th></tr><tr><td>0000b</td><td>Reserved</td></tr><tr><td>0001b</td><td>ACK</td></tr><tr><td>0010b</td><td>NRDY</td></tr><tr><td>0011b</td><td>ERDY</td></tr><tr><td>0100b</td><td>STATUS</td></tr><tr><td>0101b</td><td>STALL</td></tr><tr><td>0110b</td><td>DEV_NOTIFICATION</td></tr><tr><td>0111b</td><td>PING</td></tr><tr><td>1000b</td><td>PING_RESPONSE</td></tr><tr><td>1001b – 1111b</td><td>Reserved</td></tr></table>	<u>Value</u>	<u>Type of TP</u>	0000b	Reserved	0001b	ACK	0010b	NRDY	0011b	ERDY	0100b	STATUS	0101b	STALL	0110b	DEV_NOTIFICATION	0111b	PING	1000b	PING_RESPONSE	1001b – 1111b	Reserved
<u>Value</u>	<u>Type of TP</u>																							
0000b	Reserved																							
0001b	ACK																							
0010b	NRDY																							
0011b	ERDY																							
0100b	STATUS																							
0101b	STALL																							
0110b	DEV_NOTIFICATION																							
0111b	PING																							
1000b	PING_RESPONSE																							
1001b – 1111b	Reserved																							

8.5.1 Acknowledgement (ACK) Transaction Packet

This TP is used for two purposes:

- For IN endpoints, this TP is sent by the host to request data from a device as well as to acknowledge the previously received data packet.
- For OUT endpoints, this TP is sent by a device to acknowledge receipt of the previous data packet sent by the host, as well as to inform the host of the number of data packet buffers it has available after receipt of this packet.

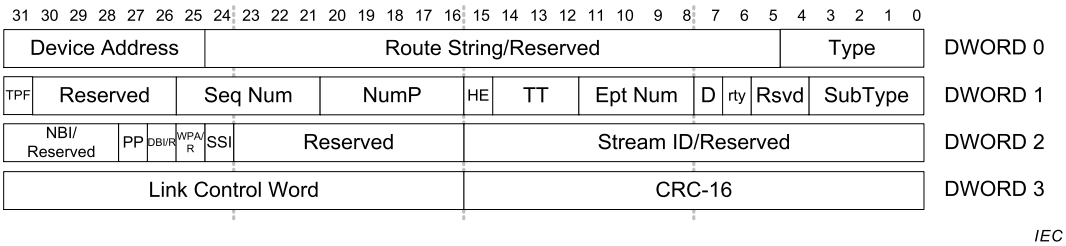


Figure 8-18 – ACK Transaction Packet

Table 8-13 – ACK TP Format

Width (bits)	Offset (DW:bit)	Description																		
20	0:5	Route String/Reserved. This field is only used by hubs. In conjunction with the hub depth, it is used to route a packet to the correct downstream port. Refer to Section 8.9 for details. When sent by a device, this field is Reserved.																		
7	0:25	Device Address. This field specifies the device, via its address, that is the recipient or the source of the TP. Refer to Section 8.8.																		
4	1:0	SubType. This field shall be set to ACK for an ACK TP.																		
2	1:4	Reserved (Rsvd).																		
1	1:6	Retry Data Packet (rty). This field is used to signal that the host or a device did not receive a data packet or received a corrupted data packet and requests the transmitter to resend one or more data packets starting at the specified sequence number.																		
1	1:7	Direction (D). This field defines the direction of an endpoint within the device that is the source or recipient of this TP. Refer to Section 8.8. <table><tr><th><u>Value</u></th><th><u>Direction of Data Flow</u></th></tr><tr><td>0b</td><td>Host to Device</td></tr><tr><td>1b</td><td>Device to Host</td></tr></table>	<u>Value</u>	<u>Direction of Data Flow</u>	0b	Host to Device	1b	Device to Host												
<u>Value</u>	<u>Direction of Data Flow</u>																			
0b	Host to Device																			
1b	Device to Host																			
4	1:8	Endpoint Number (Ept Num). This field determines an endpoint within the device that is the source or recipient of this TP. Refer to Section 8.8.																		
3	1:12	Transfer Type (TT). When operating at Gen 1 speed, the value of this field is Reserved and shall be set to zero. When not operating at Gen 1 speed, this field is defined as follows: <table><tr><th><u>Value</u></th><th><u>Meaning</u></th></tr><tr><td>100b</td><td>Control Transfer Type</td></tr><tr><td>101b</td><td>Isochronous Transfer Type</td></tr><tr><td>110b</td><td>Bulk Transfer Type</td></tr><tr><td>111b</td><td>Interrupt Transfer Type</td></tr><tr><td>001b</td><td>Reserved</td></tr><tr><td>010b</td><td>Reserved</td></tr><tr><td>011b</td><td>Reserved</td></tr><tr><td>000b</td><td>Unknown for ACKs and deferred DPs originating from SuperSpeed bus instances. Reserved for all other ACKs and DPs. Refer to chapter 10.</td></tr></table>	<u>Value</u>	<u>Meaning</u>	100b	Control Transfer Type	101b	Isochronous Transfer Type	110b	Bulk Transfer Type	111b	Interrupt Transfer Type	001b	Reserved	010b	Reserved	011b	Reserved	000b	Unknown for ACKs and deferred DPs originating from SuperSpeed bus instances. Reserved for all other ACKs and DPs. Refer to chapter 10.
<u>Value</u>	<u>Meaning</u>																			
100b	Control Transfer Type																			
101b	Isochronous Transfer Type																			
110b	Bulk Transfer Type																			
111b	Interrupt Transfer Type																			
001b	Reserved																			
010b	Reserved																			
011b	Reserved																			
000b	Unknown for ACKs and deferred DPs originating from SuperSpeed bus instances. Reserved for all other ACKs and DPs. Refer to chapter 10.																			
1	1:15	Host Error (HE). This field is only valid when the ACK TP is sent from the host to a device. This bit shall be set if the host was unable to accept a valid data packet due to internal host issues. When the host sets this field, it must also set the Retry Data Packet field for a non-isochronous transaction.																		
5	1:16	Number of Packets (NumP). This field is used to indicate the number of Data Packet buffers that the receiver can accept. The value in this field shall be less than or equal to the maximum burst size supported by the endpoint as determined by the value in the bMaxBurst field in the Endpoint Companion Descriptor (refer to Section 9.6.7).																		
5	1:21	Sequence Number (Seq Num). This field is used to identify the sequence number of the next expected data packet.																		
5	1:26	Reserved.																		
1	1:31	TP Follows (TPF). When operating at Gen 1 speed, this field is reserved. When not operating at Gen 1 speed, this bit shall be set only if the device intends to send a Device Notification TP following this TP.																		

Width (bits)	Offset (DW:bit)	Description
16	2:0	Stream ID/Reserved. If this ACK TP is targeted at a Bulk endpoint that supports Streams (i.e., a Stream pipe), this field contains a Stream ID value between 1 and 65 535. The Stream ID value of 0 is reserved for Stream pipes and the TP shall be considered invalid if a 0 value is received. All other pipe types shall treat this field as reserved. The usage of this field is class dependent. This field shall be set to zero if the Bulk endpoint does not support Streams. Refer to Section 8.12.1.4 for more information on Stream IDs.
8	2:16	Reserved.
1	2:24	Support Smart Isochronous (SSI). This field is only valid for ISO endpoints. For OUT endpoints, the DBI , WPA and NBI fields are valid only if this field is set to one and the lpf field is set to zero. In the case of IN endpoints, the host does not set the lpf field and hence it just has to set the SSI field if the host supports smart isochronous scheduling. It informs the device that this host controller supports advanced isochronous scheduling functionality that can be used by the device to drive its link to lower power states in between the times that the host is polling the ISO endpoint in its service interval. In the case the host is transferring data to an OUT endpoint, it is the responsibility of the host controller that it only sets this field to one when the lpf field is set to zero. Device response when both these fields are set to one is undefined. In the case the host is transferring data from an IN endpoint, then if the device responds with a DP that has the lpf field set to one, then it can ignore the value in this field (and other related fields) and simply wait for the host to PING the device again before the endpoint is serviced
1	2:25	Will Ping Again (WPA/Reserved). This field is only valid for ISO endpoints and is only valid when the SSI field is set to one. If this field is set to one, then the host controller will send a PING TP before it services the endpoint again.
1	2:26	Data in this Bus Interval is done (DBI/Reserved). This field is only valid for ISO endpoints and is only valid when the SSI field is set to one. If this field is set to one, then the host controller is done performing transactions to this endpoint in the current bus interval. Note: WPA has a higher priority than this field. When a host sets the WPA field, the device can safely ignore the value in this field as the host will PING the device before resuming transactions to this endpoint.
1	2:27	Packets Pending (PP). This field can only be set by the Host. If the field is set, then the host is ready to receive another DP from this endpoint/Stream. Where the endpoint is identified by the Endpoint Number and Direction fields, and if this is a Stream endpoint, then the Stream is identified by the Stream ID field. If this field is cleared, then the host is not ready to receive any more DPs for this Endpoint/Stream. If no endpoints on this device have packets pending, then the device can use this information to aggressively power manage its upstream link, e.g., set the link to a lower power U1 or U2 state.
4	2:28	Number of Bus Intervals (NBI/Reserved). This field is only valid for ISO endpoints and is only valid when the SSI field is set to one, the WPA field is set to zero and the DBI field is set to one. The value in this field informs the device the number of bus intervals after which the host controller will perform transactions to the endpoint again. The value in this field indicates to the endpoint that the host controller will service the endpoint in the bus interval with a value equal to (current bus interval + value in NBI field + 1).

8.5.2 Not Ready (NRDY) Transaction Packet

This TP can only be sent by a device for a non-isochronous endpoint. An OUT endpoint sends this TP to the host if it has no packet buffer space available to accept the DP sent by the host. An IN endpoint sends this TP to the host if it cannot return a DP in response to an ACK TP sent by the host.

Only the fields that are different from an ACK TP are described in this section.

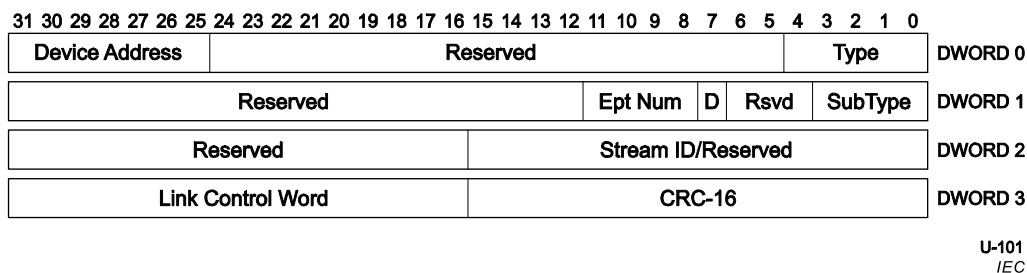


Figure 8-19 – NRDY Transaction Packet

Table 8-14 – NRDY TP Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType . This field shall be set to <i>NRDY</i> .
3	1:4	Reserved (Rsvd) .
20	1:12	Reserved .
5	2:27	Reserved .

8.5.3 Endpoint Ready (ERDY) Transaction Packet

This TP can only be sent by a device for a non-isochronous endpoint. It is used to inform the host that an endpoint is ready to send or receive data packets. Only the fields that are different from an ACK TP are described in this section.

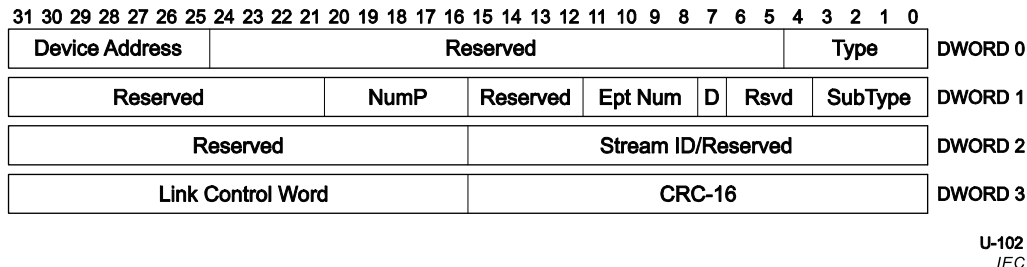


Figure 8-20 – ERDY Transaction Packet

Table 8-15 – ERDY TP Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType . This field shall be set to <i>ERDY</i> .
3	1:4	Reserved (Rsvd) .
4	1:12	Reserved .
5	1:16	Number of Packets (NumP) . For an OUT endpoint, refer to Table 8-13 for the description of this field. For an IN endpoint this field is set by the endpoint to the number of packets it can transmit when the host resumes transactions to it. This field shall not have a value greater than the maximum burst size supported by the endpoint as indicated by the value in the bMaxBurst field in the Endpoint Companion Descriptor. Note that the value reported in this field may be treated by the host as informative only.
11	1:21	Reserved .
5	2:27	Reserved .

8.5.4 STATUS Transaction Packet

This TP can only be sent by the host. It is used to inform a control endpoint that the host has initiated the Status stage of a control transfer. This TP shall only be sent to a control endpoint. Only the fields that are different from an ACK TP are described in this section.

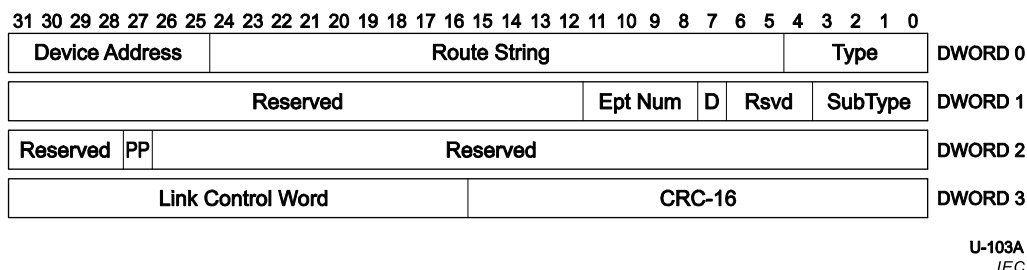


Figure 8-21 – STATUS Transaction Packet

Table 8-16 – STATUS TP Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType . This field shall be set to <i>STATUS</i> .
3	1:4	Reserved (Rsvd) .
52	1:12	Reserved .

8.5.5 STALL Transaction Packet

This TP can only be sent by an endpoint on the device. It is used to inform the host that the endpoint is halted or that a control transfer is invalid. Only the fields that are different from an ACK TP are described in this section.

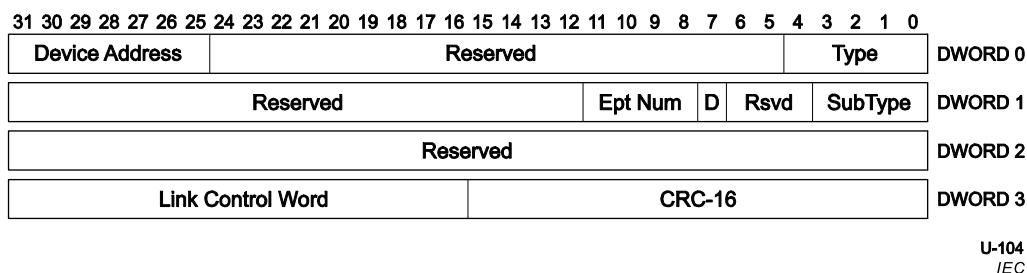


Figure 8-22 – STALL Transaction Packet

Table 8-17 – STALL TP Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType . This field shall be set to <i>STALL</i> .
3	1:4	Reserved (Rsvd) .
52	1:12	Reserved .

8.5.6 Device Notification (DEV_NOTIFICATION) Transaction Packet

This TP can only be sent by a device. It is used by devices to inform the host of an asynchronous change in a device or interface state, e.g., to identify the function within a device that caused the device to perform a remote wake operation. This TP is not sent from a

particular endpoint but from the device in general. Only the fields that are different from an ACK TP are described in this section.

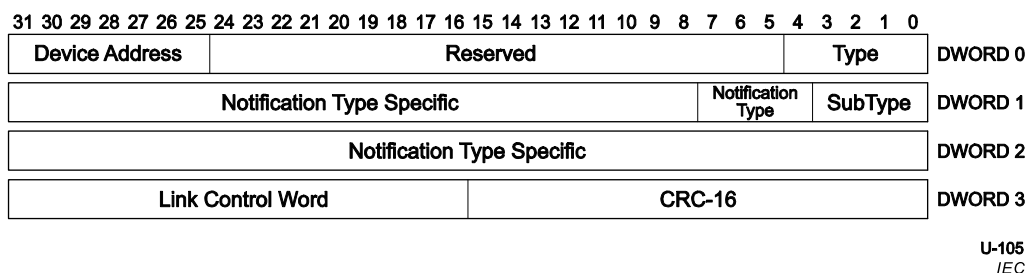


Figure 8-23 – Device Notification Transaction Packet

Table 8-18 – Device Notification TP Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description																
4	1:0	SubType. This field shall be set to <i>DEV_NOTIFICATION</i> .																
4	1:4	Notification Type. The field identifies the type of the device notification. <table><tr><th><u>Value</u></th><th><u>Type of Notification Packet</u></th></tr><tr><td>0000b</td><td>Reserved</td></tr><tr><td>0001b</td><td>FUNCTION_WAKE</td></tr><tr><td>0010b</td><td>LATENCY_TOLERANCE_MESSAGE</td></tr><tr><td>0011b</td><td>BUS_INTERVAL_ADJUSTMENT_MESSAGE</td></tr><tr><td>0100b</td><td>HOST_ROLE_REQUEST²</td></tr><tr><td>0101b</td><td>SUBLINK_SPEED³</td></tr><tr><td>0110b – 1111b</td><td>Reserved</td></tr></table>	<u>Value</u>	<u>Type of Notification Packet</u>	0000b	Reserved	0001b	FUNCTION_WAKE	0010b	LATENCY_TOLERANCE_MESSAGE	0011b	BUS_INTERVAL_ADJUSTMENT_MESSAGE	0100b	HOST_ROLE_REQUEST ²	0101b	SUBLINK_SPEED ³	0110b – 1111b	Reserved
<u>Value</u>	<u>Type of Notification Packet</u>																	
0000b	Reserved																	
0001b	FUNCTION_WAKE																	
0010b	LATENCY_TOLERANCE_MESSAGE																	
0011b	BUS_INTERVAL_ADJUSTMENT_MESSAGE																	
0100b	HOST_ROLE_REQUEST ²																	
0101b	SUBLINK_SPEED ³																	
0110b – 1111b	Reserved																	

8.5.6.1 Function Wake Device Notification

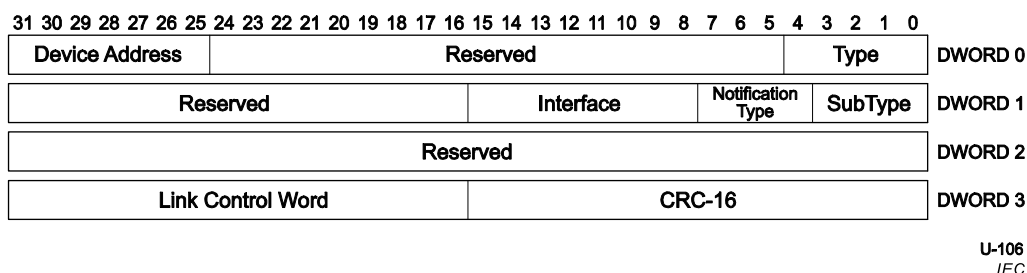


Figure 8-24 – Function Wake Device Notification

² This Notification Type value shall be reserved for OTG use. Refer to Section 5.5 of the USB 3.0 OTG and EH Supplement for the definition of the respective Device Notification TP.

³ This Device Notification is required for devices not operating at Gen 1 Speed. This Device Notification is optional for devices operating at Gen 1 speed.

Table 8-19 – Function Wake Device Notification

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType . This field shall be set to <i>DEV_NOTIFICATION</i> .
4	1:4	Notification Type . FUNCTION_WAKE
8	1:8	Interface . This field identifies the first interface in the function that caused the device to perform a remote wake operation.
48	1:16	Reserved .

8.5.6.2 Latency Tolerance Message (LTM) Device Notification

Latency Tolerance Message Device Notification is an optional normative feature enabling more power efficient platform operation.

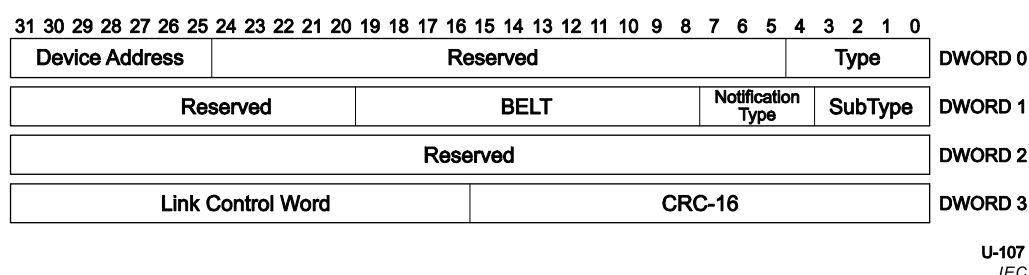


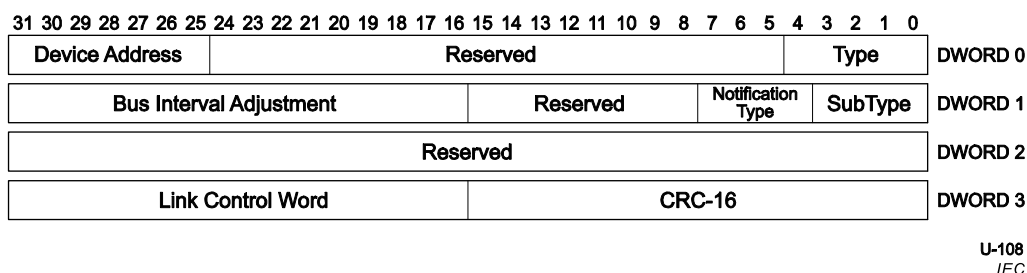
Figure 8-25 – Latency Tolerance Message Device Notification

Table 8-20 – Latency Tolerance Message Device Notification

Width (bits)	Offset (DW:bit)	Description																
4	1:0	SubType . This field shall be set to <i>DEV_NOTIFICATION</i> .																
4	1:4	Notification Type . LATENCY_TOLERANCE_MESSAGE.																
12	1:8	BELT. This field describes the Best Effort Latency Tolerance value, representing the time in nanoseconds that a device can wait for service before experiencing unintended operational side effects. <table><tr><th>Bits</th><th>Description</th></tr><tr><td>9:0</td><td>LatencyValue (ns)</td></tr><tr><td>11:10</td><td>LatencyScale</td></tr></table> <table><tr><th>Value</th><th>Description</th></tr><tr><td>00b</td><td>Reserved</td></tr><tr><td>01b</td><td>LatencyValue is to be multiplied by 1024</td></tr><tr><td>10b</td><td>LatencyValue is to be multiplied by 32,768</td></tr><tr><td>11b</td><td>LatencyValue is to be multiplied by 1,048,576</td></tr></table>	Bits	Description	9:0	LatencyValue (ns)	11:10	LatencyScale	Value	Description	00b	Reserved	01b	LatencyValue is to be multiplied by 1024	10b	LatencyValue is to be multiplied by 32,768	11b	LatencyValue is to be multiplied by 1,048,576
Bits	Description																	
9:0	LatencyValue (ns)																	
11:10	LatencyScale																	
Value	Description																	
00b	Reserved																	
01b	LatencyValue is to be multiplied by 1024																	
10b	LatencyValue is to be multiplied by 32,768																	
11b	LatencyValue is to be multiplied by 1,048,576																	
44	1:20	Reserved .																

8.5.6.3 Bus Interval Adjustment Message Device Notification

The Bus Interval Adjustment Message Device Notification is only defined for devices operating at Gen 1 speed.

**Figure 8-26 – Bus Interval Adjustment Message Device Notification****Table 8-21 – Bus Interval Adjustment Message Device Notification**

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType. This field shall be set to <i>DEV_NOTIFICATION</i> .
4	1:4	Notification Type. BUS_INTERVAL_ADJUSTMENT_MESSAGE.
8	1:8	Reserved.
16	1:16	Bus Interval Adjustment. This field is a two's complement value ranging from -32768 to +32767 expressed in BusIntervalAdjustmentGranularity units.

8.5.6.4 Function Wake Notification

A function may signal that it wants to exit from device suspend (after transitioning the link to U0) or function suspend by sending a Function Wake Device Notification to the host if it is enabled for remote wakeup. Refer to Section 9.2.5 for more details.

8.5.6.5 Latency Tolerance Messaging

Latency Tolerance Messaging is an optional normative USB power management feature that utilizes reported BELT (Best Effort Latency Tolerance) values to enable more power efficient platform operation.

The BELT value is the maximum time (factoring in the service needs of all configured endpoints) for leaving a device without service from the host. Specifically, the BELT value is the time between the host's receipt of an ERDY from a device, and the host's transmission of the response to the ERDY.

Devices indicate whether they are capable of sending LTM TPs using the **LTM Capable** field in the SUPERSPEED_USB Device Capability descriptor in the BOS descriptor (refer to Section 9.6.2). The LTM Enable (refer to Section 9.4.9) feature selector enables (or disables) an LTM capable device to send LTM TPs.

8.5.6.5.1 Optional Normative LTM and BELT Requirements

General Device Requirements

- LTM TPs shall be originated only by peripheral devices.
- LTM TPs apply to all endpoint types except for isochronous endpoints. For interrupt endpoints the BELT value only applies while the endpoint is in a flow control condition.
- Once a BELT value has been sent to the host by a device, all configured endpoints for that device shall expect to be serviced within the specified BELT time.
- A device shall send an LTM TP with a value of tBELTdefault in the **BELT** field in response to any change in state of LTM_Enable within the timing specified by tMinLTMStateChange.

- A device shall ensure that its BELT value is determined frequently enough that it is able to provide reasonable estimate of the device's service latency tolerance prior to its need to change BELT value. In addition, the following conditions shall be met:
 - 1) The maximum number of LTM TPs is bounded by tBeltRepeat.
 - 2) Each LTM TP shall have a different BELT value.
- The system shall default to a BELT of 1 ms for all devices (refer to Table 8-36).
- The minimum value for a BELT is 125 μ s (refer to Table 8-36).

Device Requirements Governing Establishment of BELT Value

- The LTM mechanism shall utilize U1SEL and U2SEL to provide devices with system latency information (see Section 9.4.12 – Set SEL). In this context, the system latency is the time between when a device transmits an ERDY and when it will receive a transaction packet (type is direction-specific) from the host when the deepest allowed link state is U1 or U2. These values are used by the device to properly adjust their BELT value, factoring in their location within the USB link topology.
 - 1) Devices that allow their link to enter U1, but not U2, shall subtract the U1 System Exit Latency (U1SEL) from its total latency tolerance and send the resultant value as the **BELT** field value in an LTM TP.
 - 2) Devices that allow their link to enter U1 and U2, shall subtract U2SEL from its total latency tolerance and send the resulting value as the **BELT** field value in an LTM TP.

8.5.6.6 Bus Interval Adjustment Message

The Bus Interval Adjustment Message may be sent only by devices operating at Gen 1 speed and shall be ignored by hosts that are not operating at Gen 1 speed. Note that this notification will be deprecated in a future release.

This device notification may be sent by a device to request an increase or decrease in the length of the bus interval. This would typically be used by a device trying to synchronize the host's bus interval clock with an external clock. Bus interval adjustment requests are relative to the current bus interval. For example, if a device requests an increase of one BusIntervalAdjustmentGranularity unit and then later requests an increase of two BusIntervalAdjustmentGranularity units the overall increase by the host would be three BusIntervalAdjustmentGranularity units.

The host shall support adjustments through an absolute range of -37 268 to 37 267 BusIntervalAdjustmentGranularity units. A device shall not request adjustments more than once every eight bus intervals. A device shall not send another bus interval adjustment request until it has waited long enough to accurately observe the effect of the previous bus interval adjustment request on the timestamp value in subsequent ITPs. A device shall not make a single BusIntervalAdjustment request for more than $\pm 4\,096$ units. A device may make multiple BusIntervalAdjustment requests over time for a combined total of more than 4096 units. A device shall not request a bus interval adjustment unless the device received an ITP within the past 125 μ s, the ITP contained a **Bus Interval Adjustment Control** field with a value equal to zero or the device's address and the device is in the Address or Configured state.

Only one device can control the bus interval length at a time. The host controller implements a first come first serve policy for handling bus interval adjustment requests as described in this section. When the host controller begins operation it shall transmit ITPs with the **Bus Interval Adjustment Control** field set to zero. When the host controller first receives a bus interval adjustment control request, it shall set the **Bus Interval Adjustment Control** field in subsequent ITPs to the address of the device that sent the request. The host shall ignore bus interval adjustment requests from all other devices once the **Bus Interval Adjustment Control** field is set to a non-zero address. If the controlling device is disconnected, the host controller shall reset the **Bus Interval Adjustment Control** field to zero. The host controller may provide a way for software to override default bus interval adjustment control field

behavior and select a controlling device. The host controller shall begin applying bus interval adjustments within two bus intervals from when the bus interval adjustment request is received.

The smallest bus interval adjustment (one `BusIntervalAdjustmentGranularity`) requires the host to make an average adjustment of eight high speed bit times every 4 096 bus intervals. The host is allowed to make this adjustment in a single bus interval such that the clock used to generate ITP times and bus interval boundaries does not need a period smaller than eight high speed bit times. The host shall make bus interval adjustments at regular intervals. When the host is required to make an average of one or more eight high speed bit time adjustments every 4 096 bus intervals the adjustments shall be evenly distributed as defined by the following constraints:

- Intervals that contain one more eight high speed bit time adjustment than other intervals are referred to as maximum adjustment bus intervals.
- The number of eight high speed bit time adjustments made in any bus interval shall not be more than one greater than the number of high speed bit time adjustments made in any other bus interval.
- The distance in bus intervals between consecutive maximum adjustment bus intervals shall not vary by more than one bus interval.

The even distribution and average adjustment requirements for bus interval adjustments shall apply from one bus interval after a bus interval adjustment request is received by the host until the bus interval where a subsequent valid bus interval adjustment request is received by the host.

The following is an example of valid host behavior for a specific bus interval adjustment request. After power on, the host receives a bus interval adjustment request for a bus interval decrease of 10 `BusIntervalAdjustmentGranularity` units in bus interval $X - 1$. The host controller uses a clock with a period of eight high speed bit times to drive a counter that produces timestamps and bus interval boundaries. The host controller adds an extra eight high speed bit time clock tick to its counter in each of the following bus intervals: $X + 409$, $X + 819$, $X + 1228$, $X + 1638$, $X + 2048$, $X + 2457$, $X + 2867$, $X + 3276$, $X + 3686$, $X + 4096$, $X + 4505$,...

8.5.6.7 Sublink Speed Device Notification

Sublink Speed Device Notification TPs are reported by the device to identify the characteristics of its link connection. Sublink Speed Device Notification TPs shall be generated by Enhanced SuperSpeed devices not operating at Gen 1 speed upon entering the *Address USB Device State*.

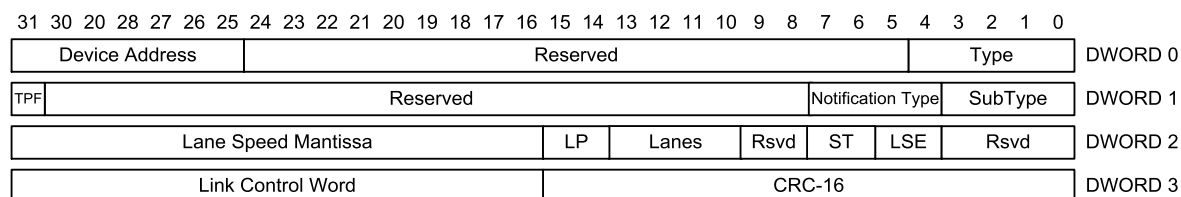
$\text{Lane Speed} = \text{Lane Speed Mantissa} \times 1000^{\text{Lane Speed Exponent}}$

$\text{Sublink Speed} = \text{Lane Speed} \times (\text{Lane Count} + 1)$

- The device shall inform the host that a Device Notification TP shall follow a `SET_ADDRESS` request by setting the *TP Follows* (TPF) flag in the Status stage ACK TP.
- The Rx Sublink Speed Device Notification TP shall be the next TP transmitted by a device after setting the *TP Follows* (TPF) flag in a Status stage ACK TP.
- The device shall inform the host that a second Device Notification TP shall follow the Rx Sublink Speed Device Notification TP by setting the *TP Follows* (TPF) flag in the Rx Sublink Speed Device Notification TP.
- The Tx Sublink Speed Device Notification TP shall be the next TP transmitted by a device after setting the *TP Follows* (TPF) flag in the Rx Sublink Speed Device Notification TP.

Asymmetric Lane Types may only be reported by SuperSpeed Interchip (SSIC) devices. A Symmetric link is one that has the same Lane Speed and number of lanes for both the Rx and

Tx Sublinks. Enhanced SuperSpeed devices shall only support Symmetric links and shall only support one lane per sublink.



U-108A

IEC

Figure 8-27 – Sublink Speed Device Notification

Table 8-22 – Sublink Speed Device Notification

Width (bits)	Offset (DW:bit)	Description													
4	1:0	SubType . This field shall be set to <i>DEV_NOTIFICATION</i> .													
4	1:4	Notification Type . SUBLINK_SPEED.													
23	1:8	Reserved .													
2	2:4	Lane Speed Exponent (LSE). This field defines the base 1000 exponent, that shall be applied to the Lane Speed Mantissa (LSM) when calculating the maximum Lane Speed represented by this Sublink Speed Device Notification. <table><tr><th>Value</th><th>Description</th></tr><tr><td>00b</td><td>Bits per second</td></tr><tr><td>01b</td><td>Kb/s</td></tr><tr><td>10b</td><td>Mb/s</td></tr><tr><td>11b</td><td>Gb/s</td></tr></table>	Value	Description	00b	Bits per second	01b	Kb/s	10b	Mb/s	11b	Gb/s			
Value	Description														
00b	Bits per second														
01b	Kb/s														
10b	Mb/s														
11b	Gb/s														
2	2:6	Sublink Type (ST). This field identifies whether this Sublink Speed Device Notification defines a symmetric or asymmetric bit rate. This field also indicates if this Sublink Speed Device Notification defines the receive or transmit bit rate. Note that the Sublink Speed Attributes shall be paired, i.e. an Rx immediately followed by a Tx, and both Attributes shall define the same value for the SSID. <table><tr><th>Bit</th><th>Value</th><th>Description</th></tr><tr><td rowspan="2">0</td><td>0</td><td>Symmetric. Rx and Tx Sublinks have the same number of lanes and operate at the same speed.</td></tr><tr><td>1</td><td>Asymmetric. Rx and Tx Sublink have different number of lanes and/or operate at different speeds.</td></tr><tr><td rowspan="2">1</td><td>0</td><td>Sublink operates in Receive mode</td></tr><tr><td>1</td><td>Sublink operates in Transmit mode</td></tr></table>	Bit	Value	Description	0	0	Symmetric. Rx and Tx Sublinks have the same number of lanes and operate at the same speed.	1	Asymmetric. Rx and Tx Sublink have different number of lanes and/or operate at different speeds.	1	0	Sublink operates in Receive mode	1	Sublink operates in Transmit mode
Bit	Value	Description													
0	0	Symmetric. Rx and Tx Sublinks have the same number of lanes and operate at the same speed.													
	1	Asymmetric. Rx and Tx Sublink have different number of lanes and/or operate at different speeds.													
1	0	Sublink operates in Receive mode													
	1	Sublink operates in Transmit mode													
2	2:8	Reserved .													
4	2:10	Lane Count (Lanes). This field identifies the number of Lanes associated with a Sublink. The <i>Lane Count</i> value is a zero based value; e.g., a Lane Count of 0 means 1 lane, 1 = 2 lanes, etc.													
2	2:14	Link Protocol (LP). This field identifies the protocol supported by the link. <table><tr><th>Value</th><th>Description</th></tr><tr><td>00b</td><td>Reserved</td></tr><tr><td>01b</td><td>SuperSpeedPlus</td></tr><tr><td>11b-10b</td><td>Reserved</td></tr></table>	Value	Description	00b	Reserved	01b	SuperSpeedPlus	11b-10b	Reserved					
Value	Description														
00b	Reserved														
01b	SuperSpeedPlus														
11b-10b	Reserved														
16	2:16	Lane Speed Mantissa (LSM). This field defines the mantissa that shall be applied to the LSE when calculating the Lane Speed represented by this Sublink Speed Device Notification.													

**NOTE**

This specification includes features to support SSIC capabilities, such as asymmetric link speeds and multiple lanes.

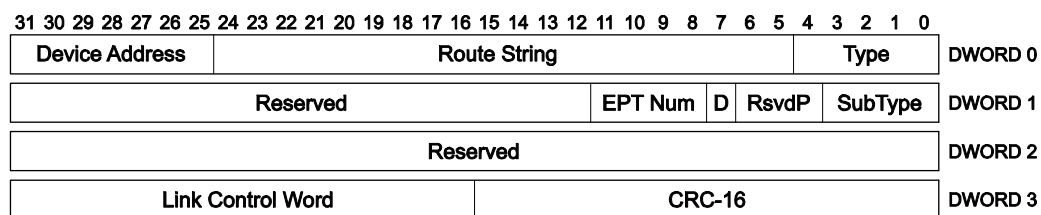
*The Enhanced SuperSpeed bus does not support Asymmetric link speeds or multiple lanes. An Enhanced SuperSpeed device shall only set the **Sublink Type** to Symmetric and shall set the **Lane Count** field to zero.*

8.5.7 PING Transaction Packet

This TP can only be sent by the host. It is used by the host to transition all links in the path to a device back to U0 prior to initiating an isochronous transfer. Refer to Appendix C for details on the usage of this TP. Only the fields that are different from an ACK TP are described in this section.

A device shall respond to the PING TP by sending a PING_RESPONSE TP (refer to Section 8.8) to the host within the tPingResponse time (refer to Table 8-36). Note that the device shall not validate the EP_NUM and Direction fields and simply copy them to the respective fields in the PING_RESPONSE TP.

A device shall keep its link in U0 until it receives a subsequent packet from the host, or until the tPingTimeout time (refer to Table 8-36) elapses.



U-109
IEC

Figure 8-28 – PING Transaction Packet**Table 8-23 – PING TP Format (differences with ACK TP)**

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType. This field shall be set to <i>PING</i> .
3	1:4	Reserved.
52	1:12	Reserved.

8.5.8 PING_RESPONSE Transaction Packet

This TP can only be sent by a device in response to a PING TP sent by the host. A PING_RESPONSE TP shall be sent for each PING TP received. Refer to Appendix C for details on the usage of this TP. Only the fields that are different from an ACK TP are described in this section.

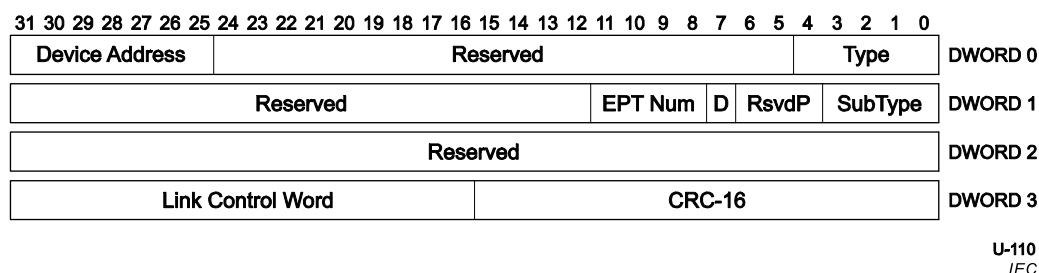


Figure 8-29 – PING_RESPONSE Transaction Packet

Table 8-24 – PING_RESPONSE TP Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description
4	1:0	SubType . This field shall be set to <i>PING_RESPONSE</i> .
3	1:4	Reserved .
1	1:7	Direction (D) . This field shall be set to the value of the Direction field in the PING TP for which this PING_RESPONSE TP is being sent.
4	1:8	Endpoint Number (Ept Num) . This field shall be set to the value of the Ept Num field in the PING TP for which this PING_RESPONSE TP is being sent.
52	1:12	Reserved .

8.6 Data Packet (DP)

This packet can be sent by either the host or a device. The host uses this packet to send data to a device. Devices use this packet to return data to the host in response to an ACK TP. All data packets are comprised of a Data Packet Header and a Data Packet Payload. Only the fields that are different from an ACK TP are described in this section.

Data packets traverse the direct path between the host and a device. Note that it is permissible to send a data packet with a zero length data block; however, it shall have a CRC-32.

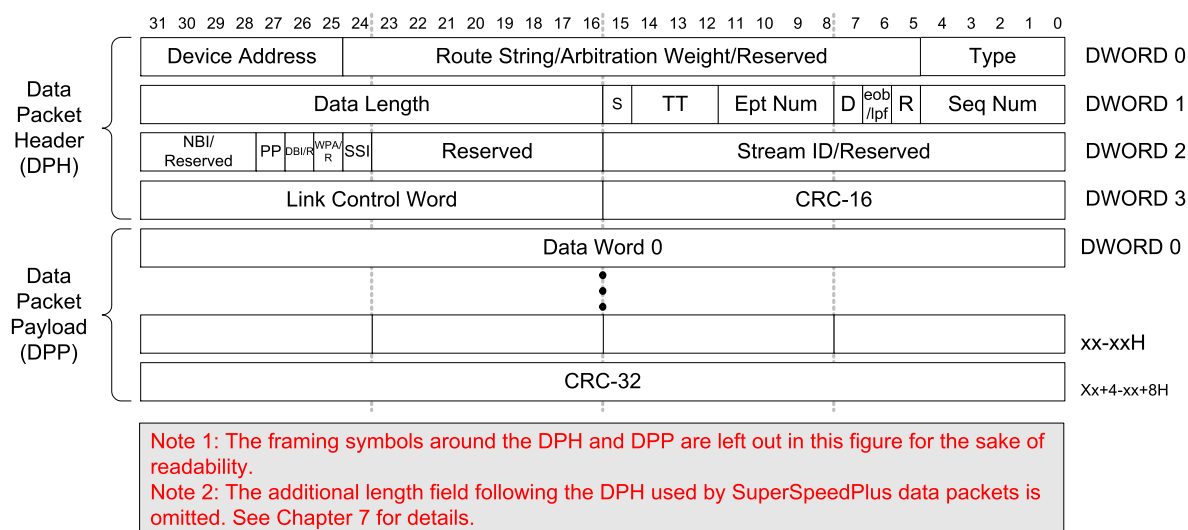


Figure 8-30 – Example Data Packet

Table 8-25 – Data Packet Format (Differences with ACK TP)

Width (bits)	Offset (DW:bit)	Description
20	0:5	When operating at Gen 1 speed: The definition of the Route String/Arbitration Weight/Reserved field is as defined in the ACK TP Route String/Reserved field and is only used by hubs. When not operating at Gen 1 speed:: The Route String/Arbitration Weight/Reserved field redefines the ACK TP Route String/Reserved field. For downstream flowing packets (packets from the host to a device), this field is interpreted as a Route String. Refer to Section 8.9 for details. For upstreaming flowing asynchronous packets (e.g. those sent by a device to a host) on SuperSpeedPlus bus segments, the lower 16-bits (bits 5 through 20) of this field indicate the Arbitration Weight. The remaining bits are Reserved and shall be set to zero. Refer to Section 10.8.6.1 for details.
5	1:0	Sequence Number (Seq Num). This field is used to identify the sequence number of the DP. Note that the sequence number wraps around at 31.
1	1:5	Reserved (R) .
1	1:6	End Of Burst (EOB)/Last Packet Flag (LPF). For non-isochronous endpoints, this field is referred to as EOB and for isochronous endpoints this field is referred to as LPF. For non-isochronous IN endpoints, this field is used to identify that this is the last packet of a burst. When a device is ready to continue the transfer, it shall send an ERDY TP to signal the host. Note that an endpoint shall re-evaluate the EOB value in a retried DP. The EOB field shall be set in the last packet of a burst if: • The device returns fewer than the number of packets requested in the NumP field of the last ACK TP it received and this packet is not a short packet. • If the device cannot meet tMaxBurstInterval for a device operating at Gen 1 speed, or tGen2MaxBurstInterval for device operating at Gen 2 speed and this packet is not a short packet. Note that a device is not required to set this field to a 1b when it transmits a short packet even if it will be returning fewer than the number of packets requested in the NumP field of the last ACK TP it received. It is only required to set this field to a 1b if it wants to enter the flow control state after completing the current transfer with this short packet. For non-isochronous OUT and control endpoints, this field shall be set to zero. For isochronous endpoints this field is used to identify that this is the last packet of the last burst in the current service interval. LPF can be set by a device and the host. Please refer to Section 8.12.6 for the usage of this field when the target or source of this DP is an isochronous endpoint.
4	1:8	Endpoint Number (Ept Num) . This field determines an endpoint within the device that is the source or recipient of this DP.
1	1:15	Setup (S) . This field is set by the host to indicate that this DP is a Setup data packet. This field can only be set by the host.
16	1:16	Data Length . This field is used to indicate the number of bytes in the DPP excluding the data CRC-32.
1	2:27	Packets Pending (PP) . This field may only be set by the Host. If this field is set, then the host has one or more DPs available for transmission to this endpoint/Stream. Where the endpoint is identified by the Endpoint Number and Direction fields, and if this is a Stream endpoint then the Stream is identified by the Stream ID field. If the field is cleared, then this is the last DP that the host has available for transmission to the target endpoint/Stream. If no endpoints on this device have packets pending, then the device can use this information to aggressively power manage its upstream link, e.g., set the link to a lower power U1 or U2 state.
xx	4:0	Data Block . This field contains the data in the DPP. The size of this field in bytes is indicated by the value in the Data Length field.
32	4:0 + xx	CRC-32. The data CRC is calculated over the data block of the DPP. Refer to Section 7.2.1.2.2 for the polynomial used to calculate this value. Note that this field is not necessarily aligned on a DWORD boundary as the data block length may not be a multiple of four.

8.7 Isochronous Timestamp Packet (ITP)

The Isochronous Timestamp Packet (ITP) shall be multicast on all links in U0 that have completed Port Configuration.

The value in the **Type** field is *Isochronous Timestamp Packet* for an ITP. ITPs are used to deliver timestamps from the host to all active devices. ITPs carry no addressing or routing information and are multicast by hubs to all of their downstream ports with links in the U0 state and that have completed Port Configuration. A device shall not respond to an ITP. ITPs are used to provide host timing information to devices for synchronization. Note that any device or hub may receive an ITP. The host shall transmit an ITP on a root port link if and only if the link is already in U0. Only the host shall initiate an ITP transmission. The host shall not bring a root port link to U0 for the purpose of transmitting an ITP. The host shall transmit an ITP in every bus interval within tTimestampWindow from a bus interval boundary if the root port link is in U0. The host shall begin transmitting ITPs within tIsochronousTimestampStart from when the host root port's link enters U0 from the polling state. An ITP may be transmitted in between packets in a burst. If a device receives an ITP with the delayed flag (DL) set in the link control word, the timestamp value may be significantly inaccurate and may be ignored by the device.

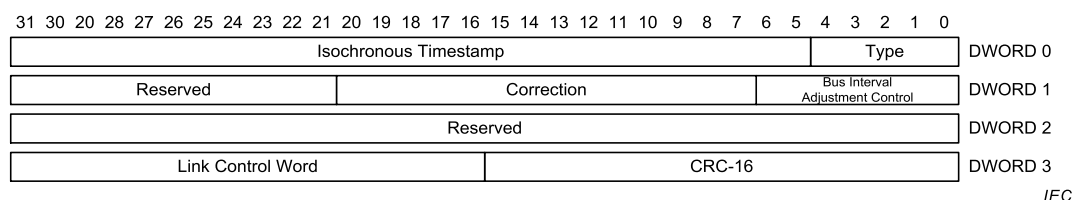


Figure 8-31 – Isochronous Timestamp Packet

Table 8-26 – Isochronous Timestamp Packet Format

Width (bits)	Offset (DW:bit)	Description						
27	0:5	Isochronous Timestamp (ITS). The isochronous timestamp field is used to identify the current time value from the perspective of the host transmitting the ITP. The timestamp field is split into two sub-fields: <table><tr><th>Bits</th><th>Description</th></tr><tr><td>13:0</td><td>Bus interval counter. The current 125 us (one Microframe interval) counter. The count value rolls over to zero when the value reaches 0x3FFF and continues to increment.</td></tr><tr><td>26:14</td><td>Delta. The time delta from the start of the current ITP packet to the previous bus interval boundary. This value is a number of tIsochTimestampGranularity units. The value used shall specify the delta that comes closest to the previous bus interval boundary without going before the boundary.</td></tr></table> NOTE If a packet starts exactly on a bus interval boundary, the delta time is set to 0.	Bits	Description	13:0	Bus interval counter. The current 125 us (one Microframe interval) counter. The count value rolls over to zero when the value reaches 0x3FFF and continues to increment.	26:14	Delta. The time delta from the start of the current ITP packet to the previous bus interval boundary. This value is a number of tIsochTimestampGranularity units. The value used shall specify the delta that comes closest to the previous bus interval boundary without going before the boundary.
Bits	Description							
13:0	Bus interval counter. The current 125 us (one Microframe interval) counter. The count value rolls over to zero when the value reaches 0x3FFF and continues to increment.							
26:14	Delta. The time delta from the start of the current ITP packet to the previous bus interval boundary. This value is a number of tIsochTimestampGranularity units. The value used shall specify the delta that comes closest to the previous bus interval boundary without going before the boundary.							
7	1:0	Bus Interval Adjustment Control. This field specifies the address of the device that controls the bus interval adjustment mechanism. Upon reset, power-up, or if the device is disconnected, the host shall set this field to zero.						
14	1:7	Correction. This field specifies the negative delay in tIsochTimestampGranularity units that the ITP has accumulated passing through PTM capable hubs. This field shall be set to 0 by the host.						
43	1:21	Reserved.						

The ITS value in the ITP shall have an accuracy of ± 1 tIsochTimestampGranularity units of the value of the host clock (for ITP generation) measured when the first framing symbol of the ITP is transmitted by the host. The requirements with respect to ITPs for a hub that supports Precision Time Management (PTM) are described in Section 10.9.4.4.1.

8.8 Addressing Triple

Data Packets and most Transaction Packets provide access to specific data flow using a composite of three fields. They are the **Device Address**, the **Endpoint Number**, and the **Direction** fields.

Upon reset and power-up, a device's address defaults to a value of zero and shall be programmed by the host during the enumeration process with a value in the range from 1 to 127. Device address zero is reserved as the default address and may not be assigned to any other use.

Devices may support up to a maximum of 15 IN and 15 OUT endpoints (as indicated by the **Direction** field) apart from the required default control endpoint that has an endpoint number set to zero.

8.9 Route String Field

The **Route String** is a 20-bit field in downstream directed packets that the hub uses to route each packet to the designated downstream port. It is composed of a concatenation of the downstream port numbers (4 bits per hub) for each hub traversed to reach a device. The hub uses a Hub Depth value multiplied by four as an offset into the Route String to locate the bits it uses to determine the downstream port number. The Hub Depth value is determined and assigned to every hub during the enumeration process.



IEC

Figure 8-32 – Route String Detail

In Figure 8-32, the value in Hub@Tier1 field is the downstream port number of the hub connected directly to one of the root ports on the host to which a second hub is attached and so on.

8.9.1 Route String Port Field

This 4-bit wide field in the **Route String** represents the port in the hub being addressed.

8.9.2 Route String Port Field Width

The **Route String** Port field width is fixed at 4 bits, limiting the maximum number of ports a hub may support to 15.

8.9.3 Port Number

The specific port on a hub to which the packet is directed is identified by the value in the Route String Port field. When addressing the hub controller then the Port Number field at the hub's tier level shall be set to zero in the Route String. The hub's downstream ports are addressed beginning with one and count up sequentially.

8.10 Transaction Packet Usages

TPs are used to report the status of data transactions and can return values indicating successful reception of data packets, command acceptance or rejection, flow control, and halt conditions.

8.10.1 Flow Control Conditions

This section describes the interaction between the host and a device when an endpoint returns a flow control response. The flow control is at an end-to-end level between the host and the endpoint on the device. Only bulk, control and interrupt endpoints may send flow control responses. Isochronous endpoints shall not send flow control responses.

An IN endpoint shall be considered to be in a flow control condition if it returns one of the following responses to an ACK TP:

- Responding with an NRDY TP; note that an endpoint shall wait until it receives an ACK TP for the last DP it transmitted before it can send an NRDY TP
- Sending a DP with the **EOB** field set to 1 in the DPH

An OUT endpoint shall be considered to be in a flow control condition if it returns one of the following responses to a DP:

- Responding with an NRDY TP
- Sending an ACK TP with the **NumP** field set to 0

The **Packets Pending** field is only valid when set by the host and does not affect whether or not an endpoint enters the flow control state. Refer to Section 8.11 for further details on host and device TP responses.

When an endpoint is in a flow control condition, it shall send an ERDY TP to be moved back into the active state. Further, if the endpoint is an IN endpoint, then it shall wait until it receives an ACK TP for the last DP it transmitted before it can send an ERDY TP. When an endpoint is not in a flow control condition, it shall not send an ERDY TP unless the endpoint is a Bulk endpoint that supports streams. Refer to Section 8.12.1.4.2 and Section 8.12.1.4.3 for further information about when a Bulk endpoint that supports streams can send an ERDY TP. The host may resume transactions to any endpoint – even if the endpoint had not returned an ERDY TP after returning a flow control response. To ensure that the host and the device continue to operate normally, a host shall ignore ERDY TPs from an endpoint that is not in a flow control state. If the host continues, or resumes, transactions to an endpoint, the endpoint shall re-evaluate its flow control state and respond appropriately.

8.10.2 Burst Transactions

The Enhanced SuperSpeed architecture allows bursting of DPs between a host and device.

SuperSpeedPlus architecture has features that create additional burst conditions for DPs:

- Multiple simultaneous IN transactions
- Hubs with additional buffering and with local arbitration decisions
- Hubs with different speed downstream facing port links.

8.10.2.1 Enhanced SuperSpeed Burst Transactions

The Enhanced SuperSpeed USB protocol allows the host to continually send data to a device or receive data from a device as long as the device can receive the data or transmit the data. The number of packets an endpoint on a device can send or receive at a time without an intermediate ACK TP is reported by the device in the endpoint companion descriptor (refer to Section 9.6.7) for that endpoint. An endpoint that reports more than one packet in its maximum burst size is considered to be able to support “Burst” Transactions.

While bursting the following rules apply:

- The maximum number of packets that can be sent in a burst prior to receiving an acknowledgement is limited to the minimum of the maximum burst size (see the definition

of **bMaxBurst** in Table 9-26) of the endpoint and the value of the **NumP** field in the last ACK TP or ERDY received by the endpoint or the host, minus the number of packets that the endpoint or the host has already sent after the packet acknowledged by the last ACK TP.

Note that host may re-initialize the maximum number of DPs that can be sent/received in a burst to the maximum burst size of the endpoint whenever the endpoint is initialized or the host is resuming transactions to an endpoint after a flow control condition.

- Each individual packet in the burst shall have a data payload of maximum packet size. Only the last packet in a burst may be of a size smaller than the reported maximum packet size. If the last one is smaller, then the same rules for short packets apply to a short packet at the end of a burst (refer to Section 8.10.2).
- The burst transaction continues as long as the **NumP** field in the ACK TP is not set to zero and each packet has a data payload of maximum packet size.
- The **NumP** field can be incremented at any time by the host or a device sending the ACK TP as long as the device or host wants to continue receiving data. The only requirement is that the **NumP** field shall not have a value greater than the maximum burst supported by the device. However, for an ISOC IN endpoint, refer to Section 8.12.6, for additional requirements on how to change **NumP** for each burst.
- If a device or host sending an ACK TP decrements the **NumP** field, then it shall do so by no more than one. For example, if the previous ACK TP had a value of five in the **NumP** field, then the next ACK TP to acknowledge the next packet received shall have a value of no less than four in the **NumP** field. The only exceptions to this rule are:
 - 1) If the device can receive the data but cannot accept any more data, then it shall send an ACK TP with the **NumP** field set to zero.
 - 2) The host shall send an ACK TP with the **NumP** field set to zero in response to a device sending a DP with the **EOB** field set or that is a short packet (see Section 8.10.2). However, if the host receives a short packet with **EOB** = 0 and the host has another transfer to initiate with the same endpoint, then the host may instead send an ACK TP with the **NumP** field set to a non-zero value.
 - 3) The host may send an ACK TP with the **rtty** bit set to one and the **NumP** field set to any value less than the maximum burst that the endpoint is capable of, including zero, in response to a device sending a DP with a DPP error (See Section 8.11.2).

8.10.2.2 SuperSpeedPlus Burst Transactions

The SuperSpeedPlus architecture has the following additional requirements for burst transactions.

The SuperSpeedPlus architecture defines signaling rates faster than SuperSpeed. In order to maximize performance of the bus, when operating at Gen 2 speed, Enhanced SuperSpeed devices and hosts shall be limited to **tGen2MaxBurstInterval** for the time between DPs being bursted from a device endpoint to the host or from the host to a device endpoint.

Since the SuperSpeedPlus architecture allows multiple INs, a single SuperSpeedPlus device can be ready to burst multiple DPs from multiple endpoints whenever the link is available. A SuperSpeedPlus device operating at Gen 2 speed shall be limited to **tGen2MaxDeviceMultiPacketInterval** for the time between DPs being concurrently bursted from different device endpoints to the host.

In the SuperSpeedPlus architecture, DPs for different endpoints or devices can be buffered and reordered with respect to each other as they pass through SuperSpeedPlus hubs, due to other traffic that may be contending for use of path. When a SuperSpeedPlus hub has several DPs buffered for a link operating at Gen 2 speed, it shall transmit those DPs with a maximum of **tGen2MaxHubMultiPacketInterval** for the time between DPs, whether those DPs are for the same or different devices or endpoints.

8.10.3 Short Packets

Enhanced SuperSpeed retains the semantics of short packet behavior that USB 2.0 supports. When the host or a device receives a DP with the **Data Length** field shorter than the maximum packet size for that endpoint it shall deem that that transfer is complete.

In the case of an IN transfer, a device shall stop sending DPs after sending a short DP. The host shall respond to the short DP with an ACK TP with the **NumP** field set to zero unless it has another transfer for the same endpoint in which case it may set the **NumP** field as mentioned in Section 8.10.2. The host shall schedule transactions to the endpoint on the device when another transfer is initiated for that endpoint.

In the case of an OUT transaction, the host may stop sending DPs after sending a short DP. The host shall schedule transactions to an endpoint on the device when another transfer is initiated for that endpoint. Note that this shall be the start of a new burst to the endpoint.

8.10.4 SuperSpeedPlus Transaction Reordering

On a SuperSpeedPlus bus instance, TPs shall be transmitted before DPs (for both periodic and asynchronous packets), if there are TPs and DPs ready for transmission.

TPs use Type 1 Link Credits.

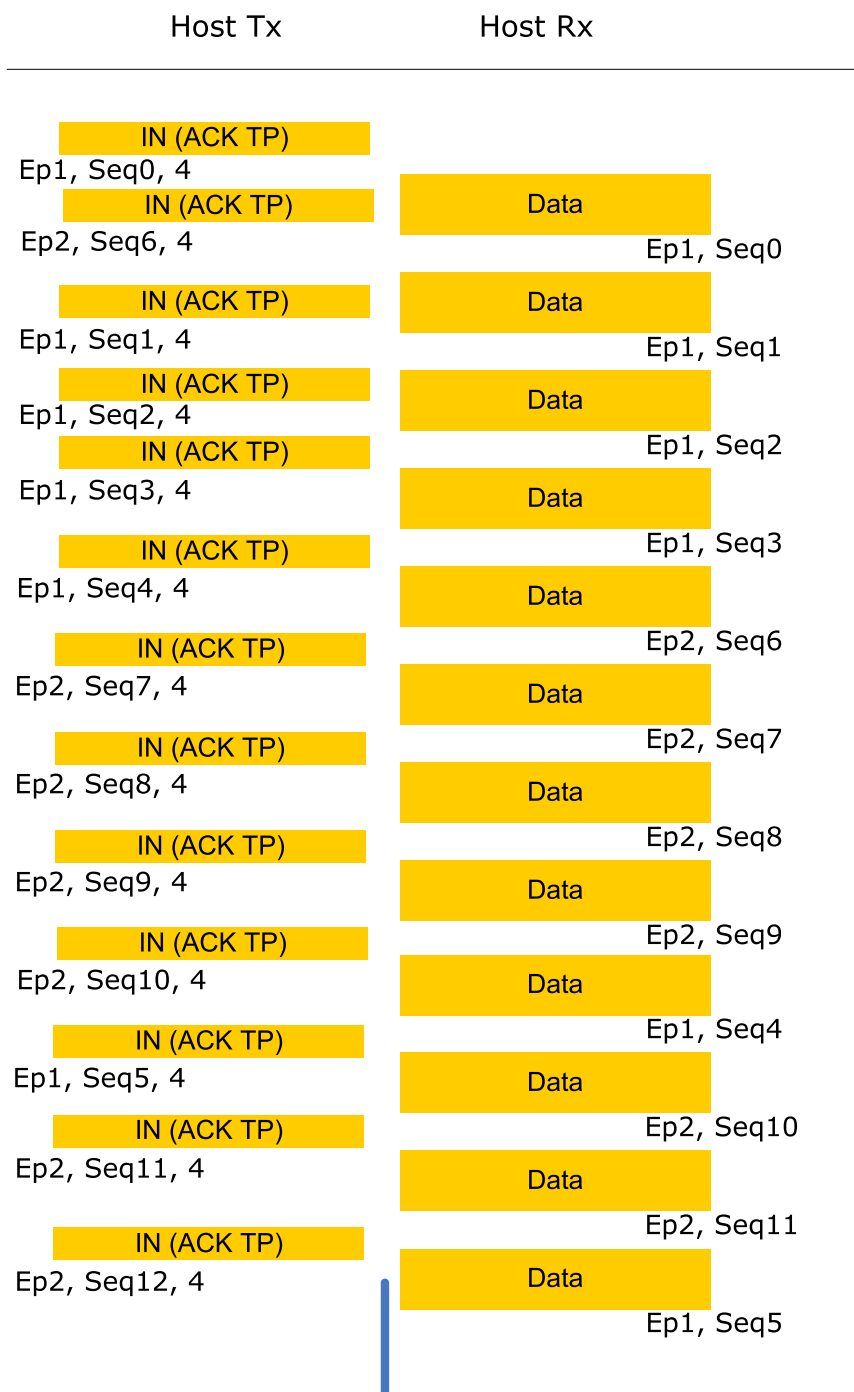
Hosts and devices shall set the Transfer Type (TT) field in ACK and Data Packet Header (DPH) packets that they originate on SuperSpeedPlus bus instances. Upward flowing DPHs from an Asynchronous endpoint have an Arbitration Weight (AW) field. SuperSpeedPlus Devices shall set the AW field to zero.

SuperSpeedPlus hubs and devices shall select periodic data packets before asynchronous data packets for transmission on the link.

Periodic DPs use Type 1 Link Credits. Asynchronous DPs use Type 2 Link Credits.

The order that DPs are returned is independent of the order in which the IN transactions from different endpoints were initiated on the bus. TPs and DPs involving the same endpoint shall be delivered in the order they were transmitted from the host or device.

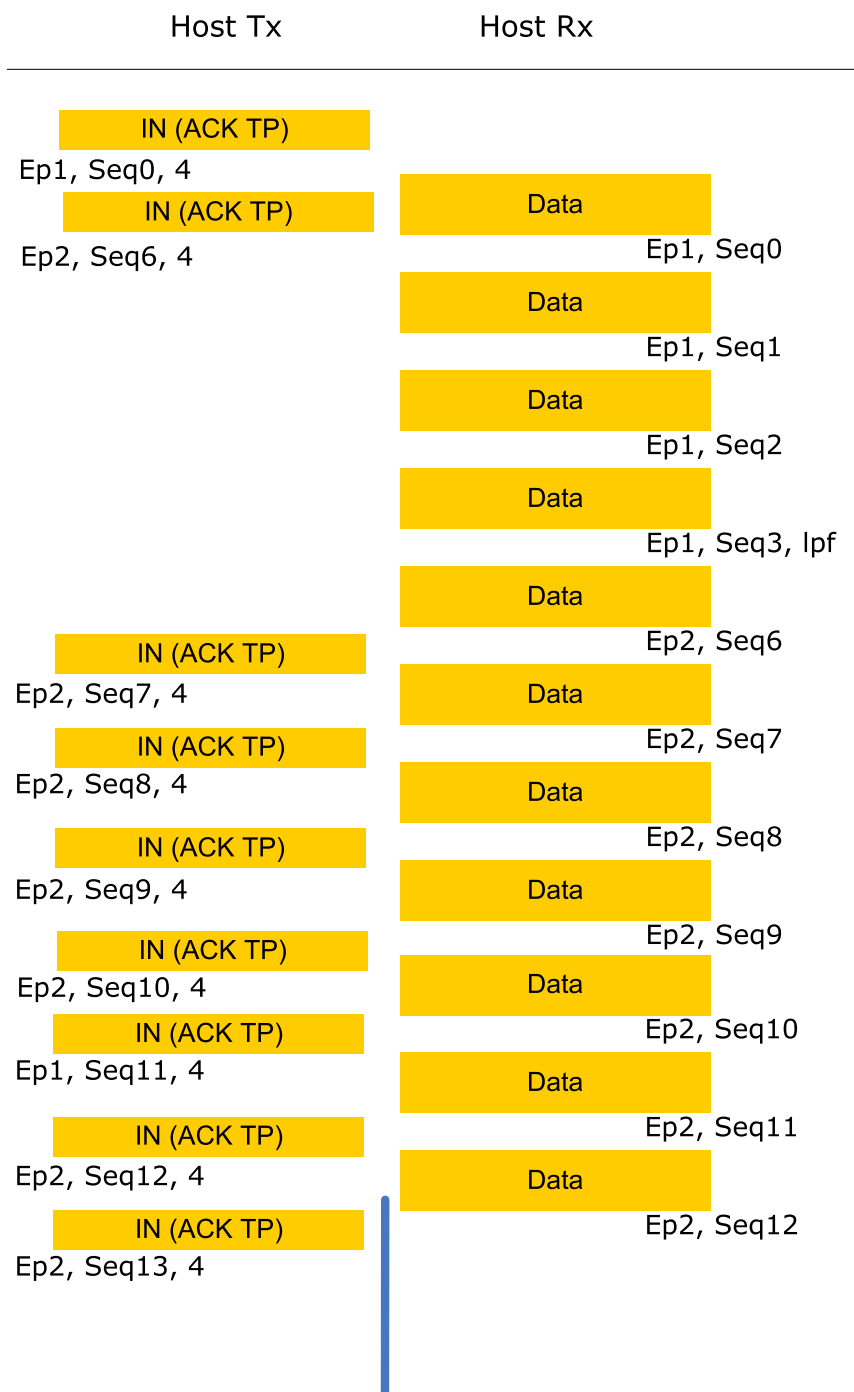
BULK IN



IEC

Figure 8-33 – Sample Concurrent BULK IN Transactions

BULK IN and ISOC IN



IEC

Figure 8-34 – Sample Concurrent BULK and Isochronous IN Transactions

8.11 TP or DP Responses

Transmitting and receiving devices shall return DPs or TPs as detailed in Table 8-27 through Table 8-29. Not all TPs are allowed, depending on the transfer type and depending on the direction of flow of the TP.

8.11.1 Device Response to TP Requesting Data

Table 8-27 shows the possible ways a device shall respond to a TP requesting data for bulk, control, and interrupt endpoints. A TP is considered to be invalid if one or more of the following conditions exist:

- It has an incorrect Device Address
- Its endpoint number and direction does not refer to an endpoint that is part of the current configuration
- It does not have the expected sequence number
- Its TT does not match the endpoint type (for a device not operating at Gen 1 speed).

**Table 8-27 – Device Responses to TP Requesting Data
(Bulk, Control, and Interrupt Endpoints)**

Invalid TP Received	TP Received with Deferred Bit Set	Device Tx Endpoint Halt Feature Set	Device Ready to Transmit Data	Action Taken
Yes	Do not care	Do not care	Do not care	The device shall ignore the TP.
No	Yes	Yes	Do not care	The device shall send an ERDY TP.
No	Yes	No	No	The device shall not respond. It shall send an ERDY TP when it is ready to resume.
No	Yes	No	Yes	The device shall send an ERDY TP indicating that it is ready to send data.
No	No	Yes	Do not care	Issue STALL TP
No	No	No	No	Issue NRDY TP
No	No	No	Yes	Start transmitting DPs with sequence numbers requested by the host

An IN endpoint shall wait until it receives an ACK TP for the last DP it transmitted before it can send an STALL TP.

8.11.2 Host Response to Data Received from a Device

Table 8-28 shows the host responses to data received from a device for bulk, control, and interrupt endpoints. The host is able to return only an ACK TP. A DPH is considered to be invalid if any of the following conditions exist:

- It has an incorrect Device Address
- Its endpoint number and direction does not refer to an endpoint that is part of the current configuration
- It does not have the expected sequence number
- Its Data length in the DPH is greater than the endpoint's maximum packet size
- Its TT does not match the endpoint type (from a device not operating at Gen 1 speed).

In Table 8-28, DPP Error may be due to one or more of the following:

- CRC incorrect
- DPP aborted
- DPP missing
- Data length in the DPH does not match the actual data payload length

**Table 8-28 – Host Responses to Data Received from a Device
(Bulk, Control, and Interrupt Endpoints)**

DPH has Invalid Values	Data Packet Payload Error	Host Can Accept Data	TP Returned by Host
Yes	Do not care	Do not care	Discard data and do not send any TP.
No	Yes	Do not care	Discard data and send an ACK TP with the Retry bit set, requesting for zero or more DPs with the Sequence Number field set to the sequence number of the DP that was corrupted.
No	No	No	Discard data; send an ACK TP with the Retry bit set requesting for one or more DPs with the Sequence Number field set to the sequence number of the DP that the host was unable to receive. The ACK TP shall have the Host Error bit set to one to indicate that the host was unable to accept the data.
No	No	Yes	Accept data and send an ACK TP requesting for zero or more DPs with the Sequence Number field set to the sequence number of the next DP expected. This is also an implicit acknowledgement that this DP was received successfully.

8.11.3 Device Response to Data Received from the Host

TP responses by a device to data received from the host for bulk, control, and interrupt endpoints are shown in Table 8-29. A DPH is considered to be invalid if one or more of the following conditions exist:

- It has an incorrect Device Address
- Its endpoint number and direction does not refer to an endpoint that is part of the current configuration
- It does not have the expected sequence number
- Its Data length in the DPH is greater than the endpoint's maximum packet size
- Its TT does not match the endpoint type (for a device not operating at Gen 1 speed).

In Table 8-29, DPP Error may be due to one or more of the following:

- CRC incorrect
- DPP aborted
- DPP missing
- Data length in the DPH does not match the actual data payload length

NOTE Receipt of an ACK TP indicates to the host the DP with the previous sequence number was successfully received by a device as well as the number of data packet buffers the device has available to receive any pending DPs the host has. A device shall send an ACK TP for each DP successfully received.

**Table 8-29 – Device Responses to OUT Transactions
(Bulk, Control, and Interrupt Endpoints)**

DPH has Invalid Values	DPH has Deferred Bit Set	Receiver Halt Feature Set	Data Packet Payload Error	Device Can Accept Data	TP Returned by Device
Yes	Do not care	Do not care	Do not care	Do not care	Discard DP.
No	Yes	Yes	Do not care	Do not care	The device shall send an ERDY TP.
No	Yes	No	Do not care	No	The device shall not respond. It shall send an ERDY TP when it is ready to resume.
No	Yes	No	Do not care	Yes	The device shall send an ERDY TP.
No	No	Yes	Do not care	Do not care	The device shall send a STALL TP.
No	No	No	Do not care	No	Discard DP, send an NRDY TP.
No	No	No	Yes	Yes	Discard DP, send an ACK TP with the sequence number of the DP expected (thereby indicating that the DP was not received), the Retry bit set and the number of DPs that the device can receive for this endpoint.
No	No	No	No	Yes	Send an ACK TP indicating the sequence number of the next DP expected (thereby indicating that this DP was received successfully) and the number of DPs that the device can receive for this endpoint.

8.11.4 Device Response to a SETUP DP

A SETUP DP is a special DP that is identified by the **Setup** field set to one and addressed to any control endpoint. SETUP is a special type of host-to-device data transaction that permits the host to initiate a command that the device shall perform. Upon receiving a SETUP DP, a device shall respond as shown in Table 8-30.

A SETUP DPH shall be considered invalid if it has any one of the following:

- Incorrect Device Address
- Endpoint number and direction does not refer to an endpoint that is part of the current configuration
- Endpoint number does not refer to a control endpoint
- Non-zero sequence number
- Data length is not set to eight
- TT does not match the endpoint type (for a device not operating at Gen 1 speed).

In Table 8-30, DPP Error may be due to one or more of the following:

- CRC incorrect
- DPP aborted
- DPP missing
- Data length in the Setup DPH does not match the actual data payload length.

**Table 8-30 – Device Responses to SETUP Transactions
(Only for Control Endpoints)**

DPH has Invalid Values	DPH has Deferred Bit Set	Data Packet Payload Error	TP Returned by Device
Yes	N/A	N/A	Discard DP.
No	Yes	N/A	The device shall send an ERDY TP indicating that it is ready to receive the SETUP DP.
No	No	Yes	Discard SETUP DP, send an ACK TP with the sequence number set to zero, the rty bit set and the NumP field set to one.
No	No	No	Send an ACK TP with the sequence number set to one (thereby indicating that this SETUP DP was received successfully). The value in the NumP field indicates to the host whether the device wants to flow control the Data/Status stage or not. Refer to Section 8.12.2 for details.

8.12 TP Sequences

The packets that comprise a transaction vary depending on the endpoint type. There are four endpoint types: bulk, control, interrupt, and isochronous.

8.12.1 Bulk Transactions

The bulk transaction type is characterized by its ability to guarantee error-free delivery of data between the host and a device by means of error detection and retry. Bulk transactions use a two-phase transaction consisting of TPs and DPs. Under certain flow control and halt conditions, the data phase may be replaced with a TP. The **TT** field shall be set to Bulk by hosts and peripheral devices operating above Gen 1 speed; see Table 8-13.

8.12.1.1 State Machine Notation Information

This section shows detailed host and device endpoint state machines required to advance the Protocol on an IN or OUT pipe. The diagrams should not be taken as a required implementation, but to specify the required behavior.

Figure 8-35 shows the legend for the state machine diagrams. A circle with a three line border indicates a reference to another (hierarchical) state machine. A circle with a two-line border indicates an initial state. A circle with a single line border is a simple state.

A diamond (joint) is used to join several transitions to a common point. A joint allows a single input transition with multiple output transitions or multiple input transitions and a single output transition. All conditions on the transitions of a path involving a joint must be true for the path to be taken. A path is simply a sequence of transitions involving one or more joints.

A transition is labeled with a block with a line in the middle separating the (upper) condition and the (lower) actions. A transition without a line is a condition only. The condition is required to be true to take the transition. The actions are performed if the transition is taken. The syntax for actions and conditions is VHDL. A circle includes a name in bold and optionally one or more actions that are performed upon entry to the state.

Transitions using a solid arrow are generated by the host. Transitions using a dashed arrow are generated by a device. Transitions using a dot-dot-dash arrow are generated by the either a device or the host.

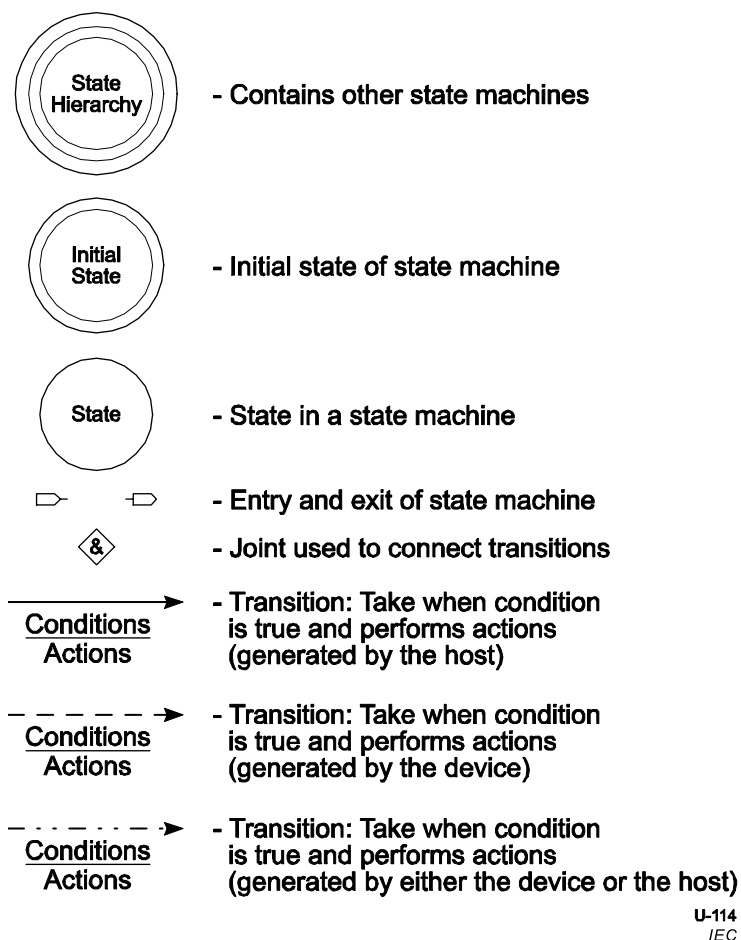


Figure 8-35 – Legend for State Machines

8.12.1.2 Bulk IN Transactions

When the host is ready to receive bulk data, it sends an ACK TP to a device indicating the sequence number and number of packets it expects from the device. A Bulk endpoint shall respond as defined in Section 8.11.1.

The host shall send an ACK TP for each valid DP it receives from a device. A device does not need to wait for the ACK TP to send the next DP to the host if the previous ACK TP indicated that the host expected the device to send more than one DP (depending on the value of the **Number of Packets** field in the TP). The ACK TP implicitly acknowledges the last DP with the previous sequence number as being successfully received by the host and also indicates to the device the next DP with the sequence number and number of packets the host expects from the device. If the host detects an error while receiving any of the DPs, it shall send an ACK TP with the sequence number value set to the first DP that was received with an error with the Retry bit set, even if subsequent packets in the burst asked for by the host were received without error. A device is required to resend all DPs starting from the sequence number set in the ACK TP in which the Retry bit set.

The host expects the first DP to have a sequence number set to zero when it starts the first transfer from an endpoint after the endpoint has been initialized (via a Set Configuration, Set Interface, or a ClearFeature (ENDPOINT_HALT) command – refer to Chapter 9 for details on these commands). The second DP sent by the device from that endpoint shall have a sequence number set to one; the third DP has a sequence number set to two, and so on until sequence number 31. The next DP after sequence number 31 uses a sequence number of zero. An endpoint on the device keeps incrementing the sequence number of the packets it transmits unless it receives an ACK TP with the Retry bit set to one that indicates that it has to retransmit an earlier DP.

If the host asks for multiple DPs from a device and the device does not have that number of DPs available to send at the time, the device shall send the last DP with the **End Of Burst** flag in the DPH set to one. Note that it is not necessary to set the **End Of Burst** flag if the DP sent to the host has a payload that is less than the MaxPacketSize for that endpoint.

A transfer is complete when a device sends all the data that is expected by the host or it sends a DP with a payload that is less than the MaxPacketSize. When the host wants to start a new transfer, it shall send another ACK TP with the next sequence number and number of DPs expected from a device. For example, if the DP with the payload less than MaxPacketSize was two, the host shall initiate the next transfer by sending an ACK TP with the expected sequence number set to three.

8.12.1.3 Bulk OUT Transactions

When the host is ready to transmit bulk data, it sends one or more DPs to a device. If a DPH with valid values (valid device address, endpoint number, and direction as well as the expected sequence number) is received by a device, it shall respond as defined in Section 8.11.3.

The host always initializes the first DP sequence number to zero in the first transfer it performs to an endpoint after the endpoint is initialized (via a Set Configuration, Set Interface, or ClearFeature (ENDPOINT_HALT) command – refer to Chapter 9 for details on these commands). The second DP has a sequence number set to one; the third DP has a sequence number set to two; and so on until 31. The next DP after sequence number 31 uses a sequence number of zero. The host keeps incrementing the sequence number of the DPs it transmits unless it receives an ACK TP with the Retry bit set to one that indicates that it has to retransmit an earlier packet.

A transfer is complete when the host sends all the data it has to a device; however, the last DP of the transfer may or may not have a payload which is equal to the MaxPacketSize of the endpoint. When the host wants to start a new transfer it shall send another DP, with the next sequence number, targeted at an endpoint in the device.

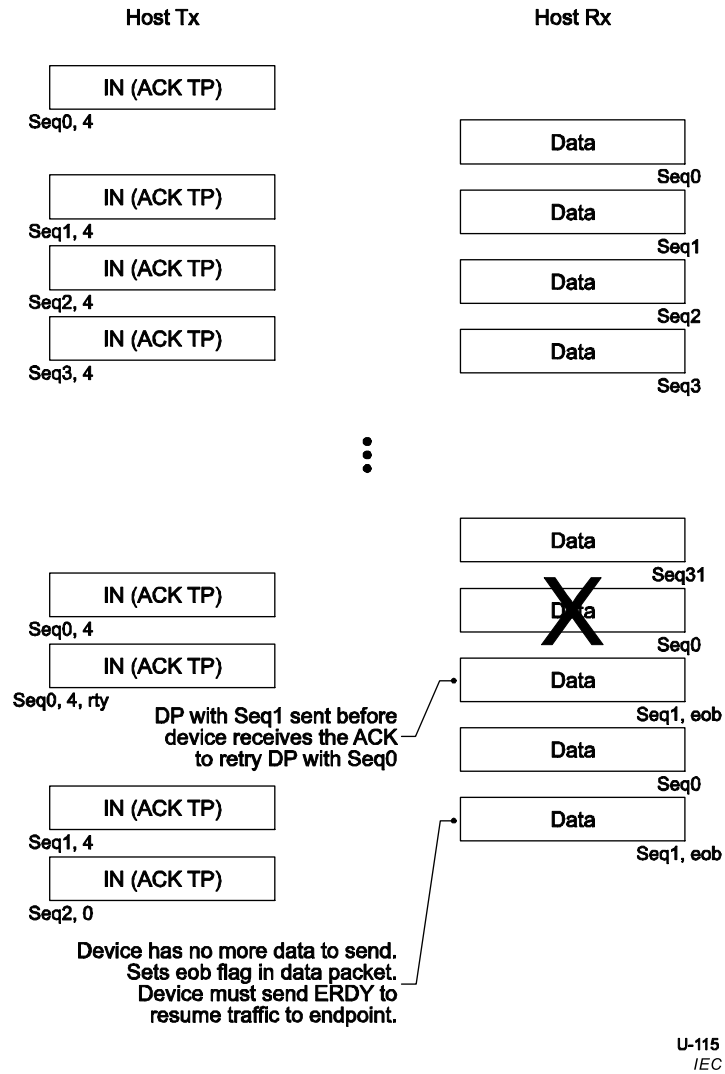


Figure 8-36 – Sample BULK IN Sequence

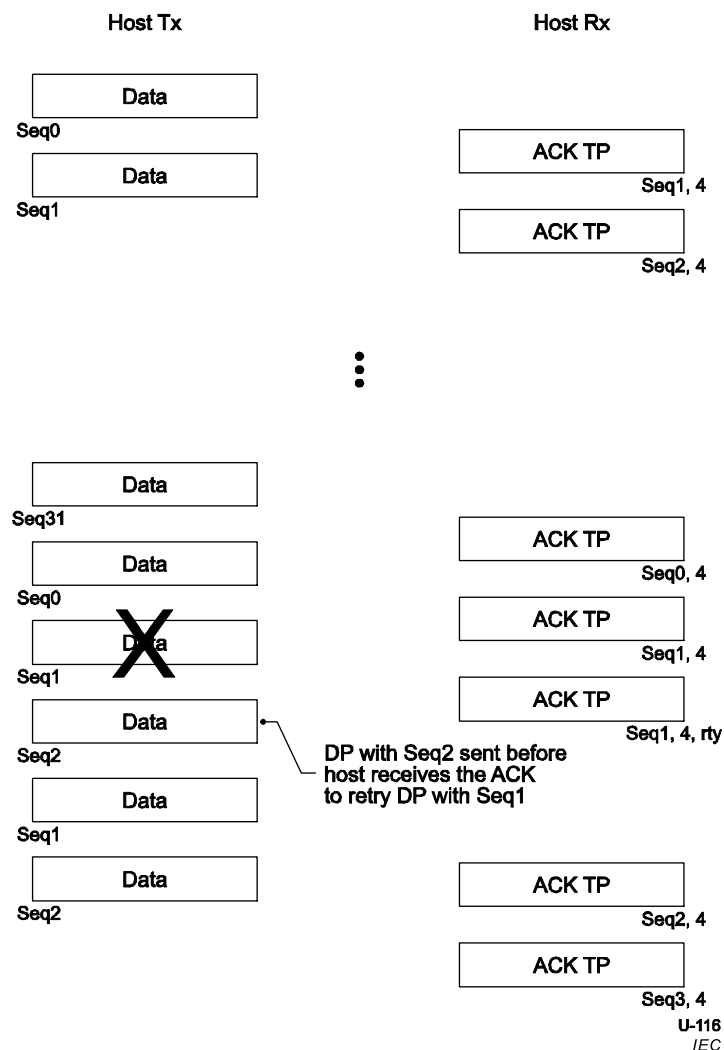


Figure 8-37 – Sample BULK OUT Sequence

8.12.1.4 Bulk Streaming Protocol

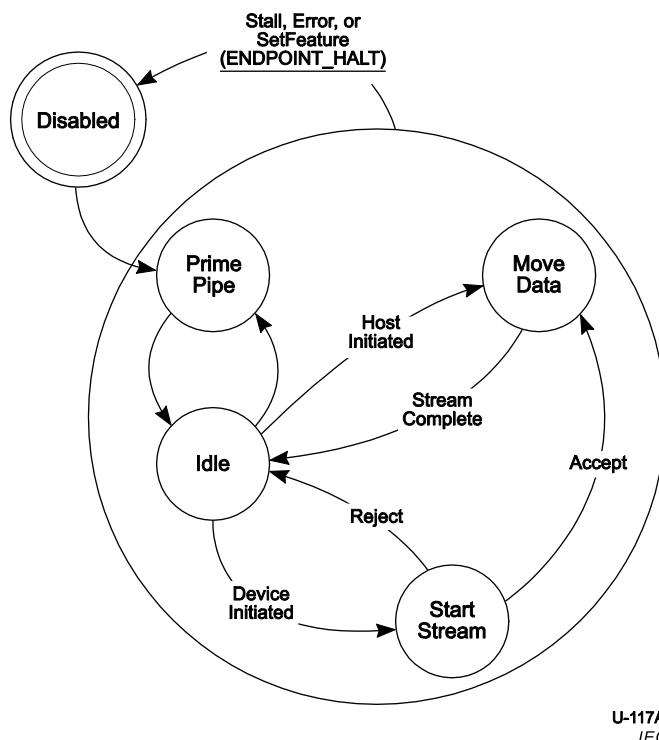
The Stream Protocol adheres to the semantics of the standard Enhanced SuperSpeed Bulk protocol, so the packet exchanges on an Enhanced SuperSpeed bulk pipe that supports Streams are indistinguishable from an Enhanced SuperSpeed bulk pipe that does not. The Stream Protocol is managed strictly through manipulation of the *Stream ID* field in the packet header.

NOTE Device Class defined methods are used for coordinating the Stream IDs that are used by the host to select Endpoint Buffers and by the device to select the Function Data associated with a particular Stream. Typically this is done via an out-of-band mechanism (e.g., another endpoint) that is used to pass the list of "Active Stream IDs" between the host and the device.

NOTE The Stream state machines illustrate a 1:1 relationship between sending a DP and receiving an ACK. Logically this is true; however, Enhanced SuperSpeed burst capabilities allows up to MaxBurst outstanding ACKs between the host and a device so temporally there may be a "many to 1" relationship. Bursts are managed on a Stream pipe identically to how they are managed on a normal Bulk pipe. Refer to Section 8.10.2 for more information on Burst Transactions.

NOTE As described in this section, the Stream Protocol applies to the state of the "pipe" and is described as single entity. In reality, the Stream Protocol is being tracked independently by the host at one end of the pipe and the device at the other. So at any instant in time the two ends may momentarily be out of phase due to packet propagation delays between the host and the device.

NOTE If a Retry is requested and the host cannot continue retransmission of a DP during the current burst, the host shall return to the endpoint at the next available opportunity within the constraints of the transfer type.



U-117A
IEC

Figure 8-38 – General Stream Protocol State Machine (SPSM)

Figure 8-38 illustrates the basic state transitions of the Stream Protocol State Machine (SPSM). This section describes the general transitions of the SPSM as they apply to both IN and OUT endpoints. Detailed operation of the SPSM for IN and OUT endpoints is described in subsequent sections.

Disabled – This is the initial state of the pipe after it is configured, as well as the state that is transitioned to if an error is detected in any of the other states. The first time an Endpoint Buffer is assigned to the pipe, the host shall transition the SPSM to the Prime Pipe state. If the **Disabled** state was entered due to an error, then the error condition must be removed by software intervention before the state may be exited.

Note that an error (e.g., Stall) or a SetFeature(ENDPOINT_HALT) request shall transition any SPSM state to the **Disabled** state. If an error condition is detected by the host (e.g., Stall, tHostTransactionTimeouts, etc.), the host shall transition its SPSM for the endpoint to the Disabled state. In the case where the host detects an error, not asserted by device (e.g., tHostTransactionTimeouts), the host shall transition the device's SPSM to the Disabled state by issuing a SetFeature(ENDPOINT_HALT) request to the device.

Prime Pipe – A transition to this state is always initiated by host, and informs a device that an Endpoint Buffer set has been added or modified by software. After exiting this state, any Active Stream IDs previously considered Not Ready by the device shall now be considered Ready.

NOTE To minimize bus transactions, the host controller limits transitions to the Prime Pipe state to one transition per Idle state entry. This means that while in the Idle state only a single transition to the Prime Pipe state will be generated even if Endpoint Buffers for multiple streams become ready. And since the Prime Pipe state does not specify which Stream(s) are ready, all Active Stream IDs are set to Ready by a Prime Pipe. The device is responsible for testing all Active Stream IDs (as described above) by sending the appropriate ERDYs after returning to Idle. Note that Device Class defined constraints may be used to limit the number of Active Stream IDs that need to be tested at any point in time.

Idle – A transition to this state indicates that there is no Current Stream (CStream) selected. In this state, the SPSM is waiting for a transition to Prime Pipe or Move Data initiated by the Host, or a transition to Start Stream initiated by the Device. The object of the Host and Device Initiated transitions is to start moving data for a Stream. A host initiated transition to Move

Data is referred to as a Host Initiated Move Data or HIMD. All Active Stream IDs are set to Ready by a HIMD.

Start Stream – This state is always initiated by a Device, and informs the host that the device wants to begin moving data on a selected Stream. The device may initiate a transition to this state anytime it has a Ready Stream ID. If the device selected Stream is accepted by the host, then the pipe enters the Move Data state. If the device selected Stream ID is rejected by the host, the pipe returns to Idle state and the selected Stream ID shall temporarily be considered Not Ready by the device. Note that a device maintains a list of the "Active" Stream IDs. An Active Stream ID may be Ready or Not Ready. The device is informed of the Active Stream IDs by the host through an out-of-band mechanism (typically a separate OUT endpoint).

Move Data – In this state, Stream data is transferred. The Current Stream is set when the SPSM transitions to this state. The SPSM transitions to the Idle state when the Stream transfer is complete, or if the host or device decides to terminate the Stream transfer because they have temporarily exhausted their data or buffer space. The transition to Idle invalidates the Current Stream for the pipe.

NOTE The general rule is that a Stream state machine advances only due to the reception of a good DP or TP. For example, if a DP is received with a bad DPP, a Stream state machine shall perform any retries in the current state, and advance only if a good packet is transferred.

8.12.1.4.1 Stream IDs

A 16-bit field *Stream ID* field is reserved in DP headers and in ACK, NRDY, and ERDY TPs for passing SIDs between the host and a device. Specific SID values that are reserved by the Stream Protocol and other SID notations are:

- **NoStream** – This SID indicates that no Stream ID is associated with the respective bus packet and the Stream ID field should not be interpreted as referencing a valid Stream. The *NoStream* SID value is FFFFh.
- **Prime** – This SID is used to define transitions into and out of the Prime Pipe state. As with *NoStream*, no Stream ID is associated with the respective bus packet and the Stream ID field should not be interpreted as referencing a valid Stream. The *Prime* SID value is FFFEh.
- **Stream *n*** – Where *n* is a value between 1 and 65 533 (FFFDh). This notation is used to reference a valid Stream ID. The Stream ID field in the packet header is valid if it uses this notation. Valid *Stream n* SID values are between 1 and 65 533 (FFFDh), where the numeric value is identical to *n*.
- **Stream 0** – This value is reserved and not used by a pipe that supports Streams. The *Stream 0* SID value is 0000h. Its use is required by a standard bulk pipe.
- **CStream** – represents the value of the "Current" Stream ID assigned to the pipe. A *CStream* value is maintained by both the host and a device. The Stream Protocol ensures that the *CStream* values are consistent in the host and the device. Valid values are *NoStream* or *Stream n*.
- **LCStream** – represents the value of the CStream SID assigned to the pipe before the last state transition. An *LCStream* value is maintained by the host. Valid values are *Prime*, *NoStream*, or *Stream n*. For example, while the pipe in the Move Data state *CStream* = *Stream n*, when the pipe transitions from Move Data to Idle state, *LCStream* is set to *Stream n*, and *CStream* is set to *NoStream*, thus *LCStream* records the "Last CStream" value.

Stream n SID values are assigned by the host and passed to a device (typically through an out-of-band, Device Class defined method). The value of a *Stream n* SID shall be treated as a "logical value" by a device, i.e., the device should not infer any meaning from the value or modify it.

NOTE The Bulk IN and OUT Stream Protocols below describe simplified state machines that do not explicitly detail the burst feature of Enhanced SuperSpeed endpoints which allows DPs to be sent without receiving an ACK. An implementation shall extend these state machines to manage bursting.

The following Sections (8.12.1.4.2 to 8.12.1.4.5) separate the Stream state machines into four cases for the device and host ends of a Stream pipe. Sections 8.12.1.4.2 and 8.12.1.4.3 describe the device end state machines. Sections 8.12.1.4.4 and 8.12.1.4.5 describe the host end state machines. And for each end of the pipe a separate section describes the respective IN and OUT operations.

The subsections in each Stream state machine section describe the state machine's respective states. The subsections begin with a description of the purpose and general characteristics of the state, followed by a discussion of each of the state's exit transitions. A paragraph that describes a state's exit transition is preceded with a unique *condition* or *action* label of the associated exit transition in the previous state diagram figure.

NOTE The U1 or U2 Timeouts in the path between the host and a device should be set to values that will prevent a transition to a U1 or U2 state for normal responses to Data Transactions. Refer to Section 0 for more Data Transaction timing information.

NOTE In the Stream state machine sections, the state names are overloaded, e.g., The **Idle** state is defined in all four state machine descriptions. The **INMvData Host** state is defined in both the device and host IN state machine sections, etc. The states are related in that they may occur at either end of a Stream pipe; however, each Stream state machine section describes an independent state machine, so the conditions and actions associated with the states are distinct in each section.

NOTE A transition condition that is italicized shall be interpreted as a comment, not a required condition. For example, the "*Stream n Active and Ready*" text of the **Idle** to **Start Stream** transition of Figure 8-39.

NOTE Any *CStream* data payload may be zero-length. The use of zero-length DPs on a Stream pipe (other than for Prime Pipe or Start Stream reject operations) is defined by the Device Class associated with the endpoint.

NOTE An IN Data or Burst Transaction is terminated with an ACK TP with NumP = 0. This ACK TP is referred to as a "Terminating ACK" in the following sections.

8.12.1.4.2 Device IN Stream Protocol

This section defines the Enhanced SuperSpeed packet exchanges that transition the device side of the Stream Protocol from one state to another on an IN bulk endpoint.

In the following text, a Device IN Stream state transition is assumed to occur at the point the device sends the first bit of the first symbol of a state machine related message to the host, or at the point the device first decodes state machine related message from the host.

For an IN pipe, Endpoint Buffers in the host receive Function Data from a device.

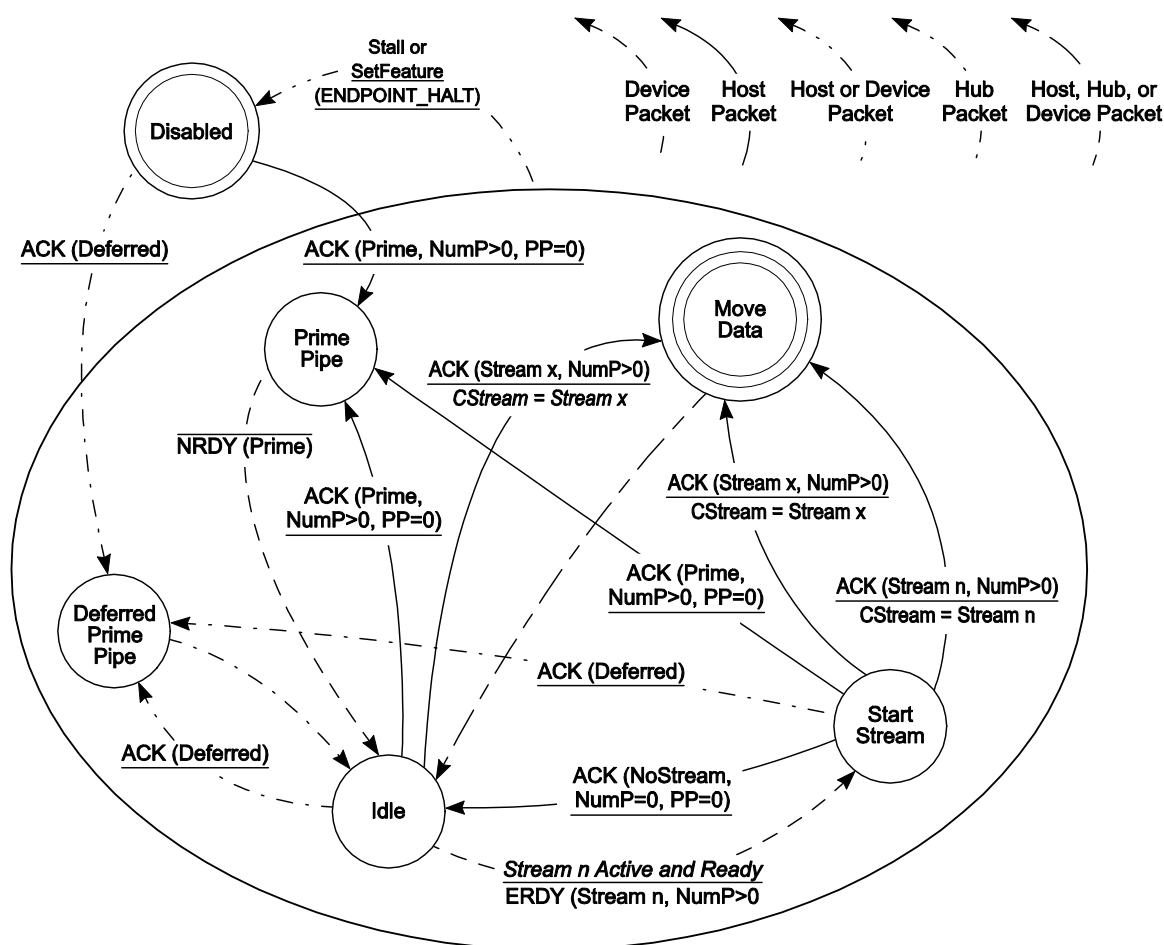
U-175
IEC

Figure 8-39 – Device IN Stream Protocol State Machine (DISPSM)

8.12.1.4.2.1 Disabled

After an endpoint is configured or receives a SetFeature(ENDPOINT_HALT) request, the pipe is in the **Disabled** state.

ACK(Prime, NumP>0, PP=0) – If an ACK TP with the Stream ID field set to *Prime* is received, then the device shall transition the pipe to the **Prime Pipe** state. This transition occurs after the initial Endpoint Buffers are assigned to the pipe by system software.

ACK(Deferred) – If an ACK with the Deferred (DF) flag set is received, then the device shall transition the pipe to the **Deferred Prime Pipe** state. This packet is received when the link has transitioned to a U1 or U2 state while waiting for the initial Endpoint Buffer assignment.

8.12.1.4.2.2 Prime Pipe

The **Prime Pipe** state informs the device that the Endpoint Buffers have been assigned to one or more Streams; however, it does not specify which Stream(s). In this state, the device shall set all Active Streams to Ready. After returning to the **Idle** state the device shall issue an ERDY to start a specific Stream from its list of Active Streams.

NRDY(Prime) – Upon entering the **Prime Pipe** state, the device shall generate an NRDY TP with its Stream ID field set to *Prime* and transition to the **Idle** state.

8.12.1.4.2.3 Deferred Prime Pipe

The **Deferred Prime Pipe** state informs the device that the Endpoint Buffers have been assigned to one or more Streams; however, the link has transitioned to a U1 or U2 state while waiting. In this state, the device shall set all Active Streams to Ready. After returning to the **Idle** state the device shall issue an ERDY to start a specific Stream from its list of Active Streams.

No Condition – Upon entering the **Deferred Prime Pipe** state, the device shall immediately transition to the **Idle** state. This is the only **Deferred Prime Pipe** exit transition in Figure 8-39.

8.12.1.4.2.4 Idle

In the **Idle** state, the pipe is waiting for a Stream selection (e.g., a transition to **Start Stream** or **Move Data**) or a notification from the host that a Stream Endpoint Buffer has been added or modified for the pipe (i.e., transition to **Prime Pipe**). Note that upon the initial entry in to **Idle** (i.e., from **Disabled**), only the device may initiate a Stream selection.

ERDY(Stream n , NumP>0) – To initiate a Stream selection, the device generates an ERDY TP with its Stream ID set to *Stream n* and a NumP value > 0, and transitions to the **Start Stream** state, where *Stream n* is the Stream ID proposed by the device. A device may initiate this transition when it wishes to start a Stream transfer, regardless of whether the pipe is in a flow control condition or not. The device maintains a list of *Active and Ready* Streams that it may generate ERDYs for. The method that a device uses for Stream selection is outside the scope of this specification and is normally defined by the Device Class associated with the pipe. Note that the value of the ERDY NumP field reflects the amount of Endpoint Data the device has available for *Stream n* .

ACK(Prime, NumP>0, PP=0) – If an ACK TP with a Stream ID equal to *Prime* is received from the host, the device shall transition to the **Prime Pipe** state.

ACK(Stream x , NumP>0) – With this transition the host proposes the Stream ID *Stream x* to the device. If an ACK TP with a Stream ID not equal to *Prime* is received from the host, the device shall transition to the **Move Data** state. The host may initiate this transition when it wishes to start a Stream transfer and is referred to as a *Host Initiated Move Data* or **HIMD**. A HIMD indicates the specific Stream that the Endpoint Buffer had been changed for. The device shall set *Stream x* to Ready due to this transition. After entering the **Move Data** state, the device may reject the proposed Stream with an NRDY or accept the proposed Stream with a DP. Upon transitioning to the **Move Data** state the device sets *CStream* to the value of the received Stream ID (*Stream x*). Typically *Stream x* will be equal to the Stream ID (*Stream n*) in the last ERDY generated by the device. *Stream x* may not be equal to *Stream n* if one of the race conditions described below occurs, because the host drops ERDYs under these conditions. PP should equal 1.

ACK(Deferred) – If an ACK with the Deferred (DF) flag set is received, then the device shall transition the pipe to the **Deferred Prime Pipe** state. This packet is received when the link has transitioned to a U1 or U2 state and the host has attempted a HIMD.

8.12.1.4.2.5 Start Stream

In the **Start Stream** state, the device is waiting for the host to accept or reject the Active and Ready Stream selection that it has proposed.

ACK(Stream n , NumP>0) – If an ACK TP with a Stream ID equal to *Stream n* is received, the host has accepted the device's proposal for starting *Stream n* and the device shall transition to the **Move Data** state. Upon transitioning to the **Move Data** state the device sets *CStream* to the value of the received Stream ID (*Stream n*). PP should equal 1.

ACK(NoStream, NumP=0, PP=0) – If an ACK TP with a Stream ID equal to *NoStream* is received, the host has rejected the device's proposal for starting *Stream n* and the device shall transition to the **Idle** state. The device shall set *Stream n* to Not Ready due to this transition. The host shall reject a proposal from a device if there are no Endpoint Buffers available for it.

ACK(Prime, NumP>0, PP=0) – If an ACK TP with a Stream ID equal to *Prime* is received, a race condition has occurred. The host has entered the **Prime Pipe** state to inform the device that the Endpoint Buffers for one or more Streams have been updated, at the same time that the device has attempted to initiate a Stream transfer, and their respective messages have passed each other on the link. During this condition, the device is in the **Start Stream** state and the host is in the **Prime Pipe** state. To resolve this condition, the device shall transition to the **Prime Pipe** state.

ACK(Stream *x*, NumP>0) – If an ACK TP with a Stream ID equal to *Stream x* is received, a race condition has occurred. The host has entered the **Move Data** state to initiate a transfer on *Stream x*, at the same time that the device has attempted to initiate a transfer on *Stream n*, and their respective messages have passed each other on the link. During this condition, the device is in the **Start Stream** state and the host is in the **Move Data** state. To resolve this condition, the device shall transition to the **Move Data** state. The device shall set *Stream x* to Ready due to this transition. Upon transitioning to the **Move Data** state the device sets *CStream* to the value of the received Stream ID (*Stream x*). PP should equal 1.

ACK(Deferred) – If an ACK with the Deferred (DF) flag set is received, then the device shall transition the pipe to the **Deferred Prime Pipe** state. This packet is received when the link has transitioned to a U1 or U2 state while waiting for a host response to the Start Stream request. Note that this transition can occur only if the tERDYTimeout has been exceeded.

NOTE The statement “PP should equal 1” in the **Idle** and **Start Stream** states, does not require the device to verify that PP equals 1 for the respective transition; however, if a device does check the condition it should halt the EP if PP is not equal to 1.

8.12.1.4.2.6 Move Data

In the Device IN **Move Data** state, *CStream* is set to the same value at both ends of the pipe and the pipe may actively move data. The details of the bus transactions executed in the **Move Data** state and its exit conditions are defined in the Device IN Move Data State Machine defined below.



may generate this transition upon initial entry into the DIMDSM to reject a HMD, or during a Stream transfer due to unexpected internal conditions where it wants to flow control *CStream*.

8.12.1.4.2.8 INMvData Host

In this state the device has just sent a DP to the host and has more Function Data available for *CStream*. The device waits in this state for an acknowledgement from the host for the last DP that it sent.

ACK(*CStream*, NumP>0, PP=1) – If the device receives an ACK with NumP > 0 and PP = 1, then it shall transition to the **INMvData Device** state. This is the host response if the current burst is not complete and it has more Endpoint Buffer space available for a *CStream* DP from the device. Note that the Retry (Rty=1) flag may be set in this packet if the host detected an error in the last DP from the device. If Rty is set, then the device shall return the DP with the appropriate Sequence Number the next time it sends a DP. If a DP error is detected, the host may continue the current burst until all retries are exhausted or a good DP is received. If the host cannot continue the current burst, the host shall initiate another burst to this endpoint at the next available opportunity within the constraints of the transfer type.

ACK(*CStream*, NumP=0, PP=1) – If the device receives an ACK with NumP = 0 and PP = 1, then it shall transition to the **INMvData Burst End** state. This is the host response if it has more Endpoint Buffer space available for another *CStream* DP; however, it must terminate the current burst from the device. Note that during the **INMvData Host** to **INMvData Device** transitions, the device should see NumP decrement towards 0 as the burst reaches completion. Note that the Retry (Rty=1) flag may be set in this packet if the host detected an error in the last DP from the device.

ACK(*CStream*, NumP=0, PP=0) – If the device receives an ACK with NumP = 0, and PP = 0, then it shall transition to the **Idle** state, exiting the DIMDSM. This is the host response to a DP when it has accepted the last DP because it has exhausted its *CStream* Endpoint Buffer space. The device shall set *CStream* to Not Ready due to this transition. During the **INMvData Host** to **INMvData Device** transitions, the device should see NumP decrement towards 0 as the Endpoint Buffer is exhausted.

NOTE Receiving an ACK with NumP > 0 and PP = 0 is an illegal combination in the **INMvData Host** state and the device should halt the EP if detected.

8.12.1.4.2.9 INMvData Device Terminate

This state is entered because the device has just sent the last DP that it has available for *CStream*, e.g., it has exhausted its *CStream* Function Data. In this state the device waits for an acknowledgement from the host for the last DP of the **Move Data** transfer.

ACK(*CStream*, NumP=0, No Rty) – If the device receives an ACK with NumP = 0 and Rty = 0, then it shall transition to the **Idle** state, exiting the DIMDSM. This is the normal host response (Terminating ACK) for acknowledging the successful reception of the last DP for *CStream* from the device.

ACK(*CStream*, NumP>0, PP=1, Rty) – If the device receives an ACK with Rty = 1, then it shall transition to the **INMvData Device** state and resend the appropriate DP. This is the host response if an error was detected on the DP from the device and the burst was not complete.

ACK(*CStream*, NumP=0, PP=1, Rty) – If the device receives an ACK with Rty = 1 and NumP = 0, then it shall transition to the **INMvData Burst End** state and wait for the host to initiate the next burst. This is the host response if an error was detected on the DP from the device but the burst was complete. The host shall continue the retry process in the next burst.

8.12.1.4.2.10 INMvData Burst End

This state is entered because the host has terminated a burst on a stream pipe. In this state the device waits for an ACK TP that signifies the start of another burst.

ACK(CStream, NumP>0, PP=1) – If the device receives an ACK with NumP > 0 and PP = 1, then it shall transition to the **INMvData Device** state. Note, if the Rty flag was set when the state was entered, then it shall be set upon exit.

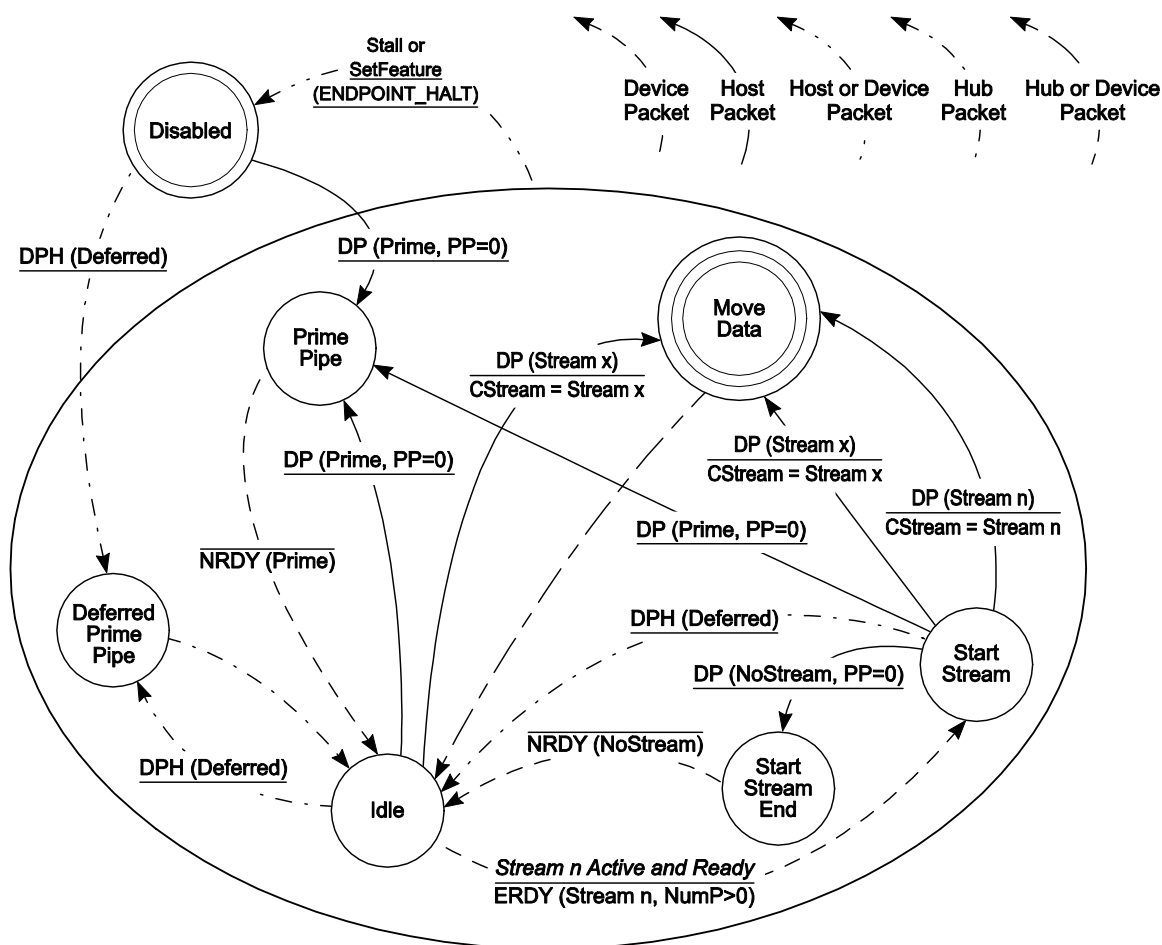
ACK(Deferred) – If an ACK with the Deferred (DF) flag set is received, then the device shall transition to the **Idle** state, exiting the DIMDSM. This transition occurs when the link has entered to a U1 or U2 state while waiting for the host to restart a burst, and this transition becomes more likely as the transfer activity associated with other devices increases.

8.12.1.4.3 Device OUT Stream Protocol

This section defines the Enhanced SuperSpeed packet exchanges that transition the device side of the Stream Protocol from one state to another on an OUT bulk endpoint.

In the following text, a Device OUT Stream state transition is assumed to occur at the point the device sends the first bit of the first symbol of a state machine related message to the host, or at the point the device first decodes a state machine related message from the host.

For an OUT pipe, Endpoint Data in the host is transmitted to Function Buffers in a device. Unless otherwise stated, a DP will contain Endpoint Data.



U-177
IEC

Figure 8-41 – Device OUT Stream Protocol State Machine (DOSPSM)

8.12.1.4.3.1 Disabled

After an endpoint is configured or receives a SetFeature(ENDPOINT_HALT) request, the pipe is in the **Disabled** state.

DP(Prime, PP=0) – If a DP with the Stream ID field set to *Prime* is successfully received, then the device shall transition the pipe to the **Prime Pipe** state. The DPP shall contain a zero-length data payload. This transition occurs after the initial Endpoint Buffers are assigned to the pipe by system software. Note, if an error is detected in the DP data (even though it is zero-length) the device shall remain in the **Disabled** state, and issue ACK(Prime, NumP>0, Rty) packets, retrying until a DP(Prime) is successfully received. This case is not illustrated in the figure above.

DPH(Deferred) – If a DP with the Deferred (DF) flag set is received, then the device shall transition the pipe to the **Deferred Prime Pipe** state. This packet is received when the link has transitioned to a U1 or U2 state while waiting for the initial Endpoint Data assignment.

8.12.1.4.3.2 Prime Pipe

The **Prime Pipe** state informs the device that the Endpoint Data has been assigned to one or more Streams; however, it does not specify which Stream(s). After returning to the **Idle** state the device shall issue an ERDY to start a specific Stream from its list of Active and Ready Streams.

NRDY(Prime) – Upon entering the **Prime Pipe** state, the device shall return an NRDY TP with its Stream ID field set to *Prime* and immediately transition to the **Idle** state.

8.12.1.4.3.3 Deferred Prime Pipe

The **Deferred Prime Pipe** state informs the device that the Endpoint Data has been assigned to one or more Streams; however, the link has transitioned to a U1 or U2 state while waiting.

No Condition – Upon entering the **Deferred Prime Pipe** state, the device shall immediately transition to the **Idle** state.

8.12.1.4.3.4 Idle

In the **Idle** state, the pipe is waiting for a Stream selection (e.g., a transition to **Start Stream** or **Move Data**) or a notification from the host that Endpoint Data has been added or modified for the pipe (i.e., transition to **Prime Pipe**). Note that upon the initial entry in to **Idle**, only the device may initiate a Stream selection.

ERDY(Stream *n*, NumP>0) – To initiate a Stream selection, the device generates an ERDY TP with its Stream ID set to *Stream n* and a NumP value > 0, and transitions to the **Start Stream** state, where *Stream n* is the Stream ID proposed by the device. A device may initiate this transition when it wishes to start a Stream transfer, regardless of whether the pipe is in a flow control condition or not. The device maintains a list of *Active and Ready* Streams that it may generate ERDYs for. The method that a device uses for Stream selection is outside the scope of this specification and is normally defined by the Device Class associated with the pipe. Note that the value of ERDY NumP reflects the amount of Endpoint Buffer space the device has available for *Stream n*.

DP(Prime, PP=0) – If a DP with a Stream ID equal to *Prime* is successfully received, the device shall transition to the **Prime Pipe** state. The DPP shall contain a zero-length data payload. Note, if an error is detected in the DP data the device shall remain in the **Idle** state, and issue ACK(Prime, NumP>0, Rty) packets, retrying until a DP(Prime) is successfully received. This case is not illustrated in the Figure above. The DPP shall contain a zero-length data payload.

DP(Stream x) – With this transition the host proposes the Stream ID *Stream x* to the device. If a DP with a Stream ID not equal to *Prime* is received from the host, the device shall transition to the **Move Data** state. The host may initiate this transition when it wishes to start a Stream transfer and is referred to as a *Host Initiated Move Data* or **HIMD**. A HIMD indicates the specific Stream that the Endpoint Data had been changed for. The device shall set *Stream x* to Ready due to this transition. After entering the **Move Data** state, the device may reject the proposed Stream with an NRDY or accept the proposed Stream with an ACK TP. Upon transitioning to the **Move Data** state the device sets *CStream* to the value of the received Stream ID (*Stream x*). The DPP shall contain the first data payload for the Stream. Typically *Stream x* will be equal to the Stream ID (*Stream n*) in the last ERDY generated by the device. *Stream x* may not be equal to *Stream n* if one of the race conditions described below occurs, because the host drops ERDYS under these conditions.

DPH(Deferred) – If a DPH with the Deferred (DF) flag set is received, then the device shall transition to the **Deferred Prime Pipe** state. This packet may be received when the link has transitioned to a U1 or U2 state and the host has attempted a transition to **Prime Pipe** or **Move Data** (a HIMD).

8.12.1.4.3.5 Start Stream

In the **Start Stream** state, the device is waiting for the host to accept or reject the Active and Ready Stream selection that it has proposed.

DP(Stream n) – If a DP with a Stream ID equal to *Stream n* is received: the host has accepted the device's proposal for starting *Stream n* and provided the first packet of *Stream n* data, and the device shall transition to the **Move Data** state. Upon transitioning to the **Move Data** state the device sets *CStream* to the value of the received Stream ID (*Stream n*). The DPP shall contain the first data payload for *CStream*.

DP(NoStream, PP=0) – If a DP with a Stream ID equal to *NoStream* is successfully received, the host has rejected the device's proposal for starting *Stream n* and the device shall transition to the **Start Stream End** state. The DPP shall contain a zero-length data payload. The host shall reject a proposal from a device if there is no Endpoint Data available for the Stream. The device shall set *Stream n* to Not Ready due to this transition. Note, if an error is detected in the DP data the device shall remain in the **Start Stream** state, and issue ACK(NoStream, NumP>0, Rty) packets, retrying until a DP(NoStream) is successfully received. This case is not illustrated in the Figure above.

DP(Prime, PP=0) – If a DP with a Stream ID equal to *Prime* is received, a race condition has occurred. The host has entered the **Prime Pipe** state to inform the device that Endpoint Data for one or more Streams has been posted, at the same time that the device has attempted to initiate a Stream transfer, and their respective messages have passed each other on the link. The DPP shall contain a zero-length data payload. During this condition, the device is in the **Start Stream** state and the host is in the **Prime Pipe** state. To resolve this condition, the device shall transition to the **Prime Pipe** state. Note, if an error is detected in the DP data the device shall transition to the **Prime Pipe** state and perform any retries there.

DP(Stream x) – If a DP with a Stream ID not equal to *Stream n* , *Prime* or *NoStream* (e.g., equal to *Stream x*) is received, a race condition has occurred. The host has entered the **Move Data** state to initiate a transfer on *Stream x* , at the same time that the device has attempted to initiate a transfer on *Stream n* , and their respective messages have passed each other on the link. During this condition, the device is in the **Start Stream** state and the host is in the **Move Data** state. To resolve this condition, the device shall transition to the **Move Data** state. The device shall set *Stream x* to Ready due to this transition. Upon transitioning to the **Move Data** state the device sets *CStream* to the value of the received Stream ID (*Stream x*). The DPP shall contain the first data payload for *CStream*. The device may accept or reject the Stream proposed by the host when in the **Move Data** state.

DPH(Deferred) – If a DPH with the Deferred (DF) flag set is received, then the device shall transition the pipe to the **Idle** state. This packet is received when the link has transitioned to a

8.12.1.4.3.6 Start Stream End

NRDY(NoStream) – The device shall generate an NRDY with the Stream ID equal to *NoStream* and transition to the **Idle** state.

8.12.1.4.3.7 Move Data

The diagram illustrates the OOB protocol sequence with the following components and transitions:

- States:**
 - OUTMvData Host** (bottom circle)
 - OUTMvData Host Terminate** (top circle)
 - OUTMvData Device** (bottom right circle)
 - Hub** (central point with arrows)
- Transitions and Messages:**
 - OUTMvData Host** to **OUTMvData Host Terminate**: DP (CStream, PP=0)
 - OUTMvData Host Terminate** to **OUTMvData Host**: DP (CStream, PP=1)
 - OUTMvData Host** to **OUTMvData Device**: DP (CStream, PP=1)
 - OUTMvData Device** to **OUTMvData Host**: ACK (CStream, NumP>0)
 - OUTMvData Device** to **OUTMvData Host Terminate**: ACK (CStream, NumP>0)
 - OUTMvData Device** to **Hub**: NRDY (CStream)
 - Hub** to **OUTMvData Device**: ACK (CStream, NumP=0)
 - Hub** to **OUTMvData Host Terminate**: ACK (CStream, NumP=0)
 - Hub** to **OUTMvData Host Terminate**: NRDY (CStream)
 - Hub** to **OUTMvData Device**: DPH (Deferred)
- External Elements:**
 - PP=0** and **PP=1** are shown as inputs to the **OUTMvData Host** state.
 - At the top right, three dashed arrows point away from the **Hub** labeled **Device Packet**, **Host Packet**, and **Hub Packet**.

Figure 8-42 – Device OUT Move Data State Machine (DOMDSM)

The Device OUT Move Data State Machine (DOMDSM) is entered from the **Start Stream** or **Idle** states as described above.

The DOMDSM allows either the device to terminate the Move Data operation because it has exhausted its Function Buffer space associated with a Stream or the host to terminate the Move Data operation because it has exhausted its Endpoint Data associated with a Stream.

PP = 0 – Upon entry into the DOMDSM, if the host has only one packet of Endpoint Data available for the Stream then PP will equal 0 in the first DP received by the Device, and it shall transition to the **OUTMvData Device Terminate** state.

PP = 1 – Upon entry into the DOMDSM, if the host has more than one packet of Endpoint Data available for the Stream then PP will equal 1 in the first DP received by the Device, and it shall transition to the **OUTMvData Device** state.

The DOMDSM always exits to the **Idle** state. The Retry (Rty=1) flag shall never be set in a packet that causes a DOMDSM exit. A Stream pipe remains in the **Move Data** state during packet retries.

NOTE The *Stream ID* value shall be *CStream* for all packets exchanged in the **Move Data** state. If a *Stream ID* value other than *CStream* is detected while in the DOMDSM the device should halt the endpoint.

NOTE if *CStream* is not Active upon initially entering the **Move Data** state, the device may reject the Stream proposal with an NRDY or STALL the pipe, as defined by the associated Device Class.

8.12.1.4.3.8 OUTMvData Device

This state is initially entered from the **Start Stream** state or the **Idle** state. In this state the device acknowledges the last DP sent by the host or it may reject a HIMD from the host.

ACK(*CStream*, NumP>0) – If the device has more Function Buffer space available for *CStream*, then it shall send an ACK TP to the host with NumP > 0 and transition to the **OUTMvData Host** state. Note that the Retry (Rty) flag may be set in this packet if the device detected an error in the last DP from the host. The host shall continue the current burst until all retries are exhausted or a positive acknowledgement (Rty=0) is received. This transition shall indicate that the data payload of the previously received DP has been accepted by the endpoint for *CStream*.

ACK(*CStream*, NumP=0) – If the device has no more Endpoint Buffer space available for *CStream*, then it shall generate an ACK TP with NumP = 0, exit the DOMDSM and transition to the **Idle** state. This transition allows the device to exit from the **Move Data** state if its Endpoint Buffer space is exhausted. This transition shall indicate that the data payload of the previously received DP has been accepted by the endpoint for *CStream*.

NRDY(*CStream*) – The device may also terminate further *CStream* transfers by sending an NRDY with its Stream ID set to *CStream*, transitioning to the **Idle** state, exiting the DOMDSM. The device may generate this transition upon initial entry into the DOMDSM to reject a HIMD, or during a Stream transfer due to unexpected internal conditions where it wants to flow control *CStream*. This transition shall indicate that the data payload of the previously received DP has been dropped.

8.12.1.4.3.9 OUTMvData Host

In this state the host has just received an ACK TP from the device for a previous DP and has more Endpoint Data available for *CStream*. The host generates a DP in this state. The pipe will also wait in this state between bursts from the host.

DP(*CStream*, PP=1) – If the device receives a DP with PP = 1, then it shall transition to the **OUTMvData Device** state. The DPP shall contain a *CStream* data payload. This is the host response if it has more than one Max Packet Size of Endpoint Data available for *CStream*.

DP(*CStream*, PP=0) – If the device receives a DP with PP = 0, then it shall transition to the **OUTMvData Host Terminate** state. The DPP shall contain a *CStream* data payload. This is

the host response if it has exhausted the Endpoint Data that it has available for *CStream*. The length of the DP will be less than or equal to one Max Packet Size.

DPH(Deferred) – If a DPH with the Deferred (DF) flag set is received, then the device shall transition to the **Idle** state, exiting the DOMDSM. This packet is received when the link has transitioned to a U1 or U2 state while waiting for the next DP from the host. There is no DPP associated with a deferred DPH.

8.12.1.4.3.10 OUTMvData Host Terminate

This state is entered because the host has just sent the last DP that it has available for *CStream*, e.g., it has exhausted its *CStream* Endpoint Data. In this state the device acknowledges the last DP from the host for the **Move Data** transfer.

ACK(*CStream*, NumP=0) – If the device has also exhausted its Function Buffer space, then it shall generate an ACK TP with NumP = 0 and transition to the **Idle** state, exiting the DOMDSM.

ACK(*CStream*, NumP>0) – If the device has not exhausted its Function Buffer space, then it shall generate an ACK TP with NumP > 0, and transition to the **Idle** state, exiting the DOMDSM. The device shall set *CStream* to Not Ready due to this transition.

ACK(*CStream*, NumP>0, Rty) – If an error was detected on the last DP by the device, then it shall generate an ACK TP with NumP > 0 and Rty = 1, so that the host will retry the last DP. The device shall then transition to the **OUTMvData Host** state.

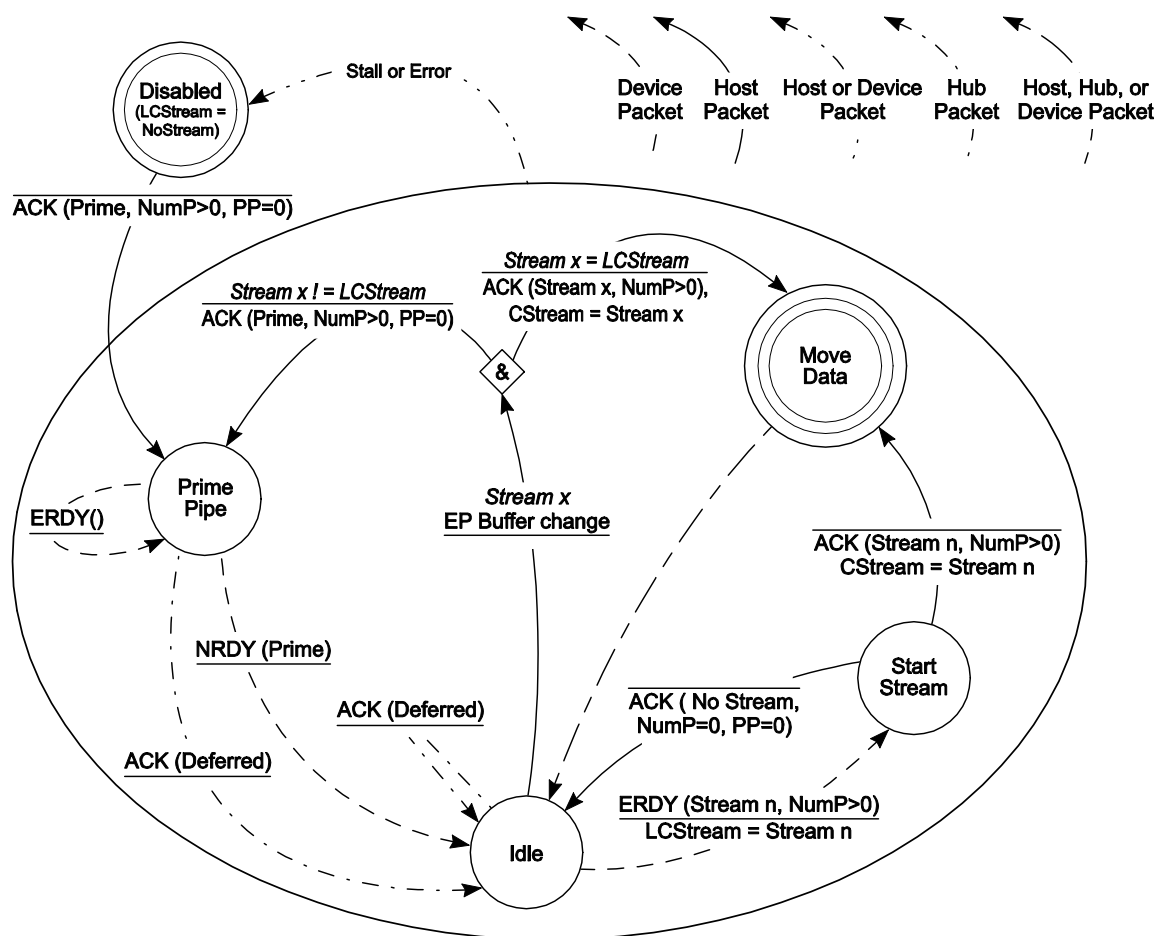
NRDY(*CStream*) – The device may flow control on the last *CStream* transfer by sending an NRDY with its Stream ID set to *CStream* and transition to the **Idle** state, exiting the DOMDSM. The device may generate this transition due to unexpected internal conditions where it wants to flow control *CStream*.

8.12.1.4.4 Host IN Stream Protocol

This section defines the Enhanced SuperSpeed packet exchanges that transition the host side of the Stream Protocol from one state to another on an IN bulk endpoint.

In the following text, a Host IN Stream state transition is assumed to occur at the point the host sends the first bit of the first symbol of a state machine related message to the device, or at the point the host first decodes state machine related message from the device.

For an IN pipe, Endpoint Buffers in the host receive Function Data from a device.



U-179
IEC

Figure 8-43 – Host IN Stream Protocol State Machine (HISPSM)

8.12.1.4.4.1 Disabled

After an endpoint is configured or an endpoint error condition (Stall, *tHostTransactionTimeout*, etc.) request, the pipe is in the **Disabled** state and *LCStream* is initialized to *NoStream*.

ACK(Prime, NumP>0, PP=0) – When the initial Endpoint Buffers are assigned to the pipe by system software, the host shall send an ACK TP with the Stream ID field set to *Prime* to the device, and transition the pipe to the **Prime Pipe** state.

8.12.1.4.4.2 Prime Pipe

The **Prime Pipe** state informs the device that the Endpoint Buffers have been assigned to one or more Streams.

NRDY(Prime) – If the host receives an NRDY TP with its Stream ID field set to *Prime*, it shall transition to the **Idle** state. This transition is the normal termination of a Prime Pipe operation.

ACK(Deferred) – If an ACK with the Deferred (DF) flag set is received, then the host shall transition the pipe to the **Idle** state. This packet may be received when the link has transitioned to a U1 or U2 state while the pipe was waiting for its initial Endpoint Buffer assignment. e.g. after an **ACK(Prime, NumP>0, PP=0)** has been generated in the **Disabled** state.

ERDY() – If an ERDY is received, a race condition has occurred. During this condition, the device is in the **Start Stream** state and the host is in the **Prime Pipe** state. The host has entered the **Prime Pipe** state to inform the device that the Endpoint Buffers for one or more

Streams have been updated, at the same time that the device has attempted to initiate a Stream transfer, and their respective messages have passed each other on the link. To resolve this condition, the host shall remain in the **Prime Pipe** state and wait for an NRDY(Prime) from the device.

8.12.1.4.4.3 Idle

In the **Idle** state, the pipe is waiting for a Stream selection (e.g., a transition to **Start Stream** or **Move Data**) or a notification from the host that Stream Endpoint Data has been added or modified for the pipe (i.e., transition to **Prime Pipe**). Note that upon the initial entry in to **Idle** (i.e., from **Disabled**), only the device may initiate a Stream selection.

ERDY(Stream n , NumP>0) – If an ERDY is received, the host shall transition to the **Start Stream** state. The device generates an ERDY to select a specific Stream (*Stream n*) that it expects the host to begin IN transactions on. A device may initiate this transition when it wishes to start a Stream transfer, regardless of whether it had previously flow controlled the pipe or not. Note that the value of the ERDY NumP field reflects the amount of Endpoint Data the device has available for *Stream n* . The value of the ERDY NumP is informative and the method that a device uses for Stream selection is outside the scope of this specification and is normally defined by the Device Class associated with the pipe. Upon transitioning to the **Start Stream** state the host sets *LCStream* to the value of *Stream n* .

ACK(Deferred) – If an ACK with the Deferred (DF) flag set is received, then the host shall remain in the **Idle** state. This packet is received if the link has transitioned to a U1 or U2 state when the host rejects a Start Stream request from the device (i.e., due to an ACK(NoStream, NumP=0, PP=0)). This case only occurs if tERDYTimeout is exceeded.

Stream x EP Buffer Change – This transition occurs if the state of one or more Endpoint Buffers has changed in the host. The host evaluates (at the Joint “&”) the ID of the Stream that software presents to the host controller (*Stream x*) and transitions to the **Prime Pipe** or **Move Data** states. This is an optimization that allows the host to transition the Stream pipe directly to the **Move Data** state, rather than going through the **Prime Pipe**, **Start Stream**, **Move Data** sequence, and is referred to as a *Host Initiated Move Data* or **HIMD**. The specific algorithm used to make this decision is host specific.

&

ACK(Prime, NumP>0, PP=0) – If the transition to **Prime Pipe** is selected, then the host shall generate a ACK TP with the Stream ID = *Prime*, NumP > 0, and PP = 0, and transition to the **Prime Pipe** state. Note that the host asserts a non-zero NumP value so that the device may respond with an NRDY. If NumP = 0, the device would consider it a Terminating ACK and not respond. Typically the **Prime Pipe** transition will be selected when the Stream that has just had its host Endpoint Buffers modified is not the same Stream that the device has last selected, e.g., *Stream x != LCStream*.

ACK(Stream x , NumP>0) – If the transition to **Move Data** is selected, then the host shall generate a ACK TP with the Stream ID = *Stream x* and NumP > 0, and transition to the **Move Data** state. Upon transitioning to the **Move Data** state the host sets *CStream* to the value of *Stream x* . Typically the **Move Data** transition will be selected when the Stream that has just had its Endpoint buffers modified is the same Stream as the one that the device last selected, e.g., *Stream x = LCStream*. PP shall equal 1 because the host is capable of receiving another DP from the device. This transition optionally may be disabled in some hosts, and some Device Classes may not process this transition (e.g., Mass Storage UASP).

8.12.1.4.4.4 Start Stream

In the **Start Stream** state, the device has sent an ERDY proposing to the host that it initiate an IN transfer for *Stream n* and it is waiting for the host to accept or reject the Stream selection.

ACK(Stream n , NumP>0) – If the host has accepted the device's proposal for starting *Stream n* , then it shall transmit an ACK TP with a Stream ID equal to *Stream n* , and transition to the **Move Data** state. Upon transitioning to the **Move Data** state the host sets *CStream* to the value of *Stream n* . The host shall accept a Stream proposal from a device if there are Endpoint Buffers available to receive the Function Data for the Stream. PP shall equal 1 because the host is capable of receiving another DP from the device.

ACK(NoStream, NumP=0, PP = 0) – If the host rejects the device's proposal for starting *Stream n* , then it shall transmit an ACK TP with a Stream ID equal to *NoStream*, and transition to the **Idle** state. The host shall reject a Stream proposal from a device if there are no Endpoint Buffers available to receive the Function Data for the Stream.

8.12.1.4.4.5 Move Data

In the Host IN **Move Data** state, *CStream* is set to the same value at both ends of the pipe and the pipe may actively move data. The details of the bus transactions executed in the **Move Data** state and its exit conditions are defined in the Host IN Move Data State Machine defined below.

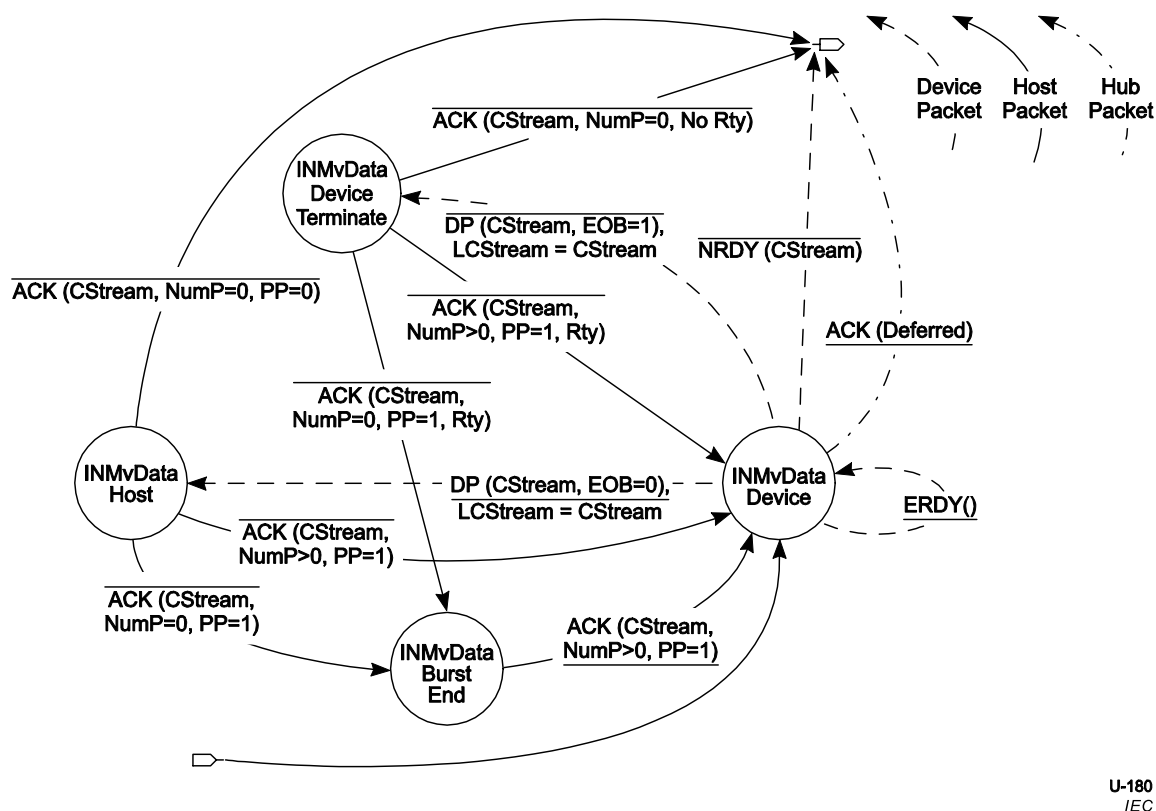


Figure 8-44 – Host IN Move Data State Machine (HIMDSM)

The Host IN Move Data State Machine (HIMDSM) is entered from the **Start Stream** or **Idle** states as described above. The entry into the HIMDSM immediately transitions to the **INMvData Device** state. The HIMDSM allows either the device to terminate the Move Data operation because it has exhausted its Function Data associated with a Stream or the host to terminate the Move Data operation because it has exhausted its Endpoint Buffer space associated with a Stream.

The HIMDSM always exits to the **Idle** state. The Retry (Rty=1) flag shall never be set in a packet that causes a HIMDSM exit. A Stream pipe remains in the Move Data state during packet retries.

NOTE The *Stream ID* value shall be *CStream* for all packets exchanged in the **Move Data** state, except the **INMvData Device** substate ERDY transition. For the identified substates, if a *Stream ID* value other than *CStream* is detected while in the HIMDSM the host should halt the endpoint.

8.12.1.4.4.6 INMvData Device

This state is initially entered from the **Start Stream** state or the **Idle** state. In this state the host is waiting for a DP from the device or a rejection of a HIMD.

DP(*CStream*, EOB=0) – If the host receives a DP with EOB = 0, it shall copy the DP data to the Endpoint Buffer associated with the Stream and transition to the **INMvData Host** state. The DPP shall contain a *CStream* data payload. This transition occurs when the device returns IN data and has more Function Data to send. Upon transitioning to the **INMvData Host** state the host sets *LCStream* to the value of *CStream*. This action updates *LCStream* with the value of *CStream* if the device accepts a HIMD, i.e., *LCStream* records the last Stream that was of interest to the device.

DP(*CStream*, EOB=1) – If the host receives a DP with EOB = 1, it shall copy the DP data to the Endpoint Buffer associated with the Stream and transition to the **INMvData Device Terminate** state. The DPP shall contain a *CStream* data payload. This transition occurs when device returns IN data and has no more Function Data to send, e.g., it is terminating the Move Data operation because this DP exhausts the Function Data available for this Stream. Upon transitioning to the **INMvData Device Terminate** state the host sets *LCStream* to the value of *CStream*. This action updates *LCStream* with the value of *CStream* if the device accepts a HIMD, i.e., *LCStream* records the last Stream that was of interest to the device.

NRDY(*CStream*) – If the host receives an NRDY, it shall exit the HIMDSM and transition to the **Idle** state. This transition may occur upon initial entry into the HIMDSM when the device rejects a HIMD, or during a Stream transfer due to unexpected internal device conditions where it wants to flow control *CStream*.

ACK(Deferred) – If the host receives an ACK with the Deferred (DF) flag set, then it shall exit the HIMDSM and transition to the **Idle** state. This packet shall be received if a link in the path between the host and the device has transitioned to a U1 or U2 state. There are two cases when this transition may occur: 1) the host has attempted a HIMD, and 2) between bursts. Case 1 is likely to occur if there has been a long host delay in obtaining buffers for the Stream. Case 2 may occur if there is a lot of endpoint activity on other devices delaying the time between bursts. The device treats this transition like a Prime Pipe and will send an ERDY to restart the stream when it receives the Deferred ACK forwarded to it by a hub.

ERDY() – If an ERDY is received, a race condition has occurred. During this condition, the device is in the **Start Stream** state and the host is in the **Move Data** state. The host has entered the **Move Data** state as the result of a HIMD, at the same time that the device has attempted to initiate a Stream transfer, and their respective messages have passed each other on the link. To resolve this condition, the host shall remain in the **INMvData Device** state and wait for a DP or an NRDY from the device.

8.12.1.4.4.7 INMvData Host

In this state the host has received a DP from a device that has more Function Data available for *CStream*. The host responds with an acknowledgement after copying the received data to the Endpoint Buffer space associated with the Stream.

ACK(*CStream*, NumP>0, PP=1) – If more Endpoint Buffer space is available for the Stream and the host is continuing the current burst to the device, then the host shall generate an ACK TP with NumP > 0 and PP = 1, and transition to the **INMvData Device** state. If the host detected an error on the last DP from the device, then the Rty flag shall be set. The host may continue the **INMvData Host** to **INMvData Device** loop until all retries are exhausted or a good packet is received by the device. If the current burst terminates before all retries are exhausted, the host may transition to the **INMvData Burst End** state (with Rty=1) and return

to the **INMvData Device** state (with *Rty*=1) at the next available opportunity to continue the retry process within the constraints of the endpoint type.

ACK(*CStream*, *NumP*=0, *PP*=1) – If more Endpoint Buffer space is available for the Stream; however, the host must terminate the current burst to the device, then the host shall generate an ACK TP with *NumP* = 0 and *PP* = 1, and transition to the **INMvData Burst End** state. If the host detected an error on the last DP from the device, then the *Rty* flag shall be set.

ACK(*CStream*, *NumP*=0, *PP*=0) – If the host did not detect an error on the last DP received from the device and the Endpoint Buffer space available for the Stream is exhausted, then the host shall generate an ACK TP with *NumP* = 0 and *PP* = 0, and transition to the **Idle** state, exiting the HIMDSM. This transition informs the device the host has exhausted its Endpoint Buffer space for the Stream.

8.12.1.4.4.8 INMvData Burst End

This state is entered because the host has terminated a burst on a stream pipe. The host will exit this state when it is ready to start another burst. If this state was entered while retrying (*Rty* = 1), then the host shall continue the retry process within the constraints of the endpoint when exiting the state.

ACK(*CStream*, *NumP*>0, *PP*=1) – When ready to start another burst to the device on *CStream*, the host shall generate an ACK with *NumP* > 0 and *PP* = 1 and transition to the **INMvData Device** state. Note, if the *Rty* flag was set when the state was entered, then it shall be set upon exit.

8.12.1.4.4.9 INMvData Device Terminate

In this state the host has received the last DP from a device for this Move Data operation because the device has exhausted the Function Data it has available for *CStream*. The host responds with an acknowledgement after copying the received data to the Endpoint Buffer space associated with the Stream and exits the HIMDSM. If the DP received from the device is bad, then retries may be performed within the constraints of the endpoint type.

ACK(*CStream*, *NumP*=0, No *Rty*) – If the DP received from the device is good, then the host generates an ACK with *NumP* = 0 and *Rty* = 0, and transitions to the **Idle** state, exiting the HIMDSM.

ACK(*CStream*, *NumP*>0, *PP*=1, *Rty*) – If the DP received from the device is bad and the current burst is not complete, then the host shall generate an ACK with *NumP* > 0, *PP* = 1 and *Rty* = 1, and transition to the **INMvData Device** state. The host may continue the **INMvData Device Terminate** to **INMvData Device** loop until all retries are exhausted or a good packet is received.

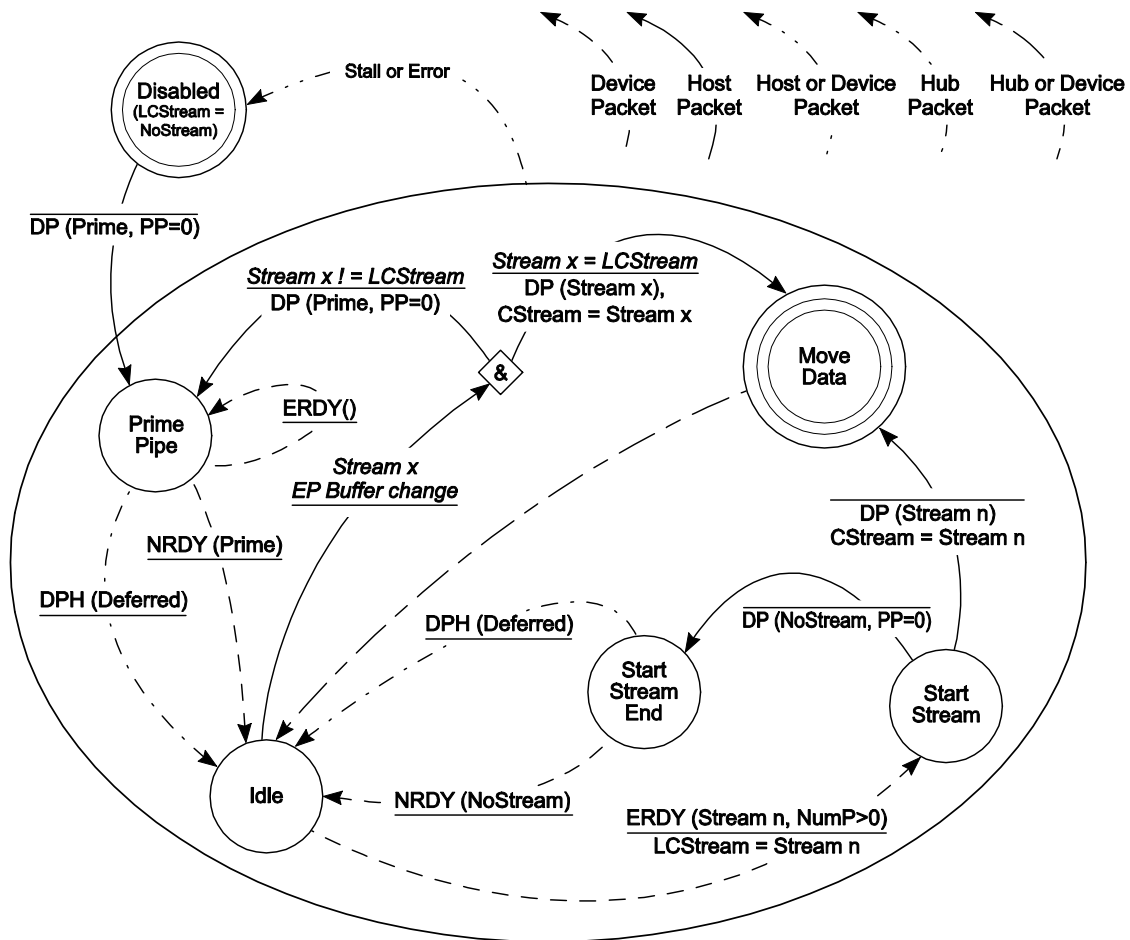
ACK(*CStream*, *NumP*=0, *PP*=1, *Rty*) – If the DP received from the device is bad and the current burst is complete, then the host shall generate an ACK with *NumP* = 0, *PP* = 1 and *Rty* = 1, and transition to the **INMvData Burst End** state. The host shall continue the retry process in the next burst.

8.12.1.4.5 Host OUT Stream Protocol

This section defines the Enhanced SuperSpeed packet exchanges that transition the host side of the Stream Protocol from one state to another on an OUT bulk endpoint.

In the following text, a Host OUT Stream state transition is assumed to occur at the point the host sends the first bit of the first symbol of a state machine related message to the device, or at the point the host first decodes state machine related message from the device.

For an OUT pipe, Function Buffers in the device receive Endpoint Data from the host.



U-181
IEC

Figure 8-45 – Host OUT Stream Protocol State Machine (HOSPSM)

8.12.1.4.5.1 Disabled

After an endpoint is configured or receives a SetFeature(ENDPOINT_HALT) request, the pipe is in the **Disabled** state and *LCStream* is initialized to *NoStream*.

DP(Prime, PP=0) – When the initial Endpoint Data is assigned to the pipe by system software, the host shall send a zero-length DP with the Stream ID field set to *Prime* to the device, and transition the pipe to the **Prime Pipe** state. The DPP shall contain a zero-length data payload.

8.12.1.4.5.2 Prime Pipe

The **Prime Pipe** state informs the device that Endpoint Buffers have been assigned to one or more Streams. Note, this state is entered when the host transmits a DP(Prime) from the **Disabled** or the **Idle** state. If an error is detected in the DP data by the device, the device shall issue ACK(Prime, NumP>0, Rty) packet, retrying until a DP(Prime) is successfully received. The host may retransmit the DP(Prime) and shall remain in the **Prime Pipe** state until the device successfully receives the DP(Prime) and returns an NRDY(Prime), or the retries for the pipe are exhausted and the host halts the pipe. This case is not illustrated in the Figure above.

NRDY(Prime) – If the host receives an NRDY TP with its Stream ID field set to *Prime*, it shall transition to the **Idle** state. This transition is the normal termination of a Prime Pipe operation.

DPH(Deferred) – If a DPH with the Deferred (DF) flag set is received, then the host shall transition the pipe to the **Idle** state. This packet may be received when the link has transitioned to the U1 or U2 state while the pipe waiting for its initial Endpoint Buffer

assignment, e.g., after a DP(Prime, PP=0) has been generated in the **Disabled** state. There is no DPP associated with a deferred DPH.

ERDY() – If an ERDY is received, a race condition has occurred. During this condition, the device is in the **Start Stream** state and the host is in the **Prime Pipe** state. The host has entered the **Prime Pipe** state to inform the device that the Endpoint Buffers for one or more Streams have been updated, at the same time that the device has attempted to initiate a Stream transfer, and their respective messages have passed each other on the link. To resolve this condition, the host shall remain in the **Prime Pipe** state and wait for an NRDY from the device.

8.12.1.4.5.3 Idle

In the **Idle** state, the pipe is waiting for a Stream selection (e.g., a transition to **Start Stream** or **Move Data**) or a notification from the host that Stream Endpoint Data has been added or modified for the pipe (i.e., transition to **Prime Pipe**). Note that upon the initial entry in to **Idle** (i.e., from **Disabled**), only the device may initiate a Stream selection.

ERDY(Stream n , NumP>0) – If an ERDY is received, the host shall transition to the **Start Stream** state. The device generates an ERDY to select a specific Stream (*Stream n*) that it expects the host to begin OUT transactions on. A device may initiate this transition when it wishes to start a Stream transfer, regardless of whether it had previously flow controlled the pipe or not. Note that the value of the ERDY NumP field reflects the amount of Endpoint Buffer space the device has available for *Stream n* . The method that a device uses for Stream selection is outside the scope of this specification and is normally defined by the Device Class associated with the pipe. Upon transitioning to the **Start Stream** state the host sets *LCStream* to the value of *Stream n* .

Stream x EP Buffer Change – This transition occurs if Endpoint Data has been posted for one or more Streams in the host. The host evaluates (at the Joint “&”) the ID of the Stream that software presents to the host controller (*Stream x*) and transitions to the **Prime Pipe** or **Move Data** states. This is an optimization that allows the host to transition the Stream pipe directly to the **Move Data** state, rather than going through the **Prime Pipe**, **Start Stream**, **Move Data** sequence, and is referred to as a *Host Initiated Move Data* or **HIMD**. The specific algorithm used to make this decision is host specific.

&

DP(Prime, PP=0) – If the transition to **Prime Pipe** is selected, then the host shall generate a DP with the Stream ID = *Prime* and PP = 0, and transition to the **Prime Pipe** state. The DPP shall contain a zero-length data payload. Typically, the **Prime Pipe** transition will be selected when the Stream that has just had its Endpoint Data modified is not the same Stream that the device has last selected, e.g., *Stream x != LCStream*.

DP(Stream x) – If the transition to **Move Data** is selected, then the host shall generate a DP with the Stream ID = *Stream x* , and transition to the **Move Data** state. Upon transitioning to the **Move Data** state the host sets *CStream* to the value of *Stream x* . The DPP shall contain the first data payload for *CStream*. Typically the **Move Data** transition will be selected when the Stream that has just had its Endpoint Data modified is the same Stream as the one that the device last selected, e.g., *Stream x = LCStream*. The value of PP shall depend on the amount of Endpoint data the host has available. If the host has more than Max Packet Size Endpoint Data available for the Stream, then PP = 1 else PP = 0. This transition may be optionally be disabled in some hosts, and some Device Classes may not process this transition (e.g., Mass Storage UASP).

8.12.1.4.5.4 Start Stream

In the **Start Stream** state, the device has sent an ERDY proposing to the host that it initiate an OUT transfer for *Stream n* and it is waiting for the host to accept or reject the Stream selection.

DP(Stream *n*) – If the host has accepted the device's proposal for starting *Stream n*, then it shall transmit a DP with a Stream ID equal to *Stream n*, and transition to the **Move Data** state. Upon transitioning to the **Move Data** state the host sets *CStream* to the value of *Stream n*. The DPP shall contain the first data payload for *CStream*. The host shall accept a Stream proposal from a device if there is Endpoint Data available for the Stream.

DP(NoStream, PP = 0) – If the host rejects the device's proposal for starting *Stream n*, then it shall transmit a DP with a Stream ID equal to *NoStream*, and transition to the **Start Stream End** state. The DPP shall contain a zero-length data payload. The host shall reject a Stream proposal from a device if there is no Endpoint Data available to send for the Stream.

8.12.1.4.5.5 Start Stream End

In the **Start Stream End** state, the host has rejected a proposed Stream ID from the device because there was no Endpoint Data available for *Stream n*. Note, this state is entered when the host transmits a DP(NoStream) from the **Start Stream** state. If an error is detected in the DP data by the device, the device shall issue ACK(NoStream, NumP>0, Rty) packet, retrying until a DP(NoStream) is successfully received. The host may retransmit the DP(NoStream) and shall remain in the **Start Stream End** state until the device successfully receives the DP(NoStream) and returns an NRDY(NoStream), or the retries for the pipe are exhausted and the host halts the pipe. This case is not illustrated in the Figure above.

NRDY(NoStream) – If an NRDY with the Stream ID equal to *NoStream* is received, the host shall transition to the **Idle** state.

DPH(Deferred) – If a DPH with the Deferred (DF) bit set is received, the host shall transition to the **Idle** state. This packet is received when the link has transitioned to a U1 or U2 state while waiting for a host response to the Start Stream request. Note that this transition can occur only if the tERDYTimeout has been exceeded. There is no DPP associated with a deferred DPH.

8.12.1.4.5.6 Move Data

In the Host OUT **Move Data** state, *CStream* is set to the same value at both ends of the pipe and the pipe may actively move data. The details of the bus transactions executed in the **Move Data** state and its exit conditions are defined in the Host OUT Move Data State Machine defined below.

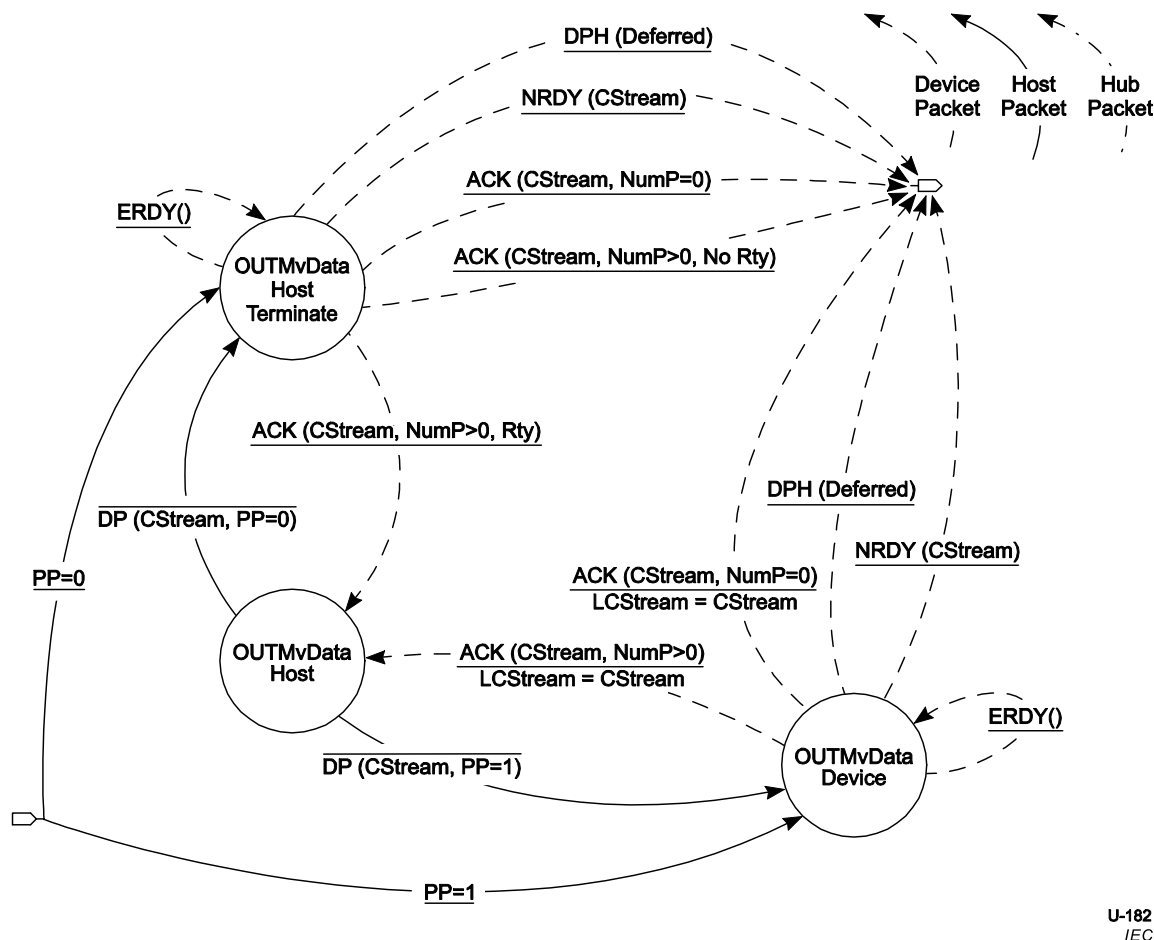


Figure 8-46 – Host OUT Move Data State Machine (HOMDSM)

The Host OUT Move Data State Machine (HOMDSM) is entered from the **Start Stream** or **Idle** states as described above. The entry into the HOMDSM immediately transitions to the **OUTMvData Device** state. The HOMDSM allows either the device to terminate the Move Data operation because it has exhausted its Function Buffer space associated with a Stream or the host to terminate the Move Data operation because it has exhausted its Endpoint Data associated with a Stream.

PP = 0 – Upon entry into the HOMDSM, if the host has only one packet of Endpoint Data available for the Stream then PP will equal 0 in the first DP sent to the Device, and it shall transition to the **OUTMyData Device Terminate** state.

PP = 1 – Upon entry into the HOMDSM, if the host has more than one packet of Endpoint Data available for the Stream then PP will equal 1 in the first DP sent to the Device, and it shall transition to the **OUTMyData Device** state.

The HOMDSM always exits to the **Idle** state. The Retry (Rty=1) flag shall never be set in a packet that causes a HOMDSM exit. A Stream pipe remains in the Move Data state during packet retries.

NOTE The *Stream ID* value shall be *CStream* for all packets exchanged in the **Move Data** state, except the **OUTMvData Device** substate ERDY transition. For the identified substates, if a *Stream ID* value other than *CStream* is detected while in the HOMDSM the host should halt the endpoint.

8.12.1.4.5.7 OUTMvData Device

This state is initially entered from the **Start Stream** state or the **Idle** state. In this state the host is waiting for an ACK TP or a rejection of a HIMD from the device.

ACK(CStream, NumP>0) – If the host receives an ACK TP with NumP > 0, it shall transition to the **OUTMvData Host** state. This transition occurs when device has more Function Buffer space available for the stream. If the device detected an error on the last DP from the host, then the Retry (Rty) flag shall be set. If a Retry is requested, the host shall continue the current burst until all retries are exhausted or a good packet is transmitted. The host shall set *LCStream* = *CStream*. This action updates *LCStream* with the value of *Stream* x if the device accepts a HIMD, i.e., *LCStream* records the last Stream that was of interest to the device.

ACK(CStream, NumP=0) – If the host receives an ACK TP with NumP = 0, it shall transition to the **Idle** state, exiting the HOMDSM. This transition occurs when device has no more Function Buffer space available for the Stream, e.g., it is terminating the Move Data operation because the last DP exhausted its Function Buffer space. The host shall set *LCStream* = *CStream*. This action updates *LCStream* with the value of *Stream* x if the device accepts a HIMD, i.e., *LCStream* records the last Stream that was of interest to the device.

NRDY(CStream) – If the host receives an NRDY, it shall transition to the **Idle** state, exiting the HOMDSM. This transition may occur upon initial entry into the HOMDSM when the device rejects a HIMD, or during a Stream transfer due to unexpected internal device conditions where it wants to flow control *CStream*.

DPH(Deferred) – If the host receives a DPH with the Deferred (DF) flag set, then it shall transition to the **Idle** state, exiting the HOMDSM. This packet may be received when the link has transitioned to a U1 or U2 state and the host has attempted a HIMD or between bursts on the OUT pipe, if there is a lot of endpoint activity on other devices and the Ux Timeouts in the path to this device are set to short values. When this transition occurs the host will wait in the **Idle** state for an ERDY from the device to restart the stream. There is no DPP associated with a deferred DPH.

ERDY() – If an ERDY is received, a race condition has occurred. During this condition, the device is in the **Start Stream** state and the host is in the **Move Data** state. The host has entered the **Move Data** state as the result of a HIMD, at the same time that the device has attempted to initiate a Stream transfer, and their respective messages have passed each other on the link. To resolve this condition, the host shall remain in the **OUTMvData Device** state and wait for an ACK or an NRDY from the device.

8.12.1.4.5.8 OUTMvData Host

In this state the host has received an ACK TP from a device and the device has more Function Buffer space available for *CStream*. The host responds with a DP containing Endpoint Data associated with the Stream. The pipe will also wait in this state between bursts from the host. Note, that the DP retry process may span bursts.

DP(CStream, PP=1) – If more Endpoint Data is available for the Stream and the host is continuing the current burst to the device, then the host shall generate a DP with PP = 1, and transition to the **OUTMvData Device** state. The DPP shall contain a *CStream* data payload. If the Rty flag was set in the last ACK from the device, then the host shall resend the appropriate DP until all retries are exhausted or a good DP is acknowledged by the device.

DP(CStream, PP=0) – If the Endpoint Data available for the Stream is exhausted by transmitting this DP, then the host shall generate a DP with PP = 0, and transition to the **OUTMvData Host Terminate** state. The DPP shall contain a *CStream* data payload. This transition informs the device the host has exhausted its Endpoint Data for the Stream.

8.12.1.4.5.9 OUTMvData Host Terminate

In this state the host has just exhausted the Endpoint Data that it has available for *CStream* and sent the last DP for the Stream. The host is waiting for an acknowledgement for the last DP of the Stream.

ACK(CStream, NumP=0) – If the host receives an ACK TP with NumP = 0 and Rty = 0, then the host shall transition to the **Idle** state, exiting the HOMDSM. This transition occurs when the device has successfully received the last DP, and both the host and the device have exhausted their respective Endpoint Data and Function Buffer space at the same time.

ACK(CStream, NumP>0, No Rty) – If the host receives an ACK TP with NumP > 0, PP = 0, and Rty = 0, then the host shall transition to the **Idle** state, exiting the HOMDSM. This transition occurs when the device has successfully received the last DP, and the host has exhausted its Endpoint Data for the *CStream*, but the device still has more Function Buffer space available.

ACK(CStream, NumP>0, Rty) – If the host receives an ACK TP with NumP > 0 and Rty = 1, then the host shall transition the **OUTMvData Host** state and resend the appropriate DP. This transition occurs when the last packet received by the device was bad, and a Retry is required. The host shall continue the **OUTMvData Host Terminate** to **OUTMvData Host** loop until all retries are exhausted or a good DP is acknowledged by the device.

NRDY(CStream) – If the host receives an NRDY, it shall transition to the **Idle** state, exiting the HOMDSM. This transition may occur during a Stream transfer due to unexpected internal device conditions where it wants to flow control *CStream*.

DPH(Deferred) – If a DPH with the Deferred (DF) flag set is received, then the host shall transition to the **Idle** state, exiting the HOMDSM. This packet is received when the link had transitioned to the U1 or U2 state before the last DP was sent by the host. There is no DPP associated with a deferred DPH.

8.12.2 Control Transfers

Control transfers have a minimum of two transaction stages: Setup and Status. A control transfer may optionally contain a Data stage between the Setup and Status stages. The direction of the Data stage is indicated by the **bmRequestType** field which is present in the first byte of the data payload of the Setup packet. During the Setup stage, a SETUP transaction is used to transmit information to a control endpoint of the device. SETUP transactions are similar in format to a Bulk OUT transaction but have the **Setup** field set to one in the DPH along with the **Data Length** field set to eight. In addition, the Setup packet always uses a Data sequence number of zero. A device receiving a Setup packet shall respond as defined in Section 8.11.4. The **Direction** field shall be set to zero in TPs or DPs exchanged between the host and any control endpoint on the device regardless of the stage or direction of the control transfer. The **TT** field shall be set to Control by hosts and devices operating above Gen 1 speed; see Table 8-13.

If the endpoint successfully received the SETUP packet, it may return an ACK TP with the **NumP** field set to zero if it wants to flow control the control transfer. A device shall send an ERDY when it is ready to resume the control transfer (either the Data or Status stage). Note that an endpoint may return an ACK TP with the **NumP** field set to zero in response to a SETUP packet if it wants to flow control the control transfer. A device must send an ERDY to start the Data or Status stage. Note that the host may resume transactions to any endpoint – even if the endpoint had not returned an ERDY TP after returning a flow control response.

The Data stage, if present, of a control transfer consists of one or more IN or OUT transactions and follows the same protocol rules as bulk transfers except that the **Direction** field shall always be set to zero. The Data stage always starts with the sequence number set to zero. All the transactions in the Data stage shall be in the same direction (i.e., all INs or all OUTs). The maximum amount of data to be sent during the data stage and its direction are specified during the Setup stage. If the amount of data exceeds the data packet size, the data is sent in multiple data packets that carry the maximum packet size. Any remaining data is sent as a residual in the last data packet.

Note that all control endpoints only support a burst of one and hence the host can only send or receive one packet at a time to or from a control endpoint.

The Status stage of a control transfer is the last transaction in the sequence. The status stage transaction is identified by a TP with the SubType set to *STATUS*. In response to a STATUS TP with zero in the **Deferred** bit, a device shall send an NRDY, STALL, or ACK TP. If a device sends an NRDY TP, the host shall wait for it to send an ERDY TP for that control endpoint before sending another STATUS TP to the device. However, the host may resume transactions to any endpoint – even if the endpoint had not returned an ERDY TP after returning a flow control response. If the **Deferred** bit is set in the STATUS TP, then the device shall send an ERDY TP to indicate to the host that is ready to complete the status stage of the control transfer.

Figure 8-47 and Figure 8-48 show the transaction order, the data sequence number value, and the data packet types for control read and write sequences.

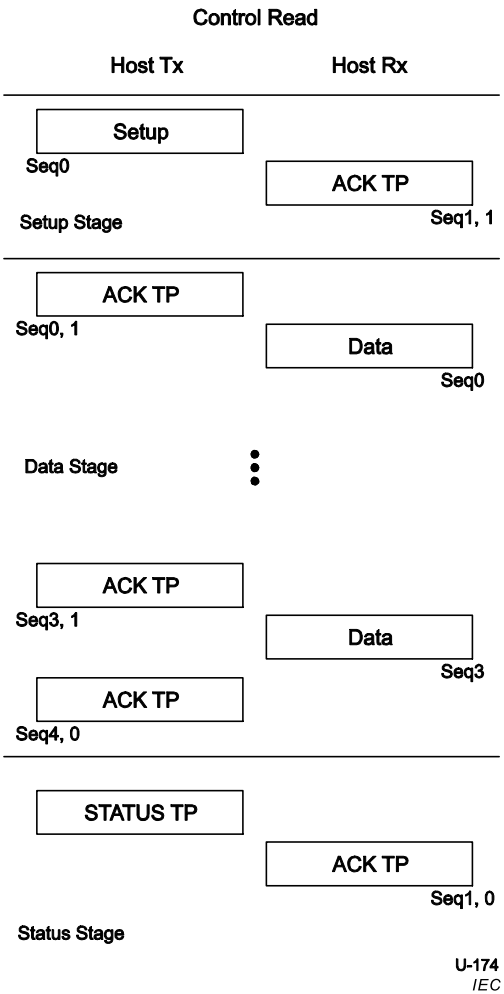


Figure 8-47 – Control Read Sequence

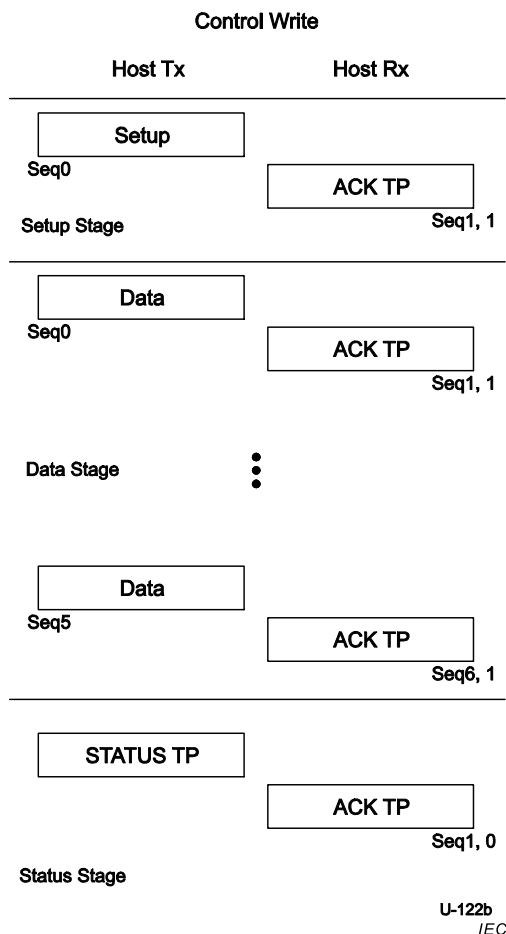


Figure 8-48 – Control Write Sequence

When a STALL TP is sent by a control endpoint in either the Data or Status stages of a control transfer, a STALL TP shall be returned on all succeeding accesses to that endpoint until a SETUP DP is received. An endpoint shall return an ACK TP when it receives a subsequent SETUP DP. For control endpoints, if an ACK TP is returned for the SETUP transaction, the host expects that the endpoint has automatically recovered from the condition that caused the STALL and the endpoint shall operate normally.

8.12.2.1 Reporting Status Results

During the Status stage, a device reports to the host the outcome of the previous Setup and Data stages of the transfer. Three possible results may be returned:

- The command sequence completed successfully.
- The command sequence failed to complete.
- The device is still busy completing the command.

Status reporting is always in the device-to-host direction. Table 8-31 summarizes the type of responses required for each. All Control transfers return status in the TP that is returned to the host in response to a STATUS TP transaction.

Note that even though the status reporting is always in the device-to-host direction, the STATUS TP shall be treated as an OUT transaction. A host may start performing IN transactions to another endpoint without waiting for the response for the STATUS TP.

Table 8-31 – Status Stage Responses

Status Response	TP Sent by Device
Request completes	ACK TP
Request has an error	STALL TP
Device is busy	NRDY TP

The host shall send a STATUS TP to the control pipe to initiate the Status stage. The pipe's handshake response to this TP indicates the current status. An NRDY TP indicates that a device is still processing the command and that the device shall send an ERDY TP when it completes the command. An ACK TP indicates that a device has completed the command and is ready to accept a new command. A STALL TP indicates that a device has an error that prevents it from completing the command.

The **NumP** field of the ACK TP sent by a control endpoint on the device shall be set to zero. However, this is not considered a flow control condition for a control endpoint.

If during a Data stage a control pipe is sent more data or is requested to return more data than was indicated in the Setup stage, it shall return a STALL TP. If a control pipe returns a STALL TP during the Data stage, there shall not be a Status stage for that control transfer.

8.12.2.2 Variable-length Data Stage

A control pipe may have a variable-length data phase in which the host requests more data than is contained in the specified data structure. When all of the data structure is returned to the host, a device indicates that the Data stage is ended by returning a DP that has a payload less than the maximum packet size for that endpoint.

Note that if the amount of data in the data structure that is returned to the host is less than the amount requested by the host and is an exact multiple of maximum packet size then a control endpoint shall send a zero length DP to terminate the data stage.

8.12.2.3 STALL TPs Returned by Control Pipes

Control pipes have the unique ability to return a STALL TP due to problems in control transfers. If a device is unable to complete a command, it returns a STALL TP in the Data and/or Status stages of the control transfer. Unlike the case of a functional stall, protocol stall does not indicate an error with the device. The protocol STALL condition lasts until the receipt of the next SETUP DP, and the device shall return a STALL TP in response to any IN or OUT transaction on the pipe until the SETUP DP is received. In general, protocol stall indicates that the request or its parameters are not understood by a device and thus provides a mechanism for extending USB requests.

Devices do not support functional stall on a control pipe.

8.12.3 Bus Interval and Service Interval

For all periodic (interrupt and isochronous) endpoints, the interval at which an endpoint must be serviced is called a "Service Interval". In this specification the term "Bus Interval" is used to refer to a one Microframe interval as defined in the USB 2.0 specification.

8.12.4 Interrupt Transactions

The interrupt transfer type is used for infrequent data transfers with a bounded service period. It supports a reliable data transport with guaranteed bounded latency. It offers guaranteed constant data rate as long as data is available. If an error is detected in the data delivered, the host is not required to retry the transaction in the same service interval. However, if a

device is momentarily unable to transmit or receive the data (i.e., responds with an NRDY TP), the host shall resume transactions to an endpoint only after it receives an ERDY TP from that device for that endpoint.

Interrupt transactions are very similar to bulk transactions – but are limited to a burst of three DPs in each service interval. The **TT** field shall be set to Interrupt by hosts and devices operating above Gen 1 speed; see Table 8-13. The host shall continue to perform transactions to an interrupt endpoint at the agreed upon service interval as long as a device accepts data (in the case of OUT endpoints) or returns data (in the case of IN endpoints). The host is required to send an ACK TP for every DP successfully received in the service interval even if it is the last DP in that service interval. The final ACK TP shall acknowledge the last DP received and shall have the **Number of Packets** field set to zero. If an error occurs while performing transactions to an interrupt endpoint in the current service interval, then the host is not required to retry the transaction in the current service interval but the host shall retry the transaction in the next service interval at the latest.

8.12.4.1 Interrupt IN Transactions

When the host wants to start an Interrupt IN transaction to an endpoint, it sends an ACK TP to the endpoint with the expected sequence number and the number of packets it expects to receive from the endpoint. If an interrupt endpoint is able to send data in response to the ACK TP from the host, it may send up to the number of packets requested by the host within the same service interval. The host shall respond to each of the DPs with an ACK TP indicating successful reception of the data or an ACK TP requesting the DP to be retried in case the DPP was corrupted.

Note that the host expects the first DP to have its sequence number set to zero when it starts the first transfer from a specific endpoint, after the endpoint has been initialized (via a Set Configuration or Set Interface or ClearFeature (ENDPOINT_HALT) command – refer to Chapter 9 for details on these commands).

An interrupt endpoint shall respond to TPs received from the host as described in Section 8.11.1. As long as a device returns data in response to the host sending ACK TPs and the transfer is not complete, the host shall continue to send ACK TPs to the device during every service interval for that endpoint.

The host shall stop performing transactions to an endpoint on the device when any of the following happen:

- The endpoint responds with an NRDY or STALL TP.
- All the data for the transfer is successfully received.
- The endpoint sets the EOB flag in the last DP sent to the host.

When an endpoint receives an ACK TP from the host and cannot respond by sending data, it shall send an NRDY (or STALL in case of an internal endpoint or device error) TP to the host. The host shall not perform any more transactions to the endpoint in subsequent service intervals.

The host shall resume interrupt transactions to an endpoint that responded with a flow control response in a previous service interval only after it receives an ERDY TP from the endpoint. This notifies the host about the endpoint's readiness to transmit data again. Once the host receives the ERDY TP, it shall send an IN request (via an ACK TP) to the endpoint no later than twice the service interval as determined by the value of the **Interval** field in the interrupt endpoint descriptor. An interrupt endpoint responds by returning either the DP (the sequence number of the packet being one more than the sequence number of the last successful data sent) or, should it be unable to return data, an NRDY or a STALL TP.

If a device receives a deferred interrupt IN TP, and the device needs to send interrupt IN data, the device shall respond with an ERDY TP and keep its link in U0 until it receives the

subsequent interrupt transaction from the host, or until $t_{\text{PingTimeout}}$ (refer to Table 8-36) time elapses.

As in the case of Bulk transactions, the sequence number is continually incremented with each packet sent by an interrupt endpoint. When the sequence number reaches 31 it wraps around to zero.

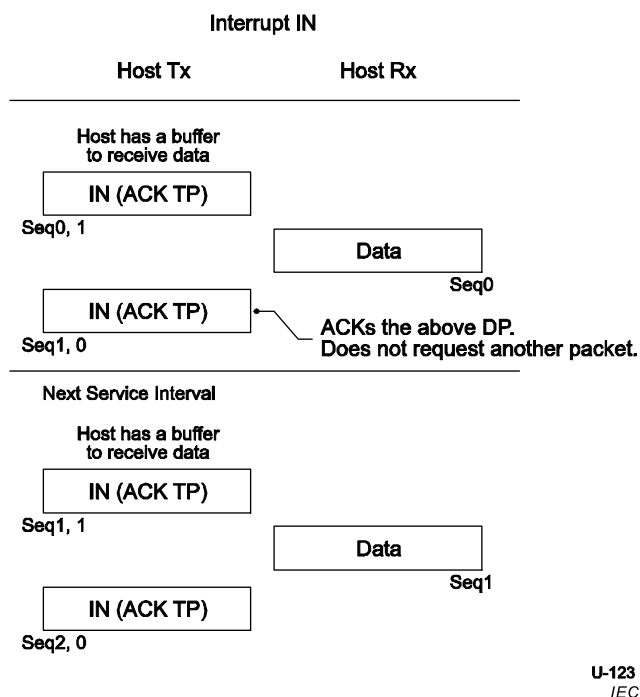


Figure 8-49 – Host Sends Interrupt IN Transaction in Each Service Interval

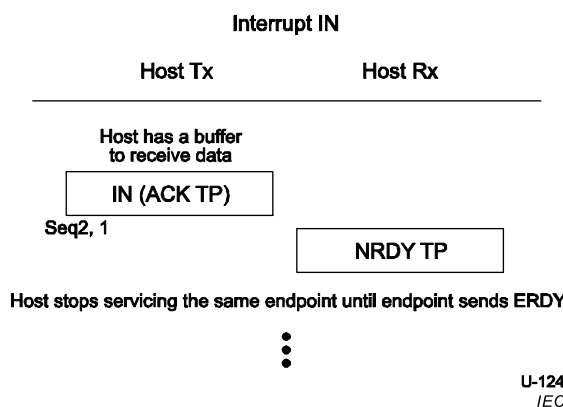


Figure 8-50 – Host Stops Servicing Interrupt IN Transaction Once NRDY is Received

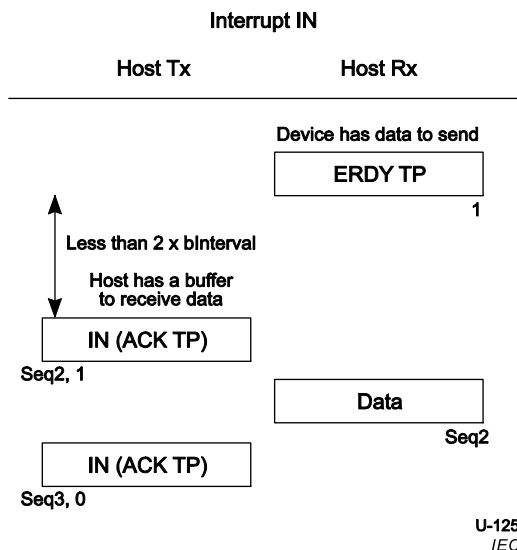


Figure 8-51 – Host Resumes IN Transaction after Device Sent ERDY

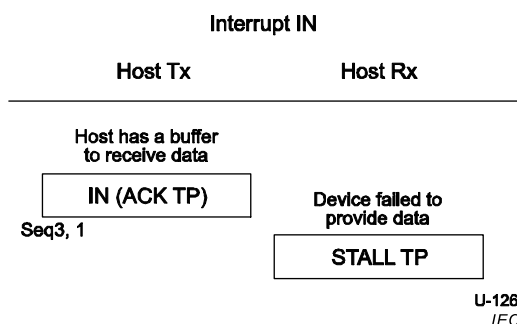


Figure 8-52 – Endpoint Sends STALL TP

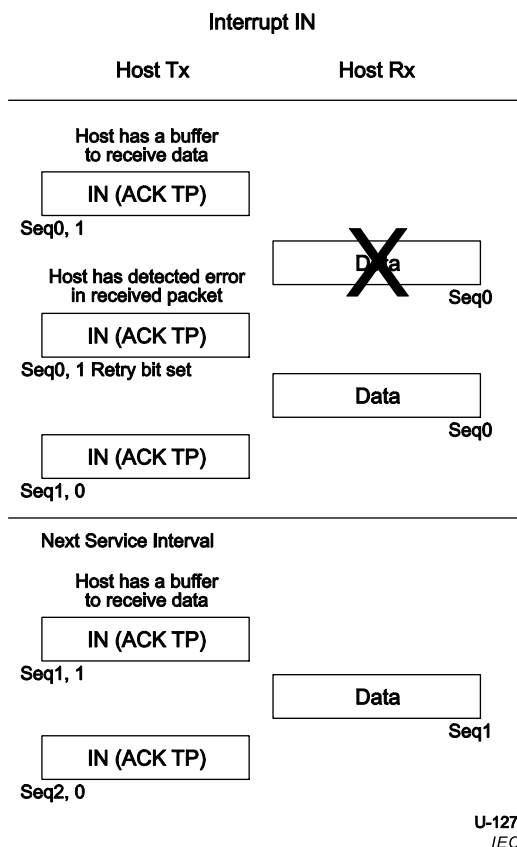


Figure 8-53 – Host Detects Error in Data and Device Resends Data

NOTE In Figure 8-53 the host retries the data packet received with an error in the same service interval. It is not required to do so and may retry the transaction in the next service interval.

8.12.4.2 Interrupt OUT Transactions

When the host wants to start an Interrupt OUT transaction to an endpoint, it sends the first DP with the expected sequence number. The host may send more packets to the endpoint in the same service interval if the endpoint supports a burst size greater than one. If an endpoint was able to receive that data from the host, it sends an ACK TP to acknowledge the successful receipt of data.

Note that the host always initializes the first DP sequence number to zero in the first transfer it performs to an endpoint after the endpoint is initialized (via a Set Configuration or Set Interface or ClearFeature (ENDPOINT_HALT) command – refer to Chapter 9 for details on these commands).

As long as a device returns ACK TPs in response to the host sending data packets and the transfer is not complete, the host shall continue to send data to the device during every service interval for that endpoint. A device shall acknowledge the successful reception of the DP or ask the host to retry the transaction if the data packet was corrupted.

In response to the OUT data sent by the host an interrupt endpoint shall respond as described in Section 8.11.3.

When an endpoint receives data from the host, and it cannot receive data momentarily, it shall send an NRDY (or STALL in case of an internal endpoint or device error) TP to the host. The host shall not perform any more transactions to the endpoint in subsequent service intervals.

A host shall only resume interrupt transactions to an endpoint that responded with a flow control response after it receives an ERDY TP from that endpoint. This notifies the host about the endpoint's readiness to receive data again. Once the host receives an ERDY TP, the host shall transmit the data packet to the endpoint no later than twice the service interval as determined by the value of the **Interval** field in the interrupt endpoint descriptor for that endpoint.

If a device receives a deferred interrupt OUT DPH, and the device needs to receive interrupt OUT data, the device shall respond with an ERDY TP and keep its link in U0 until it receives the subsequent interrupt transaction from the host, or until tPingTimeout (see Table 8-36) elapses.

As in the case of Bulk transactions, the sequence number is continually incremented with each packet sent by host. When the sequence number reaches 31 it wraps around to zero.

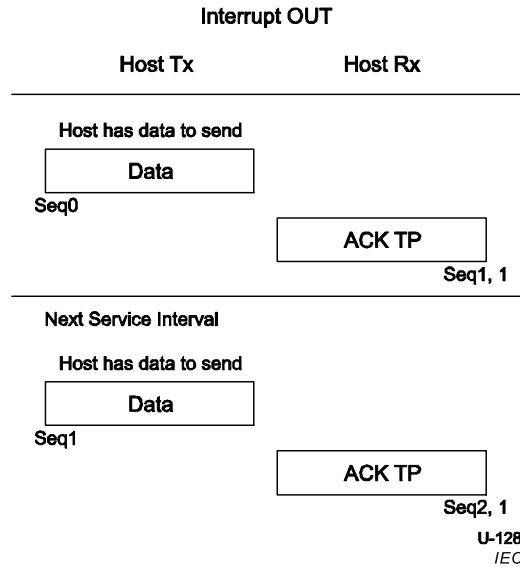


Figure 8-54 – Host Sends Interrupt OUT Transaction in Each Service Interval

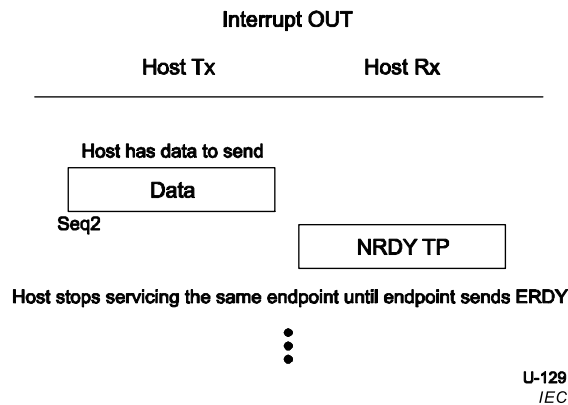


Figure 8-55 – Host Stops Servicing Interrupt OUT Transaction Once NRDY is Received

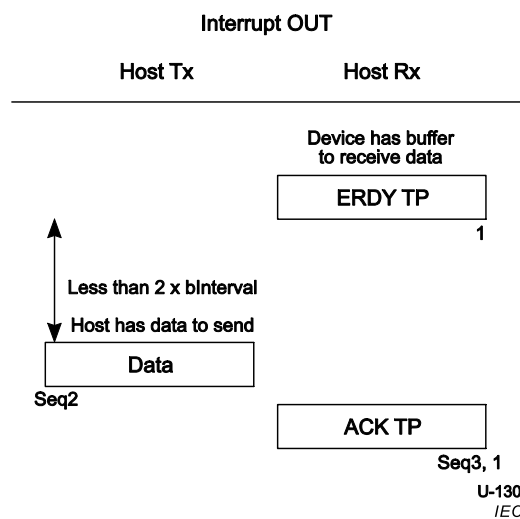


Figure 8-56 – Host Resumes Sending Interrupt OUT Transaction After Device Sent ERDY

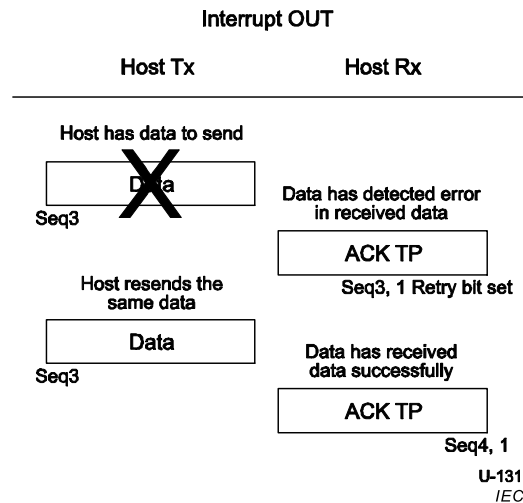


Figure 8-57 – Device Detects Error in Data and Host Resends Data

NOTE In Figure 8-57 the host retries the data packet received with an error in the same service interval. It is not required to do so and may retry the transaction in the next service interval.

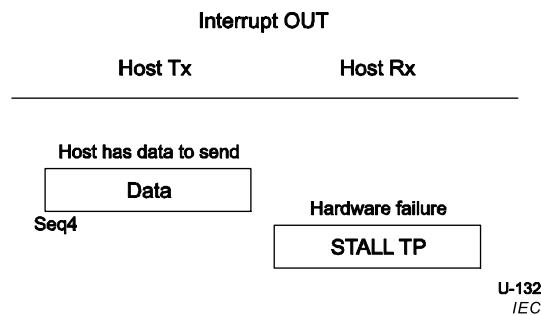


Figure 8-58 – Endpoint Sends STALL TP

8.12.5 Host Timing Information

USB 3.0 host controllers do not broadcast regular start of frame (SOF) packets to all devices on a Enhanced SuperSpeed USB link. Host timing information is sent by the host via isochronous timestamp packets (ITP) when the root port link is in U0 around a bus interval boundary. Hubs forward isochronous timestamp packets (with any necessary modifications as described in Section 10.9.4.4) to any downstream port with a link in U0 and which has completed Port Configuration. The host shall provide isochronous timestamps based on a non-spread clock. Devices are responsible for keeping the link in U0 around bus interval boundaries when isochronous timestamps are required for device operation. A device should never keep the link in U0 for the sole purpose of receiving timestamps unless the timestamps are required for device operation.

NOTE A device will receive an isochronous timestamp if its link is in U0 around a bus interval boundary. This means that devices without any isochronous endpoints or need for synchronization may discard isochronous timestamp packets without negative side effects.

The timing information is sent in an isochronous timestamp packet around each bus interval boundary and communicates the current bus interval and the time from the start of the timestamp packet to the previous bus interval. Isochronous endpoints request a service interval of one Bus Interval $\times 2^n \mu\text{s}$, where n is an integer value from 0 to 15 inclusive.

ITPs communicate timing information such that all isochronous endpoints receive the same bus interval boundaries. The host shall keep service interval boundaries aligned for all endpoints at all times unless the host link enters U3. ITPs issued after the host root port link exits U3 may be aligned with boundaries from before the host root port link entered U3. The host shall begin transmitting ITPs within `tlIsochronousTimestampStart` from when the host root

port's link enters U0 after the link was in U3. Figure 8-59 shows an example with an active isochronous IN endpoint and isochronous OUT endpoint connected below the same USB 3.0 host controller. The service interval for the isochronous IN endpoint is X and the service interval for the isochronous OUT endpoint is $2X$. Note that the host is free to schedule an isochronous IN (via an ACK TP) or isochronous OUT data anywhere within the appropriate service interval. A device will detect the start of new service interval by detecting the rollover of least significant bits in the Bus Interval Counter. The number of bits that need to be monitored for rollover is defined by bInterval . For example, if service interval is equal to two Bus Intervals, the beginning of the service interval is defined by the transition of the least significant bit of the Bus Interval Counter to '0'. If service interval is 4 Bus Intervals, the service interval is defined by the transition of the least significant two bits of the Bus Interval Counter to '0', and so on.

If bInterval is one, a device will detect the start of the service interval when the value of the least significant bit of Bus Interval Counter changes.

A device shall not assume that transactions occur at the same location within each service interval. The host shall schedule isochronous transactions such that they do not cross service interval boundaries.

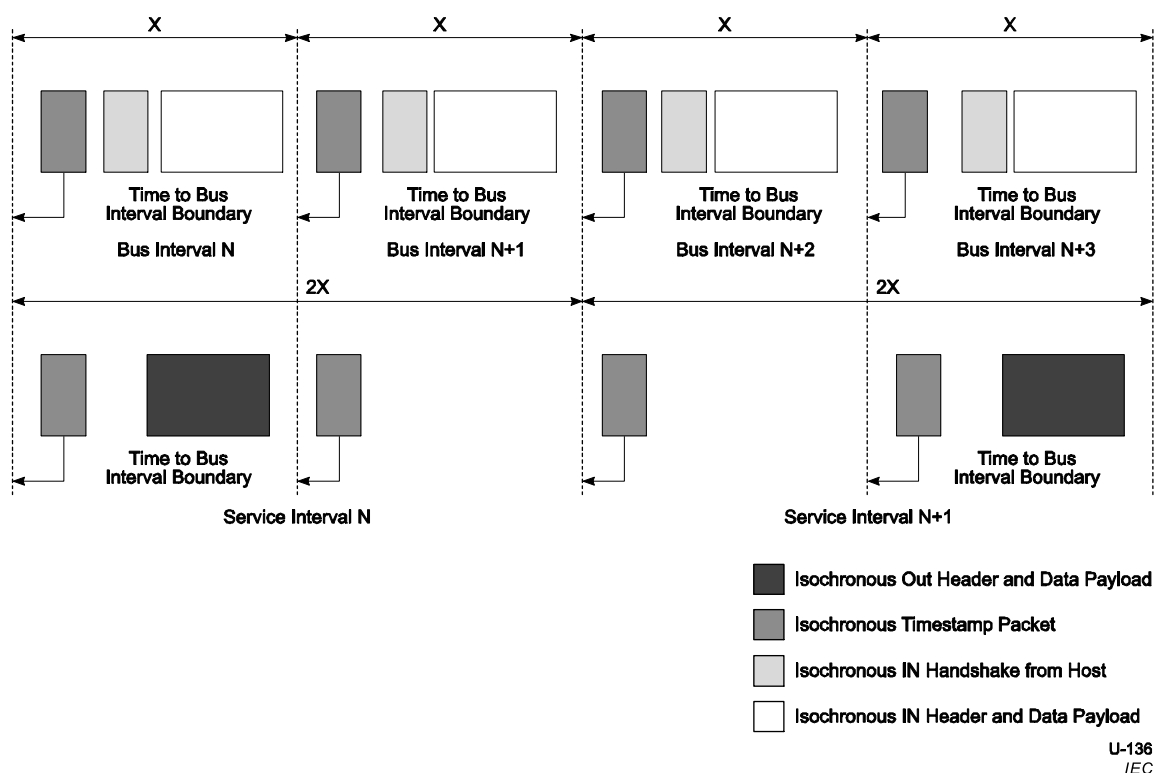


Figure 8-59 – Multiple Active Isochronous Endpoints with Aligned Service Interval Boundaries

8.12.6 Isochronous Transactions

The following sections define the Isochronous transaction protocols for SuperSpeed and SuperSpeedPlus devices. The SuperSpeedPlus protocol relaxes restrictions on how a host may schedule Isochronous transactions to/from a SuperSpeedPlus device and adds the ability to pipeline transaction requests to an endpoint in order to improve the efficiency of the bus.

8.12.6.1 Enhanced SuperSpeed Isochronous Transactions

IN isochronous transactions are shown in Figure 8-60 and OUT isochronous transactions are shown in Figure 8-61. For INs, the host issues an ACK TP followed by a data phase in which the endpoint transmits data for INs. For OUTs, the host simply transmits data when there is

data to be sent in the current service interval. Isochronous transactions do not support retry capability. The **TT** field shall be set to Isochronous by hosts and peripheral devices operating above Gen 1 speed; see Table 8-13.

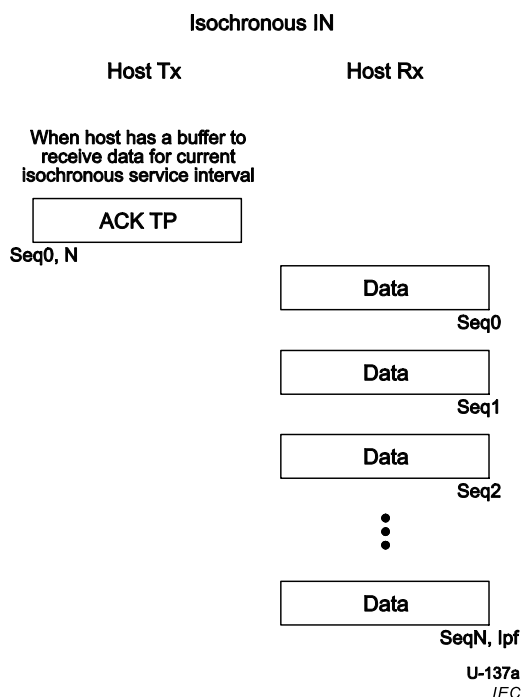


Figure 8-60 – Enhanced SuperSpeed Isochronous IN Transaction Format

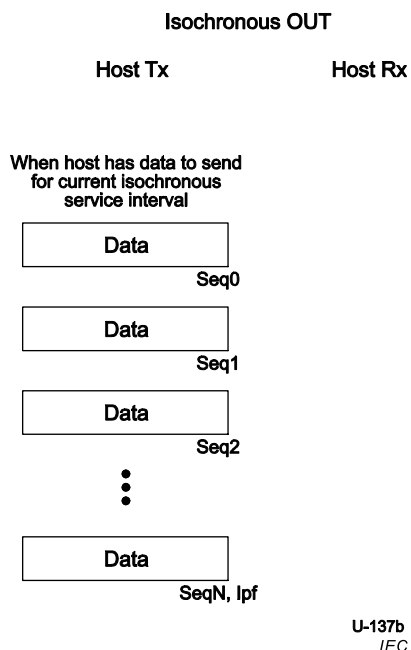


Figure 8-61 – Enhanced SuperSpeed Isochronous OUT Transaction Format

The first DP or ACK TP in each service interval shall start with the sequence number set to 0.

For isochronous transactions that include multiple data packets in a service interval the sequence number is increased by one for each subsequent DP. The DP after sequence number 31 uses a sequence number of zero.

For IN transactions, the current ACK TP **Seq Num** field shall be set to the value of the sum of the **Seq Num** and **NumP** fields in the previous ACK TP as long as all the data for the service interval has not been returned. The equation used to set the current sequence is given below:

$$Seq\ Num[i + 1] = Seq\ Num[i] + NumP[i]$$

A device with an isochronous endpoint shall be able to send or receive the number of packets indicated in its endpoint and endpoint companion descriptors per service interval. The host shall be able to accept and send up to 48 DPs per service interval for devices operating at Gen 1 speed and up to 96 DPs for devices operating at Gen 2 speed.

The last packet in the service interval shall be sent with the *lpf* field set to 1 and can be less than or equal to MaxPacketSize bytes. Each packet except the last packet in the service interval shall be sent with the *lpf* field set to 0 and shall be equal to MaxPacketSize bytes. If there is no data to send to an isochronous OUT endpoint during a service interval, the host does not send anything during the interval. If a device with an isochronous IN endpoint does not have data to send when an isochronous IN ACK TP is received from the host, it shall send a zero length data packet.

Figure 8-62 and Figure 8-63 show sample isochronous IN and OUT transactions for endpoints that have requested 2 000 bytes of bandwidth per service interval (i.e., no more than two packets can be sent or received each service interval).

If the host is not able to send isochronous OUT data during the specified interval due to an error condition, the host discards the data and notifies host software of the error. If the host is not able to send an isochronous ACK TP during the specified service interval due to an error condition, the host notifies host software of the error.

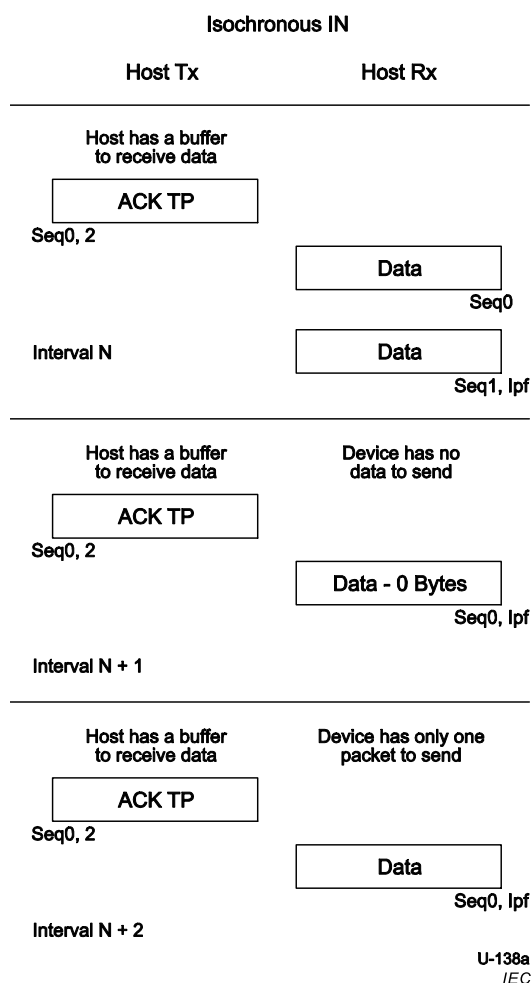


Figure 8-62 – Sample Enhanced SuperSpeed Isochronous IN Transaction

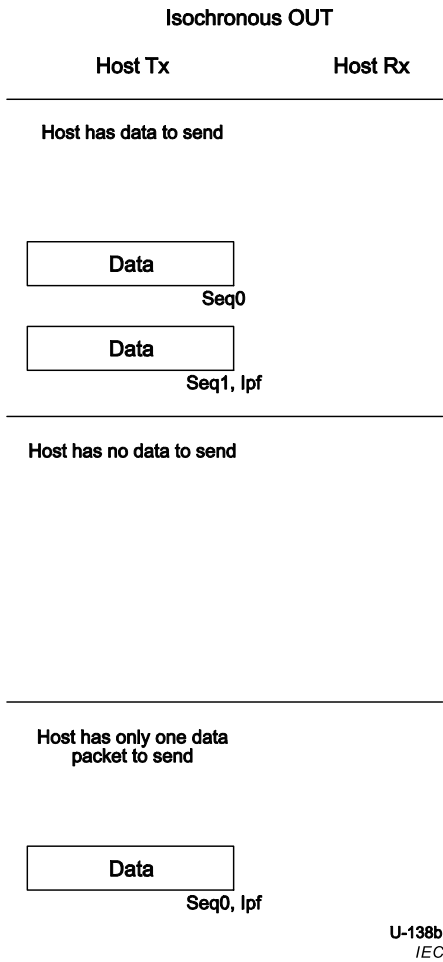


Figure 8-63 – Sample Enhanced SuperSpeed Isochronous OUT Transaction

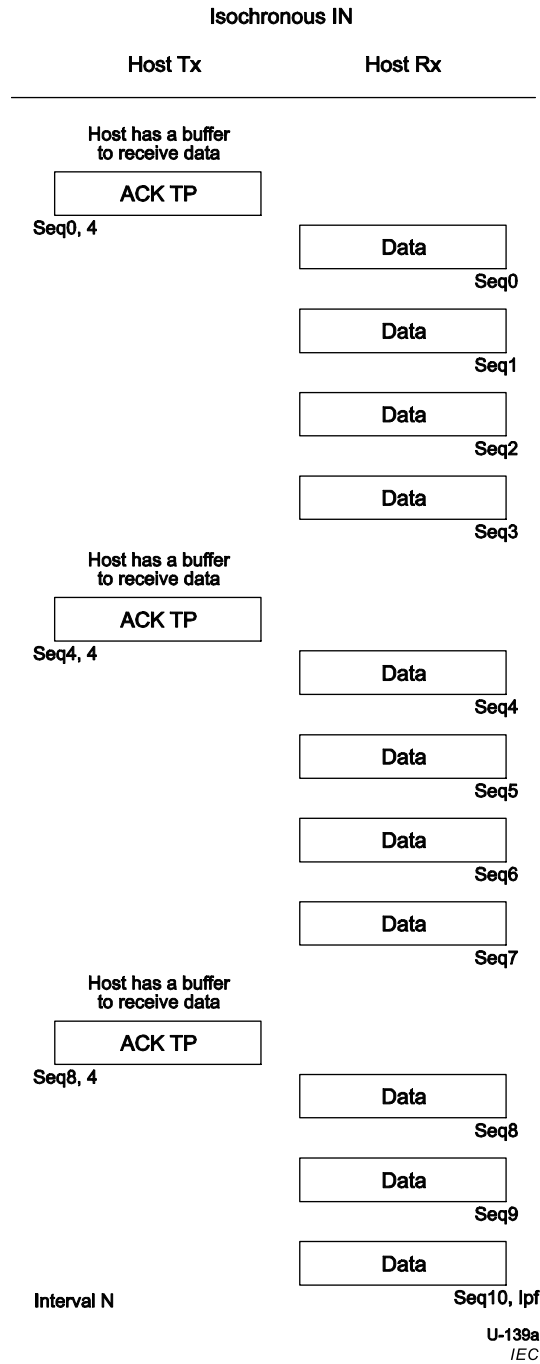


Figure 8-64 – Sample Enhanced SuperSpeed Isochronous IN Transaction

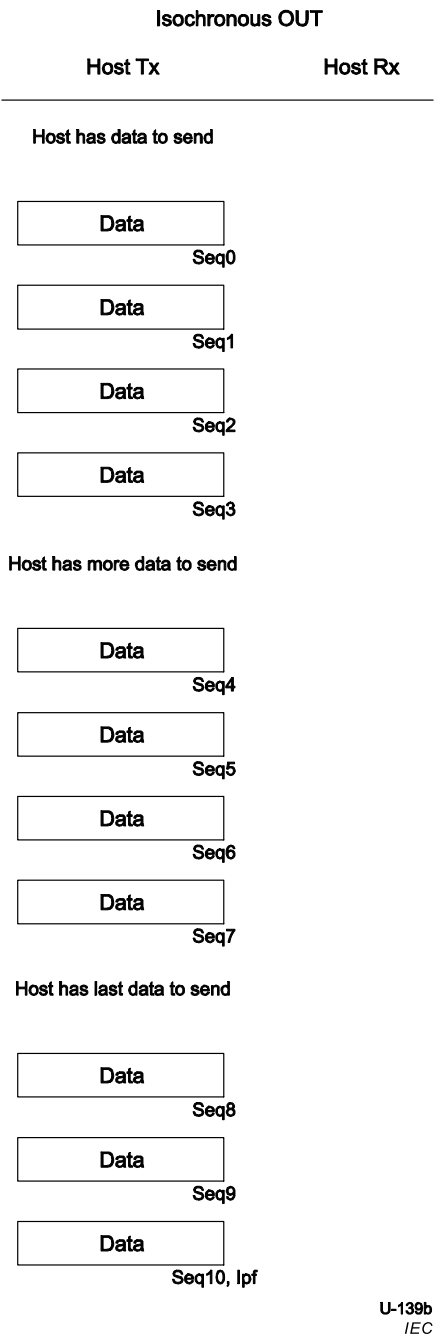


Figure 8-65 – Sample Enhanced SuperSpeed Isochronous OUT Transaction

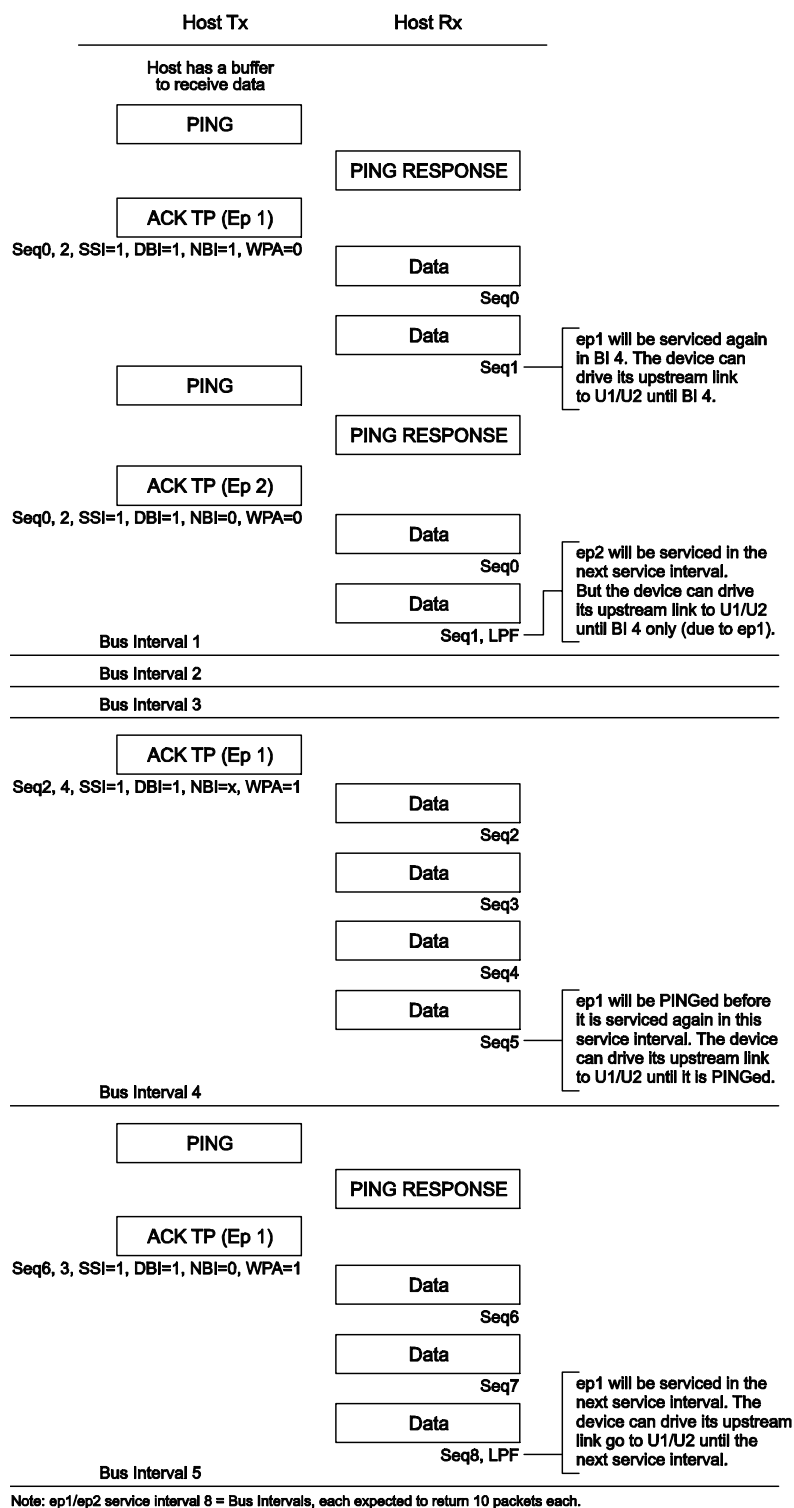
8.12.6.1.1 Smart Isochronous Scheduling Protocol

Figure 8-66 and Figure 8-67 show sample isochronous IN and OUT transactions with smart Isochronous scheduling to endpoints that have service intervals of 8. In the isochronous IN example below the host is only sending one ACK TP with the SSI and DBI field set to non-zero values when asking for data from the endpoint. It should be noted that a host may send multiple ACK TPs with only the last ACK TP in the current bus interval having the SSI and DBI fields set to non-zero values.

The **SSI**, **WPA**, **DBI** and **NBI** fields (described in Table 8-13) are provided in addition to the lpf to give devices more information about when the host plans to transfer isochronous data thus allowing them to more aggressively manage their upstream link. The DBI is used to tell the device that the host has no more data to transfer during the current bus interval. The WPA field, when set to one, informs the device that the host will send a PING TP to the device before it initiates a data transfer on the endpoint again.

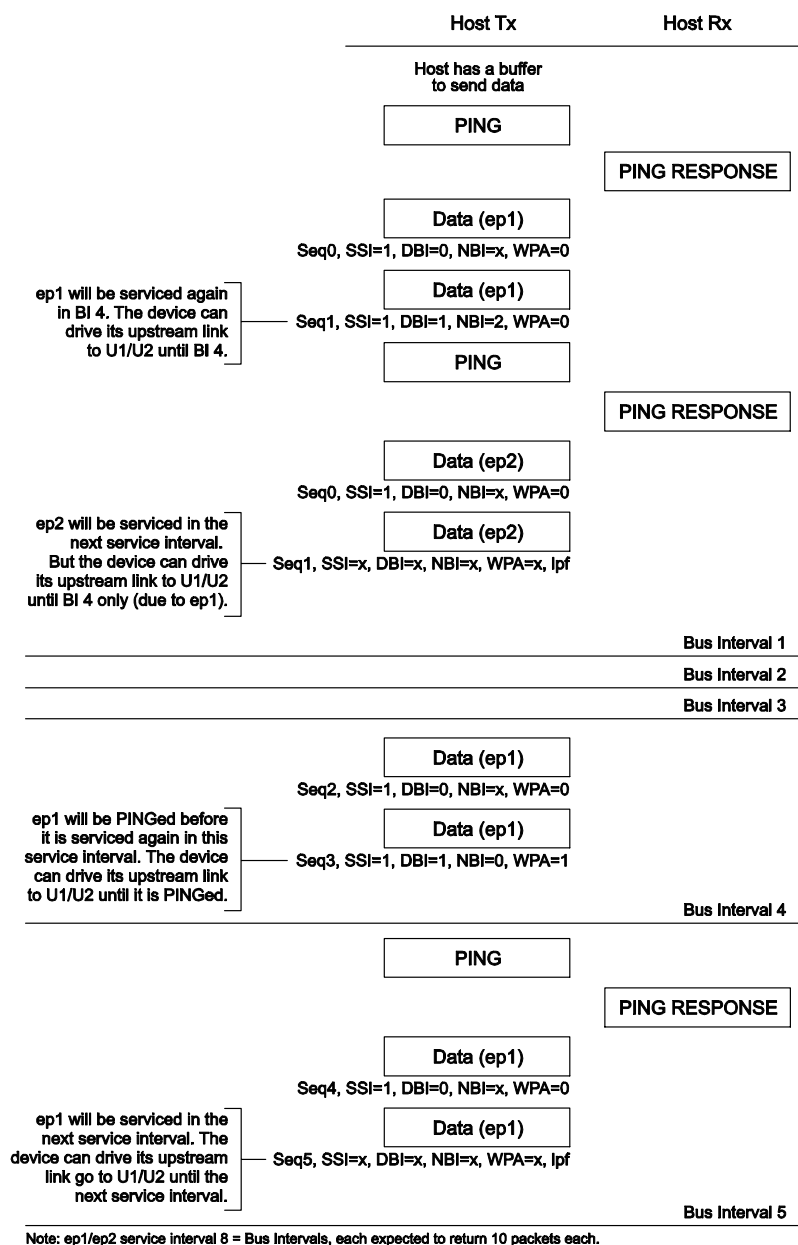
The **NBI** value provides the device additional information (when **DBI** is set to one and **WPA** is set to zero) that it may use to more aggressively manage its upstream port. The value is used to determine the bus interval number (see Table 8-13) that the host will initiate another data transfer on the endpoint. In this case, the host will not be required to send a PING TP before it resumes transfers to the endpoint; it is the device's responsibility to manage its upstream port's link accordingly.

Note that the **SSI** and related fields are only valid and may only be used by a host to inform a device about the manner in which it will service a particular isochronous endpoint on a device within a service interval. A host is always required to send a PING and wait for a PING_RESPONSE before servicing an isochronous endpoint before the start of each service interval.



U-172
IEC

Figure 8-66 – Sample Smart Enhanced SuperSpeed Isochronous IN Transaction



U-173
IEC

Figure 8-67 – Sample Smart Enhanced SuperSpeed Isochronous OUT Transaction

8.12.6.2 Host Flexibility in Performing SuperSpeed Isochronous Transactions

A host targeting an endpoint on a SuperSpeed bus instance shall adhere to the requirements in this section.

The host is allowed some flexibility in performing isochronous service during a service interval. The host may transfer all the DPs to or from an endpoint as a single isochronous burst transaction or it may split the transfer into smaller bursts of two, four, or eight DPs followed by a final isochronous burst with the remaining DPs for that service interval. The host shall not perform isochronous transactions in any other way. For isochronous endpoints that have a multiplier value greater than one, these rules apply to the burst transactions associated with each multiplier value separately. A device shall support all possible host burst transactions allowed by these rules. For example, if an isochronous IN endpoint requests a maximum number of packets in a burst of 11 and the host has 11 packets to receive from the endpoint during a service interval there are four possible ways the host could perform the transaction:

- Request a single burst of 11 packets

- Request a burst of eight followed by a burst of three
- Request two bursts of four followed by a burst of three
- Request five bursts of two followed by a burst of one
- Request 11 bursts of one.

Taking the above example a step further, if the isochronous IN endpoint requests a maximum number of packets in a burst of 11 and a Mult of 2 (in essence requesting three bursts of 11) and the host has buffer space to receive 33 packets from the endpoint during a service interval, then the host can use any combination of the above mentioned options to transfer the three sets of 11 packets to the endpoint.

8.12.6.3 SuperSpeedPlus Isochronous Transactions

8.12.6.3.1 Pipelined Isochronous IN Transactions

SuperSpeedPlus hosts may perform Isochronous transactions to an Enhanced SuperSpeed Isochronous endpoint following the rules outlined in Section 8.12.6.1. However, when performing IN transactions to SuperSpeedPlus endpoints, a SuperSpeedPlus host is allowed to send multiple IN ACK TPs requesting more data from the endpoint before the endpoint has returned all the data previously requested. The host shall not request more outstanding DPs than the max burst size reported in the endpoints' descriptors.

If a SuperSpeedPlus endpoint reports a Max Burst Size of 'M' in its descriptors then a SuperSpeedPlus host can send the following sequence of IN ACK TPs to the device without waiting for the device to return all the DPs asked for in the initial IN ACK TP:

Table 8-32 – ACK TP and DPs for Pipelined Isochronous IN Transactions

Host to Device	Device to Host
IN ACK TP (SeqN = 0, NumP = N) Where N ≤ M	DP (Seq0) DP (Seq1)
IN ACK TP (SeqN = N, NumP = X) Where X ≤ M – N + Number of DPs received	DP (Seq2) DP (Seq3) DP (Seq4)
IN ACK TP (SeqN = N + X, NumP = Y) Where Y ≤ M – X + Number of DPs received	. . . DP (SeqM) DP (SeqM+1)

As can be seen in Table 8-32 the **Seq Num** field is updated in the same manner as it is SuperSpeedPlus Isochronous IN transactions however, they are "Pipelined". Pipelined refers to the ability of SuperSpeedPlus hosts to send the next IN ACK TP before the first one has completed. A SuperSpeedPlus host may continue to send Pipelined IN ACK TPs with the following three caveats:

- The number of outstanding packets requested from the endpoint cannot be greater than the Max Burst Size of the endpoint.
- The number of outstanding packets requested from the endpoint cannot exceed the total amount of data expected from the endpoint in that Service Interval.
- The SuperSpeedPlus host shall stop sending pipelined IN ACK TPs for the current service interval once it receives an end of data indication from the endpoint.

If at any time the endpoint returns a DP with the *lpf* bit set, the SuperSpeedPlus host shall not expect any more packets from the endpoint for this SI. The host shall treat this condition as the termination of Isochronous transactions for this SI for this endpoint. The endpoint shall discard any additional IN ACK TPs it had received or receives in this SI.

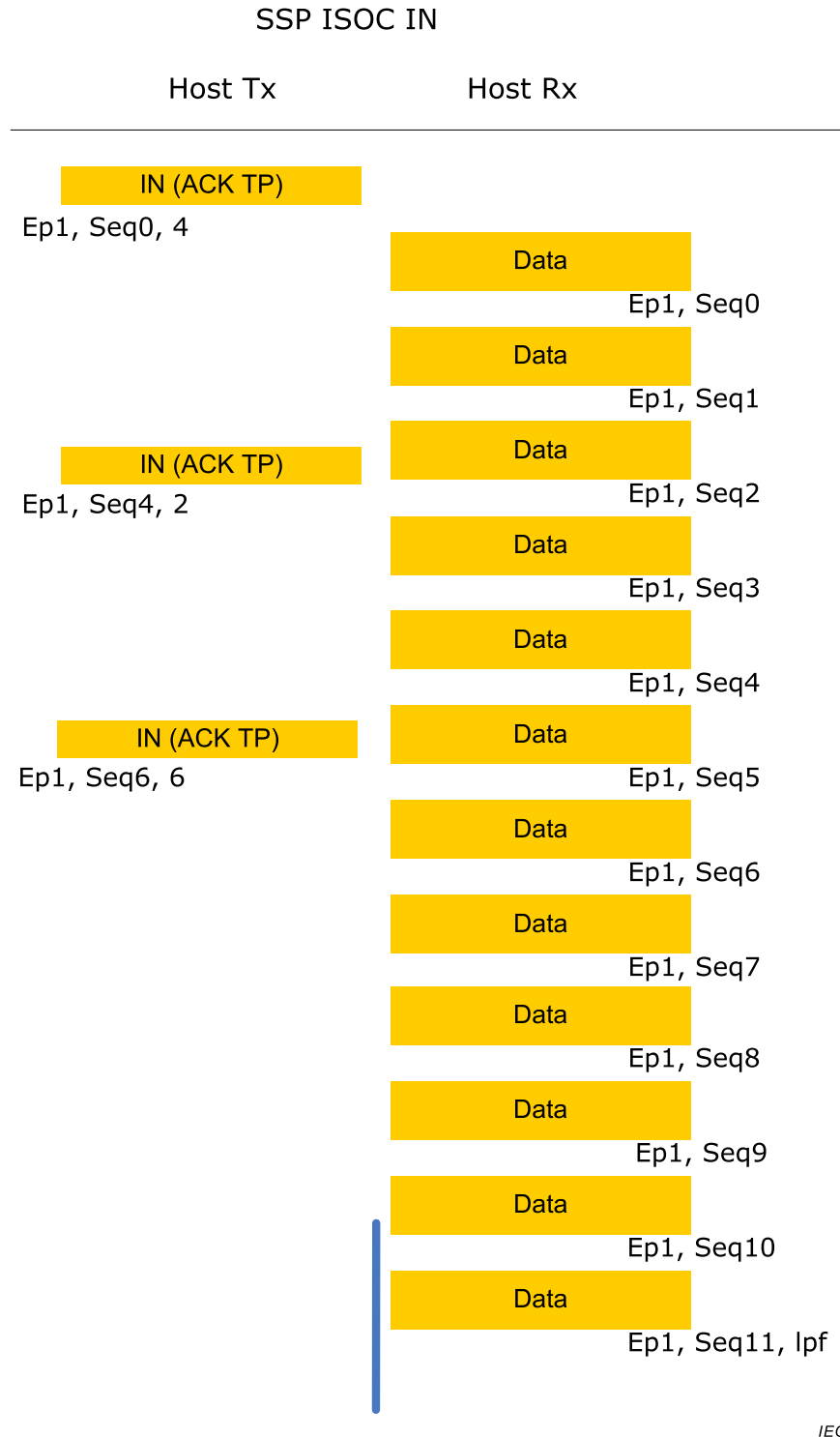


Figure 8-68 – Sample Pipeline Isochronous IN Transactions

8.12.6.4 Host Flexibility in Performing SuperSpeedPlus Isochronous Transactions

A SuperSpeedPlus host is allowed to perform Isochronous transactions without the restrictions mentioned in Section 8.12.6.2 when performing transfers to/from a SuperSpeedPlus Isochronous endpoint. The SuperSpeedPlus host may transfer all the DPs to

or from an endpoint in bursts of any size as long as the number of outstanding packets is less than or equal to the max burst size advertised by the endpoint in its descriptors.

A device shall support all possible pipelined Isochronous IN transactions allowed by these rules.

8.12.6.5 Device Response to Isochronous IN Transactions

Table 8-33 lists the possible responses a device may make in response to an ACK TP. An ACK TP is considered to be invalid if any of the following conditions exist:

- It has an incorrect Device Address
- Its endpoint number and direction does not refer to an endpoint that is part of the current configuration
- It does not have the expected sequence number
- It has the deferred bit set in it
- Its TT does not match the endpoint type (for devices not operating at Gen 1 speed).

Table 8-33 – Device Responses to Isochronous IN Transactions

ACK TP Received Invalid	Device Can Transmit Data	Action Taken
Yes	Do not care	Return no response
No	No	Return zero length data packet with sequence number 0
No	Yes	Return N data packets with sequence numbers 0 to N-1. Each packet except the last shall be MaxPacketSize bytes. The last packet can be less than or equal to MaxPacketSize bytes. The last packet shall have the LPF flag set.

8.12.6.6 Host Processing of Isochronous IN Transactions

Table 8-34 lists the host processing of data from an IN transaction. The host never returns a response to isochronous IN data received. In Table 8-34, DP Error may be due to one or more of the following:

- CRC-32 incorrect
- DPP aborted
- DPP missing
- DPH TT is not set to Isochronous (from a device not operating at Gen 1 speed)
- Data length in the DPH does not match the actual data payload length.

If the host receives a corrupted data packet, it discards the remaining data in the current service interval and informs host software of the error.

Table 8-34 – Host Responses to IN Transactions

Data Packet Error	Host Can Accept Data	Host Data Processing
Yes	N/A	Discard data
No	No. (This should never happen for a compliant host implementation.)	Discard data
No – Data Packet Has Expected Sequence Number	Yes	Accept data
No – Data Packet Does Not Have Expected Sequence Number.	Yes	Discard data

8.12.6.7 Device Response to an Isochronous OUT Data Packet

Table 8-35 lists the device processing of data from an OUT data packet. A device never returns a TP in response. In Table 8-35, DP Error may be due to one or more of the following:

- CRC-32 incorrect
- DPP aborted
- DPP missing
- DPH TT is not set to Isochrnous (for a device not operating at Gen 1 speed)
- Data length in the DPH does not match the actual data payload length
- Deferred bit set in the DPH

Table 8-35 – Device Responses to OUT Data Packets

Data Packet Error	Expected Sequence Number	Device Can Accept Data	Device Data Processing
Yes	Do not care	Do not care	Discard data
No	Yes	Yes	Accept data
No	Yes	No	Data discarded
No	No	No	Data discarded. Device may discard any additional data for current service interval.
No	No	Yes	Data discarded. Device may discard any additional data for current service interval.

8.13 Timing Parameters

Table 8-36 lists the minimum and/or maximum times a device shall adhere to when responding to various types of packets it receives. It also lists the default and minimum times a device may set in Latency Tolerance messages as well as the minimum time after receipt of certain TPs and when it can initiate a U1 or U2 entry. In addition, it lists the maximum time between DPs a device must adhere to while bursting.

All txxxResponse (e.g., tNRDYResponse), tMaxBurstInterval and tGen2MaxBurstInterval times are all timings that a host/device shall meet when the host/device has nothing else to send on its downstream/upstream link.

Table 8-36 – Timing Parameters

Name	Description	Min	Max	Units
tPortConfiguration	Maximum duration from when the port enters U0 to when it has completed the exchange of LMPs. In case of tiebreaker (refer to Table 8-7), both ports shall reset their timers and restart them for each LMP exchange until the tiebreaker is resolved.		20	µs
tPingTimeout	Timeout after the device receives a ping from the host and when it can initiate or accept U1 or U2 requests. This parameter is measured in terms of the maximum of all the service intervals for all isochronous endpoints within the device.	2		Service intervals
tPingResponse	Time between device reception of the last framing symbol of a ping and the first framing symbol of the PING_RESPONSE		400	ns
tBELTDefault	Default for best effort latency tolerance	1		ms
tBELTmin	Minimum value of best effort latency tolerance allowed in a Latency Tolerance Message	125		µs
tNRDYorSTALLResponse	Time between device reception of the last framing symbol for an ACK TP or a DPP or a STATUS TP and the first framing symbol of the NRDY or STALL response		400	ns
tDPResponse	Time between device reception of the last framing symbol for an ACK TP and the first framing symbol of the DP response		400	ns
tACKResponse	Time between device reception of the last framing symbol for a DPP or a STATUS TP and the first framing symbol of the ACK response		400	ns
tHostACKResponse	Time between host reception of the last framing symbol for a DPP and the first framing symbol of the ACK response		3	µs
tERDYTimeout	Timeout after the device sends an ERDY to the host and when it can initiate or accept a U1 or U2 request if not serviced	500		ms
tNotification	Period at which the device shall send a function wake notification if the device has not been accessed (since sending the last function wake notification)	2500		ms
tMaxBurstInterval	When the device is operating at Gen 1 speed, time between DPs when the device endpoint is bursting to the host or the host is bursting to the device endpoint.		100	ns
tTimestampWindow	The host shall transmit an isochronous timestamp from a bus interval boundary to tTimestampWindow after the bus interval boundary if the root port's link is in U0.	0	8	µs
tIsochTimestampGranularity	The granularity of isochronous timestamps	8	8	USB 2.0 High-Speed bit times
BusIntervalAdjustmentGranularity	The adjustment unit for device requested changes to the bus interval		4.0690104 ¹	ps
tIsochronousTimestampStart	Time by which the host shall start transmitting isochronous timestamps after a root port link enters U0 from polling or after the root port link enters U0 after the link was in U3		250	µs
tBELTRepeat	Duration within which devices are limited to send more than two LTM TPs	1		ms

Name	Description	Min	Max	Units
tMinLTMStateChange	Time by which the peripheral device must send an LTM notification after completion of request to enable or disable LTM_Enable feature selector		10	ms
tHostTransactionTimeout	For control, bulk, and interrupt transactions, this is defined as the time without receiving a response to the last DP or ACK TP that the host sent out before the host shall assume that the transaction has failed and halt the endpoint. For Isochronous IN transactions, this is defined as the time without receiving a response to the ACK TP that the host sent. The timer is initialized and restarts counting whenever the host receives each DP that was requested by the ACK TP. If a timeout occurs, the host shall not perform any more transactions to the endpoint in the current service interval. The host shall not halt the endpoint and shall restart transactions to the endpoint in the next service interval. No retries shall be performed.	7.6 ²	25	ms
tGen2MaxBurstInterval	When the device is operating at Gen 2 speed, time between DPs being bursted from the device endpoint to the host. When the host is operating at Gen 2 speed, time between DPs being bursted from the host to the device endpoint.		50	ns
tGen2MaxDeviceMultiPacketInterval	When the device link is operating at Gen 2 speed, time between DPs being concurrently bursted from different device endpoints to the host.		50	ns
tGen2MaxHubMultiPacketInterval	When the hub link is operating at Gen 2 speed, time between DPs on a link when the hub has buffered DPs to transmit for that link.		50	ns
tHostTPFTimeout	When the host times out waiting for Device Notification packet after receiving TP with TPF set.	7.6	25	ms
tDeviceTPFNotification	Time after sending last symbol of TP with TPF set until sending first symbol of corresponding Device Notification packet		400	ns
tITPRegenerationLimit	The jitter introduced by hub in Delta subfield of a transmitted ITP.	-0	-33	µs
tLDMRequestTimeout	Time by which an upstream facing port shall timeout a previously sent LDM Request.		125	µs
tLDMResponseDelay	Time by which a Responder shall time out the generation of a LDM Response LMP.	125		µs
tLDMResponseTime	Time between receiving the last framing symbol of a LDM TS Request LMP and transmitting the first framing symbol of a LDM TS Response LMP.		3	µs
tTPTransmissionDelay	The default transmission delay time used by software for transmitting a TP.	40	40	ns

¹ (tIsochTimestampGranularity/4096)

² This value was chosen to be greater than the link Pending HP Credit timeout (see Chapter 7)

9 Device Framework

A device may be divided into three layers:

- The bottom layer is a bus interface that transmits and receives packets.
- The middle layer handles routing data between the bus interface and various endpoints on the device. As in USB 2.0, the endpoint is the ultimate consumer or provider of data. It may be thought of as a source or sink for data. The characteristics of an endpoint; e.g., the endpoint's transfer type, the maximum payload (MaxPacketSize), and the number of packets (Burst Size) it can receive or send at a time are described in the endpoint's descriptor.
- The top layer is the functionality provided by the serial bus device, for instance, a mouse or video camera interface.

This chapter describes the common attributes and operations of the middle layer of a device. These attributes and operations are used by the function-specific portions of the device to communicate through the bus interface and ultimately with the host.

9.1 USB Device States

A device has several possible states. Some of these states are visible to the USB and the host, while others are internal to the device. This section describes those states.

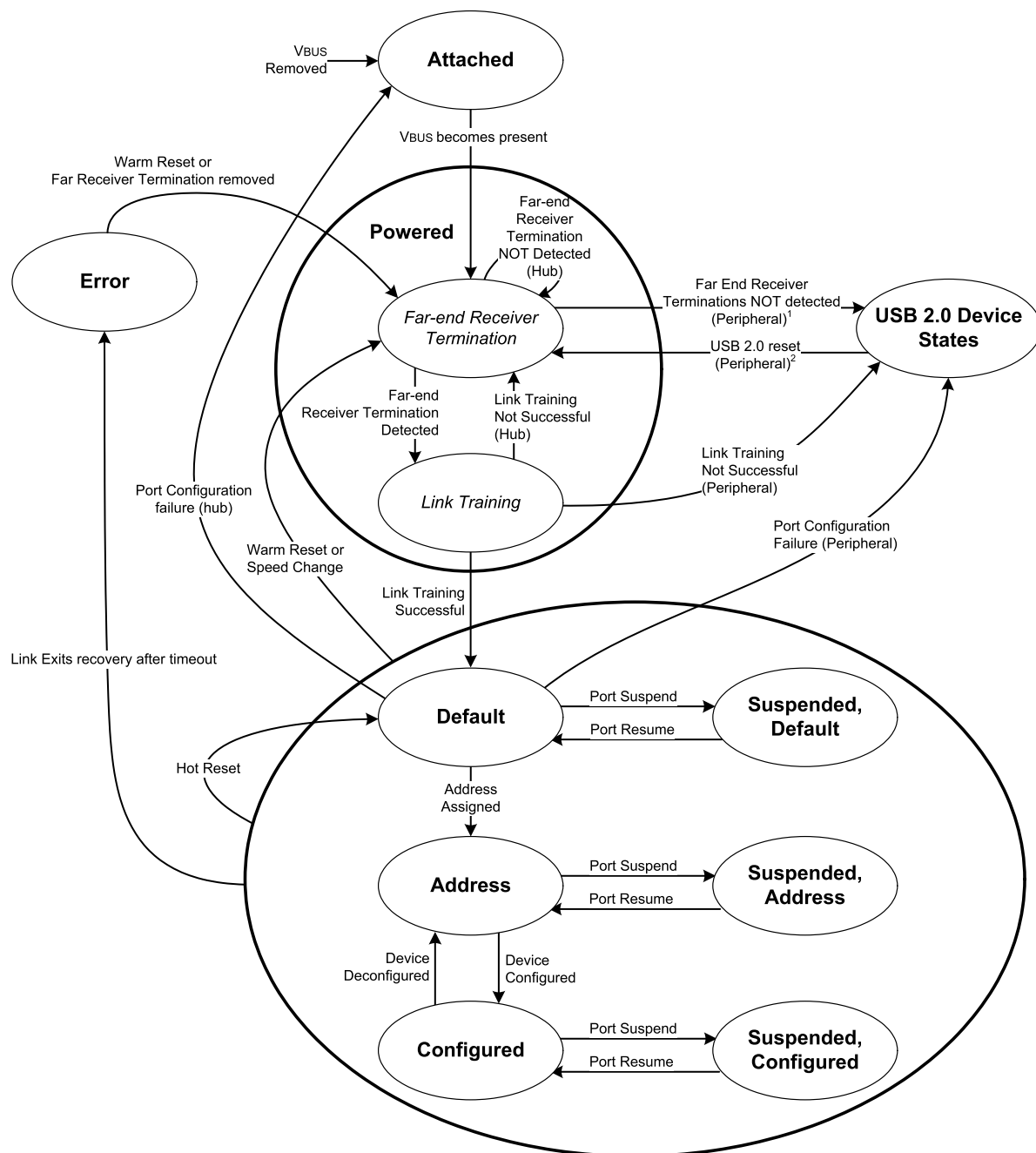
9.1.1 Visible Device States

This section describes device states that are externally visible (see Figure 9-1). Table 9-1 summarizes the visible device states.



NOTE

Devices perform a reset operation in response to reset signaling on the upstream facing port. When reset signaling has completed, the device is reset. The reset signaling depends on the link state. Refer to Section 7.3 for details.



¹ Refer to Note 2 in Figure 10-25 Peripheral Device Upstream Port State Machine.

² Refer to sections 10.16.2.6 and 10.16.2.7 for the conditions that cause this transition.

IEC

**Figure 9-1 – Peripheral State Diagram and Hub State Diagram
(Enhanced SuperSpeed Portion Only)**

Figure 9-1 is a combined state diagram for both peripherals and hubs. Note that a USB Hub has two discrete state diagrams, one for the Enhanced SuperSpeed portion shown in Figure 9-1 and another for the non-SuperSpeed portion that may be found in Figure 9-1 in the *USB 2.0 Specification*.

Table 9-1 – Visible Enhanced SuperSpeed Device States

Attached	Powered	Default	Address	Configured	Suspended ¹	Error	State
No	--	--	--	--	--	--	Device is not attached to the USB. Other attributes are not significant.
Yes	No	--	--	--	--	--	Device is attached to the USB, but is not powered. Other attributes are not significant.
Yes	Yes	No	--	--	--	--	Device is attached to the USB and powered and its upstream link has not successfully completed training.
Yes	Yes	Yes	No	--	--	--	Device is attached to the USB and powered and has been reset, but has not been assigned a unique address. Device responds at the default address.
Yes	Yes	Yes	Yes	No	--	--	Device is attached to the USB, powered, has been reset, and a unique device address has been assigned. Device is not configured.
Yes	Yes	Yes	Yes	Yes	No	--	Device is attached to the USB, powered, has been reset, has a unique address, is configured, and is not suspended. The host may now use the function provided by the device.
Yes	Yes	Yes	--	--	Yes	--	Device is, at minimum, in the Default state (attached to the USB, is powered and its upstream link has been successfully trained) and its upstream link has been set to U3 by its upstream link partner. It may also have a unique address and be configured for use. However, because the device is suspended, the host may not use the device's function.
Yes	Yes					Yes	Device is attached to the USB, powered, and a link timeout error has occurred.

¹ Suspended from the Default, Address, or Configured state.

9.1.1.1 Attached

A device may be attached or detached from the USB. The state of a device when it is detached from the USB is not defined by this specification. This specification only addresses required operations and attributes once the device is attached.

9.1.1.2 Powered

Devices may obtain power from an external source and/or from the USB through the hub to which they are attached. Externally powered devices are termed self-powered. Although self-powered devices may already be powered before they are attached to the USB, they are not considered to be in the Powered state until they are attached to the USB and VBUS is applied to the device.

A device may support both self-powered and bus-powered configurations. Some device configurations support either power source. Other device configurations may be available only if the device is self-powered. Devices report their power source capability through the configuration descriptor. The current power source is reported as part of a device's status. Devices may change their power source at any time, e.g., from self- to bus-powered. If a configuration is capable of supporting both power modes, the power maximum reported for that configuration is the maximum the device will draw from VBUS in either mode. The device shall observe this maximum, regardless of its mode. If a configuration supports only one power mode and the power source of the device changes, the device will lose its current configuration and address and return to the Powered state. If a device operating at Gen X speed is self-powered and its current configuration requires more than 150 mA, then if the device switches to being bus-powered, it shall return to the Powered state. Self-powered hubs that use VBUS to power the Hub Controller are allowed to remain in the Configured state if local power is lost. Note that the maximum power draw for a device operating at a USB 2.0 speed is governed by the limits set in the USB 2.0 specification.

A hub port shall be powered in order to detect port status changes, including attach and detach. Bus-powered hubs do not provide any downstream power until they are configured, at which point they will provide power as allowed by their configuration and power source. A device shall be able to be addressed within a specified time period from when power is initially applied (refer to Chapter 7). After an attachment to a port has been detected, the host may reset the port, which will also reset the device attached to the port.

While in the Powered state, a hub or peripheral device may be in one of two substates: "Far-end Receiver Termination" or "Link Training".

9.1.1.2.1 Far-end Receiver Termination Substate

A peripheral device shall transition to a USB 2.0 Device State as per the conditions defined in Note 2 of Figure 10-26 if Far-end Receiver Terminations are not detected.

A hub shall remain in the Far-end Receiver Termination substate if Far-end Receiver Terminations are not detected.

If Far-end Receiver Terminations are detected, a hub or peripheral device shall transition to the Link Training substate.

9.1.1.2.2 Link Training Substate

A peripheral device shall transition to USB 2.0 Device States if Link Training fails.

A hub shall transition to the Far-end Receiver Termination substate if Link Training fails.

If Link Training is successful, a hub or peripheral device shall transition to the Default state.

9.1.1.3 Default

When operating at Gen X speed, after the device has been powered, it shall not respond to any bus transactions until its link has successfully trained. The device is then addressable at the default address.

An Enhanced SuperSpeed device determines whether it will operate at Gen X speed as a part of the connection process (see the Device Connection State Diagram in Chapter 10 for more details).

A USB device shall reset successfully at one of the supported USB 2.0 speeds when in an USB 2.0 only electrical environment. After the device is successfully reset, the device shall also respond successfully to device and configuration descriptor requests and return appropriate information according to the requirements laid out in the USB 2.0 specification. The device may or may not be able to support its intended functionality when operating in the USB 2.0 mode.

A hub or peripheral device shall transition to the Powered::Far-end Receiver Termination substate if the hub or peripheral device receives a Warm Reset or the hub or device initiates a speed change or a speed change is initiated on the hub or peripheral device's upstream facing port.

A peripheral device shall transition to a USB 2.0 Device State if Port Configuration fails. Refer to Section 8.4.6

A hub shall transition to the Attached state if Port Configuration fails. Note that it is necessary to physically remove and reapply VBUS to transition a hub out of the Attached state.

9.1.1.4 Address

All devices use the default address when initially powered, or after the device has been reset. Each device is assigned a unique address by the host after reset. A device maintains its assigned address while suspended.

A device responds to requests on its default pipe whether the device is currently assigned a unique address or is using the default address.

A hub or peripheral device shall transition to the Powered::Far-end Receiver Termination substate if the hub or peripheral device receives a Warm Reset or the hub or device initiates a speed change or a speed change is initiated on the hub or peripheral device's upstream facing port.

9.1.1.5 Configured

Before a device's function may be used, the device shall be configured. From the device's perspective, configuration involves correctly processing a SetConfiguration() request with a non-zero configuration value. Configuring a device or changing an alternate setting causes all of the status and configuration values associated with all the endpoints in the affected interfaces to be set to their default values. This includes resetting the sequence numbers of any endpoint in the affected interfaces to zero. On initial entry into the configured state a device shall default to the fully functional D0 State.

A hub or peripheral device shall transition to the Powered::Far-end Receiver Termination substate if the hub or peripheral device receives a Warm Reset or the hub or device initiates a speed change or a speed change is initiated on the hub or peripheral device's upstream facing port.

9.1.1.6 Suspended

In order to conserve power, devices automatically enter the Suspended state (one of Suspended Default, Address, or Configured) when they observe that their upstream link is being driven to the U3 state (refer to Section 7.2.4.2.4). Refer to Section 9.2.5.2 for the state that a device maintains while it is suspended.

Attached devices shall be prepared to suspend at any time from the Default, Address, or Configured states. A device shall enter the Suspended state when the hub port it is attached to is set to go into U3. This is referred to as selective suspend.

A device exits suspend mode when it observes wake-up signaling (refer to Section 6.9.1 and Section 7.5.9) on its upstream port. A device may also request the host to exit suspend mode or selective suspend by driving resume signaling (refer to Section 6.9.1 and Section 7.5.9) and sending a Function Wake Notification (refer to Section 8.5.6) on its upstream link to indicate remote wakeup. The ability of a device to signal remote wakeup is optional. If a device is capable of remote wakeup, the device shall support the ability of the host to enable and disable this capability. When the device is reset, remote wakeup shall be disabled. Refer to Section 9.2.5 for more information.

9.1.1.7 Error

This state is entered if the device is in the Default, Address, Configured, or Suspended state and its link exits the Recovery state due to a timeout. A Warm Reset or removal of Far-end Receiver Terminations shall recover from this error condition and transition the device to the Powered::Far-end Receiver Termination substate.

9.1.2 Bus Enumeration

When a device is attached to or removed from the USB, the host uses a process known as bus enumeration to identify and manage the device state changes necessary. When a device is attached to a powered port the following actions are taken (note, these actions apply whether the attached device is a peripheral device or hub device):

- 1) The hub to which the device is now attached informs the host of the event via a reply on its status change pipe (refer to Section 10.13.1). At this point, the device has been reset, is in the Default state and the port to which it is attached is enabled and ready to respond to control transfer requests on the default control pipe.
- 2) The host determines the exact nature of the change by querying the hub.
- 3) Now that the host knows the port to which the new device has been attached, the host then may reset the device again if it wishes, but it is not required to do so.
- 4) If the host resets the port, the hub performs the required reset processing for that port (refer to Section 10.3.1.6). When the reset is completed, the port will be back in the enabled state.
- 5) The device is now in the Default state and can draw no more than 150 mA from VBUS. All of its registers and state have been reset and it answers to the default address.
- 6) The host assigns a unique address to the device, moving the device to the Address state.
- 7) Before the device receives a unique address, its default control pipe is still accessible via the default address. The host reads the device descriptor to determine the actual maximum data payload size that can be used by this device's default pipe.
- 8) The host shall set the isochronous delay to inform the device of the delay from the time a host transmits a packet to the time it is received by the device.
- 9) The host shall inform the device of the system exit latency using the Set SEL request. Device shall accept a Set SEL request whether it is LTM capable or not and whether LTM is enabled or not.
- 10) The host reads the configuration information from the device by reading each configuration from zero to $n-1$, where n is the number of configurations. This process may take several milliseconds to complete.
- 11) Any time after this, the host can set the U1/U2 timeout for the downstream port on which the device is connected using the Set Port Feature (PORT_U1_TIMEOUT/PORT_U2_TIMEOUT).
- 12) Based on the configuration information and how the device will be used, the host assigns a configuration value to the device. The device is now in the Configured state and all of

the endpoints in this configuration have taken on their described characteristics. The device may now draw the amount of VBUS power described in its descriptor for the selected configuration. From the device's point of view, it is now ready for use.

When the device is detached, the hub again sends a notification to the host. Detaching a device disables the port to which it had been attached and the port moves into the Disconnected state (refer to Section 10.3.1.2). Upon receiving the detach notification, the host will update its local topological information.

9.2 Generic Device Operations

All devices support a common set of operations. This section describes those operations.

9.2.1 Dynamic Attachment and Removal

Devices may be attached or detached at any time. The hub provides the attachment point or downstream port and is responsible for reporting any change in the state of the port.

The hub resets and enables the hub downstream port where the device is attached upon detection of an attachment, which also has the effect of resetting the device. A reset device has the following characteristics:

- Its USB address is set to zero (the default USB address)
- It is not configured
- It is not suspended

When a device is removed from a hub port, the hub disables the port where the device was attached, the port moves into the `DSPORT.Disconnected` state (refer to Section 10.3.1.2) and notifies the host of the removal.

9.2.2 Address Assignment

When a device is attached, the host is responsible for assigning a unique address to the device. Before assigning an address, the host may explicitly reset the device, however; note that the device implicitly gets reset during the connection process before the host is notified of a device being attached to the port.

9.2.3 Configuration

A device shall be configured before its function(s) may be used. The host is responsible for configuring a device. The host typically requests configuration information from the device to determine its capabilities.

As part of the configuration process, the host sets the device configuration and, where necessary, selects the appropriate alternate settings for the interfaces.

Within a single configuration, a device may support multiple interfaces. An interface is a related set of endpoints that present a single feature or function of the device to the host. The protocol used to communicate with this related set of endpoints and the purpose of each endpoint within the interface may be specified as part of a device class or vendor-specific definition.

In addition, an interface within a configuration may have alternate settings that redefine the number or characteristics of the associated endpoints. If this is the case, the device shall support the `GetInterface()` request to report the current alternate setting for the specified interface and `SetInterface()` request to select the alternate setting for the specified interface.

Within each configuration, each interface descriptor contains fields that identify the interface number and the alternate setting. Interfaces are numbered from zero to one less than the

number of concurrent interfaces supported by the configuration. Alternate settings range from zero to one less than the number of alternate settings for a specific interface. The default setting when a device is initially configured is alternate setting zero.

In support of adaptive device drivers that are capable of managing a related group of devices, the device and interface descriptors contain *Class*, *SubClass*, and *Protocol* fields. These fields are used to identify the function(s) provided by a device and the protocols used to communicate with the function(s) on the device. A class code is assigned to a group of related devices that has been characterized as a part of a USB Class Specification. A class of devices may be further subdivided into subclasses and, within a class or subclass, a protocol code may define how the host software communicates with the device.

NOTE The assignment of class, subclass, and protocol codes shall be coordinated but is beyond the scope of this specification.

9.2.4 Data Transfer

Data may be transferred between an endpoint within a device and the host in one of four ways. Refer to Chapter 4 for the definition of the four types of transfers. An endpoint number may be used for different types of data transfers in different alternate settings. However, once an alternate setting is selected (including the default setting of an interface), a device endpoint uses only one data transfer method until a different alternate setting is selected.

9.2.5 Power Management

Power management on devices involves the issues described in the following sections.

9.2.5.1 Power Budgeting

USB bus power is a limited resource. During device enumeration, a host evaluates a device's power requirements. If the power requirements of a particular configuration exceed the power available to the device, host software shall not select that configuration.

Devices shall limit the power they consume from VBUS to one unit load or less until configured. When operating at Gen X speed, 150 mA equals one unit load. Suspended devices, whether configured or not, shall limit their bus power consumption as to the suspend mode power requirements in the USB 2.0 specification. Depending on the power capabilities of the port to which the device is attached, an Enhanced SuperSpeed device operating at Gen X speed may be able to draw up to six unit loads from VBUS after configuration. The amount of current draw for Enhanced SuperSpeed devices are increased to 150 mA for low-power devices and 900 mA for high-power devices when operating at Gen X speed.

Device power management is comprised of Device Suspend and Function Suspend. Device Suspend refers to a device-wide state that is entered when its upstream link is placed in U3. Function Suspend refers to a state of an individual function within a device. Suspending a device with more than one function effectively suspends all the functions within the device.

Note that placing all functions in the device into Function Suspend does not suspend the device. A device is suspended only when its upstream link is placed in U3.

9.2.5.2 Changing Device Suspend State

Device Suspend is entered and exited intrinsically as part of the suspend entry and exit processes (refer to Section 9.1.1.6). The minimum device state information that shall be maintained through the duration of each Suspended USB Device State is listed in Table 9-2.

Table 9-2 – Preserved USB Suspend State Parameters

Parameter	Suspended USB Device State ^{1, 2}	
	Address	Configured
U1_SEL/U1_PEL/U2_SEL/U2_PEL	Yes	Yes
HALT_ENDPOINT	N/A	Yes
FUNCTION REMOTE WAKEUP	Yes	Yes
Isochronous Delay	N/A	Yes
U2_Inactivity_Timeout	Yes	Yes
Force_LinkPM_Accept	Yes	Yes
U1/U2 Enable	Yes	Yes
LTM_ENABLE	Yes	Yes
LDM_ENABLE	Yes	Yes
Data Sequence	N/A	Yes
Hub Depth	Yes	Yes
Downstream U1_Inactivity Timeout/ U2_Inactivity Timeout (Applicable to hubs)	Yes	Yes
Port Configuration Information	Yes	Yes
Device Configuration/Interface Setting Information	N/A	Yes
Device Address	Yes	Yes
Header Sequence Number (HSN)	Yes	Yes
Downstream port state	Yes	Yes
Link Speed	Yes	Yes

¹ No parameters other than HSN, are preserved in the *Default* Suspended USB Device State.

² “Yes” indicates a parameter that shall be preserved in the respective Suspended USB Device State.

Some additional Class specific device state information may also be retained during suspend.

A device shall send a Function Wake Notification after driving resume signaling (refer to Section 6.9.1 and Section 7.5.9). If the device has not been accessed for longer than *tNotification* (refer to Section 8.13) since sending the last Function Wake Notification, the device shall send the Function Wake Notification again until it has been accessed.

Device classes may require additional information to be retained during suspend, beyond what is identified in this specification and is beyond the scope of this specification. Devices can optionally remove power from circuitry that is not needed while in suspend.

9.2.5.3 Function Suspend

The Function Suspend state is a reduced power state associated with an individual function. The function may or may not be part of a composite device.

A function may be placed into Function Suspend independently of other functions within a composite device. A device may be transitioned into Device Suspend regardless of the Function Suspend state of any function within the device. Function Suspend state is retained while in Device Suspend and throughout the Device Suspend entry and exit processes.

9.2.5.4 Changing Function Suspend State

Functions are placed into Function Suspend using the `FUNCTION_SUSPEND` feature selector (see Table 9-7). The `FUNCTION_SUSPEND` feature selector also controls whether the function may initiate a function remote wakeup. Whether a function is capable of initiating a Function Remote Wake is determined by the status returned when the first interface in that function is queried using a Get Status command (refer to Section 9.4.5).

Remote wakeup (i.e., wakeup from a Device Suspend state) is enabled when any function within a device is enabled for function remote wakeup (note the distinction between “function remote wake” and “remote wake”). The `DEVICE_REMOTE_WAKEUP` feature selector is ignored and not used by Enhanced SuperSpeed devices.

A function may signal that it wants to exit from Function Suspend by sending a Function Wake Notification to the host if it is enabled for function remote wakeup. This applies to single function devices as well as multiple function (i.e., composite) devices. If the link is in a non-U0 state, then the device must transition the link to U0 prior to sending the remote wake message. If a remote wake event occurs in multiple functions, each function shall send a Function Wake Notification. If the function has not been accessed for longer than `tNotification` (refer to Section 8.13) since sending the last Function Wake Notification, the function shall send the Function Wake Notification again until it has been accessed.

When all functions within a device are in Function Suspend and the `PORT_U2_TIMEOUT` field (refer to Section 10.16.2.10) is programmed to `0xFF`, the device shall initiate U2 after 10 ms of link inactivity.

9.2.6 Request Processing

With the exception of `SetAddress()` requests (refer to Section 9.4.6), a device may begin processing a request as soon as the device receives the Setup Packet. The device is expected to “complete” processing of the request before it allows the Status stage to complete successfully. Some requests initiate operations that take many milliseconds to complete. For such requests, the device class is required to define a method other than Status stage completion to indicate that the operation has completed. For example, a reset on a hub port may take multiple milliseconds to complete depending on the status of the link attached to the port. The `SetPortFeature(PORT_RESET)` (refer to Section 10.16.2.10) request “completes” when the reset on the port is initiated. Completion of the reset operation is signaled when the port’s status change is set to indicate that the port is now enabled. This technique prevents the host from having to poll for completion when it is known that the operation will take a relatively long period of time to complete.

9.2.6.1 Request Processing Timing

All devices are expected to handle requests in a timely manner. USB sets an upper limit of 5 seconds for any command to be processed. This limit is not applicable in all instances. The limitations are described in the following sections. It should be noted that the limitations are intended to encompass a wide range of implementations. If all devices in a USB system used the maximum allotted time for request processing, the user experience would suffer. For this reason, implementations should strive to complete requests in times that are as short as possible.

9.2.6.2 Reset/Resume Recovery Time

After a port is successfully reset or resumed, the USB system software is allowed to access the device attached to the port immediately and it is expected to respond to data transfers.

9.2.6.3 Set Address Processing

After the reset or resume, when a device receives a `SetAddress()` request, the device shall be able to complete processing of the request and be able to successfully complete the Status

stage of the request within 50 ms. In the case of the SetAddress() request, the Status stage successfully completes when the device sends an ACK Transaction Packet in response to Status stage STATUS Transaction Packet.

After successful completion of the Status stage, the device shall be able to accept Setup packets addressed to the new address. In addition, after successful completion of the Status stage, the device shall not respond to transactions sent to the old address (unless, of course, the old address and the new address are the same).

9.2.6.4 Standard Device Requests

For standard device requests that require no Data stage, a device shall be able to complete the request and be able to successfully complete the Status stage of the request within 50 ms of receipt of the request. This limitation applies to requests targeted to the device, interface, or endpoint.

For standard device requests that require a data stage transfer to the host, the device shall be able to return the first data packet to the host within 500 ms of receipt of the request. For subsequent data packets, if any, the device shall be able to return them within 500 ms of successful completion of the transmission of the previous packet. The device shall then be able to successfully complete the status stage within 50 ms after returning the last data packet.

For standard device requests that require a data stage transfer to the device, the 5-second limit applies. This means that the device shall be capable of accepting all data packets from the host and successfully completing the Status stage if the host provides the data at the maximum rate at which the device can accept it. Delays between packets introduced by the host add to the time allowed for the device to complete the request.

9.2.6.5 Class-specific Requests

Unless specifically exempted in the class document, all class-specific requests shall meet the timing limitations for standard device requests. If a class document provides an exemption, the exemption may only be specified on a request-by-request basis.

A class document may require that a device respond more quickly than is specified in this section. Faster response may be required for standard and class-specific requests.

9.2.6.6 Speed Dependent Descriptors

An Enhanced SuperSpeed device shall be capable of operating at one of the USB 2.0 defined speeds. The device always knows its operational speed as part of connection processing (refer to Section 10.1.1 or Section 10.1.2 for more details on the connection process). A device operates at a single speed after completing the reset sequence. In particular, there is no speed switch during normal operation. However, an Enhanced SuperSpeed device may have configurations that are speed dependent. That is, it may have some configurations that are only possible when operating at Gen X speed or some that are only possible when operating at high-speed. Enhanced SuperSpeed devices shall support reporting the speeds at which they can operate. Note that a USB hub is the only device that is allowed to operate at both USB 2.0 and Gen X speed simultaneously.

An Enhanced SuperSpeed device responds with descriptor information that is valid for the current operating speed. For example, when a device is asked for configuration descriptors, it only returns those for the current operating speed (e.g., high speed). Note that the device shall report the other speeds it can operate via its BOS descriptor (refer to Section 9.6.2).

Note that when operating at USB 2.0 speeds, the device shall report the other USB 2.0 speeds it supports using the standard mechanism defined in the USB 2.0 specification in addition to reporting the other speeds supported by the device in its BOS descriptor. Devices

with a value of at least 0210H in the *bcdUSB* field of their device descriptor shall support GetDescriptor (BOS Descriptor) requests.



NOTE

These descriptors are not retrieved unless the host explicitly issues the corresponding GetDescriptor requests.

9.2.7 Request Error

When a request not defined for the device is inappropriate for the current setting of the device or has values that are not compatible with the request is received, a Request Error exists. The device deals with the Request Error by returning a STALL Transaction Packet in response to the next Data stage transaction or in the Status stage of the message. It is preferred that the STALL Transaction Packet be returned at the next Data stage transaction to avoid unnecessary bus activity.

9.3 USB Device Requests

All devices respond to requests from the host on the device's Default Control Pipe. These requests are made using control transfers. The request and the request's parameters are sent to the device in the Setup packet. The host is responsible for establishing the values passed in the fields listed in Table 9-3. Every Setup packet has 8 bytes.

Table 9-3 – Format of Setup Data

Offset	Field	Size	Value	Description
0	<i>bmRequestType</i>	1	Bitmap	Characteristics of request: D7: Data transfer direction 0 = Host-to-device 1 = Device-to-host D6...5: Type 0 = Standard 1 = Class 2 = Vendor 3 = Reserved D4...0: Recipient 0 = Device 1 = Interface 2 = Endpoint 3 = Other 4...30 = Reserved 31 = Vendor Specific
1	<i>bRequest</i>	1	Value	Specific request (refer to Table 9-4)
2	<i>wValue</i>	2	Value	Word-sized field that varies according to request
4	<i>wIndex</i>	2	Index or Offset	Word-sized field that varies according to request; typically used to pass an index or offset
6	<i>wLength</i>	2	Count	Number of bytes to transfer if there is a Data stage

9.3.1 *bmRequestType*

This bitmapped field identifies the characteristics of the specific request. In particular, this field identifies the direction of data transfer in the second stage of the control transfer. The state of the *Direction* bit is ignored if the *wLength* field is zero, signifying there is no Data stage.

USB defines a series of standard requests that all devices shall support. These are listed in Table 9-4. In addition, a device class may define additional requests. A device vendor may also define requests supported by the device.

Requests may be directed to the device, an interface on the device, or a specific endpoint on a device. This field also specifies the intended recipient of the request. When an interface is specified, the *wIndex* field identifies the interface. When an endpoint is specified, the *wIndex* field identifies the endpoint.

9.3.2 bRequest

This field specifies the particular request. The *Type* bits in the *bmRequestType* field modify the meaning of this field. This specification defines values for the *bRequest* field only when the bits are reset to zero, indicating a standard request (refer to Table 9-4).

9.3.3 wValue

The contents of this field vary according to the request. It is used to pass a parameter to the device, specific to the request.

9.3.4 wIndex

The contents of this field vary according to the request. It is used to pass a parameter to the device, specific to the request.

The *wIndex* field is often used in requests to specify an endpoint or an interface. Figure 9-2 shows the format of *wIndex* when it is used to specify an endpoint.

D7	D6	D5	D4	D3	D2	D1	D0
Direction	Reserved (Reset to Zero)			Endpoint Number			
D15	D14	D13	D12	D11	D10	D7	D8
Reserved (Reset to Zero)							

U-081
IEC

Figure 9-2 – *wIndex* Format when Specifying an Endpoint

The *Direction* bit is set to zero to indicate the OUT endpoint with the specified *Endpoint Number* and to one to indicate the IN endpoint. In the case of a control pipe, the request should have the *Direction* bit set to zero but the device may accept either value of the *Direction* bit.

Figure 9-3 shows the format of *wIndex* when it is used to specify an interface.

D7	D6	D5	D4	D3	D2	D1	D0
Interface Number							
D15	D14	D13	D12	D11	D10	D7	D8
Reserved (Reset to Zero)							

U-082
IEC

Figure 9-3 – *wIndex* Format when Specifying an Interface

9.3.5 wLength

This field specifies the length of the data transferred during the second stage of the control transfer. The direction of data transfer (host-to-device or device-to-host) is indicated by the *Direction* bit of the *bmRequestType* field. If this field is zero, there is no data transfer stage.

On an input request, a device shall never return more data than is indicated by the *wLength* value; it may return less. On an output request, *wLength* will always indicate the exact amount of data to be sent by the host. Device behavior is undefined if the host should send more or less data than is specified in *wLength*.

9.4 Standard Device Requests

This section describes the standard device requests defined for all devices. Table 9-4 outlines the standard device requests, while Table 9-5 and Table 9-6 give the standard request codes and descriptor types, respectively.

Devices shall respond to standard device requests, even if the device has not yet been assigned an address or has not been configured. If a standard request defines a persistent parameter that can be modified, the reset/default value for that parameter, unless otherwise specified, is zero.

Table 9-4 – Standard Device Requests

bmRequestType	bRequest	wValue		wIndex		wLength	Data
00000000B 00000001B 00000010B	CLEAR_FEATURE	Feature Selector		Zero Interface Endpoint		Zero	None
10000000B	GET_CONFIGURATION	Zero		Zero		One	Configurati on Value
10000000B	GET_DESCRIPTOR	Descriptor Type and Descriptor Index		Zero or Language ID		Descriptor Length	Descriptor
10000001B	GET_INTERFACE	Zero		Interface		One	Alternate Interface
10000000B 10000001B 10000010B	GET_STATUS	Zero	Status Type	Zero Interface Endpoint		Two	Device, Interface, or Endpoint Status
00000000B	SET_ADDRESS	Device Address		Zero		Zero	None
00000000B	SET_CONFIGURATION	Configuration Value		Zero		Zero	None
00000000B	SET_DESCRIPTOR	Descriptor Type and Descriptor Index		Zero or Language ID		Descriptor Length	Descriptor
00000000B 00000001B 00000010B	SET_FEATURE	Feature Selector		Options	Zero Interface Endpoint	Zero	None
00000001B	SET_INTERFACE	Alternate Setting		Interface		Zero	None
00000000B	SET_ISOCH_DELAY	Delay in ns		Zero		Zero	None
00000000B	SET_SEL	Zero		Zero		Six	Exit Latency Values
10000010B	SYNCH_FRAME	Zero		Endpoint		Two	Frame Number

Table 9-5 – Standard Request Codes

bRequest	Value
GET_STATUS	0
CLEAR_FEATURE	1
Reserved for future use	2
SET_FEATURE	3
Reserved for future use	4
SET_ADDRESS	5
GET_DESCRIPTOR	6
SET_DESCRIPTOR	7
GET_CONFIGURATION	8
SET_CONFIGURATION	9
GET_INTERFACE	10
SET_INTERFACE	11
SYNCH_FRAME	12
SET_ENCRYPTION	13
GET_ENCRYPTION	14
SET_HANDSHAKE	15
GET_HANDSHAKE	16
SET_CONNECTION	17
SET_SECURITY_DATA	18
GET_SECURITY_DATA	19
SET_WUSB_DATA	20
LOOPBACK_DATA_WRITE	21
LOOPBACK_DATA_READ	22
SET_INTERFACE_DS	23
SET_SEL	48
SET_ISOCH_DELAY	49

Table 9-6 – Descriptor Types

Descriptor Types	Value
DEVICE	1
CONFIGURATION	2
STRING	3
INTERFACE	4
ENDPOINT	5
Reserved	6
Reserved	7
INTERFACE_POWER ⁴	8
OTG	9
DEBUG	10
INTERFACE_ASSOCIATION	11
BOS	15
DEVICE CAPABILITY	16
SUPERSPEED_USB_ENDPOINT_COMPANION	48
SUPERSPEEDPLUS_ISOCHRONOUS_ENDPOINT_COMPANION	49

Feature selectors are used when enabling or setting features, such as function remote wakeup, specific to a device, interface, or endpoint. The values for the feature selectors are given in Table 9-7.

Table 9-7 – Standard Feature Selectors

Feature Selector	Recipient	Value
ENDPOINT_HALT	Endpoint	0
FUNCTION_SUSPEND	Interface	0
DEVICE_REMOTE_WAKEUP	Device	1
TEST_MODE	Device	2
b_hnp_enable	Device	3
a_hnp_support	Device	4
a_alt_hnp_support	Device	5
WUSB_DEVICE	Device	6
U1_ENABLE	Device	48
U2_ENABLE	Device	49
LTM_ENABLE	Device	50
B3_NTF_HOST_REL ⁵	Device	51
B3_RSP_ENABLE ⁵	Device	52
LDM_ENABLE	Device	53

⁴ The INTERFACE_POWER descriptor is defined in the current revision of the *USB Interface Power Management Specification*.

⁵ This Feature Selector value shall be reserved for OTG use. Refer to Section 6.4 of the *USB 3.0 OTG and EH Supplement* for its definition.

If an unsupported or invalid request is made to a device, the device responds by returning a STALL Transaction Packet in the Data or Status stage of the request. If the device detects the error in the Setup stage, it is preferred that the device returns a STALL Transaction Packet at the earlier of the Data or Status stage. Receipt of an unsupported or invalid request does not cause the *Halt* feature on the control pipe to be set. If, for any reason, the device becomes unable to communicate via its Default Control Pipe due to an error condition, the device shall be reset to clear the condition and restart the Default Control Pipe. **Clear Feature**

This request is used to clear or disable a specific feature.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B 00000001B 00000010B	CLEAR_FEATURE	Feature Selector	Zero Interface Endpoint	Zero	None

Feature selector values in *wValue* shall be appropriate to the recipient. Only device feature selector values may be used when the recipient is a device, only interface feature selector values may be used when the recipient is an interface, and only endpoint feature selector values may be used when the recipient is an endpoint.

Refer to Table 9-7 for a definition of which feature selector values are defined for which recipients.

A ClearFeature() request that references a feature that cannot be cleared, that does not exist, or that references an interface or an endpoint that does not exist, will cause the device to respond with a Request Error.

If *wLength* is non-zero, then the device behavior is not specified.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: This request is valid when the device is in the Address state; references to interfaces, or to endpoints other than the Default Control Pipe, shall cause the device to respond with a Request Error.

Configured state: This request is valid when the device is in the Configured state.



NOTE

The device shall process a Clear Feature (U1_Enable or U2_Enable or LTM_Enable) only if the device is in the configured state.

9.4.2 Get Configuration

This request returns the current device configuration value.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000000B	GET_CONFIGURATION	Zero	Zero	One	Configuration Value

If the returned value is zero, the device is not configured.

If *wValue*, *wIndex*, or *wLength* are not as specified above, then the device behavior is not specified.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: The value zero shall be returned.

Configured state: The non-zero *bConfigurationValue* of the current configuration shall be returned.

9.4.3 Get Descriptor

This request returns the specified descriptor if the descriptor exists.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000000B	GET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero or Language ID (refer to Section 9.6.9)	Descriptor Length	Descriptor

The *wValue* field specifies the descriptor type in the high byte (refer to Table 9-6) and the descriptor index in the low byte. The descriptor index is used to select a specific descriptor (only for configuration and string descriptors) when several descriptors of the same type are implemented in a device. For example, a device can implement several configuration descriptors. For other standard descriptors that can be retrieved via a *GetDescriptor()* request, a descriptor index of zero shall be used. The range of values used for a descriptor index is from 0 to one less than the number of descriptors of that type (excluding string descriptors) implemented by the device.

The *wIndex* field specifies the Language ID for string descriptors or is reset to zero for other descriptors. The *wLength* field specifies the number of bytes to return. If the descriptor is longer than the *wLength* field, only the initial bytes of the descriptor are returned. If the descriptor is shorter than the *wLength* field, the device indicates the end of the control transfer by sending a short packet when further data is requested.

The standard request to a device supports four types of descriptors: device, configuration, BOS (Binary device Object Store), and string. As noted in Section 9.2.6.6, a device operating at Gen X speed reports the other speeds it supports via the BOS descriptor and shall not support the *device_qualifier* and *other_speed_configuration* descriptors. A request for a configuration descriptor returns the configuration descriptor, all interface descriptors, endpoint descriptors and endpoint companion descriptors (when operating at Gen X speed) for all of the interfaces in a single request. The first interface descriptor follows the configuration descriptor. The endpoint descriptors for the first interface follow the first interface descriptor. In addition, Enhanced SuperSpeed devices shall return Endpoint Companion descriptors for each of the endpoints in that interface to return the endpoint capabilities required for Enhanced SuperSpeed devices, which would not fit inside the existing endpoint descriptor footprint. If there are additional interfaces, their interface descriptor, endpoint descriptors, and endpoint companion descriptors (when operating at Gen X speed) follow the first interface's endpoint and endpoint companion (when operating at Gen X speed) descriptors.

This specification also defines a flexible and extensible framework for describing and adding device-level capabilities to the set of USB standard specifications. The BOS descriptor (refer to Section 9.6.2) defines a root descriptor that is similar to the configuration descriptor, and is the base descriptor for accessing a family of related descriptors. A host can read a BOS descriptor and learn from the *wTotalLength* field the entire size of the device-level descriptor set, or it can read in the entire BOS descriptor set of device capabilities. There is no way for a host to read individual device capability descriptors. The entire set can only be accessed via reading the BOS descriptor with a *GetDescriptor()* request and using the length reported in the *wTotalLength* field.

Class-specific and/or vendor-specific descriptors follow the standard descriptors they extend or modify.

All devices shall provide a device descriptor and at least one configuration descriptor. If a device does not support a requested descriptor, it responds with a Request Error.

Default state: This is a valid request when the device is in the Default state.

Address state: This is a valid request when the device is in the Address state.

Configured state: This is a valid request when the device is in the Configured state.

9.4.4 Get Interface

This request returns the selected alternate setting for the specified interface.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000001B	GET_INTERFACE	Zero	Interface	One	Alternate Setting

Some devices have configurations with interfaces that have mutually exclusive settings. This request allows the host to determine the currently selected alternate setting.

If *wValue* or *wLength* are not as specified above, then the device behavior is not specified.

If the interface specified does not exist, then the device responds with a Request Error.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: A Request Error response is given by the device.

Configured state: This is a valid request when the device is in the Configured state.

9.4.5 Get Status

This request returns status for the specified recipient.

bmRequestType	bRequest	wValue		wIndex	wLength	Data
10000000B 10000001B 10000010B	GET_STATUS	Zero	Status Type	Zero Interface Endpoint	Status Type Length	Device, Interface, or Endpoint Status

The *Recipient* bits of the *bmRequestType* field specify the desired recipient. The data returned is the current status of the specified recipient. If the recipient is an endpoint, then the lower byte of *wIndex* identifies the endpoint whose status is being queried. If the recipient is an interface, then the lower byte of *wIndex* identifies the interface whose status is being queried.

Only a Device is allowed as the Recipient for a PTM Status request.

The *wValue* field specifies the Status type in the low order byte (refer to Table 9-8) and the high order byte is reserved. The Status Type is used to select a specific status register when several types of status registers are implemented in a device.

Table 9-8 – Standard Status Type Codes

Status Type	Value	Status Type Length	Description
STANDARD_STATUS	00H	2	Returns Standard Status Request information
PTM_STATUS	01H	4	Returns PTM Status Request information
Reserved	02-FFH		Reserved for future use

If *wLength* is not as specified above or if *wIndex* is non-zero for a device status request, then the behavior of the device is not specified.

If an interface or an endpoint is specified that does not exist, then the device responds with a Request Error.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: If an interface or an endpoint other than the Default Control Pipe is specified, then the device responds with a Request Error.

Configured state: If an interface or an endpoint that does not exist is specified, then the device responds with a Request Error.

A GetStatus() request to a device returns the information shown in Figure 9-1.

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to Zero)			LTM Enable	U2 Enable	U1 Enable	Remote Wakeup	Self-Powered
D15	D14	D13	D12	D11	D10	D7	D8
Reserved (Reset to Zero)							

U-083
IEC

Figure 9-4 – Information Returned by a Standard GetStatus() Request to a Device

The status fields defined Figure 9-4 are returned by a STANDARD_STATUS type request to a Device recipient.

The *Self Powered* field indicates whether the device is currently self-powered. If D0 is reset to zero, the device is bus-powered. If D0 is set to one, the device is self-powered. The *Self Powered* field may not be changed by the SetFeature() or ClearFeature() requests.

The *Remote Wakeup* field is reserved and must be set to zero by Enhanced SuperSpeed devices. Enhanced SuperSpeed devices use the *Function Remote Wake* enable/disable field to indicate whether they are enabled for Remote Wake.

The *U1 Enable* field indicates whether the device is currently enabled to initiate U1 entry. If D2 is set to zero, the device is disabled from initiating U1 entry, otherwise; it is enabled to initiate U1 entry. The *U1 Enable* field can be modified by the SetFeature() and ClearFeature() requests using the U1_ENABLE feature selector. This field is reset to zero when the device is reset.

The *U2 Enable* field indicates whether the device is currently enabled to initiate U2 entry. If D3 is set to zero, the device is disabled from initiating U2 entry, otherwise; it is enabled to initiate U2 entry. The *U2 Enable* field can be modified by the SetFeature() and ClearFeature() requests using the U2_ENABLE feature selector. This field is reset to zero when the device is reset.

The *LTM Enable* field indicates whether the device is currently enabled to send Latency Tolerance Messages. If D4 is set to zero, the device is disabled from sending Latency Tolerance Messages, otherwise; it is enabled to send Latency Tolerance Messages. The *LTM Enable* field can be modified by the SetFeature() and ClearFeature() requests using the LTM_ENABLE feature selector. This field is reset to zero when the device is reset.

A GetStatus() request to the first interface in a function returns the information shown in Figure 9-5.

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to Zero)						Function Remote Wakeup	Function Remote Wake Capable
D15	D14	D13	D12	D11	D10	D7	D8
Reserved (Reset to Zero)							

U-084
IEC

Figure 9-5 – Information Returned by a Standard GetStatus() Request to an Interface

The status fields defined by Figure 9-5 are returned by a STANDARD_STATUS type request to an Interface recipient.

The *Function Remote Wake Capable* field indicates whether the function supports remote wake up. The *Function Remote Wakeup* field indicates whether the function is currently enabled to request remote wakeup. The default mode for functions that support function remote wakeup is disabled. If D1 is reset to zero, the ability of the function to signal remote wakeup is disabled. If D1 is set to one, the ability of the function to signal remote wakeup is enabled. The *Function Remote Wakeup* field can be modified by the SetFeature() requests using the *FUNCTION_SUSPEND* feature selector. This *Function Remote Wakeup* field is reset to zero when the function is reset.

A GetStatus() request to any other interface in a function shall return all zeros.

A GetStatus() request to an endpoint returns the information shown in Figure 9-6.

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to Zero)							Halt
D15	D14	D13	D12	D11	D10	D7	D8
Reserved (Reset to Zero)							

U-085
IEC

Figure 9-6 – Information Returned by a Standard GetStatus() Request to an Endpoint

The status fields defined by Figure 9-6 are returned by a STANDARD_STATUS type request to an Endpoint recipient.

The *Halt* feature is required to be implemented for all interrupt and bulk endpoint types. If the endpoint is currently halted, then the Halt feature is set to one. Otherwise, the Halt feature is reset to zero. The Halt feature may optionally be set with the SetFeature(ENDPOINT_HALT) request. When set by the SetFeature() request, the endpoint exhibits the same stall behavior as if the field had been set by a hardware condition. If the condition causing a halt has been removed, clearing the *Halt* feature via a ClearFeature(ENDPOINT_HALT) request results in the endpoint no longer returning a STALL Transaction Packet. Regardless of whether an endpoint has the *Halt* feature set, a ClearFeature(ENDPOINT_HALT) request always results in the data sequence being reinitialized to zero, and if Streams are enabled, the Stream State Machine shall be reinitialized to the *Disabled* state. The *Halt* feature is reset to zero after

either a SetConfiguration() or SetInterface() request even if the requested configuration or interface is the same as the current configuration or interface.

Enhanced SuperSpeed devices do not support functional stall on control endpoints and hence do not require the *Halt* feature be implemented for any control endpoints.

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to Zero)						PTM Valid	PTM Enable
D15	D14	D13	D12	D11	D10	D9	D8
Reserved (Reset to Zero)							
D23	D22	D21	D20	D19	D18	D17	D16
PTM Link Delay (low byte)							
D31	D30	D29	D28	D27	D26	D25	D24
PTM Link Delay (high byte)							

U-085a
IEC

Figure 9-7 – Information Returned by a PTM GetStatus() Request to an Endpoint

The status fields defined by Figure 9-7 are returned by a PTM_STATUS type request to a Device recipient.

The *LDM Enabled* flag indicates whether the device is currently enabled to participate in Precision Time Measurement (PTM). If D5 is set to zero, the device is disabled from executing the LDM protocol and providing a local bus interval boundary reference, otherwise; it is enabled to execute the LDM protocol. The *LDM Enabled* flag can be modified by the SetFeature() and ClearFeature() requests using the LDM_ENABLE feature selector. This field shall be set to one when the device is reset, allowing a PTM capable device to automatically attempt to participate in LDM with its upstream partner. If a Requestor is unable to successfully establish LDM Timestamp Exchanges in its Responder, then the *LDM Enabled* field shall be cleared to zero.

The *LDM Valid* field indicates whether the LDM Link Delay is valid, otherwise; it is invalid. *LDM Valid* shall be zero if *LDM Enabled* is zero.

The *LDM Link Delay* field is in tIsochTimestampGranularity units. If *LDM Valid* is one, then the *LDM Link Delay* field defines the link delay value measured by the PTM LDM mechanism. If *LDM Valid* is one, then the *LDM Link Delay* field shall be set to zero. Refer to section 8.4.8.5.1.

9.4.6 Set Address

This request sets the device address for all future device accesses.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_ADDRESS	Device Address	Zero	Zero	None

The *wValue* field specifies the device address to use for all subsequent accesses.

The Status stage after the initial Setup packet assumes the same device address as the Setup packet. The device does not change its device address until after the Status stage of this request is completed successfully. Note that this is a difference between this request and

all other requests. For all other requests, the operation indicated shall be completed before the Status stage.

If the specified device address is greater than 127, or if *wIndex* or *wLength* is non-zero, then the behavior of the device is not specified.

- Default state: If the address specified is non-zero, then the device shall enter the Address state; otherwise, the device remains in the Default state (this is not an error condition).
- Address state: If the address specified is zero, then the device shall enter the Default state; otherwise, the device remains in the Address state but uses the newly-specified address.
- Configured state: Device behavior when this request is received while the device is in the Configured state is not specified.

9.4.7 Set Configuration

This request sets the device configuration.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_CONFIGURATION	Configuration Value	Zero	Zero	None

The lower byte of the *wValue* field specifies the desired configuration. This configuration value shall be zero or match a configuration value from a configuration descriptor. If the configuration value is zero, the device is placed in its Address state. The upper byte of the *wValue* field is reserved.

If *wIndex*, *wLength*, or the upper byte of *wValue* is non-zero, then the behavior of this request is not specified.

- Default state: Device behavior when this request is received while the device is in the Default state is not specified.
- Address state: If the specified configuration value is zero, then the device remains in the Address state. If the specified configuration value matches the configuration value from a configuration descriptor, then that configuration is selected and the device enters the Configured state. Otherwise, the device responds with a Request Error.
- Configured state: If the specified configuration value is zero, then the device enters the Address state. If the specified configuration value matches the configuration value from a configuration descriptor, then that configuration is selected and the device remains in the Configured state. Otherwise, the device responds with a Request Error.

9.4.8 Set Descriptor

This request is optional and may be used to update existing descriptors or new descriptors may be added.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_DESCRIPTOR	Descriptor Type and Descriptor Index	Language ID (refer to Section 9.6.7) or zero	Descriptor Length	Descriptor

The *wValue* field specifies the descriptor type in the high byte (refer to Table 9-6) and the descriptor index in the low byte. The descriptor index is used to select a specific descriptor (only for configuration and string descriptors) when several descriptors of the same type are

implemented in a device. For example, a device can implement several configuration descriptors. For other standard descriptors that can be set via a SetDescriptor() request, a descriptor index of zero shall be used. The range of values used for a descriptor index is from 0 to one less than the number of descriptors of that type (excluding string descriptors) implemented by the device.

The *wIndex* field specifies the Language ID for string descriptors or is reset to zero for other descriptors. The *wLength* field specifies the number of bytes to transfer from the host to the device.

The only allowed values for descriptor type are device, configuration, and string descriptor types.

If this request is not supported, the device will respond with a Request Error.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: If supported, this is a valid request when the device is in the Address state.

Configured state: If supported, this is a valid request when the device is in the Configured state.

9.4.9 Set Feature

This request is used to set or enable a specific feature.

bmRequestType	bRequest	wValue	wIndex		wLength	Data
00000000B 00000001B 00000010B	SET_FEATURE	Feature Selector	Suspend Options	Zero Interface Point	Zero	None

Feature selector values in *wValue* shall be appropriate to the recipient. Only device feature selector values may be used when the recipient is a device; only interface feature selector values may be used when the recipient is an interface; and only endpoint feature selector values may be used when the recipient is an endpoint. If the recipient is an endpoint, then the lower byte of *wIndex* identifies the endpoint.

Refer to Table 9-7 for a definition of which feature selector values are defined for which recipients.

The FUNCTION_SUSPEND feature is only defined for an interface recipient. The lower byte of *wIndex* shall be set to the first interface that is part of that function.

The U1/U2_ENABLE feature is only defined for a device recipient and *wIndex* shall be set to zero. Setting the U1/U2_ENABLE feature allows the device to initiate U1/U2 entry respectively. A device shall support the U1/U2_ENABLE feature when in the Configured state only. System software must not enable the device to initiate U1 if the time for U1 System Exit Latency initiated by Host plus one Bus Interval time is greater than the minimum of the service intervals of any periodic endpoints in the device. In addition, system software must not enable the device to initiate U2 if the time for U2 System Exit Latency initiated by Host plus one Bus Interval time is greater than the minimum of the service intervals of any periodic endpoints in the device.

The LTM_ENABLE feature is only defined for a device recipient and *wIndex* shall be set to zero. Setting the LTM_ENABLE feature allows the device to send Latency Tolerance Messages. A device shall support the LTM_ENABLE feature if it is in the Configured state and supports the LTM capability.

The LDM_ENABLE feature is only defined for a device recipient and *wIndex* shall be set to zero. Setting the LDM_ENABLE feature allows the device to execute the LDM protocol. A device shall support the LDM_ENABLE feature if it is in the Address or Configured states and supports the PTM capability.

A SetFeature() request that references a feature that cannot be set or that does not exist causes a STALL Transaction Packet to be returned in the Status stage of the request.

Table 9-9 – Suspend Options

Bit	Description
0	Value Meaning
	0 Normal operation state (default)
	1 Low power suspend state
1	Value Meaning
	0 Function Remote Wake Disabled (Default)
	1 Function Remote Wake Enabled
2-7	Reserved

If the feature selector is *FUNCTION_SUSPEND*, then the most significant byte of *wIndex* is used to specify Suspend options. The recipient of a SetFeature (FUNCTION_SUSPEND...) shall be the first interface in the function; and, hence, the *bmRequestType* shall be set to one. The valid encodings for the *FUNCTION_SUSPEND* suspend options are listed in Table 9-9.

If *wLength* is non-zero, then the behavior of the device is not specified.

If an endpoint or interface is specified that does not exist, then the device responds with a Request Error.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: If an interface or an endpoint other than the Default Control Pipe is specified then the device responds with a Request Error. If the device receives a SetFeature(U1/U2 Enable or LTM Enable or LDM Enable or FUNCTION_SUSPEND), then the device responds with a Request Error.

Configured state: This is a valid request when the device is in the Configured state.

9.4.10 Set Interface

This request allows the host to select an alternate setting for the specified interface.

bmRequestType	bRequest	WValue	wIndex	wLength	Data
00000001B	SET_INTERFACE	Alternate Setting	Interface	Zero	None

Some devices have configurations with interfaces that have mutually exclusive settings. This request allows the host to select the desired alternate setting. If a device only supports a default setting for the specified interface, then a STALL Transaction Packet may be returned in the Status stage of the request. This request cannot be used to change the set of configured interfaces (the SetConfiguration() request shall be used instead).

If the interface or the alternate setting does not exist, then the device responds with a Request Error. If *wLength* is non-zero, then the behavior of the device is not specified.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: The device shall respond with a Request Error.

Configured state: This is a valid request when the device is in the Configured state.

9.4.11 Set Isochronous Delay

This request informs the device of the delay from the time a host transmits a packet to the time it is received by the device.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_ISOCH_DELAY	Delay in ns	Zero	Zero	None

The *wValue* field specifies a delay from 0 to 65 535 ns. This delay represents the time from when the host starts transmitting the first framing symbol of the packet to when the device receives the first framing symbol of that packet. The *wValue* field shall be calculated as follows.

$$wValue = (\text{sum of } wHubDelay \text{ values}) + (tPTTransmissionDelay * (\text{number of hubs} + 1))$$

Where, a *wHubDelay* value is provided by the Enhanced SuperSpeed Hub Descriptor of each hub in the path, respectively, and *tPTTransmissionDelay* is defined in Table 8-36.

If *wIndex* or *wLength* is non-zero, then the behavior of this request is not specified.

Default state: This is a valid request when the device is in the Default state.

Address state: This is a valid request when the device is in the Address state.

Configured state: This is a valid request when the device is in the Configured state.

9.4.12 Set SEL

This request sets both the U1 and U2 System Exit Latency and the U1 or U2 exit latency for all the links between a device and a root port on the host.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_SEL	Zero	Zero	Six	Exit Latency Values

The latency values are sent to the device in the data stage of the control transfer in the following format:

Offset	Name	Meaning
0	U1SEL	Time in μs for U1 System Exit Latency
1	U1PEL	Time in μs for U1 Device to Host Exit Latency
2	U2SEL	Time in μs for U2 System Exit Latency
4	U2PEL	Time in μs for U2 Device to Host Exit Latency

Figure C.2 in Appendix C illustrates the total latency a device may experience. The components of latency include the following:

- t1: the time to transition all links in the path to the host to U0 when the transition is initiated by the device
- t2: the time for the ERDY to traverse the interconnect hierarchy from the device to the host

- t3: the time for the host to consume the ERDY and transmit a response to that request
- t4: the time for the response to traverse the interconnect hierarchy from the host to the device

The U1SEL and U2SEL values represent the total round trip path latency when transitioning the links between the device and host from U1 or U2 respectively to U0 under worst-case circumstances when the transition is initiated by the device. This is the sum of times t1, t2, and t4.

The U1PEL and U2PEL values represent the device to host latency to transition the entire path of links between the device and host from U1 or U2 respectively to U0 under worst-case circumstances when the transition is initiated by the device. This time includes only t1.

For more information, refer to Section C.1.5.1.

If *wIndex* or *wValue* is not set to zero or *wLength* is not six, then the behavior of the device is not specified.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: This is a valid request when the device is in the Address state.

Configured state: This is a valid request when the device is in the Configured state.

9.4.13 Synch Frame

This request is used to set and then report an endpoint's synchronization frame.

bmRequestType	bRequest	WValue	wIndex	wLength	Data
10000010B	synch_frame	Zero	Endpoint	Two	Frame Number

When an endpoint supports isochronous transfers, the endpoint may also require per-frame transfers to vary in size according to a specific pattern. The host and the endpoint must agree on which frame the repeating pattern begins. The number of the frame in which the pattern began is returned to the host.

If an Enhanced SuperSpeed device supports the Synch Frame request, it shall internally synchronize itself to the zeroth microframe and have a time notion of classic frame. Only the frame number is used to synchronize and reported by the device endpoint (i.e., no microframe number). The endpoint must synchronize to the zeroth microframe.

This value is only used for isochronous data transfers using implicit pattern synchronization. If *wValue* is non-zero or *wLength* is not two, then the behavior of the device is not specified.

If the specified endpoint does not support this request, then the device will respond with a Request Error.

Default state: Device behavior when this request is received while the device is in the Default state is not specified.

Address state: The device shall respond with a Request Error.

Configured state: This is a valid request when the device is in the Configured state.

9.4.14 Events and Their Effect on Device Parameters

This section lists the various parameters and the effect on those parameters when the device receives a control transfer command or when it observes a bus reset on the bus. An X

denotes that the parameter is reset to its default value when the said event occurs. A Y denotes that the particular Parameter is modified by the event.

Control transfers and events not identified in the table shall not affect the value of parameters shown in Table 9-10.

Table 9-10 – Device Parameters and Events

Parameter	Event							
	Warm Reset	Hot Reset	Set Address 0	Set Address	Set Configuration	Set Interface	ClearFeature (STALL)	Disconnect
Device Address	X	X	X	Y				X
Device Configuration Value	X	X	X		Y			X
Alternate Interface Setting	X	X	X		X	Y		X
U1_SEL/U1_PEL/ U2_SEL/U2_PEL	X	X	X					X
HALT_ENDPOINT	X	X	X		X	X (if the EP is affected)	X	X
FUNCTION REMOTE WAKEUP	X	X	X		X	X		X
Isochronous Delay	X	X	X					X
U2_Inactivity_Timeout	X	X	X					X
Force_LinkPM_Accept	X	X	X					X
U1/U2 Enable	X	X	X					X
LTM_ENABLE	X	X	X					X
LDM_ENABLE	X	X	X					X
HeaderSequence Number related to DPs	X	X	X		X	X (if the EP is affected)	X	X
Hub Depth	X	X	X		X			X
Downstream U1_Inactivity Timeout/ U2_Inactivity Timeout (Applicable to host and hub)	X	X	X					X (Hub Upstream or Hub/Host Downstream)
Port Configuration Information	X							X

9.5 Descriptors

Devices report their attributes using descriptors. A descriptor is a data structure with a defined format. Each descriptor begins with a byte-wide field that contains the total number of bytes in the descriptor followed by a byte-wide field that identifies the descriptor type.

Using descriptors allows concise storage of the attributes of individual configurations because each configuration may reuse descriptors or portions of descriptors from other configurations that have the same characteristics. In this manner, the descriptors resemble individual data records in a relational database.

Where appropriate, descriptors contain references to string descriptors that provide displayable information describing a descriptor in human-readable form. The inclusion of string descriptors is optional. However, the reference fields within descriptors are mandatory. If a device does not support string descriptors, string reference fields shall be reset to zero to indicate no string descriptor is available.

If a descriptor returns with a value in its length field that is less than defined by this specification, the descriptor is invalid and should be rejected by the host. If the descriptor returns with a value in its length field that is greater than defined by this specification, the extra bytes are ignored by the host, but the next descriptor is located using the length returned rather than the length expected.

A device may return class- or vendor-specific descriptors in two ways:

- 1) If the class or vendor specific descriptors use the same format as standard descriptors (i.e., start with a length byte and followed by a type byte), they shall be returned interleaved with standard descriptors in the configuration information returned by a `GetDescriptor(Configuration)` request. In this case, the class or vendor-specific descriptors shall follow a related standard descriptor they modify or extend.
- 2) If the class or vendor specific descriptors are independent of configuration information or use a non-standard format, a `GetDescriptor()` request specifying the class or vendor specific descriptor type and index may be used to retrieve the descriptor from the device. A class or vendor specification will define the appropriate way to retrieve these descriptors.

9.6 Standard USB Descriptor Definitions

The standard descriptors defined in this specification may only be modified or extended by revision of this specification.

9.6.1 Device

A device descriptor describes general information about a device. It includes information that applies globally to the device and all of the device's configurations. A device has only one device descriptor.

The device descriptor of an Enhanced SuperSpeed device shall have a version number of 3.1 (0310H). The device descriptor of an Enhanced SuperSpeed device operating in one of the USB 2.0 modes shall have a version number of 2.1 (0210H).

The *bcdUSB* field contains a BCD version number. The value of the *bcdUSB* field is 0xJJMN for version JJ.M.N (JJ – major version number, M – minor version number, N – sub-minor version number), e.g., version 2.1.3 is represented with value 0213H and version 3.0 is represented with a value of 0300H.

The *bNumConfigurations* field indicates the number of configurations at the current operating speed. Configurations for the other operating speed are not included in the count. If there are specific configurations of the device for specific speeds, the *bNumConfigurations* field only reflects the number of configurations for a single speed, not the total number of configurations for both speeds.

If the device is operating at Gen X speed, the *bMaxPacketSize0* field shall be set to 09H (see Table 9-11) indicating a 512-byte maximum packet. An Enhanced SuperSpeed device shall not support any other maximum packet sizes for the default control pipe (endpoint 0) control endpoint.

All devices have a default control pipe. The maximum packet size of a device's default control pipe is described in the device descriptor. Endpoints specific to a configuration and its interface(s) are described in the configuration descriptor. A configuration and its interface(s)

do not include an endpoint descriptor for the default control pipe. Other than the maximum packet size, the characteristics of the default control pipe are defined by this specification and are the same for all Enhanced SuperSpeed devices.

The *bNumConfigurations* field identifies the number of configurations the device supports. Table 9-11 shows the standard device descriptor.

Table 9-11 – Standard Device Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	DEVICE Descriptor Type
2	<i>bcdUSB</i>	2	BCD	USB Specification Release Number in Binary-Coded Decimal (i.e., 2.10 is 210H). This field identifies the release of the USB Specification with which the device and its descriptors are compliant.
4	<i>bDeviceClass</i>	1	Class	Class code (assigned by the USB-IF). If this field is reset to zero, each interface within a configuration specifies its own class information and the various interfaces operate independently. If this field is set to a value between 1 and FEH, the device supports different class specifications on different interfaces and the interfaces may not operate independently. This value identifies the class definition used for the aggregate interfaces. If this field is set to FFH, the device class is vendor-specific.
5	<i>bDeviceSubClass</i>	1	SubClass	Subclass code (assigned by the USB-IF). These codes are qualified by the value of the <i>bDeviceClass</i> field. If the <i>bDeviceClass</i> field is reset to zero, this field shall also be reset to zero. If the <i>bDeviceClass</i> field is not set to FFH, all values are reserved for assignment by the USB-IF.
6	<i>bDeviceProtocol</i>	1	Protocol	Protocol code (assigned by the USB-IF). These codes are qualified by the value of the <i>bDeviceClass</i> and the <i>bDeviceSubClass</i> fields. If a device supports class-specific protocols on a device basis as opposed to an interface basis, this code identifies the protocols that the device uses as defined by the specification of the device class. If this field is reset to zero, the device does not use class-specific protocols on a device basis. However, it may use class-specific protocols on an interface basis. If this field is set to FFH, the device uses a vendor-specific protocol on a device basis.
7	<i>bMaxPacketSize0</i>	1	Number	Maximum packet size for endpoint zero. The <i>bMaxPacketSize0</i> value is used as the exponent for a $2^{bMaxPacketSize0}$ value; e.g., a <i>bMaxPacketSize0</i> of 4 means a Max Packet size of 16 ($2^4 \rightarrow 16$). 09H is the only valid value in this field when operating at Gen X speed.
8	<i>idVendor</i>	2	ID	Vendor ID (assigned by the USB-IF)
10	<i>idProduct</i>	2	ID	Product ID (assigned by the manufacturer)
12	<i>bcdDevice</i>	2	BCD	Device release number in binary-coded decimal
14	<i>iManufacturer</i>	1	Index	Index of string descriptor describing manufacturer
15	<i>iProduct</i>	1	Index	Index of string descriptor describing product
16	<i>iSerialNumber</i>	1	Index	Index of string descriptor describing the device's serial number
17	<i>bNumConfigurations</i>	1	Number	Number of possible configurations

9.6.2 Binary Device Object Store (BOS)

This section defines a flexible and extensible framework for describing and adding device-level capabilities to the set of USB standard specifications. As mentioned above, there exists a device descriptor, but all device-level capability extensions are defined using the following framework.

The BOS descriptor defines a root descriptor that is similar to the configuration descriptor, and is the base descriptor for accessing a family of related descriptors. A host can read a BOS descriptor and learn from the *wTotalLength* field the entire size of the device-level descriptor set, or it can read in the entire BOS descriptor set of device capabilities. The host accesses this descriptor using the GetDescriptor() request. The descriptor type in the GetDescriptor() request is set to BOS (see Table 9-12). There is no way for a host to read individual device capability descriptors. The entire set can only be accessed via reading the BOS descriptor with a GetDescriptor() request and using the length reported in the *wTotalLength* field.

Table 9-12 – BOS Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of descriptor
1	<i>bDescriptorType</i>	1	Constant	BOS Descriptor type
2	<i>wTotalLength</i>	2	Number	Length of this descriptor and all of its sub descriptors
4	<i>bNumDeviceCaps</i>	1	Number	The number of separate device capability descriptors in the BOS

Individual technology-specific or generic device-level capabilities are reported via Device Capability descriptors. The format of the Device Capability descriptor is defined in Table 9-13. The Device Capability descriptor has a generic header, with a sub-type field (*bDevCapabilityType*) which defines the layout of the remainder of the descriptor. The codes for *bDevCapabilityType* are defined in Table 9-14.

Table 9-13 – Format of a Device Capability Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor.
1	<i>bDescriptorType</i>	1	Constant	Descriptor type: DEVICE CAPABILITY Type.
2	<i>bDevCapabilityType</i>	1	Number	Valid values are listed in Table 9-14.
3	<i>Capability-Dependent</i>	Var	Variable	Capability-specific format.

Device Capability descriptors are always returned as part of the BOS information returned by a GetDescriptor(BOS) request. A Device Capability cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

Table 9-14 – Device Capability Type Codes

Capability Code	Value	Description
Wireless_USB	01H	Defines the set of Wireless USB-specific device level capabilities
USB 2.0 EXTENSION	02H	USB 2.0 Extension Descriptor
SUPERSPEED_USB	03H	Defines the set of SuperSpeed USB specific device level capabilities
CONTAINER_ID	04H	Defines the instance unique ID used to identify the instance across all operating modes
PLATFORM	05H	Defines a device capability specific to a particular platform/operating system
POWER_DELIVERY_CAPABILITY	06H	Defines the various PD Capabilities of this device
BATTERY_INFO_CAPABILITY	07H	Provides information on each battery supported by the device
PD_CONSUMER_PORT_CAPABILITY	08H	The consumer characteristics of a port on the device
PD_PROVIDER_PORT_CAPABILITY	09H	The provider characteristics of a port on the device
SUPERSPEED_PLUS	0AH	Defines the set of SuperSpeed Plus USB specific device level capabilities
PRECISION_TIME_MEASUREMENT	0BH	Precision Time Measurement (PTM) Capability Descriptor
Wireless_USB_Ext	0CH	Defines the set of Wireless USB 1.1-specific device level capabilities
Reserved	00H, 0D-FFH	Reserved for future use

The following section defines the USB 2.0 Extension Descriptor, the SuperSpeed USB Device Capability, the SuperSpeedPlus Capability and the Container ID (if supported) that a USB device shall return when operating at Gen X speed or in any of the USB 2.0 speeds.

9.6.2.1 USB 2.0 Extension

An Enhanced SuperSpeed device shall include the USB 2.0 Extension descriptor and shall support LPM when operating in USB 2.0 High-Speed mode.

Table 9-15 – USB 2.0 Extension Descriptor

Offset	Field	Size	Value	Description										
0	<i>bLength</i>	1	Number	Size of descriptor										
1	<i>bDescriptorType</i>	1	Constant	DEVICE CAPABILITY Descriptor type										
2	<i>bDevCapabilityType</i>	1	Constant	Capability type: USB 2.0 EXTENSION										
3	<i>bmAttributes</i>	4	Bitmap	Bitmap encoding of supported device level features. A value of one in a bit location indicates a feature is supported; a value of zero indicates it is not supported. Encodings are: <table><tr><th><u>Bit</u></th><th><u>Encoding</u></th></tr><tr><td>0</td><td>Reserved. Shall be set to zero.</td></tr><tr><td>1</td><td>LPM. A value of one in this bit location indicates that this device supports the Link Power Management protocol.</td></tr><tr><td></td><td>Enhanced SuperSpeed devices shall set this bit to one.</td></tr><tr><td>31:2</td><td>Reserved. Shall be set to zero.</td></tr></table>	<u>Bit</u>	<u>Encoding</u>	0	Reserved. Shall be set to zero.	1	LPM. A value of one in this bit location indicates that this device supports the Link Power Management protocol.		Enhanced SuperSpeed devices shall set this bit to one.	31:2	Reserved. Shall be set to zero.
<u>Bit</u>	<u>Encoding</u>													
0	Reserved. Shall be set to zero.													
1	LPM. A value of one in this bit location indicates that this device supports the Link Power Management protocol.													
	Enhanced SuperSpeed devices shall set this bit to one.													
31:2	Reserved. Shall be set to zero.													

9.6.2.2 SuperSpeed USB Device Capability

This section defines the required device-level capabilities descriptor which shall be implemented by all Enhanced SuperSpeed devices. This capability descriptor cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

Table 9-16 – SuperSpeed Device Capability Descriptor

Offset	Field	Size	Value	Description																		
0	<i>bLength</i>	1	Number	Size of descriptor																		
1	<i>bDescriptorType</i>	1	Constant	DEVICE CAPABILITY Descriptor type																		
2	<i>bDevCapabilityType</i>	1	Constant	Capability type: SUPERSPEED_USB																		
3	<i>bmAttributes</i>	1	Bitmap	Bitmap encoding of supported device level features. A value of one in a bit location indicates a feature is supported; a value of zero indicates it is not supported. Encodings are: <table><tr><th>Bit</th><th>Encoding</th></tr><tr><td>0</td><td>Reserved. Shall be set to zero.</td></tr><tr><td>1</td><td>LTM Capable. A value of one in this bit location indicates that this device has is capable of generating Latency Tolerance Messages.</td></tr><tr><td>7:2</td><td>Reserved. Shall be set to zero.</td></tr></table>	Bit	Encoding	0	Reserved. Shall be set to zero.	1	LTM Capable. A value of one in this bit location indicates that this device has is capable of generating Latency Tolerance Messages.	7:2	Reserved. Shall be set to zero.										
Bit	Encoding																					
0	Reserved. Shall be set to zero.																					
1	LTM Capable. A value of one in this bit location indicates that this device has is capable of generating Latency Tolerance Messages.																					
7:2	Reserved. Shall be set to zero.																					
4	<i>wSpeedsSupported</i>	2	Bitmap	Bitmap encoding of the speed supported by this device. <table><tr><th>Bit</th><th>Encoding</th></tr><tr><td>0</td><td>If this bit is set, then the device supports operation at low-Speed USB.</td></tr><tr><td>1</td><td>If this bit is set, then the device supports operation at full-Speed USB.</td></tr><tr><td>2</td><td>If this bit is set, then the device supports operation at high-Speed USB.</td></tr><tr><td>3</td><td>If this bit is set, then the device supports operation at Gen 1 speed.</td></tr><tr><td>15:4</td><td>Reserved. Shall be set to zero.</td></tr></table>	Bit	Encoding	0	If this bit is set, then the device supports operation at low-Speed USB.	1	If this bit is set, then the device supports operation at full-Speed USB.	2	If this bit is set, then the device supports operation at high-Speed USB.	3	If this bit is set, then the device supports operation at Gen 1 speed.	15:4	Reserved. Shall be set to zero.						
Bit	Encoding																					
0	If this bit is set, then the device supports operation at low-Speed USB.																					
1	If this bit is set, then the device supports operation at full-Speed USB.																					
2	If this bit is set, then the device supports operation at high-Speed USB.																					
3	If this bit is set, then the device supports operation at Gen 1 speed.																					
15:4	Reserved. Shall be set to zero.																					
6	<i>bFunctionalitySupport</i>	1	Number	The lowest speed at which all the functionality supported by the device is available to the user. For example if the device supports all its functionality when connected at full speed and above then it sets this value to 1. Refer to the <i>wSpeedsSupported</i> field for valid values that can be placed in this field.																		
7	<i>bU1DevExitLat</i>	1	Number	U1 Device Exit Latency. Worst-case latency to transition from U1 to U0, assuming the latency is limited only by the device and not the device’s link partner. This field applies only to the exit latency associated with an individual port, and does not apply to the total latency through a hub (e.g., from downstream port to upstream port). The following are permissible values: <table><tr><th>Value</th><th>Meaning</th></tr><tr><td>00H</td><td>Zero.</td></tr><tr><td>01H</td><td>Less than 1 μs</td></tr><tr><td>02H</td><td>Less than 2 μs</td></tr><tr><td>03H</td><td>Less than 3 μs</td></tr><tr><td>04H</td><td>Less than 4 μs</td></tr><tr><td>...</td><td>...</td></tr><tr><td>0AH</td><td>Less than 10 μs</td></tr><tr><td>0BH – FFH</td><td>Reserved</td></tr></table> For a hub, this is the value for both its upstream and downstream ports.	Value	Meaning	00H	Zero.	01H	Less than 1 μs	02H	Less than 2 μs	03H	Less than 3 μs	04H	Less than 4 μs	...	...	0AH	Less than 10 μs	0BH – FFH	Reserved
Value	Meaning																					
00H	Zero.																					
01H	Less than 1 μs																					
02H	Less than 2 μs																					
03H	Less than 3 μs																					
04H	Less than 4 μs																					
...	...																					
0AH	Less than 10 μs																					
0BH – FFH	Reserved																					

Offset	Field	Size	Value	Description																		
8	wU2DevExitLat	2	Number	U2 Device Exit Latency. Worst-case latency to transition from U2 to U0, assuming the latency is limited only by the device and not the device's link partner. Applies to all ports on a device. The following are permissible values: <table><tr><th><u>Value</u></th><th><u>Meaning</u></th></tr><tr><td>0000H</td><td>Zero</td></tr><tr><td>0001H</td><td>Less than 1 μs</td></tr><tr><td>0002H</td><td>Less than 2 μs</td></tr><tr><td>0003H</td><td>Less than 3 μs</td></tr><tr><td>0004H</td><td>Less than 4 μs</td></tr><tr><td>...</td><td>...</td></tr><tr><td>07FFH</td><td>Less than 2047 μs</td></tr><tr><td>0800H – FFFFH</td><td>Reserved</td></tr></table> For a hub, this is the value for both its upstream and downstream ports.	<u>Value</u>	<u>Meaning</u>	0000H	Zero	0001H	Less than 1 μs	0002H	Less than 2 μs	0003H	Less than 3 μs	0004H	Less than 4 μs	...	...	07FFH	Less than 2047 μs	0800H – FFFFH	Reserved
<u>Value</u>	<u>Meaning</u>																					
0000H	Zero																					
0001H	Less than 1 μs																					
0002H	Less than 2 μs																					
0003H	Less than 3 μs																					
0004H	Less than 4 μs																					
...	...																					
07FFH	Less than 2047 μs																					
0800H – FFFFH	Reserved																					

9.6.2.3 Container ID

This section defines the device-level Container ID descriptor which shall be implemented by all USB hubs, and is optional for other devices. If this descriptor is provided when operating in one mode, it shall be provided when operating in any mode. This descriptor may be used by a host in order to identify a unique device instance across all operating modes. If a device can also connect to a host through other technologies, the same Container ID value contained in this descriptor should also be provided over those other technologies in a technology specific manner.

This capability descriptor cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

Table 9-17 – Container ID Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of descriptor
1	<i>bDescriptorType</i>	1	Constant	DEVICE CAPABILITY Descriptor type
2	<i>bDevCapabilityType</i>	1	Constant	Capability type: CONTAINER_ID
3	<i>bReserved</i>	1	Number	This field is reserved and shall be set to zero.
4	<i>ContainerID</i>	16	UUID	This is a 128-bit number that is unique to a device instance that is used to uniquely identify the device instance across all modes of operation. This same value may be provided over other technologies as well to allow the host to identify the device independent of means of connectivity. Refer to IETF RFC 4122 for details on generation of a UUID.

9.6.2.4 Platform Descriptor

The Platform Descriptor contains a 128-bit UUID value that is defined and published independently by the platform/operating system vendor, and is used to identify a unique platform specific device capability. The descriptor may also contain one or more bytes of data associated with the capability.

Table 9-18 – Platform Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of descriptor
1	<i>bDescriptorType</i>	1	Constant	DEVICE CAPABILITY Descriptor type
2	<i>bDevCapabilityType</i>	1	Constant	Capability type: PLATFORM
3	<i>bReserved</i>	1	Number	This field is reserved and shall be set to zero.
4	<i>PlatformCapabilityUUID</i>	16	UUID	This is a 128-bit number that uniquely identifies a platform specific capability of the device.
20	<i>CapabilityData</i>	Variable	Binary	This is a variable-length field containing data associated with the platform specific capability. This field may be zero bytes in length.

9.6.2.5 SuperSpeedPlus USB Device Capability

This section defines the required device-level capabilities descriptor which shall be implemented by all SuperSpeedPlus devices. This capability descriptor cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

Table 9-19 – SuperSpeedPlus Descriptor

Offset	Field	Size	Value	Description										
0	<i>bLength</i>	1	Number	Size of descriptor										
1	<i>bDescriptorType</i>	1	Constant	DEVICE CAPABILITY Descriptor type										
2	<i>bDevCapabilityType</i>	1	Constant	Capability type: SUPERSPEED_PLUS										
3	<i>bReserved</i>	1	Number	This field is reserved and shall be set to zero.										
4	<i>bmAttributes</i>	4	Bitmap	Bitmap encoding of supported SuperSpeedPlus features: <table><tr><td>Bit</td><td>Description</td></tr><tr><td>4:0</td><td>Sublink Speed Attribute Count (SSAC). The number of Sublink Speed Attribute bitmaps. A SuperSpeedPlus device shall report at least one SSAC. The number of Sublink Speed Attributes = SSAC + 1.</td></tr><tr><td>8:5</td><td>Sublink Speed ID Count (SSIC). The number of unique Sublink Speed IDs supported by the device. The number of Sublink Speed IDs = SSIC + 1.</td></tr><tr><td>31:9</td><td>Reserved.</td></tr></table>	Bit	Description	4:0	Sublink Speed Attribute Count (SSAC). The number of Sublink Speed Attribute bitmaps. A SuperSpeedPlus device shall report at least one SSAC. The number of Sublink Speed Attributes = SSAC + 1.	8:5	Sublink Speed ID Count (SSIC). The number of unique Sublink Speed IDs supported by the device. The number of Sublink Speed IDs = SSIC + 1.	31:9	Reserved.		
Bit	Description													
4:0	Sublink Speed Attribute Count (SSAC). The number of Sublink Speed Attribute bitmaps. A SuperSpeedPlus device shall report at least one SSAC. The number of Sublink Speed Attributes = SSAC + 1.													
8:5	Sublink Speed ID Count (SSIC). The number of unique Sublink Speed IDs supported by the device. The number of Sublink Speed IDs = SSIC + 1.													
31:9	Reserved.													
8	<i>wFunctionalitySupport</i>	2	Number	The device shall support full functionality at all reported bandwidths at or above the minimum bandwidth described via this field. <table><tr><td>Bit</td><td>Description</td></tr><tr><td>3:0</td><td>Sublink Speed Attribute ID (SSID). This field indicates the minimum lane speed</td></tr><tr><td>7:4</td><td>Reserved. Shall be set to zero.</td></tr><tr><td>11:8</td><td>Min Rx Lane Count. This field indicates the minimum receive lane count.</td></tr><tr><td>15:12</td><td>Min Tx Lane Count. This field indicates the minimum transmit lane count.</td></tr></table>	Bit	Description	3:0	Sublink Speed Attribute ID (SSID). This field indicates the minimum lane speed	7:4	Reserved. Shall be set to zero.	11:8	Min Rx Lane Count. This field indicates the minimum receive lane count.	15:12	Min Tx Lane Count. This field indicates the minimum transmit lane count.
Bit	Description													
3:0	Sublink Speed Attribute ID (SSID). This field indicates the minimum lane speed													
7:4	Reserved. Shall be set to zero.													
11:8	Min Rx Lane Count. This field indicates the minimum receive lane count.													
15:12	Min Tx Lane Count. This field indicates the minimum transmit lane count.													
9	<i>wReserved</i>	2	Number	Reserved. Shall be set to zero.										

Offset	Field	Size	Value	Description																																	
12	<i>bmSublinkSpeedAttr[0]</i>	4	Bitmap	Sublink Speed Attribute. Bitmap encoding of a Sublink's characteristics: <table><thead><tr><th>Bit</th><th>Description</th></tr></thead><tbody><tr><td>3:0</td><td>Sublink Speed Attribute ID (SSID). This field is an ID that uniquely identifies the speed of the sublink. Note that a maximum of 16 unique SSIDs may be defined.</td></tr><tr><td>5:4</td><td>Lane Speed Exponent (LSE). This field defines the base 10 exponent times 3, that shall be applied to the Lane Speed Mantissa (LSM) when calculating the maximum bit rate represented by this Lane Speed Attribute.</td></tr></tbody></table> LSE Value Bit Rate <table><tbody><tr><td>0</td><td>Bits per second</td></tr><tr><td>1</td><td>Kb/s</td></tr><tr><td>2</td><td>Mb/s</td></tr><tr><td>3</td><td>Gb/s</td></tr></tbody></table> 7:6 Sublink Type (ST). This field identifies whether the Sublink Speed Attribute defines a symmetric or asymmetric bit rate. This field also indicates if this Sublink Speed Attribute defines the receive or transmit bit rate. Note that the Sublink Speed Attributes shall be paired, i.e. an Rx immediately followed by a Tx, and both Attributes shall define the same value for the SSID. <table><thead><tr><th>Bit</th><th>Value</th><th>Description</th></tr></thead><tbody><tr><td rowspan="2">6</td><td>0</td><td>Symmetric. Rx and Tx Sublinks have the same number of lanes and operate at the same speed.</td></tr><tr><td>1</td><td>Asymmetric. Rx and Tx Sublink have different number of lanes and/or operate at different speeds.</td></tr><tr><td rowspan="2">7</td><td>0</td><td>Sublink operates in Receive mode</td></tr><tr><td>1</td><td>Sublink operates in Transmit mode</td></tr></tbody></table> 13:8 Reserved. 15:14 Link Protocol (LP). This field identifies the protocol supported by the link. LP Value Protocol <table><tbody><tr><td>0</td><td>SuperSpeed</td></tr><tr><td>1</td><td>SuperSpeedPlus</td></tr><tr><td>3-2</td><td>Reserved.</td></tr></tbody></table> 31:16 Lane Speed Mantissa (LSM). This field defines the mantissa that shall be applied to the LSE when calculating the maximum bit rate represented by Lane Speed Attribute.	Bit	Description	3:0	Sublink Speed Attribute ID (SSID). This field is an ID that uniquely identifies the speed of the sublink. Note that a maximum of 16 unique SSIDs may be defined.	5:4	Lane Speed Exponent (LSE). This field defines the base 10 exponent times 3, that shall be applied to the Lane Speed Mantissa (LSM) when calculating the maximum bit rate represented by this Lane Speed Attribute.	0	Bits per second	1	Kb/s	2	Mb/s	3	Gb/s	Bit	Value	Description	6	0	Symmetric. Rx and Tx Sublinks have the same number of lanes and operate at the same speed.	1	Asymmetric. Rx and Tx Sublink have different number of lanes and/or operate at different speeds.	7	0	Sublink operates in Receive mode	1	Sublink operates in Transmit mode	0	SuperSpeed	1	SuperSpeedPlus	3-2	Reserved.
Bit	Description																																				
3:0	Sublink Speed Attribute ID (SSID). This field is an ID that uniquely identifies the speed of the sublink. Note that a maximum of 16 unique SSIDs may be defined.																																				
5:4	Lane Speed Exponent (LSE). This field defines the base 10 exponent times 3, that shall be applied to the Lane Speed Mantissa (LSM) when calculating the maximum bit rate represented by this Lane Speed Attribute.																																				
0	Bits per second																																				
1	Kb/s																																				
2	Mb/s																																				
3	Gb/s																																				
Bit	Value	Description																																			
6	0	Symmetric. Rx and Tx Sublinks have the same number of lanes and operate at the same speed.																																			
	1	Asymmetric. Rx and Tx Sublink have different number of lanes and/or operate at different speeds.																																			
7	0	Sublink operates in Receive mode																																			
	1	Sublink operates in Transmit mode																																			
0	SuperSpeed																																				
1	SuperSpeedPlus																																				
3-2	Reserved.																																				
12+(4*SSAC)	<i>bmSublinkSpeedAttr[1-SSAC]</i>	4	Bitmap	Sublink Speed Attribute. Additional Lane Speed Attributes associated with this device.																																	

9.6.2.6 Precision Time Measurement

This section defines the required device-level capabilities descriptor which shall be implemented by all hubs and devices that support the PTM capability. This capability descriptor cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

Table 9-20 – PTM Capability Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of descriptor
1	<i>bDescriptorType</i>	1	Constant	DEVICE CAPABILITY Descriptor type
2	<i>bDevCapabilityType</i>	1	Constant	Capability type: PRECISION_TIME_MEASUREMENT

9.6.3 Configuration

The configuration descriptor describes information about a specific device configuration. The descriptor contains a *bConfigurationValue* field with a value that, when used as a parameter to the SetConfiguration() request, causes the device to assume the described configuration.

The descriptor describes the number of interfaces provided by the configuration. Each interface may operate independently. For example, a Video Class device might be configured with two interfaces, each providing 64-MBps bi-directional channels that have separate data sources or sinks on the host. Another configuration might present the Video Class device as a single interface, bonding the two channels into one 128-MBps bi-directional channel.

When the host requests the configuration descriptor, all related interface, endpoint, and endpoint companion descriptors are returned (refer to Section 9.4.3).

A device has one or more configuration descriptors. Each configuration has one or more interfaces and each interface has zero or more endpoints. An endpoint is not shared among interfaces within a single configuration unless the endpoint is used by alternate settings of the same interface. Endpoints may be shared among interfaces that are part of different configurations without this restriction.

Once configured, devices may support limited adjustments to the configuration. If a particular interface has alternate settings, an alternate may be selected after configuration. Table 9-21 shows the standard configuration descriptor.

Table 9-21 – Standard Configuration Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	CONFIGURATION Descriptor Type
2	<i>wTotalLength</i>	2	Number	Total length of data returned for this configuration. Includes the combined length of all descriptors (configuration, interface, endpoint, and class- or vendor-specific) returned for this configuration
4	<i>bNumInterfaces</i>	1	Number	Number of interfaces supported by this configuration
5	<i>bConfigurationValue</i>	1	Number	Value to use as an argument to the SetConfiguration() request to select this configuration
6	<i>iConfiguration</i>	1	Index	Index of string descriptor describing this configuration
7	<i>bmAttributes</i>	1	Bitmap	Configuration characteristics: D7: Reserved (set to one) D6: Self-powered D5: Remote Wakeup D4...0: Reserved (reset to zero) D7 is reserved and shall be set to one for historical reasons. A device configuration that uses power from the bus and a local source reports a non-zero value in <i>bMaxPower</i> to indicate the amount of bus power required and sets D6. The actual power source at runtime may be determined using the GetStatus(DEVICE) request (refer to Section 9.4.5). If a device configuration supports remote wakeup, D5 is set to one.
8	<i>bMaxPower</i>	1	mA	Maximum power consumption of the device from the bus in this specific configuration when the device is fully operational. Expressed in 2-mA units when the device is operating in high-speed mode and in 8-mA units when operating at Gen X speed. (i.e., 50 = 100 mA when operating at high-speed and 50 = 400 mA when operating at Gen X speed). Note: A device configuration reports whether the configuration is bus-powered or self-powered. Device status reports whether the device is currently self-powered. If a device is disconnected from its external power source, it updates device status to indicate that it is no longer self-powered. A device may not increase its power draw from the bus, when it loses its external power source, beyond the amount reported by its configuration. If a device can continue to operate when disconnected from its external power source, it continues to do so. If the device cannot continue to operate, it shall return to the Powered state.

9.6.4 Interface Association

The Interface Association Descriptor is used to describe that two or more interfaces are associated to the same function. An “association” includes two or more interfaces and all of their alternate setting interfaces. A device must use an Interface Association descriptor for each device function that requires more than one interface. An Interface Association descriptor is always returned as part of the configuration information returned by a

GetDescriptor(Configuration) request. An interface association descriptor cannot be directly accessed with a GetDescriptor() or SetDescriptor() request. An interface association descriptor must be located before the set of interface descriptors (including all alternate settings) for the interfaces it associates. All of the interface numbers in the set of associated interfaces must be contiguous. Table 9-22 shows the standard interface association descriptor. The interface association descriptor includes function class, subclass, and protocol fields. The values in these fields can be the same as the interface class, subclass, and protocol values from any one of the associated interfaces. The preferred implementation, for existing device classes, is to use the interface class, subclass, and protocol field values from the first interface in the list of associated interfaces.

Table 9-22 – Standard Interface Association Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	INTERFACE ASSOCIATION Descriptor
2	<i>bFirstInterface</i>	1	Number	Interface number of the first interface that is associated with this function
3	<i>bInterfaceCount</i>	1	Number	Number of contiguous interfaces that are associated with this function
4	<i>bFunctionClass</i>	1	Class	Class code (assigned by USB-IF). A value of zero is not allowed in this descriptor. If this field is FFH, the function class is vendor-specific. All other values are reserved for assignment by the USB-IF.
5	<i>bFunctionSubClass</i>	1	SubClass	Subclass code (assigned by USB-IF). If the <i>bFunctionClass</i> field is not set to FFH, all values are reserved for assignment by the USB-IF.
6	<i>bFunctionProtocol</i>	1	Protocol	Protocol code (assigned by USB-IF). These codes are qualified by the values of the <i>bFunctionClass</i> and <i>bFunctionSubClass</i> fields.
7	<i>iFunction</i>	1	Index	Index of string descriptor describing this function



NOTE

Since this particular feature was not included in earlier versions of the USB specification, there is an issue with how existing USB operating system implementations will support devices that use this descriptor. It is strongly recommended that device implementations utilizing the interface association descriptor use the Multi-interface Function Class codes in the device descriptor. This allows simple and easy identification of these devices and allows on some operating systems, installation of an upgrade driver that can parse and enumerate configurations that include the Interface Association Descriptor. The Multi-interface Function Class code is documented at <http://www.usb.org/developers/docs>.

9.6.5 Interface

The interface descriptor describes a specific interface within a configuration. A configuration provides one or more interfaces, each with zero or more endpoint descriptors. When a configuration supports more than one interface, the endpoint descriptors for a particular interface follow the interface descriptor in the data returned by the GetConfiguration() request. As mentioned earlier in this chapter, Enhanced SuperSpeed devices shall return Endpoint Companion descriptors for each of the endpoints in that interface to return additional information about its endpoint capabilities. The Endpoint Companion descriptor shall immediately follow the endpoint descriptor it is associated with in the configuration information. An interface descriptor is always returned as part of a configuration descriptor. Interface descriptors cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

An interface may include alternate settings that allow the endpoints and/or their characteristics to be varied after the device has been configured. The default setting for an interface is always alternate setting zero. The `SetInterface()` request is used to select an alternate setting or to return to the default setting. The `GetInterface()` request returns the selected alternate setting.

Alternate settings allow a portion of the device configuration to be varied while other interfaces remain in operation. If a configuration has alternate settings for one or more of its interfaces, a separate interface descriptor and its associated endpoint and endpoint companion (when reporting its Enhanced SuperSpeed configuration) descriptors are included for each setting.

If a device configuration supported a single interface with two alternate settings, the configuration descriptor would be followed by an interface descriptor with the *bInterfaceNumber* and *bAlternateSetting* fields set to zero and then the endpoint and endpoint companion (when reporting its Enhanced SuperSpeed configuration) descriptors for that setting, followed by another interface descriptor and its associated endpoint and endpoint companion descriptors. The second interface descriptor's *bInterfaceNumber* field would also be set to zero, but the *bAlternateSetting* field of the second interface descriptor would be set to one.

If an interface uses only the Default Control Pipe, no endpoint descriptors follow the interface descriptor. In this case, the *bNumEndpoints* field shall be set to zero.

An interface descriptor never includes the Default Control Pipe in the number of endpoints. Table 9-23 shows the standard interface descriptor.

Table 9-23 – Standard Interface Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	INTERFACE Descriptor Type
2	<i>bInterfaceNumber</i>	1	Number	Number of this interface. Zero-based value identifying the index in the array of concurrent interfaces supported by this configuration.
3	<i>bAlternateSetting</i>	1	Number	Value used to select this alternate setting for the interface identified in the prior field
4	<i>bNumEndpoints</i>	1	Number	Number of endpoints used by this interface (excluding the Default Control Pipe). If this value is zero, this interface only uses the Default Control Pipe.
5	<i>bInterfaceClass</i>	1	Class	Class code (assigned by the USB-IF). A value of zero is reserved for future standardization. If this field is set to FFH, the interface class is vendor-specific. All other values are reserved for assignment by the USB-IF.
6	<i>bInterfaceSubClass</i>	1	SubClass	Subclass code (assigned by the USB-IF). These codes are qualified by the value of the <i>bInterfaceClass</i> field. If the <i>bInterfaceClass</i> field is reset to zero, this field shall also be reset to zero. If the <i>bInterfaceClass</i> field is not set to FFH, all values are reserved for assignment by the USB-IF.
7	<i>bInterfaceProtocol</i>	1	Protocol	Protocol code (assigned by the USB). These codes are qualified by the value of the <i>bInterfaceClass</i> and the <i>bInterfaceSubClass</i> fields. If an interface supports class-specific requests, this code identifies the protocols that the device uses as defined by the specification of the device class. If this field is reset to zero, the device does not use a class-specific protocol on this interface. If this field is set to FFH, the device uses a vendor-specific protocol for this interface.
8	<i>iInterface</i>	1	Index	Index of string descriptor describing this interface

9.6.6 Endpoint

Each endpoint used for an interface has its own descriptor. This descriptor contains the information required by the host to determine the bandwidth requirements of each endpoint. An endpoint descriptor is always returned as part of the configuration information returned by a `GetDescriptor(Configuration)` request. An endpoint descriptor cannot be directly accessed with a `GetDescriptor()` or `SetDescriptor()` request. There is never an endpoint descriptor for endpoint zero. Table 9-24 shows the standard endpoint descriptor.

Table 9-24 – Standard Endpoint Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	ENDPOINT Descriptor Type
2	<i>bEndpointAddress</i>	1	Endpoint	The address of the endpoint on the device described by this descriptor. The address is encoded as follows: Bit 3...0: The endpoint number Bit 6...4: Reserved, reset to zero Bit 7: Direction, ignored for control endpoints 0 = OUT endpoint 1 = IN endpoint
3	<i>bmAttributes</i>	1	Bitmap	This field describes the endpoint's attributes when it is configured using the <i>bConfigurationValue</i>. Bits 1..0: Transfer Type 00 = Control 01 = Isochronous 10 = Bulk 11 = Interrupt If an interrupt endpoint, bits 5..2 are defined as follows: Bits 3..2: Reserved Bits 5..4: Usage Type 00 = Periodic 01 = Notification 10 = Reserved 11 = Reserved If isochronous, they are defined as follows: Bits 3..2: Synchronization Type 00 = No Synchronization 01 = Asynchronous 10 = Adaptive 11 = Synchronous Bits 5..4 Usage Type 00 = Data endpoint 01 = Feedback endpoint 10 = Implicit feedback Data endpoint 11 = Reserved If not an isochronous or interrupt endpoint, bits 5..2 are reserved and shall be set to zero. All other bits are reserved and shall be reset to zero. Reserved bits shall be ignored by the host.
4	<i>wMaxPacketSize</i>	2	Number	Maximum packet size this endpoint is capable of sending or receiving when this configuration is selected. For control endpoints this field shall be set to 512. For bulk endpoint types this field shall be set to 1024. For interrupt and isochronous endpoints this field shall be set to 1024 if this endpoint defines a value in the bMaxBurst field greater than zero. If the value in the bMaxBurst field is set to zero then this field can have any value from 0 to 1024 for an isochronous endpoint and 1 to 1024 for an interrupt endpoint.
6	<i>bInterval</i>	1	Number	Interval for servicing the endpoint for data transfers. Expressed in 125-μs units. For Enhanced SuperSpeed isochronous and interrupt endpoints, this value shall be in the range from 1 to 16. However, the valid ranges are 8 to 16 for Notification type Interrupt endpoints. The <i>bInterval</i> value is used as the exponent for a $2^{(bInterval-1)}$ value; e.g., a <i>bInterval</i> of 4 means a period of 8 ($2^{(4-1)} \rightarrow 2^3 \rightarrow 8$). This field is reserved and shall not be used for Enhanced SuperSpeed bulk or control endpoints.

The *bmAttributes* field provides information about the endpoint's Transfer Type (bits 1..0) and Synchronization Type (bits 3..2). For interrupt endpoints, the Usage Type bits (bits 5..4) indicate whether the endpoint is used for infrequent notifications that can tolerate varying latencies (bits 5..4 = 01b), or if it regularly transfers data in consecutive service intervals or is dependent on bounded latencies (bits 5..4 = 00b). For example, a hub's interrupt endpoint would specify that it is a notification type, while a mouse would specify a periodic type. For endpoints that sometimes operate in infrequent notification mode and at other times operate in periodic mode then this field shall be set to Periodic (bits 5..4 = 00b). These values may be used by software to determine appropriate power management settings. See Appendix C for details on how this value may effect power management. In addition, for isochronous endpoints the Usage Type bit (bits 5..4) indicate whether this is an endpoint used for normal data transfers (bits 5..4 = 00b), whether it is used to convey explicit feedback information for one or more data endpoints (bits 5..4 = 01b) or whether it is a data endpoint that also serves as an implicit feedback endpoint for one or more data endpoints (bits 5..4=10b).

If the endpoint is used as an explicit feedback endpoint (bits 5..4 = 01b), then the Transfer Type shall be set to isochronous (bits 1..0 = 01b) and the Synchronization Type shall be set to No Synchronization (bits 3..2 = 00b).

A feedback endpoint (explicit or implicit) needs to be associated with one (or more) isochronous data endpoints to which it provides feedback service. The association is based on endpoint number matching. A feedback endpoint always has the opposite direction from the data endpoint(s) it services. If multiple data endpoints are to be serviced by the same feedback endpoint, the data endpoints shall have ascending ordered—but not necessarily consecutive—endpoint numbers. The first data endpoint and the feedback endpoint shall have the same endpoint number (and opposite direction). This ensures that a data endpoint can uniquely identify its feedback endpoint by searching for the first feedback endpoint that has an endpoint number equal or less than its own endpoint number.

Example: Consider the extreme case where there is a need for five groups of OUT asynchronous isochronous endpoints and at the same time four groups of IN adaptive isochronous endpoints. Each group needs a separate feedback endpoint and the groups are composed as shown in Table 9-25.

Table 9-25 – Example of Feedback Endpoint Numbers

OUT Group	Number of OUT Endpoints	IN Group	Number of IN Endpoints
1	1	6	1
2	2	7	2
3	2	8	3
4	3	9	4
5	3		

The endpoint numbers can be intertwined as illustrated in Figure 9-8.

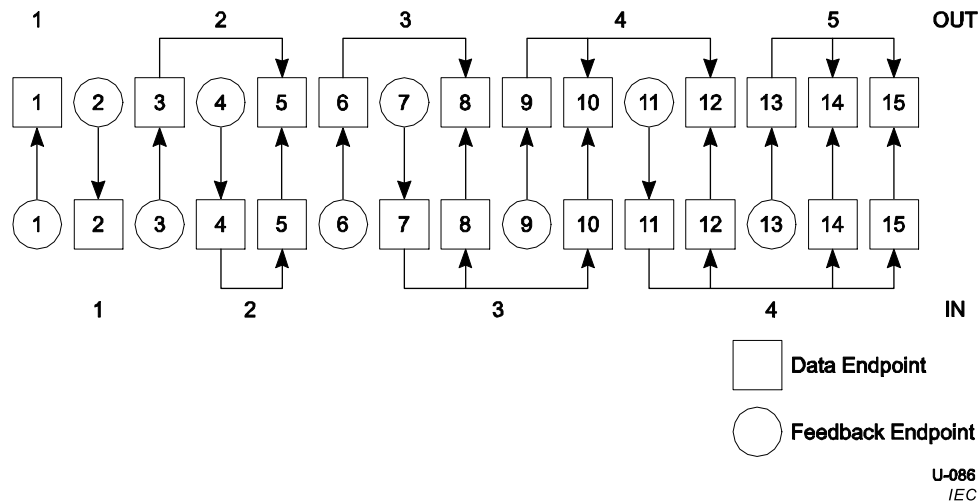


Figure 9-8 – Example of Feedback Endpoint Relationships

For high-speed bulk and control OUT endpoints, the *bInterval* field is only used for compliance purposes; the host controller is not required to change its behavior based on the value in this field.

9.6.7 SuperSpeed Endpoint Companion

This descriptor shall only be returned by Enhanced SuperSpeed devices that are operating at Gen X speed. Each endpoint described in an interface is followed by a SuperSpeed Endpoint Companion descriptor. This descriptor is returned as part of the configuration information returned by a `GetDescriptor(Configuration)` request and cannot be directly accessed with a `GetDescriptor()` or `SetDescriptor()` request. The Default Control Pipe does not have an Endpoint Companion descriptor. The Endpoint Companion descriptor shall immediately follow the endpoint descriptor it is associated with in the configuration information.

Table 9-26 – SuperSpeed Endpoint Companion Descriptor

Offset	Field	Size	Value	Description																		
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes																		
1	<i>bDescriptorType</i>	1	Constant	SUPERSPEED_USB_ENDPOINT_COMPANION Descriptor Type																		
2	<i>bMaxBurst</i>	1	Number	The maximum number of packets the endpoint can send or receive as part of a burst. Valid values are from 0 to 15. A value of 0 indicates that the endpoint can only burst one packet at a time and a value of 15 indicates that the endpoint can burst up to 16 packets at a time. For endpoints of type control this shall be set to 0.																		
3	<i>bmAttributes</i>	1	Bitmap	If this is a Bulk Endpoint: <table><thead><tr><th>Bits</th><th>Description</th></tr></thead><tbody><tr><td>4:0</td><td>MaxStreams. The maximum number of streams this endpoint supports. Valid values are from 0 to 16, where a value of 0 indicates that the endpoint does not define streams. For the values 1 to 16, the number of streams supported equals $2^{\text{MaxStream}}$.</td></tr><tr><td>7:5</td><td>Reserved. These bits are reserved and shall be set to zero.</td></tr></tbody></table> If this is a Control or Interrupt Endpoint: <table><thead><tr><th>Bits</th><th>Description</th></tr></thead><tbody><tr><td>7:0</td><td>Reserved. These bits are reserved and shall be set to zero.</td></tr></tbody></table> If this is an isochronous endpoint: <table><thead><tr><th>Bits</th><th>Description</th></tr></thead><tbody><tr><td>1:0</td><td>Mult. A zero based value that determines the maximum number of packets within a service interval that this endpoint supports. Maximum number of packets = (bMaxBurst + 1) x (Mult + 1) The maximum value that can be set in this field is 2. This field shall be set to zero if the bMaxBurst field is set to zero.</td></tr><tr><td>6:2</td><td>Reserved. These bits are reserved and shall be set to zero.</td></tr><tr><td>7</td><td>SSP ISO Companion. If this field is set to one then a SuperSpeedPlus Isochronous Endpoint Companion descriptor shall immediately follow this descriptor and the value in the Mult field shall be ignored. The actual Mult shall be determined as follows: $\text{dwBytesPerInterval} / \text{bMaxBurst} / \text{wMaxPacketSize}$ rounded up to the nearest integer value..</td></tr></tbody></table>	Bits	Description	4:0	MaxStreams. The maximum number of streams this endpoint supports. Valid values are from 0 to 16, where a value of 0 indicates that the endpoint does not define streams. For the values 1 to 16, the number of streams supported equals $2^{\text{MaxStream}}$.	7:5	Reserved. These bits are reserved and shall be set to zero.	Bits	Description	7:0	Reserved. These bits are reserved and shall be set to zero.	Bits	Description	1:0	Mult. A zero based value that determines the maximum number of packets within a service interval that this endpoint supports. Maximum number of packets = (bMaxBurst + 1) x (Mult + 1) The maximum value that can be set in this field is 2. This field shall be set to zero if the bMaxBurst field is set to zero.	6:2	Reserved. These bits are reserved and shall be set to zero.	7	SSP ISO Companion. If this field is set to one then a SuperSpeedPlus Isochronous Endpoint Companion descriptor shall immediately follow this descriptor and the value in the Mult field shall be ignored. The actual Mult shall be determined as follows: $\text{dwBytesPerInterval} / \text{bMaxBurst} / \text{wMaxPacketSize}$ rounded up to the nearest integer value..
Bits	Description																					
4:0	MaxStreams. The maximum number of streams this endpoint supports. Valid values are from 0 to 16, where a value of 0 indicates that the endpoint does not define streams. For the values 1 to 16, the number of streams supported equals $2^{\text{MaxStream}}$.																					
7:5	Reserved. These bits are reserved and shall be set to zero.																					
Bits	Description																					
7:0	Reserved. These bits are reserved and shall be set to zero.																					
Bits	Description																					
1:0	Mult. A zero based value that determines the maximum number of packets within a service interval that this endpoint supports. Maximum number of packets = (bMaxBurst + 1) x (Mult + 1) The maximum value that can be set in this field is 2. This field shall be set to zero if the bMaxBurst field is set to zero.																					
6:2	Reserved. These bits are reserved and shall be set to zero.																					
7	SSP ISO Companion. If this field is set to one then a SuperSpeedPlus Isochronous Endpoint Companion descriptor shall immediately follow this descriptor and the value in the Mult field shall be ignored. The actual Mult shall be determined as follows: $\text{dwBytesPerInterval} / \text{bMaxBurst} / \text{wMaxPacketSize}$ rounded up to the nearest integer value..																					

Offset	Field	Size	Value	Description
4	<i>wBytesPerInterval</i>	2	Number	The total number of bytes this endpoint will transfer every service interval (SI). This field is only valid for periodic endpoints. For isochronous endpoints: a) If the <i>SSP ISO Companion</i> bit in the bmAttributes field is set to zero, this value is used to reserve the bus time in the schedule, required for the frame data payloads per SI. The pipe may, on an ongoing basis, actually use less bandwidth than that reserved. The device reports, if necessary, the actual bandwidth used via its normal, non-USB defined mechanisms. b) If the <i>SSP ISO Companion</i> bit in the bmAttributes field is set to one, this field shall be set to one, and the total number of bytes this endpoint will transfer shall be reported via the endpoint's SuperSpeedPlus Isochronous Endpoint Companion descriptor. <i>wBytesPerInterval</i> is reserved and must be set to zero for control and bulk endpoints.

9.6.8 SuperSpeedPlus Isochronous Endpoint Companion

This descriptor contains additional endpoint characteristics that are only defined for endpoints of devices operating at above Gen 1 speed. This descriptor shall only be returned (as part of the devices' complete configuration descriptor) by an Enhanced SuperSpeed device that is operating at above Gen 1 speed. This descriptor shall be returned for each Isochronous endpoint that requires more than 48K bytes per Service Interval.

This descriptor is returned as part of the configuration information returned by a GetDescriptor(Configuration) request and cannot be directly accessed with a GetDescriptor() or SetDescriptor() request.

The SuperSpeedPlus Isochronous Endpoint Companion descriptor shall immediately follow the SuperSpeed Endpoint Companion descriptor that follows the Isochronous endpoint descriptor in the configuration information.

When an alternate setting is selected that has an Isochronous endpoint that has a SuperSpeedPlus Isochronous Endpoint Companion descriptor the endpoint shall operate with the characteristics as described in the SuperSpeedPlus Isochronous Endpoint Companion descriptor.

Table 9-27 – SuperSpeedPlus Isochronous Endpoint Companion Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	SUPERSPEEDPLUS_ISOCHRONOUS_ENDPOINT_COMPANION Descriptor Type
3	<i>wReserved</i>	2	Number	Reserved. Shall be set to zero.
4	<i>dwBytesPerInterval</i>	4	Number	The total number of bytes this endpoint will transfer every service interval (SI). This value is used to reserve the bus time in the schedule, required for the frame data payloads per SI. The pipe may, on an ongoing basis, actually use less bandwidth than that reserved. The device reports, if necessary, the actual bandwidth used via its normal, non-USB defined mechanisms.

9.6.9 String

String descriptors are optional. As noted previously, if a device does not support string descriptors, all references to string descriptors within device, configuration, and interface descriptors shall be reset to zero.

String descriptors use UNICODE UTF16LE encodings as defined by *The Unicode Standard, Worldwide Character Encoding, Version 5.0*, The Unicode Consortium, Addison-Wesley Publishing Company, Reading, Massachusetts (<http://www.unicode.org>). The strings in a device may support multiple languages. When requesting a string descriptor, the requester specifies the desired language using a 16-bit language ID (LANGID) defined by the USB-IF. The list of currently defined USB LANGIDs can be found at <http://www.usb.org/developers/docs.html>. String index zero for all languages returns a string descriptor that contains an array of 2-byte LANGID codes supported by the device. Table 9-28 shows the LANGID code array. A device may omit all string descriptors. Devices that omit all string descriptors shall not return an array of LANGID codes.

The array of LANGID codes is not NULL-terminated. The size of the array (in bytes) is computed by subtracting two from the value of the first byte of the descriptor.

Table 9-28 – String Descriptor Zero, Specifying Languages Supported by the Device

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	N+2	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	STRING Descriptor Type
2	<i>wLANGID[0]</i>	2	Number	LANGID code zero
...	...	...	...	...
N	<i>wLANGID[x]</i>	2	Number	LANGID code x

The UNICODE string descriptor (shown in Table 9-29) is not NULL-terminated. The string length is computed by subtracting two from the value of the first byte of the descriptor.

Table 9-29 – UNICODE String Descriptor

Offset	Field	Size	Value	Description
0	<i>bLength</i>	1	Number	Size of this descriptor in bytes
1	<i>bDescriptorType</i>	1	Constant	STRING Descriptor Type
2	<i>bString</i>	N	Number	UNICODE encoded string

9.7 Device Class Definitions

All devices shall support the requests and descriptor definitions described in this chapter. Most devices provide additional requests and, possibly, descriptors for device-specific extensions. In addition, devices may provide extended services that are common to a group of devices. In order to define a class of devices, the following information shall be provided to completely define the appearance and behavior of the device class.

9.7.1 Descriptors

If the class requires any specific definition of the standard descriptors, the class definition shall include those requirements as part of the class definition. In addition, if the class defines a standard extended set of descriptors, they shall also be fully defined in the class definition. Any extended descriptor definitions shall follow the approach used for standard descriptors; for example, all descriptors shall begin with a length field.

9.7.2 Interface(s)

When a class of devices is standardized, the interfaces used by the devices shall be included in the device class definition. Devices may further extend a class definition with proprietary features as long as they meet the base definition of the class.

9.7.3 Requests

All of the requests specific to the class shall be defined.

10 Hub, Host Downstream Port, and Device Upstream Port Specification

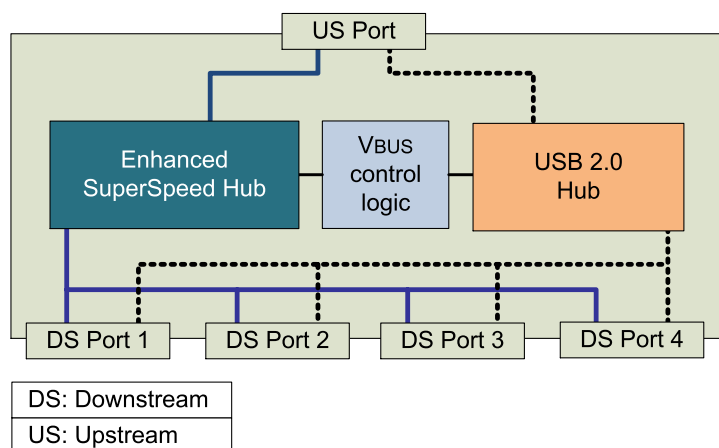
This chapter describes the architectural requirements for a hub that supports both Enhanced SuperSpeed and USB 2.0 and is referred to as a “USB hub”. The chapter also describes differences between functional requirements for a host downstream port and a hub downstream port as well as differences between a peripheral upstream port and a hub upstream port. The chapter contains the description of the Enhanced SuperSpeed hub. An Enhanced SuperSpeed hub supports all Gen X speeds. This chapter includes descriptions of the SuperSpeed sub-blocks (the SuperSpeed repeater/forwarder and the SuperSpeed Hub Controller) as well as the SuperSpeedPlus sub-blocks (the SuperSpeedPlus Upstream Controller, the SuperSpeedPlus Downstream Controller and the SuperSpeedPlus Hub Controller). This chapter also describes the hub's operation for error recovery, reset, suspend/resume, hub request behavior, and hub descriptors. The USB 2.0 hub sub-block is described in the *Universal Serial Bus Specification, Revision 2.0*.

The hub specification chapter along with the *Universal Serial Bus Specification, Revision 2.0* supply the information needed for an implementer to design a hub that conforms to this revision of the USB specification.

10.1 Hub Feature Summary

Hubs provide the electrical interface between USB devices and the host. Hubs are directly responsible for supporting many of the attributes that make USB user friendly and hide its complexity from the user. Listed below are the major aspects of USB functionality that hubs support:

- Connectivity behavior
- Power management
- Device connect/disconnect detection
- Bus fault detection and recovery
- Enhanced SuperSpeed and USB 2.0 (high-speed, full-speed, and low-speed) device support



IEC

Figure 10-1 – USB Hub Architecture

When a USB hub connects on its upstream facing port at Gen 1 speed, it shall operate (and be referred to) as a SuperSpeed hub (including operating its downstream ports at no higher than Gen 1 speed).

When a USB hub connects on its upstream facing port at a speed above Gen 1 speed, it shall operate (and be referred to) as a SuperSpeedPlus hub.

When the hub upstream facing port is attached to an electrical environment that is only operating at high-speed or full-speed, then Enhanced SuperSpeed connectivity is not available to devices attached to downstream facing ports.

Figure 10-1 shows a high level block diagram of a four port USB hub and the locations of its upstream and downstream facing ports. A USB hub is the logical combination of two hubs: a USB 2.0 hub and an Enhanced SuperSpeed hub. Each hub operates independently on a separate data bus. Typically, the only signal shared logic between them is to control VBUS. If either the USB 2.0 hub or Enhanced SuperSpeed hub controllers requires a downstream port to be powered, power is turned on for the port. A USB hub connects on both interfaces upstream whenever possible. All exposed downstream ports on a USB hub shall support both Enhanced SuperSpeed and USB 2.0 connections. Host controller ports may have different requirements.

Figure 10-2 shows the SuperSpeed portion of a USB hub consisting of a Hub Repeater/Forwarder section and a Hub Controller section.

The USB 2.0 portion of a USB hub shall meet all requirements of the *USB 2.0 Specification* unless specific exceptions are noted.

The SuperSpeed Hub Repeater/Forwarder is responsible for connectivity setup and tear-down. It also supports exception handling, such as bus fault detection and recovery and connect/disconnect detect. The SuperSpeed Hub Controller provides the mechanism for host-to-hub communication. Hub-specific status and control commands permit the host to configure a hub and to monitor and control its individual downstream facing ports.

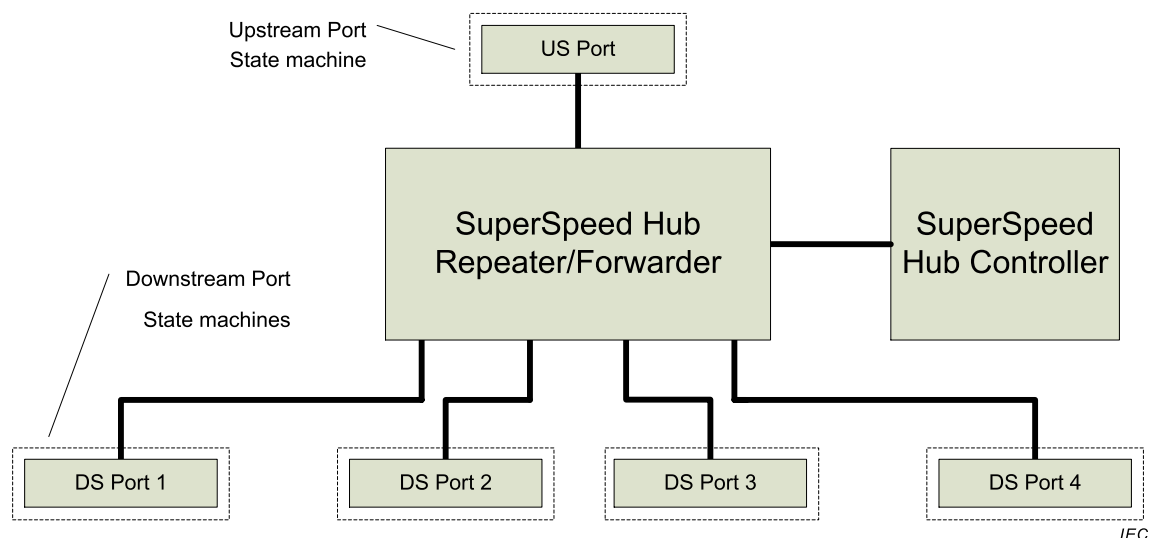


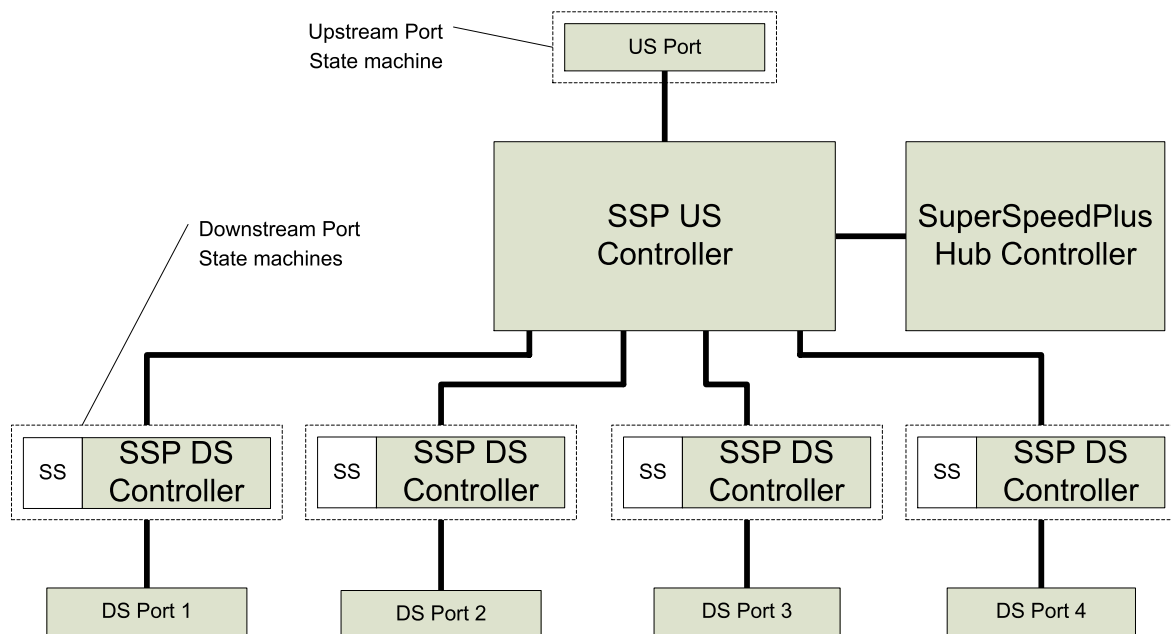
Figure 10-2 – SuperSpeed Portion of the USB Hub Architecture

As shown in Figure 10-3, the SuperSpeedPlus hub portion consists of three functional components: the SuperSpeedPlus Upstream Controller, the SuperSpeedPlus Downstream Controller and the SuperSpeedPlus Hub Controller. All subsequent references in this specification are to components of the Enhanced SuperSpeed hub unless otherwise noted.

The SuperSpeedPlus Upstream (SSP US) Controller is responsible for the behavior of the upstream port, buffering for packets being received from the upstream link, buffering and arbitrating packets waiting to be transmitted on the upstream link, and for routing packets to the appropriate downstream port's Downstream Controller (or to the hub controller).

The SuperSpeedPlus Downstream (SSP DS) Controller is responsible for the behavior of the downstream port, buffering for packets being received from the downstream link, buffering and arbitrating packets waiting to be transmitted on the downstream link and for routing packets to the Upstream Controller.

The SuperSpeedPlus Hub Controller provides the same mechanism for host-to-hub communication that the SuperSpeed Hub Controller does.



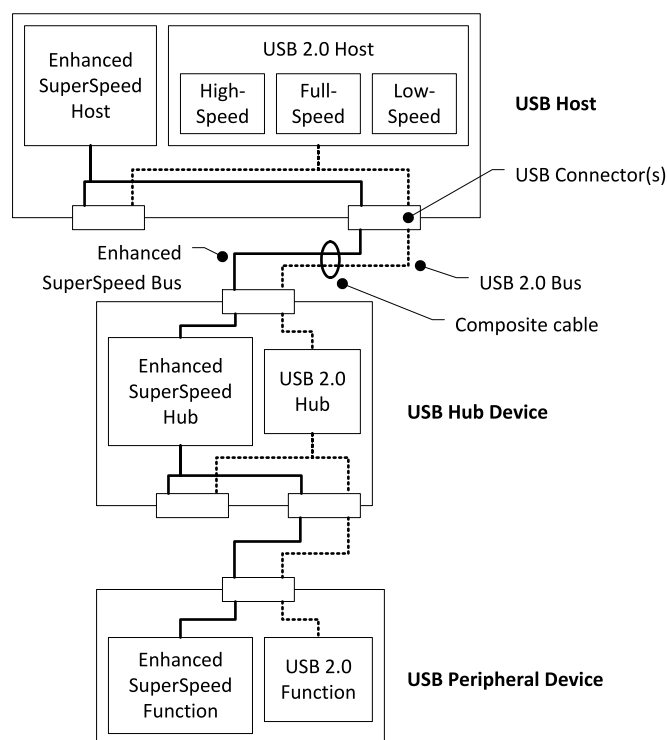
IEC

Figure 10-3 – SuperSpeedPlus Portion of the Hub Architecture

Unlike USB peripheral devices, a USB hub connects upstream on both the Enhanced SuperSpeed bus and USB 2.0 bus. Connections may be enabled or disabled under the control of system software for a USB hub's downstream ports. If a USB hub upstream port is not connected on either USB 2.0 bus or Enhanced SuperSpeed bus, the hub does not provide power to the downstream ports unless it supports the *USB Implementers Forum, Inc.'s Battery Charging Specification*. Refer to Section 10.3.1.1 for a detailed discussion on when a hub is allowed to remove VBUS from a downstream facing port. This specification allows self-powered and bus-powered hubs.

The following sections present the typical flow for connection management in various types of systems for the simple topology shown in Figure 10-4 when the host system is first powered on.

NOTE These connection examples outline cases where the system operates as expected. The handling of error cases are specified later in this chapter.



IEC

Figure 10-4 – Simple USB Topology

10.1.1 Connecting to an Enhanced SuperSpeed Capable Host

When the host is powered off, the hub does not provide power to its downstream ports unless the hub supports charging applications (refer to Section 10.3.1.1).

When a hub is connected to a powered port and it detects Enhanced SuperSpeed connectivity, by default the following is the typical sequence of events:

- The upstream facing port will train at the fastest speed supported by its link partner as defined in the link chapter.
- Simultaneously, the hub powers its downstream ports and trains each link at the fastest speed supported by its link partner.
- If a downstream port trained at a higher speed than the upstream port then the downstream port shall retrain at a speed no faster than the upstream port.
- Hub connects both as an Enhanced SuperSpeed hub device and as a high-speed hub device.
- Host system begins hub enumeration at high-speed and Gen X speed.
- Host system begins device enumeration at Gen X speed.

10.1.2 Connecting to a USB 2.0 Host

When the host is powered off, the hub does not provide power to its downstream ports unless the hub supports charging applications (refer to Section 10.3.1.1).

When the host is powered on and there is no Gen X speed support, the following is the typical sequence of events:

- Hub detects VBUS and connects as a high-speed hub device.
- Host system begins hub enumeration at high-speed.

- Hubs power downstream ports when directed by software (USB 2.0) with Gen X connectivity disabled.
- Device connects at high-speed.
- Host system begins device enumeration at high-speed.

10.1.3 Hub Connectivity

Hubs exhibit different connectivity behavior depending on whether they are propagating data packet header/data packet payload traffic, other packet traffic, resume signaling, or are in an Idle state.

The hub contains one port that shall always connect in the upstream direction (referred to as the upstream facing port) and one or more downstream facing ports. Upstream connectivity/routing is defined as being towards the host and downstream connectivity/routing is defined as being towards a device.

There are differences in the packet connectivity/routing behavior for Enhanced SuperSpeed hubs operating at Gen 1 speed or at above Gen 1 speed.

Section 10.1.3.1 describes how a USB hub routes packets it receives. Section 10.1.3.2 describes the connectivity behavior for SuperSpeed hubs. Section 10.1.3.3 describes the packet routing behavior for SuperSpeedPlus hubs.

10.1.3.1 Routing Information

Packets received on the hub upstream port are routed based on information contained in a 20-bit field (Route String) in the packet header. The route string is used in conjunction with a hub depth value by the hub to identify the target port for a downstream directed packet. The hub depth value is assigned by software using the Set Hub Depth request. The hub shall ignore the route string and assume all packets are routed directly to the hub, until the hub enters the configured state and the hub's depth is set. The hub's upstream port shall be represented by port number zero while the downstream ports shall begin with port number one and count up sequentially.

The hub shall set the route string of upward flowing packets to;

- Zero, when the upward flowing packet was originated by the hub controller. These could be packets in response to a packet routed to the hub controller; e.g. DP in response to IN/ACK TP or packets such as an ERDY after a previous NRDY response by the hub controller.
- The route string value of a corresponding downward flowing packet, when the downward flowing packet has been marked as deferred by this hub controller.
- The aggregate arbitration weight of the hub, for a SuperSpeedPlus hub as described in Section 10.8.7.

Figure 10-5 illustrates the use of route strings in an example topology with five levels of four port USB hubs. The hub depth value for each level of hub is illustrated in the figure. Each hub and each device in the topology contains the route string that would be used to route a packet to that device/hub. For each hub depth, the octet in the route string that determines the routing target at that hub depth is shown in bold and a larger font size than the rest of the route string. The host root port is not included in the 20-bit route string.

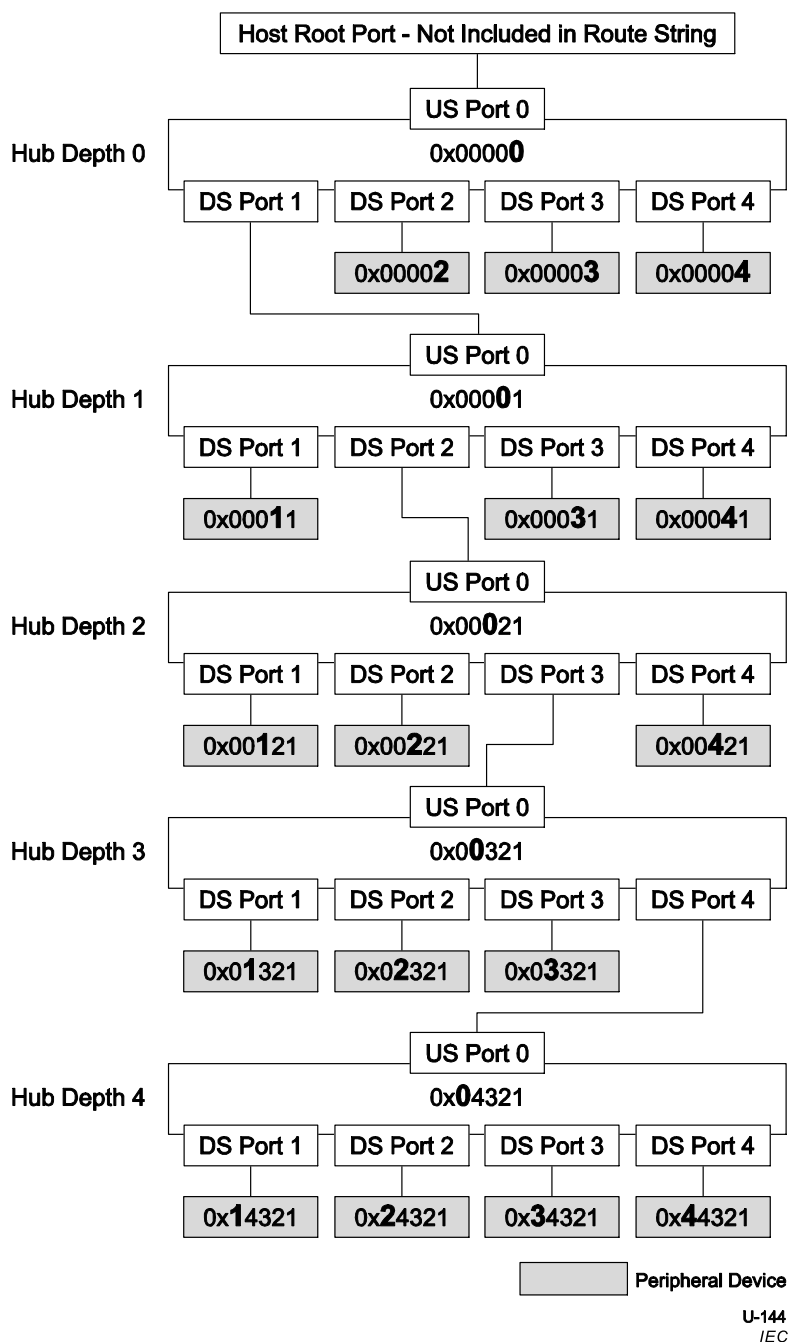


Figure 10-5 – Route String Example

10.1.3.2 SuperSpeed Hub Packet Signaling Connectivity

The SuperSpeed hub repeater/forwarder contains buffering for header and data packets. A SuperSpeed hub repeater/forwarder does not use the repeater-only model used for high-speed connectivity in a USB 2.0 hub. This change allows multiple downstream devices to send asynchronous messages simultaneously without data loss and for some traffic to be stored and delivered when it is directed to downstream ports when the links are not in U0.

Figure 10-6 shows the high level packet signaling connectivity behavior for SuperSpeed hubs in the upstream and downstream directions. Later sections describe the SuperSpeed hub internal buffering and connectivity in more detail. A SuperSpeed hub also has an Idle state, during which the SuperSpeed hub makes no connectivity. When in the Idle state, all of the SuperSpeed hub's ports (upstream plus downstream) are U1, U2 or in U0 receiving and transmitting logical idles waiting for the start of the next packet.

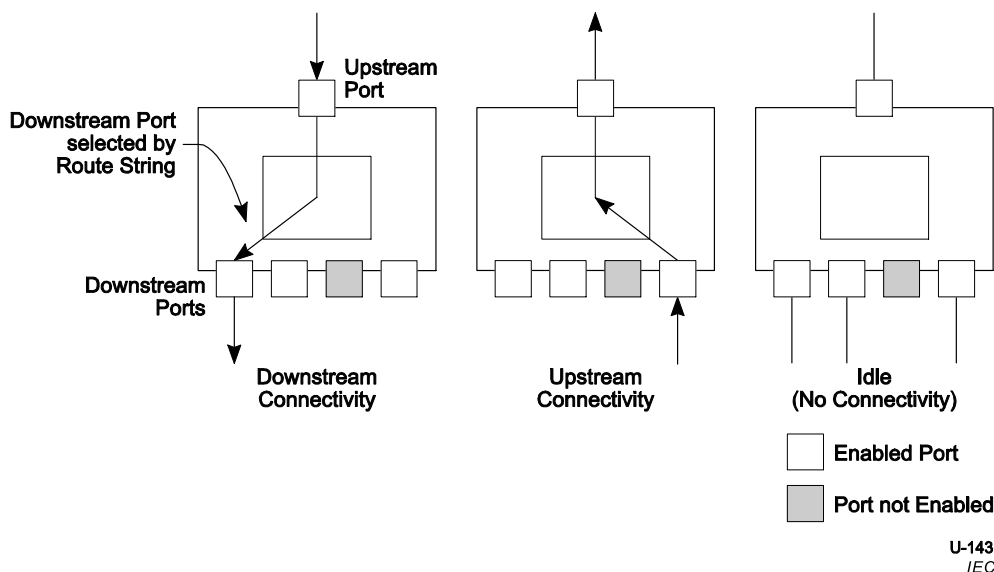


Figure 10-6 – SuperSpeed Hub Signaling Connectivity

If a downstream facing port is enabled (i.e., in a state where it can propagate signaling through the hub) and the SuperSpeed hub detects the start of a packet on that port, the SuperSpeed hub begins to store the packet header. The SuperSpeed hub transmits the valid header packet received on the downstream port upstream, but not to any other downstream facing ports. This means that when a device operating at Gen 1 speed or a SuperSpeed hub transmits a packet upstream, only those SuperSpeed hubs in a direct line between the transmitting device and the host will see the packet.

All packets except Isochronous Timestamp Packets (ITP) are unicast in the downstream direction; SuperSpeed hubs operate using a direct connectivity model. This means that when the host or SuperSpeed hub transmits a packet downstream, only those SuperSpeed hubs in a direct line between the host and recipient device will see the packet.

10.1.3.3 SuperSpeedPlus Hub Packet Routing

The SuperSpeedPlus hub contains buffering for header and data packets. A SuperSpeedPlus hub does not use the repeater-only model used for high-speed connectivity in a USB 2.0 hub or the connectivity based repeater/forwarding model of a SuperSpeed hub. This change allows support for the additional features of SuperSpeedPlus operation.

If a downstream facing port is enabled (i.e., in a state where it can transmit and receive packets) and the hub detects the start of a packet on that port, the hub shall begin to store the packet header. The hub shall route the valid header packet received on the downstream port to the upstream port, but not to any other downstream facing ports. This means that when a device or a hub transmits a packet upstream, only those hubs in a direct line between the transmitting device and the host will see the packet.

All packets except Isochronous Timestamp Packets (ITP) are unicast in the downstream direction; hubs operate using a direct routing model. This means that when the host or hub transmits a packet downstream, only those hubs in a direct line between the host and recipient device will see the packet.

10.1.4 Resume Connectivity

Hubs exhibit different connectivity behaviors for upstream- and downstream-directed resume signaling. A hub does not propagate resume signaling from its upstream facing port to any of its downstream facing ports unless a downstream facing port is suspended and has received resume signaling since it was suspended. Figure 10-7 illustrates hub upstream and downstream resume connectivity.

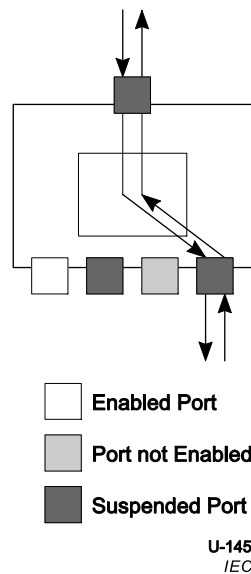


Figure 10-7 – Resume Connectivity

If a hub upstream port is suspended and the hub detects resume signaling from a suspended downstream facing port, the hub propagates that signaling upstream and does not reflect that signaling to any of the downstream facing ports (including the downstream port that originated resume signaling). If a hub upstream port is not suspended and the hub detects resume signaling from a suspended downstream facing port, the hub reflects resume signaling to the downstream port. Note that software shall not initiate a transition to U3 on the upstream port of a hub unless it has already initiated transitions to U3 on all enabled downstream ports. A detailed discussion of resume connectivity appears in Section 10.10.

10.1.5 Hub Fault Recovery Mechanisms

Hubs are the essential USB component for establishing connectivity between the host and other devices. It is vital that any connectivity faults be prevented if possible and detected in the unlikely event they occur.

Hubs must also be able to detect and recover from lost or corrupted packets that are addressed to the Hub Controller. Because the Hub Controller is, in fact, another USB device, it shall adhere to the same rules as other USB devices, as described in Chapter 8.

10.1.6 Hub Buffer Architecture

The buffering behaviors of an Enhanced SuperSpeed hub operating at Gen 1 or at above Gen 1 speeds are different. Section 10.1.6.1 summarizes the buffering behavior of the hub when operating at Gen 1 speed. Section 10.1.6.2 summarizes the buffering and arbitration behavior of the hub when operating at above Gen 1 Speed.

10.1.6.1 SuperSpeed Hub Buffer Architecture

The SuperSpeed hub has header packet buffers associated with its upstream and downstream ports. It also has data packet payload (DPP) buffers for upstream and downstream data flows. See Section 10.7 or Section 10.7.4 for more details.

10.1.6.1.1 SuperSpeed Hub Header Packet Buffer Architecture

Figure 10-8 shows the logical representation of a typical header packet buffer implementation for a SuperSpeed hub. Logically, a SuperSpeed hub has separate header packet buffers associated with each port for both upstream and downstream traffic. When a SuperSpeed hub receives a header packet on its upstream port, it routes the header packet to the appropriate downstream header packet buffer for transmission (unless the header packet is for the hub).

When the SuperSpeed hub receives a non-LMP header packet on a downstream port, it routes the header packet to the upstream port header packet buffer for transmission. Header packets are kept in the SuperSpeed hub header packet buffers after transmission until link level acknowledgement (LGOOD_n) for the header packet is received. This allows the SuperSpeed hub to retry the header packets if necessary to ensure that header packets are received correctly at the link level. The header packet buffers also allow a SuperSpeed hub to store the header packets until they can be forwarded when the header packet is directed to a downstream link that is a low power link state. SuperSpeed hubs store the header packet and deliver it once the link becomes active.

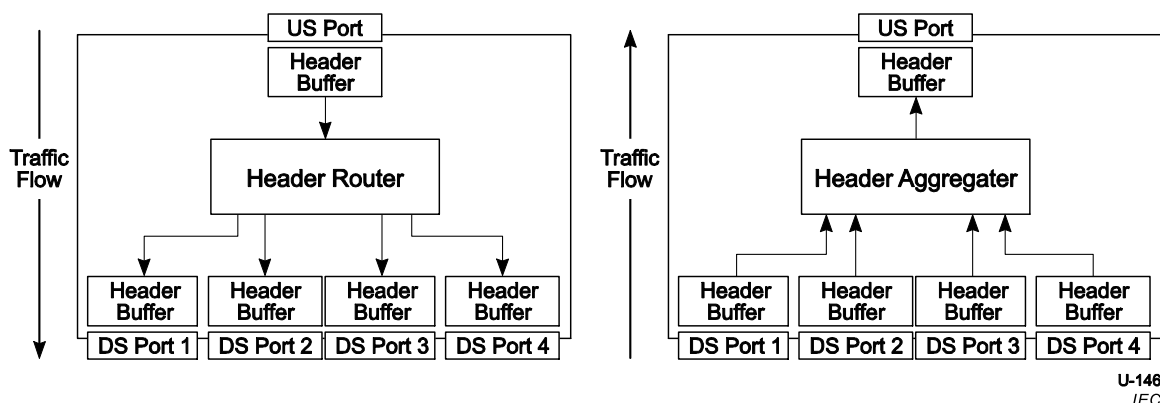
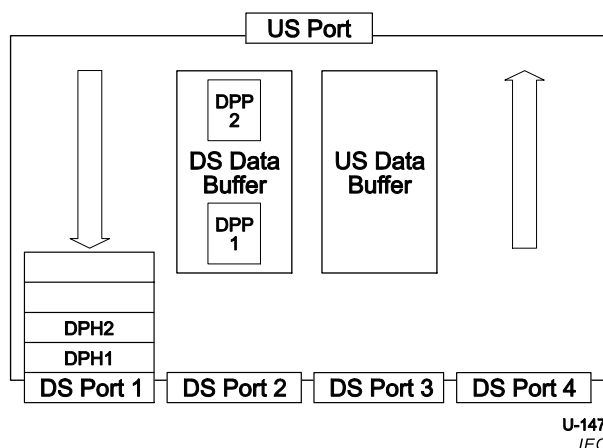


Figure 10-8 – Typical SuperSpeed Hub Header Packet Buffer Architecture

10.1.6.1.2 Hub Data Buffer Architecture



**Figure 10-9 – SuperSpeed Hub Data Buffer Traffic
(Header Packet Buffer Only Shown for DS Port 1)**

Figure 10-9 shows the logical representation of the data buffer architecture in a typical SuperSpeed hub. SuperSpeed hubs provide independent buffering for data packet payloads (DPP) in both the upstream and downstream directions. The Enhanced SuperSpeed Architecture allows concurrent transactions to occur in both the upstream and downstream directions. In the figure, two data packets are in progress in the downstream direction. The SuperSpeed hub can store more than one data packet payload at the same time. In rare occurrences where data packet payloads are discarded because buffering is unavailable, the end-to-end protocol will recover by retrying the transaction. The isochronous protocol does not include retries. However, discard errors are expected less frequently than bit errors on the physical bus.

NOTE Data packet headers are stored and handled in the same fashion as other header packet packets using the header packet buffers. DPPs are handled using the separate data buffers.

10.1.6.2 SuperSpeedPlus Hub Buffer Architecture

The SuperSpeedPlus hub has significantly more data packet header (DPH) and data packet payload (DPP) buffering than a SuperSpeed hub. Downstream ports can operate at a different speed than the upstream port and there can be multiple DPs simultaneously in transit on different downstream ports-. Therefore, DPs may have to be buffered until they can be transmitted out of the hub. Since there can be multiple DPs buffered in a hub awaiting transmission on a port, the SuperSpeedPlus hub also has local arbitration rules to select the packet to be transmitted next on a port. There are specific buffering requirements for upstream and downstream traffic. See Section 10.8 for more details.

10.2 Hub Power Management

10.2.1 Link States

The hub is required to support U0, U1, U2, and U3 on all ports (upstream and downstream).

10.2.2 Hub Downstream Port U1/U2 Timers

The hub is required to have inactivity timers for both U1 and U2 on each downstream port. The timeout values are programmable and may be set by the host software. A timeout value of zero means the timer is disabled. The default value for the U1/U2 timeouts is zero. The U1 and U2 timeout values for all downstream ports reset to the default values on PowerOn Reset or when the hub upstream port is reset. The U1 and U2 timeout values for a downstream port reset to the default values when the port receives a SetPortFeature request for a port reset. The downstream port state machines presented in this chapter describe the specific operational rules when U1 and/or U2 timeouts are enabled.

- Hub downstream ports shall accept U1 or U2 entry initiated by a link partner except when the corresponding U1/U2 timeout is set to zero or there is pending traffic directed to the downstream port.
- If a hub has received a valid packet on its upstream port that is routed to a downstream port, it shall reject U1 or U2 link entry attempts on the downstream port until the packet has been successfully transmitted. A hub may also reject U1 or U2 link entry attempts on downstream ports if the hub is receiving a packet but has not determined the packet's destination. A hub implementation shall ensure no race condition exists where a header packet that has not been deferred is queued for transmission on a downstream port with a link that is in U1, U2, or is in the process of entering U1, U2.
- Hub downstream ports shall reject all U1 and U2 entry requests if the corresponding timeout is set to zero.
- The hub inactivity timers for U1 and U2 shall not be reset by an Isochronous Timestamp Packet (ITP).

10.2.3 Downstream/Upstream Port Link State Transitions

The hub shall evaluate the link power state of its downstream ports such that it propagates the highest link state of any of its downstream ports to its upstream port when there is no pending upstream traffic. U0 is the highest link state, followed by U1, then U2, then U3, then Rx.Detect, and then eSS.Disabled. The order of the other link states is undefined and implementation dependent. If an upstream port link state transition would result in an upstream port link state that has been disabled by software, the hub shall transition the upstream port link to the next highest U-state that is enabled. The hub never automatically attempts to transition the hub upstream port to U3 or lower state.

The downstream port state machines presented in this chapter provide the specific timing requirements for changing the upstream port link state in response to downstream port link state changes.

The hub also shall initiate a link state transition on the appropriate downstream port whenever it receives a packet that is routed to downstream port that is not in U0. The hub upstream port

state machines provided in this chapter provide the specific timing requirements for these transitions.

If enabled, port status change interrupts, e.g., due to a connect event on a downstream port, will cause the upstream link to initiate a transition to U0.

10.3 Hub Downstream Facing Ports

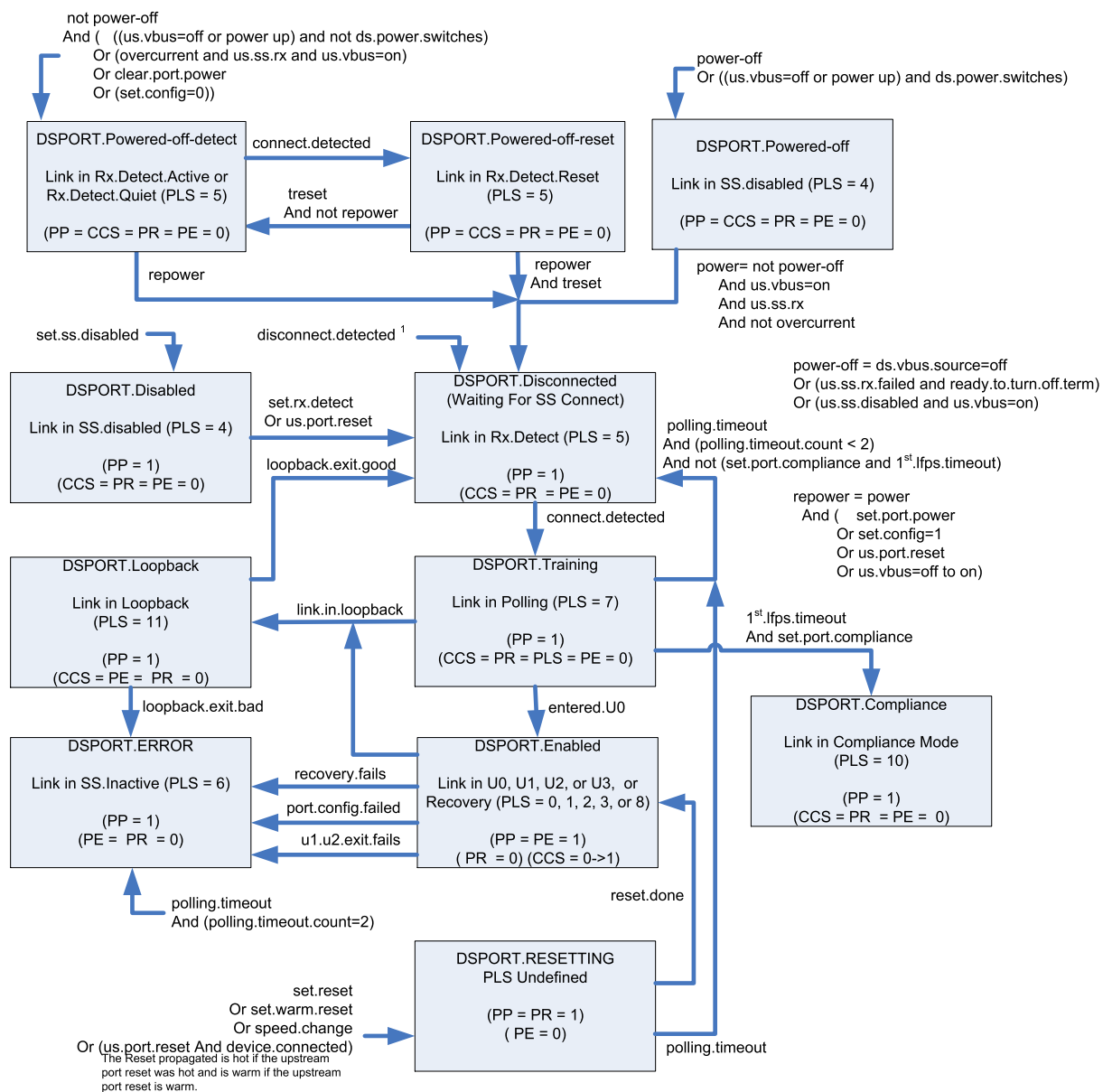
The following sections provide a functional description of a state machine that exhibits the correct required behavior for a downstream facing port.

Figure 10-10 is an illustration of the downstream facing port state machine. Each of the states is described in Section 10.4.2. In the diagram below, some of the entry conditions into states are shown without origin. These conditions have multiple origin states and the individual transitions lines are not shown to simplify the diagram. The description of the entered state indicates from which states the transition is applicable.



NOTE

For the root hub, the signals from the upstream facing port state machines are implementation dependent.



Port Status Field:
Notation Field Name
 PP PORT_POWER
 CCS PORT_CONNECTION
 PR PORT_RESET
 PLS PORT_LINK_STATE
 PE PORT_ENABLE

Note:
 Clear Port Feature (PORT_ENABLED) and Set Port Feature (PORT_ENABLED) are not used for SS Ports

1 This direct transition may only occur from a DSPORT state whose link is in the SS.Inactive, Rx.Detect.Active (during DSPORT.RESETTING), U1, U2, or U3 state.

IEC

Figure 10-10 – Downstream Facing Hub Port State Machine

Table 10-1 – Downstream Facing Hub Port State Machine Diagram Legend

Key	Description
1st.lfps.timeout	First LFPS Timeout after Receiving SetPortFeature(PORT_LINK_STATE)=Compliance Mode
clear.port.power	Received ClearPortFeature(PORT_POWER)
connect.detected	Connect Detected
device.connected	Device is connected as reported by LTSSM receiver detection, except when detected for DSPORT.Powered-off-reset or DSPORT.Powered-off-detect states.
disconnect.detected	Disconnect Detected
ds.power.switches	Hub reports non-zero value in bPwrOn2PwrGood field of hub descriptor. Hub supports power switching.
ds.vbus.source = off	Downstream port VBUS is Off due to loss of Local Power Source when hub is self-powered.
ds.vbus.source = on	Downstream port VBUS may be On. Local Power Source is On or the hub is bus-powered.
entered.U0	Link Transitions from Polling.Idle to U0
link.in.loopback	Loopback bit set in received TS2 ordered sets
loopback.exit.bad	Loopback exit LFPS handshake failed (applies only if Downstream Port is loopback master)
loopback.exit.good	Loopback exit LFPS handshake successful
overcurrent	Over-Current is active.
polling.timeout	Any Polling substate times out
polling.timeout.count	See chapter 7 for definition of cPollingTimeout
port.config.failed	Port Configuration Fails (refer to Section 8.4.6)
ready.to.turn.off.term	Ready to turn off receiver termination. Warm reset tReset duration was met or downstream port is power switched or no device is connected or port was in a powered on state and not performing a warm reset. These conditions ensure that a device connected to this downstream port will not drop to Compliance when termination is removed.
recovery.fails	Link Exits Recovery after Timeout
repower	Repowering conditions defined in Section 10.3.1.10 are met
reset.done	Reset Completes Successfully
set.config = 0	Received SetConfig(0) request
set.config = 1	Received SetConfig(1) request
set.port.compliance	Received SetPortFeature(PORT_LINK_STATE) = Compliance Mode.
set.port.power	Received SetPortFeature(PORT_POWER)
set.reset	Received SetPortFeature(PORT_RESET)
set.rx.detect	Received SetPortFeature(PORT_LINK_STATE) = Rx.Detect
set.ss.disabled	Received SetPortFeature(PORT_LINK_STATE) = eSS.Disabled
set.warm.reset	Received SetPortFeature(PORT_BH_RESET)
speed.change	The upstream port trained to a lower speed than a downstream port speed.
triset	Warm Reset has been signaled for tReset duration.
us.port.reset	Received inband reset on Upstream Port
us.ss.disabled	Upstream port transitioned to eSS.Disabled.
us.ss.rx	Far End Receiver Terminations are present on the Upstream Port's link or were present when Upstream VBUS was most recently on.
us.ss.rx.failed	Hub's upstream port link has attempted eight consecutive Rx.Detect events without detecting far-end receiver termination
us.vbus = off	Upstream port VBUS is Off
us.vbus = on	Upstream port VBUS is On
u1.u2.exit.fails	U1 or U2 Exit Fails

10.3.1 Hub Downstream Facing Port State Descriptions

10.3.1.1 DSPORT.Powered-off

The DSPORT.Powered-off state is a logical powered off state. This is a default state where DSPORT port will be after power-up if the hub supports power switching. The hub may still be required or choose to provide VBUS for a downstream port in the DSPORT.Powered-off state. Detailed requirements for presence of VBUS are covered later in this section.

A port shall transition into this state if any of the following situations occur:

- From any state when VBUS is removed from the hub upstream port and the hub supports power switching on the DS ports.
- Any “power-off” condition is met:
 - From any state when local power is lost to the port.
 - From any state if the hub’s upstream port link transitions to the eSS.Disabled state and Upstream PORT VBUS is on.
 - From any state if the hub’s upstream port link has attempted eight consecutive Rx.Detect events without detecting far-end receiver terminations and the receiver termination (near-end) of the hub’s downstream port is ready to be turned off.

The downstream port’s termination is considered to be ready to turn off when any of following conditions is met.

- Warm reset signaling has completed.
- The port is power switched.
- The port is in a powered on state and not performing a warm reset
- No device is connected.

A port shall remain in the DSPORT.Powered-off state until the following conditions are met.

- The hub’s Upstream Port link has detected far-end receiver terminations. Note that this requires Upstream Port VBUS to be on.
- No “power-off” condition is true.
- No Overcurrent condition is active.

If a hub was configured while the local power supply was present and then if local power is lost, the hub shall place all ports in the Powered-off state if power remains to run the hub controller.

In the DSPORT.Powered-off state, the port's link is in the eSS.Disabled state.

Table 10-2 shows the allowed state of VBUS for hub downstream ports for possible states of the hub upstream port and logical port power for a downstream port. The table covers the case where the hub has adequate power to provide power for the downstream ports (local power source is present). For a hub that does not implement per port power control, all downstream ports that will be affected by removing VBUS shall be in a state where power may be off (refer to Table 10-2) before the hub removes VBUS.

NOTE A hub may provide power to all its downstream ports all of the time to support applications such as battery charging from a USB port. Such hubs must ensure that Enhanced SuperSpeed devices on its downstream-facing ports attempt Enhanced SuperSpeed connection once upstream VBUS is seen by the hub. The recommended method to achieve this is to cycle Vbus off for a duration or by actively discharging so that it is seen to be off by the downstream device.

Table 10-2 – Downstream Port VBUS Requirements

Hub Upstream Port Connection Status	Downstream Port Enhanced SuperSpeed Port Power Off (PORT_POWER = 0) USB 2.0 Port Power On (PORT_POWER = 1)	Downstream Port Enhanced SuperSpeed Port Power On (PORT_POWER = 1) Downstream Port USB 2.0 Port Power Off (PORT_POWER = 0)	Downstream Port USB 2.0 and Enhanced SuperSpeed Port Power Off (PORT_POWER = 0)
Enhanced SuperSpeed	On ¹	On	May be off
USB 2.0	On	May be off	May be off
Enhanced SuperSpeed and USB 2.0	On	On	May be off
No VBUS	May be off	May be off	May be off

¹ If the hub upstream port is unable to connect on the USB 2.0 bus, the downstream port VBUS may be off in this state.

10.3.1.2 DSPORT.Disconnected (Waiting for eSS Connect)

A port transitions to this state in any of the following situations:

- From the DSPORT.Powered-off state when the hub's Upstream Port link has detected far-end receiver terminations, Upstream Port VBUS is on (implied by receiver detection), no power-off condition is met and no overcurrent condition exists.
- From any state that can and does detect a disconnect, except from DSPORT.Powered-off-detect.
- From the DSPORT.Powered-off-reset state when conditions for Repowering defined in Section 10.3.1.10 are met and the DSPORT.Powered-off-reset state has been maintained for tReset.
- From the DSPORT.Powered-off-detect state when conditions for Repowering defined in Section 10.3.1.10 are met.
- From the DSPORT.Resetting state when a port's link times out from Rx.Detect.Active during a reset. That is, it detects a disconnect.
- From the DSPORT.Disabled state when a SetPortFeature(PORT_LINK_STATE) Rx.Detect request is received for the port.
- From the DSPORT.Disabled state when the hub's upstream port is reset. Note: The hub shall issue a Warm Reset on the downstream port, if a device is detected in the first Rx.Detect after entering this state, even if the upstream port reset is a hot reset.
- From the DSPORT.Powered-off state or DSPORT.Disabled state when the hub's upstream port is reset. Note: The hub shall issue a Warm Reset on the downstream port after it has transitioned the port to the DSPORT.RxDetect state and detected a far-end receiver, even if the upstream port reset is a hot reset
- From the DSPORT.Resetting state if the port's link times out from any Polling substate during a reset.
- From the DSPORT.Training state if the port's link times out from any Polling substate and the cPollingTimeout is less than 2 and the port is not enabled to enter compliance or Polling substate which timed out is not Polling.LFPS. See definition of PollingTimeout in Section 7.5.4.2 and Section 10.16.2.10 defining Set Port Feature for enabling compliance entry.
- From the DSPORT.Loopback state if the port's link performs a successful LFPS handshake in Loopback.Exit.

In this state, the port's link shall be in the Rx.Detect state.

NOTE The port's link shall still perform connection detection normally from the Rx.Detect if the hub upstream port's link is in U3.

10.3.1.3 DSPORT.Training

A port transitions to this state from the DSPORT.Disconnected state when far-end receiver terminations are detected.

In this state, the port's link shall be in the Polling state.

10.3.1.4 DSPORT.ERROR

A port shall transition to this state only when an Enhanced SuperSpeed device is connected and a serious error condition occurred while attempting to operate the link.

A port transitions to this state in any of the following situations:

- From the DSPORT.Enabled state if the link enters recovery and times out without recovering.
- From the DSPORT.Enabled state if U1 or U2 exit fails.
- From the DSPORT.Loopback state if the port is the loopback master and the LFPS handshake in Loopback.Exit fails.
- From DSPORT.Enabled if Port Configuration fails as described in Section 8.4.6.
- From the DSPORT.Training state if the port's link times out from any Polling substate and cPollingTimeout is 2. See 7.5.4.2 for details of SetioncPollingTimeout.

In this state, the port's link shall be in the eSS.Inactive state.

10.3.1.5 DSPORT.Enabled

A port transitions to this state in any of the following situations:

- From the Training state when the port's link successfully enters U0.
- From the DSPORT.Resetting state when a reset completes successfully.

A port in the DSPORT.Enabled state will propagate packets in both the upstream and the downstream direction after its Current Connect Status (CCS) is set. When the hub downstream port first transitions to the DSPORT.Enabled state after a power on or warm reset, it shall transmit a port configuration LMP as defined in Section 8.4.6. If CCS was set before entering the DSPORT.Enabled state, it will remain set. If CCS was not set, then it shall be set only after the port configuration LMP exchange succeeds.

When the hub downstream port first transitions to the DSPORT.Enabled state after a power on reset, the value for the U1 and U2 inactivity timers shall be reset to zero.

The link shall be in U0 when the enabled state is entered.

If the hub upstream port's link is in U3 when the downstream port enters DSPORT.Enabled and the hub is not enabled for remote wakeup, the downstream port shall initiate a transition to U3 on its link within tDSPORTEnabledToU3.

Section 10.4 provides a state machine that shows a functionally correct implementation for a downstream port managing different link states within the DSPORT.Enabled state.

10.3.1.6 DSPORT.Resetting

A downstream port transitions to the DSPORT.Resetting state in any of the following situations:

- From the DSPORT.Error state when a SetPortFeature(PORT_RESET) request or SetPortFeature(BH_PORT_RESET) is received, the port shall send a warm reset on the downstream port link.
- From the DSPORT.Enabled state and the port's link is in any state when a SetPortFeature(BH_PORT_RESET) is received. In this situation the port shall initiate a Warm Reset on the downstream port link.
- From any state except for DSPORT.Powered-off-reset or DSPORT.Powered-off-detect or DSPORT.Powered-off or DSPORT.Disabled or DSPORT.Disconnected if the hub detects a Reset on its Upstream Port. In this situation, the port shall initiate a Hot/Warm Reset on the downstream port link depending on the type of Reset detected on the hub's upstream port and depending on the current state of the downstream port. This transition shall occur before the upstream port link transitions to U0.
- From any state except for DSPORT.Powered-off, DSPORT.Powered-off-reset, DSPORT.Powered-off-detect, DSPORT.Disabled or DSPORT.Disconnected when it receives a SetPortFeature(PORT_RESET) or SetPortFeature(BH_PORT_RESET). If the downstream port is in the DSPORT.Powered-off, DSPORT.Powered-off-reset, DSPORT.Powered-off-detect, DSPORT.Disabled or DSPORT.Disconnected state and it receives one of the above requests, the request is ignored.
- From the DSPORT.Enabled state and the port's link state is in any state other than U3 when a SetPortFeature(PORT_RESET) is received. In this situation the port shall initiate a Hot Reset on the downstream port link.
- From the DSPORT.Enabled state and the port's link state is in U3 when a SetPortFeature(PORT_RESET) is received. In this situation the port shall initiate a Warm Reset on the downstream port link.

NOTE If the port initiates a hot reset on the link and the hot reset fails during the link Recovery state, a warm reset will be automatically tried. Refer to the Link Chapter for details on this process. The port stays in the DSPORT.Resetting state throughout this process until the warm reset completes.

When the downstream port link enters Rx.Detect.Active during a warm reset, the hub shall start a timer to count the time it is in Rx.Detect.Active or Rx.Detect.Quiet. If this timer exceeds tTimeForResetError while the link remains in Rx.Detect, the port shall transition to the DSPORT.Disconnected state.

10.3.1.7 DSPORT.Compliance

A port transitions to this state in any of the following situations:

- When the link enters the Compliance Mode state.

10.3.1.8 DSPORT.Loopback

A port transitions to this state in any of the following situations:

- From the DSPORT.Training state if the loopback bit is set in the received TS2 ordered sets.

In this state, the port's link shall be in the Loopback state.

10.3.1.9 DSPORT.Disabled

A port transitions to this state when the port receives a SetPortFeature(PORT_LINK_STATE) eSS.Disabled request.

In this state, the port's link shall be in the eSS.Disabled state.

10.3.1.10 DSPORT.Powered-off-detect

This state is entered when the downstream power state is logically off and an Enhanced SuperSpeed connection, rather than a USB 2.0 connection, is desired. To ensure that an

Enhanced SuperSpeed connection is established, unlike the DSPORT.Powered-off state, terminations are maintained while in this state. This is the default DSPORT state at power-up if the hub does not support power switching. This state shall perform far-end receiver detection with the link in Rx.Detect, until any of the following conditions are true:

- A receiver is detected.
- Any condition to “power-off” is met.
- The conditions to “repower” the port as described below are met.

A port shall transition into this state from the DSPORT.Powered-off-reset state when tReset time has been met and the conditions to “repower” are not met.

All the following conditions shall be met for “repower”:

- All “power” conditions are met.
- SetPortFeature(PORT_POWER) request is received,
Or SetConfig(1) request is received,
Or Upstream Port Reset is detected,
Or Upstream Port VBUS transitioned from off to on.

Note that Upstream Port VBUS is considered to have transitioned from off to on when it is on at power-up.

When no “power-off” condition is met and any of the following conditions are true, this state is entered regardless of the previous state.

- Overcurrent condition is detected either on this port or globally and Upstream Port Far-end Receiver Terminations are present and Upstream VBUS is on. Note: If Upstream VBUS is turned off while overcurrent is active port transitions to Powered-off state immediately (without waiting for tReset to complete) if ds power switches are supported.
- Upstream Port VBUS is off and the hub does not support power switching.
- The hub receives a ClearPortFeature(PORT_POWER) request for this port. In this case, power is removed from the port only if it would not impact the low-speed, full-speed, or high-speed operation on any of the downstream ports on the hub and would not impact SS operation on any ports other than the target port.
- The hub upstream port receives a SetConfiguration(0) request. In this case the downstream port will stay in this state or transition between this state and DSPORT.Powered-off-reset state regardless of other conditions until the hub is reset or the hub upstream port receives a non-zero SetConfiguration request.

10.3.1.11 DSPORT.Powered-off-reset

This state is entered when the downstream power state is logically off and an Enhanced SuperSpeed connection, rather than a USB 2.0 connection, is desired. To ensure that an Enhanced SuperSpeed connection is established, unlike the DSPORT.Powered-off state, the terminations are maintained while in this state, and to avoid a link training failure, which would allow the downstream device to drop into Compliance Mode or USB 2.0 operation, Warm Reset signaling shall be driven for tReset duration. This state shall drive Warm Reset with the link in the Rx.Detect.Reset substate, until the tReset duration is met.

This state is entered from DSPORT.Powered-off-detect whenever a far end receiver is detected.

10.3.2 Disconnect Detect Mechanism

Disconnect detection mechanisms are covered in Section 7.5.

10.3.3 Labeling

USB system software uses port numbers to reference an individual port with a ClearPortFeature or SetPortFeature request. If a vendor provides a labeling to identify individual downstream facing ports, then each port connector shall be labeled with its respective port number. The port numbers assigned to a specific port by the hub shall be consistent between the USB 2.0 hub and Enhanced SuperSpeed hub.

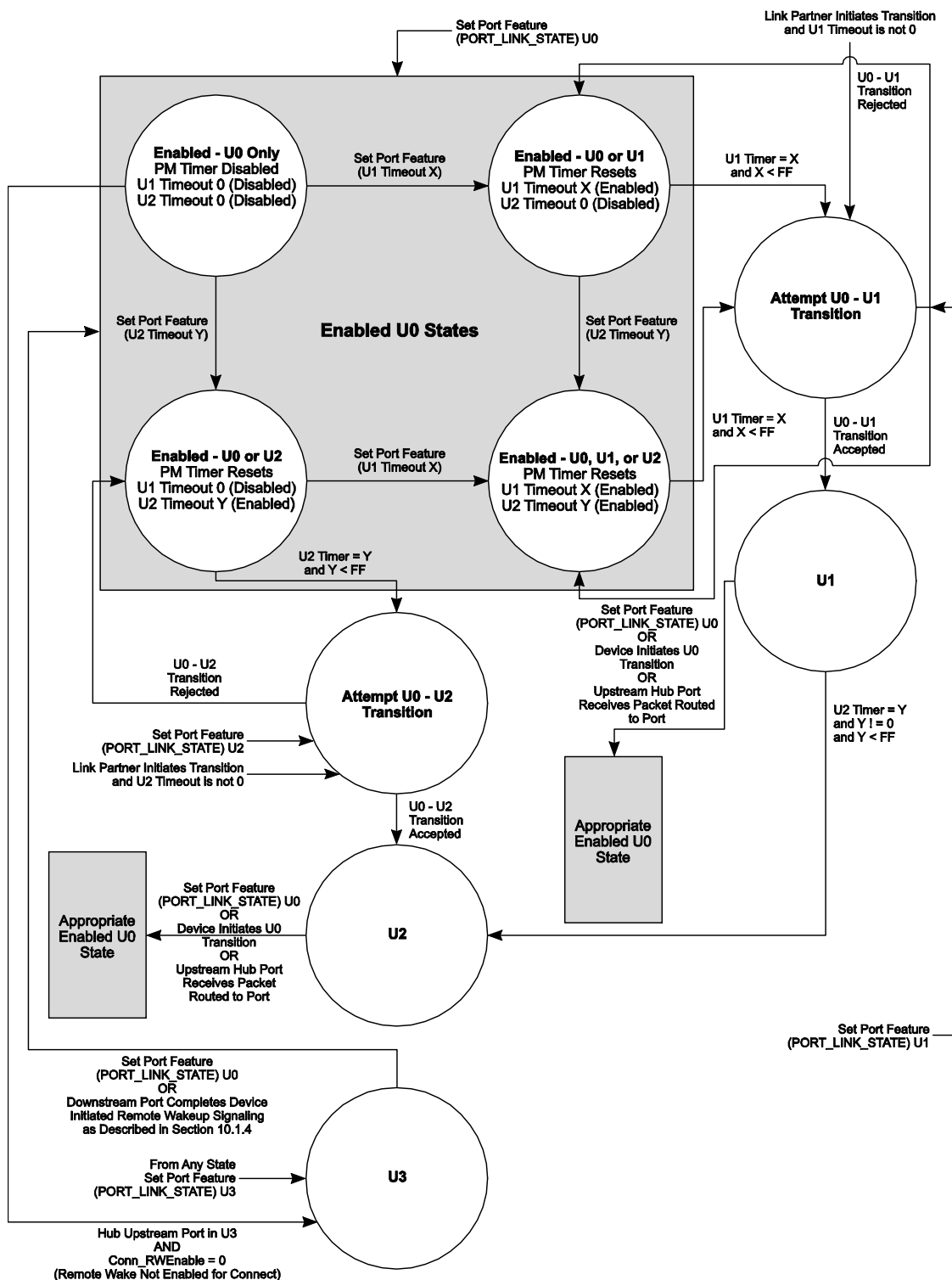
10.4 Hub Downstream Facing Port Power Management

The following sections provide a functional description of a state machine that exhibits correct link power management behavior for a downstream facing port.

Figure 10-11 is an illustration of the downstream facing port power management state machine. Each of the states is described in Section 10.4.2. In Figure 10-11, some of the entry conditions into states are shown without origin. These conditions have multiple origin states and the individual transitions lines are not shown so that the diagram can be simplified. The description of the entered state indicates from which states the transition is applicable.

10.4.1 Downstream Facing Port PM Timers

Each downstream port maintains logical inactivity timers for keeping track of when U1 and U2 timeouts are exceeded. The U1 or U2 timeout values may be set by software with a SetPortFeature(PORT_U1_TIMEOUT) or SetPortFeature(PORT_U2_TIMEOUT) command at any time. The PM timers are reset to 0 every time a SetPortFeature(PORT_U1_TIMEOUT) or SetPortFeature(PORT_U2_TIMEOUT) request is received. The timers shall be reset every time a packet of any type except an isochronous timestamp packet is sent or received by the port's link. The U1 timer shall be accurate to $\pm 1/-0 \mu\text{s}$. The U2 timer shall be accurate to $\pm 500/-0 \mu\text{s}$. Other requirements for the timer are defined in the downstream port PM state machine descriptions.



U-149
IEC

Figure 10-11 – Downstream Facing Hub Port Power Management State Machine

10.4.2 Hub Downstream Facing Port State Descriptions

10.4.2.1 Enabled U0 States

There are four enabled U0 states that differ only in the values that are configured for the U1 and U2 timeouts. The port behaves as follows for the various combinations of U1 and U2 timeout values:

U1_TIMEOUT = 0, U2_TIMEOUT = 0

- This is the default state before the hub has received any SetPortFeature(PORT_U1/U2_TIMEOUT) requests for the port.
- The port's link shall reject all U1 or U2 transition requests by the link partner.
- The PM timers may be disabled and the PM timer values shall be ignored.
- The port's link shall not attempt to initiate transitions to U1 or U2.

U1_TIMEOUT = X > 0, U2_TIMEOUT = 0

- The port's link shall reject all U2 transition requests by the link partner.
- The PM timers shall be reset when this state is entered and is active.
- The port's link shall accept U1 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port.
- If the U1 timeout is 0xFF, the port shall be disabled from initiating U1 entry but shall accept U1 entry requests by the link partner unless the hub has one or more packets/link commands to transmit on the port.
- If the U1 timeout is not 0xFF and the U1 timer reaches X, the port's link shall initiate a transition to U1.

U1_TIMEOUT = 0, U2_TIMEOUT = Y > 0

- The port's link shall reject all U1 transition requests by the link partner.
- The PM timers shall be reset when this state is entered and is active.
- The port's link shall accept U2 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port.
- If the U2 timeout is 0xFF, the port shall be disabled from initiating U2 entry but shall accept U2 entry requests by the link partner unless the hub has one or more packets/link commands to transmit on the port.
- If the U2 timeout is not 0xFF and the U2 timer reaches Y, the port's link shall initiate a direct transition from U0 to U2. In this case, PORT_U2_TIMEOUT represents an amount of inactive time in U0.

U1_TIMEOUT = X > 0, U2_TIMEOUT = Y > 0

- The PM timers are reset when this state is entered and is active.
- The port's link shall accept U1 or U2 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port.
- If the U1 timeout is 0xFF, the port shall be disabled from initiating U1 entry but shall accept U1 entry requests by the link partner unless the hub has one or more packets/link commands to transmit on the port.
- If the U1 timeout is not 0xFF and the U1 timer reaches X, the port's link shall initiate a transition to U1.
- If the U2 timeout is 0xFF, the port shall be disabled from initiating U2 entry but shall accept U2 entry requests by the link partner unless the hub has one or more packets/link commands to transmit on the port.

A port transitions to one of the Enabled U0 states (depending on the U1 and U2 Timeout values) in any of the following situations:

- From any state if the hub receives a SetPortFeature(PORT_LINK_STATE) U0 request.
- From U1 if the link partner successfully initiates a transition to U0.
- From U2 if the link partner successfully initiates a transition to U0.

- From U1 if the hub successfully initiates a transition to U0 after receiving a packet routed to the port.
- From U2 if the hub successfully initiates a transition to U0 after receiving a packet routed to the port
- From an attempt to transition from the U0 to the U1 state if the downstream port's link partner rejects the transition attempt
- From an attempt to transition from the U0 to the U2 state if the downstream port's link partner rejects the transition attempt
- From U3 if the upstream port of the hub receives wakeup signaling and the hub downstream port being transitioned received wakeup signaling while it was in U3.
- From U3 if the downstream port's link partner initiated wake signaling and the upstream hub port's link is not in U3.

NOTE Refer to Section 10.1.4 for details on cases where a downstream port's link partner initiates remote wakeup signaling.

10.4.2.2 Attempt U0 – U1 Transition

In this state, the port attempts to transition its link from the U0 state to the U1 state.

A port shall attempt to transition to the U1 state in any of the following situations:

- The U1 timer reaches the U1 timeout value.
- The hub receives a SetPortFeature(PORT_LINK_STATE) U1 request.
- The downstream port's link partner initiates a U0-U1 transition.

If the transition attempt fails, the port returns to the appropriate enabled U0 state. However, if this state was entered due to a SetPortFeature request, the port continues to attempt the U0-U1 transition on its link.

NOTE That the SetPortFeature request is typically only used for U1 entry for test purposes.

10.4.2.3 Attempt U0 – U2 Transition

In this state, the port attempts to transition the link from the U0 state to the U2 state.

A port shall attempt to transition to the U2 state in any of the following situations:

- The U2 timer reaches the U2 timeout value.
- The hub receives a SetPortFeature(PORT_LINK_STATE) U2 request.
- The downstream port's link partner initiates a U0-U2 transition.

If the transition attempt fails, the port returns to the appropriate enabled U0 state. However, if this state was entered due to a SetPortFeature request, the port continues to attempt the U0-U2 transition.

NOTE That the SetPortFeature request is typically only used for U2 entry for test purposes.

10.4.2.4 Link in U1

Whenever a downstream port enters U1 and all downstream ports are now in the U1 or a lower power state, the hub shall initiate a transition to U1 on the upstream port within tHubPort2PortExitLat if the upstream port is enabled for U1.

The U2 timer is reset to zero and started when the Link enters U1.

If the U2 timeout is not 0xFF and the U2 timer reaches Y, the port's link shall initiate a direct transition from U1 to U2. In this case, PORT_U2_TIMEOUT represents an amount of time in U1.

Whenever a downstream port or its link partner initiates a transition from U1 to one of the Enabled U0 states and the upstream port is not in U0, the hub shall initiate a transition to U0 on the upstream port within tHubPort2PortExitLat of when the transition was initiated on the downstream port. If the upstream port is in U0, it shall remain in U0 while the downstream port transitions to U0.

10.4.2.5 Link in U2

The following rules apply when a downstream port enters U2:

- If all downstream ports are now in the U2 or a lower power state, the hub shall initiate a transition to U2 on the upstream port within tHubPort2PortExitLat, if the upstream port is enabled for U2. If U2 is not enabled on the upstream port, but U1 is enabled, the hub shall initiate a transition to U1 with the same timing requirements.
- If all downstream ports are now in the U1 or lower power state, the hub shall initiate a transition to U1 on the upstream port within tHubPort2PortExitLat, if the upstream port is enabled for U1.

Whenever a downstream port or its link partner initiates a transition from U2 to one of the Enabled U0 states and the hub upstream port is not in U0:

- If the hub upstream port's link is in U2, the hub shall initiate a transition to U0 on the upstream port's link within tHubPort2PortExitLat of when the transition was initiated on the downstream port.
- If the hub upstream port's link is in U1, the hub upstream port shall initiate a transition to U0 within tHubPort2PortExitLat + U2DevExitLat-U1DevExitLat of when the transition was initiated on the downstream port.

10.4.2.6 Link in U3

The following rules apply when a downstream port enters U3:

- If all downstream ports are now in the U2 or U3, the hub shall initiate a transition to the lowest enabled power state above U3 on the upstream port within tHubPort2PortExitLat.
- If all downstream ports are now in the U1 or lower power state, the hub shall initiate a transition to U1 on the upstream port within tHubPort2PortExitLat, if the upstream port is enabled for U1.

Refer to Section 10.3.1.5 for a detailed description of the transition from Enabled – U0 Only to the U3 state.

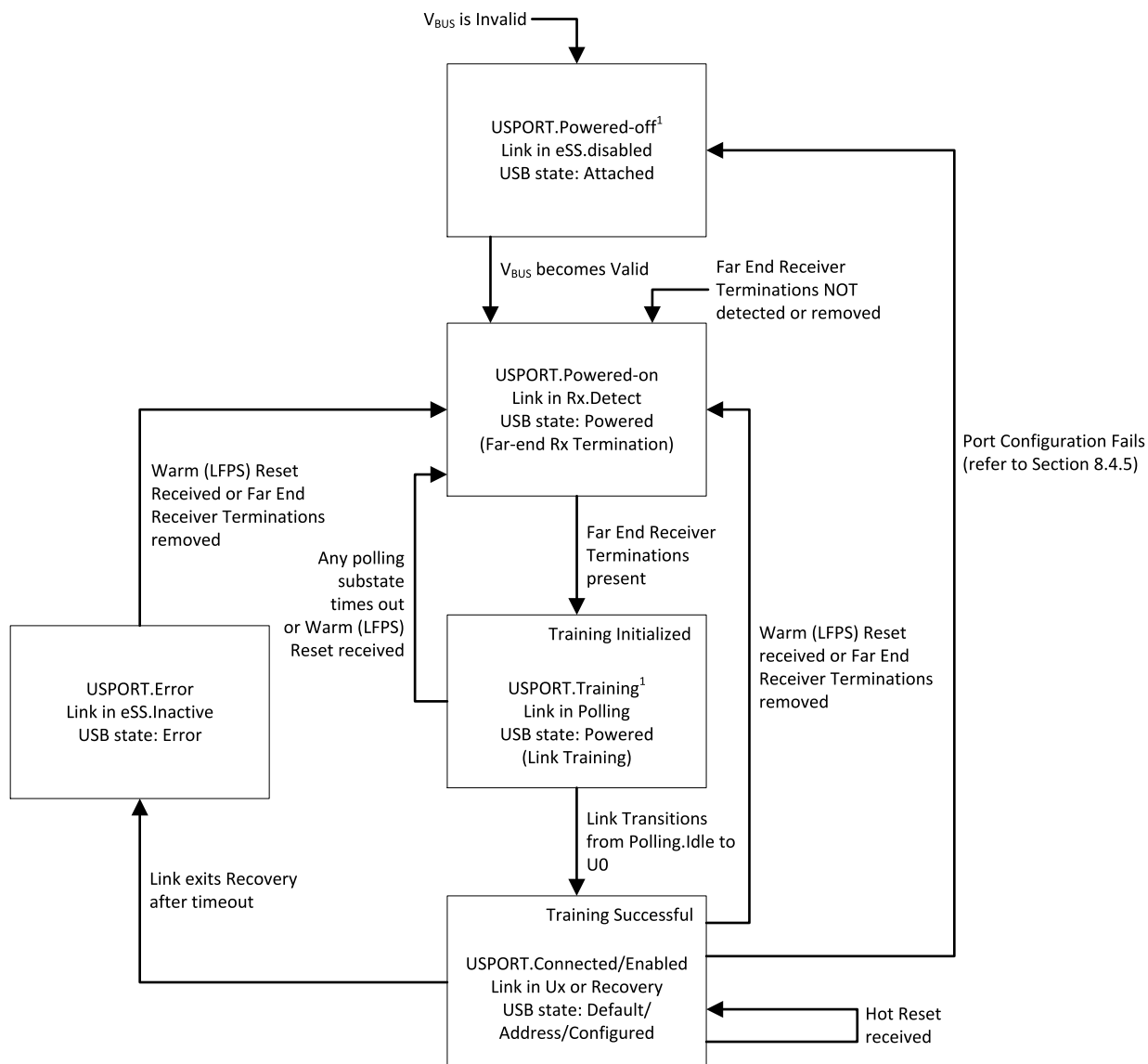
NOTE If the upstream port of the hub receives a packet that is routed to a downstream port that is in U3, the packet is silently discarded. The hub shall perform normal link level acknowledgement of the header packet in this case.

10.5 Hub Upstream Facing Port

The following sections provide a functional description of a state machine that exhibits correct behavior for a hub upstream facing port. These sections also apply to the upstream facing port on a device unless exceptions are specifically noted. An upstream port shall only attempt to connect to the Enhanced SuperSpeed bus and the USB 2.0 bus as described by the upstream port state machine in the following sections.

Figure 10-12 is an illustration of the upstream facing port state machine. Each of the states is described in Section 10.5.1. In Figure 10-12, some of the entry conditions into states are shown without origin. These conditions have multiple origin states and the individual

transitions lines are not shown so that the diagram can be simplified. The description of the entered state indicates from which states the transition is applicable.



¹ If Port Configuration fails, the port shall transition to the USPORT.Powered-off state with the link in eSS.Disabled state and USB Device in the Attached state. V_{BUS} may still be present on the upstream port. V_{BUS} must be toggled to transition to the USPORT.Powered state.

U-150A

IEC

Figure 10-12 – Upstream Facing Hub Port State Machine

10.5.1 Upstream Facing Port State Descriptions

Refer to Figure 9-1 for hub USB states.

10.5.1.1 USPORT.Powered-off

The USPORT.Powered-off state is the default state for an upstream facing port.

A port shall transition into this state if any of the following situations occur:

- From any state when V_{BUS} is invalid.
- From any state if far-end receiver terminations are not detected.

- From the USPORT.Connected/Enabled state if the Port Configuration process fails.

In this state, the port's link shall be in the eSS.Disabled state and the corresponding hub USB state shall be Attached.

NOTE If the port enters this state because far end receiver terminations are not detected and VBUS is present, it may immediately transition to USPORT.Powered on without removing near end terminations.

10.5.1.2 USPORT.Powered-on

A port shall transition into this state in any of the following situations:

- From the USPORT.Powered-off state when VBUS becomes valid.
- From the USPORT.Error state when the link receives a warm reset or if Far-end Terminations are removed.
- From the USPORT.Connected/Enabled state when the link receives a Warm Reset.
- From the USPORT.Training state if the port's link times out from any Polling substate or if the port receives a Warm (LFPS) Reset.

In this state, the port's link shall be in the Rx.Detect state. The corresponding hub USB state shall be Powered (Far-end Receiver Termination substate). While in this state, if the USB 2.0 portion of the hub enters the suspended state, the total hub current draw from VBUS shall not exceed the suspend current limit.

10.5.1.3 USPORT.Training

A port transitions to this state from the USPORT.Powered-on state when Enhanced SuperSpeed far-end receiver terminations are detected.

In this state, the port's link shall be in the Polling state. The corresponding hub USB state shall be Powered (Link Training substate).

10.5.1.4 USPORT.Connected/Enabled

A port transitions to this state from the USPORT.Training state when its link enters U0 from Polling.Idle. A port remains in this state during hot reset. When a hot reset is completed, the corresponding hub USB state shall transition to Default.

In this state, the port's link shall be in the U0, U1, U2, U3, or Recovery state. The corresponding hub USB state shall be Default, Address, or Configured.

When the link enters U0 the port shall start the port configuration process as defined in Section 8.4.6.

The port may send link management packets or link commands but shall not transmit any other packets except to respond to default control endpoint requests while in the USPORT.Connected state.

10.5.1.5 USPORT.Error

A port transitions to this state when a serious error condition occurred while attempting to operate the link. A port transitions to this state in any of the following situations:

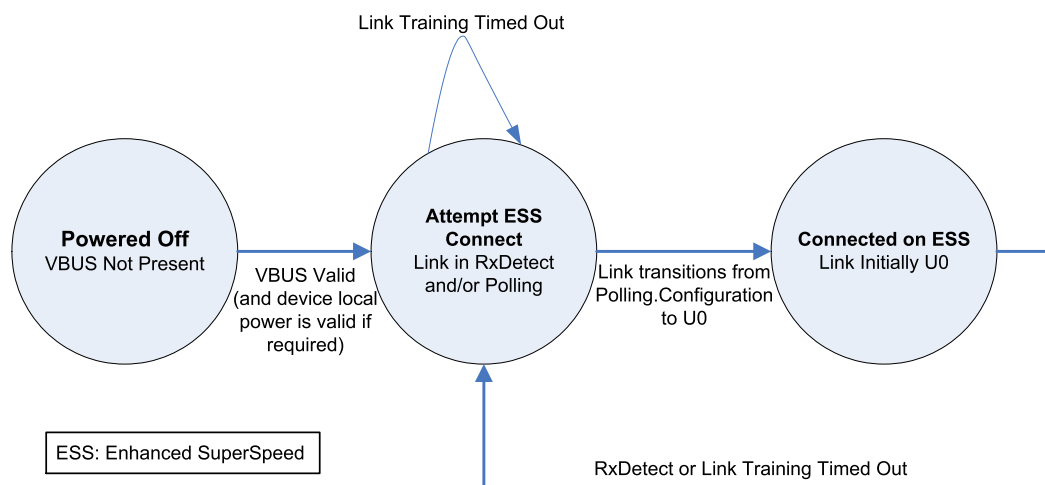
- From the USPORT.Connected/Enabled state if the link enters Recovery and times out without recovering.

In this state, the port's link shall be in the eSS.Inactive state. The corresponding hub USB state shall be Error.

A port exits the Error state only if a Warm Reset is received on the link or if Far-end Receiver Terminations are removed.

10.5.2 Hub Connect State Machine

The following sections provide a functional description of a state machine that exhibits correct hub behavior for when to connect on the Enhanced SuperSpeed bus or the USB 2.0 bus. For a hub, the connection logic for the Enhanced SuperSpeed bus and the USB 2.0 bus are completely independent. The hub shall follow the USB 2.0 specification for connecting on USB 2.0. Figure 10-13 is an illustration of the hub connect state machine for an Enhanced SuperSpeed hub. Each of the states is described in Section 10.5.2.1.



IEC

Figure 10-13 – Hub Connect (HCONNECT) State Machine

10.5.2.1 Hub Connect State Descriptions

10.5.2.2 HCONNECT.Powered-off

The HCONNECT.Powered-off state is the default state for a hub device. A hub device shall transition into this state if the following situation occurs:

- From any state when VBUS is removed.

In this state, the hub upstream port's link shall be in the eSS.Disabled state.

10.5.2.3 HCONNECT.Attempt ESS Connect

A hub shall transition into this state if any of the following situations occur:

- From the HCONNECT.Powered-off state when VBUS becomes valid (and local power is valid if required).
- From the HCONNECT.Connected on ESS state if Rx.Detect or Link Training time out.

In this state, the hub's upstream port Enhanced SuperSpeed link is in Rx.Detect or Polling.

10.5.2.4 HCONNECT.Connected on ESS

A port shall transition into this state if the following situation occurs:

- From the HCONNECT.Attempt ESS Connect when the link transitions from polling to U0.

In this state the hub's upstream port Enhanced SuperSpeed link is in U0, U1, U2, U3, Inactive, Rx.Detect, Recovery, or Polling.

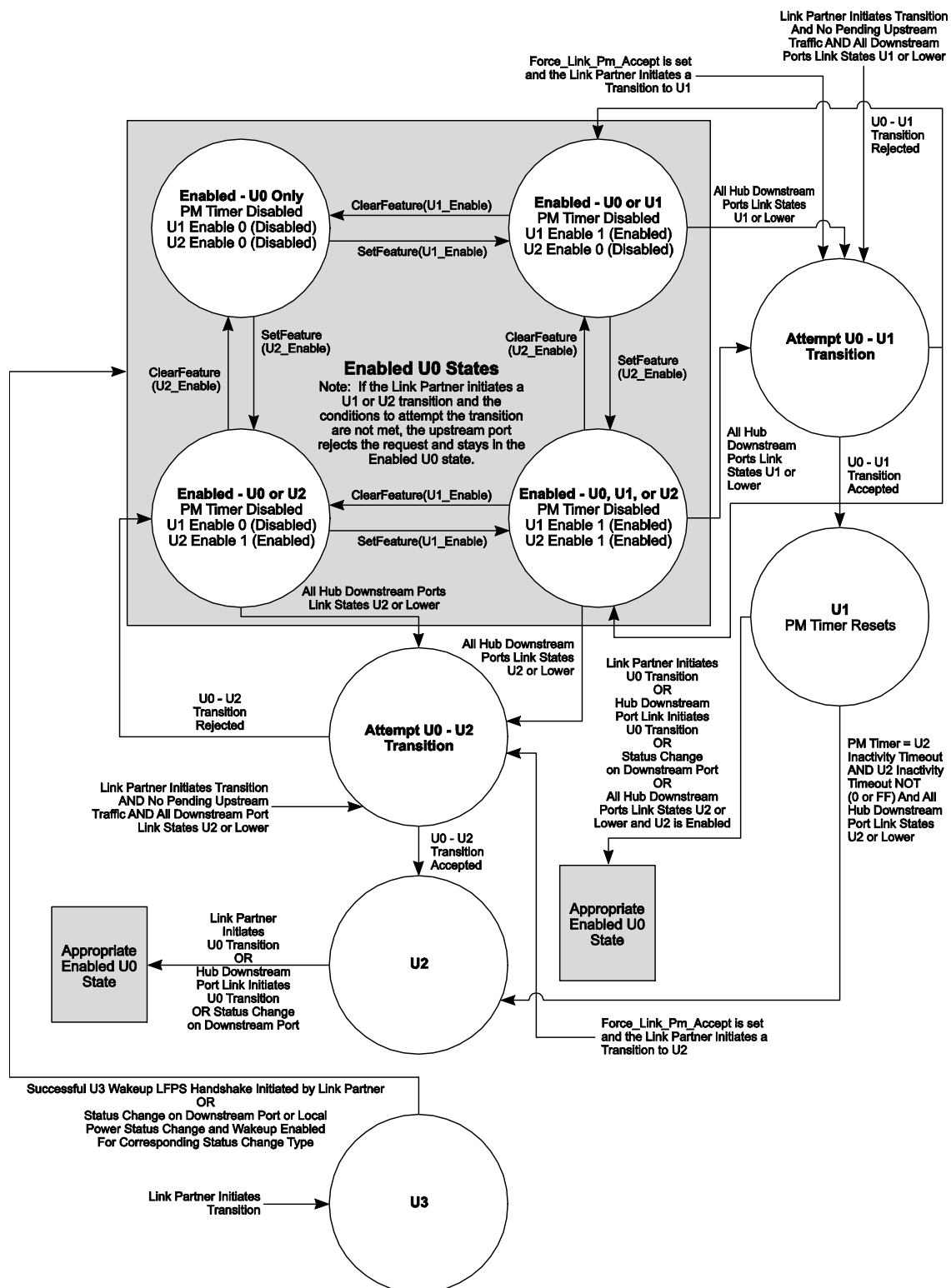
10.6 Upstream Facing Port Power Management

The following sections provide a functional description of a state machine that exhibits correct link power management behavior for a hub upstream facing port.

Figure 10-14 is an illustration of the upstream facing port power management state machine. Each of the states is described in Section 10.6.2. In Figure 10-14, some of the entry conditions into states are shown without origin. These conditions have multiple origin states and the individual transitions lines are not shown so that the diagram can be simplified. The description of the entered state indicates from which states the transition is applicable.

If there is a status change on any downstream port, the hub shall initiate a transition on the upstream port's link to U0 if the upstream port is in U1 or U2.

If there is a status change on any downstream port and the hub upstream port's link is in U3, the hub behavior is specified by the current remote wakeup mask settings. Refer to Section 10.16.2.10 for more details.



U-166
IEC

Figure 10-14 – Upstream Facing Hub Port Power Management State Machine

10.6.1 Upstream Facing Port PM Timer

The hub upstream port maintains a logical PM timer for keeping track of when the U2 inactivity timeout is exceeded. No standard U1 inactivity timeout is defined. The U2 inactivity timeout is set when a U2 Inactivity Timeout LMP is received. The PM timer is reset when the

hub upstream port link enters U1. The PM timer shall be accurate to +500/–0 µs. Other requirements for the timer are defined in the upstream port PM state machine descriptions.

10.6.2 Hub Upstream Facing Port State Descriptions

10.6.2.1 Enabled U0 States

There are four enabled U0 states that differ only in the U1 and U2 Enable settings. The following rules apply globally to all Enabled U0 states:

- The upstream port shall not initiate a transition to U1 or U2 if there are pending packets to transmit on the upstream port.
- The upstream port shall accept U1 or U2 transitions from the link partner if the Force_LinkPM_Accept bit is set to one (refer to Section 8.4.2).

The port behaves as follows for the various combinations of U1 and U2 Enable values:

U1_ENABLE = 0, U2_ENABLE = 0

- This is the default state before the hub has received any SetFeature(U1/U2_ENABLE) requests.
- The PM timer may be disabled and the PM timer values shall be ignored.
- The port's link shall accept U1 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port or one or more of the hub downstream ports has a link in U0 or recovery.
- The port's link shall accept U2 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port or one or more of the hub downstream ports has a link in U0, U1, or recovery.
- The port's link shall not attempt to initiate transitions to U1 or U2.

U1_ENABLE = 1, U2_ENABLE = 0

- The port's link shall not initiate a U2 transition.
- The port's link shall accept all U2 entry requests by the link partner unless the hub has one or more packets/link commands to transmit on the port or one or more of the hub downstream ports has a link in U0, U1 or recovery.
- The port's link shall accept U1 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port or one or more of the hub downstream ports has a link in U0 or recovery.
- The PM timer may be disabled and the PM timer values shall be ignored.
- The port's link shall initiate a transition to U1 if all the hub downstream ports are in U1 or a lower link state.

U1_ENABLE = 0, U2_ENABLE = 1

- The port's link shall not initiate a U1 transition.
- The port's link shall accept all U1 entry requests by the link partner unless the hub has one or more packets/link commands to transmit on the port or one or more of the hub downstream ports has a link in U0 or recovery.
- The port's link shall accept U2 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port or one or more of the hub downstream ports has a link in U0, U1, or recovery.
- The PM timer may be disabled and the PM timer values shall be ignored.
- The port's link shall initiate a transition to U2 if all the hub downstream ports are in U2 or a lower link state.

U1_ENABLE = 1, U2_ENABLE = 1

- The port's link shall accept U1 or U2 entry requests by its link partner unless the hub has one or more packets/link commands to transmit on the port.
 - A U1 entry request shall not be accepted if one or more of the hub downstream ports has a link in U0 or recovery.
 - A U2 entry request shall not be accepted if one or more of the hub downstream ports has a link in U0, U1, or recovery.
- The port's link shall initiate a transition to U1 if all the hub downstream ports are in U1 or a lower link state unless the conditions for U2 entry are satisfied.
- The port's link shall initiate a transition to U2 if all the hub downstream ports are in U2 or a lower link state. Note that if the port is already in U1, then the port shall transition to U0 before transitioning to U2.
- The PM timer may be disabled and the PM timer values shall be ignored.

A port transitions to one of the Enabled U0 states (depending on the U1 and U2 Enable values) in any of the following situations:

- From U1 if the link partner successfully initiates a transition to U0.
- From U2 if the link partner successfully initiates a transition to U0.
- From U1 if there is a status change on a downstream port.
- From U2 if there is a status change on a downstream port.
- From U1 if a hub downstream port's link initiates a transition to U0.
- From U2 if a hub downstream port's link initiates a transition to U0.
- From an attempt to transition from the U0 to the U1 state if the upstream port's link partner rejects the transition attempt
- From an attempt to transition from the U0 to the U2 state if the upstream port's link partner rejects the transition attempt
- From U3 if the upstream port of the hub receives wakeup signaling.
- From U3 if there is a status change on a downstream port or a local power status change and remote wakeup is enabled for the corresponding event type.

10.6.2.2 Attempt U0 – U1 Transition

In this state the port attempts to transition its link from the U0 state to the U1 state.

A port shall attempt to transition to the U1 state in any of the following situations:

- U1 entry is requested by the link partner and there is no pending traffic on the port and all the hub downstream port's links are in U1 or a lower state.
- All the hub downstream ports are in U1 or a lower link state and there is no pending traffic to transmit on the upstream port and U1_ENABLE is set to one.
- U1 entry is requested by the link partner and Force_LinkPM_Accept bit is set.

If the transition attempt fails (an LXU is received or the link goes to recovery), the port returns to the appropriate enabled U0 state.

10.6.2.3 Attempt U0 – U2 Transition

In this state, the port attempts to transition the link from the U0 state to the U2 state.

A port shall attempt to transition to the U2 state in any of the following situations:

- U2 entry is requested by the link partner and there is no pending traffic on the port and all the hub downstream port's links are in U2 or a lower state.
- All the hub downstream ports are in U2 or a lower link state and there is no pending traffic to transmit on the upstream port and U2_ENABLE is set to one.
- U2 entry is requested by the link partner and Force_LinkPM_Accept bit is set.

If the transition attempt fails (an LXU is received or the link goes to recovery), the port returns to the appropriate enabled U0 state.

10.6.2.4 Link in U1

The PM timer is reset when this state is entered and is active.

A port transitions to U1:

- After sending an LAU to accept a transition initiated by the link partner.
- After receiving an LAU from the link partner after initiating an attempt to transition the link to U1

If the U2 inactivity timeout is not 0xFF or 0x00, and the PM timer reaches the U2 inactivity timeout, the port's link shall initiate a transition from U1 to U2.

10.6.2.5 Link in U2

The link is in U2.

A port transitions to U2:

- After sending an LAU to accept a transition initiated by the link partner.
- After receiving an LAU from the link partner after initiating an attempt to transition the link to U2

10.6.2.6 Link in U3

The link is in U3.

A port transitions to U3:

- After sending an LAU to accept a transition initiated by the link partner.

10.7 SuperSpeed Hub Header Packet Forwarding and Data Repeater

The SuperSpeed Hub uses a store and forward model for header packets and a repeater model for data that combined provide the following general functionality.

In the downstream direction:

- Validates header packets
- Sets up connection to selected downstream port
- Forwards header packets to downstream ports
- Forwards data payload to downstream port if present
- Sets up and tears down connectivity on packet boundaries

In the upstream direction:

- Validates header packets
- Sets up connection to upstream port

- Forwards header packets to the upstream port
- Forwards data packet payload to upstream port if present
- Sets up and tears down connectivity on packet boundaries

10.7.1 SuperSpeed Hub Elasticity Buffer

There are no direct specifications for elasticity buffer behavior in a SuperSpeed hub. However, note that a SuperSpeed hub must meet the requirements in Section 10.7.3 for the maximum variation in propagation delay for header packets that are forwarded from the upstream port to a downstream port.

10.7.2 SKP Ordered Sets

A SuperSpeed hub transmits SKP ordered sets, following the rules for all transmitters in Chapter 6, for all transmissions.

10.7.3 Interpacket Spacing

When a SuperSpeed hub originates or forwards packets, Data packet headers and data packet payloads shall be sent as required in Section 7.2.1.

When a SuperSpeed hub forwards a header packet downstream and the downstream port link is in U0 when the header packet is received on the hub upstream port the propagation delay variation shall not be more than $t_{PropagationDelayJitterLimit}$.

10.7.4 SuperSpeed Header Packet Buffer Architecture

The specification does not require a specific architecture for the header packet buffers in a SuperSpeed hub. An example architecture that meets the functional requirements of this specification is shown in Figure 10-15 and Figure 10-16 to illustrate the functional behavior of a SuperSpeed hub. Figure 10-15 shows a SuperSpeed hub with a four header packet Rx buffer for the upstream port and a four header packet Tx buffer for each of the downstream ports. Figure 10-16 shows a four header packet Rx buffer for each of the downstream ports and a four header packet Tx buffer for the upstream port. The buffers shown in Figure 10-15 and Figure 10-16 are independent physical buffers.

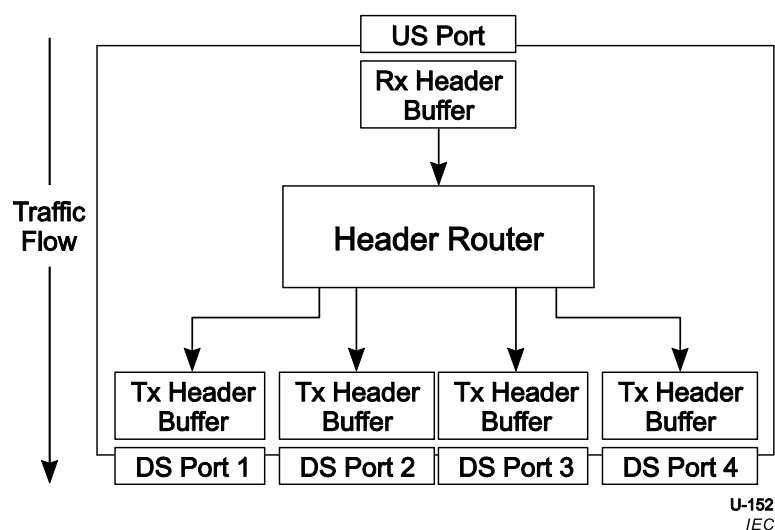


Figure 10-15 – Example SS Hub Header Packet Buffer Architecture – Downstream Traffic

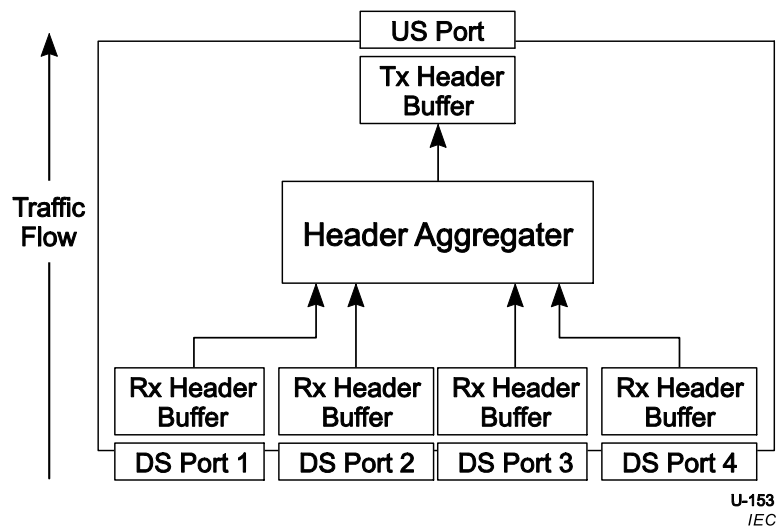


Figure 10-16 – Example SS Hub Header Packet Buffer Architecture – Upstream Traffic

The following lists functional requirements for a SuperSpeed hub buffer architecture with the assumption in each case that only the indicated port on the hub is receiving or transmitting header packets:

- A SuperSpeed hub starting with all header packet buffers empty shall be able to receive at least eight header packets directed to the same downstream port that is not in U0 before its upstream port runs out of header packet flow control credits.
- A SuperSpeed hub that receives a header packet on its upstream port that is routed to a downstream port shall immediately route the header packet to the appropriate downstream port header packet buffer (if space in that buffer is available) regardless of the state of any other downstream port header packet buffers or the state of the upstream port Rx header packet buffer. For example, a hub Tx header packet buffer for downstream port 1 is full and the hub has three more header packets routed to downstream port 1 in the hub upstream port Rx header packet buffer. If the hub now receives a header packet routed to downstream port 2, it must immediately route the header packet to the downstream port 2 Tx header packet buffer.
- A SuperSpeed hub starting with all header packet buffers empty shall be able to receive at least eight header packets on the same downstream port directed for upstream transmission when the upstream port is not in U0.
- Header packets transmitted by a downstream port shall be transmitted in the order they were received on the upstream port.
- Header packets transmitted by an upstream port from the same downstream port shall be transmitted in the order they were received on that downstream port.

Section 10.9 provides detailed functional state machines for the upstream and downstream port Tx and Rx header packet buffers in a hub implementation.

The SuperSpeed hub shall have at least 1 080 bytes of buffering for data packets received on the upstream port.

The SuperSpeed hub shall have at least 1 080 bytes of shared buffering for data packets received on all downstream ports.

10.7.5 SuperSpeed Packet Connectivity

The SuperSpeed hub packet repeater/forwarder must re-clock the packets in both directions. Re-clocking means that the repeater extracts the data from the received stream and retransmits the stream using its own local clock.

10.8 SuperSpeedPlus Store and Forward Behavior

The SuperSpeedPlus Hub provides the following general functionality.

In the downstream direction:

- Receives and validates packet
- Forwards packet to appropriate downstream port
- Selects next packet to transmit on (each) downstream port

In the upstream direction:

- Receives and validates packet
- Forwards packet to the upstream port
- Selects next packet to transmit on the upstream port

10.8.1 Hub Elasticity Buffer

There are no direct specifications for elasticity buffer behavior in a hub. However, note that a hub must meet the requirements in Section 10.7.3 for the maximum variation in propagation delay.

10.8.2 SKP Ordered Sets

A SuperSpeedPlus hub transmits SKP ordered sets, following the rules for all transmitters in Chapter 6, for all transmissions.

10.8.3 Interpacket Spacing

When a hub originates or forwards packets, DPHs and their corresponding DPPs shall be sent as required in Section 7.2.1.

The SuperSpeedPlus hub has several aspects to its store and forward behavior including buffering, arbitration among packets to be forwarded upstream, and modifications of packets during forwarding.

10.8.4 Upstream Flowing Buffering

The SuperSpeedPlus hub shall provide buffering for 16x 1KB Control/Bulk DPP buffers and 16x 1KB Interrupt/Isochronous DPP buffers for each DFP receiver. The SuperSpeedPlus hub shall provide buffering for 16x Control/Bulk header buffers and 16x TP/Interrupt/Isochronous header buffers per DFP receiver. These buffers shall be used to hold packets received from downstream ports that are awaiting transmission on the upstream port.

Buffer space is required for each downstream port since there can be packets simultaneously arriving on each downstream port while there is a packet being transmitted on the upstream port. Further, the hub arbitration rules (see Section 10.8.6) can delay when a packet received on a downstream port can be transmitted on the upstream port.

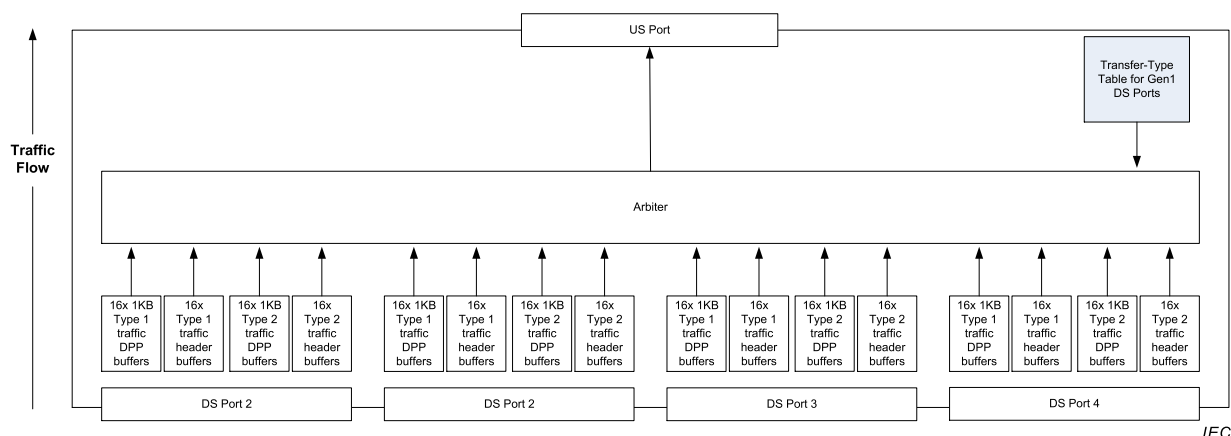


Figure 10-17 – Logical Representation of Upstream Flowing Buffers

10.8.5 Downstream Flowing Buffering

The SuperSpeedPlus hub shall provide buffering for 18x 1KB Control/Bulk DPP buffers and 18x 1KB Interrupt/Isochronous DPP buffers per hub. The SuperSpeedPlus hub shall provide buffering for 18x Control/Bulk header buffers and 18x TP/Interrupt/Isochronous header buffers per hub.

Buffering for downstream flowing traffic is primarily present to provide a rate matching function due to the different possible upstream port and downstream port speeds. Therefore, it is provided for each hub and not for each downstream port. However, the organization and function of this buffering shall allow packets to be received from the upstream port and then subsequently transmitted on multiple downstream ports simultaneously and in a different order than the order in which they were received. That is, this buffering cannot be organized as a single, simple FIFO.

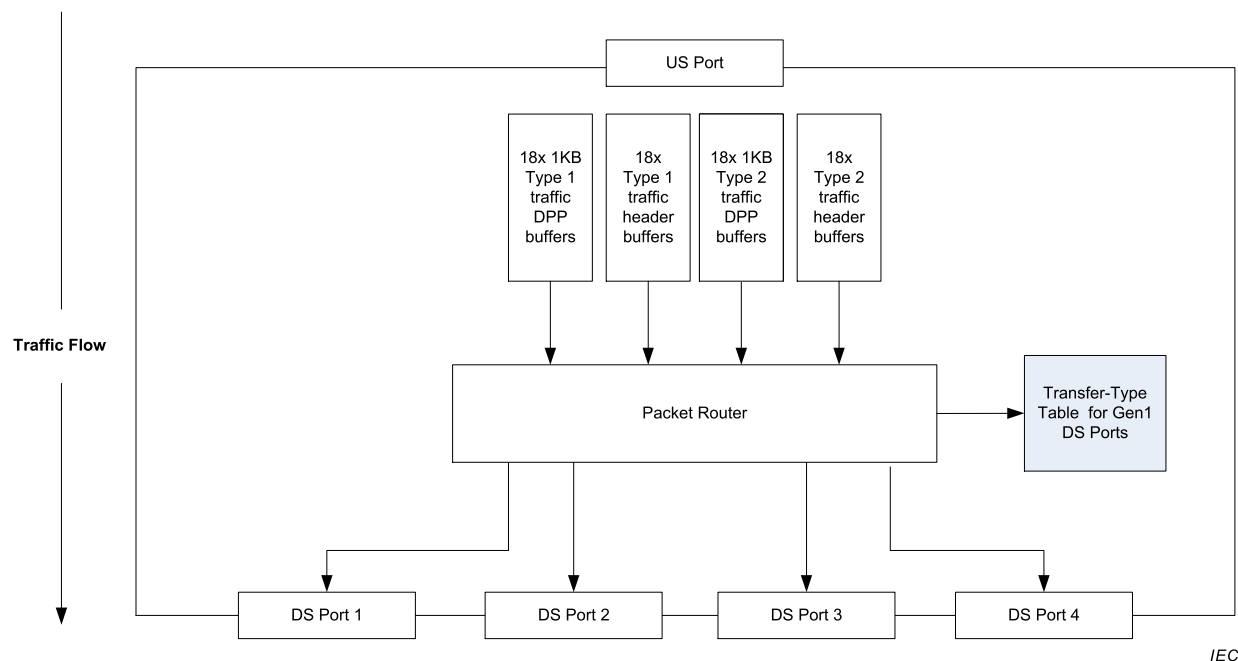


Figure 10-18 – Logical Representation of Downstream Flowing Buffers

10.8.6 SuperSpeedPlus Hub Arbitration of Packets

10.8.6.1 Arbitration Weight

The i^{th} downstream facing port (DFPi) has an arbitration weight (AW) associated with it. This weight shall be set to

$$\text{DFPi.AW} = \text{DFPi.link_speed} / \text{ArbitrationWeightBase}$$

For example, a port link operating at 5 Gb/s will have an AW of 4. A port link operating at 10 Gb/s will have an AW of 8.

10.8.6.2 Direction Independent Packet Selection

When there are multiple packets buffered that are ready to be transmitted out of the hub, the SuperSpeedPlus hub has to select which packet to transmit next.

There are several selection rules that are independent of direction of packet flow.

The SuperSpeedPlus hub has additional rules that are specific for upstream and downstream flowing packet reception and selection (see the next two sections).

A TP shall only be considered as a possible candidate after it has been fully received and validated.

A buffered TP shall be selected for transmission before any buffered DPs. TPs shall be selected in the order in which they were buffered for a port (e.g. FIFO). When selecting a TP to transmit on the hub upstream facing port, there is no specific ordering requirement for TPs buffered from different downstream ports.

A buffered Interrupt or Isochronous DP shall be selected for transmission before any buffered Control or Bulk DPs.

Once a hub starts transmitting a packet on a port, it shall continue transmitting that packet until the packet transmission is complete. With respect to the following arbitration rules, there is no “pre-emption” of the transmission of one packet for the transmission of another packet.

If a DP is being received on a port and the port to which it is to be routed has no other packets buffered nor has a packet currently being transmitted, the hub shall begin transmitting the packet on the destination port before the DP is fully received. Transmission of the DP shall not begin transmission until sufficient bytes have been received, so that transmitter under-run is avoided.

10.8.6.3 Downstream Flowing Packet Reception and Selection

For downstream flowing traffic, buffered Isochronous and Interrupt DPs destined to be transmitted on the same downstream port shall be selected to be transmitted in the same order as they were received on the upstream port. Control and Bulk DPs buffered for transmission on the same downstream port shall be selected for transmission in the same order as they were received on the upstream port. TPs buffered for transmission on the same downstream port shall be selected for transmission in the same order as they were received on the upstream port.

10.8.6.4 Upstream Flowing Packet Reception and Selection

When the Upstream Controller needs to select a packet to transmit on the upstream port, any fully buffered packets from downstream ports are candidates for the next packet to transmit. However, some packets still being received and not fully buffered can also be candidates.

To select the next DP for transmission on the upstream port, the Upstream Controller shall use:

- A weighted round robin arbitration behavior to select the next Control/Bulk DP buffered from the hub downstream ports.
- A simple round robin arbitration behavior to select the next Interrupt/Isochronous DP buffered from the hub downstream ports.

The next section describes when an incompletely buffered DP that is still being received can be a candidate. The section after that describes the upstream weighted round robin arbitration mechanism.

10.8.6.4.1 Partially Buffered DP Selection Candidate

A DP (call it RCV_DP) shall be considered as a possible candidate, after the DPH has been fully received and validated and all of the following conditions are true:

- Let ALT_P be the candidate packet that would have been selected from the current set of fully buffered packets (i.e. when not considering RCV_DP as a possible candidate). RCV_DP would be selected when compared to ALT_P.
- The time remaining to fully receive this DPP is less than the time it will take to transmit ALT_P.
- Enough of the DPP has been received to ensure that upstream port transmitter under-run will not occur during the transmission of this packet.

For example, if there are only buffered Bulk DPs from other downstream ports and an Isochronous DP is being received on one downstream port, the Upstream Controller shall select the Isochronous DP as the next packet to transmit on the upstream port; as long as the remaining time to receive the Isochronous DP is less than the time required to transmit the Bulk DP and there is a sufficient amount of the Isochronous DPP already received.

10.8.6.4.2 Upstream Weighted Round Robin Arbitration

When the Upstream Controller needs to select the next Control/Bulk DP to transmit on the upstream port, the Upstream Controller uses the following selectPacket() algorithm to determine the next DP.

In the selectPacket() algorithm pseudo code, once a packet is selected, the algorithm is complete and any remaining steps in the algorithm are ignored for the selection of the current packet to transmit upstream.

Across invocations of the selectPacket algorithm, retain the value of *i* and curr_weight. Initial values of *i*=-1 and curr_weight=0.

The *i*th downstream facing port is DFPI. The candidate packet for the *i*th DFP is CPi.

selectPacket() algorithm:

- 1) If there are no buffered packets, set *i*=-1 and curr_weight=0 and don't select a packet.

NOTE The upstream port will await the arrival of a packet on some DFPI.

- 2) For each DFPI, identify a candidate packet CPi for the DFPI:
 - a) if there is a Control or Bulk DP buffered, set CPi to be the first one that had been buffered.
- 3) If there is only one DFPI with packets buffered for upstream transmission:
 - a) Set *i* = port index
 - b) Set cw = CPi.AW

- c) Select CPi
- d) exit
- 4) While true
 - a) $i = (i + 1) \bmod \text{num_ports}$
 - b) if ($i == 0$) then
 - i) compute the Greatest Common Divisor (GCD) of all the buffered CPi.AW
 - ii) $\text{curr_weight} = \text{curr_weight} - \text{GCD}$
 - iii) if ($\text{curr_weight} \leq 0$) then
 - 1) $\text{curr_weight} = \text{max of CPi.AWs for all buffered CPi}$
 - 2) if ($\text{curr_weight} == 0$) then there is no packet to select
 - c) if ($\text{DFPi.AW} \geq \text{curr_weight}$) then
 - i) select CPi
 - ii) exit

10.8.7 SuperSpeedPlus Upstream Flowing Packet Modifications

When the upstream port of a SuperSpeedPlus hub is operating at greater than Gen 1 speed and the hub Downstream Controller receives a valid IN/ACK TP that is routed to a downstream port (DFPi) that is operating at Gen 1 speed, the Downstream Controller shall:

- 1) Save the transfer type (SAVE_TT) of the TP for that DFPi.

When the hub Downstream Controller receives a valid DPH packet from DFPi, the Downstream Controller shall:

- 1) If the transfer type for this DFPi has been saved and the DPH is not a deferred DPH, set the transfer type of the DP to the saved value (DFPi.SAVE_TT).
- 2) If the AW field value of the received DPH is zero and the transfer type is Control or Bulk, modify the AW field of the received DPH by setting the DPH.AW field to DFPi.AW
- 3) If the DPH was modified, recompute the CRC-16 for the DPH.

This packet modification shall be done when the packet is received.

When the hub Upstream Controller selects (as described in Section 10.8.6.4) a Control/Bulk packet (S_DP) to transmit on the upstream port and there are multiple downstream ports (DFPi) with buffered Control/Bulk DPs awaiting transmission, the Upstream Controller shall:

- 1) For each DFPi, determine a candidate buffered Control/Bulk DP (C_DPi) for that DFPi that would be selected for upstream transmission if there were no other DFPi's with buffered Control/Bulk DPs.
- 2) Compute the sum (SUM_AW) of the AWs of the C_DPi's.
- 3) If the SUM_AW is different than the current value of the S_DP DPH.AW, modify the AW field of the S_DP DPH by replacing the AW value with SUM_AW
- 4) If the DPH was modified, recompute the CRC-16 for the S_DP DPH.

This modification shall be done before the packet is routed to the upstream port for transmission.

Note that in the above descriptions, a packet may appear to have its CRC-16 recomputed twice. Hub implementations are encouraged to be structured so that the correct CRC-16 value only needs to be computed once after all required modifications have been made.

10.8.8 SuperSpeedPlus Downstream Controller

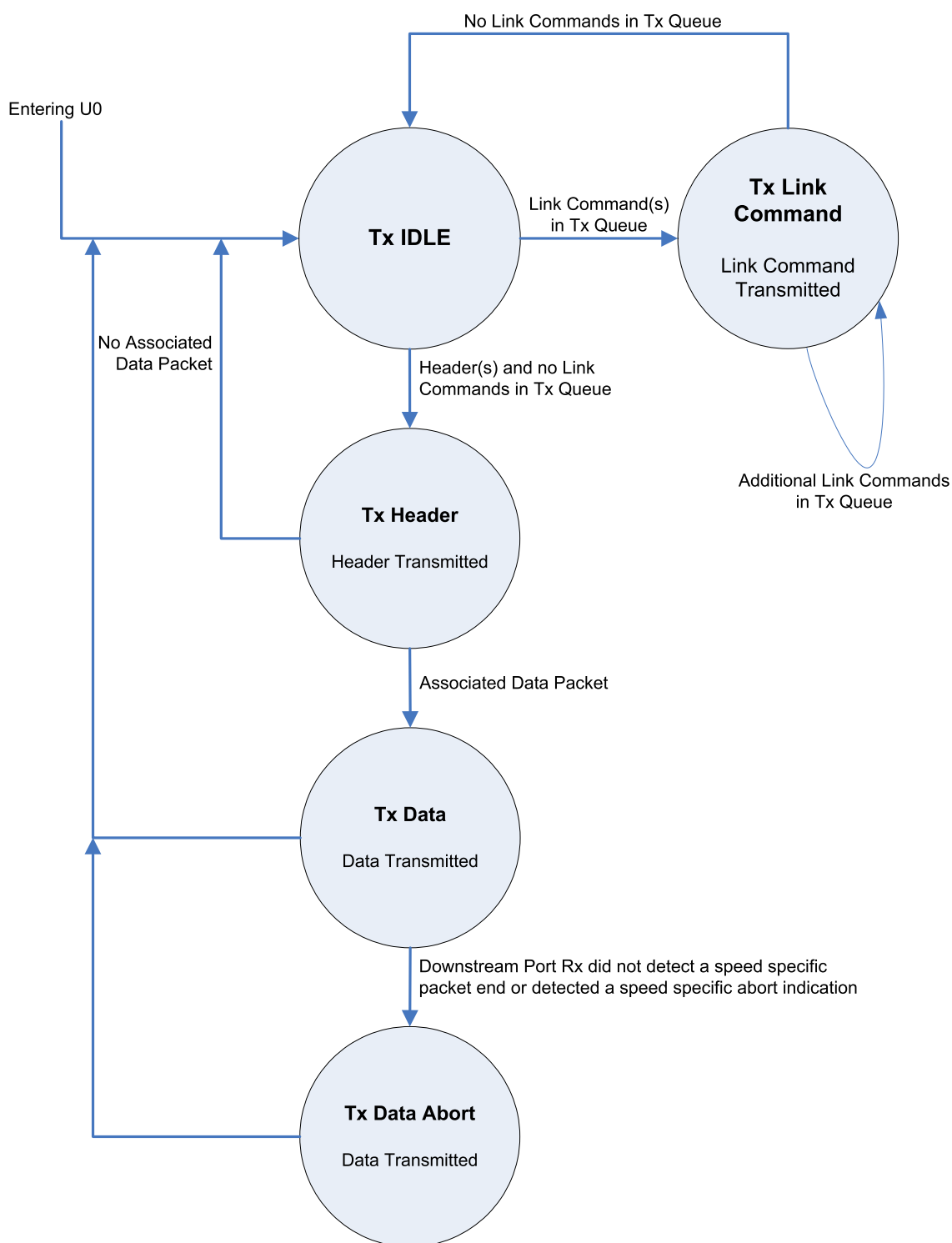
The Downstream Controller for each downstream port shall be responsible for updating the ITP fields as described in Section 8.4.8.8 before forwarding the ITP on all downstream ports in U0. See Chapter 8 for the format of an ITP.

10.9 Port State Machines

In the following descriptions of port state machines, there are references to the first or last symbol of a header packet. The first symbol of a header packet is the first DPHP or SHP (Section 7.2.1.1). The last symbol of a SuperSpeedPlus DPH header packet is the last byte of the replicated length (if present) or the last byte of the LCW (if the replicated length field is not present). The last symbol of a SuperSpeedPlus non-DPH header packet and all SuperSpeed header packets is the last byte of the LCW.

10.9.1 Port Transmit State Machine

This section describes the functional requirements of the upstream and downstream facing port Transmit (Tx) state machines. Upstream and downstream ports shall adhere to all requirements of the link layer (see Chapter 7).



IEC

Figure 10-19 – Port Transmit State Machine

10.9.2 Port Transmit State Descriptions

10.9.2.1 Tx IDLE

In the Tx IDLE state, the port transmitter is actively transmitting idle symbols. A port transmitter shall transition to the Tx IDLE state in any of the following situations:

- From the Tx Data, Tx Data Abort, or Tx Header state after packet transmission is completed.

- From the Tx Link Command state after a link command is transmitted and there are no other link commands awaiting transmission.
- As the default state when the link enters U0.

10.9.2.2 Tx Header

In the Tx Header state, the port transmitter is actively transmitting a header packet.

A port transmitter shall transition to the Tx Header state in any of the following situations:

- From the Tx IDLE state when there are one or more header packets queued for transmission and there are no link commands queued for transmission.

10.9.2.3 Tx Data

In the Tx Data state, the port transmitter is actively transmitting a DPP. After transmitting the DPP, the port transmitter may remove the DPP from hub storage. A hub shall not retransmit a DPP under any circumstances.

A port transmitter shall transition to the Tx Data state from the Tx Header state when there is a DPP associated with the DPH that was transmitted. The DPP transmission shall begin immediately after transmission of the last symbol of the DPH.

10.9.2.4 Tx Data Abort

In the Tx Data abort state, the port transmitter aborts the normal transmission of a DPP by performing speed specific abort processing (see Section 7.2.1.2.3). The port transmitter then removes the DPP from hub storage.

In the case where the hub is simultaneously receiving a DPP into the hub and transmitting the same DPP out of the hub:

- An upstream port transmitter shall transition to the Tx Data Abort state from the Tx Data state when the downstream port receiving the DPP detects a speed specific abort indication.
- A downstream port transmitter shall transition to the Tx Data Abort state from the Tx Data state when the upstream port receiving the DPP detects a speed specific abort indication.

10.9.2.5 Tx Link Command

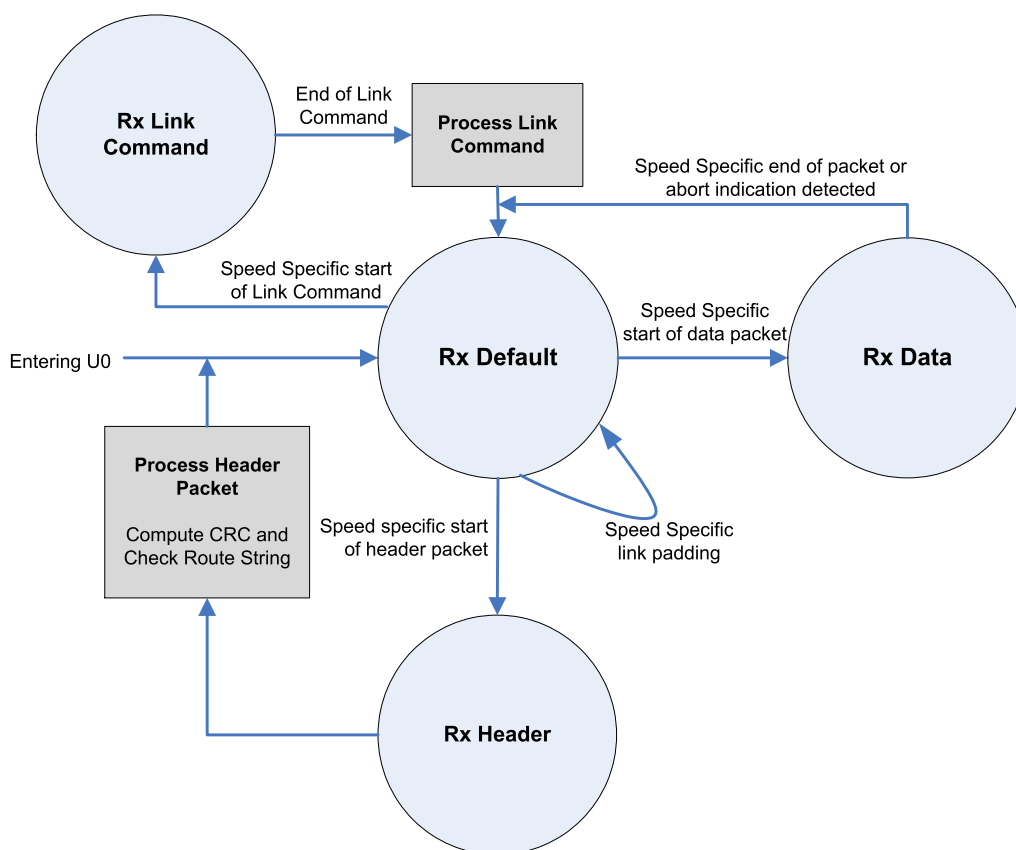
In the Tx Link Command state, the port transmitter is actively transmitting a link command.

A port transmitter shall transition to the Tx Link Command state in any of the following situations:

- From the Tx IDLE state when there are one or more link commands queued for transmission.
- From the Tx Link Command state when there are additional link commands queued for transmission.

10.9.3 Port Receive State Machine

This section describes the functional requirements of the upstream and downstream facing port receiver (Rx) state machine.



IEC

Figure 10-20 – Upstream Facing Port Rx State Machine

10.9.4 Port Receive State Descriptions

10.9.4.1 Rx Default

In the Rx Default state, the port receiver is actively receiving symbols and looking for the speed specific beginning of a valid packet or a link command.

A port receiver shall transition to the Rx Default state in any of the following situations:

- From the Rx Data state when a speed specific end of packet or abort indication is detected.
- From the Rx Header state when the last symbol of the header packet is received.
- After receiving a link command.
- As the default state when the link enters U0.

10.9.4.2 Rx Data

In the Rx Data state, the port receiver is actively processing symbols and looking for the speed specific indication of the end of a packet or the occurrence of an abort condition.

A port shall transition to the Rx Data state when it receives a speed specific start of packet indication.

When the port detects an error before the end of the DPP as defined in Section 7.2.4.1.6, it performs speed specific abort processing (see Section 7.2.1.2.3).

In the case where the hub is simultaneously receiving a DPP into the hub and transmitting the same DPP out of the hub, the corresponding port transmitter shall be given an indication of the abort condition so that it can perform speed specific abort processing.

If the DPP is not being actively transmitted out of the hub,

- For an upstream port receiver, the hub shall buffer a speed specific aborted DP for the appropriate downstream port.
- For a downstream port receiver, the hub shall buffer a speed specific aborted DP on the upstream port.

10.9.4.3 Rx Header

In the Rx header state, the port receiver is actively processing received symbols until the last header packet symbol is received.

A port shall transition to the Rx Header state when it detects the speed specific beginning of a header packet.

The port shall validate CRC-16, the Link Control Word CRC-5, check the route string (only if this is an upstream port) and header packet type within four symbol times after the last symbol of the header packet is received.

Implementations may have to begin the CRC calculation as the header is being received and check the route string before the header packet is verified to meet this requirement.

10.9.4.4 Process Header Packet

When the last symbol of a header packet is received the port shall perform all processing necessary for the header packet. Any such processing shall not block the port from immediately returning to the Rx Default state.

As described in the link chapter, when the last symbol of a header packet is received in the Rx header packet state and either the header packet CRC-16 or Link Control Word CRC-5 is determined to be invalid, the link layer won't pass the header packet to the hub.

Table 10-3 summarizes the actions of the hub when it receives a packet on its upstream port that is targeted for a valid downstream port.

Table 10-3 – Downstream Flowing Header Packet Processing Actions

DFP LTSSM State	Packet Type		
	ITP	PING	Other TPs/DPs
U0	Queue	Queue	Queue
U1/U2	Silently Discarded	Queue (Wake)	Queue (Wake, DF, drop DPP)
Recovery	Queue	Queue	Queue
Others	Silently Discarded	Silently Discarded	Silently Discarded

The steps described in the next 4 sections depend on:

- whether the hub is operating as a SuperSpeed hub or a SuperSpeedPlus hub, and
- whether the port processing is being done for an upstream or downstream facing port.

10.9.4.4.1 SuperSpeed Hub Upstream Facing Port

- The header packet is not an ITP and not a PING and is routed to a downstream port that is in U1 or in U2:
 - 1) The hub initiates U0 entry on the appropriate downstream port link. U0 entry shall be initiated no later than $t_{DownLinkStateChange}$ from when the hub received the first symbol of the header packet.
 - 2) If the header packet is not already marked deferred:
 - a) The header packet is marked deferred and the Link Control Word CRC-5 is re-calculated for the deferred header packet. If the deferred header packet is a DPH, the corresponding DPP is silently discarded.
 - b) A copy of the header packet is modified to include the hub's hub depth, marked as deferred and with the Link Control Word CRC-5 recalculated is queued for transmission on the upstream port. Note that the route string in this deferred header packet is preserved and not set to zero.
 - 3) The deferred header packet (see Section 7.2.4.1.4) is queued for transmission on the appropriate downstream port.
- If the header packet is a PING and is routed to a downstream port that is in U0 or is in U1 or is in U2 or is in Recovery:
 - 1) If the appropriate downstream port link is in U1 or in U2, the hub initiates U0 entry on the appropriate downstream port link. U0 entry shall be initiated no later than $t_{DownLinkStateChange}$ from when the hub received the first symbol of the header packet.
 - 2) The header packet is queued for transmission on the appropriate downstream port.
- If the header packet is not an ITP and not a PING and is routed to a downstream port that is in U0 or in Recovery:
 - 1) If the downstream port Tx header packet buffer queue is not empty (there is at least one header packet in the queue that has not been completely transmitted) or no link credit is available for transmission on the downstream port, the header packet is marked delayed and the Link Control Word CRC-5 is re-calculated for the modified header packet.
 - 2) The header packet is queued for transmission on the appropriate downstream port.

NOTE If the queue for the appropriate downstream port is full, the header packet is queued as soon as a space is available for the appropriate downstream port. The hub shall process subsequent header packets while a downstream port buffer is full if they are directed to a different downstream port.
- If the header packet is an ITP then for each downstream port:
 - 1) The ITP is silently discarded for any downstream port with a link not in U0 and not in Recovery.
 - 2) The Delta and Correction fields in the ITP shall be updated to account for the measured delay of propagating the ITP through the hub.
 - a) If the delay introduced by the hub exceeds the $t_{PropagationDelayJitterLimit}$, then the header packet shall be marked Delayed (DL) and the correct Link Control Word CRC-5 is re-calculated for modified header packet.
 - b) If the *Delta* subfield overflowed, the ITP shall not be queued, otherwise the header packet shall be queued for transmission on each downstream port that has completed Port Configuration and is in U0 or in Recovery.

NOTE If the queue for the appropriate downstream port is full, the header packet is queued as soon as a space is available in the appropriate downstream port queue. The hub shall process subsequent header packets while a downstream port queue is full if they are directed to a different downstream port.
- If the header packet is routed to a disabled or nonexistent downstream port or to a downstream port that is in not in U0 and not in U1 and not in U2 and not in Recovery:
 - 1) The header packet is removed from the RX header packet queue.

- 2) The header packet is silently discarded.
- 3) If the header packet is a DPH the corresponding DPP is silently discarded.
- If the header packet is routed to the hub controller:
 - 1) The header packet is processed by the hub controller.
 - 2) The header packet is removed from the RX header packet queue.
 - 3) A response to the header packet is queued for transmission on the upstream port, if required.

10.9.4.4.2 SuperSpeedPlus Hub Upstream Facing Port

- The header packet is not an ITP and not a PING and is routed to a downstream port that is in U1 or in U2:
 - 1) The hub initiates U0 entry on the appropriate downstream port link. U0 entry shall be initiated no later than $t_{DownLinkStateChange}$ from when the hub received the first symbol of the header packet.
 - 2) If the header packet is not already marked deferred:
 - a) The header packet is marked deferred and the Link Control Word CRC-5 is re-calculated for the deferred header packet. If the deferred header packet is a DPH, the corresponding DPP is silently discarded.
 - b) A copy of the header packet is modified to include the hub's hub depth, marked as deferred and with the Link Control Word CRC-5 recalculated is buffered awaiting arbitration for transmission on the upstream port. Note that the route string in this deferred header packet is preserved and not set to zero.
 - 3) The deferred header packet (see Section 7.2.4.1.4) is buffered awaiting arbitration (see Section 10.8.6.4) for transmission on the appropriate downstream port.
- If the header packet is a PING and is routed to a downstream port that is in U0 or is in U1 or is in U2 or is in Recovery:
 - 1) If the appropriate downstream port link is in U1 or in U2, the hub initiates U0 entry on the appropriate downstream port link. U0 entry shall be initiated no later than $t_{DownLinkStateChange}$ from when the hub received the first symbol of the header packet.
 - 2) The header packet is buffered awaiting arbitration for transmission on the appropriate downstream port.
- If the header packet is not an ITP and not a PING and is routed to a downstream port that is in U0 or in Recovery:
 - 1) If the downstream port is currently transmitting a packet or there is at least one packet buffered that will be selected before this packet or no link credit is available for transmission on the downstream port, the header packet is marked delayed and the Link Control Word CRC-5 is re-calculated for modified header packet.
 - 2) The header packet is buffered awaiting arbitration for transmission on the appropriate downstream port.
- If the header packet is an ITP then for each downstream port:
 - 1) The ITP is silently discarded for any downstream port with a link not in U0 and not in Recovery.
 - 2) The Delta and Correction fields in the ITP shall be updated to account for the measured delay of propagating the ITP through the hub.
 - a) If the delay introduced by the hub exceeds the $t_{PropagationDelayJitterLimit}$, then the header packet shall be marked Delayed (DL) and the correct Link Control Word CRC-5 is re-calculated for modified header packet.
 - b) If the *Delta* subfield overflowed, the ITP shall not be buffered, otherwise the header packet is buffered awaiting arbitration for transmission on each downstream port that has completed Port Configuration and is in U0 or in Recovery.

- If the header packet is routed to a disabled or nonexistent downstream port or to a downstream port that is not in U0 and not in U1 and not in U2 and not in Recovery:
 - 1) The header packet is removed from the Upstream Receive buffer.
 - 2) The header packet is silently discarded.
 - 3) If the header packet is a DPH the corresponding DPP is silently discarded.
- If the downstream port to which the packet is being routed is operating at Gen 1 speed and the header packet is a valid IN/ACK, save the transfer type (DFP.SAVE_TT) of the IN/ACK. The DFP.SAVE_TT is preserved until the next IN/ACK is received that is routed to the same downstream port. See Section 10.9.4.4.4.
- If the header packet is routed to the hub controller:
 - 1) The header packet is processed by the hub controller.
 - 2) The header packet is removed from the Upstream Receive buffer.
 - 3) A response to the header packet is buffered awaiting arbitration for transmission on the upstream port if required.

10.9.4.4.3 SuperSpeed Hub Downstream Facing Port

- The header packet is queued for transmission on the upstream port.

If the queue for the upstream port is full, the header packet is queued as soon as a space is available in the upstream port queue. The hub shall process subsequent header packets while the upstream port queue is full. If header packets have been received on more than one downstream port or are queued to be sent by the hub controller when a space becomes available in the upstream port header packet queue, the hub shall prioritize a non-data packet header over a data packet header packet if one is waiting at the front of a downstream queue or from the hub controller. Otherwise, the arbitration algorithm the hub uses is not specified.

NOTE These arbitration requirements only apply across multiple downstream ports and the hub controller. For a single source (downstream port or hub controller), packets must be transmitted in the order received or generated.

10.9.4.4.4 SuperSpeedPlus Hub Downstream Facing Port

- If a valid DP is received then:
 - 1) If the port is operating at Gen 1 speed then set the transfer type of the DP to the value of DFP.SAVE_TT. See Section 10.9.4.4.2.
 - 2) If the transfer type is asynchronous and the AW field value is zero, modify the AW field of the received DPH by setting the DPH.AW field to DFP.AW. See Section 10.8.6.
- The header packet is buffered awaiting arbitration for transmission on the upstream port (see Section 10.8.6).

10.9.4.5 Rx Link Command

In the Rx Link Command state, the port receiver is actively processing received symbols and looking for the speed specific indication of the end of a link command.

A port shall transition to the Rx Link Command state when it receives a valid speed specific indication of the beginning of a link command.

10.9.4.6 Process Link Command

Once the link command is received, the port shall perform all additional processing necessary for the link command. Any such processing shall not block the port from immediately returning to the Rx Default state.

10.10 Suspend and Resume

Hubs must support suspend and resume both as a USB device and in terms of propagating suspend and resume signaling. Global suspend/resume refers to the entire bus being suspended or resumed without affecting any hub's downstream facing port states; selective suspend/resume refers to a downstream facing port of a hub being suspended or resumed without affecting the hub state. Enhanced SuperSpeed hubs only support selective suspend and resume. They do not support global suspend and resume. Selective suspend/resume is implemented via requests to a hub. Device-initiated resume is called remote-wakeup.

The hub follows the same suspend requirements as an Enhanced SuperSpeed device on its upstream facing port.

When a hub downstream port link is in the U3 state, the following requirements apply to the hub if it receives wakeup signaling from its link partner on that downstream port:

- If the hub upstream port's link is not in U3, the hub shall drive remote wakeup signaling on the downstream link where the wakeup signaling was received in tHubDriveRemoteWakeDownstream.
- If the hub upstream port's link is in U3, the hub shall drive wakeup signaling on its upstream port in tHubPropRemoteWakeUpstream.
- If the hub upstream port is in the process of entering U3, the hub shall wait until the U3 entry is completed, before driving wakeup signaling on its upstream port in tHubPropRemoteWakeUpstream.

When a hub upstream port's link enters the U3 state and one of its downstream links is in U0/U1/U2/Recovery and has received a remote wake, the hub shall automatically drive remote wakeup on upstream port in tHubPropRemoteWakeUpstream.

When a hub upstream port's link is in the U3 state and it receives wakeup signaling from its link partner on the hub upstream port's link, the hub shall automatically drive remote wakeup to any downstream ports that are in U3 and have received remote wakeup signaling since entering U3.

If the hub upstream port's link is in U3, the hub shall drive wakeup signaling on its upstream port due to connect (when the downstream port enters DSPORT.Enabled), disconnect, or overcurrent events, if the hub is enabled for remote wakeup.

When the hub receives a SetPortFeature(PORT_LINK_STATE) U0 for a downstream port with a link in U3, the hub shall drive remote wakeup signaling on the link in tHubDriveRemoteWakeDownstream.

10.11 Hub Upstream Port Reset Behavior

Reset signaling to a hub is defined only in the downstream direction, which is at the hub's upstream facing port. The reset signaling mechanism required of the hub is described in Chapter 6.

A suspended hub shall interpret the start of reset as a wakeup event; it shall be awake and have completed its reset sequence by the end of reset signaling.

After completion of a Warm Reset, the entire hub returns to the default state.

After completion of a Hot Reset, the hub returns to the default state except port configuration information is maintained for the upstream port.

Irrespective of how the hub was reset, the hub needs to propagate reset as described in Section 10.3.1.6 and not just transition those downstream ports to the default state.

10.12 Hub Port Power Control

Self-powered hubs may have power switches that control delivery of power to downstream facing ports but it is not required. A hub with power switches can switch power to all ports as a group/gang, to each port individually, or have an arbitrary number of gangs of one or more ports.

A hub indicates whether or not it supports power switching by the setting of the Logical Power Switching Mode field in *wHubCharacteristics*. If a hub supports per-port power switching, then the power to a port is turned on or off as specified in Table 10-2. If a hub supports ganged power switching, then the power to all ports in a gang is turned on when power is required to be on for any port in the gang. The power to a gang is not turned off unless all ports in a gang are in a state that allows power to be removed as specified in Table 10-2. The power to a port is not turned on by a *SetPortFeature(PORT_POWER)* if both *C_HUB_LOCAL_POWER* and *Local Power Status* (in *wHubStatus*) are set to one at the time when the request is executed and the *PORT_POWER* feature would be turned on. A hub that supports charging applications may keep power on at other times. Refer to Section 10.3.1.1 for more details on allowed behavior for a hub that supports charging applications.

Although a self-powered hub is not required to implement power switching, the hub shall support the Powered-off states for all ports.

For a hub with no power switches, *bPwrOn2PwrGood* shall be set to zero.

10.12.1 Multiple Gangs

A hub may implement any number of power and/or over-current gangs. A hub that implements more than one over-current and/or power switching gang shall set both the Logical Power Switching Mode and the Over-current Reporting Mode to indicate that power switching and over-current reporting are on a per port basis (these fields are in *wHubCharacteristics*).

When an over-current condition occurs on an over-current protection device, the over-current is signaled on all ports that are protected by that device. When the over-current is signaled, all the ports in the group are placed in the *DSPORT.Powered-off* or the *DSPORT.Powered-off-reset* state, and the *C_PORT_OVER_CURRENT* field is set to one on all the ports. When port status is read from any port in the group, the *PORT_OVER_CURRENT* field will be set to one as long as the over-current condition exists. The *C_PORT_OVER_CURRENT* field shall be cleared in each port individually.

When multiple ports share a power switch, setting *PORT_POWER* on any port in the group will cause the power to all ports in the group to turn on. It will not, however, because the other ports in that group to leave the *DSPORT.Powered-off* or the *DSPORT.Powered-off-reset* state. When all the ports in a group are in the *DSPORT.Powered-off* state or the hub is not configured, the power to the ports is turned off.

If a hub implements both power switching and over-current, it is not necessary for the over-current groups to be the same as the power switching groups.

If an over-current condition occurs and power switches are present, then all power switches associated with an over-current protection circuit shall be turned off. If multiple over-current protection devices are associated with a single power switch, then that switch will be turned off when any of the over-current protection circuits indicates an over-current condition.

10.13 Hub Controller

The Hub Controller is logically organized as shown in Figure 10-21.

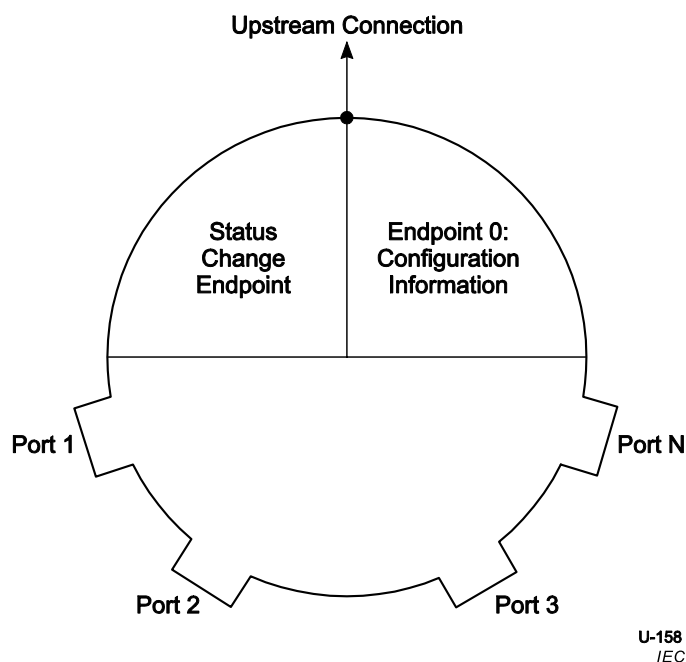


Figure 10-21 – Example Hub Controller Organization

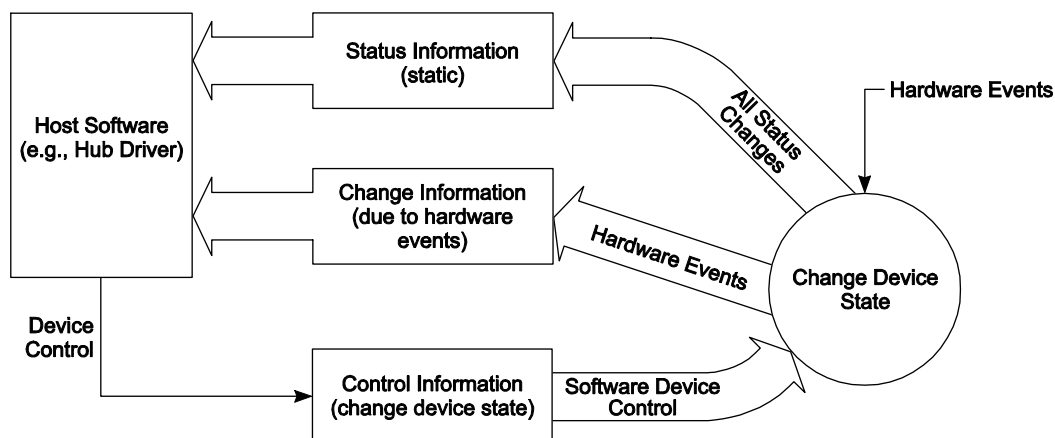
10.13.1 Endpoint Organization

The Hub Class defines one additional endpoint beyond the default control pipe, which is required for all hubs: the Status Change endpoint. This endpoint has the maximum burst size set to one. The host system receives port and hub status change notifications through the Status Change endpoint. The Status Change endpoint is an interrupt endpoint. If no hub or port status change bits are set, then the hub returns an NRDY when the Status Change endpoint receives an IN (via an ACK TP) request. When a status change bit is set, the hub will send an ERDY TP to the host. The host will subsequently ask the Status Change endpoint for the data, which will indicate the entity (hub or port) with a change bit set. The USB system software can use this data to determine which status registers to access in order to determine the exact cause of the status change interrupt.

10.13.2 Hub Information Architecture and Operation

Figure 10-22 shows how status, status change, and control information relate to device states. Hub descriptors and Hub/Port Status and Control are accessible through the default control pipe. The Hub descriptors may be read at any time. When a hub detects a change on a port or when the hub changes its own state, the Status Change endpoint transfers data to the host in the form specified in Section 10.13.4.

Hub or port status change bits can be set because of hardware or software events. When set, these bits remain set until cleared directly by the USB system software through a ClearPortFeature() request or by a hub reset. While a change bit is set, the hub continues to report a status change when the Status Change endpoint is read until all change bits have been cleared by the USB system software.



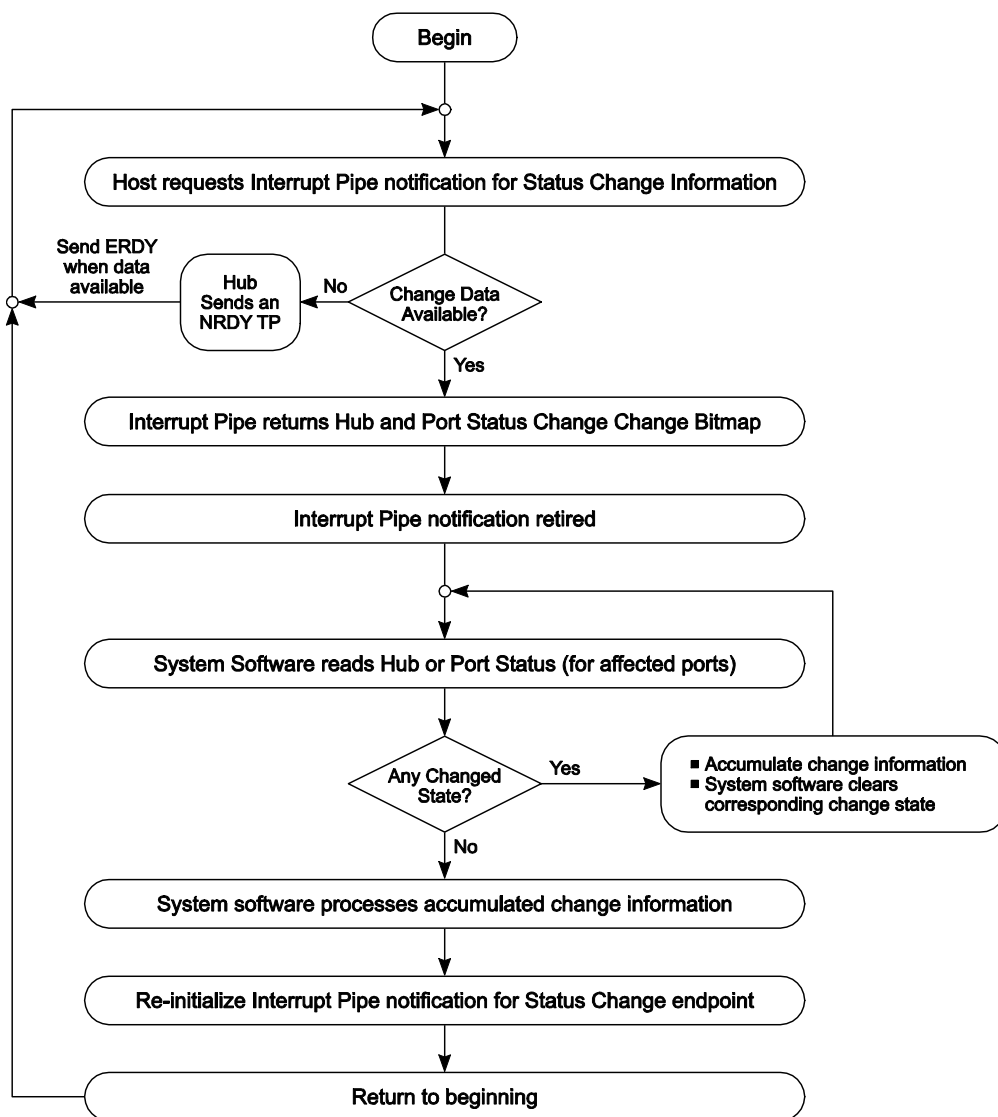
U-159
IEC

Figure 10-22 – Relationship of Status, Status Change, and Control Information to Device States

The USB system software uses the interrupt pipe associated with the Status Change endpoint to detect changes in hub and port status.

10.13.3 Port Change Information Processing

Hubs report a port's status through port commands on a per-port basis. The USB system software acknowledges a port change by clearing the change state corresponding to the status change reported by the hub. The acknowledgment clears the change state for that port so future data transfers to the Status Change endpoint do not report the previous event. This allows the process to repeat for further changes (see Figure 10-23).



U-160
IEC

Figure 10-23 – Port Status Handling Method

10.13.4 Hub and Port Status Change Bitmap

The Hub and Port Status Change Bitmap, shown in Figure 10-24, indicates whether the hub or a port has experienced a status change. This bitmap also indicates which port(s) have had a change in status. The hub returns this value on the Status Change endpoint. Hubs report this value in byte-increments. For example, if a hub has six ports, it returns a byte quantity, and reports a zero in the invalid port number field locations. The USB system software is aware of the number of ports on a hub (this is reported in the hub descriptor) and decodes the Hub and Port Status Change Bitmap accordingly. The hub reports any changes in hub status in bit zero of the Hub and Port Status Change Bitmap.

The Hub and Port Status Change Bitmap size is two bytes. Hubs report only as many bits as there are ports on the hub. A USB hub may have no more than nMaxHubPorts.

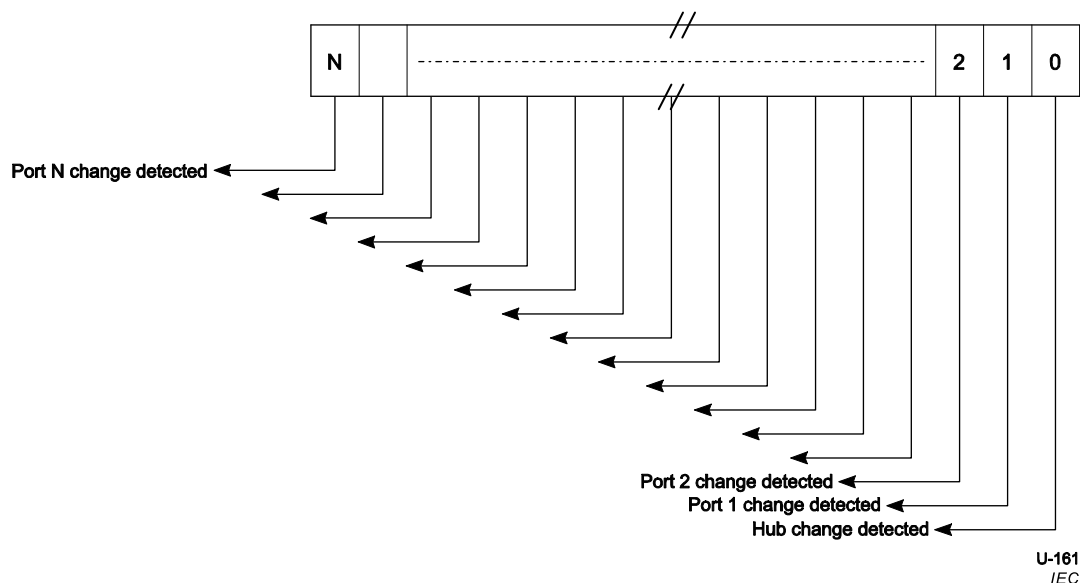


Figure 10-24 – Hub and Port Status Change Bitmap

Any time any of the Status Changed bits are non-zero, an ERDY is returned (if an NRDY was previously sent) notifying the host that the Hub and Port Status Change Bitmap has changed. Figure 10-25 shows an example creation mechanism for hub and port change bits.

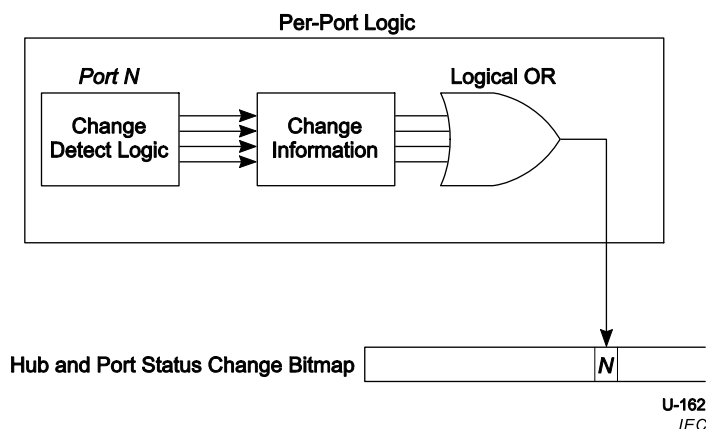


Figure 10-25 – Example Hub and Port Change Bit Sampling

10.13.5 Over-current Reporting and Recovery

USB devices shall be designed to meet applicable safety standards. Usually, this will mean that a self-powered hub implements current limiting on its downstream facing ports. If an over-current condition occurs, it causes a status and state change in one or more ports. This change is reported to the USB system software so that it can take corrective action.

A hub may be designed to report over-current as either a port or a hub event. The hub descriptor field *wHubCharacteristics* is used to indicate the reporting capabilities of a particular hub (refer to Section 10.15.2.1). The over-current status bit in the hub or port status field indicates the state of the over-current detection when the status is returned. The over-current status change bit in the Hub or Port Change field indicates if the over-current status has changed.

When a hub experiences an over-current condition, it shall place all affected ports in the DSPORT.Powered-off-reset state. If a hub has per-port power switching and per-port current limiting, an over-current condition on one port may still cause the power on another port to fall below specified minimums. In this case, the affected port is placed in the

DSPORT.Powered-off-reset state and C_PORT_OVER_CURRENT is set for the port, but PORT_OVER_CURRENT is not set. If the hub has over-current detection on a hub basis, then an over-current condition on the hub will cause all ports to enter any DSPORT.Powered-off-reset state. However, in this case, neither C_PORT_OVER_CURRENT nor PORT_OVER_CURRENT is set for the affected ports.

Host recovery actions for an over-current event should include the following:

- 1) Host gets change notification from hub with over-current event.
- 2) Host extracts appropriate hub or port change information (depending on the information in the change bitmap).
- 3) Host waits for over-current status bit to be cleared to 0.
- 4) Host cycles power to on for all of the necessary ports (e.g., issues a SetPortFeature(PORT_POWER) request for each port).
- 5) Host re-enumerates all affected ports.

10.13.6 Enumeration Handling

The hub device class commands are used to manipulate its downstream facing port state. When a device is attached, the device attach event is detected by the hub and reported on the Status Change endpoint. The host will accept the status change report and may request a SetPortFeature(PORT_RESET) on the port. The GetPortStatus request invoked by the host will return a PORT_CONNECTION indication along with the PORT_SPEED field set to zero if the downstream facing port has a Enhanced SuperSpeed device connected.

When the device is detached from the port, the port reports the status change through the Status Change endpoint. Then the process is ready to be repeated on the next device attach detect.

10.14 Hub Configuration

Hubs are configured through the standard USB device configuration commands. A hub that is not configured behaves like any other device that is not configured with respect to power requirements and addressing. A hub is required to power its downstream ports based on several factors, including whether the hub supports power switching and charging applications. Refer to Section 10.3.1.1 for details on when a hub is required to provide power to downstream ports. Configuring a hub enables the Status Change endpoint. Part of the configuration process is setting the hub depth which is used to compute an index (refer to Section 10.16.2.9) into the Route String (refer to Section 8.9). The hub depth is used to derive the offset into the Route String (in a TP or DP) that the hub shall use to route packets received on its upstream port. The USB system software may then issue commands to the hub to switch port power on and off at appropriate times.

The USB system software examines hub descriptor information to determine the hub's characteristics. By examining the hub's characteristics, the USB system software ensures that illegal power topologies are not allowed by not powering on the hub's ports if doing so would violate the USB power topology. The device status and configuration information can be used to determine whether the hub can be used in a given topology. Table 10-4 summarizes the information and how it can be used to determine the current power requirements of the hub.

Table 10-4 – Hub Power Operating Mode Summary

Configuration Descriptor		Hub Device Status (Self Power)	Explanation
MaxPower	bmAttributes (Self Powered)		
0	0	N/A	N/A This is an illegal combination.
0	1	0	N/A A device which is only self-powered, but does not have local power, cannot connect to the bus and communicate.
0	1	1	Self-powered only hub and local power supply is good. Hub status also indicates local power good. Hub functionality is valid anywhere depth restriction is not violated.
>0	0	N/A	Bus-powered only hub. Downstream facing ports may not be powered unless allowed in current topology. Hub device status reporting self-powered is meaningless if <i>bmAttributes.self-powered</i> is zero.
>0	1	0	This hub is capable of both self- and bus-powered operating modes. It is currently only available as a bus-powered hub.
> 0	1	1	This hub draws power from both the bus and its local power supply. It is currently available as a self-powered hub.

A self-powered hub has a local power supply, but may optionally draw one unit load from its upstream connection. This allows the interface to function when local power is not available (refer to Section 11.4.1.1). When local power is removed (either a hub-wide over-current condition or local supply is off), a hub of this type remains in the Configured state but transitions all ports (whether removable or non-removable) to the Powered-off state. While local power is off, all port status and change information read as zero and all SetPortFeature() requests are ignored (request is treated as a no-operation). The hub will use the Status Change endpoint to notify the USB system software of the hub event (refer to Section 10.13.4 for details on hub status).

The *MaxPower* field in the configuration descriptor is used to report to the system the maximum power the hub will draw from VBUS when the configuration is selected. The external devices attaching to the hub will report their individual power requirements.

A compound device may power both the hub electronics and the permanently attached devices from VBUS. The entire load may be reported in the hubs' configuration descriptor with the permanently attached devices each reporting self-powered, with zero *MaxPower* in their respective configuration descriptors.

A bus powered hub shall be able to supply any power not used by the hub electronics or permanently attached devices for the selected configuration to the exposed downstream ports. The hub shall be able to provide the power with any split across the exposed downstream ports (i.e., if the hub can provide 600 mA to two exposed downstream ports, it must be able to provide 450 mA to one and 150 mA to the other, 300 mA to each, etc.).

NOTE Software shall ensure that at least 150 mA is available for each exposed downstream port on a bus powered hub.

10.15 Descriptors

Hub descriptors are derived from the general USB device framework. Hub descriptors describe a hub device and the ports on that hub. The host accesses hub descriptors through the hub's default control pipe.

The USB specification (refer to Chapter 8) defines the following descriptors:

- Device Level Descriptors
- Configuration
- Interface
- Endpoint
- String (optional)

The hub class defines additional descriptors (refer to Section 10.15.2). In addition, vendor-specific descriptors are allowed in the USB device framework. Hubs support standard USB device commands as defined in Chapter 8.

A hub is the only device that is allowed to function at high-speed and a Gen X speed at the same time. This specification only defines the descriptors a hub shall report on the Enhanced SuperSpeed bus.

Note that an Enhanced SuperSpeed hub shall always support the Get Descriptor (BOS) (refer to Section 9.6.2) when operating at either the Gen X speed or at USB 2.0 speeds.

10.15.1 Standard Descriptors for Hub Class

The hub class pre-defines certain fields in standard USB descriptors. Other fields are either implementation-dependent or not applicable to this class.

A hub has a device descriptor with a *bDeviceProtocol* field set to 3 and an interface descriptor with a *bInterfaceProtocol* field set to 0.

Hub Descriptors for USB hub operating at Gen 1 speed

Device Descriptor (SuperSpeed information)

<i>bLength</i>	18
<i>bDescriptorType</i>	DEVICE Descriptor type
<i>bcdUSB</i>	310H
<i>bDeviceClass</i>	HUB_CLASSCODE (9)
<i>bDeviceSubClass</i>	0
<i>bDeviceProtocol</i>	3
<i>bMaxPacketSize0</i>	9
<i>bNumConfigurations</i>	1

BOS Descriptor

<i>bLength</i>	5
<i>bDescriptorType</i>	BOS Descriptor type
<i>wTotalLength</i>	73
<i>bNumDeviceCaps</i>	3

USB 2.0 Extension

<i>bLength</i>	7
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	USB 2.0 EXTENSION
<i>bmAttributes</i>	2

SuperSpeed USB Device Capability

<i>bLength</i>	10
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	SUPERSPEED_USB
<i>bmAttributes</i>	Implementation-dependent
<i>wSpeedsSupported</i>	14
<i>bFunctionalitySupport</i>	1
<i>bU1DevExitLat</i>	Implementation-dependent
<i>wU2DevExitLat</i>	Implementation-dependent

ContainerID

<i>bLength</i>	20
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	CONTAINER_ID
<i>bReserved</i>	0
<i>ContainerID</i>	Implementation-dependent

The hub shall also return SuperSpeedPlus USB Device Capability and Precision Time Measurement capability as defined for a hub operating at Gen 2 speeds.

Configuration Descriptor (SuperSpeed information)

<i>bLength</i>	9
<i>bDescriptorType</i>	CONFIGURATION Descriptor type
<i>wTotalLength</i>	31
<i>bNumInterfaces</i>	1
<i>bConfigurationValue</i>	X
<i>iConfiguration</i>	Y
<i>bmAttributes</i>	Z
<i>bMaxPower</i>	The maximum amount of bus power the hub will consume in this configuration

Interface Descriptor

<i>bLength</i>	9
<i>bDescriptorType</i>	INTERFACE Descriptor type
<i>bInterfaceNumber</i>	0
<i>bAlternateSetting</i>	0
<i>bNumEndpoints</i>	1
<i>bInterfaceClass</i>	HUB_CLASSCODE (9)
<i>bInterfaceSubClass</i>	0
<i>bInterfaceProtocol</i>	0
<i>iInterface</i>	1

Endpoint Descriptor (for Status Change Endpoint)

<i>bLength</i>	7
<i>bDescriptorType</i>	ENDPOINT Descriptor type
<i>bEndpointAddress</i>	Implementation-dependent; Bit 7: Direction = In(1)
<i>bmAttributes</i>	Transfer Type = Interrupt (19)
<i>wMaxPacketSize</i>	2
<i>bInterval</i>	8 (maximum allowable interval)

Endpoint Companion Descriptor (for Status Change Endpoint)

<i>bLength</i>	6
<i>bDescriptorType</i>	SUPERSPEED_USB_ENDPOINT_COMPANION Descriptor type
<i>bMaxBurst</i>	0
<i>bmAttributes</i>	0
<i>wBytesPerInterval</i>	2

Hub Descriptors for USB hub operating at Gen 2 speed**Device Descriptor (SuperSpeedPlus information)**

<i>bLength</i>	18
<i>bDescriptorType</i>	DEVICE Descriptor type
<i>bcdUSB</i>	310H
<i>bDeviceClass</i>	HUB_CLASSCODE (9)
<i>bDeviceSubClass</i>	0
<i>bDeviceProtocol</i>	3
<i>bMaxPacketSize0</i>	9
<i>bNumConfigurations</i>	1

BOS Descriptor

<i>bLength</i>	5
<i>bDescriptorType</i>	BOS Descriptor type
<i>wTotalLength</i>	73
<i>bNumDeviceCaps</i>	3

USB 2.0 Extension

<i>bLength</i>	7
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	USB 2.0 EXTENSION
<i>bmAttributes</i>	2

SuperSpeed USB Device Capability

<i>bLength</i>	10
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	SUPERSPEED_USB
<i>bmAttributes</i>	Implementation-dependent
<i>wSpeedsSupported</i>	14
<i>bFunctionalitySupport</i>	1
<i>bU1DevExitLat</i>	Implementation-dependent
<i>wU2DevExitLat</i>	Implementation-dependent

SuperSpeedPlus USB Device Capability

<i>bLength</i>	28
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	SUPERSPEED_PLUS
<i>bReserved</i>	0
<i>bmAttributes</i>	13H
<i>wFunctionalitySupport</i>	1
<i>wReserved</i>	0
<i>bmSublinkSpeedAttr[0]</i>	00050030H
<i>bmSublinkSpeedAttr[1]</i>	000500B0H
<i>bmSublinkSpeedAttr[2]</i>	000A4031H
<i>bmSublinkSpeedAttr[3]</i>	000A40B1H

ContainerID

<i>bLength</i>	20
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	CONTAINER_ID
<i>bReserved</i>	0
<i>ContainerID</i>	Implementation-dependent

Precision Time Measurement

<i>bLength</i>	3
<i>bDescriptorType</i>	DEVICE CAPABILITY Descriptor type
<i>bDevCapabilityType</i>	PRECISION_TIME_MEASUREMENT

Configuration Descriptor (SuperSpeedPlus information)

<i>bLength</i>	9
<i>bDescriptorType</i>	CONFIGURATION Descriptor type
<i>wTotalLength</i>	31
<i>bNumInterfaces</i>	1
<i>bConfigurationValue</i>	X
<i>iConfiguration</i>	Y
<i>bmAttributes</i>	Z
<i>bMaxPower</i>	The maximum amount of bus power the hub will consume in this configuration

Interface Descriptor

<i>bLength</i>	9
<i>bDescriptorType</i>	INTERFACE Descriptor type
<i>bInterfaceNumber</i>	0
<i>bAlternateSetting</i>	0
<i>bNumEndpoints</i>	1
<i>bInterfaceClass</i>	HUB_CLASSCODE (9)
<i>bInterfaceSubClass</i>	0
<i>bInterfaceProtocol</i>	0
<i>iInterface</i>	I

Endpoint Descriptor (for Status Change Endpoint)

<i>bLength</i>	7
<i>bDescriptorType</i>	ENDPOINT Descriptor type
<i>bEndpointAddress</i>	Implementation-dependent; Bit 7: Direction = In(1)
<i>bmAttributes</i>	Transfer Type = Interrupt (19)
<i>wMaxPacketSize</i>	2
<i>bInterval</i>	8 (maximum allowable interval)

Endpoint Companion Descriptor (for Status Change Endpoint)

<i>bLength</i>	6
<i>bDescriptorType</i>	SUPER SPEED_USB_ENDPOINT_COMPANION Descriptor type
<i>bMaxBurst</i>	0
<i>bmAttributes</i>	0
<i>wBytesPerInterval</i>	2

10.15.2 Class-specific Descriptors

10.15.2.1 Hub Descriptor

Table 10-5 outlines the various fields contained in the hub descriptor.

Table 10-5 – Enhanced SuperSpeed Hub Descriptor

Offset	Field	Size	Description
0	<i>bDescLength</i>	1	Number of bytes in this descriptor, including this byte. (12 bytes)
1	<i>bDescriptorType</i>	1	Descriptor Type, value: 2AH for Enhanced SuperSpeed hub descriptor
2	<i>bNbrPorts</i>	1	Number of downstream facing ports that this hub supports. The maximum number of ports of ports a hub can support is 15.
3	<i>wHubCharacteristics</i>	2	D1...D0: Logical Power Switching Mode 00: Ganged power switching (all ports' power at once) 01: Individual port power switching 1X: Reserved D2: Identifies a Compound Device 0: Hub is not part of a compound device. 1: Hub is part of a compound device. D4...D3: Over-current Protection Mode 00: Global Over-current Protection. The hub reports over-current as a summation of all ports' current draw, without a breakdown of individual port over-current status. 01: Individual Port Over-current Protection. The hub reports over-current on a per-port basis. Each port has an over-current status. 1X: No Over-current Protection. This option is allowed only for bus-powered hubs that do not implement over-current protection. D15...D5: Reserved
5	<i>bPwrOn2PwrGood</i>	1	Time (in 2-ms intervals) from the time the power-on sequence begins on a port until power is good on that port. The USB system software uses this value to determine how long to wait before accessing a powered-on port. This value is set to zero if power-switching is not supported by the hub.
6	<i>bHubContrCurrent</i>	1	Maximum current requirements of the Hub Controller electronics when the hub is operating on both USB 2.0 and Enhanced SuperSpeed expressed in units of aCurrentUnit (i.e., 50 = 50 × aCurrentUnit mA). Note that the encoding of this field is different if the encoding used is the USB 2.0 specification for USB 2.0 hubs. A USB hub shall report the current requirements when it is only operating on USB 2.0 (not Enhanced SuperSpeed) in the USB 2.0 hub descriptor.

Offset	Field	Size	Description																										
7	<i>bHubHdrDecLat</i>	1	Hub Packet Header Decode Latency. Worst case latency for hubs whose upstream link is in U0 to decode the header of a downstream flowing TP or DP packet and initiate a transition to U0 on the relevant downstream port. The time is measured from receipt of the last symbol of the header packet by the upstream port until the hubs starts LFPS on the intended downstream port. This field is used to calculate the total path exit latency through a hub. The following are permissible values: <table><thead><tr><th><u>Value</u></th><th><u>Meaning</u></th></tr></thead><tbody><tr><td>00H</td><td>Much less than 0.1 μs.</td></tr><tr><td>01H</td><td>0.1 μs</td></tr><tr><td>02H</td><td>0.2 μs</td></tr><tr><td>03H</td><td>0.3 μs</td></tr><tr><td>04H</td><td>0.4 μs</td></tr><tr><td>05H</td><td>0.5 μs</td></tr><tr><td>06H</td><td>0.6 μs</td></tr><tr><td>07H</td><td>0.7 μs</td></tr><tr><td>08H</td><td>0.8 μs</td></tr><tr><td>09H</td><td>0.9 μs</td></tr><tr><td>0AH</td><td>1.0 μs</td></tr><tr><td>0BH – FFH</td><td>Reserved</td></tr></tbody></table>	<u>Value</u>	<u>Meaning</u>	00H	Much less than 0.1 μs.	01H	0.1 μs	02H	0.2 μs	03H	0.3 μs	04H	0.4 μs	05H	0.5 μs	06H	0.6 μs	07H	0.7 μs	08H	0.8 μs	09H	0.9 μs	0AH	1.0 μs	0BH – FFH	Reserved
<u>Value</u>	<u>Meaning</u>																												
00H	Much less than 0.1 μs.																												
01H	0.1 μs																												
02H	0.2 μs																												
03H	0.3 μs																												
04H	0.4 μs																												
05H	0.5 μs																												
06H	0.6 μs																												
07H	0.7 μs																												
08H	0.8 μs																												
09H	0.9 μs																												
0AH	1.0 μs																												
0BH – FFH	Reserved																												
8	<i>wHubDelay</i>	2	This field defines the maximum delay in nanoseconds a hub introduces while forwarding packets in either direction. Note that the maximum value a hub is allowed to report in this field is tHubDelay. See Section 10.19.																										
10	<i>DeviceRemovable</i>	2	Indicates if a port has a removable device attached. This field is reported on byte-granularity. Within a byte, if no port exists for a given location, the bit field representing the port characteristics shall be 0. Bit value definition: 0B – Device is removable. 1B – Device is non-removable. This is a bitmap corresponding to the individual ports on the hub: Bit 0: Reserved for future use Bit 1: Port 1 Bit 2: Port 2 Bit <i>n</i>: Port <i>n</i> (implementation-dependent, up to a maximum of 15 ports)																										

10.16 Requests

10.16.1 Standard Requests

Hubs have tighter constraints on request processing timing than specified in Section 9.2.6 for standard devices because they are crucial to the “time to availability” of all devices attached to the USB. The worst case request timing requirements are listed below (they apply to both Standard and Hub Class requests):

- Completion time for requests with no data stage: 50 ms

- Completion times for standard requests with data stage(s):
 - Time from setup packet to first data stage: 50 ms
 - Time between each subsequent data stage: 50 ms
 - Time between last data stage and status stage: 50 ms

Because hubs play such a crucial role in bus enumeration, it is recommended that hubs average response times be less than 5 ms for all requests.

Table 10-6 outlines the various standard device requests.

Table 10-6 – Hub Responses to Standard Device Requests

bRequest	Hub Response
clear_feature	Standard
GET_CONFIGURATION	Standard
GET_DESCRIPTOR	Standard
GET_INTERFACE	Undefined. Hubs are allowed to support only one interface.
GET_STATUS	Standard
SET_ADDRESS	Standard
SET_CONFIGURATION	Standard
SET_DESCRIPTOR	Optional
set_feature	Standard
SET_INTERFACE	Undefined. Hubs are allowed to support only one interface.
SET_ISOCH_DELAY	Standard
SET_SEL	Standard
synch_frame	Undefined. Hubs are not allowed to have isochronous endpoints.

A hub is required to accept all “Standard” requests without error. A hub shall not respond with a request error to a well-formed SET_ISOC_DELAY request. A hub is not required to retain or process the delay value. Optional requests that are not implemented shall return a STALL in the Data stage or Status stage of the request.

10.16.2 Class-specific Requests

The hub class defines requests to which hubs respond, as outlined in Table 10-7. Table 10-8 defines the hub class request codes. All requests in the table below except SetHubDescriptor() are mandatory.

Table 10-7 – Hub Class Requests

Request	bmRequestType	bRequest	wValue	wIndex	wLength	Data
ClearHubFeature	00100000B	CLEAR_FEATURE	Feature Selector	Zero	Zero	None
ClearPortFeature	00100011B	CLEAR_FEATURE	Feature Selector	Port	Zero	None
GetHubDescriptor	10100000B	GET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero or Language ID	Descriptor Length	Descriptor
GetHubStatus	10100000B	GET_STATUS	Zero	Zero	Four	Hub Status and Change Status
GetPortStatus	10100011B	GET_STATUS	Zero	Port	Four	Port Status and Change Status
GetPortErrorCount	10100011B	GET_PORT_ERR_COUNT	Zero	Port	Two	Number of Link Errors on this port
SetHubDescriptor	00100000B	SET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero or Language ID	Descriptor Length	Descriptor
SetHubFeature	00100000B	SET_FEATURE	Feature Selector	Zero	Zero	None
SetHubDepth	00100000B	SET_HUB_DEPTH	Hub Depth	Zero	Zero	None
SetPortFeature	00100011B	SET_FEATURE	Feature Selector	Selector, Timeout, Port	Zero	None

Table 10-8 – Hub Class Request Codes

bRequest	Value
get_status	0
clear_feature	1
Reserved (used in previous specifications for get_state)	2
set_feature	3
ReseRVED	4-5
get_descriptor	6
set_descriptor	7
ReseRVED (used in USB 2.0 specification)	8-11
SET_HUB_DEPTH	12
GET_PORT_ERR_COUNT	13

Table 10-9 gives the valid feature selectors for the hub class. Refer to Section 10.16.2.1 and Section 10.16.2.8 for a description of the features.

Table 10-9 – Hub Class Feature Selectors

Feature Selector	Recipient	Value
c_hub_local_power	Hub	0
c_hub_over_current	Hub	1
port_connection	Port	0
port_over_current	Port	3
port_reset	Port	4
PORT_LINK_STATE	Port	5
port_power	Port	8
c_port_connection	Port	16
c_port_over_current	Port	19
c_port_reset	Port	20
ReseRVED (used in USB 2.0 specification)	Port	21
PORT_U1_TIMEOUT	Port	23
PORT_U2_TIMEOUT	Port	24
C_PORT_LINK_STATE	Port	25
C_PORT_CONFIG_ERROR	Port	26
PORT_REMOTE_WAKE_MASK	Port	27
BH_PORT_RESET	Port	28
C_BH_PORT_RESET	Port	29
FORCE_LINKPM_ACCEPT	Port	30

10.16.2.1 Clear Hub Feature

This request resets a value reported in the hub status.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100000B	CLEAR_FEATURE	Feature Selector	Zero	Zero	None

Clearing a feature disables that feature; refer to Table 10-9 for the feature selector definitions that apply to the hub as a recipient. If the feature selector is associated with a status change, clearing that status change acknowledges the change. This request format is used to clear either the C_HUB_LOCAL_POWER or C_HUB_OVER_CURRENT features.

It is a Request Error if *wValue* is not a feature selector listed in Table 10-9 or if *wIndex* or *wLength* are not as specified above.

If the hub is not configured, the hub's response to this request is undefined.

10.16.2.2 Clear Port Feature

This request resets a value reported in the port status.

bmRequestType	bRequest	wValue	wIndex		wLength	Data
00100011B	CLEAR_FEATURE	Feature Selector	Selector	Port	Zero	None

The port number shall be a valid port number for that hub, greater than zero. The port field is located in bits 7..0 of the *wIndex* field.

Clearing a feature disables that feature or starts a process associated with the feature; refer to Table 10-9 for the feature selector definitions. If the feature selector is associated with a status change, clearing that status change acknowledges the change. This request format is used to clear the following features:

- PORT_POWER
- C_PORT_CONNECTION
- C_PORT_RESET
- C_PORT_OVER_CURRENT
- C_PORT_LINK_STATE
- C_PORT_CONFIG_ERROR
- C_BH_PORT_RESET
- FORCE_LINKPM_ACCEPT

Clearing the PORT_POWER feature causes the port to be placed in the DSPORT.Powered-off-reset state and may, subject to the constraints due to the hub's method of power switching, result in power being removed from the port. When in the DSPORT.Powered-off or the DSPORT.Powered-off-detect or the DSPORT.Powered-off-reset state, the only requests that are valid when this port is the recipient are Get Port Status (refer to Section 10.16.2.6) and Set Port Feature (PORT_POWER) (refer to Section 10.16.2.10).

Clearing the FORCE_LINKPM_ACCEPT feature causes the port to de-assert the Force_LinkPM_Accept bit in Set Link Function LMPs. If the Force_LinkPM_Accept bit is not asserted on the port, the hub shall treat this request as a functional no-operation.

It is a Request Error if *wValue* is not a feature selector listed in Table 10-9, if *wIndex* specifies a port that does not exist, or if *wLength* is not as specified above. It is not an error for this request to try to clear a feature that is already cleared (the hub shall treat this as a functional no-operation).

If the hub is not configured, the hub's response to this request is undefined.

10.16.2.3 Get Hub Descriptor

This request returns the hub descriptor.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10100000B	GET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero	Descriptor Length	Descriptor

The GetDescriptor() request for the hub class descriptor follows the same usage model as that of the standard GetDescriptor() request (refer to Chapter 9). The standard hub descriptor is denoted by using the value *bDescriptorType* defined in Section 10.15.2.1. All hubs are required to implement one hub descriptor, with descriptor index zero.

If *wLength* is larger than the actual length of the descriptor, then only the actual length is returned. If *wLength* is less than the actual length of the descriptor, then only the first *wLength* bytes of the descriptor are returned; this is not considered an error even if *wLength* is zero.

It is a Request Error if *wValue* or *wIndex* are other than as specified above.

If the hub is not configured, the hub's response to this request is undefined.

10.16.2.4 Get Hub Status

This request returns the current hub status and the states that have changed since the previous acknowledgment.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10100000B	GET_STATUS	Zero	Zero	Four	Hub Status and Change Status

The first word of data contains the *wHubStatus* field (refer to Table 10-10). The second word of data contains the *wHubChange* field (refer to Table 10-11).

It is a Request Error if *wValue*, *wIndex*, or *wLength* are other than as specified above.

If the hub is not configured, the hub's response to this request is undefined.

Table 10-10 – Hub Status Field, *wHubStatus*

Bit	Description
0	Local Power Source: This is the source of the local power supply. This field indicates whether hub power (for other than the SIE) is being provided by an external source or from the USB. This field allows the USB system software to determine the amount of power available from a hub to downstream devices. 0 = Local power supply good 1 = Local power supply lost (inactive)
1	Over-current: If the hub supports over-current reporting on a hub basis, this field indicates that the sum of all the ports' current has exceeded the specified maximum and all ports have been placed in the Powered-off-reset state. If the hub reports over-current on a per-port basis or has no over-current detection capabilities, this field is always zero. The hub shall only report over-current if it is physically unable to meet the sum of all ports' current draws. For more details on over-current protection, refer to the USB 2.0 Specification, Section 7.2.1.2.1. 0 = No over-current condition currently exists. 1 = A hub over-current condition exists.
2-15	Reserved

There are no defined feature selector values for these status bits and they can neither be set nor cleared by the USB system software.

Table 10-11 – Hub Change Field, *wHubChange*

Bit	Description
0	Local Power Status Change (C_HUB_LOCAL_POWER): This field indicates that a change has occurred in the hub's Local Power Source field in <i>wHubStatus</i> . This field is initialized to zero when the hub receives a bus reset. 0 = No change has occurred to Local Power Status. 1 = Local Power Status has changed.
1	Over-Current Change (C_HUB_OVER_CURRENT): This field indicates if a change has occurred in the Over-Current field in <i>wHubStatus</i> . This field is initialized to zero when the hub receives a bus reset. 0 = No change has occurred to the Over-Current Status. 1 = Over-Current Status has changed.
2-15	Reserved

Hubs may allow setting of these change bits with SetHubFeature() requests in order to support diagnostics. If the hub does not support setting of these bits, it shall either treat the SetHubFeature() request as a Request Error or as a functional no-operation. When set, these bits may be cleared by a ClearHubFeature() request. A request to set a feature that is already set or to clear a feature that is already clear has no effect and the hub shall treat this as a functional no-operation.

10.16.2.5 Get Port Error Count

This request returns the number of link errors detected by the hub on the port indicated by *wIndex*. This value is reset to zero whenever the device goes through a Reset (refer to Section 7.3) or at power up.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10100011B	GET_PORT_ERR_COUNT	Zero	Port	Two	Number of Link Errors

The port number shall be a valid port number for that hub, greater than zero.

It is a Request Error if *wValue* or *wLength* are other than as specified above or if *wIndex* specifies a port that does not exist.

If the hub is not configured, the behavior of the hub in response to this request is undefined.

10.16.2.6 Get Port Status

This request returns the current port status and the current value of the port status change bits.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10100011B	GET_STATUS	Port Status Type	Port	Port Status Length	Port Status and Change Status

The port number shall be a valid port number for that hub, greater than zero.

The first word of PORT_STATUS or EXT_PORT_STATUS data contains the *wPortStatus* field (refer to Table 10-13). The second word of PORT_STATUS or EXT_PORT_STATUS data contains the *wPortChange* field (refer to Table 10-14). An EXT_PORT_STATUS request shall return an additional dword of data that contains *dwExtPortStatus* field (refer to Table 10-15).

The bit locations in the *wPortStatus* and *wPortChange* fields correspond in a one-to-one fashion where applicable.

The *wValue* field specifies the Port Status Type in the low order byte (refer to Table 10-11 and the high order byte is reserved).

Table 10-12 – Port Status Type Codes

Port Status Type	Value (wValue, low-byte)	Port Status Length (wLength)	Description
PORT_STATUS	00H	4	Returns Port Status Request information
PD_STATUS ⁶	01H	8	PD Status of the specified port on a PDUSB Hub.
EXT_PORT_STATUS	02H	8	Returns Extended Port Status Request information
Reserved	03-FFH	NA	Reserved for future use

It is a Request Error if the Port Status Type equals EXT_PORT_STATUS and the hub that does not define a SuperSpeedPlus USB Capability descriptor, or if the Port Status Type equals a reserved value, or if *wValue* or *wLength* are other than as specified in Table 10-7, or if *wIndex* specifies a port that does not exist.

If the hub is not configured, the behavior of the hub in response to this request is undefined.

⁶ Refer to the USB Power Delivery Specification Revision 1.0

10.16.2.6.1 Port Status Bits

Table 10-13 – Port Status Field, *wPortStatus*

Bit	Description																												
0	Current Connect Status (PORT_CONNECTION): This field reflects whether or not a device is currently connected to this port. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No device is present</td></tr> <tr> <td>1</td><td>A device is present on this port</td></tr> </table>	Value	Meaning	0	No device is present	1	A device is present on this port																						
Value	Meaning																												
0	No device is present																												
1	A device is present on this port																												
1	Port Enabled/Disabled: This field indicates whether the port is enabled. Ports can be disabled by either a fault condition (disconnect event or other fault condition) or by the USB system software. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>Port is disabled</td></tr> <tr> <td>1</td><td>Port is enabled</td></tr> </table>	Value	Meaning	0	Port is disabled	1	Port is enabled																						
Value	Meaning																												
0	Port is disabled																												
1	Port is enabled																												
2	Reserved																												
3	Over-current (PORT_OVER_CURRENT): If the hub reports over-current conditions on a per-port basis, this field indicates that the current drain on the port exceeds the specified maximum. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No over-current condition exists on this port</td></tr> <tr> <td>1</td><td>An over-current condition exists on this port</td></tr> </table>	Value	Meaning	0	No over-current condition exists on this port	1	An over-current condition exists on this port																						
Value	Meaning																												
0	No over-current condition exists on this port																												
1	An over-current condition exists on this port																												
4	Reset (PORT_RESET): This field is set when the host wishes to reset the attached device. It remains set until the reset signaling is turned off by the hub. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>Reset signaling not asserted</td></tr> <tr> <td>1</td><td>Reset signaling asserted</td></tr> </table>	Value	Meaning	0	Reset signaling not asserted	1	Reset signaling asserted																						
Value	Meaning																												
0	Reset signaling not asserted																												
1	Reset signaling asserted																												
5-8	Port Link State (PORT_LINK_STATE): This field reflects the current state of the link attached to this port. A new state is not reflected until the link state transition to that state is complete. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0x00</td><td>Link is in the U0 State</td></tr> <tr> <td>0x01</td><td>Link is in the U1 State</td></tr> <tr> <td>0x02</td><td>Link is in the U2 State</td></tr> <tr> <td>0x03</td><td>Link is in the U3 State</td></tr> <tr> <td>0x04</td><td>Link is in the eSS.Disabled State</td></tr> <tr> <td>0x05</td><td>Link is in the Rx.Detect State</td></tr> <tr> <td>0x06</td><td>Link is in the eSS.Inactive State</td></tr> <tr> <td>0x07</td><td>Link is in the Polling State</td></tr> <tr> <td>0x08</td><td>Link is in the Recovery State</td></tr> <tr> <td>0x09</td><td>Link is in the Hot Reset State</td></tr> <tr> <td>0xA</td><td>Link is in the Compliance Mode State</td></tr> <tr> <td>0xB</td><td>Link is in the Loopback State</td></tr> <tr> <td>0xC-0xF</td><td>Reserved</td></tr> </table>	Value	Meaning	0x00	Link is in the U0 State	0x01	Link is in the U1 State	0x02	Link is in the U2 State	0x03	Link is in the U3 State	0x04	Link is in the eSS.Disabled State	0x05	Link is in the Rx.Detect State	0x06	Link is in the eSS.Inactive State	0x07	Link is in the Polling State	0x08	Link is in the Recovery State	0x09	Link is in the Hot Reset State	0xA	Link is in the Compliance Mode State	0xB	Link is in the Loopback State	0xC-0xF	Reserved
Value	Meaning																												
0x00	Link is in the U0 State																												
0x01	Link is in the U1 State																												
0x02	Link is in the U2 State																												
0x03	Link is in the U3 State																												
0x04	Link is in the eSS.Disabled State																												
0x05	Link is in the Rx.Detect State																												
0x06	Link is in the eSS.Inactive State																												
0x07	Link is in the Polling State																												
0x08	Link is in the Recovery State																												
0x09	Link is in the Hot Reset State																												
0xA	Link is in the Compliance Mode State																												
0xB	Link is in the Loopback State																												
0xC-0xF	Reserved																												

Bit	Description																		
9	Port Power (PORT_POWER): This field reflects a port's logical, power control state. Because hubs can implement different methods of port power switching, this field may or may not represent whether power is applied to the port. The device descriptor reports the type of power switching implemented by the hub. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>This port is in the Powered-off state</td></tr> <tr> <td>1</td><td>This port is not in the Powered-off state</td></tr> </table>	Value	Meaning	0	This port is in the Powered-off state	1	This port is not in the Powered-off state												
Value	Meaning																		
0	This port is in the Powered-off state																		
1	This port is not in the Powered-off state																		
10-12	Negotiated speed of the Enhanced SuperSpeed Device Attached to this port (PORT_SPEED): This field is valid only if an Enhanced SuperSpeed device is attached. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>Enhanced SuperSpeed</td></tr> <tr> <td>1</td><td>Reserved</td></tr> <tr> <td>2</td><td>Reserved</td></tr> <tr> <td>3</td><td>Reserved</td></tr> <tr> <td>4</td><td>Reserved</td></tr> <tr> <td>5</td><td>Reserved</td></tr> <tr> <td>6</td><td>Reserved</td></tr> <tr> <td>7</td><td>Reserved</td></tr> </table>	Value	Meaning	0	Enhanced SuperSpeed	1	Reserved	2	Reserved	3	Reserved	4	Reserved	5	Reserved	6	Reserved	7	Reserved
Value	Meaning																		
0	Enhanced SuperSpeed																		
1	Reserved																		
2	Reserved																		
3	Reserved																		
4	Reserved																		
5	Reserved																		
6	Reserved																		
7	Reserved																		
13-15	Reserved																		

10.16.2.6.1.1 PORT_CONNECTION

This bit is set to one when the port is in the DSPORT.Enabled state. In DSPORT.Resetting or DSPORT.Error state it maintains the value from prior state.

SetPortFeature(PORT_CONNECTION) and ClearPortFeature(PORT_CONNECTION) requests shall not be used by the USB system software and shall be treated as no-operation requests by hubs.

10.16.2.6.1.2 PORT_ENABLE

This bit is set to one when the downstream port is in the DSPORT.Enabled state and is set to zero otherwise.

Note that the USB 2.0 ClearPortFeature (PORT_ENABLE) request is not supported by Enhanced SuperSpeed hubs and cannot be used by USB system software to disable a port.

10.16.2.6.1.3 PORT_OVER_CURRENT

This bit is set to one while an over-current condition exists on the port and set to zero otherwise.

If the voltage on this port is affected by an over-current condition on another port, this bit is set to one and remains set to one until the over-current condition on the affecting port is removed. When the over-current condition on the affecting port is removed, this bit is set to zero.

Over-current protection is required on self-powered hubs (it is optional on bus-powered hubs) as outlined in Section 10.12.

The SetPortFeature(PORT_OVER_CURRENT) and ClearPortFeature(PORT_OVER_CURRENT) requests shall not be used by the USB system software and may be treated as no-operation requests by hubs.

10.16.2.6.1.4 PORT_RESET

This bit is set to one while the port is in the DSPORT.Resetting state. This bit is set to zero in all other downstream port states.

A SetPortFeature(PORT_RESET or BH_PORT_RESET) request will initiate the DSPORT.Resetting state if the conditions in Section 10.3.1.6 are met.

The ClearPortFeature(PORT_RESET) request shall not be used by the USB system software and may be treated as a no-operation request by hubs.

10.16.2.6.1.5 PORT_LINK_STATE

This field reflects the current state of the link.

The SetPortFeature(PORT_LINK_STATE) request may be issued by the USB system software at any time but will have an effect only as specified in Section 10.16.2.10.

The ClearPortFeature(PORT_LINK_STATE) requests shall not be used by the USB System software and may be treated as no-operation requests by hubs.

10.16.2.6.1.6 PORT_POWER

This bit reflects the current logical power state of a port. This bit is implemented on all ports whether or not actual port power switching devices are present.

While this bit is zero, the port is in the DSPORT.Powered-off state, the DSPORT.Powered-off-detect state, or the DSPORT.Powered-off-reset state. Similarly, anything that causes this port to go to any of these three states will cause this bit to be set to zero.

A SetPortFeature(PORT_POWER) will set this bit to one unless both C_HUB_LOCAL_POWER and Local Power Status (in *wHubStatus*) are set to one in which case the request is treated as a functional no-operation.

10.16.2.6.1.7 PORT_SPEED

This value in this field is only valid when the PORT_ENABLE bit is set to one and the Port Status Type is set to PORT_STATUS. A value of zero in this field indicates that an Enhanced SuperSpeed device is attached. All other values in this field are reserved.

System Software can determine the actual speed at which the device is operating by using the Get Port Status request with the Port Status Type set to EXT_PORT_STATUS (see Section 10.16.2.6.3).

This field can only be read by USB system software.

10.16.2.6.2 Port Status Change Bits

Port status change bits are used to indicate changes in port status bits that are not the direct result of requests. Port status change bits can be cleared with a ClearPortFeature() request or by a hub reset. Hubs may allow setting of the status change bits with a SetPortFeature() request for diagnostic purposes. If a hub does not support setting of the status change bits, it may either treat the request as a Request Error or as a functional no-operation. Table 10-14 describes the various bits in the *wPortChange* field.

Table 10-14 – Port Change Field, *wPortChange*

Bit	Description						
0	Connect Status Change (C_PORT_CONNECTION): Indicates a change has occurred in the port's Current Connect Status. The hub device sets this field as described in Section0. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No change has occurred to Current Connect status</td></tr> <tr> <td>1</td><td>Current Connect status has changed</td></tr> </table>	Value	Meaning	0	No change has occurred to Current Connect status	1	Current Connect status has changed
Value	Meaning						
0	No change has occurred to Current Connect status						
1	Current Connect status has changed						
1-2	Reserved						
3	Over-Current Indicator Change (C_PORT_OVER_CURRENT): This field applies only to hubs that report over-current conditions on a per-port basis (as reported in the hub descriptor). <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No change has occurred to Over-Current Indicator</td></tr> <tr> <td>1</td><td>Over-Current Indicator has changed</td></tr> </table> If the hub does not report over-current on a per-port basis, then this field is always zero.	Value	Meaning	0	No change has occurred to Over-Current Indicator	1	Over-Current Indicator has changed
Value	Meaning						
0	No change has occurred to Over-Current Indicator						
1	Over-Current Indicator has changed						
4	Reset Change (C_PORT_RESET): This field is set when reset processing for any type of reset on this port is complete. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No change</td></tr> <tr> <td>1</td><td>Reset complete</td></tr> </table>	Value	Meaning	0	No change	1	Reset complete
Value	Meaning						
0	No change						
1	Reset complete						
5	BH Reset Change (C_BH_PORT_RESET): This field is set when a warm reset processing on this port is complete <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No change</td></tr> <tr> <td>1</td><td>Reset complete</td></tr> </table>	Value	Meaning	0	No change	1	Reset complete
Value	Meaning						
0	No change						
1	Reset complete						
6	Port Link State Change (C_PORT_LINK_STATE): This field is set when the port link status has changed as described below. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>No change</td></tr> <tr> <td>1</td><td>Link Status has changed</td></tr> </table>	Value	Meaning	0	No change	1	Link Status has changed
Value	Meaning						
0	No change						
1	Link Status has changed						
7	Port Config Error (C_PORT_CONFIG_ERROR): This field is set when the port fails to configure its link partner. <table> <tr> <th>Value</th><th>Meaning</th></tr> <tr> <td>0</td><td>Port Link Configuration was successful</td></tr> <tr> <td>1</td><td>Port Link Configuration was unsuccessful</td></tr> </table>	Value	Meaning	0	Port Link Configuration was successful	1	Port Link Configuration was unsuccessful
Value	Meaning						
0	Port Link Configuration was successful						
1	Port Link Configuration was unsuccessful						
8-15	Reserved						

10.16.2.6.2.1 C_PORT_CONNECTION

This bit is set to one when the PORT_CONNECTION bit changes.

This bit shall be set to zero by a ClearPortFeature(C_PORT_CONNECTION) request or while logical port power is off.

10.16.2.6.2.2 C_PORT_OVER_CURRENT

This bit is set to one when the PORT_OVER_CURRENT bit changes from zero to one or from one to zero. This bit is also set if the port is placed in the DSPORT.Powered-off-reset state due to an over-current condition on another port.

This bit shall be set to zero by a ClearPortFeature(C_PORT_OVER_CURRENT) request.

10.16.2.6.2.3 C_PORT_RESET

This bit is set to one when the port transitions from the DSPORT.Resetting state to the DSPORT.Enabled state for any type of reset.

This bit shall be set to zero by a ClearPortFeature(C_PORT_RESET) request, or while logical port power is off.

10.16.2.6.2.4 C_PORT_BH_RESET

This bit is set to one when the port transitions from the DSPORT.Resetting state to the DSPORT.Enabled state for a Warm Reset only.

This bit shall be cleared by a ClearPortFeature(C_PORT_BH_RESET) request, or while logical port power is off.

10.16.2.6.2.5 C_PORT_LINK_STATE

This bit is set to one when the port's link completes a transition from the U3 state to the U0 state as a result of a SetPortFeature(Port_Link_State) request or completes a transition to Loopback state or to Compliance or to eSS.Inactive with Rx terminations present. This bit is not set to one due to transitions from U3 to U0 as a result of remote wakeup signaling received on a downstream facing port.

This bit will be cleared by a ClearPortFeature(C_PORT_LINK_STATE) request, or while logical port power is off.

10.16.2.6.2.6 C_PORT_CONFIG_ERROR

This bit is set to one if the link connected to the port could not be successfully configured, e.g., if two downstream only capable ports are connected to each other or if the link configuration could not be completed. In addition, the port shall transition to the DSPORT.Error state when this occurs.

This bit will be cleared by a ClearPortFeature(C_PORT_CONFIG_ERROR) request, or while logical port power is off.

10.16.2.6.3 Extended Port Status Bits

The extended port status bits are returned only if the Port Status Type of a Get Port Status request is set to EXT_PORT_STATUS.

Note that for Enhanced SuperSpeed devices the "Port Speed" is the Link Speed multiplied by Lane Count.

Table 10-15 – Extended Port Status Field, *dwExtPortStatus*

Bit	Description
0-3	Rx Sublink Speed ID (RX_SUBLINK_SPEED_ID): Indicates the negotiated Rx Sublink speed of the device attached to this port. This field is valid only if an Enhanced SuperSpeed device is attached.
4-7	Tx Sublink Speed ID (TX_SUBLINK_SPEED_ID): Indicates the negotiated Tx Sublink speed of the device attached to this port. This field is valid only if an Enhanced SuperSpeed device is attached.
8-11	Rx Lane Count (RX_LANE_COUNT): Zero based value of the negotiated number of Rx lanes of the device attached to this port. This field is valid only if an Enhanced SuperSpeed device is attached.
12-15	Tx Lane Count (TX_LANE_COUNT): Zero based value of the negotiated number of Tx lanes of the device attached to this port. This field is valid only if an Enhanced SuperSpeed device is attached.
12-31	Reserved

10.16.2.6.3.1 TX_SUBLINK_SPEED_ID and RX_SUBLINK_SPEED_ID

The value in this field is only valid when the PORT_ENABLE bit is set to one. The Lane Speed (i.e. bit rate of a single lane) is determined by evaluating the parameters of the *Sublink Speed Attribute* in the SuperSpeedPlus USB Capability descriptor whose *Sublink Speed Attribute ID* value matches the Sublink Speed ID value, e.g. if the Sublink Speed Attribute LSE and LSM fields equal 3 and 10, respectively, then the link is operating at 10 Gb/s. All values not referenced by a *Sublink Speed Attribute* are reserved.

This field can only be read by USB system software.

10.16.2.6.3.2 TX_LANE_COUNT and RX_LANE_COUNT

This value in this field is only valid when the PORT_ENABLE bit is set to one. The speed of a port is determined by multiplying the Sublink Speed (as defined by the SUBLINK_SPEED_ID) by the Lane Count.

This field can only be read by USB system software.

10.16.2.7 Set Hub Descriptor

This request overwrites the hub descriptor.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100000B	SET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero	Descriptor Length	Descriptor

The SetDescriptor request for the hub class descriptor follows the same usage model as that of the standard SetDescriptor request (refer to the framework chapter). The standard hub descriptor is denoted by using the value *bDescriptorType* defined in Section 10.15.2.1. All hubs are required to implement one hub descriptor with descriptor index zero.

This request is optional. This request writes data to a class-specific descriptor. The host provides the data that is to be transferred to the hub during the data transfer stage of the control transaction. This request writes the entire hub descriptor at once.

Hubs shall buffer all the bytes received from this request to ensure that the entire descriptor has been successfully transmitted from the host. Upon successful completion of the bus transfer, the hub updates the contents of the specified descriptor.

It is a Request Error if *wIndex* is not zero or if *wLength* does not match the amount of data sent by the host. Hubs that do not support this request respond with a STALL during the Data stage of the request.

If the hub is not configured, the hub's response to this request is undefined.

10.16.2.8 Set Hub Feature

This request sets a value reported in the hub status.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100000B	SET_FEATURE	Feature Selector	Zero	Zero	None

Setting a feature enables that feature; refer to Table 10-9 for the feature selector definitions that apply to the hub as recipient. Status changes may not be acknowledged using this request.

It is a Request Error if *wValue* is not a feature selector listed in Table 10-9 or if *wIndex* or *wLength* are not as specified above.

If the hub is not configured, the hub's response to this request is undefined.

10.16.2.9 Set Hub Depth

This request sets the value that the hub uses to determine the index into the Route String Index for the hub.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00100000B	SET_HUB_DEPTH	Hub Depth	Zero	Zero	None

wValue has the value of the Hub Depth. The Hub Depth left shifted by two is the offset into the Route String that identifies the lsb of the Route String Port Field for the hub.

It is a Request Error if *wValue* is greater than 4 or if *wIndex* or *wLength* are not as specified above.

If the hub is not configured, the hub's response to this request is undefined.

10.16.2.10 Set Port Feature

This request sets a value reported in the port status.

bmRequestType	bRequest	wValue	wIndex		wLength	Data
00100011B	SET_FEATURE	Feature Selector	Selector or Timeout Value or Remote Wake Mask	Port	Zero	None

The port number shall be a valid port number for that hub, greater than zero. The port number is in the least significant byte (bits 7..0) of the *wIndex* field. The most significant byte of *wIndex* is zero, except when the feature selector is PORT_U1_TIMEOUT or PORT_U2_TIMEOUT or PORT_LINK_STATE or PORT_REMOTE_WAKE_MASK.

Setting a feature enables that feature or starts a process associated with that feature; see Table 10-9 for the feature selector definitions that apply to a port as a recipient. Status change may not be acknowledged using this request. Features that can be set with this request are:

- PORT_RESET
- BH_PORT_RESET
- PORT_POWER
- PORT_U1_TIMEOUT
- PORT_U2_TIMEOUT
- PORT_LINK_STATE
- PORT_REMOTE_WAKE_MASK
- FORCE_LINKPM_ACCEPT

When the feature selector is PORT_U1_TIMEOUT, the most significant byte (bits 15..8) of the *wIndex* field specifies the Timeout value for the U1 inactivity timer. Refer to Section 10.4.2.1 for a detailed description of how the U1 inactivity timer value is used.

The following are permissible values:

Table 10-16 – U1 Timeout Value Encoding

Value	Description
00H	Zero (Default)
01H	1 μ s
02H	2 μ s
03H	3 μ s
...	...
7FH	127 μ s
80H-FEH	Reserved
FFH	Infinite

When the feature selector is PORT_U2_TIMEOUT, the most significant byte (bits 15..8) of the *wIndex* field specifies the Timeout value for the U2 inactivity timer. The port's link shall send an LMP to its link partner with the specified timeout value after receiving a Set Port Feature request with the PORT_U2_TIMEOUT feature selector. Refer to Section 10.4.2.1 for a detailed description of how the U2 inactivity timer value is used.

The following are permissible values:

Table 10-17 – U2 Timeout Value Encoding

Value	Description
00H	Zero (Default)
01H	256 μ s
02H	512 μ s
03H	768 μ s
...	...
FEH	65.024 ms
FFH	Infinite

NOTE It is the responsibility of software to properly set the U2 timeout for a downstream port that is connected to a hub. Inconsistent link states could result if the timeout is not set properly. It is recommended that software should set the upstream U2 timeout to at least twice the value of the U2 timeout of the downstream ports on the hub.

When the feature selector is `PORT_LINK_STATE`, the most significant byte (bits 15..8) of the *wIndex* field specifies the U state the host software wants to put the link connected to the port into. This request is only valid when the `PORT_ENABLE` bit is set and the `PORT_LINK_STATE` is not set to `eSS.Disabled`, `Rx.Detect` or `eSS.Inactive` except as noted below:

- If the value is 0, then the hub shall transition the link to U0 from any of the U states.
- If the value is 1, then host software wants to transition the link to the U1 State. The hub shall attempt to transition the link to U1 from U0. If the link is in any state other than U0 when a request is received with a value of 1, the behavior is undefined.
- If the value is 2, then the host software wants to transition the link to the U2 State. The hub shall attempt to transition the link to U2 from U0. If the link is in any state other than U0 when a request is received with a value of 2, the behavior is undefined.
- If the value is 3, then host software wants to selectively suspend the device connected to this port. The hub shall transition the link to U3 from any of the other U states using allowed link state transitions. If the port is not already in the U0 state, then it shall transition the port to the U0 state and then initiate the transition to U3. While this state is active, the hub does not propagate downstream-directed traffic to this port, but the hub will respond to resume signaling from the port.
- If the value is 4 (`eSS.Disabled`), the hub shall transition the link to `eSS.Disabled`. The request is valid at all times when the value is 4. The downstream port shall transition to the `DSPORT.Disabled` state after this request is received.
- If the value is 5 (`Rx.Detect`), the hub shall transition the link to `Rx.Detect`. This request is only valid when the downstream port is in the `DSPORT.Disabled` state. If the link is in any other state when a request is received with this value, the behavior is undefined. The downstream port shall transition to the `DSPORT.Disconnected` state after this request is received.
- If the value is 10 (Enable Compliance Mode), the hub shall enable entry into Compliance Mode for the next attach. This request is valid only when the downstream port is in the `DSPORT.Disconnected` state. If the link is in any other state when a request is received with this value, the behavior is undefined. Entry into Compliance Mode is disabled once the link enters Compliance Mode or Polling.LFPS succeeds.
- The hub shall respond with a Request Error if it sees any other value in the upper byte of the *wIndex* field.

When the feature selector is `PORT_REMOTE_WAKE_MASK`, the most significant byte (bits 15..8) of the *wIndex* field specifies the conditions that would cause the hub to signal a remote wake event on its upstream port. The encoding for the port remote wake mask is given below:

Table 10-18 – Downstream Port Remote Wake Mask Encoding

Bit	Description
0	Conn_RWEnable <u>Value</u> <u>Meaning</u> 0 The hub is disabled from signaling a remote wakeup due to a connect event on this port; connect events that occur during suspend must still be detected and reported after the resume process has completed (due to some other event) as a C_PORT_CONNECTION port status change. 1 The hub is enabled to signal a remote wakeup due to a connect event on the port and if Function Remote Wake is also enabled.
1	Disconn_RWEnable <u>Value</u> <u>Meaning</u> 0 The hub is disabled from signaling a remote wakeup due to a disconnect event on this port; disconnect events that occur during suspend must still be detected and reported after the resume process has completed (due to some other event) as a C_PORT_CONNECTION port status change. 1 The hub is enabled to signal a remote wakeup due to a disconnect event on the port and if Function Remote Wake is also enabled.
2	OC_RWEnable <u>Value</u> <u>Meaning</u> 0 The hub is disabled from signaling a remote wakeup due to an over-current event on this port; over-current events that occur during suspend must still be detected and reported after the resume process has completed (due to some other event) as a C_PORT_OVER_CURRENT port status change. Note that a hub that does not support per-port over current detection/reporting will signal remote-wakeup for an over-current event unless all ports have OC-RWEnable set to 0. 1 The hub is enabled to signal a remote wakeup due to an over-current event on the port and if Function Remote Wake is also enabled.
3-7	These bits are reserved and must be set to zero.

Note that after power on or after the hub is reset, the remote wake mask is set to zero (i.e., the mask is enabled).

The hub shall meet the following requirements:

- If the port is in the Powered-off state, the hub shall treat a SetPortFeature(PORT_RESET) request as a functional no-operation.
- If the port is not in the Enabled state, the hub shall treat a SetPortFeature(PORT_LINK_STATE) U3 request as a functional no-operation.
- If the port is not in the Powered-off state, the hub shall treat a SetPortFeature(PORT_POWER) request as a functional no-operation.
- If the port is not in the Enabled state, the hub shall treat a SetPortFeature(FORCE_LINKPM_ACCEPT) request as a functional no-operation.

When the feature selector is BH_PORT_RESET, the hub shall initiate a warm reset (refer to Section 7.4.2) on the port that is identified by this command. The state of the port after this reset shall be the same as the state after a SetPortFeature(PORT_RESET). On completion of a BH_PORT_RESET, the hub shall set the C_BH_PORT_RESET field to one in the PortStatus for this port.

It is a Request Error if *wValue* is not a feature selector listed in Table 10-9, if *wIndex* specifies a port that does not exist, or if *wLength* is not as specified above.

If the hub is not configured, the hub's response to this request is undefined.

10.17 Host Root (Downstream) Ports

The root ports of a USB host have similar functional requirements to the downstream ports of a USB hub. This section summarizes which requirements also apply to the root port of a host and identifies any additional or different requirements.

A host root port shall follow the requirements for a downstream facing hub port in Section 10.2 except for Section 10.2.3.

A host root port shall follow the requirements for a downstream facing hub port in Section 10.3 with the following exceptions and additions:

- None of the transitions and/or transition conditions based on the state of the hub upstream port apply to a root port.
- A host shall have control mechanisms in the host interface that allow software to achieve equivalent behavior to hub downstream port behavior in response to SetPortFeature or ClearPortFeature requests documented in Section 10.3.
- A host shall implement port status bits consistent with the downstream port state descriptions in Section 10.3.
- A host is required to provide a mechanism to correlate each USB 2.0 port with any Enhanced SuperSpeed port that shares the same physical connector. Note that this is similar to the requirement for USB hubs in Section 10.3.3.

A host root port shall follow the requirements for a downstream facing hub port in Section 10.4 with the same general exceptions already noted in this section.

A host shall implement port status bits through the host interface that are equivalent to all port status bit definitions in this chapter.

A host shall have mechanisms to achieve equivalent control over its root ports as provided by the SetPortFeature, ClearPortFeature, and GetPortStatus requests documented in this chapter.

10.18 Peripheral Device Upstream Ports

The upstream port of a USB peripheral device has similar functional requirements to the upstream port of a USB hub. This section summarizes which requirements also apply to the upstream port of a peripheral device and identifies any additional or different requirements.

10.18.1 Peripheral Device Upstream Ports

A peripheral device shall follow the requirements for an upstream facing hub port in Section 10.5 with the following exceptions and additions:

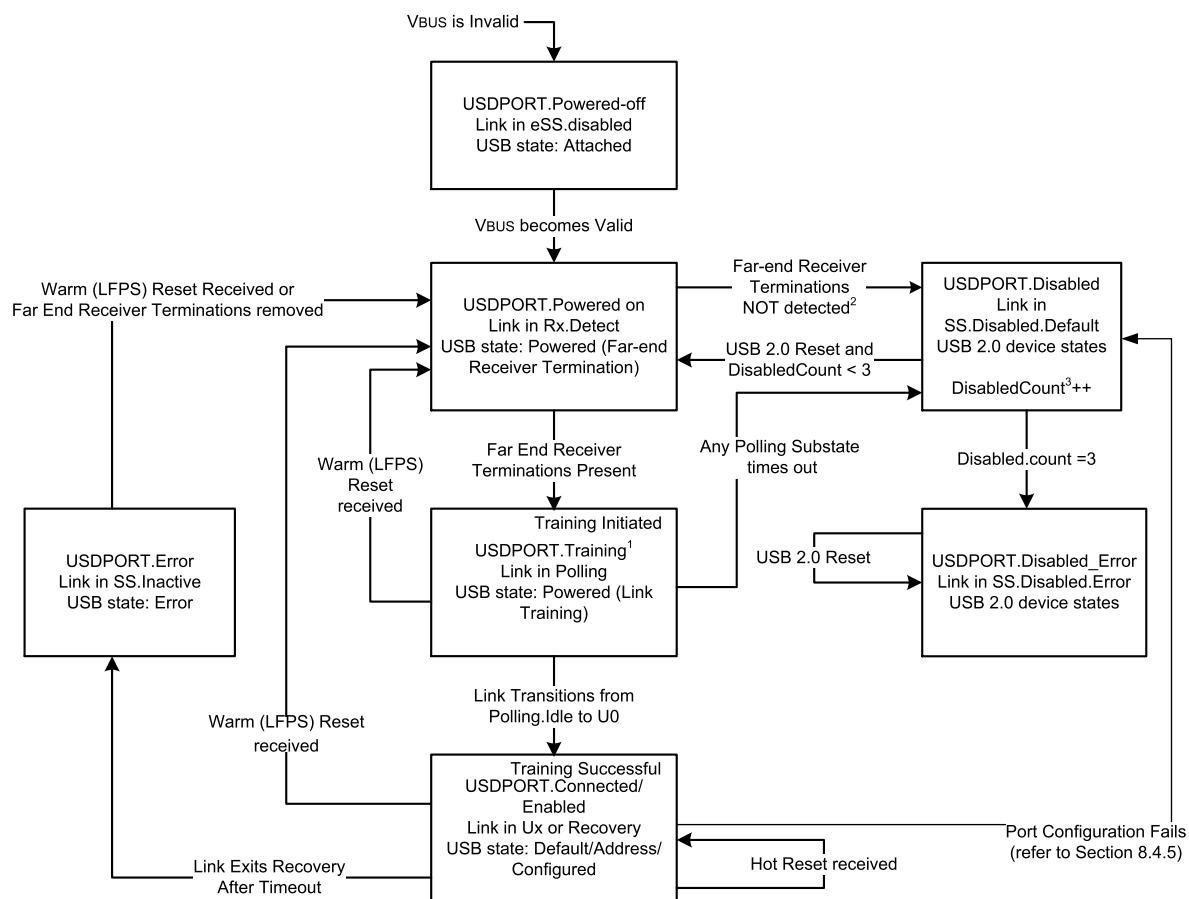
- A peripheral device shall not attempt to connect on the USB 2.0 interface when the port is in the USPORT.Connected state.
- A peripheral device shall not attempt to connect on the USB 2.0 interface unless the port has entered the USPORT.Powered-off state and VBUS is still present as shown in Figure 10-12.
- If a device is connected on the USB 2.0 interface and it receives a USB 2.0 bus reset, the device shall enter the USPORT.Powered-On state within tCheckSuperSpeedOnReset time.
- After a USB 2.0 reset, if the Enhanced SuperSpeed port enters the USPORT.Training state, the device shall disconnect on the USB 2.0 interface within tUSB2SwitchDisconnect time.

A device shall follow the requirements for an upstream facing hub port in Section 10.6 with the following exceptions and additions:

- None of the conditions related to downstream port apply.
- A peripheral device initiates transitions to U1 or U2 when otherwise allowed based on vendor specific algorithms.

10.18.2 Peripheral Device Upstream Port State Machine

The following sections provide a functional description of a state machine that exhibits correct peripheral device behavior for when to connect on Enhanced SuperSpeed or USB 2.0. Figure 10-26 is an illustration of the peripheral device upstream port state machine.



¹ Peripheral Device must disconnect on USB2.0 within tUSB2SwitchDisconnect of entering this state

² If USPORT.Powered on was entered from any state except USPORT.Disabled then this transition shall take place if Far-end Receiver Terminations (RRX-DC) are not detected after 8 successive Rx.Detect.Quiet to Rx.Detect.Active transitions. If USPORT.Powered on was entered from the USPORT.Disabled state, then this transition shall take place the first time that Far-end Receiver Terminations are not detected in the Rx.Detect.Active substate.

³ Disabled count is incremented each time the "Disabled" state is entered from the "Training Initiated" state. Disabled count is reset to '0' each time the link completes Port Configuration.

IEC

Figure 10-26 – Peripheral Upstream Device Port State Machine

10.18.2.1 USDPORT.Powered-off

The USDPORT.Powered-off state is the default state for a peripheral device. A peripheral device shall transition into this state if any of the following situations occur:

- From any state when VBUS is invalid.

In this state, the port's link shall be in the eSS.Disabled state and the USB 2.0 pull-up is not applied. The corresponding peripheral USB state shall be Attached.

10.18.2.2 USDPORT.Powered on

A port shall transition into this state if any of the following situations occur:

- From the USDPORT.Powered-off state when VBUS becomes valid (and local power is valid if required).
- From the USDPORT.Error state when the link receives a warm reset or Far-end terminations are removed.
- From the USDPORT.Connected/Enabled state when the link receives a Warm Reset.
- From the USDPORT.Disabled state if the port receives a USB 2.0 reset.
- From the USDPORT.Training state when the link receives a Warm Reset.

In this state, the port's link shall be in the Rx.Detect state. The corresponding peripheral device USB state shall be Powered (Far-end Receiver Termination substate).

If the transition is from the USDPORT.Disabled state the USB 2.0 pull-up shall remain enabled. If the transition is from any other state the USB 2.0 pull-up shall not be enabled.

10.18.2.3 USDPORT.Training

A port transitions to this state from the USDPORT.Powered-on state when Enhanced SuperSpeed Far-end Receiver Terminations are detected.

In this state, the port's link shall be in the Polling state. The corresponding peripheral device USB state shall be Powered (Link Training substate).

As noted in Figure 10-26, the peripheral device shall disconnect on USB 2.0 within $t_{USB2SwitchDisconnect}$ after entering this state.

10.18.2.4 USDPORT.Connected/Enabled

A port transitions to this state from the USDPORT.Training state when its link enters U0 from Polling.Idle. A port remains in this state during hot reset. When a hot reset is completed, the corresponding peripheral device USB state shall transition to Default.

In this state, the Enhanced SuperSpeed link is in U0, U1, U2, U3 or Recovery and the USB 2.0 pull-up is not applied. The corresponding peripheral device USB state shall be Default, Address, or Configured.

10.18.2.5 USDPORT.Error

A port transitions to this state when a serious error condition occurred while attempting to operate the link. A port transitions to this state if the following situation occurs:

- From the USDPORT.Connected/Enabled state if the link enters Recovery and times out without recovering.

In this state, the port's link shall be in the eSS.Inactive state. The corresponding peripheral device USB state shall be Error.

A port exits the USDPORT.Error state only if a Warm Reset is received on the link or if Far-end Receiver Terminations are removed.

10.18.2.6 USDPORT.Disabled

A port transitions to this state

- From the USDPORT.Powered on state when Far-end Receiver Terminations are not detected as per the rules described below:

If USDPORT.Powered on was entered from any state except USDPORT.Disabled then this transition shall take place if Far-end Receiver Terminations (RRX-DC) are not detected after eight successive Rx.Detect.Quiet to Rx.Detect.Active transitions.

If USDPORT.Powered on was entered from the USDPORT.Disabled state, then this transition shall take place the first time that Far-end Receiver Terminations are not detected in the Rx.Detect.Active substate.

- From the USDPort.Training state if a timeout occurs on any Polling substate (see Figure 7-18).
- From the USDPort.Connected state, if the Port Configuration process times out (see Section 8.4.6).

A count (*Disabled_count*) shall be maintained of each entry into the USDPort.Disabled state.

- Count is initialized to “0” upon power on reset.
- The count is incremented upon each entry into the USDPort.Disabled state from the USDPORT.Training Initiated state.
- If the count equals 3, the port transitions to the Disabled.Error state.
- The count is reset to “0” upon a successful completion of the Port Configuration process.

In this state, the port's link shall be in the eSS.Disabled state. The corresponding peripheral device USB state shall be USB 2.0 Device States.

10.18.2.7 USDPORT.Disabled_Error

A port transitions to this state from the USDPort.Disabled state when *Disabled_Count* = 3

- The port shall remain in USDPORT.Disabled_Error state if the port's link receives a USB 2.0 reset.

In this state, a fatal error has been detected on the port's link and the link shall be in the eSS.Disabled state. The corresponding peripheral device USB state shall be USB 2.0 Device States.

10.19 Hub Chapter Parameters

Table 10-19 includes a complete list of the parameters used in the hub chapter.

Table 10-19 – Hub Parameters

Name	Description	Min	Max	Units
tDownLinkStateChange	Time from receiving the first symbol of a header packet directed to a downstream port that is in a low power link state to initiating a return to U0 on the downstream link. A hub reports the actual delay via the wHubDelay field in Enhanced SuperSpeed Hub Descriptor.	0	400	ns
sDataSymbolsBabble	The number of symbols in a data packet payload after the DPPSTART ordered set without a Data Packet Payload ending frame ordered set or DPPABORT ordered set that shall cause a device to detect the packet is invalid.	1030	N/A	symbols
tHubPropRemoteWakeUpstream	Time from start of remote wakeup signaling on the downstream port a hub to when the hub must propagate the remote wakeup signaling on its upstream port if the upstream port link is in U3.	0	1	ms
tHubDriveRemoteWakeDownstream	Time from receiving a SetPortFeature(PORT_LINK_STATE) U0 for a downstream port with a link in U3 to driving remote wakeup signaling on the link.	0	1000	ns
tHubPort2PortExitLat	Time from a downstream port's link initiating a U-state change to when a hub must initiate a U-state change on the upstream port's link (when required).	0	1	µs
aCurrentUnit	Unit for reporting the current draw of hub controller circuitry in the hub descriptor.		4	mA
nMaxHubPorts	Maximum number of ports on a USB hub.		15	Ports
tTimeForResetError	If the downstream port link remains in RxDetect.Active or RxDetect.Quiet for this length of time during a warm reset, the reset is considered to have failed.	100	200	ms
tCheckSuperSpeedOnReset	Time from when a device (not a hub) detects a USB 2.0 bus reset to when the device port must enter the USPORT.Powered-On state.	0	1	ms
tUSB2SwitchDisconnect	Time from when a device (not a hub) enters USPORT.Training to when the device must disconnect on the USB 2.0 interface if the device is connected on USB 2.0.	0	1	ms
tPropagationDelayJitterLimit	Variation from the minimum time between when the last symbol of a header packet routed to a downstream port with a link in U0 is received on a hub upstream port and when the first symbol of the header packet is transmitted on the hub downstream port. ITP propagation shall meet tPropagationDelayJitterLimit for all downstream ports that transmit the ITP.	-0	+8	ns
nSkipSymbolLimit	Average number of symbols between transmitted SKP ordered sets.	354	354	Symbols

Name	Description	Min	Max	Units
tHubDelay	When both the upstream and the downstream port are operating at the same speed this timing defines the maximum delay in nanoseconds a hub can introduce while forwarding packets in either direction. The time is measured from receipt of the first symbol of the packet by the receiving port until the transmitting port sends the first symbol of the packet, when both the receiving and transmitting links are in U0 and the following conditions are met: • No Link Commands are in flight. • Remote Rx Header Buffer Credit Count of the transmitting port is not zero. • Tx Header Buffer of the transmitting port is empty. A hub reports the actual delay via the wHubDelay field in Enhanced SuperSpeed Hub Descriptor.		400	ns
tHubArbitrationDelay	When both the upstream and the downstream port are operating at the same speed this timing defines the maximum delay in nanoseconds a hub can introduce while forwarding packets in either direction. The time is measured from receipt of the last symbol of the packet by the receiving port until the transmitting port sends the first symbol of the packet, when both the receiving and transmitting links are in U0 and the following conditions are met: • No Link Commands are in flight. • Remote Rx Header Buffer Credit Count of the transmitting port is not zero. • Tx Header Buffer of the transmitting port is empty. A hub reports the actual delay via the wHubDelay field in Enhanced SuperSpeed Hub Descriptor.		400	ns
tDSPORTEnabledToU3	Time from when a downstream port enters DSPORT.ENABLED when the upstream hub port is in U3 and remote wakeup is disabled to when the downstream port shall initiate a transition to the U3 link state.	0	1	s
ArbitrationWeightBase	The value is used as the denominator to calculate the arbitration weight. See Section 10.8.6.1.	1.25	1.25	Gbps

11 Interoperability and Power Delivery

This chapter defines interoperability and power delivery requirements for USB 3.1. Areas covered include USB 3.1 host and device support for USB 2.0 operation, and USB 3.1 VBus power consumption limits.

Table 11-1 lists the compatibility matrix for USB 3.1 and USB 2.0. The implication of identifying a host port as supporting USB 3.1 is that both hardware and software support for USB 3.1 is in place; otherwise the port shall only be identified as a USB 2.0 port.

Table 11-1 – USB 3.0 and USB 2.0 Interoperability

USB Host Port	USB Device Capability	Connected Mode
USB 2.0	USB 2.0	USB 2.0 high-speed, full-speed, or low-speed
	USB 3.1	USB 2.0 high-speed, full-speed, or low-speed
USB 3.1	USB 2.0	USB 2.0 high-speed, full-speed, or low-speed
	USB 3.1	USB 3.1 SuperSpeed or SuperSpeedPlus mode

It should be noted that USB 3.1 devices are not required to be backward compatible with USB 1.1 host ports although supporting full-speed and low-speed modes are allowed.

11.1 USB 3.1 Host Support for USB 2.0

USB 3.1-capable ports on hosts shall also support USB 2.0 operation in order to enable backward compatibility with USB 2.0 devices. It should be noted, however, that USB 3.1-capable hosts are not required to support Enhanced SuperSpeed operation on all of the ports available on the host, i.e., some USB 3.1-capable hosts may have a mix of USB 2.0-only and USB 3.1-capable ports.

To address the situation where a USB 3.1 device is connected to a USB 2.0-only port on a USB 3.1-capable host, after establishing a USB 2.0 high-speed connection with the device, it is recommended that the host inform the user that the device will support Enhanced SuperSpeed operation if it is moved to a USB 3.1-capable port on the same host. If a USB 3.1 device is connected to a USB 3.1-capable host via a USB 2.0 hub, it is recommended that the host inform the user that the device will support Enhanced SuperSpeed operation if it is moved to an appropriate host port or if the hub is replaced with a USB 3.1 hub.

When a USB 3.1 hub is connected to a host's USB 3.1-capable port, both USB 3.1 Enhanced SuperSpeed and USB 2.0 high-speed bus connections shall be allowed to connect and operate in parallel. There is no requirement for a USB 3.1-capable host to support multiple parallel connections to peripheral devices.

The USB 2.0 capabilities of a USB 3.1 host shall be designed to the USB 2.0 specification and shall meet the USB 2.0 compliance requirements.

11.2 USB 3.1 Hub Support for USB 2.0

All ports, both upstream and downstream, on USB 3.1 hubs shall support USB 2.0 operation in order to enable backward compatibility with USB 2.0 devices.

When another USB 3.1 hub is connected in series with a USB 3.1 hub, both SuperSpeed and USB 2.0 high-speed bus connections shall be allowed to connect and operate in parallel. There is no requirement for a USB 3.1 hub to support multiple parallel connections to peripheral devices.

Within a USB 3.1 hub, both the Enhanced SuperSpeed and USB 2.0 hub devices shall implement in the hub framework a common standardized ContainerID to enable software to identify the physical relationship of the hub devices. The ContainerID descriptor is a part of the BOS descriptor set.

The USB 2.0 capabilities of a USB 3.1 hub shall be designed to the USB 2.0 specification and shall meet the USB 2.0 compliance requirements.

11.3 USB 3.1 Device Support for USB 2.0

In most cases, backward compatible operation at USB 2.0 signaling is supported by USB 3.1 devices in order that higher capability devices are still useful with lesser capable hosts and hubs. For product installations where support for USB 3.1 operation can be independently assured between the device and the host, such as internal devices that are not user accessible, device support for USB 2.0 may not be necessary. USB 3.1 device certification requirements require support for USB 2.0 for all user attached devices.

For any given USB 3.1 peripheral device within a single physical package, only one USB connection mode, either Enhanced SuperSpeed or a USB 2.0 speed but not both, shall be established for operation with the host.

Peripheral devices may implement in the device framework a common standardized ContainerID to enable software to identify all of the functional components of a specific device and independent of which speed bus it appears on. All devices within a compound device that support ContainerID shall return the same ContainerID.

The USB 2.0 capabilities of a USB 3.1 device shall be designed to the USB 2.0 specification and shall meet the USB 2.0 compliance requirements. Note that a USB 3.1 device operating in one of the USB 2.0 modes must return 0210H in the bcd version field of the device descriptor.

11.4 Power Distribution

This section describes the USB 3.1 power distribution specification. The USB 2.0 power distribution requirements still apply when a USB 3.1 device is operating at high-speed, full-speed, or low-speed. Note that a USB 3.1 peripheral device shall not draw more than 100 mA until it detects far-end Rx terminations in the unconfigured state.

11.4.1 Classes of Devices and Connections

USB 3.1 provides power over two connectors: the Standard-A connector and the MicroAB connector (when the ID pin is connected to ground). The following sections focus on the power delivery requirements for the Standard-A connector.

The power source and sink requirements of different device classes can be simplified with the introduction of the concept of a unit load. A unit load for Enhanced SuperSpeed has been redefined to be 150 mA. The number of unit loads a device can draw is an absolute maximum, not an average over time. A device may be either low-power at one unit load or high-power, consuming up to six unit loads. All devices default to low-power when first powered. The transition to high-power is under software control. It is the responsibility of software to ensure adequate power is available before allowing devices to consume high-power.

The USB supports a range of power sourcing and power consuming agents; these include the following:

- **Root port hubs:** Are directly attached to the USB Host Controller. Hub power is derived from the same source as the Host Controller. Systems that obtain operating power externally, either AC or DC, must be capable of supplying at least six unit loads to each port. Such ports are called high-power ports. Battery-powered systems may supply either one or six unit loads. Ports that can supply only one unit load are termed low-power ports.
- **Self-powered hubs:** Power for the internal functions and downstream facing ports does not come from VBUS. However, the USB interface of the hub may draw up to one unit load from VBUS on its upstream facing port to allow the interface to function when the remainder of the hub is powered down. Hubs that obtain operating power externally (not from VBUS) must supply six unit loads to each port.
- **Low-power bus-powered devices:** All power to these devices comes from VBUS. They may draw no more than one unit load at any time.

- **High-power bus-powered devices:** All power to these devices comes from VBUS. They must draw no more than one unit load upon power-up and may draw up to six unit loads after being configured.
- **Ports may support the USB Charging Specification.**
- **Self-powered devices:** May draw up to one unit load from VBUS to allow the USB interface to function when the remainder of the function is powered down. All other power comes from an external (not from VBUS) source.

No device shall supply (source) current on VBUS at its upstream facing port at any time. From VBUS on its upstream facing port, a device may only draw (sink) current. Devices must also ensure that the maximum operating current drawn by a device is one unit load until configured.

11.4.1.1 Self-powered Hubs

Self-powered hubs have a local power supply that furnishes power to any non-removable functions and to all downstream facing ports, as shown in Figure 11-1. Power for the Hub Controller, however, may be supplied from the upstream VBUS (a “hybrid” powered hub) or the local power supply. The advantage of supplying the Hub Controller from the upstream supply is that communication from the host is possible even if the device’s power supply remains off. This makes it possible to differentiate between a disconnected and an unpowered device. If the hub draws power for its upstream facing port from VBUS, it may not draw more than one unit load.

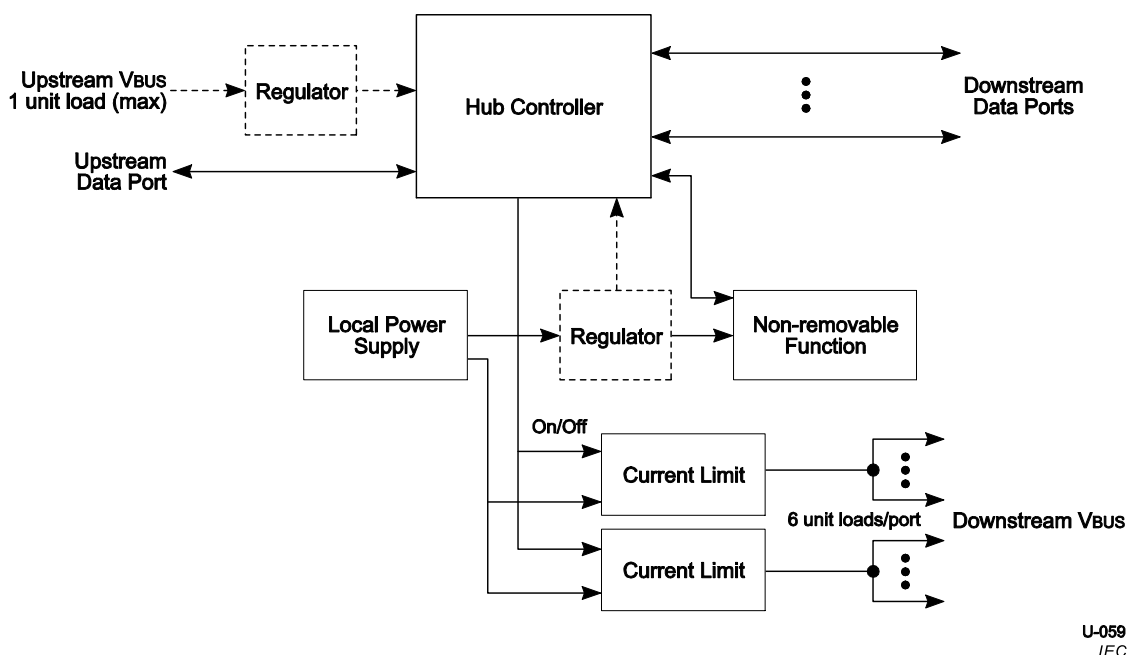


Figure 11-1 – Compound Self-powered Hub

The maximum number of ports that can be supported is limited by the capability of the local VBUS supply.

11.4.1.1.1 Over-current Protection

The host and all self-powered hubs must implement over-current protection for safety reasons, and the hub must have a way to detect the over-current condition and report it to the USB software. Should the aggregate current drawn by a gang of downstream facing ports exceed a preset value, the over-current protection circuit removes or reduces power from all affected downstream facing ports. The over-current condition is reported through the hub to the Host Controller, as described in Section 10.13.5. The preset value cannot exceed 5,0 A and must be sufficiently higher than the maximum allowable port current or time delayed such that transient currents (e.g., during power up or dynamic attach or reconfiguration) do not trip the

over-current protector. If an over-current condition occurs on any port, subsequent operation of the USB is not guaranteed, and once the condition is removed, it may be necessary to reinitialize the bus as would be done upon power-up. The over-current limiting mechanism must be resettable without user mechanical intervention. Polymeric PTCs and solid-state switches are examples of methods that can be used for over-current limiting.

11.4.1.2 Low-power Bus-powered Devices

A low-power device is one that draws up to one unit load from the USB cable when operational. Figure 11-2 shows a typical bus-powered, low-power device, such as a mouse. Low-power regulation can be integrated into the device silicon. Low-power devices must be capable of operating with input VBUS voltages as low as 4,00 V, measured at the plug end of the cable.

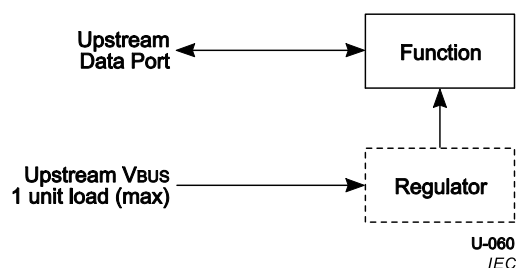


Figure 11-2 – Low-power Bus-powered Function

11.4.1.3 High-power Bus-powered Devices

A device is defined as being high-power if, when fully powered, it draws over one but no more than six unit loads from the USB cable. A high-power device requires staged switching of power. It must first come up in a reduced power state of less than one unit load. At bus enumeration time, its total power requirements are obtained and compared against the available power budget. If sufficient power exists, the remainder of the device may be powered on. High-power devices shall be capable of operating with an input voltage as low as 4,00 V. They must also be capable of operating at full power (up to six unit loads) with an input voltage of 4,00 V measured at the device side of the B-series receptacle.

A typical high-power device is shown in Figure 11-3. The device's electronics have been partitioned into two sections. The device controller contains the minimum amount of circuitry necessary to permit enumeration and power budgeting. The remainder of the device resides in the function block.

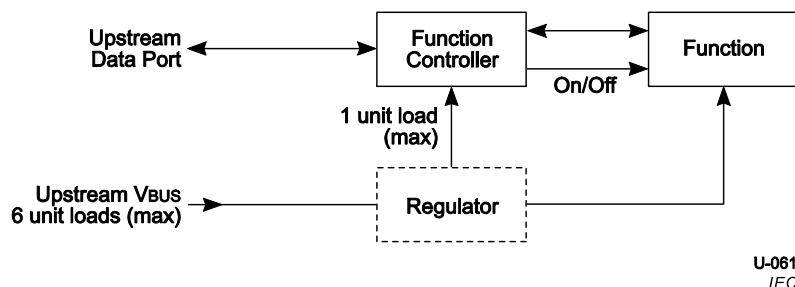


Figure 11-3 – High-power Bus-powered Function

11.4.1.4 Self-powered Devices

Figure 11-4 shows a typical self-powered device. The device controller is powered either from the upstream bus via a low-power regulator or from the local power supply. The advantage of the former scheme is that it permits detection and enumeration of a self-powered device whose local power supply is turned off. The maximum upstream power that the device controller can draw is one unit load, and the regulator block must implement inrush current limiting. The amount of power that the device block may draw is limited only by the local

power supply. Because the local power supply is not required to power any downstream bus ports, it does not need to implement current limiting, soft start, or power switching.

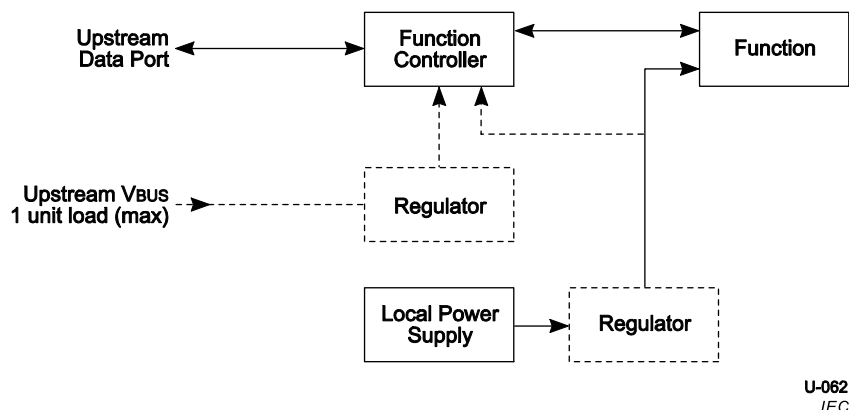


Figure 11-4 – Self-powered Function

11.4.2 Steady-State Voltage Drop Budget

The steady-state voltage drop budget is derived from the following assumptions:

- The nominal $5\text{ V} \pm 5\%$ source (host or hub) is 4,75 V to 5,25 V.
- The voltage supplied at the connector of hub or root ports shall be between 4,45 V to 5,25 V.
- The maximum voltage drop (for detachable cables) between the A-series plug and B-series plug on VBUS is 171 mV.
- The maximum current for the calculations is 0,9 A.
- The maximum voltage drop for all cables between upstream and downstream on GND is 171 mV.
- The maximum voltage drop for all mated connectors is 27 mV.
- All hubs and peripheral devices shall be able to provide configuration information with as little as 4,00 V at the device end of their B-series receptacle. Both low and high-power devices need to be operational with this minimum voltage.

Figure 11-5 shows the minimum allowable voltages. Note that under transient conditions, the supply at the device can drop to 3,67 V for a brief moment.

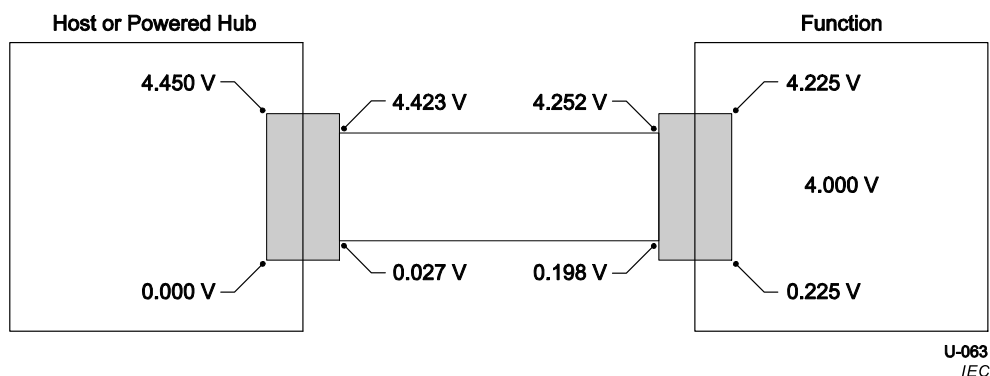
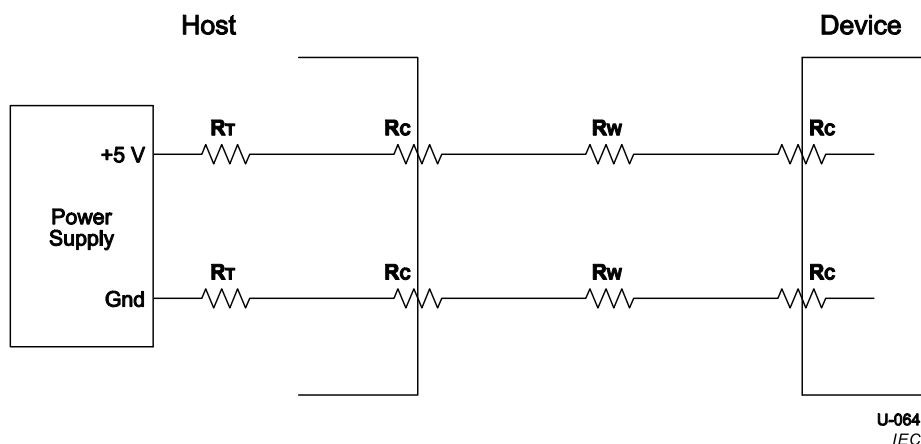


Figure 11-5 – Worst-case Voltage Drop Topology (Steady State)

NOTE The following assumptions were used in Figure 11-5:

- 3 meter cable assembly with A-series and B-series plugs
- #22AWG wire used for power and ground (0.019 Ω /foot)

- A-series and B-series plug/receptacle pair have a contact resistance of 30 mΩ
- Wire ~380 mΩ series resistance
- IR Drop at device = $((2 \times 30 \text{ m}\Omega) + 190 \text{ m}\Omega) \times 900 \text{ mA} \times 2$ or 0,450 V



Rt Host Trace Resistance: 0.167 Ω

Rc Mated Connector Resistance: 0.030 Ω

Rw Cable Resistance: 0.190 Ω

Figure 11-6 – Worst-case Voltage Drop Analysis Using Equivalent Resistance

11.4.3 Power Control During Suspend/Resume

All USB devices may draw up to 2,5 mA during suspend. When configured, bus-powered compound devices may consume a suspend current of up to 12,5 mA. This 12,5 mA budget includes 2,5 mA suspend current for the internal hub plus 2,5 mA suspend current for each port on that internal hub having attached internal functions, up to a maximum of four ports. When computing suspend current, the current from VBUS through the bus pull-up and pull-down resistors must be included.

While in the Suspend state, a device may briefly draw more than the average current. The amplitude of the current spike cannot exceed the device power allocation 150 mA (or 900 mA). A maximum of 1,0 second is allowed for an averaging interval. The average current cannot exceed the average suspend current limit (ICCS, see Table 11-2) during any 1,0-second interval. The profile of the current spike is restricted so the transient response of the power supply (which may be an efficient, low-capacity, trickle power supply) is not overwhelmed. The rising edge of the current spike must be no more than 100 mA/μs. Downstream facing ports must be able to absorb the 900 mA peak current spike and meet the voltage droop requirements defined for inrush current during dynamic attach. Figure 11-7 illustrates a typical example profile for an averaging interval.

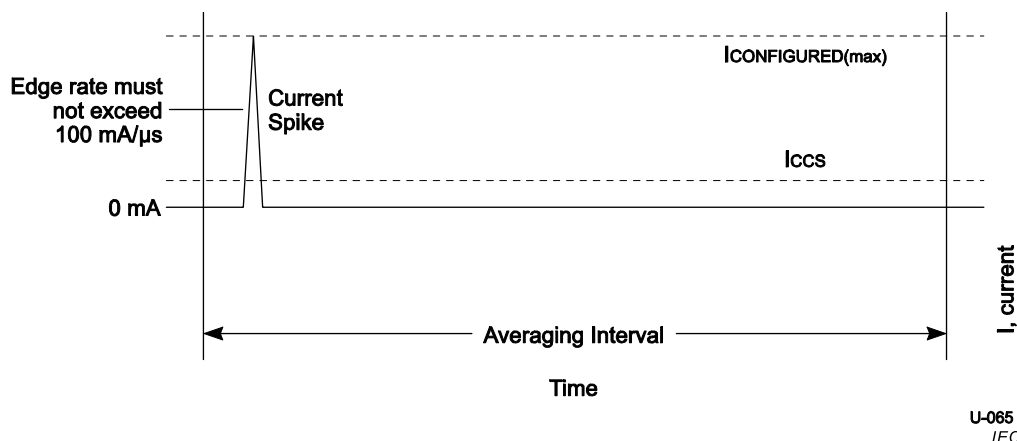


Figure 11-7 – Typical Suspend Current Averaging Profile

Devices are responsible for handling the bus voltage reduction due to the inductive and resistive effects of the cable. When a hub is in the Suspend state, it must still be able to provide the maximum current per port (six unit loads per port for self-powered hubs). This is necessary to support remote wakeup-capable devices that will power-up while the remainder of the system is still suspended. Such devices, when enabled to do remote wakeup, must drive resume signaling upstream within 10 ms of starting to draw the higher, non-suspend current. Devices not capable of remote wakeup must not draw the higher current when suspended.

When devices wakeup, either by themselves (remote wakeup) or by seeing resume signaling, they must limit the inrush current on VBUS. The device must have sufficient on-board bypass capacitance or a controlled power-on sequence such that the current drawn from the hub does not exceed the maximum current capability of the port at any time while the device is waking up.

11.4.4 Dynamic Attach and Detach

The act of plugging or unplugging a hub or peripheral device must not affect the functionality of another device on other segments of the network. Unplugging a device will stop any transactions in progress between that device and the host. However, the hub or root port to which this device was attached will recover from this condition and will alert the host that the port has been disconnected.

11.4.4.1 Inrush Current Limiting

When a peripheral device or hub is plugged into the network, it has a certain amount of on-board capacitance between VBUS and ground. In addition, the regulator on the device may supply current to its output bypass capacitance and to the device as soon as power is applied. Consequently, if no measures are taken to prevent it, there could be a surge of current into the device which might pull the VBUS on the hub below its minimum operating level. Inrush currents can also occur when a high-power device is switched into its high-power mode. This problem must be solved by limiting the inrush current and by providing sufficient capacitance in each hub to prevent the power supplied to the other ports from going out of tolerance. An additional motivation for limiting inrush current is to minimize contact arcing, thereby prolonging connector contact life.

The maximum droop possible in the hub VBUS is 330 mV. In order to meet this requirement, the following conditions must be met:

- The maximum load (CRPB) that can be placed at the downstream end of a cable is 10 μF in parallel with as small as a 27 Ω resistance. The 10 μF capacitance represents any bypass capacitor directly connected across the VBUS lines in the device plus any

capacitive effects visible through the regulator in the device. The $27\ \Omega$ resistance represents one unit load of current drawn by the device during connect.

- If more bypass capacitance is required in the device, then the device must incorporate some form of VBUS surge current limiting, such that it matches the characteristics of the above load.
- The hub downstream facing port VBUS power lines must be bypassed (CHPB) with no less than $120\ \mu\text{F}$ of low-ESR capacitance per hub. Standard bypass methods should be used to minimize inductance and resistance between the bypass capacitors and the connectors to reduce droop. The bypass capacitors themselves should have a low dissipation factor to allow decoupling at higher frequencies.

The upstream facing port of a hub is also required to meet the above requirements.

A high-power bus-powered device that is switching from a lower power configuration to a higher power configuration must not cause droop $> 330\ \text{mV}$ on the VBUS at its upstream hub. The device can meet this by ensuring that changes in the capacitive load it presents do not exceed $10\ \mu\text{F}$.

11.4.4.2 Dynamic Detach

When a device is detached from the network with power flowing in the cable, the inductance of the cable will cause a large flyback voltage to occur on the open end of the device cable. This flyback voltage is not destructive. Proper bypass measures on the hub ports will suppress any coupled noise. This will require some low capacitance, very low inductance bypass capacitors on each hub port connector. The flyback voltage and the noise it creates are also moderated by the bypass capacitance on the device end of the cable. Also, there must be some minimum capacitance on the device end of the cable to ensure that the inductive flyback on the open end of the cable does not cause the voltage on the device end to reverse polarity. A minimum of $1,0\ \mu\text{F}$ is recommended for bypass across VBUS.

11.4.5 VBUS Electrical Characteristics

Table 11-2 – DC Electrical Characteristics

Parameter	Symbol	Conditions	Min.	Max.	Units
Supply Voltage:					
Port (downstream connector)	VBUS		4,45	5,25	V
Port (upstream connector)	VBUS		4,0		V
Supply Current:					
High-power Hub Port (out)	ICCPRT		900		mA
Low-power Hub Port (out)	ICCUPT		150		mA
High-power Peripheral Device (in)	ICCHPF			900	mA
Low-power Peripheral Device (in)	ICCLPF			150	mA
Unconfigured Device (in)	ICCINIT			150	mA
Suspended High-power Device	ICCS			2,5	mA

11.4.6 Powered-B Connector

The Powered-B connector was defined by USB 3.0 has been deprecated.

11.4.7 Wire Gauge Table

Table 11-3 is a table of VBUS/Gnd wire gauges showing the relationship between gauge and maximum length in order to achieve the previously cited voltage drop values. The user should note that these lengths are the maximum length possible to meet the voltage drop budget,

thus gauges smaller and lengths greater than the table values will fail to deliver the expected voltage value.

Table 11-3 – VBus/Gnd Wire Gauge vs. Maximum Length

American Stranded Wire Gauge (AWG) on VBus/Gnd	Ohms Per 100 Meters (Maximum)	Maximum Cable Length (Meters)
28	23,20	0,8
26	14,60	1,3
24	9,09	2,0
22	5,74	3,0
20	3,58	5,3

A Gen 1 Symbol Encoding

Table A.1 shows the byte-to-Symbol encodings for data characters. Table A.2 shows the Symbol encodings for the Special Symbols. RD- and RD+ refer to the Running Disparity of the Symbol sequence on a per-Lane basis.

Table A.1 – 8b/10b Data Symbol Codes

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D0.0	00	000 00000	100111 0100	011000 1011
D1.0	01	000 00001	011101 0100	100010 1011
D2.0	02	000 00010	101101 0100	010010 1011
D3.0	03	000 00011	110001 1011	110001 0100
D4.0	04	000 00100	110101 0100	001010 1011
D5.0	05	000 00101	101001 1011	101001 0100
D6.0	06	000 00110	011001 1011	011001 0100
D7.0	07	000 00111	111000 1011	000111 0100
D8.0	08	000 01000	111001 0100	000110 1011
D9.0	09	000 01001	100101 1011	100101 0100
D10.0	0A	000 01010	010101 1011	010101 0100
D11.0	0B	000 01011	110100 1011	110100 0100
D12.0	0C	000 01100	001101 1011	001101 0100
D13.0	0D	000 01101	101100 1011	101100 0100
D14.0	0E	000 01110	011100 1011	011100 0100
D15.0	0F	000 01111	010111 0100	101000 1011
D16.0	10	000 10000	011011 0100	100100 1011
D17.0	11	000 10001	100011 1011	100011 0100
D18.0	12	000 10010	010011 1011	010011 0100
D19.0	13	000 10011	110010 1011	110010 0100
D20.0	14	000 10100	001011 1011	001011 0100
D21.0	15	000 10101	101010 1011	101010 0100
D22.0	16	000 10110	011010 1011	011010 0100
D23.0	17	000 10111	111010 0100	000101 1011
D24.0	18	000 11000	110011 0100	001100 1011
D25.0	19	000 11001	100110 1011	100110 0100
D26.0	1A	000 11010	010110 1011	010110 0100
D27.0	1B	000 11011	110110 0100	001001 1011
D28.0	1C	000 11100	001110 1011	001110 0100
D29.0	1D	000 11101	101110 0100	010001 1011
D30.0	1E	000 11110	011110 0100	100001 1011
D31.0	1F	000 11111	101011 0100	010100 1011
D0.1	20	001 00000	100111 1001	011000 1001
D1.1	21	001 00001	011101 1001	100010 1001
D2.1	22	001 00010	101101 1001	010010 1001
D3.1	23	001 00011	110001 1001	110001 1001

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D4.1	24	001 00100	110101 1001	001010 1001
D5.1	25	001 00101	101001 1001	101001 1001
D6.1	26	001 00110	011001 1001	011001 1001
D7.1	27	001 00111	111000 1001	000111 1001
D8.1	28	001 01000	111001 1001	000110 1001
D9.1	29	001 01001	100101 1001	100101 1001
D10.1	2A	001 01010	010101 1001	010101 1001
D11.1	2B	001 01011	110100 1001	110100 1001
D12.1	2C	001 01100	001101 1001	001101 1001
D13.1	2D	001 01101	101100 1001	101100 1001
D14.1	2E	001 01110	011100 1001	011100 1001
D15.1	2F	001 01111	010111 1001	101000 1001
D16.1	30	001 10000	011011 1001	100100 1001
D17.1	31	001 10001	100011 1001	100011 1001
D18.1	32	001 10010	010011 1001	010011 1001
D19.1	33	001 10011	110010 1001	110010 1001
D20.1	34	001 10100	001011 1001	001011 1001
D21.1	35	001 10101	101010 1001	101010 1001
D22.1	36	001 10110	011010 1001	011010 1001
D23.1	37	001 10111	111010 1001	000101 1001
D24.1	38	001 11000	110011 1001	001100 1001
D25.1	39	001 11001	100110 1001	100110 1001
D26.1	3A	001 11010	010110 1001	010110 1001
D27.1	3B	001 11011	110110 1001	001001 1001
D28.1	3C	001 11100	001110 1001	001110 1001
D29.1	3D	001 11101	101110 1001	010001 1001
D30.1	3E	001 11110	011110 1001	100001 1001
D31.1	3F	001 11111	101011 1001	010100 1001
D0.2	40	010 00000	100111 0101	011000 0101
D1.2	41	010 00001	011101 0101	100010 0101
D2.2	42	010 00010	101101 0101	010010 0101
D3.2	43	010 00011	110001 0101	110001 0101
D4.2	44	010 00100	110101 0101	001010 0101
D5.2	45	010 00101	101001 0101	101001 0101
D6.2	46	010 00110	011001 0101	011001 0101
D7.2	47	010 00111	111000 0101	000111 0101
D8.2	48	010 01000	111001 0101	000110 0101
D9.2	49	010 01001	100101 0101	100101 0101
D10.2	4A	010 01010	010101 0101	010101 0101
D11.2	4B	010 01011	110100 0101	110100 0101
D12.2	4C	010 01100	001101 0101	001101 0101
D13.2	4D	010 01101	101100 0101	101100 0101
D14.2	4E	010 01110	011100 0101	011100 0101

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D15.2	4F	010 01111	010111 0101	101000 0101
D16.2	50	010 10000	011011 0101	100100 0101
D17.2	51	010 10001	100011 0101	100011 0101
D18.2	52	010 10010	010011 0101	010011 0101
D19.2	53	010 10011	110010 0101	110010 0101
D20.2	54	010 10100	001011 0101	001011 0101
D21.2	55	010 10101	101010 0101	101010 0101
D22.2	56	010 10110	011010 0101	011010 0101
D23.2	57	010 10111	111010 0101	000101 0101
D24.2	58	010 11000	110011 0101	001100 0101
D25.2	59	010 11001	100110 0101	100110 0101
D26.2	5A	010 11010	010110 0101	010110 0101
D27.2	5B	010 11011	110110 0101	001001 0101
D28.2	5C	010 11100	001110 0101	001110 0101
D29.2	5D	010 11101	101110 0101	010001 0101
D30.2	5E	010 11110	011110 0101	100001 0101
D31.2	5F	010 11111	101011 0101	010100 0101
D0.3	60	011 00000	100111 0011	011000 1100
D1.3	61	011 00001	011101 0011	100010 1100
D2.3	62	011 00010	101101 0011	010010 1100
D3.3	63	011 00011	110001 1100	110001 0011
D4.3	64	011 00100	110101 0011	001010 1100
D5.3	65	011 00101	101001 1100	101001 0011
D6.3	66	011 00110	011001 1100	011001 0011
D7.3	67	011 00111	111000 1100	000111 0011
D8.3	68	011 01000	111001 0011	000110 1100
D9.3	69	011 01001	100101 1100	100101 0011
D10.3	6A	011 01010	010101 1100	010101 0011
D11.3	6B	011 01011	110100 1100	110100 0011
D12.3	6C	011 01100	001101 1100	001101 0011
D13.3	6D	011 01101	101100 1100	101100 0011
D14.3	6E	011 01110	011100 1100	011100 0011
D15.3	6F	011 01111	010111 0011	101000 1100
D16.3	70	011 10000	011011 0011	100100 1100
D17.3	71	011 10001	100011 1100	100011 0011
D18.3	72	011 10010	010011 1100	010011 0011
D19.3	73	011 10011	110010 1100	110010 0011
D20.3	74	011 10100	001011 1100	001011 0011
D21.3	75	011 10101	101010 1100	101010 0011
D22.3	76	011 10110	011010 1100	011010 0011
D23.3	77	011 10111	111010 0011	000101 1100
D24.3	78	011 11000	110011 0011	001100 1100
D25.3	79	011 11001	100110 1100	100110 0011

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D26.3	7A	011 11010	010110 1100	010110 0011
D27.3	7B	011 11011	110110 0011	001001 1100
D28.3	7C	011 11100	001110 1100	001110 0011
D29.3	7D	011 11101	101110 0011	010001 1100
D30.3	7E	011 11110	011110 0011	100001 1100
D31.3	7F	011 11111	101011 0011	010100 1100
D0.4	80	100 00000	100111 0010	011000 1101
D1.4	81	100 00001	011101 0010	100010 1101
D2.4	82	100 00010	101101 0010	010010 1101
D3.4	83	100 00011	110001 1101	110001 0010
D4.4	84	100 00100	110101 0010	001010 1101
D5.4	85	100 00101	101001 1101	101001 0010
D6.4	86	100 00110	011001 1101	011001 0010
D7.4	87	100 00111	111000 1101	000111 0010
D8.4	88	100 01000	111001 0010	000110 1101
D9.4	89	100 01001	100101 1101	100101 0010
D10.4	8A	100 01010	010101 1101	010101 0010
D11.4	8B	100 01011	110100 1101	110100 0010
D12.4	8C	100 01100	001101 1101	001101 0010
D13.4	8D	100 01101	101100 1101	101100 0010
D14.4	8E	100 01110	011100 1101	011100 0010
D15.4	8F	100 01111	010111 0010	101000 1101
D16.4	90	100 10000	011011 0010	100100 1101
D17.4	91	100 10001	100011 1101	100011 0010
D18.4	92	100 10010	010011 1101	010011 0010
D19.4	93	100 10011	110010 1101	110010 0010
D20.4	94	100 10100	001011 1101	001011 0010
D21.4	95	100 10101	101010 1101	101010 0010
D22.4	96	100 10110	011010 1101	011010 0010
D23.4	97	100 10111	111010 0010	000101 1101
D24.4	98	100 11000	110011 0010	001100 1101
D25.4	99	100 11001	100110 1101	100110 0010
D26.4	9A	100 11010	010110 1101	010110 0010
D27.4	9B	100 11011	110110 0010	001001 1101
D28.4	9C	100 11100	001110 1101	001110 0010
D29.4	9D	100 11101	101110 0010	010001 1101
D30.4	9E	100 11110	011110 0010	100001 1101
D31.4	9F	100 11111	101011 0010	010100 1101
D0.5	A0	101 00000	100111 1010	011000 1010
D1.5	A1	101 00001	011101 1010	100010 1010
D2.5	A2	101 00010	101101 1010	010010 1010
D3.5	A3	101 00011	110001 1010	110001 1010
D4.5	A4	101 00100	110101 1010	001010 1010

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D5.5	A5	101 00101	101001 1010	101001 1010
D6.5	A6	101 00110	011001 1010	011001 1010
D7.5	A7	101 00111	111000 1010	000111 1010
D8.5	A8	101 01000	111001 1010	000110 1010
D9.5	A9	101 01001	100101 1010	100101 1010
D10.5	AA	101 01010	010101 1010	010101 1010
D11.5	AB	101 01011	110100 1010	110100 1010
D12.5	AC	101 01100	001101 1010	001101 1010
D13.5	AD	101 01101	101100 1010	101100 1010
D14.5	AE	101 01110	011100 1010	011100 1010
D15.5	AF	101 01111	010111 1010	101000 1010
D16.5	B0	101 10000	011011 1010	100100 1010
D17.5	B1	101 10001	100011 1010	100011 1010
D18.5	B2	101 10010	010011 1010	010011 1010
D19.5	B3	101 10011	110010 1010	110010 1010
D20.5	B4	101 10100	001011 1010	001011 1010
D21.5	B5	101 10101	101010 1010	101010 1010
D22.5	B6	101 10110	011010 1010	011010 1010
D23.5	B7	101 10111	111010 1010	000101 1010
D24.5	B8	101 11000	110011 1010	001100 1010
D25.5	B9	101 11001	100110 1010	100110 1010
D26.5	BA	101 11010	010110 1010	010110 1010
D27.5	BB	101 11011	110110 1010	001001 1010
D28.5	BC	101 11100	001110 1010	001110 1010
D29.5	BD	101 11101	101110 1010	010001 1010
D30.5	BE	101 11110	011110 1010	100001 1010
D31.5	BF	101 11111	101011 1010	010100 1010
D0.6	C0	110 00000	100111 0110	011000 0110
D1.6	C1	110 00001	011101 0110	100010 0110
D2.6	C2	110 00010	101101 0110	010010 0110
D3.6	C3	110 00011	110001 0110	110001 0110
D4.6	C4	110 00100	110101 0110	001010 0110
D5.6	C5	110 00101	101001 0110	101001 0110
D6.6	C6	110 00110	011001 0110	011001 0110
D7.6	C7	110 00111	111000 0110	000111 0110
D8.6	C8	110 01000	111001 0110	000110 0110
D9.6	C9	110 01001	100101 0110	100101 0110
D10.6	CA	110 01010	010101 0110	010101 0110
D11.6	CB	110 01011	110100 0110	110100 0110
D12.6	CC	110 01100	001101 0110	001101 0110
D13.6	CD	110 01101	101100 0110	101100 0110
D14.6	CE	110 01110	011100 0110	011100 0110
D15.6	CF	110 01111	010111 0110	101000 0110

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D16.6	D0	110 10000	011011 0110	100100 0110
D17.6	D1	110 10001	100011 0110	100011 0110
D18.6	D2	110 10010	010011 0110	010011 0110
D19.6	D3	110 10011	110010 0110	110010 0110
D20.6	D4	110 10100	001011 0110	001011 0110
D21.6	D5	110 10101	101010 0110	101010 0110
D22.6	D6	110 10110	011010 0110	011010 0110
D23.6	D7	110 10111	111010 0110	000101 0110
D24.6	D8	110 11000	110011 0110	001100 0110
D25.6	D9	110 11001	100110 0110	100110 0110
D26.6	DA	110 11010	010110 0110	010110 0110
D27.6	DB	110 11011	110110 0110	001001 0110
D28.6	DC	110 11100	001110 0110	001110 0110
D29.6	DD	110 11101	101110 0110	010001 0110
D30.6	DE	110 11110	011110 0110	100001 0110
D31.6	DF	110 11111	101011 0110	010100 0110
D0.7	E0	111 00000	100111 0001	011000 1110
D1.7	E1	111 00001	011101 0001	100010 1110
D2.7	E2	111 00010	101101 0001	010010 1110
D3.7	E3	111 00011	110001 1110	110001 0001
D4.7	E4	111 00100	110101 0001	001010 1110
D5.7	E5	111 00101	101001 1110	101001 0001
D6.7	E6	111 00110	011001 1110	011001 0001
D7.7	E7	111 00111	111000 1110	000111 0001
D8.7	E8	111 01000	111001 0001	000110 1110
D9.7	E9	111 01001	100101 1110	100101 0001
D10.7	EA	111 01010	010101 1110	010101 0001
D11.7	EB	111 01011	110100 1110	110100 1000
D12.7	EC	111 01100	001101 1110	001101 0001
D13.7	ED	111 01101	101100 1110	101100 1000
D14.7	EE	111 01110	011100 1110	011100 1000
D15.7	EF	111 01111	010111 0001	101000 1110
D16.7	F0	111 10000	011011 0001	100100 1110
D17.7	F1	111 10001	100011 0111	100011 0001
D18.7	F2	111 10010	010011 0111	010011 0001
D19.7	F3	111 10011	110010 1110	110010 0001
D20.7	F4	111 10100	001011 0111	001011 0001
D21.7	F5	111 10101	101010 1110	101010 0001
D22.7	F6	111 10110	011010 1110	011010 0001
D23.7	F7	111 10111	111010 0001	000101 1110
D24.7	F8	111 11000	110011 0001	001100 1110
D25.7	F9	111 11001	100110 1110	100110 0001
D26.7	FA	111 11010	010110 1110	010110 0001

Data Byte Name	Data Byte Value (hex)	Bits HGF EDCBA (binary)	Current RD- abcdei fghj (binary)	Current RD+ abcdei fghj (binary)
D27.7	FB	111 11011	110110 0001	001001 1110
D28.7	FC	111 11100	001110 1110	001110 0001
D29.7	FD	111 11101	101110 0001	010001 1110
D30.7	FE	111 11110	011110 0001	100001 1110
D31.7	FF	111 11111	101011 0001	010100 1110

Table A.2 – 8b/10b Special Character Symbol Codes

Data Byte Name	Data Byte Value	Bits HGF EDCBA	Current RD – abcdei fghj	Current RD + abcdei fghj
K28.0	1C	000 11100	001111 0100	110000 1011
K28.1	3C	001 11100	001111 1001	110000 0110
K28.2	5C	010 11100	001111 0101	110000 1010
K28.3	7C	011 11100	001111 0011	110000 1100
K28.4	9C	100 11100	001111 0010	110000 1101
K28.5	BC	101 11100	001111 1010	110000 0101
K28.6	DC	110 11100	001111 0110	110000 1001
K28.7	FC	111 11100	001111 1000	110000 0111
K23.7	F7	111 10111	111010 1000	000101 0111
K27.7	FB	111 11011	110110 1000	001001 0111
K29.7	FD	111 11101	101110 1000	010001 0111
K30.7	FE	111 11110	011110 1000	100001 0111

NOTE Only a small fraction of the possible K-characters are defined in this table. Any K-character that decodes to a value that is not in Table A.2 shall be returned as Decode_Error_Substitution (K28.4). Refer to Section 6.3.1.4 and Table 6-1 for more information.

B Symbol Scrambling

B.1. Data Scrambling

The following subroutines encode and decode an 8-bit value contained in “inbyte” with the LFSR. This is presented as one example only; there are many ways to obtain the proper output. This example demonstrates how to advance the LFSR eight times in one operation and how to XOR the data in one operation. Many other implementations are possible but they must all produce the same output as that shown here.

The following algorithm uses the “C” programming language conventions, where “<<” and “>>” represent the shift left and shift right operators, “>” is the compare greater than operator, and “^” is the exclusive or operator, and “&” is the logical “AND” operator.

```
/*
this routine implements the serial descrambling algorithm in parallel form
for the LSFR polynomial: x^16+x^5+x^4+x^3+1
this advances the LSFR 8 bits every time it is called
this requires fewer than 25 xor gates to implement (with a static register)
```

The XOR required to advance 8 bits/clock is:

bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	8	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7
				8	9	10	11	12	13	14	15					
					8	9	10	11	12	13	14	15				
						8	9	10	11	12	13	14	15			

The serial data is just the reverse of the upper byte:

bit	0	1	2	3	4	5	6	7
	15	14	13	12	11	10	9	8

*/

```
int scramble_byte(int inbyte)
{

static int scrambit[16];
static int bit[16];
static int bit_out[16];
static unsigned short lfsr = 0xffff; // 16 bit short for polynomial
int i, outbyte;

    if (inbyte == COMMA)    // if this is a comma
    {
        lfsr = 0xffff;      // reset the LFSR
        return (COMMA);     // and return the same data
    }

    if (inbyte == SKIP)     // don't advance or encode on skip
        return (SKIP);

    for (i=0; i<16; i++)    // convert LFSR to bit array for legibility
        bit[i] = (lfsr >> i) & 1;

    for (i=0; i<8; i++)    // convert byte to be scrambled for legibility
        scrambit[i] = (inbyte >> i) & 1;

    // apply the xor to the data
    if (! (inbyte & 0x100) &&    // if not a KCODE, scramble the data
        ! (TrainingSequence == TRUE)) // and if not in the middle of
    {                               // a training sequence
        scrambit[0] ^= bit[15];
        scrambit[1] ^= bit[14];
        scrambit[2] ^= bit[13];
        scrambit[3] ^= bit[12];
        scrambit[4] ^= bit[11];
        scrambit[5] ^= bit[10];
        scrambit[6] ^= bit[9];
        scrambit[7] ^= bit[8];
    }
}
```

```
// Now advance the LFSR 8 serial clocks
bit_out[ 0] = bit[ 8];
bit_out[ 1] = bit[ 9];
bit_out[ 2] = bit[10];
bit_out[ 3] = bit[11] ^ bit[ 8];
bit_out[ 4] = bit[12] ^ bit[ 9] ^ bit[ 8];
bit_out[ 5] = bit[13] ^ bit[10] ^ bit[ 9] ^ bit[ 8];
bit_out[ 6] = bit[14] ^ bit[11] ^ bit[10] ^ bit[ 9];
bit_out[ 7] = bit[15] ^ bit[12] ^ bit[11] ^ bit[10];
bit_out[ 8] = bit[ 0] ^ bit[13] ^ bit[12] ^ bit[11];
bit_out[ 9] = bit[ 1] ^ bit[14] ^ bit[13] ^ bit[12];
bit_out[10] = bit[ 2] ^ bit[15] ^ bit[14] ^ bit[13];
bit_out[11] = bit[ 3]           ^ bit[15] ^ bit[14];
bit_out[12] = bit[ 4]           ^ bit[15];
bit_out[13] = bit[ 5];
bit_out[14] = bit[ 6];
bit_out[15] = bit[ 7];
lfsr = 0;
for (i=0; i <16; i++) // convert the LFSR back to an integer
    lfsr += (bit_out[i] << i);

outbyte = 0;
for (i= 0; i<8; i++) // convert data back to an integer
    outbyte += (scrambit[i] << i);

return outbyte;
}

/* NOTE THAT THE DESCRAMBLE ROUTINE IS IDENTICAL TO THE SCRAMBLE ROUTINE
this routine implements the serial descrambling algorithm in parallel form
this advances the lfsr 8 bits every time it is called
this uses fewer than 25 xor gates to implement (with a static register)
The XOR tree is the same as the scrambling routine
*/
```

```
int unscramble_byte(int inbyte)
{
    static int descrambit[8];
    static int bit[16];
    static int bit_out[16];
    static unsigned short lfsr = 0xffff; // 16 bit short for polynomial
    int outbyte, i;

    if (inbyte == COMMA) // if this is a comma
    {
        lfsr = 0xffff; // reset the LFSR
        return (COMMA); // and return the same data
    }

    if (inbyte == SKIP) // don't advance or encode on skip
        return (SKIP);

    for (i=0; i<16;i++) // convert the LFSR to bit array for legibility
        bit[i] = (lfsr >> i) & 1;

    for (i=0; i<8; i++) // convert byte to be de-scrambled for legibility
        descrambit[i] = (inbyte >> i) & 1;

    // apply the xor to the data
    if (! (inbyte & 0x100) && // if not a KCODE, scramble the data
        ! (TrainingSequence == TRUE)) // and if not in the middle of
    { // a training sequence
        descrambit[0] ^= bit[15];
        descrambit[1] ^= bit[14];
        descrambit[2] ^= bit[13];
        descrambit[3] ^= bit[12];
        descrambit[4] ^= bit[11];
        descrambit[5] ^= bit[10];
        descrambit[6] ^= bit[9];
        descrambit[7] ^= bit[8];
    }
}
```

```
// Now advance the LFSR 8 serial clocks
bit_out[ 0] = bit[ 8];
bit_out[ 1] = bit[ 9];
bit_out[ 2] = bit[10];
bit_out[ 3] = bit[11] ^ bit[ 8];
bit_out[ 4] = bit[12] ^ bit[ 9] ^ bit[ 8];
bit_out[ 5] = bit[13] ^ bit[10] ^ bit[ 9] ^ bit[ 8];
bit_out[ 6] = bit[14] ^ bit[11] ^ bit[10] ^ bit[ 9];
bit_out[ 7] = bit[15] ^ bit[12] ^ bit[11] ^ bit[10];
bit_out[ 8] = bit[ 0] ^ bit[13] ^ bit[12] ^ bit[11];
bit_out[ 9] = bit[ 1] ^ bit[14] ^ bit[13] ^ bit[12];
bit_out[10] = bit[ 2] ^ bit[15] ^ bit[14] ^ bit[13];
bit_out[11] = bit[ 3]           ^ bit[15] ^ bit[14];
bit_out[12] = bit[ 4]           ^ bit[15];
bit_out[13] = bit[ 5];
bit_out[14] = bit[ 6];
bit_out[15] = bit[ 7];
lfsr = 0;
for (i=0; i <16; i++) // convert the LFSR back to an integer
    lfsr += (bit_out[i] << i);

outbyte = 0;
for (i= 0; i<8; i++) // convert data back to an integer
    outbyte += (descrambit[i] << i);

return outbyte;
}
```

The initial 16-bit values of the LFSR for the first 128 LFSR advances following a reset are listed below:

	0, 8	1, 9	2, A	3, B	4, C	5, D	6, E	7, F
00	FFFF	E817	0328	284B	4DE8	E755	404F	4140
08	4E79	761E	1466	6574	7DBD	B6E5	FDA6	B165
10	7D09	02E5	E572	673D	34CF	CB54	4743	4DEF
18	E055	40E0	EE40	54BE	B334	2C7B	7D0C	07E5
20	E5AF	BA3D	248A	8DC4	D995	85A1	BD5D	4425
28	2BA4	A2A3	B8D2	CBF8	EB43	5763	6E7F	773E
30	345F	5B54	5853	5F18	14B7	B474	6CD4	DC4C
38	5C7C	70FC	F6F0	E6E6	F376	603B	3260	64C2
40	CB84	9743	5CBF	B3FC	E47B	6E04	0C3E	3F2C
48	29D7	D1D1	C069	7BC0	CB73	6043	4A60	6FFA
50	F207	1102	01A9	A939	2351	566B	6646	4FF6
58	F927	3081	85B0	AC5D	478C	82EF	F3F2	E43B
60	2E04	027E	7E72	79AE	A501	1A7D	7F2A	2197
68	9019	0610	1096	9590	8FCD	D0E7	F650	46E6
70	E8D6	C228	3AB2	B70A	129F	9CE2	FC3C	2B5C
78	5AA3	AF6A	70C7	CDF0	E3D5	C0AB	B9C0	D9C1

An 8-bit value of 0 repeatedly encoded with the LFSR after reset produces the following consecutive 8-bit values:

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	FF	17	C0	14	B2	E7	02	82	72	6E	28	A6	BE	6D	BF	8D
10	BE	40	A7	E6	2C	D3	E2	B2	07	02	77	2A	CD	34	BE	E0
20	A7	5D	24	B1	9B	A1	BD	22	D4	45	1D	D3	D7	EA	76	EE
30	2C	DA	1A	FA	28	2D	36	3B	3A	0E	6F	67	CF	06	4C	26
40	D3	E9	3A	CD	27	76	30	FC	94	8B	03	DE	D3	06	52	F6
50	4F	88	80	95	C4	6A	66	F2	9F	0C	A1	35	E2	41	CF	27
60	74	40	7E	9E	A5	58	FE	84	09	60	08	A9	F1	0B	6F	62
70	17	43	5C	ED	48	39	3F	D4	5A	F5	0E	B3	C7	03	9D	9B
80	8B	0D	8E	5C	33	98	77	AE	2D	AC	0B	3E	DA	0B	42	7A
90	7C	D1	CF	A8	1C	12	EE	41	C2	3F	38	7A	0D	69	F4	01
A0	DA	31	72	C5	A0	D7	93	0E	DC	AF	A4	55	E7	F0	72	16
B0	68	D5	38	84	DD	00	CD	18	9E	CA	30	59	4C	75	1B	77
C0	31	C5	ED	CF	91	64	6E	3D	FE	E8	29	04	CF	6C	FC	C4
D0	0B	5E	DA	62	BA	5B	AB	DF	59	B7	7D	37	5E	E3	1A	C6
E0	88	14	F5	4F	8B	C8	56	CB	D3	10	42	63	04	8A	B4	F7
F0	84	01	A0	01	83	49	67	EE	3E	2A	8B	A4	76	AF	14	D5
100	4F	AC	60	B6	79	D6	62	B7	43	E7	E5	2A	40	2C	6E	7A
110	56	61	63	20	6A	97	4A	38	05	E5	DD	68	0D	78	4C	53
120	8B	D6	86	57	B2	AA	1A	80	18	DC	BA	FC	03	A3	4B	30

C Power Management

This appendix offers a system level overview of Enhanced SuperSpeed's power management features and capabilities. The following topics are also included:

- Examples of how end to end low power link state exit latencies are calculated
- A discussion of device-initiated link power management policies
- An example device implementation for Latency Tolerance Messaging
- System power considerations for SuperSpeed versus High Speed device interfaces

C.1. SuperSpeed Power Management Overview

The SuperSpeed architecture has been defined with platform power efficiency as a primary objective. Some of the key power efficiency enhancements include:

- Elimination of continuous device polling
- Elimination of broadcast packet transmission through hubs
- Introduction of link power management states enabling aggressive power savings when idle
- Host and device initiated transition to low power states
- Device and individual Function level suspend capabilities enabling devices to remove power from all, or only those portions of their circuitry that are not in use

C.1.1. Link Power Management

Link power management enables a link to be placed into a lower power state when the link partners are idle. The longer a pair of link partners remain idle, the deeper the power savings that can be achieved by progressing from U0 (link active) to U1 (link standby with fast exit), to U2 (link standby with slower exit), and finally to U3 (suspend).

After being configured by software, the U1 and U2 link states are entered and exited via hardware autonomous control. Hardware autonomous transitions for the U1 and U2 link states enable faster response times. This, in turn, translates to better power savings when entering a power saving state, and less impact on the operational state when exiting. The U3 link state however is entered only under software control, typically after a software inactivity timeout, and is exited either by software (host initiated exit) or hardware (remote wakeup). The U3 link state is directly coupled to the device's suspend state (refer to Section C.1.4).

C.1.1.1. Summary of Link States

Table C.1 provides a summary characterization of the SuperSpeed link states.

Table C.1 – Link States and Characteristics Summary

Link State	Description	Characteristics	State Transition Initiator	Device Clock Gen On/Off	Typical Exit Latency Range
U0	Link active	Link operational state	N/A	On	N/A
U1	Link idle – fast exit	Rx and Tx circuitry quiesced	Hardware ¹	On or Off	µs
U2	Link idle – slower exit	Clock generation circuitry may additionally be quiesced	Hardware ¹	On or Off ²	µs – ms
U3	Link suspend	Interface (e.g., Physical Layer) power may be removed	Entry: Software only Exit: Hardware or Software	Off	ms

NOTES:

- 1) It is possible, under system test conditions, to instrument software initiated U1 and U2 state transitions.
- 2) From a power efficiency perspective it is desirable for devices to turn off their clock generation circuitry (e.g., their PLL) during the U2 link state.

C.1.1.2. U0 – Link Active

U0 is the fully operational, link active state. Packets of any type may be communicated over a link that is in the U0 State.

C.1.1.3. U1 – Link Idle with Fast Exit

U1 is a power saving state this is characterized by fast transition time back to the U0 State. Note that the predominant latency, when transitioning from the U1 → U0 state is imposed by the time that is required to achieve symbol lock between the two link partners.

C.1.1.3.1. U1 Entry

Either link partner for a given link can request a transition to the U1 link state. All downstream ports (hub or root ports) track inactivity using an inactivity timer mechanism. When a port's inactivity timer expires, if enabled, it requests transition to the U1 state. Upstream ports may also initiate U1 entry based on device specific policies.

When a port initiates U1 entry, its link partner may either accept or reject the request. The link level U0 → U1 transition process consists of one port transmitting an LGO_U1 link command, and its link partner responding with either an LAU (accept the request) or an LXU (reject the request) link command.

The most typical reason for rejecting a U1 transition request would be because the requesting port's link partner has some activity which will shortly require a packet transmission. Downstream ports would also reject a U1 transition request if not enabled to accept U1 transition requests. Rejection of a U1 transition request by an upstream port simply resets and restarts the requesting link partner's inactivity timer.

System software configures and then enables each device to initiate U1 entry. The primary programming parameters involved in setting up hardware autonomous link state management include:

U1DevExitLat – Parameter used by devices to report their maximum U1 to U0 exit latency (refer to Chapter 9 for details).

PORT_U1_TIMEOUT – Sets the value for the downstream port's U1 inactivity timer. When specifying a value between the range 0x01-0xFE it also enables the downstream port to send U1 entry transition requests to its link partner (refer to Chapter 10 for details).

U1_Enable – Enables an upstream port to initiate requests for transition into U1 (refer to Chapter 9 for details).

The U1_Enable feature controls whether that particular upstream port may initiate U1 entry. Regardless of whether the upstream port is enabled for U1 entry initiation or not, it still responds to requests for U1 entry from its link partner, either accepting or rejecting the transition request.

The following table illustrates the relationship between the downstream timeout and the upstream U1_Enable to configure the platform for any combination of port link state management. For example, if *U1_Enable* is enabled, and *PORT_U1_TIMEOUT* is set to FFH, then only the upstream port may initiate requests for transition to U1.

PORT_U1_TIMEOUT	U1_Enable	Initiating Port
01H-FEH	Disabled	Downstream Port only
FFH	Enabled	Upstream Port only
01H-FEH	Enabled	Either Port
0H or FFH	Disabled	Neither Port

For detailed information regarding the specifics of the U1 entry process, refer to Chapter 7 of this specification.

C.1.1.3.2. Exiting the U1 State

There are two ways to exit the U1 state. The link can return to the active U0 state or it can transition into a deeper power savings state (U2).

Transitioning from U1 → U0

Either link partner can initiate a transition from U1 → U0. This transition is normally initiated when a packet needs to be transmitted, such as an IN message from the host, or an ERDY message from a device. The transition process is initiated by first signaling a Low Frequency Periodic Signaling (LFPS) handshake. This is followed by link recovery and training sequences. Refer to Chapter 6 and 7 respectively.

Transitioning from U1 → U2

This transitions the link to an even lower power state and is triggered by a second inactivity timer (U2 inactivity timer). When a link enters U1, this starts the U2 inactivity timer. If the U2 inactivity timer expires while the link is still in U1, then both link partners transition silently from U1 → U2 without additional bus activity. The next sections discuss this in more detail.

C.1.1.4. U2 – Link Idle with Slow Exit

The purpose of the U2 link state is to use less power than the U1 state, however at the cost of increased exit latency. For example, clock generation circuitry may be quiesced in order to save additional power in comparison with U1. Under some implementation-specific circumstances, this may not make sense. For example, a hub may share one PLL across all of its ports so the PLL cannot be quiesced unless all of its ports are in U2.

The primary parameters involved in configuring a port for U2 link transitions include:

U2DevExitLat – Parameter used by devices to report their maximum U2 to U0 exit latency (refer to Chapter 9 for details).

PORT_U2_TIMEOUT – Sets the value of a downstream port's U2 inactivity timer. When specified with a value in the range 0x01-0xFE it also enables the downstream port to initiate U2 entry transition requests to its link partner (refer to Chapter 10 for details).

U2_Enable – Enables an upstream port to initiate requests for transitions into U2 (refer to Chapter 9 for details).

The U2_Enable feature controls whether an upstream port may initiate U2 entry from U0. Regardless of whether the upstream port is enabled for U2 entry initiation or not, it still responds to requests for U2 entry from its link partner, either accepting or rejecting the transition request.

The following table illustrates the relationship between the downstream timeout and the upstream U2_Enable to configure the platform for any combination of port link state management. For example, if U2_Enable is enabled, and PORT_U2_TIMEOUT is set to FFH, then only the upstream port may initiate requests for transition to U2.

PORT_U2_TIMEOUT	U2_Enable	Initiating Port
01H-FEH	Disabled	Downstream Port only
FFH	Enabled	Upstream Port only
01H-FEH	Enabled	Either Port
0H or FFH	Disabled	Neither Port

Transitioning from U1 → U2

U2 is typically entered directly from U1 as mentioned earlier. The U2 inactivity timer starts when a link enters U1, and when the U2 inactivity timer expires both link partners silently transition from U1 → U2.

Both link partners must be configured with the same U2 inactivity timeout value. This is done by first having software program the U2 inactivity timeout in the downstream port. The downstream port then sends an LMP containing the U2 inactivity timeout value to its link partner. Any changes to this value on the downstream port are also updated with the link partner in the same manner (refer to Chapter 10).

Note that U2 inactivity timer synchronization between link partners can never be perfect so there can be brief periods of time where one port is in U2 while its link partner is still in U1. However given that the U1 → U0 and U2 → U0 state transition processes are compatible with one another this corner condition is handled cleanly.

Transitioning directly to U2 from U0

A downstream port can be configured for direct, hardware autonomous transition from U0 → U2 by programming its U1 inactivity timer to zero while programming its U2 inactivity timer with a non-zero value in the range 0x01 – 0xFE. In this case, when the U2 inactivity timer expires, the downstream port initiates U2 entry from U0.

When a port initiates U2 entry from U0, its link partner may either accept or reject the request. The link level U0 → U2 transition process consists of one port transmitting an LGO_U2 Link

Command, and its link partner responding with either an LAU (accept the request) or an LXU (reject the request) Link Command.

Exiting U2

Exiting U2 can only result in a link state transition to U0. Either link partner can initiate U2 exit, which is initiated when a packet needs to be transmitted. The exit process is similar to that of a U1 → U0 transition, consisting of a Low Frequency Periodic Signaling handshake followed by link recovery and training.

C.1.1.5. U3 – Link Suspend

The U3 state is a deep power saving state where portions of device power may be removed, except as needed to perform the following functions:

- For upstream ports in devices and hubs:
 - Warm Reset signaling detection
 - Wakeup signaling detection (for host initiated wakeup)
 - Wakeup signaling transmission (for remote wakeup capable devices)
- For downstream ports in hubs and hosts:
 - Warm Reset generation
 - Disconnect event detection
 - Wakeup signaling detection (for remote wakeup)
 - Wakeup signaling transmission (for host initiated wakeup)

The purpose of U3 is to minimize power consumption during device or system suspend.

VBUS remains active during U3. Any power rail switching by devices is implementation specific and beyond the scope of this specification.

Entering the U3 State

U3 entry may only be initiated by the host (refer to Chapter 10 for details). Software typically implements an inactivity timeout for the purpose of placing a function into suspend following a long period of inactivity.

When a downstream port initiates a U3 entry request, its link partner is not allowed to reject it. The downstream port sends an LGO_U3 Link Command, and its link partner responds with an LAU Link Command (refer to Chapter 7 for details).

Exiting the U3 State

The only legitimate link state transition from U3 is U3 → U0, and either link partner can initiate it.

Host software initiates U3 exit on a downstream port by issuing a SetPortFeature(PORT_LINK_STATE) request to the desired downstream port. Upstream ports (e.g., upstream port of a hub, or a peripheral device), initiate U3 exit in response to a remote wakeup event. An example of this would be an incoming Wake on LAN packet for a USB attached network interface device. The exit process consists of a Low Frequency Periodic Signaling (LFPS) handshake followed by link recovery and training.

Device Initiated U3 Exit

If the exit was initiated by a device, the specific function within the device that initiated the wakeup would follow up the LFPS triggered transition to U0 by sending a Function Wake device notification packet to the host.

Host initiated U3 Exit

For host initiated U3 exit, the LFPS handshake process allows devices up to 20 ms to complete, allowing sufficient time for a device to turn on a switched power rail if implemented (refer to Chapter 6 for details).

The software interface associated with U3 consists of a set of port controls and function controls. The port controls, e.g., initiating U3 on a hub downstream port, are described in Section C.1.2.3. The function controls, e.g., enabling a function (device) for remote wakeup, are described in Section C.1.4.1.

C.1.2. Link Power Management for Downstream Ports

Hubs play several critical roles in link power management. They perform the following functions:

- Coordinate the upstream port link power management state with that of their downstream ports.
- Handle packet deferral, where the hub tells the host that a packet was sent to a downstream port that is not currently in U0.
- Provide inactivity timers on downstream ports to initiate U1 and U2 entry.

C.1.2.1. Link State Coordination and Management

A hub monitors its downstream ports' link states and keeps its upstream port in the lowest power link state it can without allowing it to be in a lower power state than any of its downstream ports. The intent for this policy is to ensure that the path to the host, (i.e., the upstream port), is as active as the most active of its downstream ports.

When a device initiates a transition from a low power state back to U0 on a hub downstream port, the hub begins transitioning its upstream port to U0 immediately, in parallel with the downstream port.

For packets traveling downstream, hubs must first receive the packet, determine which of its downstream ports the packet is targeted at, and then only initiate a transition to U0 for that particular downstream port.

USB 3.0 hubs use a unicast packet transmission model and this improves the power efficiency of the platform in every instance where a hub is deployed. USB 3.1 hubs use a store and forward packet transmission model with multiple INs in flight. However, this does not impact the link power management as the packets are routed only through the ports directly between the host and device.

C.1.2.2. Packet Deferring

Packet deferring is a mechanism that enables efficient bus utilization while supporting aggressive link power management. Packet deferring achieves this by enabling a hub to respond on behalf of a downstream device whose link is in a low power state. This allows the host to make forward progress while the device is brought back to the active state.

Hub's Role in Packet Deferring

After receiving a header packet from the host and detecting a packet deferring condition, the hub informs the host of this by sending a deferred header packet back to the host. The hub also sends the original header packet to the device, with the deferred field asserted, once it is brought back to the U0 state. The host treats this deferred header packet as it would receipt of an NRDY, and so is then free to initiate transfers with other devices' endpoints instead of waiting for the sleeping link to return to U0 (refer to Chapter 10 for details).

Device's Role in Packet Deferring

When a device receives an IN or an OUT header packet with the deferred field asserted, it prepares for the transfer, sends an ERDY to the host when ready, and keeps its link in U0 until the transfer occurs.

The host ultimately responds to the ERDY by rescheduling the original transfer.

C.1.2.3. Software Interface

The software interface for downstream port power management consists of the following port controls and status fields:

- PORT_LINK_STATE feature and port status field
- PORT_REMOTE_WAKE_MASK feature
- C_PORT_LINK_STATE port status change bit
- PORT_U1_TIMEOUT feature
- PORT_U2_TIMEOUT feature

The PORT_LINK_STATE

This feature is used to request a link state change from any current U-state to any next U-state. In a normal operating environment, this feature is used solely to request U0 → U3 and U3 → U0 transitions. It can be used for test purposes though, to request other state transitions.

The PORT_REMOTE_WAKE_MASK

This feature is used to mask each remote wakeup event that might be originated at a downstream port.

Note that if remote wake notifications for connect, disconnect, or over current events are disabled, these events are still captured and reported as port status change events after the host or hub is resumed.

C_PORT_LINK_STATE

This flag is used to signal completion of a transition from U3 → U0. Specifically, assertion of this flag results from a host initiated wakeup on a downstream port.

Once the C_PORT_LINK_STATE flag is set, a port status change event is sent to system software indicating that the downstream port and its link partner have completed the transition to the U0 state.

Note that C_PORT_LINK_STATE is not asserted in the event of a remote wakeup. As discussed previously, in the event of a Remote Wakeup the associated function sends the host a Function Wake device notification packet.

PORT_U1_TIMEOUT

This feature is used to enable and disable U1 entry on downstream ports. It also specifies the U1 inactivity timeout value.

Timeout Value	U1 Transition Capabilities	U1 Transition Initiation
00H	U1 transitions disabled; will not initiate or accept link partner requests	n/a
01H-FEH	U1 transitions enabled; port will initiate and accept link partner requests	Following timer expiration using specified Timeout Value
FFH	U1 transition initiation disabled; will accept transition requests from its link partner	n/a

PORT_U2_TIMEOUT

This feature is used to enable and disable U2 on downstream ports, and also to set an inactivity timeout for initiating a transition to U2.

Timeout Value	U2 Transition Capabilities	U2 Transition Initiation
00H	U2 transitions disabled; will not initiate or accept link partner requests	n/a
01H-FEH	U2 transitions enabled; port will initiate and accept link partner requests	Following timer expiration using specified Timeout Value
FFH	Direct U1 → U2 transition is disabled; will accept U2 transition requests (if current link state is U0) from its link partner	n/a

Other hub port controls can impact link power management behavior, e.g., the PORT_RESET feature, but are not covered here (refer to Chapter 10 for details).

C.1.3. Other Link Power Management Support Mechanisms

C.1.3.1. Packets Pending Flag

Devices may use the Packets Pending flag (refer to Chapter 8) to help decide when to place their link in a low power state. The Packets Pending flag provides an indication of whether the host controller has any additional packets to transfer on the schedule associated with a given non-Stream endpoint. When there are no more packets pending for all non-Stream endpoints on a device, the device may place its link in a low power state immediately.

For Stream endpoints, the Packet Pending flag is an indication of whether the host controller has any additional packets to transfer on the schedule associated with a given Stream. When there are no more packets pending for any Streams and for all endpoints on a device, the device may place its link in a low power state immediately.

C.1.3.2. Support for Isochronous Transfers

If a link is in a low power state when an isochronous transfer is scheduled, the latency to transition to U0 from source to destination could potentially delay the transfer beyond its subscribed isochronous service interval.

To ensure that isochronous service contract guarantees are satisfied, a SuperSpeed mechanism (Ping) has been defined to bring all paths between the host and an isochronous endpoint to U0 as a routine step in servicing isochronous endpoints.

The host controller, with sufficient information to know how long any given path might take to become fully active, factors this link path exit latency into its isochronous service scheduler and uses the Ping protocol to ensure that the links are brought to a fully active state in time to meet the isochronous service contract.

The ping process consists of the following:

- The host sends a PING packet to a device.
- Hubs route the PING packet toward the targeted device.
- The device responds to the PING packet by sending a PING_RESPONSE packet to the host.
- The device keeps its link in U0 until it receives a subsequent packet from the host.

After sending a PING packet to one device and prior to receiving a PING_RESPONSE packet, the host may transfer data with other devices. Much like the Packet Deferring mechanism, the Ping mechanism enables efficient bus utilization while at the same time supporting significant power savings.

C.1.3.3. Support for Interrupt Transfers

If any links between the host and a scheduled interrupt endpoint are in a low power state, the latency to transition the end to end pathway to U0 could potentially delay the transfer beyond the subscribed interrupt service interval. A host controller, possessing knowledge of link path exit latency between itself and any given device within the link hierarchy, is able to schedule interrupt transfers far enough in advance to compensate for these latencies.

C.1.4. Device Power Management

Device power management is directed primarily under software control, with various hardware mechanisms to support it. Device power management consists of some function level mechanisms plus some device and hub mechanisms.

C.1.4.1. Function Suspend

A function may be placed into function suspend independent of other functions using the FUNCTION_SUSPEND feature. The FUNCTION_SUSPEND feature is also used to enable function remote wakeup (refer to Chapter 9 for details).

If a composite device has at least one of its functions in suspend while other functions remain active, i.e., the device's upstream port is not in U3, a mechanism is needed for a suspended function to signal a remote wakeup. This is done with the Function Wake device notification packet (refer to Chapter 8 for details).

C.1.4.2. Device Suspend

Device suspend is a device-wide state entered and exited intrinsically as a result of a device's upstream port entering and exiting the U3 state. A device may be transitioned into device suspend regardless of the function suspend state of any function within the device.

Devices may implement a switched power rail and remove power from large portions of the device while in suspend. Some device state information must be retained in a persistent state during suspend (refer to Chapter 9 for details).

C.1.4.3. Host Initiated Suspend

Suspending a Device

The host transitions a device into the suspend state according to the following sequence:

- Enable remote wakeup, if needed
- A SetPortFeature(PORT_LINK_STATE, U3) request is issued to the downstream port of which the targeted device is its link partner.
- The downstream port initiates U3 entry by sending an LGO_U3 link command.
- The device sends an LAU link command (acceptance is non-negotiable).
- The downstream port sends an LPMA link command.
- Both link partners transition their transmitters to electrical idle and enter the U3 state (refer to Chapter 7 for details).

Suspending the USB Link Hierarchy

A link hierarchy of devices and hubs is placed into suspend on a device by device basis under host software control. First, all peripheral devices in the hierarchy are placed into suspend. Then the hubs connected to the peripheral devices are placed into suspend. This is repeated all the way up the hierarchy until reaching the root ports of the USB hierarchy.

Note that by using the same technique a select subset of a given USB link hierarchy could be suspended rather than the whole of it if so desired.

C.1.4.4. Host Initiated Wake from Suspend

Host initiated wake from suspend (U3 → U0) of an individual device, group of devices, or of the entire link hierarchy is accomplished using the same repetitive process, one link at a time.

Figure C.1 illustrates the Host initiated Wake Sequence.

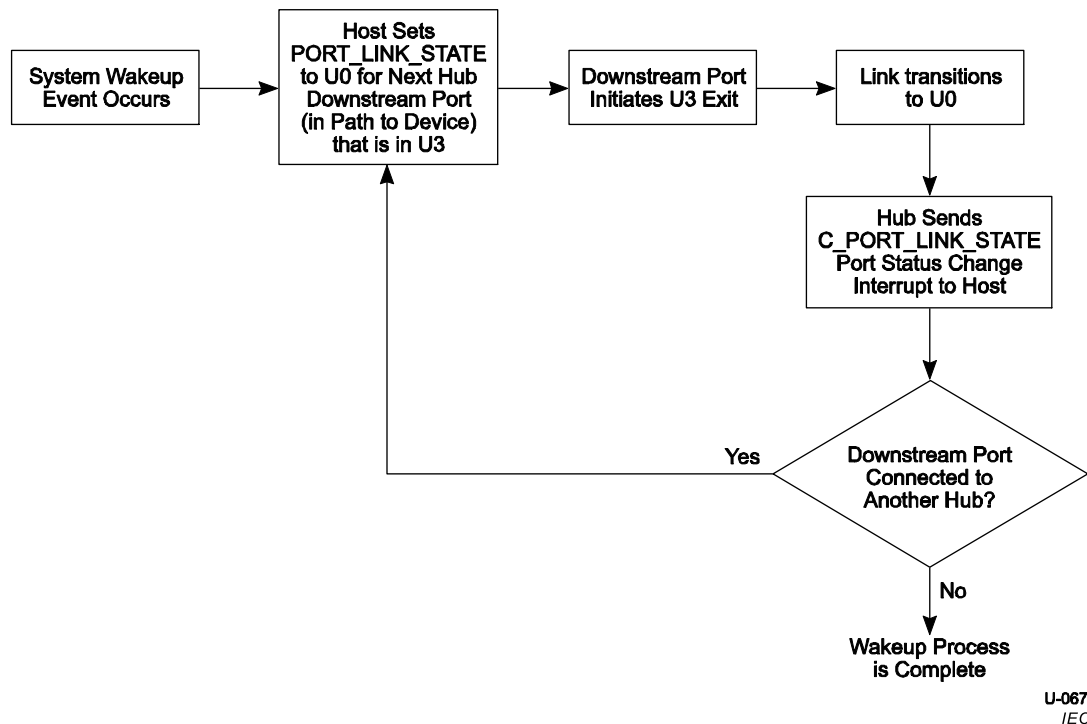


Figure C.1 – Flow Diagram for Host Initiated Wakeup

C.1.4.5. Device Initiated Wake from Suspend

A device initiated transition from suspend (U3 → U0) follows the sequence outlined below:

- 1) The device transmits LFPS wakeup signaling to its link partner.
- 2) The LFPS signaling is propagated upstream until it reaches the root hub or a hub that is not in U3. This hub is referred to as the Controlling Hub.
- 3) The Controlling Hub then automatically reflects LFPS wakeup signaling on the downstream port which had received (from the opposite direction) the wakeup signaling.
- 4) Each hub in the direct path to the remote wakeup device propagates the wakeup signaling downstream on the hub downstream port that had received wakeup signaling. As the wakeup signaling is propagated downstream, each link completes the LFPS handshake and transitions to U0 (refer to Chapters 6 and 7 for details).
- 5) After all of the links between the Controlling Hub and the remote wakeup device transition to U0, the function within the remote wakeup device that had originated the remote wakeup sends a Function Wake device notification packet to the host. This in turn causes a software interrupt, and in the service of this interrupt the function suspend state is cleared for that function.

C.1.5. Platform Power Management Support

The Latency Tolerance Message (LTM) feature allows a platform to make dynamic tradeoffs between power and performance. It enables this, in cooperation with devices, without imposing additional cost.

The LTM protocol enables USB devices to inform the host how long they can tolerate lack of service before experiencing unintended side effects. Each device provides this information in the form of a Best Effort Latency Tolerance (BELT) value. A given device's BELT value is derived considering all configured endpoints, typically conforming to the endpoint with the lowest latency tolerance.

LTM provides the ability for a device to dynamically change its BELT value to more accurately reflect, for example, long periods of anticipated idle time. The platform can potentially take advantage of this insight and, along with other system-related information, conserve more energy at the system level without running the risk of unintended side effects.

C.1.5.1. System Exit Latency and BELT

A device's reported BELT value has to comprehend not only its own intrinsic design characteristics, such as its internal buffering, but also factor in other associated end to end latencies between itself and the host. These would include other latencies associated with the time required to awaken sleeping links, the number of hubs between the device and the host, host processing time, packet propagation delays, etc. The system provides the device with additional system latency information, through the SET_SEL request, such that the device's intrinsic BELT value can be adjusted downward to account for these other factors. Refer to Chapter 8 for detailed LTM specification and to Chapter 9 for specification details regarding the SET_SEL request.

Device implementation determines the total latency that a device can tolerate. The primary factors are the amount of data that the device is required to produce or consume, and the amount of buffering on the device. The total device latency tolerance must be allocated among different system components.

Figure C.2 illustrates the total latency a device may experience within the context of LTM. The latency is the sum of parameters t_1 , t_2 , t_3 , and t_4 :

t_1 : the time to transition all links in the path to the host to U0 when the transition is initiated by the device

t_2 : the time for the ERDY to traverse the interconnect hierarchy from the device to the host

t_3 : the time for the host to consume the ERDY and transmit a response to that request

t_4 : the time for the response to traverse the interconnect hierarchy from the host to the device

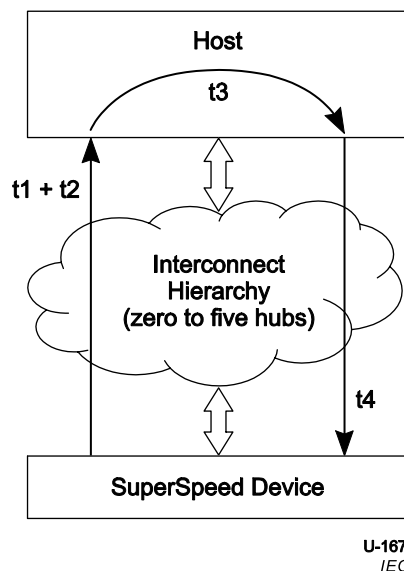


Figure C.2 – Device Total Intrinsic Latency Tolerance

Devices may calculate their BELT value by subtracting U1SEL or U2SEL (refer to the SET_SEL request in Chapter 9) from their total intrinsic latency tolerance. If the device allows its link to enter U2, then the device calculates its BELT by subtracting U2SEL from its total intrinsic latency tolerance. If the device does not allow its link to enter U2 but does allow its link to enter U1, then the device calculates its BELT by subtracting U1SEL from its total intrinsic latency tolerance.

U1SEL and U2SEL are calculated and programmed by host software. Example calculations for t_1 are provided in Section C.2. For LTM purposes t_2 and t_4 should be calculated by host software as follows:

- For t_2 , a hub may delay forwarding the ERDY by up to one maximum packet size (approximately $2.1 \mu\text{s}$ including framing) when there is a transfer in progress. Each additional hub will delay forwarding the ERDY by up to approximately t_{HubDelay} to transfer the packet. The value of t_2 is determined as follows:
 - If there are zero hubs in the direct path between the device and the host, then t_2 is zero
 - If there are one or more hubs in the direct path between the device and the host, then t_2 is approximately $2.1 \mu\text{s} + t_{\text{HubDelay}} \times (\text{number of hubs} - 1)$
- For t_4 , a hub may delay forwarding a packet by up to approximately t_{HubDelay} . The value of t_4 is approximately t_{HubDelay} times the number of hubs in the direct path between the device and the host).

C.1.5.2. Maximum Exit Latency and PING

The host controller is provided with a Maximum Exit Latency (MEL) value that it uses to schedule a PING relative to a periodic transfer. The Maximum Exit Latency must comprehend worst case round trip delay of sending a PING to a device and receiving the PING_RESPONSE from it.

The Maximum Exit Latency factors in the end to end latencies between host and the device. These would include other latencies associated with the time required to awaken sleeping links, the number of hubs between the host and the device, device processing time, packet propagation delays, etc.

The Maximum Exit Latency (t_{MEL}) is the sum of parameters t_{MEL1} , t_{MEL2} , t_{MEL3} , and t_{MEL4} .

C.1.5.2.1. Maximum Exit Latency t_1 (t_{MEL1})

The t_{MEL1} delay is the time to transition all links in the path to the device to U0 when the transition is initiated by the host. The method for calculating MEL t_1 delay is described in Sections C.2.1.1 and C.2.2.1.

C.1.5.2.2. Maximum Exit Latency t_2 (t_{MEL2})

t_{MEL2} is the sum of the t_{HubDelay} values for each hub in the path, and $t_{\text{TPTransmissionDelay}}$ across each link in the path is calculated as:

$$t_{\text{MEL2}} = (\text{sum of } w_{\text{HubDelay}} \text{ values}) + (t_{\text{TPTransmissionDelay}} \times (\text{number of hubs} + 1)).$$

Where, a w_{HubDelay} value is provided by the SuperSpeed Hub Descriptor of each hub in the path, respectively, and $t_{\text{TPTransmissionDelay}}$ is defined in Table 8-36.

C.1.5.2.3. Maximum Exit Latency t_3 (t_{MEL3})

The t_{MEL3} delay is the time for the device to receive the PING and generate the PING_RESPONSE, which is defined by $t_{\text{PingResponse}}$. Refer to Table 8-36.

C.1.5.2.4. Maximum Exit Latency t_4 (tMEL4)

The tMEL4 delay is the time for the PING_RESPONSE to traverse the interconnect hierarchy from the device to the host. Since wHubDelay defines the downstream and upstream delay through a hub, the propagation delay of a PING_RESPONSE upstream is identical to that of a PING downstream delay (tMEL2), with one exception. In the upstream path a TP may be queued behind a Max Packet Size DP, so an additional 2.1 μs of delay is included to comprehend the “congestion jitter”. tMEL4 is calculated as:

$$t_{\text{MEL4}} = t_{\text{MEL2}} + 2.1 \mu\text{s}.$$

C.2. Calculating U1 and U2 End to End Exit Latencies

This section provides examples of how to calculate the exit latency (i.e., time to transition from a non-U0 state to a U0 state) spanning the end to end path between a device and the host. Examples are given for both device initiated exit and host initiated exit.

Figure C.3 depicts a SuperSpeed hierarchy and calls out the relevant parameters used in the calculations that follow.

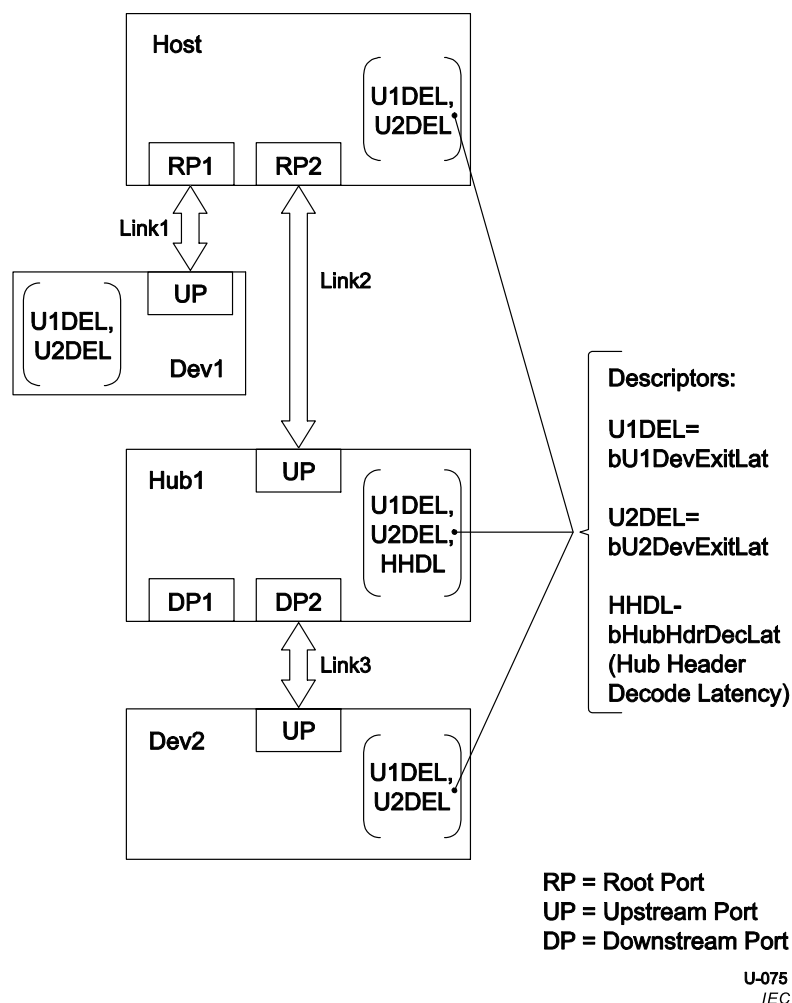


Figure C.3 – Host to Device Path Exit Latency Calculation Examples

C.2.1. Device Connected Directly to Host

C.2.1.1. Host Initiated Transition

In this example, a peripheral device (Dev1) and a host controller root port (RP1) are link partners communicating over a link (Link1).

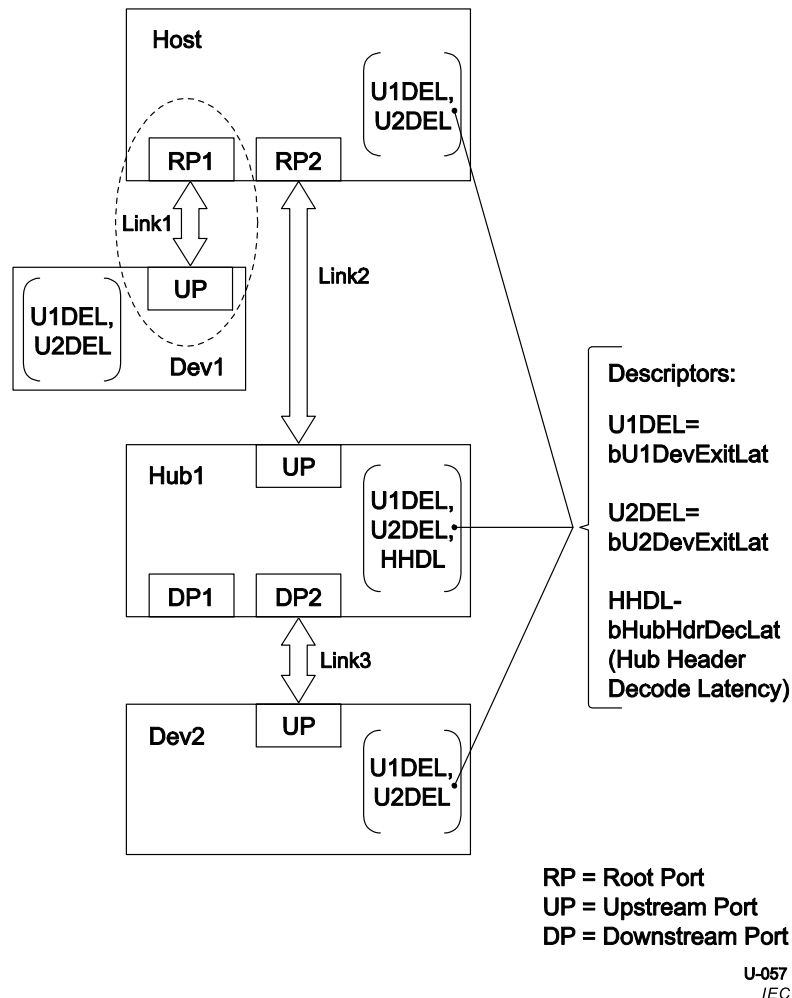


Figure C.4 – Device Connected Directly to a Host

U1 → U0 Transition Latency

The host initiates the transition by transmitting LFPS. At this point, both link partners' transitions from U1 → to U0 are executed in parallel.

The exit latency is characterized by the largest of the two device exit latencies: Dev1:U1DEL and RP1:U1DEL

U2 → U0 Transition Latency

In this example it is assumed that at least one of the link partners (e.g., the RP1) is enabled for U2. The host initiates the transition by transmitting LFPS. At this point both link partners execute transition from U2 → to U0 in parallel.

The exit latency is characterized by the largest of the two device exit latencies: Dev1:U2DEL and RP1:U2DEL

C.2.1.2. Device Initiated Transition

These transition latencies are the same as for the host initiated cases.

- U1 End to End Exit Latency is **the larger of Dev1:U1DEL and RP1:U1DEL**
- U2 End to End Exit Latency is **the larger of Dev1:U2DEL and RP1:U2DEL**

C.2.2. Device Connected Through a Hub

In this example a peripheral device (Dev2) is connected to one of a hub's downstream ports (DP2) via a link (Link3), which in turn is connected to a host controller's root port (RP2) via a link (Link2).

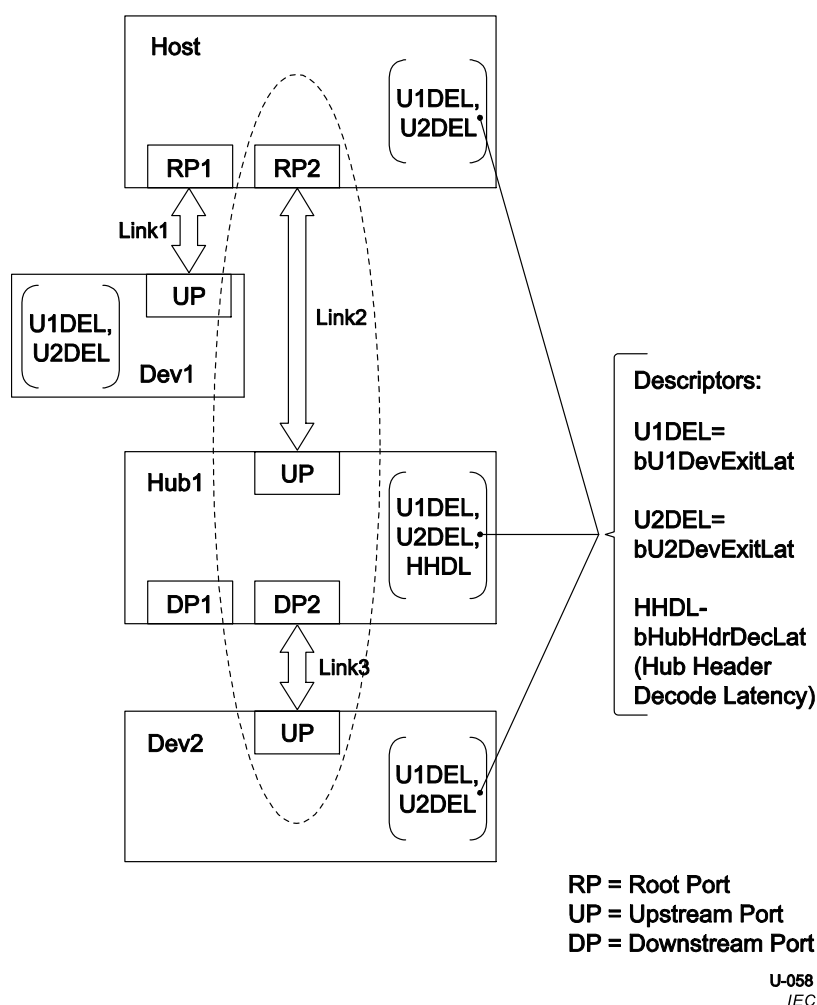


Figure C.5 – Device Connected Through a Hub

C.2.2.1. Host Initiated Transition

U1 → U0 Transition Latency

This example highlights the end to end latency incurred when transitioning both Link2 and Link3 from U1 → U0. For the purposes of this example, it is assumed that all link partners are enabled for U1, and that both Link2 and Link3 are currently in the U1 state.

Figure C.6 shows the chronological sequence of events. Following the figure is a more detailed description of each stage of the multi-hop link state transition.

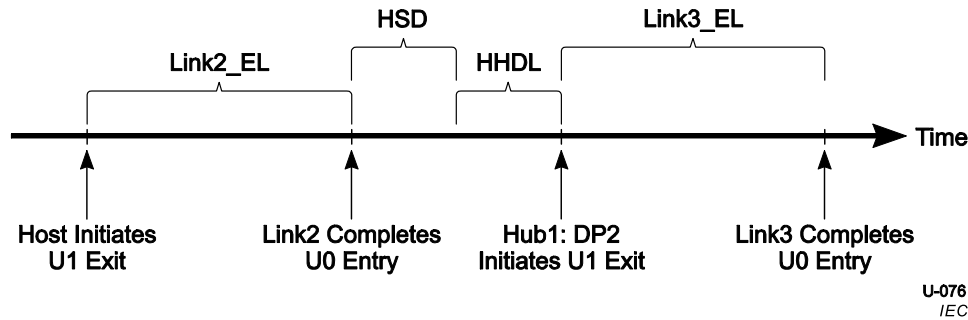


Figure C.6 – Downstream Host to Device Path Exit Latency with Hub

- 1) The host is prepared to send a packet to Dev2, however it first needs to bring Link2 out of U1 before it is able to send the packet. The host begins the process by transmitting LFPS on RP2 to Hub1's upstream port (UP) which then starts both link partners transitioning in parallel towards U0. This latency is characterized by the larger of **RP2:U1DEL** and **Hub1:U1DEL**.
- 2) Once the Link2 partners are in U0, the host schedules the packet targeting Dev2. After a Host Scheduling Delay (HSD) the packet is then sent over Link2 where it then is routed to Link3. This routing incurs the latency associated with the Hub having to parse the packet header to determine the target downstream port for the packet. This latency is characterized by the hub parameter **HHDL**.
- 3) The final hop requires Hub1:DP2 to signal LFPS over Link3 at which point the final component of the end to end latency is executed by the Link3 partners in parallel. This final ingredient to the end to end latency is characterized by the larger of **Hub1:U1DEL** and **Dev2_UP:U1DEL**.

The total latency for end to end link transition to U0 can be summarized as:

$$\text{Max}(\text{RP2:U1DEL}, \text{Hub1:U1DEL}) + \text{HSD} + \text{HUB1:HHDL} + \text{Max}(\text{Hub1:U1DEL},$$

U2 → U0 Transition Latency

This example highlights the end to end latency incurred when transitioning both Link2 and Link3 from U2 → U0. For the purposes of this example it is assumed that all link partners are enabled for U2 and that both Link2 and Link3 are currently in the U2 state.

- 1) The host is prepared to send a packet to Dev2, however it first needs to bring Link2 out of U2 before it is able to send the packet. The host begins the process by transmitting LFPS to Hub1:UP which then starts both link partners transitioning in parallel towards U0. This latency is characterized by the larger of **RP2:U2DEL** and **Hub1:U2DEL**.
- 2) Once the Link2 partners are in U0 the host sends its packet targeting Dev2 over Link2 where it then needs to be routed to Link3. This incurs the latency associated with the Hub having to parse the packet header to determine the target downstream port for the packet. This latency is characterized by the hub parameter **HHDL**.
- 3) The final hop requires Hub1:DP2 to signal LFPS over Link3 at which point the final component of the end to end latency is executed by the Link3 partners in parallel. This final ingredient to the end to end latency is characterized by the larger of **Hub1:U2DEL** and **Dev2_UP:U2DEL**.

The total exit latency for end to end link transition to U0 can be summarized as:

$$\text{Max}(\text{RP2:U2DEL}, \text{Hub1:U2DEL}) + \text{HSD} + \text{Hub1:HHDL} + \text{Max}(\text{Hub1:U2DEL}, \text{Dev2_UP:U2DEL})$$

C.2.2.2. Device Initiated Transition

This section provides some examples for calculating end to end exit latencies for device initiated exit.

U1 → U0 Transition Latency

This example highlights the end to end latency incurred when transitioning both Link2 and Link3 from U1 → U0. For the purposes of this example it is assumed that all link partners are enabled for U1 and that both Link2 and Link3 are currently in the U1 state.

Figure C.7 depicts the end to end link state transition in chronological order. Following the figure a more detailed description of the sequence is provided.

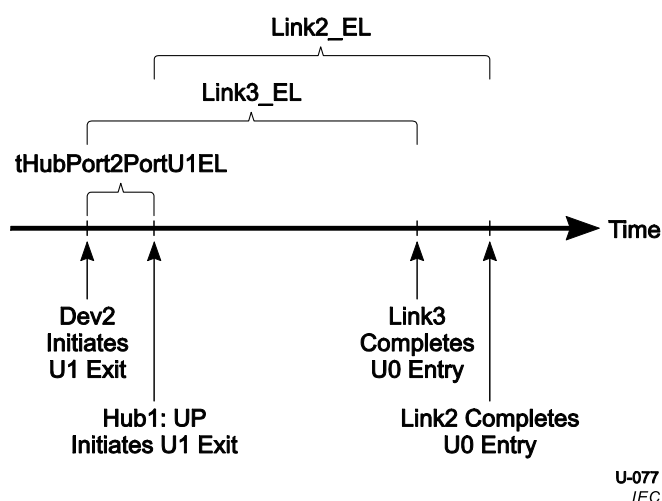


Figure C.7 – Upstream Device to Host Path Exit Latency with Hub

- 1) Dev2 is prepared to send a packet upstream to the host. However, it first needs to bring Link3 out of U1 and back to U0 before it is able to send the packet. Dev2 begins the process by transmitting LFPS to Hub1:DP2 which then starts both link partners transitioning in parallel towards U0. The Link3 exit latency (**Link3_EL**) is characterized by the larger of **Dev2_UP:U1DEL** and **Hub1_DP2:U1DEL**.
- 2) After a latency of **tHubPort2PortU1EL**, the time it takes the hub to determine that one of its downstream ports is awakening, the hub then begins signaling LFPS on Link2 to initiate transition of the Link2 partners (hub's upstream port and host controller root port RP2) to U0.
- 3) The last component of the end to end exit latency is characterized by the larger of **Hub1_UP:U1DEL** and **RP2:U1DEL** which represents the Link2 exit latency (**Link2_EL**).

End to end exit latency = Max(Link3_EL, (Link2_EL + tHubPort2PortU1ExitLat))

U2 → U0 Transition Latency

This example highlights the end to end latency incurred when transitioning both Link2 and Link3 from U2 → U0. For the purposes of this example it is assumed that all link partners are enabled for U2 and that both Link2 and Link3 are currently in the U2 state.

- 1) Dev2 is prepared to send a packet upstream to the host. However, it first needs to bring Link3 out of U2 and back to U0 before it is able to send the packet. Dev2 begins the process by transmitting LFPS to Hub1:DP2 which then starts both link partners transitioning in parallel towards U0. The Link3 exit latency (**Link3_EL**) is characterized by the larger of **Dev2_UP:U2DEL** and **Hu1b_DP2:U2DEL**.

- 2) After a latency of ***tHubPort2PortU2EL***, the time it takes the hub to determine that one of its downstream ports is awakening, the hub then begins signaling LFPS on Link2 to initiate transition of the Link2 partners (hub's upstream port and host controller root port RP2) to U0.
- 3) The last component of the end to end exit latency is characterized by the larger of ***Hub1_UP:U2DEL*** and ***RP2:U2DEL*** which represents the Link2 exit latency (***Link2_EL***).

End to end exit latency = Max(Link3_EL, Link2_EL + tHubPort2PortU2EL)

C.3. Device-Initiated Link Power Management Policies

Power savings resulting from the effective use of link power management can have a significant impact on system power consumption. For example, without using link power management, the average battery life of a typical notebook computer could be decreased by as much as 15 %.

Both devices and downstream ports can initiate U1 and U2 entry.

- **Downstream ports** have inactivity timers used to initiate U1 and U2 entry. Downstream port inactivity timeouts are programmed by system software.
- **Devices** may have additional information available that they can use to decide to initiate U1 or U2 entry more aggressively than inactivity timers.

This section describes policies for devices to initiate U1 or U2.

C.3.1. Overview and Background Information

Devices can save significant power by initiating U1 or U2 more aggressively rather than waiting for downstream port inactivity timeouts. For example, an isochronous device may substantially increase U1 residency by initiating U1 upon completion of isochronous transfers within each service interval.

Devices can use the following information to help determine when to initiate U1 or U2 due to endpoint idle conditions:

- The type of device endpoints and related flags (refer to Chapter 8)
 - Packets Pending flag, used with bulk endpoints
 - End of Burst flag, used with interrupt endpoints
 - Last Packet flag, used with isochronous endpoints
- Protocol level endpoint flow control conditions, e.g., an endpoint having sent an NRDY
- Device class and device implementation
- U1 and U2 device-to-host exit latencies

U1 and U2 device-to-host exit latencies are the total latency to transition all links in the path between the device and the host to U0, when exit is initiated by the device. The device may assume a device-to-host exit latency based on the worst case exit latency (device connected five hubs deep). The device may alternatively use the device-to-host exit latency provided with the U1PEL and U2PEL fields of the SET_SEL request (refer to Chapter 9).

C.3.2. Entry Conditions for U1 and U2

A device should initiate U1 or U2 when idle conditions are met for all its endpoints. A device typically initiates U1. However, if a device is able to determine that its link will not be needed for a long time, then the device may be able to initiate U2. For example, WiFi has a protocol

where its radio may be shut off for long periods, e.g., 100 ms, and since the link is not needed during this time it may be placed in U2.

A device should initiate U2 if it is able to determine its link is not needed for a period of time that exceeds the U2 device-to-host exit latency (plus an appropriate guard band). The device should initiate U1 in all other cases.

Devices should consider the device-to-host exit latency when determining whether to initiate U1 and U2 entry. The host-to-device exit latency and the device-to-host exit latency are both considered by host software when determining whether to enable U1 or U2 on each link. Devices are enabled to initiate U1 and U2 with the U1_Enable and U2_Enable feature selectors (refer to Chapter 9).

The following subsections offer recommendations for determining when an endpoint is idle, or does not need to use the link for a known period, based on endpoint type. Idle conditions may be determined in other implementation specific ways.

C.3.2.1. Control Endpoints

A control endpoint is idle when all of the following conditions are met:

- Device is in the configured state
- Device is not in the midst of a control transfer
- Either an NRDY was sent, or the Packets Pending flag was set to zero in the last ACK packet received from the host
- Device does not have a pending ERDY

C.3.2.2. Bulk Endpoints

A bulk endpoint is idle when both of the following conditions are met:

- Either an NRDY was sent, or the Packets Pending flag was set to zero in the last ACK packet received from the host
- Device does not have a pending ERDY

Some devices can also determine that their link is not needed for a known period of time. For example, a mass storage device may need to spin up a spindle to service a request. Since the spin up time can be hundreds of milliseconds, the device should place its link in U2.

After a device has sent an ERDY associated with a bulk endpoint, the link should be kept in U0 until the host sends a request in response to the ERDY (or until the tERDYTimeout occurs, refer to Section 8.13).

C.3.2.3. Interrupt Endpoints

An interrupt endpoint is idle when both of the following conditions are met:

- Either an NRDY was sent, or the Packets Pending flag was set to zero in the last ACK packet received from the host.
- Device does not have a pending ERDY.

After a device has sent an ERDY associated with an interrupt endpoint, the link should be kept in U0 until the host sends a request in response to the ERDY (or until the tERDYTimeout occurs) in order to achieve the subscribed interrupt service latency. However, when all transfers for a given service interval have been completed, the endpoint will not need the link until the next service interval. The device may be able to place its link in U1 or U2 during this

time. The End of Burst flag can be used to determine when all transfers for a given service interval are completed. Note that hosts are required to initiate interrupt transfers far enough ahead of a transfer window to meet subscribed service requirements.

C.3.2.4. Isochronous Endpoints

An isochronous endpoint is idle when all transfers for a given service interval have been completed, as indicated by the Last Packet flag. The endpoint will not need the link until the next service interval. Note that the host is required to send a PING packet far enough ahead of a transfer window to meet subscribed service requirements.

C.3.2.5. Devices That Need Timestamp Packets

When a device needs timestamp information, it needs to ensure that its link is in U0 when the next bus interval boundary is reached in order to receive a timestamp packet. If the device's link is not in U0, it should transition to U0 prior to the next bus interval boundary. The device must track when the bus interval boundary will occur. The device initiates a transition to U0 a period of time before the bus interval boundary occurs, where the period of time is the device-to-host link exit latency.

C.4. Latency Tolerance Message (LTM) Implementation Example

Computer systems typically maintain a high state of readiness to service devices even when the computer system is idle. LTM supports a mechanism for a system to reduce its state of readiness with the cooperation of Enhanced SuperSpeed devices. This may result in substantial system power savings without requiring additional cost to devices.

This section provides a device implementation example for LTM support. This example is based on a model using two device Latency Tolerance states, an active state and an idle state. Each state has a different Best Effort Latency Tolerance (BELT).

In the following subsections, first a description of BELT and its relationship to overall system exit latency is given. This is followed by a description of a device state machine implementation example.

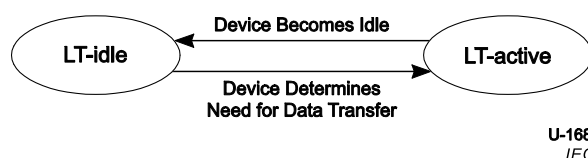
C.4.1. Device State Machine Implementation Example

This section describes an example of a typical device implementation. It assumes the device implementation supports both U1 and U2 in conjunction with LTM.

In this example, two device Latency Tolerance states (LT-states) are defined:

- **LT-idle state:** the device is idle and can tolerate a larger latency from the system (this is the default state).
- **LT-active state:** the device has determined a need to perform data transfers with the host and wants a shorter latency from the system.

A state machine is illustrated in Figure C.8.



U-168
IEC

Figure C.8 – LT State Diagram

The device described by this implementation example is designed to accommodate the worst case value for U1SEL during LT-active, and the worst case value for U2SEL during LT-idle.

The following device design goals are to be met:

- Design for a minimum LTM BELT of 1 ms when in LT-idle
- Design for a minimum LTM BELT of 125 μ s when in LT-active

C.4.1.1. LTM-Idle State BELT

The device determines its LT-idle state BELT value by subtracting U2SEL from the total latency it can tolerate. To achieve a minimum LT-idle state BELT of 1 ms, the total latency the device must be able to tolerate is 1 ms plus the worst case value for U2SEL, or a total of approximately 3,1 ms (refer to Section C.1.5.1). The worst case U2SEL is based on a worst case device-to-host U2 exit latency of 2,053 ms for t1 (2,047 ms device exit latency plus 1 μ s for each of five hubs), plus 0,003 ms for t2, plus 0,001 ms for t4, plus some guard band.

For system implementations where U2SEL is less than its worst case value, the device reports a BELT value larger than 1 ms.

C.4.1.2. LTM-Active State BELT

The device determines its LT-active state BELT value by subtracting U1SEL from the total latency it can tolerate. To achieve a minimum LT-active state BELT of 125 μ s, the total latency the device must be able to tolerate is 125 μ s plus the maximum value for U1SEL, or a total of approximately 145 μ s. The worst case U1SEL is based on a worst case device-to-host U1 exit latency of 15 μ s for t1 (10 μ s device exit latency plus 1 μ s for each of five hubs), plus 3,1 μ s for t2, plus 1,3 μ s for t4, plus some guard band. This assumes the device will not allow its link to enter U2 prior to changing its state to LT-idle. If the device will allow its link to enter U2 when in LT-active, then the total latency the device must be able to tolerate is 125 μ s plus the worst case value for U2SEL.

For system implementations where U1SEL is less than its worst case value, the device reports a BELT value larger than 125 μ s.

C.4.1.3. Transitioning Between LT-States

When a device transitions between LT-states, the device sends an LTM Transaction Packet (TP) with an updated BELT. The device should send all BELT updates as soon as possible after a change in LT-state.

C.4.1.3.1. Transitioning From LT-idle to LT-active

Devices transition from LT-idle to LT-active when the device determines that a bulk or interrupt data transfer needs to occur. Some examples are given below:

- The host initiates a bulk OUT transfer with the Packets Pending flag asserted to a flash drive device. As a result of receiving this OUT request, the device transitions to LT-active and sends an updated BELT to the host.
- The host initiates a bulk IN transfer with the Packets Pending flag asserted to a hard disk drive device with its spindle currently spun down and the requested data not in a cache on the hard disk drive. As a result of receiving this IN request, the device determines that a bulk data transfer has been initiated by the host. However, the device will service the IN request after its spindle spins up, which may take substantially longer than the last reported BELT. The device may delay transitioning to LT-active. When the device

determines that the spindle will complete its spin up within the last reported BELT, the device transitions to LT-active and sends an updated BELT to the host.

- A Network Interface Controller device begins receiving data on its network interface that requires a bulk IN data transfer with the host. As a result of receiving data on its network interface, the device transitions to LT-active, begins to transition its link to U0 (if not already in U0), and sends both an ERDY and an updated BELT to the host.
- A multi-touch Human Interface Device is set up with an interrupt endpoint. When human input is detected the device transitions to LT-active, begins transitioning its link to U0 (if not already in U0), and sends both an ERDY and an updated BELT to the host.

In some cases, a transition from LT-idle to LT-active is not appropriate even though the device needs to transmit to the host. For example:

- The host sends a GetStatus request to a device. Since it is a control transfer and not a bulk or an interrupt transfer, the device remains in the LT-idle state.

When the device transitions from LT-idle to LT-active, the device sends an LTM TP with a BELT of at least tBELTmin.

C.4.1.3.2. Transitioning From LT-active to LT-idle

When a device determines that it is idle, it transitions from LT-active to LT-idle. The method used in this example for device idle detection is based on U2 entry. When the device is in LT-active, the device transitions to LT-idle just prior to when its link will enter U2.

In preparation for a transition from U0 to U2, a device in LT-active should perform the following actions:

- 1) The device transitions to LT-idle.
- 2) The device sends an LTM with a BELT value of at least tBELTdefault.
- 3) The device initiates a transition to U2 from U0.

In preparation for a transition from U1 to U2, a device in LT-active should perform the following actions:

- 1) The device transitions its link to U0 prior to U2 entry.
- 2) The device transitions to LT-idle.
- 3) The device sends an LTM with a BELT value of at least tBELTdefault.
- 4) The device initiates a transition to U2 from U0.

In the latter case, since the device must transition its link to U0 prior to U2 entry, the device must detect the U2 inactivity timer expiration enough in advance to avoid the possibility that its link partner will have already transitioned from U1 directly to U2. For example, the device may initiate a transition to U0 1 μ s before the U2 inactivity timer expires.

C.4.2. Other Considerations

The following are additional considerations associated with device support of LTM:

- The BELT represents a latency tolerance for an entire peripheral device. The BELT value must be aggregated across all endpoints within the device, including all functions within a composite device. The smallest BELT value across all endpoints should be selected. For LTM purposes, isochronous endpoints are ignored when determining the BELT value.
- If LTM is supported by a device, LTM should be disabled prior to placing the device into suspend (refer to the PORT_LINK_STATE feature selector in Chapter 10). Devices send an updated LTM when LTM is enabled, or immediately before LTM is disabled, as defined in Chapter 8. Disabling LTM in this way ensures that a suspended device does not keep the system in a high state of readiness, wasting power.

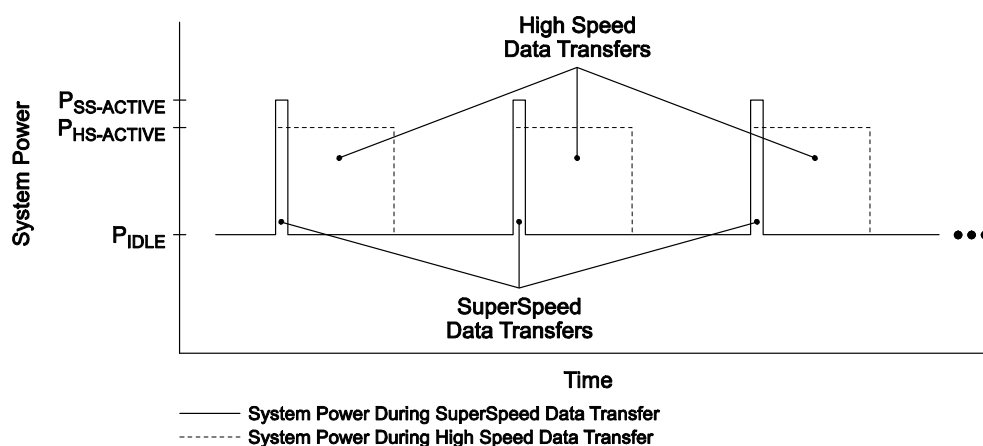
C.5. SuperSpeed vs. High Speed Power Management Considerations

Some devices may operate well with a High Speed (480 Mbps) interface, but can substantially reduce system power consumption if implemented with a SuperSpeed interface. In addition to device power consumption, system power consumption should be considered when selecting the interface for a new device design.

When a device is actively transferring data, system components are also transferring that data. For some systems, the power consumption of system components is much larger than a USB device's contribution to the system's power consumption.

Under typical circumstances, the faster the data transfer completes, the faster system components can return to a low power state. Transferring data faster can save power, on average, over time. Examples of system components include a Host Controller, a DRAM controller, DRAM components, a microprocessor with a cache that needs to snoop DRAM accesses, etc.

Figure C.9 illustrates a sample device that has an average data transfer rate of 20 MBps when actively in use. The figure shows the system power consumption when the device is operating in SuperSpeed mode and also in High Speed mode.



U-169
IEC

Figure C.9 – System Power during SuperSpeed and High Speed Device Data Transfers

When no data transfer is taking place the system power consumption is P_{IDLE} . P_{IDLE} is approximated to be the same in both SuperSpeed and High Speed modes. Link power management considerations are ignored for simplicity of illustration.

When a data transfer is taking place, the system power is $P_{SS-ACTIVE}$ and $P_{HS-ACTIVE}$ for SuperSpeed and High Speed modes respectively. The difference between $P_{SS-ACTIVE}$ and $P_{HS-ACTIVE}$ is due to the physical layer interface power of the device and its link partner (no hubs present).

Data transfers complete roughly ten times faster in SuperSpeed mode than in High Speed mode. This causes the average system power in High Speed mode to be much larger than the average system power in SuperSpeed mode. The difference in average system power may be as high as 50 % during a data transfer. This can have a major impact on the battery life of mobile systems.

D Example Packets

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Device Address 000_0001b								Reserved 0000_0000_0000_0000_0000b																		Type 00100b					
Reserved 000_0000_0000b										NumP 00001b				Reserved 0000b				Ept Num 0000b			D 0	Rsvd 000b			SubType 0011b						
Reserved 00000b				Reserved 000_0000_0000_0000_0000_0000b																											
Link Control Word 11110b					0	0	001b			000b			010b			CRC-16 10001111b								01001110b							

U-164
IEC

Figure D.1 – Sample ERDY Transaction Packet

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0																																					
Device Address 000_0100b								Routing String 0000_0000_0000_0010_0001b																Type 01000b													
Data Length 0000_0000_0000_1001b																S 0		Rsvd 000b		Ept Num 0001b				D 0		EOB 1		R 0		Seq Num 00010b							
Rsvd 0000b		PP 1		Reserved 000_0000_0000_0000_0000_0000b																																	
Link Control Word 01011b												0		0		010b		000b		001b		CRC-16 00011000b								11101101b							
Byte 3 0111_0110b								Byte 2 0101_0100b								Byte 1 0011_0010b								Byte 0 0001_0000b													
Byte 7 1111_1110b								Byte 6 1101_1100b								Byte 5 1011_1010b								Byte 4 1001_1000b													
0110_0010b								CRC32 1111_0011b								1000_1011b								Byte 8 0001_0000b													
																								CRC32 0000_0111b													

U-165
IEC

Figure D.2 – Sample Data Packet

128b/132b Control block								
Block Header 1100 Block #1	0th SKP	1st SKP	2nd SKP	3rd SKP	4th SKP	5th SKP	6th SKP	7th SKP
	8th SKP	9th SKP	10th SKP	11th SKP	12th SKPEND	13th Scrambler Seed	14th Scrambler Seed	15th Scrambler Seed
128b/132b Data block								
Block Header 0011 Block #2	0th IS	1st SLC	2nd SLC	3rd SLC	4th EPC	5th Link Command Word	6th Link Command Word	7th Link Command Word
	8th Link Command Word	9th IS	10th IS	11th SHP	12th SHP	13th SHP	14th EPF	15th Header
128b/132b Data block								
Block Header 0011 Block #3	0th Header	1st Header	2nd Header	3rd Header	4th Header	5th Header	6th Header	7th Header
	8th Header	9th Header	10th Header	11th CRC	12th CRC	13th Link Command Word	14th Link Command Word	15th IS
128b/132b Data block								
Block Header 0011 Block #4	0th IS	1st SLC	2nd SLC	3rd SLC	4th EPC	5th Link Command Word	6th Link Command Word	7th Link Command Word
	8th Link Command Word	9th IS	10th IS	11th IS	12th IS	13th IS	14th IS	15th IS

IEC

Figure D.3 – Example placement of Gen 2 SKP Block, Idle Symbols, Link Command and Header Packet

128b/132b Data block								
Block Header 0011	0th IS	1st IS	2nd IS	3rd DHPH	4th DHPH	5th DHPH	6th EPF	7th DHPH
Block #1	8th DHPH	9th DHPH	10th DHPH	11th DHPH	12th DHPH	13th DHPH	14th DHPH	15th DHPH
128b/132b Data block								
Block Header 0011	0th DHPH	1st DHPH	2nd DHPH	3rd CRC	4th CRC	5th Link Command Word	6th Link Command Word	7th Length Field Replica
Block #2	8th Length Field Replica	9th Length Field Replica	10th Length Field Replica	11th SDP	12th SDP	13th SDP	14th EPF	15th Data
128b/132b Data block								
Block Header 0011	0th Data	1st Data	2nd Data	3rd Data	4th Data	5th CRC	6th CRC	7th CRC
Block #3	8th CRC	9th END	10th END	11th END	12th EPF	13th IS	14th IS	15th DHPH
128b/132b Data block								
Block Header 0011	0th DHPH	1st DHPH	2nd EPF	3rd DHPH	4th DHPH	5th DHPH	6th DHPH	7th DHPH
Block #4	8th DHPH	9th DHPH	10th DHPH	11th DHPH	12th DHPH	13th DHPH	14th DHPH	15th CRC
128b/132b Data block								
Block Header 0011	0th CRC	1st Link Command Word	2nd Link Command Word	3rd Length Field Replica	4th Length Field Replica	5th Length Field Replica	6th Length Field Replica	7th SDP
Block #5	8th SDP	9th SDP	10th EPF	11th CRC	12th CRC	13th CRC	14th CRC	15th END
128b/132b Data block								
Block Header 0011	0th END	1st END	2nd EPF	3rd IS	4th IS	5th IS	6th IS	7th IS
Block #6	8th IS	9th IS	10th IS	11th IS	12th IS	13th IS	14th IS	15th IS

IEC

Figure D.4 – Example placement of Gen 2 Data Packets and Idle Symbols

E Repeaters

E.1. Overview

E.1.1. Definitions

In this document, the following definitions apply:

Repeater refers to any active component that acts on a signal in order to increase the physical lengths and/or interconnect loss over which the signal can be transmitted successfully. The category of repeaters includes both retimers and re-drivers, which are defined below.

Retimer refers to a component that contains a clock-data recovery (CDR) circuit that “retimes” the signal. The retimer latches the signal into a synchronous memory element before re-transmitting it. It is used to extend the physical length of the system without accumulating high frequency jitter by creating separate clock domains on either side of the retimer.

Re-driver refers to an analog component that operates on the signal without re-timing it. This may include equalization, amplification, and transmitter. The re-driver does not include a CDR.

Device captive retimer refers to a retimer that is located on the same printed circuit as the device silicon. The retimer is said to be associated with the device.

Host captive retimer refers to a retimer that is located on the same printed circuit as the host silicon. In this case, the retimer is said to be associated with the host.

Link segment refers to a transmitter-channel-receiver combination between a downstream port and a retimer, and upstream port and a retimer, or between two retimers. This is demonstrated in Figure E.1.

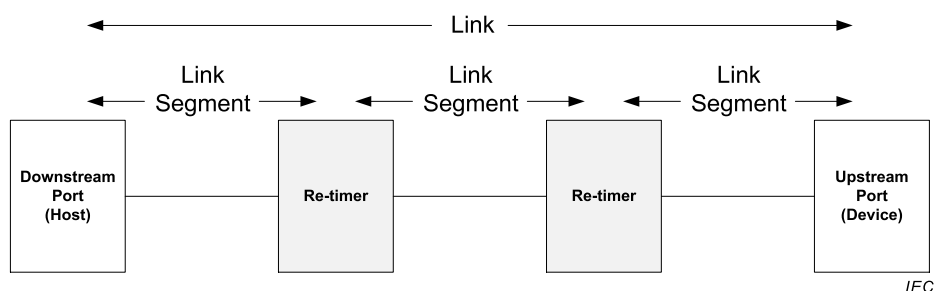


Figure E.1 – Link segment definition

E.1.2. Scope

E.1.2.1. Retimers

This appendix defines the requirements for CDR-based (re-timing) repeaters, a.k.a. retimers, and includes requirements for both the physical and link layers. A retimer may be used on a printed circuit board in conjunction with a host or device, or as part of a cable assembly.

The USB usage model, which matches hosts, devices and cables at the time of usage, allows for the construction of systems that include retimers on all three components. The requirements set forth in this appendix comprehend the use of up to four retimers in a system.

Figure E.2 depicts examples of USB systems that employ repeaters. Note that the figure does not depict all allowed configurations.

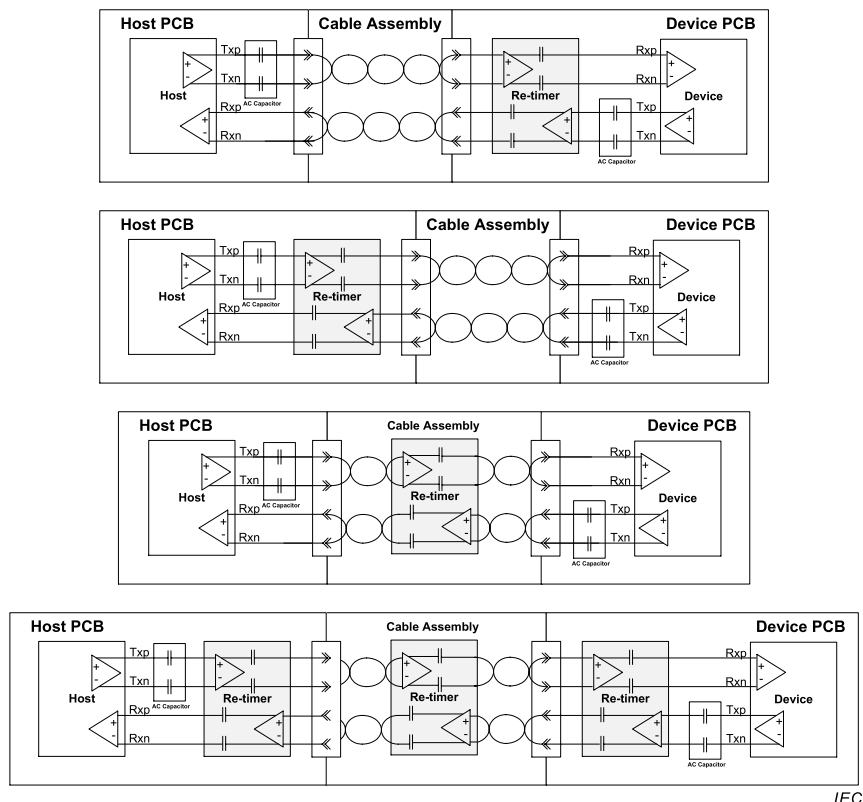


Figure E.2 – Retimer implementation examples

E.1.2.2. Re-drivers

Analog re-drivers are beyond the scope of this document.

E.2. Retimer Architectural Overview and Requirement

The primary motivation of a retimer is to relay the communications by re-generating the signal to the quality matching the transmitter requirement defined in Chapter 6. To achieve this, the incoming data at the receiver is recovered and re-transmitted by the retimer with the transmit clock generated based on a local reference clock. Given this basic retimer operation, some retimer specific architectural considerations and requirements need to be addressed and are covered by this section.

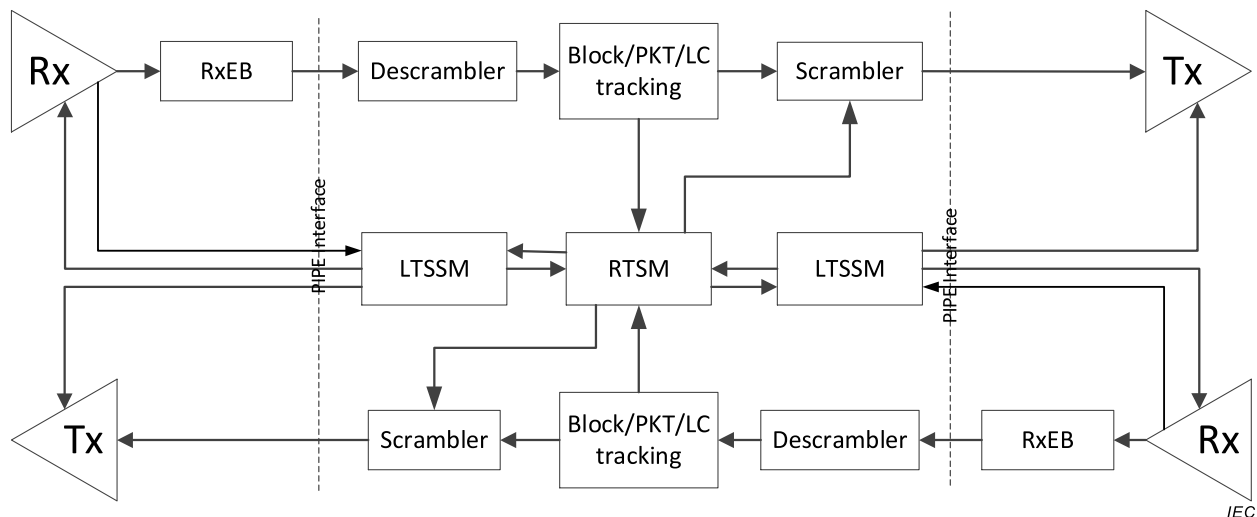


Figure E.3 – Example high level retimer architecture in Gen 2 operation

E.2.1. Architectural Overview

Shown in Figure E.3 is a high level retimer architectural example for Gen 2 operation. Although the data path architecture from a receiver to its transmitter for Gen 2 operation maybe different from Gen 1 operation, the architectural framework between Gen 1 operation and Gen 2 operation is the same. Refer to Figure E.7 for a retimer architectural example for Gen 1 operation.

Each retimer consists of a downstream port and an upstream port to perform clock data recovery at its receivers and data transmission at its transmitters. Each port has its own LTSSM to manage the operation in various link states, and both LTSSMs are very similar to LTSSM defined in chapter 7, but with differences unique to retimer operations that are described in Section E.2.3. A retimer state machine (RTSM) is employed to coordinate the operation between its upstream port and downstream port. In addition, the RTSM is also responsible to serve the following management functions:

- Link state detection and retimer power management.
- Data path management and clock offset compensation.
- Packet/link command detection, error detection and correction (if applicable).

E.2.2. General requirements

A retimer shall include the capability to successfully train and operate in both SuperSpeed Gen 1 and Gen 2 modes. This includes implementation of all training and power states defined in Chapters 6 and 7.

The timing budgets, signal levels, and compliance specifications defined in Chapter 6 apply to retimers. As with SuperSpeed and SuperSpeedPlus hosts and devices, retimers shall implement spread spectrum clocking (SSC) in compliance with the requirements contained in Section 6.5.3 and 6.5.4 of the base specification.

E.2.2.1. Physical Layer Requirements

Retimers shall meet the transmitter requirements defined in Section 6.7 including:

- the normative requirements defined in Table 6-17;
- the normative eye mask requirements defined in Table 6-19.

The informative transmitter requirements defined in Table 6-18 also apply to retimers. Table 6-15 in Section 6.5 defines transmitter jitter targets which apply to retimers.

The transmitter shall implement the low frequency periodic signaling (LFPS) and LFPS-based pulse modulation (LBPM) features defined in Section 6.9, meeting all requirements defined in that section.

E.2.2.2. Link Layer Requirements

The purpose for retimers to implement partial link layer function includes the following.

- TS2 ordered set decoding. A retimer shall decode the link configuration field to determine the link operation during Polling.Idle or Recovery.Idle. Refer to Section E.2.3 for retimer specific link configuration and operation.
- Link command tracking. A retimer shall track the link command for link power management.
- IS and DPP tracking. A retimer operating at SuperSpeed shall track IS and DPP for the clock offset compensation. Refer to Section E.4.1 for details.

Some additional rule of the link layer operation may apply.

- A retimer shall correct any single symbol error detected in any framing order sets when forwarding.
- A retimer may optionally correct the link command that contains one invalid link command field replica.

E.2.3. Retimer Operation

The primary responsibility of a retimer shall include the following:

- Connectivity detect and monitoring. A retimer upon power-up shall perform the presence detect of its remote link partners, being a hub downstream port, or an upstream port, or a retimer port. This is achieved by performing far-end receiver termination detection periodically. A retimer shall also monitor the operation and be aware when its link partners may be disconnected or disabled.
- Participation of link initialization and training. A retimer, upon establishing connection with its link partners, shall participate the link initialization and training. This includes:
 - Speed negotiation: a retimer in Polling substates shall either forward or re-generate and monitor the Polling.LFPS signal and LBPMs in order to determine the data rate its link partners has agreed upon completion of speed negotiation, and configure itself towards the negotiated data rate.
 - Equalization training: a retimer shall transmit TSEQ OS based on its local reference clock to train the receivers of its link partners, while training its own receiver equalization.
 - Bit/symbol/block alignment and link configuration: a retimer shall perform bit/symbol/block alignment to achieve end to end link training and decode TS2 OS in order to determine what the link configuration is. This includes: U0, Hot Reset, Local Loopback, and PassThrough Loopback. Clock offset compensation shall be performed in these Polling substates. Refer to Section E.4 for details of clock offset compensations in SS and SSP operations.
- Traffic relay. A retimer in U0 shall relay the traffic in both directions. This includes eSS packets and IS. Clock offset compensation shall be performed accordingly in this state depending on the rate it is operating. Refer to Section E.4 for details of clock offset compensations in SS and SSP operations.
- LFPS relay. LFPS signal may be detected under all link states except eSS.Disabled. A retimer shall process and re-transmit the LFPS signals meeting the electrical and timing

requirements defined in Section 6.9. Refer to Section E.2 for special treatment of LFPS transmission under different link states.

- Link power management. While forwarding the traffic, the retimer shall track the link command to determine if link is preparing to enter a low power link state. When in a low power state, the retimer shall also monitor the connectivity of its link partners and prepare for link state transition.
- Error recovery. While the retimer may perform error correction at its inbound traffic (U0 only), such as single-bit block header error, single/double-bit SSP SKP symbol error, and etc., to preserve the quality of link operation, it shall leave the decision of entry to Recovery to the host and device.

E.3. Retimer Training and Status State Machine (RTSSM)

The RTSSM shown in Figure E.4 is similar to the LTSSM defined in Chapter 7. The main differences between RTSSM and LTSSM are summarized in the following.

- eSS.Inactive is combined with Rx.Detect as the behaviors of a retimer in those two states are the same. A retimer, even upon detecting the presence of its far-end link partners, will not transition to Polling until Polling.LFPS is received.
- eSS.Disabled is defined in the LTSSM for a self-powered peripheral device where it may operate at USB2 mode. The eSS operation is disabled and the link may enter the lowest power state. For retimer operation, unless it is aware of the link configuration, it will not be able to take the advantage of this state. However, it is kept for implementations capable of managing the retimer operation under proprietary mechanism, which is out of the scope of this specification.
- PassThrough Loopback is introduced for a retimer. It is a special operation pertained to a retimer only when the LTSSM is in Loopback. A retimer in this state passes the inbound traffic directly as if it is in U0.
- Local Loopback is named to contrast PassThrough Loopback. The purpose and the operation of a retimer in this state are the same as Loopback defined in LTSSM.



Figure E.4 – Retimer Training and Status State Machine

In the following subsections, the details of RTSSM are described. The operations that are common to LTSSM will not be elaborated.

It is worth mentioning that the main purpose of RTSSM is to describe and define the retimer operation under various LTSSM states. The external behavior of the RTSSM shall reflect that of the LTSSM. The implementation may vary regarding RTSSM optimization.

E.3.1. Rx.Detect

Rx.Detect is the default power-on state of the retimer. The retimer's responsibility is to detect the presence of its link partners by performing far-end receiver termination periodically and mirror the presence status of its link partners.

E.3.1.1. Rx.Detect Requirement

- The retimer's receiver terminations at both ports shall present high impedance to ground of $Z_{RX-HIGH-IMP-DC-POS}$ defined in Table 6-21. Note that a retimer may not have the knowledge to which port a host or a device is connected upon power-on.
- The retimer shall initiate a far-end receiver termination detection at both ports every 12 ms.
- The retimer, upon detecting far-end low-impedance receiver termination (R_{RX-DC}) defined in Table 6-21, shall enable its low-impedance receiver termination (R_{RX-DC}) at its respective port to mirror the presence of its link partner.
- The retimer shall forward Polling.LFPS it may receive in this state.

- The retimer shall forward and decode Warm Reset it may receive. The retimer shall ensure the duration of forwarded Warm Reset meeting the timing requirement defined in Section 6.9.3. Note that the retimer may also map its port receiving Warm Reset as USP, and its port transmitting Warm Reset as DSP.

E.3.1.2. Exit from Rx.Detect

- A retimer shall transition to Polling if Polling.LFPS is received.

E.3.2. eSS.Disabled

eSS.Disabled is an optional state where no eSS activity is enabled. The retimer is at its lowest possible power state. Note that there is no defined mechanism for entry to and exit from eSS.Disabled. It is implementation's responsibility to manage eSS.Disabled based on its capabilities.

E.3.2.1. eSS.Disabled Requirements

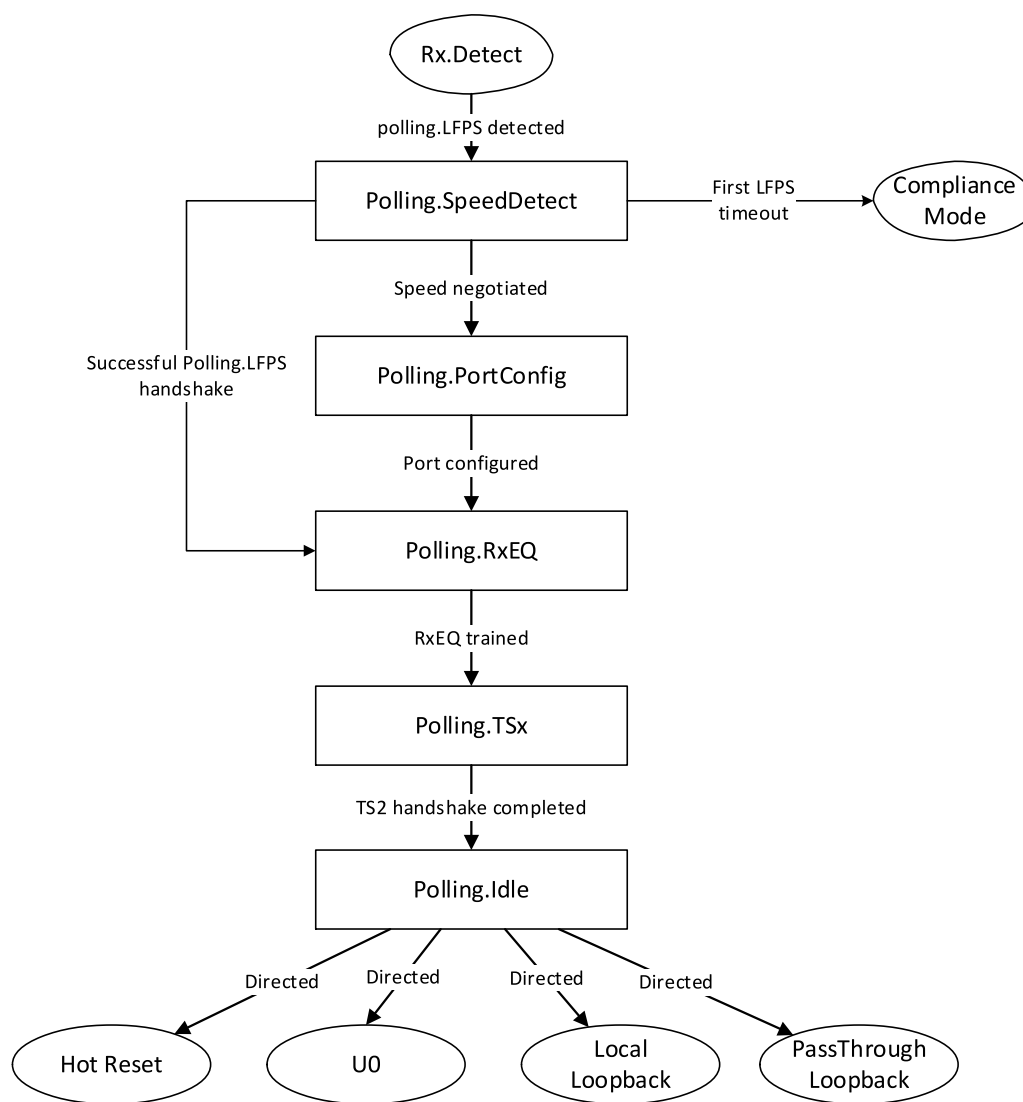
- The retimer shall present high impedance to ground of $Z_{RX-HIGH-IMP-DC-POS}$ defined in Table 6-21 at both ports.
- The retimer shall be in its lowest power state.

E.3.2.2. Exit from eSS.Disabled

- The retimer shall transition to Rx.Detect upon direction.

E.3.3. Polling

Polling is a state for retimer to participate in speed negotiation, link training and determine the state of operation upon exit from Polling. A simplified Polling substate machine of the retimer is shown in Figure E.5. Note that the transition to Rx.Detect due to Warm Reset is not shown.



IEC

Figure E.5 – Polling Substate Machine

E.3.3.1. Polling.SpeedDetect

Polling.SpeedDetect is a substate that covers the following substates in LTSSM.

- Polling.LFPS
- Polling.LFPSPlus
- Polling.PortMatch

The retimer's responsibility in this substate includes:

- Relay the LFPS based signals include Polling.LFPS, SCD1/SCD2, and LBPM.
- Decode the LFPS signal to determine the negotiated data rate.

E.3.3.1.1. Polling.SpeedDetect Requirements

- The retimer shall decode the following received LFPS signals and monitor the status and progression of the LTSSM.
 - Polling.LFPS

- SCD1/SCD2
- LBPM
- Warm Reset
- The retimer shall forward the LFPS signals meeting the electrical and timing requirements defined in Section 6.9. The retimer shall perform the following when forwarding the LFPS signals.
 - If it's Polling.LFPS or SCD1/SCD2, the retimer shall qualify between Polling.LFPS and SCD1/SCD2. The retimer may store before forward between four to eight Polling.LFPS to determine if it's Polling.LFPS with non-variant tRepeat, or SCD1/SCD2.
 - If it's Warm Reset, it shall ensure that the forwarded Warm Reset shall meet the timing specification defined in Section 6.9. The retimer shall start forwarding Warm Reset upon finding it is neither Polling.LFPS, nor SCD1/SCD2, nor LBPM. Note that the retimer may not be able to declare Warm Reset until the minimum duration of LFPS qualifies itself as Warm Reset. The retimer shall not wait until Warm Reset declaration before start the forwarding. Also note that the retimer shall follow similar approach when in other link states.
- The retimer shall start the tPollingLFPSTimeout timer upon receiving Polling.LFPS.
- The retimer shall start the tPollingLBPM LFPSTimeout timer upon observing LBPM.
- The retimer shall implement a tPollingSCDLFPSTimeout timer to monitor the absence of LFPS signal after the completion of SuperSpeed Polling.LFPS handshake.
- The retimer shall enable the transition path to Compliance Mode at its other port if only SCD2 is detected.

E.3.3.1.2. Exit from Polling.SpeedDetect

- Both ports of the retimer shall enter Compliance Mode upon timeout of the tPollingLFPSTimeout timer and the following three conditions are met.
 - SCD2 is received upon entry to the substate.
 - No Polling.LFPS handshake is achieved.
 - The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.
- The retimer shall transition to Polling.RxEQ for SS operation if the following two conditions are met.
 - Successful Polling.LFPS handshake is observed.
 - Upon timeout of the tPollingSCDLFPSTimeout timer.
- The retimer shall transition to Polling.PortConfig if it has observed successful LBPM handshake for port match.
- The retimer shall transition to Rx.Detect if one of the following conditions is met.
 - Warm Reset is detected.
 - The tPollingLBPM LFPSTimeout timer has expired.

E.3.3.2. Polling.PortConfig

Polling.PortConfig is a substate for the retimer to configure itself towards the negotiated data rate it has detected. The operation of the retimer is the same.

E.3.3.2.1. Polling.PortConfig Requirements

- The retimer shall not forward any PHYReady LBPMs before it is ready for SSP training.
- The retimer shall monitor and forward the received LBPMs until a successful handshake is reached. Note that the forwarded LBPMs shall meet the electrical and timing requirements defined in Section 6.9.

- The retimer shall start the tPollingLBPM LFPSTimeout timer upon entry to this substate.

E.3.3.2.2. Exit from Polling.PortConfig

- The retimer shall transition Polling.RxEQ if it has observed successful PHYReady LBPM handshake.
- The retimer shall transition to Rx.Detect if one of the following conditions is met.
 - Warm Reset is detected.
 - The tPollingLBPM LFPSTimeout timer has expired.

E.3.3.3. Polling.RxEQ

Polling.RxEQ is a substate for eSS receiver equalization training. The training mechanism is the same as defined in LTSSM except the exit.

E.3.3.3.1. Polling.RxEQ Requirements

- The lane polarity detection and correction for SS operation shall be enabled.
- The retimer shall generate TSEQ OS based on its own reference clock.
- The retimer shall not forward any TSEQ OS it has received.
- Upon completing the number of TSEQ OS transmission, the retimer shall continue the TSEQ OS transmission until TS1 OS at its other port has been recovered and ready to be forwarded.
- The retimer shall start a 12-ms timer upon entry to the substate.

E.3.3.3.2. Exit from Polling.RxEQ

- The retimer shall transition to Polling.TSx if all the following conditions are met.
 - Its receiver equalization is trained.
 - TS1 OS is received.
 - The operation of the retimer EB is established and the TS1 OS is ready to be forwarded.
- The retimer shall transition to Rx.Detect if Warm Reset is one of the following conditions is met:
 - Warm Reset is detected.
 - The expiration of the 12-ms timer and the transition conditions to Polling.TSx are not met.

E.3.3.4. Polling.TSx

Polling.TSx is a substate where the retimer tracks the TS1/TS2 OS to achieve end to end symbol/block alignment.

E.3.3.4.1. Polling.TSx Requirements

- The lane polarity detection and correction shall be completed.
- The retimer shall monitor the status and progression of the LTSSM.
- The retimer shall perform the clock offset compensation based on the following.
 - For SS operation, the retimer shall perform the clock offset compensation as defined in Section E.4.1.

- For SSP operation, the retimer shall perform the clock offset compensation as defined in Section 6.4.3.
- The retimer shall maintain synchronicity of the scrambler/descrambler operation with its link partners.
- The retimer shall maintain DC balance of its forwarded TS1/TS2 OS.
- The retimer shall not perform error correction within its received TS1/TS2 OS. This includes block headers in SSP operation.
- The retimer shall start the `tPollingActiveTimeout` timer upon entry to this substate.
- The retimer shall start the `tPollingConfigurationTimeout` timer upon observing TS2 OS at both ports.

E.3.3.4.2. Exit from Polling.TSx

- The retimer shall transition to `Polling.Idle` upon observing successful TS2 handshake.
- The retimer shall disable its eSS transceivers and remain in this substate if both of the following two conditions are met.
 - The `tPollingActiveTimeout` timer has expired for three consecutive times. Note this implies the retimer has remained in this substate for 36 ms.
 - No successful TS1 and/or TS2 OS handshakes have been observed.Note that this is an error state where retimer can remain until Warm Reset is received.
- The retimer shall transition to `Rx.Detect` if one of the following conditions is met.
 - Warm Reset is detected.
 - The `tPollingActiveTimeout` timer has expired.

E.3.3.5. The `tPollingConfigurationTimeout` timer has expired.Polling.Idle

`Polling.Idle` is a substate where the retimer decodes TS2 OS and decides the next operation state.

E.3.3.5.1. Polling.Idle Requirements

- The retimer shall decode the link configuration field in TS2 OS and configure itself to the corresponding operation state.
- The retimer in SS operation shall not delete any idle symbols for the overflow clock offset compensation.
- The retimer shall manage its scrambler operation according to the link configuration field in TS2 OS.
- The retimer shall start the `tPollingIdleTimeout` timer to monitor the progression of LTSSM.

E.3.3.5.2. Exit from Polling.Idle

- The retimer shall transition to `U0` if successful idle symbol handshake is observed.
- The retimer shall transition to `PassThrough Loopback` if the Loopback bit (bit 2 of the link configuration field) in the link configuration field is asserted.
- The retimer shall transition to `Local Loopback` as the loopback slave if the retimer loopback bit (bit 4 in the link configuration field) is asserted. Note that it is illegal to have both bit4 and bit 2 asserted. If both bits are asserted, the retimer shall give priority to `PassThrough Loopback`.
- The retimer shall transition to `Hot Reset` if the Reset bit is asserted.
- The retimer shall transition to `Rx.Detect` if one of the following conditions is met.

- Warm Reset is detected.
- The tPollingIdleTimeout timer has expired.

E.3.4. Compliance Mode

- The retimer operation in Compliance Mode is the same as LTSSM.

E.3.5. U0

U0 is the normal operational state where packets are forwarded by the retimer in both directions.

E.3.5.1. U0 Requirements

- The retimer shall monitor and decode the link command to participate retimer and link power management.
- The retimer shall monitor LMP based link initialization upon entry to U0 in order to determine its port orientation towards a hub downstream port or device upstream port.
- The retimer shall decode U2 Inactivity timeout LMP from host, when acknowledged by device with ACK.
- The retimer shall maintain its scrambler synchronization with its link partners.
- The retimer shall maintain DC balance on its outbound traffic.
- The retimer shall perform the clock offset compensation as defined in Section E.4.
- The retimer shall not initiate entry to Recovery due to bit errors at its inbound traffic.
- The retimer shall remain in this state even if the bit/symbol lock or block alignment is lost. Under this scenario, the retimer shall perform necessary bit/symbol lock and block alignment. The retimer shall transmit IS until inbound traffic is recovered, or entry to Recovery is detected.
- The retimer in SS operation shall perform single symbol error correction on any framing OS.
- The retimer in SSP operation shall perform error correction if the following bit errors are detected.
 - Single-bit block header error.
 - Single/double-bit SKP symbol error.
 - Single-symbol framing OS error.
 - Replica link command word error.

E.3.5.2. Exit from U0

- The retimer shall transition to U1 upon successful completion of LGO_U1 entry sequence. Refer to Section 7.2.4.2 for details.
- The retimer shall transition to U2 upon successful completion of LGO_U2 entry sequence. Refer to Section 7.2.4.2 for details.
- The retimer shall transition to U3 upon successful completion of LGO_U3 entry sequence. Refer to Section 7.2.4.2 for details.
- The retimer shall transition to Recovery upon observing TS1 OS.
- The retimer shall transition to Rx.Detect when Warm Reset is detected.

E.3.6. U1

U1 is a low power state where no packets are observed and the retimer is in a standby state.

E.3.6.1. U1 Requirements

- The retimer shall distinguish between Ping.LFPS, and U1 LFPS exit signal.
- The retimer, upon detecting an inbound LFPS signal that is not Ping.LFPS, shall initiate simultaneous U1 LFPS exit by performing the following.
 - It shall respond with U1_Exit LFPS handshake.
 - It shall forward the received LFPS signal outbound to its other link partner meeting the electrical and timing requirements defined in Section 6.9.
 - It shall monitor the duration of the received LFPS signal to determine if it is Warm Reset. The retimer may declare Warm Reset if the received LFPS signal is more than 2-ms.
- Note that the beginning of Warm Reset will be treated by the retimer as U1 LFPS exit signal, and exit from U1 to Recovery may be achieved. However, the ensuing declaration of Warm Reset detection shall bring the retimer to Rx.Detect, if it has not been there already.
- The retimer shall enable an U2 inactivity timer upon entry to this state if the U2 inactivity timer has a non-zero timeout value.
- The retimer shall enable a 300-ms timer (tU1PingTimeout). This timer will be reset and restarted when a Ping.LFPS is received.
- The retimer shall transmit Ping.LFPS at its upstream port independently to Ping.LFPS received at its downstream port.
- The retimer shall declare device disconnect upon timeout of the tU1PingTimeout timer, and perform the following.
 - It shall disable its receiver termination at its upstream port.
 - It shall stop transmitting Ping.LFPS at its upstream port.
- Note that the propagation of disconnect declaration eliminates a potential race contention between a hub downstream port and the retimer. If the hub downstream port declare device disconnect ahead of the retimer, it may enter Rx.Detect first and exit from Rx.Detect to Polling after performing far-end receiver termination detection, as the retimer has not declared the device disconnect and still has its low-impedance receiver termination presented at its upstream port.

E.3.6.2. Exit from U1

- The retimer shall transition to Recovery upon successful completion of a LFPS handshake meeting the U1 LFPS exit handshake signaling in Section 6.9.2
- The retimer shall transition to U2 upon the timeout of the U2 inactivity timer defined in Sections 10.4.2.4 and 10.6.2.4.
- The retimer shall transition to Rx.Detect if one of the following three conditions is met.
 - Upon the 300-ms timer (tU1PingTimeout) expiration and removal of the receiver termination at its own corresponding port mirroring the far-end receiver termination.
 - Upon detecting Warm Reset.
 - Upon the 2-ms LFPS handshake timer timeout (tNoLFPSResponseTimeout) and a successful LFPS handshake meeting the U1 LFPS exit handshake signaling in Section 6.9.2 is not achieved.

E.3.7. U2

U2 is a link state where more power saving is allowed for the retimer as compared to U1, but with an increased exit latency. The operation of the retimer is the same as is defined in LTSSM with a few exceptions.

E.3.7.1. U2 Requirements

- The retimer shall perform simultaneous U2 LFPS exit handshake upon detecting U2 LFPS exit signals.
- The retimer's downstream port shall distinguish between Ping.LFPS, and U1/U2 LFPS exit signal.
- The retimer, upon detecting an inbound LFPS signal that is not Ping.LFPS, shall initiate simultaneous U2 LFPS exit by performing the following.
 - It shall respond with U2_Exit LFPS handshake.
 - It shall forward the received LFPS signal outbound to its other link partner meeting the electrical and timing requirements defined in Section 6.9.
 - It shall monitor the duration of the received LFPS signal to determine if it is Warm Reset.
- Note that the beginning of Warm Reset will be treated by the retimer as U2 LFPS exit signal, and exit from U2 to Recovery may be achieved. However, the ensuing declaration of Warm Reset detection shall bring the retimer to Rx.Detect, if it has not been there already.

E.3.7.2. Exit from U2

- The retimer shall transition to Recovery upon successful completion of a LFPS handshake meeting the U2 LFPS exit signaling defined in Section 6.9.2.
- The retimer shall transition to Rx.Detect if one of the following three conditions is met.
 - Upon detection of a far-end high-impedance receiver termination ($Z_{RX-HIGH-IMP-DC-POS}$) defined in Table 6-21 and removal of the receiver termination at its own corresponding port mirroring the far-end receiver termination.
 - Upon detecting Warm Reset.
 - Upon the 2-ms LFPS handshake timer timeout ($t_{NoLFPSResponseTimeout}$) and a successful LFPS handshake meeting the U2 LFPS exit handshake signaling in Section 6.9.2 is not achieved.

E.3.8. U3

U3 is a link state where a device is put into a suspend state. Significant link and retimer powers can be saved. The retimer operation is the same as is defined in LTSSM except that U3 LFPS exit is propagated.

E.3.8.1. U3 Requirements

- The retimer shall perform propagated U3 LFPS exit, rather than simultaneous U1/U2 LFPS exit. This is primarily due to the fact that a port may not be ready to respond in time, and attempt U3 LFPS exit when it is ready. The retimer shall perform the following to facilitate the propagated U3 LFPS exit.
 - Upon detecting the U3 LFPS exit signal, the retimer shall propagate the received U3 LFPS exit signal within 100-us.
- NOTE If the retimer is implemented based on the PIPE specification, it shall not wait for the PIPE direction to start the LFPS signal transmission. Instead, while updating the Rx status to indicate the LFPS reception, it shall establish the operating condition of its LFPS transmitter by concurrently powering on its LFPS clock source and ramping up its DC operating point, and start propagating the U3 LFPS exit signal when it is ready to transmit the LFPS signal.
- The retimer shall start propagating the U3 LFPS exit handshake signal only if it has detected the U3 LFPS exit handshake signal.
 - The retimer shall establish its eSS operating condition within 2-ms upon detecting U3 LFPS exit signal.

- Upon observing the successful completion of U3 LFPS exit handshake, if the retimer has not established its eSS operating condition, it shall continue the LFPS transmission until it is ready for eSS operation.
- Note: situation may exist that the retimer's eSS operating condition may not be established upon successful U3 LFPS exit handshake. To make sure that the maximum gap of electrical idle between the LFPS transmission and the eSS transmission does not exceed the specification defined in Section 6.9.2, the retimer shall continue the LFPS transmission until it is ready to transmit eSS signal.
- The retimer shall remain in U3 if the U3 LFPS exit handshake has failed.

E.3.8.2. Exit from U3

- The retimer shall transition to Recovery if the following two conditions are both met.
 - Upon observing the successful completion of the U3 LFPS exit handshake meeting the U3 wakeup signaling defined in Section 6.9.2.
 - The retimer is ready for eSS operation.
- The retimer shall remain in U3 upon failure to achieve a successful U3 LFPS exit handshake meeting the U3 wakeup signaling in Section 6.9.2.
- The retimer shall transition to Rx.Detect if one of the following three conditions is met.
 - Upon detection of a far-end high-impedance receiver termination ($Z_{RX-HIGH-IMP-DC-POS}$) defined in Table 6-21 and removal of the receiver termination at its own corresponding port mirroring the far-end receiver termination.
 - Upon detecting Warm Reset.

E.3.9. Recovery

The retimer operation in Recovery is the same as its operation in Polling.Active, Polling.Configuration, and Polling.Idle. There is only one behavioral difference upon entry to Recovery.

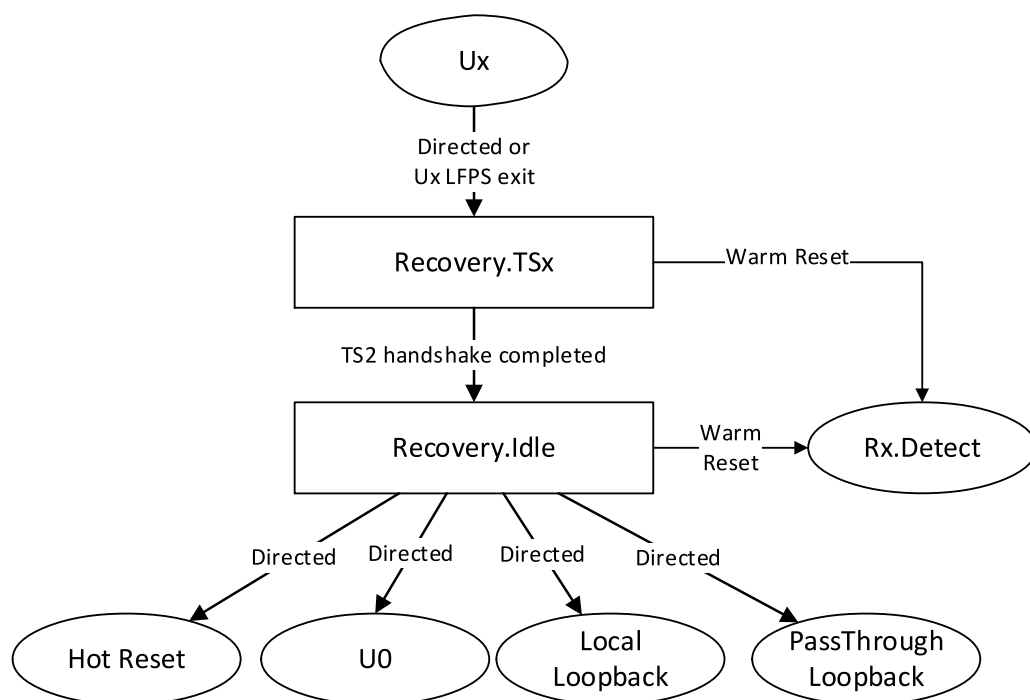
- Upon entry to Recovery, the retimer shall transmit its locally generated TS1 OS to train its link partners.

NOTE in case of exit from a low power state, the retimer may not have its receiver trained in time to forward the received TS1 OS. To avoid unmanaged electrical idle between the end of Ux LFPS exit signal and the start of TS1 OS, the retimer shall fill the gap with the locally generated TS1 OS.

- Upon completing the receiver training at both of its ports, the retimer shall switch the outbound TS1 OS to the received TS1 OS or TS2 OS.

NOTE situation exists that the retimer may have its receiver trained later than its link partner. This may result in retimer missing any TS1 OS to be forwarded. Under this situation, the retimer shall not forward any TS2 OS until its other receiver has been trained.

Recovery contains only two substates, Recovery.TSx and Recovery.Idle as shown in Figure E.6. The exit conditions in the two substates are the same as those in Polling.TSx and Polling.Idle.



IEC

Figure E.6 – Recovery Substate Machine

E.3.10. PassThrough Loopback

PassThrough Loopback is a retimer only state as defined in Section E.3.3.5. In this state, the retimer operation shall be the same as if it's U0, except no error correction is allowed. The retimer does not need to implement the two substates.

E.3.10.1. PassThrough Loopback Requirements

- The retimer shall monitor the LTSSM progression.
- The retimer shall not perform any error correction while looping through the traffic.
- The retimer in SS operation shall perform the overflow clock offset compensation based on SKP OS. Note that in loopback, the loopback master will insert enough SKP OS for the retimer to perform the overflow clock offset compensation.
- The retimer shall implement and start the tLoopbackExitTimeout timer upon detecting Loopback LFPS exit signal.

E.3.10.2. Exit from PassThrough Loopback

- The retimer shall transition to Rx.Detect if any one of the following conditions is met.
 - Upon detecting successful Loopback LFPS exit handshake.
 - Upon timeout of the tLoopbackExitTimeout timer.
 - Upon detecting Warm Reset.

E.3.11. Local Loopback

Local Loopback is the same as the Loopback state defined in LTSSM. The retimer always operates as a loopback slave.

E.3.11.1. Local Loopback Requirements

- The retimer shall configure the two ports in the following.
 - It shall configure its port receiving TS2 OS with the Loopback bit asserted in Local Loopback.
 - It shall configure its other port in Rx.Detect.
- The retimer shall implement the two substates defined in LTSSM.
- The retimer operation shall meet the requirements defined in LTSSM.

E.3.11.2. Exit from Local Loopback.Active

- The retimer shall transition to Rx.Detect upon detecting Warm Reset at its port in Rx.Detect. Note that the retimer will not declare Warm Reset at its port in Local Loopback.Active since the beginning of the Warm Reset will be treated as the start of the Loopback LFPS exit signal.
- The retimer shall transition to Local Loopback.Exit upon detection of Loopback LFPS exit signal.

E.3.11.3. Exit from Local Loopback.Exit

- The retimer shall transition to Rx.Detect if one of the following conditions is met.
 - Upon detecting Warm Reset.
 - Upon completing the Loopback LFPS exit handshake meeting Loopback LFPS exit signaling defined in Section 6.9.2.

E.3.12. Hot Reset

Hot Reset is a state where no actions need taken for the retimer. The retimer's responsibility is to monitor the progression of Hot Reset until its completion. The retimer does not need to implement two substates in Hot Reset.

E.3.12.1. Hot Reset Requirements

- The retimer shall track and monitor the progression of LTSSM.
- The retimer in SS operation shall not delete any inbound TS2 OS with the Reset bit de-asserted in order to perform the over clock offset compensation. Note that the TS2 OS with the Reset bit de-asserted is used for Hot Reset.Active exit handshake.
- The retimer in SS operation shall not delete any inbound IS to perform the overflow clock offset compensation. Note that IS used for Hot Reset.Exit exit handshake.
- The retimer shall start the tHotResetActiveTimeout timer upon entry to the state.
- The retimer shall disable and reset the tHotResetActiveTimeout timer and start the tHotResetExitTimeout timer upon detecting successful TS2 OS based Hot Reset.Active handshake.

E.3.12.2. Exit from Hot Reset

- The retimer shall transition to U0 upon observing successful Hot Reset.Exit handshake.
- The retimer shall transition to Rx.Detect if one of the following conditions is met.
 - Upon timeout of the tHotResetActiveTimeout timer and no successful Hot Reset.Active handshake is observed.
 - Upon timeout of the tHotResetExitTimeout timer and no successful Hot Reset.Exit handshake is observed.
 - Upon detecting Warm Reset.

E.4. Clock Offset Compensation

Retimers are expected to implement a reference clock in the generation of the high frequency local data clock. The reference clock may be common with the host or device with which the retimer is associated, or it may be an independent reference clock. This section describes the requirement and operation of the retimer clock offset compensation based on independent reference clock, and with the maximum number of four retimers in the link topology.

E.4.1. Gen 1 Operation

Retimer clock offset compensation for Gen 1 operation is not based on the mechanism defined in Chapter 6 using SKP OS to manage a receiving port's EB. This is mainly due to the fact that the number of SKP OS inserted at a transmitting port is budgeted only for the receiving port to manage its EB. No additional SKP OS is considered to account for retimer clock offset compensation. Therefore, a retimer needs to perform its clock offset compensation without consuming any SKP OS it has received. This section describes the means a retimer may employ to achieve clock offset compensation. An example architectural concept is also provided.

E.4.1.1. Underflow Clock Offset Compensation

The underflow clock offset compensation refers to a condition that the average throughput of an inbound traffic is lower than that of an outbound traffic based on the local clock frequency of the retimer. This implies a potential underflow situation where the retimer is forwarding the data out faster than the data received. The treatment for the underflow clock offset compensation is consistent with the mechanism defined in Chapter 6 using SKP OS.

- A retimer shall insert SKP OS to perform the underflow clock offset compensation.
- A retimer shall follow SKP OS insertion rule defined in Chapter 6.
- A retimer shall implement its EB that can tolerate the worst case underflow clock offset for at least the duration of a DP with the maximum DPP size.

E.4.1.2. Overflow Clock Offset Compensation

The overflow clock offset compensation refers to a condition that the average throughput of an inbound traffic is higher than that of an outbound traffic based on the local clock frequency of the retimer. This implies a potential overflow situation where the retimer is forwarding the data out slower than the data received. Given no additional SKP OS is budgeted for a retimer to consume, the treatment for the overflow clock offset compensation is different and shall be performed in the following way.

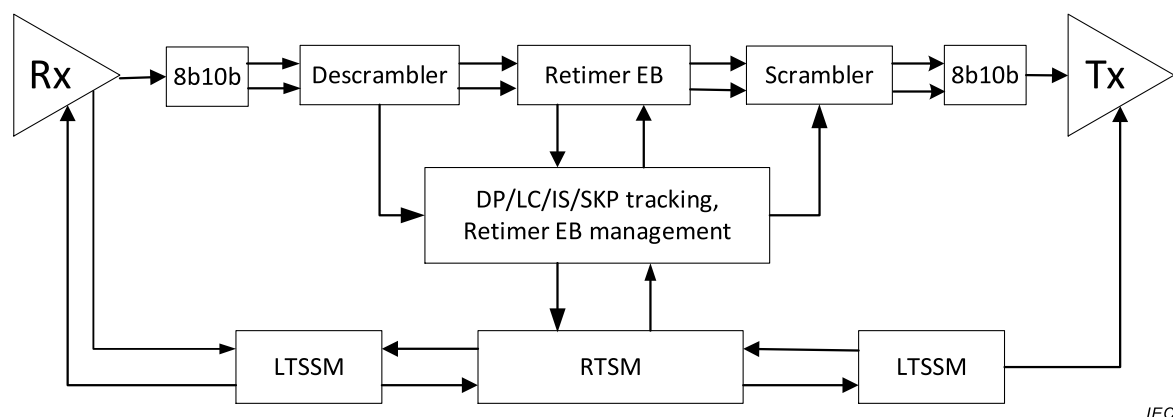
- A retimer shall implement its EB that can tolerate at least eight consecutive DPs of the maximum DPP size under the worst case overflow clock offset.
- A retimer in Polling.Active, Polling.Configuration, Recovery.Active, Recovery.Configuration, and Hot Reset shall employ TS1 OS, or TS2 OS removal to perform the overflow clock offset compensation, while still not violating the exit conditions defined by LTSSM.
- A retimer in U0 shall employ IS removal to perform the overflow clock offset compensation. A retimer shall adhere to the following rules.
 - A retimer shall maintain the scrambler synchronization with its link partner.
 - A retimer shall maintain the running disparity when the forwarded data stream is 8b/10b encoded at the transmitter.
 - There shall be no restriction regarding the number of IS removal to perform overflow clock offset compensation.
 - The IS removal shall only be employed for the overflow clock offset compensation, and a retimer shall not remove more IS than is necessary.

NOTE The preservation of IS necessary for consumption by additional retimers that may exist.

- A retimer is allowed to drop or truncate the DPP of a DP for the overflow clock offset compensation if and only if the following conditions are met.
 - It has forwarded eight consecutive DPs of the maximum DPP size.
 - Its EB has reached the overflow limit.
 - It has no IS to consume.
- A retimer shall not drop any link commands and header packets to perform the overflow clock offset compensation.
- A retimer may re-organize the SKP OS it received and re-construct them in different orders as long as the SKP OS insertion in its forwarded data stream meets the requirements defined in Chapter 6 and Chapter 7 in terms of SKP OS insertion interval and locations of SKP OS insertion.
- In case of multiple retimer coexistence, a retimer may perform the overflow clock offset compensation by deleting the SKP OS if the retimer is able to identify that those SKP OS are added by other retimers.

NOTE A retimer may implement a SKP OS count to track the SKP OS insertion interval, and based on that to determine if the SKP OS is added by retimers.

Shown in Figure E.7 is a conceptual example block diagram of a retimer in SuperSpeed operation.



IEC

Figure E.7 – Example block diagram of a retimer operating at SuperSpeed

E.4.2. Gen 2 Operation

Retimers shall perform its clock offset compensation as defined in Section 6.4.3.2.

E.5. Compliance

E.5.1. Host and Device Product Compliance

Host and device products with retimers shall meet the transmitter compliance requirements defined in Section 6.7.3 and the receiver jitter tolerance requirements defined in Section 6.8.5 of the base specification. During all host or device product compliance testing the retimer shall be in the normal operation state (U0).

E.5.2. Component-Level Retimer Compliance

Retimer products may also undergo component level compliance testing. The transmitter and receiver compliance requirements specified in Sections 6.7.3 and 6.8.5 apply to component

level compliance testing. When undergoing component level compliance testing for the transmitter, the retimer shall generate appropriate compliance patterns. When undergoing component level compliance testing for the receiver, the retimer shall be placed into loopback mode.

NOTE All Engineering Change Notices (ECN) and Errata documents as of May 01, 2015 that pertain to this core specification follow this page.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB 3.1 CTLE

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
1) Change to the reference CTLE transfer function for Gen 2 operation to add 3 dB of amplification.
2) Define a single, normative transmitter equalization requirement for Gen 2 operation, along with compliance patterns (CP13, CP14, CP15) to allow measurement of the equalization coefficients.
3) Definition of the LFSR15 polynomial for CP12 in Table 6-13. We overlooked defining the polynomial.

Benefits as a result of the changes:
1) Allows SuperSpeed Gen 2 systems to support an additional 3 dB of loss (10 dB host/6dB cable assembly/7 dB device) in order to minimize the need for re-timers.
2) Standardize the LFSR15 polynomial that is used and clarification on rate.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No existing systems exist.

An analysis of the hardware implications:
1) Will allow host system designs for Gen 2 operation to have up to 10 dB of loss. Design impact is that Gen 2 transmitters must meet a normative specification, and Gen 2 receivers are expected to have at least 3 dB of peak gain (per the reference CTLE transfer function). Both are small impact to designs.
2) Using a consistent polynomial assures that the LFSR15 pattern can be decoded if so desired.

An analysis of the software implications:
No software implications.

An analysis of the compliance testing implications:
The ECN results in changes to the Gen 2 reference CTLE that is used for transmitter compliance testing and receiver testing calibration, and change to the amount of transmitter equalization used in receiver compliance testing. The change will also require redesign of the load board for compliance testing of devices.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) Section 6.4.4, Table 6-13, page 159

From Text:

- 1) In the table, patterns CP0 through CP8 are transmitted at Gen 1 rate, while CP9 through CP12 are transmitted at Gen 2 rate.

Table 6-13 – Compliance Pattern Sequences

Compliance Pattern	Value	Description
CP0	D0.0 scrambled	A pseudo-random data pattern that is exactly the same as logical idle (refer to Chapter 7) but does not include SKP sequences.
CP1	D10.2	Nyquist frequency
CP2	D24.3	Nyquist/2
CP3	K28.5	COM pattern
CP4	LFPS	The low frequency periodic signaling pattern
CP5	K28.7	With de-emphasis
CP6	K28.7	Without de-emphasis
CP7	50-250 1's and 0's	With de-emphasis. Repeating 50-250 1's and then 50-250 0's.
CP8	50-250 1's and 0's	With without de-emphasis. Repeating 50-250 1's and then 50-250 0's.
CP9		Pseudo-random data pattern (see section 6.4.4.1)
CP10	AAh	Nyquist pattern at 10 Gb/s. This is not 128b132b encoded.
CP11	CCh	Nyquist/2 at 10 Gb/s, This is not 128b132b encoded.
CP12	LFSR15	Uncoded LFSR15 for PHY level testing and fault isolation. This is not 128b132b encoded.

NOTE Unless otherwise noted, scrambling is disabled for compliance patterns.

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

- 1) In the table, patterns CP0 through CP8 are transmitted at Gen 1 rate, while CP9 through CP16 are transmitted at Gen 2 rate.

Table 6-13 – Compliance Pattern Sequences

Compliance Pattern	Value	Description
CP0	D0.0 scrambled	A pseudo-random data pattern that is exactly the same as logical idle (refer to Chapter 7) but does not include SKP sequences.
CP1	D10.2	Nyquist frequency
CP2	D24.3	Nyquist/2
CP3	K28.5	COM pattern
CP4	LFPS	The low frequency periodic signaling pattern
CP5	K28.7	With de-emphasis
CP6	K28.7	Without de-emphasis
CP7	50-250 1's and 0's	With de-emphasis. Repeating 50-250 1's and then 50-250 0's.
CP8	50-250 1's and 0's	Without de-emphasis. Repeating 50-250 1's and then 50-250 0's.
CP9		Pseudo-random data pattern (see section 6.4.4.1)
CP10	AAh	Nyquist pattern at 10 Gb/s. This is not 128b132b encoded.
CP11	CCh	Nyquist/2 at 10 Gb/s, This is not 128b132b encoded.
CP12	LFSR15	Uncoded LFSR15 for PHY level testing and fault isolation. This is not 128b132b encoded. The polynomial is $x^{15}+x^{14}+1$.
CP13	64 1's and 0's	With pre-shoot defined in section 6.7.5.2 (no de-emphasis). Repeating 64 1's and then 64 0's at 10 Gb/s. This is not 128b132b encoded.
CP14	64 1's and 0's	With de-emphasis defined in section 6.7.5.2 (no pre-shoot). Repeating 64 1's and then 64 0's at 10 Gb/s. This is not 128b132b encoded.
CP15	64 1's and 0's	With pre-shoot and de-emphasis defined in section 6.7.5.2. Repeating 64 1's and then 64 0's at 10 Gb/s. This is not 128b132b encoded.
CP16	64 1's and 0's	No de-emphasis or pre-shoot. Repeating 64 1's and then 64 0's at 10 Gb/s. This is not 128b132b encoded.

NOTE Unless otherwise noted, scrambling is disabled for compliance patterns.

USB 3.1 ENGINEERING CHANGE NOTICE

(b) Section 6.7.1, Table 6-17, page 166

From Text:

6.7.1 Transmitter Electrical Parameters

Peak (p) and peak-peak (p-p) are defined in Section 6.6.2.

Table 6-17 – Transmitter Normative Electrical Parameters

Symbol	Parameter	Gen 1 (5.0 GT/s)	Gen 2 (10 GT/s)	Units	Comments
UI	Unit Interval	199.94 (min) 200.06 (max)	99.97 (min) 100.03 (max)	ps	The specified UI is equivalent to a tolerance of ± 300 ppm for each device. Period does not account for SSC induced variations.
$V_{TX-DIFF-PP}$	Differential p-p Tx voltage swing	0.8 (min) 1.2 (max)	0.8 (min) 1.2 (max)	V	Nominal is 1 V p-p
$V_{TX-DIFF-PP-LOW}$	Low-Power Differential p-p Tx voltage swing	0.4 (min) 1.2 (max)	0.4 (min) 1.2 (max)	V	Refer to Section 6.7.2. There is no de-emphasis requirement in this mode. De-emphasis is implementation specific for this mode.
$V_{TX-DE-RATIO}$	Tx de-emphasis	3.0 (min) 4.0 (max)	Not applicable	dB	Nominal is 3.5 dB for Gen 1 operation. Gen 2 transmitter equalization recommendations are described in section 6.7.5.2.
$R_{TX-DIFF-DC}$	DC differential impedance	72 (min) 120 (max)	72 (min) 120 (max)	Ω	
$V_{TX-RCV-DETECT}$	The amount of voltage change allowed during Receiver Detection	0.6 (max)	0.6 (max)	V	Detect voltage transition should be an increase in voltage on the pin looking at the detect signal to avoid a high impedance requirement when an “off” receiver’s input goes below ground.
$C_{AC-COUPLING}$	AC Coupling Capacitor	75 (min) 200 (max)	75 (min) 265 (max)	nF	All Transmitters shall be AC coupled. The AC coupling is required either within the media or within the transmitting component itself.
$t_{CDR_SLEW_MAX}$	Maximum slew rate	10	Not applicable	ms/s	See the jitter white paper for details on this measurement. This is a df/ft specification; refer to Section 6.5.4 for details.
SSC_{dfdt}	SSC df/dt	Not applicable	1250 (max)	ppm/ μ s	See note 1.

NOTES:

1. Measured over a 0.5 μ s interval using CP10. The measurements shall be low pass filtered using a filter with 3 dB cutoff frequency that is 60 times the modulation rate. The filter stopband rejection shall be greater or equal to a second order low-pass of 20 dB per decade. Evaluation of the maximum df/dt is achieved by inspection of the low-pass filtered waveform.

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

6.7.1 Transmitter Electrical Parameters

Peak (p) and peak-peak (p-p) are defined in Section 6.6.2.

Table 6-17 – Transmitter Normative Electrical Parameters

Symbol	Parameter	Gen 1 (5.0 GT/s)	Gen 2 (10 GT/s)	Units	Comments
UI	Unit Interval	199.94 (min) 200.06 (max)	99.97 (min) 100.03 (max)	ps	The specified UI is equivalent to a tolerance of ± 300 ppm for each device. Period does not account for SSC induced variations.
$V_{TX-DIFF-PP}$	Differential p-p Tx voltage swing	0.8 (min) 1.2 (max)	0.8 (min) 1.2 (max)	V	Nominal is 1 V p-p
$V_{TX-DIFF-PP-LOW}$	Low-Power Differential p-p Tx voltage swing	0.4 (min) 1.2 (max)	0.4 (min) 1.2 (max)	V	Refer to Section 6.7.2. There is no de-emphasis requirement in this mode. De-emphasis is implementation specific for this mode.
$V_{TX-DE-RATIO}$	Tx de-emphasis	3.0 (min) 4.0 (max)	See section 6.7.5.2.	dB	Nominal is 3.5 dB for Gen 1 operation. Gen 2 transmitter equalization recommendations requirements are described in section 6.7.5.2.
$R_{TX-DIFF-DC}$	DC differential impedance	72 (min) 120 (max)	72 (min) 120 (max)	Ω	
$V_{TX-RCV-DETECT}$	The amount of voltage change allowed during Receiver Detection	0.6 (max)	0.6 (max)	V	Detect voltage transition should be an increase in voltage on the pin looking at the detect signal to avoid a high impedance requirement when an “off” receiver’s input goes below ground.
$C_{AC-COUPLING}$	AC Coupling Capacitor	75 (min) 200 (max)	75 (min) 265 (max)	nF	All Transmitters shall be AC coupled. The AC coupling is required either within the media or within the transmitting component itself.
$t_{CDR_SLEW_MAX}$	Maximum slew rate	10	Not applicable	ms/s	See the jitter white paper for details on this measurement. This is a df/ft specification; refer to Section 6.5.4 for details.
SSC_{dfdt}	SSC df/dt	Not applicable	1250 (max)	ppm/ μ s	See note 1.

NOTES:

1. Measured over a 0.5 μ s interval using CP10. The measurements shall be low pass filtered using a filter with 3 dB cutoff frequency that is 60 times the modulation rate. The filter stopband rejection shall be greater or equal to a second order low-pass of 20 dB per decade. Evaluation of the maximum df/dt is achieved by inspection of the low-pass filtered waveform.

USB 3.1 ENGINEERING CHANGE NOTICE

(c) Section 6.7.5, page 168

From Text:

6.7.5 Informative Transmitter De-emphasis

6.7.5.1 Gen 1 (5GT/s)

The channel budgets and eye diagrams were derived using a $V_{TX-DE-RATIO}$ of transmit de-emphasis for both the Host and the Device reference channels. An example differential peak-to-peak de-emphasis waveform is shown in Figure 6-20.

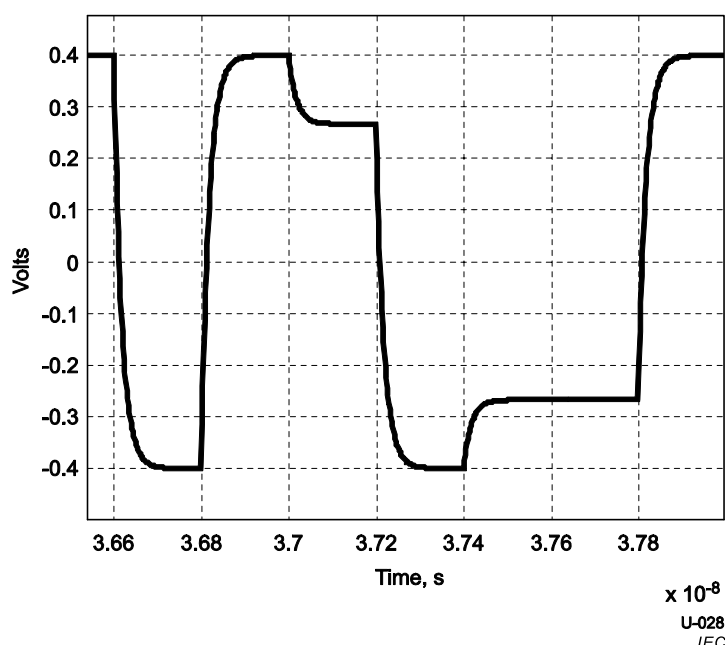


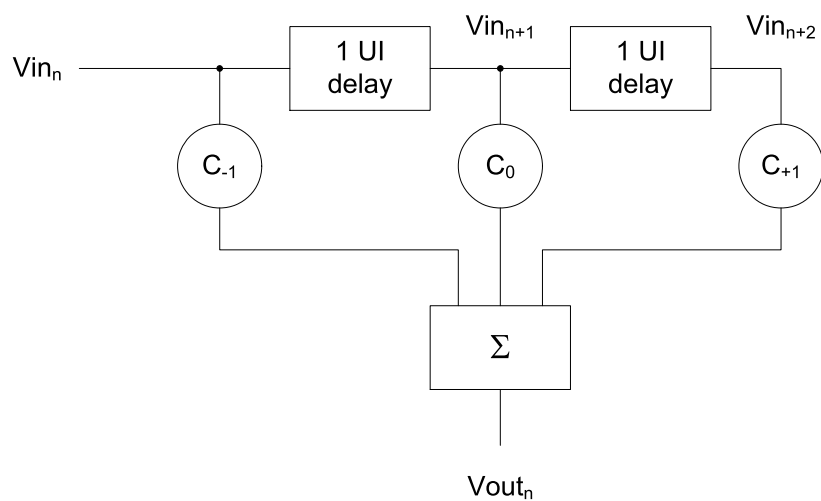
Figure 6-20 – De-Emphasis Waveform

6.7.5.2 Gen 2 (10GT/s)

Gen 2 transmitters employ a 3-tap FIR-based equalizer, the structure of which is shown in Figure 6-21. An example waveform from the 3-tap equalizer is shown in Figure 6-22. In the figure, the pre-cursor (V_c) is referred to as pre-shoot, while the post-cursor (V_b) is referred to as de-emphasis. This convention allows pre-shoot and de-emphasis to be defined independently of one another. The maximum swing, V_d , is also shown to illustrate that, when both $C+1$ and $C-1$ are nonzero, the swing of V_a does not reach the maximum as defined by V_d . Figure 6-22 is shown as an example of TxEQ and is not intended to represent the signal as it would appear for measurement purposes.

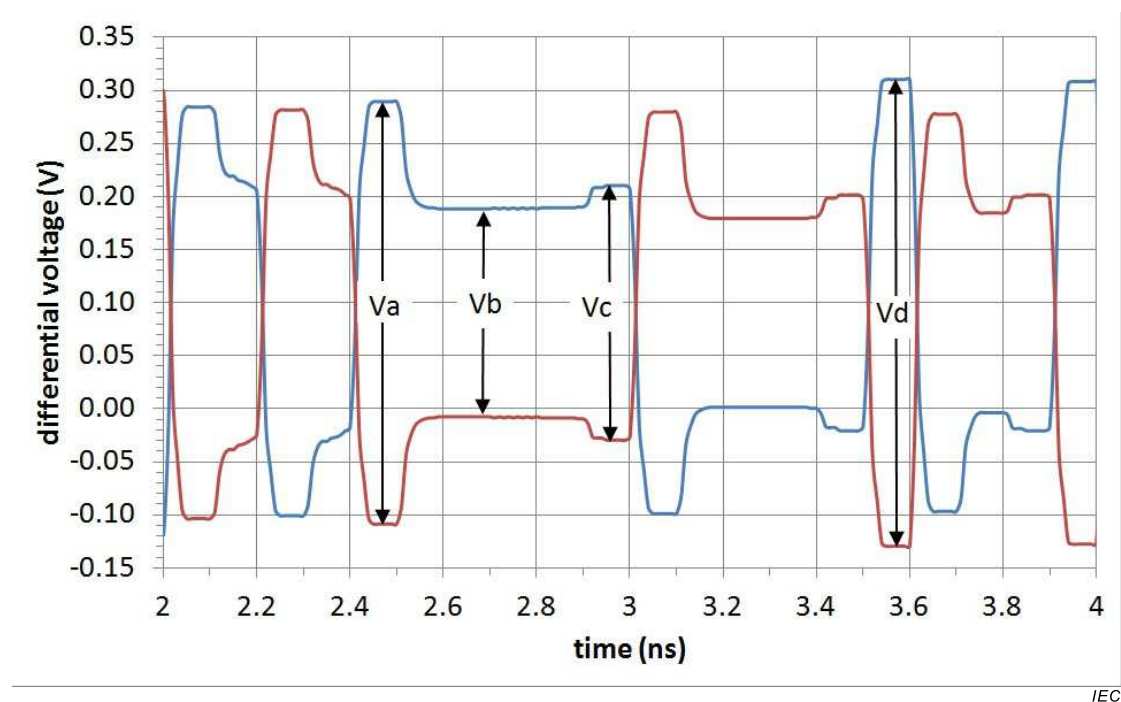
Table 6-20 provides recommended (informative) tap coefficient values (C_{-1} and C_1) along with the corresponding pre-shoot, de-emphasis and output amplitudes. The host/device loss referred to in the table refers to the differential insertion loss in the conductor path from the silicon die pad to the connector, and includes parasitic I/O capacitance, the chip package (routing, vias and I/O pins), and printed circuit board (routing and vias).

USB 3.1 ENGINEERING CHANGE NOTICE



IEC

Figure 6-21 – 3-tap Transmit Equalizer Structure



IEC

Preshoot = $20\log(V_c/V_b)$
De-emphasis = $20\log(V_b/V_a)$

Figure 6-22 – Example Output Waveform for 3-tap Transmit Equalizer

USB 3.1 ENGINEERING CHANGE NOTICE

Table 6-20 – Informative Gen 2 Transmitter Equalization Settings

Host/Device Loss @ 5 GHz (dB)	<3,5	≥3,5
C ₋₁	0,000	-0,100
C ₁	-0,125	-0,125
Preshoot (dB)	0.0	2.7
De-emphasis (dB)	-2,5	-3,3
Va/Vd	1,000	0,800
Vb/Vd	0,750	0,550
Vc/Vd	0,750	0,750

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

6.7.5 Informative Transmitter De-emphasis

6.7.5.1 Gen 1 (5GT/s)

The channel budgets and eye diagrams were derived using a $V_{TX-DE-RATIO}$ of transmit de-emphasis for both the Host and the Device reference channels. An example differential peak-to-peak de-emphasis waveform is shown in Figure 6-20.

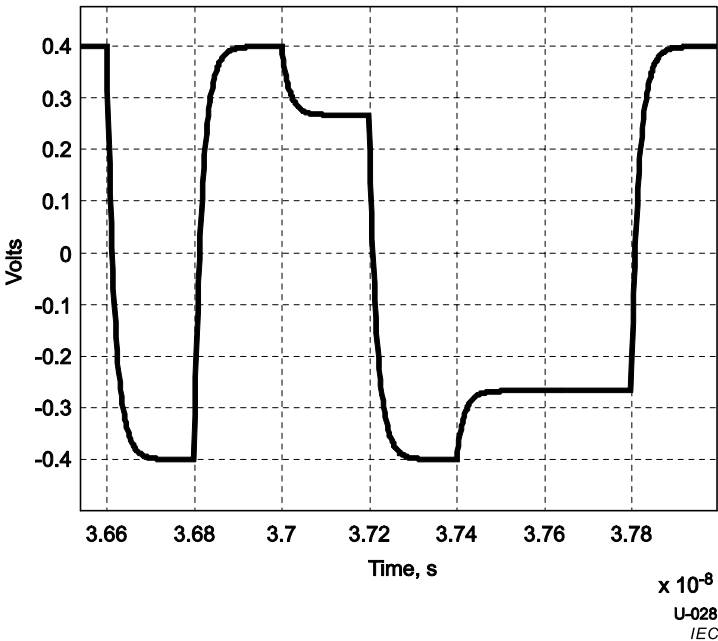


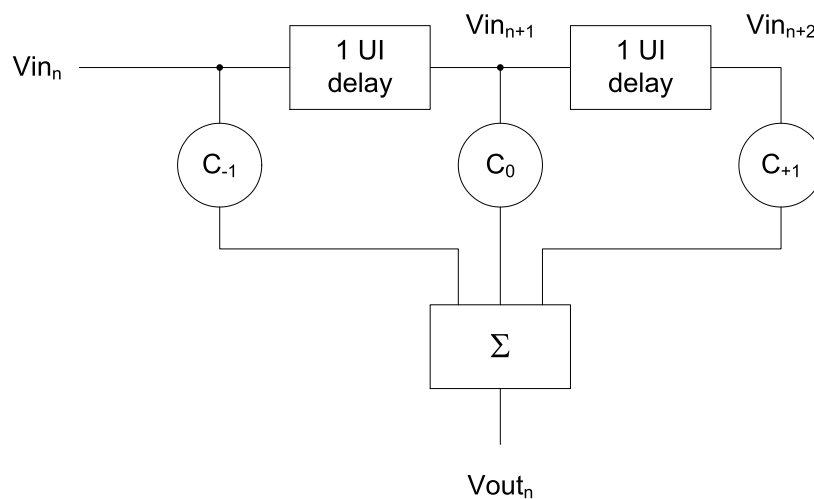
Figure 6-20 – De-Emphasis Waveform

6.7.5.2 Gen 2 (10GT/s)

Gen 2 transmitters employ a 3-tap FIR-based equalizer, the structure of which is shown in Figure 6-21. An example waveform from the 3-tap equalizer is shown in Figure 6-22. In the figure, the pre-cursor (V_c) is referred to as pre-shoot, while the post-cursor (V_b) is referred to as de-emphasis. This convention allows pre-shoot and de-emphasis to be defined independently of one another. The maximum swing, V_d , is also shown to illustrate that, when both C_{+1} and C_{-1} are nonzero, the swing of V_a does not reach the maximum as defined by V_d . Figure 6-22 is shown as an example of TxEQ and is not intended to represent the signal as it would appear for measurement purposes.

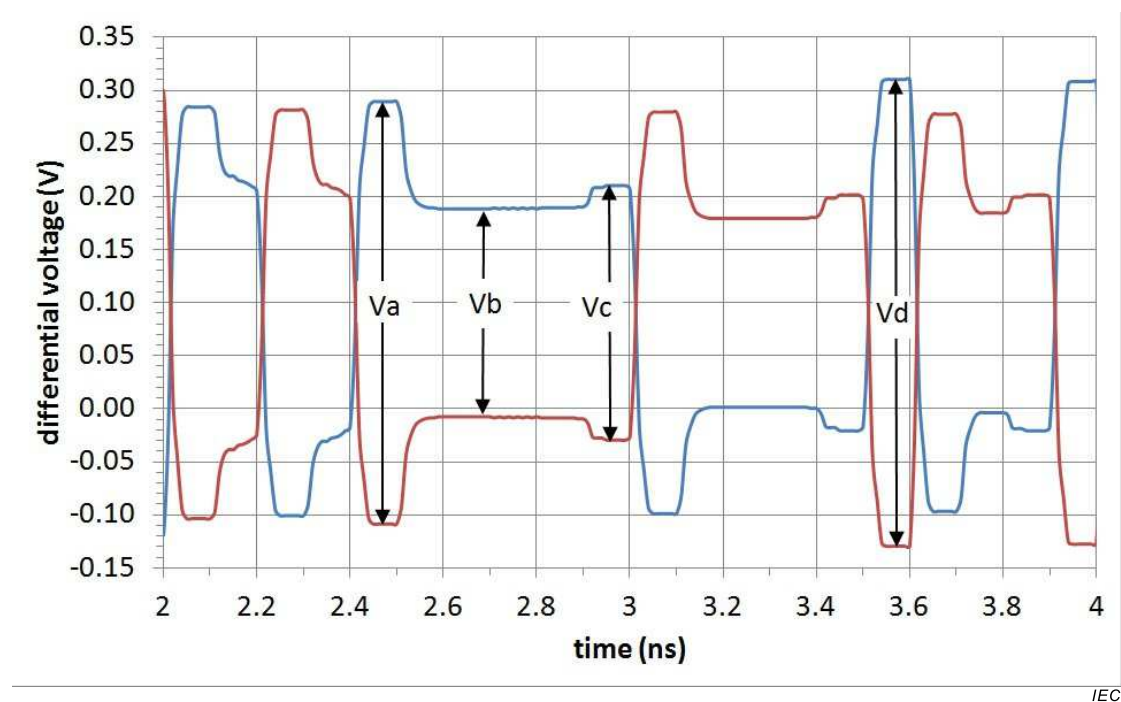
Table 6-20 provides the recommended (informative) normative pre-shoot and de-emphasis tap coefficient values (C_{-1} and C_1) along with the corresponding tap coefficient values (C_{-1} and C_1) pre-shoot, de-emphasis and output amplitudes. The host/device loss referred to in the table refers to the differential insertion loss in the conductor path from the silicon die pad to the connector, and includes parasitic I/O capacitance, the chip package (routing, vias and I/O pins), and printed circuit board (routing and vias).

USB 3.1 ENGINEERING CHANGE NOTICE



IEC

Figure 6-21 – 3-tap Transmit Equalizer Structure



IEC

Preshoot = $20\log(V_c/V_b)$
De-emphasis = $20\log(V_b/V_a)$

Figure 6-22 – Example Output Waveform for 3-tap Transmit Equalizer

USB 3.1 ENGINEERING CHANGE NOTICE

Table 6-20 – Informative Gen 2 Transmitter Equalization Settings

Parameter	Value	Comments
Preshoot (dB)	2.2±1.0	Normative requirement
De-emphasis (dB)	-3.1±1.0	Normative requirement
C_{-1}	-0.083	Informative – for reference only
C_1	-0.125	Informative – for reference only
Nominal Boost (dB)	4.7	Informative – for reference only
Va/Vd	0.834	Informative – for reference only
Vb/Vd	0.584	Informative – for reference only
Vc/Vd	0.750	Informative – for reference only

Measurement of the preshoot and de-emphasis is done using CP13, CP14 and CP15. Va, Vc and Vb are obtained using CP13, CP14 and CP15 as shown in Figure 6-23. Preshoot and de-emphasis are calculated using equations (10) and (11).

$$preshoot = 20 \log_{10} \left(\frac{V_{CP14}}{V_{CP15}} \right) = 20 \log_{10} \left(\frac{-C_{-1} + C_0 + C_1}{C_{-1} + C_0 + C_1} \right) \quad (10)$$

$$deemphasis = 20 \log_{10} \left(\frac{V_{CP15}}{V_{CP13}} \right) = 20 \log_{10} \left(\frac{C_{-1} + C_0 + C_1}{C_{-1} + C_0 - C_1} \right) \quad (11)$$

A transmitter must satisfy equation (12) during transmission of compliance patterns and during normal operation.

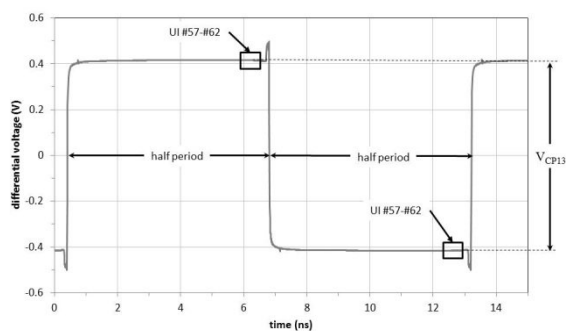
$$|C_{-1}| + |C_0| + |C_1| = 1 \quad (12)$$

Satisfying equations (10) and (12) means that during transmission of the CP13 pattern, the transmitter moves the transistor legs for the de-emphasis tap (C_1) into the cursor tap (C_0). Similarly, during transmission of CP14, the transmitter moves the transistor legs for the preshoot tap (C_{-1}) into the cursor tap (C_0).

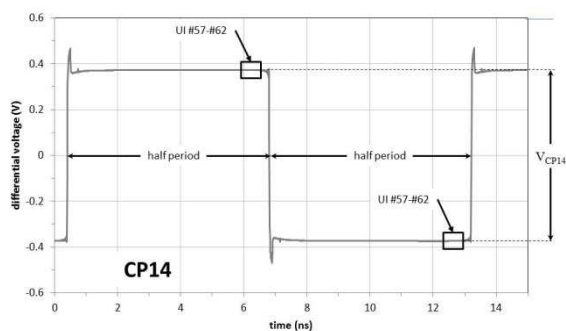
During measurement, ISI and switching effects are minimized by restricting the portion of the curve over which voltage is measured to the last few UI of each half cycle, as illustrated in the Figure 6-23. High frequency noise is mitigated by averaging over multiple readings until the peak-to-peak noise over the area of interest is less than 2 % of the magnitude of the swing.

In addition, a transmitter's output voltage swing with no equalization is obtained by measuring the peak-to-peak voltage using the CP16 compliance pattern as shown in Figure 6-22.

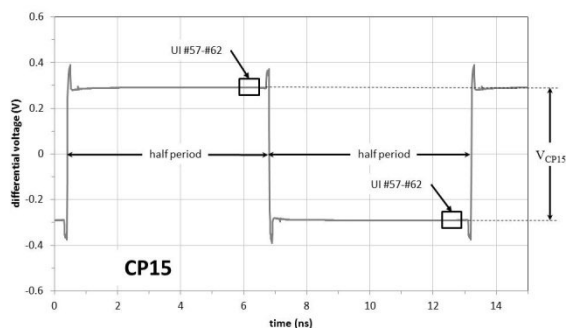
USB 3.1 ENGINEERING CHANGE NOTICE



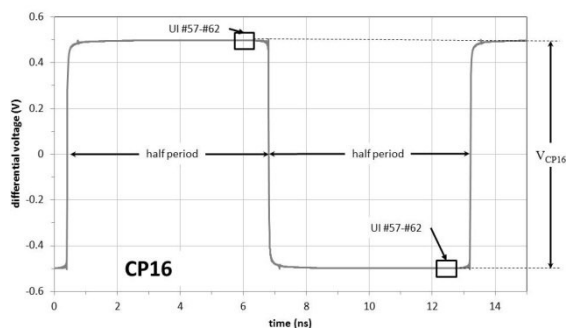
(a) CP13 (with preshoot only)



(b) CP14 (with de-emphasis only)



(c) CP15 (with preshoot and de-emphasis)



(d) CP16 (without preshoot or de-emphasis)

IEC

Figure 6-23 – Example waveforms for measuring transmitter equalization

NOTE All remaining figures in chapter 6 (Figure 6-23 and above) will require renumbering. The equations in subsequent chapters also require renumbering.

USB 3.1 ENGINEERING CHANGE NOTICE

(d) Section 6.8.2.2.1, Figure 6-25, page 173

From Text:

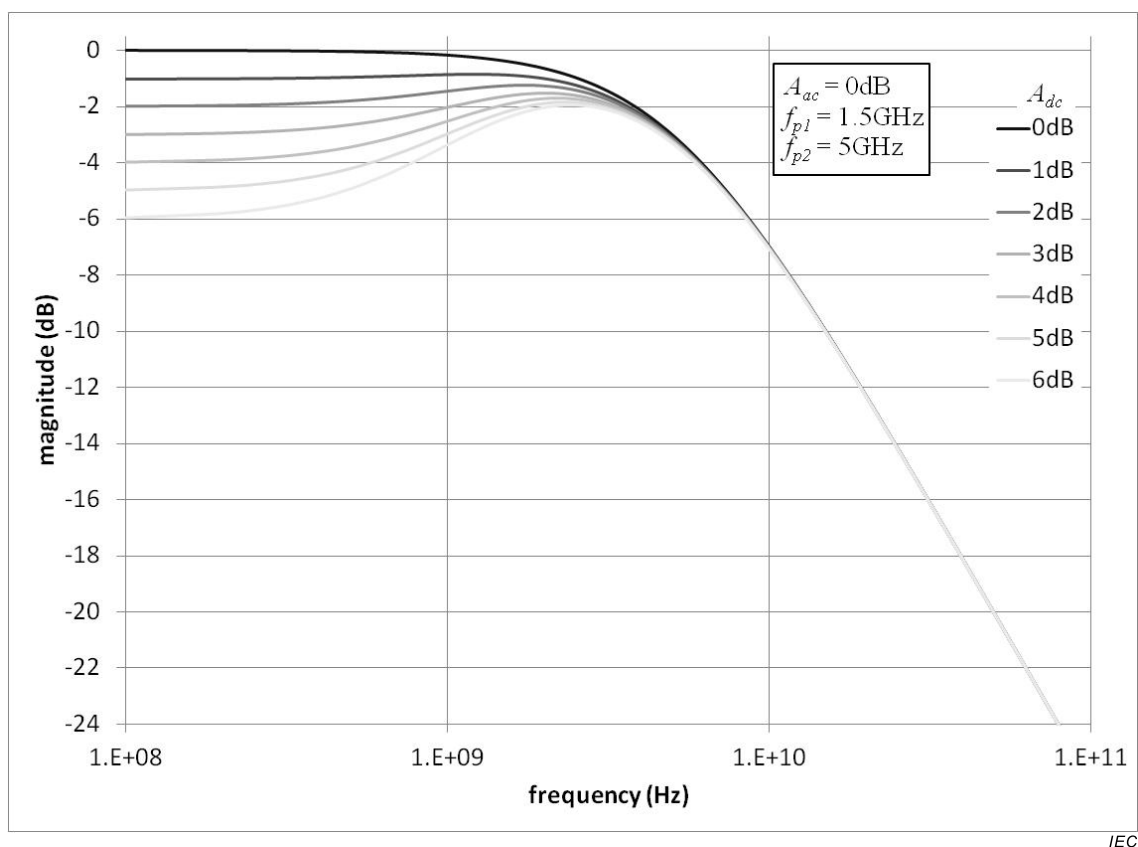


Figure 6-25 – Gen 2 Compliance Rx EQ Transfer Function

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

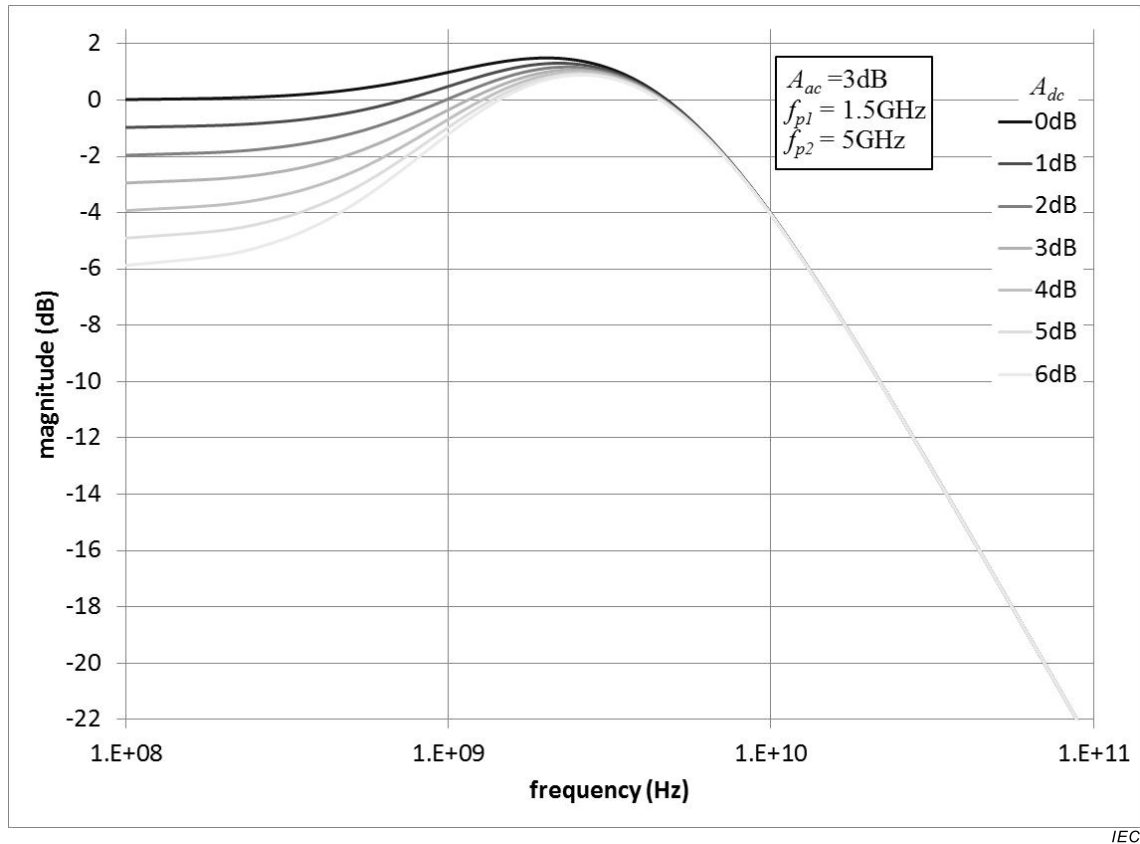


Figure 6-25 – Gen 2 Compliance Rx EQ Transfer Function

USB 3.1 ENGINEERING CHANGE NOTICE

(e) Section 6.8.2.2.1, Figure 6-27, page 177

From Text:

Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing

Symbol	Parameter	Gen 1 (5GT/s)	Gen 2 (10GT/s)	Units	Notes
f1	Tolerance corner	4,9	7,5	MHz	
J _{Rj}	Random Jitter	0,0121	0,01308	UI rms	1
J _{Rj_p-p}	Random Jitter peak- peak at 10 ⁻¹²	0,17	0,184	UI p-p	1,4
J _{Pj_500kHz}	Sinusoidal Jitter	2	4,76	UI p-p	1,2,3
J _{Pj_1MHz}	Sinusoidal Jitter	1	2,03	UI p-p	1,2,3
J _{Pj_2MHz}	Sinusoidal Jitter	0,5	0,87	UI p-p	1,2,3
J _{Pj_4MHz}	Sinusoidal Jitter	N/A	0,37	UI p-p	1,2,3
J _{Pj_f1}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_50MHz}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_100MHz}	Sinusoidal Jitter	N/A	0,17	UI p-p	1,2,3
V _{full_swing}	Transition bit differential voltage swing	0,75	0,8	V p-p	1
V _{EQ_level}	Non transition bit voltage (equalization)	-3	Preshoot=2.7 De-emphasis= -3.3	dB	1

NOTES:

1. All parameters measured at TP1. The test point is shown in Figure 6-19.
2. Due to time limitations at compliance testing, only a subset of frequencies can be tested. However, the Rx is required to tolerate Pj at all frequencies between the compliance test points.
3. During the Rx tolerance test, SSC is generated by test equipment and present at all times. Each J_{Pj} source is then added and tested to the specification limit one at a time.
4. Random jitter is also present during the Rx tolerance test, though it is not shown in Figure 6-20.
5. The JTOL specs for Gen 2 comprehend jitter peaking with re-timers in the system and has a 25 dB/decade slope.

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing

Symbol	Parameter	Gen 1 (5GT/s)	Gen 2 (10GT/s)	Units	Notes
f1	Tolerance corner	4,9	7,5	MHz	
J _{Rj}	Random Jitter	0,0121	0,01308	UI rms	1
J _{Rj_p-p}	Random Jitter peak- peak at 10 ⁻¹²	0,17	0,184	UI p-p	1,4
J _{Pj_500kHz}	Sinusoidal Jitter	2	4,76	UI p-p	1,2,3
J _{Pj_1MHz}	Sinusoidal Jitter	1	2,03	UI p-p	1,2,3
J _{Pj_2MHz}	Sinusoidal Jitter	0,5	0,87	UI p-p	1,2,3
J _{Pj_4MHz}	Sinusoidal Jitter	N/A	0,37	UI p-p	1,2,3
J _{Pj_f1}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_50MHz}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_100MHz}	Sinusoidal Jitter	N/A	0,17	UI p-p	1,2,3
V _{full_swing}	Transition bit differential voltage swing	0,75	0,8	V p-p	1
V _{EQ_level}	Non transition bit voltage (equalization)	-3	Preshoot=2,2 De-emphasis= -3,1	dB	1

NOTES:

1. All parameters measured at TP1. The test point is shown in Figure 6-19.
2. Due to time limitations at compliance testing, only a subset of frequencies can be tested. However, the Rx is required to tolerate Pj at all frequencies between the compliance test points.
3. During the Rx tolerance test, SSC is generated by test equipment and present at all times. Each J_{Pj} source is then added and tested to the specification limit one at a time.
4. Random jitter is also present during the Rx tolerance test, though it is not shown in Figure 6-20.
5. The JTOL specs for Gen 2 comprehend jitter peaking with re-timers in the system and has a 25 dB/decade slope.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: HSEQ

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Addresses one extreme corner case in SSP operation where ambiguity of Header Sequence Number Advertisement may arise after entry to U0 from Recovery. In SSP operation, all header sequence numbers (0~7) are used. Header Sequence Number Advertisement is one and only one block acknowledgement in USB3 operation. There exist two scenarios that result in link going to Recovery. 1. All 8 HPs transmitted were corrupted and corrupted in a way that HPSTART OS are not recognizable and LBAD is impossible. This leads to entry to Recovery. The Header Sequence Number is LGOOD_4 (assuming the 1st header packet has HSEQ# of 5). 2. All 8 HPs transmitted were received successfully, but all 8 LGOOD_n are corrupted and corrupted in a way that SLCSTART OS are not recognizable. This will also lead to entry to Recovery. The same Header Sequence Number of LGOOD_4 is received. Although this is an extreme corner case, it does result in ambiguity where the port transmitting all 8 HPs needs to decide if all 8 HPs needs to be flushed or re-transmitted.

Benefits as a result of the changes:
Resolve behavioral uncertainty

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No certified SSP product yet

An analysis of the hardware implications:
Minor change is needed. Header sequence number range increased by one bit.

An analysis of the software implications:
None

An analysis of the compliance testing implications:
Minor. Extend header sequence number range from 0~7 to 0~15.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

Section 7.2.1.1.3

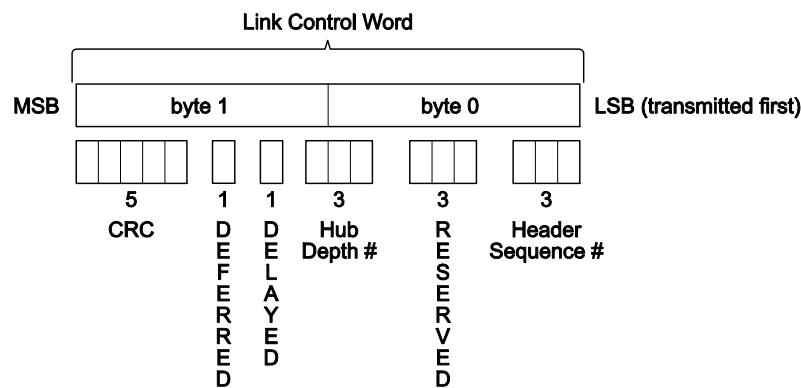
From Text:

The Link Control Word shall contain a 3-bit Header Sequence Number, 3-bit Reserved, a 3-bit Hub Depth Index, a Delayed bit (DL), a Deferred bit (DF), and a 5-bit CRC-5.

To Text:

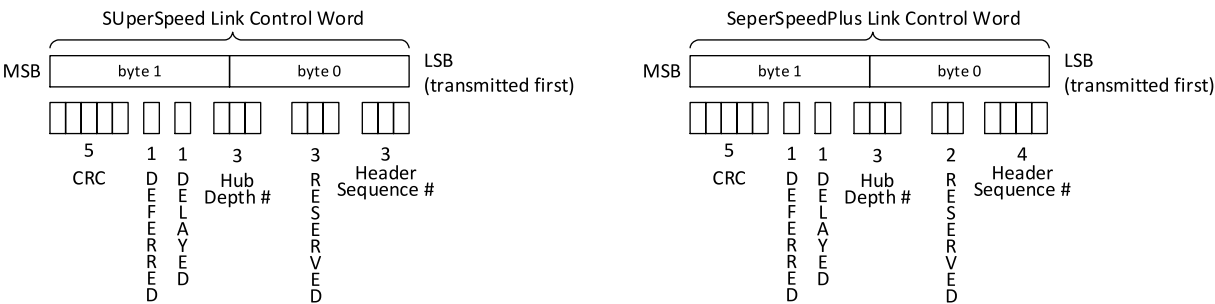
In SuperSpeed operation, the Link Control Word shall contain a 3-bit Header Sequence Number, 3-bit Reserved, a 3-bit Hub Depth Index, a Delayed bit (DL), a Deferred bit (DF), and a 5-bit CRC-5. In SuperSpeedPlus operation, the Link Control Word shall contain a 4-bit Header Sequence Number, 2-bit Reserved, a 3-bit Hub Depth Index, a Delayed bit (DL), a Deferred bit (DF), and a 5-bit CRC-5.

From Figure 7-7



U-040
IEC

To Figure 7-7



IEC

Section 7.2.4.1.1

From Text:

- 1) A port shall initiate the Header Sequence Number Advertisement by transmitting LGOOD_n with “n” equal to the Rx Header Sequence Number minus one.

USB 3.1 ENGINEERING CHANGE NOTICE

NOTE The decrement is based on modulo-8 operation.

- 2) A port shall set its initial ACK Tx Header Sequence Number to the Sequence Number received during the Rx Header Sequence Number Advertisement plus one.

NOTE The increment is based on modulo-8 operation.

To Text:

- 1) A port shall initiate the Header Sequence Number Advertisement by transmitting LGOOD_n with “n” equal to the Rx Header Sequence Number minus one.

NOTE The decrement is based on modulo-8 operation in SuperSpeed operation, and modulo-16 in SuperSpeedPlus operation.

- 2) A port shall set its initial ACK Tx Header Sequence Number to the Sequence Number received during the Rx Header Sequence Number Advertisement plus one.

NOTE The increment is based on modulo-8 operation in SuperSpeed operation, and modulo-16 in SuperSpeedPlus operation.

Section 7.2.4.1.1

From Text:

- c) If a port enters U0 from Recovery, it shall flush all the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have been sent before Recovery except for those with the Header Sequence Number greater than (modulo 8) the Header Sequence Number received in Header Sequence Number Advertisement.

NOTE If for example, the Header Sequence Number Advertisement of LGOOD_1 is received, a port shall flush the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers with Header Sequence Numbers of 1, 0, 7, 6.

To Text:

- b) If a port enters U0 from Recovery, it shall flush all the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have been sent before Recovery except for those with the Header Sequence Number greater than (modulo 8 in SuperSpeed operation, and modulo-16 in SuperSpeedPlus operation) the Header Sequence Number received in Header Sequence Number Advertisement.

NOTE If for example in SuperSpeed operation, the Header Sequence Number Advertisement of LGOOD_1 is received, a port shall flush the header packets in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers with Header Sequence Numbers of 1, 0, 7, 6.

Section 7.2.4.1.2

From Text:

- Header packets shall be sent with the Header Sequence Number in the numerical order from 0 to 7, and back to 0. LGOOD_n received out of the numerical order is considered as missing of a link command, and the transition to Recovery shall be initiated.

To Text:

- Header packets in SuperSpeed operation shall be sent with the Header Sequence Number in the numerical order from 0 to 7, and back to 0. Header packets in SuperSpeedPlus operation shall be sent with the Header Sequence Number in the numerical order from 0 to 15, and back to 0. LGOOD_n received out of the numerical order is considered as missing of a link command, and the transition to Recovery shall be initiated.

USB 3.1 ENGINEERING CHANGE NOTICE

Section 7.2.4.1.10

From Text:

- A port shall maintain every header packet transmitted within its Tx Header Buffer or Type 1/Type 2 Tx Header Buffer until it receives an LGOOD_n. Upon receiving LGOOD_n, a port shall do one of the following:
 - 1) If LGOOD_n is the Header Sequence Number Advertisement and a port is entering U0 from Recovery, a port shall flush all the header packets retained in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have their Header Sequence Numbers equal to or less than the received Header Sequence Number, and initialize its ACK Tx Header Sequence Number to be the received Header Sequence Number plus one.

NOTE The comparison and increment are based on modulo-8 operation.

- 2) If a port receives an LGOOD_n and this LGOOD_n is not Header Sequence Number Advertisement, it shall flush the header packet in its Tx Header Buffer or Type 1/Type 2 Tx Header Buffer with its Header Sequence Number matching the received Header Sequence Number and increment the ACK Tx Header Sequence Number by one based on modulo-8 operation.

To Text:

- A port shall maintain every header packet transmitted within its Tx Header Buffer or Type 1/Type 2 Tx Header Buffer until it receives an LGOOD_n. Upon receiving LGOOD_n, a port shall do one of the following:
 - 1) If LGOOD_n is the Header Sequence Number Advertisement and a port is entering U0 from Recovery, a port shall flush all the header packets retained in its Tx Header Buffers or Type 1/Type 2 Tx Header Buffers that have their Header Sequence Numbers equal to or less than the received Header Sequence Number, and initialize its ACK Tx Header Sequence Number to be the received Header Sequence Number plus one.

NOTE The comparison and increment are based on modulo-8 operation in SuperSpeed operation, and modulo-16 in SuperSpeedPlus operation.

- 2) If a port receives an LGOOD_n and this LGOOD_n is not Header Sequence Number Advertisement, it shall flush the header packet in its Tx Header Buffer or Type 1/Type 2 Tx Header Buffer with its Header Sequence Number matching the received Header Sequence Number and increment the ACK Tx Header Sequence Number by one based on modulo-8 operation in SuperSpeed operation, and modulo-16 in SuperSpeedPlus operation.

USB 3.1 ENGINEERING CHANGE NOTICE

Section 7.2.2.2

From Table 7-4:

Table 11-4 – Link Command Bit Definitions

Class		Type	b6~4	Sub-Type
b10~9	Link Command	b8~7		b3~0
00	LGOOD_n LRTY LBAD LCRD_x LCRD1_x LCRD2_x	00: LGOOD_n	Reserved (000)	b3: Reserved b2~0: HP Sequence Number 000: LGOOD_0 001: LGOOD_1 010: LGOOD_2 011: LGOOD_3 100: LGOOD_4 101: LGOOD_5 110: LGOOD_6 111: LGOOD_7
		01: LCRD_x LCRD1_x LCRD2_x		b3: Reserved b2: Credit series 0: LCRD_x or LCRD1_x 1: LCRD2_x b1~0: Rx Header Buffer Credit 00: LCRD_A/LCRD1_A/LCRD2_A 01: LCRD_B/LCRD1_B/LCRD2_B 10: LCRD_C/LCRD1_C/LCRD2_C 11: LCRD_D/LCRD1_D/LCRD2_D
		10: LRTY 11: LBAD		Reserved (0000)
01	LGO_Ux LAU LXU LPMA	00: LGO_Ux		0001: LGO_U1 0010: LGO_U2 0011: LGO_U3 Others: Reserved
		01: LAU 10: LXU 11: LPMA		Reserved (0000)
10	LDN LUP	00: LUP 11: LDN Others: Reserved		Reserved (0000)
11: Reserved	Reserved	Reserved (0000)		Reserved (0000)

USB 3.1 ENGINEERING CHANGE NOTICE

To Table 7-4:

Table 11-5 – Link Command Bit Definitions

Class		Type	b6~4	Sub-Type
b10~9	Link Command	b8~7		b3~0
00	LGOOD_n LRTY LBAD LCRD_x LCRD1_x LCRD2_x	00: LGOOD_n	Reserved (000)	SuperSpeed operation: b3: Reserved; b2~0: HP Sequence Number 000: LGOOD_0, 001: LGOOD_1, ...111: LGOOD_7 SuperSpeedPlus operation: b3~0: HP Sequence Number 0000: LGOOD_0, 0001: LGOOD_1, ... 1111: LGOOD_15
		01: LCRD_x LCRD1_x LCRD2_x		b3: Reserved b2: Credit series 0: LCRD_x or LCRD1_x 1: LCRD2_x b1~0: Rx Header Buffer Credit 00: LCRD_A/LCRD1_A/LCRD2_A 01: LCRD_B/LCRD1_B/LCRD2_B 10: LCRD_C/LCRD1_C/LCRD2_C 11: LCRD_D/LCRD1_D/LCRD2_D
		10: LRTY 11: LBAD		Reserved (0000)
01	LGO_Ux LAU LXU LPMA	00: LGO_Ux		0001: LGO_U1 0010: LGO_U2 0011: LGO_U3 Others: Reserved
		01: LAU 10: LXU 11: LPMA		Reserved (0000)
10	LDN LUP	00: LUP 11: LDN Others: Reserved		Reserved (0000)
11: Reserved	Reserved	Reserved (0000)		Reserved (0000)

Section 7.2.2.2

From Text:

USB 3.1 ENGINEERING CHANGE NOTICE

Successful header packet transactions between the two link partners require proper header packet acknowledgement. Rx Header Buffer Credit exchange facilitates link flow control. Header packet acknowledgement and Rx Header Buffer Credit exchange are realized using different link commands. LGOOD_n (n = 0 to 7) and LBAD are used to acknowledge whether a header packet has been received properly or not. LRTY is used to signal that a header packet is re-sent.

To Text:

Successful header packet transactions between the two link partners require proper header packet acknowledgement. Rx Header Buffer Credit exchange facilitates link flow control. Header packet acknowledgement and Rx Header Buffer Credit exchange are realized using different link commands. LGOOD_n and LBAD are used to acknowledge whether a header packet has been received properly or not. LRTY is used to signal that a header packet is re-sent.

Section 7.2.2.2

From Table 7-5:

Link Command	Definition – See Sections 7.2.4.1, 0, and 7.5.6 for detailed use and requirements.
LGOOD_n	n (0, 1, 2,7): Header Sequence Number. Sent by a port receiving a header packet when all of the following conditions are true: – The header packet has a valid structure and can be recognized by the receiver. – CRC-5 and CRC-16 are valid. – Header Sequence Number in the received header packet matches the expected Rx Header Sequence Number. – An Rx Header Buffer in the receiver is available for storing the received header packet. Mismatch between a Header Sequence Number in the received header packet and the expected Rx Header Sequence Number will result in a port transitioning to Recovery. Received by a port sending a header packet. This is an acknowledgement from a link partner that a header packet with the Header Sequence Number of "n" is received properly. Receipt of LGOOD_n mismatching the expected ACK Tx Header Sequence Number will result in a port transitioning to Recovery. Also sent by a port upon entry to U0 as the Header Sequence Number Advertisement to initialize the ACK Tx Header Sequence Number of the two ports. Refer to Section 7.2.4.1 for details.

To Table 7-5:

USB 3.1 ENGINEERING CHANGE NOTICE

Link Command	Definition – See Sections 7.2.4.1, 0, and 7.5.6 for detailed use and requirements.
LGOOD_n	n: Header Sequence Number. Sent by a port receiving a header packet when all of the following conditions are true: – The header packet has a valid structure and can be recognized by the receiver. – CRC-5 and CRC-16 are valid. – Header Sequence Number in the received header packet matches the expected Rx Header Sequence Number. – An Rx Header Buffer in the receiver is available for storing the received header packet. Mismatch between a Header Sequence Number in the received header packet and the expected Rx Header Sequence Number will result in a port transitioning to Recovery. Received by a port sending a header packet. This is an acknowledgement from a link partner that a header packet with the Header Sequence Number of “n” is received properly. Receipt of LGOOD_n mismatching the expected ACK Tx Header Sequence Number will result in a port transitioning to Recovery. Also sent by a port upon entry to U0 as the Header Sequence Number Advertisement to initialize the ACK Tx Header Sequence Number of the two ports. Refer to Section 7.2.4.1 for details.

Section 7.2.4.1.2

From Text:

- Header packets shall be sent with the Header Sequence Number in the numerical order from 0 to 7, and back to 0. LGOOD_n received out of the numerical order is considered as missing of a link command, and the transition to Recovery shall be initiated.

To Text:

- Header packets shall be sent with the Header Sequence Number in the numerical order from 0 to its maximum value, and back to 0. LGOOD_n received out of the numerical order is considered as missing of a link command, and the transition to Recovery shall be initiated.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB 3.1 wHubDelay

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Restore original 3.0 definition of wHubDelay so that hubs can report it and system software can use it consistently. 3.0 defined wHubDelay as the delay between receiving a header packet on one side (up or down) as indicated by having received the last symbol of the packet and starting to transmit a header packet on the other side (down or up). 3.1 redefined wHubDelay as the delay between the start of receiving a header packet and the start of transmitting it. In SuperSpeed operation, that's a 40 ns difference and unnecessarily complicates hubs and system software that calculates round trip times.

Benefits as a result of the changes:
Simplify hubs and system software that calculates round trip times by using a consistent definition for wHubDelay. A SuperSpeed host that uses wHubDelay will expect it to be reported as it is defined in 3.0; so, a SuperSpeedPlus hub operating in SuperSpeed mode should report the value with that definition in contradiction to 3.1. Restoring the wHubDelay from 3.0 avoids the contradiction, complications and confusion.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
None. There would be without this change.

An analysis of the hardware implications:
Simplifies design migration from 3.0 to 3.1.

An analysis of the software implications:
Simplifies new software and avoids need to update old software.

An analysis of the compliance testing implications:
TD 10.127 won't have to use different wHubDelay definitions for 3.0 and 3.1

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

Table 10-19. Hub Parameters

From Text:

Name	Description	Min	Max	Units
tHubDelay	When both the upstream and the downstream port are operating at the same speed this timing defines the maximum delay in nanoseconds a hub can introduce while forwarding packets in either direction. The time is measured from receipt of the first symbol of the packet by the receiving port until the transmitting port sends the first symbol of the packet, when both the receiving and transmitting links are in U0 and the following conditions are met: – No Link Commands are in flight. – Remote Rx Header Buffer Credit Count of the transmitting port is not zero. – Tx Header Buffer of the transmitting port is empty. A hub reports the actual delay via the wHubDelay field in Enhanced SuperSpeed Hub Descriptor.		400	ns

To Text:

Name	Description	Min	Max	Units
tHubDelay	When both the upstream and the downstream port are operating at the same speed this timing defines the maximum delay in nanoseconds a hub can introduce while forwarding header packets in either direction. The time is measured from receipt of the last symbol of the header packet by the receiving port until the transmitting port sends the first symbol of the header packet, when both the receiving and transmitting links are in U0 and the following conditions are met: – No Link Commands are in flight. – Remote Rx Header Buffer Credit Count of the transmitting port is not zero. – Tx Header Buffer of the transmitting port is empty. A hub reports the actual delay via the wHubDelay field in Enhanced SuperSpeed Hub Descriptor.		400	ns

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB 3.1 LTM

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional proposed:
The USB 3.1 Specification changes in this ECN are intended to facilitate use of Latency Tolerance Messaging (LTM) to achieve greater system power savings. This objective is already described in Appendix C.4 of the USB 3.1 Specification: “Computer systems typically maintain a high state of readiness to service devices even when the computer system is idle. LTM supports a mechanism for a system to reduce its state of readiness with the cooperation of Enhanced SuperSpeed devices. This may result in substantial system power savings without requiring additional cost to devices.” However, LTM as currently defined enables devices to advertise a BELT value less than tBELTdefault, which would generally increase system power consumption. To facilitate use of LTM for greater system power savings, two USB 3.1 Specification changes are requested: 1) Tighter definition of the legal range of BELT values permitted in Latency Tolerance Message Device Notifications. 2) Elimination of the recommendation to disable LTM before suspending a device. The objective of this recommendation was to ensure that BELT is no less than tBELTdefault while the device is suspended, and thereby avoid wasting power. With LTM BELT constrained to values greater than tBELTdefault, disabling LTM is no longer necessary.

Benefits as a result of the changes:
1) Constraining BELT to values greater than or equal to tBELTdefault will give software implementers confidence that enabling LTM cannot increase power consumption. 2) Eliminating the requirement to disable LTM before suspending a device will reduce software complexity, which will further increase software vendors' confidence in implementing LTM support.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
The changes herein have no known impact on systems that currently conform to the USB specification. LTM has not been widely enabled, in part due to the concern about increased power consumption.

An analysis of the hardware implications:
The changes herein have no known impact to existing designs. Device designs must already accommodate tBELTdefault when LTM is not enabled by software.

An analysis of the software implications:
The changes herein should simplify software support, as software won't have to toggle LTM_Enable when suspending and resuming a device.

An analysis of the compliance testing implications:
Any compliance testing exposure to BELT should be changed to reflect the change in BELT range specified below.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) Section 8.5.6.5.1, ‘Optional Normative LTM and BELT Requirements’

From Text:

General Device Requirements

...

- The minimum value for a BELT is 125 μ s (refer to Table 8-36).

To Text:

General Device Requirements

...

- The minimum value for a BELT is 1 ms (refer to Table 8-36).

(b) Table 8-36. Timing Parameters

From Text:

Name	Description	Min	Max	Units
...	...	...	...	...
tBELTmin	Minimum value of best effort latency tolerance allowed in a Latency Tolerance Message	125		μ s
...	...	...	...	...

To Text:

Name	Description	Min	Max	Units
...	...	...	...	...
tBELTmin	Minimum value of best effort latency tolerance allowed in a Latency Tolerance Message	1		ms
...	...	...	...	...

(c) Appendix C.4.1, ‘Device State Machine Implementation Example’

From Text:

In this example, two device Latency Tolerance states (LT-states) are defined:

- **LT-idle state:** the device is idle and can tolerate a larger latency from the system (this is the default state).
- **LT-active state:** the device has determined a need to perform data transfers with the host and wants a shorter latency from the system.

A state machine is illustrated in Figure C-8.

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

In this example, two device Latency Tolerance states (LT-states) are defined:

- **LT-idle state:** the device is idle and can tolerate a larger latency from the system (this is the default state).
- **LT-active state:** the device has determined a need to perform data transfers with the host and wants a shorter latency from the system.

A state machine for this example is illustrated in Figure C-8.

NOTE the state machine described in this example is implementation-specific and is not intended to constrain implementers' design choices in any way. For example, another implementation might use the same BELT value at all times.

(d) Appendix C.4.1, 'Device State Machine Implementation Example'

From Text:

The following device design goals are to be met:

- Design for a minimum LTM BELT of 1 ms when in LT-idle
- Design for a minimum LTM BELT of 125 μ s when in LT-active

To Text:

The following device design goals are to be met:

- Design for a minimum LTM BELT of 3 ms when in LT-idle
- Design for a minimum LTM BELT of 1 ms when in LT-active

(e) Appendix C.4.1.1, 'LTM-Idle State BELT'

From Text:

To achieve a minimum LT-idle state BELT of 1 ms, the total latency the device must be able to tolerate is 1 ms plus the worst case value for U2SEL, or a total of approximately 3,1 ms (refer to Section C.1.5.1). The worst case U2SEL is based on a worst case device-to-host U2 exit latency of 2,053 ms for t1 (2,047 ms device exit latency plus 1 μ s for each of five hubs), plus 0,003 ms for t2, plus 0,001 ms for t4, plus some guard band.

For system implementations where U2SEL is less than its worst case value, the device reports a BELT value larger than 1 ms.

To Text:

To achieve a minimum LT-idle state BELT of 3 ms, the total latency the device must be able to tolerate is 3 ms plus the worst case value for U2SEL, or a total of approximately 5,1 ms (refer to Section C.1.5.1). The worst case U2SEL is based on a worst case device-to-host U2 exit latency of 2,053 ms for t1 (2,047 ms device exit latency plus 1 μ s for each of five hubs), plus 0,003 ms for t2, plus 0,001 ms for t4, plus some guard band.

For system implementations where U2SEL is less than its worst case value, the device reports a BELT value larger than 3 ms.

USB 3.1 ENGINEERING CHANGE NOTICE

(f) Appendix C.4.1.2, 'LTM-Active State BELT'

From Text:

To achieve a minimum LT-active state BELT of 125 μ s, the total latency the device must be able to tolerate is 125 μ s plus the maximum value for U1SEL, or a total of approximately 145 μ s. The worst case U1SEL is based on a worst case device-to-host U1 exit latency of 15 μ s for t1 (10 μ s device exit latency plus 1 μ s for each of five hubs), plus 3,1 μ s for t2, plus 1,3 μ s for t4, plus some guard band. This assumes the device will not allow its link to enter U2 prior to changing its state to LT-idle. If the device will allow its link to enter U2 when in LT-active, then the total latency the device must be able to tolerate is 125 μ s plus the worst case value for U2SEL.

For system implementations where U1SEL is less than its worst case value, the device reports a BELT value larger than 125 μ s.

To Text:

To achieve a minimum LT-active state BELT of 1 ms, the total latency the device must be able to tolerate is 1 ms plus the maximum value for U1SEL, or a total of approximately 1,020 ms. The worst case U1SEL is based on a worst case device-to-host U1 exit latency of 15 μ s for t1 (10 μ s device exit latency plus 1 μ s for each of five hubs), plus 3,1 μ s for t2, plus 1,3 μ s for t4, plus some guard band. This assumes the device will not allow its link to enter U2 prior to changing its state to LT-idle. If the device will allow its link to enter U2 when in LT-active, then the total latency the device must be able to tolerate is 1 ms plus the worst case value for U2SEL.

For system implementations where U1SEL is less than its worst case value, the device reports a BELT value larger than 1 ms.

(g) Appendix C.4.2, 'Other Considerations'

From Text:

The following are additional considerations associated with device support of LTM:

- The BELT represents a latency tolerance for an entire peripheral device. The BELT value must be aggregated across all endpoints within the device, including all functions within a composite device. The smallest BELT value across all endpoints should be selected. For LTM purposes, isochronous endpoints are ignored when determining the BELT value.
- If LTM is supported by a device, LTM should be disabled prior to placing the device into suspend (refer to the PORT_LINK_STATE feature selector in Chapter 10). Devices send an updated LTM when LTM is enabled, or immediately before LTM is disabled, as defined in Chapter 8. Disabling LTM in this way ensures that a suspended device does not keep the system in a high state of readiness, wasting power.

To Text:

- The BELT represents a latency tolerance for an entire peripheral device. The BELT value must be aggregated across all endpoints within the device, including all functions within a composite device. The smallest BELT value across all endpoints should be selected. For LTM purposes, isochronous endpoints are ignored when determining the BELT value.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: Polling LFPS

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
To modify the operational behavior of a SSP port connecting with a SS port in the Polling.LFPS substate to achieve the following: 1) Clarify SSP behavior switching from SCD1 to Polling.LFPS 2) Relax transition conditions from Polling.LFPS to Polling.RxEQ for a SSP port to achieve minimum exit delay by not requiring additional 20 Polling.LFPS transmission if exit conditions to Polling.RxEQ are met.
Benefits as a result of the changes:
1) Clarify behavioral requirement of a SSP port when connecting to a SS port. 2) Backwards compatibility to legacy SS operation.
An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
Implementation changes are expected.
An analysis of the hardware implications:
Logic only to cover behavioral changes when SSP switching to SS operation.
An analysis of the software implications:
None
An analysis of the compliance testing implications:
Corresponding changes to be expected.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

From Text:

7.5.4.3.1 Polling.LFPS Requirements

- Upon entry, a LFPS receiver shall be enabled to receive the Polling.LFPS signals defined in Section 6.9.1.
- Upon entry, a port shall establish its LFPS operating condition within 80 μ s.
- A downstream port shall disable its transition path to Compliance Mode upon PowerOn Reset or Warm Reset.
- A downstream port shall enable its transition path to Compliance Mode, if directed.
- An upstream port always has its transition path to Compliance Mode enabled upon PowerOn Reset.
- A port in SuperSpeed operation shall transmit Polling.LFPS.
- An upstream port in SuperSpeedPlus operation shall transmit SCD1 defined in Table 6-32. If no signature of SCD1 is found in sixteen consecutive Polling.LFPS received, the port shall switch to SuperSpeed operation and transmit Polling.LFPS instead of SCD1.
- A downstream port in SuperSpeedPlus operation shall transmit SCD1 defined in Table 6-32 if Compliance Mode is disabled. If no signature of SCD1 is found in sixteen consecutive Polling.LFPS received, the port shall switch to SuperSpeed operation and transmit Polling.LFPS instead of SCD1.
- A downstream port in SuperSpeedPlus operation shall transmit SCD2 if Compliance Mode is enabled.
- Note: In case a retimer is connected to the downstream port, SCD2 is used for the retimer to enable Compliance Mode. Refer to Appendix F for details.
- A port in SuperSpeedPlus operation shall implement a 60- μ s timer ($t_{\text{PollingSCDLFPSTimeout}}$) to monitor the absence of LFPS signal after the completion of SuperSpeed Polling.LFPS handshake-
- A port in SuperSpeedPlus operation shall be ready for SuperSpeed operation if it has detected that its link partner operates at SuperSpeed.
- Note: There is no time allocated for a SuperSpeedPlus port to re-configure itself for SuperSpeed operation upon detection of its link partner operating at SuperSpeed. A SuperSpeedPlus port shall switch to SuperSpeed operation as quickly as possible for its receiver equalization training. Any time for configuration will result in receiving fewer TSEQ ordered sets from its link partner.
- A port shall disable its transition path to Compliance Mode when it has successfully completed Polling.LFPS handshake or has entered Compliance Mode.
- A 360-ms timer ($t_{\text{PollingLFPSTimeout}}$) shall be started upon entry to the substate.
- The operating condition of an eSS PHY shall be established when a port is ready to exit to Polling.RxEQ.
- An eSS receiver in SuperSpeed operation may optionally be enabled to receive TSEQ ordered sets for receiver equalizer training.
- Note: The port first entering Polling.RxEQ will start transmitting TSEQ ordered sets while the other port is still in Polling.LFPS. Enabling a SuperSpeed receiver in Polling.LFPS will allow a port to start the receiver equalizer training while completing the requirement for Polling.LFPS exit handshake.

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

7.5.4.3.1 Polling.LFPS Requirements

- Upon entry, a LFPS receiver shall be enabled to receive the Polling.LFPS signals defined in Section 6.9.1.
- Upon entry, a port shall establish its LFPS operating condition within 80 μ s.
- A downstream port shall disable its transition path to Compliance Mode upon PowerOn Reset or Warm Reset.
- A downstream port shall enable its transition path to Compliance Mode, if directed.
- An upstream port shall always have its transition path to Compliance Mode enabled upon PowerOn Reset.
- A port in SuperSpeed operation shall transmit Polling.LFPS.
- An upstream port in SuperSpeedPlus operation shall transmit SCD1 defined in Table 6-32. It shall perform in one of the following ways if no signature of SCD1 or SCD2 is detected within the received Polling.LFPS bursts.

- 1) If it has received sixteen or more consecutive Polling.LFPS bursts and the $t_{\text{PollingSCDLFPSTimeout}}$ timer has not expired, it shall switch to SuperSpeed operation and transmit Polling.LFPS instead of SCD1.

NOTE This may imply that its SuperSpeed link partner is sensitive to varying t_{Repeat} of the Polling.LFPS bursts of SCD1.

- 2) If the $t_{\text{PollingSCDLFPSTimeout}}$ timer has expired, it shall switch to SuperSpeed operation in preparation to transition to Polling.RxEQ when all exit conditions are met.

NOTE This may imply that its SuperSpeed link partner may enter Polling.LFPS first, and may be insensitive to varying t_{Repeat} of SCD1 or SCD2, and has met all the exit conditions to Polling.RxEQ.

- A downstream port in SuperSpeedPlus operation shall transmit SCD1 defined in Table 6-32 if Compliance Mode is disabled. It shall perform in one of the following ways if no signature of SCD1 or SCD2 is detected within the received Polling.LFPS bursts.
- 1) If it has received sixteen or more consecutive Polling.LFPS bursts and the $t_{\text{PollingSCDLFPSTimeout}}$ timer has not expired, it shall switch to SuperSpeed operation and transmit Polling.LFPS instead of SCD1.

NOTE This may imply that its SuperSpeed link partner is sensitive to varying t_{Repeat} of the Polling.LFPS bursts of SCD1.

- 2) If the $t_{\text{PollingSCDLFPSTimeout}}$ timer has expired, it shall switch to SuperSpeed operation in preparation to transition to Polling.RxEQ when all exit conditions are met.

NOTE This may imply that its SuperSpeed link partner may enter Polling.LFPS first, and may be insensitive to varying t_{Repeat} of SCD1 or SCD2, and has met all the exit conditions to Polling.RxEQ.

- A downstream port in SuperSpeedPlus operation shall transmit SCD2 if Compliance Mode is enabled.

NOTE In case a retimer is connected to the downstream port, SCD2 is used for the retimer to enable Compliance Mode. Refer to Appendix E for details.

- A port in SuperSpeedPlus operation shall implement a 60- μ s timer ($t_{\text{PollingSCDLFPSTimeout}}$) to monitor the absence of LFPS signal after the completion of SuperSpeed Polling.LFPS handshake. During this period, the port shall continue the transmission of Polling.LFPS or SCD1 until its expiration.
- A port in SuperSpeedPlus operation shall be ready for SuperSpeed operation if it has detected that its link partner operates at SuperSpeed.

NOTE There is no time allocated for a SuperSpeedPlus port to re-configure itself for SuperSpeed operation upon detection of its link partner operating at SuperSpeed. A SuperSpeedPlus port shall switch to SuperSpeed

USB 3.1 ENGINEERING CHANGE NOTICE

operation as quickly as possible for its receiver equalization training. Any time for configuration will result in receiving fewer TSEQ ordered sets from its link partner.

- A port shall disable its transition path to Compliance Mode when it has successfully completed Polling.LFPS handshake or has entered Compliance Mode.
- A 360-ms timer ($t_{\text{PollingLFPSTimeout}}$) shall be started upon entry to the substate.
- The operating condition of an eSS PHY shall be established when a port is ready to exit to Polling.RxEQ.
- An eSS receiver in SuperSpeed operation may optionally be enabled to receive TSEQ ordered sets for receiver equalizer training.

NOTE The port first entering Polling.RxEQ will start transmitting TSEQ ordered sets while the other port is still in Polling.LFPS. Enabling a SuperSpeed receiver in Polling.LFPS will allow a port to start the receiver equalizer training while completing the requirement for Polling.LFPS exit handshake.

From Text:

7.5.4.3.2 Exit from Polling.LFPS

- The port in SuperSpeed operation shall transition to Polling.RxEQ when the following three conditions are met:
 - 1) At least 16 consecutive Polling.LFPS bursts meeting the Polling.LFPS specification defined in Section 6.9 are sent.
 - 2) Two consecutive Polling.LFPS bursts are received.
 - 3) Four consecutive Polling.LFPS bursts are sent after receiving one Polling.LFPS burst.
- The port in SuperSpeedPlus operation shall transition to Polling.LFPSPlus if two SCD1 are transmitted after one SCD1 or SCD2 as defined in Section 6.9.4.2 is received.
- The port in SuperSpeedPlus operation shall transition to Polling.RxEQ and switch to SuperSpeed operation if the following conditions are met:
 - 1) At least two consecutive Polling.LFPS bursts are received.
 - 2) Twenty Polling.LFPS bursts are transmitted, and no SCD1 is detected.

NOTE This condition guarantees that, in the case of a port in SuperSpeedPlus operation connecting to a port in SuperSpeed operation, a port in SuperSpeed operation will receive twenty consecutive Polling.LFPS to exit from this substate if it is unable to recognize Polling.LFPS with varying t_{Repeat} in SCD1.

- 3) No LFPS signal for more than $t_{\text{PollingSCDLFPSTimeout}}$ is observed.

NOTE This condition implies the SuperSpeed link partner has entered Polling.RxEQ transmitting TSEQ ordered sets.

- An upstream port shall transition to Compliance Mode upon the 360-ms timer timeout ($t_{\text{PollingLFPSTimeout}}$) and the following two conditions are met:
 - 1) The port has never successfully completed Polling.LFPS after PowerOn Reset.
 - 2) The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.

NOTE If the very first attempt in Polling.LFPS handshake fails after PowerOn Reset, it implies that a passive test load may be present and compliance test should be initiated. If the very first attempt in Polling.LFPS handshake succeeds after PowerOn Reset, it implies the presence of the Enhanced SuperSpeed ports on each side of the link and no compliance test is intended. Therefore, any subsequent handshake timeout in Polling.LFPS when the link is retrained is only an indication of link training failure, not a signal to enter Compliance Mode.

- A downstream port shall transition to Compliance Mode upon the 360-ms timer timeout ($t_{\text{PollingLFPSTimeout}}$) if the following three conditions are met:
 - 1) The Compliance Mode is enabled.
 - 2) The port has never successfully completed Polling.LFPS handshake after Compliance Mode is enabled.

USB 3.1 ENGINEERING CHANGE NOTICE

3) The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.

NOTE In case Compliance mode is disabled, a downstream port may enter Rx.Detect attempting Polling.LFPS handshake again, or enter eSS.Inactive for SW intervention based on the count value of cPollingTimeout.

- A downstream port shall transition to Rx.Detect upon the 360-ms timer timeout (tPollingLFPSTimeout) if cPollingTimeout is less than two and Compliance Mode is disabled.
- A downstream port shall transition to eSS.Inactive upon the 360-ms timer timeout (tPollingLFPSTimeout) and cPollingTimeout is two.
- An upstream port of a hub shall transition to Rx.Detect upon the 360-ms timer timeout (tPollingLFPSTimeout) after having trained once since PowerOn Reset and the conditions to transition to Polling.RxEQ are not met.
- A peripheral device shall transition to eSS.Disabled upon the 360-ms timer timeout (tPollingLFPSTimeout) after having trained once since PowerOn Reset and the conditions to transition to Polling.RxEQ are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

To Text:

7.5.4.3.3 Exit from Polling.LFPS

- The port in SuperSpeed operation shall transition to Polling.RxEQ when the following three conditions are met:
 - 1) At least 16 consecutive Polling.LFPS bursts meeting the Polling.LFPS specification defined in Section 6.9 are sent.
 - 2) Two consecutive Polling.LFPS bursts are received.
 - 3) Four consecutive Polling.LFPS bursts are sent after receiving one Polling.LFPS burst.
- The port in SuperSpeedPlus operation shall transition to Polling.LFPSPlus if two SCD1 are transmitted after one SCD1 or SCD2 as defined in Section 6.9.4.2 is received.
- The port in SuperSpeedPlus operation shall transition to Polling.RxEQ and switch to SuperSpeed operation if the following conditions are met:
 - 1) No SCD1 or SCD2 is detected within the received Polling.LFPS bursts.
 - 2) At least four consecutive SCD1 are transmitted.
 - 3) Two consecutive Polling.LFPS bursts are received.
 - 4) One SCD1 or four consecutive Polling.LFPS bursts are transmitted after receiving one Polling.LFPS burst.
 - 5) No LFPS signal for more than tPollingSCDLFPSTimeout is observed.

NOTE This condition implies the SuperSpeed link partner has entered Polling.RxEQ transmitting TSEQ ordered sets.

- An upstream port shall transition to Compliance Mode upon the 360-ms timer timeout (tPollingLFPSTimeout) and the following two conditions are met:
 - 1) The port has never successfully completed Polling.LFPS after PowerOn Reset.
 - 2) The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.

NOTE If the very first attempt in Polling.LFPS handshake fails after PowerOn Reset, it implies that a passive test load may be present and compliance test should be initiated. If the very first attempt in Polling.LFPS handshake succeeds after PowerOn Reset, it implies the presence of the Enhanced SuperSpeed ports on each side of the link and no compliance test is intended. Therefore, any subsequent

USB 3.1 ENGINEERING CHANGE NOTICE

handshake timeout in Polling.LFPS when the link is retrained is only an indication of link training failure, not a signal to enter Compliance Mode.

- A downstream port shall transition to Compliance Mode upon the 360-ms timer timeout (tPollingLFPSTimeout) if the following three conditions are met:
 - 1) The Compliance Mode is enabled.
 - 2) The port has never successfully completed Polling.LFPS handshake after Compliance Mode is enabled.
 - 3) The condition to transition to Polling.RxEQ or Polling.LFPSPlus is not met.

NOTE In case Compliance mode is disabled, a downstream port may enter Rx.Detect attempting Polling.LFPS handshake again, or enter eSS.Inactive for SW intervention based on the count value of cPollingTimeout.

- A downstream port shall transition to Rx.Detect upon the 360-ms timer timeout (tPollingLFPSTimeout) if cPollingTimeout is less than two and Compliance Mode is disabled.
- A downstream port shall transition to eSS.Inactive upon the 360-ms timer timeout (tPollingLFPSTimeout) and cPollingTimeout is two.
- An upstream port of a hub shall transition to Rx.Detect upon the 360-ms timeout (tPollingLFPSTimeout) after having trained once since PowerOn Reset and the conditions to transition to Polling.RxEQ are not met.
- A peripheral device shall transition to eSS.Disabled upon the 360-ms timeout (tPollingLFPSTimeout) after having trained once since PowerOn Reset and the conditions to transition to Polling.RxEQ are not met.
- A downstream port shall transition to eSS.Disabled when directed.
- A downstream port shall transition to Rx.Detect when directed to issue Warm Reset.
- An upstream port shall transition to Rx.Detect when Warm Reset is detected.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB 3.1 PTM Value

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
The PTM value was left out in the list of LMP Types. This ECN sets a value for the same.

Benefits as a result of the changes:
Implementers will be able to correctly send/receive a PTM LMP.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
This ECN is required for PTM to work successfully

An analysis of the hardware implications:
This ECN is required for PTM to work successfully

An analysis of the software implications:
None.

An analysis of the compliance testing implications:
This ECN is required to be able to test PTM implementations

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

From Text: Table 8-3 Link Management Packet Subtype Field

0111b-1111b	Reserved
-------------	----------

To Text:

0111b	Precision Time Measurement
1000b-1111b	Reserved

USB 3.1 ENGINEERING CHANGE NOTICE

Title: SKP OS Bytes During Compliance Clarification

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Correct typographical errors in Table 6-12, which defines the SKP ordered set for SuperSpeed Gen 2. First, the table refers to values for SKP OS bytes during compliance. However, SKP OS transmission is not allowed in compliance mode. The proposed change resolves this inconsistency. Second, there is a reference to sending a parity bit but this functionality is not defined in USB 3.1. Once again, removing this text fixes this issue.

Benefits as a result of the changes:
Correction to incorrect specifications.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No impact

An analysis of the hardware implications:
None

An analysis of the software implications:
None

An analysis of the compliance testing implications:
None

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) From Text (and location): Table 6-12, Section 6.4.2, page 158

2*N+1	00-FFh	(i) If prior block was a Data Block: Bit[7] = Even Data Parity Bit[6:0] = LFSR[22:16] (ii) Else: Bit[7] = ~LFSR[22] Bit[6:0] = LFSR[22:16]
2*N+2	00-FFh	(i) If LTSSM state is Compliance mode: Error_Status[7:0] (ii) Else LFSR[15:8]
2*N+3	00-FFh	(i) If LTSSM state is Compliance mode: ~Error_Status[7:0] (ii) Else LFSR[7:0]

(a) To Text (and location): Table 6-12, Section 6.4.2, page 158

2*N+1	40-BFh	Bit[7] = ~LFSR[22] Bit[6:0] = LFSR[22:16]
2*N+2	00-FFh	LFSR[15:8]
2*N+3	00-FFh	LFSR[7:0]

(b) From Text (and location): Section 6.4.2, page 156

- 1) The Data parity bit should be even parity for last three symbols in the SKP OS. The parity is a check of the LFSR seed value.

(c) From Text (and location): Section 6.4.2, page 156

(no text – item 5 deleted)

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB3.1 SKP Ordered Set Definition

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Section 6.4.3.2 contains the SKP Order Set Rules for Gen2 operation. The current SKP OS set definition is geared toward minimizing the BW overhead of SKP OS, but requires re-timers to meet exceptionally stringent elastic buffer design requirements. The current definition specifies that 1 SKP OS (12 SKPs) shall be transmitted once every 22 to 90 blocks, which requires elastic buffers to meet a very stringent SKP removal quantization error threshold. If a given elastic buffer exceeds the SKP removal quantization error threshold, then downstream elastic buffers may be starved of SKPs, leading to an elastic buffer overflow condition. There is also a concern that the worst case dynamic PPM profile across the re-timers has not been discovered and could lead to more stringent/resource intensive requirements for re-timers/endpoints. To mitigate these risk factors, this ECN changes the SKP OS definition to guarantee that each elastic buffer receives enough SKPs in a SKP OS to fully compensate for SI drift that occurred between a SKP OS interval, regardless of the SKP removal quantization error each elastic buffer introduces. In order to accomplish this, the transmitted SKP OS shall contain 20 SKPs (4 SKPs * 5 Elastic Buffers), and be transmitted on average of once every 40 blocks since the last transmitted SKP Ordered Set. The maximum SI drift over 40 blocks is less than 4. Therefore, upon receiving a SKP OS each re-timer in the link will be allowed to add 4 SKPs, remove 4 SKPs, or make no adjustments. This definition guarantees that each elastic buffer will have the opportunity to remove 4 SKPs for every received SKP OS, even when all other elastic buffers in the link remove 4 SKPs.

Benefits as a result of the changes:
This change drastically reduces the re-timer elastic buffer design requirements and the overall risk associated with clock frequency compensation for USB3.1.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No certified SSP product yet.

An analysis of the hardware implications:
Re-timer design based on USB3.1 specification has not begun at this point, so this will be the baseline. Host and Device will need to make moderate modification to SKP OS handling/processing logic.

An analysis of the software implications:
None

An analysis of the compliance testing implications:
Compliance testing complexity for SKP OS will be drastically reduced. In order to verify that the SKP OS definition is being met, the compliance test will need to make sure a port either removes 4 SKPs, adds 4 SKPs or does nothing for each received SKP Ordered set.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

Section 6.3.2.2 Normative 128b/132b Decode Rules

From Text:

The physical layer shall encode the data on a per block basis. Each block shall comprise a 4-bit Block Header and a 128-bit payload. The 4-bit header is set to 0011b for data and 1100b for control blocks. This header format allows for the correction of single bit errors in the header information.

To Text:

The physical layer shall encode the data on a per block basis. Each block, except for the SKP Ordered Set control block, shall comprise a 4-bit Block Header and a 128-bit payload. The SKP Ordered Set control block shall be comprised of a 4-bit Block Header and a 192-bit payload. The 4-bit header is set to 0011b for data and 1100b for control blocks. This header format allows for the correction of single bit errors in the header information.

USB 3.1 ENGINEERING CHANGE NOTICE

Section 6.4.3 Elasticity Buffer and SKP Ordered Set

From Text:

The Enhanced SuperSpeed architecture supports a separate reference clock source on each side of the Enhanced SuperSpeed link. The accuracy of each reference clock is required to be within ± 300 ppm. This gives a maximum frequency difference between the two devices of the link of ± 600 ppm. In addition, SSC creates a frequency delta that has a maximum difference of 5000 ppm. The total magnitude of the frequency delta can range from -5300 to 300 ppm. This frequency delta is managed by an elasticity buffer that consumes or inserts SKP ordered sets.

SKP Ordered Sets shall be used to compensate for frequency differences between the two ends of the link. The transmitter sends SKP ordered sets at an average of every 354 symbols. However, SKP ordered sets shall not be inserted within any packet. The transmitter is allowed to buffer the SKP ordered sets up to a maximum of four SKP ordered sets. For Gen 1 operation the receiver shall implement an elasticity buffer capable of buffering (or starving) eight symbols of data. For Gen 2 operation, due to the presence of retimers along the signal path, a receiver shall tolerate not receiving any SKP Symbols for up to 180 blocks. Thus during Gen 2 operation a receiver must implement an elasticity buffer capable of buffering (or starving) twenty two symbols of data.

To Text:

The Enhanced SuperSpeed architecture supports a separate reference clock source on each side of the Enhanced SuperSpeed link. The accuracy of each reference clock is required to be within ± 300 ppm. This gives a maximum frequency difference between the two devices of the link of ± 600 ppm. In addition, SSC creates a frequency delta that has a maximum difference of 5 000 ppm. The total magnitude of the frequency delta can range from -5 300 to 300 ppm. This frequency delta is managed by an elasticity buffer that consumes or inserts SKP ordered sets.

SKP Ordered Sets shall be used to compensate for frequency differences between the two ends of the link.

For Gen1 operation, the transmitter sends SKP ordered sets at an average of every 354 symbols. However, SKP ordered sets shall not be inserted within any packet. The transmitter is allowed to buffer the SKP ordered sets up to a maximum of four SKP ordered sets. For Gen 1 operation the receiver shall implement an elasticity buffer capable of buffering (or starving) eight symbols of data.

For Gen 2 operation, the average interval between transmitted SKP Ordered Sets is 40 blocks. However, SKP Ordered Sets shall not be inserted within any packet. Consequently, the transmitter is allowed to buffer up to three SKP Ordered Sets. For Gen 2 operation the receiver shall implement an elasticity buffer capable of buffering (or starving) eleven symbols of data.

USB 3.1 ENGINEERING CHANGE NOTICE

Section 6.4.3.2 SKP Rules (Host/Device/Hub) for Gen 2 Operation

From Text:

Table 6-12 describes the layout of the SKP Ordered Set when using 128b/132b encoding. A transmitted SKP Ordered Set always starts out as 16 Symbols long. The granularity for which SKP Symbols can be added or removed by a Port is two symbols. A port may add or remove more than 2 SKP symbols, but the number of SKP symbols that is added or removed shall be a multiple of two. This includes retimers within the signal path. Thus, a receiver may receive a SKP OS with anywhere from 0 to thirty six SKP symbols with the number of SKP symbols being a multiple of two. A SKP OS with 0 SKP symbols has only a SKPEND symbol followed by the three symbols that describe the LFSR state. Another impact of receiving variable length SKP OS is that a receiver is always allowed to add up to 12 SKP symbols to any SKP OS regardless of the length of the received SKP OS.

The SKPEND Symbol indicates the last four Symbols of SKP Ordered Set so that receivers can identify the location of the next Block Header in the bit stream. The three Symbols following the SKPEND Symbol contain different information depending on the LTSSM state.

A receiver must always perform single bit error correction on the SKP and SKPEND (and all other special) symbols. However, since the Hamming distance between the SKP and SKPEND symbols is 8, once a receiver has determined that it is dealing with a non-empty SKP OS (by proper detection of a first SKP symbol) it may be beneficial to use multiple bit (up to 3-bit) error correction in differentiating between a SKP and a SKPEND symbol.

Table 11-6 – Gen 2 SKP Ordered Set

Symbol Number	Value	Description
0 through $2*N-1$ [N can be 0 through 18]	CCh	SKP Symbol Symbol 0 is the SKP Ordered Set Identifier Note: for an empty SKP OS, the first symbol will be a SKPEND.
$2*N$	33h	SKPEND Symbol
$2*N+1$	00-FFh	(i) If prior block was a Data Block: Bit[7] = Even Data Parity Bit[6:0] = LFSR[22:16] (ii) Else: Bit[7] = $\sim$ LFSR[22] Bit[6:0] = LFSR[22:16]
$2*N+2$	00-FFh	(i) If LTSSM state is Compliance mode: Error_Status[7:0] (ii) Else LFSR[15:8]
$2*N+3$	00-FFh	(i) If LTSSM state is Compliance mode: $\sim$ Error_Status[7:0] (ii) Else LFSR[7:0]

The following rules apply for SKP insertion for Gen 2 operation:

- 1) A transmitter shall keep a running count of the number of transmitted blocks since the last SKP Ordered set. The value of this count will be referred to as Y. The value of Y is reset whenever the transmitter enters Polling.Active or when a SKP OS is transmitted.
- 2) Once the count, Y, gets to 21 a transmitter must insert a SKP OS at the next legitimate opportunity. The fastest a transmitter can insert SKP OS is once every 22 blocks. Situations that delay the immediate insertion of a SKP OS are the following: a transmitter shall not interrupt a data packet or a SYNC OS to insert a SKP OS. In the worst case it may take 90 blocks before there is an opportunity to insert a SKP OS. In Gen 2 operation

USB 3.1 ENGINEERING CHANGE NOTICE

there is no accumulation of SKP OS, each time a SKP OS is transmitted the SKP counter, Y, is reset to 0.

- 3) SKP Ordered Sets do not count as interruptions when monitoring for Ordered Sets (i.e., consecutive TS1, TS2 Ordered Sets in Polling and Recovery).
- 4) SYNC ordered sets have priority of SKP ordered sets. A SKP OS that is scheduled to be sent at the same time as a SYNC OS shall have to be delayed until the SYNC OS is transmitted.
- 5) The Data parity bit should be even parity for last three symbols in the SKP OS. The parity is a check of the LFSR seed value.

To Text:

Table 6-12 describes the layout of the SKP Ordered Set for Gen2 operation. A transmitted SKP Ordered Set is 24 symbols. The granularity for which SKP Symbols can be added or removed is four SKP symbols. Upon receiving a SKP ordered set, a re-timer shall perform one and only one of the following adjustments: add four SKPs, remove four SKPs, or make no adjustment. Thus, a received SKP OS can have anywhere from 4 to 36 SKP symbols with the number of SKP symbols being a multiple of four.

The SKPEND Symbol indicates the last four Symbols of SKP Ordered Set so that receivers can identify the location of the next Block Header in the bit stream. The three Symbols following the SKPEND Symbol contain the transmitter LFSR state.

A receiver shall always perform single bit error correction on the SKP and SKPEND (and all other special) symbols. However, since the Hamming distance between the SKP and SKPEND symbols is 8, once a receiver has determined that it is dealing with a SKP OS (by proper detection of a first SKP symbol) it may be beneficial to use multiple bit (up to 3-bit) error correction in differentiating between a SKP and a SKPEND symbol.

Table 11-7 – Gen 2 SKP Ordered Set

Symbol Number	Value	Description
0 through 4*N-1 [N can be 1 through 9]	CCh	SKP Symbol Symbol 0 is the SKP Ordered Set Identifier
4*N	33h	SKPEND Symbol
4*N+1	40-BFh	Bit[7] = ~LFSR[22] Bit[6:0] = LFSR[22:16]
4*N+2	00-FFh	LFSR[15:8]
4*N+3	00-FFh	LFSR[7:0]

NOTE The transmitted LFSR state is intended for use by test equipment vendors needing to re-synch their data scramblers. The transmitted LFSR state is not intended to be used by ports in normal operation.

The following rules apply for SKP insertion for Gen 2 operation:

- 1) A port shall keep a running count of the number of transmitted blocks since the last SKP Ordered Set. The value of this count will be referred to as Y. The value of Y is reset whenever the transmitter enters Polling.Active. Y is not incremented for transmitted SKP Ordered Sets.
- 2) A port shall calculate the integer result of Y/40 when an opportunity to insert a SKP Ordered Set arises. The integer result of Y/40 is the number of accumulated SKP Ordered

USB 3.1 ENGINEERING CHANGE NOTICE

Sets that need to be transmitted – this value will be referred to as Z. The value of Z can be either 0, 1, 2, or 3.

NOTE The non-integer remainder of the $Y/40$ SKP calculation shall not be discarded and shall be used in the calculation to schedule the next SKP Ordered Set.

- 3) Unless otherwise specified, when the LTSSM is not in the loopback state, a transmitter shall insert Z SKP Ordered Sets immediately after each transmitted SYNC, TS1, TS2, SDS, LMP, Header Packet, Data Packet, or Logical idle. When the LTSSM is in the Loopback state, the Loopback Master transmitter shall insert $2 \cdot Z$ SKP Ordered Sets immediately after each transmitted SYNC, TS1, TS2, SDS, LMP, Header Packet, Data Packet, or Logical idle. A transmitter shall not transmit SKP Ordered Sets at any other time.
- 4) SKP Ordered Sets do not count as interruptions when monitoring for Ordered Sets (i.e., consecutive TS1, TS2 Ordered Sets in Polling and Recovery).
- 5) SYNC ordered sets have priority over SKP ordered sets. A SKP OS that is scheduled to be sent at the same time as a SYNC OS shall be delayed until the SYNC OS is transmitted.
- 6) The Data parity bit should be even parity for last three symbols in the SKP OS. The parity is a check of the LFSR seed value.

USB 3.1 ENGINEERING CHANGE NOTICE

6.8.4 Receiver Loopback

From Text:

The entry and exit process for receiver loopback is described in Chapter 7.

Receiver loopback must be retimed. Direct connection from the Rx amplifier to the transmitter is not allowed for loopback mode. The receiver shall continue to process SKPs as appropriate. SKP symbols shall be consumed or inserted as required for proper clock tolerance compensation. Over runs or under runs of the clock tolerance buffers will reset the buffers to the neutral position.

During loopback the receiver shall process the Bit Error Rate Test (BERT) commands.

Loopback shall occur in the 10-bit domain for Gen 1 operation and in the 132-bit domain for Gen 2 operation. No error correction is allowed. All symbols shall be transmitted as received with the exception of SKP and BERT commands.

To Text:

The entry and exit process for receiver loopback is described in Chapter 7.

Receiver loopback must be retimed. Direct connection from the Rx amplifier to the transmitter is not allowed for loopback mode. The receiver shall continue to process SKP Ordered Sets as appropriate. For Gen1 operation, SKP symbols shall be consumed or inserted as required for proper clock tolerance compensation. For Gen2 operation, the receiver can either add 4 SKPs, remove 4 SKPs or make no adjustment to the received SKP Ordered Set. The modified SKP Ordered Set shall meet the requirements specified in Section 6.4.3.2 (i.e. must contain between 4 and 36 SKPs followed by the SKPEND Symbol and 3 Symbols that proceed the SKPEND.) Over runs or under runs of the clock tolerance buffers will reset the buffers to the neutral position.

During loopback the receiver shall process the Bit Error Rate Test (BERT) commands.

Loopback shall occur in the 10-bit domain for Gen 1 operation and in the 132-bit domain for Gen 2 operation. No error correction is allowed. All symbols shall be transmitted as received with the exception of SKP and BERT commands.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: SLC IS SDS

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
1) The SLC symbol is the same as IS. It is 5A. In the case with EPF corruption LCSTART OS will lose single symbol error tolerance. 2) There are two different symbol definitions for SDS, one newly defined, the other from PCIe3. Both are DC balanced, but need to eliminate one.

Benefits as a result of the changes:
The changes: 1) Change SLC symbol from 5AH to 4BH. The criteria for SLC change are, (1). Tolerate to single symbol error with LCSTART OS; (2). DC balance 2) Use E1h as SDS identifier to make it consistent with PCIe3.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
Design change needed

An analysis of the hardware implications:
Design change needed

An analysis of the software implications:
None

An analysis of the compliance testing implications:
Change needed

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

Section 6.3.3

From Text:

Symbol	Name	Gen 1 Symbol	Gen 2 Symbol	Description
SKP	Skip	K28.1	CCh	Compensates for different bit rates between two communicating ports. SKPs may be dynamically inserted or removed from the data stream. For SuperSpeedPlus operation, unscrambled
SKPEND	Skip End	Not applicable	33h	Marks the boundary between SKP symbols and the remainder of the SKP OS. Unscrambled.
SDP	Start Data Packet	K28.2	96h	Marks the start of a Data Packet Payload. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
EDB	End Bad	K28.3	69h	Marks the end of a nullified Packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SUB	Decode Error Substitution	K28.4	Not applicable	Symbol substituted by the 8b/10b decoder when a Decode error is detected.
COM	Comma	K28.5	Not applicable	Used for symbol alignment.
-----	-----	K28.6	Not applicable	Reserved
SHP	Start Header Packet	K27.7	9Ah	Marks the start of a Data Packet (Gen 1 operation only), Transaction Packet or Link Management Packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
DPHP	Start Data Packet Header	Not applicable	95h	Marks the start of a Data Packet (SuperSpeedPlus only). Scrambled and transmitted only in data block.
END	End	K29.7	65h	Marks the end of a packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SLC	Start Link Command	K30.7	5Ah	Marks the start of a Link Command. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
EPF	End Packet Framing	K23.7	36h	Marks the end of a packet framing. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SDS	Start of Data Stream	Not applicable	63h	Marks the start of an SDS Ordered Set. Unscrambled.

USB 3.1 ENGINEERING CHANGE NOTICE

To Text:

Symbol	Name	Gen 1 Symbol	Gen 2 Symbol	Description
SKP	Skip	K28.1	CCh	Compensates for different bit rates between two communicating ports. SKPs may be dynamically inserted or removed from the data stream. For SuperSpeedPlus operation, unscrambled
SKPEND	Skip End	Not applicable	33h	Marks the boundary between SKP symbols and the remainder of the SKP OS. Unscrambled.
SDP	Start Data Packet	K28.2	96h	Marks the start of a Data Packet Payload. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
EDB	End Bad	K28.3	69h	Marks the end of a nullified Packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SUB	Decode Error Substitution	K28.4	Not applicable	Symbol substituted by the 8b/10b decoder when a Decode error is detected.
COM	Comma	K28.5	Not applicable	Used for symbol alignment.
-----	-----	K28.6	Not applicable	Reserved
SHP	Start Header Packet	K27.7	9Ah	Marks the start of a Data Packet (Gen 1 operation only), Transaction Packet or Link Management Packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
DHP	Start Data Packet Header	Not applicable	95h	Marks the start of a Data Packet (SuperSpeedPlus only). Scrambled and transmitted only in data block.
END	End	K29.7	65h	Marks the end of a packet. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SLC	Start Link Command	K30.7	4Bh	Marks the start of a Link Command. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
EPF	End Packet Framing	K23.7	36h	Marks the end of a packet framing. For SuperSpeedPlus operation, scrambled and transmitted only in data block.
SDS	Start of Data Stream	Not applicable	E1h	Marks the start of an SDS Ordered Set. Unscrambled.

Section 6.4.1.2.2

From:

Table 11-8 – SDS Ordered Set

Symbol Number	Symbol	Description
0	E1h	SDS Ordered Set Identifier
2 through 15	55h	Body of SDS Ordered Set

USB 3.1 ENGINEERING CHANGE NOTICE

To:

Table 11-9 – SDS Ordered Set

Symbol Number	Symbol	Description
0 through 3	E1h	SDS Ordered Set Identifier
4 through 15	55h	Body of SDS Ordered Set

USB 3.1 ENGINEERING CHANGE NOTICE

Title: SSP ping.LFPS tRepeat Requirement

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Clarify the Superspeed Plus ping.LFPS tRepeat requirement.
Benefits as a result of the changes:
Eliminate any potential confusion for Gen 2 operation. For SuperSpeed, the tRepeat requirement for ping.LFPS is explicitly defined. It is not explicitly defined for SuperSpeed Plus, leading to questions from spec adopters.
An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No impact to Gen 2 operation, it clarifies the intent of the specification.
An analysis of the hardware implications:
No hardware implications for Gen 2.
An analysis of the software implications:
None
An analysis of the compliance testing implications:
No compliance testing implications. This feature was never used for compliance testing.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) From, in Section 6.9.1, Table 6-29, page 180

Table 6-29 – LFPS Transmitter Timing for SuperSpeed Designs¹

	tBurst				tRepeat		
	Min	Typ	Max	Minimum Number of LFPS Cycles ²	Min	Typ	Max
Polling.LFPS	0.6 µs	1.0 µs	1.4 µs		6 µs	10 µs	14 µs
Ping.LFPS ⁸	40 ns		200 ns	2	160 ms	200 ms	240 ms
Ping.LFPS for SuperSpeedPlus ⁹	40 ns		160 ns	2			
tReset ³	80 ms	100 ms	120 ms				
U1 Exit ^{4,5}	600 ns ⁷		2 ms				
U2 / Loopback Exit ^{4,5}	80 µs ⁷		2 ms				
U3 Wakeup ^{4,5}	80 µs ⁷		10 ms				

(a) To, in Section 6.9.1, Table 6-29, page 180

Table 6-29 – LFPS Transmitter Timing for SuperSpeed Designs¹

	tBurst				tRepeat		
	Min	Typ	Max	Minimum Number of LFPS Cycles ²	Min	Typ	Max
Polling.LFPS	0.6 µs	1.0 µs	1.4 µs		6 µs	10 µs	14 µs
Ping.LFPS ⁸	40 ns		200 ns	2	160 ms	200 ms	240 ms
Ping.LFPS for SuperSpeedPlus ⁹	40 ns		160 ns	2	160 ms	200 ms	240 ms
tReset ³	80 ms	100 ms	120 ms				
U1 Exit ^{4,5}	600 ns ⁷		2 ms				
U2 / Loopback Exit ^{4,5}	80 µs ⁷		2 ms				
U3 Wakeup ^{4,5}	80 µs ⁷		10 ms				

USB 3.1 ENGINEERING CHANGE NOTICE

Title: SSP System Jitter Budget

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Change to the 10 Gbps system jitter budget. The change reduces the random jitter (RJ) budget for both transmitters and receivers from 18.4 ps to 14.1 ps each, and gives 6.1 ps relief to the receiver deterministic jitter (DJ) budget.

Benefits as a result of the changes:
Reduces the design difficulty for SuperSpeed Gen 2 receivers by giving them more budget for deterministic jitter. The reduction in RJ is based upon comparison with other industry specs. For example, the jitter spec for PCIe4 and 10G KR are closer to 1 ps RMS @ 1e-12BER. This ECR proposes to reduce the RJ budget from 18.4 ps to 14.1 ps at 1e-12BER (1.00 ps RMS), which is consistent with those specs.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No existing systems exist.

An analysis of the hardware implications:
No existing products on the market. While this proposal is intended to make Gen 2 receivers easier to design, we acknowledge that there is potential impact to in-progress designs. The PHY development companies represented in the PHY WG (Intel, AMD, Synopsys) all support this change, recognizing the benefit to their design efforts.

An analysis of the software implications:
None

An analysis of the compliance testing implications:
The proposal results in changes to the Gen 2 eye mask for transmitter compliance and to the random jitter input for receiver jitter tolerance testing. Since the Gen 2 compliance program is not yet in place, these impacts are minimal as they will be incorporated into the CTS as we develop it.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) From, in Section 6.5.1, Table 6-15, page 160

Table 6-15 – Informative Jitter Budgeting at the Silicon Pads

Jitter Contribution (ps)	Gen 1 (5 GT/s)			Gen 2 (10 GT/s)		
	Rj ^{1,2}	Dj ³	Tj ⁴ at 10 ⁻¹²	Rj ^{1,2}	Dj ³	Tj ⁴ at 10 ⁻¹²
Tx ⁶	2,42	41	75	1,31	17	35,4
Media ⁵	2,13	45	75	0,00	36	36,0
Rx	2,42	57	91	1,31	21	39,4
Total:	4,03	143	200	1,85	74	100

NOTES:

- 1) Rj is the sigma value assuming a Gaussian distribution.
- 2) Rj Total is computed as the Root Sum Square of the individual Rj components.
- 3) Dj budget is using the Dual Dirac method.
- 4) Tj at a 10⁻¹² BER is calculated as 14.068 * Rj + Dj.
- 5) The media budget includes the cancellation of ISI from the appropriate Rx equalization function.
- 6) Tx is measured after application of the JTF.

(a) To, in Section 6.5.1, Table 6-15, page 160

Table 6-15 – Informative Jitter Budgeting at the Silicon Pads

Jitter Contribution (ps)	Gen 1 (5 GT/s)			Gen 2 (10 GT/s)		
	Rj ^{1,2}	Dj ³	Tj ⁴ at 10 ⁻¹²	Rj ^{1,2}	Dj ³	Tj ⁴ at 10 ⁻¹²
Tx ⁶	2,42	41	75	1,00	17	31,1
Media ⁵	2,13	45	75	0,00	36	36,0
Rx	2,42	57	91	1,00	27,1	41,2
Total:	4,03	143	200	1,41	80,1	100

NOTES:

- 1) Rj is the sigma value assuming a Gaussian distribution.
- 2) Rj Total is computed as the Root Sum Square of the individual Rj components.
- 3) Dj budget is using the Dual Dirac method.
- 4) Tj at a 10⁻¹² BER is calculated as 14.068 * Rj + Dj.
- 5) The media budget includes the cancellation of ISI from the appropriate Rx equalization function.
- 6) Tx is measured after application of the JTF.

USB 3.1 ENGINEERING CHANGE NOTICE

(b) From, in Section 6.7.3, Table 6-19, page 168

Table 6-19 – Normative Transmitter Eye Mask at Test Point TP1

Signal Characteristic	5GT/s			10GT/s			Units	Note
	Minimum	Nominal	Maximum	Minimum	Nominal	Maximum		
Eye Height	100		1 200	70		1 200	mV	2,4
Dj			0,43			0,530	UI	1,2,3
Rj			0,23			0,184	UI	1,2,3,5
Tj			0,66			0,714	UI	1,2,3

NOTES:

- 1) Measured over 10^6 consecutive UI and extrapolated to 10^{-12} BER.
- 2) Measured after receiver equalization function.
- 3) Measured at end of reference channel and cables at TP1 in Figure 6-19.
- 4) The eye height is to be measured at the minimum opening over the range from the center of the eye ± 0.05 UI.
- 5) The Rj specification is calculated as 14.069 times the RMS random jitter for 10^{-12} BER.

(b) To, in Section 6.7.3, Table 6-19, page 168

Table 6-19 – Normative Transmitter Eye Mask at Test Point TP1

Signal Characteristic	5GT/s			10GT/s			Units	Note
	Minimum	Nominal	Maximum	Minimum	Nominal	Maximum		
Eye Height	100		1 200	70		1 200	mV	2,4
Dj			0,43			0,530	UI	1,2,3
Rj			0,23			0,141	UI	1,2,3,5
Tj			0,66			0,671	UI	1,2,3

NOTES:

- 1) Measured over 10^6 consecutive UI and extrapolated to 10^{-12} BER.
- 2) Measured after receiver equalization function.
- 3) Measured at end of reference channel and cables at TP1 in Figure 6-19.
- 4) The eye height is to be measured at the minimum opening over the range from the center of the eye ± 0.05 UI.
- 5) The Rj specification is calculated as 14.069 times the RMS random jitter for 10^{-12} BER.

USB 3.1 ENGINEERING CHANGE NOTICE

(c) From, in Section 6.8.5, Table 6-27, page 178

Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing

Symbol	Parameter	Gen 1 (5GT/s)	Gen 2 (10GT/s)	Units	Notes
f1	Tolerance corner	4,9	7,5	MHz	
J _{Rj}	Random Jitter	0,012 1	0,013 08	UI rms	1
J _{Rj_p-p}	Random Jitter peak- peak at 10 ⁻¹²	0,17	0,184	UI p-p	1,4
J _{Pj_500kHz}	Sinusoidal Jitter	2	4,76	UI p-p	1,2,3
J _{Pj_1Mhz}	Sinusoidal Jitter	1	2,03	UI p-p	1,2,3
J _{Pj_2MHz}	Sinusoidal Jitter	0,5	0,87	UI p-p	1,2,3
J _{Pj_4MHz}	Sinusoidal Jitter	N/A	0,37	UI p-p	1,2,3
J _{Pj_f1}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_50MHz}	Sinusoidal Jitter	0,2	0,17	UI p-p	1,2,3
J _{Pj_100MHz}	Sinusoidal Jitter	N/A	0,17	UI p-p	1,2,3
V _{full_swing}	Transition bit differential voltage swing	0,75	0,8	V p-p	1
V _{EQ_level}	Non transition bit voltage (equalization)	-3	Preshoot=2,7 De-emphasis= -3,3	dB	1

NOTES:

- 1) All parameters measured at TP1. The test point is shown in Figure 6-19.
- 2) Due to time limitations at compliance testing, only a subset of frequencies can be tested. However, the Rx is required to tolerate Pj at all frequencies between the compliance test points.
- 3) During the Rx tolerance test, SSC is generated by test equipment and present at all times. Each J_{Pj} source is then added and tested to the specification limit one at a time.
- 4) Random jitter is also present during the Rx tolerance test, though it is not shown in Figure 6-20.
- 5) The JTOL specs for Gen 2 comprehend jitter peaking with re-timers in the system and has a 25 dB/decade slope.

USB 3.1 ENGINEERING CHANGE NOTICE

(c) To, in Section 6.8.5, Table 6-27, page 178

Table 6-27 – Input Jitter Requirements for Rx Tolerance Testing

Symbol	Parameter	Gen 1 (5GT/s)	Gen 2 (10GT/s)	Units	Notes
f1	Tolerance corner	4,9	7,5	MHz	
J _{Rj}	Random Jitter	0,012 1	0,010 0	UI rms	1
J _{Rj_p-p}	Random Jitter peak- peak at 10 ⁻¹²	0,17	0,184	UI p-p	1,4
J _{Pj_500kHz}	Sinusoidal Jitter	2	4,76	UI p-p	1,2,3
J _{Pj_1Mhz}	Sinusoidal Jitter	1	2,03	UI p-p	1,2,3
J _{Pj_2MHz}	Sinusoidal Jitter	0,5	0,87	UI p-p	1,2,3
J _{Pj_4MHz}	Sinusoidal Jitter	N/A	0,37	UI p-p	1,2,3
J _{Pj_f1}	Sinusoidal Jitter	0.2	0,17	UI p-p	1,2,3
J _{Pj_50MHz}	Sinusoidal Jitter	0.2	0,17	UI p-p	1,2,3
J _{Pj_100MHz}	Sinusoidal Jitter	N/A	0,17	UI p-p	1,2,3
V _{full_swing}	Transition bit differential voltage swing	0,75	0,8	V p-p	1
V _{EQ_level}	Non transition bit voltage (equalization)	-3	Preshoot=2,7 De-emphasis= -3,3	dB	1

NOTES:

- 1) All parameters measured at TP1. The test point is shown in Figure 6-19.
- 2) Due to time limitations at compliance testing, only a subset of frequencies can be tested. However, the Rx is required to tolerate Pj at all frequencies between the compliance test points.
- 3) During the Rx tolerance test, SSC is generated by test equipment and present at all times. Each J_{Pj} source is then added and tested to the specification limit one at a time.
- 4) Random jitter is also present during the Rx tolerance test, though it is not shown in Figure 6-20.
- 5) The JTOL specs for Gen 2 comprehend jitter peaking with re-timers in the system and has a 25 dB/decade slope.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB3.1 tHubDriveResume

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Add new parameter <code>tHubDriveResume</code> to use in place of <code>tHubDriveRemoteWakeDownstream</code> for initiating U3 exit, resume, on downstream ports. This new parameter has a much larger max value.

Benefits as a result of the changes:
Reusing the remote wake parameter for resume resulted in unnecessary restriction on resume timing. Need reasonable value that is testable for compliance. While in U3 a port will likely enter pipe pwrdown state of P3 with a low frequency suspend clock. In order to meet the required 20 ns delay between signaling LFPS for Ux exit and the start of SS signaling for recovery (defined in 6.9.2), designs will also likely switch back to the SS clock before starting to signal LFPS. The time to return to the SS clock depends on how slow the suspend clock is. For example, given 5 clocks at a 32 kHz rate for the process return to P0, switch clocks and start U3 exit signaling would take over 156 us. Though such a large delay has not been observed, there is no need to prevent it and doing so could enable greater power savings in future products by use of an even slower clock.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
No effect on use of u3 exit request for normal operation. Already observed that up to an additional 9 us to exit U3 has not impacted efficient and robust return to U0 when a DSP is directed to exit U3.

An analysis of the hardware implications:
None. Whereas, requiring compliance to original value would have.

An analysis of the software implications:
None

An analysis of the compliance testing implications:
Update test TD 10.115.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

Table 10-19, Hub Parameters

To Text: add parameter

Name	Description	Min	Max	Units
tHubDriveResume	Time from receiving a SetPortFeature(PORT_LINK_STATE) U0 for a downstream port with a link in U3 to driving resume signaling on the link.	0	400	μs

10.10 Suspend and Resume

From Text:

When a hub upstream port's link is in the U3 state and it receives wakeup signaling from its link partner on the hub upstream port's link, the hub shall automatically drive remote wakeup to any downstream ports that are in U3 and have received remote wakeup signaling since entering U3.

To Text:

When a hub upstream port's link is in the U3 state and it receives wakeup signaling from its link partner on the hub upstream port's link, the hub shall automatically drive remote wakeup handshake or resume to any downstream ports that are in U3 and have received remote wakeup signaling since entering U3.

From Text:

When the hub receives a SetPortFeature(PORT_LINK_STATE) U0 for a downstream port with a link in U3, the hub shall drive remote wakeup signaling on the link in tHubDriveRemoteWakeDownstream.

To Text:

When the hub receives a SetPortFeature(PORT_LINK_STATE) U0 for a downstream port with a link in U3, the hub shall drive resume signaling on the link in tHubDriveResume.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: TSEQ Gen2 Clarification

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Clarification of the TSEQ rules for Gen 2 operation.

Benefits as a result of the changes:
Eliminate confusion about the number of times that the Gen 2 TSEQ ordered set is transmitted and about the periodicity of the training pattern.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
Applies to Gen 2 only. No existing systems with Gen 2 implementation, so no impact.

An analysis of the hardware implications:
Ensures TSEQ is implemented correctly.

An analysis of the software implications:
None

An analysis of the compliance testing implications:
Ensures test equipment suppliers implement TSEQ algorithms correctly.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) From Text (and location): Section 6.4.1.2.2, Page 154

6.4.1.2.2 Training Sequence Values for Gen 2 Operation

Transmitters are required to track the running DC Balance of the bits transmitted on the wire (after scrambling) for TSEQ, TS1 and TS2 Ordered Sets. The running DC Balance is the difference between the number of 1s transmitted and the number of 0s transmitted.

(a) To Text (and location): Section 6.4.1.2.2, Page 154

6.4.1.2.2 Training Sequence Values for Gen 2 Operation

The TSEQ training sequence is transmitted 524 288 times to allow for testing many coefficient settings.

Transmitters are required to track the running DC Balance of the bits transmitted on the wire (after scrambling) for TSEQ, TS1 and TS2 Ordered Sets. The running DC Balance is the difference between the number of 1s transmitted and the number of 0s transmitted.

USB 3.1 ENGINEERING CHANGE NOTICE

(b) From Text (and location): Section 7.5.4.7.1, Page 242

7.5.4.7.1 Polling.RxEQ Requirements

The detection and correction of the lane polarity inversion in SuperSpeed operation shall be enabled, as is described in Section 6.4.2.

- The port shall transmit the corresponding TSEQ ordered sets in Table 6-2.

(b) To Text (and location): Section 7.5.4.7.1, Page 242

7.5.4.7.1 Polling.RxEQ Requirements

- The detection and correction of the lane polarity inversion in SuperSpeed operation shall be enabled, as is described in Section 6.4.2.
- The port shall transmit the corresponding TSEQ ordered sets defined in Table 6-2 for SuperSpeed operation, or Table 6-8 for SuperSpeedPlus operation.

USB 3.1 ENGINEERING CHANGE NOTICE

(c) From Text (and location): Section 7.5.4.7.2, Page 242

7.5.4.7.2 Exit from Polling.RxEQ

- The port in SuperSpeed operation shall transition to Polling.Active after 65 536 consecutive TSEQ ordered sets defined in Table 6-2 are transmitted.
- The port in SuperSpeedPlus operation shall transition to Polling.Active after 262 143 TSEQ ordered sets defined in Table 6-8 are transmitted. Refer to Section 6.4.1.2.1 for SYNC ordered set insertion while transmitting TSEQ ordered sets.

(c) To Text (and location): Section 7.5.4.7.2, Page 242

7.5.4.7.2 Exit from Polling.RxEQ

- The port in SuperSpeed operation shall transition to Polling.Active after 65 536 consecutive TSEQ ordered sets defined in Table 6-2 are transmitted.
- The port in SuperSpeedPlus operation shall transition to Polling.Active after 524 288 TSEQ ordered sets defined in Table 6-8 are transmitted. Refer to Section 6.4.1.2.1 for SYNC ordered set insertion while transmitting TSEQ ordered sets.

USB 3.1 ENGINEERING CHANGE NOTICE

(d) From Text (and location): Section 6.8.1, Page 170

The 3rd paragraph of Section 6.8.1:

During Gen 2 operation, the training period is ~8 ms. The training pattern is periodic with a period of 16 384 132-bit blocks (2162688UI). The much longer pattern greatly increases the richness of the pattern compared to Gen 1. The Gen 2 training pattern spectrum is essentially white. Due to the length of the Gen 2 training interval and the potential desire to examine the data, SKPs are inserted during polling.TSEQ.

(d) To Text (and location): Section 6.8.1, Page 170

During Gen 2 operation, the training period is ~8 ms. The training pattern is periodic with a period of 16 385 132-bit blocks (16384 TSEQ blocks plus a SYNC OS block). The much longer pattern greatly increases the richness of the pattern compared to Gen 1. The Gen 2 training pattern spectrum is essentially white. Due to the length of the Gen 2 training interval and the potential desire to examine the data, SKPs are inserted during polling.TSEQ.

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB 3.1 VBUS Max Limit

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes:
Update the USB3.1 specification to align with the expanded VBUS voltage definition in the USB-IF Type-C specification which defines VBUS as having a Max supply voltage of 5,50 V DC. The Type-C cable and connector definition specifies a minimal current carrying capacity of 3A. This increased capacity creates a situation where losses in cables, connectors, PCB traces and power supply components will result in the VBUS level available at the consumer end of the cable that is below the minimum acceptable level for operation. As a result, in order to have an implementable Type-C specification it is necessary to boost the upper limit for VBUS at the supply side to 5,5 V DC from the present spec value of 5,25 V DC.

Benefits as a result of the changes:
OEM's can design products that support the higher power levels of the Type-C connector and provide VBUS levels at or above 5,25 V to ensure that the VBUS voltage that is available at the far-side connection is within a suitable working range for USB products, and is also at a high enough level to charge batteries in as short a time interval as possible to ensure customer satisfaction.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
The impact is difficult to assess as there is a wide range of component vendors supporting USB, and also a wide variation of design methodologies by OEM's. There are certainly going to be a subset of existing USB products that may be sensitive to operation at VBUS levels of up to 5,5 V. The sensitivity will be mitigated in many cases by cable losses, so the devices themselves may never experience the 5,5 V condition. In the cases where there is a direct attach, then connector losses are minimized and an increased potential will exist for a device that is not tolerant of 5,5 V operation to experience overstress. The overstress results will be device specific and is impossible to outline in this ECN but is not expected to result in a safety situation of any kind.

An analysis of the hardware implications:
New designs adhering to this ECN will need to consider the new 5,5 V VBUS max level during the component selection process.

An analysis of the software implications:
There are no known software implications.

An analysis of the compliance testing implications:
Compliance will need to make the small modifications necessary to accommodate the new upper limit.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change

(a) Section 11.4.2 Steady State Voltage Drop Budget

From Text:

The nominal 5 V $\pm$ 5 % source (host or hub) is 4,75 V to 5,25 V

To Text:

- 1) The nominal 5 V – 5 % +10 % source (host or hub) is 4,75 V to 5,50 V

USB 3.1 ENGINEERING CHANGE NOTICE

(b) Section 11.4.5 VBUS Electrical Characteristics.

Table 11-2, Supply Voltage: Port (downstream connector) VBUS, Max.

From Text:

Change From: 5,25 V

Table 11-2. DC Electrical Characteristics

Parameter	Symbol	Conditions	Min.	Max.	Units
Supply Voltage:					
Port (downstream connector)	VBUS		4.45	5.25	V
Port (upstream connector)	VBUS		4.0		V
Supply Current:					
High-power Hub Port (out)	ICCPRT		900		mA
Low-power Hub Port (out)	ICCUPT		150		mA
High-power Peripheral Device (in)	ICCHPF			900	mA
Low-power Peripheral Device (in)	ICCLPF			150	mA
Unconfigured Device (in)	ICCINIT			150	mA
Suspended High-power Device	ICCS			2.5	mA

To Text:

Change To: 5,50 V

Table 11-2. DC Electrical Characteristics

Parameter	Symbol	Conditions	Min.	Max.	Units
Supply Voltage:					
Port (downstream connector)	VBUS		4.45	5.50	V
Port (upstream connector)	VBUS		4.0		V
Supply Current:					
High-power Hub Port (out)	ICCPRT		900		mA
Low-power Hub Port (out)	ICCUPT		150		mA
High-power Peripheral Device (in)	ICCHPF			900	mA
Low-power Peripheral Device (in)	ICCLPF			150	mA
Unconfigured Device (in)	ICCINIT			150	mA
Suspended High-power Device	ICCS			2.5	mA

USB 3.1 ENGINEERING CHANGE NOTICE

NOTICE: Any Company or Companies submitting an ECR proposal must be one of the following: a Promoter, Contributor, or Adopter of the USB 3.0 Specification for which the ECR is being submitted. If a group of Companies is submitting an ECR proposal, each company must be either a Promoter, Contributor, or Adopters of the USB 3.1 Specification.

SPECIFICATION REVISIONS AND ADDENDA: At any point in time, there shall only be one current version of the USB 3.1 Specification, termed the production version. At the same time, there may also be proposed revisions to the specification's design which are not yet approved and shall be held confidential as deemed necessary by the USB 3.1 Promoters and within the Group of Working Committee(s).

PROCEDURES FOR SUBMITTING PROPOSALS: Both members of the USB Implementers Forum as a whole and members of the USB 3.1 Promoters may submit requests to revise the USB 3.1 Specification and design guides. Such a request may be rejected or may result in a USB 3.1 Engineering Change Notice (ECN), which is the official way USB specifications may be changed.

FORMAT OF PROPOSAL: The originator of a request to alter the USB 3.1 Specification may do so in writing, either through electronic mail or printed copy. For an electronic copy of this form call 1-503-619-0426 or Fax 1-503-644-6708. Although faxed proposals are acceptable, the original document must be sent to the Secretary of the USB Implementers Forum (see below) after the Fax is sent.

RESUBMISSION AND APPEAL: The originator of a request that was not approved can redraft the original request. Rewritten proposal will be treated as a new proposal and will be evaluated using the procedures described above. The originator of a request that was not approved can also submit an appeal to the USB 3.1 Promoters. The appeal must be made in writing and addressed to the Secretary of the USB Implementers Forum.

ABOUT THE ENGINEERING CHANGE REQUEST FORM:

The Purpose of this Engineering Change Request Form is to expedite the review process of the proposal by providing explanations, background information, and examples of the proposed changes at a high level. This form serves as an executive summary to the actual proposal.

STEPS ON HOW TO SUBMIT A USB 3.1 ENGINEERING CHANGE REQUEST:

- 1) Please fill out the Engineering Change Request Form on the next page completely.
- 2) Give a minimum of 2-3 sentences for each description on the form.
- 3) Attach the form on top of your detailed proposal to serve as a cover sheet.
- 4) Send the entire document with the written proposal to the Secretary of the USB-IF at:

USB Implementers Forum

Attn: Traci Donnell

3855 SW 153rd Drive

Beaverton, OR 97006

E-mail: admin@usb.org

FAX#: (503) 644-6708

USB 3.1 ENGINEERING CHANGE NOTICE

Name: Mark Bohm Email: mark.bohm@microchip.com

Company: Microchip Mailstop: SC12-332

Address: 10900-B Stonelake Blvd, Suite 100

City: Austin State/Province: TX

Country: USA Zip/Postal Code: 78759

Phone: 512-427-0330 FAX: 512-334-1984

Name: _____ Email: _____

Company: _____ Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

Name: _____ Email: _____

Company: _____ Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

Name: _____ Email: _____

Company: _____ Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

USB 3.1 ENGINEERING CHANGE NOTICE

Title: USB 3.1 VBUS Max Limit

Applied to: USB_3_1r1.0_07_31_2013

Brief description of the functional changes proposed:
--

Update the USB3.1 specification to align with the expanded VBUS voltage definition in the USB-IF Type-C specification which defines VBUS as having a Max supply voltage of 5,50 V DC.
--

The Type-C cable and connector definition specifies a minimal current carrying capacity of 3 A. This increased capacity creates a situation where losses in cables, connectors, PCB traces and power supply components will result in the VBUS level available at the consumer end of the cable that is below the minimum acceptable level for operation. As a result, in order to have an implementable Type-C specification it is necessary to boost the upper limit for VBUS at the supply side to 5,5 V DC from the present spec value of 5,25 V DC.

Benefits as a result of the proposed changes:
--

OEM's can design products that support the higher power levels of the Type-C connector and provide VBUS levels at or above 5,25 V to ensure that the VBUS voltage that is available at the far-side connection is within a suitable working range for USB products, and is also at a high enough level to charge batteries in as short a time interval as possible to ensure customer satisfaction.
--

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
--

The impact is difficult to assess as there is a wide range of component vendors supporting USB, and also a wide variation of design methodologies by OEM's.
--

There are certainly going to be a subset of existing USB products that may be sensitive to operation at VBUS levels of up to 5,5 V. The sensitivity will be mitigated in many cases by cable losses, so the devices themselves may never experience the 5,5 V condition. In the cases where there is a direct attach, then connector losses are minimized and an increased potential will exist for a device that is not tolerant of 5,5 V operation to experience overstress.

The overstress results will be device specific and is impossible to outline in this ECR but is not expected to result in a safety situation of any kind.

An analysis of the hardware implications:
--

New designs adhering to this ECR will need to consider the new 5,5 V VBUS max level during the component selection process

An analysis of the software implications:
--

There are no known software implications.
--

An analysis of the compliance testing implications:
--

Compliance will need to make the small modifications necessary to accommodate the new upper limit.

USB 3.1 ENGINEERING CHANGE NOTICE

Actual Change Requested

(a) Section 11.4.2 Steady State Voltage Drop Budget

From Text:

The nominal 5 V $\pm$ 5 % source (host or hub) is 4,75 V to 5,25 V

To Text:

The nominal 5 V – 5 % +10 % source (host or hub) is 4,75 V to 5,50 V

USB 3.1 ENGINEERING CHANGE NOTICE

(b) Section 11.4.5 VBUS Electrical Characteristics.

Table 11-2, Supply Voltage: Port (downstream connector) VBUS, Max.

From Text:

Change From: 5,25 V

Table 11-2. DC Electrical Characteristics

Parameter	Symbol	Conditions	Min.	Max.	Units
Supply Voltage:					
Port (downstream connector)	VBUS		4.45	5.25	V
Port (upstream connector)	VBUS		4.0		V
Supply Current:					
High-power Hub Port (out)	ICCPRT		900		mA
Low-power Hub Port (out)	ICCUPT		150		mA
High-power Peripheral Device (in)	ICCHPF			900	mA
Low-power Peripheral Device (in)	ICCLPF			150	mA
Unconfigured Device (in)	ICCINIT			150	mA
Suspended High-power Device	ICCS			2.5	mA

To Text:

Change To: 5,50 V

Table 11-2. DC Electrical Characteristics

Parameter	Symbol	Conditions	Min.	Max.	Units
Supply Voltage:					
Port (downstream connector)	VBUS		4.45	5.50	V
Port (upstream connector)	VBUS		4.0		V
Supply Current:					
High-power Hub Port (out)	ICCPRT		900		mA
Low-power Hub Port (out)	ICCUPT		150		mA
High-power Peripheral Device (in)	ICCHPF			900	mA
Low-power Peripheral Device (in)	ICCLPF			150	mA
Unconfigured Device (in)	ICCINIT			150	mA
Suspended High-power Device	ICCS			2.5	mA

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

3, rue de Varembé
PO Box 131
CH-1211 Geneva 20
Switzerland

Tel: + 41 22 919 02 11
Fax: + 41 22 919 03 00
info@iec.ch
www.iec.ch